ORDERED

Architecting Commerce Platforms

Željko Obrenović

ORDERED: Architecting Commerce Platforms

Željko Obrenović

This book is available at https://leanpub.com/ordered

This version was published on 2026-11-07

This is a Leanpub book. Leanpub empowers authors and publishers with the Lean Publishing process. Lean Publishing is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2026 Željko Obrenović

Table of Contents

ORDERED: Architecting Commerce Platforms . . . . . **i**

Preface . . . . . **ii**

Introduction: Why Commerce Architecture Matters . . . . . **iii**
- Who This Book Is For iv
- The Argument: A Promise-Keeping Machine v
- The Order's Journey as the Spine vii
- Four Scale Stages, and Why Every Chapter Compares Them viii
- How the Book Is Organized ix
- Start With the Decision in Front of You xii
- How to Read the Evidence xvi
- What the Book Draws On xviii
- Kestrel & Pine and the Toolkit xviii
- What the Book Leaves to Others xix
- The Failure Patterns xx
- A Living Journal xx
- Where to Go Next xxi
- Sources and Evidence xxi

Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong . . . . . **xxii**
- Sixteen Patterns to Recognize xxiii
- Where the Patterns Strike xlvii
- How the Patterns Reinforce Each Other l
- Using the Patterns li
- Sources and Evidence li

Meet Kestrel & Pine . . . . . **liv**
- Kestrel & Pine at a Glance lv
- How Kestrel & Pine Grew, 2016–2027 lvi
- The Catalog and the Products lviii
- Channels and Customers lviii
- The Volumes Every Design Is Sized Against lix

The Systems Landscape in January 2027 . . . lx
The People . . . lxiii
The Problems Kestrel & Pine Starts 2027 With . . . lxv
The Waypoint Program . . . lxvii
The Recurring Worked Examples . . . lxx
How the Chapters Use Kestrel & Pine . . . lxxii
Sources and Evidence . . . lxxii

Part I — COMMERCE: What a Commerce Platform Is . . . 1
Chapters in this part . . . 6
Sources and Evidence . . . 7

1. Follow the Order: What Happens Between "Buy" and "Delivered" . . . 8
Setting the Scene at Kestrel & Pine: Four Drawings of One Order . . . 10
The Journey in Thirteen Steps . . . 11
The Things an Order Is Made Of . . . 13
Promises and Handoffs . . . 15
One Order, Three Status Lines . . . 17
Eva's Jacket, System by System . . . 20
Why the Joins Are Where Things Break . . . 23
In Practice: What the Public Record Says . . . 25
Across Scale Stages: When It Fits and When It Limits . . . 26
When It Goes Wrong at Each Step . . . 26
What to Take Away . . . 28
Questions to Ask . . . 29
Sources and Evidence . . . 30

2. Who Sells What to Whom: Business Models and the Economics That Shape Architecture . . . 32
Setting the Scene at Kestrel & Pine: One Jacket, Four Ways to Sell It . . . 34
The Business Models, Defined by the People Who Report Them . . . 34
What Each Model Changes in the Promise Chain . . . 37
The Unit Economics of an Order . . . 41
The Same Jacket in Three Lines: A Worked Example . . . 46
Seasonality and the Peak . . . 49
The Four Scale Stages . . . 49
The Choices: One Stack, Many Stacks or a Shared Core . . . 54
In Practice: What the Public Record Says . . . 56
When It Goes Wrong . . . 57
What to Take Away . . . 58
Questions to Ask . . . 59
Sources and Evidence . . . 59

3. The Map of a Commerce Platform: Capabilities and Layers 62
Setting the Scene at Kestrel & Pine: A Whiteboard in February 2027 64
What a Capability Is, and Why It Comes Before the Systems 65
The Map: Six Layers 66
How the Pieces Talk: The Typical Flows 75
Three Ways to Cover the Map 78
Across Scale Stages: When It Fits and When It Limits 81
Reading a Vendor's Product Page Against the Map 83
In Practice: Kestrel & Pine's Systems on the Map 83
When It Goes Wrong 87
What to Take Away 87
Questions to Ask 88
Sources and Evidence 89

4. Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record . 91
Setting the Scene at Kestrel & Pine: "Where Does Stock Live?" 93
Facts, Records and Copies 95
Bounded Contexts: Why the Warehouse's Order Is Not the Ledger's Order 96
The Entity–Owner Table 98
Four Ways to Assign Ownership 104
Ownership in a Suite and Across Vendors 106
When Ownership Is Not Decided: The Five Truth Patterns 107
Across Scale Stages: When It Fits and When It Limits 108
In Practice 110
When It Goes Wrong 113
What to Take Away 114
Questions to Ask 114
Sources and Evidence 115

5. Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026 . . 117
Setting the Scene at Kestrel & Pine: Two Pitches in One Week 119
Seven Kinds of Platform 120
How Platforms Are Priced, and What the Price Tells You 127
What Changed, 2023 to 2026 130
How to Read a Vendor's Claims 131
The Trade-offs Between Categories 133
Across Scale Stages: Which Categories Fit Which Stage 134
Kestrel & Pine's Place on the Map 135
When It Goes Wrong 136
What to Take Away 137
Questions to Ask 138
Sources and Evidence 139

Part II — CATALOG: Products, Prices and Promises 142

Chapters in this part 145
Sources and Evidence 146

6. What Are We Selling? Products, Variants and the Catalog 147
Setting the Scene at Kestrel & Pine: The Season That Was Typed Four Times 149
What a Product Is: Four Things That Share a Name 150
Who Owns Which Field: Producers, Consumers and the System of Record 153
How the Pieces Fit Together With a PIM 157
The Choices 159
Trade-offs 159
Across Scale Stages: When It Fits and When It Limits 161
In Practice 163
When It Goes Wrong 165
What to Take Away 166
Questions to Ask 167
Sources and Evidence 167

7. What Does It Cost? Price Lists, Contract Prices, Currencies and Tax 170
Setting the Scene at Kestrel & Pine: The Price Drift 171
What a Price Is Made Of 173
Tax, Markets and Currencies: Why €199 Is Three Different Prices 176
The Price Rules Europe Writes Into the System 177
How the Pieces Fit Together 178
The Choices 181
Trade-offs 182
Across Scale Stages: When It Fits and When It Limits 184
In Practice 186
When It Goes Wrong 188
What to Take Away 189
Questions to Ask 190
Sources and Evidence 191

8. Discounts, Coupons and Loyalty: Promotions Without Chaos 193
Setting the Scene at Kestrel & Pine: The Black Friday Stacking Bug 195
What a Promotion Is, and What It Touches 197
How the Pieces Fit Together 198
Gift Cards, Store Credit and Points: Ledgers, Not Promotions 202
The Choices 204
Across Scale Stages: When It Fits and When It Limits 206
In Practice 208
When It Goes Wrong 210

What to Take Away . . . 211
Questions to Ask . . . 212
Sources and Evidence . . . 213

9. Finding the Product: Search, Navigation and Merchandising . . . 215
Setting the Scene at Kestrel & Pine: The Waterproof Jacket Nobody Could Find . . . 217
Three Ways to Find a Product . . . 218
How the Pieces Fit Together: The Index as a Copy . . . 220
Reviews and Ratings as Discovery and Trust Data . . . 225
The Choices . . . 226
Across Scale Stages: When It Fits and When It Limits . . . 228
In Practice . . . 230
When It Goes Wrong . . . 230
What to Take Away . . . 231
Questions to Ask . . . 233
Sources and Evidence . . . 233

10. Is It in Stock? Inventory, Availability and the Promise You Can Keep . . . 235
Setting the Scene at Kestrel & Pine: Friday 27 November 2026, 19:40 . . . 237
What "In Stock" Means: Five Numbers, Not One . . . 239
When a Unit Becomes Promised: Reservations and Allocations . . . 241
Many Locations, Many Channels . . . 243
How the Pieces Fit: The Availability Service . . . 245
The Choices . . . 249
Trade-offs . . . 250
Across Scale Stages: When It Fits and When It Limits . . . 251
In Practice . . . 252
When It Goes Wrong . . . 254
What to Take Away . . . 256
Questions to Ask . . . 257
Sources and Evidence . . . 257

Part III — ORDER: Cart, Checkout, Payment and the Order Record . . . 260
Chapters in this part . . . 263
Sources and Evidence . . . 264

11. From Cart to Confirmation: Checkout Orchestration . . . 265
Setting the Scene at Kestrel & Pine: Three Requests for the Checkout . . . 267
The Cart Is a Draft, the Checkout Is a Commitment . . . 268
What Checkout Must Call . . . 269
The Clock and the Detour . . . 272

Guest, Account and the Express Wallets . . . 274
Abandoned Carts and the Business Metric . . . 274
The Choices: Four Ways to Run a Checkout . . . 275
How Protected Checkout Is: Shopify's Timeline, 2023–2026 . . . 276
What Changes When the Buyer Is a Company . . . 278
Across Scale Stages: When It Fits and When It Limits . . . 278
Eva's Checkout, Step by Step . . . 280
When It Goes Wrong . . . 283
What to Take Away . . . 284
Questions to Ask . . . 285
Sources and Evidence . . . 286

12. Taking the Money: Payment Providers, Authorization, Capture and Refunds . . . 288
Setting the Scene at Kestrel & Pine: Three Weeks in January . . . 290
What a Payment Service Provider Does . . . 291
The Order Depends on a Message It Does Not Control . . . 295
Strong Customer Authentication and 3-D Secure . . . 298
Refunds, Cancels and Chargebacks: Money Flowing Backward . . . 299
Fraud and Card Data: Keep the Attack Out and the Card Number Out . . . 301
The Choices: Four Ways to Take Payments . . . 302
Stores, Wholesale and Marketplaces: Four Feeds, Not One . . . 304
Across Scale Stages: When It Fits and When It Limits . . . 304
In Practice . . . 306
When It Goes Wrong . . . 307
What to Take Away . . . 307
Questions to Ask . . . 308
Sources and Evidence . . . 308

13. The Order Record: Lifecycle, State and the Order Management System . . . 311
Setting the Scene at Kestrel & Pine: Order #1052877 . . . 313
What an Order Record Holds . . . 314
Several State Machines on One Record . . . 316
Changing a Placed Order . . . 318
Routing, Sourcing and Splits . . . 318
Many Channels, One Record . . . 320
The OMS as a Product . . . 321
The Choices . . . 325
Trade-offs . . . 326
Across Scale Stages: When It Fits and When It Limits . . . 327
In Practice . . . 328
When It Goes Wrong . . . 329
What to Take Away . . . 331
Questions to Ask . . . 332

Sources and Evidence . . . 333

14. Tax, Invoices and the Books: From Order to Ledger . . . 335
Setting the Scene at Kestrel & Pine: Sanne's April Close . . . 337
What the Commerce System Owes Finance . . . 337
VAT in Plain Words . . . 339
Tax at Checkout and Tax at Invoice . . . 341
The Invoice as a Legal Document . . . 341
E-Invoicing at a Glance . . . 343
From Order to Ledger: What Gets Posted . . . 344
Reconciliation: Orders, Payouts and the Bank . . . 345
The Choices . . . 347
Across Scale Stages: When It Fits and When It Limits . . . 348
Eva's Order in the Books, and Ardennes Trail Co.'s Peppol Invoice . . . 350
When It Goes Wrong . . . 352
What to Take Away . . . 353
Questions to Ask . . . 354
Sources and Evidence . . . 354

Part IV — FULFILLMENT: From Warehouse to Doorstep and Back . . . 357
Chapters in this part . . . 360
Sources and Evidence . . . 361

15. Pick, Pack, Ship: Warehouse Management and Fulfillment . . . 362
Setting the Scene: Duiven's Day . . . 364
What a Warehouse Management System Does . . . 365
How the Warehouse Connects to Everything Else . . . 367
The Choices: Who Runs the Warehouse Software . . . 373
Across Scale Stages: When It Fits and When It Limits . . . 375
In Practice . . . 377
When It Goes Wrong . . . 378
What to Take Away . . . 379
Questions to Ask . . . 380
Sources and Evidence . . . 381

16. Getting It There: Carriers, Labels, Tracking and the Delivery Promise . . . 383
Setting the Scene: A Sentence in the Theme . . . 384
The Vocabulary of Getting It There . . . 385
How the Pieces Fit Together . . . 387
The Choices . . . 393
Across Scale Stages: When It Fits and When It Limits . . . 395
In Practice . . . 397

When It Goes Wrong . . . 399
What to Take Away . . . 400
Questions to Ask . . . 401
Sources and Evidence . . . 402

17. Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment . . . 404
Setting the Scene at Kestrel & Pine: The Utrecht Pilot and a Deferred Decision . . . 406
What a Store Adds to the Architecture . . . 407
How the Pieces Fit Together . . . 410
The Choices . . . 417
Across Scale Stages: When It Fits and When It Limits . . . 420
In Practice . . . 421
When It Goes Wrong . . . 424
What to Take Away . . . 425
Questions to Ask . . . 426
Sources and Evidence . . . 426

18. The Way Back: Returns, Exchanges, Refunds and Reverse Logistics . . . 429
Setting the Scene at Kestrel & Pine: One Item in Four . . . 431
What a Return Is, Step by Step . . . 432
How the Pieces Fit Together . . . 435
The Law: Fourteen Days, Three Times Over . . . 438
The Choices . . . 440
Returns From Marketplaces and Stores . . . 442
Across Scale Stages: When It Fits and When It Limits . . . 443
In Practice . . . 444
When It Goes Wrong . . . 447
What to Take Away . . . 448
Questions to Ask . . . 449
Sources and Evidence . . . 450

19. Where Is My Order? Notifications, Service and the Customer Record . . . 451
Setting the Scene at Kestrel & Pine: Eleven Tickets, Six Days, One Order . . . 453
What the Customer Needs to Know, and Which System Knows It . . . 456
How the Pieces Fit Together . . . 459
The Choices . . . 463
Across Scale Stages: When It Fits and When It Limits . . . 467
In Practice . . . 469
When It Goes Wrong . . . 473
What to Take Away . . . 474
Questions to Ask . . . 475
Sources and Evidence . . . 475

Part V — CHANNELS: Selling Everywhere . 477

Chapters in this part . . . 480
Sources and Evidence . . . 481

20. Selling on Someone Else's Platform: Marketplaces and Channel Integration . . 482
Setting the Scene at Kestrel & Pine: Two Days of Stale Stock . . . 484
What a Marketplace Is, and the Two Ways to Sell on One . . . 485
What the Marketplace Measures You On . . . 487
How the Pieces Fit Together . . . 489
Channel Allocation: The Number You Send Is a Promise . . . 492
The Choices . . . 495
Across Scale Stages: When It Fits and When It Limits . . . 497
In Practice . . . 499
When It Goes Wrong . . . 502
What to Take Away . . . 503
Questions to Ask . . . 504
Sources and Evidence . . . 505

21. Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals 507
Setting the Scene at Kestrel & Pine: The 38% . . . 509
What Is Different When the Buyer Is a Company . . . 510
Who Owns What . . . 514
How a Price Becomes a Promise . . . 518
Credit, Terms and the Buyer's Own Controls . . . 520
The Choices . . . 521
Across Scale Stages: When It Fits and When It Limits . . . 524
In Practice . . . 525
When It Goes Wrong . . . 527
What to Take Away . . . 529
Questions to Ask . . . 530
Sources and Evidence . . . 530

22. Documents Between Companies: EDI, Punchout and E-Invoicing 532
Setting the Scene at Kestrel & Pine: A Deduction Instead of a Payment . . . 534
What EDI Is and Why It Has Not Gone Away . . . 535
The Documents: Order, Despatch Advice, Invoice . . . 536
Identifiers: What, Where and This Pallet . . . 538
Transport: Point-to-Point or Network . . . 539
Mapping, Onboarding and the Chargeback Economy . . . 541
How EDI Documents Reach the Order Record and the Ledger . . . 541
Punchout: When the Buyer's System Owns the Order . . . 543
E-Invoicing: The Invoice Becomes a Transmitted Legal Document . . . 544
The Mandate Calendar . . . 546

The Choices . . . 549
Across Scale Stages: When It Fits and When It Limits . . . 550
In Practice . . . 552
When It Goes Wrong . . . 553
What to Take Away . . . 554
Questions to Ask . . . 555
Sources and Evidence . . . 555

23. The Back Office: ERP Integration and Who Is the Hub . . . 558
Setting the Scene at Kestrel & Pine: The Connector Nobody Could Read . . . 560
What an ERP Does in a Commerce Business . . . 562
What Flows Between Commerce and the ERP . . . 563
Who Is the Hub? Three Answers and a Non-Answer . . . 567
Batch or Events: The ERP's Rhythm . . . 569
Connectors as Products . . . 570
Master Data: Who May Create and Change What . . . 571
Trade-offs . . . 573
Across Scale Stages: When It Fits and When It Limits . . . 574
In Practice . . . 576
What Never Belongs in the ERP, and the Partner as a Dependency . . . 577
When It Goes Wrong . . . 578
What to Take Away . . . 579
Questions to Ask . . . 580
Sources and Evidence . . . 580

Part VI — INTEGRATION: Making Separate Systems Behave Like One . . . 582
Chapters in this part . . . 585
Sources and Evidence . . . 586

24. APIs, Events, Files and Middleware: How Commerce Systems Talk . . . 587
Setting the Scene at Kestrel & Pine: Thirty-Four Lines Nobody Had Drawn . . . 589
What an Integration Is, and Why the Joins Cost Money . . . 591
The Four Styles: File, Shared Database, Direct Call, Message . . . 592
Five Ways to Trigger an Exchange . . . 593
Point to Point, Hub, or the Vendor's Connector . . . 596
Choosing Per Flow . . . 598
The Trade-Offs . . . 602
Across Scale Stages: When It Fits and When It Limits . . . 604
In Practice . . . 605
The Integration Inventory and Integration Debt . . . 608
When It Goes Wrong . . . 609

What to Take Away . . . 610
Questions to Ask . . . 610
Sources and Evidence . . . 611

25. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation 613
Setting the Scene at Kestrel & Pine: The First Reconciliation . . . 615
Eventual Consistency in Plain Words . . . 617
At Least Once: Why Duplicates Are the Default . . . 618
Idempotency: Making a Repeat Harmless . . . 618
The Outbox: Never Losing the Event Between the Database and the Queue . . . 620
Retries, Backoff and the Dead-Letter Queue . . . 621
Sagas and Compensation: Undoing What Cannot Be Rolled Back . . . 622
Reconciliation as a First-Class Component . . . 623
What Each Flow Needs . . . 626
The Choices and Their Trade-Offs . . . 628
Across Scale Stages: When It Fits and When It Limits . . . 630
In Practice . . . 632
When It Goes Wrong . . . 633
What to Take Away . . . 633
Questions to Ask . . . 634
Sources and Evidence . . . 635

26. Promises Between Systems: API Contracts, Webhooks and Versioning . . . 637
Setting the Scene: The Deprecated Endpoint . . . 639
What a Contract Promises . . . 639
Changing Without Breaking . . . 642
Versioning: Dated, Semantic or in the Media Type . . . 643
Deprecation: A Date Pair, Monitoring and Consent . . . 645
Webhooks Done Well . . . 646
Rate Limits and Partner APIs . . . 648
Who Owns a Contract . . . 649
The Choices and Their Trade-offs . . . 650
Across Scale Stages: When It Fits and When It Limits . . . 651
In Practice: The Contract Card for the bol Stock Flow . . . 652
When It Goes Wrong . . . 654
What to Take Away . . . 655
Questions to Ask . . . 656
Sources and Evidence . . . 656

27. One Customer, One Product: Identity and Master Data Across Systems . . . 659
Setting the Scene at Kestrel & Pine: Four Evas . . . 660
Why One Customer and One Product Matter . . . 662
How the Pieces Fit Together . . . 666
Matching and Merging . . . 670

The Choices . . . 673
Trade-offs . . . 674
Across Scale Stages: When It Fits and When It Limits . . . 676
In Practice . . . 677
When It Goes Wrong . . . 679
What to Take Away . . . 681
Questions to Ask . . . 682
Sources and Evidence . . . 682

28. Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics 684
Setting the Scene: Three Answers to One Question . . . 686
Operational Data and Analytical Data . . . 688
Getting the Data Out . . . 689
One Definition: Metrics and the Metrics Layer . . . 691
What the Data Is For . . . 693
Real-Time Dashboards and When They Mislead . . . 694
The Choices and Their Trade-offs . . . 695
Across Scale Stages: When It Fits and When It Limits . . . 697
In Practice: Waypoint W7 at Kestrel & Pine . . . 698
When It Goes Wrong . . . 702
What to Take Away . . . 703
Questions to Ask . . . 704
Sources and Evidence . . . 704

Part VII — PLATFORM: How Commerce Platforms Are Built . . . 707
Chapters in this part . . . 710
Sources and Evidence . . . 712

29. Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says . . . 713
"Are We Behind?" at Kestrel & Pine . . . 715
Four Styles in Plain Words . . . 716
What the Famous Examples Actually Show . . . 719
What Each Style Costs . . . 723
Modularity Sized to the Team . . . 726
Across Scale Stages: When It Fits and When It Limits . . . 728
In Practice: Noor's Memo . . . 730
When It Goes Wrong . . . 731
What to Take Away . . . 732
Questions to Ask . . . 733
Sources and Evidence . . . 734

30. Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation 737
Two Rate Limits on One Friday Evening 739
Why Platforms Share: Tenancy in Plain Words 739
How a Shared Platform Keeps Tenants Apart 742
Rate Limits: The Tenancy Tool You Meet 744
One Tenant or Many 748
Tenancy in the SaaS You Also Rent 751
The Merchant's Choices and Their Trade-Offs 752
Across Scale Stages: When It Fits and When It Limits 753
In Practice: Kestrel & Pine's API Budget for 2027 755
When It Goes Wrong 756
What to Take Away 757
Questions to Ask 758
Sources and Evidence 758

31. Apps, Plugins and Functions: Extensibility and Ecosystem Governance 761
Setting the Scene at Kestrel & Pine: Forty-One Apps and One Spreadsheet 763
Five Ways to Extend a Platform 764
The App Store: Business Model and Governance 769
What an App May Touch: The Writer Problem 770
The Forked Platform: When the Core Has Been Changed 772
The Choices: Where Custom Behavior Should Live 774
Across Scale Stages: When It Fits and When It Limits 775
In Practice 777
When It Goes Wrong 780
What to Take Away 782
Questions to Ask 783
Sources and Evidence 783

32. Delivering the Storefront: Themes, Headless, Rendering and Performance . . . 786
Setting the Scene at Kestrel & Pine: The Headless Proposal 788
What a Storefront Does, and What It Does Not Own 788
Three Rendering Models 790
Cache the Shell, Fetch the Promise 792
Performance: What the Evidence Says, With Dates 795
The Choices and Their Trade-Offs 797
Across Scale Stages: When It Fits and When It Limits 799
In Practice 801
When It Goes Wrong 803
What to Take Away 805
Questions to Ask 806
Sources and Evidence 807

33. Black Friday Every Year: Peak Load, Resilience and Recovery 809

Setting the Scene at Kestrel & Pine: Reading the Post-Mortem . . . 810
What a Peak Is, and Why the Merchant's Is Worse Than the Platform's . . . 811
The Chain Under Load: Where a Peak Day Breaks . . . 813
Preparing: Capacity, Forecasts and Load Tests . . . 816
Running the Peak: Queues, Backpressure, Caches and Switches . . . 817
The Choices and Their Trade-offs . . . 821
Observing the Day: Orders per Minute as the Health Signal . . . 822
Across Scale Stages: When It Fits and When It Limits . . . 824
In Practice: Kestrel & Pine's 2027 Peak Plan . . . 826
When It Goes Wrong: Recovery and Reconciliation . . . 827
What to Take Away . . . 828
Questions to Ask . . . 829
Sources and Evidence . . . 829

Part VIII — TRUST: Security, Privacy and Regulation . . . 832
Chapters in this part . . . 835
Sources and Evidence . . . 836

34. Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers . . . 837
Setting the Scene at Kestrel & Pine: The Card-Testing Night . . . 839
What Attackers Want From a Shop . . . 840
Shared Responsibility: What the Platform Does and What You Must Do . . . 842
Card Data: Where PCI DSS Draws Its Line . . . 843
Card Testing: The Attack and the Controls . . . 846
Account Takeover, Bots and Scraping . . . 849
Skimmers: The Attack in the Shopper's Browser . . . 849
Admin Accounts, Keys and the Apps . . . 850
The Choices: Four Security Postures . . . 851
Across Scale Stages: When It Fits and When It Limits . . . 852
In Practice: Kestrel & Pine's 2027 Security Baseline . . . 854
When It Goes Wrong . . . 856
What to Take Away . . . 856
Questions to Ask . . . 857
Sources and Evidence . . . 858

35. Whose Data Is It? Privacy, Consent and Retention . . . 860
Setting the Scene at Kestrel & Pine: The Request From Ghent . . . 862
What Personal Data a Commerce Platform Holds, and Under Which Rules . . . 864
Controller, Processor and the Commerce Supply Chain . . . 865
Lawful Bases: The Order Versus the Marketing . . . 867
Consent, Cookies and Tracking . . . 868

Data Subject Requests Across Many Systems . . . 871
Retention: The Person Versus the Transaction . . . 872
Sharing, Transfers and the Store . . . 875
The Choices . . . 875
Across Scale Stages: When It Fits and When It Limits . . . 877
In Practice . . . 878
When It Goes Wrong . . . 879
What to Take Away . . . 881
Questions to Ask . . . 882
Sources and Evidence . . . 882

36. Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation . . . 885
Setting the Scene at Kestrel & Pine: Four Rules Through Four Doors . . . 887
How to Read a Law as a Requirement . . . 888
The Rules, by What They Touch . . . 892
The Regulatory Calendar for 2027 . . . 895
The Long Fuse: Why Deadlines Arrive Late . . . 901
The Choices: Four Ways to Run Compliance . . . 902
Across Scale Stages: When It Fits and When It Limits . . . 904
In Practice . . . 906
When It Goes Wrong . . . 907
What to Take Away . . . 907
Questions to Ask . . . 908
Sources and Evidence . . . 908

Part IX — JOURNEYS: One Order, End to End . . . 911
Chapters in this part . . . 914
Sources and Evidence . . . 915

37. Journey 1: Eva's Jacket, From Search to Doorstep . . . 916
The Order, the Setting and the Rules of This Walk . . . 917
Fifteen Steps in One Table . . . 919
Before the Buy: Search, Product Page and "In Stock" . . . 924
Checkout: Six Promises Under a Clock . . . 925
The Order Becomes a Record: Platform to OMS to WMS . . . 928
In Transit: Tracking, Notifications and "Where Is My Order" . . . 929
The Money and the Books . . . 930
What Each Handoff Requires . . . 931
The Same Order in January 2027 . . . 933
When It Goes Wrong Along This Path . . . 934

What to Take Away . . . 935
Questions to Ask . . . 936
Sources and Evidence . . . 937

38. Journey 2: The Jacket Comes Back . . . 939
The Return, the Setting and the Rules of This Walk . . . 941
Twelve Steps in One Table . . . 942
The Request: Eligibility, Reason and the Choice . . . 948
The Way Back: Label, Drop-off and the Scan That Ends a Right . . . 950
At the Dock: Receiving, Inspection and Disposition . . . 950
The Exchange Order . . . 951
The Refund Alternative: One Writer, One Key . . . 952
The Books: Credit Note, New Invoice, Reconciliation . . . 953
Two Variants: The Marketplace Return and the Store Return . . . 954
What Each Handoff Requires . . . 955
The Same Return in January 2027 . . . 956
When It Goes Wrong Along This Path . . . 957
What to Take Away . . . 958
Questions to Ask . . . 959
Sources and Evidence . . . 960

39. Journey 3: A Chain Reorders by EDI, From Order to Invoice . . . 961
The Order, the Setting and the Rules of This Walk . . . 963
Fifteen Steps in One Table . . . 966
ORDERS In: From Message to Order Record . . . 971
Allocation: Whose Stock Is It? . . . 972
Pick to Pallet, Label and Despatch Advice: The Document That Costs Money . . . 973
Invoice, Terms and the Money Thirty Days Later . . . 974
What Each Handoff Requires . . . 976
The Failure Path: The ASN Chargeback . . . 977
The Variant: A Belgian Shop's Peppol Invoice . . . 979
The Same Order in January 2027 . . . 980
When It Goes Wrong Along This Path . . . 981
What to Take Away . . . 982
Questions to Ask . . . 983
Sources and Evidence . . . 983

40. Journey 4: Black Friday, Hour by Hour . . . 986
The Week Before the Evening . . . 987
Friday 27 November 2026, Hour by Hour . . . 989
Saturday to Monday: Finding Out What Was Sold . . . 995
The 2027 Plan: The Same Evening, Rehearsed . . . 997
When It Goes Wrong in 2027 . . . 1003
What to Take Away . . . 1004

Questions to Ask . . . 1005
Sources and Evidence . . . 1005

Part X — EVOLUTION: Decisions That Change With Scale . . . 1007
Chapters in this part . . . 1010
Sources and Evidence . . . 1012

41. Build, Buy or Assemble: Choosing and Changing Platforms . . . 1013
The 2027 Platform Review at Kestrel & Pine . . . 1014
Four Ways to Cover the Map . . . 1015
What to Build: Differentiation Against Commodity . . . 1019
Total Cost of Ownership: The Lines the Quote Leaves Out . . . 1020
Lock-In and the Cost of Leaving . . . 1027
Decision Criteria . . . 1028
Trade-Offs . . . 1028
Across Scale Stages: When It Fits and When It Limits . . . 1030
In Practice . . . 1032
When It Goes Wrong . . . 1033
What to Take Away . . . 1034
Questions to Ask . . . 1035
Sources and Evidence . . . 1035

42. Moving Without Burning the Bridge: Replatforming and Migration . . . 1038
Setting the Scene: Two Hubs Beside a Running Shop . . . 1040
Reasons to Replatform, and Reasons That Are Not . . . 1040
Three Documented Failures, and What They Share . . . 1042
The Strangler Approach for Commerce . . . 1044
How the Pieces Fit During a Migration . . . 1046
Data Migration Is Five Migrations . . . 1047
Cutover and Rollback . . . 1051
Migrating the Integrations . . . 1052
Across Scale Stages: When It Fits and When It Limits . . . 1052
In Practice: The OMS and PIM Introductions at Kestrel & Pine . . . 1054
When It Goes Wrong . . . 1057
What to Take Away . . . 1058
Questions to Ask . . . 1059
Sources and Evidence . . . 1059

43. What Changes at Each Scale: From First Store to Group . . . 1061
Setting the Scene at Kestrel & Pine: The Stage Map . . . 1063
What a Scale Stage Is, and Is Not . . . 1064
The Map: Four Stages Side by Side . . . 1066

How Fresh Must a Fact Be? Requirements by Stage . . . 1072
Stage by Stage . . . 1073
Multi-Brand and Multi-Country: The Group's Decision Table . . . 1077
The Transition Signals . . . 1078
In Practice: Kestrel & Pine's Path, Priced . . . 1081
When It Goes Wrong: Too Early, Too Late, Sideways . . . 1082
What to Take Away . . . 1082
Questions to Ask . . . 1083
Sources and Evidence . . . 1084

44. Who Builds and Who Owns: Teams, Vendors, Agencies and Governance . . . 1086
Setting the Scene at Kestrel & Pine: The Ownership Map . . . 1088
What Owning Means . . . 1089
The Ownership Map . . . 1091
Team Shapes for a Commerce Merchant . . . 1092
Working With Agencies and Implementation Partners . . . 1095
Vendor Management: What the Contract Must Say . . . 1096
Governance That Fourteen People Will Follow . . . 1099
The Product Manager in a Merchant . . . 1100
Engineering Practices on Rented Software . . . 1100
Across Scale Stages: When It Fits and When It Limits . . . 1101
In Practice: Kestrel & Pine, Before and After . . . 1103
When It Goes Wrong . . . 1104
What to Take Away . . . 1105
Questions to Ask . . . 1106
Sources and Evidence . . . 1107

45. Assistants and Agents: AI in Commerce and What Is Real in 2026 . . . 1109
Setting the Scene at Kestrel & Pine: The Agent Test . . . 1111
Two Kinds of AI, and Why the Difference Is Architectural . . . 1112
What AI Already Does in Commerce . . . 1113
Assistants in the Admin: The Secret Agent Wears a Badge . . . 1114
Agentic Commerce: Fourteen Months, Told in Order . . . 1114
The Protocols in Plain Words . . . 1116
Europe: The Shopper Still Authenticates the Cart . . . 1119
What a Merchant Must Have Before an Agent Can Buy . . . 1120
The Choices and Their Trade-offs . . . 1123
Across Scale Stages: When It Fits and When It Limits . . . 1124
In Practice: Waypoint's W8 at Kestrel & Pine . . . 1126
When It Goes Wrong . . . 1128
What to Take Away . . . 1129
Questions to Ask . . . 1130
Sources and Evidence . . . 1130

Part XI — PRACTICE: How Others Built It 1133

Chapters in this part . . . 1136
Sources and Evidence . . . 1137

46. Case: Shopify, the Modular Monolith That Serves Millions of Merchants . . . 1138
How to Read This Case . . . 1140
The Business: Where the Money Comes From . . . 1140
The Monolith That Stayed: 2019 to 2024 . . . 1142
The Shop as the Unit of Isolation: Pods and Moves . . . 1143
Storefront Delivery . . . 1145
Checkout: Freedom Withdrawn on a Schedule . . . 1146
Payments: Orchestration, Not a License . . . 1147
Inventory States, Briefly . . . 1148
The Ecosystem: Apps, Scopes, Webhooks and Versions . . . 1148
Peak Days With Definitions . . . 1150
AI and the Agent Surface: Rebuilt Within a Year . . . 1152
What Transfers to a Merchant, and What Does Not . . . 1153
Where the Evidence Stops . . . 1155
What to Take Away . . . 1156
Questions to Ask . . . 1157
Sources and Evidence . . . 1158

47. Case: Amazon, From Monolith to Services to the Everything Store . . . 1160
How to Read a Case Study in This Book . . . 1162
2001: The Front End Hits a Wall . . . 1162
The Cart That Never Says No . . . 1164
Teams, Ownership and the Contract as the Only Door . . . 1166
Fulfillment as a Platform . . . 1167
The Promise Is the Product . . . 1169
Other Shops, Other Agents: 2022 to 2026 . . . 1171
A Nuance: When One Process Beat a Distributed Design . . . 1173
What a Merchant Can Learn, and What Only Amazon Can Do . . . 1173
Where the Evidence Stops . . . 1175
What to Take Away . . . 1175
Questions to Ask . . . 1176
Sources and Evidence . . . 1177

48. Case: Zalando, Microservices, Platform Strategy and the Return to Discipline . 1180
2015: Jimmy, Radical Agility and the Split . . . 1182
The Cost of Distributed Data, Stated Early . . . 1184
The API Guild and the Guidelines . . . 1184
The Return to Discipline, 2018 to 2026 . . . 1187
2025: When the Event Stream Fell Behind . . . 1190

The Platform Pieces: What Survived . . . 1191
A Retailer Becoming a Platform . . . 1193
AI at Zalando: Discovery First, Then Identity . . . 1195
What Zalando Teaches . . . 1196
Where the Evidence Stops . . . 1197
What to Take Away . . . 1197
Questions to Ask . . . 1198
Sources and Evidence . . . 1199

49. Case: commercetools, Shopware and the Composable Movement . . . 1202
What Composable Promised, 2020–2023 . . . 1204
commercetools: From Flagship to Layers . . . 1205
Shopware: The Mid-Market Path . . . 1209
Adobe, the Forks and Commerce.com: Composable as a Deployment Option . . . 1211
The Alliance Rewrites Its Principles . . . 1212
What the Movement's Own Data Say . . . 1213
What Composable Actually Costs . . . 1214
When Composable Is Right . . . 1216
What Kestrel & Pine Takes From It . . . 1217
Where the Evidence Stops . . . 1217
What to Take Away . . . 1218
Questions to Ask . . . 1219
Sources and Evidence . . . 1219

Reference . . . 1222

Domain Coverage: From Three Model Families to Forty-Nine Chapters . . . 1223
Where the Models Come From . . . 1224
Coverage by Capability . . . 1227
Coverage by Model . . . 1239
Is Anything Uncovered? . . . 1250
Where the Book Departs From the Models . . . 1256
The Models' Personas and the Book's Readers . . . 1259
How to Use This Map . . . 1266
Sources and Evidence . . . 1266

Toolkit: Cards and Templates for Commerce Architecture Decisions . . . 1268
Find the Template for the Question on Your Desk . . . 1269
How the Records Connect . . . 1271
Tool 1: Capability Map . . . 1272
Tool 2: Entity–Owner Table . . . 1273
Tool 3: Integration Inventory . . . 1275
Tool 4: Integration Contract Card . . . 1276

Tool 5: Reconciliation Card . . . 1277
Tool 6: Availability (ATP) Card . . . 1279
Tool 7: Order State Card . . . 1280
Tool 8: Delivery Promise Card . . . 1281
Tool 9: Channel Allocation Card . . . 1282
Tool 10: EDI Partner Card . . . 1284
Tool 11: App Governance Card . . . 1285
Tool 12: Peak Plan and Degradation Ladder . . . 1287
Tool 13: Regulatory Calendar . . . 1288
Tool 14: Build, Buy or Assemble Decision Card . . . 1290
Tool 15: Migration Plan Card . . . 1291
Tool 16: Architecture Decision Record . . . 1293
Keeping the Cards True . . . 1294
Sources and Evidence . . . 1295

Glossary: Plain-Language Definitions . . . 1296
How to Use This Glossary . . . 1297
Preface — Evidence Labels, Take-Away Tiers and Failure Patterns . . . 1297
Part I — COMMERCE: What a Commerce Platform Is . . . 1303
Part II — CATALOG: Products, Prices and Promises . . . 1311
Part III — ORDER: Cart, Checkout, Payment and the Order Record . . . 1320
Part IV — FULFILLMENT: From Warehouse to Doorstep and Back . . . 1327
Part V — CHANNELS: Selling Everywhere . . . 1335
Part VI — INTEGRATION: Making Separate Systems Behave Like One . . . 1342
Part VII — PLATFORM: How Commerce Platforms Are Built . . . 1352
Part VIII — TRUST: Security, Privacy and Regulation . . . 1358
Part IX — JOURNEYS: One Order, End to End . . . 1362
Part X — EVOLUTION: Decisions That Change With Scale . . . 1363
Part XI — PRACTICE: How Others Built It . . . 1368
Standards, Protocols and Regulations . . . 1371
Kestrel & Pine Names (Fictional) . . . 1386
Alphabetical Index . . . 1396
Where to Go Next . . . 1402
Sources and Evidence . . . 1402

Bibliography: Every Source the Book Cites . . . 1403
How to Read This Register . . . 1404
Topic Index . . . 1407
Commerce and business models . . . 1409
Catalog, pricing and search . . . 1419
Checkout, payments, tax and orders . . . 1428
Fulfillment, shipping, stores, returns and service . . . 1436
Marketplaces, B2B, EDI and ERP . . . 1444

Integration, contracts, identity and data 1451
Platform engineering, tenancy, extensibility, storefront and peak 1459
Security, privacy and regulation 1468
Journeys 1474
Evolution, organization and AI 1475
Company records and case evidence 1483
Sources by Chapter 1491

ORDERED: Architecting Commerce Platforms

Željko Obrenović

AI-assisted: researched and written with Claude Code, images generated with the Gemini API. How this works

How commerce systems fit together, why the architectural decisions behind them matter, and how those decisions evolve as a business grows — from the catalog, the cart and the payment to the warehouse, the stores, the marketplaces and the back office, following one order from discovery to delivery and back. Written for readers new to commerce architecture, platform-neutral, with a fictional running example and dated public evidence. This is a living journal and a work in progress: I revise it as I learn more through my work.

Preface

Introduction: Why Commerce Architecture Matters

IN THIS SECTION, YOU WILL: Learn what this book argues, who it is for, how its eleven parts follow one order, how to find the chapter for your decision, and how to read its evidence.

KEY POINTS:

- **A commerce platform is a promise-keeping machine.** It shows a product and promises a price, "in stock", a delivery date and a return; then a dozen systems owned by different teams and vendors have to keep those promises. It is not one product you buy but the whole set of systems that make and keep promises to a customer.
- **Most commerce failures are a promise one system made and another could not keep.** A storefront sells a jacket the warehouse no longer has; two systems refund the same return; three systems report three statuses for one order. The failures live in the joins between systems, rarely inside one of them.

- **Architecture decides who may make which promise, how the others learn of it, and what happens when they disagree.** The right answer changes with the size and shape of the business, so the book follows one fictional merchant as it grows, compares every decision across four scale stages, and says in each chapter what is established, what depends on context and what is the author's view.

On a Tuesday evening in April 2027, at 21:14, **Eva Janssen** in Nijmegen orders a Ridgewalker shell jacket, women's, size M, in moss, for €199, and two pairs of merino hiking socks at €18 each. She pays €235 by iDEAL, the Dutch bank-transfer payment method. Order **#1072455** is confirmed in under a minute.

In that minute the shop made her **five promises**: this product exists as described; it costs €199; it is in stock; it will arrive on Thursday; and she can send it back if it does not fit. No single system can keep all five. The price lives in one system, the stock count in the warehouse's system, the delivery date depends on the warehouse's pick waves and the carrier's network, the money sits with a payment provider, and the return right runs through a returns portal and, eventually, the finance system. **The page that made the promises owns almost none of the facts behind them.**

Five months earlier, on the busiest evening of 2026, the same shop broke the "in stock" promise 1,340 times. The jacket sold out in the warehouse at 19:40, but the storefronts and marketplaces kept selling it for up to two hours, because each one was looking at a copy of the stock count that was refreshed hourly. Every system did what it had been built to do. **The failure lived between them.**

The shop is **Kestrel & Pine**, a fictional outdoor brand this book follows from its first architecture into its second. Eva's jacket is the recurring order: almost every chapter says where on its way from search to doorstep, and back, the chapter sits. This book is about how the systems behind that order fit together, why the decisions about them matter, and how those decisions change as a business grows.

Who This Book Is For

This book is for people **new to commerce architecture** who need to understand how the systems behind selling fit together before they can make or judge decisions about them. You may have just joined:

- **A retailer, a brand or a wholesaler**, as an engineer, product manager, analyst or engineering lead, and found a dozen systems that nobody drew.
- **A commerce software vendor**, building one of those systems (a warehouse system, a returns portal, a marketplace connector), and need to understand the systems on either side of yours.

- **An agency or a consultancy**, asked to recommend a platform, a migration or an integration.
- **A leadership role**, as a CTO, head of ecommerce or finance or operations director, and need to know which technology questions are really business questions.

What you need to know already. General software literacy: you know roughly what an API (a way for one program to call another), a database, a message queue and a web page are. You do not need to have built a shop.

What you do not need to know. Commerce vocabulary and architecture vocabulary. A **SKU** (stock keeping unit, one sellable variant such as "jacket, women's, M, moss"), **ATP** (available-to-promise, the quantity you may still sell), an **OMS** (order management system), a **WMS** (warehouse management system), a **PSP** (payment service provider), **EDI** (electronic data interchange, structured documents between companies), a **system of record** (the one system whose copy of a fact wins), **idempotency** (handling the same message twice with the same result as once) and **eventual consistency** (copies that agree only after a delay) are each explained the first time they appear in each chapter, and collected in the Glossary: Plain-Language Definitions with the chapter that owns each one.

The book speaks from the position of **an architect who advises growing merchants**. When I give my judgement, I say so in the first person, with my reasons and the conditions under which I would decide differently.

The Argument: A Promise-Keeping Machine

A commerce platform is usually described as a product you subscribe to, license or assemble. This book uses the word differently: **a commerce platform is the whole set of systems that make and keep promises to a customer**: the storefront, the catalog, pricing, promotions, search, inventory, checkout, payments, the order record, tax and invoicing, the warehouse, the carriers, the stores, the returns process, customer service, the marketplaces, the wholesale portal, the EDI links, the ERP (enterprise resource planning, the system that runs finance, purchasing and the books) and the data warehouse.

Seen that way, the platform's job is simple to state and hard to do. **It makes promises in one place and keeps them in another**. The storefront promises; the warehouse, the carrier, the payment provider and the finance system deliver. Between the promise and its keeping sits a handoff, a message from one system to another, and every handoff can be late, lost, duplicated or misread.

That is why the book's central claim is about joins, not components: **most commerce failures are a promise one system made and another could not keep**. The Black Friday oversell was a storefront promising from a two-hour-old copy. Kestrel &

Pine's Three-Status Order, which showed *Delivered* on the platform, *In picking* in the warehouse and *Not invoiced* in the ERP for six days, was three honest systems describing three different objects. Its Double Refund, 212 returns refunded twice, was two systems each entitled to refund the same return.

Architecture, in this book, is the discipline of answering three questions for every promise:

Question	What it means in practice	Where the book answers it first
Who may make this promise?	Which system owns the fact the promise rests on (the stock count, the price, the order's status), so its copy wins when copies disagree	Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record
How do the others learn of it?	Whether they call it, receive an event, read a nightly file or copy it by hand, and how fresh their copy must be	APIs, Events, Files and Middleware: How Commerce Systems Talk
What happens when they disagree?	Which messages can arrive twice or not at all, what a repeat does, and which job compares the copies and repairs them	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation

A one-store brand and a group of brands in six countries need different answers, which is why the book follows one merchant as it grows.

Figure 1: *A commerce platform makes promises in one place and keeps them in another, so most failures happen at a join between systems, not inside one of them.*

The Order's Journey as the Spine

The book is organized around **the order's journey**: the path one order takes from a shopper's search to the doorstep, back again as a return, and into the books. Eva's jacket is that order. Its canonical facts never change: ordered Tuesday 13 April 2027 at 21:14, picked and packed at Kestrel & Pine's Duiven warehouse on Wednesday, delivered by PostNL on Thursday 15 April, and exchanged for a size L through the returns portal on Monday 19 April.

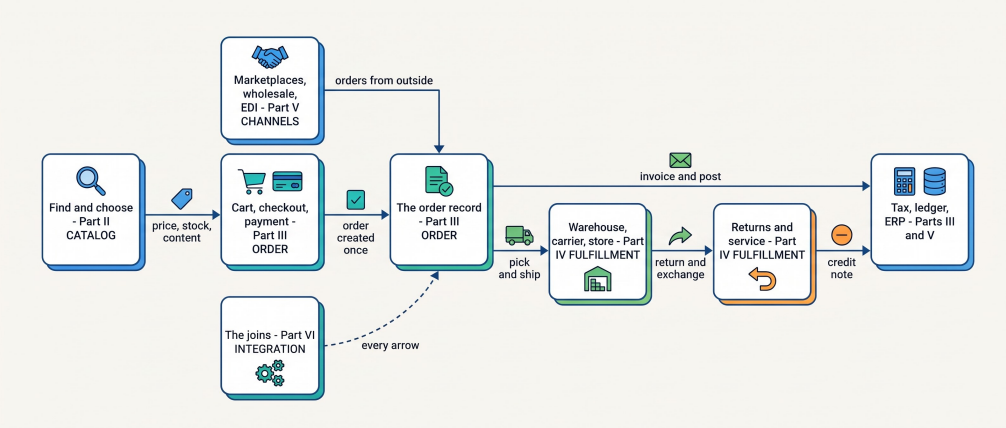


Figure 2: *Eva's order crosses Parts II to V in the order it happens; Part VI covers every arrow between them, and Parts VII to XI step back to look at the platform, the risks, the whole journey and how the decisions change with scale.*

Parts II to V follow the order in the sequence it happens; Part VI is about the arrows between systems. Each chapter has a short paragraph near its top headed "**Where we are in the order's journey**", so you always know which step of Eva's order you are looking at, or how a chapter off the order's path bears on it.

Four Scale Stages, and Why Every Chapter Compares Them

The right architecture for a merchant with one shop and a back room is the wrong one for a group of brands in six countries, and the reverse is just as true. The book therefore compares its decisions across **four scale stages**:

Stage	Rough size	Typical shape	Kestrel & Pine
1 — Starting	under about €2 million revenue	One channel, a hosted platform, the founders running operations, a third-party logistics provider (3PL) or a back room	2016–2018
2 — Growing	about €2–20 million	Two or three channels, a first store or wholesale line, first integrations (accounting or ERP, 3PL, a marketplace), a small team and an agency	2019–2022

Stage	Rough size	Typical shape	Kestrel & Pine
3 — Omnichannel	about €20–200 million	Stores, wholesale, marketplaces and several countries, an own warehouse, a dozen or more systems; the joins become the problem	2023 to now
4 — Group	over about €200 million, or several brands or business units	Several brands, countries or business units; platform teams; shared services against autonomy; acquisitions	not yet

The revenue bands are illustrative markers, not thresholds. **The shape of the business decides the stage**: a €15 million brand with stores, wholesale and two countries already has Omnichannel problems.

Every chapter in Parts II to VIII has a section titled "**Across Scale Stages: When It Fits and When It Limits**", with a table that says, for each stage, what is typical, what becomes necessary and which warning signs show you have outgrown an approach. The most common mistake in commerce architecture is not a bad design but one that was right two stages ago, or will be right two stages from now. Kestrel & Pine's 2026 troubles are mostly Growing-stage designs, such as an hourly stock sync built for fewer channels, still running under Omnichannel load. What Changes at Each Scale: From First Store to Group gathers all the stage tables in one place.

How the Book Is Organized

Three preface pages, eleven parts and four reference pages. Each part asks one question:

Section	The question it answers	Chapters
Preface	What is the book for, how do commerce platforms go wrong, and which merchant do the examples follow?	This page, Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Meet Kestrel & Pine
Part I — COMMERCE: What a Commerce Platform Is	What is a commerce platform, who sells what to whom, and which system owns which fact?	1–5: the order's journey, business models, the capability map, systems of record, the 2026 platform landscape
Part II — CATALOG: Products, Prices and Promises	What does the shop sell, at what price and discount, how do shoppers find it, and is it really in stock?	6–10: product information, pricing, promotions, search, inventory and availability
Part III — ORDER: Cart, Checkout, Payment and the Order Record	How does a cart become a paid order with one record, and how does that order reach the books?	11–14: checkout, payments, order management, tax and invoicing
Part IV — FULFILLMENT: From Warehouse to Doorstep and Back	How is the promise kept in the physical world, and what is the customer told along the way?	15–19: warehouse, shipping, stores, returns, service and notifications
Part V — CHANNELS: Selling Everywhere	How do orders from marketplaces, business customers and trading partners join the same order record?	20–23: marketplaces, B2B, EDI and e-invoicing, the ERP
Part VI — INTEGRATION: Making Separate Systems Behave Like One	How do separate systems talk, recover, keep their contracts, mean the same customer and count the same order?	24–28: integration styles, events and reconciliation, contracts and versioning, identity and master data, analytics

Section	The question it answers	Chapters
Part VII — PLATFORM: How Commerce Platforms Are Built	How are the platforms a merchant rents built, shared, extended, rendered and kept up on the peak day?	29–33: architecture styles, multi-tenancy, extensibility, storefronts, peak load
Part VIII — TRUST: Security, Privacy and Regulation	What do attackers take, whose data is it, and which laws turn into data requirements with a date?	34–36: security, privacy, regulation
Part IX — JOURNEYS: One Order, End to End	What does the whole book look like for one order, one return, one EDI reorder and one Black Friday?	37–40: four journeys, each told on two architectures
Part X — EVOLUTION: Decisions That Change With Scale	Which decisions change with scale: platform choice, migration, stage, ownership and AI?	41–45: build or buy, replatforming, scaling stages, organizing engineering, AI and agents
Part XI — PRACTICE: How Others Built It	What did Shopify, Amazon, Zalando and the composable vendors actually build, and what transfers?	46–49: four dated cases
Reference	Where do the research maps land, which templates help, what does a term mean, and where is the source?	Domain Coverage: From Three Model Families to Forty-Nine Chapters, Toolkit: Cards and Templates for Commerce Architecture Decisions, Glossary: Plain-Language Definitions, Bibliography: Every Source the Book Cites

Part I is the one part not to skip. It introduces the vocabulary, the capability map and the entity–owner table that every later chapter points back to.

Every chapter in Parts II to VIII answers the same five questions: **what problem this solves and why it matters** (business first), **how the components work together**, **which**

choices exist, **what trade-offs they introduce**, and **when the approach fits and when it limits**. It opens with a scene at Kestrel & Pine and ends with "What to Take Away", questions to ask in a design review, and its sources.

The four journeys of Part IX deserve a word. Each one takes a Kestrel & Pine story with fixed dates and amounts, Eva's order (Journey 1: Eva's Jacket, From Search to Doorstep), her exchange (Journey 2: The Jacket Comes Back), the Bergzicht Outdoor chain's spring reorder by EDI (Journey 3: A Chain Reorders by EDI, From Order to Invoice) and Black Friday (Journey 4: Black Friday, Hour by Hour), and runs it twice: through Kestrel & Pine's late-2027 architecture and through its January 2027 one. **The dates do not change; what changes is whether the promises matched them**. If you learn best from a whole example, read a journey early and come back to the chapters it names.

Start With the Decision in Front of You

The parts are a learning path for a first reading. If a decision is already on your desk, find the row that matches it, read the chapter in **Start with**, and follow **Then** if you need more. Return to Part I whenever a term or an owner is unclear.

If you need to decide…	Start with	Then
Whether to replatform, stay, or add systems beside the current platform, and how to move without a big-bang cutover	Build, Buy or Assemble: Choosing and Changing Platforms	Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Moving Without Burning the Bridge: Replatforming and Migration, Case: commercetools, Shopware and the Composable Movement

If you need to decide…	Start with	Then
Why channels keep overselling, and which system should own stock	Is It in Stock? Inventory, Availability and the Promise You Can Keep	Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Journey 4: Black Friday, Hour by Hour
Whether you need an order management system, and what it should own	The Order Record: Lifecycle, State and the Order Management System	Follow the Order: What Happens Between "Buy" and "Delivered", Where Is My Order? Notifications, Service and the Customer Record, Journey 1: Eva's Jacket, From Search to Doorstep
Whether you need a PIM, and who should own product content	What Are We Selling? Products, Variants and the Catalog	One Customer, One Product: Identity and Master Data Across Systems, Selling on Someone Else's Platform: Marketplaces and Channel Integration, Finding the Product: Search, Navigation and Merchandising
Why the same product shows different prices in different channels	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax	Discounts, Coupons and Loyalty: Promotions Without Chaos, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Promises Between Systems: API Contracts, Webhooks and Versioning

If you need to decide…	Start with	Then
How to stop refunds being paid twice or reconciled by hand	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics	Taking the Money: Payment Providers, Authorization, Capture and Refunds, Tax, Invoices and the Books: From Order to Ledger, Journey 2: The Jacket Comes Back
How to connect marketplaces without a second order system	Selling on Someone Else's Platform: Marketplaces and Channel Integration	The Order Record: Lifecycle, State and the Order Management System, Is It in Stock? Inventory, Availability and the Promise You Can Keep, Promises Between Systems: API Contracts, Webhooks and Versioning
How to serve wholesale customers and EDI trading partners, and what role the ERP should play	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals	Documents Between Companies: EDI, Punchout and E-Invoicing, The Back Office: ERP Integration and Who Is the Hub, Journey 3: A Chain Reorders by EDI, From Order to Invoice
How to tame dozens of hand-built integrations with no owners	APIs, Events, Files and Middleware: How Commerce Systems Talk	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Promises Between Systems: API Contracts, Webhooks and Versioning, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance

If you need to decide…	Start with	Then
Whether to go microservices, composable or headless	Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says	Delivering the Storefront: Themes, Headless, Rendering and Performance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline
How to survive the next Black Friday	Black Friday Every Year: Peak Load, Resilience and Recovery	Journey 4: Black Friday, Hour by Hour, Is It in Stock? Inventory, Availability and the Promise You Can Keep, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation
Which apps, scripts or AI agents may write stock, prices or orders	Apps, Plugins and Functions: Extensibility and Ecosystem Governance	Who Builds and Who Owns: Teams, Vendors, Agencies and Governance, Assistants and Agents: AI in Commerce and What Is Real in 2026, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers

One row starts somewhere you might not expect: overselling starts with inventory, not with the storefront or the marketplace connector, because the fix is deciding which system owns stock, not making the copies faster.

How to Read the Evidence

Architecture writing becomes unreliable when facts, habits, guesses, opinions, examples and estimates blur into one voice. Every chapter keeps **six kinds of statement** apart:

Kind of statement	What it is	How to recognize it
Documented fact	A company's public statement, a standard, a regulation, a date, a number from a filing or documentation	Linked inline, with the **year in the sentence** ("Zalando's March 2025 engineering post…"), and listed under Sources and Evidence. Regulations cite the official text and the dates they apply from.
Industry practice	What merchants and vendors commonly do	Phrased as "commonly" or "a typical pattern is", with no invented percentages
Reference-architecture inference	How a system is plausibly built, reasoned from engineering principles	When it concerns a real company, it is labeled **INFERENCE, NOT DOCUMENTED**
Recommendation	My judgement as an architect advising growing merchants	First person ("I would…"), with the reasoning and the conditions under which it does not hold
Illustration	An example set at Kestrel & Pine	Labeled fictional in each chapter; its numbers agree across the book
Assumption	An illustrative volume, latency, cost or percentage that is neither documented nor a Kestrel & Pine fact	Marked "assume…" in the text or *(assumption)* in a table; follow the reasoning, do not use it as a benchmark

Two distinctions are easy to lose. A Kestrel & Pine figure, such as the 24% item return

rate, is an illustration, not an industry benchmark. And a recommendation that sounds like a fact, such as "the warehouse is the system of record for physical stock", is still a recommendation, with reasons and exceptions.

Why claims carry dates. An undated claim quietly goes stale. In **March 2025** Zalando's engineering blog described replacing a price-and-stock event stream, which could fall up to 30 minutes behind during Cyber Week, with a synchronous read API (Zalando Engineering, 2025); a chapter describing the earlier design without its date would teach the opposite lesson (Case: Zalando, Microservices, Platform Strategy and the Return to Discipline). On **3 November 2026** bol announced that its Offers API v10 stops on 1 February 2027 (bol partner platform, 2026), and Shopify releases a new API version every quarter (Shopify API versioning, accessed November 2026). Laws run on a timetable too: the EU rule that a price reduction must be shown against the lowest price of the previous 30 days has applied since 28 May 2022 (Directive (EU) 2019/2161). So every company and regulatory claim carries its date, evidence older than about three years is flagged, and announced features are kept apart from generally available ones.

Three tiers of take-away

Every chapter ends its argument with "**What to Take Away**", split into three tiers, so you can tell how much weight each conclusion carries:

Tier	What it holds	How to use it
Established	What the field agrees on, with the evidence or standard behind it (for example, that a message between systems may arrive more than once, so the receiver must handle repeats)	Treat as a starting point; argue with it only with evidence
Depends on context	Recommendations that hold under stated conditions: a scale stage, a business model, a team size, a vendor	Check the condition against your own business before you adopt it
The author's synthesis	My own position, marked as such, with the reasoning in one clause	Weigh it, disagree with it, and write down why

What the Book Draws On

The book uses three kinds of material, and no others.

- **Public, dated sources.** Filings, engineering posts, developer documentation, standards such as EDIFACT and GS1, legal texts and published research, cited with their year and collected in the Bibliography: Every Source the Book Cites.
- **Verified research briefs.** For each subject area a brief collected the public evidence and tagged each claim by how well it is verified; chapters use only verified or widely reported claims, or claims they checked themselves.
- **Domain models, used as maps only.** Productscape domain models in my sibling repository, structured descriptions of a merchant platform of the Shopify kind, a retail marketplace of the Amazon kind and a set of specialist commerce products, served as research maps of capabilities, roles and data. **They are models inferred from public product behavior, not descriptions of any company**, and nothing in them is used as a fact about Shopify, Amazon or any vendor. Domain Coverage: From Three Model Families to Forty-Nine Chapters maps every capability in them to the chapter that covers it, and lists what the book adds that the models lack, from integration and reconciliation to scale stages and failure patterns.

Vendor-neutral by design. The book recommends architectures, not products. Kestrel & Pine's systems are named by role ("the platform", "the ERP", "the WMS", "the marketplace connector"), never by vendor. Real companies appear as dated cases with their sources: **Shopify** (Case: Shopify, the Modular Monolith That Serves Millions of Merchants), **Amazon** (Case: Amazon, From Monolith to Services to the Everything Store), **Zalando** (Case: Zalando, Microservices, Platform Strategy and the Return to Discipline), and **commercetools and Shopware** with the composable movement around them (Case: commercetools, Shopware and the Composable Movement). They show what was decided, why and at what size; none of them is presented as the answer.

No internal material. Nothing here comes from confidential documents or inside knowledge of any company. Where a chapter needs a situation the public record cannot supply, it uses Kestrel & Pine.

Kestrel & Pine and the Toolkit

Kestrel & Pine is fictional. It is an outdoor apparel and equipment brand from Arnhem, founded in 2016, with **€48 million of revenue in 2026**: €27 million from online storefronts in the Netherlands, Belgium and Germany, €12 million from 1,100 wholesale accounts, €6 million from six stores and €3 million from three marketplaces (bol, Amazon.de and Zalando). It ships from its own warehouse in Duiven and runs thirteen systems joined by 34 integrations, with **no order management system and no**

product information system: orders live in four systems and product data in four. In 2026 that landscape produced ten incidents the chapters reuse, from the Black Friday Oversell (about €96,000) and the Double Refund (€19,400 refunded twice) to the ASN Chargeback (€14,200 deducted by its largest wholesale customer for mislabeled pallets).

In January 2027 **Noor Visser** joins as Head of Engineering. She is the reader's stand-in: experienced in software, new to this merchant's commerce landscape. In March 2027 she launches **Waypoint**, a €1.1 million program that decides ownership first and buys the missing hubs second: the warehouse owns physical stock, an order management system becomes the one order record, a product information system (PIM) owns product content, integrations get owners, contracts and reconciliation, and the company **stays on its hosted platform** rather than replatforming. Its eight decisions, W1 to W8, are canonical; chapters explain them and never change them.

Kestrel & Pine is a composite of patterns common among growing omnichannel brands, not a disguise for any company; Meet Kestrel & Pine introduces it in full. When Noor decides, ask what the equivalent would be in your business.

The **Toolkit: Cards and Templates for Commerce Architecture Decisions** collects the book's working templates, sixteen in all, from a capability map and an entity–owner table to an integration contract card, a reconciliation card, a peak plan and an architecture decision record. Start with the one that changes the decision you face this week. The Glossary: Plain-Language Definitions defines the book's terms and names the chapter that owns each one.

What the Book Leaves to Others

Three neighboring subjects appear only where they touch commerce architecture, and each has a book of its own in this collection:

- **Payments and ledgers in depth**. This book covers taking the money as a merchant does: authorization, capture, refunds, webhooks and the match of orders against payouts (Taking the Money: Payment Providers, Authorization, Capture and Refunds, Tax, Invoices and the Books: From Order to Ledger). How card networks, acquirers and double-entry ledgers work inside is the subject of *RECONCILED*, starting with Payments for Finance Architects: Eleven Reasons the Numbers Disagree.
- **Running a marketplace**. This book covers selling on marketplaces as a merchant (Selling on Someone Else's Platform: Marketplaces and Channel Integration). Operating one, with its two-sided economics, liquidity, ranking and trust, is the subject of *LISTED*, starting with Liquidity Is the Product: Network Effects, Local Markets and Marketplace Metrics.
- **The internals of AI agents**. This book covers what AI assistants and shopping agents do in commerce as of 2026 and what a merchant must have in place before

one touches stock, prices or orders (Assistants and Agents: AI in Commerce and What Is Real in 2026). How agents are built, given tools and kept within bounds is the subject of *Architecting Agentic Systems*, starting with Tool Ecosystems, MCP and the Tool Broker.

The Failure Patterns

Commerce platforms break promises through recurring patterns, each of which looked reasonable when chosen. The next page, Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, names **sixteen** of them in four families:

- **Truth**, a fact with two writers or no owner: **The Phantom Stock**, **Schrödinger's Order**, **The Familiar Stranger**, **The Boomerang** and **The Price Nobody Can Explain**.
- **Integration**, a message with no key, no contract or no owner: **The Spaghetti Junction**, **The Message in a Bottle** and **The Pulled Rug**.
- **Platform and change**, a decision copied from a bigger company or made on one date: **The First Snowfall**, **The Giant's Boots**, **The Burned Bridge**, **The Forked Platform** and **One Size Fits None**.
- **Governance**, a dependency, a deadline or an actor nobody owns: **The Lynchpin**, **The Long Fuse** and **The Secret Agent**.

The patterns travel in chains that usually start at the joins: a Spaghetti Junction of unowned connectors breeds Messages in a Bottle, which breed Phantom Stock and Boomerangs. Kestrel & Pine's Black Friday 2026 met two at once: **The Phantom Stock** in the two-hour-old copy and **The First Snowfall** in a stock chain that met real load for the first time on the day itself. Every chapter opens by naming the patterns it works on, so you can use the list as an index.

A Living Journal

This book is a living journal and a work in progress, which I revise as I learn more through my work. Its **research cut-off is 7 November 2026**: facts were checked against public sources up to that date unless a chapter says otherwise. Time-sensitive facts, such as API versions, program closures, regulatory dates and market positions, carry their date in the sentence; check the source before relying on one. The story at Kestrel & Pine is set in 2027, a year after the cut-off, so where it touches real dates (an API retirement, a law's date of application) it uses the dates published by November 2026.

Every chapter has a "View spec" link in its byline that opens the specification it was written against, including its open questions. Read the book as current thinking with its limits stated, not as a finished text.

I developed the book through AI-mediated authoring: I set the concept, the structure, the chapter contracts and the arguments; AI tools supported research against public sources, drafting from those contracts, consistency checks across chapters and editing; I reviewed and decided. The specs and the scripts that build this site are in the book's GitHub repository. The content, its judgements and its errors are my responsibility.

Where to Go Next

If you read in order, turn next to Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong for the sixteen patterns, then to Meet Kestrel & Pine, so that every chapter's opening scene lands in a company you know, and begin Part I with Follow the Order: What Happens Between "Buy" and "Delivered". If a decision is waiting, use the routing table above and come back to Part I when a term is unclear. Either way, keep Eva's five promises in mind: this jacket, at this price, in stock, on Thursday, returnable. Every chapter is about one of them, and about which system keeps it.

Sources and Evidence

Company, product and investor sources

- From event-driven to API: Zalando's price and stock pipeline, Zalando Engineering, 2025: event delays of up to 30 minutes during Cyber Week and the move to a synchronous read API.
- bol partner platform, bol, 2026: the announcement on 3 November 2026 that Offers API v10 stops on 1 February 2027.
- API versioning, Shopify developer documentation, n.d., accessed 2026-11: quarterly API versions and their support period.

Standards, regulation and official documentation

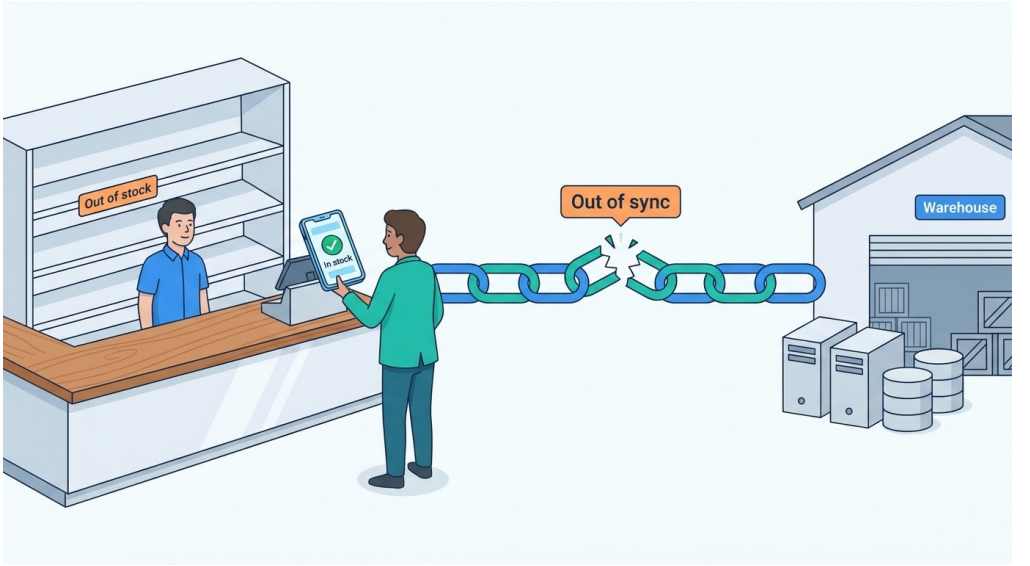
- Directive (EU) 2019/2161 (Omnibus), European Parliament and Council, 2019: the 30-day prior-price rule for price reductions, applied from 28 May 2022.

Research and practitioner sources

- Productscape domain models (author's sibling repository, 2026): research maps of capabilities, roles and data, inferred from public product behavior and mapped in Domain Coverage: From Three Model Families to Forty-Nine Chapters. Not a description of any company's systems.

Company, regulatory and research sources for each topic are listed in the chapter that owns the topic and collected in the Bibliography: Every Source the Book Cites. Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong



IN THIS SECTION, YOU WILL: Learn sixteen named ways commerce platforms break promises, recognize each by its warning sign, see what it costs in public cases and at the book's fictional merchant, and find the chapters that prevent or repair it.

KEY POINTS:

- **Nearly every pattern is a promise one system made and another could not keep.** A storefront promises a unit the warehouse no longer has, a portal promises a refund the payment provider already sent, a page promises a delivery the warehouse's cut-off cannot meet. The sixteen patterns are the recurring ways the promise and its keeper come apart.
- **The patterns fall into four families, and the family tells you where to look.** Truth patterns are a fact with two writers; integration patterns

are a message with no key, no contract or no owner; platform-and-change patterns are a decision copied from a bigger company or made on one date; governance patterns are a dependency, a deadline or an actor nobody owns.

- **The patterns travel in chains, and the chains start at the joins**. A Spaghetti Junction breeds Messages in a Bottle, which breed Phantom Stock, Schrödinger's Orders and Boomerangs; a Lynchpin sets up a Pulled Rug; a Giant's Boots ends as a Forked Platform or a Burned Bridge. Fixing the downstream symptom rarely holds; naming the origin usually does.

The Introduction: Why Commerce Architecture Matters describes a commerce platform as a **promise-keeping machine**: it shows a product, promises a price, a delivery and a return, takes the money, and then has to keep every promise across a dozen systems owned by different teams and vendors. This page names the ways those promises break, so the rest of the book can point back to them.

None of the patterns implies incompetence. **Most are the residue of reasonable decisions**, taken one at a time by people accountable for one step: the manager who installed an app, the agency that wrote a script. The failure is the accumulation, and only someone accountable for the whole landscape sees it.

Each pattern gets a quote (fictional and unattributed), its mechanism and cost, an illustration from **Kestrel & Pine, the fictional outdoor brand this book follows** (see Meet Kestrel & Pine; its incidents are invented), a dated public example where the record supports it, the remedy direction and the chapters that address it. Mapping a public case to a pattern is the author's reading.

Sixteen Patterns to Recognize

The patterns fall into four families: **truth** (five facts with no agreed owner: stock, order status, customer, return, price), **integration** (three), **platform and change** (five decisions about load, shape, upgrade, migration and tenancy) and **governance** (three).

Pattern	Warning sign	Family
The Phantom Stock	Two channels sell the same last unit; "out of stock" cancellations rise in busy weeks.	Truth
Schrödinger's Order	Support checks three systems to say where an order is, and they disagree.	Truth
The Familiar Stranger	Nobody can say whether the store customer and the online customer are the same person, because each system counts customers its own way.	Truth
The Boomerang	Refunds, restocking and credit notes are reconciled by hand after the return arrives, and sometimes twice.	Truth
The Price Nobody Can Explain	The same product carries different prices in different channels, and nobody can say which system set which price or why.	Truth
The Spaghetti Junction	Every pair of systems, sometimes every customer, has its own hand-built connector with its own owner, or none.	Integration
The Message in a Bottle	Messages between systems are sent once and trusted; customers find the duplicates and the gaps.	Integration
The Pulled Rug	A vendor's API or a partner's document format changes and an integration breaks before anyone is told.	Integration

Pattern	Warning sign	Family
The First Snowfall	The busiest day of the year is the first time the system, and its integrations, meet real load.	Platform and change
The Giant's Boots	An architecture is copied from a company a hundred times larger, or from a trend, without its reasons.	Platform and change
The Burned Bridge	Everything moves to a new platform on one date, with no way back.	Platform and change
The Forked Platform	The platform's core or theme has been customized so deeply that it can no longer be upgraded, and every vendor release is a project.	Platform and change
One Size Fits None	Several brands or business units are forced onto one stack before anyone has measured the benefit, or kept apart when they should share.	Platform and change
The Lynchpin	A capability customers depend on runs through one vendor, one connector, one agency or one person, with no exit plan.	Governance
The Long Fuse	A legal deadline (e-invoicing, accessibility, consumer rules) is discovered when it is months away.	Governance

Pattern	Warning sign	Family
The Secret Agent	A script, app, bot or AI agent changes orders, stock, prices or money without a named owner, its own identity or a person's approval.	Governance

A warning sign is a prompt to look, not a verdict: **ask for the number that would confirm it**.

The Phantom Stock

"The site said we had twelve. The warehouse had none since seven o'clock."

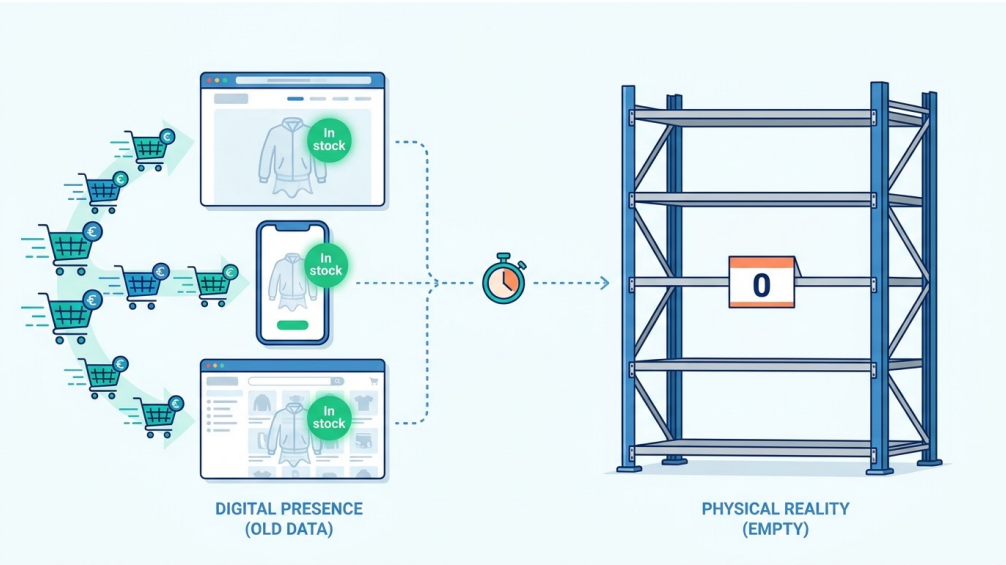


Figure 1: *The Phantom Stock: every channel sells from an aging copy of a stock number, so the shelf can be empty while the screens still say "in stock".*

What it looks like. The storefront's stock number is a copy of unknown age; click-and-collect promises what the store sold that afternoon.

Why it happens. The system that knows stock is rarely the one that sells it, so every channel gets a copy through syncs nobody designed as a whole, and the copy grows writers (an app, an import). **A fact with two writers has no owner**.

What it costs. Canceled orders, vouchers, marketplace penalties and seller ratings. Stock records are less accurate than assumed: the classic field study found 65% of about

370,000 inventory records inaccurate at one US retailer's stores (DeHoratius and Raman, *Management Science*, 2008; dated, still the field's reference). Even commercetools documents that its available quantity is eventually consistent "with a delay of up to 10 seconds" (n.d., accessed 2026-11).

At Kestrel & Pine (fictional). On Friday 27 November 2026 the Ridgewalker jacket sold out at the Duiven warehouse at 19:40; the storefronts and the marketplaces kept selling it for up to two hours, because stock traveled WMS → ERP every 15 minutes, ERP → platform hourly, platform → marketplace connector hourly. **1,340 units** oversold, **1,120 orders** canceled or delayed, about **€96,000** including goodwill vouchers and marketplace penalties. In December the Utrecht click-and-collect pilot promised **37** items the store had already sold, because store stock reached the ERP only nightly.

In public. Shopify's August 2026 developer changelog moved draft-order and transfer holds from its `reserved` to its `committed` inventory state (5 August 2026): the stock vocabulary changes under the apps that read it.

Remedy direction. One writer per location's stock, changes published as events, available-to-promise computed in one place, copies reconciled so drift is measured (Kestrel & Pine's W1).

Where this book helps: Is It in Stock? Inventory, Availability and the Promise You Can Keep owns available-to-promise, reservations and buffers; Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record names the owner per fact; Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation measures the drift.

Schrödinger's Order

"The customer sees Delivered. The warehouse says In picking. Finance says it was never invoiced."

Figure 2: *Schrödinger's Order: each system keeps its own status for the same order, and each is honest and wrong at once.*

What it looks like. The platform's status is set by the first message it received; the ERP waits for one that never came; a split shipment has no status at all.

Why it happens. The money, each shipment and each return have their own state. Without one record that keeps them apart, every system's status is honest and wrong.

What it costs. Service load first: "where is my order" (WISMO) is the largest single category of e-commerce support contacts, at 18% of incoming requests in one helpdesk vendor's data (Gorgias, updated 13 November 2023). Then money: invoices that never post and duplicate shipments.

At Kestrel & Pine (fictional). Order **#1052877** in December 2026, a fleece and a pair of socks: the fleece shipped on 7 December and the socks on 14 December after replenishment. The platform showed *Delivered* (the agency's flow had marked the whole order fulfilled on the first warehouse confirmation), the WMS showed *In picking* and the ERP *Not invoiced*, for six days, while one customer opened **11 support tickets**.

In public. Shopify's help documentation keeps separate order, payment and fulfillment status families on one order, and states that without a tracking number "no shipping updates are received" (n.d., accessed 2026-11).

Remedy direction. One order record for every channel, with separate states for money, each shipment and returns, and a summary computed from the least-advanced shipment (Kestrel & Pine's W2).

Where this book helps: The Order Record: Lifecycle, State and the Order Management System owns the order record and its state model; Where Is My Order? Notifications,

Service and the Customer Record builds the service view; The Back Office: ERP Integration and Who Is the Hub stops the ERP from being a second order system.

The Familiar Stranger

"Is the Eva who bought a jacket online the Eva on the store loyalty list? Probably."

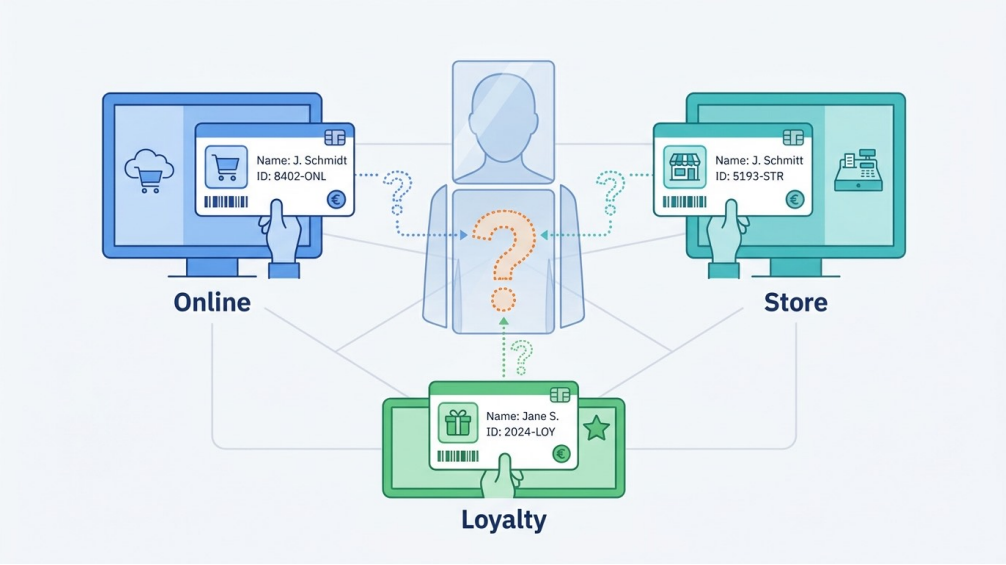


Figure 3: *The Familiar Stranger: each channel keeps its own customer record, so one person becomes several strangers nobody can match.*

What it looks like. A store cannot see an online order; a data-subject request becomes a search across nine systems.

Why it happens. Each channel's system has its own customer table, keyed on what it collects. Matching them is a project nobody owns, because every system works without it.

What it costs. Duplicate marketing, blind service, and loyalty built on a wrong count. Under the GDPR a data-subject request must be answered within one month, extendable by two (Regulation (EU) 2016/679, Article 12(3), 2016); a merchant that cannot find the person in all its systems cannot meet it.

At Kestrel & Pine (fictional). Eva Janssen, the recurring order's customer, exists as an online account, a store customer in the POS loyalty list, a newsletter subscriber and a returns-portal contact, with no shared identifier. When she returns her jacket to the Nijmegen store, the store cannot see that she is the online Eva. In 2027 the ecommerce manager and the head of engineering agreed not to launch an online loyalty program until the customer record is unified.

In public. Shopify's developer documentation marks its legacy customer accounts deprecated as of 26 February 2026 (accessed 2026-11): identity is a model to rebuild, not a table to keep.

Remedy direction. A customer master with one identifier per person, written matching rules, channel records linked to it, privacy requests served from it. Loyalty comes after the match.

Where this book helps: One Customer, One Product: Identity and Master Data Across Systems owns the customer master and matching; Whose Data Is It? Privacy, Consent and Retention serves data-subject requests across systems; Discounts, Coupons and Loyalty: Promotions Without Chaos explains why loyalty waits for it.

The Boomerang

"We refunded it. Then the portal refunded it. The customer kept both, and who can blame her."

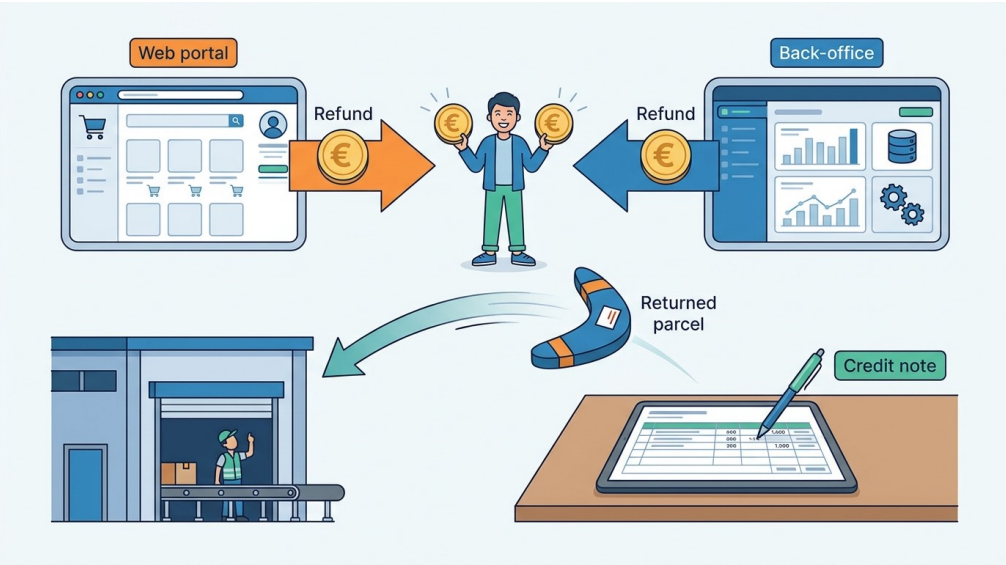


Figure 4: *The Boomerang: a return touches goods, money and books with three owners, so the refund can be paid twice and the credit note made by hand.*

What it looks like. Two systems can issue a refund; stock rises at the dock scan; credit notes come from a spreadsheet.

Why it happens. A return touches goods, money and books, with three owners. The way back was assembled from parts, and nothing says which event releases the refund.

What it costs. The NRF put the 2025 US online return rate at 19.3% of online sales, with 9% of all returns fraudulent (October 2025). Stripe notes a "risk of double refund" for

bank-debit methods, and Mollie "blocks duplicate refunds made in a short time frame" (both n.d., accessed 2026-11).

At Kestrel & Pine (fictional). Over three weeks in January 2027, **212 returns** were refunded twice: once by the returns portal through the platform's API and once by hand by support in the payment provider's console; **€19,400** went out twice, most of it recovered by asking customers to transfer it back. In the Black Friday week before, **190** returned jackets were refunded at €159.20 instead of the €143.28 actually paid, about €3,000, because the order-level discount had never been allocated to lines in the order export.

In public. Shopify's returns documentation separates the *return* (the buyer's intent and the money) from the *reverse fulfillment order* (the warehouse's work) and names four dispositions, restocked, processing required, not restocked and missing (n.d., accessed 2026-11): money and stock are two records.

Remedy direction. One return record; the warehouse's disposition as the single trigger for restock, refund and credit note; one system that refunds, keyed on the return id; a daily refund match. Kestrel & Pine's February 2027 fix.

Where this book helps: The Way Back: Returns, Exchanges, Refunds and Reverse Logistics owns the return record, dispositions and refund reconciliation; Taking the Money: Payment Providers, Authorization, Capture and Refunds owns refunds and keys; Tax, Invoices and the Books: From Order to Ledger books credit notes.

The Price Nobody Can Explain

"Germany is at 179. Everyone else is at 199. Which one did we mean?"

Figure 5: *The Price Nobody Can Explain: prices live in several systems with their own calendars and imports, so the same product carries several prices and nobody can say why.*

What it looks like. Prices live in five systems with five campaign calendars; discounts stack unconfigured; a refund uses a price nobody paid.

Why it happens. Several roles set prices under several rules, each selling system keeps its own number, imports fail silently, and promotion settings outlive their campaign.

What it costs. Margin, parity penalties and regulatory exposure: since 28 May 2022 the EU's Omnibus rules require any price-reduction announcement to show the "prior price", the lowest price in at least the 30 days before the reduction (Directive (EU) 2019/2161, 2019), and the Dutch regulator ACM announced supervision of those rules from 1 January 2023 (6 December 2022).

At Kestrel & Pine (fictional). In October 2026 the German storefront sold the Ridgewalker jacket at **€179** for **nine days** while the marketplace listing and the stores charged €199: the restore row of a campaign import had failed in an agency script that logged the error and reported success. **410 orders**, about **€8,200** of margin lost, a price-parity warning from a marketplace. In Black Friday week a 20% outerwear discount stacked with a 10% welcome code on **2,860** orders because a combination setting from an October campaign had never been reverted: about **€52,000**.

In public. The Omnibus rule (2022) and ACM's supervision (2023) make a 30-day price history per SKU and channel a legal requirement.

Remedy direction. One owner per price kind, price events with effective dates, campaigns as data, a 30-day price history, promotions with an evaluation order and stop rule, daily reconciliation, prices frozen on the order.

Where this book helps: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax owns price ownership, price lists and the price history; Discounts, Coupons and Loyalty: Promotions Without Chaos owns stacking rules and line-level allocation; Selling on Someone Else's Platform: Marketplaces and Channel Integration handles parity and channel price rules.

The Spaghetti Junction

"Thirty-four integrations. Thirteen have no owner. Six of them write stock."

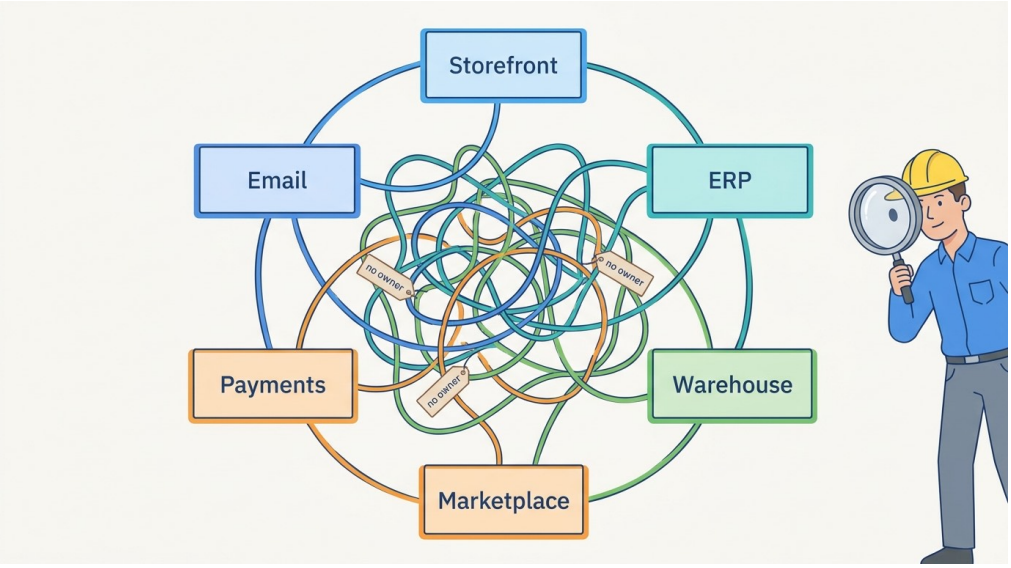


Figure 6: *The Spaghetti Junction: each connector was the fastest way to meet one need, until nobody can list them or say who owns them.*

What it looks like. Flows on the integration platform, agency scripts and vendor plugins move the same fact along several paths. Nobody can produce the list.

Why it happens. Each connector was the fastest way to meet one need. Point-to-point is cheap the first ten times.

What it costs. Undiagnosable incidents, unplannable upgrades, and money: the MACH Alliance's own February 2026 survey of 600 enterprise technology decision-makers found 37% citing integration complexity as a primary concern (18 February 2026).

At Kestrel & Pine (fictional). The February 2027 integration inventory counted **34 integrations**: 14 on the iPaaS, 9 agency scripts, 11 vendor-built connectors or plugins; **13 had no named owner**; only 3 had a reconciliation job, none for stock or orders; **four flows wrote stock** into the platform, and the head of engineering's capability map

found **six writers** of Duiven stock in all. The pre-order app that wrote inventory beside the WMS feed (March 2026) is the pattern at its smallest.

In public. Zalando's engineering blog described how its developer platform, built on Backstage, replaced "more than 100 disconnected interfaces" with one portal holding over 40,000 registered entities (3 August 2023).

Remedy direction. An integration inventory with an owner and a contract per flow, shared facts moving as events through one place, a reconciliation for every copy (Kestrel & Pine's W5).

Where this book helps: APIs, Events, Files and Middleware: How Commerce Systems Talk owns the styles and the inventory; The Back Office: ERP Integration and Who Is the Hub untangles the ERP's flows; Who Builds and Who Owns: Teams, Vendors, Agencies and Governance assigns the owners.

The Message in a Bottle

"We sent the order to the warehouse. If it didn't arrive, that's on them."

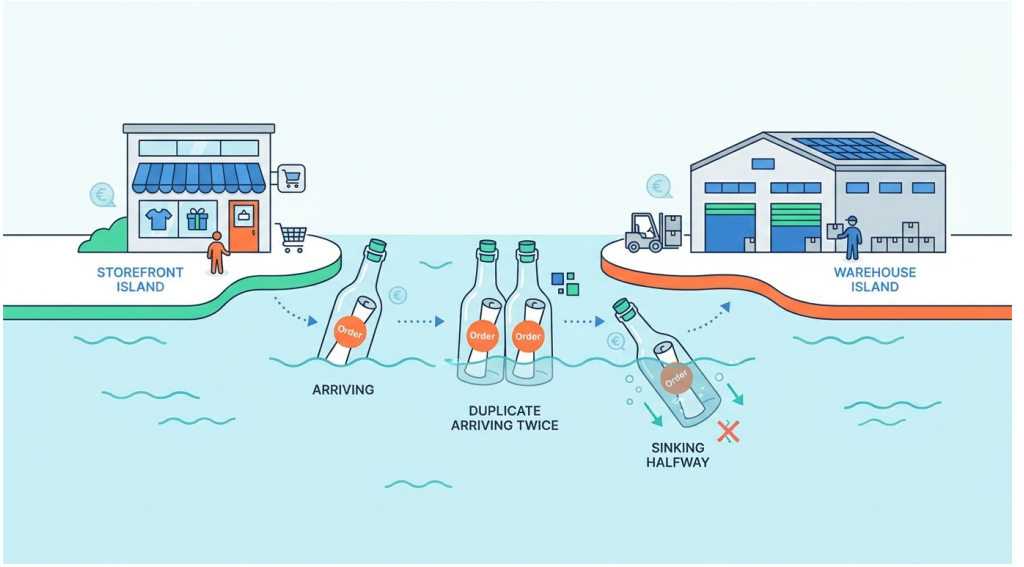


Figure 7: *The Message in a Bottle: messages between systems arrive once, twice, late or never, and a design that trusts them to arrive once finds the duplicates and gaps through customers.*

What it looks like. A retry creates a second order or refund; a dropped message leaves an order paid and never picked.

Why it happens. Delivery is at least once, sometimes twice, late or never, and vendors say so. Shopify's webhook documentation says an app "might receive the same webhook

more than once", and retries an unanswered webhook 8 times over 4 hours before deleting the subscription; Stripe retries for up to three days; Mollie calls a webhook ten times over 26 hours and then stops (all n.d., accessed 2026-11). Integrations assume exactly once, because on a quiet day it holds.

What it costs. Duplicates, lost orders, drifting copies and peak backlogs. Zalando's engineering blog reported that its event-driven price and stock pipeline, reprocessing payloads of which "over 90%" were unchanged, delayed events "by up to 30 minutes" during Cyber Week and left consumers with "divergent versions of product data", before it was replaced by a synchronous read API for the critical path (7 March 2025).

At Kestrel & Pine (fictional). On Black Friday 2026 the iPaaS flow that created orders in the WMS one at a time was throttled by the WMS vendor's rate limit; the queue backed up for **four hours**, and roughly **8,000 orders** missed the next-day delivery the storefront had promised.

In public. Stripe's fulfillment guide: "You can't rely on triggering fulfillment only from your checkout landing page, because it's not guaranteed customers visit that page" (n.d., accessed 2026-11).

Remedy direction. An idempotency key from the business event on every message, a record of handled events, state fetched from its owner, a dead-letter queue, a daily reconciliation.

Where this book helps: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation owns idempotency, outbox, dead letters and reconciliation; APIs, Events, Files and Middleware: How Commerce Systems Talk chooses the style; Taking the Money: Payment Providers, Authorization, Capture and Refunds applies the rules to money.

The Pulled Rug

"They announced it in November. The e-mail went to someone who left in 2025."

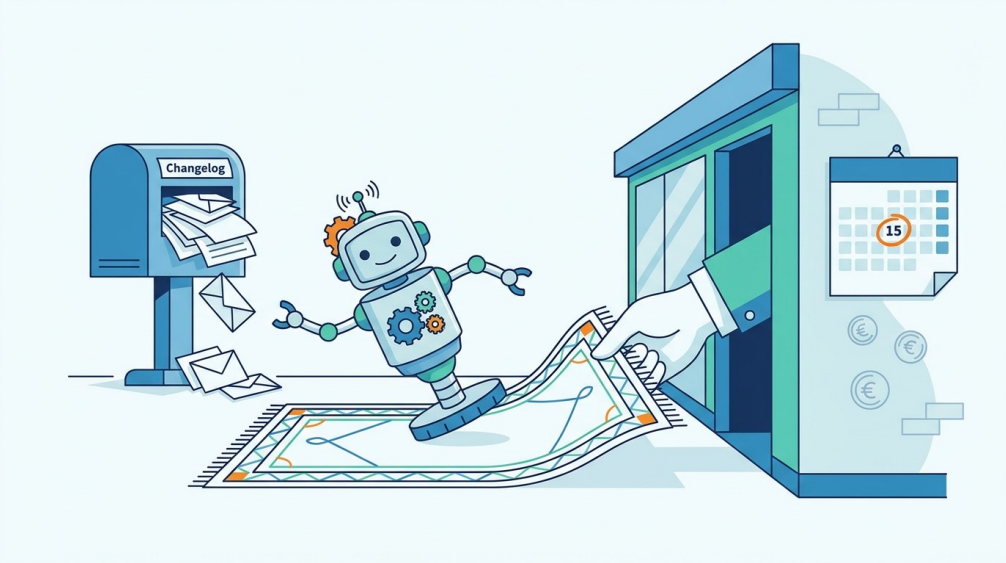


Figure 8: *The Pulled Rug: vendors announce their changes, but without a record of the dependency and someone reading the changelog, the integration breaks first and the news arrives later.*

What it looks like. A script keeps calling a retired version; chargebacks arrive before the new mapping; changelogs reach a mailbox nobody reads.

Why it happens. Vendors announce changes, but the merchant has no record of the dependency, no reader and no test. The contract exists on one side only.

What it costs. Stale stock, seller-account strikes, chargebacks, emergency rewrites. The calendars are public: Amazon's Marketplace Web Service was "no longer available" from 1 April 2024, with all private developers told to migrate to the Selling Partner API by 31 March 2024 (SP-API changelog, 2023–2024); bol's partner platform announced on 3 November 2026 that version 10 of its Offers API would be switched off on 1 February 2027.

At Kestrel & Pine (fictional). On 8 February 2027 the marketplace connector retired the API version a 2021 agency script used to push stock for the bol listings, announced in November 2026 to a contact who had left the agency in 2025. The script logged errors and kept running; stock to bol was stale for **two days**. In Q3 2026 Bergzicht Outdoor, the wholesale chain, deducted **€14,200** for despatch advices whose pallet labels did not match what arrived.

In public. Shopify's API versioning policy, quarterly releases with each version supported for at least 12 months and a nine-month overlap (n.d., accessed 2026-11), is one calendar among many.

Remedy direction. Pinned versions and sunset dates per integration, a role mailbox for changelogs, partner cards with owners, contract tests, despatch advices generated from

what was packed.

Where this book helps: Promises Between Systems: API Contracts, Webhooks and Versioning owns versions, sunsets and contract tests; Documents Between Companies: EDI, Punchout and E-Invoicing owns the partner card and the despatch advice.

The First Snowfall

"It worked fine all year. Then Friday happened."

Figure 9: *The First Snowfall: the busiest day is the first time the system and its integrations meet real load, and it is usually the joins, not the storefront, that give way.*

What it looks like. The storefront holds; the stock sync and the order queue do not.

Why it happens. Peak is 20 to 40 times an ordinary hour, and every design decision was made against the ordinary hour. Load tests, when they exist, test the storefront and not the joins.

What it costs. Oversells and missed promises on the day the brand is measured. Target's new site, launched in 2011 after it left Amazon's platform, crashed on 13 September 2011 under the Missoni launch and was "down most of the day", with a second 2.5-hour outage on 25 October (MPR News, 2011). Shopify's 2025 readiness post describes tests at 150% of the previous year's peak, reaching 200 million requests a minute (20 November 2025).

At Kestrel & Pine (fictional). Black Friday week 2026, 23–29 November: **31,000** online orders, **10.6%** of the year's; a peak hour on Friday of **2,300 orders**, 38 a minute, with storefront traffic about **41×** an ordinary hour. The platform held. The stock sync chain

and the WMS connector queue did not: the oversell above, a four-hour backlog, 8,000 late orders, a stacking bug worth about €52,000. Kestrel & Pine had no kill switch to disable guest card payments without a deploy.

In public. Shopify's 2025 Black Friday weekend peaked at $5.1 million a minute (2 December 2025); the merchant's own connectors are the merchant's to size.

Remedy direction. Test the joins at a multiple of last year's peak; switches to degrade without a deploy; a dated change freeze; a named runbook.

Where this book helps: Black Friday Every Year: Peak Load, Resilience and Recovery owns the peak plan, degradation and switches; Journey 4: Black Friday, Hour by Hour walks the 2026 Friday hour by hour and the 2027 plan; Pick, Pack, Ship: Warehouse Management and Fulfillment sizes the order queue.

The Giant's Boots

"Amazon does it with microservices. Zalando does it with 4,000 applications. So should we."

Figure 10: *The Giant's Boots: copying the architecture of a company a hundred times larger brings its costs without its reasons.*

What it looks like. A 14-person team runs a dozen composable vendors; a €48 million merchant builds a headless storefront. The giant's reasons are absent; the costs are not.

Why it happens. The giants publish good work, and vendors sell the shape that produces the most work. "Do we have that problem" is easy to skip.

What it costs. Cost and slower change without the benefit. The reversals are public: Segment described in 2018 growing to over 140 services, three engineers spending most of their time keeping them running, and consolidating back into one (10 July 2018); an Amazon Prime Video team reported in 2023 that a serverless monitoring design supported about 5% of expected load and that one process cut cost by 90% (InfoQ, 3 May 2023; the original post has been removed).

At Kestrel & Pine (fictional). In February 2027 the head of engineering received two pitches citing the Black Friday Oversell: an outside consultancy's composable rebuild replacing the platform (about **€3.2 million** over three years) and the agency's headless rebuild of the three storefronts on the current platform (about **€240,000** and nine months, plus a second runtime to operate). She declined both after measuring. Waypoint's W4, stay on the SaaS platform and surround it, is the decision not to wear the boots.

In public. Zalando's 2025 replacement of an event stream with a synchronous read API on its critical path (7 March 2025) is a giant narrowing a choice others copied from it.

Remedy direction. Count channels, warehouses, countries, teams and facts with two writers; add a capability when a measured problem demands it.

Where this book helps: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says compares the styles with their reasons; Build, Buy or Assemble: Choosing and Changing Platforms works the stay-and-surround decision; What Changes at Each Scale: From First Store to Group gives the stage test.

The Burned Bridge

"Go-live is 19 November. The old platform is switched off the same day."

Figure 11: *The Burned Bridge: moving everything on one date with the old platform switched off leaves no way back when something goes wrong.*

What it looks like. Everything moves at once; the old platform is switched off to save a month's subscription; a contract sets the date, weeks before peak.

Why it happens. A big bang looks cheaper than running two systems, and each risk was fine in the demo.

What it costs. Lost sales, sometimes a reversal. Finish Line switched to new e-commerce site technology on 19 November 2012 and reverted to the former platform, "which it had kept operable", 17 days later, at a cost of about $3 million in quarterly sales (Retail Dive, 8 January 2013; dated).

At Kestrel & Pine (fictional). Waypoint replaced nothing on one date. The OMS came in seam by seam in 2027: a shadow copy of all orders in August, a parallel run on 1–14 September, marketplace orders on 15 September, D2C orders on 13 October, a change freeze around Black Friday, wholesale orders on 8 December, the old paths retired in January 2028. The PIM went live one storefront at a time.

In public. Target's 2011 launch weeks before its holiday season, and its two outages (MPR News, 2011), is a date chosen against the calendar.

Remedy direction. Migrate by seams, keep the old path operable until the new one has run a peak, run in parallel, rehearse rollback, design exit in at purchase.

Where this book helps: Moving Without Burning the Bridge: Replatforming and Migration owns seams, parallel runs and rollback; Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says explains the strangler pattern; Case: commercetools, Shopware and the Composable Movement reads the public replatforming record.

The Forked Platform

"We can't take the upgrade. The theme and the checkout changes would all have to be redone."

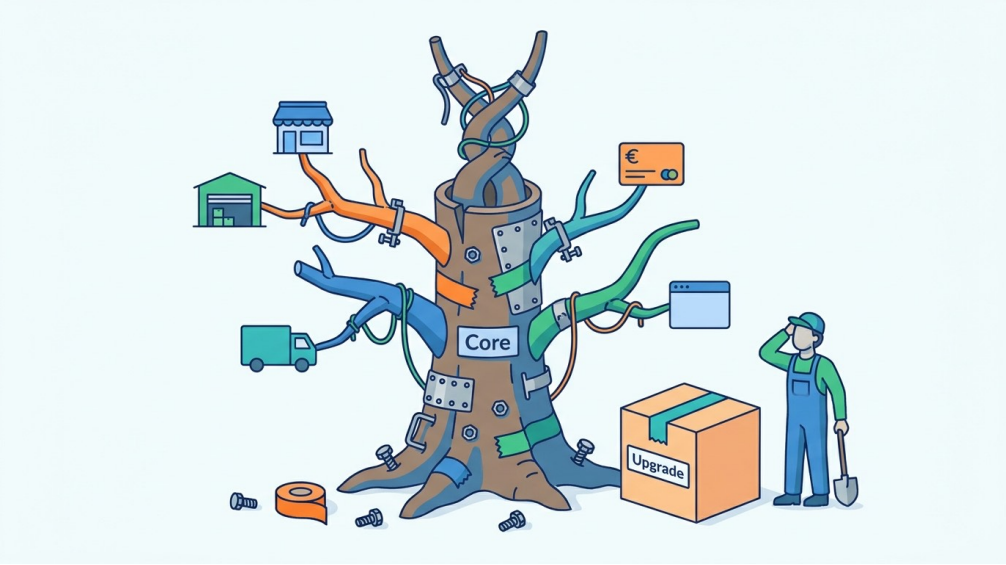


Figure 12: *The Forked Platform: customize the core deeply enough and every vendor upgrade becomes a project.*

What it looks like. Agency edits to files the vendor has since replaced; scripts inside the checkout; apps nobody uses.

Why it happens. Each customization was the fastest route, and nobody owns upgradeability.

What it costs. Lost features, security exposure, a forced migration on the vendor's date. Adobe Commerce gives each release line three years of standard support, and from 1 June 2027 cloud environments on unsupported versions "may be taken offline" (lifecycle policy, accessed 2026-11). Shopify's `checkout.liquid` customizations stopped working on 13 August 2024 (announced 13 February 2023), and Shopify Scripts stopped executing on 30 June 2026 (changelog, 9 April 2026).

At Kestrel & Pine (fictional). The storefront theme has been customized by the agency since 2017; the checkout has not, which is one reason the decision to stay on the platform (W4) is possible. The **41 installed apps**, of which 22 were installed by the agency, 6 are unused and 3 wrote inventory, are the smaller fork: the April 2027 app governance rule gives every app a named owner and takes away the right to write stock.

In public. The sunset calendars above.

Remedy direction. Upgradeability with an owner; supported extension points only; a register of customizations and apps; upgrades budgeted as routine.

Where this book helps: Apps, Plugins and Functions: Extensibility and Ecosystem Governance owns extension points, app governance and the register; Delivering the Storefront: Themes, Headless, Rendering and Performance separates theme from core; From Cart to Confirmation: Checkout Orchestration keeps the checkout unforked.

One Size Fits None

"All three brands go on the group platform next year. The business case is 'synergies'."

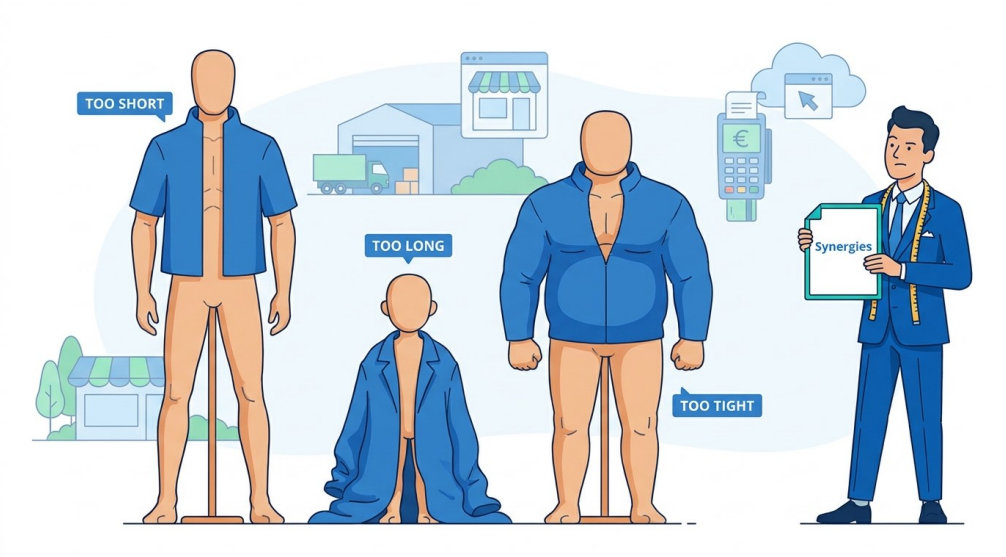


Figure 13: *One Size Fits None: forcing different brands onto one stack before anyone measures the benefit, or keeping them apart when they should share, decides on cost and misses the operational differences.*

What it looks like. One tenant serves brands with different catalogs, rules and peaks, and every change is a negotiation; or two brands run two of everything because nobody measured what sharing would save.

Why it happens. The decision is made on cost, and the operational differences are invisible in the business case.

What it costs. Sharing: slower change, one brand's peak degrading another. Separation: duplicated contracts and integrations.

At Kestrel & Pine (fictional, illustrative scene). Kestrel & Pine runs its three storefronts in **one tenant** of its platform, which works because they share a catalog,

a warehouse, a season and a price (€199 for the jacket in all three countries). The April 2027 stage map names the signals that would change the answer, a second brand, a fourth country or revenue past about €200 million, none planned for 2027–2028; what a second brand would share is deferred to the 2029 review on purpose, to be answered with measurements.

In public. Zalando completed its acquisition of ABOUT YOU on 11 July 2025 as a dual-brand consumer business "with shared logistics and payment infrastructure", with group synergies targeted at around €100 million a year (corporate.zalando.com, 11 July 2025).

Remedy direction. Decide sharing per capability: share what has scale economics and the same rules (payments, logistics), keep apart what differs by brand, and write the benefit as a number first.

Where this book helps: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation owns tenancy and isolation; What Changes at Each Scale: From First Store to Group names the Group-stage signals; Case: Zalando, Microservices, Platform Strategy and the Return to Discipline reads a group's sharing decisions.

The Lynchpin

"Only the EDI consultant understands the mappings. He's on holiday."

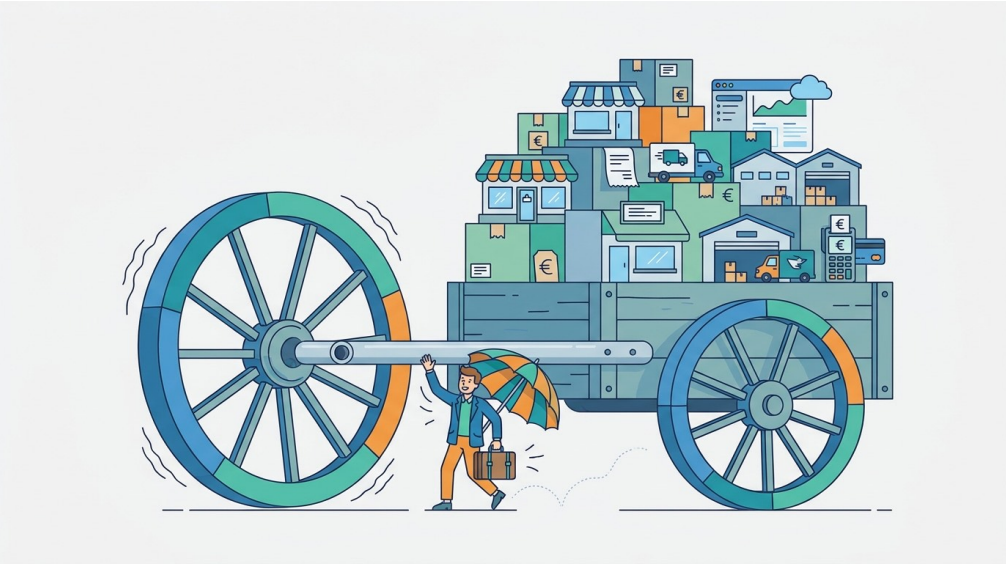


Figure 14: *The Lynchpin: when a capability customers rely on runs through one vendor, connector, agency or person, the whole cart depends on one pin.*

What it looks like. A connector documented nowhere; mappings in one consultant's head.

Why it happens. Specialists are efficient, and an exit plan has no visible return until the day it is needed.

What it costs. Outages as long as one person's absence, and programs that end on the vendor's date: Shopify sold its logistics business to Flexport in a deal that closed on 6 June 2023, and Zalando's Connected Retail program, through which physical shops sold branch stock on its platform, ends by the end of June 2027 (TextilWirtschaft, 24 April 2026).

At Kestrel & Pine (fictional). The EDI mappings for Bergzicht Outdoor, worth €2.1 million a year, were built in 2025 by one external consultant; when despatch advices and pallet labels disagreed, the fix waited on him. The ERP partner's platform-to-ERP connector was undocumented and could be retired only after the lead integration engineer wrote its contract in Q3 2027. The February 2027 ownership map found **5 integrations with no named owner**, among them the scripts behind the Deprecated Endpoint and the Price Drift.

In public. Connected Retail's end (announced April 2026, effective June 2027) is a channel retired by the platform that owned it.

Remedy direction. An ownership map for every system, integration and app; contracts in the merchant's hands; a second person per mapping; exit cost assessed at purchase.

Where this book helps: Who Builds and Who Owns: Teams, Vendors, Agencies and Governance owns the ownership map; Build, Buy or Assemble: Choosing and Changing Platforms prices exit at purchase; Documents Between Companies: EDI, Punchout and E-Invoicing puts the partner knowledge on a card.

The Long Fuse

"Belgium wants structured invoices from January. It's October."

Figure 15: *The Long Fuse: a legal deadline found only months before it lands leaves too little time to meet it.*

What it looks like. Finance learns of a mandate from a newsletter; the storefront has never had an accessibility audit.

Why it happens. Rules land per topic and per country, nobody in a 14-person team reads the Official Journal, and each rule arrives without an owner.

What it costs. Fines, blocked invoices, emergency prices. The dates are public: Belgium's structured B2B e-invoicing obligation applied from 1 January 2026, announced 18 September 2024, with fines of €1,500, €3,000 and €5,000 for lacking the technical means (einvoice.belgium.be); the European Accessibility Act applied to e-commerce services from 28 June 2025 (Directive (EU) 2019/882); PCI DSS requirements 6.4.3 and 11.6.1 on payment-page scripts became mandatory on 31 March 2025; Germany has required businesses to receive e-invoices since 1 January 2025.

At Kestrel & Pine (fictional). The fuse was caught once: Peppol e-invoices have gone to Belgian business customers since January 2026, issued and numbered in the ERP through the EDI provider as access point. The 2027 regulatory calendar, kept by the head of engineering with the finance director, turns each remaining rule (accessibility of the three storefronts and the checkout widgets, the Dutch e-invoicing plan for 2030, packaging rules, the product passport) into a dated requirement with an owner.

In public. The Belgian mandate, announced September 2024 for January 2026: fifteen months of notice, then fines.

Remedy direction. A regulatory calendar with a named owner, listing each rule's countries, date and systems; vendor conformance as a procurement item.

Where this book helps: Rules Turned Into Requirements: Consumer, Payment,

Invoicing and Platform Regulation owns the calendar; Documents Between Companies: EDI, Punchout and E-Invoicing and Tax, Invoices and the Books: From Order to Ledger implement e-invoicing; Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers covers the PCI requirements.

The Secret Agent

"Something changed 600 stock levels on Tuesday night. We think it was the pre-order app."

Figure 16: *The Secret Agent: a script, app or AI agent that writes orders, stock, prices or money with no owner, no identity of its own and no approval changes the business in the dark.*

What it looks like. Scripts and apps share one admin's API key and write beside the official feeds.

Why it happens. Apps are easy to add, hard to see and broadly permitted by default; AI agents arrive through the same door with more autonomy.

What it costs. Phantom stock, price drift and, at worst, a compromised checkout: Sansec reported on 25 June 2024 that the `cdn.polyfill.io` script, embedded by more than 100,000 sites, had been sold in February 2024 and was injecting malware (Sansec, 2024). Visa reported 31 issuers and 4 merchants live with agent-initiated transactions in Europe (2 July 2026), and the agent protocols (Agentic Commerce Protocol releases through April 2026; Universal Commerce Protocol, launched 11 January 2026) carry an identity and a mandate for the agent.

At Kestrel & Pine (fictional). In March 2026 a pre-order app adjusted inventory in the platform while the WMS feed also wrote it: two weeks of phantom reservations on **600 variants**. In October 2026 the agency's price-import script failed on the row that restored the German price, logged the error and reported success. Waypoint's answer is W5 (every writer named) and W8 (agent readiness, not agents: accurate product data, availability and order status by API first, no shopper-facing AI agent in 2027).

In public. Polyfill.io (June 2024) is checkout code changing hands unseen.

Remedy direction. Every writing actor with an owner, its own credentials and stated rights; least privilege; human approval over a threshold; a person at the gate for AI agents.

Where this book helps: Apps, Plugins and Functions: Extensibility and Ecosystem Governance owns app governance and permissions; Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers covers API keys and script monitoring; Assistants and Agents: AI in Commerce and What Is Real in 2026 covers agents, mandates and W8.

Where the Patterns Strike

Across the four scale stages of What Changes at Each Scale: From First Store to Group, most patterns are born at Growing and paid for at Omnichannel (the author's judgement, not a survey).

Pattern	Starting	Growing	Omnichannel	Group
The Phantom Stock	Rare: one channel, one stock	**Born:** second channel, first sync	**Hurts:** stores, marketplaces, own warehouse	Per brand and shared pool
Schrödinger's Order	Rare: the platform is the record	**Born:** the first 3PL or ERP copy	**Hurts:** orders in four systems	Group reporting disagrees by brand
The Familiar Stranger	Rare: one customer table	**Born:** the first store or marketplace	**Hurts:** service and loyalty blocked	Cross-brand identity
The Boomerang	Rare: a refund button	**Born:** the first returns portal or 3PL	**Hurts:** returns from portal, stores, marketplaces	Shared reverse logistics

Pattern	Starting	Growing	Omnichannel	Group
The Price Nobody Can Explain	Rare	**Born**: wholesale price list, first campaign import	**Hurts**: five price copies, parity rules	Group price governance
The Spaghetti Junction	Absent	**Born**: connectors one at a time	**Hurts**: 30-plus flows, unknown writers	Platform team or chaos
The Message in a Bottle	Hidden by the platform	**Born**: the first custom integration	**Hurts**: at peak and in refunds	Event platform discipline
The Pulled Rug	Vendor handles it	**Born**: first marketplace or partner API	**Hurts**: many calendars, one mailbox	Contract management function
The First Snowfall	Platform absorbs it	**Born**: first big campaign with integrations	**Hurts**: joins fail at 40× load	Shared peak across brands
The Giant's Boots	**Born**: the wrong platform category	Agency-driven over-architecture	**Hurts**: composable estate for a small team	Appropriate at last, with reasons
The Burned Bridge	Low cost to migrate	**Born**: first replatforming	**Hurts**: migration touches a dozen systems	Brand-by-brand migrations
The Forked Platform	Rare	**Born**: theme and checkout edits, apps	**Hurts**: upgrades become projects	Per-brand forks
One Size Fits None	Absent	Rare	**Born**: second country or line in one tenant	**Hurts**: brands forced together or apart

Pattern	Starting	Growing	Omnichannel	Group
The Lynchpin	The founder	**Born**: the agency, the one consultant	**Hurts**: EDI, ERP connector, unowned flows	Vendor concentration
The Long Fuse	Platform defaults, per country	**Born**: second country, first B2B mandate	**Hurts**: many rules, many countries	Compliance function
The Secret Agent	Apps with wide permissions	**Born**: scripts and apps that write	**Hurts**: six writers of stock	Agents at scale

Kestrel & Pine's April 2027 stage map attributes eight of its ten incidents to Growing-stage designs still running in 2026. The remedy is rarely a new platform; it is ownership, contracts and reconciliation.

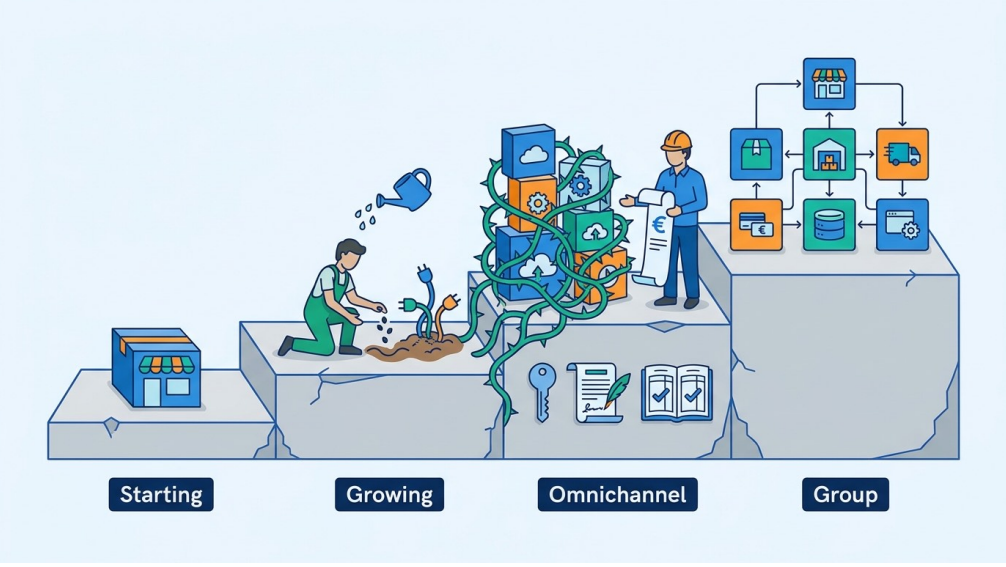


Figure 17: *Most failure patterns are born at the Growing stage and paid for at the Omnichannel stage, and the remedy is rarely a new platform; it is ownership, contracts and reconciliation.*

How the Patterns Reinforce Each Other

The integration chain. A Spaghetti Junction produces Messages in a Bottle, and those produce the Phantom Stock, Schrödinger's Order, the Boomerang and, through a price import, the Price Nobody Can Explain. The First Snowfall makes all of them visible in one evening.

The governance chain. A Lynchpin sets up the Pulled Rug: nobody reads the calendar for a flow nobody owns. The Secret Agent is a Lynchpin with write access; the Long Fuse is a deadline with no owner.

The platform chain. The Giant's Boots lead to the Forked Platform or the Burned Bridge. **The identity thread**, the Familiar Stranger, runs under all of them.

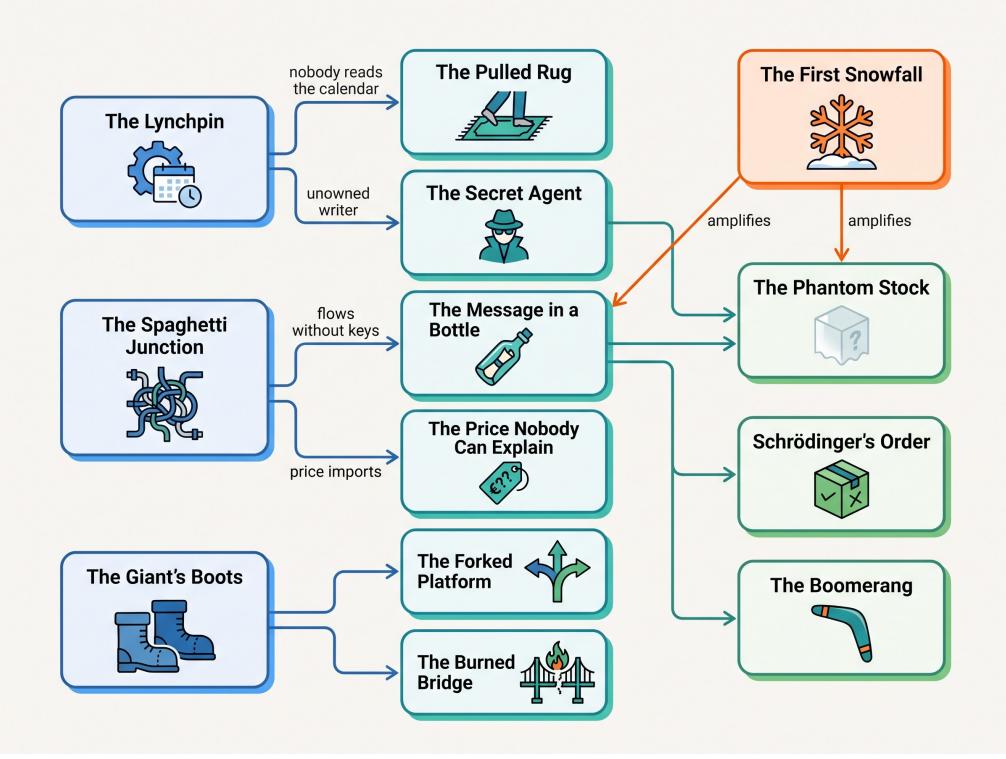


Figure 18: *The patterns travel in chains: the integration chain runs from the Spaghetti Junction through the Message in a Bottle to phantom stock, undecidable orders and double refunds; the governance chain runs from the Lynchpin to the Pulled Rug and the Secret Agent; the platform chain runs from the Giant's Boots to the fork or the burned bridge; the First Snowfall amplifies whatever is already there.*

Of Kestrel & Pine's ten incidents, eight are a fact with two writers; two are what the outside world does to a landscape nobody watches. So Waypoint decides ownership

first, then adds the hubs, then makes the copies trustworthy, and keeps the platform.

Using the Patterns

In design reviews. Walk one order through the design with the step table of Journey 1: Eva's Jacket, From Search to Doorstep and ask at every handoff which pattern could occur and what prevents it.

In vendor demos. Ask to see the pattern, not the feature: a split shipment's status, a refund sent twice, what an app may write.

In due diligence. Ask for the integration inventory, the writers of stock, prices and money, the version register, the regulatory calendar and the last reconciliation reports. Each missing document is a pattern with no owner.

The next page introduces the merchant this book follows: its business lines, systems, people, and the ten incidents above as they were lived. Continue with Meet Kestrel & Pine.

Sources and Evidence

Company, product and investor sources

- Draft order and transfer shipment inventory is moving from reserved to committed — Shopify developer changelog, 2026-08-05; a stock state's meaning changing under its readers.
- Order status and Customer notifications — Shopify Help Center, n.d., accessed 2026-11; separate status families; no tracking number, no updates.
- Customer accounts — Shopify developer documentation, n.d., accessed 2026-11; legacy customer accounts deprecated as of 2026-02-26.
- Returns apps — Shopify developer documentation, n.d., accessed 2026-11; return versus reverse fulfillment order; dispositions.
- Webhooks and HTTPS delivery — Shopify developer documentation, n.d., accessed 2026-11; duplicate and unordered deliveries; 8 retries over 4 hours.
- API versioning — Shopify developer documentation, n.d., accessed 2026-11; quarterly versions, 12 months of support, nine months of overlap.
- The checkout.liquid theme file is being deprecated — Shopify changelog, 2023-02-13; the 2024-08-13 sunset.
- Shopify Scripts deprecation — Shopify developer changelog, 2026-04-09; script editing ends 2026-04-15, execution 2026-06-30.
- BFCM readiness 2025 — Shopify Engineering, 2025-11-20; load tests at 150% of the previous peak, 200 million requests per minute, 80,000+ checkouts per minute.

- Black Friday Cyber Monday 2025 data — Shopify, 2025-12-02; $14.6 billion and $5.1 million per minute.
- Shopify completes sale of Shopify Logistics to Flexport — Shopify, 2023-06-06; a vendor exiting a capability.
- Inventory — commercetools API documentation, n.d., accessed 2026-11; the 10-second availability consistency window.
- Refunds, Webhooks and Fulfill orders with Checkout — Stripe documentation, n.d., accessed 2026-11; double-refund risk, three-day retries, fulfillment from webhooks.
- Refunds and Webhooks — Mollie documentation, n.d., accessed 2026-11; duplicate blocking, "do not retry automatically", ten retries over 26 hours.
- Important Amazon MWS removal dates — Amazon SP-API changelog, 2023–2024; MWS unavailable from 2024-04-01.
- bol partner platform — bol, 2026-11-03; Offers API v10 switched off 2027-02-01.
- From event-driven to API: Zalando's price and stock pipeline — Zalando Engineering, 2025-03-07; 30-minute event delays at Cyber Week, divergent copies, the synchronous read API.
- Sunrise: Zalando's developer platform based on Backstage — Zalando Engineering, 2023-08-03; "more than 100 disconnected interfaces" replaced.
- Zalando and ABOUT YOU successfully complete transaction — Zalando, 2025-07-11; shared logistics and payment infrastructure, about €100 million of synergies.
- Zalando stellt Connected Retail ein — TextilWirtschaft, 2026-04-24; the program ends by the end of June 2027.
- Goodbye Microservices: from 100s of problem children to 1 superstar — Twilio Segment, 2018-07-10; 140+ services consolidated into one (dated).
- Prime Video switched from serverless to EC2 and ECS to save costs — InfoQ, 2023-05-03; 90% cost reduction; the original post has been removed.
- MACH Alliance research: 6x more organizations achieve AI ROI with a composable foundation — MACH Alliance, 2026-02-18; 37% cite integration complexity as a primary concern (600 respondents).
- Adobe Commerce lifecycle policy — Adobe, accessed 2026-11; three years of standard support; cloud enforcement from 2027-06-01.
- Target's website crashes again — MPR News (AP), 2011-10-25; the Missoni crash of 2011-09-13 and the second outage (dated).
- Finish Line pulls a U-turn online after losing $3 million — Retail Dive, 2013-01-08; the 17-day reversal (dated).
- Visa press release on agentic transactions in Europe — Visa UK, 2026-07-02; 31 issuers and 4 merchants live.
- Universal Commerce Protocol announcement — Google, 2026-01-11; launch of the agent checkout protocol.
- Agentic Commerce Protocol — OpenAI and Stripe, specification releases through 2026-04-17.

- Polyfill supply chain attack hits 100K+ sites — Sansec, 2024-06-25; the script domain sold in February 2024.
- How to automate WISMO requests — Gorgias, updated 2023-11-13; WISMO as 18% of requests.

Standards, regulation and official documentation

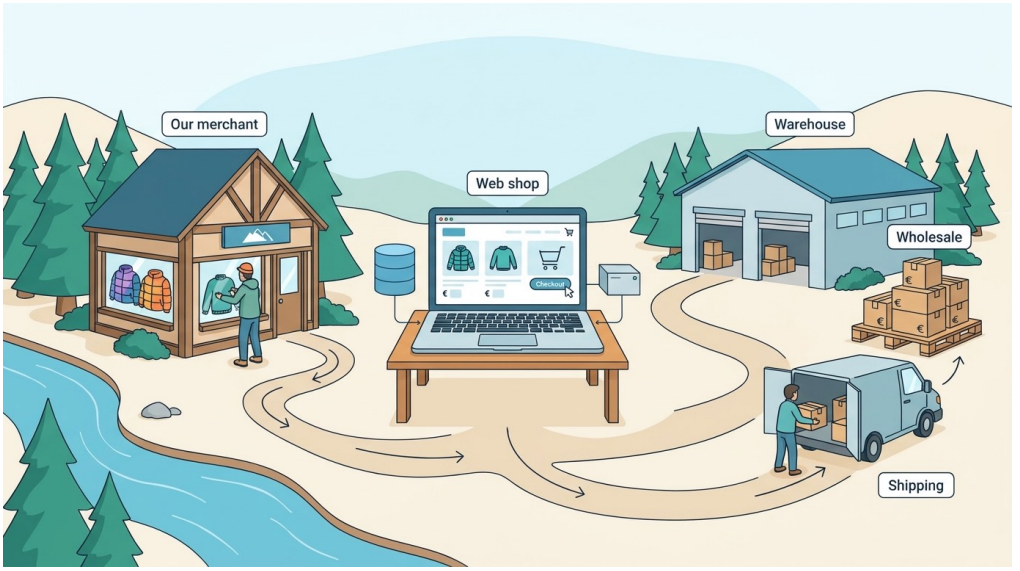
- Directive (EU) 2019/2161 (Omnibus) — Official Journal, 2019-12-18; the 30-day prior-price rule, applied from 2022-05-28.
- ACM wil einde aan nepkortingen na komst strengere regels — Authority for Consumers and Markets, 2022-12-06; supervision from 2023-01-01.
- Regulation (EU) 2016/679 (GDPR) — Official Journal, 2016-05-04; Article 12(3), one month to answer a data-subject request.
- Directive (EU) 2019/882 (European Accessibility Act) — Official Journal, 2019-06-07; application from 2025-06-28.
- Structured electronic invoices between companies are compulsory from 2026 and FAQ — FPS Finance Belgium, 2024-09-18 and n.d., accessed 2026-11; the mandate and the fines.
- E-Rechnung FAQ — German Federal Ministry of Finance, accessed 2026-11; receiving since 2025-01-01, issuing transition to end 2027.
- Guidance for PCI DSS e-commerce requirements effective after 31 March 2025 — PCI Security Standards Council, 2025; mandatory from 2025-03-31.

Research and practitioner sources

- Are inventory records inaccurate? Evidence from retail — DeHoratius and Raman, *Management Science* 54(4), 2008; 65% of records inaccurate (dated).
- 2025 Retail Returns Landscape — NRF with Happy Returns, 2025-10-15; 19.3% online return rate, 9% fraudulent.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Meet Kestrel & Pine



IN THIS SECTION, YOU WILL: Meet Kestrel & Pine, the fictional outdoor brand this book follows, its systems, its people, its 2026 incidents, the Waypoint program and the recurring order every chapter returns to.

KEY POINTS:

- **Kestrel & Pine is a fictional composite, not a disguised company**. It borrows the shape of many growing omnichannel brands so the book can show real tensions with consistent numbers. Every name, price, figure and incident is invented and recorded in one canonical ledger.
- **Four channels, thirteen systems, no order hub and no product hub**. Orders live in four systems, product data in four, stock in four, and the chain that copies stock between them takes up to two hours. Nearly every incident on this page is that gap, seen from a different side.
- **Waypoint decides ownership first and buys the hubs second**. The 2027 program names a system of record for stock, orders and product content, adds an availability service, an order management system and a product information system beside the existing platform, and sets integration standards, without replatforming.

This book tests every argument against **one fictional merchant, detailed enough to be realistic and consistent enough to follow across sixty-seven pages**. A toy example is too simple to show why the hard decisions are hard; a real company's internals cannot be verified. So the book invents a merchant, writes its facts down once, and holds every chapter to them. This page is that record in readable form; the failure patterns of Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong appear here as the incidents Kestrel & Pine lived through.

Kestrel & Pine is fictional. It is a composite of *patterns* common among growing brands: a web shop that became a company, stores added later on a separate till system, wholesale customers who want paper-free ordering, marketplaces added for reach, a warehouse of its own, and an integration landscape built one connector at a time. It is **not a portrait of any real brand or retailer**. Its systems are named by role (the platform, the ERP, the WMS), never by vendor, which also keeps the book platform-neutral. Real marketplaces (bol, Amazon.de, Zalando) and carriers (PostNL, bpost, DHL, DPD) appear as channels and services; Kestrel & Pine's dealings with them are invented.

If you are new to commerce, this page is also your first tour of its vocabulary. Every term is explained when it first appears, and the chapter that owns it is linked.

Kestrel & Pine at a Glance

Kestrel & Pine B.V. is an outdoor apparel and equipment brand headquartered in **Arnhem, the Netherlands**, founded in **2016** by **Mara Lindqvist** and **Teun Baas**. The founders hold a majority; **Veluwe Capital**, a fictional Dutch growth investor, has held a minority stake since **2022**. It sells its own brand (about **75%** of revenue: jackets, fleeces, trail trousers, merino base layers, socks, packs) and selected third-party equipment brands (about **25%**: footwear, tents, stoves, accessories) in the Netherlands, Belgium (since 2020) and Germany (since 2022), in euros throughout. In 2026 it had revenue of **€48.0 million**, a gross margin of **54%**, EBITDA of about **€3.4 million** and **310 employees** (stores 96, warehouse 88, head office 126), of whom **14** work in technology. Technology costs about **€2.6 million** a year, **5.4%** of revenue.

The revenue comes from four business lines. This is the table the whole book keeps coming back to, because each line wants something different from the architecture (Who Sells What to Whom: Business Models and the Economics That Shape Architecture explains why).

Line	Revenue 2026	Orders or transac-tions	Average value	Notes
Online D2C storefronts (NL, BE, DE)	**€27.0M** (NL €18.5M, DE €5.5M, BE €3.0M)	**293,000 orders**	**€92**	Free shipping over €75; "ordered before 22:00, delivered tomorrow" in NL
Wholesale (B2B)	**€12.0M**	**9,400 orders**	**€1,277**	**1,100 active accounts** (NL 640, BE 210, DE 250); 30-day payment terms
Stores	**€6.0M**	**118,000 transac-tions**	**€51**	Six stores
Marketplaces	**€3.0M**	**43,000 orders**	**€70**	bol, Amazon.de, Zalando through a channel connector

D2C (direct-to-consumer) means the brand sells to the shopper through its own storefronts rather than through a retailer. **B2B** (business-to-business) means selling to other businesses, here outdoor shops that resell the products. A €51 store receipt, a €92 web order and a €1,277 wholesale order are not the same kind of transaction, even when they contain the same jacket.

How Kestrel & Pine Grew, 2016–2027

The history matters because every stage left a system behind, chosen for the stage it was bought in. The book describes four **scale stages** (Starting, Growing, Omnichannel, Group; What Changes at Each Scale: From First Store to Group owns them), and Kestrel & Pine has passed through three.

Years	What happened	Stage
2016–2018	Founded in Arnhem; one Dutch web shop on a hosted SaaS commerce platform (software as a service: rented by the month and run by the vendor); one payment provider; orders shipped by a third-party logistics company (3PL, a warehouse that stores and ships on a merchant's behalf) in Tiel; an agency retained for the theme and apps from 2017.	1 — Starting
2019–2022	First store in Arnhem (2019) on a cloud point-of-sale system (POS, the till software that records store sales) from a different vendor; Belgium (2020); the Utrecht store, the marketplace connector, the integration platform and GTINs on every own-brand item (2021); Germany, Veluwe Capital's investment and Ruben Alves as lead integration engineer (2022).	2 — Growing
2023–2026	The Duiven warehouse on its own WMS, the ERP and the platform's enterprise tier with B2B features (2023); Amsterdam, Nijmegen and Eindhoven stores and the data warehouse (2023–24); Antwerp, EDI with Bergzicht Outdoor and the returns portal (2025); Peppol e-invoicing, the Utrecht click-and-collect pilot and the Black Friday Oversell (2026).	3 — Omnichannel
2027	Noor Visser joins as Head of Engineering (January); Waypoint is approved (March).	3 — Omnichannel

The revenue bands behind the stages are rough markers (Starting under about €2 million, Growing to about €20 million, Omnichannel to about €200 million, Group beyond that or several brands). The *shape* decides the stage: Kestrel & Pine became Omnichannel in 2023, when it had stores, wholesale, marketplaces, three countries and its own warehouse at the same time, and a dozen systems to join. The joins are where its problems now live.

The Catalog and the Products

Kestrel & Pine sells **3,400 products** in **26,000 sellable variants**. A product is "the Ridgewalker shell jacket, women's"; a variant is one size in one color of it. The book calls a sellable variant a **SKU** (stock-keeping unit, the thing you can count on a shelf and put in a parcel). About **17,000 variants are active** at any time; two main seasons a year plus carry-over lines bring about **1,200 new variants** per season. Every own-brand SKU has carried a **GTIN** (Global Trade Item Number, the number behind the EAN-13 barcode) since 2021; third-party SKUs use their brand's GTIN.

The hero product is the **Ridgewalker** shell jacket, in women's and men's cuts, six sizes and four colors including "moss". A few prices recur through the book, all fictional:

Item	Consumer price (incl. VAT)	Wholesale price (excl. VAT)	Landed cost
Ridgewalker shell jacket	**€199**	**€95**	**€58**
Trailhead fleece	€119	€57	
Merino hiking socks	**€18**	**€8.50**	

Where the product data lives is the first sign of trouble. In January 2027 it lives in **three places**, the platform (storefront content and variants), the ERP item master (purchasing and costs) and shared spreadsheets (seasonal enrichment and translations), with a **fourth copy** in the marketplace connector for listings. There is **no PIM** (product information management system: one place where product content and attributes are kept and published to every channel). What Are We Selling? Products, Variants and the Catalog works this out.

Channels and Customers

Online. Three localized storefronts (the Netherlands; Belgium in Dutch and French; Germany) run in **one tenant** of the platform, meaning one account on the vendor's shared service with one configuration, theme and set of apps. The Dutch storefront promises next-day delivery for orders before **22:00**, a promise written into the page rather than computed from the warehouse's state. Shoppers pay by iDEAL, card, PayPal, Klarna, Bancontact or Apple Pay.

Stores. Six stores, each holding about **1,800 variants**: Arnhem (2019, the flagship next to head office), Utrecht (2021; click-and-collect pilot since 2026, meaning order online and pick up in store), Amsterdam and Nijmegen (2023), Eindhoven (2024) and Antwerp

(2025, where Belgian fiscal receipt rules apply). Store stock is tracked in the POS and copied to the ERP **nightly**. There is **no link between the POS and the platform**, so the click-and-collect pilot is checked by phone and e-mail. Store card payments go through a separate terminal provider (Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment).

Wholesale. **1,100 active accounts**, mostly independent outdoor shops. **62%** of their orders arrive through the platform's B2B portal (company accounts, price lists, payment terms); **38%** arrive by e-mail or phone and are keyed into the ERP by the wholesale team (Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals). The largest account is **Bergzicht Outdoor**, a fictional Dutch outdoor chain with **38 shops** and a distribution center in **Zwolle**, worth **€2.1 million** a year. Since 2025 Bergzicht orders by **EDI** (electronic data interchange, the long-established standard for exchanging business documents such as orders and invoices between companies' systems without anyone typing them). Belgian business customers have received **Peppol** e-invoices (a European network for structured electronic invoices) since January 2026 (Documents Between Companies: EDI, Punchout and E-Invoicing).

Marketplaces. **bol** (NL and BE), **Amazon.de** and **Zalando** (DE and NL), all through **the marketplace connector**, a channel integration service that publishes listings, pulls orders and pushes stock levels. About **40%** of marketplace units for Germany are fulfilled by the marketplace's own fulfillment program from stock Kestrel & Pine sends there in advance, a location outside its control (Selling on Someone Else's Platform: Marketplaces and Channel Integration).

The Volumes Every Design Is Sized Against

Chapters quote these 2026 figures when they size a queue, a sync interval or a peak plan. All are illustrative.

Measure	2026 value
Online D2C orders	**293,000** (about 800 a day on average)
Marketplace orders	**43,000**
Wholesale orders	**9,400**
Store transactions	**118,000**
Active variants	about **17,000** of 26,000
Shipped from Duiven	**336,000** parcels plus **9,400** wholesale shipments; **1.9 million units**

Measure	2026 value
Returns	**72,000** parcels; item return rate **24%**, order return rate **19%** on D2C; **28%** of returns are exchanges; **61%** of returned items resold as new
Black Friday week (23–29 November)	**31,000** D2C orders, **10.6%** of the year's
Peak hour (Friday 27 November, 20:00–21:00)	**2,300** orders, **38 per minute**; storefront traffic about **41×** an average hour
Stock sync delay at peak	up to **two hours** from a sale to the stock shown elsewhere; the WMS connector queue backed up for **four hours**
Installed platform apps	**41**, of which **6** unused and **3** write to inventory
Integrations	**34** (point-to-point and iPaaS flows)

Two numbers deserve a pause. A 24% item return rate means one item comes back for every four shipped, so the return path is a main road, not a side street (The Way Back: Returns, Exchanges, Refunds and Reverse Logistics). And the peak hour is 41 times an ordinary one: a design that works at 800 orders a day meets 2,300 in one hour, once a year, which is what the pattern **The First Snowfall** warns about (Black Friday Every Year: Peak Load, Resilience and Recovery).

The Systems Landscape in January 2027

When Noor Visser arrives in January 2027, this is what she finds. Each system is named by its role.

Area	System	Since	Notes
Storefront, cart, checkout, B2B portal	**the platform**, a hosted SaaS commerce platform, enterprise tier since 2023	2016	Three storefronts in one tenant; theme customized by the agency; 41 apps

Area	System	Since	Notes
Finance and back office	**the ERP** (enterprise resource planning system: the books, purchasing, inventory valuation, invoicing, item master, manual wholesale order entry)	2023	The platform-to-ERP connector was built by the ERP partner
Warehouse	**the WMS** (warehouse management system: receiving, picking, packing and shipping at Duiven)	2023	Orders in from the platform, stock out to the ERP, via the iPaaS
Order management	**none**	—	Orders live in the platform, the connector, the ERP and the POS
Product information	**none**	—	Product data in the platform, the ERP, spreadsheets and the connector
Marketplaces	**the marketplace connector**	2021	Listings, orders, hourly stock sync
Stores	**the POS**	2019	Store stock to the ERP nightly; no link to the platform
Returns	**the returns portal** (SaaS)	2025	Labels, reasons, exchanges; refunds through the platform's API
Shipping	**the shipping service** (multi-carrier labels and tracking)	2023	Inside the WMS flow

Area	System	Since	Notes
Payments	**the PSP** (payment service provider: the company that takes the shopper's payment and settles the money to the merchant)	2016	A separate terminal provider for stores since 2019
EDI	**the EDI provider** (a value-added network that relays EDI documents between trading partners)	2025	The order, despatch advice and invoice messages (ORDERS, DESADV, INVOIC) with Bergzicht; Peppol e-invoices
Integration	**the iPaaS** (integration platform as a service: a hosted tool for building and running the flows that move data between systems) plus agency scripts and vendor connectors	2021	34 integrations; owned by Ruben Alves
Data	**the data warehouse** and a BI tool (business intelligence: reports and dashboards)	2024	Three definitions of "order" in use

Two rows say "none", and they are the point of this page. There is no **OMS** (order management system: one place that holds every order from every channel, with one status model, and steers it through payment, fulfillment, delivery and return). And there is no PIM. So an order is a platform record if Eva placed it online, a connector record if she bought on bol, an ERP record if a shop e-mailed it, and a POS receipt if she bought in Nijmegen; a product is described four times. Nobody decided this. It is what buying one good system per need, one need at a time, produces (Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record and The Order Record: Lifecycle, State and the Order Management System take it from here).

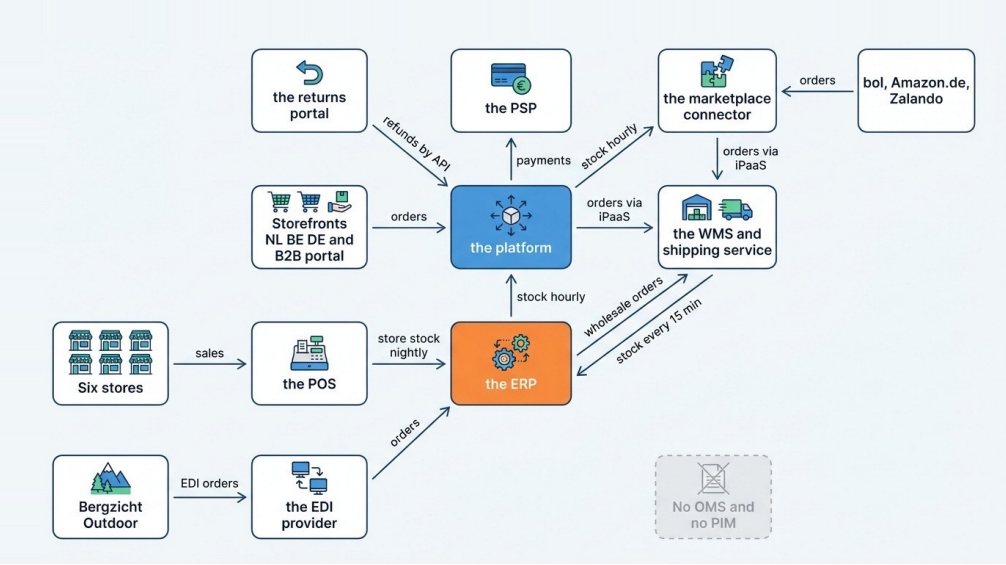


Figure 1: *Kestrel & Pine's January 2027 landscape: four channels create orders in four different systems, stock travels from the WMS to the ERP to the platform to the marketplace connector in quarter-hourly and hourly hops, and no system holds all orders or all product data.*

The **stock sync chain** in the middle of the figure is the mechanism behind the Black Friday Oversell. The WMS knows what is physically on the shelf. It tells the ERP every 15 minutes; the ERP tells the platform every hour; the platform tells the connector every hour; the connector tells the marketplaces. A sale at Duiven can take up to **two hours** to reduce the number a shopper sees on Amazon.de, and in those two hours every channel believes it can still sell the unit. Is It in Stock? Inventory, Availability and the Promise You Can Keep owns the fix.

The People

Each person owns something the architecture depends on, and most of the book's tensions are between their legitimate goals. All are fictional.

Person	Role	Pronouns	Owns
Mara Lindqvist	Co-founder and CEO	she/her	The Waypoint budget
Teun Baas	Co-founder, Chief Product and Brand	he/him	The catalog and the seasons
Noor Visser	Head of Engineering since January 2027	she/her	**The reader's stand-in.** Previously lead engineer at a logistics software company; proposed Waypoint
Daniel Okafor	Head of Operations	he/him	The warehouse, the stores and the carriers
Sanne de Wit	Finance Director	she/her	The ERP, invoicing and month-end
Milan Petrov	Ecommerce and Marketplaces Manager	he/him	The storefronts, the apps and the marketplace connector
Lea Moreau	Head of Wholesale	she/her	The B2B portal and the EDI relationship
Ruben Alves	Lead Integration Engineer since 2022	he/him	The iPaaS and the integration inventory
Hanna Berg	Data Lead	she/her	The data warehouse and metric definitions
Eva Janssen	A customer in Nijmegen	she/her	The recurring order
Pieter Smit	Buyer at Bergzicht Outdoor	he/him	The spring reorder
the agency	External storefront and apps agency since 2017	—	The theme, the apps and several scripts
the ERP partner	External ERP implementation partner	—	The platform-to-ERP connector

Noor leads a technology team of **14**: herself, nine engineers (four integration and back-end, three storefront and apps, two data), two product managers and two application

support specialists. When a chapter says "you", it usually means Noor: new to this company, not new to engineering, and asked to decide about systems she did not choose.

The Problems Kestrel & Pine Starts 2027 With

When Noor starts, every system works and the company is profitable. The problems are in the joins. Ten incidents define her starting position; each is an instance of a named pattern from Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, and each is worked in full by the chapter that owns it.

1. **The Black Friday Oversell** (Friday 27 November 2026). The Ridgewalker sold out at Duiven at 19:40; the storefronts and marketplaces kept selling it for up to two hours. **1,340 units** oversold, **1,120 orders** canceled or delayed, about **€96,000** in cost including goodwill vouchers and marketplace penalties. *The Phantom Stock* and *The First Snowfall*; Is It in Stock? Inventory, Availability and the Promise You Can Keep, Journey 4: Black Friday, Hour by Hour.
2. **The Three-Status Order** (December 2026). Order **#1052877** showed *Delivered* in the platform, *In picking* in the WMS and *Not invoiced* in the ERP for six days; one customer opened **11 support tickets**. *Schrödinger's Order*; The Order Record: Lifecycle, State and the Order Management System, Where Is My Order? Notifications, Service and the Customer Record.
3. **The Double Refund** (January 2027). Over three weeks, **212 returns** were refunded twice: once by the returns portal through the platform's API and once by hand in the PSP console. **€19,400** went out twice, most of it recovered. *The Boomerang* and *The Message in a Bottle*; The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Taking the Money: Payment Providers, Authorization, Capture and Refunds.
4. **The Price Drift** (October 2026). The German storefront sold the Ridgewalker at **€179** for **nine days** while the marketplace listing and the stores charged **€199**; a price-list sync had failed silently. **410 orders**, about **€8,200** of margin lost, and a price-parity warning from a marketplace. *The Price Nobody Can Explain*; What Does It Cost? Price Lists, Contract Prices, Currencies and Tax.
5. **The ASN Chargeback** (Q3 2026). Bergzicht deducted **€14,200** for despatch advices whose pallet labels did not match what arrived. A despatch advice (**DESADV** in EDIFACT, the UN message standard most European EDI uses; also called an ASN, advance shipping notice) tells the buyer what is on its way and on which pallets, each identified by an **SSCC** (serial shipping container code, the 18-digit number on a pallet label); Kestrel & Pine's DESADV came from the ERP's view of the order, not from what the warehouse packed. *The Pulled Rug* (the contract) and *The Lynchpin* (one EDI consultant); Documents Between Companies: EDI, Punchout and E-Invoicing.

6. **The Store Stock Mirage** (December 2026). Click-and-collect at Utrecht promised **37** items the store had already sold, because store stock reaches the ERP only nightly. *The Phantom Stock*, from the store side; Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
7. **The App That Wrote Stock** (March 2026). A pre-order app adjusted inventory in the platform while the WMS feed also wrote it: two weeks of phantom reservations on **600 variants**. *The Secret Agent* (an unowned writer) and *The Spaghetti Junction*; Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
8. **The Card-Testing Night** (August 2026). **48,000** bot attempts with stolen card numbers hit the checkout in one night; **€2,900** in PSP fees for declined authorizations, and the PSP paused the account for 40 minutes. A trust failure rather than one of the sixteen patterns; Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
9. **The Familiar Stranger**. Eva Janssen exists as an online account, a POS loyalty customer, a newsletter subscriber and a returns-portal contact, with no shared identifier, so nobody can say she is one person. *The Familiar Stranger*; One Customer, One Product: Identity and Master Data Across Systems.
10. **The Deprecated Endpoint** (February 2027). The marketplace connector retired an API version on a published date; the agency script that read it had not been updated, and stock to bol was stale for **two days**. *The Pulled Rug*; Promises Between Systems: API Contracts, Webhooks and Versioning.

Eight of the ten are the same problem: a fact (stock, status, price, identity) that lives in more than one system, with no agreed owner and no reliable way for the copies to learn of a change. The other two are what the outside world does to a landscape nobody watches as a whole.

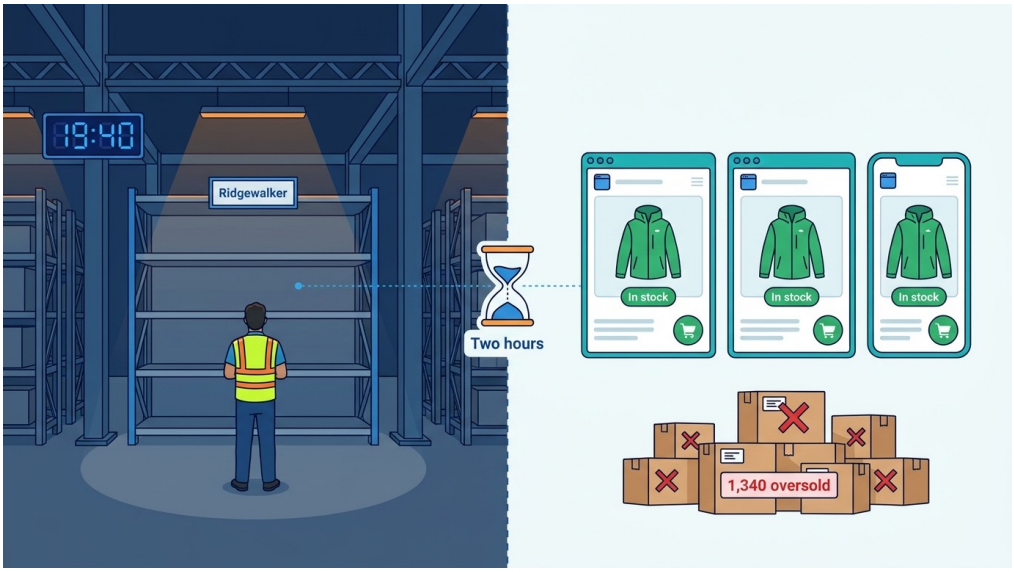


Figure 2: *The Black Friday Oversell: the Ridgewalker sold out at Duiven at 19:40, but the storefronts and marketplaces kept selling it from a copy up to two hours old, and 1,340 units were oversold.*

The Waypoint Program

In **March 2027** Mara Lindqvist approved **Waypoint**, on Noor's proposal: **€1.1 million** over 2027–2028 on top of the running technology spend. The decisions are canonical; chapters explain them and never change them.

Decision	What changes	When
W1 — Stock ownership	The WMS becomes the **system of record** for physical stock at Duiven (the one system whose value is the truth when copies disagree); the POS for store stock; the marketplace fulfillment stock becomes a tracked external location. An **availability service**, bought as a module of the OMS below, computes **available-to-promise** per channel (what a channel may still sell: physical stock minus what is already promised or reserved) from stock events. The sync chain is retired.	Q4 2027
W2 — One order record	A bought **OMS** becomes the order hub: every channel's order (platform, marketplaces, EDI, manual wholesale, later stores) is created there, with one state model and one order number family. The platform keeps the checkout and the storefront order history.	Q4 2027
W3 — Product information	A **PIM** becomes the system of record for product content and attributes; the ERP keeps costs and purchasing; the platform and the connector become consumers.	Q3 2027

Decision	What changes	When
W4 — Stay and surround	Kestrel & Pine **stays on its SaaS platform**; no replatforming. Capabilities the platform does not own well (OMS, PIM, availability) are added beside it. The platform's own POS is evaluated in 2028.	2027; review 2029
W5 — Integration standards	Every integration gets a named owner, a contract (fields, direction, frequency, system of record), **idempotent** handling of repeated messages (receiving a message twice has the same effect as once) and a reconciliation job; the inventory is kept current. Stock, orders and prices move by **events** (a message sent the moment something changes, instead of a periodic copy); products by scheduled publication; finance by daily batch.	From 2027
W6 — EDI compliance	SSCC labels generated by the WMS at pack time; DESADV sent from the WMS pack confirmation, not from the ERP; one EDI partner card per trading partner.	2027
W7 — One definition of an order	The data warehouse adopts one metric definition for order, revenue and return, owned by the Data Lead.	2027

Decision	What changes	When
W8 — Agent readiness, not agents	Accurate product data (via the PIM), availability and delivery promise by API, and order status by API are prerequisites; no shopper-facing AI agent in 2027.	2027

The order is the argument. Waypoint decides **who owns each fact** first (W1, W3, W7), then adds the hubs the landscape lacks (the availability service, the OMS, the PIM), then makes the copies trustworthy (W5, W6). It does not replace the platform (W4), because the platform is not what failed; the joins failed. Build, Buy or Assemble: Choosing and Changing Platforms works the stay-and-surround decision, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation the integration standards, Assistants and Agents: AI in Commerce and What Is Real in 2026 W8.

The Recurring Worked Examples

Five stories recur through the book. Their facts never change, so you can follow one order across many chapters and the numbers will always agree.

Eva's jacket, the recurring order. On **Tuesday 13 April 2027 at 21:14**, Eva Janssen in Nijmegen orders on the Dutch storefront: one **Ridgewalker shell jacket, women's, size M, color moss (€199)** and **two pairs of merino hiking socks (€18 each)**. Total **€235**, free shipping, paid by **iDEAL** (the Dutch bank-transfer method most online shoppers there use). Order number **#1072455**; because she ordered before 22:00, delivery is promised for tomorrow, Wednesday. The order is picked and packed at Duiven on **Wednesday 14 April**, shipped with PostNL and delivered **Thursday 15 April**, a day later than promised, because Duiven's last pick wave leaves at 21:00 (Pick, Pack, Ship: Warehouse Management and Fulfillment). On **Monday 19 April** she requests an **exchange of the jacket for size L** through the returns portal (too small). The return is received and inspected on **22 April**, graded resellable, and the exchange order ships the same day. If a chapter needs the refund variant, it is a refund of **€199** by iDEAL reversal, booked as a credit note.

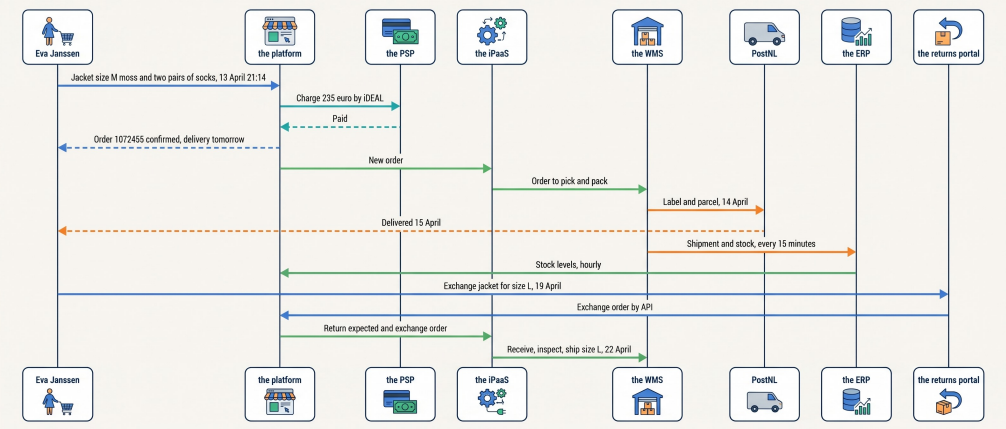


Figure 3: *Eva's jacket crosses seven systems in nine days: the platform sells and takes the money, the iPaaS carries the order to the WMS, PostNL delivers, stock trickles back through the ERP, and the exchange re-enters through the returns portal; every arrow is a handoff that can be late, lost or duplicated.*

This is the order Follow the Order: What Happens Between "Buy" and "Delivered" walks step by step and Journey 1: Eva's Jacket, From Search to Doorstep and Journey 2: The Jacket Comes Back work in full. Every chapter says near its top where in this journey it sits.

Bergzicht's spring reorder. An EDI ORDERS message arrives on **Tuesday 6 April 2027** for **240 units** across **18 SKUs** (Ridgewalker jackets and Trailhead fleeces), worth **€19,860** excluding VAT. It is picked to **three pallets** with SSCC labels; the DESADV is sent at pack confirmation; the pallets reach the Zwolle distribution center on **9 April**; the INVOIC (the EDI invoice) is sent the same day; payment is due **9 May**, on 30-day terms. Journey 3: A Chain Reorders by EDI, From Order to Invoice and Documents Between Companies: EDI, Punchout and E-Invoicing.

The marketplace variant. On 13 April 2027, the day of Eva's order, one Amazon.de customer orders a Ridgewalker that Kestrel & Pine ships itself from Duiven (seller-fulfilled), and another orders one that the marketplace's fulfillment program ships from the stock sent there in advance. The two orders look alike to the customers and nothing alike to the systems. Selling on Someone Else's Platform: Marketplaces and Channel Integration.

Black Friday 2026. The week of 23–29 November and its Friday evening: 31,000 orders, a peak hour of 2,300, the sell-out at 19:40, the two-hour stock lag and the four-hour WMS queue, with the numbers in the volumes table and in problem 1. Journey 4: Black Friday, Hour by Hour, Black Friday Every Year: Peak Load, Resilience and Recovery and Pick, Pack, Ship: Warehouse Management and Fulfillment.

The stock reconciliation. In **May 2027**, the first weekly reconciliation between the WMS and the platform (a job that compares the two systems' quantities and lists the

differences) found **2.3%** of active variants with a different quantity, most left behind by the three apps that wrote stock. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation.

The 2026 incidents are the evidence behind Waypoint. The 2027 examples happen while Waypoint runs and test its priorities: ownership before hubs, contracts before connectors, reconciliation for every copy.

How the Chapters Use Kestrel & Pine

Each chapter opens with a 2027 scene at Kestrel & Pine, then answers its question with concepts, dated public evidence from real companies and standards, and a Kestrel & Pine worked example. Chapters in Parts II to VIII end with a table across the four scale stages, using Kestrel & Pine's own path as one column. The journeys of Part IX — JOURNEYS: One Order, End to End follow the recurring orders end to end; the case studies of Part XI — PRACTICE: How Others Built It leave Kestrel & Pine and test the book against the public record.

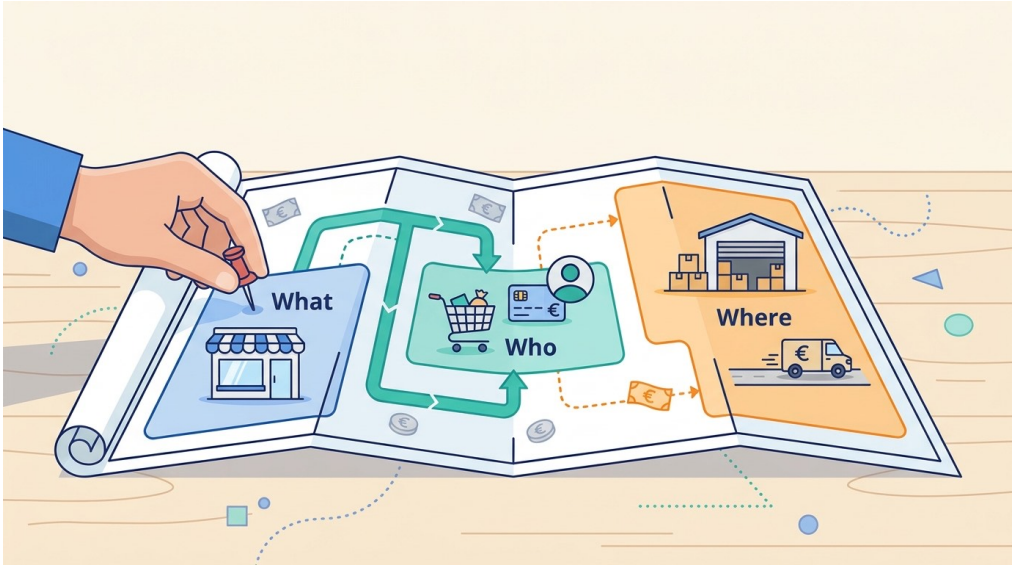
Three conventions hold. **Kestrel & Pine is labeled fictional** the first time it appears in a chapter. **Its systems are named by role**, never by vendor, so nothing in the book reads as a recommendation for or against a product. **Its facts come from one canonical record**, so the 1,340 units, the €19,400 and order #1072455 mean the same on every page; numbers that are neither Kestrel & Pine's nor documented are labeled as assumptions.

With the company, its problems and its program introduced, Part I begins where Noor began in her first week: by following one order through every system it touches, and asking at each handoff which system made the promise and which one has to keep it. Continue with Part I — COMMERCE: What a Commerce Platform Is.

Sources and Evidence

Kestrel & Pine, Bergzicht Outdoor, Veluwe Capital and every person named here are fictional; all figures are illustrative.

Part I — COMMERCE: What a Commerce Platform Is



IN THIS PART, YOU WILL: Follow one order through every system it touches, see why ways of selling shape systems, draw the map of a commerce platform, decide which system owns each fact, and sort the 2026 platform market into its kinds.

KEY POINTS:

- **A commerce platform is a promise-keeping machine, not one product.** The storefront promises a product, a price, "in stock", a delivery date and a return right; the warehouse, the carrier, the payment provider and the finance system have to keep them. Every handoff between them is a message that can be late, lost, duplicated or misread, and that is where platforms break.

- **Who sells what to whom decides the systems**. A brand selling to consumers, a wholesaler, a marketplace seller and a retailer with stores make different promises, so they need different systems; a merchant running several lines needs a shared core for the facts they share and separate edges for the promises they do not.
- **Draw the map in capabilities, then name one record per fact**. About two dozen jobs in six layers, each the home of one business fact, with every other value a copy that learns from it. The market covers that map in seven kinds of platform, priced mostly on your sales; choose by the integration work you can own, because every kind moves that work rather than removing it.

FAILURE PATTERNS THIS PART ADDRESSES:

- **The Phantom Stock** — the first promise in the journey, "in stock", is made from a copy of the warehouse's count that nobody has named as a copy.
- **Schrödinger's Order** — the warehouse's, the storefront's and the ledger's order are three objects, and no system owns the view across them.
- **The Spaghetti Junction** — connectors multiply when nobody has drawn which capability feeds which.
- **The Giant's Boots** — the landscape is full of architectures built for companies a hundred times larger, pitched after every incident.

A shop shows a jacket, promises a price, says it is in stock, promises delivery tomorrow and a return within 14 days, and takes the money. Keeping those promises takes a dozen systems: a storefront, a payment provider, a warehouse, carriers, a returns portal, a finance system, usually a store till and a marketplace connector. **They are owned by different teams and vendors, and none can see the whole order**. This part is the business before the technology: what the promises are, who makes them, which systems keep them, and what the market sells to do so.

Figure 1: *The storefront writes the promises in seconds; the warehouse, the carrier, the payment provider and the finance system have to honour them over days.*

Part I sits above the order's path rather than at one step of it. Its first chapter walks the book's recurring order, Eva Janssen's jacket, end to end for the first time; its second shows the same jacket leaving the warehouse four ways, to a consumer, a retail chain, a marketplace shopper and a store counter. The three chapters after that are about the whole platform: the map of its parts, the ownership of its facts, and the vendors that sell them.

The merchant the book follows, Kestrel & Pine, is fictional: an outdoor brand from Arnhem with €48 million of 2026 revenue, thirteen systems, 34 integrations, 41 apps, no order hub and no product hub. Its warehouse stock had six writers, its orders were born in four systems, its product data lived in four places. **The claim of this part is that nearly every one of its ten 2026 incidents was a promise one system made and another could not keep, and that the remedy is a decision about ownership, not a faster sync or a new platform.** Its Waypoint program of March 2027 is that decision, written down.

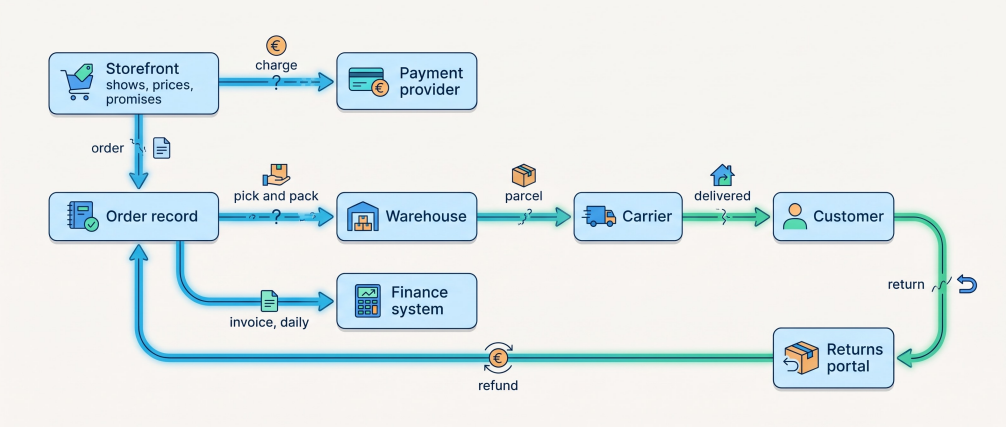


Figure 2: *The order's journey as a chain of promises: the storefront makes them early, the other systems keep them late, and every arrow is a handoff that can be late, lost, duplicated or misread.*

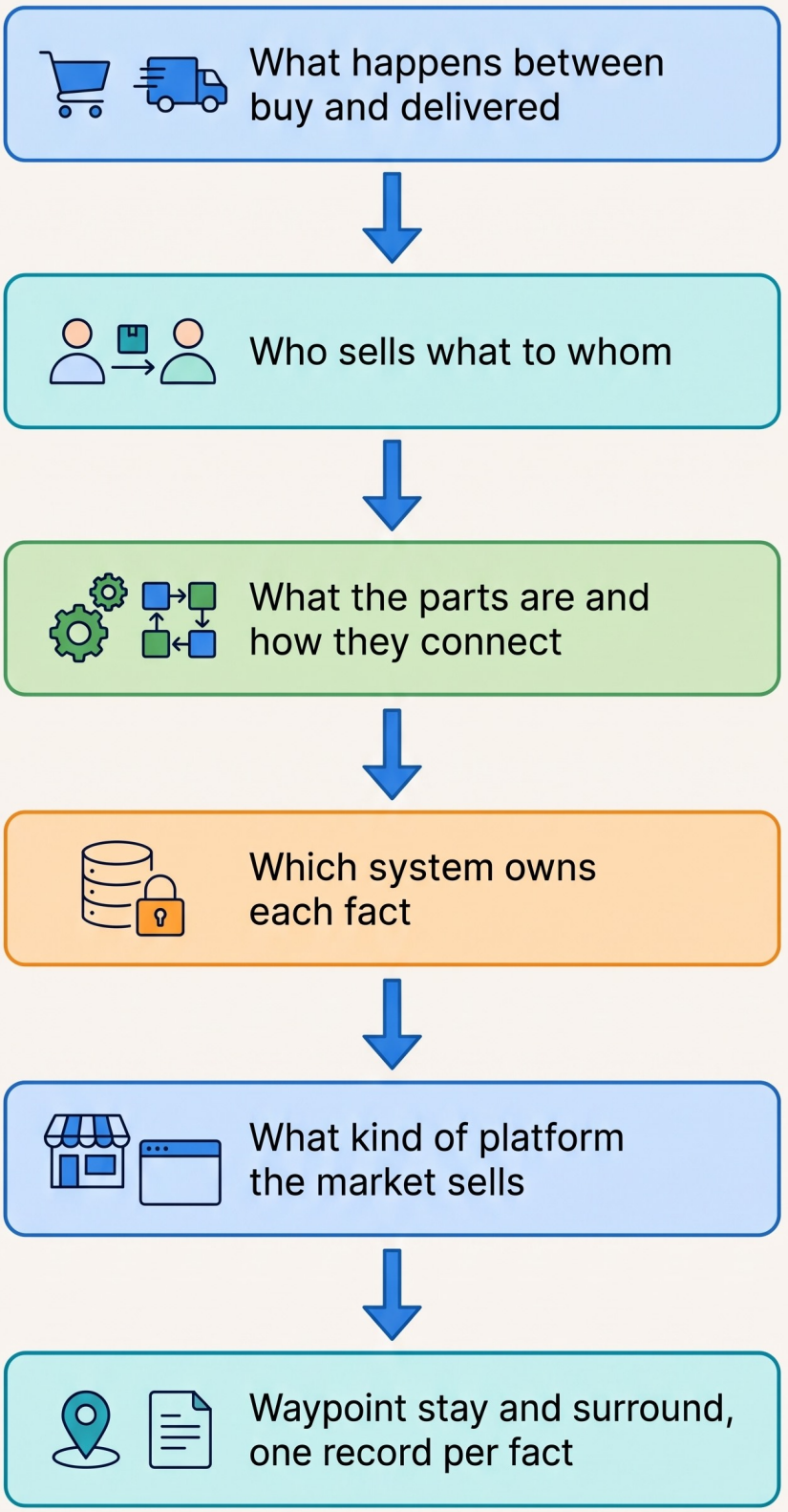

What happens between buy and delivered
Who sells what to whom
What the parts are and how they connect
Which system owns each fact
What kind of platform the market sells
Waypoint stay and surround, one record per fact

Figure 3: *The five questions of Part I in reading order: each answer is the vocabulary the next question needs, and together they are the reasoning behind Kestrel & Pine's 2027 decisions.*

Chapters in this part

Follow the Order: What Happens Between "Buy" and "Delivered" decides **what has to happen, in which systems, for one online order to be found, bought, paid, shipped, delivered, returned and accounted for**: thirteen steps, a promise at each, a handoff between most, and three status lines (money, goods, returns) that no single "status" field can hold. The case is Eva's jacket, order #1072455 of 13 April 2027, through seven systems in nine days, and the Three-Status Order that showed *Delivered*, *In picking* and *Not invoiced* at once.

Who Sells What to Whom: Business Models and the Economics That Shape Architecture decides **why a brand selling to consumers, a wholesaler, a retailer with stores, a marketplace seller and a marketplace operator need different systems**: a business model is a decision about who makes which promise, each number in an order's economics pulls on a different part of the platform, and the four scale stages are shapes, not revenue bands. The case is one Ridgewalker jacket sold four ways, €199 to a consumer and €95 to Bergzicht Outdoor, and the Price Drift that sold it at €179 in Germany for nine days.

The Map of a Commerce Platform: Capabilities and Layers decides **what the parts of a commerce platform are, what each one owns and how they usually connect**: about two dozen capabilities in six layers, the fact each is the home for, the flows between them, and the three ways vendors cover the map. The case is Noor Visser's February 2027 whiteboard, with four empty boxes and six writers of warehouse stock.

Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record decides **which system is right when several hold the same fact, and how the others find out**: the system of record, the copy, the writer, the bounded context inside which "order" has one meaning, and the entity–owner table that changes as the business grows. The case is "where does stock live?", answered six ways in March 2027, and decision W1, which made the warehouse system the record for physical stock.

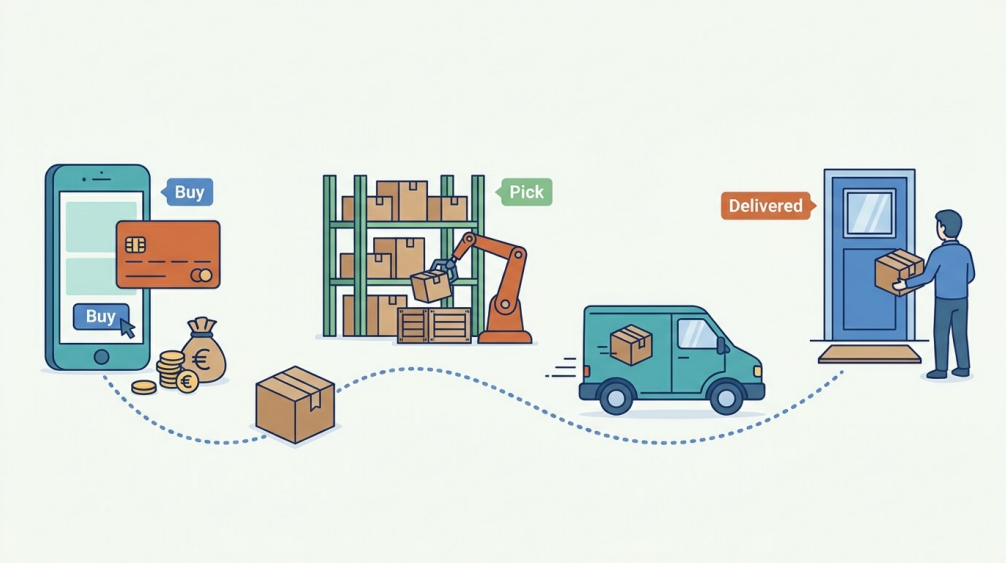
Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026 decides **what kinds of platforms exist, who sells them, how they are priced and how the market changed up to 2026**: seven categories, priced mostly on sales rather than software, and a 2023–2026 convergence on selling layers beside a kept core. The case is the two rebuild pitches of February 2027, a consultancy's composable replacement and the agency's headless storefront, and decision W4, stay and surround.

Begin with Follow the Order: What Happens Between "Buy" and "Delivered".

Sources and Evidence

Every public claim in this part is sourced and dated in the chapter that makes it, under that chapter's Sources and Evidence. Kestrel & Pine is fictional; recommendations and labeled assumptions are the author's judgement.

1. Follow the Order: What Happens Between “Buy” and “Delivered”



IN THIS CHAPTER, YOU WILL: Follow one online order through every step and every system it touches, and learn the words the rest of the book uses for the things, the promises and the handoffs along the way.

KEY POINTS:

- **An order is a chain of promises, not a database row.** Between “buy” and “delivered” the shop promises a product, a price, an availability, a delivery date, a return right and a refund. Each promise is made by one system and has to be kept by another, often owned by a different team or vendor.
- **Every step hands the order to a different system, and a handoff is a message.** Storefront, payment provider, warehouse, carrier, returns portal and finance system each hold their own copy of the order. Messages between them can be late, lost, duplicated or misread, and nearly every commerce failure is one of those four.

- **The order has three status lines, not one.** Money, goods and returns each move through their own states. Reducing them to one "status" string is how a support agent ends up with three systems that disagree.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **Schrödinger's Order** — the order's three status lines live in three systems, and an integration that flattens them into one word makes them disagree.
- **The Message in a Bottle** — every handoff in the journey is a message sent from one system to another, and a message sent once and trusted is the cheapest way to lose or duplicate an order.
- **The Phantom Stock** — the first promise in the journey, "in stock", is made by the storefront from a number the warehouse produced up to two hours earlier.

Ask a shopper what happens when they press "buy" and they will say: the money leaves my account and a parcel arrives. Ask the people who run a merchant and you get a different answer from each one. The ecommerce manager sees an order in the storefront's admin screen. The warehouse sees a pick list. The finance director sees an invoice and, days later, a payout. The support agent sees a ticket asking where the parcel is. **They are all describing the same order, and none of them can see the whole of it.**

That gap is the subject of this book, and this chapter is its map. An online order passes through about thirteen steps, from the moment a shopper searches for a jacket to the moment the accountant closes the month. At most of those steps the order moves from one system to another: storefront to payment provider and back, storefront to warehouse, warehouse to carrier, carrier to the customer's phone, and, if the jacket does not fit, back again through a returns portal, the warehouse and the finance system. **At every step a promise is made to the customer, and at almost every step the system that made the promise is not the one that has to keep it.**

This is the gentlest chapter in the book, deliberately. It introduces the things an order is made of, the systems that hold them, and two words the rest of the book leans on: **promise** and **handoff**.

Where we are in the order's journey. This chapter covers the whole of it. Eva Janssen's jacket, the order the book follows from Meet Kestrel & Pine onward, is walked here end to end for the first time, system by system; the chapters of Part IX — JOURNEYS: One Order, End to End walk it again in full detail once the parts in between have explained each system.

Setting the Scene at Kestrel & Pine: Four Drawings of One Order

In her second week as Head of Engineering, in January 2027, Noor Visser asked four colleagues at Kestrel & Pine to draw what happens to an online order. She gave each a whiteboard and twenty minutes.

Milan Petrov, who runs the storefronts and marketplaces, drew the platform (the hosted software-as-a-service commerce platform that runs the web shop) in the middle, with arrows to the payment provider and to “the warehouse”. Daniel Okafor, Head of Operations, drew the warehouse system at Duiven in the middle, with orders arriving from “the integration thing” and parcels leaving with PostNL. Sanne de Wit, the Finance Director, drew the ERP (the enterprise resource planning system that keeps the books) in the middle, with invoices going out and payouts coming in. Ruben Alves, the lead integration engineer, drew thirty-four boxes and did not finish.

Four drawings, four centers, and not one of them wrong. Each person drew the part of the journey their system owns, which is the part they can see. What nobody had drawn, because nobody owned it, was the order’s path *between* the boxes: which message carries it from one to the next, what each message promises, and what happens when a message is late.

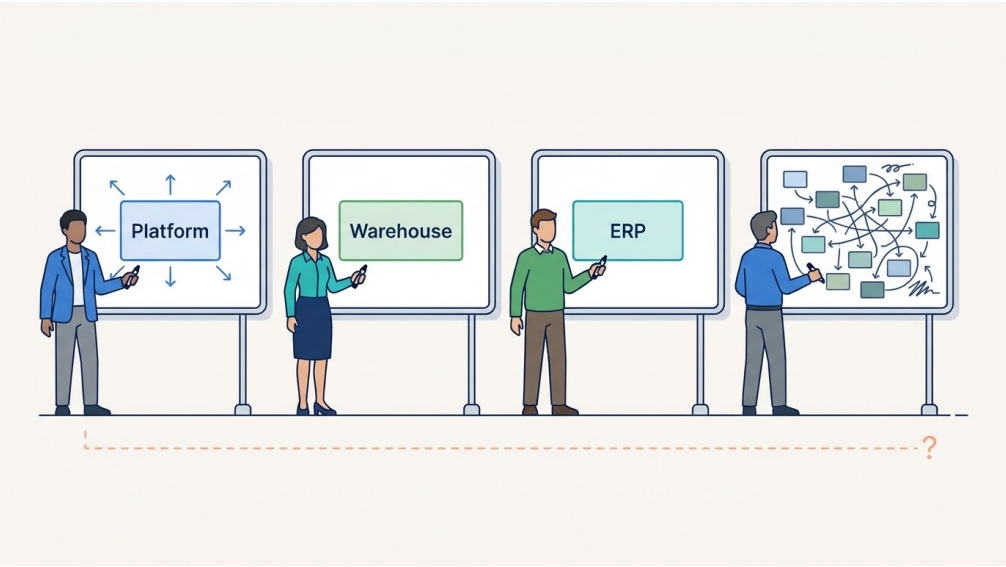


Figure 1: *Four people drew the same order around four different centers; none of them drew the path between the boxes, because nobody owned it.*

Noor’s response was to pick one order and follow it, with a notebook, through every system it touched. The order the book uses for that walk is Eva’s jacket, placed in April

2027. Before following it, the chapter needs the thirteen steps, the things an order is made of, and the two words that describe how it moves.

The Journey in Thirteen Steps

Every consumer order, on every platform, passes through roughly the same steps. The names vary by company; the shape does not. The table lists each step, what happens, the **promise** the merchant makes to the customer at that step, and which system usually has to keep it.

#	Step	What happens	The promise made	Who must keep it
1	**Discovery**	The shopper finds the product through search, a category page, an advertisement or a marketplace listing	"We sell this, and it is what the page says"	Product information, search (Finding the Product: Search, Navigation and Merchandising)
2	**Product page**	Price, sizes, colors, images, stock and delivery promise are shown	"This price, this variant, in stock, delivered by then"	Pricing, inventory, delivery promise
3	**Cart**	The shopper collects items; the cart is a draft, not a commitment	"We will hold this draft for you"	The platform
4	**Checkout**	Identity, address, delivery option, final price with tax and shipping, payment method	"You pay exactly this, delivered there, by then"	Pricing, tax, shipping rates, availability

#	Step	What happens	The promise made	Who must keep it
5	**Payment**	The payment service provider (PSP) takes the money or reserves it	"Your money is safe and matches the order"	The PSP, the platform
6	**Order creation**	The order record is created, numbered and confirmed; stock is committed to it	"The order exists and is being kept"	The platform or an order management system
7	**Fulfillment**	The warehouse picks, packs and labels the parcel	"What you ordered is in the box"	The warehouse system
8	**Shipping**	The carrier collects the parcel and scans it along the way	"It is on its way, here is where"	The carrier, the shipping service
9	**Delivery**	The parcel is handed over, left at a pickup point, or delivery fails	"Delivered when we said"	The carrier, the notification layer
10	**Service**	The customer asks "where is my order", changes an address or cancels	"We know where your order is"	Service tools over all of the above
11	**Return**	The customer sends an item back; the warehouse receives and inspects it	"You can send it back within 14 days"	Returns portal, warehouse

#	Step	What happens	The promise made	Who must keep it
12	**Refund or exchange**	Money goes back or a replacement ships	"Your money is back within 14 days"	The PSP, the platform, the warehouse
13	**Accounting**	Invoice, credit note, payout reconciliation, month-end	"The books agree with what happened"	The ERP, finance

Three things about the table matter before any technology is named. **The promises are made early and kept late**: steps 1 to 6 take minutes, steps 7 to 13 take days or weeks. **The system that makes a promise is rarely the one that keeps it**: the product page promises "in stock" from a number it was given, and the warehouse has to find the unit. And **steps 10 to 13 exist for every order**, not only the problem ones: at Kestrel & Pine, 24% of items sold online came back in 2026, and every order became an invoice. A journey that ends at "delivered" is half a journey.

The Things an Order Is Made Of

Before following an order through systems, it helps to name the things the systems hold. These are the **entities** of commerce: the nouns that every platform, in its own words, keeps records of.

Thing	In plain words	Where the book explains it
Product	What the shop sells, as the shopper thinks of it: "the Ridgewalker jacket"	What Are We Selling? Products, Variants and the Catalog
Variant or **SKU**	One sellable version of a product: the Ridgewalker jacket, women's, size M, color moss. SKU (stock keeping unit) is the identifier of that one thing. Kestrel & Pine has 3,400 products and 26,000 SKUs	What Are We Selling? Products, Variants and the Catalog

Thing	In plain words	Where the book explains it
Price	What a variant costs in one channel, country and currency, including or excluding tax	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
Stock or **inventory**	How many units of a SKU exist at a location, and how many of those may still be promised	Is It in Stock? Inventory, Availability and the Promise You Can Keep
Cart	The shopper’s draft: a list of variants and quantities with no commitment on either side	From Cart to Confirmation: Checkout Orchestration
Order	The record of what was bought, by whom, for how much, to be delivered where. It gets a number, and it is the thing every later system refers to	The Order Record: Lifecycle, State and the Order Management System
Payment	The money side of the order: what was reserved, taken, refunded, and the references the payment provider gave for each	Taking the Money: Payment Providers, Authorization, Capture and Refunds
Fulfillment order	The instruction to one warehouse: ship these lines of this order to this address	Pick, Pack, Ship: Warehouse Management and Fulfillment
Shipment	A physical parcel with a carrier label and a tracking number, and the stream of carrier scans that follow it	Getting It There: Carriers, Labels, Tracking and the Delivery Promise

Thing	In plain words	Where the book explains it
Return	The customer's request to send items back, and the warehouse's record of what arrived and what was done with it	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics
Invoice and **credit note**	The legal documents: what was sold, with how much tax, under which sequential number; and the mirror document that reduces it after a return	Tax, Invoices and the Books: From Order to Ledger
Customer	The person or company behind the order, as each system knows them	One Customer, One Product: Identity and Master Data Across Systems

Two of these are often merged by newcomers. **A cart is not an order**. A cart is a draft that commits nobody, and most are abandoned: the Baymard Institute's running average across fifty studies published between 2006 and 2025 is a 70.22% cart-abandonment rate (page updated 28 October 2026). An order is a commitment on both sides. **An order is not a shipment**. One order can become two parcels from two locations, and the fulfillment order is the piece in between: the warehouse's unit of work.

Promises and Handoffs

Two words carry the argument of this book, and this chapter owns their definitions.

A **promise** is a statement the merchant makes to the customer that some system will later have to make true. "In stock" is a promise. "€199" is a promise. "Ordered before 22:00, delivered tomorrow" is a promise. "Free returns within 14 days" is a promise. Promises are made by the systems closest to the customer (the storefront, the checkout, the returns portal) and kept by the systems closest to the goods and the money (the warehouse, the carrier, the payment provider, the finance system). **A commerce platform, in the sense this book uses, is the whole set of systems that make and keep those promises**, not one product with that name on the invoice.

A **handoff** is the moment responsibility for the order passes from one system to another. A handoff is always carried by a message: an API call, a file, a webhook, a queue entry, an EDI document (electronic data interchange: a structured document exchanged

between two companies’ systems), sometimes an e-mail. The storefront hands the order to the payment provider (“charge €235”); the payment provider hands the result back (“paid”); the storefront hands the order to the warehouse (“pick these three lines”). Each handoff has a sender, a receiver, a message format, a timing and, if things are done well, an owner.

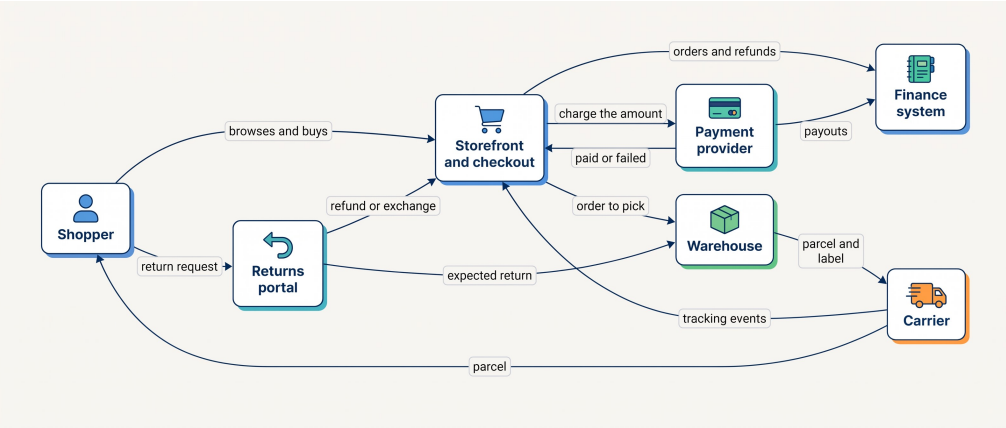


Figure 2: *The order’s journey as handoffs: every arrow is a message from one system to another, and every message can be late, lost, duplicated or misread.*

Each handoff carries requirements the business sets, usually without saying so: how fresh the information must be, how consistent with the sender’s copy, how fast it must arrive, and what happens if the receiver is down. The table states them for the main handoffs of a consumer order; the targets are illustrative, and a real merchant sets them from its own promises.

Handoff	Freshness the promise needs	Consistency	Speed	If the receiver is down
Warehouse stock → storefront “in stock”	Seconds to minutes during a sale; hours on a quiet Tuesday *(assumption)*	The storefront’s number may lag but must never exceed the warehouse’s	Urgent in aggregate, not per message	The storefront keeps selling from a stale number: the Phantom Stock

Handoff	Freshness the promise needs	Consistency	Speed	If the receiver is down
Checkout → payment provider → checkout	Immediate	Exactly one charge per order	Seconds; the shopper is waiting	Checkout cannot complete; the shopper leaves
Storefront → warehouse (order to pick)	Before the cut-off the delivery promise depends on	Every paid order, exactly once, with the right lines	Minutes; it decides whether "tomorrow" holds	Orders queue; the promise slips silently
Warehouse → storefront (shipped, tracking number)	At label print	One shipment record per parcel	Minutes; it triggers the customer's e-mail	The customer sees "being prepared" after the parcel has left
Returns portal → warehouse → payment provider (refund)	On inspection, not on request	Exactly one refund per return	Within 14 days by EU law	A refund issued by hand in the PSP console: the Double Refund
Storefront → finance system (invoice, payment, refund)	Daily is usually enough	Every order, invoice and credit note exactly once, with stable identifiers	Hours to a day	Month-end does not reconcile

Every merchant has all of these rows; what differs is whether anyone has written them down. That is why Part VI — INTEGRATION: Making Separate Systems Behave Like One is about integration.

One Order, Three Status Lines

A newcomer's instinct is to give an order one status: *new*, *paid*, *shipped*, *delivered*, *returned*. Platforms learned long ago that this does not work, because the money, the

goods and the return move independently.

Shopify’s documentation (n.d., accessed November 2026) keeps three status families on one order: an *order status* (open, archived, canceled), a *payment status* (from authorized through paid to refunded, eight values) and a *fulfillment status* (from unfulfilled through in progress to fulfilled, six values). commercetools’ order documentation (n.d., accessed November 2026) does the same with an `OrderState`, a `PaymentState` and a `ShipmentState`, plus separate states for each returned item. Neither vendor’s “fulfilled” means “delivered”; delivery is known only if the carrier’s events are fed back.

In plain words: an order can be *paid and not shipped*, *shipped and not paid* (an invoice customer), *delivered and partly refunded*, or *returned and not yet refunded*, and each of those is a normal state, not an error. The goods line alone, for a simple order, looks like this:

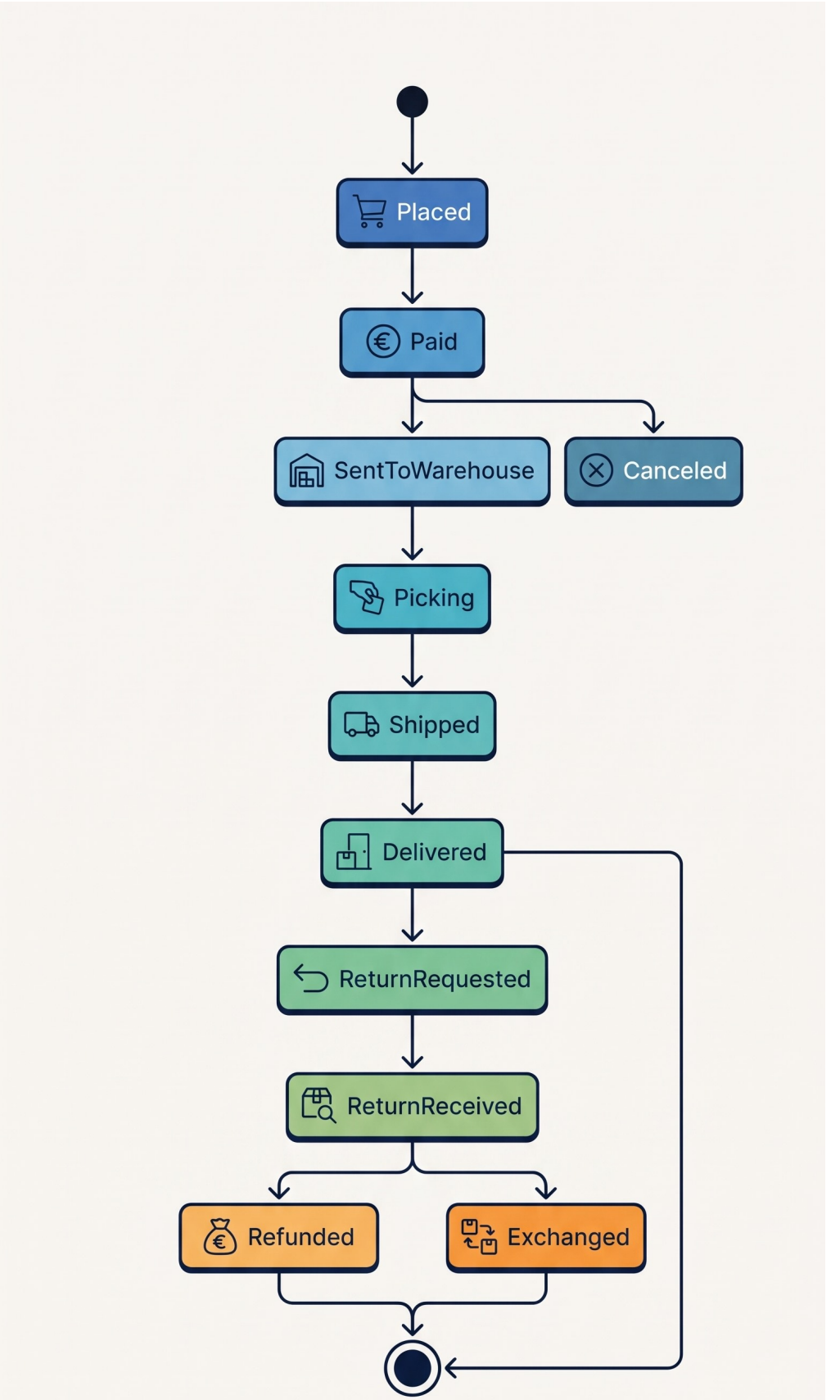

Placed
Paid
SentToWarehouse
Canceled
Picking
Shipped
Delivered
ReturnRequested
ReturnReceived
Refunded
Exchanged

Figure 3: *The goods line of a simple consumer order; money and returns have their own lines, and a support agent needs all three to answer “where is my order”.*

The Three-Status Order at Kestrel & Pine was exactly this. Order #1052877, a fleece and a pair of socks, shipped in two parcels because the socks were short in the pick location: the fleece on 7 December 2026, the socks a week later. The integration flow that connected the platform to the WMS had no notion of a partial shipment, so it marked the whole order fulfilled on the first shipment confirmation, and the platform showed *Delivered* once the carrier scanned the fleece. The warehouse management system (the WMS) tracks an order as a whole and showed *In picking* until the socks shipped. The ERP invoices on a WMS “order complete” message and showed *Not invoiced.* All three were true. None was the order’s status, because no system held the order’s status; each held one line of it, and no system could produce the sentence “your order ships in two parcels”. The Order Record: Lifecycle, State and the Order Management System explains how an order management system becomes the place that holds all three, and Where Is My Order? Notifications, Service and the Customer Record follows the eleven tickets.

Eva’s Jacket, System by System

Now the walk. Kestrel & Pine is fictional, and so is Eva Janssen, a customer in Nijmegen; every number below is fixed for the whole book, so it will match every later chapter.

On **Tuesday 13 April 2027 at 21:14**, Eva orders on the Dutch storefront: one Ridgewalker shell jacket, women’s, size M, color moss, at **€199**, and two pairs of merino hiking socks at **€18** each. The total is **€235**; shipping is free because the order is over €75. She pays by **iDEAL**, the Dutch bank-transfer method. The order number is **#1072455**. Because she ordered before the 22:00 cut-off on the storefront, the confirmation promises delivery tomorrow, Wednesday. The iPaaS in the figure is the integration tool that carries messages between Kestrel & Pine’s systems; step 6 explains it.

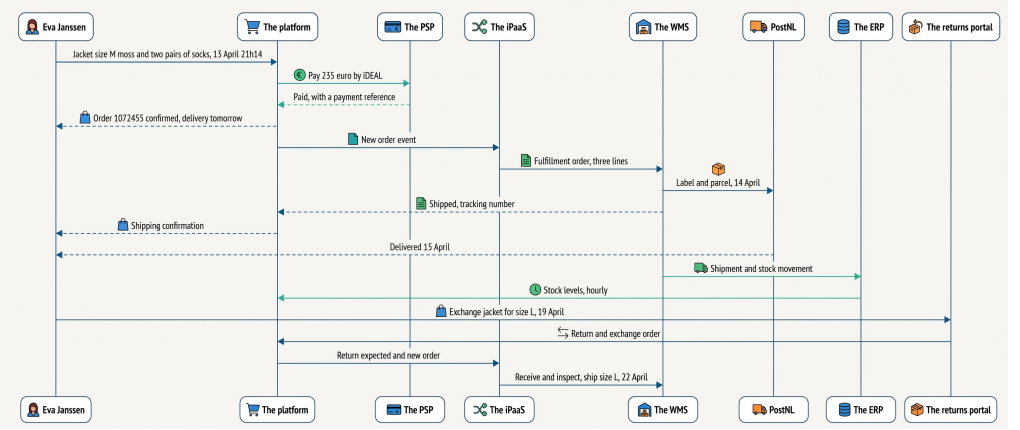


Figure 4: *Eva's jacket crosses seven systems in nine days; every arrow is a handoff, and each one is a place where a later chapter's problems live.*

Steps 1 and 2, discovery and product page (the platform). Eva searched for "waterproof jacket" on the Dutch storefront. The product page showed the moss color in size M as in stock, at €199, deliverable tomorrow. The price came from the platform's own price list. The stock number came from the platform's inventory field, which the ERP updated up to an hour earlier from a number the WMS gave it up to fifteen minutes before that. The delivery promise came from a static rule in the theme (the storefront's page templates): before 22:00 means tomorrow. Three promises, three sources, and only the price is the platform's own.

Steps 3 and 4, cart and checkout (the platform). The cart held three lines. At checkout the platform asked for Eva's address, offered home delivery by PostNL at €0, computed the total of €235 including Dutch VAT (value-added tax) at the standard rate of 21%, and offered iDEAL, cards, PayPal, Klarna, Bancontact and Apple Pay. This is the step the book calls **checkout orchestration**: in a few seconds the platform has called pricing, promotions, tax, availability, shipping rates and the payment provider, and turned a draft into a set of promises.

Step 5, payment (the PSP). The platform asked the payment provider (the **PSP**, payment service provider: the company that takes the shopper's money and later pays it out to the merchant) to collect €235 by iDEAL. iDEAL is different from a card: Mollie's documentation (n.d., accessed November 2026) describes it as an instant charge, captured automatically, with no chargeback and a 15-minute session timeout. So within a minute the PSP told the platform, by a **webhook** (a message the PSP pushes to a web address the merchant registered), that the payment was complete, with a reference. For a card payment this step has two halves, an **authorization** that reserves the money and a **capture** that takes it, which Taking the Money: Payment Providers, Authorization, Capture and Refunds explains.

Step 6, order creation (the platform). Strictly, the platform created order #1072455 a moment before the iDEAL result arrived, with the payment *pending*, and the webhook then marked it paid; steps 5 and 6 overlap. It committed one moss M jacket and two pairs of socks from its own available quantity, sent Eva a confirmation e-mail and published an order event to the **iPaaS** (integration platform as a service: the hosted tool Kestrel & Pine uses to move data between systems). The order now exists in one system. Within an hour it will exist in three.

Step 7, fulfillment (the WMS). The iPaaS created a **fulfillment order** in the warehouse management system (the **WMS**, the software that runs the physical work at Duiven: where stock sits, who picks what, into which box). On Wednesday 14 April the jacket and the socks were picked in a morning wave, packed in one box and labeled. The WMS reported the shipment and the tracking number back through the iPaaS to the platform, which sent Eva a shipping confirmation, and to the ERP, which recorded the stock movement.

Steps 8 and 9, shipping and delivery (PostNL). PostNL collected the parcel on Wednesday evening and delivered it on **Thursday 15 April**. Its scans (accepted, sorted, out for delivery, delivered) reached the platform through the shipping service inside the WMS flow, and the order status page moved to *Delivered*. The promise made on Tuesday evening was broken by a day. The “before 22:00” rule dates from the years when a third-party warehouse in Tiel shipped the orders; at Duiven the last pick wave starts at 21:00, so an order placed at 21:14 is picked the next morning and leaves with the evening collection. Nobody updated the sentence on the page, and Eva did not complain, which is why nobody noticed. Getting It There: Carriers, Labels, Tracking and the Delivery Promise replaces the rule with a computed promise.

Step 10, service. Eva did not call. Had she asked “where is my order” on Wednesday evening, the agent would have worked from what the support team calls “the five tabs”: the platform’s order screen, the WMS, the shipping service’s tracking page, the PSP console and the returns portal, with the ERP visible only to finance.

Step 11, return (the returns portal and the WMS). On **Monday 19 April** Eva used the returns portal to request an **exchange** of the jacket for size L: too small. The portal, a separate SaaS (software as a service) product, issued a PostNL return label and told the platform, through its API, that a return was expected and a new order for size L should be created. The portal has made two promises: that Eva may return the jacket, which EU law backs (Directive 2011/83/EU gives a consumer 14 days from receiving goods to withdraw from a distance purchase without giving a reason), and that size L will ship, which depends on size L being in stock.

Step 12, exchange (the WMS, the platform). The return was received at Duiven on **22 April**, inspected and graded resellable; the WMS reported the receipt, the platform closed the return, and the held exchange order for size L shipped the same day. Had Eva chosen a refund, the PSP would have returned €199 by iDEAL reversal and the ERP would have issued a credit note.

Step 13, accounting (the ERP). On dispatch, the ERP generated an invoice for order #1072455: €235 gross, of which €194.21 net revenue and €40.79 VAT at 21%. When the PSP paid out Tuesday’s sales a few days later, finance matched the payout batch to the orders in it, fees deducted. The exchange produced a credit note for the size M jacket and a new invoice for size L, netting to zero. Under the accounting standard IFRS 15, revenue is recognized when the goods are delivered, not when Eva paid, which is why finance cares about step 9 and not only step 5. Tax, Invoices and the Books: From Order to Ledger owns this.

The table says what each system knew at each moment. Read it down a column and you see one system’s world; across a row, why Noor’s colleagues drew four pictures.

Moment	The platform	The PSP	The WMS	The ERP	The returns portal
Tue 21:14, paid	Order #1072455, paid, unfulfilled	Payment, €235, complete	Nothing yet	Nothing yet	Nothing
Wed morning, picked	Paid, unfulfilled	Complete	Fulfillment order in picking	Order copy, not invoiced	Nothing
Wed afternoon, shipped	Paid, fulfilled, tracking number	Complete	Shipped	Invoice issued, stock reduced	Nothing
Mon 19 April	Delivered, return open, exchange order held	Complete	Return expected	Invoiced	Return requested, label issued
22 April	Return closed, exchange shipped	Complete	Return received, resellable; size L shipped	Credit note and new invoice	Return complete

Six systems, six copies of the order, each correct about its own part. That is normal. The question the book keeps asking is which copy is the one to believe when two disagree, and how the others learn when it changes. Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record names that copy the **system of record**.

Why the Joins Are Where Things Break

Each arrow in Figure 2 is a message. Messages can go wrong in only four ways, and all four have happened at Kestrel & Pine.

Late. The stock number on the product page was up to two hours old because it had traveled WMS → ERP → platform → marketplace connector in fifteen-minute and hourly hops. On the evening of Black Friday 2026 the Ridgewalker jacket sold out in the warehouse at 19:40 and the storefronts and marketplaces kept selling it for up to two hours: 1,340 units oversold, 1,120 orders canceled or delayed, about €96,000 in cost. The same evening the queue carrying orders to the WMS backed up for four hours.

Lost. In February 2027 the marketplace connector (the software that sends listings and stock to marketplaces such as bol, the Dutch and Belgian marketplace, and brings their orders back) retired an API version on a published date. The agency script that pushed the platform’s stock figures into the connector still used the retired version, so the connector kept sending bol its last figures for two days. A message that is not sent produces no error.

Duplicated. Over three weeks in January 2027, 212 returns were refunded twice: once by the returns portal through the platform’s API, and once by a support agent in the PSP’s console who could not see that the first refund had happened. €19,400 went out twice; most was recovered. Two senders, one receiver, no shared record of what had already been done.

Misread. The Three-Status Order was three systems reading the same two-parcel order differently, each consistently with the messages it had received: “shipped” meant “all shipped” to one and “first parcel shipped” to another.

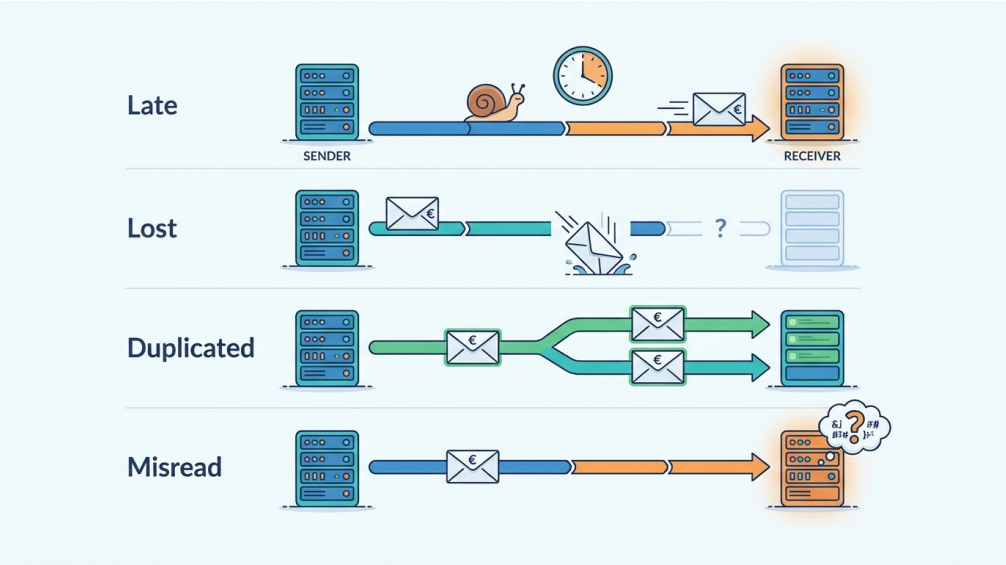


Figure 5: *Every handoff is a message, and a message can fail in only four ways: late, lost, duplicated or misread.*

Two ideas, introduced here and worked in Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, are the field’s answer. **Idempotency** means that receiving the same message twice has the same effect as receiving it once: a second “refund order #1072455” request must not produce a second refund. Stripe’s documentation (n.d., accessed November 2026) describes the mechanism on the payment side: a client sends an idempotency key with each request, and a retry with the same key returns the saved result instead of acting again. **Reconciliation** means regularly

comparing two systems' copies and listing the differences: the platform's orders against the WMS's shipments, the returns against the refunds, the PSP's payouts against the invoices. The first weekly reconciliation Kestrel & Pine ran, in May 2027, found 2.3% of active variants with different stock quantities in the WMS and the platform. Neither idea is advanced; both are routinely skipped, because each integration was built to make the happy path work.

In Practice: What the Public Record Says

Vendor documentation and regulation describe the journey's steps with surprising precision.

Payment success is learned by message, not by page. Stripe's checkout fulfillment guide (n.d., accessed November 2026) says a merchant "can't rely on triggering fulfillment only from your checkout landing page, because it's not guaranteed customers visit that page", and that the fulfillment function must "correctly handle being called multiple times with the same Checkout Session ID". commercetools' checkout documentation (n.d., accessed November 2026) creates the order *before* the payment provider has answered, with the payment pending, and warns to start fulfilling "only if the Payment contains a Transaction" with a successful state. In both designs the handoff at step 5 is asynchronous: the order waits for a message.

Messages arrive more than once and out of order. Stripe's webhook documentation (n.d., accessed November 2026) retries delivery "for up to three days", "doesn't guarantee the delivery of events in the order that they're generated", and warns that endpoints "might occasionally receive the same event more than once", so receivers should track event identifiers. Mollie's webhook (n.d., accessed November 2026) carries only a payment identifier and no status, so that a forged call can never mark an order paid; the receiver must fetch the status, and Mollie retries ten times over about 26 hours. These are the vendors' own descriptions of the Message in a Bottle, and of its remedy.

The warehouse handoff is a handshake. Shopify's fulfillment-service documentation (n.d., accessed November 2026) sends a fulfillment request that the warehouse's service must accept or reject; fulfillments "can't be created for a fulfillment order before the fulfillment service accepts", and once accepted, a cancellation becomes a *request* the warehouse may refuse. That handshake is what closes the order's edit window.

Tracking is the customer's view of the journey. Shopify's customer-notification page (n.d., accessed November 2026) lists the default messages: order confirmation (which cannot be switched off), shipping confirmation at fulfillment, and "out for delivery" and "delivered" when a carrier reports them. Its order-status page warns that "if you fulfill an order without a tracking number, then no shipping updates are received". Gorgias, a helpdesk vendor, reported from its own customer base that "where is my order" requests were about 18% of incoming support contacts (article updated November 2023; one vendor's data, now approaching three years old).

The law fixes the outer bounds of three promises. Directive 2011/83/EU on consumer rights (2011, in force) requires delivery “not later than 30 days from the conclusion of the contract” unless agreed otherwise (Article 18); gives the consumer 14 days from receiving the goods to withdraw without reason (Article 9); and requires the trader to reimburse “not later than 14 days” after being informed, by the same means of payment, while allowing the trader to wait until the goods or proof of sending arrive (Article 13). Each clause is a date the order record must be able to compute.

The books are the last handoff. Odoo’s accounting documentation (version 18, accessed November 2026), representative of how ERPs behave, assigns an invoice its number from a sequence on confirmation, after which “an invoice can no longer be updated”; a credit note “is the only legal method for canceling, refunding, or modifying a validated invoice”. Eva’s exchange therefore produced two new documents, not an edit.

Across Scale Stages: When It Fits and When It Limits

The thirteen steps do not change with the size of the merchant; what changes is how many systems share them and how many channels feed them. At the **Starting** stage (Kestrel & Pine in 2016–2018) one hosted platform makes and keeps most promises itself, and the PSP, a third-party warehouse and the carrier are the only outside systems. At the **Growing** stage (2019–2022) a second channel creates orders outside the platform, an ERP or a 3PL (a third-party logistics provider that warehouses and ships for the merchant) holds a copy of orders and stock, and “which system is right?” is asked for the first time. At the **Omnichannel** stage (2023 onward) four channels create orders in four systems and stock sits in a warehouse, six stores and a marketplace’s fulfillment center; the joins carry the business, and an order hub, an availability service and written integration contracts become necessary. At the **Group** stage several brands or countries run the journey over shared services with local differences. Kestrel & Pine is at stage 3, and its 2027 problems are stage-3 problems: not any one system, but the handoffs between a dozen. Who Sells What to Whom: Business Models and the Economics That Shape Architecture defines the stages; What Changes at Each Scale: From First Store to Group works them.

When It Goes Wrong at Each Step

None of these needs a disaster; all happen on normal weeks.

Step	Ordinary failure	How it is detected	How it is recovered
Product page	Stock shown is stale; price differs from another channel	Oversell cancellations; a marketplace's price-parity warning	Stock events instead of periodic sync; one price owner (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax)
Checkout	Payment succeeded but no order was created (the shopper closed the tab)	Payments in the PSP with no matching order	A job that finds paid-without-order and creates or refunds; the webhook as the trigger
Payment	Webhook arrives twice or never; the PSP pauses the account (the Card-Testing Night, 40 minutes)	Duplicate order numbers; orders stuck in *pending*	Idempotent handling by event identifier; a daily PSP-to-orders reconciliation
Order to warehouse	Order arrives late, never, or twice	Orders paid but not picked past the cut-off; two parcels for one order	Queue monitoring on age, not only on errors; the request-accept handshake
Fulfillment and shipping	Wrong item picked; tracking number not passed back; parcel lost	Returns marked "wrong item"; "where is my order" tickets; carrier exceptions	Scan-to-verify at pack; tracking number required on the shipped message

Step	Ordinary failure	How it is detected	How it is recovered
Return and refund	Refund issued twice; refund without receipt; restock without inspection	Returns-to-refunds reconciliation; PSP refunds not linked to a return	One refund path, triggered by the inspection event; one refund per return
Accounting	Invoice skipped or duplicated; payout does not match orders	Month-end three-way match of orders, PSP payouts and bank	Idempotent order-to-invoice posting; credit notes, never edits

The common thread: **detection is a comparison** between two systems' copies, and **recovery is a message sent again safely**. A merchant that can compare and can safely resend is most of the way to a reliable journey; one that can do neither will meet every pattern in Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong in turn.

What to Take Away

Established.

- An order's money, goods and return move through separate states; the major platforms model them as separate status families on one record (Shopify's order, payment and fulfillment statuses; commercetools' order, payment and shipment states), and "fulfilled" never means "delivered" without carrier events.
- Payment outcomes arrive as messages that can be delayed, duplicated and re-ordered; payment providers document this and tell receivers to deduplicate by event identifier and to treat the message, not the landing page, as the trigger for fulfillment.
- EU consumer law fixes three promises for every distance sale: delivery within 30 days unless agreed, a 14-day right of withdrawal from receipt, and a refund within 14 days of notice (Directive 2011/83/EU).
- Invoices are numbered sequentially at posting and never edited; a return produces a credit note.

Depends on context.

- Whether the platform, an ERP or an order management system should hold the order record depends on how many channels create orders and how many locations fulfill them. One channel and one warehouse: the platform’s order is enough. Several of each: an order hub becomes necessary (The Order Record: Lifecycle, State and the Order Management System).
- Whether a static delivery promise is acceptable depends on how often it is broken. “Before 22:00, tomorrow” served Kestrel & Pine for years; it fails for every order placed after Duiven’s last pick wave, Eva’s included, and worst when the warehouse queue backs up, which is when the promise matters most (Getting It There: Carriers, Labels, Tracking and the Delivery Promise).
- How fresh stock must be on the product page depends on how fast the merchant sells its last units. Hourly is fine for a slow-moving catalog and ruinous on a peak evening (Is It in Stock? Inventory, Availability and the Promise You Can Keep).

The author’s synthesis.

- I treat a commerce platform as the whole set of systems that make and keep promises to a customer, and I judge an architecture by its handoffs, not by its boxes: ten good systems with no owner for the messages between them make a worse platform than five ordinary systems with written contracts for every arrow.
- Before any ownership decision, I follow one real order through every system with a notebook, as Noor did. The table of what each system knows at each moment is the cheapest architecture document a merchant can produce, and the one most merchants have never produced.

Questions to Ask

Take these into your first week, a design review or a vendor demo.

1. Can someone draw the order’s path through every system, with the message that carries each handoff and the person who owns it?
2. For a given order number, which systems hold a copy, and which copy is believed when they disagree?
3. How does the storefront learn that the warehouse sold the last unit, and how long does that take on the busiest evening of the year?
4. What triggers fulfillment: a page, a webhook or a scheduled job? What happens when the trigger arrives twice, or never?
5. When the warehouse starts picking, what stops an order edit, a cancellation or a second fulfillment order?
6. How many ways can a refund be issued, and do they share a record of what has already been refunded?

7. Which of the merchant's promises (stock, price, delivery date, return right) are computed from data, and which are rules someone typed in years ago?
8. When was the last time anyone compared two systems' copies of the same orders, and what did they find?

The journey is the same for every merchant, but the systems that run it and the promises they make differ sharply by who is selling what to whom. A brand selling its own jackets to consumers, a wholesaler shipping pallets to a retail chain, a marketplace seller and the marketplace itself run different journeys over the same thirteen steps, with different economics pulling on every one of them. Who Sells What to Whom: Business Models and the Economics That Shape Architecture takes that up next.

Sources and Evidence

Company, product and investor sources

- Checkout fulfillment, Stripe, n.d., accessed 2026-11. Fulfillment must not rely on the landing page; the fulfillment function must handle repeated calls with the same session identifier.
- Receive Stripe events in your webhook endpoint, Stripe, n.d., accessed 2026-11. Retries for up to three days, no ordering guarantee, duplicate deliveries, deduplication by event identifier.
- Idempotent requests, Stripe, n.d., accessed 2026-11. Idempotency keys and saved responses on retry.
- Webhooks, Mollie, n.d., accessed 2026-11. Identifier-only payload, ten retries over about 26 hours.
- iDEAL, Mollie, n.d., accessed 2026-11. Instant charge, automatic capture, no chargeback, 15-minute session timeout.
- Capture a payment, Adyen, n.d., accessed 2026-11. Authorization reserves funds; capture by default immediately or manually on shipment.
- Payments lifecycle, commercetools, n.d., accessed 2026-11. Order created with a pending payment; fulfill only on a successful transaction.
- Orders, commercetools, n.d., accessed 2026-11. Separate order, payment, shipment and return-item states.
- Order status, Shopify, n.d., accessed 2026-11. Order, payment and fulfillment status families.
- Build for fulfillment services, Shopify, n.d., accessed 2026-11. The fulfillment request handshake; cancellations after acceptance are requests.
- Customer notifications, Shopify, n.d., accessed 2026-11. Default transactional messages and the events that trigger them.
- Understanding order status pages, Shopify, n.d., accessed 2026-11. Five customer-facing states; no updates without a tracking number.

- Customer invoices and Credit notes and refunds, Odoo, version 18 documentation, accessed 2026-11. Sequential numbering on confirmation, immutability, credit notes as the only correction.
- How to automate WISMO requests, Gorgias, 2023 (updated November 2023). "Where is my order" at about 18% of incoming requests in one helpdesk vendor's data.

Standards, regulation and official documentation

- Directive 2011/83/EU on consumer rights, European Union, Official Journal, 2011. Delivery within 30 days (Article 18), 14-day withdrawal (Article 9), refund within 14 days (Article 13), passing of risk (Article 20).
- VAT rates and exemptions, Netherlands Enterprise Agency (business.gov.nl), n.d., accessed 2026-11. Dutch standard VAT rate of 21%.
- IFRS 15 Revenue from Contracts with Customers, IFRS Foundation, n.d., accessed 2026-11. Revenue recognized when the performance obligation is satisfied.

Research and practitioner sources

- 50 cart abandonment rate statistics, Baymard Institute, 2026 (updated 28 October 2026). Average documented abandonment rate of 70.22% across fifty studies.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

2. Who Sells What to Whom: Business Models and the Economics That Shape Architecture

IN THIS CHAPTER, YOU WILL: Learn the commerce business models, the unit economics of an order and the four scale stages, and see how each one pulls on the architecture behind it.

KEY POINTS:

- **A business model is a decision about who makes which promise**. Selling to consumers, to retailers, through a marketplace or as a marketplace changes who captures the order, who sets the price, who is the legal seller, who holds the stock and who ships. The systems follow from that, not from the technology.
- **Each number in the unit economics of an order pulls on a different part of the platform**. Conversion pulls on checkout, basket size on promotions, margin on price control, returns on the reverse flow, payment fees on the provider choice, and the peak day on capacity.

A merchant that knows its numbers knows where its architecture must be strong.

- **Scale stages are defined by shape, not revenue.** A merchant moves from Starting to Growing to Omnichannel to Group by adding channels, locations and brands, and vendors' own price tiers confirm it. Each transition turns the joins between systems into the problem, which is why the order hub, the product hub and integration standards arrive together.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **One Size Fits None** — business lines with different promises are forced onto one stack, or kept apart, before anyone has worked out what they share.
- **The Giant's Boots** — an architecture is copied from a marketplace operator or a hundred-times-larger retailer whose business model, and therefore whose problems, the merchant does not have.
- **The First Snowfall** — seasonality is a property of the business model, and the merchant that has not measured its peak meets it in production.

Every commerce platform is built to keep promises, and the promises depend on who is selling what to whom. A brand selling its own jackets on its own web shop promises a price, a delivery date and a return right, and keeps all three itself. A wholesaler selling to a retail chain promises a net price per account, a delivery to a distribution center on a pallet with the right labels, and an invoice the buyer's system can read. A seller on a marketplace promises whatever the marketplace's rules require and is penalized by the marketplace when it fails. A marketplace operator promises the shopper an experience across thousands of sellers and promises the sellers a payout. **These are different businesses, and the systems that run them are different because the promises are.**

Newcomers often meet the models as marketing labels: B2C, D2C, B2B, B2B2C, omnichannel, marketplace. This chapter defines each by what it changes operationally, in the words of the companies and statistics agencies that report them, then turns to the numbers that decide how much money an order makes and which part of the platform each number pulls on. It ends with the four scale stages the rest of the book uses, and with the question Kestrel & Pine faces: **four business lines, one profit-and-loss statement (P&L), and systems that each serve one line well.**

Where we are in the order's journey. This chapter sits beside the order's path rather than on it. Eva's jacket, the order from Follow the Order: What Happens Between "Buy" and "Delivered", is one of four ways the same Ridgewalker jacket leaves Kestrel & Pine's warehouse: Eva's is the consumer line; Bergzicht Outdoor's spring reorder is the wholesale line; the Amazon.de order of 13 April 2027 is the marketplace line; a sale in the Antwerp store is the fourth. Each runs the thirteen steps differently.

Setting the Scene at Kestrel & Pine: One Jacket, Four Ways to Sell It

In February 2027, before proposing the Waypoint program, Noor Visser sat with Sanne de Wit, the Finance Director, over the 2026 accounts. Kestrel & Pine had earned **€48.0 million** in revenue at a **54%** gross margin: **€27.0 million** from the online storefronts in the Netherlands, Germany and Belgium, **€12.0 million** from wholesale, **€6.0 million** from six stores and **€3.0 million** from three marketplaces. One company, one warehouse at Duiven, one catalog of 3,400 products.

Then Noor asked each head of a business line what they needed from technology in 2027.

Milan Petrov (storefronts and marketplaces) wanted checkout conversion, a computed delivery date on the product page, and stock on the marketplaces that did not lag two hours. Lea Moreau (wholesale) wanted every account's contract price to be right in the B2B portal, the 38% of wholesale orders still keyed by hand from e-mail into the ERP (the enterprise resource planning system that keeps the books and the wholesale orders) to arrive electronically, and the exchange of electronic documents with Bergzicht (EDI, electronic data interchange) to stop producing chargebacks. Daniel Okafor (operations) wanted pallets and parcels to stop competing for the same pickers in the same wave, and store stock visible to the web shop. Sanne wanted one definition of an order so that month-end took days, not weeks.

None of them was asking for a feature. **Each was describing the promise chain of a different business model**, and the systems Kestrel & Pine had bought, one per need, served each model partly and none of them wholly. The question Noor took to the Waypoint proposal was not "which platform" but "what do these four lines share, and what must stay separate?" This chapter gives the vocabulary to answer it.

The Business Models, Defined by the People Who Report Them

Marketing uses the model names loosely. Statistics agencies and annual reports use them precisely, because they have to count something. The table lifts their definitions.

Model	Working definition	Who reports it that way
B2C (business to consumer)	Sales to private consumers over websites or apps	Eurostat counts "web sales" to private consumers separately from sales to enterprises and public authorities (data extracted February 2026)
D2C (direct to consumer)	A brand selling through channels it operates itself: its own web shop, its own stores	VF Corporation's fiscal 2026 annual report (May 2026): direct-to-consumer "includes VF-operated retail stores, brand e-commerce sites, concession retail locations and other digital platforms", 44% of its revenue
Wholesale (B2B for a brand)	Selling to retailers and distributors who resell	Columbia Sportswear's 2025 annual report (February 2026) separates its wholesale channel (department store chains, internet retailers, international distributors) from its direct-to-consumer stores and sites
B2B e-commerce (business to business)	Business buyers ordering electronically, by web shop or by structured messages between systems	Eurostat splits "web sales" from "EDI-type sales": in 2024, EDI-type sales were 11.07% of EU enterprise turnover and web sales 8.39%, so most business ordering is still by structured message, not by web shop

Model	Working definition	Who reports it that way
B2B2C (business to business to consumer)	A brand sells through a partner's consumer channel while keeping control of its product, content and price	Zalando's 2025 annual report (March 2026) describes a partner business that lets brands sell on Zalando "while maintaining full control over their offer, content and pricing"
Marketplace selling	Selling on someone else's marketplace	Eurostat: web sales through marketplaces were 1.30% of EU enterprise turnover in 2024, against 7.08% through own websites or apps
Marketplace operating	Running the marketplace and taking a fee from sellers	Amazon's 2025 annual report (February 2026): "We offer programs that enable sellers to sell their products in our stores, and fulfill orders using our services. We are not the seller of record in these transactions"; third-party seller services were $172.2 billion of net sales
Omnichannel	Stores and digital channels sharing stock, orders and customers	Target's fiscal 2025 annual report (March 2026): stores fulfilled 97.6% of its merchandise sales, including shipment from stores, order pickup and same-day delivery

Model	Working definition	Who reports it that way
Subscription	Recurring orders under a standing agreement, or a paid membership that changes the promises	Amazon reports "subscription services" of $49.6 billion in 2025 (Prime and digital subscriptions); Target's paid Target Circle 360 membership gives "unlimited same-day delivery on eligible orders over $35"

Three terms in the table need plain words. The **seller of record** is the party that legally sells to the customer: the one whose name is on the invoice, who owes the VAT (value-added tax) and who answers for the consumer's rights. A marketplace operator usually is not, which is why Amazon says so in its annual report. **GMV** (gross merchandise value) is the value of orders passing through a platform, including what the platform does not keep: Shopify's 2025 annual report (February 2026) defines it as "the total dollar value of orders facilitated through our platform" net of refunds. **EDI-type sales** are orders received as structured messages between systems, which Documents Between Companies: EDI, Punchout and E-Invoicing explains; the point here is that for business customers they are the majority.

The models are not exclusive. Kestrel & Pine runs four at once, which is what "omnichannel" means in practice, and most merchants above the Starting stage run at least two.

What Each Model Changes in the Promise Chain

Follow the Order: What Happens Between "Buy" and "Delivered" listed the promises an order carries: a product, a price, an availability, a delivery, a return right, a refund, an invoice. A business model is a decision about who makes and keeps each of them. The table says who, per model; the architectural consequence follows in the last column.

Promise	D2C brand	Wholesale	Marketplace seller	Marketplace operator	Store (omnichannel)
Who captures the order	The brand's storefront	The buyer's system (EDI, punchout) or the brand's B2B portal	The marketplace; the seller imports it	The marketplace, for thousands of sellers	The till (the POS, point of sale)
Who sets the price	The brand, per market	Negotiated per account; price lists and terms	The seller, under parity rules and repricing pressure	The sellers; the operator sets fees	The brand, but often a separate price list
Who is seller of record	The brand	The brand (to the retailer)	The seller	Usually not the operator	The brand
Who holds the stock	The brand's warehouse or a 3PL (a third-party logistics warehouse)	The brand's warehouse; pallets, not parcels	The seller, or the marketplace's fulfillment program	The sellers; the operator in its fulfillment program	The store, as a location of its own
Who ships	The brand via carriers	The brand to a distribution center, with labels the buyer requires	The seller or the marketplace	The sellers or the operator	The customer carries it; or the store ships

Promise	D2C brand	Wholesale	Marketplace seller	Marketplace operator	Store (omnichannel)
Who handles returns	The brand's portal and warehouse	Rare; defects and chargebacks instead	The marketplace's policy, debited to the seller	The operator's policy	The store, often for online orders too
Who invoices	The brand, per order	The brand, per shipment, often electronically	The seller (or the marketplace on its behalf)	The operator invoices fees to sellers	The till, with fiscal rules per country

Promise	D2C brand	Wholesale	Marketplace seller	Marketplace operator	Store (omnichannel)
Architecture consequence	The brand owns the whole chain and every system in it	Order capture moves into the buyer's systems; prices and terms per account are the hard part	Someone else's catalog format and order format become the contract	Tenancy, payouts and seller-of-record questions; a different book (Selling on Someone Else's Platform: Marketplaces and Channel Integration and LISTED's Rules of the Marketplace: DSA, DAC7 and Vertical Regulation)	Store stock becomes sellable online; the till becomes a system

A few words in the table need plain definitions. **Punchout** lets a business buyer open the supplier's catalog from inside the buyer's own purchasing system and send the filled cart back there for approval. **Parity rules** are a marketplace's terms that a seller may not offer the same product cheaper elsewhere; **repricing** is the automatic adjustment of marketplace prices against competing sellers. **Tenancy** is the vendor's job of running many sellers or merchants on one shared system while keeping their data apart.

Two rows deserve emphasis. **Who captures the order** decides where the order record is born; a merchant with orders born in four systems needs a fifth to hold them (The Order Record: Lifecycle, State and the Order Management System). **Who holds the stock**

decides how many locations the availability calculation must cover; a marketplace's fulfillment program is a location the merchant can see but not count (Is It in Stock? Inventory, Availability and the Promise You Can Keep).

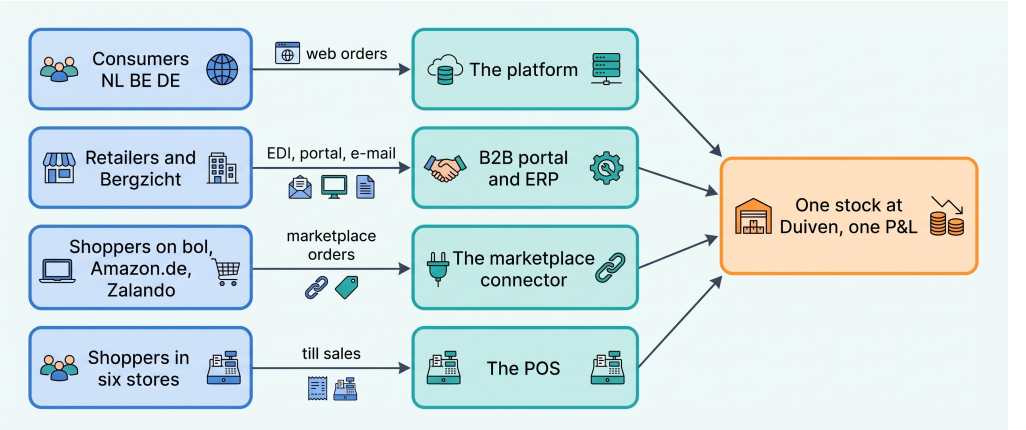


Figure 1: *Kestrel & Pine's four business lines capture orders in four systems and draw on one stock and one profit-and-loss statement; what the lines share is the stock and the money, and what they do not share is the promise.*

The **merchant's view and the vendor's view** of the same platform differ in the same way. Shopify's 2025 annual report (February 2026) notes that while most of its merchants subscribe to its Basic and Grow plans, the majority of its GMV comes from merchants on Shopify Plus and enterprise offerings. For the vendor, the product is a tenancy and GMV is the metric. For the merchant, the platform is one system among a dozen and the order is the metric. A merchant reading a vendor's roadmap should remember which metric it was written for.

The Unit Economics of an Order

How much money does one order make? The answer decides which promises a merchant can afford, and each number in it pulls on a different part of the architecture. The table gives the metrics, a dated public benchmark for each, Kestrel & Pine's own figure, and the part of the platform the metric pulls on.

Metric	In plain words	Public benchmark (dated)	Kestrel & Pine 2026	What it pulls on
Conversion rate	Purchases divided by visits	Median 1.3%, mean 1.9% across 438 Shopify stores in the 90 days to 31 October 2026 (Littledata, November 2026)	Not recorded *(assume about 1.8%)*	Storefront speed, checkout steps, visible total cost (From Cart to Confirmation: Checkout Orchestration, Delivering the Storefront: Themes, Headless, Rendering and Performance)
Cart abandonment	Carts started and not bought	70.22% average of 50 studies, 2006–2025 (Baymard, October 2026); top reasons: extra costs 40%, slow delivery 20%, forced account 18%, site errors 17%	—	The same: the top reasons are architecture failures, not design failures

Metric	In plain words	Public benchmark (dated)	Kestrel & Pine 2026	What it pulls on
Average order value (AOV)	Revenue per order	Shopify merchants' average cart $114.70 over the 2025 Black Friday weekend (December 2025); Zalando's average basket €62.8 in 2025 (March 2026)	€92 online, €70 market-places, €51 stores, €1,277 wholesale	Promotions, bundles, free-shipping thresholds (Discounts, Coupons and Loyalty: Promotions Without Chaos); a wholesale order is a different object, not a bigger basket
Gross margin	Revenue minus cost of goods, as a share of revenue	VF 54.8% (FY2026), Columbia 50.5% (2025), Nike 42.9% (FY2026), Zalando 39.9% (2025), Target 27.9% (2025)	54%	Markdown and price control: one price owner (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax)

Metric	In plain words	Public benchmark (dated)	Kestrel & Pine 2026	What it pulls on
Return rate	Share of items or orders that come back; say which	19.3% of US online sales projected for 2025 (the National Retail Federation, NRF, with Happy Returns, October 2025); 24.4% of online apparel orders in a 2023 survey (Coresight; evidence age)	24% of items, 19% of orders online; 72,000 return parcels; 61% resold as new	The reverse flow: restock, refund, credit note across systems (The Way Back: Returns, Exchanges, Refunds and Reverse Logistics)
Shipping cost and threshold	What a parcel costs and when it is free	Extra costs are the top abandon-ment reason (Baymard); Zalando charges a “minimum order fee” on small orders (2025 annual report)	Free over €75; 336,000 parcels	The delivery promise and carrier choice (Getting It There: Carriers, Labels, Tracking and the Delivery Promise)

Metric	In plain words	Public benchmark (dated)	Kestrel & Pine 2026	What it pulls on
Payment fees	What the PSP (payment service provider) keeps per transaction	EU interchange capped at 0.2% debit and 0.3% credit for consumer cards since 9 December 2015 (Regulation 2015/751); Dutch list prices, n.d., accessed November 2026: Stripe 1.5% + €0.25, Mollie 1.80% + €0.25, Shopify Payments 1.9% to 1.4% + €0.25, iDEAL about €0.29–0.32 per payment	Not recorded	PSP choice and platform tier (Taking the Money: Payment Providers, Authorization, Capture and Refunds)

Metric	In plain words	Public benchmark (dated)	Kestrel & Pine 2026	What it pulls on
Customer acquisition cost	Marketing spend per new customer	Warby Parker spent $110.2 million on marketing against $871.9 million revenue in 2025 (12.6%), up from 11.7% in 2023 (annual report, February 2026)	Not recorded	Analytics and identity: knowing who is new (Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics)

Two observations follow for the newcomer. First, **the abandonment reasons are an architecture list**. Unknown shipping costs until the last step, slow delivery promises, forced account creation and site errors are each a failure of a system to make a promise early and keep it. Second, **payment fees are an architecture decision disguised as a finance one**. For a merchant at €27 million online revenue, the gap between 1.9% and 1.4% plus a fixed fee is six figures a year, and whether the platform forces its own payment gateway, and at what fee, is one of the platform's pricing levers.

The Same Jacket in Three Lines: A Worked Example

Kestrel & Pine's Ridgewalker jacket sells at **€199** to a consumer (including 21% Dutch VAT), at **€95** excluding VAT to a wholesale account, and at €199 on Amazon.de; its landed cost is **€58**. The table works one unit through three lines. Unmarked figures are Kestrel & Pine's own (fictional); everything marked *(assumption)* is illustrative and not a benchmark.

Line	Online D2C (Netherlands)	Wholesale (Bergzicht)	Marketplace (Amazon.de, seller-fulfilled)
Price the customer pays	€199 incl. VAT	€95 excl. VAT	€199 incl. VAT
Net revenue after VAT	€164.46	€95.00	€167.23 at German VAT of 19%
Landed cost	€58.00	€58.00	€58.00
Gross profit per unit	€106.46 (65% of net)	€37.00 (39%)	€109.23
Payment or channel fee	iDEAL, about €0.30 *(from list prices)*	None; invoice on 30-day terms	Referral fee: Amazon's US pricing page lists 15% for most categories (n.d., accessed November 2026); *(assume 15% of €199, €29.85)*
Outbound shipping	One parcel, *(assume €5.50)*	240 units on three pallets, *(assume €0.60 per unit)*	One parcel, *(assume €7.50 cross-border)*
Returns	24% of items; *(assume €7.50 per return in carriage and handling, €1.80 per unit sold)*	Rare; chargebacks instead (€14,200 in Q3 2026)	Marketplace policy; *(assume as D2C)*
Marketing	*(assume 12% of net revenue, in the range Warby Parker reports: €19.74)*	A sales team, not per order	Sponsored listings; *(assume 5%, €8.36)*
Contribution per unit *(illustrative)*	**about €79**	**about €36**	**about €62**

Line	Online D2C (Netherlands)	Wholesale (Bergzicht)	Marketplace (Amazon.de, seller-fulfilled)
What the line needs from the platform	Fast checkout, computed delivery date, one price per market, a returns flow that restocks and refunds once	Contract prices per account, electronic order capture, pallet picking with the labels the buyer demands, an invoice the buyer's system reads	A listing in the marketplace's format, stock per channel with a buffer, parity with the storefront price, the marketplace's return and performance rules

Read across, the table says why the four heads of business lines asked for different things. The D2C unit earns the most and spends it on marketing, parcels and returns, so its platform must convert, promise and take returns well. The wholesale unit earns less than half as much but ships 240 at a time, on one invoice, with no returns, so its platform must get prices and documents right and nothing else. The marketplace unit pays a fee for the traffic D2C buys with marketing, and lives under rules it did not write. **One jacket, three businesses, three sets of promises, and the architecture has to serve all three without pretending they are the same.**

Figure 2: *One Ridgewalker jacket, three businesses: about €79 per unit online, €36*

wholesale and €62 on a marketplace, each with its own promises for the platform to keep.

Seasonality and the Peak

Every consumer model has a peak, and the peak is the design load. The US Census Bureau's quarterly e-commerce release (August 2026) shows the shape in one number: e-commerce was 18.3% of retail sales in the fourth quarter of 2025, not seasonally adjusted, against 15.8% in the third. Adobe Analytics reported $44.2 billion of US online spending in Cyber Week 2025, with $16 million per minute between 8 and 10 pm on Cyber Monday (December 2025). Shopify reported $14.6 billion of GMV over the 2025 Black Friday weekend with a peak of $5.1 million per minute (December 2025), which is the moment a hosted platform earns its fee and a self-run one needs a load test.

Kestrel & Pine's peak is smaller and no gentler. Black Friday week 2026 brought **31,000** online orders, **10.6%** of the year's, and the peak hour on Friday 27 November carried **2,300** orders, **38 per minute**, with storefront traffic about **41 times** an average hour. The platform held. The stock sync chain and the warehouse queue did not, and the Black Friday Oversell followed. Two architectural facts come out of this. **Capacity is a property of the whole chain**, including every integration, not of the storefront, which Black Friday Every Year: Peak Load, Resilience and Recovery works. And **the peak sets the calendar**: Hobbycraft, a UK retailer, postponed a website relaunch from before Christmas 2012 to 2013 because, in its chief executive's words, "we felt it was too risky to launch the website in peak trading" (Retail Week, February 2013; evidence more than a decade old, and still the rule). Wholesale has its own seasons, set by retailers' buying calendars; Bergzicht's spring reorder in April is one.

The Four Scale Stages

The book compares every decision across four **scale stages**. This chapter owns the term: a scale stage is a **shape of business**, described by how many channels create orders, how many locations hold stock, how many countries and brands are served, and who runs the technology. The revenue bands are rough markers; the shape decides.

Stage	Typical size	Shape	Business models typically run	What becomes necessary	Warning signs of outgrowing it
1 — Starting	Under about €2 million	One channel, a hosted platform, founder-run operations, a 3PL or a back room	One model: D2C, or a single marketplace, or a small B2B shop	The platform's built-in features; an accounting export; a PSP	Orders keyed by hand into anything; the founder as the integration
2 — Growing	About €2–20 million	Two or three channels, first store or wholesale line, first integrations, a small team and an agency	A second model is added, usually wholesale, a marketplace or a store	An ERP or accounting system, a 3PL or warehouse system, a marketplace connector; one agreed order number	The first oversell between two systems; an agency script nobody owns

Stage	Typical size	Shape	Business models typically run	What becomes necessary	Warning signs of outgrowing it
3 — Omnichannel	About €20–200 million	Stores, wholesale, marketplaces and several countries; own warehouse; a dozen or more systems	Three or four models on one stock and one catalog	An order hub (an OMS, order management system), a product hub (a PIM, product information management system), an availability service, integration standards, one metric definition	Orders in four systems; two-hour stock lag; three definitions of “order”; support agents with five tabs

Stage	Typical size	Shape	Business models typically run	What becomes necessary	Warning signs of outgrowing it
4 — Group	Over about €200 million, or several brands or business units	Multiple brands, countries or units; platform teams; acquisitions	All models, per brand or country, with different mixes	Shared services where the benefit is measured; autonomy where it is not; vendor-scale concerns (tenancy, extensibility)	A brand forced onto a shared stack that does not fit, or kept apart at twice the cost

Kestrel & Pine passed through the first three: Starting in 2016–2018 with one Dutch web shop and a 3PL, Growing from 2019 with the first store, Belgium, Germany, the marketplace connector and the integration platform, and Omnichannel from 2023 with its own warehouse, the ERP, more stores and wholesale on the platform's business-to-business features. The Omnichannel stage is where its 2027 problems live, because that is the stage at which three or four models share one stock and one catalog, and the joins between a dozen systems become the business.

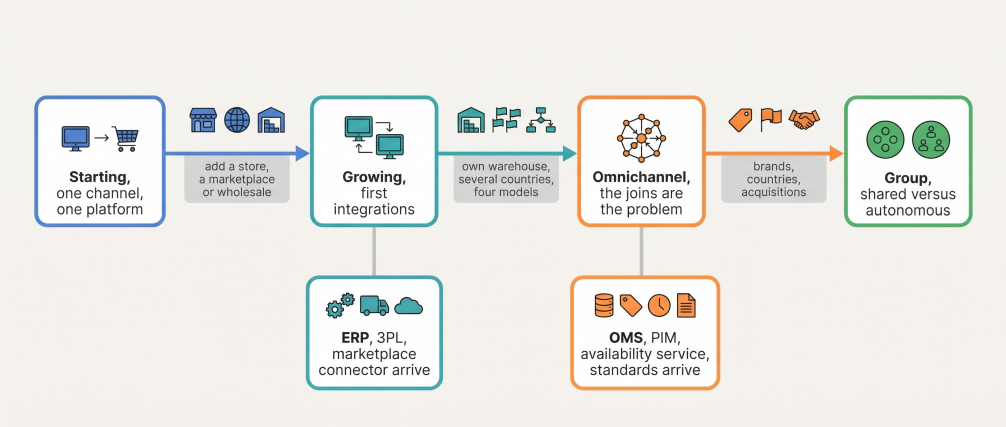


Figure 3: *Each stage transition adds a channel or a location and, with it, a join between systems; the hubs arrive at the Omnichannel stage because that is where the joins outnumber the boxes.*

Vendors confirm the stages in their price lists. Shopify's pricing page (n.d., accessed November 2026) runs from Basic at €28 a month (€21 paid yearly) to Plus "from €2,100" a month on one- or three-year terms, and its fee for using a third-party payment gateway falls from 2% on Basic to 0.2% on Plus. Shopware's pricing page (n.d., accessed November 2026) makes the Community Edition free until the shops linked to an account exceed €1 million of GMV in a year, after which a paid plan starts "from €600" a month, and states that pricing "is based on your gross merchandise value". Shopify's annual report says most merchants are on its two smallest plans while most GMV comes from Plus and enterprise. In plain words: a merchant crosses from a self-service plan to a negotiated contract at about the Growing-to-Omnichannel transition, and the fee cliffs and feature gates in a vendor's tiers are a map of which decisions the vendor expects at each size.

When the hubs arrive. No public source gives a revenue threshold for an order management system, a product information system or a warehouse management system, and the vendors' own guidance points at shape instead. Akeneo, a PIM vendor, names three signs (undated blog, accessed November 2026): product content scattered across teams, manual and ad hoc processes that make launching a new channel slow, and inconsistent product data in filters, search and translations. Practitioner guides for OMS converge on channel count and stock-location count, not order volume: several sales channels, and stock in a warehouse, stores or a marketplace's fulfillment center at once. My reading, offered as judgement: the PIM becomes necessary when the catalog is published to a third channel in a third format; the OMS when stock sits in a second location that must be sold from; the WMS (warehouse management system) when an own warehouse outgrows the ERP's stock module. All three conditions arrive at the Omnichannel stage, which is why Kestrel & Pine's Waypoint program introduces a PIM, an OMS and an availability service in the same year. What Changes at Each Scale: From First Store to Group works each transition in detail.

The Choices: One Stack, Many Stacks or a Shared Core

A merchant running several business models has three realistic ways to arrange its systems. Each is right somewhere.

One stack for every line. The consumer platform also runs the wholesale portal, the stores on its own point-of-sale product and the marketplace listings through its channel features; everything shares one catalog, one stock figure and one order list. It is the simplest to run and the cheapest to integrate, and it fits while the lines are small and alike. It limits when one line's promises do not fit the platform's model: contract prices per account with approvals, pallets with the buyer's labels, fiscal receipt rules in a store, or marketplace order formats the connector does not speak.

A stack per line. Each line buys the systems its model needs, each with its own orders, prices and stock, and gets the best fit for its promises. The cost is the joins: four order records, four prices for one jacket and four stock figures, which is Kestrel & Pine in January 2027 and the origin of the Price Drift, the Three-Status Order and the Store Stock Mirage.

A shared core with line-specific edges. The facts every line shares, the product, the price list, the stock and the order record, are owned once, in systems that serve all lines: a product hub, a pricing source, an availability service and an order hub. The promises that differ per line are made at the edges: the consumer checkout, the B2B portal, the till, the marketplace listing. This is the Waypoint shape, and the one most Omnichannel merchants arrive at. It costs more systems and more integration discipline than one stack, and it only pays once the joins have become the problem.

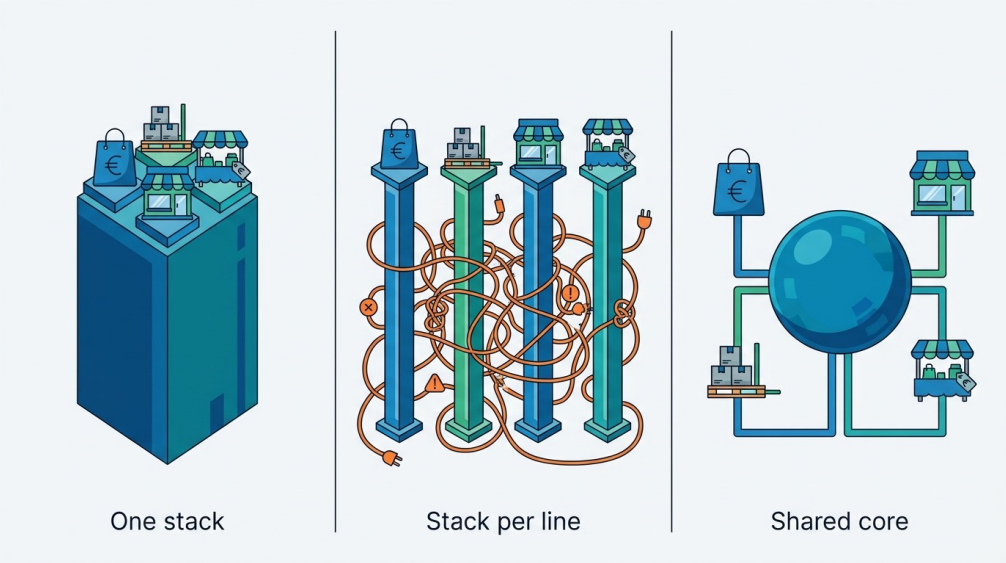


Figure 4: *One stack is simplest while the lines are alike, a stack per line multiplies the joins, and a shared core owns the common facts once while each line keeps its own edge.*

Criterion	One stack for every line	A stack per line	Shared core, line-specific edges
Number of order records	One	One per line	One, in the hub; the edges keep their own views
Fit of each line's promises	Only as good as the platform's model of that line	Best per line	Edges fit the line; the core fits all
Integration load	Lowest	Highest and unplanned	High but designed: every join has an owner and a contract
Typical stage	Starting, early Growing	Late Growing by accident	Omnichannel by decision; Group with per-brand edges
Warning sign	A line's team working around the platform in spreadsheets	Two prices for one jacket; support with five tabs	A core that becomes a bottleneck for every change

Criterion	One stack for every line	A stack per line	Shared core, line-specific edges
Kestrel & Pine	2016–2018	2021–2026	Waypoint, 2027–

My recommendation, with its conditions: **stay on one stack as long as every line's promises fit it, and move to a shared core the moment a second system starts writing a fact the platform also writes**, because that is the moment the joins start costing money. Do not move to a stack per line on purpose; it is only ever where a merchant ends up by buying one good system per need. And do not copy the shape of a marketplace operator, which runs tenancy, payouts and seller-of-record logic a brand never needs: the Giant's Boots.

In Practice: What the Public Record Says

Stores are the larger channel, and the architecture must treat them as one. Online sales were 13.5% of German retail in the latest HDE Online-Monitor (2026 edition, for 2025) and 17.1% of US retail in the second quarter of 2026 (Census Bureau, August 2026, seasonally adjusted). For a retailer with stores, the store is still where most sales happen; Target's annual report (March 2026) says its stores fulfilled 97.6% of its merchandise sales, including digital orders shipped from stores, picked up or delivered same-day. That is an order management decision at national scale, though Target does not name its system.

D2C is a channel mix with its own economics, and it can shrink. Nike's fiscal 2026 annual report (July 2026) reports NIKE Direct revenue of $17.7 billion against $18.8 billion the year before, "primarily driven by a decrease in traffic", and attributes its gross margin movement partly to "channel mix". VF's direct-to-consumer share is 44%. The lesson for an architect is that a brand's mix of D2C and wholesale moves year to year, and the platform must serve both without a replatform each time.

Marketplace operating is a different business. Amazon is not the seller of record for third-party sales and earned $172.2 billion from seller services in 2025 (annual report, February 2026); Marketplace Pulse estimated third-party sellers at about 61–62% of Amazon's units and 69% of an estimated $830 billion of 2025 GMV (February 2026; an analyst estimate). Zalando's partner business was 32.1% of its GMV in 2025 (annual report, March 2026). An operator's platform is built around sellers, listings, fees and payouts, which is why the book leaves it to Case: Amazon, From Monolith to Services to the Everything Store, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline and LISTED.

Business customers order by message, not by web shop. Eurostat's enterprise statistics (data extracted February 2026) put EDI-type sales at 11.07% of EU enterprise

turnover against 8.39% for web sales; Forrester's 2022 forecast (evidence age: more than three years) expected US B2B e-commerce to reach about $3 trillion by 2027 while tracking EDI separately at about a fifth of transactions. A wholesale line meets EDI before it meets a B2B web shop, as Kestrel & Pine's did.

Growth has normalized. The Dutch Thuiswinkel Markt Monitor (March 2026) reported Dutch online consumer spending of €35.7 billion in 2025, down 1%, and the German bevh (January 2026) reported online goods sales of €83.1 billion, up 3.2%. With growth in single digits, the Growing and Omnichannel stages compete on margin, returns and operations rather than on traffic, which is why the chapters on orders, returns and data matter more than they did in 2021.

When It Goes Wrong

Each model fails in its own way when it shares systems with another, and Kestrel & Pine has lived most of them.

Two lines, one jacket, two prices. The Price Drift of October 2026: a price-list sync to the German storefront failed silently, and the jacket sold at €179 for nine days while the marketplaces and stores charged €199. 410 orders, about €8,200 of margin, and a price-parity warning from a marketplace whose rules forbid a seller's own site undercutting the listing. Detection came from the marketplace, not from Kestrel & Pine. The remedy is one price owner per market and a reconciliation that compares channels daily (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax).

Wholesale on a consumer platform. The platform's B2B features carry 62% of wholesale orders; the other 38% arrive by e-mail and phone and are keyed into the ERP by Lea's team, so a third of the wholesale line's orders are born in a system the consumer line never sees. The ASN Chargeback of Q3 2026, €14,200 deducted by Bergzicht because the pallet labels did not match the despatch advice (the electronic advance shipping notice, ASN, that tells the buyer what is on each pallet), is a wholesale promise (the labels the buyer's warehouse requires) that no consumer system knows how to make (Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, Documents Between Companies: EDI, Punchout and E-Invoicing).

The store as an afterthought. Store stock reached the ERP nightly and never reached the platform, so the Utrecht click-and-collect pilot promised 37 items in December 2026 that the store had already sold: the Store Stock Mirage. The store line carries no parcel and no return parcel per order, and its stock was invisible to the channel that could have sold it (Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment).

Returns across lines. 24% of online items come back, and in January 2027 212 of them were refunded twice, once by the returns portal and once by hand, because two tools, the returns portal and the PSP's console, could each issue a refund and neither could see the other's. A return rate that high is a business-model fact, and the platform's reverse flow

has to be designed for it, not bolted on (The Way Back: Returns, Exchanges, Refunds and Reverse Logistics).

Detection in every case was the same: someone compared two systems' copies of the same fact. Recovery in every case was the same: decide who owns the fact and make the others consumers of it, which is Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record.

What to Take Away

Established.

- The models are defined operationally by the companies and agencies that report them: D2C as brand-operated channels (VF, Nike), wholesale as sales to resellers (Columbia), marketplace operating as not being the seller of record (Amazon), B2B2C as the partner owning the storefront while the brand keeps the offer and price (Zalando), omnichannel as stores fulfilling digital orders (Target, 97.6%).
- Business electronic ordering is mostly by structured message: EDI-type sales exceeded web sales as a share of EU enterprise turnover in 2024 (Eurostat).
- The order's economics are measurable and public benchmarks exist for each: conversion around 1–2%, cart abandonment about 70%, online return rates around a fifth and higher for apparel, interchange capped in the EU, PSP list prices published; the peak quarter carries a visibly larger share of sales (Census, Adobe, Shopify).

Depends on context.

- Whether one stack can serve several lines depends on whether each line's promises fit the platform's model; a small wholesale line on a consumer platform's business features works until contract pricing, approvals, pallets or EDI appear.
- Whether a merchant has reached the Omnichannel stage depends on shape, not revenue: three or four models on one stock, several locations and a dozen systems. Vendors' tier boundaries (Shopify Plus, Shopware's €1 million GMV threshold) are a reasonable proxy, not a definition.
- Whether payment fees are worth negotiating depends on volume and method mix; at a €92 basket and 293,000 orders, a half-point difference is material; at the Starting stage it is not.

The author's synthesis.

- I define a business model, for architecture purposes, as the answer to "who makes which promise": once that table is filled in for every line, the systems-of-record decisions mostly write themselves, and a vendor's model label stops mattering.

- I treat the unit-economics table as a map of where the architecture must be strong: a merchant with a 24% return rate must design the reverse flow first, and one with a 41-times peak hour must design the whole chain's capacity, not the storefront's.

Questions to Ask

1. For each business line, who captures the order, who sets the price, who is the seller of record, who holds the stock, who ships, who takes returns and who invoices?
2. Which facts do the lines share (product, price list, stock, order, customer), and which system owns each today?
3. What are the conversion rate, basket, margin, return rate, shipping cost and payment fee per line, and which of them has nobody measured?
4. What is the busiest hour of the year, how many times an average hour is it, and has every integration, not only the storefront, been tested at that load?
5. Which stage is the business in by shape, and which transition is next: a new channel, a new location, a new country, a new brand?
6. Where in the vendor's price list does the business sit, and which fee cliff or feature gate is nearest?
7. If a line's promises do not fit the shared stack, is the team working around it in spreadsheets, and who knows?
8. Which architecture is being copied, from whom, and does that company have the same business model?

The models and the numbers say what a commerce platform must do; they do not yet say what its parts are. The next chapter lays out the capability map: every part of a commerce platform, what each owns, and how they usually connect, so that the four business lines can be placed on it and the gaps seen. Continue with The Map of a Commerce Platform: Capabilities and Layers.

Sources and Evidence

Company, product and investor sources

- VF Corporation Form 10-K, fiscal 2026, VF Corporation / SEC, 2026. Definition of direct-to-consumer; 44% of revenue; gross margin 54.8%.
- NIKE, Inc. Form 10-K, fiscal 2026, NIKE / SEC, 2026. NIKE Direct revenue decline driven by traffic; channel mix; gross margin 42.9%.
- Columbia Sportswear Form 10-K, 2025, Columbia Sportswear / SEC, 2026. Wholesale vs direct-to-consumer channels; gross margin 50.5%.

- Zalando SE Annual Report 2025, Zalando SE, 2026. Partner business definition and 32.1% share of GMV; average basket €62.8; minimum order fee; gross margin 39.9%.
- Amazon.com, Inc. Form 10-K, 2025, Amazon / SEC, 2026. Not the seller of record for third-party sales; third-party seller services $172.2 billion; subscription services $49.6 billion.
- Amazon GMV surpassed $800 billion in 2025, Marketplace Pulse, 2026. Analyst estimate of third-party share of units and GMV.
- Target Corporation Form 10-K, fiscal 2025, Target / SEC, 2026. Stores fulfilled 97.6% of merchandise sales; Target Circle 360; gross margin 27.9%.
- Shopify Inc. Form 10-K, 2025, Shopify / SEC, 2026. GMV definition; most merchants on Basic and Grow, most GMV from Plus and enterprise.
- Shopify pricing, Shopify, n.d., accessed 2026-11. Plan prices in euros, Plus from €2,100, third-party gateway fees by tier, Shopify Payments rates.
- Shopware pricing, Shopware, n.d., accessed 2026-11. Community Edition free to €1 million GMV; Rise from €600; GMV-based pricing.
- BFCM data 2025, Shopify, 2025. $14.6 billion GMV, $5.1 million per minute peak, $114.70 average cart.
- Adobe: Cyber Monday hits record $14.25 billion, Adobe, 2025. Cyber Week $44.2 billion; $16 million per minute.
- Warby Parker Form 10-K, 2025, Warby Parker / SEC, 2026. Marketing expense 12.6% of revenue; gross margin 54.0%.
- Stripe pricing (Netherlands), Mollie pricing and Adyen pricing, n.d., accessed 2026-11. Card and iDEAL list prices.
- Amazon selling fees, Amazon, n.d., accessed 2026-11. Referral fee of 15% for most categories (US page).
- Top 3 signs you need a PIM, Akeneo, n.d., accessed 2026-11 (vendor content). Signs a PIM is needed.
- Hobbycraft to complete delayed digital roll-out by summer, Retail Week, 2013. Relaunch postponed past peak trading (evidence age).

Standards, regulation and official documentation

- E-commerce statistics (enterprises), Eurostat, 2026 (data extracted February 2026). Web sales vs EDI-type sales; marketplace vs own-site shares; B2B vs B2C web sales.
- Quarterly Retail E-Commerce Sales, 2nd Quarter 2026, US Census Bureau, 2026. 17.1% adjusted share; 18.3% vs 15.8% not adjusted in Q4 and Q3 2025.
- Regulation (EU) 2015/751 on interchange fees, European Union, Official Journal, 2015. Caps of 0.2% and 0.3% for consumer cards, applied from 9 December 2015.
- HDE Online-Monitor, Handelsverband Deutschland, 2026. Online share of German retail 13.5%.

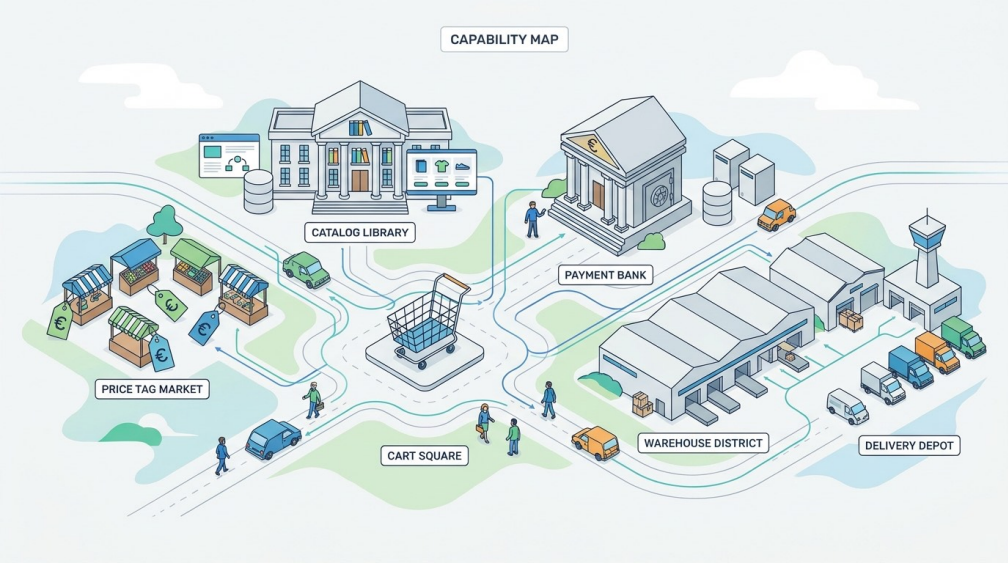
- VAT rates and exemptions, Netherlands Enterprise Agency, n.d., accessed 2026-11, and § 12 UStG, German Federal Ministry of Justice, consolidated text. Dutch 21% and German 19% standard VAT rates used in the worked example.

Research and practitioner sources

- Average ecommerce conversion rate, Littledata, 2026. Median 1.3%, mean 1.9% across 438 Shopify stores.
- 50 cart abandonment rate statistics, Baymard Institute, 2026. 70.22% average; reasons for abandonment.
- 2025 Retail Returns Landscape, National Retail Federation and Happy Returns, 2025. 19.3% of online sales returned.
- The true cost of apparel returns, Coresight Research, 2023. 24.4% online apparel return rate (survey of 100; evidence age).
- Forrester: US B2B e-commerce will reach an estimated $3 trillion by 2027, Forrester, 2022. B2B e-commerce forecast; EDI tracked separately (evidence age).
- Thuiswinkel Markt Monitor 2025, NielsenIQ for Thuiswinkel.org, 2026. Dutch online spending €35.7 billion, down 1%.
- bevh press releases, bevh, 2026. German online goods sales €83.1 billion in 2025.
- 8 signs your brand needs an order management system, Opollo, n.d., accessed 2026-11 (vendor content). Channel and location count as OMS triggers.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

3. The Map of a Commerce Platform: Capabilities and Layers



IN THIS CHAPTER, YOU WILL: Learn the capabilities a commerce platform is made of, what each one owns, how they connect, and how to place any vendor or system on the map.

KEY POINTS:

- **A capability is a job the business needs done, not a product.** "Know what we can still sell" is a capability; the warehouse system, the ERP (the back-office system that keeps the books) and an app are three systems that may each claim it. Draw the map in capabilities first, then place systems on it, and the gaps and overlaps appear by themselves.
- **Every capability owns one kind of business fact, and most of the others only copy it.** Product content, price, stock, order, payment, shipment, return, invoice and customer each need one home; the flows between capabilities are how the copies learn about changes. Where two systems both believe they own a fact, that is where the money leaks.

- **Vendors cover the map in three ways, and none covers all of it**. A suite covers many boxes with one vendor's data model; a platform plus apps covers the center and rents the edges; a best-of-breed assembly buys one product per box and makes the joins your problem. The right mix depends on the business shape and the stage, and a vendor's product page is read best with the map beside it.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Spaghetti Junction** — connectors multiply when nobody has drawn which capability feeds which, so every new need gets its own wire.
- **The Giant's Boots** — a map copied from a far larger company fills boxes that a smaller merchant does not need and cannot run.
- **The Lynchpin** — a capability that nobody has placed on the map is usually running through one vendor, one script or one person without anyone noticing.

Ask five people at a merchant what "the e-commerce platform" is and you get five answers. To the storefront manager it is the web shop and its apps. To finance it is whatever sends orders into the books. To the warehouse it is the thing that drops pick lists into the WMS (the warehouse management system that runs receiving, picking and packing). To the wholesale team it is a portal with price lists. To the agency it is a theme (the storefront's page templates) and an API. **All five are right, because a commerce platform is not one system. It is a set of capabilities, and in any merchant past its first year those capabilities are spread over a dozen systems from several vendors.**

This creates a practical problem before any technical one. When a need appears ("we want click-and-collect", "the marketplace wants stock every five minutes", "finance needs one order count") the first question is which capability it belongs to, which system holds that capability today, and which other systems depend on it. Without a map those questions are answered by whoever is in the room, and the answer is usually a new connector or a new app. That is how a landscape reaches 34 integrations and 41 apps, as Kestrel & Pine's did, with no single decision that anyone would call wrong.

This chapter draws the map. It names the capabilities, groups them into layers, says what business fact each one owns, shows the flows that join them, compares the three ways vendors cover the map, and then places Kestrel & Pine's systems on it, gaps included. Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record takes the next step and decides who owns each fact when several systems hold it; Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026 surveys the vendors.

Where we are in the order's journey. This chapter does not sit at one step of Eva's jacket; it sits above all of them. Every step in Follow the Order: What Happens Between "Buy" and "Delivered", from the search for a jacket to the credit note for the one that came back, is a capability on this map, and each handoff between steps is a flow on it.

Setting the Scene at Kestrel & Pine: A Whiteboard in February 2027

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), had thirteen systems and no picture of them when Noor Visser started as Head of Engineering in January 2027. Ruben Alves, the lead integration engineer, kept an inventory of the 34 integrations. Milan Petrov knew the 41 apps. Sanne de Wit knew what reached the ERP. Nobody had the whole.

In February, in her first month, Noor drew it on a whiteboard, in an unusual order. She did not start with the systems. She started with the jobs the company needed done to sell a jacket: show it, price it, say whether it is in stock, take the order, take the money, pick it, ship it, take it back, invoice it, count it. About two dozen boxes. Only then did she write the thirteen system names into them.

Three things were visible within an hour that had not been visible in five years of buying systems one at a time:

- **Two boxes were empty**. No system held the enriched product record (a product information management system, PIM); product content lived in the platform, the item master of the ERP (the enterprise resource planning system: the books, purchasing and stock valuation), shared spreadsheets and the marketplace connector. No system held every order (an order management system, OMS); orders lived in the platform, the connector, the ERP and the point-of-sale system (POS, the store's till software).
- **One box had six writers**. Warehouse stock was written by the WMS feed, by hand in the ERP, by hand in the platform and by three apps, with the POS writing store stock separately. The Black Friday Oversell of November 2026 (1,340 units oversold, 1,120 orders canceled or delayed, about €96,000) and the App That Wrote Stock of March 2026 (phantom reservations on 600 variants) were both on that box.
- **Two more boxes were nobody's**. Customer identity (is the store customer the online customer?) and availability (what each channel may still sell) had no system and no owner. So, in a smaller way, did the delivery promise: a sentence on the web page, "ordered before 22:00, delivered tomorrow", computed from nothing and owned by the shipping box only on paper.

The whiteboard became the first working document of the Waypoint program approved in March. The rest of this chapter is that whiteboard, generalized.

What a Capability Is, and Why It Comes Before the Systems

A **capability** is a job the business needs done, named without reference to who does it. "Calculate the price a given customer pays for a given product today" is a capability. So is "know how many units of a variant are physically on a shelf in Duiven". A **system** is a piece of software that does one or more of those jobs; a **vendor product** is a system someone sells, usually claiming several capabilities at once. A **layer** is a group of capabilities that share a purpose and a rhythm: the ones a shopper touches, the ones that describe what is for sale, the ones that move money and orders, the ones that move boxes, the ones that talk to partners, and the ones underneath everything.

The distinction matters because systems and vendor products overlap, and capabilities do not. The ERP, the WMS and the commerce platform all have a stock field; there is still only one job called "know what is physically on the shelf", and only one system should be doing it. When you draw the map in capabilities, an overlap shows as one box with two names in it, and a gap shows as an empty box. When you draw it in systems, overlaps and gaps are invisible, because every system looks complete from the inside.

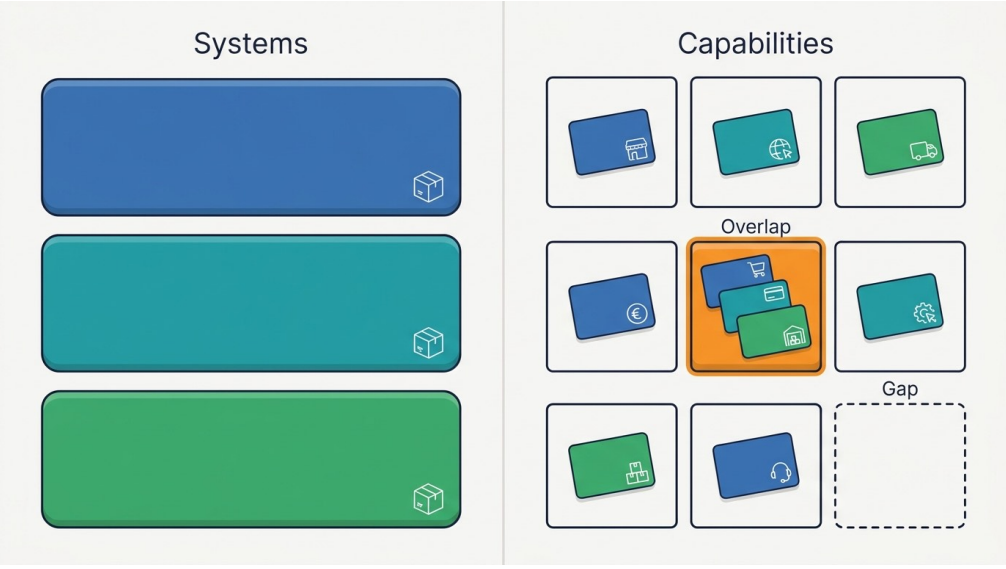


Figure 1: *Drawn as systems, every box looks complete; drawn as capabilities, an overlap shows as one box with several names and a gap as an empty box.*

Two more terms. A **business fact** is something the business needs to be true about the world: this variant has this GTIN (the Global Trade Item Number behind its barcode), this customer pays this price, this order has shipped. Most capabilities exist to create or keep one kind of fact. And a **flow** is the movement of a fact, or a request about it,

from one capability to another: the order flows from checkout to order management to the warehouse; the stock level flows back. The map is boxes (capabilities) and arrows (flows), and later chapters are mostly about one box or one arrow each.

The Map: Six Layers

The map below is the author's synthesis, built from how the specialist vendors describe their own products (an OMS vendor's and a PIM vendor's definitions are quoted later in this chapter) and from the capability inventories of several commerce product families. It is a teaching map, not a standard: other authors cut the layers differently, and a vendor's architecture diagram will usually show its own product as the center.

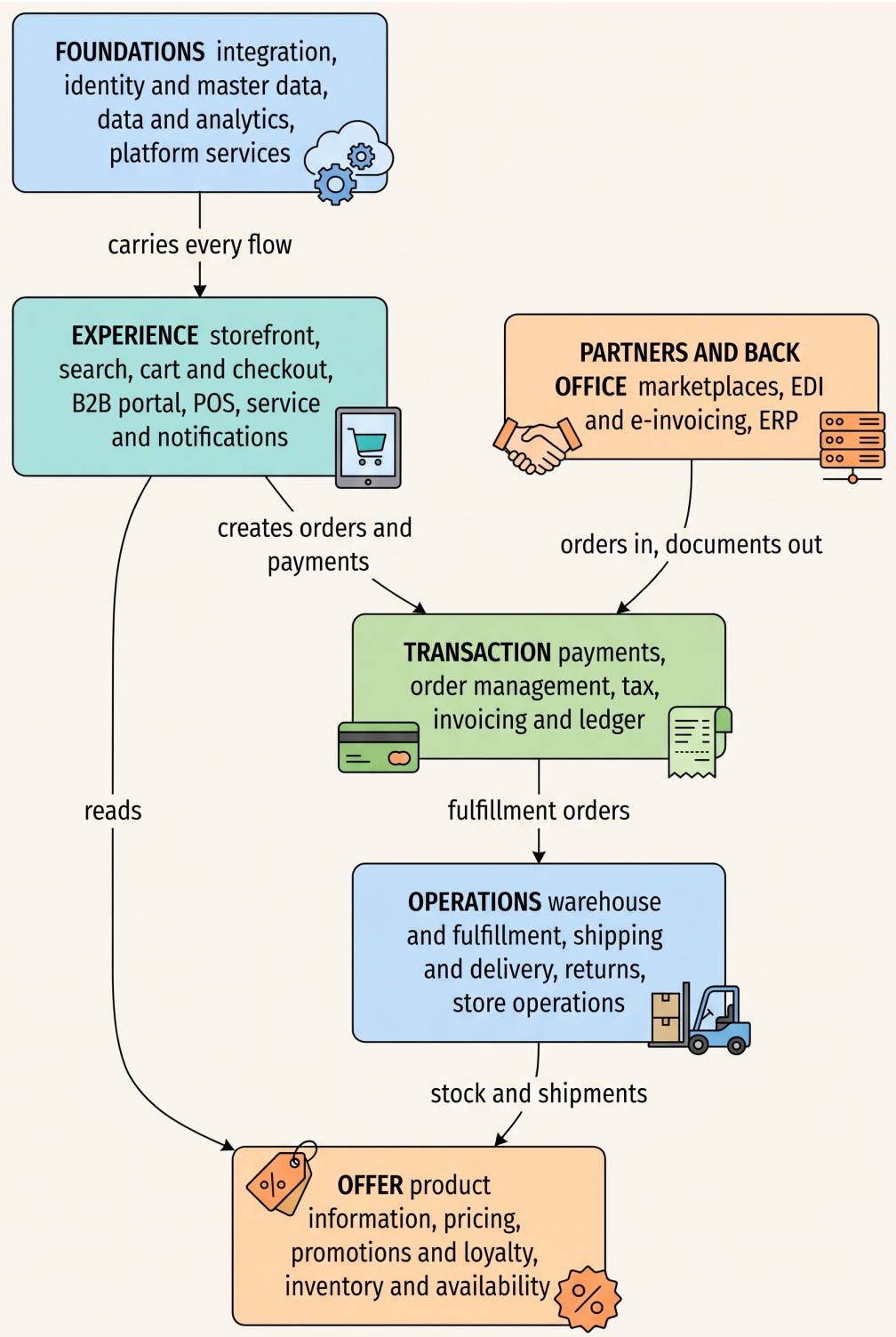

FOUNDATIONS integration, identity and master data, data and analytics, platform services
carries every flow
EXPERIENCE storefront, search, cart and checkout, B2B portal, POS, service and notifications
PARTNERS AND BACK OFFICE marketplaces, EDI and e-invoicing, ERP
creates orders and payments
orders in, documents out
TRANSACTION payments, order management, tax, invoicing and ledger
reads
fulfillment orders
OPERATIONS warehouse and fulfillment, shipping and delivery, returns, store operations
stock and shipments
OFFER product information, pricing, promotions and loyalty, inventory and availability

Figure 2: *Six layers of a commerce platform: the experience layer reads the offer layer and creates transactions, transactions become operations, operations feed stock back into the offer, partners inject orders and documents from outside, and the foundations carry everything.*

From the shopper inward: the **experience** layer is everything a shopper, store customer, wholesale buyer or service agent touches. The **offer** layer says what is for sale, at what price, under which promotion and whether it is available; it makes the promises the rest must keep. The **transaction** layer is the order and the money. The **operations** layer is the physical world: warehouse, carrier, store shelf, return parcel. The **partners and back office** layer holds systems that belong to other organizations or to finance: marketplaces, trading partners that speak EDI (electronic data interchange, the long-established standard for exchanging orders and invoices between companies' systems) and the ERP (enterprise resource planning system: the books, purchasing, stock valuation). The **foundations** are what no shopper sees and every layer depends on.

What each capability owns

This table is the chapter's main deliverable. For each capability it gives the job in plain words, the business fact it should own (the thing it is the natural home for), and the chapter that works it out. Terms used in the table are defined in those chapters; the one-line jobs are enough to place a system.

Layer	Capability	The job, in plain words	The fact it is the natural home for	Chapter
Experience	Storefront	Show products and content to shoppers on web and app, fast and accessible	Pages and themes (not the product data they show)	Delivering the Storefront: Themes, Headless, Rendering and Performance
Experience	Search and discovery	Let shoppers find products by query, filter and recommendation	The search index, a rebuildable copy of the catalog	Finding the Product: Search, Navigation and Merchandising

Layer	Capability	The job, in plain words	The fact it is the natural home for	Chapter
Experience	Cart and checkout	Hold a shopper's draft order and turn it into a paid order in a few steps	The cart (short-lived) and the checkout session	From Cart to Confirmation: Checkout Orchestration
Experience	B2B (business-to-business) buyer experience	Let a company's buyers order at their prices, with approvals, quotes and reorders	Company accounts, buyer roles, quotes	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals
Experience	Point of sale	Sell in a store, offline if needed, with receipts and store stock	Store transactions and store stock	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
Experience	Customer service and notifications	Tell the customer what is happening and show an agent one view of it	Notification events sent; the service view (a copy)	Where Is My Order? Notifications, Service and the Customer Record
Offer	Product information	Hold products, variants, attributes, media and translations once, publish to every channel	Product content and the product identifiers	What Are We Selling? Products, Variants and the Catalog

Layer	Capability	The job, in plain words	The fact it is the natural home for	Chapter
Offer	Pricing	Compute the price a given customer pays for a given product in a given channel today	Price lists, contract prices, effective dates	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
Offer	Promotions and loyalty	Apply discounts, coupons, points and store credit without breaking price or margin	Promotion rules; store credit and gift card balances (liabilities)	Discounts, Coupons and Loyalty: Promotions Without Chaos
Offer	Inventory and availability	Know what is physically where, and compute what each channel may still sell	Available-to-promise per channel (derived from physical stock)	Is It in Stock? Inventory, Availability and the Promise You Can Keep
Transaction	Payments	Take the shopper's money, refund it, and reconcile it with payouts	Payment transactions (in the PSP, the payment service provider; the order keeps a reference)	Taking the Money: Payment Providers, Authorization, Capture and Refunds

Layer	Capability	The job, in plain words	The fact it is the natural home for	Chapter
Transaction	Order management	Hold every order from every channel, in one state model, and steer it to fulfillment and back	The order record and its state	The Order Record: Lifecycle, State and the Order Management System
Transaction	Tax, invoicing and ledger	Turn an order into tax, a legal invoice and entries in the books	Invoices, credit notes, ledger postings	Tax, Invoices and the Books: From Order to Ledger
Operations	Warehouse and fulfillment	Receive, store, pick, pack and confirm shipment of physical goods	Physical stock on hand and its movements	Pick, Pack, Ship: Warehouse Management and Fulfillment
Operations	Shipping and delivery	Choose carriers, print labels, track parcels, promise a delivery date	Shipments, tracking events, the delivery promise	Getting It There: Carriers, Labels, Tracking and the Delivery Promise
Operations	Returns and exchanges	Take goods back, inspect, grade, refund or exchange, restock	Return requests, inspection results, dispositions	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics

Layer	Capability	The job, in plain words	The fact it is the natural home for	Chapter
Operations	Store operations	Run the shelf: store stock counts, transfers, click-and-collect, in-store returns	Store stock (shared with POS)	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
Partners	Marketplaces and channel integration	List, sell and fulfill on other companies' platforms, in their formats	Listings and marketplace orders (copies of product and order)	Selling on Someone Else's Platform: Marketplaces and Channel Integration
Partners	EDI and e-invoicing	Exchange orders, despatch advices and invoices with large partners and tax authorities as structured documents	Sent and received documents (immutable once sent)	Documents Between Companies: EDI, Punchout and E-Invoicing
Back office	ERP and back office	Keep the books, buy from suppliers, value the stock, hold the item and customer masters	Ledger, purchasing, stock value, item identity	The Back Office: ERP Integration and Who Is the Hub

Layer	Capability	The job, in plain words	The fact it is the natural home for	Chapter
Foundations	Integration	Move facts between systems reliably: styles, contracts, retries, reconciliation	The integration inventory and each flow's contract	APIs, Events, Files and Middleware: How Commerce Systems Talk, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Promises Between Systems: API Contracts, Webhooks and Versioning
Foundations	Identity and master data	Decide that two records are the same customer or product, and keep masters in step	Customer identity; the cross-reference of product identifiers	One Customer, One Product: Identity and Master Data Across Systems
Foundations	Data and analytics	Count the same thing the same way everywhere	Metric definitions; the warehouse (a copy of everything)	Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics

Layer	Capability	The job, in plain words	The fact it is the natural home for	Chapter
Foundations	Platform services	Serve many merchants on shared infrastructure: tenancy, extensibility, peak capacity, security	The platform's own configuration, apps and limits	Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Black Friday Every Year: Peak Load, Resilience and Recovery, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers

Two columns deserve a note. The "fact it is the natural home for" column is the hinge to the next chapter: when two systems both hold a fact, the capability map says which one *should* be its home, and Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record says how to make that decision stick. And several entries say "a copy": the search index, the service view, the listing, the data warehouse. Those capabilities hold copies by design, which is fine as long as everyone knows they are copies and they are rebuilt from the home, never edited in place.

Facts that are derived, not recorded

A few capabilities own something that is computed rather than entered, and newcomers find this the hardest idea on the map. **Availability** is the clearest case. Physical stock on hand is a recorded fact: someone counted it in Duiven. What a channel may still sell (industry practice calls it **available-to-promise**, or ATP: on hand, minus what is already promised to unshipped orders, minus any buffer the merchant holds back) is a derived fact. The warehouse does not own it, because it does not know what the storefront has promised; the storefront does not own it, because it does not know what the warehouse has. Something in between has to compute it from both, and that something is the inventory-and-availability capability, whether it is a module of an OMS, a feature of the platform or a service of its own. The delivery promise ("delivered Thursday") is derived the same way from cut-off times, carrier lead times and location. Derived facts need an owner too, and their owner is the computation, which is why a hand-typed override in a derived number is one of the surest ways to create the Phantom Stock.

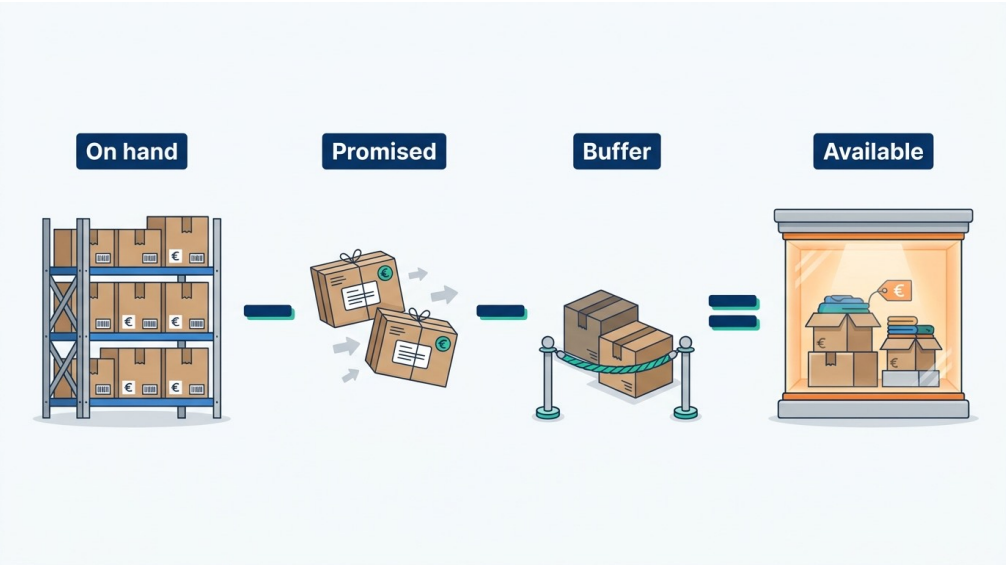


Figure 3: *Stock on hand is counted; what a channel may still sell is computed from on hand, what is already promised and the buffer, so its owner is the computation.*

How the Pieces Talk: The Typical Flows

Boxes are half the map. The arrows between them carry a small number of fact types, each with its own rhythm. The table below lists the flows almost every merchant above the first stage runs, the direction they usually take, and the requirements that decide how they should be built. "Freshness" is how old a copy may be before it causes harm;

"consistency" is whether the copies must agree exactly or only eventually; the last two columns say what happens when the flow is slow or down. The requirements are the author's judgement from industry practice; the next chapters refine them.

Flow	From → to	Freshness needed	Consistency	If slow	If down
Product content	Product information → storefront, marketplaces, EDI partners	Minutes to hours	Eventual; completeness matters more than speed	A new product appears late	Channels keep yesterday's catalog
Prices	Pricing → storefront, POS, marketplaces, B2B portal	Minutes; exact at a promotion's start and end	Exact across channels (a stale price is a broken promise, and on marketplaces a parity breach)	The Price Drift	Channels sell at the last known price
Stock and availability	Warehouse, stores, external locations → availability → every channel	Seconds to minutes at peak	Eventual, with a buffer and a reconciliation	Oversell window grows with the lag	Channels sell blind or stop selling

Flow	From → to	Freshness needed	Consistency	If slow	If down
Orders	Checkout, marketplaces, EDI, POS → order management → warehouse, ERP	Seconds to minutes	Exact: never lost, never doubled	Late picking, missed cut-off	Orders queue; nothing ships
Payments	Checkout → PSP → order management, ledger	Seconds at authorization; days at settlement	Exact to the cent, reconciled against payouts	Checkout slows, conversion drops	Checkout stops
Shipments and tracking	Warehouse and carriers → order management → notifications, service	Minutes	Eventual	"Where is my order" tickets rise	Customers unaware; service blind
Returns and refunds	Returns portal → warehouse, order management, PSP, ledger	Hours	Exact for money; eventual for stock	Refunds late, complaints	The Boomerang begins
Finance	Order management, PSP → ERP ledger	Daily batch is normal	Exact, reconciled per period	Month-end slips	Books lag, nothing else does

Two patterns stand out. **Only three flows need exactness in near real time: prices, orders and payments.** Everything else tolerates lag if there is a reconciliation (a job

that compares two systems' copies and reports the differences) behind it. And **finance works in periods, not seconds**: the ledger can be fed nightly and nobody loses a sale, which is why The Back Office: ERP Integration and Who Is the Hub argues that the ERP's rhythm must never become the storefront's availability rhythm.

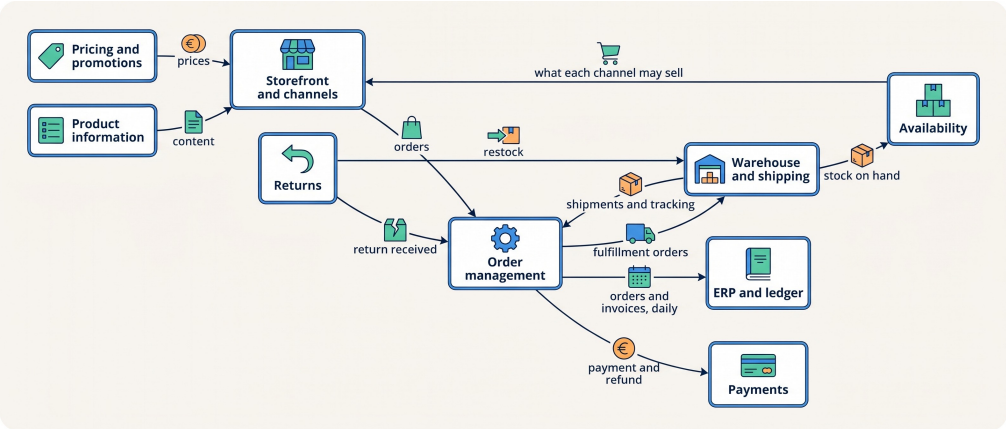


Figure 4: *The flows of a typical merchant: product, price and availability flow outward to the channels; orders flow inward to order management and down to the warehouse; stock, shipments and returns flow back; finance receives a daily batch.*

Every arrow is a place where one system's fact becomes another system's copy, and a copy can be late, lost or duplicated, which is why the joins, not the boxes, are where commerce platforms fail. Part VI — INTEGRATION: Making Separate Systems Behave Like One is about the arrows.

Three Ways to Cover the Map

No merchant builds all of this, and no vendor sells all of it, whatever the brochure says. In practice the map is covered in one of three ways, often mixed.

A suite. One vendor supplies most of the experience, offer and transaction layers, and sometimes order management and POS, in one data model. On 6 July 2026 Salesforce described its Agentforce Commerce release as one platform across B2C, B2B, point of sale and order management, with agents "natively connected to catalog, inventory, and orders". Adobe's October 2026 information center lists three ways to run Adobe Commerce (on-premises, on Adobe's cloud infrastructure, or as a fully managed cloud service) and a separate SaaS, Commerce Optimizer, that "can work alongside an existing Adobe Commerce or third-party commerce backend". The suite's promise is that the joins inside the covered boxes are the vendor's problem. Its limit is that the warehouse, the carriers, the PSP, the marketplaces, the EDI partners and the ERP are still outside,

and the vendor's data model decides what "order" and "product" mean for everything inside.

A platform plus apps. A hosted SaaS platform (software the vendor runs for thousands of merchants at once, rented by the month) covers storefront, catalog, cart, checkout, basic inventory and basic order handling, and an app store rents the rest: reviews, returns, search, subscriptions, pre-orders, loyalty. Shopify's pricing page in November 2026 lists plans from €28 to a Plus tier "from €2,100" a month, with lower card rates on higher tiers; the platform is also a payments company, as is the POS vendor Lightspeed, whose fiscal 2026 results showed transaction-based revenue of $815.1 million against subscription revenue of $370.7 million. Hosted platforms tend to retreat from the operations layer: Shopify completed the sale of most of Shopify Logistics to Flexport on 6 June 2023 (an old event, but the direction has held). The center is excellent and cheap to start; the apps are many small vendors, each writing into your data (Kestrel & Pine's 41 apps, three of them writing stock), and the operations and partner layers are yours to assemble.

Best-of-breed, or composable. One product per box, chosen for that box, joined by APIs and an integration platform. The MACH Alliance (founded in June 2020 by commercetools, Contentstack, EPAM and Valtech; the acronym stands for microservices, API-first, cloud-native and headless; headless means the storefront is a separate application that talks to the commerce engine only through APIs) made this the named alternative to the suite. The specialist layers are real products with their own definitions: Manhattan describes an OMS as a platform "designed to track and manage the lifecycle of an order from start to finish", and Akeneo describes a PIM as a system "to centralize, manage, enrich, and distribute product information for commerce" (both n.d., accessed November 2026). Each box is best in its class and replaceable; the cost is the joins. The Alliance's own February 2026 survey of 600 enterprise technology leaders found 37% naming integration complexity as a primary concern, a figure against the interest of the people who published it.

Two variants are worth naming. **ERP-centered commerce** makes the ERP the center and the storefront a front end; Microsoft's documentation of 22 September 2026 says its Business Central Shopify Connector "is preinstalled for all new sign-ups and trials", which is the clearest public sign that ERP plus hosted storefront is a mainstream small-merchant shape. And a **specialist group** pre-integrates several best-of-breed products under one owner: Katapull, a Dutch group, announced on 31 March 2025 that five companies had joined forces and on 7 October 2026 added a WMS vendor as its eighth label, listing on its site an ERP-and-webshop product, a POS, a WMS, a marketplace integrator, an EDI provider, a returns product, a shipping product and an online-performance agency. On the map that is six operational and partner boxes from one group, with the storefront platform still bought elsewhere; the public pages give no figures beyond customer and user counts, and the book uses nothing else about it.

Trade-offs

	Suite	Platform plus apps	Best-of-breed	Specialist group
Boxes covered by one vendor	Many (experience, offer, transaction, often OMS and POS)	The center (storefront to basic orders)	One each	Operations and partners, pre-joined
Where the joins live	Inside the vendor, except operations and partners	Between platform and apps; all of operations and partners	Everywhere; the integration layer is the architecture	Inside the group; the join to the storefront is yours
Who defines "order" and "product"	The suite	The platform, with app fields bolted on	You, per box	The group's ERP or hub
Cost shape	Negotiated, often linked to sales	Subscription plus payment fees plus apps plus agency	Per product, per order, per request; plus integration	Per product
Time to first sale	Months	Days	Months	Weeks
Fits	Group-stage and large omnichannel retailers with implementation partners	Starting and Growing merchants; Omnichannel merchants who surround the platform	Omnichannel and Group merchants with a platform team	Growing and early Omnichannel merchants in the group's home market

	Suite	Platform plus apps	Best-of-breed	Specialist group
Limits	The vendor's model; cost at scale; upgrade projects when customized	App sprawl; the operations layer is unassembled; fees grow with sales	Integration cost and skills; vendor count	One owner across several layers; exit from the group

The table is the author's judgement. My rule for a merchant that is not a vendor: **cover the center with one product you do not customize, cover operations with products built for operations, and own the joins yourself**. Kestrel & Pine's Waypoint decision W4, stay on the SaaS platform and surround it with an OMS, a PIM and an availability service, is that rule applied; Build, Buy or Assemble: Choosing and Changing Platforms works the economics.

Across Scale Stages: When It Fits and When It Limits

The map does not change with scale. What changes is how many boxes have their own system, and which ones. The table uses the four stages defined in Who Sells What to Whom: Business Models and the Economics That Shape Architecture.

Stage	Boxes with their own system	Typical shape	Warning sign that the shape is ending
1 — Starting	Three or four: a hosted platform covers experience, offer and transaction; a PSP; an accounting package; the portal of a 3PL (a third-party warehouse that ships on the merchant's behalf)	The platform is the map	A second channel (first store, first marketplace) with its own order source
2 — Growing	Six to nine: an ERP or accounting system claims items and finance; a marketplace connector; a POS; an integration tool	Platform plus apps, with the ERP as a second center	Two systems writing stock; a spreadsheet as the product master
3 — Omnichannel	Twelve or more: own WMS, returns portal, shipping service, EDI provider, data warehouse; then OMS, PIM, availability	The joins become the architecture; the empty boxes (OMS, PIM, availability) must be filled	Support checks three systems for one order; a sync chain measured in hours
4 — Group	The same map per brand or country, with some boxes shared (PIM, OMS, data, integration) and some kept separate	Shared services versus brand autonomy	Brands forced onto one stack before the benefit is measured, or kept apart when they should share

Kestrel & Pine is at stage 3 with four boxes still empty. The judgement: fill a box with a dedicated system when the capability has more than one producer and more than one consumer and the system that holds it today cannot carry the workflow. A PIM is unnecessary at stage 1 and overdue at stage 3 with four channels; an OMS is overkill with one channel and one warehouse and overdue the day a second order source appears.

Reading a Vendor's Product Page Against the Map

A product page lists features. The map lets you turn a feature list into three questions.

1. **Which boxes does it claim?** Mark them. A channel-integration product that claims "order management" usually means it imports marketplace orders and pushes status back; that is a flow on the map, not the order-management box. A platform that claims "inventory" usually means it holds a stock number per location; that is a copy of the warehouse's fact, not the warehouse.
2. **Which fact does it own, and which does it copy?** For each claimed box, ask whether the product would be the home of that fact or a reader of it. Fluent Commerce, an OMS vendor, states the boundary on its own site: "your ERP stays the system of record" (n.d., accessed November 2026). A vendor that cannot answer the question for its own product is telling you something.
3. **What is the unit behind the numbers?** Vendor claims count different things. "1,300 marketplaces" (ChannelEngine's site, November 2026) counts connectors that exist, not connections that are maintained. "4,500+ customers" and "27,500+ users" (Katapull's about page) differ by a factor of six because users are seats. GMV (gross merchandise value, the value of orders passing through a platform before refunds and before the vendor's cut) is sales volume, not the vendor's revenue; Shopify's reported 2025 GMV of about $378 billion is roughly thirty times its revenue of about $11.6 billion. Ask for the unit, the date and the definition before comparing two vendors; Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026 has the full list of traps.

The same three questions apply inside your own company, to the systems you already have. Most landscapes contain a system that was bought for one box and has quietly grown into three.

In Practice: Kestrel & Pine's Systems on the Map

This is Noor's February whiteboard as a table, reduced to the boxes that were not simply covered. The platform covers storefront, search, cart, checkout, promotions and the B2B portal; the WMS, the shipping service, the returns portal, the marketplace connector, the

EDI provider and the ERP each cover their own box; the platform vendor supplies the platform services. Systems are named by role, as the book does throughout.

Capability	System in January 2027	Status on the map
Product information	**Nobody**. Platform, ERP item master, spreadsheets, connector	**Empty box, four copies**
Order management	**Nobody**. Orders in the platform, the connector, the ERP, the POS	**Empty box, four order sources**
Inventory and availability	**Nobody computes availability**. Stock on hand in the WMS, ERP, platform, POS	**Empty box for availability; six writers of stock**
Identity and master data	**Nobody**	**Empty box**; Eva Janssen exists four times
Pricing	Platform price lists; ERP for wholesale; connector for marketplaces	Overlap: three writers; the Price Drift (€179 for nine days, about €8,200 lost)
Payments	The PSP online; a terminal provider in stores	Covered, but refunds also issued by hand in the PSP console (the Double Refund, €19,400)
ERP and back office	The ERP (since 2023)	Covered, and also used as order entry and stock master: five jobs in one box
B2B buyer experience	The platform's B2B features	62% of wholesale orders; 38% keyed into the ERP by hand
Point of sale, store operations	The POS (separate vendor, since 2019)	Store stock reaches the ERP nightly; no link to the platform
Shipping and delivery	The shipping service inside the WMS flow	The delivery promise is static text, computed from nothing

Capability	System in January 2027	Status on the map
EDI and e-invoicing	The EDI provider, for Bergzicht Outdoor and for Peppol (the European network for exchanging e-invoices)	One consultant knows the mappings (The Lynchpin)
Integration	The iPaaS (integration platform as a service, a hosted tool for building the flows between systems) plus agency scripts and vendor connectors; 34 integrations	Covered without standards
Data and analytics	The data warehouse and a BI (business intelligence) reporting tool	Three definitions of "order"

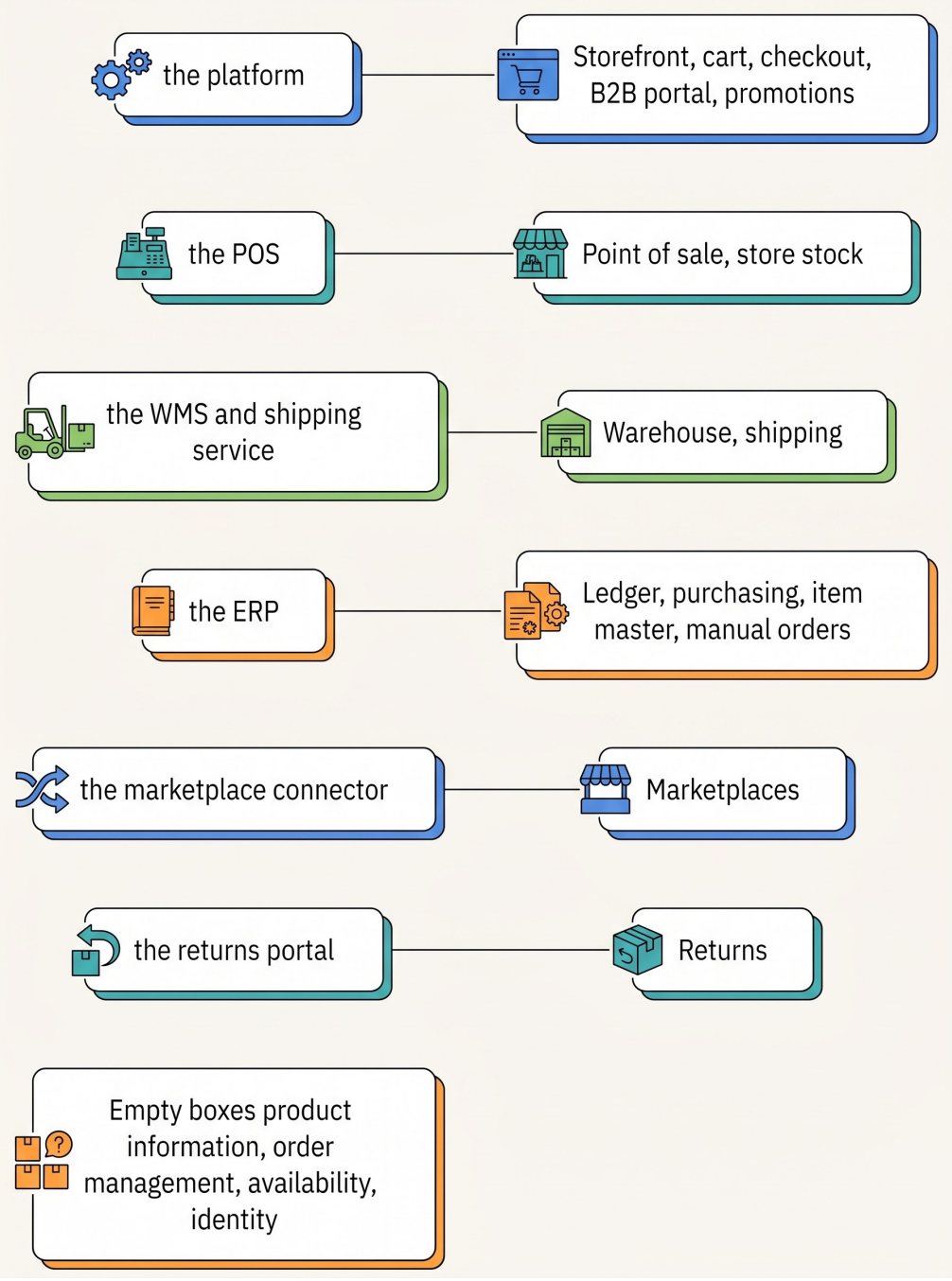


Figure 5: *Kestrel & Pine's thirteen systems fill most of the map, but four boxes have no system: product information, order management, availability and customer identity. Every 2026 incident sits on an empty box or on a box with several writers.*

The Waypoint decisions read as a plan to fill the empty boxes and thin the overlaps:

a PIM for product information (W3, Q3 2027), an OMS for order management with an availability service as one of its modules (W1 and W2, Q4 2027), the WMS as the single writer of physical stock (W1), and integration standards for the arrows (W5). Identity remains open; One Customer, One Product: Identity and Master Data Across Systems takes it up.

When It Goes Wrong

The map fails in four ways, and each shows up on the map before it shows up in the business.

An overlap: two systems own one fact. The App That Wrote Stock is the textbook case. A pre-order app adjusted inventory in the platform while the WMS feed also wrote it; for two weeks 600 variants carried phantom reservations. Detection is a *write audit*: for each box, list every system, app, script and person that can change its fact. If the list has more than one entry for a recorded fact, that is an overlap. Recovery is to name the single writer and re-publish the fact from it, which is exactly what the first weekly reconciliation in May 2027 did when it found 2.3% of active variants with a different quantity between the WMS and the platform (an illustrative figure).

A gap: nobody owns the fact. The gap fills itself with spreadsheets and with copies that start acting as records. Kestrel & Pine's product content lived in four places because no box was named for it; its order status lived in four places because none was. Detection is the empty box on the map. Recovery is to name a home before buying a system for it, because a bought OMS fed by four unreconciled order sources is a fifth copy.

A box held by a vendor who sells a different box. A channel connector that is also used as the stock master, or an ERP used as the order hub, works at the stage it was bought in and becomes the bottleneck later, when the warehouse needs stock in minutes and the ERP posts hourly. Detection: compare the box the system was bought for with the boxes it now holds. The ERP that holds purchasing, the ledger, the item master, manual wholesale orders and the stock master is doing five jobs, two of them badly.

The map copied from a giant. A merchant at stage 2 that draws a stage-4 map, with a system per box and a platform team to run them, has bought the Giant's Boots. The boxes are the same; the number of systems that should fill them is not. Below the producer-and-consumer test above, the box stays inside the platform.

What to Take Away

Established.

- A commerce platform is a set of capabilities spread over several systems, not one system; the specialist layers (OMS, PIM, WMS, returns, search, channel

integration, iPaaS, payments) are distinct product categories with their own vendors and definitions, as the vendors' own November 2026 pages show.
- No vendor covers the whole map. Suites stop at the warehouse, the carrier, the PSP, the marketplace and the ERP; hosted platforms have retreated from logistics (Shopify's 2023 sale of Shopify Logistics) and earn largely from payments (Lightspeed's fiscal 2026 revenue mix); composable assemblies make integration the architecture (the MACH Alliance's own 2026 survey: 37% cite integration complexity).
- Business-to-business ordering is a first-class capability, not an add-on: Eurostat's 2024 figures put EDI-type sales at 11.07% of EU enterprise turnover against 8.39% for web sales.

Depends on context.

- How many boxes get a dedicated system depends on the stage and the shape: one channel and one warehouse can live inside a hosted platform; four channels and an own warehouse need an OMS, a PIM and an availability service. The trigger is the producer-and-consumer test, not revenue.
- Suite versus platform-plus-apps versus best-of-breed depends on the team: a suite needs an implementation partner, best-of-breed needs a platform team, platform-plus-apps needs app governance. A fourteen-person team should expect to own joins, not boxes.
- Whether a specialist group (several pre-integrated labels under one owner) is a good deal depends on how much of the operations layer you need from one owner and what leaving the group would cost.

The author's synthesis.

- Draw the map in capabilities before systems, and mark three things on it: empty boxes, boxes with more than one writer, and boxes whose fact is derived. Those marks predict the next incident better than any vendor roadmap.
- For a merchant that is not a vendor, the durable shape is a center you do not customize, operations products built for operations, and joins you own. Covering a box is cheap; owning the joins between boxes is the work, and it is the work nobody can buy for you.

Questions to Ask

In a design review, a vendor demo or your first week:

1. Which capability on the map is this system or product for, and which other boxes has it quietly grown into?

2. For each fact it holds, is it the home or a copy? If a copy, where is the home and how does the copy learn about changes?
3. Which boxes on our map are empty, and what fills them today (a spreadsheet, a person, a habit)?
4. Which boxes have more than one writer? List every system, app, script and person that can change the fact.
5. Which of our facts are derived (availability, the delivery promise, a price after promotions), and which computation owns each one?
6. Which flows need exactness in near real time (prices, orders, payments), and which only need a reconciliation behind them?
7. What unit is behind each number on the vendor's page (GMV, merchants, users, implementations, connectors), and as of what date?
8. If this vendor disappeared, which boxes would go dark, and which joins would have to be rebuilt?
9. Which box would we fill next if we grew one stage, and what sign would tell us it is time?

The map says what the parts are and what each should own. It does not yet say what happens when two parts hold the same fact and disagree, which is Kestrel & Pine's actual condition in 2027 and the subject of Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record.

Sources and Evidence

Company, product and investor sources

- As AI Agents Transform Commerce, Salesforce Unleashes Its Biggest Agentforce Commerce Release Yet, Salesforce, 2026 — a suite covering B2C, B2B, POS and order management in one platform.
- Adobe Commerce: Platform and services, Adobe, 2026 — three deployment options and Commerce Optimizer working beside an existing backend.
- Shopify pricing, Shopify, n.d., accessed 2026-11 — plan tiers, card rates and transaction fees of a hosted platform.
- Shopify Completes Sale of Shopify Logistics to Flexport, Shopify, 2023 — a hosted platform's retreat from the operations layer (older than three years; the direction has held).
- Shopify Q4 2025 financial results, Shopify, 2026 — 2025 revenue; GMV as reported by Digital Commerce 360, 2026.
- Lightspeed fourth quarter and full year 2026 results, Lightspeed, 2026 — transaction revenue versus subscription revenue.
- Get started with the Shopify Connector, Microsoft, 2026 — the ERP connector preinstalled for new Business Central sign-ups.

- What is an OMS?, Manhattan Associates, n.d., accessed 2026-11 — an OMS vendor's definition of the order-management capability.
- Akeneo, Akeneo, n.d., accessed 2026-11 — a PIM vendor's definition of the product-information capability.
- Fluent Commerce order management system, Fluent Commerce, n.d., accessed 2026-11 — "your ERP stays the system of record".
- ChannelEngine, ChannelEngine, n.d., accessed 2026-11 — "over 1,300 marketplaces" as an example of a connector-count claim.
- Katapull and Over Katapull, Katapull, n.d., accessed 2026-11; Katapull breidt Commerce Hub uit met overname van Picqer, Katapull, 2026; Vijf softwarebedrijven bundelen krachten in Katapull, Dutch IT Channel, 2025 — the specialist-group shape, from public pages only.
- EPAM joins the MACH Alliance as a founding member, EPAM, 2020 — the founding of the MACH Alliance and the acronym's expansion.
- MACH Alliance Enterprise Technology Report, MACH Alliance, 2026 — 37% of respondents cite integration complexity.

Standards, regulation and official documentation

- E-commerce statistics, Eurostat, 2026 (2024 reference year) — EDI-type sales 11.07% versus web sales 8.39% of EU enterprise turnover.

Research and practitioner sources

- None beyond the vendor and official sources above; the layers, the flows table and the trade-offs are the author's synthesis.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

4. Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record

IN THIS CHAPTER, YOU WILL: Decide, for every business fact a commerce platform holds, which system is its record, which are copies, and what happens when they disagree.

KEY POINTS:

- **Copies are unavoidable; unowned copies are the failure.** A price, a stock level, an order status and a customer each live in several systems. The question is not how to have one copy but which one wins when they disagree, and only one may. That is a decision per fact, not per system and not per company.
- **"Order", "product" and "stock" mean different things in different systems, and that is not a bug.** A bounded context is the boundary inside which a word has one meaning; the warehouse's order and the ledger's order are different objects that share a number. Design the translation at the boundary instead of forcing one model on everyone.

- **Ownership changes with scale, and the ERP is the usual casualty.** At the start the platform owns everything but the books. By the omnichannel stage physical stock belongs to the warehouse, orders to an order hub, product content to a PIM (a product information system), and the ERP (the back-office system that keeps the books) keeps finance. Kestrel & Pine's Waypoint program is that reassignment, written down.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Phantom Stock** — two channels sell the same last unit because each reads a different copy of the stock level and neither is the record.
- **Schrödinger's Order** — support finds three statuses in three systems because each system's "order" is a different object and nobody owns the view across them.
- **The Price Nobody Can Explain** — a price drifts between channels because three systems may write it and none is named as the source.

A shopper sees "in stock" and the warehouse has none. The finance director sees three order counts in three reports. A support agent sees *Delivered* in one system and *In picking* in another. Each of these is a disagreement between two copies of one fact, and in each case the people involved cannot say which copy to believe. **Nearly every commerce failure that is not a crash is a disagreement between copies, and nearly every one of those is a fact for which nobody has named the record.**

The usual reaction is to add a synchronization: copy the number more often, in both directions, from more systems. That makes the copies agree more of the time and makes the disagreements, when they come, impossible to resolve, because now there are four writers instead of two. The remedy is a decision, not more copying: for this fact, this system is the record, these systems hold copies, copies learn from the record this way, and when they disagree the record wins.

This chapter makes that decision teachable. It defines records, copies and bounded contexts in plain words, builds the entity–owner table that The Map of a Commerce Platform: Capabilities and Layers prepared for, compares the four ways merchants assign ownership, shows how the assignment changes with scale, and works Kestrel & Pine's stock problem from six writers to one. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation later shows how copies are kept honest; this chapter decides who they answer to.

Where we are in the order's journey. Every number Eva Janssen sees on 13 April 2027 is a copy. The "in stock" next to the Ridgewalker jacket at 21:14 is a copy of the warehouse's count. The €199 is a copy of a price list. The *Shipped* she sees on 14 April is a copy of what the WMS (the warehouse management system that runs picking and packing at Duiven) confirmed. This chapter is about where each of those copies comes from, and which system they answer to when they are wrong.

Setting the Scene at Kestrel & Pine: "Where Does Stock Live?"

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), had been selling the Ridgewalker jacket for years without anyone being able to answer that question in one sentence. In early March 2027, the week before Mara Lindqvist approved the Waypoint program, Noor Visser put it to the people who ran each system. She got six honest answers.

Who	Where stock lives, in their words	What the number actually is
Daniel Okafor, Operations	"In the WMS. The pickers scan it."	Units on hand per bin at Duiven; updated at every scan
Sanne de Wit, Finance	"In the ERP. That is the valued stock."	A quantity per item in the ERP (the enterprise resource planning system: the books, purchasing, stock valuation), refreshed from the WMS every 15 minutes, plus manual adjustments, plus store stock loaded nightly
Milan Petrov, Ecommerce	"In the platform. That is what the storefront sells from."	A sellable quantity per variant, refreshed hourly from the ERP, also written by three apps and, occasionally, by hand
The store managers	"In the POS."	Units per store in the POS (the point-of-sale system, the store's till software), copied to the ERP overnight
Milan again, for marketplaces	"The connector pushes it to bol, Amazon.de and Zalando every hour."	A copy of the platform's copy

Who	Where stock lives, in their words	What the number actually is
Daniel again, for Germany	"Some of it is in the marketplace's warehouse."	Stock sent to the marketplace's fulfillment program in advance, in a location outside the company's control

Six places, six numbers. Ruben Alves pulled the logs for the Ridgewalker women's size M in moss for 20:00 on Friday 27 November 2026, twenty minutes after the warehouse sold its last unit, and found six different quantities, from zero in the WMS to a healthy double-digit number on the storefront (an illustrative reconstruction). Every system was working. The chain that copied the number took up to two hours, and in those two hours every copy was a lie that the channels sold from. That was the Black Friday Oversell: 1,340 units oversold, 1,120 orders canceled or delayed, about €96,000.

Noor's question was not "how do we sync faster". It was "**which of the six is right?**" Daniel answered for everyone: "The shelf." The WMS counts the shelf. Waypoint decision W1 followed: the WMS is the system of record for physical stock at Duiven, the POS for store stock, the marketplace program's stock is a tracked external location, and an availability service computes what each channel may sell from those three. Everything else becomes a copy.

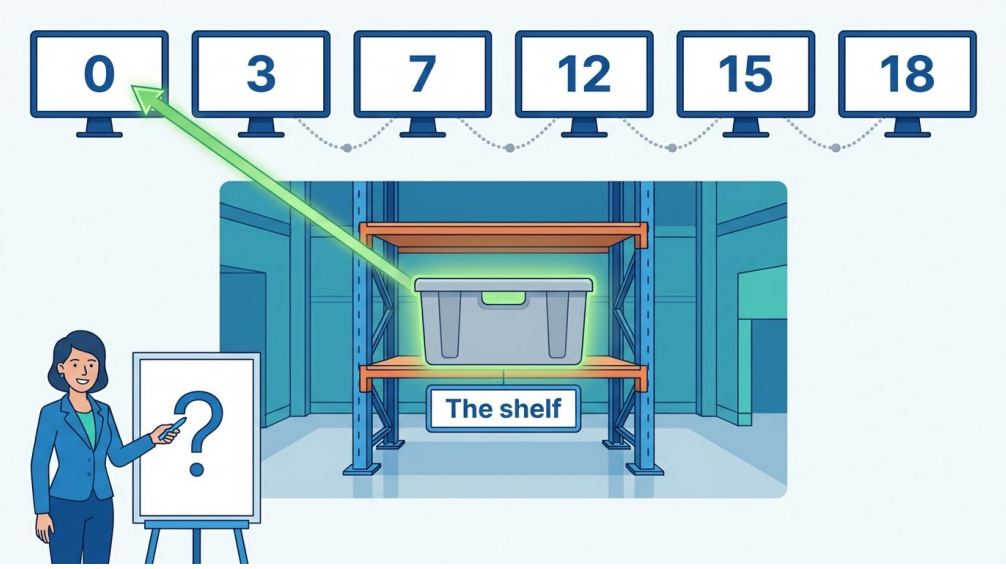


Figure 1: *Six systems, six honest numbers for one jacket at 20:00 on Black Friday; Noor's question was not how to sync them faster but which one is right, and the answer was the*

shelf.

Facts, Records and Copies

Three words carry this chapter.

A **business fact** is something the business needs to be true about the world: this variant has this barcode, this customer pays this price, there are 14 units of this size in Duiven, order #1072455 has shipped. The business acts on facts: it promises delivery, takes money, prints invoices.

A **system of record** for a fact is the one system whose value is treated as true when copies disagree, and the only system in which the fact is changed directly. Your bank's ledger is the record for your balance; the figure in your budgeting app is a copy. In commerce the shelf count in the WMS is the record for physical stock; the number on the product page is a copy. The finance book RECONCILED works the same idea for money in Systems of Record: Ending the Doppelgänger Record.

A **copy** is the same fact held in any other system. Copies are not a mistake. The storefront must hold a stock number to render a page in milliseconds; the data warehouse must hold every order to count them; the marketplace must hold the price to show it. The rule is only that a copy is known to be a copy: it is refreshed from the record, it is never edited to "fix" a disagreement, and when it differs from the record it is the copy that is wrong.

A **writer** is any system, app, script, job or person that can change a fact in a given system. The practical test of ownership is the writer list. If the record has one writer and the copies have none except the refresh from the record, ownership is real. If a copy has writers of its own, it is not a copy; it is a second record, and the two will diverge.

One phrase deserves a warning. "Single source of truth" is right only if read as *one record per fact*; it is wrong, and expensive, if read as *one system for every fact.* No system is the natural home of both the shelf count and the ledger posting. ("Master data", the slowly changing reference facts such as products, customers, suppliers and locations, follows the same rules; One Customer, One Product: Identity and Master Data Across Systems owns it.)

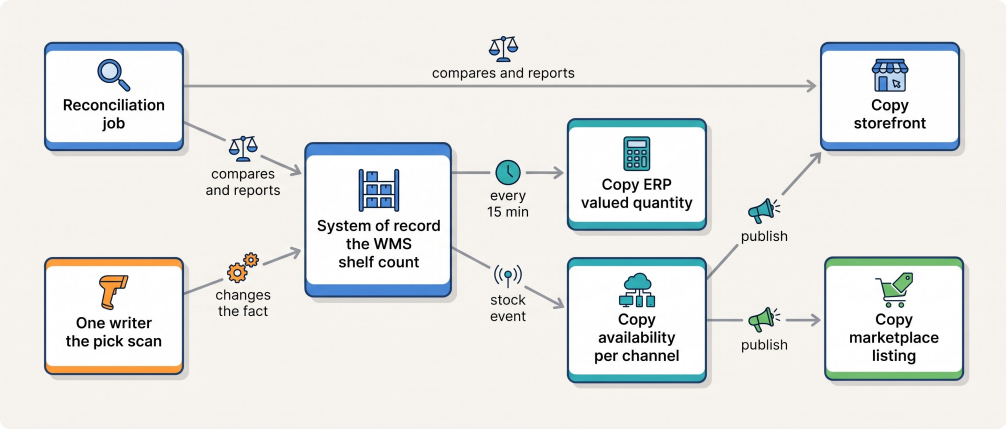


Figure 2: *One fact, one writer, one record and several copies: the shelf count changes only through the pick scan, every other system holds a copy refreshed from the record, and a reconciliation job compares record and copies so that drift is found instead of discovered by customers.*

Bounded Contexts: Why the Warehouse's Order Is Not the Ledger's Order

Ownership is harder than it looks because the same word names different things in different systems. **Bounded context** is the term Eric Evans gave this in *Domain-Driven Design* (2003): a boundary inside which a model, and the words in it, have one agreed meaning. Martin Fowler's 2014 note calls it "a central pattern in Domain-Driven Design", repeats Evans' claim that "total unification of the domain model for a large system will not be feasible or cost-effective", and adds, from experience, "Time and time again I see this confusion recur with polysemes like 'Customer' and 'Product' " (a polyseme is a word with several related meanings).

In plain words: the warehouse, the storefront and the ledger are three bounded contexts. Each has its own "order", and the three are not the same object.

Context	What "order" means there	What it contains	Its states
Storefront and checkout	What the customer bought, at what price, with what promise	Lines with consumer prices including VAT, promotions, a payment, a delivery promise, an address	Placed, paid, (customer-visible) shipped, delivered
Order management	The order's life across channels	One record per order from any channel, allocation to a location, state transitions, edits, cancellations	Placed, paid, allocated, picking, shipped, delivered, canceled, returned, refunded
Warehouse	Work to do	A fulfillment order: pick lines, bins, a packing station, one or more parcels; a split order is two of these	Released, in picking, packed, shipped
ERP	Something to invoice and post	A sales order with net amounts, VAT, a debtor, a receivable	Open, invoiced, paid

The Three-Status Order of December 2026 was order #1052877 showing *Delivered* in the platform, *In picking* in the WMS and *Not invoiced* in the ERP for six days, and 11 support tickets from one customer. Read against the table, the three statuses were not three bugs; they were three contexts, each honest about its own object, with nothing translating between them and nobody owning the view across them. The order had two lines, a fleece that shipped on 7 December and socks that waited a week for replenishment. The warehouse's "in picking" was true of an order whose second line was still waiting to be picked; the platform's "delivered" was true of the fleece's parcel, the only one the carrier had scanned; the ERP's "not invoiced" was true because it invoices on the WMS's "order complete" message, which could not come until the socks shipped. The failure was the absence of an owner for the cross-context fact "where is this order", which is what an order management system (OMS) is for, and why Waypoint decision W2 makes a bought

OMS the order hub with one state model and one order number family.

The same happens with "product" (content for the storefront; attributes and translations in a PIM, a product information management system; an item with cost and supplier in the ERP; a SKU, a stock-keeping unit or sellable variant, with dimensions, weight and bin in the WMS) and with "customer" (an account with an e-mail in the platform; a loyalty card in the POS; a debtor with payment terms in the ERP; a contact in the returns portal). Eva Janssen exists four times, with no shared identifier, because four contexts each created the customer they needed. That is the Familiar Stranger, worked in One Customer, One Product: Identity and Master Data Across Systems.

Two design rules follow. First, **translate at the boundary** rather than forcing one model on every system: the connector between the OMS and the WMS turns an order into a fulfillment order, and the connector into the ERP turns a shipment into something to invoice; the translation is named work with an owner, not an accident of field mapping. Second, **share identifiers, not models**: the order number, the SKU and the GTIN (the Global Trade Item Number behind the barcode) let the contexts refer to the same thing while describing it differently. Promises Between Systems: API Contracts, Webhooks and Versioning treats the translations as contracts.

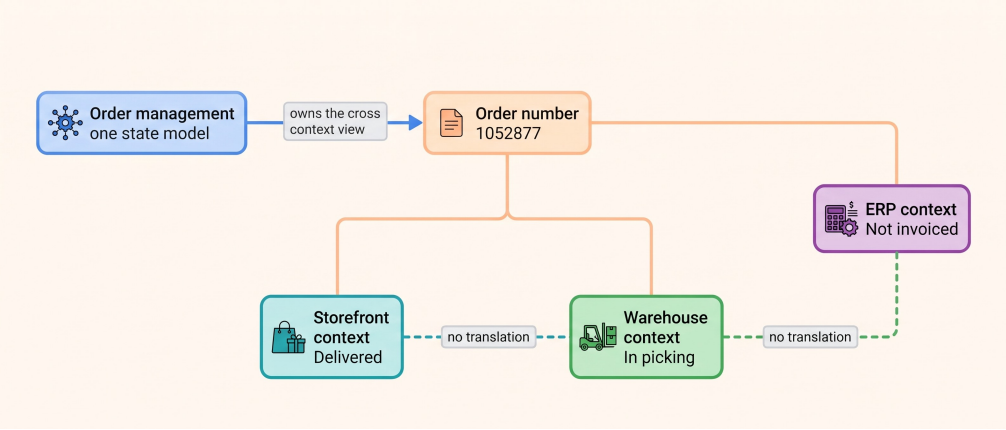


Figure 3: *The Three-Status Order: three bounded contexts each hold an honest status of their own object, share only an order number, and have no owner of the view across them; the order management system in Waypoint's design is that owner.*

The Entity–Owner Table

The deliverable of this chapter is a table with one row per business fact. For each fact it names the system of record, who may write it there, where copies are allowed, how the copies learn about changes, and how disagreement is detected. The version below is the author's recommendation for a merchant at the omnichannel stage with its own

warehouse, stores, wholesale and marketplaces; the stage table later in the chapter says how it differs earlier and later. It assumes a PIM and an OMS exist; without them the platform holds those rows.

Fact	System of record	Who may write	Copies live in	How copies learn	Disagreement found by
Product identity (SKU, GTIN, unit, supplier, cost)	ERP item master	Buying team	PIM, platform, WMS, connector, EDI partners (trading partners that exchange structured documents by electronic data interchange)	Scheduled publication on creation and change	Item count and GTIN check, weekly
Product content (names, attributes, media, translations)	PIM	Product and content teams; translators	Platform, marketplaces, EDI product catalogs	Publication on approval; nightly full sync as a safety net	Completeness and channel comparison, nightly
Consumer price per market	Pricing capability (the platform's price lists at this stage)	Pricing owner, with effective dates	Storefronts, POS, marketplace listings, connector	Event on change; exact at promotion start and end	Price comparison per channel, hourly

Fact	System of record	Who may write	Copies live in	How copies learn	Disagreement found by
Wholesale and contract price	ERP price lists	Wholesale team	B2B (business-to-business) portal, EDI price catalogs	Price events with effective dates	Per-account comparison, weekly
Physical stock per location	WMS (warehouse), POS (stores), tracked external location (marketplace program)	Scans, counts and receipts in that system only	Availability service, ERP, data warehouse	Stock events per SKU and location	Cycle counts in the record; WMS-to-platform reconciliation, weekly
Availability per channel (derived)	Availability service	The computation only; no manual edits	Storefronts, marketplaces, B2B portal	Publication on every stock or reservation event	Oversell and cancellation rate, daily
Order	OMS	Order creation from channels; state changes from fulfillment, payment and returns events	Platform order history, WMS, ERP, service tool, warehouse	Order events	Order count per channel per day, OMS versus each source, daily
Payment	PSP	The PSP, on instructions from checkout and the OMS	The order (a reference and a status)	Payment webhooks	Settlement-to-order reconciliation per payout

Fact	System of record	Who may write	Copies live in	How copies learn	Disagreement found by
Shipment and tracking	WMS and shipping service (shipment); carrier (events)	Pack confirmation; carrier scans	OMS, notifications, service tool	Shipment and tracking events	Shipped-not-delivered aging, daily
Return	Returns portal (request); WMS (receipt and grade); OMS (resolution)	The customer, the inspector, the OMS	ERP (credit note), PSP (refund)	Return events	Refunds versus credit notes versus receipts, daily
Invoice, credit note, ledger	ERP	Posting from OMS and PSP batches; finance	Reports	Daily batch	Period close
Customer (consumer)	Platform account	The customer; service	POS, newsletter tool, returns portal, warehouse	Sync on order; periodic merge	Duplicate rate, monthly
Customer (business)	ERP debtor	Wholesale and finance teams	B2B portal, EDI partner cards	Scheduled sync	Terms and credit-limit comparison, weekly

Fact	System of record	Who may write	Copies live in	How copies learn	Disagreement found by
Marketing consent	With the customer record, with evidence of when and where	The customer	Newsletter tool	Sync on change	Consent audit, quarterly

Several rows carry lessons that newcomers miss.

- **Stock has three records, not one**, because the warehouse, the stores and the marketplace's program each count a different shelf. Ownership is per fact *and per location.*
- **Availability has no human writer**. It is computed from the stock records and the open orders. A hand edit to a channel's available quantity is a second writer, and it is how the App That Wrote Stock produced phantom reservations on 600 variants.
- **Payment's record is outside the company**. The PSP (payment service provider, the company that takes the shopper's money and pays it out to the merchant) holds the transaction; the order holds a reference. A refund issued by hand in the PSP console, outside the returns flow, is a write to the record that the OMS does not know about, which is the Double Refund: 212 returns refunded twice, €19,400.
- **The business customer and the consumer have different records**. Microsoft's October 2026 documentation for its Business Central Shopify Connector says it plainly: decide "which system is the system of record for customer data", notes that most businesses manage customer records, payment terms and credit limits in the ERP, and warns that "Mixing both directions without a clear ownership rule can lead to conflicting records."

The "disagreement found by" column is the part most tables leave out. A record with copies and no reconciliation (a job that compares two systems' values and reports the differences) is a record on trust, and trust is what the Black Friday chain ran on. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation designs those jobs; here the rule is only that every copy row has one.

How fresh must a copy be?

Ownership decides who wins; it does not decide how quickly the copies must follow. That depends on what a stale copy costs, and it differs sharply by fact.

Fact	A copy may be stale for	Agreement required	Why
Availability shown to a channel	Seconds at peak; minutes normally	Eventual, within a buffer	Every minute of staleness is an oversell window; Kestrel & Pine's two hours produced 1,340 oversold units
Consumer price	Minutes; exact at a promotion's start and end	Exact across channels	A stale price is a broken promise, and on marketplaces a parity breach; the Price Drift ran nine days
Order existence	Seconds to minutes	Exact: never lost, never doubled	A missed order is a missed cut-off; a doubled one ships twice
Order status	Minutes	Eventual	Stale status creates "where is my order" contacts, not losses
Product content	Hours	Eventual	A late description is a missed sale, not a wrong one
Ledger postings	A day	Exact per period	Finance works in periods

The table is the author's judgement from industry practice and Kestrel & Pine's incidents. Its point is that the record for availability needs an event-driven copy (an **event** is a message sent the moment something changes, as opposed to a periodic copy) while the record for ledger postings can be fed by a nightly batch, and that choosing one rhythm

for all facts is how an ERP's hourly cycle became a storefront's availability latency.

Four Ways to Assign Ownership

Merchants assign ownership in one of four ways. Three are decisions; the fourth is what happens without one.

The ERP as hub. The ERP is the record for products, stock, orders and customers, and the web shop is a front end that reads from it and posts orders into it. Microsoft's connector documentation of October 2026 encodes this: "Business Central acts as the system of record for inventory", pushed to Shopify one way on a recommended cycle of every 15 to 30 minutes; customers are managed in Business Central and exported; orders flow the other way, from the shop into the ERP. This fits one channel, one warehouse and a wholesale-heavy business, because the ERP already holds the terms, the prices and the valued stock. It stops fitting when stores, marketplaces and a WMS each need stock in minutes and the ERP's batch rhythm cannot serve them.

The platform as hub. The commerce platform is the record for orders, customers and stock; the ERP subscribes to the result for finance. This is the natural shape of a direct-to-consumer (D2C) brand on a hosted platform with one or two channels: the platform's order is the order, its customer is the customer, its stock number is the stock. It stops fitting when orders are created elsewhere (a POS from another vendor, EDI from a retail chain, a marketplace connector) and when stock is counted elsewhere (an own warehouse on a WMS), because the platform's model cannot hold those orders well and its stock number becomes a copy with its own writers.

Ownership per fact, with an order hub. Each fact gets the record that counts it: the WMS for physical stock, an OMS for orders and for availability, a PIM for content, the ERP for identity, cost and finance, the platform for the storefront and checkout. This is the entity–owner table above. Vendors in the specialist layers state the boundary in their own words: Fluent Commerce's site says "your ERP stays the system of record" while its OMS decides where orders go (n.d., accessed November 2026), and OneStock, another OMS vendor, argues that the ERP "is the master for product data (if no dedicated PIM system exists)" and that the OMS "is the master for enterprise inventory availability" (n.d., accessed November 2026; a vendor arguing for its own category). It fits the omnichannel and group stages, where several channels create orders and several locations hold stock. It costs an extra system and an extra integration surface, and it is wrong at the first two stages, where there is nothing to orchestrate.

No decision. Systems are joined pairwise as needs arise, often with two-way syncs so that "both are always up to date". Every system becomes a writer of every fact it holds; disagreements are resolved by whoever notices, in whichever system they are looking at. This is Kestrel & Pine in January 2027: 34 integrations, six writers of stock, four sources of orders, four copies of product data, three definitions of "order" in the data

warehouse. It is the default because it is never chosen; each connector was a reasonable answer to one request.

	ERP as hub	**Platform as hub**	**Per fact, with an order hub**	**No decision**
Record for stock	ERP	Platform	WMS, POS, external location; availability derived	Everyone
Record for orders	ERP	Platform	OMS	Each channel's system
Record for product content	ERP	Platform	PIM	Spreadsheets
Record for finance	ERP	ERP	ERP	ERP
Fits	One channel, one warehouse, wholesale-heavy, small team	D2C-first on a hosted platform, one or two channels	Several channels, several stock locations, own warehouse	Nobody, but it is where most merchants are
Breaks when	Batch cadence meets real-time availability; several channels write one monolith	Orders and stock originate outside the platform	Too early: cost without orchestra-tion need	The first busy week
Copies tolerated	Few; the shop is a view	Few; the ERP is a ledger	Many, each named with a refresh and a rec-onciliation	Unknown

The table is the author's judgement, consistent with the vendor statements above. My rule: **choose the hub by where orders and stock originate, not by which system is largest**. If every order starts in the platform and every unit sits in one warehouse the

platform or the ERP can be the hub. The day a second order source or a second stock location appears, ownership per fact becomes necessary, and the longer the decision waits the more writers each fact collects.

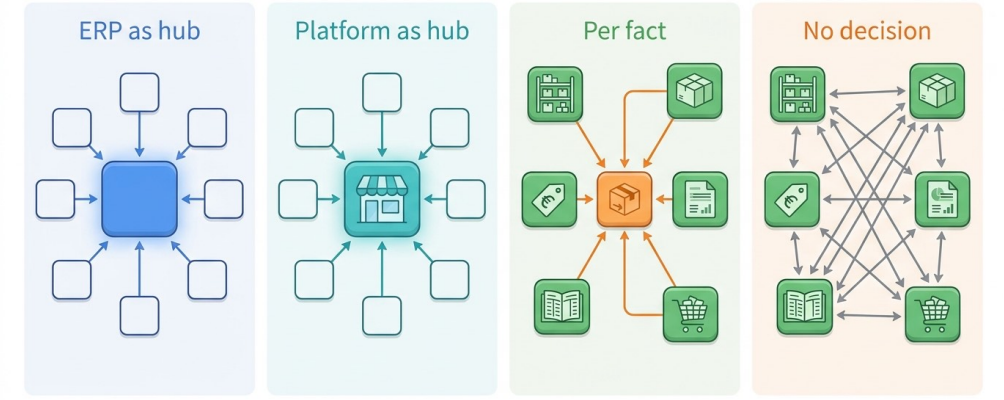


Figure 4: *Three of the four ways to assign ownership are decisions; the fourth is the tangle that forms when each connector answers one request, and the second order source or stock location is the moment to choose ownership per fact.*

Ownership in a Suite and Across Vendors

In a suite, the vendor has made most of these decisions for you. Salesforce's July 2026 release describes one platform across B2C, B2B, point of sale and order management with agents "natively connected to catalog, inventory, and orders": inside that boundary the suite's order is the order and its inventory is the inventory. A vendor-built connector makes the decisions configurable: the Business Central connector asks, per entity, which direction is the record (products: the merchant chooses; inventory: the ERP; orders: the shop). The entity–owner table becomes settings.

Two things remain yours even in a suite. First, the systems outside it: the PSP, the carriers, the marketplaces, a POS or WMS from another vendor, the EDI partners. Their facts are records the suite copies, and the suite's copies need the same refresh and reconciliation as anyone's. Second, the decision itself: the suite decides what it owns, not whether that is right for your business. Shopify's enterprise guidance of August 2026 states the discipline that holds across vendors in two sentences: "Document every field that moves between systems, and name the system of record for each" and "Decide in advance what happens when the two systems disagree."

When Ownership Is Not Decided: The Five Truth Patterns

Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong names five patterns in a family it calls Truth. Each is a fact without an owner.

Pattern	The fact with no record	Kestrel & Pine's incident	The owner Waypoint named
The Phantom Stock	Physical stock, and availability per channel	The Black Friday Oversell (1,340 units, about €96,000); the Store Stock Mirage (37 items promised that Utrecht had sold)	WMS and POS for stock; the availability service for what each channel may sell (W1)
Schrödinger's Order	The order's state across contexts	The Three-Status Order (#1052877, six days, 11 tickets)	The OMS, one state model, one order number family (W2)
The Familiar Stranger	Customer identity	Eva Janssen as four unlinked records	Not yet decided; One Customer, One Product: Identity and Master Data Across Systems
The Boomerang	The refund and its link to the return	The Double Refund (212 returns, €19,400 twice)	The OMS as the owner of the return's resolution; refunds only through the flow (W2, W5)

Pattern	The fact with no record	Kestrel & Pine's incident	The owner Waypoint named
The Price Nobody Can Explain	The consumer price per market	The Price Drift (€179 for nine days, 410 orders, about €8,200)	A single price source per market, published by event (W5); What Does It Cost? Price Lists, Contract Prices, Currencies and Tax

The pattern that is not in the family, The Secret Agent, is the ownership failure seen from the writer's side: a script, app or person that changes a fact without being on the writer list. The App That Wrote Stock was a Secret Agent and a Phantom Stock at once.

Across Scale Stages: When It Fits and When It Limits

The entity–owner table is not written once. Each stage brings a new system that claims facts the previous owner held, and the moment of handover is when two writers exist. The stages are the book's four (What Changes at Each Scale: From First Store to Group).

Stage	Typical ownership	What becomes necessary	Warning signs of outgrowing it
1 — Starting	The platform is the record for product, price, stock, order and customer; an accounting package for the books; the portal of a 3PL (a third-party warehouse that ships on the merchant's behalf) holds a copy of stock	Nothing beyond one e-mail per customer and a monthly payout reconciliation	A 3PL or an accounting system starts writing stock or customers back
2 — Growing	An ERP arrives and claims items, business customers and valued stock; the platform keeps D2C customers and orders; a POS brings store stock and a second customer key	Decide the product master and the stock master now; one direction per entity; a daily order and stock reconciliation	Two systems write stock; a spreadsheet becomes the product master; the first oversell
3 — Omnichannel	The WMS owns physical stock, an OMS owns orders and availability, a PIM owns content, the ERP keeps identity, cost and finance, the platform keeps checkout	The full table with a refresh and a reconciliation per row; matching rules for customers; the ERP's rhythm decoupled from availability	Support checks three systems for one order; a stock sync chain measured in hours; refunds issued in two places

Stage	Typical ownership	What becomes necessary	Warning signs of outgrowing it
4 — Group	One table per brand or country, with shared records where sharing pays (PIM, customer master, OMS) and separate ones where it does not; survivorship rules per attribute	A master data function with a merge log; governance that decides what is shared before a stack is forced on a brand	One Size Fits None: brands forced onto one record before the benefit is measured, or kept apart when they should share

Kestrel & Pine moved from stage 2 to stage 3 in 2023 and kept stage 2's ownership for four years: the ERP as stock master for a company with its own WMS, the platform as order record for a company with four order sources. The judgement: **reassign ownership at the stage transition, not after the first incident**. The sign that it is time is always the same, a second system that originates orders or counts stock.

In Practice

Public examples. The vendor statements quoted above (Microsoft's connector documentation of 2026, Shopify's ERP guidance of August 2026, Fluent Commerce and OneStock) are the clearest public cases of records being named per entity. At a much larger scale, a 2020 account by a former Walmart engineering director on Confluent's blog describes Walmart's inventory backend partitioning its event stream so that "each item store always goes to a specific partition", which ensured "that only one consumer deals with that item store combination": one writer per entity, enforced by the plumbing. The account (May 2020) is six years old, is the author's own view with Walmart not endorsing it, and describes a group-stage system; the principle is the one in Figure 2.

Kestrel & Pine's stock, before and after W1. The table reconstructs the writer list for the Ridgewalker jacket's warehouse stock in January 2027 and under the Waypoint design from Q4 2027.

	January 2027	After W1 (Q4 2027)
Record for Duiven stock	None named; the ERP by habit	The WMS
Writers of Duiven stock	WMS feed; manual ERP adjustments; manual platform edits; three apps	Scans, counts and receipts in the WMS
Record for store stock	The POS, copied to the ERP nightly	The POS, publishing stock events
Record for stock in the marketplace's program	None; invisible until the marketplace reported it	A tracked external location fed by the marketplace's reports
Availability per channel	Each channel's copy of the ERP's copy of the WMS	Computed by the availability service from the three stock records and open orders, with a buffer per channel
How copies learn	WMS to ERP every 15 minutes, ERP to platform hourly, platform to connector hourly	Stock events, seconds; the sync chain retired
Disagreement found by	Customers, on 27 November 2026	Weekly WMS-to-platform reconciliation; cycle counts in the WMS

Replay the Black Friday evening against the right-hand column. At 19:40 the last Ridgewalker women's M moss is scanned at a packing station; the WMS emits a stock event; the availability service recomputes the channel quantities to zero within seconds and publishes them; the storefronts and the connector stop selling the variant before 19:41. The 1,340 oversold units become a handful, from orders already in checkout at 19:40, which is what the per-channel buffer exists to absorb. The first weekly reconciliation in May 2027 found 2.3% of active variants with a different quantity between the WMS and the platform (illustrative), most left behind by the three apps; the apps' write access was removed, and the number is the reconciliation's baseline.

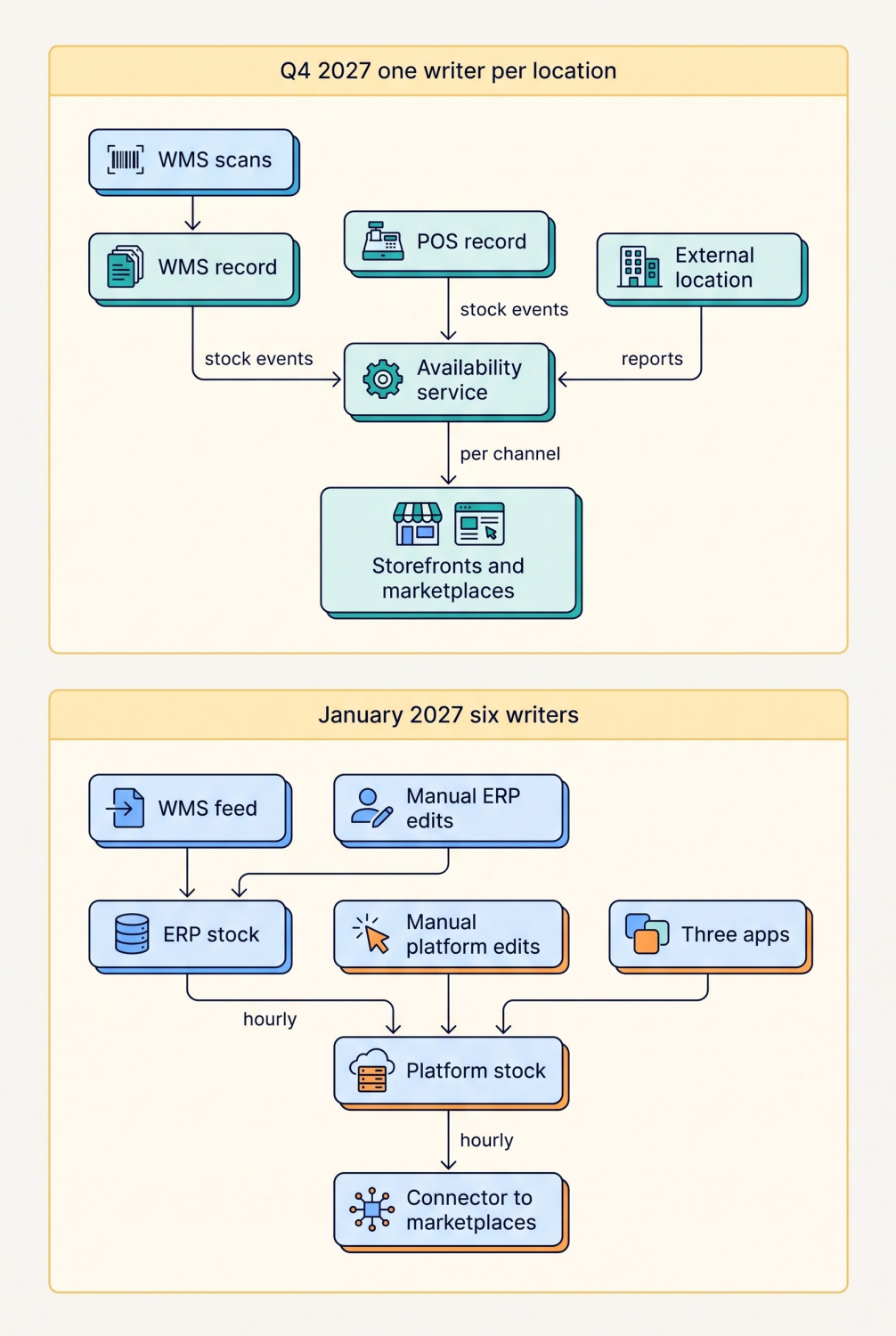

Q4 2027 one writer per location
WMS scans
WMS record
POS record
External location
stock events
stock events
Availability service
reports
per channel
Storefronts and marketplaces
January 2027 six writers
WMS feed
Manual ERP edits
ERP stock
Manual platform edits
Three apps
hourly
Platform stock
hourly
Connector to marketplaces

Figure 5: *Kestrel & Pine's warehouse stock before and after Waypoint decision W1: six writers feeding a two-hour chain of copies become one writer per location, an availability service that computes what each channel may sell, and copies that learn by event.*

When It Goes Wrong

Ownership decided is not ownership kept. Five things go wrong, each with a detection and a recovery.

A second writer appears. An app, a script, an integration "fix" or a well-meaning colleague starts changing a fact in a copy or beside the record. The App That Wrote Stock is the case. Detection is a periodic *write audit*: for each row of the entity–owner table, list every credential, app scope, job and role that can change the fact, and compare it with the writer column. Recovery is to remove the write, then re-publish the fact from the record to every copy.

A copy goes stale. The refresh breaks or slows; the copy keeps serving its last value. Kestrel & Pine's two-hour chain was stale by design, and the Deprecated Endpoint of February 2027 made the connector's copy stale for two days when an API version was retired. Detection is a *lag metric per copy* (time since last successful refresh, alarmed against the freshness table) and the reconciliation's difference count. Recovery is a full re-publication from the record once the refresh is repaired, not a hand correction of the copy.

A copy is edited by hand. Support changes an order's status in the platform to calm a customer; an operator types a stock quantity into the storefront; a refund is issued in the PSP console. Each is a write to a copy, or to a record outside the flow that owns it; the Double Refund was the last of these. Detection: change logs on copies, reviewed for human edits; refunds reconciled against credit notes daily. Recovery: route the action through the owner (the OMS issues the refund; the WMS corrects the count) and close the side door.

A copy is mistaken for the record. The data warehouse's order count is quoted as the company's; the connector's stock number is treated as the warehouse's. Detection: ask, for any disputed number, which row of the table it comes from. Recovery is the table itself, published where people can see it, and Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics's one definition of an order.

The record is wrong. The shelf count in the WMS differs from the shelf. No ownership rule fixes this; the record needs its own correction process (cycle counts, receipts, damage write-offs), performed in the record by its writers. Correcting the copy instead produces a right copy and a wrong record, and the next refresh undoes the correction.

What to Take Away

Established.

- Several systems holding one fact is normal, and the field's remedy is to name a system of record per fact and treat the rest as copies; vendors that document their integrations say so in those words (Microsoft 2026, Shopify 2026, Fluent Commerce and OneStock 2026).
- Different systems legitimately model "order", "product" and "customer" differently; Evans' bounded context (2003) and Fowler's 2014 restatement hold that unifying one model across a large system is not feasible or cost-effective, so the work is translation at the boundary and shared identifiers.
- One writer per entity is enforceable mechanically: a single refresh direction per entity in a connector, or a single consumer per item-and-location partition in an event stream (Walmart's 2020 account).

Depends on context.

- Which system is the hub depends on where orders and stock originate: the ERP or the platform can own everything at one channel and one warehouse; ownership per fact with an order hub becomes necessary at the second order source or the second stock location.
- Whether the business customer's record is the ERP or the platform depends on who manages terms and credit limits; the consumer customer's record is usually the platform until a store or a loyalty program adds a second key.
- How fresh a copy must be depends on the fact: seconds for availability at peak, exactness at promotion boundaries for price, a day for ledger postings; one rhythm for all facts is wrong in both directions.

The author's synthesis.

- Write the entity–owner table before buying any hub, and include the writer list and the reconciliation per row; a bought OMS fed by four unreconciled order sources is a fifth copy, and a written table with no reconciliation is a record on trust.
- Reassign ownership at the stage transition rather than after the incident: the day a second system originates orders or counts stock, the previous owner's claim has already expired, whatever the configuration still says.

Questions to Ask

1. For this fact, which system is the record, and is that written down anywhere a new colleague could find it?

2. Who, and what, can write this fact in that system and in every copy? Is there more than one entry on the list?
3. How does each copy learn about a change, how old may it be, and what alarms when it is older?
4. What job compares the record with each copy, how often, and who receives the differences?
5. When a customer sees one value and the record holds another, what does support do, and in which system?
6. Which of our facts are derived (availability, the delivery promise), and can anyone edit the derived number by hand?
7. What does "order" mean in each of our systems, and which system owns the view across them?
8. Which system claims a fact today because of the stage we were in when we bought it, not the stage we are in now?

With the parts mapped and their owners named, the next question is what the market sells to fill the boxes, who sells it, how it is priced, and how the landscape changed up to 2026, which is Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.

Sources and Evidence

Company, product and investor sources

- Synchronize inventory with Shopify, Microsoft, 2026 — "Business Central acts as the system of record for inventory"; one-way push every 15–30 minutes.
- Synchronize items and products with Shopify, Microsoft, 2026 — choosing the record per entity ("To Shopify" or "From Shopify").
- Synchronize customers and companies with Shopify, Microsoft, 2026 — "decide which system is the system of record for customer data"; mixed directions cause conflicting records.
- Synchronize orders with Shopify, Microsoft, 2026 — orders originate in the shop and are imported into the ERP.
- ERP integration: what it is and how it works, Shopify, 2026 — "name the system of record for each" field; "Decide in advance what happens when the two systems disagree" (vendor guidance).
- Fluent Commerce order management system, Fluent Commerce, n.d., accessed 2026-11 — "your ERP stays the system of record" (vendor).
- OMS vs ERP: the architecture shift reshaping commerce, OneStock, n.d., accessed 2026-11 — the OMS as master for availability, the ERP for product data absent a PIM (vendor).

- As AI Agents Transform Commerce, Salesforce Unleashes Its Biggest Agentforce Commerce Release Yet, Salesforce, 2026 — a suite that owns catalog, inventory and orders inside one boundary.
- Real-time inventory management at Walmart using Kafka, Suman Pattnaik on Confluent's blog, 2020 — one consumer per item-store partition (older than three years; author's own views).

Standards, regulation and official documentation

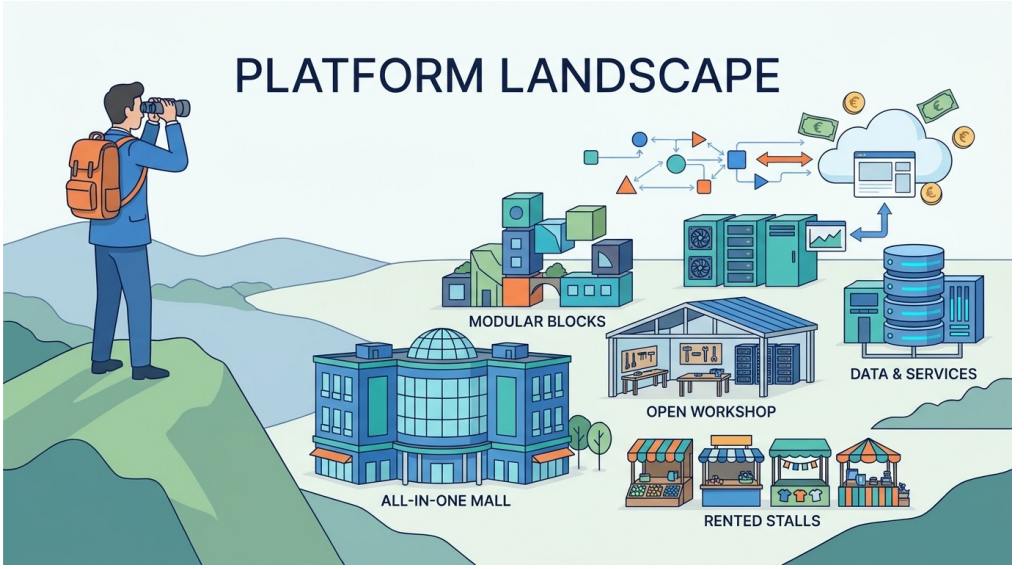
- None; this chapter's sources are vendor documentation and practitioner literature.

Research and practitioner sources

- Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003 — the bounded context (older than three years; the concept is unchanged).
- BoundedContext, Martin Fowler, 2014 — bounded context as DDD's central strategic pattern; unification of a large model is not feasible or cost-effective; "Customer" and "Product" as polysemes (older than three years; the concept is unchanged).
- Systems of Record: Ending the Doppelgänger Record, RECONCILED, 2026 — the finance-side treatment of records and copies.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

5. Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026



IN THIS CHAPTER, YOU WILL: Learn the seven kinds of commerce platform, how each is priced, what changed between 2023 and 2026, and how to read a vendor's claims before comparing two of them.

KEY POINTS:

- **The category decides where the work lands.** A hosted platform keeps the core in the vendor's hands and pushes variation to apps; a suite keeps integration inside one vendor's data model; open source moves hosting and upgrades to you; composable moves the integration burden to you. Pick the category by who you have to carry that work.
- **Commerce platforms are priced on your sales, not on their software.** Shopify's FY2025 filing puts 76% of its revenue in fees that move with merchant sales; Lightspeed's FY2026 transaction revenue is more than twice its subscriptions; Shopware, Saleor and BigCommerce tie

their plans to gross merchandise value. The subscription is the entry ticket.
- **Between 2023 and 2026 the market converged on "keep your back-end, add a layer".** commercetools, Adobe and Salesforce now sell modules that sit beside an existing stack, Shopware absorbed integration and payments into the platform, and Commerce.com reported that replatforming demand is soft. Read every 2026 pitch, "composable" and "agentic" included, as a layer to add, not a reason to move.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Giant's Boots** — the landscape is full of architectures built for companies a hundred times larger; this chapter sorts them by the merchant they fit.
- **The Forked Platform** — the categories differ in who applies upgrades, and the ones that let you change the core are the ones that let you lose the upgrade path.
- **The Lynchpin** — a vendor's revenue mix, ownership and strategic reviews are public signals of how long a capability will be funded; this chapter shows where to read them.

Every merchant runs on a commerce platform, and most people who join a merchant are told its name on the first day and nothing else. The name is not the useful fact. The useful fact is **what kind of thing it is**: whether the vendor or the merchant runs the servers, whether the storefront and the engine are one product or two, whether the price is a flat fee or a share of sales, and whether the vendor will still fund the product in three years. Two platforms with similar feature lists can have opposite cost curves and opposite exit paths.

This matters in money. A platform that charges a percentage of sales is cheap at €1 million of revenue and expensive at €50 million. A platform that lets an agency change its core is flexible in year one and unupgradable in year four. None of this appears in a feature comparison, and all of it decides what the platform costs and how long it lasts.

This chapter is a map. It sorts the market into **seven categories**, explains how each is priced, lists what changed between 2023 and 2026 with dates and sources, and ends with a reading guide for vendor claims. It recommends no product; the decision method belongs to Build, Buy or Assemble: Choosing and Changing Platforms. The map builds on The Map of a Commerce Platform: Capabilities and Layers and the ownership rules of Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record.

Where we are in the order's journey. This chapter sits off the order's path. Eva's jacket, the recurring order of Follow the Order: What Happens Between "Buy" and "Delivered", starts life on Kestrel & Pine's platform: the storefront shows it, the cart holds it, the checkout takes the €235. Which category that platform belongs to decides which later steps the platform can do itself and which must be bought and joined on.

Setting the Scene at Kestrel & Pine: Two Pitches in One Week

Kestrel & Pine, the fictional outdoor brand this book follows (Meet Kestrel & Pine), runs its storefronts, cart, checkout and wholesale portal on **the platform**: a hosted SaaS commerce platform used since 2016, on the enterprise tier since 2023, at **€2,300 a month plus payment and transaction fees**, with three localized storefronts in one tenant and 41 installed apps. Around it sit a dozen systems bought one at a time: the ERP (the finance and purchasing system), the WMS (the warehouse system), the POS (the store tills), the marketplace connector, the returns portal, the iPaaS (the integration tool) and more. Technology spend in 2026 was about **€2.6 million**, 5.4% of revenue.

In the second week of February 2027, Noor Visser, Head of Engineering since January, sat through two pitches. On Tuesday an outside consultancy proposed replacing the platform with a **composable** assembly: an API-only commerce engine, a separately built storefront, a search service, a content system and an integration layer, about **€3.2 million** over eighteen months (illustrative), with the promise that Kestrel & Pine would "own its architecture". On Thursday the agency that has built the storefront theme since 2017 proposed something smaller: keep the platform, but rebuild the three storefronts as a **headless** application (a storefront that is a separate program talking to the platform only through its API) on the platform vendor's own hosting, with a separate content system behind it, about **€240,000** and nine months (illustrative). Both decks used the Black Friday Oversell of November 2026 (1,340 units oversold, about €96,000 in cost) as the reason to act.

Mara Lindqvist, the CEO, asked Noor three questions afterwards. *What kind of platform are we on now? What would each of these cost us in year three? And what would a move fix that nothing else fixes?* Noor did not have the vocabulary for the first question on the spot, which is why this chapter exists. By March her answer had become Waypoint decision **W4: stay and surround**. What follows is the map she drew.

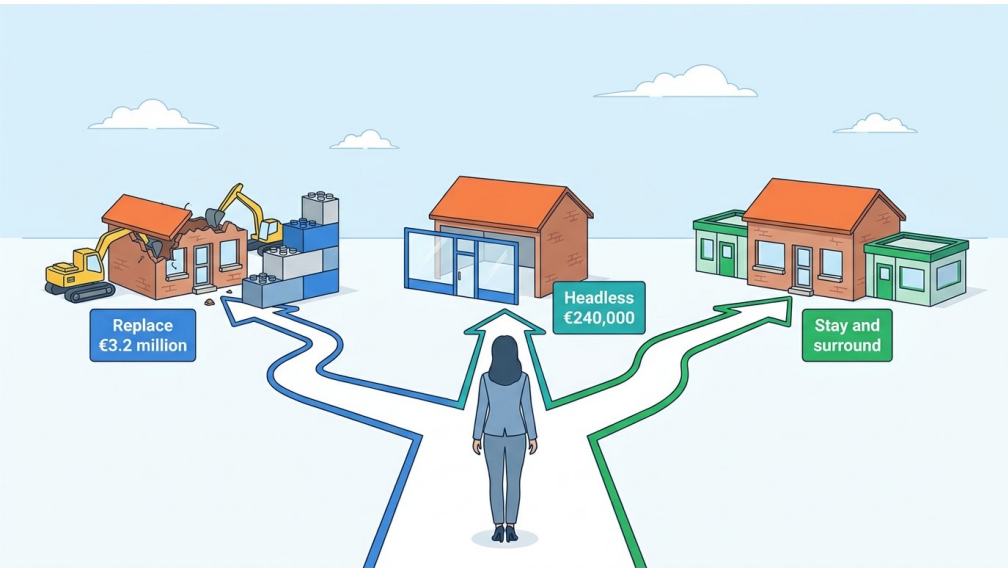


Figure 1: *Two pitches in one week: a €3.2 million replacement and a €240,000 headless rebuild, against the third answer Noor reached by March, Waypoint's W4, stay and surround.*

Seven Kinds of Platform

A **commerce platform** is the software that shows products, holds a cart, takes an order and the payment, and hands the order to whoever fulfills it. Some products stop there; others bundle warehouse, order management or finance too. The market sells this in seven shapes. The table is my synthesis; the categories overlap, and several vendors sell in more than one (Adobe sells on-premises, platform-as-a-service and SaaS).

Category	What you buy	Who runs it	Where the integration work lands
Hosted SaaS platform (Shopify, BigCommerce, Lightspeed, CCV Shop)	A storefront, cart, checkout and app store shared with many merchants	The vendor	On apps, webhooks and connectors you install

Category	What you buy	Who runs it	Where the integration work lands
Enterprise suite (SAP, Salesforce, Adobe)	One vendor's bundle of commerce, order management and content modules	The vendor's cloud, via a partner	Inside the vendor's data model, while you stay inside it
Open source and its forks (Magento and Mage-OS, WooCommerce, Shopware, PrestaShop)	Source code to host, extend and upgrade yourself; paid editions optional	You or your agency	On your agency, with upgrades as the recurring cost
Composable and headless vendor (commercetools, Elastic Path, Medusa, Saleor)	API-only commerce services; you assemble the rest	The vendor for the services; you for the rest	On you, by design
ERP-centered commerce (Odoo, JTL, PlentyONE, Business Central, NetSuite)	A storefront attached to the ERP that already owns stock, orders and finance	The vendor's cloud or your servers	Inside the ERP, while its pace is fast enough
Specialist layer (OMS, PIM, WMS, returns, search, channel, iPaaS, payments vendors)	One capability	The vendor	On the connectors between the layer and your platform
Specialist group (Katapull)	Several specialist products under one owner, pre-integrated	The group	Among the group's products; on you at its edge

Four terms carry the table. **SaaS** (software as a service) is software the vendor runs for many customers at once; you configure it and the vendor upgrades everyone on the same

day. A **suite** is one vendor's set of modules designed to work together. **Headless** means the storefront (what shoppers see) is separated from the commerce engine (catalog, cart, orders) and talks to it only through an **API** (application programming interface: the contract by which one program calls another). **Composable** means buying commerce as separate services, from one or several vendors, and assembling them. A **webhook**, which several tables mention, is a notification a platform sends to another system when something changes.

The column that matters most is the fourth. **Every category moves the integration work somewhere; none removes it.** Kestrel & Pine's 34 integrations exist because it chose a hosted platform and bought specialist layers around it. A suite would have fewer joins and one larger contract; a composable assembly more joins, all its own. The choice is who carries the work, at what price, with what freedom to change.

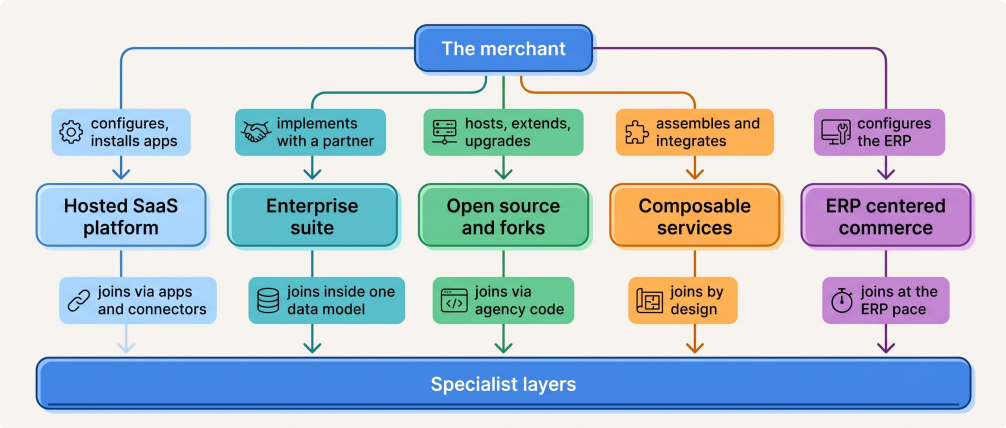


Figure 2: *Each platform category asks the merchant for a different kind of work and joins the specialist layers in a different place; the integration burden moves, it never disappears.*

Hosted SaaS platforms

The hosted platform is the default for a merchant that starts online, and the category with the most public evidence, because two of its vendors are listed. **Shopify**'s annual report for the year ended 31 December 2025 (filed 11 February 2026) reports gross merchandise volume (**GMV**, the total value of orders through the platform, which is not the vendor's revenue) of **$378.4 billion** and revenue of **$11.56 billion**. Its euro pricing page in November 2026 lists plans from Basic at €28 a month to Plus "starting at €2,100 a month", card rates from 1.9% + €0.25 down to 1.4% + €0.25, and a fee of 2% down to 0.2% on orders paid through another payment provider; larger Plus merchants may pay "a variable platform fee based on your revenue and business model". The same filing says most merchants are on the two cheapest plans while most GMV comes from Plus

and enterprise merchants: one product, two populations (Case: Shopify, the Modular Monolith That Serves Millions of Merchants takes this apart).

BigCommerce renamed its parent **Commerce.com** on 31 July 2025, keeping BigCommerce as the engine. Its November 2026 pricing page assigns plans by trailing-twelve-month GMV, with a 0.9% charge above $33,333 a month on the Scale plan: a subscription that upgrades you when you sell more is a GMV price in disguise. Its results for the quarter ended 30 June 2026 (filed 6 August 2026) showed revenue of $84.5 million, up 0.1%, and management saying B2C replatforming "remains softer than historically".

Lightspeed closed a strategic review on 6 February 2025 and chose to "focus on growing retail in North America and hospitality in Europe"; its FY2026 results (21 May 2026) showed transaction-based revenue of $815.1 million against subscription revenue of $370.7 million. Its Dutch eCom product still sells (€68 to €259 a month, November 2026).

> **INFERENCE, NOT DOCUMENTED:** Lightspeed has announced no change to its European e-commerce product. Reading the strategic review as a vendor-risk signal for a Dutch merchant on it, because European online retail is outside the vendor's stated growth engines, is my inference.

Below these sit the Dutch **CCV Shop** (€36 to €129 a month, "not a percentage of your sales") and the site builders.

Enterprise suites

A suite sells to large merchants through implementation partners, with negotiated prices, so its public evidence is thinner. **SAP Commerce Cloud** is described here only from its open-source storefront project Spartacus, which talks to the platform through its REST API (accessed November 2026), and from an October 2026 analyst note on SAP Connect 2026 that reports an "SAP Commerce Cloud, cloud ERP edition" for midsize organizations and warns buyers to "separate products that are available now from forward-looking demonstrations". SAP's product page refused automated fetching; I quote no feature list for it.

Salesforce announced on 6 July 2026 "its biggest Agentforce Commerce release yet": one platform across B2C, B2B, point of sale and order management, with three generally available agents "natively connected to catalog, inventory, and orders". Its pricing page shows "Contact for pricing" for every edition; the trade describes Salesforce as priced on a percentage of GMV, but the only written source here is a competitor's page, so treat that as industry belief.

Adobe Commerce is the clearest public example of one product in three deployment shapes. Adobe's information center (updated 21 October 2026) lists "Adobe Commerce on-premises" (you run everything), "Adobe Commerce on cloud infrastructure" (platform-as-a-service: Adobe runs the hosting, you own custom code) and "Adobe

Commerce as a Cloud Service" (fully managed SaaS with continuous updates). Beside them, **Adobe Commerce Optimizer** is a SaaS for storefront, catalog and merchandising that "can work alongside an existing Adobe Commerce or third-party commerce backend"; partner reports place both launches in June 2025. The middle option is where the **Forked Platform** pattern lives: the vendor runs the servers, the merchant owns the code, and deep customization can block every upgrade (Apps, Plugins and Functions: Extensibility and Ecosystem Governance).

Open source and its forks

Open source means the code is published under a license that lets you run and change it. It does not mean free: hosting, upgrades, security patches and the agency that applies them are the recurring price. Adobe bought Magento in 2018 and renamed Magento Commerce to Adobe Commerce in 2021; **Magento Open Source** continues, and a community **fork** (a copy maintained separately) called **Mage-OS** describes itself as an "independent, community-driven distribution of Magento Open Source", run by an association founded in 2022. Mage-OS 4.0.0 shipped on 30 October 2026; 3.6.0, two weeks earlier, was a security port of an Adobe patch, which is the dependency chain an open-source merchant lives with: Adobe publishes a fix, the distribution ports it, the agency applies it, the merchant tests it. **WooCommerce**, the open-source commerce plugin for WordPress, claims "more than 4 million stores" and prices hosting at $25 to $350 a month (November 2026).

Shopware, from Germany, has the most public evidence and the most instructive pricing. Its Community Edition is free; the paid plans (Rise "from €600 a month", Evolve "from €2,400 a month", November 2026) are "based on your gross merchandise value", and a fair-usage rule requires a paid plan once the shops on one account pass €1 million of GMV in a rolling year. So "open source" is free below the Growing stage and GMV-priced above it. The EHI Retail Institute's 2026 report found 12.5% of the 1,000 largest German B2C shops on Shopware, first place for the fifth year. In October 2025 PayPal raised its stake to more than 40%; on 10 June 2026 Shopware launched **Shopware Payments** on PayPal's infrastructure and **Nexus**, an orchestration layer it positions as replacing middleware. Case: commercetools, Shopware and the Composable Movement follows Shopware in detail.

Composable and headless vendors

commercetools, founded in Munich in 2006 and the best-known composable vendor, reported on 22 April 2026 that it had passed a "$100B GMV run-rate" with more than 600 million orders a year (a run-rate annualizes a recent period; it is not a full-year figure). On 26 February 2025 it had cut about 10% of staff after missing growth targets; on 21 January 2026 it launched **AgenticLift**, which lets companies "layer agentic commerce capabilities on top of your existing commerce solution"; on 15 July 2026 it began selling

its cart-and-order engine and its product catalog as standalone modules that integrate "with their existing technology stack". The flagship of composable now sells layers for stacks that are not composable. **Saleor**, an open-source entry, prices its cloud from $1,599 a month up to $200,000 of monthly GMV with 0.8% above (November 2026): API-only vendors price on GMV too.

MACH is the acronym a vendor alliance coined in June 2020 for Microservices, API-first, Cloud-native SaaS and Headless; the MACH Alliance's founders were commercetools, Contentstack, EPAM and Valtech. By November 2026 its principles page lists three principles (Open, Composable, Connected) and no longer defines microservices or headless. Here it is enough to know that "composable" and "MACH" name a category and a sales argument, not a verdict; the movement's rise, turn and own survey data are the subject of Case: commercetools, Shopware and the Composable Movement.

ERP-centered commerce

An **ERP** (enterprise resource planning system) keeps the books: purchasing, stock valuation, invoices, the general ledger. ERP-centered commerce makes the ERP the owner of products, stock and orders and treats the webshop as a front end. **Odoo** sells its eCommerce app free within a per-user ERP price (€19.90 or €35.90 per user per month, November 2026): the webshop costs nothing, the seats do. **JTL** (Germany) markets the ERP as the core with shop, WMS, marketplaces, EDI and POS around it; **PlentyONE** (formerly plentymarkets, renamed November 2024) bundles PIM, OMS, WMS and a free shop; **NetSuite SuiteCommerce** is described in Oracle's help pages as three products on one account. The strongest signal that this category is mainstream comes from Microsoft: its documentation dated 22 September 2026 says the Shopify Connector app "is preinstalled for all new sign-ups and trials" of Dynamics 365 Business Central. An ERP vendor shipping a connector to a hosted platform by default is "ERP as hub, storefront in front" declared as the expected shape.

The strength of this category at the Starting and Growing stages is one system and no integration problem. Its limit arrives at the Omnichannel stage, when stores, marketplaces and a warehouse each need stock in near real time and the ERP's batch cadence cannot serve them (The Back Office: ERP Integration and Who Is the Hub).

Specialist layers and specialist groups

Every category above is surrounded by **specialist layers**, each owning one capability and selling to every platform. The book's vocabulary comes from them, so the table defines each once.

Layer	What it owns, in plain words	Examples (vendor pages, accessed November 2026)
OMS (order management system)	The order after checkout: which warehouse or store fulfills it, what to promise, how to route a return	Manhattan Active Omni; Fluent Commerce ("150+ implementations"); Kibo
PIM (product information management)	The product record: attributes, completeness rules, enrichment, distribution to channels	Akeneo; inRiver ("1,600+ brands")
WMS (warehouse management system)	Physical stock and its movements: receiving, put-away, picking, packing	Manhattan; Picqer (NL, €379 a month with 3,000 orders, then €0.28 per order)
Returns	The reverse order: portal, rules, exchanges, labels, refunds	Loop ("5,000+ brands"); Narvar; Returnista (NL)
Search and discovery	Query understanding, ranking, merchandising rules	Algolia ($0.50 per 1,000 extra requests); Constructor
Channel integration	Listings, offers, stock and orders across marketplaces	ChannelEngine (NL, "over 1,300 marketplaces"); Rithum; Mirakl
iPaaS (integration platform as a service)	The flows between systems: connectors, mapping, retries, errors	Celigo; Boomi; Workato
PSP (payment service provider)	Taking the payment and settling the money	Adyen (interchange + 0.60% + $0.13); Stripe (1.5% + €0.25 on EEA cards in NL); Mollie (iDEAL €0.32)

OMS and PIM are the two layers Kestrel & Pine lacks and the two Waypoint buys (W2 and W3). An OMS decides which location fulfills an order and what to promise; a WMS decides where goods sit and how to pick them; The Order Record: Lifecycle, State and the Order Management System and Pick, Pack, Ship: Warehouse Management and Fulfillment keep the two apart.

A **specialist group** is a third shape beside suite and composable: several specialist products under one owner, kept as separate brands but pre-integrated. The Dutch group **Katapull** is the public example. Its site (accessed November 2026) names eight labels, one per role: Logic4 (ERP and webshop), Vendit (POS, point of sale: the store till software), Picqer (WMS), EffectConnect (marketplaces), DeliveryMatch (shipping), Returnista (returns), Transus (EDI, electronic data interchange: structured business documents between trading partners) and Highbiza (an online-performance agency). Five companies joined forces on 31 March 2025; Transus joined at a date no public page gives; Returnista followed on 16 February 2026 and Picqer, the eighth label, on 7 October 2026. The about page claims "4,500+ customers in 40+ countries"; no revenue, GMV or ownership figures are public, and I infer none. The group reduces the joins a merchant must build, at the cost of one owner across several layers.

How Platforms Are Priced, and What the Price Tells You

A platform's price is an architectural fact, because it tells you what the vendor's product really is. Seven pricing models recur across all seven categories.

Model	How it works	Dated examples	What it means for you
Flat subscription tier	Fixed monthly fee; limits rise with the tier	Shopify Basic €28 to Advanced €384; CCV Shop €36 to €129; Lightspeed eCom NL €68 to €259 (all Nov 2026)	Cheap to start; limits force upgrades
Subscription with GMV thresholds	Your tier is set by trailing sales; a percentage applies above a cap	BigCommerce 0.9% above $33,333 a month; Saleor 0.8% above $200,000 a month; Shopware paid above €1 million of GMV (Nov 2026)	Cost grows with revenue; the vendor takes a share once you are large

Model	How it works	Dated examples	What it means for you
GMV share (enterprise)	A negotiated percentage of sales, often with a floor	Shopify Plus "variable platform fee" above a $2,300 to $2,500 base (Nov 2026); Salesforce, widely described as 1% to 3% (unconfirmed)	Raises exit cost; wholesale (large orders, low margin) must negotiate separately
Payment fee share	The platform earns on card processing and charges a **transaction fee** per order when you use another PSP	Shopify 0.2% to 2% third-party fee; BigCommerce 0.6% to 2%; Lightspeed $815 million transaction revenue against $371 million subscriptions (FY2026)	Payments are the platform's core revenue; your PSP choice is constrained by the platform contract
Per order, **user**, **request**	A fee per unit above an included quota	Picqer €0.28 per extra order; Odoo €19.90 per user; Algolia $0.50 per 1,000 extra searches; Shopware Nexus €10 per 1,000 executions (Nov 2026)	Peak days and catalog size cost money

Model	How it works	Dated examples	What it means for you
Per transaction (payments)	One blended rate, or **interchange** (the card schemes' part of the fee) plus a margin	Adyen interchange + 0.60% + $0.13; Stripe NL 1.5% + €0.25; Mollie iDEAL €0.32 (Nov 2026)	Blended is simpler; interchange-plus is cheaper at volume but needs reconciliation (Taking the Money: Payment Providers, Authorization, Capture and Refunds)
Free core, paid services	Open-source software with hosting, extensions or editions sold	WooCommerce 0% revenue share; Mage-OS; Medusa ("no extra licenses or GMV-based tax")	Cost moves to the agency and to operations; you own upgrades

The pattern across the rows: **a vendor prices on the thing it sits in the path of**. GMV pricing assumes the vendor owns checkout; per-order pricing assumes it sees every order; per-request pricing assumes it is in the request path. This is why Shopify's FY2025 filing reports 76% of revenue as "merchant solutions" that are "directionally correlated with the level of GMV", with 65.6% of GMV through its own payment service, and why Lightspeed is, by revenue mix, a payments company with software attached.

A Kestrel & Pine illustration. Kestrel & Pine's records put the platform at €2,300 a month, €27,600 a year, "plus payment and transaction fees". Assume (an illustration, not one of Kestrel & Pine's records) that the platform's checkout carries the €27.0 million of D2C (direct-to-consumer) revenue plus the 62% of wholesale orders placed through the B2B (business-to-business) portal, about €7.4 million: roughly €34.4 million of GMV a year. At a 0.2% third-party transaction fee, the kind a top-tier hosted plan charges when the merchant brings its own PSP, that is about €69,000 a year, two and a half times the subscription; at a negotiated 1% platform fee on GMV, about €344,000, twelve times. Neither is what Kestrel & Pine pays; the point is that **the subscription line is the smallest line**, and a comparison that puts "€2,300 a month" against a competitor's list price compares entry tickets. Build, Buy or Assemble: Choosing and Changing Platforms builds the full cost model.

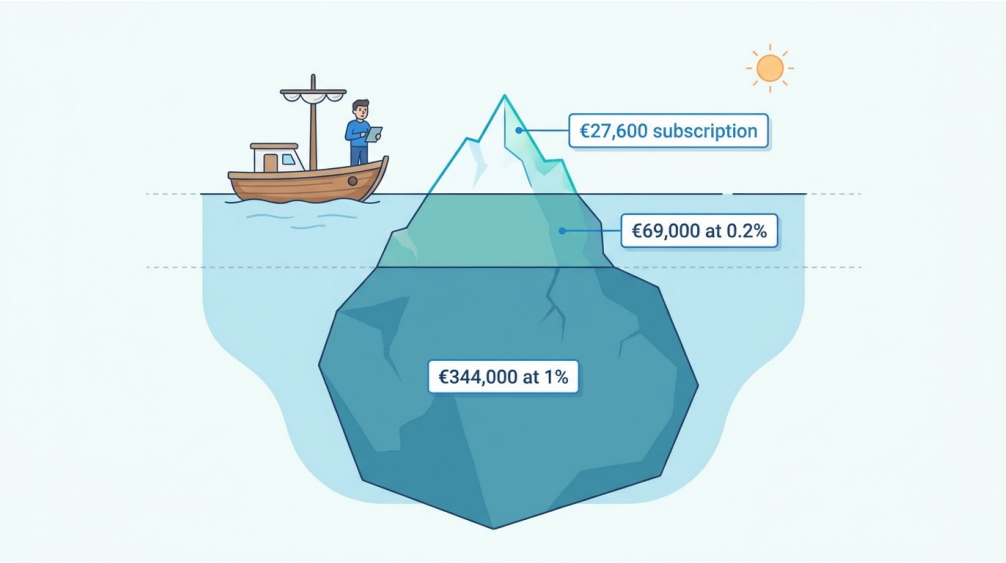


Figure 3: *The subscription line is the smallest line: on about €34.4 million of GMV a year, a 0.2% fee is two and a half times the €27,600 subscription and a 1% fee twelve times.*

What Changed, 2023 to 2026

The three years before this book's research cut-off (7 November 2026) moved in one direction: vendors retreated from physical logistics, pulled payments into the platform, and began selling layers beside a merchant's existing stack rather than replacing it.

Date	Event	What it tells a merchant
6 June 2023	Shopify completed the sale of most of Shopify Logistics to Flexport for a 13% equity stake	Platform vendors keep orchestration and push physical logistics to partners
26 February 2025	commercetools cut about 10% of staff after missing growth targets (TechCrunch)	Pure composable stopped selling itself

Date	Event	What it tells a merchant
June 2025	Adobe shipped Commerce as a Cloud Service and Commerce Optimizer, the latter "alongside an existing … third-party commerce backend"	A suite vendor sells SaaS layers beside a kept backend
21 January 2026	commercetools launched AgenticLift "on top of your existing commerce solution"	The composable founder sells a layer for non-composable stacks
10 June 2026	Shopware launched Nexus (orchestration replacing middleware) and Shopware Payments	An open-source platform absorbs integration and payments
6 July 2026	Salesforce shipped three agents "natively connected to catalog, inventory, and orders"	A suite vendor sells agents, not decomposition
15 July 2026	commercetools began selling its order engine and catalog as standalone modules "without replacing their existing platforms"	The composable flagship sells itself by the piece
6 August 2026	Commerce.com reported flat revenue and B2C replatforming "softer than historically"	Replatforming demand itself is weak

The newest surface is the **AI assistant as a channel**. Shopify's Agentic plan (accessed November 2026) has "no monthly fee" and syndicates products to ChatGPT, Gemini, Copilot and Perplexity so shoppers "check out in conversations", with "no Shopify online store required". For a merchant this is one more channel, priced as a channel; Assistants and Agents: AI in Commerce and What Is Real in 2026 separates what is available from what is announced.

How to Read a Vendor's Claims

Comparing two vendors starts with the **unit, the date and the definition** behind every number.

Claimed unit	Dated example	The trap
GMV	Shopify $378.4 billion in 2025; commercetools "$100B run-rate" in April 2026	A run-rate is not a full year; BigCommerce's plan GMV is "gross order value minus 10%", not its reported GMV
Merchants, customers, brands, users	Shopify's filing counts a merchant as a paying shop, so one brand with three stores counts three times; commercetools "500+ enterprises" in April 2026 and "more than 600" in July; Katapull "4,500+ customers" and "27,500+ users"	Dormant installs inflate open-source counts; "users" exceeds "customers" by a factor of five to ten
Marketplaces, integrations, apps	ChannelEngine "over 1,300 marketplaces"; Shopify "more than 21,000 apps" (31 December 2025)	A listing proves a connector exists, not that it is maintained or used
Surveys, analyst charts, performance claims	MACH Alliance, February 2026: 78% of mature adopters see "clear ROI from AI"; Gartner's 2025 quadrant rated 19 vendors, Forrester's Q3 2026 wave 10; Shopware Nexus "cut integration costs by 40%" (June 2026)	The vendor chose the sample; vendor counts change yearly and absence is not a verdict; unaudited claims are marketing

Market-share data need the same care. W3Techs, which crawls websites, reported on 7 November 2026 that WooCommerce runs 47.3% of sites with a detectable e-commerce technology, Shopify 32.5% and Shopware 0.2%; the EHI study of the 1,000 largest German shops put Shopware first. Both are true; they count different populations. One official statistic reframes the landscape: Eurostat's enterprise figures for 2024 (published February 2026) put web sales at 8.39% of EU enterprise turnover and sales by **EDI**-type messages (the business documents of Documents Between Companies: EDI, Punchout and E-Invoicing) at 11.07%. By value, more European enterprise e-commerce runs over EDI than over webshops, which is why Part V is not a side topic.

The Trade-offs Between Categories

The categories differ on five criteria a newcomer can check. The table is my judgment, from the evidence above and from patterns common among growing merchants.

Criterion	Hosted SaaS	Enterprise suite	Open source	Composable	ERP-centered	Specialist group
Time to first sale	Days	Months	Weeks to months	Months; a storefront must be built	Weeks	Weeks
Who upgrades	The vendor, everyone at once	The vendor or partner, per release	You or your agency	The vendor for services; you for what you built	The vendor	Each label's vendor
Integration burden	On apps and connectors	Low inside; high at the edge	On your agency	On you, by design	Low until the ERP's cadence is too slow	Low among labels; on you at the edge
Cost shape	Subscription plus a share of sales and payments	Negotiated, often GMV-linked, plus partner fees	Agency and hosting; GMV-priced editions above a threshold	Subscription with GMV caps plus the build	Per user plus ERP license	Per product
Exit cost	Moderate: export, rebuild theme and apps	High: one contract, one data model	Moderate to high: custom code tied to the core	High: every join is yours to rebuild	High: the ERP is the hub	High across several labels at once

The rule I use: choose the category by the team you have, not the team you want. A composable assembly needs people who build and run storefronts, integrations and services. A hosted platform with specialist layers needs people who own connectors and contracts. A suite needs a budget and a vendor manager. The category you cannot staff is the wrong one, whatever its feature list says.

Across Scale Stages: Which Categories Fit Which Stage

The four stages are the book's (Starting, Growing, Omnichannel, Group); Who Sells What to Whom: Business Models and the Economics That Shape Architecture defines them and What Changes at Each Scale: From First Store to Group works them.

Stage	Typical category	What becomes necessary	Warning signs of outgrowing it
1 — Starting (under about €2M)	Hosted SaaS or a site builder; an ERP-centered package for stock-heavy sellers; a 3PL (third-party logistics provider: a warehouse that ships for you)	The platform's own apps and payment service	The first store, wholesale account or marketplace; the first spreadsheet holding product data the platform cannot
2 — Growing (about €2M to €20M)	Hosted SaaS with the first specialist layers (accounting or ERP, a channel connector, a returns tool); open source with an agency	A named owner per integration; the platform's GMV tier and transaction fee reviewed against a PSP of your own	Pricing thresholds crossed in one year; the agency's customizations block an upgrade; stock in three systems

Stage	Typical category	What becomes necessary	Warning signs of outgrowing it
3 — Omnichannel (about €20M to €200M)	Hosted SaaS or open source surrounded by OMS, PIM, WMS and channel layers; a specialist group; a suite for the partner-led	Systems of record per fact; an OMS and a PIM; integration standards; composable layers per capability around a kept core	The joins fail at peak; a replatforming pitch after every incident; nobody knows how many apps write stock
4 — Group (over about €200M, or several brands)	A suite with a platform team, or a composable core with product teams	Tenancy and isolation; shared services against brand autonomy; versioned contracts for in-house APIs	One stack forced on every brand before the benefit is measured (One Size Fits None); a platform team that cannot keep the assembled services upgraded

The judgment: **at the Starting and Growing stages, "composable" is a vendor category, not an instruction**; at the Omnichannel stage it is something you do per capability, around a core you keep; only at the Group stage, with a platform team, is an API-only core realistic. The layers the 2026 vendors now sell beside existing stacks are the shape an Omnichannel merchant can absorb.

Kestrel & Pine's Place on the Map

Noor's answer to Mara's first question fits in one figure. The platform is a hosted SaaS product (€2,300 a month plus fees); the ERP (€3,900 a month) is an ERP-centered product used as the back office, not as the hub; the WMS (€2,100), the marketplace connector (€900), the returns portal (€650 plus €0.95 per return), the iPaaS (€1,200), the POS, the PSP, the EDI provider and the shipping service are specialist layers; and the two things Kestrel & Pine lacks, an OMS and a PIM, are specialist layers too.

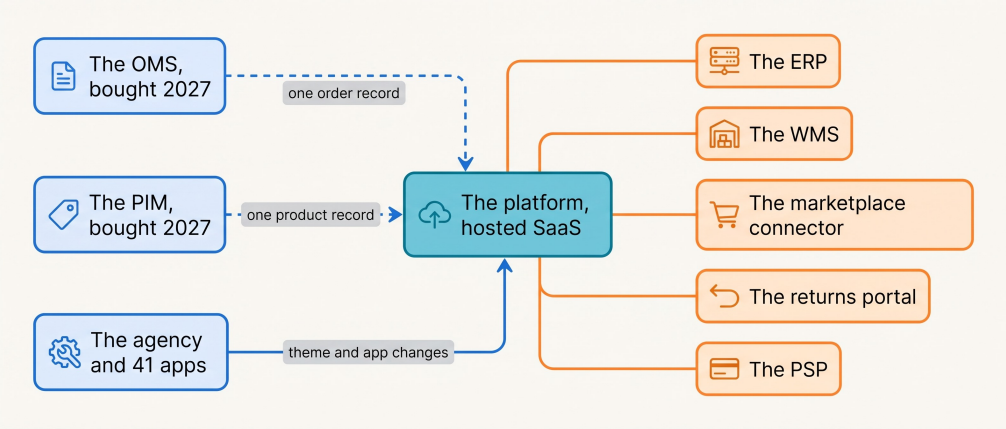


Figure 4: *Kestrel & Pine is a hosted platform surrounded by specialist layers; Waypoint adds the two layers it lacks beside the platform rather than replacing the center.*

Read against the landscape, the two February pitches look different. The composable pitch asked a 14-person technology team, three of whom work on storefront and apps, to build and run a storefront, a search integration, a content system and every join between them. It would have fixed the Black Friday Oversell only through an availability service mentioned in one bullet, and that service can be bought as a module of the OMS and connected to the existing platform (W1). That is **The Giant's Boots**: an architecture from a company with a product-engineering organization, proposed to a merchant with an integration team. It also had one date for the move, the cutover from the old platform to the new: **The Burned Bridge**. The headless pitch kept the platform but added a second application to build and run, and it would not have touched the stock chain that caused the Oversell at all. Its argument was speed; when Noor measured how fast the product pages responded for real shoppers, the slow part traced to scripts from eleven installed apps, not to the storefront's architecture, and a quarter of agency time on a performance pass fixed it (Delivering the Storefront: Themes, Headless, Rendering and Performance). Noor declined both.

W4 says stay on the hosted platform, add the OMS, the PIM and the availability service beside it, and review in 2029. The honest cost is the one the trade-off table names: the fee grows with sales, the PSP choice is constrained by the platform contract, and every added layer is a join that W5 must govern. Noor's note to Mara ended: *we are not choosing a platform; we are choosing which work we are willing to own.*

When It Goes Wrong

A platform category fails slowly, in four ways the public record illustrates.

The vendor stops funding your region or product. Lightspeed's February 2025 strategic review named two growth engines, neither of them European online retail.

Detection: read your vendors' results, strategic reviews and ownership changes once a year, and watch the revenue mix; a vendor whose transaction revenue outgrows its subscriptions will steer product toward payments. Recovery: an exit plan per platform, written before it is needed (Build, Buy or Assemble: Choosing and Changing Platforms).

The core is forked and cannot upgrade. Platform-as-a-service deployments and every open-source platform let the merchant or its agency change the core; the **Forked Platform** is the result when each release becomes a project and each security patch arrives months late. Detection: ask how many vendor releases behind the platform is. Recovery: move extensions to sandboxed or API-based mechanisms (Apps, Plugins and Functions: Extensibility and Ecosystem Governance).

The price crosses three thresholds in one year. A merchant moving from Growing to Omnichannel typically crosses the platform's GMV tier, the PSP's volume pricing and its WMS and channel tools' per-order quotas at about the same time, and discovers that "€2,300 a month" was never the price. Detection: a yearly total-cost review listing every GMV-, order- and request-priced line. Recovery: negotiate on actual GMV, with the B2B share separated.

An acquisition changes the product. PayPal's stake in Shopware became Shopware Payments within eight months; Katapull added two labels in 2026. Neither is a failure, but each changes what the platform will push. Detection: ownership in the yearly vendor review. Recovery: contracts that name the capability you depend on and its support period, so **The Lynchpin** has a date attached.

What to Take Away

Established.

- The market sells commerce in seven shapes, and the shape decides who runs the servers, who applies upgrades and where the integration work lands (Adobe's three deployment options, October 2026; Shopware's editions, November 2026).
- Platforms are priced mainly on sales and payments, not on software: Shopify's FY2025 filing (76% of revenue correlated with GMV), Lightspeed's FY2026 revenue mix, and the GMV-based plans of BigCommerce, Saleor and Shopware (November 2026).
- Between 2023 and 2026 the leading vendors moved to selling layers beside existing stacks (Adobe Commerce Optimizer, June 2025; commercetools AgenticLift, January 2026, and standalone modules, July 2026; Salesforce agents, July 2026), while Commerce.com reported replatforming demand "softer than historically" (August 2026).

Depends on context.

- A hosted platform with specialist layers fits from Starting through Omnichannel **if** the merchant can staff integration ownership; it limits when the GMV fee and the PSP constraint outgrow the margin, a Group-stage question.
- ERP-centered commerce fits Starting and Growing merchants with heavy stock and few channels; it limits at the Omnichannel stage, when stores, marketplaces and a warehouse need stock faster than the ERP's batch cadence.
- A composable core fits a Group-stage merchant with product and platform teams; at any smaller stage, adopt composable per capability (search, OMS, PIM) around a kept core.
- A specialist group reduces joins and concentrates vendor risk in one owner; it fits where one exit plan covering several layers is acceptable.

The author's synthesis.

- Choose the category by the work you can own, because every category moves the integration burden rather than removing it, and the burden you cannot staff fails on your busiest day.
- Treat every 2026 pitch, "composable" and "agentic" included, as a layer to add beside the stack you have, because that is what the vendors themselves now sell and what a merchant below the Group stage can absorb.

Questions to Ask

1. Which of the seven categories is our platform, and on which deployment option: who runs the servers, and who applies upgrades?
2. What did we pay the platform last year in total, including transaction and payment fees, and what share of that is a percentage of sales?
3. When did we last read our platform vendor's annual results, ownership changes and strategic announcements, and what did its revenue mix tell us?
4. How many vendor releases behind is our platform, and what blocks the upgrade?
5. For every "500+ customers" or "1,300 marketplaces" in a vendor deck, what is the unit, the date and the definition?
6. Which capabilities do we lack as layers (OMS, PIM, availability), can each be added beside the platform, and who of the people we have would own the joins?

Part I ends here, with the business, the capability map, the ownership of facts and the market in view. Part II — CATALOG: Products, Prices and Promises starts the catalog: the product record, the price, the promotion, the search result and the stock figure that every channel shows, beginning with What Are We Selling? Products, Variants and the Catalog and the PIM Kestrel & Pine buys in Q3 2027.

Sources and Evidence

Company, product and investor sources

- Shopify Annual Report on Form 10-K for the fiscal year ended 31 December 2025, Shopify Inc. via SEC EDGAR, 2026 — GMV and merchant definitions, FY2025 figures, revenue mix, Shopify Payments penetration, plan mix, app count.
- Shopify pricing (EUR), Shopify, n.d., accessed 2026-11 — plan prices, card rates and third-party transaction fees.
- Shopify Plus plan, Shopify Help Center, n.d., accessed 2026-11 — Plus base price and the variable platform fee.
- Shopify Agentic plan, Shopify, n.d., accessed 2026-11 — the no-monthly-fee agentic channel plan.
- Shopify completes sale of Shopify Logistics to Flexport, Shopify, 2023 — the retreat from physical logistics (more than three years old; the event, not the current state, is cited).
- Commerce.com Q2 2026 results, exhibit 99.1, Commerce.com via SEC EDGAR, 2026 — revenue, GMV, the "softer than historically" replatforming remark.
- BigCommerce rebrands as Commerce.com, Digital Commerce 360, 2025 — the rename and the three brands.
- BigCommerce pricing, BigCommerce, n.d., accessed 2026-11 — GMV-threshold plans, plan GMV definition and payment-provider fees.
- Lightspeed announces results of strategic review and Q3 FY2025 results, Lightspeed, 2025 — the two growth engines.
- Lightspeed Q4 and full-year FY2026 results, Lightspeed, 2026 — transaction versus subscription revenue.
- Lightspeed eCom pricing (NL), Lightspeed, n.d., accessed 2026-11 — Dutch tiers.
- CCV Shop pricing, CCV Shop, n.d., accessed 2026-11 — flat tiers, "not a percentage of your sales".
- Spartacus repository, SAP on GitHub, n.d., accessed 2026-11 — SAP Commerce Cloud's composable storefront.
- SAP's CX vision gets GA timeline at Connect 2026, SoftwareReviews, 2026 — the cloud ERP edition and the "available now versus demonstrations" warning.
- Salesforce's biggest Agentforce Commerce release yet, Salesforce, 2026 — the three agents and the "AI influenced 20%" statistic.
- Salesforce Commerce pricing, Salesforce, n.d., accessed 2026-11 — "Contact for pricing" on every edition.
- Adobe Commerce platforms and services, Adobe Experience League, 2026 — the three deployment options and Commerce Optimizer.
- Adobe Commerce as a Cloud Service overview, Adobe Experience League, 2026 — managed SaaS with continuous updates.

- Demystifying Adobe Commerce as a Cloud Service, Rightpoint, 2025 — the June 2025 launch timing (partner report).
- Magento, Wikipedia, 2026 — the 2018 Adobe acquisition and the 2021 rename.
- Mage-OS home and about, Mage-OS Association, n.d., accessed 2026-11 — the distribution, the association, releases 3.6.0 and 4.0.0.
- WooCommerce pricing, WooCommerce / Automattic, n.d., accessed 2026-11 — free core, hosting and extension costs; the competitor remark on Salesforce pricing.
- PrestaShop, PrestaShop, n.d., accessed 2026-11 — the open-source platform named in the category table.
- Shopware pricing, Shopware, n.d., accessed 2026-11 — GMV-based plans and the fair-usage threshold.
- Shopware Community Day 2026, Shopware, 2026 — Nexus, Shopware Payments, MCP coverage and the integration-cost claims.
- Shopware remains market leader in Germany 2026 (EHI), Shopware, 2026 — 12.5% of the top 1,000 German B2C shops.
- Shopware a Visionary in the 2025 Gartner Magic Quadrant, Shopware, 2025 — report date and vendor count.
- PayPal increases stake in Shopware, Ecommerce News Europe, 2025 — the stake rising to about 41%.
- commercetools reports strong Q1 2026, commercetools, 2026 — the $100B run-rate, orders, countries, the CEO.
- commercetools launches AgenticLift, commercetools, 2026 — the "on top of your existing commerce solution" layer.
- commercetools launches modular commerce offerings, commercetools, 2026 — standalone Core Commerce and Product Catalog modules; the "more than 600" boilerplate.
- commercetools lays off dozens of staff, TechCrunch, 2025 — the February 2025 cut and its stated reasons.
- Elastic Path, Medusa and Saleor pricing, vendor pages, n.d., accessed 2026-11 — positioning, licensing and GMV-capped pricing.
- Odoo pricing, JTL and PlentyONE, vendor pages, 2026 — per-user pricing and ERP-centered positioning.
- Get started with the Shopify Connector, Microsoft Learn, 2026 — the connector preinstalled in Business Central.
- SuiteCommerce solutions, Oracle NetSuite help, n.d., accessed 2026-11 — the three SuiteCommerce products.
- Katapull home, about and Picqer acquisition, Katapull, 2026 — the eight labels, the claimed counts, the October 2026 deal.
- Vijf softwarebedrijven bundelen krachten in Katapull, Dutch IT Channel, 2025, and Katapull breidt e-commerce platform uit met overname Returnista, Dutch IT Channel, 2026 — the formation and the Returnista acquisition.

- Specialist-layer vendor pages, n.d., accessed 2026-11: Manhattan Active Omni, Fluent Commerce, Kibo, Forrester Wave Q3 2026, Akeneo, inRiver, Picqer pricing, Loop, Algolia pricing, ChannelEngine, Celigo, Adyen pricing, Stripe pricing (NL), Mollie pricing — the definitions, claims and prices in the specialist-layer table.

Standards, regulation and official documentation

- E-commerce statistics (enterprises), Eurostat, 2026 — web sales versus EDI-type sales as shares of EU enterprise turnover, 2024 reference year.
- EPAM joins the newly formed MACH Alliance as a founding member, EPAM Systems, 2020 — the acronym's origin and founding members (more than three years old; cited as the founding document).
- MACH principles, MACH Alliance, n.d., accessed 2026-11 — the three principles as stated in 2026.
- MACH Alliance Enterprise Technology Report, MACH Alliance, 2026 — the survey figures used as an example of commissioned-survey claims.

Research and practitioner sources

- Usage statistics of e-commerce technologies, W3Techs, 2026 — website-based usage shares and their limits.
- BigCommerce named a Challenger in the 2025 Gartner Magic Quadrant, GlobeNewswire via Nasdaq, 2025 — the 19-vendor count.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Part II — CATALOG: Products, Prices and Promises

IN THIS PART, YOU WILL: Learn how a commerce platform decides what it sells, what it costs, what is discounted, how shoppers find it and whether it is really in stock, and who owns each of those facts.

KEY POINTS:

- **Catalog facts have different owners and different clocks.** Product content changes with the season and tolerates a day's lag; a price changes daily and carries legal weight in the EU; a discount is decided in the cart; stock changes by the second. One system of record (the system whose value wins when copies disagree) per kind of fact, one path per clock.

- **Every channel shows a copy, and a copy needs a source, an effective date and a reconciliation.** The German storefront at €179, the search index saying "in stock" and the marketplace selling two hours after the shelf emptied were all copies nobody compared with their source.
- **A promise is computed by a rule, at a moment, in one place, and then frozen.** The price is selected at checkout and copied to the invoice; the discount is computed once in the cart and allocated to every line; the last unit is committed to exactly one order.

FAILURE PATTERNS THIS PART ADDRESSES:

- **The Spaghetti Junction** — product content typed into four systems by four teams, so one color has three spellings and no owner.
- **The Price Nobody Can Explain** — one jacket, eleven prices, seven places, and no rule saying which is the source of which.
- **The Phantom Stock** — "in stock" computed from a copy of a copy, two hours behind the warehouse, on the busiest evening of the year.

Before a shop can take an order, it has to make four promises on one page: this is the product, this is what it costs, this is what is off today, and we have one for you. A fifth sits in the search box: type what you want and we will find it. Part I drew the map of a commerce platform and said every fact needs an owner. Part II is **everything the shopper sees before the cart**, and the first test of that rule, because these facts look alike and are not. A description has many producers and tolerates lag as long as it is complete; a price must be the same in every channel on the same day and, in Europe, keep its own history; a discount is only final once the cart knows the second item and the code; and a stock figure is stale within minutes at peak. Kestrel & Pine, the fictional outdoor brand this book follows, held each of these facts in four or more places in 2026, and each chapter opens with what that cost.

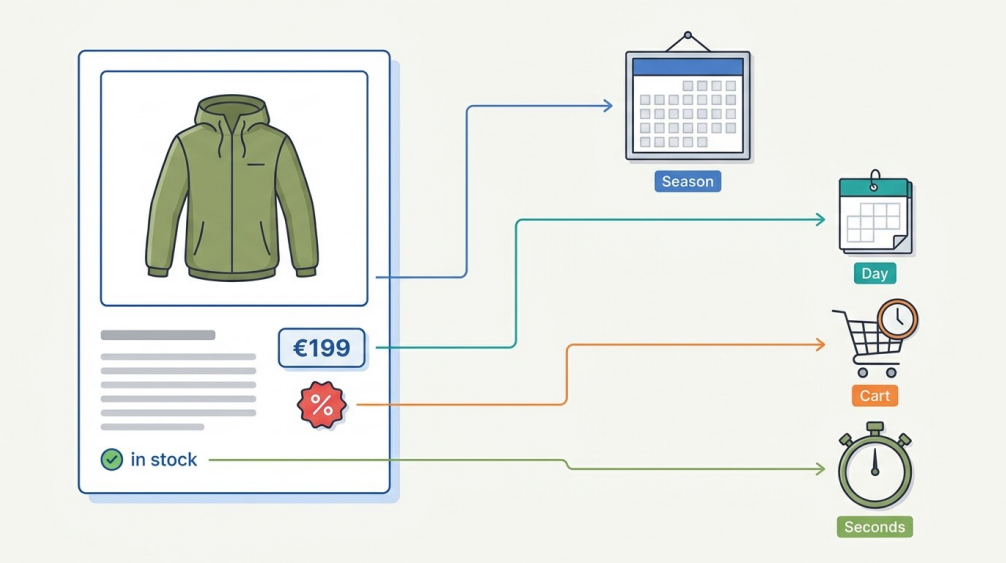


Figure 1: *Four facts that sit side by side on one product page run on four different clocks, which is why each needs its own owner and its own path to every channel.*

Where we are in the order's journey. Eva's jacket, the recurring order of Meet Kestrel & Pine, has not been ordered yet. Before Eva clicks "add to cart" on 13 April 2027, the Ridgewalker women's, size M, moss, must exist as a variant with a title, a Dutch description, a barcode number and a warehouse code; the search must find it from the words she types; the platform must select €199 from several candidate prices; the free-shipping rule must look at her €235 basket; and something must compute that a unit is available for the Dutch storefront. This part is step zero and step one of Follow the Order: What Happens Between "Buy" and "Delivered".

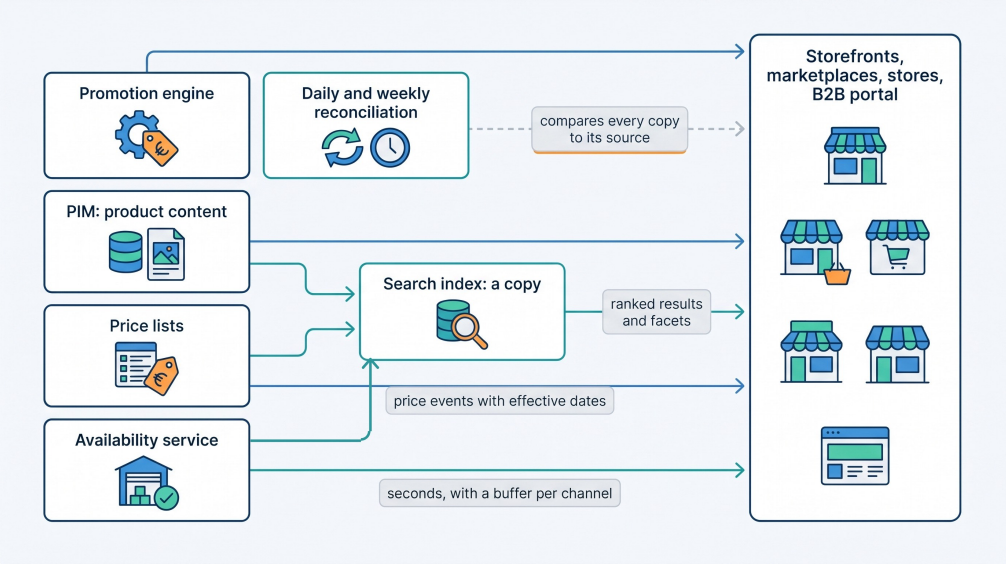


Figure 2: *The shape Part II recommends: four kinds of catalog fact with four owners and four clocks, a search index that is openly a copy of all of them, every channel a consumer, and a reconciliation that compares each copy to its source on a schedule.*

Chapters in this part

What Are We Selling? Products, Variants and the Catalog decides **what we are selling and who owns each field of it**: a product, its variants, their SKUs (the merchant's stock codes) and their GTINs (the global barcode numbers), identifiers that never change; one owner per attribute; and when a PIM, a product information management system, earns its place. The case is the Spring/Summer 2027 season typed four times, and Waypoint's W3.

What Does It Cost? Price Lists, Contract Prices, Currencies and Tax decides **where a price is selected and how every channel learns it**: one written selection rule, consumer prices in the platform, wholesale and contract prices in the ERP, every copy dated and reconciled, and price history kept because of the EU's 30-day prior-price rule. The case is the Price Drift of October 2026: €179 on the German storefront for nine days, about €8,200.

Discounts, Coupons and Loyalty: Promotions Without Chaos decides **how discounts, coupons, gift cards and points change the price without breaking the margin or the books**: rules with an order of evaluation and a stop rule, computed once in the cart and allocated to every line. The case is the Black Friday Stacking Bug of 2026: 2,860 orders, about €52,000.

Finding the Product: Search, Navigation and Merchandising decides **what goes into the search index and what is fetched fresh**: the index is a copy of content, prices, stock

and reviews, and a zero-result query is usually a missing attribute or synonym. The case is "waterproof jacket" returning nothing: a zero-result rate of 9.4% brought to 4.1%.

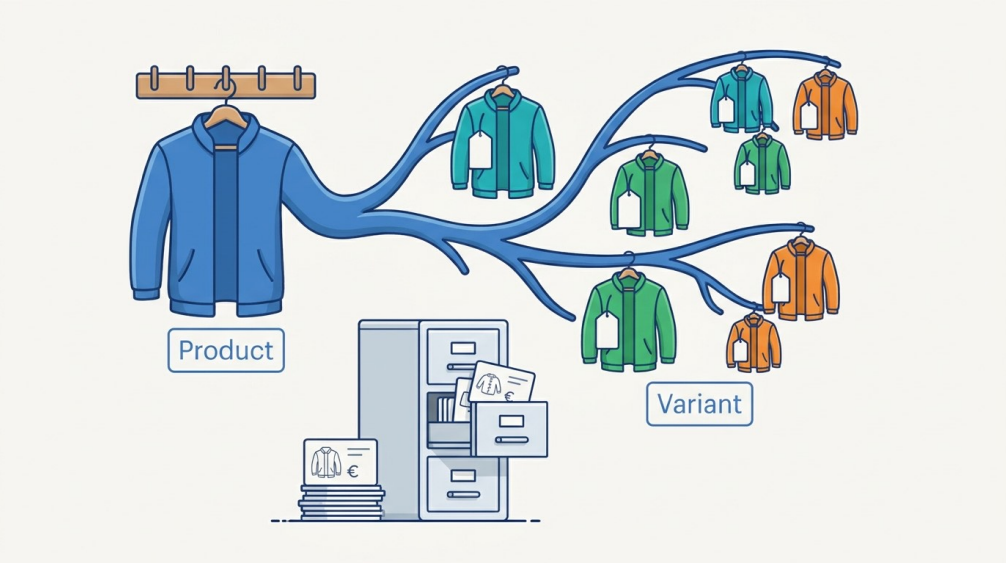
Is It in Stock? Inventory, Availability and the Promise You Can Keep decides **who counts stock, who commits it and who computes "in stock"**: on hand is counted, available is derived, one writer per stock state, stock events instead of hourly copies. The case is the Black Friday Oversell: 1,340 jackets and about €96,000 in one evening, and Waypoint's W1.

Begin with What Are We Selling? Products, Variants and the Catalog.

Sources and Evidence

Sources are cited in each chapter; this introduction makes no factual claim of its own. Kestrel & Pine is fictional, and its figures are the same in every chapter.

6. What Are We Selling? Products, Variants and the Catalog



IN THIS CHAPTER, YOU WILL: Learn how products, variants, identifiers and attributes are modeled, who should own each piece of product data, and when a PIM (product information management system) earns its place.

KEY POINTS:

- **A product is four things, and only two of them may ever change.** The product is what the shopper browses, the variant is what they buy, the SKU is the merchant's code for it and the GTIN is the world's. Titles and copy change freely; SKUs and GTINs never do, because orders, warehouses and marketplaces are keyed on them.
- **Product data has many producers and many consumers, so it needs one owner per attribute.** Purchasing, marketing, translators and suppliers write it; storefronts, marketplaces, search, the warehouse and wholesale read it. The design question is not which tool, but which system may change which field and how the others learn of it.

- **Every channel validates product data against its own rules and rejects quietly**. Google wants a stable id and a correct GTIN, bol wants an EAN and a classification "chunk", a storefront wants a translation. Syndication needs a per-channel outbox, a rejection report and a loop back to the owner.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Spaghetti Junction** — each channel gets product data through its own hand-built import, feed or spreadsheet, each with its own owner or none.
- **The Pulled Rug** — a marketplace changes its content rules or retires an API version and listings go offline before anyone is told.
- **The Giant's Boots** — a product information system is bought because larger brands have one, before the merchant has more than one producer and one consumer of product data.

Before a shop can promise a price, a delivery date or a return, it has to say what it is selling. That sounds too simple to need a chapter. It is not. **"The Ridgewalker jacket" is not one thing to a commerce platform**. It is a product the shopper browses, twenty-four variants the shopper can buy (six sizes in four colors, in a women's cut), twenty-four codes the warehouse counts, twenty-four barcodes the marketplaces scan, forty or so attributes, a dozen images, three languages of copy and a category in every channel's taxonomy. All of that has to agree, everywhere, every day of the season.

Most merchants do not design this. They type product data into whatever system needs it next: the accounting system when the item is bought, the web platform when it is sold, a spreadsheet when the season is planned, a marketplace tool when it is listed. Each copy is correct on the day it is made. Then a color is renamed, a translation is fixed, a marketplace adds a mandatory field, and the copies part ways. **The problem this chapter solves is not "how do I describe a product" but "how do I describe it once, and let every channel read the same description**."

Where we are in the order's journey. Eva's jacket, the recurring order of Meet Kestrel & Pine, starts before Eva does. Before she can search for a jacket on 13 April 2027, the Ridgewalker women's, size M, color moss must exist as a variant with a title, an image, a Dutch description, a GTIN on its barcode and a SKU the warehouse can pick. This chapter is the step before step one of Follow the Order: What Happens Between "Buy" and "Delivered": the record everything else refers to.

Setting the Scene at Kestrel & Pine: The Season That Was Typed Four Times

Kestrel & Pine is the fictional outdoor brand this book follows. It sells **3,400 products** in **26,000 sellable variants**, of which about **17,000 are active** at any time, with about **1,200 new variants** each season. In January 2027 its product data lives in **three places**: the platform (storefront content and variants), the ERP item master (the enterprise resource planning system's list of items, with purchasing data and costs) and shared spreadsheets (seasonal enrichment and translations). The marketplace connector holds a **fourth copy** for listings. There is **no PIM**.

In February 2027, Noor Visser, the new Head of Engineering, sat in on the Spring/Summer 2027 launch and wrote down what happened to one new variant of the Ridgewalker shell jacket on its way to being sold. The details are illustrative.

1. **Purchasing** created the item in the ERP: SKU, GTIN, supplier, landed cost (€58, what the item costs once delivered to the warehouse), purchasing unit. The color field said `MOSS GRN`.
2. **Teun Baas's product team** filled the "season sheet", a spreadsheet with the marketing name, the Dutch description, the fabric, the waterproof rating and the image file names. Color: `Moss`.
3. **The agency** imported the season sheet into the platform, a two-day job each season. The 20,000 mm waterproof rating stayed behind in the sheet; it never reached the platform as a field, so the storefront could not filter on it and search could not find "waterproof jacket" (Finding the Product: Search, Navigation and Merchandising).
4. **A translator** worked from a second spreadsheet. The German storefront launched with about **310 variants** still showing Dutch copy, for three weeks.
5. **Milan Petrov's marketplace team** re-mapped the new variants to bol's attribute model by hand in the connector. bol declined about **140 of the new variants** for missing mandatory attributes; the first anyone heard of it was the upload report, a day later. On the listings that went live, the color read `olive`, chosen from bol's list in 2024 and never revisited.

One color was typed four times, three ways, by four teams, and one marketplace quietly rejected a tenth of the season. Nobody had done anything wrong. **The landscape had no owner for product content, so every consumer of it had become a producer.** Waypoint's third decision, **W3**, answers this: a PIM in Q3 2027 as the system of record for product content and attributes, the ERP keeping costs and purchasing, the platform and the connector becoming consumers. This chapter explains why the order of those words matters.

Figure 1: *One color typed four times, three ways, by four teams: with no owner for product content, every consumer of it became a producer.*

What a Product Is: Four Things That Share a Name

The first thing a newcomer needs is that a "product" is a stack of four concepts, each identified differently and each stable to a different degree.

Term	What it is	Who assigns it	May it change?
Product	The sellable concept a shopper browses: "Ridgewalker shell jacket, women's"	The merchant	Title, copy and images change freely
Variant	One buyable combination of option values: size M, color moss	The merchant, from the options	Must stay stable once it has been sold
SKU	The merchant's own stock-keeping code for that variant, the thing you count on a shelf and put in a parcel	The merchant (usually the ERP)	Never

Term	What it is	Who assigns it	May it change?
GTIN	The Global Trade Item Number under the barcode, licensed from GS1, the barcode standards body, so that every company's systems mean the same item by it	A GS1 member organization	Never, and never reused for a different item

An **option** is a dimension of choice (size); an **option value** is one choice (M). A variant is one point in the grid the options define. The platforms and the standards all make this split. Shopify's developer documentation (n.d., accessed November 2026) describes the product as "the container for all product information (title, description, vendor)" and says "each variant represents a specific combination of option values, like 'Red / Small' ". The schema.org vocabulary, which search engines read from web pages, models the same thing in version 30.1 (September 2026): a `ProductGroup` is a family that differs "only along defined dimensions, such as size or color", each variant linked by `isVariantOf`.

The identifiers are where newcomers get hurt. A **SKU** is the merchant's code; nobody outside the company understands it, which is its strength. A **GTIN** is the global one. GS1, the standards body that runs the barcode system, says a GTIN is "unique, global and verifiable within the GS1 system" and that "only a GS1 Member Organisation can provide you with a GS1 GTIN" (GS1 support article, updated August 2024). The format on European retail barcodes is the 13-digit **EAN-13** (GTIN-13); a 14-digit GTIN-14 identifies a trade unit such as a carton of twelve, which is why wholesale and EDI need it and a web shop does not. Every distinct item and every packaging configuration gets its own GTIN. Kestrel & Pine has carried a GTIN on every own-brand SKU since 2021.

commercetools' product documentation (n.d., accessed November 2026) gives integrators the rule in one line: "treat the sku values as immutable identifiers". **The rule I use**: a SKU and a GTIN are promises to every other system, so they are assigned once, before the first sale, and a mistake is retired, never corrected in place. A reused SKU is a stock count that belongs to two items; a reused GTIN is a marketplace listing that shows the wrong product.

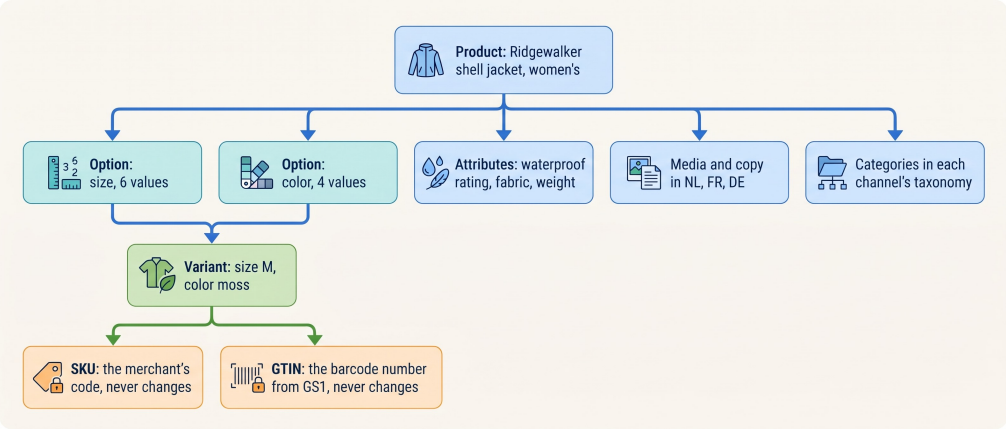


Figure 2: *One product is a grid of variants, each with two permanent identifiers, plus attributes, media and categories that change with the season; the model separates what may change from what may not.*

Attributes, attribute sets and taxonomies

An **attribute** is a typed field on a product or a variant: waterproof rating in millimeters, fabric, weight, care instructions, CE marking (the label that says a product meets EU safety rules). An **attribute set** is the list of fields a product of a given kind must carry: a jacket has a waterproof rating, a boot has a sole material, a stove has a fuel type. Attribute sets are the contract between the catalog and everything that reads it. A storefront filter, a search facet (Finding the Product: Search, Navigation and Merchandising) or a marketplace listing can only use an attribute that is filled consistently for every product of that kind. Kestrel & Pine's waterproof rating, living only in a spreadsheet, is invisible to all three.

A **taxonomy** is a fixed tree of categories a channel understands. Every channel has its own. Google's product taxonomy, the one its Shopping feeds use, is a file headed with version 2021-09-21 listing 5,595 categories; a feed may submit the numeric id or the full path such as "Apparel & Accessories > Clothing > Dresses" (Google Merchant Center help, n.d., accessed November 2026). bol, the largest Dutch and Belgian marketplace, describes its content model as a JSON file of more than 4,000 product classifications it calls "chunks", each defining the attributes a product must carry and which are mandatory (bol Retailer API documentation, n.d., accessed November 2026). The merchant therefore needs an internal taxonomy, the one it merchandises by, and a **mapping** per channel: a table that translates its categories and attribute values into each channel's. The mapping is data, not code, and it has an owner.

Bundles, digital goods, translations and lifecycle

Four product shapes break the simple grid. A **bundle** or **kit** is a product sold as a set of other products: a tent with its footprint. The components keep their own identity and stock; the bundle is a selling arrangement, not a thing on a shelf, which decides where stock lives (at the component; Is It in Stock? Inventory, Availability and the Promise You Can Keep) and how the bundle price is split back to the components on the invoice (Discounts, Coupons and Loyalty: Promotions Without Chaos). A **configurable product** is the parent that groups purchasable children; Adobe Commerce's documentation (updated October 2026) distinguishes a simple product, "a physical item with a single SKU", from a configurable one whose options each represent "a separate, simple product with a distinct SKU". A **digital good** has no shipment (a download, a license, a service) and skips the warehouse, the carrier and the usual tax treatment, which is why every platform models it as its own type. A **gift card product** is sold like a product but creates a prepaid balance, a liability in the books (Tax, Invoices and the Books: From Order to Ledger).

Translation is per product, per language, and often per channel. Shopify's documentation puts it from the merchant's side: "each published language gets its own URLs, and your store displays the translated version when translations exist" (Shopify Help Center, n.d., accessed November 2026). Kestrel & Pine sells in three countries and two languages in Belgium alone, so every description exists at least three times by design, and each translation has its own completeness status.

A product also has a **lifecycle**: draft, active, end of season, discontinued. A product is never deleted once it has been sold: orders, returns, invoices and analytics reference it for years, so "discontinued" is a state, not a deletion. commercetools enforces this mechanically: it separates staged from published data and "you can only delete unpublished products". And lifecycle is per channel: Google Merchant Center's `availability` takes `in_stock`, `out_of_stock`, `preorder` or `backorder`, and an item that is out of stock must still show its price on the landing page (product data specification, n.d., accessed November 2026). A variant can be end-of-season on the storefront and still listed on a marketplace clearing the last units.

Who Owns Which Field: Producers, Consumers and the System of Record

Now the architectural problem. Product data has **many producers**: purchasing creates the item; the product team describes it; translators localize it; suppliers provide specifications for third-party brands; the marketplace team adapts it to each channel. And it has **many consumers**: the storefronts, the search index, the marketplaces and feeds, the warehouse (which needs weight and dimensions to pick a box), the wholesale

portal, the data warehouse, and, from 2027, AI agents that need accurate product data before they can shop (Assistants and Agents: AI in Commerce and What Is Real in 2026).

Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record introduces the **system of record**: for each fact, the one system whose value is the truth when copies disagree. For product data, that decision must be made **per attribute**, not per product, because the attributes have different natural owners. The table below is the one the book uses; it is an industry-typical arrangement, not a law.

Data	System of record	Who produces it	Who reads it	Why here
Item identity: SKU, GTIN, supplier, purchasing unit, landed cost	**The ERP**	Purchasing	PIM, platform, WMS (warehouse management system), EDI (electronic data interchange with wholesale customers), finance	The item must exist and be valued before it can be bought, counted or sold
Logistic attributes: weight, dimensions, customs code	The ERP (or the PIM, if the ERP cannot hold them)	Purchasing, the warehouse at first receipt	WMS, carriers, customs	Measured once, needed for every parcel
Product content: names, descriptions, attributes, media, translations, internal categories	**The PIM** (the platform at the Starting and Growing stages)	Product team, translators, suppliers	Platform, marketplaces, feeds, search, wholesale, print	Many producers, many consumers; needs a workflow

Data	System of record	Who produces it	Who reads it	Why here
Channel mappings: category and attribute translations per channel	The PIM (or the feed tool)	The marketplace team	Connector, feed tool	Changes when a channel changes its rules
Storefront presentation: collections, theme blocks, landing pages	The platform	Ecommerce team	Storefront only	Belongs to one channel
Listings: the channel-specific product page	The channel, via the connector	Derived from the PIM and the mappings	Shoppers on that channel	A copy, by design
Price	See What Does It Cost? Price Lists, Contract Prices, Currencies and Tax			Changes at a different cadence
Stock	The WMS and the availability service, see Is It in Stock? Inventory, Availability and the Promise You Can Keep			Changes every minute

The last two rows explain why price and stock have their own chapters: they are product-related data with different freshness needs. A description can be a day old.

A price that is a day old is a broken promise with legal weight. A stock figure that is an hour old is the Black Friday Oversell. **Product content**, **prices and stock travel on separate paths** even when they end up in the same marketplace feed.

Read against this table, Kestrel & Pine in January 2027 has no owner for the "product content" row. The platform holds content because the storefront needs it; the spreadsheets because the product team works there; the ERP a name because an item must have one; the connector a re-mapped copy because bol needs its chunk. Four producers, no system of record. This is the **Spaghetti Junction**: every pair of systems has its own path, each with its own owner or none.

Requirements per flow

When product data moves between systems, the flows have different requirements (APIs, Events, Files and Middleware: How Commerce Systems Talk asks for this table whenever flows are discussed).

Flow	How fresh	How consistent	How fast	Availability need
New season into the ERP (identity, cost)	Weeks before launch	Exact: one SKU, one GTIN per variant	Batch is fine	Office hours
Enrichment into the PIM (content, media, translations)	Continuous during the season build	Workflow states: draft, enriched, translated, approved	Human speed	Office hours
Publication to the storefront and search	Minutes after approval, or on a scheduled date	Whole product at once: never a title without its variants	Minutes	Launch day matters
Syndication to marketplaces and feeds	Hours is usually acceptable for content	Per channel's rules; rejections reported back	Asynchronous, with a report	Launch and peak weeks

Two things stand out. Content tolerates lag but demands completeness: a product

published with a missing size or an untranslated description is worse than one published an hour later. And syndication is **asynchronous by nature**: the channel decides later whether it accepts the product. bol's documentation says an upload report may take an hour; Google disapproves a product with an incorrect GTIN. The architecture must expect a "no" to arrive after the "sent".

How the Pieces Fit Together With a PIM

A **PIM** (product information management system) is the editorial system where product data is created, enriched, translated and approved before it is published to channels. **Enrichment** is adding the marketing, technical and media content a channel needs beyond what the ERP holds. **Syndication** is publishing product content to each channel in that channel's format and under its rules. A **feed** is a file or API stream of product records in a channel's format; **structured data** is product information embedded in the web page itself (schema.org) so that search engines can read it.

The vendor descriptions converge. Akeneo calls a PIM "a system or solution that helps businesses manage, centralize, and distribute product information", a "central source of truth" that collects "data from your ERP, MDM, PLM, and other systems" (master data management and product lifecycle management, the systems where products are designed), enriches it and manages "catalogs for every sales channel in one system of record" (Akeneo, n.d., accessed November 2026). Pimcore describes an "open core platform for Product Experience Management" (GitHub, version 2026.1, accessed November 2026). These are vendor pages; none gives a testable threshold for when a PIM pays off. The threshold comes later in this chapter.

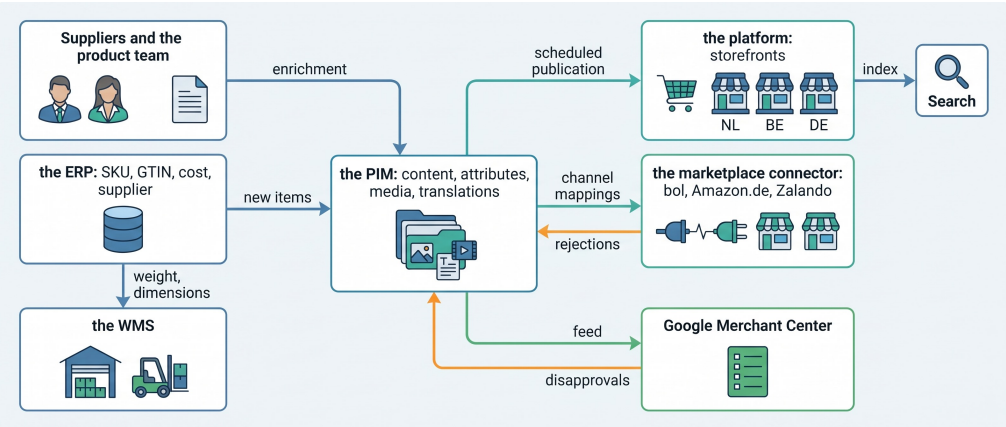


Figure 3: *With a PIM, the ERP still creates the item and keeps its cost, the PIM owns everything a shopper reads, every channel is a consumer, and each channel's rejections flow back to the owner instead of dying in an upload report.*

Three design rules make this picture work, and they are rules about ownership, not about the tool.

Rule 1: each attribute flows in one direction. The ERP creates the SKU; the PIM may not. The PIM owns the description; the platform may not edit it, only display it. Microsoft's documentation for the Business Central Shopify connector states this bluntly for a two-system case: "decide which system is the system of record for product data" before configuring the sync, choosing the direction per field, with the Shopify SKU or barcode as the mapping key (Microsoft Learn, 16 October 2026). A field that can be edited in two places will be.

Rule 2: publish whole products on a schedule, not fields on change. Content changes constantly during a season build and nobody wants half-enriched products on the storefront. Waypoint's integration standard (W5) says products move "by scheduled publication" while stock, orders and prices move by **events** (a message sent the moment something changes). commercetools builds the same idea into its product model with staged and current data; Shopify's `productSet` mutation is designed for the case where "an external system is the source of truth" and replaces a product's options and variants in one operation (Shopify developer documentation, n.d., accessed November 2026). A PIM publication needs exactly that: a whole product, atomically.

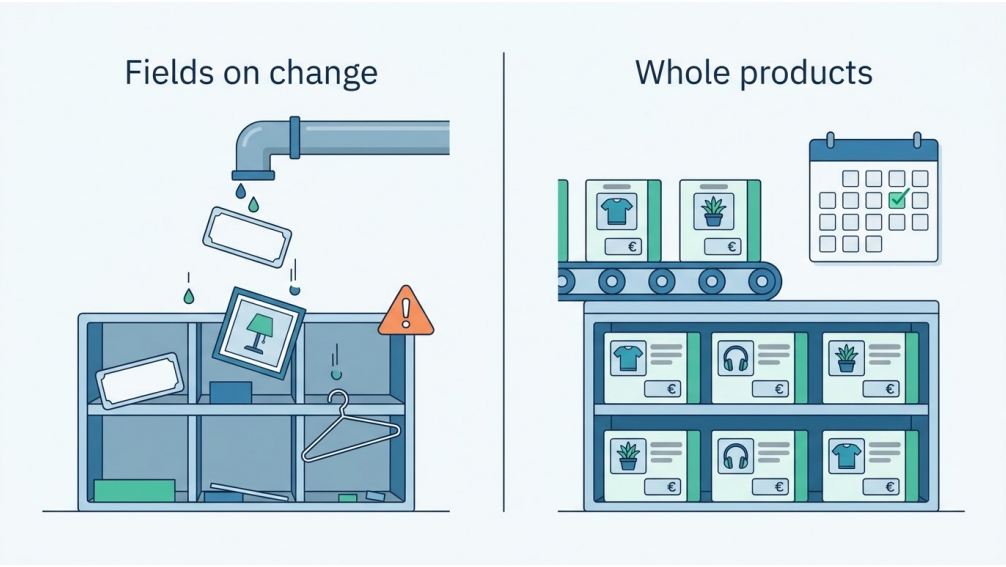


Figure 4: *Content changes all through a season build, so products are published whole on a schedule while stock, orders and prices move by events.*

Rule 3: every channel's answer comes back to the owner. A syndication path has an outbox per channel (what was sent, when, under which version of the channel's rules), a validation step that runs the channel's known rules before sending, and a rejection

report that lands in the PIM as a task for a person (Selling on Someone Else's Platform: Marketplaces and Channel Integration owns the listing operations behind it). Without the loop, 140 declined variants are a line in a report nobody reads.

The Choices

There are four realistic ways to arrange product data.

1. **The platform is the catalog**. Product data is created and edited in the commerce platform; the ERP gets a copy or just a SKU and a price; marketplaces are fed from the platform through a connector. This is every merchant's starting point, and for one channel and one language it is right.

2. **The ERP is the product master**. The ERP creates the item and holds its name, attributes and sometimes its description; the platform receives items by connector (The Back Office: ERP Integration and Who Is the Hub). Common where the ERP came first (wholesalers, manufacturers). It works for identity and cost. It fails for content, because ERP item masters were designed for purchasing, not for marketing copy in three languages with images and a translation workflow; the content ends up in spreadsheets beside the ERP, which is Kestrel & Pine's situation.

3. **Spreadsheets with discipline**. The honest name for what most Growing-stage merchants do: a season sheet, an import, a translation sheet. Cheap and flexible. Its limit is not size but the number of hands: once more than two teams write to the sheet and more than two systems read from it, there is no record of who changed what, no workflow state and no validation.

4. **A PIM as the system of record for content**. The ERP keeps identity and cost; the PIM owns content, attributes, media, translations and channel mappings, with a workflow (enrichment status, approvals, completeness per channel) and syndication; the platform and the connector become consumers. This is Waypoint's W3. It is right under the conditions the next table makes explicit, and the Giant's Boots when they do not hold.

A fifth option, a full master data management platform spanning products, customers, suppliers and locations, belongs to the Group stage and to One Customer, One Product: Identity and Master Data Across Systems.

Trade-offs

	Platform as catalog	ERP as product master	Spreadsheets with discipline	PIM as system of record
Right when	One or two channels, one language, one team editing	ERP-first business, few channels, thin content (industrial parts, wholesale-only)	Two or three channels, a small product team, content richer than the platform holds	Several channels and languages, more than one producing team, channel rules to track
Cost	Included in the platform	Included in the ERP; connector by the ERP partner	Nearly free in licenses; expensive in people at every launch	A subscription *(assumption: a few hundred to a few thousand euros a month at mid-market)* plus implementation and migration
Workflow and approvals	Minimal (draft and active)	None for content	None; versions by file name	Built in: states, completeness, per-channel readiness
Translations	Per storefront, via apps or CSV	Rarely supported well	A second sheet	Native, per locale and channel
Channel validation	The connector's, after sending	None	None	Rules per channel before sending; rejection loop

	Platform as catalog	ERP as product master	Spreadsheets with discipline	PIM as system of record
Warning sign of outgrowing it	The first marketplace rejects products in bulk; the first second language	Content lives in spreadsheets beside the ERP	A launch takes longer than two weeks; nobody knows which sheet is current	Rare; the risk is buying it too early and running it as a spreadsheet with a login

My recommendation, with its conditions. A PIM becomes necessary when product data has **more than one producer** (buying, marketing, suppliers, translators) **and more than one consumer** (web, marketplaces, print, wholesale, stores), and the platform's product record can no longer hold the workflow between them. Below that line, the platform or a disciplined spreadsheet is cheaper and the PIM is the Giant's Boots: a workflow tool for a workflow that does not exist yet. Above it, the absence of a PIM shows up as channel-specific copies that drift, and the cost of the drift (declined listings, untranslated launches, a two-week season import) usually exceeds the subscription within a season. Kestrel & Pine crossed the line around 2021–2022, when it added marketplaces (2021) and a third country (2022) to a Belgian storefront that already ran in two languages.

Two assumptions sit under that recommendation. The ERP keeps item identity and cost, so the PIM is bought for content, not as a second item master. And the merchant can name one owner for product content with authority over the product team, the translators and the marketplace team at once; at Kestrel & Pine that is Teun Baas, who owns the catalog and the seasons. Without such an owner the PIM becomes a fifth copy.

Across Scale Stages: When It Fits and When It Limits

The four stages are the ones this book uses throughout (see Meet Kestrel & Pine and, in depth, What Changes at Each Scale: From First Store to Group).

Stage	What is typical	What becomes necessary	Warning signs you have outgrown it
1 — Starting (under about €2M; Kestrel & Pine 2016–2018)	The platform is the catalog; SKUs typed by hand; GTINs bought in a block; the platform's built-in Google feed; one language	SKU and GTIN discipline from day one: one SKU per variant, never reused, never corrected in place	Variants split into separate products to get around a limit; SKUs that encode the price or the season
2 — Growing (about €2M–€20M; 2019–2022)	An ERP with its own item master; the first marketplace through a connector; a season spreadsheet; the first second language	A written ownership rule per field between platform and ERP; a mapping table per channel; the item created in the ERP before it is sold anywhere	The first marketplace declines products in bulk; the same color has three spellings; the season import takes days; translations lag launches
3 — Omnichannel (about €20M–€200M; 2023 onward)	Stores, wholesale, three marketplaces, several countries; a dozen systems reading product data; four copies	A PIM as system of record for content, the platform demoted to a channel; attribute sets as a contract with search and filters; syndication with a rejection loop; product data as a prerequisite for EDI, marketplace rules and agents	A season launch is the slowest project of the year; a marketplace rule change takes listings offline before anyone knows; the data warehouse counts products three ways

Stage	What is typical	What becomes necessary	Warning signs you have outgrown it
4 — Group (over about €200M or several brands)	Several brands, each with a catalog; shared marketplaces and retailers; print and B2B (business-to-business) catalogs	A group taxonomy with brand extensions; shared syndication with brand-level channel accounts; a master data function that allocates identifiers across brands	Two brands list the same third-party product with different GTINs; one brand's PIM becomes everyone's bottleneck; a retailer receives two feeds from one group

The judgement: **identifier discipline does not change with scale; everything else does**. A SKU and a GTIN assigned correctly at the Starting stage save a migration at every later one. The ownership rule changes once, at the Growing-to-Omnichannel boundary, when content gets its own system; after that the question is how many brands share it.

In Practice

What the platforms and channels document

Variant limits shape catalogs. As of November 2026: Shopify's historic limit was 100 variants per product; new GraphQL product APIs supporting 2,048 shipped in API version 2024-04, and on 15 October 2025 the 2,048 limit opened to all merchants, with a warning that apps not on the supported APIs "may have a downgraded or broken experience" (Shopify changelog, 15 October 2025). commercetools' 2026 release notes describe two catalog models: the Classic model keeps variants embedded in the product with a 100-variant limit, while the Modular Catalog, generally available from 29 September 2026, makes variants first-class resources with up to 10,000 per product (commercetools release notes, 2026). Both are recent and will move. The lesson is older than either: an apparel catalog of sizes times colors routinely passes 100 combinations, and a merchant that split products to fit a limit poisoned its search, feeds and analytics for years after the limit was lifted.

Channels validate, and reject. Google Merchant Center's product data specification

requires `id` (at most 50 characters; "keep it stable"), `title`, `description`, `link`, `image_link`, `price` and `availability`; `gtin` is strongly recommended and "if you submit a product with an incorrect GTIN value, your product will be disapproved"; variants need `item_group_id`; and from 31 January 2027 a 500 × 500 pixel minimum image will be enforced (Google Merchant Center help, n.d., accessed November 2026). As of November 2026, bol requires an EAN for every product, declines content without one, makes its first two enrichment levels mandatory, takes products offline where a required CE marking is missing, and updates its data model "daily". Every one of these is a rule that changes on the channel's schedule, which is the Pulled Rug seen from the catalog side.

Ownership direction is a documented setting. The Business Central Shopify connector (Microsoft Learn, 16 October 2026) asks the merchant to choose a direction per product field ("If you manage item names, descriptions, and pricing in Business Central, choose the To Shopify direction. If your merchandising team maintains product content in Shopify … choose From Shopify") and maps by SKU or barcode. That page is this chapter's argument in two sentences.

Kestrel & Pine: what W3 means in practice

Waypoint W3 introduces a bought PIM in Q3 2027 as the system of record for product content and attributes. The sequence below is this chapter's reading of that decision, with illustrative dates consistent with the rest of the book.

Ownership first. In April 2027, building on the entity–owner table of March (Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record), Noor, Teun and Milan wrote the attribute-level ownership table for the Ridgewalker as a one-page document before any vendor demo. The ERP keeps SKU, GTIN, supplier, landed cost (€58 for the Ridgewalker), purchasing unit, weight and dimensions. The PIM owns the marketing name, the descriptions in Dutch, French and German, the attribute set for jackets (waterproof rating, breathability, fabric, care, CE and other markings), media, the internal category and the mappings to Google's, bol's, Amazon.de's and Zalando's taxonomies. The platform owns collections and landing pages and nothing else about the product. The connector owns nothing; it derives listings. Color values are a controlled list in the PIM: `moss` has one spelling and a mapping to `olive` where a channel's list has no moss.

One direction per field. The ERP creates items and the PIM imports them nightly; the PIM cannot create a SKU. The PIM publishes whole products to the platform on a schedule, with a launch date per season, and the agency's season-sheet import is retired. The platform's product editor is read-only for owned fields.

Syndication with a loop. The PIM syndicates to the connector and to the Google feed; each channel's rejections return as tasks on the product, with the channel's rule quoted. A completeness score per channel (how many mandatory attributes are filled) is visible

before launch, so a repeat of bol's 140 declines becomes a list to clear before launch, not a surprise in the next day's upload report.

Migration consumer by consumer. The PIM is loaded in July 2027 from the four copies, and the load itself is a reconciliation: about **1,900 variants** with conflicting attributes are listed and resolved before anything is published. The DE storefront becomes the first consumer in August, the NL and BE storefronts and the connector follow in September, and the spreadsheets go read-only (Moving Without Burning the Bridge: Replatforming and Migration treats this as the small migration it is). The Spring/Summer 2028 season is the first built in the PIM from the start. The PIM is also the first prerequisite of **W8**, agent readiness: an AI shopping agent needs product data that is accurate, complete and the same in every channel, and the PIM is where that becomes checkable.

The numbers that matter are small. The season import drops from two agency days to a scheduled publication. The DE storefront launches with no untranslated variants because the completeness score blocks a product that is not translated. And one spelling of `moss` reaches four channels.

When It Goes Wrong

Product data fails quietly. The failures below are the ones to detect on purpose.

Silent rejection. A channel declines a product in an asynchronous report nobody reads, and the product is simply not for sale there. *Detection:* a listing status per variant per channel (sent, accepted, rejected with reason, live, offline) reconciled against what the channel reports; a daily count of variants that should be live but are not. *Recovery:* the rejection becomes a task on the owner's queue.

Identifier drift. A SKU reused for a new item, a GTIN with a wrong check digit, a variant deleted and recreated with a new id. The warehouse counts two items as one; a marketplace shows the wrong product; orders lose their link. *Detection:* a weekly cross-reference check (SKU to GTIN to platform id to connector id) across the ERP, the PIM, the platform and the connector. *Recovery:* retire the identifier, create a new one, map the old one for history; never edit in place.

The split product. Variants modeled as separate products to get around a limit, or because a color launched late. Search shows the jacket four times, `item_group_id` is wrong in feeds, analytics count four products. *Detection:* products that share a name and a brand but not a group id. *Recovery:* a merge that keeps the old URLs and ids mapped, which is a migration, not an edit.

Stale translation. A fixed Dutch description republished while the German one still carries the old claim. *Detection:* completeness per locale as a blocking state before publication. *Recovery:* publish the whole product again, in all locales.

The pulled rug. A marketplace changes its data model (bol's is updated daily), adds a mandatory attribute or retires an API version on a published date. Kestrel & Pine's

Deprecated Endpoint of February 2027 was stock, not content, but the mechanism is the same: the agency script that pushed stock to the connector called a retired API version, and bol's figures were stale for two days. *Detection:* the channel's changelog and deprecation notices read by a named owner, and a weekly listing status check that would show listings going offline. *Recovery:* the mapping is data, so a new mandatory field is a mapping change, not a code change; Promises Between Systems: API Contracts, Webhooks and Versioning owns the versioning side.

The common thread is the one Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation develops for every copy in the landscape: a copy is acceptable only with a reconciliation that compares it to its system of record on a schedule and lists the differences for a person.

What to Take Away

Established.

- A product is a product, its variants, their SKUs and their GTINs, and the identifiers are permanent. Shopify's product model, commercetools' "treat the sku values as immutable identifiers", schema.org's `ProductGroup` and GS1's GTIN rules all say so.
- Channels validate product data against their own rules and reject asynchronously: Google disapproves a wrong GTIN and requires a stable id, bol declines content without an EAN and takes products offline for missing markings. Syndication must expect and route the "no".
- Products with sales history are never deleted; "discontinued" is a lifecycle state.

Depends on context.

- The platform is the right catalog at the Starting stage and often at the Growing stage; it stops being right when there is more than one producing team and more than one consuming channel. That boundary, not revenue, is the test for a PIM.
- The ERP should own item identity and cost. Whether it also owns logistic attributes depends on whether it can hold them and whether the WMS already reads from it; at Kestrel & Pine it can and it does.
- Mappings to channel taxonomies live in the PIM when there is one and in the feed tool or connector when there is not; what must not change with the choice is that the mapping is data with an owner.
- Variant limits are a platform fact that changes (Shopify 2,048 since October 2025; commercetools 100 or 10,000 depending on model since September 2026); whether a limit matters depends on the catalog's shape, and apparel and footwear catalogs are the ones that hit it.

The author's synthesis.

- Decide the ownership table before choosing a PIM, because the PIM only works if each field has one owner and one direction; a PIM bought without the table becomes a fifth copy with a login.
- Treat syndication as an integration with an outbox, a validation step and a rejection loop, not as a feed; the "no" that arrives a day later is the message most worth routing.

Questions to Ask

1. For the product the shopper sees on your storefront, how many systems hold a copy of its description, and which one is right when they disagree?
2. Can anyone edit a product's name or description in two places? Which fields in the platform are read-only, and which should be?
3. When a SKU is typed wrong, is it corrected in place or retired and reissued? Has a SKU or a GTIN ever been reused?
4. Which attributes does a jacket (or your equivalent) carry as fields, and which live inside description text? Can the storefront filter on every attribute a shopper would filter on?
5. When a marketplace declines a product, who finds out, how long after, and where does the task land?
6. How long does a season launch take from the first item in the ERP to live listings on every channel, and which step is manual?
7. Which channel's rules changed in the last twelve months, and how did you learn of it?
8. If an AI shopping agent read your product data tomorrow, would it find the same jacket, in the same color, at the same completeness, in every channel?

With the product described once and owned per field, the next promise is the one the shopper reads under the title. A price is also product-related data, but it changes at a different cadence, it carries legal weight in Europe, and it is copied into more systems than any other fact. What Does It Cost? Price Lists, Contract Prices, Currencies and Tax takes it up with Kestrel & Pine's Price Drift of October 2026.

Sources and Evidence

Company, product and investor sources

- New product model (GraphQL Admin API), Shopify developer documentation, n.d., accessed 2026-11 — the product, option and variant model and the `productSet` mutation for an external source of truth.

- The product variant limit is now 2,048 for all merchants, Shopify changelog, 2025-10-15 — the variant limit and the warning to apps.
- 2,048 variants, Shopify blog, 2025-10-15 — why the REST API could not carry high-variant products.
- Variants and Markets: languages, Shopify Help Center, n.d., accessed 2026-11 — variants in merchant terms; translations and per-language URLs.
- Standard Product Taxonomy, Shopify, versioned 2026-10, accessed 2026-11 — an open-source taxonomy with categories, attributes and values.
- Products, commercetools documentation, n.d., accessed 2026-11 — immutable SKUs, staged and published data, deletion only when unpublished.
- Introduced Modular Catalog in public beta, 2026-06-26, and Modular Catalog is now generally available, 2026-09-29, commercetools release notes — the two catalog models and the 100 and 10,000 variant limits.
- Create a product, Adobe Experience League, updated 2026-10-08 — simple, configurable, grouped, bundle, virtual, downloadable and gift-card product types.
- What is a PIM, Akeneo, n.d., accessed 2026-11 — a vendor's definition of PIM, enrichment and syndication.
- pimcore/pimcore, Pimcore, version 2026.1, accessed 2026-11 — an open-core PIM and master-data platform.
- Synchronize items and inventory, Microsoft Learn, 2026-10-16 — "decide which system is the system of record for product data"; mapping by SKU or barcode.
- Product Content API v10, bol, n.d., accessed 2026-11 — the EAN requirement, the 4,000+ "chunks", mandatory enrichment levels, asynchronous upload reports and the daily data-model update.

Standards, regulation and official documentation

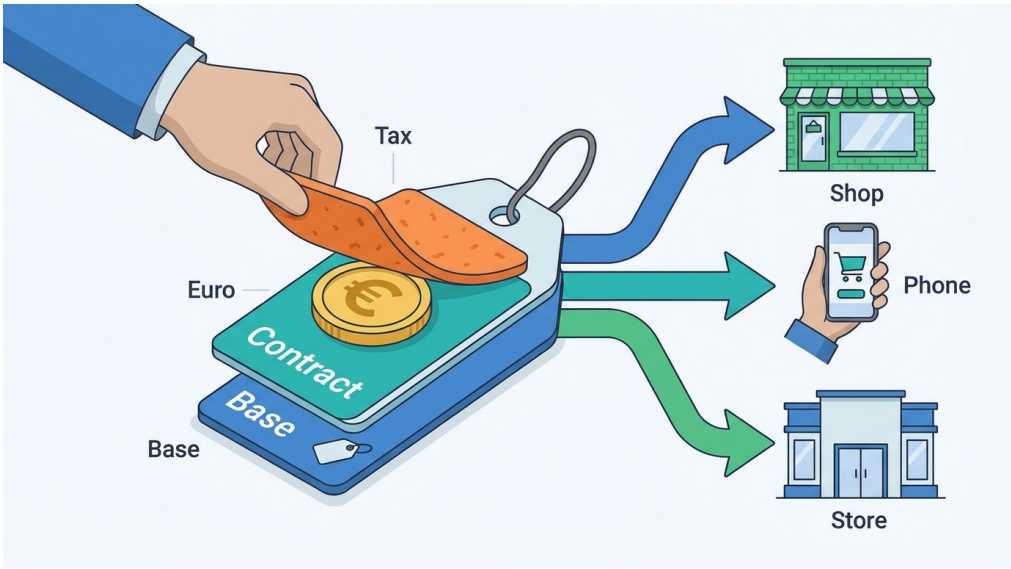
- Product, schema.org, version 30.1, 2026-09-16 — `sku`, `gtin`, `ProductGroup`, `isVariantOf` and inheritance of features.
- What is a GS1 GTIN, GS1 support, updated 2024-08-30 — the GTIN as unique, global and verifiable, issued only by a GS1 member organization.
- Global Trade Item Number, Wikipedia, accessed 2026-11 — the GTIN-8, -12, -13 and -14 formats and the rule that each trade item and packaging configuration gets its own GTIN (secondary source; GS1's own site blocked fetching).
- Product data specification and google_product_category, Google Merchant Center help, n.d., accessed 2026-11 — required fields, the GTIN disapproval rule, `item_group_id`, availability values and the image minimum from 2027-01-31.
- Google product taxonomy, Google, version 2021-09-21 — the 5,595-category tree still in use.
- Merchant listing structured data, Google Search Central, n.d., accessed 2026-11 — variant URLs and `priceValidUntil`.

Research and practitioner sources

- None beyond the documentation above; no independent study of when a PIM pays off was found, and the threshold given in this chapter is the author's judgement.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

7. What Does It Cost? Price Lists, Contract Prices, Currencies and Tax



IN THIS CHAPTER, YOU WILL: Learn what a price is made of, where it should be selected, how it reaches every channel, and what European price rules require of its history.

KEY POINTS:

- **A price is a promise with a selection rule.** One SKU (one sellable variant) carries many prices (market, currency, customer group, channel, quantity, validity). What makes them one price is a deterministic, documented rule that picks one at a defined moment, and that moment must be the same for the storefront, the checkout and the invoice.
- **Separate the engine that selects a price from the copies that show it.** Channels that cannot call the engine (marketplaces, the point of sale, EDI orders from wholesale customers) receive copies, and every copy needs a source, an effective date and a reconciliation. Kestrel & Pine's Price Drift was a copy whose restore failed silently.

- **In the EU, price history is a legal record.** Consumer prices must be shown including tax, and any announced reduction must name the lowest price of the previous 30 days, so the pricing system must answer "what did we charge, where, when" for every SKU.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Price Nobody Can Explain** — the same product carries different prices in different channels and nobody can say which system set which price or why.
- **The Message in a Bottle** — a price change is sent to a channel once and trusted; the channel that missed it keeps selling at the old price.
- **The Secret Agent** — a repricing rule in a connector, an app or a spreadsheet import changes prices without a named owner, a floor or a person's approval.

The price is the promise the shopper remembers. Ask what the Ridgewalker jacket costs at Kestrel & Pine, the fictional outdoor brand this book follows, and the honest answer is a table: **€199** to a Dutch, Belgian or German consumer, including that country's VAT; **€95** excluding VAT to an outdoor shop; a lower negotiated price to its largest wholesale customer; €199 on bol, Amazon.de and Zalando less the marketplace's commission; €199 at the till in Antwerp; and, for one week in November, a Black Friday price that must be announced against the lowest price of the previous 30 days.

These are not one price with exceptions. They are one product with a pricing rule, whose inputs are who is buying, where, through which channel, how many, on which date. Most merchants never write the rule down; they type prices into each system that needs them, and each copy is right only on the day it is typed.

Where we are in the order's journey. Eva's jacket (the recurring order of Meet Kestrel & Pine) meets this chapter on the product page, where she reads €199, and again at checkout on 13 April 2027, where the platform selects the price it will charge and freezes it on order #1072455. The invoice, the ledger (Tax, Invoices and the Books: From Order to Ledger) and the refund of her exchange (The Way Back: Returns, Exchanges, Refunds and Reverse Logistics) all reuse that frozen price. What Are We Selling? Products, Variants and the Catalog described the jacket; this chapter prices it; Discounts, Coupons and Loyalty: Promotions Without Chaos discounts it.

Setting the Scene at Kestrel & Pine: The Price Drift

In October 2026 the German storefront sold the Ridgewalker at **€179** for **nine days** while the marketplace listings and the stores charged **€199**. A price-list sync had failed silently. **410 orders** went out at the low price, about **€8,200** of margin was lost, and one

marketplace sent a price-parity warning. Meet Kestrel & Pine records the incident; what follows is this chapter's reconstruction of the mechanism, with illustrative details.

The German autumn campaign ran from 25 September to 4 October at €179, ten percent off. Three systems had to know. In 2026 the German consumer price list lived in the ERP (the enterprise resource planning system that runs finance, purchasing and invoicing), and the connector the ERP partner had built pushed it to the platform's German market price list every hour; the campaign was entered there with a validity window ending 4 October. The marketplace connector had its own campaign calendar, typed in by Milan Petrov's team, with the same dates. The stores never ran the campaign at all; the point of sale (POS) gets a weekly price file, and Germany has no store.

On 5 October the hourly push that should have restored €199 failed on an error, logged it and reported the run as successful. The connector pushed only changes since its last run, so the next hour saw nothing to send. The marketplace connector restored €199 on schedule from its own calendar. From that morning, the German storefront was the only channel at €179. Nobody noticed, because nothing compared the channels. On 14 October a marketplace notified Milan that the listing was priced above the same product on Kestrel & Pine's own site, and Sanne de Wit's weekly margin report showed German online margin down. By then 410 orders had each given away €20.

Figure 1: *The Price Drift: one restore failed silently, so for nine days one copy kept the campaign price while every other channel had moved back to €199.*

Noor Visser's April 2027 inventory of where the Ridgewalker's price lived made the pattern visible. The platform held three market price lists (NL, BE, DE) and four B2B price lists (wholesale NL, BE, DE and one for Bergzicht Outdoor). The ERP held the

wholesale and contract prices again, for invoicing, and the consumer lists that fed the DE storefront and the POS file. The marketplace connector held a channel price per marketplace. The campaign spreadsheet held the dates. **One jacket, eleven prices, seven places, no rule saying which was the source of which.** That is the Price Nobody Can Explain.

What a Price Is Made Of

A newcomer's first surprise is that a price is not a number on a product. It is a number selected from many by a rule. The dimensions below are the ones the major platforms document; the vocabulary in bold is the chapter's.

Dimension	Plain meaning	Kestrel & Pine example
SKU	The variant being priced	Ridgewalker women's, M, moss
Market (country, region)	Where the buyer is	NL, BE, DE
Currency	What the price is denominated in	Euro throughout
Customer group	Which kind of buyer	Consumer; wholesale; one named wholesale account
Channel	Through which sales path	Storefront, store, bol, Amazon.de, Zalando, B2B portal, EDI
Quantity tier	How many are bought	Wholesale price breaks per carton
Validity window	From when until when	Autumn campaign 25 September to 4 October 2026
Tax basis	Whether the number includes tax, and which tax	€199 includes 21% Dutch VAT; €95 excludes VAT

A **price list** (Shopify calls it a catalog, SAP Commerce a set of price rows, commercetools a set of standalone prices) is a named set of prices for a group of customers or a market. A **base price** is the list price for the general public. A **customer-specific price** applies to a customer group, such as wholesale. A **contract price** is agreed with one named account and outranks the list; Bergzicht Outdoor, Kestrel & Pine's largest wholesale customer, buys on a contract price list negotiated by Lea Moreau each season. A **tiered**

price (volume pricing) changes with quantity. A **quote** is an offer at a stated price with a validity date that the buyer can accept into an order; it is the one place a price becomes a promise with an expiry written on it (Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals owns quotes).

The selection rule

With these dimensions, several prices can match one purchase, so the platform needs a **price selection rule**: the order in which it tries the dimensions and what it does when more than one price still matches. commercetools documents its rule in full: a price is defined per SKU and per scope (currency, country, customer group, channel), selection walks an eight-step fallback from the most specific combination to the least, time-bound prices beat open-ended ones, and when several still match "the cheapest is selected" (commercetools pricing overview, n.d., accessed November 2026). Adobe Commerce scopes quantity tiers per website and customer group (Adobe Experience League, n.d., accessed November 2026), and SAP Commerce's long-lived Europe1 model matches price rows by product, customer group, quantity, currency and date range (SAP Javadoc for release 1811, 2018; eight years old, but the model persists).

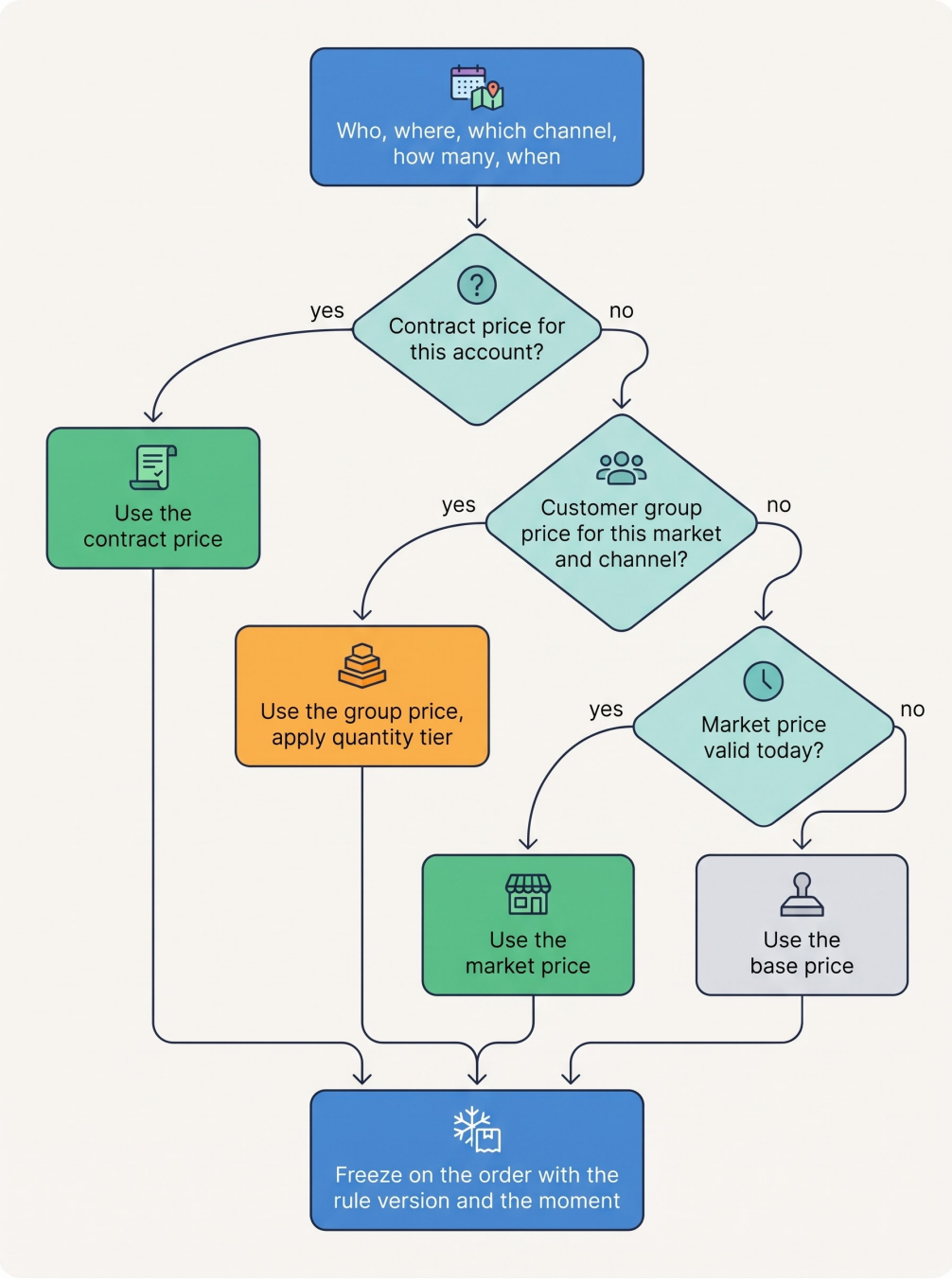


Figure 2: *A price is selected, not looked up: the rule tries the most specific price first, falls back to the base price, and freezes the result on the order together with when and by which rule it was chosen.*

The rule must be **deterministic** (the same inputs give the same price in every system

that selects) and **written down** where the people who type prices can read it, because "cheapest wins" and "most specific wins" give different answers when a campaign price and a contract price overlap.

The moment of selection

A price is also selected at a **moment**, and the moment is part of the promise. A storefront shows a price when the page loads; a cart holds it; checkout selects the price it will charge; the order freezes it; the invoice copies it. If any two of these select at different moments with different rules, the shopper sees one price and pays another.

SAP's support knowledge base records the generic bug: a campaign price that "does NOT expire after the validity end date has passed", because the validity is compared with the cart creation date, not the checkout date (SAP KBA 2267117, n.d., accessed November 2026). Adobe Commerce documents the opposite rule for purchase orders, keeping the original price if it changes after the order is created (Adobe Experience League, n.d., accessed November 2026): correct for a negotiated order, wrong for a cart.

The rule I use: display prices are selected at page load and may change; the price that counts is selected once, at order placement, from the same rule and the same data the storefront used, and is frozen on the order with the rule version and the timestamp. Quotes and purchase orders freeze earlier, for a stated validity. The invoice never selects; it copies.

Tax, Markets and Currencies: Why €199 Is Three Different Prices

In Europe a consumer price includes tax by law. Directive 98/6/EC of 16 February 1998 (published 18 March 1998) defines the selling price as "the final price for a unit of the product … including VAT and all other taxes" and requires that it "shall be indicated for all products" and be "unambiguous, easily identifiable and clearly legible". In the United States there is no federal equivalent, and sales tax depends on the delivery address, so it is added at checkout. Google's Merchant Center specification states the split operationally: for the US and Canada, prices "shouldn't include tax"; for all other countries, "include value added tax (VAT) or Goods and Services Tax (GST) in the price" (Google Merchant Center help, n.d., accessed November 2026).

Tax-inclusive means the shown price is what the consumer pays; **tax-exclusive** means tax is added later. The design consequence is that a stored price must say which it is. Kestrel & Pine sells the Ridgewalker at **€199 including VAT in all three countries**, but VAT differs: 21% in the Netherlands and Belgium, 19% in Germany (standard rates as of November 2026). So the net revenue per jacket is €164.46 in Nijmegen and €167.23 in Munich, and the margin differs by country at the same shelf price. The alternative, one

net price with the gross derived, would give the shopper €199.00 in Arnhem and €195.71 in Munich, and would break the one-price-everywhere story that marketplaces compare against. Either choice is defensible; a price stored without its tax basis is not, because then every consumer of the number guesses.

Which country's VAT applies is a tax question that Tax, Invoices and the Books: From Order to Ledger owns. The one fact a pricing design must carry is that since 1 July 2021 the EU's One Stop Shop (OSS) lets a merchant report the VAT of every EU country of delivery in one return at home, once its cross-border consumer sales pass €10,000 a year (European Commission OSS page, n.d., accessed November 2026). Kestrel & Pine passed that threshold years ago, so a €199 sale to Antwerp carries Belgian VAT inside the same €199. A wholesale sale to the same city with a valid VAT number carries none; the €95 is net and the invoice says why.

Currencies add a second axis. Kestrel & Pine prices in euros everywhere; a merchant selling in several currencies must choose between **converted prices** (base price times today's rate, rounded) and **fixed prices per market**; Shopify, for example, converts at the current exchange rate by default and rounds automatically, with fixed per-market prices as the alternative (Shopify Help Center, n.d., accessed November 2026). Converted prices move with the rate, which suits a storefront and not a marketplace that compares yesterday's price with today's. **Rounding is not cosmetic**: cents are what marketplace comparison and the 30-day reference rule see.

The Price Rules Europe Writes Into the System

Two European rules turn pricing from a merchandising choice into a record-keeping duty. Their dates matter, because "we did not know" is the Long Fuse.

The prior-price rule. Directive (EU) 2019/2161 of 27 November 2019, known as the Omnibus directive, inserted Article 6a into the 1998 price directive: "any announcement of a price reduction shall indicate the prior price", meaning "the lowest price applied by the trader during a period of time not shorter than 30 days prior to the application of the price reduction". It applies from **28 May 2022**; for widespread cross-border infringements the maximum fine must be at least 4% of annual turnover, or €2 million where turnover is unknown. Germany transposed it in § 11 of its 2022 price indication ordinance (PAngV). The Netherlands' consumer authority ACM supervised the rule from **1 January 2023**, with a worked example (6 December 2022): a price raised from €50 to €70 and cut by 35% after 25 days is really a 9% discount on the €50 reference. On 11 June 2024 ACM published fines against five web shops, among them Leen Bakker, JYSK and the Dutch Tommy Hilfiger store, because "consumers were therefore shown a fake discount" (ACM, 2024, translated).

The architectural reading is simple and demanding. **The pricing system must keep an effective-dated price history per SKU, per market and per channel, and must**

answer "lowest price in the last 30 days" at the moment a reduction is published. The reference price is itself a syndicated field, because marketplaces and feeds show the strike-through price too. A merchant whose prices live in seven places cannot answer the question reliably, which is the strongest legal argument for one source per kind of price in Europe.

Marketplace price comparison. Marketplaces compare a merchant's listing against its own web shop and against other sellers, continuously, and the legal boundary keeps moving. In November 2013 Amazon deleted its **price parity** clauses, which had forbidden sellers to sell cheaper elsewhere, after a German competition case (Bundeskartellamt, 2013; historical). On 5 February 2026 the Bundeskartellamt prohibited the opposite: Amazon's **price control mechanisms**, which removed or hid offers it judged too expensive; its case overview does not say whether the decision is final (accessed November 2026). Amazon's selling-partner API shows the mechanism from the seller's side: a `PRICING_HEALTH` notification says an offer is "ineligible for the Featured Offer (Buy Box) because its price is uncompetitive", the Buy Box being the default "add to cart" offer (Amazon SP-API documentation, n.d., accessed November 2026).

For the architecture, **channel price rules must be configuration, not code**, because the rules that bound them change on regulators' dates, and marketplace price-state events (Buy Box lost, offer suppressed, parity warning) are operational signals the pricing system should receive, not e-mails to the marketplace manager.

How the Pieces Fit Together

Four components make a pricing architecture, even when two of them live inside one product.

Component	What it is	What it owns	Example at Kestrel & Pine (2027 design)
Price data (price lists)	The numbers, with their dimensions and validity windows	The prices themselves, per kind	Market price lists in the platform; wholesale and contract lists in the ERP

Component	What it is	What it owns	Example at Kestrel & Pine (2027 design)
Pricing engine	The code that applies the selection rule to a request	The rule, its version, the moment	The platform's checkout and B2B portal; the ERP's order entry for manual and EDI orders
Price history	Every price, per SKU, market and channel, with effective dates	The 30-day reference, audit of who changed what	Kept with the price data; queried when a campaign is published
Channel price rules	Derived prices for channels that cannot call the engine	Marketplace price = base price plus uplift, within a floor and a ceiling	Configured in the connector, with a floor per product group set from the ERP's landed costs
Copies	What each channel actually shows	Nothing; they are promises made on the source's behalf	The POS price file, the marketplace listings, the EDI order confirmation

The distinction that organizes everything is **the engine versus the copies**. A channel that can call the engine at the moment of sale (the storefront, the B2B portal) needs no copy. A channel that cannot (the POS, which must work offline; a marketplace, which holds its own listing; an EDI partner, which sends its own price in the order) receives a copy, and **every copy is a promise that can drift**. The design question per channel is whether it can call or must receive, and if it must receive, from which source, with what effective date, and who reconciles it. At Kestrel & Pine the copies travel through the iPaaS (integration platform as a service, the hosted tool that runs its integrations).

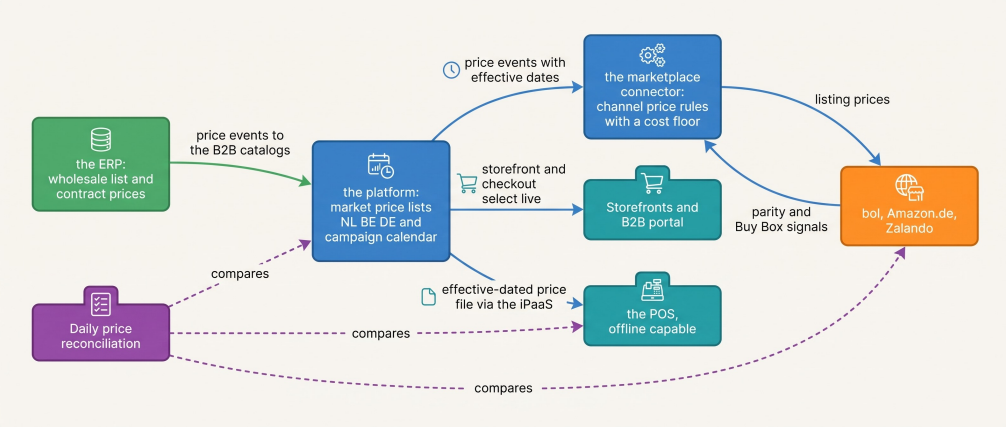


Figure 3: *Two sources, one per kind of price: the platform owns consumer prices and the campaign calendar, the ERP owns wholesale and contract prices; every other system receives effective-dated price events and is compared back to its source every day.*

Requirements per flow

Flow	How fresh	How consistent	How fast	When it fails
Storefront display	Seconds to minutes after a change	Must match what checkout will select	Page load	A campaign starts at midnight on the page and at 00:15 in checkout
Checkout selection	Live	Authoritative; frozen on the order	Under the checkout's clock, with tax and availability	A cart keeps a price a campaign ended
Marketplace listings	Minutes after a change *(assumption)*	Within channel rules and the floor	Asynchronous feeds, acknowledged	The Price Drift
Point of sale	Daily file, effective-dated	Must work offline with the last valid file	Overnight	A store sells yesterday's price for a day, by design

Flow	How fresh	How consistent	How fast	When it fails
EDI orders	The partner's price arrives in ORDERS (the EDI order message)	Must match the contract list or be flagged	Per document	A mismatch discovered on the INVOIC (the EDI invoice), as a deduction
Invoice	Copies the order	Immutable	Batch	An invoice recomputes and differs from the order

Two rows carry the design. Marketplace copies are asynchronous and must be **acknowledged**: the channel confirms the price it now holds, and that confirmation, not the sending, closes the loop. The POS copy is deliberately a day old, acceptable because it is effective-dated and reconciled, and because a till that cannot sell offline is worse than one that sells at yesterday's price.

Who may change a price

A **price change** should be an event with the SKU, the scope, the old and new values, the effective date, the author and the reason, published once by the source and consumed by every copy. Three kinds of author need rules.

People: merchandising sets consumer and campaign prices; wholesale sets contract prices; finance approves anything below a cost floor. Their rights live in the source system; the copies are read-only. Imports: a spreadsheet import is a writer with no judgement, so it needs validation (no price below the floor, no gap in validity, no restore row missing) and a report that a person reads. Automations: a connector's repricer, an app that adjusts prices or an AI agent proposing changes is the **Secret Agent** unless it has its own identity, a floor and a ceiling, a log and an owner. Channel integration platforms sell repricing with "margin guardrails" (ChannelEngine, n.d., accessed November 2026), and the guardrails are the point: a repricer runs on the channel price, bounded by a floor computed from cost and fees, and never writes back to the base price. Assistants and Agents: AI in Commerce and What Is Real in 2026 extends the rule to agents.

The Choices

Four arrangements cover what merchants actually do.

1. **The platform is the price source**. Prices are typed into the commerce platform's market and customer-group price lists; the ERP receives the price on each order; marketplaces and the POS receive copies from the platform. Right for a merchant whose prices are consumer prices and whose wholesale is small. Its limit is wholesale: contract prices, quantity breaks and the invoice's need for a price per account live naturally in the ERP, and a platform-only source forces them into a shape built for consumers.

2. **The ERP is the price source**. All price lists live in the ERP; the platform, the POS and the connector receive them. Common in ERP-first businesses, and the natural home of wholesale and contract prices. Its limit is the consumer side: campaigns with hourly start times, market-specific prices and promotion interplay need a cadence and a vocabulary ERPs do not have, and the copy to the platform becomes the slowest link at exactly the moments (a launch, Black Friday) when it matters most.

3. **A pricing service beside the platform**. A dedicated engine, bought or built, holds all price lists and the rule, exposes an API every channel calls, and publishes events to channels that cannot call. The cleanest answer and the most expensive: it adds a system, a call in the checkout path and an owner. Right when prices are genuinely complex (many markets and currencies, many customer groups, dynamic rules) or when many storefronts and apps must agree. The Giant's Boots for a merchant with three markets and one currency.

4. **Split by kind of price, with events**. Consumer prices and the campaign calendar in the platform; wholesale and contract prices in the ERP; each the system of record for its kind; everything else a consumer fed by effective-dated price events and compared daily. Two sources, but one rule and one history per kind. This is what Kestrel & Pine chose in 2027.

Per-channel pricing, where each channel owns its price, is what merchants drift into rather than choose; as a design it survives only as derived channel prices under arrangement 4.

Trade-offs

	Platform as source	ERP as source	Pricing service	Split by kind, with events
Right when	Consumer-led, wholesale small, one or two markets	Wholesale-led, few campaigns, ERP-first	Many markets, currencies, groups; many storefronts	Mixed consumer and wholesale; a hosted (SaaS) platform; Omnichannel stage
Consumer campaigns	Native: scheduled prices, markets	Slow; cadence is batch	Native	Native, in the platform
Contract and tiered prices	Possible (B2B catalogs), awkward at scale	Native	Native	Native, in the ERP
Price history for the 30-day rule	If the platform keeps it; often only current prices	Usually kept, per list	Designed in	Must be kept in both sources
Copies to reconcile	ERP, POS, connector	Platform, POS, connector	Only channels that cannot call	POS, connector, EDI
Cost	Included	Included	A system plus an owner *(assumption: the costliest option)*	Integration work and a reconciliation job
Warning sign of outgrowing it	Wholesale prices typed twice; invoices disagree with orders	Campaigns need a developer; the copy lags a launch	Rare; the risk is buying it early	The two sources start pricing the same thing (a consumer price in the ERP)

My recommendation, with its conditions, is arrangement 4 for a merchant like Kestrel & Pine, resting on three assumptions. The platform supports per-market and per-customer-group price lists with validity windows and keeps (or exports) a price history; without that, the 30-day rule has no data. The ERP invoices wholesale, so contract prices and invoices share a source. And every copy is fed by events with effective dates and compared daily, which is Waypoint's W5 applied to prices. Drop any of the three and arrangement 3 becomes the honest answer.

Across Scale Stages: When It Fits and When It Limits

The four stages are the ones this book uses throughout (Meet Kestrel & Pine; in depth, What Changes at Each Scale: From First Store to Group).

Stage	What is typical	What becomes necessary	Warning signs you have outgrown it
1 — Starting (under about €2M; Kestrel & Pine 2016–2018)	One price list in the platform; one country; tax-inclusive prices; campaigns typed by hand	A tax basis on every price; a written rule for what counts as a discount, because the 30-day rule applies from the first sale	A second country or a first wholesale customer priced in a spreadsheet
2 — Growing (about €2M–€20M; 2019–2022)	Market prices; the first wholesale list; a marketplace copy through a connector; the ERP arrives with its own lists	One source per kind of price; the first channel price rule (marketplace = base plus uplift); a reconciliation between platform and marketplace	The first marketplace shows how fast a copy drifts; the same price typed in the platform and the ERP; a campaign restore that someone has to remember

Stage	What is typical	What becomes necessary	Warning signs you have outgrown it
3 — Omnichannel (about €20M–€200M; 2023 onward)	Several markets, wholesale with contract prices, stores, three marketplaces; seven places that hold a price	Price events with effective dates; price history per SKU, market and channel; channel price rules as configuration with a cost floor; daily reconciliation; the moment of selection fixed at checkout	The Price Drift; a parity warning nobody routes; a Black Friday reference price computed by hand; the invoice differing from the order
4 — Group (over about €200M or several brands)	Several brands and currencies; per-country legal rules; group-wide marketplace accounts; a pricing team	A pricing service shared across brands with per-country rules as configuration; converted versus fixed prices per market as policy; price governance (who may change what, floors, approvals)	Two brands undercut each other on one marketplace; a currency move changes margins nobody sees; a regulator's rule implemented in three stacks

The judgement: **the tax basis and the 30-day history do not change with scale; the number of sources and copies does.** The source decision changes once, at the Growing stage, when the ERP arrives and the merchant must say which prices it owns; after that the work is in the copies.

In Practice

What the platforms document

Shopify's B2B feature prices by **catalog**, with percentage adjustments, fixed prices and volume pricing, and below the Plus plan allows "up to 3 active catalog assignments across active B2B markets" (Shopify Help Center, n.d., accessed November 2026; plan rules change). commercetools allows up to 100 embedded prices per variant or 50,000 standalone prices (commercetools documentation, n.d., accessed November 2026).

The pattern: **the number of prices grows multiplicatively** (SKUs times markets times groups times windows), so every platform moves prices out of the product into a separate store, and documents a selection rule because the growth makes one necessary.

Kestrel & Pine: the price of one jacket on 13 April 2027

In April 2027 Noor, Milan, Lea and Sanne wrote the price ownership decision on one page. The platform's market price lists are the system of record for consumer prices in NL, BE and DE and for the campaign calendar; consumer prices no longer pass through the ERP, whose role shrinks to finance (The Back Office: ERP Integration and Who Is the Hub). The ERP's price lists are the system of record for wholesale list prices and contract prices. The ERP feeds no channel directly, meaning there is no ERP-to-storefront link: it publishes those prices as events to the platform's B2B catalogs and, once the OMS (order management system) is live in Q4 2027, to the OMS. The POS, the marketplace connector and the B2B catalogs are consumers, fed by price events with effective dates through the iPaaS (W5); the connector's price rules carry a floor per product group set from the ERP's landed costs. A daily price reconciliation across all copies starts in May 2027, and the partner-built connector's price flow is retired.

On the day of Eva's order, the Ridgewalker women's, size M, moss, is priced like this:

Where	Price	Basis	Source	How it got there
NL storefront, checkout	€199	Incl. 21% VAT; net €164.46	Platform, NL market list	Selected live at 21:14, frozen on #1072455
DE storefront	€199	Incl. 19% VAT; net €167.23	Platform, DE market list	Selected live

Where	Price	Basis	Source	How it got there
bol, Amazon.de, Zalando	€199	Incl. VAT of the marketplace's country	Connector rule: the market price of the marketplace's country, uplift 0%, floor €95 *(illustrative, from the €58 landed cost plus fees)*	Price event, acknowledged by the connector
Six stores	€199	Incl. VAT of the store's country	Platform NL and BE market lists	Effective-dated daily file via the iPaaS
B2B portal, an independent shop	€95	Excl. VAT	ERP wholesale list NL	Price event to the platform's B2B catalog
Bergzicht Outdoor, by EDI	Contract price	Excl. VAT	ERP contract list	The ORDERS price is checked against the list on receipt
Eva's invoice	€199	Incl. 21% VAT	Copied from #1072455	Never recomputed

The last row makes Eva's exchange on 19 April work (Journey 2: The Jacket Comes Back): it reuses the frozen €199, whatever the list price is that day.

The Black Friday test. Suppose the NL autumn campaign of 2027 runs 20 October to 3 November at €179 (illustrative). A Black Friday reduction announced on 26 November must show the prior price as the lowest price of the previous 30 days, 27 October to 26 November, which includes a week at €179. The "was" price is **€179, not €199**, and a Black Friday price of €159 is an 11% reduction, not 20%. Before the 2027 design that answer required someone to remember the campaign; after it, the platform's price history answers it when the reduction is scheduled.

Figure 4: *The 30-day rule reads history, not memory: a campaign inside the window makes the was-price €179, not €199, so €159 is an 11% reduction, not 20%.*

What changed about the Price Drift. Under the 2027 design the German campaign is one row in the platform with a start and an end, so no hourly push has to carry the restore, and the daily reconciliation would have shown any channel still at €179 against the DE market list at €199 on the morning of 5 October. The margin at risk falls from nine days to one.

When It Goes Wrong

The silent restore. A campaign's end price never arrives, or arrives and is dropped. *Detection:* a daily reconciliation of every copy against its source; an expected-price check at campaign end; acknowledgement from each channel. *Recovery:* republish the current price as a new event; the copies are idempotent, so a repeat does no harm.

The repricer that undercuts the storefront. A connector rule chases a competitor's price below the merchant's own site, and the margin disappears. *Detection:* a floor from cost and fees; an alert when a channel price falls below the base price. *Recovery:* channel rules bounded by configuration, never writing to the base price.

Two moments of selection. The cart holds a campaign price that checkout no longer offers, or the reverse; the SAP cart-creation-date bug is the documented case. *Detection:* a test that opens a cart before a campaign boundary and checks out after it. *Recovery:* one rule, one moment, at order placement.

The reference price nobody kept. A reduction is announced against last week's list price when a cheaper campaign ran within 30 days. *Detection:* the pricing system

computes the reference from history when the reduction is scheduled and blocks a higher one. *Recovery:* correct the strike-through price in every channel, which is itself a syndicated field, and record why.

The EDI price that differs. Bergzicht's ORDERS carries the price it believes; the invoice carries the contract list's price; the difference becomes a deduction on payment. *Detection:* price validation on ORDERS receipt against the contract list; a mismatch is a task before shipment, not a discovery on the INVOIC (Documents Between Companies: EDI, Punchout and E-Invoicing).

Every one of these ends in the same component: a reconciliation that compares copies to their source on a schedule and gives a person a list. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation builds that component for every kind of copy in the landscape.

What to Take Away

Established.

- A price is selected by a rule from several candidates that differ by market, customer group, channel, quantity and validity; commercetools, Adobe Commerce and SAP Commerce each document such a rule, and every platform separates price data from the product as the count grows.
- In the EU, consumer prices must be shown including VAT and all other taxes (Directive 98/6/EC, 1998), and any announced reduction must name the lowest price of the previous 30 days (Directive 2019/2161, applied from 28 May 2022; enforced in the Netherlands from 1 January 2023 with fines published in June 2024).
- Marketplaces compare prices continuously and act on the result; Amazon dropped its price parity clauses in 2013 after a German competition case, its price-control mechanisms were prohibited in Germany in February 2026 (finality not stated as of November 2026), and the Buy Box is decided partly on price.

Depends on context.

- Whether the platform or the ERP is the source of a given kind of price depends on who prices it and who invoices it: consumer prices and campaigns belong with the storefront; wholesale and contract prices with the system that invoices wholesale. A merchant with no wholesale needs one source; a merchant with no consumer campaigns may need only the ERP.
- A dedicated pricing service is right for many markets, currencies and storefronts, and the Giant's Boots for a merchant with three euro markets.

- One gross price in every market (Kestrel & Pine's choice) simplifies marketplace comparison and costs margin where VAT is higher; one net price with derived gross does the reverse. The choice is a policy; the requirement is that the tax basis is stored with the price.

The author's synthesis.

- Treat every copy of a price as a promise made on the source's behalf: it gets a source, an effective date, an acknowledgement and a daily comparison, or it does not get made.
- Fix the moment of selection at order placement, freeze the price on the order with the rule version, and let the invoice copy; everything else about pricing can be reorganized later, but a price that differs between order and invoice cannot be explained to a customer or an auditor.

Questions to Ask

1. For one product, how many systems hold a price, and which one is the source for each kind of price (consumer, wholesale, contract, channel)?
2. Is the selection rule written down? When a campaign price and a contract price both apply, which wins, and does every system agree?
3. At which moment is the price that will be charged selected, and can a cart keep a price that checkout would no longer give?
4. Does every stored price say whether it includes tax, and which tax?
5. Can the system answer "the lowest price we charged for this SKU in this country in the last 30 days" at the moment a reduction is scheduled?
6. When a campaign ends, how does each channel learn, how do you know it learned, and what compares the channels the next morning?
7. Which automations change prices (repricers, apps, imports), who owns each, and what floor bounds them?
8. When a marketplace warns about a price, where does the warning land?

The price on the product page is only the start of what the shopper will pay. Coupons, bundles, free-shipping thresholds and loyalty points change it at checkout and must still respect the floor, the 30-day reference and the books. Discounts, Coupons and Loyalty: Promotions Without Chaos takes the frozen price from this chapter and shows what may be subtracted from it, in which order, and how the discount reaches the invoice.

Sources and Evidence

Company, product and investor sources

- B2B catalogs, Shopify Help Center, n.d., accessed 2026-11 — catalogs as price lists, volume pricing, the three-assignment limit below Plus.
- Markets: pricing and Rounding prices, Shopify Help Center, n.d., accessed 2026-11 — converted versus fixed market prices and automatic rounding.
- Pricing and discounts overview and Standalone Prices, commercetools documentation, n.d., accessed 2026-11 — price scopes, the eight-step selection fallback, "the cheapest is selected", 100 embedded and 50,000 standalone prices.
- Quotes, commercetools documentation, n.d., accessed 2026-11 — quotes as "legally binding custom offers".
- Tier pricing, Shared catalogs and Purchase order flow, Adobe Experience League, 2026 and n.d., accessed 2026-11 — tier precedence, one public shared catalog, "the original price is used".
- Europe1PriceFactory and PriceRowModel, SAP Commerce Javadoc, release 1811, 2018 (evidence age: eight years) — price-row matching and the dimensions of a price row.
- KBA 2267117: price does not expire after validity end, SAP Support, n.d., accessed 2026-11 — validity compared with the cart creation date.
- Notification type values, Amazon Selling Partner API documentation, n.d., accessed 2026-11 — `PRICING_HEALTH` and `ANY_OFFER_CHANGED`.
- ChannelEngine, ChannelEngine, n.d., accessed 2026-11 — repricing with "margin guardrails" as a connector feature.

Standards, regulation and official documentation

- Directive 98/6/EC on consumer protection in the indication of the prices of products, European Parliament and Council, 1998-02-16 (OJ 1998-03-18) — the selling price "including VAT and all other taxes".
- Directive (EU) 2019/2161, European Parliament and Council, 2019-11-27 (OJ 2019-12-18) — Article 6a, the 30-day prior price, application from 2022-05-28, minimum fines.
- § 11 PAngV, German price indication ordinance, 2022, accessed 2026-11 — the German transposition.
- ACM wil einde aan nepkortingen, ACM, 2022-12-06 — supervision from 2023-01-01 and the €50 to €70 example.
- ACM publiceert boetebesluiten webwinkels voor nepkortingen, ACM, 2024-06-11 — the five fines.
- One Stop Shop, European Commission, n.d., accessed 2026-11 — OSS from 2021-07-01 and the €10,000 threshold.

- Product data specification, Google Merchant Center help, n.d., accessed 2026-11 — tax-exclusive prices for the US and Canada, tax-inclusive elsewhere.
- Proceedings against Amazon terminated, Bundeskartellamt, 2013-11-26 (historical) — the end of price parity clauses in the EU.
- Proceedings against Amazon (overview), Bundeskartellamt, accessed 2026-11 — the 2026-02-05 prohibition of price control mechanisms and the initial €59 million disgorgement; the page does not state whether the decision is final.

Research and practitioner sources

- None beyond the documentation above; the reconstruction of the Price Drift's mechanism and the 2027 ownership decision are the author's illustration.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

8. Discounts, Coupons and Loyalty: Promotions Without Chaos

IN THIS CHAPTER, YOU WILL: Learn how promotions, coupons, gift cards and loyalty points are modeled, where the final discount must be computed, and how a discount reaches the invoice, the refund and the books.

KEY POINTS:

- **A promotion is a rule evaluated in one place, at cart time.** The product page shows an estimate; the cart engine computes the final discount from the rules, the codes and the customer's eligibility, in a declared order, with a rule that says when stacking stops. Every platform that takes promotions seriously has converged on this shape.
- **A discount is not finished until it is allocated to lines.** Returns, partial refunds, invoices and tax all work per line, so an order-level discount must be spread across the lines at order time. Skip that step and every return refunds too much.
- **Gift cards, store credit and points are money you owe, not marketing.** They are ledgers with balances that finance must reconcile to

a liability account every month; the rules for when unredeemed value becomes revenue are accounting standards, not product decisions.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Price Nobody Can Explain** — a cart whose total nobody can reconstruct from the rules is a price with no owner.
- **The Boomerang** — refunds computed from the undiscounted line price give the discount back twice.
- **The First Snowfall** — Black Friday is the first time all the promotion rules meet all the customers at once.

Every merchant discounts. A new-customer code in the newsletter, 20% off outerwear in November, free shipping over a threshold, a gift card for a birthday, points for coming back. Each looks like a marketing decision, and each is also a **change to the price that the rest of the platform must be able to explain**: to the customer who returns one item from a discounted order, to the accountant who posts the invoice, to the tax authority that wants the right VAT on the right net amount, and to the finance director who must know how much gift-card money the company holds for customers who have not spent it yet.

A promotion is a promise about a price, and promises interact. Two promotions that are each sensible alone can combine into a price below cost; a promotion that is right on the product page can be wrong in the cart, because the cart knows something the page did not (a second item, a code, a shipping address); and a promotion that was right at order time can be wrong at refund time, because the refund looked at the line, not at the order. This chapter shows how a platform keeps all three moments consistent.

This chapter owns the **promotion engine**, **stacking**, the **store credit liability** and **discount allocation**. It builds on What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, hands the cart to From Cart to Confirmation: Checkout Orchestration, the invoice to Tax, Invoices and the Books: From Order to Ledger, the refund to The Way Back: Returns, Exchanges, Refunds and Reverse Logistics and customer identity to One Customer, One Product: Identity and Master Data Across Systems.

Where we are in the order's journey. Eva's jacket, the recurring order this book follows, was bought at full price on 13 April 2027: €199 for the Ridgewalker and €18 for each pair of socks, €235 in all, with free shipping because the cart passed €75. That free shipping is the only promotion Eva used, and it is still a rule the cart evaluated. This chapter sits between the price (the previous chapter) and the cart (Part III): the step where the price the customer pays is finally decided, and the one that leaves a trace on every later step.

Setting the Scene at Kestrel & Pine: The Black Friday Stacking Bug

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), sells on a hosted SaaS commerce platform in three countries. For **Black Friday week 2026 (23–29 November)** Milan Petrov, the Ecommerce and Marketplaces Manager, configured three promotions:

- an **automatic 20% discount on all outerwear** (jackets, fleeces, trail trousers), applied to every cart without a code, valid for the week;
- the standing **welcome code**, 10% off a first order, sent to every new newsletter subscriber;
- the standing rule of **free shipping on orders over €75**.

The campaign plan said the welcome code would *not* combine with the outerwear discount: a first-time customer would get whichever was better. Nobody checked the platform's **discount combination setting**, which says which classes of discount may apply to the same order. The agency had switched it in October for a short Trailhead fleece campaign that *was* meant to combine with a code, and never switched it back. The week's figures are illustrative, invented for this chapter and consistent with the rest of the book.

What happened in the week of 23–29 November 2026	**Figure *(illustrative)***
Online direct-to-consumer (D2C) orders in the week (see Meet Kestrel & Pine)	31,000
Orders that received both the 20% outerwear discount and the 10% welcome code	2,860
Ridgewalker jacket price after both: €199 → €159.20 → €143.28	€15.92 extra discount per jacket
Margin given away beyond the plan (about €15 per affected order)	about €43,000
Orders whose discounted subtotal fell under €75 and were charged €4.95 shipping after the product page had promised it free	1,140
Shipping refunded by hand after complaints	about €5,600

What happened in the week of 23–29 November 2026	Figure *(illustrative)*
Returned Black Friday jackets refunded at €159.20 instead of €143.28, because the order-level 10% had not been allocated to the lines the returns portal reads	190 returns, about €3,000 over-refunded
Total cost	about €52,000

Three things went wrong, and they are the three things this chapter is about. The **stacking** was not what the business intended, because the rule that decides it lived in a setting, not in the campaign. The **free-shipping threshold** was evaluated on the discounted subtotal in the cart while the product page promised it on the list price. And the **order-level discount was never allocated to the lines**, so the returns portal, which reads line prices, refunded the discount back. Noor Visser, Head of Engineering since January 2027, put the incident next to the Price Drift of October 2026 and drew the same conclusion: **a price that nobody can reconstruct from the rules is a price with no owner**.

Figure 1: *The stacking bug: a combination setting left on from October let the welcome code stack on the outerwear discount, taking the jacket from €199 to €143.28.*

What a Promotion Is, and What It Touches

In plain words, a **promotion** is a rule that lowers the price a customer pays under stated conditions. A **coupon code** is a promotion the customer unlocks by typing a string; an **automatic discount** applies when the conditions are met with no code. **Eligibility** is the set of conditions: which products, customers, period, channel, minimum cart. **Stacking** is applying more than one promotion to the same cart or line. **Discount allocation** is spreading an order-level discount over the lines so that each line, and later each invoice line and return, carries its share.

The table lists the shapes every platform supports and what each needs from the rest of the system.

Promotion type	Example	What it needs to compute	What it changes
Percentage off a product or set	20% off outerwear	The product's current price and category	Line price
Fixed amount off a product	€10 off the Trailhead fleece	The product's price	Line price
Percentage or fixed amount off the order	10% off a first order with a code	The subtotal after product discounts; the customer's order history	Order total, to be allocated to lines
Buy X, get Y	Three pairs of socks for the price of two	Quantities per line; which unit is the free one	One line's price
Tiered	10% over €150, 15% over €250	The subtotal	Order total, allocated
Free or reduced shipping	Free shipping over €75	The subtotal and the shipping method	Shipping line
Bundle	Jacket plus fleece for €279	The presence of both items	Two lines, with a fixed total allocated between them

Every row touches three things the business cares about: the **price** the customer sees and pays, which in the EU must obey the prior-price rule described later; the **margin**, which disappears silently when promotions combine (the Ridgewalker at €143.28 still cleared its €58 landed cost, but gave up about a sixth of its planned Black Friday unit

margin); and the **books**, because the invoice must show the net price per line with the VAT on it, and a refund must give back what was paid for that line, not what was listed.

Two things that look like promotions are not. **Gift cards**, **store credit** and **loyalty points** are value the merchant has been paid for or has promised and not yet delivered: money it owes. They are ledgers, not rules, and get their own section.

How the Pieces Fit Together

A promotion system has five parts, whether it is a platform feature, an app or a service. The diagram shows them around the decision that matters: the final discount is computed once, in the cart, and written to the order with its allocation.

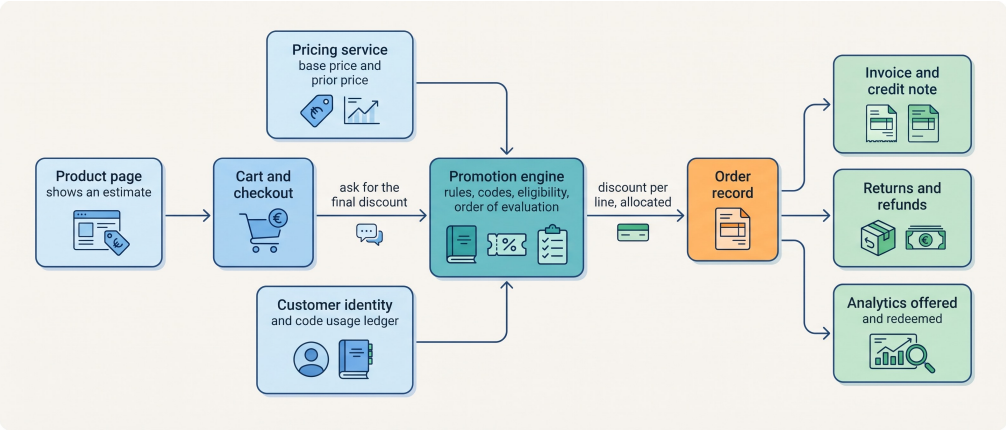


Figure 2: *The product page shows an estimate; the promotion engine computes the final discount once, at cart time, from the base price, the rules and the customer's eligibility, and writes it to the order allocated per line so that the invoice, the refund and the analytics all read the same number.*

The five parts

Promotion definitions. Each promotion is a record: its type, conditions, validity window, channel, budget or usage cap, and its position in the **order of evaluation**. The platforms that document this best make the order explicit. Shopify evaluates product discounts first, then order discounts on the revised subtotal, then shipping discounts; its help center (accessed November 2026) notes that two combined percentage *order* discounts are both calculated on the original subtotal, so 10% plus 20% on €100 gives €70, not €72. commercetools gives every cart discount a unique `sortOrder`, a stacking mode that can stop further discounts after a given one, and discount groups in which only the best saving applies. Adobe Commerce applies the highest-priority rule first and

has a "Discard subsequent rules" switch. One idea, three vendors: **promotions are data with a declared order and a stop rule**.

Eligibility and identity. "First order", "members only" and "once per customer" all need to know who the customer is. On Kestrel & Pine's platform the welcome code's "first order" check looked at the e-mail address, so a customer with two addresses was a first-time customer twice. Promotions are one of the first places a merchant discovers it has no single customer identity: the Familiar Stranger pattern, owned by One Customer, One Product: Identity and Master Data Across Systems.

The code ledger. Every code has a usage record: how many uses are allowed in total and per customer, and how many have happened. It must be strongly consistent at redemption: two checkouts redeeming the last use of a single-use code in the same second must not both succeed. It is the one part of a promotion engine that behaves like a bank account, and the reason bulk-generated single-use codes with usage caps (Adobe generates them in batches; Shopify allows 5 product or order codes and 1 shipping code per order) are the baseline defense against code sharing.

Evaluation in the cart. The engine takes the cart's lines, prices, shipping method and customer, applies the promotions in order, stops where the stop rule says, and returns a discount per line and per shipping line. Some conditions can only be evaluated late: Adobe's documentation notes that conditions on the shipping or payment method "are met only in checkout after the specific shipping/payment methods are selected". That is why the product page can only show an **estimate**, and why the estimate and the final must come from the same rules.

Allocation and events. The final step writes the discount to the order *per line*. commercetools makes the choice explicit: an absolute cart discount is distributed proportionately, evenly or individually across the lines. However it is distributed, each line then carries its net price, which the invoice and the refund read. The engine also emits events: promotion applied, code redeemed, promotion offered but not taken. The last is the one analytics needs most and platforms rarely record.

What each interaction needs

Promotions are evaluated at several moments with different requirements.

Interaction	Freshness	Consistency	Latency	Availability
Product page strike-through price	Start and end to the minute; prior price per the EU's 30-day rule	An estimate; may differ from the cart	Served from cache *(assumption)*	Must not block the page; show the list price if the engine is down

Interaction	Freshness	Consistency	Latency	Availability
Cart evaluation	Current rules and base prices	Final; the same result on every re-evaluation	Under about 300 ms, on every cart change *(assumption)*	Checkout must not fail because promotions fail; degrade to no discount with a message, or hold the checkout, by business decision
Code redemption	Current usage count	Strongly consistent; one redemption per allowed use	As the cart	As the cart
Gift card or store credit redemption	Current balance	Strongly consistent; idempotent per checkout attempt	As the cart	Decline the card if the balance service is down; never guess
Promotion publication	Scheduled start; tested against active promotions	All channels see the same rule set	Minutes	Stores and marketplaces receive a copy
Analytics	Daily	Offered and redeemed both recorded	Batch	—

The two rows that behave like money (code redemption and stored value) cannot be eventually consistent; everything else can be a cached copy with a known delay. It is the distinction Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record draws for every fact: decide which system holds the truth, and let the copies be copies.

How a discount reaches the invoice and the books

A discount leaves the order in three documents. The **invoice** shows each line at its net price after allocation, with VAT on that net amount; an order-level discount across lines with different VAT rates (a jacket at 21% and a book at 9%) must be allocated before the tax can be right, which makes allocation a tax problem as much as a promotions problem. The **credit note**, issued on a return, reverses the line at the same net price. The **ledger** records revenue net of the discount: a promotion is not an expense but revenue never earned. Tax, Invoices and the Books: From Order to Ledger walks the entries; RECONCILED's Payments-to-Ledger: Draining the Suspense Swamp goes deeper on reconciliation.

Here is the allocation on Eva's order, had she used a 10% code (she did not; the prices are her order's, the code is hypothetical):

Line	List price	Share of €235	Allocated discount	Net line price
Ridgewalker jacket, women's M, moss	€199.00	84.7%	€19.90	€179.10
Merino hiking socks × 2	€36.00	15.3%	€3.60	€32.40
Total	**€235.00**		**€23.50**	**€211.50**

When Eva returns the jacket, the refund is **€179.10**, not €199. The 190 over-refunded Black Friday returns were this calculation skipped: the returns portal read the line price (€159.20 after the product discount) because the order-level 10% had never been written to the lines, and gave back €15.92 per jacket the customer had never paid. That is the Boomerang in one table row.

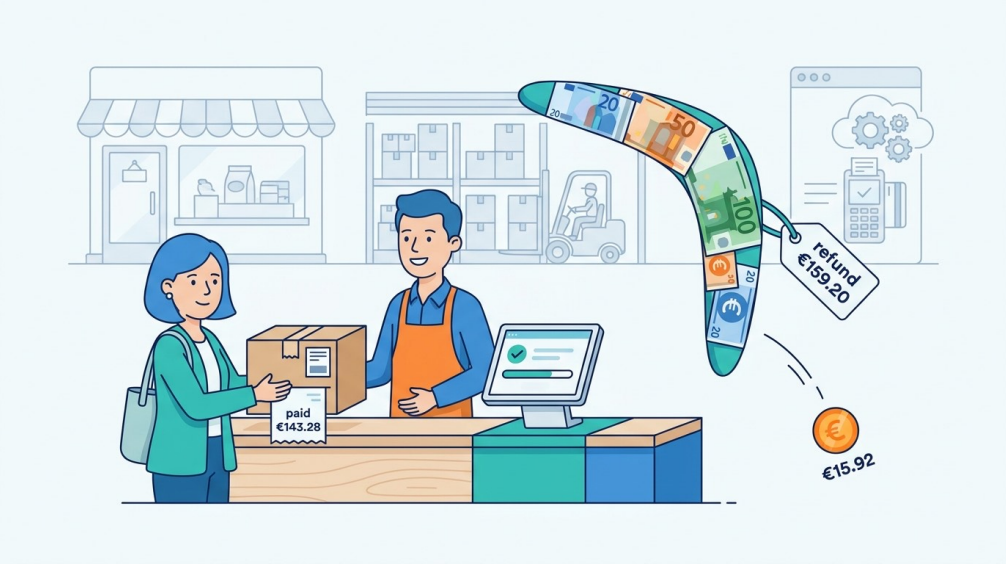


Figure 3: *The Boomerang: because the order-level 10% was never allocated to the lines, each Black Friday jacket return gave back €15.92 the customer had never paid.*

Gift Cards, Store Credit and Points: Ledgers, Not Promotions

A **gift card** is value the customer paid for in advance. **Store credit** is value the merchant granted, typically as a refund alternative or a goodwill gesture. **Loyalty points** are value the customer earned by buying. All three share a property that no promotion has: **they are a liability on the balance sheet**, money or value the merchant owes, until they are redeemed or expire.

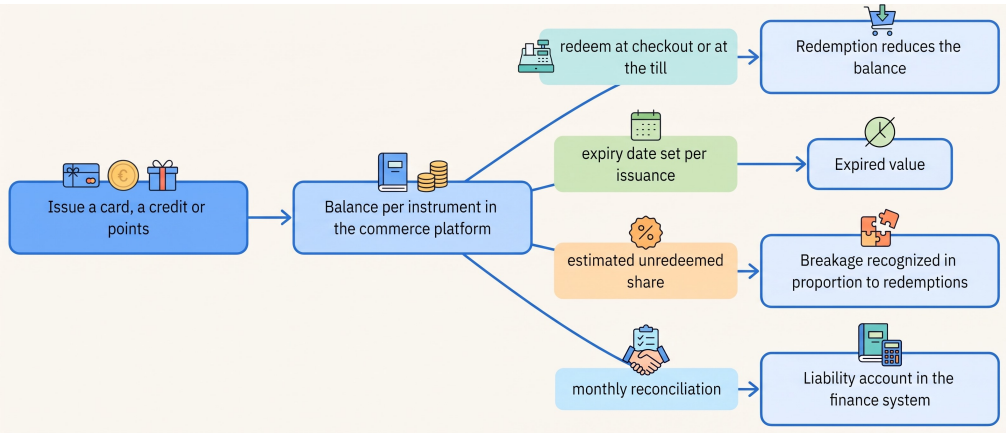


Figure 4: *Stored value is a ledger: every issuance, redemption, expiry and breakage estimate changes a balance per instrument in the commerce platform, and the sum of those balances must agree with the liability account in the finance system every month.*

The accounting is settled and drives the data model. Under IFRS 15, the international accounting standard for revenue (issued May 2014, effective from 1 January 2018, replacing IFRIC 13 on loyalty programs) and its US twin ASC 606, the sale of a gift card is a contract liability, not revenue; revenue is recognized on redemption. The part that will never be redeemed is **breakage**; the standard requires the merchant to *estimate* it and recognize it "in proportion to the pattern of exercised rights" rather than wait for cards to expire, and unclaimed value owed to a government is a liability, not revenue. Loyalty points that give the customer a **material right** (a benefit valuable enough to count as something they paid for) are a separate performance obligation, with revenue deferred until they are used.

Starbucks' FY2025 annual report (fiscal year ended 28 September 2025) shows what this means at scale: amounts loaded onto stored value cards are recorded as deferred revenue; the stored value card and loyalty liability stood at **$1,751.7 million**; breakage recognized in company-operated stores was **$200.4 million** in the year; Stars are deferred at their estimated selling price "as each Star is earned" and "generally expire after six to twelve months, depending on the market". Starbucks is a coffee chain, but the obligation has the same shape for a merchant with €200,000 outstanding.

Four design rules follow:

- **Every instrument is a ledger**. Issuance, each redemption, each expiry and each breakage estimate is an immutable record per card, credit or points account; a balance is derived, never edited (RECONCILED's Never Update, Only Append: Designing Custom Ledgers gives the pattern).
- **Expiry is an attribute of the issuance, not a global setting**, because rules differ by country and, in the US, by state. Shopify's store credit documentation (accessed November 2026) models it that way: "Store credit expiration is set per issuance".
- **Redemption is idempotent**: a repeated request has the same effect as one. The idempotency key (a unique id that lets a retried request be recognized and applied once) that protects the payment (Taking the Money: Payment Providers, Authorization, Capture and Refunds) must also cover the stored-value debit.
- **The platform holds the balance, finance holds the liability, and they are reconciled monthly**; a difference is a bug or a fraud.

Paid membership programs (Prime-style: a fee in exchange for delivery and other benefits) are a different thing again, often confused with loyalty. They are a subscription with its own revenue line and its own promises (free delivery, early access) that the promotion engine reads as eligibility conditions. Amazon's annual report for 2025 (filed February 2026) reports subscription services net sales of $49.6 billion and describes

Prime benefits as "effective worldwide marketing tools" it intends "to continue offering indefinitely"; Target's FY2025 report describes its paid Target Circle 360 membership with "unlimited same-day delivery on eligible orders over $35". For a merchant of Kestrel & Pine's size the architectural point is modest: a membership is a customer attribute with a start and end date, and a liability for benefits paid for but not yet delivered.

Kestrel & Pine's stored value in 2027

Kestrel & Pine sells gift cards online through the platform and in the stores through the POS (point of sale, the till software). Because there is no link between the POS and the platform, **an online gift card cannot be redeemed in a store, nor a store card online** (illustrative). At 31 December 2026 the outstanding balances were about **€186,000** online and **€41,000** in the stores; Sanne de Wit, the Finance Director, reconciles both to the liability account monthly from two exports. There is no online loyalty program; the stores keep a loyalty list in the POS. Noor and Milan agreed in 2027 not to launch one until the customer record is unified, because points earned by a customer the platform cannot recognize across channels are a liability it cannot redeem correctly. One cross-channel balance is a goal for after Waypoint; it needs a central balance service, which the 2028 POS evaluation (decision W4) will weigh.

The Choices

A merchant can run promotions in four ways. The fourth is listed because it is common, not because it is good.

The platform's built-in promotions. Hosted platforms ship discounts, codes, automatic promotions, gift cards and, increasingly, store credit, with documented combination rules and limits. Shopify allows at most 25 active automatic discounts (apps included); commercetools allows 100 active code-less cart discounts per project plus 100 per store, and 10 codes per cart. The limits are design signals: promotion interaction is the most common source of margin bugs. It fits any merchant whose promotions fit the platform's vocabulary.

The platform plus a promotion or loyalty app. When the business wants what the platform does not model (a tiered loyalty scheme, referral rewards, cross-channel vouchers), an app or a specialist SaaS adds it, usually by creating discounts in the platform on the customer's behalf. The app becomes a second writer of the price, with its own rules, its own view of the customer and its own failure modes. Kestrel & Pine's pre-order app wrote stock; a loyalty app is the same shape of risk, which is why Apps, Plugins and Functions: Extensibility and Ecosystem Governance treats apps as integrations.

A dedicated promotion engine shared across channels. A separate service (bought or, rarely, built) holds the definitions and evaluates them for the web storefront, the POS

and the B2B portal through one API. It is the only option in which a store and the web shop run the same promotion from the same rule, and in which a wholesale contract price and a consumer promotion are kept apart by design. It costs an integration into every checkout and a team that owns the rules.

Rules hard-coded in the theme, a script or a marketplace listing. The agency makes a banner compute "20% off" in the browser; a script adjusts marketplace prices for a sale; store staff key their own discount into the POS. Each copy drifts, nobody can say what the price was on a given day, and the Price Nobody Can Explain follows. It is not a design but the absence of one.

Criterion	Platform built-in	Platform plus app	Dedicated engine	Hard-coded rules
Where the final discount is computed	Platform cart	Platform cart, rules written by the app	The engine, called by every channel	Several places
Order of evaluation and stop rule	Documented by the vendor; set per shop	Partly inside the app	Yours to define	Undefined
Same promotion in web, store and marketplace	Web only; stores and market-places get copies	Web and sometimes POS of the same vendor	Yes, by design	No
Discount allocation to lines	Vendor-defined	As the platform does it	Yours to define, must be exported	Usually missing
Stored value across channels	Within the platform's channels	App-dependent	A balance service	No
Fits	Starting, Growing; most Om-nichannel merchants	Growing merchants with one specific need	Omnichannel with stores and B2B on different systems; Group	Nobody

My rule: **use the platform's promotions until a promotion must be the same rule in two systems the platform does not control.** For Kestrel & Pine that moment has arrived for the stores (a separate POS) and will arrive for the B2B portal if wholesale promotions start; it never arrives for marketplaces, whose promotions stay in the

marketplace's own tools. Until then the cheapest improvement is governance, not software: one owner for the combination settings, one calendar, and a rollout test against the promotions already active, which Shopify offers as a feature and any merchant can do in a staging copy.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
1 — Starting	A few codes and a free-shipping threshold in the platform; platform gift cards reconciled by finance from an export	A naming convention for codes and a note of who created each promotion	A code from an old campaign still works; nobody knows which promotions are active
2 — Growing	Automatic discounts plus codes; a welcome code; the first Black Friday with several promotions at once; store credit for refunds	An explicit combination decision per campaign; single-use bulk codes; a first breakage estimate with the accountant	The first stacking surprise; disputed refunds of discounted items; the "first order" check fooled by a second e-mail address

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
3 — Omnichannel	Web, stores, marketplaces and B2B each run their own promotions; stored value split by channel; two loyalty lists	One promotion calendar across channels; allocation exported to the OMS (order management system), the returns tool and the ERP (enterprise resource planning system, which invoices); a central stored-value balance, or a decision not to cross channels yet; identity resolved before a loyalty program	Store staff keying discounts the web shop does not know; a marketplace promotion undercutting the web shop; gift cards that work in one channel only
4 — Group	Several brands or countries with their own calendars; a shared loyalty platform; a liability ledger per legal entity	A promotion engine or shared service with per-brand rules; a group stored-value ledger reconciled per entity; holdouts as standard	Brands competing for one customer with conflicting offers; a group voucher no single system can honor; breakage estimated differently per country

The judgement: the promotion *engine* changes little with scale; evaluating a thousand rules is the same problem as evaluating ten. What changes is **how many systems must agree on the result** and **how much money the stored-value ledger holds**. Kestrel & Pine at stage 3 has the first problem acutely (three channels with their own discounts, two gift-card pools) and the second mildly (about €227,000 outstanding), so its order is

ownership and allocation first, a cross-channel balance second, a loyalty program last.

In Practice

Dated, sourced examples

The prior-price rule turns promotions into a data requirement. Directive (EU) 2019/2161, the Omnibus Directive, applied by member states from 28 May 2022, requires any announced price reduction to show the **prior price**: the lowest price the trader applied in the 30 days before the reduction. The Dutch regulator ACM announced on 6 December 2022 that it would supervise the rule from 1 January 2023, with a worked example: a price raised from €50 to €70 and cut by 35% after 25 days is really a 9% discount on the €50 reference. On 11 June 2024 ACM published fines against Leen Bakker, G-Star Raw eStore, JYSK, PVH Brands Netherlands (Tommy Hilfiger) and Koopjedeal for discounts shown against a misleading reference price; Germany's § 11 PAngV transposes the same rule. For architecture this means an effective-dated price history per SKU per country that can answer "lowest price in the last 30 days" when a promotion is published, and a strike-through price that is itself a syndicated field. Had Kestrel & Pine's Price Drift run into November, the German strike-through price for the Ridgewalker would legally have been €179, not €199.

Stored value is a liability with accounting rules. Starbucks' FY2025 10-K, above. PwC's revenue guide on unexercised rights (accessed November 2026) states the ASC 606 position that estimated breakage is recognized "in proportion to the pattern of exercised rights", that breakage not expected to occur is recognized only when the likelihood of redemption becomes remote, and that amounts due under unclaimed-property law are never breakage revenue. The IFRS Foundation's page confirms IFRS 15 replaced IFRIC 13 on customer loyalty programs; the matching IFRS 15 paragraphs B44–B47 are cited here from a secondary source, because the standard's own text was not checked for this chapter.

Promotion abuse is an identity problem. Ravelin's 2020 article on promotion abuse (older than three years, but the taxonomy holds) names multi-accounting for sign-up offers, self-referral and guessable codes as the main forms; its figure that 53% of merchants saw more abuse in a year is a vendor survey without a published sample, so treat it as directional.

Incrementality needs a holdout. Gubela, Lessmann and Jaroszewicz (EJOR, 2019) argue that coupon campaigns should be judged on revenue uplift, not conversions, because customers differ in spending and most exposed customers do not buy; Gubela and Lessmann (2021) report that of several coupon types tested, only a €15 coupon produced positive incremental revenue. Redemption rate is not incrementality, because many redeemers were already at checkout; the honest measure is the difference between an exposed group and a **holdout group** deliberately not given the offer. For the platform

this means recording which promotions were *offered*, flagging holdout customers in the engine, and one customer identity to assign them with; Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics takes it from there.

Kestrel & Pine's Black Friday 2027 plan

For Black Friday 2027 Milan and Noor wrote the promotion plan as a table, each row with an owner, an order of evaluation and a stop rule, and tested it against every active promotion in a staging copy of the platform in early November (illustrative):

Promotion	Type	Order	Combines with	Stop	Owner
20% off outerwear	Automatic product discount	1	Free shipping only	Yes	Milan
Welcome code 10%	Order code	2	Free shipping only; never with row 1 (the better one applies)	—	Milan
Free shipping over €75	Shipping	3	Everything; threshold on the *discounted* subtotal, stated on the product page	—	Daniel Okafor
Store promotion, same 20%	POS discount keyed by store managers from this table	—	—	—	Daniel

Promotion	Type	Order	Combines with	Stop	Owner
Marketplace deals	Each marketplace's own tool, priced at the web price after discount plus fee uplift	—	—	—	Milan

Two changes matter more. The order export to the WMS (warehouse management system), the returns portal and the ERP now carries the **allocated net price per line**, so a return refunds what was paid; Ruben Alves added it to the integration contract under decision W5. And every change to the platform's promotion settings, including the combination setting, needs a second person's approval. Neither cost software; both cost the week in which the bug was understood.

When It Goes Wrong

Unintended stacking. Detection: a daily report of orders with more than one discount, with the discount depth (discount as a share of list price) per order, and a threshold alert in the peak week. Recovery: honor the orders already placed, fix the setting, and compute the margin given away from the order lines, which is only possible if the allocation was written.

Estimate and final disagree. The page says free shipping, the cart charges €4.95; the page says "20% off", the cart shows 10% because the code replaced the automatic discount. Detection: log what the page rendered and what the cart computed per session, and watch abandonment at the shipping step (From Cart to Confirmation: Checkout Orchestration). Recovery: state thresholds in the terms the cart uses, and let the page ask the engine for an estimate when the customer is logged in.

The discount comes back twice. A return refunds the list price of a discounted item (the Boomerang), or a gift-card purchase is refunded to the card *and* to the payment method. Detection: a weekly reconciliation of refunds against the allocated net price of the returned lines. Recovery: a credit-note correction per case, and the allocation added to every order export.

Codes leak or are guessed. A generic code on a coupon site, or a sequential format a script can enumerate. Detection: redemptions per code per hour against the expected rate, and per identity signal (device, payment instrument, address) rather than per e-mail (LISTED's Scams, Fakes and Account Takeovers: Fraud Prevention for Listings).

Recovery: disable the code, issue single-use replacements, and accept that placed orders are placed.

Stored value debited twice, or not at all. A checkout retried after a timeout debits a gift card twice; a payment webhook (the message the payment provider sends when a payment completes) arrives late and the order is created without the debit. Detection: ledger entries per instrument that reference the checkout attempt, so a duplicate shows as two entries with one key. Recovery: one idempotency key per checkout attempt covering the payment and the stored-value debit, and a daily job matching debits to orders (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation).

A promotion ends but a copy does not. A cached page, a marketplace listing updated hourly, a price-comparison feed or a POS still showing last week's 20%. Detection: an end-of-promotion check that reads the price on every channel; a marketplace parity warning is the expensive version. Recovery: a scheduled end that every copy receives as an event, with its delivery promise in the integration contract.

What to Take Away

Established.

- Promotions are data with an explicit order of evaluation and a stop rule, computed once in the cart; the storefront shows an estimate. Shopify, commercetools and Adobe Commerce document this shape independently (accessed November 2026).
- An order-level discount must be allocated to lines at order time, because invoices, VAT, returns and partial refunds work per line.
- Gift cards, store credit and loyalty points are contract liabilities under IFRS 15 and ASC 606; breakage is estimated and recognized in proportion to redemptions (PwC's guide; Starbucks' FY2025 10-K as the worked example).
- In the EU, an announced price reduction must show the lowest price of the preceding 30 days (Directive 2019/2161, applied from 28 May 2022; Dutch fines published June 2024).

Depends on context.

- Use the platform's built-in promotions until one promotion must be the same rule in two systems the platform does not control (a separate POS, a separate B2B portal); then a shared engine earns its integration cost. Marketplace promotions stay the marketplace's own in every case.
- A central stored-value balance is worth building once customers expect to use a card in any channel and the outstanding balance makes two reconciliations a month a risk; below that, two pools and an honest sign at the till are acceptable.

- Launch a loyalty program only after the customer identity is unified across channels; a single-channel business already meets that condition.
- Measure incrementality with holdouts when the promotion budget is large enough that a 10% error in its effect costs more than withholding the offer from a few percent of customers.

The author's synthesis.

- The cheapest promotion architecture is governance: one owner for the combination settings, one calendar with an order of evaluation per campaign, and a rollout test against everything already active, because almost every promotion incident I have seen was a setting, not a system.
- Treat the allocated net line price as the order's primary price and the list price as metadata; every downstream system that reads the list price will eventually refund it.

Questions to Ask

1. Which system computes the final discount, and does the product page show that rule's estimate or its own?
2. What is the order of evaluation, where does stacking stop, and who may change that setting?
3. Does the order export carry the allocated net price per line to the warehouse, the returns tool and the ERP?
4. How does "first order" identify the customer, and what happens with a second e-mail address?
5. Can the pricing system answer "lowest price in the last 30 days" for any SKU and country when a promotion is published?
6. Where do gift-card and store-credit balances live, and who reconciles them to the liability account, how often?
7. Is a gift-card debit idempotent with the checkout attempt, or can a retry debit twice?
8. Which promotions were *offered* last quarter, not only redeemed, and could we run a holdout?
9. When a promotion ends, how does each copy (stores, marketplaces, feeds, cached pages) learn of it, and how do we check?
10. What did last Black Friday's promotions cost beyond the plan, and can we compute that from the order lines alone?

The price is now decided. The next question is how the customer found the product in the first place, and what has to be true of the catalog for a search box to answer "waterproof jacket": Finding the Product: Search, Navigation and Merchandising.

Sources and Evidence

Company, product and investor sources

- Combining discounts — Shopify Help Center, n.d., accessed 2026-11; discount classes, combination rules, order of application, the 25-automatic and 5 + 1 code limits.
- Discounts — Shopify Help Center, n.d., accessed 2026-11; discounts applied before tax, scheduled rollouts tested against existing discounts.
- Store credit — Shopify Help Center, n.d., accessed 2026-11; expiry set per issuance.
- Pricing and discounts overview — commercetools docs, n.d., accessed 2026-11; product vs cart discounts, sort order, stacking mode, discount groups, best-deal mode.
- Cart Discounts — commercetools docs, n.d., accessed 2026-11; value types and the proportionate, even and individual allocation modes.
- Limits — commercetools docs, n.d., accessed 2026-11; active cart discounts and codes per cart.
- Create a cart price rule and Cart price rules — Adobe Experience League, n.d., accessed 2026-11; priority, discard subsequent rules, coupon usage caps, late-evaluated conditions.
- Starbucks Form 10-K, fiscal 2025 — Starbucks Corp. via SEC EDGAR, 2025; stored value liability, breakage, Stars deferral and expiry.
- Amazon Form 10-K, fiscal 2025 — Amazon.com, Inc. via SEC EDGAR, 2026; subscription services net sales and the framing of Prime benefits.
- Target Form 10-K, fiscal 2025 — Target Corp. via SEC EDGAR, 2026; the Target Circle 360 paid membership.

Standards, regulation and official documentation

- Directive (EU) 2019/2161 (Omnibus) — Publications Office of the EU, 2019; Article 6a prior price, 30-day window, application from 28 May 2022.
- ACM wil einde aan nepkortingen — Autoriteit Consument & Markt, 2022; supervision from 1 January 2023 and the worked example.
- ACM publiceert boetebesluiten webwinkels voor nepkortingen — Autoriteit Consument & Markt, 2024; the June 2024 fines.
- § 11 PAngV — Bundesministerium der Justiz, n.d., accessed 2026-11; the German transposition.
- IFRS 15 Revenue from Contracts with Customers — IFRS Foundation, n.d., accessed 2026-11; issue and effective dates, replacement of IFRIC 13.
- Unexercised rights (breakage), Revenue guide 7.4 — PwC Viewpoint, n.d., accessed 2026-11; the ASC 606 breakage rules.

- Revenue from customers' unexercised rights — IFRScommunity.com, 2024; secondary summary of IFRS 15 B44–B47 and material rights.

Research and practitioner sources

- Response Transformation and Profit Decomposition for Revenue Uplift Modeling — Gubela, Lessmann and Jaroszewicz, 2019; revenue uplift as the objective for coupon targeting.
- Interpretable Multiple Treatment Revenue Uplift Modeling — Gubela and Lessmann, 2021; several coupon types, one with positive incremental revenue.
- Promotion abuse in ecommerce — Ravelin, 2020; abuse taxonomy and a vendor survey figure.
- What is incrementality — Uniqodo, 2024; holdout method, vendor content.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

9. Finding the Product: Search, Navigation and Merchandising

IN THIS CHAPTER, YOU WILL: Learn how search, navigation, merchandising, recommendations and reviews help a shopper find a product, and what the catalog, prices and stock must provide for them to work.

KEY POINTS:

- **The search index is a copy, and every copy goes stale**. It is a separate, query-shaped copy of the catalog, the prices, the availability and the reviews, each of which changes at a different pace. Deciding what goes into the index and what is fetched fresh at query time is the central design decision.
- **Search is only as good as the data under it**. A zero-result query is usually a missing attribute, a missing synonym or an unindexed product, not a weak engine. Relevance tuning, merchandising rules and AI retrieval all sit on top of product data; none of them repairs it.
- **Reviews are discovery data with provenance rules**. A rating changes what ranks and what sells, so it belongs in the index; EU and US rules

require a merchant to know and be able to show whether a review came from a real purchaser.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Phantom Stock** — search results that say "in stock" from a copy of stock that is two hours old.
- **The Message in a Bottle** — index updates sent once and trusted, so a product changed in the catalog stays unchanged in the results.
- **The Lynchpin** — relevance configured inside one hosted vendor's console, by one agency, with no record of why.

Before a shopper can buy, they must find. On most storefronts they do that in one of three ways: they **search** (type words into a box), they **navigate** (click through categories and filters) or they are **recommended** something (a carousel of "you might also like"). All three answer the same question, "which of the 17,000 active variants should this person see first?", and all three depend on the same thing: **product data that is complete, correctly structured and fresh**.

This is why a search problem is rarely a search-engine problem. The engine ranks what it is given. If the jacket's waterproof rating lives in a spreadsheet, no engine can find it by "waterproof". If the price in the index is yesterday's, the results page lies. If stock in the index is two hours old, the results page sells what the warehouse has already shipped. **The search index is a copy of several systems of record, and the chapter's work is deciding which facts to copy, how fresh each copy must be, and what to do when a copy is wrong.**

Reviews belong here for the same reason. A rating is a signal that ranks products and persuades shoppers, so it lives in the index next to the title and the price. But a review is also a claim about a purchase, and since 2022 EU law has required merchants to say whether reviews come from real purchasers and to check that they do. Reviews are discovery data with a legal provenance record.

This chapter owns the **search index**, **relevance**, the **merchandising rule** and the **facet**. It builds on What Are We Selling? Products, Variants and the Catalog (the attributes being indexed), What Does It Cost? Price Lists, Contract Prices, Currencies and Tax (the price being shown) and Discounts, Coupons and Loyalty: Promotions Without Chaos (the strike-through price), hands the truth about stock to Is It in Stock? Inventory, Availability and the Promise You Can Keep, and leaves agentic commerce to Assistants and Agents: AI in Commerce and What Is Real in 2026. The sibling book LISTED goes deeper on engines in Search Architecture for Listings: Indices, Freshness and Scale and on ranking for marketplaces in Ranking for Two Sides: Relevance, Freshness and Paid Visibility; this chapter does not repeat them.

Where we are in the order's journey. Eva's jacket, the recurring order this book follows, begins before the cart: on 13 April 2027 Eva had to find the Ridgewalker on

the Dutch storefront before she could put it in a basket. This chapter is step one of the journey, discovery, and it is the step where the catalog, the price and the stock are shown to a shopper together for the first time.

Setting the Scene at Kestrel & Pine: The Waterproof Jacket Nobody Could Find

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), sells 3,400 products in 26,000 variants on three storefronts, using the search built into its hosted platform. In mid-March 2027 Hanna Berg, the Data Lead, brought the first-quarter search report so far to Noor Visser, the new Head of Engineering, and Teun Baas, who owns the catalog. The figures below are illustrative, invented for this chapter.

What the Q1 2027 search report showed *(illustrative)*	**Figure**
Sessions that used the search box	31%
Searches that returned no results (the **zero-result rate**)	9.4%
Rank of "waterproof jacket", "waterdichte jas" and "wasserdichte Jacke" among all queries	Top ten, in all three languages
Results for those queries	None
Why	The Ridgewalker is titled "shell jacket"; its 20,000 mm waterproof rating lives only in the seasonal spreadsheet; the platform's search indexes title, description, type and tags
Searches for "regenjas" (Dutch for rain jacket)	Zero results; no synonym
Ridgewalker shown "in stock" in search results after the warehouse sold out on 27 November 2026	Up to two hours, from the platform's stock copy

The report made three things visible. The best-selling product was invisible to the words shoppers used for it, because the attribute that answered their question was not in any system the search could read: Kestrel & Pine's missing PIM (product information management system, the system of record for product content), seen from the shopper's side. The engine had no synonyms, so the Dutch word for the product category returned nothing. And on the busiest evening of the year the results page had shown a copy of

stock that was two hours old, which is the Black Friday Oversell as the shopper first met it.

Figure 1: *Search ranks only what it can read: the best seller was titled shell jacket and its 20,000 mm rating lived in a spreadsheet, so waterproof jacket found nothing.*

The first fix was cheap: in March 2027 the storefront team added synonym sets per language (waterproof, rain, shell; regenjas, jas; Regenjacke, Jacke) in the platform's search settings. The second fix waited for the Waypoint program: when the PIM goes live in Q3 2027, "waterproofness" becomes an attribute every jacket carries and the search can filter on. By Q4 2027 the zero-result rate was 4.1%. The third fix is the subject of the next chapter: availability in search must come from the availability service, not from an hourly copy.

Three Ways to Find a Product

A shopper has a need; the storefront has 17,000 active variants. Discovery is whatever bridges the two.

Way of finding	What the shopper does	What the system needs	Who tunes it
Search	Types a query ("waterproof jacket M")	Indexed text and attributes; synonyms; typo tolerance; a ranking	The search team and merchandisers
Navigation	Clicks a category, then narrows with filters (size, color, price)	A category tree; complete attributes; **facets** (filters with counts) computed from the index	The catalog team
Recommendations	Accepts a suggestion ("often bought together")	Behavior data (views, purchases) and product relations	A model, with merchandiser overrides

Some vocabulary, in plain words. The **search index** is a separate copy of the catalog built for fast matching; it is not the catalog, and it is always slightly behind it. **Indexing** is the job that copies changes into it. **Relevance** is the engine's ordering of results for a query; **ranking** is the same thing seen as a list. A **facet** is a filter with counts ("Color: moss (14)") computed from indexed attributes. A **merchandising rule** is a human override of relevance for a query or category: pin a product to the top (boost), push one down (bury), hide one, redirect a query. A **synonym set** tells the engine that "jacket" and "coat", or "waterproof" and "rain", should match each other. **Typo tolerance** lets "ridgewalkr" find the Ridgewalker. **Personalization** reorders results for one shopper from what that shopper did before.

Baymard Institute's usability research, which benchmarks large retail sites against its own guidelines (article of September 2024, updated April 2026), reports that about half of test participants prefer search to navigation, that 56% of sites fail to adequately support users' search needs, and that failure rates rise with the query type: 12% of sites fail exact product-name searches, 39% fail feature searches ("waterproof"), 43% fail use-case searches ("jacket for hiking in rain") and 66% fail non-product searches ("returns"). These are expert reviews of mostly US and UK sites, not conversion measurements, so use them to name failure modes, not to size a business case. The pattern they show is the one at Kestrel & Pine: sites handle the product name and fail the words shoppers actually use.

How the Pieces Fit Together: The Index as a Copy

The diagram shows the search service as a consumer of four systems of record, each changing at its own pace, and the storefront asking it one question.

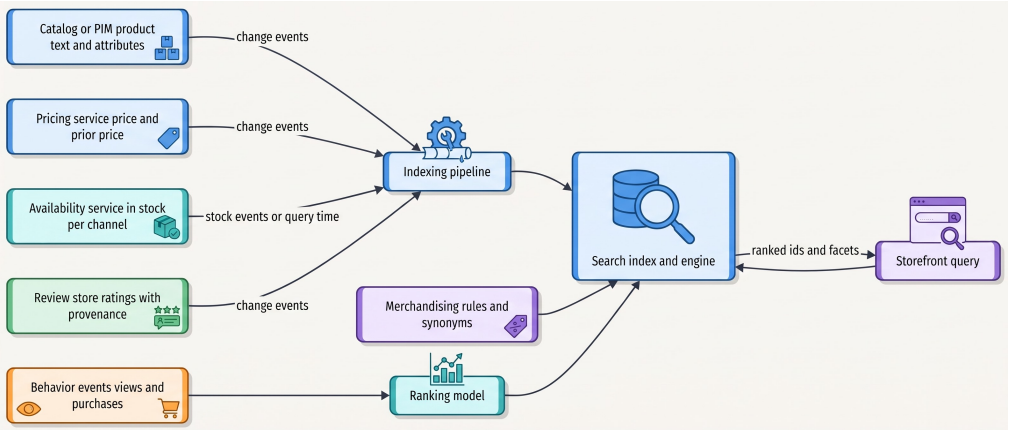


Figure 2: *The search index is a copy of the catalog, the prices, the availability and the reviews, fed by change events and overlaid with merchandising rules and a ranking model; the storefront gets back ranked identifiers and facets and fetches the fresh details separately.*

What goes in the index, and what is fetched fresh

The decision that shapes everything else is **which facts are in the index and which are joined at query time**. A fact in the index can be filtered and sorted on ("under €150", "in stock only", "best rated"), but it is only as fresh as the last indexing run. A fact fetched at query time from its system of record is fresh, but the engine cannot filter or sort on it. commercetools' Product Search (documentation accessed November 2026) shows the usual compromise: it indexes product fields, variant attributes declared searchable, prices and availability, returns product identifiers, and lets the client fetch full data separately; price filters work per currency, country, customer group and channel.

Fact	Changes how often	Freshness the shopper needs	Where it should live
Product title, description, attributes	Seasonally, with corrections	Hours	In the index

Fact	Changes how often	Freshness the shopper needs	Where it should live
Category and assortment membership	Weekly	Hours	In the index
List price and strike-through price	Daily; promotions by the minute	Minutes during a promotion	In the index for filtering; re-read from the pricing service on the product page
Availability per channel	Continuously; by the second at peak	Minutes, or at query time	A coarse flag in the index ("available"); the exact answer from the availability service at query time
Rating and review count	Daily	Hours	In the index
Behavior signals (views, purchases)	Continuously	Daily aggregates	In the ranking model, not the index

The availability row is where Kestrel & Pine was hurt. Its platform indexed the platform's own stock copy, which the stock sync chain updated hourly from the ERP (the enterprise resource planning system), which the WMS (the warehouse management system at Duiven) updated every fifteen minutes. On Black Friday evening the results page said "in stock" for up to two hours after Duiven ran out. The remedy is not a faster index; it is availability as a **stock event** (a message that stock changed, sent when it changes) from the availability service decided under Waypoint W1, with the index holding only a coarse flag and the product page asking for the exact number. Is It in Stock? Inventory, Availability and the Promise You Can Keep owns that design.

Figure 3: *The Phantom Stock seen from search: the index read a copy of a copy, so on Black Friday the results said in stock for up to two hours after the shelf at Duiven was empty.*

How the index is fed

Two patterns exist. **Scheduled sync** copies the catalog on a timer, nightly or hourly. It is simple and it is always stale by up to one interval; Algolia's Shopify B2B (business-to-business) integration documents a daily resync, with some changes taking up to two days to reach all B2B customers, and notes that product reindexing is never automatic. **Event-driven indexing** listens for change events from the catalog, the pricing service and the availability service and updates the affected records within seconds; it is the design every system in Part VI aims at, and it needs the idempotent handling of repeated messages and the reconciliation job that Waypoint W5 requires of every integration, because an index update is a message in a bottle like any other. In both patterns a **full rebuild** must remain possible, because a change to the index's shape (a new attribute, a new language, a new embedding model) cannot be applied record by record.

Interaction	Freshness	Consistency	Latency	Availability
Search query	Index as of the last update	Eventual; the product page is the truth	Under about 200 ms for results and facets *(assumption)*	If search is down, navigation must still work

Interaction	Freshness	Consistency	Latency	Availability
Attribute or text change	Minutes	Eventual	Seconds per event	Queue and retry
Price change during a promotion	Minutes	Eventual, with the pricing service re-read on the page	Seconds per event	Queue and retry
Stock change at peak	Minutes for the coarse flag	The availability service is the truth	Seconds per event; hundreds per minute at peak	Degrade to "check the product page"
Full rebuild	Hours	Atomic swap of the old and new index	Hours	Old index serves until the new one is complete

Relevance, merchandising and facets

A lexical engine matches words, with stemming (jackets → jacket), synonyms and typo tolerance; then it ranks the matches by text match, by popularity and by business rules. Elasticsearch's documentation (accessed November 2026) recommends search-time synonyms because the sets can be updated without reindexing; the distinction matters at Kestrel & Pine, where synonyms are per language and change every season.

Merchandising rules sit on top. Algolia's Rules (documentation accessed November 2026) can pin, hide, boost, bury, filter, rewrite the query or return custom data, carry up to 25 conditions, and have a validity period "so that promotions end automatically", with the vendor's own caveat that rules "are no substitute for a well-configured index". Shopify's Search & Discovery app allows at most 10 search terms per product boost. I treat merchandising rules as promotions: **data with an owner**, **a start**, **an end and a count**, reviewed on the same calendar as the campaigns they serve, because a rule that never expires is a price nobody can explain in a different costume.

Facets depend on attribute completeness, and the limits are concrete. Shopify's filters (documentation accessed November 2026) compute values and counts using only the first 1,000 results, show no filters on collections above 5,000 products, allow 25 filters per store with up to 100 values each, and do not show a price filter in a currency other than the shop's default. A "waterproof" facet with half the jackets unfilled hides the other half from a shopper who ticks it, which is why the facet's quality is decided in the PIM, not in the search console. LISTED's Ranking for Two Sides: Relevance, Freshness and Paid Visibility covers ranking signals for marketplaces with two sides; a single-merchant storefront needs only a fraction of it.

Personalization and AI retrieval, in plain words

Collaborative filtering recommends items from patterns of what other shoppers viewed or bought together, without understanding the items. Amazon's item-to-item version was described in a 2003 paper and, by Amazon's own 2019 account of its history, had already been in use for six years; the same account notes that a deep-learning model for Prime Video first underperformed both the old method and a plain bestseller list, and only beat them after being reframed. The lesson is evidence, not nostalgia: **strong baselines are hard to beat, and reframing the problem beat scaling the model**.

Vector (semantic) search matches the meaning of a query to product text through numeric embeddings, so "something for rain" can match "shell jacket". **Hybrid search** runs a lexical and a vector query and merges the two ranked lists, commonly by reciprocal rank fusion, which rewards products that rank well in either. Elasticsearch (documentation accessed November 2026) runs both in one request and recommends approximate nearest-neighbor retrieval for production; OpenSearch introduced hybrid search in version 2.11; Algolia launched NeuralSearch on 2 May 2023; Shopify's semantic search reached its Shopify and Advanced plans on 24 June 2024. Amazon's 2019 research paper on semantic product search gives the reason: "pure lexical matching via an inverted index falls short" on hypernyms, synonyms, antonyms, morphological variants and spelling errors.

Two cautions. First, embeddings are a second derived artifact: Elasticsearch's documentation states that query vectors "must have the same dimension and be created with the same model as the document vectors", so a model change is a full reindex, and product text changes make embeddings stale just as they make keywords stale. Second, every conversion lift a search vendor publishes is a customer anecdote without a published control; Algolia's launch release reported a 65% drop in zero results and up to 17% conversion uplift for one customer in a four-week test, and Constructor's home page claims a 13% lift for one retailer with no baseline or method. Present them as "the vendor reports", and measure your own zero-result rate first.

Conversational search (a shopper describing a need in sentences) is the current frontier. Zalando announced a beta fashion assistant built on ChatGPT on 19 April 2023 for selected customers in four countries, and its Q3 2026 results release (3 November 2026) says, as of 2026, more than 22 million unique users visited the Zalando Assistant in the year. What such an assistant needs from the merchant (structured attributes, fresh prices and availability, order status by API) is exactly what Waypoint W8 calls agent readiness; Assistants and Agents: AI in Commerce and What Is Real in 2026 takes it from there.

Search for wholesale: contract prices and assortments

For a wholesale account, search must answer "what can *this* buyer see, at *their* price". An **assortment** is the subset of the catalog an account may buy; a **contract price** is its

negotiated price (Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals owns both). Algolia's B2B guidance (accessed November 2026) gives the three designs: with fewer than about 100 pricing levels, store one price per tier or customer group in the index and sort by it; with more, keep per-customer prices out of the index and merge them at query time, accepting that they cannot be filtered or sorted on; or do both. Its Shopify B2B integration stores per-company-location prices in a hidden attribute that only the logged-in location can retrieve and recommends a ceiling of about 50 company locations before records hit size limits.

The rule that matters is about security, not speed: **assortment visibility must be enforced in the search service, with the filter bound to the account's token, not in the storefront**, because search results are an information leak. Kestrel & Pine's 1,100 accounts, on a handful of price tiers *(assumption)*, fit the segment-price design; a merchant with per-account prices for thousands of accounts does not.

Reviews and Ratings as Discovery and Trust Data

A review does two jobs. As **discovery data**, the average rating and the count rank products and persuade shoppers; schema.org's `Product` type (version 30.1, September 2026) carries `aggregateRating`, so the rating is also syndicated to search engines and shopping feeds. As **trust data**, a review is a claim that someone bought and used the product, and that claim is regulated.

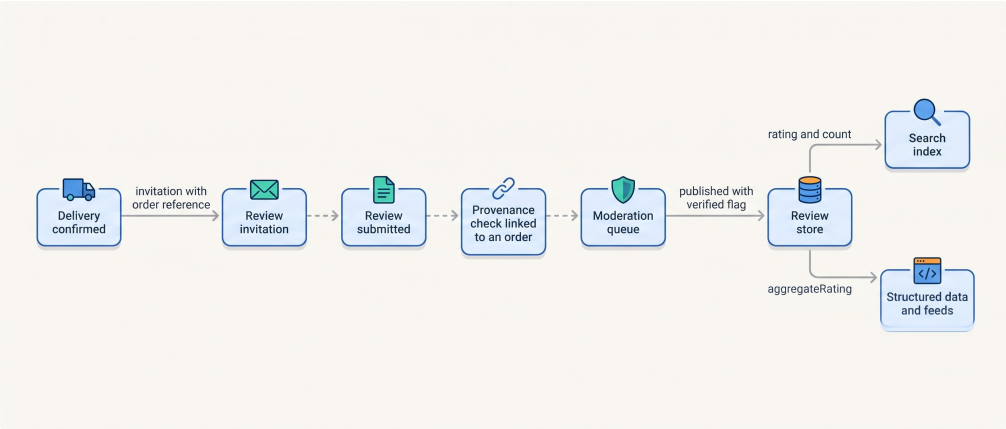


Figure 4: *A review enters through an invitation tied to a delivered order, carries its provenance through moderation, and reaches the index and the feeds as a rating with a verified-purchase flag the merchant can prove.*

Directive (EU) 2019/2161, applied from 28 May 2022, makes it material information whether reviews come from "consumers who have actually used or purchased the product", and blacklists both stating that reviews are from real purchasers "without

taking reasonable and proportionate steps to check" and "submitting or commissioning another … to submit false consumer reviews"; its recital 19 defines ranking broadly, as the relative prominence given to offers "including resulting from the use of algorithmic sequencing, rating or review mechanisms, visual highlights, or other saliency tools", and the directive makes the main parameters determining ranking material information too. In the United States, the Federal Trade Commission's rule of 14 August 2024 bans fake or AI-generated reviews, compensation "conditioned on the writing of consumer reviews" with a particular sentiment, undisclosed insider reviews and review suppression, with civil penalties. Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation places these in the wider calendar.

Four design consequences:

- **Provenance per review**. Every review records how it was collected and, where it is marked verified, the order it is linked to. Kestrel & Pine's reviews app invites customers after delivery with the order reference, so each of its 6,800 reviews in 2026 is tied to an order (illustrative); a review form open to anyone can still exist, but its reviews cannot carry the verified flag.
- **Moderation is a queue with rules, not a judgement call**. Reject on policy (off-topic, personal data, abuse), never on sentiment; the FTC rule names suppression of negative reviews as a violation.
- **Incentives are disclosed and never conditional on sentiment**. A discount code for "a review" is lawful in most places if disclosed; a code for "a five-star review" is not.
- **Ranking parameters are documented**. If "best rated" or "most popular" sorts the results, the merchant must be able to say what the sort does; that is a one-page document, kept with the merchandising rules.

Marketplaces keep their own reviews under their own rules; a merchant selling on bol, Amazon.de and Zalando has three review pools it does not own, which Selling on Someone Else's Platform: Marketplaces and Channel Integration covers. For a merchant of Kestrel & Pine's size, reviews are a case for a specialist app or service with the provenance and moderation built in, and the architectural work is making sure the rating reaches the index and the feed, and that the order link survives a platform or app change.

The Choices

The platform's built-in search. Hosted platforms ship search, filters, synonyms, boosts and, since 2024 on some plans, semantic understanding. It is free, it indexes the platform's own copy of price and stock, and its limits are published (Shopify's 25 filters and first-1,000-results counts above). It fits a merchant whose catalog fits the platform's product model and whose promises can tolerate the platform's indexing delay.

A hosted search service fed by events. Algolia, Constructor, Bloomreach and others run the index and the engine as a service; the merchant feeds them from the catalog, the pricing service and the availability service, and merchandisers tune relevance in a console. This buys relevance tooling, analytics and hybrid retrieval, and costs an integration per source, a subscription, and relevance configuration that lives in a vendor's console unless it is exported and versioned, which is the Lynchpin risk.

A self-run engine. Elasticsearch or OpenSearch, managed or self-hosted, with the index design, analyzers, synonyms, ranking and the pipeline built by the merchant's team. It is the right choice when relevance is a competitive asset, the catalog does not fit a hosted service's record model, or the data cannot leave the merchant's control. It costs a team that owns search as a product.

A search module of a composable or marketplace stack. Larger merchants and marketplaces run discovery as its own platform, with per-brand indexes and ranking experiments; LISTED's Search Architecture for Listings: Indices, Freshness and Scale describes that shape. It is a Group-stage answer.

Criterion	Platform built-in	Hosted search service	Self-run engine
Who owns relevance	The vendor's defaults plus a few boosts	Merchandisers, in the vendor's console	Your team, in code and configuration
Who owns the data pipeline	The platform	You, per source	You, per source
Freshness of price and stock	The platform's copy and schedule	As fresh as your events	As fresh as your events
B2B prices and assortments	Limited	Segment prices in the index; per-account at query time	Any design
Hybrid or semantic retrieval	Plan-gated, English first	Built in since 2023–2024	Elastic kNN and hybrid; OpenSearch 2.11+
Exit cost	None	Relevance configuration and analytics history	The whole system
Fits	Starting, Growing	Growing, Omnichannel	Omnichannel with a search team; Group

My rule: **measure the zero-result rate and the top failing queries before buying anything**. Half of Kestrel & Pine's problem was fixed with synonym sets in the platform it already had; the other half waits for the PIM, which no search vendor can replace. A hosted service earns its subscription when the merchant has merchandisers who will use the console weekly and a catalog with the attributes to tune against. A self-run engine earns its team when search is a product, not a feature.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
1 — Starting	The platform's built-in search; one language; categories and a few filters; reviews from an app	A monthly look at zero-result queries; synonyms for the five words shoppers use that the catalog does not	Shoppers search for a word in the product's name and find nothing; a facet with half the products unfilled
2 — Growing	A second language or market; the first marketplace; a hosted search app with daily sync; review invitations after delivery	Synonym sets per language; attributes filled before a facet is switched on; a zero-result and click-position report per language	Search results show prices or stock from yesterday; merchandising rules that nobody remembers creating; reviews that cannot be linked to an order

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
3 — Omnichannel	Several markets, a PIM, a B2B portal, an app and store associates all querying products; promotions by the minute	Event-driven indexing from the PIM, pricing and availability services; a coarse availability flag with query-time truth; assortment and contract-price filtering enforced in the search service; one search service for storefront, app and store tools	A hosted service's record limit hit by B2B prices; "in stock" in results and sold out on the page; a vendor console nobody can reproduce in a new environment
4 — Group	A discovery platform shared by brands; per-brand indexes and relevance; ranking experiments with holdouts; a review platform with group-wide moderation	A search team that owns the pipeline and the experiments; a documented ranking policy per brand and market; model and index versioning for embeddings	Brands fighting over shared relevance defaults; an embedding model change that silently degrades one brand's results

The judgement: the engine matters less with each stage than the pipeline feeding it. A Starting merchant is limited by vocabulary and fixes it with synonyms; a Growing merchant is limited by staleness and fixes it with events; an Omnichannel merchant is limited by how many consumers (web, app, store associates, wholesale) must see the same products at their own prices and stock, and fixes it with one search service and query-time truth; a Group is limited by ownership. Kestrel & Pine sits in the third row with second-row tooling, which is the normal condition of a merchant that grew faster than its systems.

In Practice

Deprecation is a search risk too. commercetools' older Product Projection Search is deprecated: projects created after 31 August 2026 cannot activate it, and it deactivates after 30 days without calls (documentation accessed November 2026). A search integration built on a retired API is Kestrel & Pine's Deprecated Endpoint of February 2027 in another system; Promises Between Systems: API Contracts, Webhooks and Versioning covers the discipline.

Limits are documented, so read them. Shopify's filters use the first 1,000 results for counts and allow 25 filters per store; Algolia's Shopify B2B integration suggests about 50 company locations before records hit size limits and stores only the highest volume tier; commercetools indexes up to 10,000 variants and standalone prices combined per product in its Modular Catalog. Each limit is a design constraint a merchant discovers at the worst moment unless it is read first.

Metrics have definitions. Algolia's analytics (documentation accessed November 2026) define the no-results rate as the percentage of searches with no results, click-through rate as the percentage of tracked searches with at least one click, conversion rate as the percentage with at least one conversion event, and average click position, where "smaller values are better"; keystrokes within 30 seconds aggregate into one search. Whatever engine a merchant uses, these four, per language and per channel, are the dashboard. Baymard's data show mobile and app sites performing worse than desktop, so a single blended number hides the worst channel.

Kestrel & Pine's 2027 search plan. Noor's plan for search had five lines and no new vendor (illustrative): synonym sets per language in March; a monthly review of the top 50 zero-result queries with Teun's catalog team, the way promotions are reviewed; the waterproof rating, fabric, insulation and fit attributes modeled in the PIM for Q3 with the facet switched on only once 95% of jackets carry a value; availability in the index replaced by a coarse flag fed from the availability service when it goes live, with the product page asking for the exact number; and a reviews data export that keeps the order link, so the 6,800 reviews survive any later app change. The zero-result rate fell from 9.4% to 4.1% by Q4, most of it from the synonyms. The plan's cost was a few days of configuration and one attribute model; its precondition was a PIM the company had already decided to buy for other reasons.

When It Goes Wrong

The index says in stock; the warehouse says no. Detection: a daily sample of results pages against the availability service, and the rate of "add to cart" failures on items shown available. Recovery: a coarse flag in the index and the exact answer at query time; at peak, a rule that any product sold out in the last hour is re-indexed immediately. This is the Phantom Stock in its first outfit.

A change never reached the index. A product renamed in the catalog still shows its old name; a price change made at 09:00 appears at noon; a discontinued variant still ranks. Detection: a reconciliation job that compares a sample of index records with their sources every hour, the same job Waypoint W5 requires of every integration. Recovery: re-index the differing records, and fix the lost event, which is the Message in a Bottle.

A full rebuild is needed and nobody planned for it. A new attribute, a new language or a new embedding model requires rebuilding every record. Detection is easy; the failure is doing it in place and serving a half-built index. Recovery: build the new index beside the old one and swap atomically; keep the old one until the new one is verified.

Synonyms in the wrong language. A synonym set written for Dutch applied to the German index, so "jas" matches nothing and "Jacke" matches everything. Detection: zero-result and click-position metrics per language. Recovery: one synonym set per language index, owned by someone who speaks it.

A merchandising rule outlives its campaign. Last season's pinned jacket still tops "jacket" in April. Detection: a list of active rules without an end date. Recovery: validity windows on every rule, and the same monthly review as promotions.

Review provenance is lost. The reviews app is replaced, the export has the text and the stars but not the order reference, and the "verified purchase" badge can no longer be substantiated. Detection: a count of verified reviews whose order cannot be found. Recovery: keep the order link in your own store, not only in the app's; drop the badge where it cannot be proven.

The assortment leaks. A wholesale account's search returns products from another tier, or a retail visitor's search returns B2B prices, because the filter was applied in the storefront, not in the search service. Detection: a test account per tier that queries the service directly. Recovery: token-scoped filters in the service; treat it as a security incident, which it is (Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers).

The hosted search service is down. Detection is immediate; the question is what the storefront does. Recovery: navigation and category pages must work without search, served from the platform, and the search box must fail with a message rather than an empty result, which shoppers read as "you do not sell it". Black Friday Every Year: Peak Load, Resilience and Recovery covers the general case.

What to Take Away

Established.

- The search index is a derived copy of the catalog, prices, availability and reviews; it is always behind its sources, and the design decision is which facts are in it

(filterable, sortable) and which are fetched fresh at query time (commercetools', Algolia's and Shopify's documentation, accessed November 2026, all show this shape).
- Zero-result rate, click-through rate, conversion rate and click position, per language and channel, are the measures of search; a zero-result query is most often a data gap, not an engine fault (Baymard 2024–2026; Algolia's metric definitions).
- Hybrid retrieval (lexical plus vector, merged by reciprocal rank fusion) is now standard in Elasticsearch, OpenSearch (2.11), Algolia (2023) and Shopify (2024); embeddings are a second derived artifact that must be rebuilt when the model or the text changes.
- Since 28 May 2022, EU merchants must be able to show whether reviews come from real purchasers and must take reasonable steps to check; the main parameters determining ranking are material information (Directive 2019/2161). The FTC's 2024 rule bans fake, incentivized-by-sentiment and suppressed reviews in the US.

Depends on context.

- The platform's built-in search is enough while the catalog fits the platform's product model, the storefront is in one or two languages, and the indexing delay is shorter than the promises on the page; the first promotion-by-the-minute or the first B2B tier breaks that condition.
- A hosted search service pays for itself where merchandisers will tune relevance weekly and the catalog has the attributes to tune against; without a PIM or equivalent, buy the attributes first.
- Put availability in the index as a coarse flag only when the exact answer is available at query time from an availability service; without one, a stale "in stock" in the index is worse than no flag.
- Enforce B2B assortment and contract-price visibility in the search service once more than one tier exists; a single-tier wholesale business can do it in the storefront for a while.

The author's synthesis.

- Treat search as the first consumer of the PIM and the availability service, and fund those before any search vendor; every relevance feature I have seen bought ahead of its data was tuned against gaps.
- Keep merchandising rules, synonyms and ranking parameters as versioned data with owners and end dates, outside any vendor console, because they are the part of search a merchant cannot buy back.

Questions to Ask

1. What is our zero-result rate per language, and what are the top twenty queries that return nothing?
2. Which facts are in the search index, and which are fetched fresh at query time? How old can the price and the stock in a results page be?
3. How does a catalog change reach the index: on a schedule, or as an event with retry and reconciliation?
4. Can we rebuild the whole index beside the live one and swap, and when did we last do it?
5. Who owns the synonym sets per language, and where are the merchandising rules recorded, with their end dates?
6. Which attributes do our facets depend on, and what share of products carries a value for each?
7. For wholesale accounts, where is the assortment filter enforced, and has anyone tested it with a token from another tier?
8. For every published review, can we show how it was collected and, if it is marked verified, which order it came from?
9. If we sort by "best rated" or "most popular", can we write down in one page what that sort does?
10. If the search service is down for an hour on Black Friday, what does the storefront show?

Search has shown Eva a Ridgewalker in size M, moss, "in stock". The next chapter asks what that last phrase means, who computed it, and how to make it a promise the warehouse can keep: Is It in Stock? Inventory, Availability and the Promise You Can Keep.

Sources and Evidence

Company, product and investor sources

- Product Search and Product Projection Search (deprecated) — commercetools docs, n.d., accessed 2026-11; what is indexed, identifiers returned, price filters per scope, the deprecation dates.
- Limits — commercetools docs, n.d., accessed 2026-11; indexed variants and prices per product.
- Shopify B2B integration — Algolia docs, n.d., accessed 2026-11; daily resync, per-location prices, the 50-location ceiling.
- Personalized pricing for B2B — Algolia docs, n.d., accessed 2026-11; the three B2B price designs.

- Rules overview — Algolia docs, n.d., accessed 2026-11; merchandising rule consequences, conditions and validity periods.
- Search analytics metrics — Algolia docs, n.d., accessed 2026-11; metric definitions.
- Algolia launches NeuralSearch — Algolia, 2023; launch date and a vendor-reported customer result.
- Filters and Modifying search — Shopify Help Center, n.d., accessed 2026-11; filter limits, boosts, semantic understanding.
- Semantic search is now available on more plans — Shopify changelog, 2024; the 24 June 2024 rollout.
- Constructor — Constructor, n.d., accessed 2026-11; vendor claims, cited as such.
- Zalando launches fashion assistant powered by ChatGPT — Zalando, 2023; the April 2023 beta.
- Zalando Q3 2026 results — Zalando, 2026; 22 million unique Assistant users.
- The history of Amazon's recommendation algorithm — Amazon Science, 2019; item-to-item filtering and the Prime Video baseline story (older than three years; historical).

Standards, regulation and official documentation

- Directive (EU) 2019/2161 (Omnibus) — Publications Office of the EU, 2019; review provenance, fake-review blacklist, ranking as material information, recital 19.
- FTC announces final rule banning fake reviews and testimonials — Federal Trade Commission, 2024; the rule's prohibitions.
- schema.org Product — schema.org, 2026 (v30.1); `aggregateRating` and `offers`.
- kNN search, Hybrid search and Search with synonyms — Elastic docs, n.d., accessed 2026-11; vector model constraints, hybrid retrieval, search-time synonyms.
- Hybrid search — OpenSearch docs, n.d., accessed 2026-11; hybrid search since 2.11, normalization and reciprocal rank fusion.

Research and practitioner sources

- Semantic Product Search — Nigam et al., Amazon, KDD 2019; why lexical matching falls short and the reported gains.
- Ecommerce search query types — Baymard Institute, 2024, updated 2026; failure rates by query type and the search-vs-navigation preference.
- Ecommerce search report and benchmark — Baymard Institute, n.d., accessed 2026-11; autocomplete and misspelling findings.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

10. Is It in Stock? Inventory, Availability and the Promise You Can Keep

IN THIS CHAPTER, YOU WILL: Learn what "in stock" really means, who should compute it, and why Kestrel & Pine oversold 1,340 jackets on one Friday evening.

KEY POINTS:

- **"In stock" is a computed promise, not a count.** Physical stock (on hand) is a fact owned by whoever counts it. What a channel may still sell (available-to-promise) is derived: on hand minus units promised to orders minus units held back on purpose. Every oversell is a disagreement between the fact and the derivation.
- **Oversells come from writers and lag, not from bad luck.** Several systems writing the same stock number, periodic copying instead of events, and reservation windows let two channels sell the same last unit. The fix is one writer per stock state and stock events that travel in seconds.

- **One availability service, many channels, each with its own buffer.** With more than one location and more than one channel, availability belongs to a service that receives stock events, applies per-channel rules and publishes an available figure per channel. Kestrel & Pine buys it as a module of its order management system (Waypoint W1).

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Phantom Stock** — two channels sold the same last Ridgewalker because each learned of the other's sales up to two hours late.
- **The First Snowfall** — the stock sync chain had never met a Friday with 38 orders a minute; at that rate a one-hour copy interval is 2,300 orders of blindness.
- **The Secret Agent** — a pre-order app wrote inventory in the platform while the warehouse feed wrote it too, and for two weeks nobody owned the number.

A shopper sees "In stock" next to a jacket and reads it as a promise: if I buy this now, I will get it. The merchant makes the same promise on the web shop, on a marketplace, at a store till and to a wholesale buyer, often in the same minute, from the same shelf. **"In stock" is the most repeated promise in commerce and the one most often made on stale information.**

The trouble is that the two words hide at least five numbers. How many units are physically on the shelf. How many of those are already promised to orders that have not shipped. How many are sitting in a cart. How many the merchant has deliberately held back, as a buffer, as damaged goods, or for a wholesale customer. And how many are on a truck on the way in. Only one of these is a fact anyone can count; the others are claims and derivations. **When several systems each hold their own version of the number, and each learns of changes by copying from the next one on a schedule, the promise and the shelf drift apart.** The drift is invisible on a quiet Tuesday and expensive on Black Friday.

This chapter is about computing the promise you can keep: the vocabulary, the moment a unit becomes "promised", what changes with several locations and channels, the four ways merchants compute availability, and how to choose as a business grows. Kestrel & Pine, the fictional outdoor brand this book follows, and its worst evening of 2026 are the worked example.

Where we are in the order's journey. Eva's jacket (the recurring order of this book, see Meet Kestrel & Pine) is on the product page, about to go into the cart. This chapter is the "In stock" label she sees before she clicks, the check at add-to-cart, and the moment at checkout when her unit is taken out of what the next shopper may buy. From Cart to Confirmation: Checkout Orchestration takes over at the cart; Pick, Pack, Ship: Warehouse Management and Fulfillment when the unit is picked.

Setting the Scene at Kestrel & Pine: Friday 27 November 2026, 19:40

In January 2027 Noor Visser, Kestrel & Pine's new Head of Engineering, asks Ruben Alves, the lead integration engineer, and Daniel Okafor, Head of Operations, to reconstruct the Black Friday Oversell minute by minute. The mechanism was never written down. What they draw on the whiteboard is a chain.

The warehouse at Duiven runs on **the WMS**, a warehouse management system (the software that knows which bin holds which unit and tells pickers what to pick). It is the only system that counts physical stock, and it reduces its count when a unit is picked. Every 15 minutes it sends stock levels to **the ERP** (the enterprise resource planning system: finance, purchasing, inventory valuation). Every hour the ERP sends stock to **the platform**, the hosted commerce platform behind the three storefronts, and every hour the platform sends stock to **the marketplace connector**, which pushes it to bol, Amazon.de and Zalando. A storefront sale reduced the platform's own number at once; the marketplaces learned of it only after the unit was picked and the chain had run, up to two hours later. A marketplace sale reached the storefront the same slow way.

On Friday 27 November the Ridgewalker jacket, the hero product, was the deal of the week. The reconstruction is illustrative; the totals are those recorded in Meet Kestrel & Pine.

Time (27 Nov 2026)	What happened	What the channels believed about the Ridgewalker
19:40	The WMS picks the last units; physical on hand reaches zero	Storefronts and marketplaces: in stock
19:45	WMS sends stock to the ERP (15-minute run)	ERP: zero. Platform and marketplaces: in stock
20:00–21:00	Peak hour: 2,300 orders, 38 per minute	Storefronts and marketplaces keep selling it
about 20:30	ERP sends stock to the platform (hourly run) *(illustrative time)*	Storefronts: sold out at last

Time (27 Nov 2026)	What happened	What the channels believed about the Ridgewalker
21:05	Milan Petrov, alerted by the warehouse through Daniel Okafor, sets the Ridgewalker to zero in the marketplace connector by hand; the hourly platform-to-connector run would have arrived later still	Marketplaces: sold out once the marketplaces process the push
Saturday	Support discovers what was sold against zero stock	1,340 units oversold, 1,120 orders affected

Figure 1: *The Black Friday Oversell as a relay: each hand-off waited for its schedule, so the channels kept selling for up to two hours after Duiven ran out, and 1,340 units were oversold.*

Two things made it worse. The order queue from the platform to the WMS backed up for four hours that evening (Pick, Pack, Ship: Warehouse Management and Fulfillment), so the WMS's count fell later than the sales it reflected. And three of the platform's 41 installed apps could write inventory. One, a pre-order app, had already caused the App That Wrote Stock in March 2026: two weeks of phantom reservations on about 600 variants, because the app and the WMS feed both wrote the same number and each

undid the other.

Mara Lindqvist, the CEO, asked the question every CEO asks after an oversell: "Why did the website not know?" Noor's answer, which became Waypoint decision W1, was that the website could not know, because nobody had decided who owned the number and nothing carried changes fast enough. This chapter is that answer in full.

What "In Stock" Means: Five Numbers, Not One

Every system that tracks stock separates the units that are physically present from the units it may still promise. The names differ between vendors; the arithmetic does not.

- **On hand** is the number of units physically at a location right now. It is a fact, owned by whichever system counts it: a WMS, an ERP's warehouse module, a point-of-sale system (**POS**, the till software) for a store, or the platform itself for a merchant with a back room.
- **Committed** (also **allocated**) is the part of on hand already claimed by placed orders that have not shipped. Eva's jacket becomes committed when her order is confirmed and stays committed until it ships.
- **Reserved** is a temporary claim before an order is final: a unit in a cart during a limited-stock sale, or in a draft order a sales rep is building. Reservations expire; commitments do not.
- **Held back** covers units present but deliberately not for sale: a **safety stock** buffer against counting errors and sync lag, damaged units, units awaiting quality control, units set aside for a wholesale customer.
- **Incoming** is stock on the way in: a supplier's shipment or a transfer from another location. It is not on hand until received.

From these, the number a channel may actually sell is derived. In planning vocabulary that derived number is **available-to-promise (ATP)**: the uncommitted portion of stock, and sometimes of planned receipts, that can be promised to new orders. Commerce platforms implement a simple single-period version of it, usually called "available" or "salable". As industry practice, the formula is:

> available-to-promise = on hand − committed − reserved − held back (+ incoming, only if you promise against it)

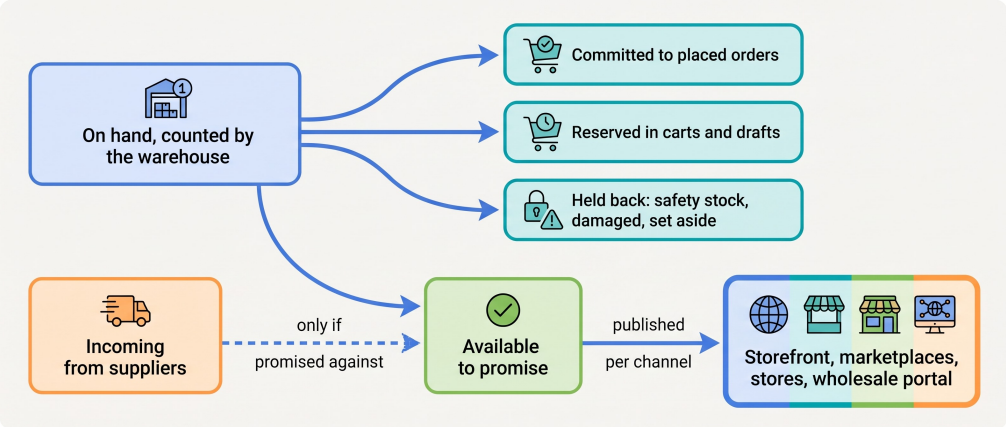


Figure 2: *On hand is the one counted fact; committed, reserved and held-back units are claims against it; available-to-promise is what remains, and it is a derivation that each channel should receive, not a number each channel should keep.*

The vendors' own vocabularies show that this is not one platform's idea. The table aligns four of them; the alignment is the author's, none of the vendors publishes such a mapping.

Concept	Shopify (developer docs, n.d.)	commercetools (API docs, n.d.)	Adobe Commerce (docs, updated 2026-08)	SAP Commerce (2022 API docs)
Physical units at a location	`on_hand`	`quantityOnStock`	stock quantity	actual stock level per warehouse
Promised to placed orders	`committed`	reserved, implicit	reservations against salable quantity	reserved via `reserve()`
Sellable now	`available`	`availableQuantity`	salable quantity per sales channel	available amount
Held back deliberately	`safety_stock`, `damaged`, `quality_control`	none built in	none built in	none built in

Concept	Shopify (developer docs, n.d.)	commercetools (API docs, n.d.)	Adobe Commerce (docs, updated 2026-08)	SAP Commerce (2022 API docs)
Location	location	supply channel	source, grouped into stocks	warehouse

Shopify's developer documentation (n.d., accessed November 2026) states the arithmetic plainly: on hand "equals the sum of inventory quantities that are in the available, committed, reserved, damaged, safety_stock, and quality_control states", and incoming is not part of on hand. Its POS gained "On hand" and "Unavailable" states on 9 February 2023, with the stated aim of reducing accidental overselling. And on 5 August 2026 Shopify moved the holds for draft orders and transfers from `reserved` to `committed`, leaving `available` and `on_hand` unchanged: an integration that subtracted `reserved` in its own calculation changed behavior that day without any code change. Even the meaning of a stock state can move under the systems that read it (Promises Between Systems: API Contracts, Webhooks and Versioning).

The warehouse owns on hand. The order-taking system owns committed. Nobody should own "available", because it is a computation. Most oversells begin when a system stores "available" as a fact and lets another system overwrite it.

When a Unit Becomes Promised: Reservations and Allocations

The platforms disagree about *when* a unit leaves the available number: when it goes into a cart, when the order is created, or when the warehouse allocates it to a pick. This is a business choice with a cost on each side, and the vendors expose it as configuration.

commercetools lets each cart choose an inventory mode (API documentation, n.d., accessed November 2026): `TrackOnly` lets an order be created "even if the Inventory Entry quantity is zero or negative"; `ReserveOnOrder` reserves line items "only for the duration of the ordering transaction" and returns an out-of-stock error if a line is gone when the order is created; `ReserveOnCart` reserves units at add-to-cart, releases them on removal and expires them after a configurable 1 to 44,640 minutes. The same documentation states that cart- and order-driven quantities are eventually consistent with the internal availability used for reservations, with a delay of up to 10 seconds. Adobe Commerce reserves "when submitting an order", creates a compensating reservation on cancellation and deducts from the specific source when the order ships (documentation, updated 20 August 2026). Shopify commits units at order creation and

keeps them committed while an order is on hold (help center, n.d.).

Moment the unit is promised	Protects	Costs	Fits
Add to cart (reserve-on-cart, with expiry)	The shopper's promise: what is in the cart will be there at checkout	Stock tied up in abandoned carts; expiry and release logic	Scarce-stock drops, limited editions
Order creation (reserve-on-order; the default in all four platforms above)	Stock utilization: only real orders claim units	"Sold out" at the last click; a window of seconds in which two checkouts can both succeed	Most merchants, most of the time
Allocation at fulfillment (the warehouse claims units when it releases the pick)	Warehouse flexibility: late sourcing, substitutions	Orders accepted that cannot be fulfilled; the oversell surfaces at the pick	Wholesale and backorder flows where confirmation comes later

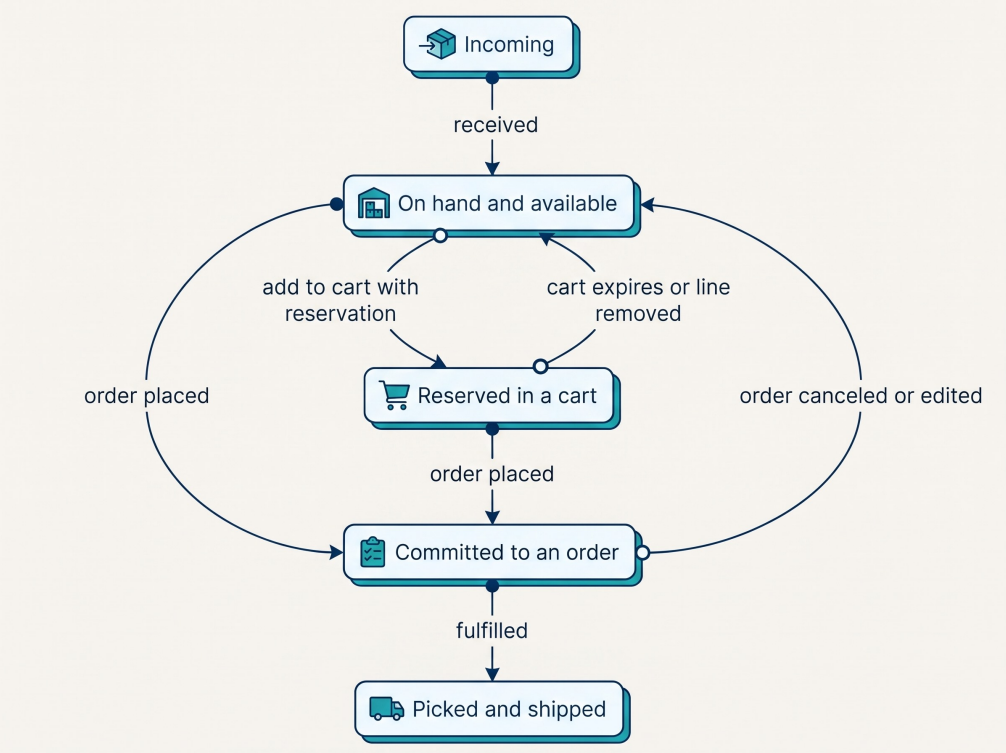


Figure 3: *A unit's life from the merchant's point of view: it is available until a cart reserves it or an order commits it, returns to available if the cart expires or the order is canceled, and leaves the count when it ships.*

commercetools' 10-second window is small, published and honest. It is also enough to oversell a hot item during a drop: at Kestrel & Pine's peak rate of 38 orders a minute, 10 seconds is six orders. **Every platform has such a window**; **most do not publish it**. Ask.

Many Locations, Many Channels

Everything above assumes one shelf and one shop. Kestrel & Pine has neither. Its stock sits at Duiven, in six stores of about 1,800 variants each, and in a marketplace's fulfillment program warehouse in Germany, which holds about 40% of the units it sells on that marketplace there. Two new concepts enter.

A **location** is any place from which stock can be promised and shipped: an own warehouse, a store, a third-party logistics provider (**3PL**, a company that stores and ships on the merchant's behalf), or a marketplace's fulfillment center. Each location has its own on-hand count and its own counting system. The marketplace's warehouse is the hardest case: the merchant sees the count the marketplace reports but cannot count it, so it must be modeled as an external location with its own buffer.

A **channel** is a place where a promise is made: a storefront, a marketplace listing, a store till, the B2B portal (business-to-business: the wholesale portal), a click-and-collect option (order online, pick up in a store). Availability is published per channel, and one physical pool may be promised to several channels at once. Three rules follow, all industry practice rather than any vendor's doctrine.

Availability per channel, with a buffer per channel. A marketplace oversell is not just a cancellation; it is a seller-performance penalty and, repeated, a suspended listing. So marketplaces usually get the largest safety buffer, so that the last units are sold where an oversell costs least. The buffer costs margin on the units it withholds, which is why it is a business setting, not an engineering constant.

Figure 4: *One pool, many promises: every channel reaches into the same stock, so availability is published per channel, with the largest buffer where an oversell costs most.*

Sourcing is a separate decision. Once a line can be fulfilled from more than one location, someone must decide which. Adobe Commerce ships two source-selection algorithms, source priority (a configured list) and distance priority (nearest source to the shipping address), per its documentation updated 20 August 2026. Shopify assigns groups of lines to a location through a *fulfillment order* (developer docs, n.d.). The moment sourcing needs capacity, cut-off times, carrier cost or split-shipment limits, the decision moves to an order management system, The Order Record: Lifecycle, State and

the Order Management System's subject.

Wholesale needs allocations, not just availability. A wholesale buyer who ordered 240 jackets for spring should not compete with Friday-evening shoppers for them. An **allocation** reserves part of the pool for a customer, a season or a channel in advance, so the storefront sees on hand minus committed minus allocations. None of the four platforms above holds per-customer allocations; they are order management territory (Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals covers the commercial side).

Two further cases promise stock that is not on hand. A **backorder** accepts an order for a unit that is out but expected; a **pre-order**, for a unit not yet released. Both are legitimate only if the incoming quantity is tracked and the date is honest; Directive 2011/83/EU, Article 18, requires delivery within 30 days unless agreed otherwise, so an open-ended backorder is a legal risk too. **Inventory valuation**, what the stock is worth in the books, stays in the ERP; it needs cost prices and accounting rules the commerce side should never hold (The Back Office: ERP Integration and Who Is the Hub).

How the Pieces Fit: The Availability Service

With several locations and channels, the clean arrangement separates three responsibilities and connects them with events. A **stock event** is a message sent the moment a stock quantity changes, saying which stock-keeping unit (**SKU**, one sellable variant such as the Ridgewalker women's M in moss), at which location, changed by how much, why, and when. A **queue** holds events until each consumer has processed them.

Component	Responsibility	Writes	Never writes
Stock-counting systems (the WMS at Duiven, the POS in each store, the marketplace fulfillment program's reports)	Count physical units; emit stock events on receipt, pick, adjustment, count	On hand per SKU per location	Committed, available

Component	Responsibility	Writes	Never writes
Order-taking systems (the platform, the marketplace connector, later the order management system, OMS)	Commit units at order creation; release on cancellation; emit order events	Committed per SKU	On hand
Availability service	Hold the latest on hand and committed per SKU per location; apply reservations, allocations and per-channel buffers; compute and publish available per channel	Available per channel (a derived, republished value)	On hand, committed
Reconciliation job	Compare each counting system's on hand and each order system's committed with the service's copies; report differences	A variance report	The numbers it compares

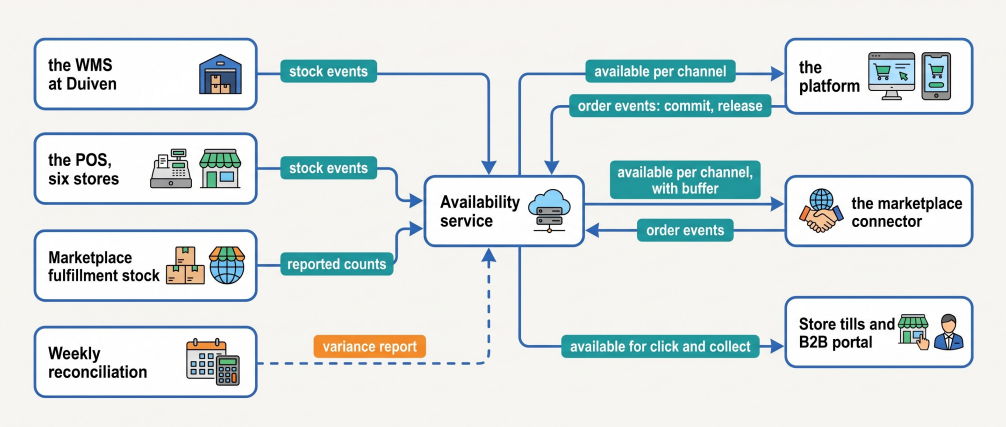


Figure 5: *The Waypoint target: counting systems send stock events, order-taking systems send commitments, one availability service derives available per channel and publishes it back, and a reconciliation job checks the copies against the counts.*

This is Waypoint decision W1 drawn as a diagram. The WMS is the system of record for physical stock at Duiven (the **system of record** is the one system whose value is the truth when copies disagree; Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record owns the idea). The POS is the system of record for store stock. The marketplace's program stock is a tracked external location. The availability service is bought as a module of the OMS Kestrel & Pine introduces under W2, not built, and the hourly chain is retired by Q4 2027.

Requirements per interaction

Different moments need different freshness, and over-specifying is as expensive as under-specifying. The targets below are the author's illustrative figures, not Kestrel & Pine facts.

Interaction	How fresh	How consistent	How fast	If the service is down
Product listing page, "In stock" badge	Seconds to a minute behind the service; stale is tolerable	Per-channel available, cached	Served from the storefront's cache; under 50 ms *(assumption)*	Show the cached value; never block the page

Interaction	How fresh	How consistent	How fast	If the service is down
Add to cart	Seconds	Check against the service's current available	Under 200 ms *(assumption)*	Allow the add; re-check at checkout
Checkout, order creation	Now	Commit atomically; the last unit goes to exactly one order	Under 500 ms *(assumption)*	Degrade to the platform's own committed count with a small buffer; alert
Marketplace stock push	Minutes; the marketplace sets the cadence	Available minus the marketplace buffer	Batched per rate limit	Push the last known value; widen the buffer
Click-and-collect at a store	Minutes behind the store till	Store on hand minus store commitments	Under 1 s *(assumption)*	Fall back to "call the store" for that store only

The checkout row carries the design. It is the only place where the last unit must go to exactly one order, so it is the one place that must call the service synchronously (ask and wait) and where the commit must be atomic (the order is created and the unit committed, or neither). Everything else can read a slightly stale copy. **Consistency where money changes hands**; **eventual consistency everywhere else**. Eventual consistency, in plain words, is the promise that if updates stop, all copies will agree soon, while accepting that for a short time they may differ; Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation owns the idea.

Events versus periodic sync

The difference between a stock event and an hourly sync is not only speed. A sync copies a snapshot, which must be applied carefully into a system that has its own claims (committed units). An event describes a change: it can be applied as a delta, kept in order per SKU and location, and deduplicated by its id.

Public examples show both ends. Shopify polls a fulfillment service's stock endpoint for on hand per SKU hourly (developer docs, n.d., accessed November 2026). Picqer, a Dutch

warehouse system for small and mid-size merchants, documents a `products.free_-stock_changed` webhook "to support real-time stock updates in a store" (n.d., accessed November 2026); a **webhook** is a push notification the source sends to a receiver's URL when something changes. Microsoft's inventory sync between Business Central and Shopify (documented 14 October 2026) is the careful middle: it compares "Shopify Stock" with "Last Calculated Stock" per variant and location and sends an update only when they differ.

At group scale the same design appears with industrial tooling. A 2020 account of Walmart's inventory backend, by its then director of engineering on Confluent's blog (4 May 2020; the author's views, not Walmart's endorsement), describes more than ten event sources normalized into one canonical inventory event, where "Each item-store combination always maps to one specific partition, so only one consumer processes it". The account is six years old; the idea, one ordered stream of changes per SKU and location, is the point.

The Choices

A growing merchant has four realistic ways to compute availability. Each is right for a stage, and each becomes a limit at the next.

1. **The platform's own stock**. The platform's inventory fields are the system of record for on hand, committed and available; apps, a same-vendor POS and a 3PL connector write to it. This is how Kestrel & Pine ran in 2016–2018 with its Tiel 3PL, and how most merchants start. It works while there is one location and the platform is the only system that takes orders.

2. **The ERP or WMS as the stock master, with periodic sync**. An ERP or WMS counts on hand; a connector copies it to the platform and from there to channels on a schedule. This is Kestrel & Pine's January 2027 landscape and the most common arrangement at the Growing stage, because every ERP and WMS vendor ships such a connector. Its limit is structural: the copy is a snapshot, the schedule is a lag, and each hop adds its own clock. The Oversell is what it does at peak.

3. **An event-driven availability service**. Counting systems emit stock events; order systems emit commitments; one service derives available per channel and publishes it. This is W1. It can be bought (a module of an OMS, a feature of an enterprise platform, a standalone inventory service) or built (a small service over a queue). The cost is integration work on every counting and ordering system, and the discipline of one writer per state.

4. **A fulfillment provider's shared pool**. The merchant hands physical stock to a logistics provider that promises it to several channels from one pool. Zalando's ZEOS announced in November 2024 that it would fulfill most of NEXT's continental-European direct-to-consumer orders through "a unified stock pool and an advanced

order management system"; by March 2026 it reported 24 markets and 18 sales channels (letter to shareholders, 12 March 2026). This moves the availability problem to the provider, at the price of the merchant's control over the count and the buffer.

Trade-offs

Criterion	1. Platform stock	2. Stock master plus sync	3. Availability service	4. Provider pool
Lag between a sale and the other channels knowing	Seconds (one system)	Minutes to hours, multiplied by hops	Seconds	Seconds within the provider; the merchant's own channels depend on the provider's feed
Number of writers of "available"	Platform plus apps (the risk)	Every connector in the chain (the risk)	One	One, not the merchant
Per-channel buffers and wholesale allocations	Basic or none	Usually none	Yes	The provider's rules
Integration cost	Low	One connector per hop	One event feed per counting and ordering system	One contract; high exit cost, the stock is physically there
When it breaks	Two apps write the same number	Peak day; a failed sync run	A lost or duplicated event; a stuck queue	The provider's outage or policy change

My rule: **stay with the platform's stock until a second system starts taking orders**

or counting stock. The day that happens, write down which system owns on hand and which owns committed, before building the first connector. Move to an availability service when any two of these are true: more than one location, more than one order-taking channel, promises that depend on location (click-and-collect, ship-from-store), or wholesale allocations. Kestrel & Pine met that test by 2023, with its own warehouse, stores, storefronts, marketplaces and wholesale, and built the sync chain instead; W1 is the overdue correction. A provider's pool fits a brand selling mainly through that provider's channels, not a merchant whose own storefronts and stores carry most of the revenue.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs the approach is outgrown
1 — Starting (Kestrel & Pine 2016–2018)	The platform's own stock fields; a 3PL or back room counts through the platform's app; reserve-on-order	A weekly count against the platform; at most one app with permission to write inventory	A 3PL or an accounting system starts "correcting" the platform's number by hand
2 — Growing (2019–2022)	An ERP or 3PL system counts on hand; a vendor connector syncs it to the platform on a schedule; the first marketplace gets a copy of the copy	A written owner for on hand and for committed; a daily reconciliation; a safety buffer on the marketplace	Out-of-stock cancellations cluster in busy weeks; two systems can both write inventory; the first "why did the website not know"

Stage	What is typical	What becomes necessary	Warning signs the approach is outgrown
3 — Omnichannel (2023–, now)	Own warehouse on a WMS, stores on a POS, a marketplace program holding stock, several channels; the chain of copies	One availability service (bought or built) fed by stock events; per-channel buffers; wholesale allocations; reconciliation weekly, daily at peak	An oversell at peak; click-and-collect checked by phone; apps writing stock; sync lag in hours; cycle counts finding double-digit variances
4 — Group	Several brands or countries, several warehouses, possibly several WMS vendors after acquisitions; a provider's pool for some markets	A shared availability service with per-brand segmentation; one ordered event stream per SKU and location; capacity-aware sourcing in the OMS	Brands competing for one pool with no allocation rules; the availability service becomes a bottleneck shared by every storefront

The approach changes with scale more than most topics in this book, because stock is the fact the most systems copy. What does not change: on hand is counted by one system per location, available is derived, and the last unit at checkout is committed atomically. A Starting merchant gets all three from the platform for free; an Omnichannel merchant must arrange them deliberately.

In Practice

The dated vendor evidence is above: Shopify's states added in 2023 and re-labeled in 2026, commercetools' 10-second window, Adobe's reservations and sourcing rules (2026), Business Central's compare-then-send sync (2026), Walmart's partition per item and store (2020). One more finding belongs here. **The counted fact is itself often wrong**. DeHoratius and Raman examined nearly 370,000 inventory records from 37 stores of one US retailer and found 65% inaccurate (*Management Science*, April 2008). The study is old, pre-dates RFID and concerns one retailer; use it as the field's classic finding, not a current rate. Later reviews cite 20% to 67% across studies. That is why

cycle counting (counting a small part of the stock on a rolling schedule, so errors are found continuously) and safety buffers exist.

Kestrel & Pine, worked

The reconstruction gave Noor three causes, and W1 answers each.

Cause 1: lag by design. Three hops, each on its own clock, up to two hours end to end. At 38 orders a minute, one hour of blindness is 2,300 orders, any of which could contain a Ridgewalker. W1 replaces the chain with stock events from the WMS to the availability service, carried by the iPaaS (the integration platform as a service that runs Kestrel & Pine's integrations) instead of passing through the ERP; the service republishes available to the platform and the connector within seconds. Of the 1,340 oversold units, about 790 were storefront orders and 550 marketplace orders (illustrative split); the marketplace share carried the penalties.

Cause 2: several writers. Six writers could change Duiven stock: the WMS feed (reaching the platform through the ERP's hourly sync), hand adjustments in the ERP, hand edits in the platform admin and three apps. Four of them were automated flows into the platform: the hourly sync and the three apps. W1 names one writer per state: the WMS writes on hand for Duiven, the POS for each store, the platform and the connector write committed, the service writes available. The apps lose their inventory permission; the pre-order app, if it survives the app review of Apps, Plugins and Functions: Extensibility and Ecosystem Governance, gets a pre-order allocation in the service instead.

Cause 3: no buffer, no reconciliation. Every channel sold down to the last unit, and nobody compared the WMS with the platform until the first weekly reconciliation in May 2027 found 2.3% of active variants with a different quantity, most left behind by the apps. After W1 the service holds a per-channel safety buffer, illustratively 3 units per SKU for marketplaces and 1 for the storefronts, and reconciliation runs weekly, daily in November.

Had W1 existed on 27 November 2026: the pick at 19:40 emits an event; the service recomputes the Ridgewalker's availability within seconds; the storefront badge flips to sold out and the connector pushes zero to the marketplaces on its next run, minutes later rather than two hours. The marketplace buffer would have pulled the listing a few sales before the shelf emptied. The oversell shrinks from 1,340 units to the handful that slip through the marketplace's own push interval. That is not zero. **No availability design reaches zero oversells; the goal is to make the window seconds wide and the buffer deliberate**.

When It Goes Wrong

Stock integrations fail in a small number of recurring ways. Each has a detection and a recovery.

Failure	What it looks like	Detection	Recovery
Duplicated stock event	The same pick applied twice; on hand too low; a sellable unit shown as sold out	Event id seen before; reconciliation variance in the "too low" direction	Idempotent handling: store processed event ids per SKU and location and skip repeats (the rule of Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation)
Out-of-order events	A receipt applied before the pick that preceded it; a brief negative or inflated count	Event timestamps or sequence numbers decreasing	Order events per SKU and location (one partition per key); reject events older than the last applied; re-fetch the current count from the counting system

Failure	What it looks like	Detection	Recovery
Snapshot overwrite	A periodic sync sets the platform's *available* to the warehouse's on hand, erasing the platform's own commitments; every sync run "un-commits" the hour's sales	Oversells that cluster right after sync runs	Set on hand, never available, from the outside, and let the platform derive available (Shopify's `inventorySetQuantities` can set either; set `on_hand`)
Two writers	The App That Wrote Stock: phantom reservations on 600 variants for two weeks	An app inventory with write-inventory permissions; variance that follows app activity	One writer per state; remove the permission; reconcile
Counting system outage	The WMS is down or its queue is stuck (the four-hour backlog); available goes stale	Queue depth; lag between order creation and WMS acknowledgment	The service keeps committing from order events and widens buffers; replay when the WMS returns
Identifier mismatch	`RW-M-MOSS` in the WMS and `rw-m-moss` in the platform; stock lands on the wrong or no variant	Events rejected or applied to unknown SKUs	One identifier per SKU across systems, mapped once (One Customer, One Product: Identity and Master Data Across Systems); Shopify's documentation warns that SKU strings are case-sensitive

Failure	What it looks like	Detection	Recovery
The count is wrong	The shelf has 3, the WMS says 5; the third order does not ship	Cycle counts; short picks	Cycle counting; safety stock sized to the measured error rate

One entry recurs in the detection column: the reconciliation job. **A stock integration without a reconciliation job is not finished**, because every delivery mechanism a merchant will meet (a webhook, a queue, a connector's retry) delivers at least once and sometimes not at all. Shopify's webhook documentation says delivery "isn't always guaranteed" and tells apps to "use reconciliation jobs to periodically fetch data" (n.d., accessed November 2026). Waypoint W5 makes the reconciliation job a condition of every integration.

What to Take Away

Established.

- On hand is a counted fact owned by one system per location; committed and reserved are claims; available is a derivation. All four vendor models in this chapter separate them, under different names.
- When a unit becomes promised is a configurable business choice (cart, order, fulfillment); the platforms default to reserve-on-order with a small eventual-consistency window, which commercetools publishes as 10 seconds.
- Counted stock is itself often wrong (65% of records in the 2008 DeHoratius and Raman study; 20–67% across later studies), so cycle counting and safety buffers are standard practice, not pessimism; and delivery of stock changes is at least once at best, so vendors instruct receivers to deduplicate and reconcile.

Depends on context.

- Keep stock in the platform while it is the only order-taking and counting system; this fails the day a 3PL, an ERP or a second channel arrives.
- A periodic sync from an ERP or WMS is acceptable at the Growing stage if the interval is minutes, the sync sets on hand rather than available, and a daily reconciliation exists; it fails at peak rates where an hour of lag is thousands of orders.
- Reserve-on-cart with short expiry is right for scarce-stock sales and wrong for ordinary catalogs, where it ties stock to abandoned carts.

- Buy the availability service as a module of an OMS when the OMS is coming anyway (Kestrel & Pine's case); build a small one when there is a strong integration team and no OMS on the horizon; use a provider's pool when the business sells mainly through that provider.

The author's synthesis.

- "Who owns the stock number" must be answered in writing before the first connector is built, because every oversell I have seen reconstructed came down to two writers or a copy on a schedule, never to a platform that could not count.
- Size the per-channel buffer from the measured count error and sync lag, and revisit it before every peak; an unmeasured buffer costs margin on quiet days and still oversells on busy ones.

Questions to Ask

1. Which system is the system of record for on hand at each location, and which system is the only writer of committed?
2. Which systems, apps and scripts can write inventory, and who owns each one?
3. At what moment does a unit leave "available": add to cart, order creation or fulfillment? What is the platform's documented consistency window?
4. How does a sale on one channel reach the others, by event or by schedule, and how long does it take at peak rate?
5. Does the outside feed set on hand or available? If available, what happens to the platform's own commitments on each run?
6. What is the safety buffer per channel, who set it, and from what measurement?
7. When did the last reconciliation between the counting system and the platform run, what variance did it find, and who fixed it?
8. What happens to checkout if the availability service is down for ten minutes?

Eva's jacket is in stock, and the unit she is about to buy will be committed to her order and to nobody else's. The next part follows that click: the cart as a draft, the checkout as a timed orchestration of every promise so far, and the payment that turns a promise into an order. Continue with Part III — ORDER: Cart, Checkout, Payment and the Order Record and From Cart to Confirmation: Checkout Orchestration.

Sources and Evidence

Company, product and investor sources

- Inventory management apps: inventory quantity names and arithmetic, Shopify developer documentation, n.d., accessed 2026-11 — the eight quantities, the on-hand sum, `inventorySetQuantities`.
- Inventory states, Shopify Help Center, n.d., accessed 2026-11 — the five merchant-facing states and their arithmetic.
- New On hand and Unavailable inventory states in POS, Shopify changelog, 2023-02-09 — dating of the states and the stated aim of reducing oversells.
- Draft order and transfer shipment inventory is moving from reserved to committed, Shopify developer changelog, 2026-08-05 — a stock state's semantics changing under integrations.
- Build for fulfillment services, Shopify developer documentation, n.d., accessed 2026-11 — hourly stock polling and case-sensitive SKUs.
- Order management apps: fulfillment orders and FulfillmentOrder object, Shopify developer documentation, n.d., accessed 2026-11 — assigned location and when it may change.
- Order status, Shopify Help Center, n.d., accessed 2026-11 — committed stock on held orders.
- Webhooks, Shopify developer documentation, n.d., accessed 2026-11 — delivery not guaranteed; reconciliation jobs.
- Inventory (InventoryEntry), Carts (InventoryMode) and Inventory APIs overview, commercetools documentation, n.d., accessed 2026-11 — quantities, the three inventory modes, reservation expiry, the 10-second window.
- Introduction to Inventory Management (updated 2026-08-14) and Source algorithms and reservations (updated 2026-08-20), Adobe Commerce documentation — sources, stocks, salable quantity, reservations, sourcing algorithms.
- StockService, SAP Commerce 2205 Javadoc, SAP, 2022 — reserve, release and actual stock level per warehouse; 2022 documentation, the model is long-lived.
- Synchronize inventory (Business Central and Shopify), Microsoft Learn, 2026-10-14 — compare-then-send sync with timestamps.
- Picqer product and API pages, Picqer, n.d., accessed 2026-11 — the `products.free_stock_changed` webhook.
- Walmart's real-time inventory management using Kafka, Suman Pattnaik on Confluent's blog, 2020-05-04 — one partition per item and store; a six-year-old account of one company's design.
- ZEOS becomes fulfilment partner of NEXT in continental Europe, Zalando SE, 2024-11-21, and Letter to shareholders 2025, Zalando SE, 2026-03-12 — the shared stock pool; 24 markets and 18 channels.

Standards, regulation and official documentation

- Directive 2011/83/EU on consumer rights, Article 18, Publications Office of the EU, 2011 — delivery within 30 days absent agreement.

Research and practitioner sources

- Inventory Record Inaccuracy: An Empirical Analysis, Nicole DeHoratius and Ananth Raman, *Management Science* 54(4), 2008 — 65% of about 370,000 records inaccurate at one retailer; the classic finding, pre-RFID.
- Open-access review citing inventory inaccuracy rates of 20–67%, *Production* journal, 2018 — range across empirical studies.
- The alignment of vendor vocabularies and the grouping of oversell causes in this chapter are the author's synthesis, not any vendor's.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Part III — ORDER: Cart, Checkout, Payment and the Order Record

IN THIS PART, YOU WILL: Follow one cart through checkout, payment, the order record and the books, and learn which system owns each step and how the others find out what it did.

KEY POINTS:

- **Checkout is the moment the shop stops promising and starts owing**. The cart is a draft that reserves nothing; checkout asks pricing, promotions, tax, availability, shipping and payment for a final answer and creates the order exactly once, before the bank has replied. It owns none of the facts it writes.

- **From then on, the truth about the order arrives by message from systems the merchant does not control.** The payment provider's result, the warehouse's shipment confirmation and the carrier's scan come late, out of order and sometimes twice. So the order keeps separate states for money, goods per parcel and returns, every message carries a key, a repeat changes nothing, and a daily job compares the copies.
- **The order ends as documents with identifiers, each issued by one system.** One writer per action that moves money, one invoice sequence per legal entity, a credit note for every correction, and a monthly match of orders, payouts and the bank.

FAILURE PATTERNS THIS PART ADDRESSES:

- **The Message in a Bottle** — a payment result or a shipment confirmation sent once and trusted, so an order is paid twice, never paid, or "delivered" while half of it is still being picked.
- **Schrödinger's Order** — the platform, the warehouse and the finance system each held an honest status of a different object, and nobody held the order's.
- **The Boomerang** — two systems could refund the same return, neither sent a key the provider could recognize, and the two paths met only in the ledger, a month later.

Part II was everything the shopper sees before the cart, each item a promise. Part III is **the moment the promises become an obligation.** In a few minutes the shop asks every system that made a promise for a final answer, takes the shopper's money, and writes one record that the warehouse, the carrier, the service desk and the accountants will all read and change. From then on the merchant owes a parcel, owes the state its tax, and owes its own books an explanation of every euro.

What makes this part different from the catalog is where the truth lives. A price is decided inside the merchant's own systems. Whether a payment succeeded is decided by a **payment service provider** (PSP: the company that offers the payment methods, collects the money through the banks and pays the merchant one net amount a day) and told to the shop by a **webhook**, an HTTP call to the merchant's server that can arrive late, out of order or twice. Whether a parcel left is decided by the warehouse; whether it arrived, by the carrier. The order is the one record every system touches, and it is held in several places at once unless someone decides otherwise. Kestrel & Pine, the fictional outdoor brand this book follows, had not decided in 2026.

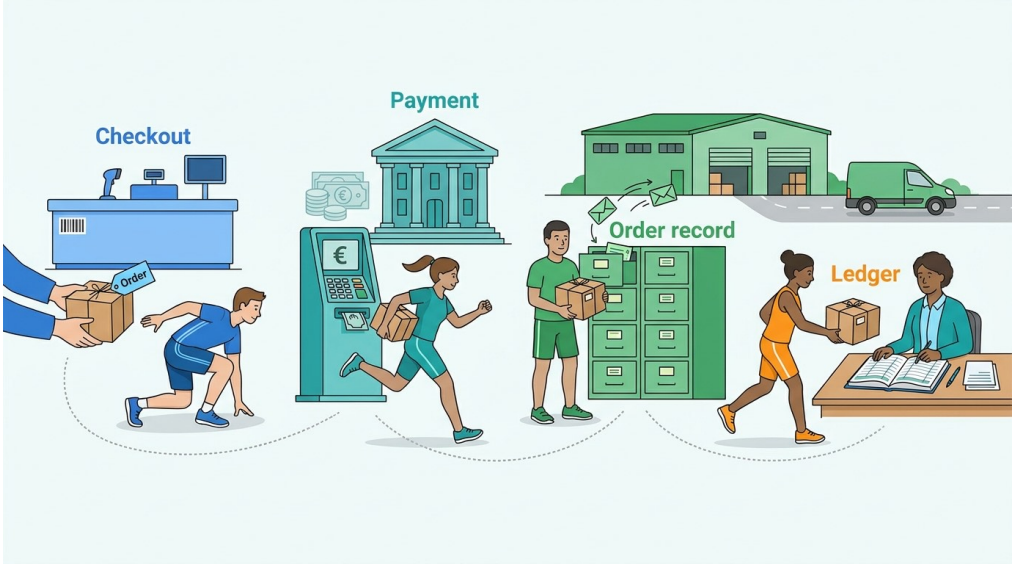


Figure 1: *One order passes through four owners in Part III: checkout creates it, the payment provider settles it, the order record holds it, and the ledger ends it.*

Where we are in the order's journey. Eva's jacket, the recurring order of Meet Kestrel & Pine, enters this part at 21:09 on Tuesday 13 April 2027 as a cart holding a Ridgewalker jacket and two pairs of socks, and leaves it as an invoice posted on 14 April and a payout batch matched at the May close. In between it becomes order #1072455 at 21:14 with its payment pending, is marked paid by the PSP's webhook seconds later, and is handed to the Duiven warehouse; under Waypoint, Kestrel & Pine's 2027 program, it will live in an order management system (OMS), which holds every channel's orders in one record. Picking, delivery and the exchange are Part IV.

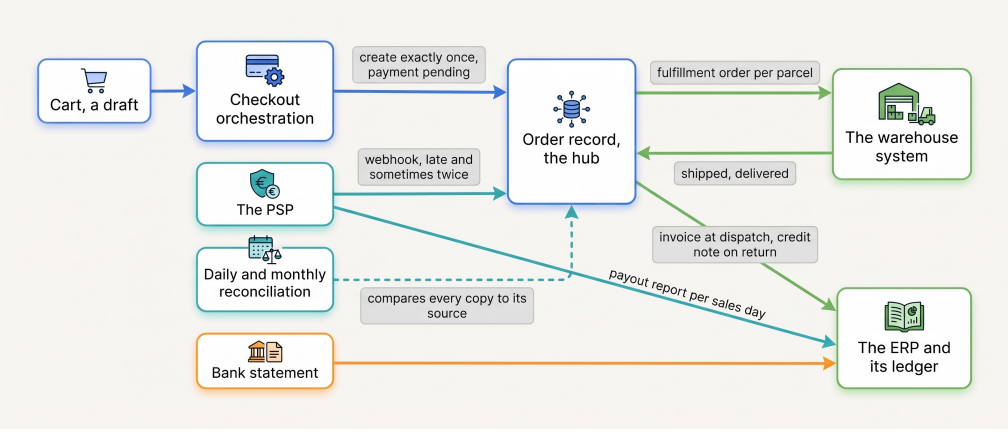


Figure 2: *The shape Part III recommends: checkout creates the order once; the order is one hub fed by messages from the payment provider, the warehouse and the carrier; the*

ledger is fed by documents, not events; and a reconciliation compares orders, payouts and the bank on a schedule.

Chapters in this part

From Cart to Confirmation: Checkout Orchestration decides **what checkout must call, in what order, and how it creates the order exactly once**: checkout is a thin, clocked orchestration that owns no price, stock or payment state; the order is created keyed on the cart and the payment, before the bank's answer, and settled by the webhook or a daily job, never by the thank-you page; and platforms withdraw the freedom to edit checkout on purpose, as Shopify's dated timeline from 2023 to 2026 shows. The case is Eva's checkout step by step, from 21:09 to order #1072455 at 21:14, and the Card-Testing Night of August 2026: 48,000 bot attempts, €2,900 in fees, a 40-minute pause of every online sale.

Taking the Money: Payment Providers, Authorization, Capture and Refunds decides **which systems may move money, with what key, and how the order learns what the provider actually did**: money moves in four stages, authorization, capture, settlement and payout, that each method bends differently (iDEAL has no capture and no chargeback; a card authorization expires in days); the order keeps its own small payment vocabulary, moved only by confirmed events; every refund has one writer and one key; and the card number never touches the merchant's servers. The case is Eva's €235 by iDEAL, and the Double Refund of January 2027: 212 returns refunded by the returns portal and again by hand in the PSP console, €19,400 out twice.

The Order Record: Lifecycle, State and the Order Management System decides **where the order lives, which states it passes through, and when an order management system is needed**: an order is one record with several state machines, money, goods per shipment and returns, not one status word; the summary of a split order is the state of its least-advanced parcel; routing and splits are cost decisions that need an owner; and an OMS becomes the hub once two of four things are true. The case is the Three-Status Order of December 2026, #1052877: *Delivered* in the platform, *In picking* in the warehouse and *Not invoiced* in the ERP for six days, 11 tickets from one customer, and Waypoint's W2.

Tax, Invoices and the Books: From Order to Ledger decides **which system computes the tax, which issues the invoice, and how finance proves every month that orders, payouts and the bank agree**: tax comes from facts the shop owns (the ship-to address, the product's category, the customer's status) and rates it should not own; the invoice is a numbered, immutable legal document issued at dispatch of each shipment by one system per legal entity and corrected only by a credit note; and documents, not webhooks, feed the ledger. The case is Eva's order in the books, net €194.21 and VAT €40.79, and Ardennes Trail Co., the Belgian wholesale customer in Namur whose invoice travels as structured data over Peppol, the European e-invoicing network.

Begin with From Cart to Confirmation: Checkout Orchestration.

Sources and Evidence

Sources are cited in each chapter; this introduction makes no factual claim of its own. Kestrel & Pine is fictional, and its figures are the same in every chapter.

11. From Cart to Confirmation: Checkout Orchestration

IN THIS CHAPTER, YOU WILL: Follow one cart through checkout, learn what checkout must call and in which order, see why platforms protect it, and choose between four ways to run it.

KEY POINTS:

- **A cart is a draft; checkout is a promise machine.** The cart reserves nothing and promises nothing. Checkout is the clocked sequence that asks pricing, promotions, tax, availability, shipping and payment for a final answer and turns the draft into an order the rest of the platform must keep.
- **Checkout should be thin, and it must create the order exactly once.** It composes calls and holds a short-lived state; it owns no prices, stock or payment state. The shopper may leave for the bank's app and never return, the payment provider's answer may arrive late and twice, so order creation is keyed on the cart and the payment, never on a page load.

- **Platforms withdraw the freedom to edit checkout, on purpose.** Shopify spent 2023 to 2026 replacing editable checkout templates and scripts with sandboxed extension points and dated deadlines. The lesson for a merchant is to customize checkout only through sanctioned mechanisms and to plan for the day the platform changes them.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Phantom Stock** — checkout promises availability from whatever stock figure it is given; on Black Friday 2026 that figure was up to two hours old.
- **The Message in a Bottle** — the payment provider's answer is a message that can arrive late, twice or never; a checkout that trusts one page load creates orders twice or not at all.
- **The Forked Platform** — a checkout customized by patching the platform's template or injecting scripts cannot follow the platform's upgrades, and the platform will one day switch it off.

Between "add to cart" and "order confirmed" lies a window of a few minutes in which a commerce platform makes every promise it will later have to keep. The price, the discount, the tax, the "in stock" badge, the "delivered tomorrow" line and the payment itself are decided in that window, by different systems, and written into one record: the order. **Checkout is where the promise-keeping machine makes its promises.**

Most of what makes checkout hard is not in checkout. The price belongs to pricing (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax), the discount to promotions (Discounts, Coupons and Loyalty: Promotions Without Chaos), the stock figure to inventory (Is It in Stock? Inventory, Availability and the Promise You Can Keep), the delivery line to shipping (Getting It There: Carriers, Labels, Tracking and the Delivery Promise) and the money to the payment provider (Taking the Money: Payment Providers, Authorization, Capture and Refunds). Checkout's own job is to ask each of them at the right moment, in the right order, and to turn the answers into exactly one order. That sounds modest. It is the most protected code on every commerce platform, because a slow, stale or repeated answer here turns directly into lost revenue, oversold stock or a shopper charged twice.

This chapter walks the window step by step, using Eva Janssen's order at Kestrel & Pine, the fictional outdoor brand this book follows, as the worked example, then compares four ways to run a checkout and reads Shopify's 2023–2026 history for how far a merchant may customize one.

Where we are in the order's journey. Eva's jacket has been found, shown and put in a cart. This chapter covers the three minutes in which the cart becomes order #1072455 on Tuesday 13 April 2027 at 21:14. Taking the Money: Payment Providers, Authorization, Capture and Refunds takes the money from the moment the payment request leaves

checkout; The Order Record: Lifecycle, State and the Order Management System takes the order record from the moment it exists.

Setting the Scene at Kestrel & Pine: Three Requests for the Checkout

In her second month, Noor Visser gets three requests about the same page.

Milan Petrov, who owns the storefronts, wants a delivery-date line in checkout ("delivered Thursday") instead of the static "ordered before 22:00, delivered tomorrow", and a gift-wrap option. The agency proposes an app for the first and a script injected into the checkout page for the second. Milan also wants fewer abandoned carts; about seven in ten carts never become orders *(illustrative, in line with Baymard's industry average).*

Daniel Okafor, who owns the warehouse, wants to know why the checkout kept selling the Ridgewalker jacket for two hours after Duiven ran out on Black Friday evening 2026.

Figure 1: *The Phantom Stock: checkout promised availability from a stock figure up to two hours old, so it kept selling a jacket Duiven no longer had.*

Sanne de Wit, who owns finance, wants the checkout to stop creating orders the payment provider later reports as unpaid, each of which she explains at month-end, and asks why the Card-Testing Night of August 2026 cost €2,900 in fees for payments that were all declined.

Noor's first move is to ask a question none of the three had asked: **what**, **exactly**, **does**

the checkout call, and in what order? The answer is the subject of this chapter; the agency's script proposal is the subject of its second half.

The Cart Is a Draft, the Checkout Is a Commitment

Three words carry the chapter, and they are easy to blur.

A **cart** is a draft of an order that the shopper can still change: product references and quantities, perhaps a coupon code, perhaps an address. It reserves nothing: a jacket in Eva's cart is still for sale to everyone else. It promises nothing: the price next to the jacket is an estimate that will be recomputed. Carts live for days or weeks; a signed-in shopper's cart follows her across devices, and a guest cart lives in the browser until it is merged into an account at sign-in. The cart also hides the first business decision about checkout: whether to **reserve stock when an item is added**. Most platforms do not; stock is committed only when the order is created, because a reservation held by every browsing shopper would make the "in stock" figure useless. Flash sales and ticketing are the exception, and they pay for it with visible countdowns.

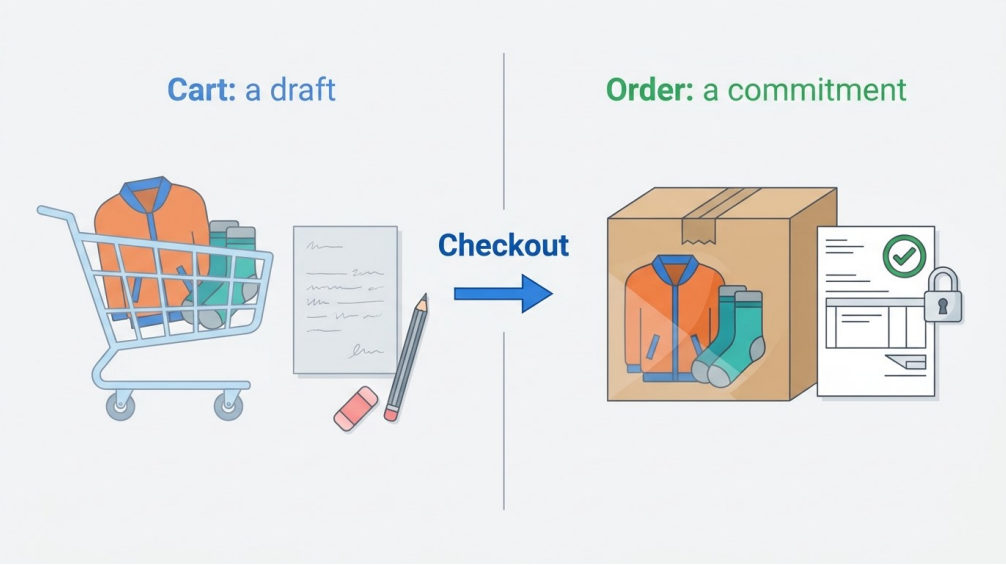


Figure 2: *The cart reserves and promises nothing; checkout turns it into an order the shop now owes.*

A **checkout** is the orchestrated sequence that turns the cart into an order. It is short (minutes), it has a clock, and it is the first moment the platform asks every other capability for a *final* answer rather than an estimate.

An **order** is the record that exists when checkout succeeds: what was bought, at what price, with what tax, delivered where and how, paid how. From that moment it belongs

to order management (The Order Record: Lifecycle, State and the Order Management System) and the checkout has no further say.

	Cart	Checkout	Order
Lifetime	Days to weeks	Minutes	Years (tax retention)
Can change	Freely, by the shopper	Only through the steps	Only by order edits with their own rules
Reserves stock	Usually not	Usually not until the final step	Yes, on creation
Price and tax	Estimates	Final computation	Frozen, with the basis recorded
Owner	Storefront or platform	Checkout orchestration	Order management

The distinction matters because every system downstream treats the order as a promise. If the checkout wrote a stale price or a stale stock figure into it, the warehouse, finance and the customer all inherit the error.

What Checkout Must Call

A checkout walks the shopper through a handful of steps, and each step is a call to a system that owns a fact. The table below is a reference-architecture view; vendor flows differ in order and in how many steps are visible, but the calls are the same. (The Baymard Institute's 2024 benchmark counted an average of 5.1 steps from cart to order review and 11.3 form fields, and argued that eight fields suffice.)

Step	What it must call	The promise it makes
Identity (guest or account)	Customer service; address book for returning customers	"We know whom to bill and where to deliver"
Address	Address validation; tax determination (the ship-to address decides the rate of VAT, value-added tax, Europe's sales tax)	"This is where it goes and what tax applies"

Step	What it must call	The promise it makes
Shipping options	Shipping-rate service or carrier rules; availability and the delivery promise	"Delivered by X for Y"
Price review	Pricing, promotions, tax engine	"You pay exactly this"
Payment	The PSP (payment service provider, the company that takes the shopper's money through many methods and pays the merchant one net amount), including authentication; fraud screening	"The money is reserved"
Confirmation	Order creation; stock commitment; notifications	"The order exists and will be kept"

Two things stand out. First, **checkout owns none of the facts it writes**. The price list lives in pricing, the rate in the tax engine, the stock figure in inventory, the delivery promise in shipping, the payment state in the PSP; checkout consumes all of them. Second, **the calls have different freshness needs**. A product description may be hours old without harm; the price should be minutes old at most; the availability figure must be current at the moment of commitment or the order is a Phantom Stock waiting to happen.

Call	How fresh	How consistent	How fast (illustrative targets)	If it is down
Price and promotions	Minutes; recomputed at the price review step	Final at order creation	Under 200 ms per cart *(assumption)*	Fail the step, never guess
Tax	Computed at price review from the ship-to address	Recorded on the order as the basis	Under 200 ms *(assumption)*	Fall back to the platform's rate table with a flag

Call	How fresh	How consistent	How fast (illustrative targets)	If it is down
Availability	Current at order creation	Committed by the order	Under 100 ms per line *(assumption)*	Degrade to "may be delayed", never to "in stock"
Shipping rates and promise	Current per address	Frozen on the order	Under 500 ms *(assumption)*	Offer the default service without a date
Payment	The PSP's answer only	Idempotent per order	Seconds, plus the authentication detour	Queue the order as pending; do not fulfil

This is why checkout is described as an **orchestration**: a coordinator that calls several services in sequence, holds a short-lived state between the calls, and completes or abandons the whole. In plain words, a conductor, not a musician.

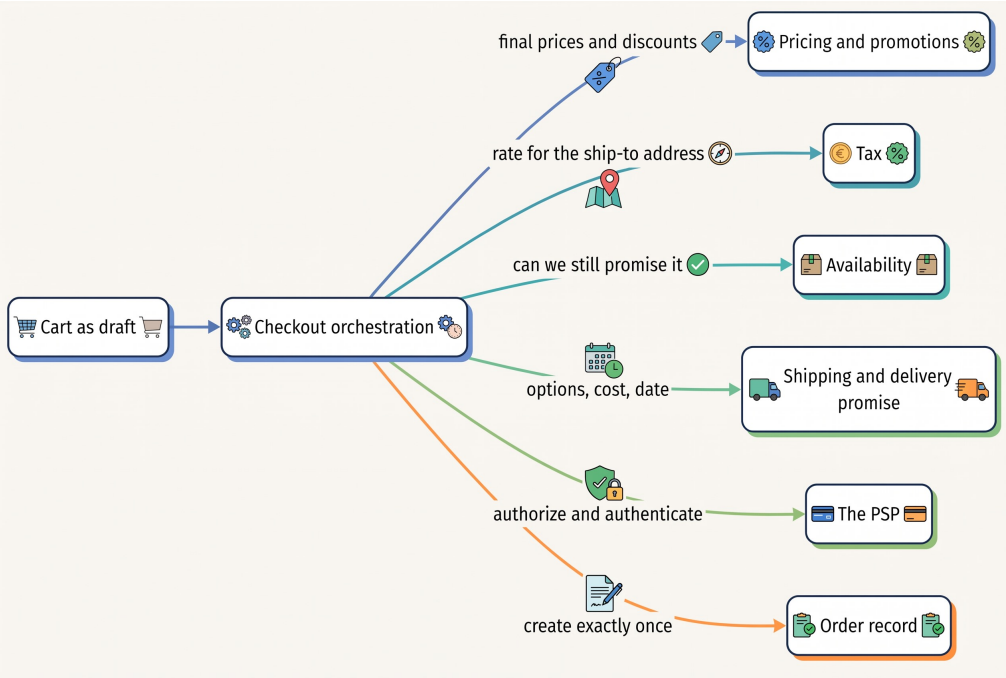


Figure 3: *Checkout owns none of the facts it writes into the order; it asks six systems for a final answer and creates the order exactly once.*

The Clock and the Detour

Checkout runs against two clocks it does not control.

The first is the **shopper's patience**. Baymard's 2026 list of abandonment reasons, from a survey of US adults who had abandoned a cart and were not merely browsing, puts "extra costs too high" at 40%, "delivery too slow" at 20%, "didn't trust the site with my card" at 19%, "forced to create an account" at 18% and "too long or complicated" at 17% (multi-select). Three of those five are architecture: showing total cost early means calling tax and shipping *on the cart*; showing a credible delivery date means calling the delivery promise; not forcing an account means the order needs a customer *reference*, not a customer *account*.

The second clock is the **payment provider's**. A payment session expires: Mollie's documentation (n.d., accessed November 2026) gives iDEAL (the Dutch method in which the shopper pays by bank transfer from her banking app) sessions 15 minutes, cards 30 minutes, PayPal 6 hours and Klarna 48 hours, and advises merchants not to predict expiry but to wait for the webhook. A card authorization, once granted, is valid for days (Stripe's and Adyen's documentation list 5 to 7 days for most card schemes). Checkout must finish inside the session; capturing the money can wait for the warehouse (Taking the Money: Payment Providers, Authorization, Capture and Refunds).

Inside the payment step sits a **detour**. For most European online payments the shopper leaves the merchant's page: to the bank's app for iDEAL, to a 3-D Secure challenge for a card (3-D Secure is the card networks' protocol for the strong customer authentication EU rules require; the shopper confirms in her banking app), to PayPal's page. She may lose her connection or never come back. Stripe's fulfillment guide states the consequence: "You can't rely on triggering fulfillment only from your checkout landing page, because it's not guaranteed customers visit that page." The platform learns the outcome from a **webhook**, an HTTP call the PSP makes to the merchant's server when something happened, and webhooks arrive late, out of order and sometimes twice (Stripe's documentation says so in as many words). Two rules follow.

Create the order exactly once, keyed on the cart and the payment. The commercetools checkout documentation (n.d., accessed November 2026) describes a flow in which the order is created with the payment transaction *pending* before the PSP's final answer; the webhook then moves it to success or failure, and the merchant should "start fulfilling an Order only if the Payment contains a Transaction" in a successful state. Stripe's guide tells the merchant to make the fulfillment function "correctly handle being called multiple times with the same Checkout Session ID". Both are the same discipline: an **idempotency key**, a unique token sent with a request so that a retry does nothing twice,

derived from the business event ("create order from cart X with payment Y"), not from a page load.

Treat the payment outcome as external truth. The order carries its own small payment vocabulary (pending, authorized, paid, failed) with the PSP's raw status beside it. Nothing in checkout decides that a payment succeeded; the PSP's signed webhook does, and a daily job asks the PSP about every order still pending after the method's expiry window, because webhooks stop retrying (Mollie after ten attempts over 26 hours, Stripe after three days). Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation owns the general pattern; checkout is its busiest application.

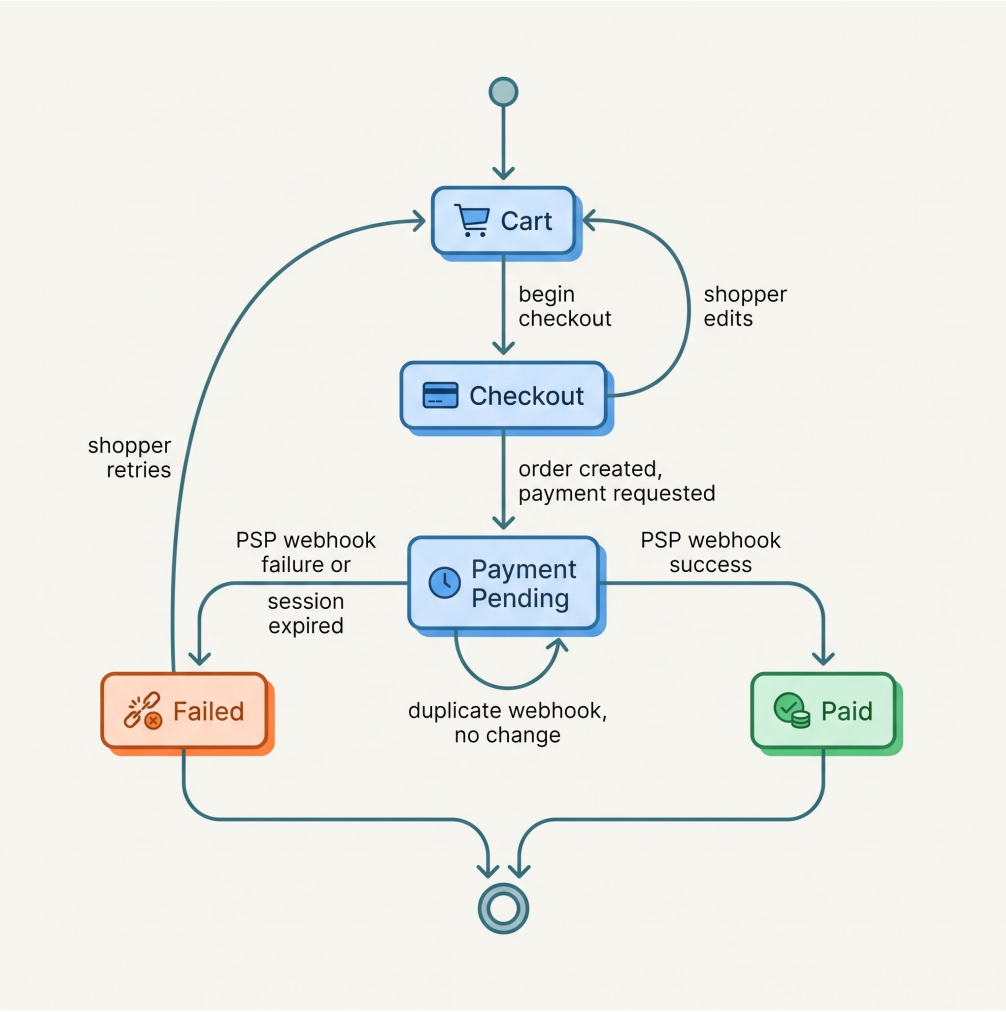


Figure 4: *The order exists before the payment outcome is known; the webhook, not the thank-you page, moves it to paid or failed, and a repeated webhook changes nothing.*

Guest, Account and the Express Wallets

Forced account creation is the fourth-largest stated reason for abandonment in Baymard's 2026 list. The architectural consequence is small and important: identity is optional at checkout and attachable afterwards. An order needs an e-mail address and a delivery address, not a login. The guest order can be claimed by an account later, one of the matching problems One Customer, One Product: Identity and Master Data Across Systems works out (Eva exists at Kestrel & Pine as an online account, a store loyalty customer, a newsletter subscriber and a returns-portal contact with no shared identifier).

Express wallets short-cut the steps. Apple Pay's developer overview (n.d., accessed November 2026) describes a purchase completed "from a product page with a single tap, without the need to add to a cart or fill out a form", with the merchant never receiving the card number. Google Pay's web documentation describes a payment sheet that returns a payment token to the merchant's server, which forwards it to the PSP. Shopify's Shop Pay stores e-mail, payment method and addresses; in Shopify's Black Friday–Cyber Monday reporting of 2 December 2025, Shop Pay carried 32% of orders across its merchants that weekend (a vendor-defined figure). The wallet supplies identity, address and a payment token in one callback. It does **not** remove the orchestration: pricing, tax, availability and shipping are still called, now from inside the wallet's callback and under a stricter clock.

Abandoned Carts and the Business Metric

Checkout has one business metric: **conversion**, the share of carts (or sessions) that become orders. Baymard's running list (updated 28 October 2026) gives an average documented abandonment rate of 70.22% across fifty studies from 2006 to 2025, with individual studies between 55% and 84%; the studies define "cart" and "abandonment" differently, so the figure is a direction, not a benchmark. Baymard's claim that fixing solvable usability issues could lift conversion by about 35% is its own estimate from usability testing, not a controlled experiment.

An **abandoned cart** is a cart started and not converted within some window. Recovering it (the reminder e-mail an hour later) has two preconditions: the cart is persisted server-side, not only in the browser, and it is attached to an identity the merchant may contact, which under EU privacy rules means a consent question (Whose Data Is It? Privacy, Consent and Retention). The architecture can do three things for conversion that no button color can: show the full price, including tax and shipping, before checkout begins (call tax and shipping rates on the cart page); show a delivery date the warehouse can keep (a computed promise, Getting It There: Carriers, Labels, Tracking and the Delivery Promise); and never fail the payment step because an internal call was slow.

The Choices: Four Ways to Run a Checkout

A merchant has four realistic options, and they map closely to the scale stages.

1. **The platform's hosted checkout, as is.** The platform renders, runs and protects the checkout; the merchant configures payment methods, shipping rules and branding. The merchant's PCI DSS scope (the card industry's security standard for anyone who touches card data) is the smallest possible, because the card fields are served from the PSP's domain and the merchant's servers never see a card number. Nothing to build and nothing to break.

2. **The hosted checkout with sanctioned extensions.** The platform exposes named places where apps may add a block, validate a cart, customize delivery or payment options or run a sandboxed function, each with a fixed input, a fixed output and a time budget. Shopify's checkout extensibility is the best-documented example (next section); commercetools' API Extensions are a synchronous variant, a call to the merchant's endpoint before a cart or order is saved, with a default two-second timeout after which the whole call fails. The merchant gets the delivery-date line and the gift-wrap option without owning the page.

3. **A custom or headless checkout on the platform's APIs.** The merchant or its agency builds the checkout pages on the platform's cart, checkout and payment APIs: full control of the experience, full responsibility for the orchestration rules above, for every script on the payment page and for keeping up with API versions (Promises Between Systems: API Contracts, Webhooks and Versioning). Card fields stay the PSP's hosted fields if the merchant wants the short PCI questionnaire.

4. **An own checkout service over composable capabilities.** The merchant runs a checkout service that calls separate pricing, promotion, tax, availability, shipping and payment services, often from different vendors, and creates the order in an order management system: the composable style of Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says applied to the most sensitive flow. It is right when one checkout logic must serve several storefronts, an app, a point of sale and a B2B portal, and it costs a team.

	1. Hosted as is	2. Hosted plus extensions	3. Custom on platform APIs	4. Own checkout service
Who runs the orchestration	The platform	The platform	The merchant, on the platform's APIs	The merchant

	1. Hosted as is	2. Hosted plus extensions	3. Custom on platform APIs	4. Own checkout service
Customization	Configuration and branding	Named extension points, time-boxed	Anything, at the merchant's risk	Anything
PCI DSS scope	Smallest (hosted page)	Smallest, if extensions cannot touch the payment fields	Small only with hosted fields; every page script is a liability	Depends on design; a service that touches card numbers is the long question-naire
Upgrade risk	None	Low: extensions are versioned	High: API versions, changed flows	Owned entirely by the merchant
Team needed	None	An app developer or agency	Front-end and integration engineers	A platform team
Fits	Starting; most Growing	Growing; Omnichan-nel on a SaaS platform	Omnichannel with a strong storefront team	Group; vendors

My rule: **take the most protected option that still gives you the two or three customizations the business needs, and get those through sanctioned extension points**. The reasons to leave option 2 are real (a checkout shared by web, app and store; a B2B flow the platform cannot express) and rarer than agencies suggest. Every freedom you take in checkout is a page you now keep working at 41 times normal load, under the card industry's rules, through every platform release.

How Protected Checkout Is: Shopify's Timeline, 2023–2026

The clearest public record of a platform taking checkout *away* from its merchants is Shopify's, and it is worth reading as a sequence of dates.

On 13 February 2023 Shopify's changelog announced that `checkout.liquid`, the template file that let Shopify Plus merchants edit the checkout's markup, would stop working for the information, shipping and payment pages on 13 August 2024; checkouts not upgraded "will automatically revert to a standard, uncustomized Shopify Checkout". The replacement, checkout extensibility, was described as "app-based, upgrade-safe, higher-converting, and integrated with Shop Pay". The thank-you and order-status pages followed: Shopify's developer documentation (n.d., accessed November 2026) records that `checkout.liquid` and "additional scripts" were sunset for those pages on 28 August 2025 for Plus stores, and its help center names 26 August 2026 as the deadline for non-Plus stores. Shopify Scripts, the Ruby customizations that ran inside checkout, were retired on 30 June 2026 according to the help center. As of November 2026, "advanced checkout customization features are available only on the Shopify Plus plan"; the mechanisms are UI extensions in defined slots, Shopify Functions (merchant logic compiled to WebAssembly and run by the platform inside the purchase flow, limited to 11 million instructions per run for carts of up to 200 lines, per the API reference for version 2026-10), web pixels for tracking and a branding API.

> **INFERENCE, NOT DOCUMENTED:** Shopify has not published its reasons beyond the changelog wording. The arc, free-form template and script access withdrawn over three and a half years with dated deadlines and replaced by sandboxed extension points with hard limits, is consistent with a platform that wants one checkout it can change under every merchant at once, keep inside the card industry's rules and keep fast at peak.

The timeline teaches three things that apply to any platform.

First, **a time budget is how a platform protects checkout from the merchant's own code**. Shopify's instruction count and commercetools' two-second extension timeout are the same idea: your logic may run inside checkout, but only for so long, and if it fails the platform decides what happens (commercetools fails the whole call; Shopify's Functions cannot call the network at all).

Second, **the payment page is where the card industry looks**. PCI DSS v4.0.1 added requirements for an inventory of scripts on the payment page and for tamper detection (6.4.3 and 11.6.1, effective 31 March 2025 per the PCI Security Standards Council's blog of 30 January 2025, which also removed them from the shortest self-assessment questionnaire in exchange for a declaration that the site is not susceptible to script attacks). The agency's gift-wrap script is exactly the kind of third-party script that rule exists for; a skimmer that compromises any script on that page reads every card typed into it (Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers).

Third, **a platform will withdraw an extension point, and it will tell you the date**. The Forked Platform pattern is usually described for customized open-source cores, but a checkout customized through a mechanism the vendor has announced it will retire is the

same pattern on a SaaS platform (software rented by the month and run by the vendor). The remedy is an inventory of every checkout customization, its mechanism, and the vendor's support clock for that mechanism (Apps, Plugins and Functions: Extensibility and Ecosystem Governance).

What Changes When the Buyer Is a Company

A business customer changes four steps and removes one. Identity is a **company account** with several contacts and roles, not a person; the buyer may need an approver before the order is final. Prices come from a **contract price list**, and the price review shows amounts *excluding* VAT. The address step collects the company's VAT identification number, and a valid number from another EU country switches the tax treatment to the **reverse charge** (the seller charges 0% and the buyer accounts for the VAT itself; Tax, Invoices and the Books: From Order to Ledger). The payment step often disappears: the order is placed on **payment terms** (net 30 days, at Kestrel & Pine) with a purchase-order number, and "payment" becomes a receivable in the ERP, not a PSP event. Shopify's Admin API (version 2026-10) types these terms as net, fixed date, on receipt or on fulfillment; Microsoft's documentation for its Business Central–Shopify connector (14 October 2026) tells merchants to decide which system owns payment terms and credit limits, because "mixing both directions without a clear ownership rule can lead to conflicting records". A **credit check** against the view of the ERP (enterprise resource planning system: the back-office system that keeps the books and each business customer's credit limit) of the customer's balance is the one call a consumer checkout never makes.

Kestrel & Pine's B2B portal runs on the platform's company accounts, price lists and net terms; 62% of wholesale orders arrive through it. Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals works the whole picture.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting (under about €2M)	The hosted checkout as is; one PSP; guest checkout on; hosted payment page	Configuration only: home-market payment methods, shipping rules, legal texts	Requests to "edit the checkout page" for features apps provide; a script in the checkout
2 — Growing (€2M–€20M)	Hosted checkout plus apps and functions: delivery options, a validation rule, express wallets; a second country brings a second VAT rate and payment mix	Customizations only through sanctioned extension points, inventoried with the vendor's support dates; tax and shipping on the cart; abandoned-cart recovery with consent	Checkout slows at peak because of an extension; a plan-tier gate blocks a needed change; two apps both write to the order at creation
3 — Omnichannel (€20M–€200M)	Checkout as a channel-independent capability: web, app and B2B portal share pricing, tax and availability; order created before the PSP's final answer and settled by webhook	One availability service behind every channel's checkout; one owner for order creation; a daily payment reconciliation job; a peak-day degradation ladder that never includes the payment step	The Black Friday Oversell; orders that exist but stay unpaid for days; the Card-Testing Night

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
4 — Group	One checkout service with per-brand configuration, or several platform checkouts behind shared pricing, tax and payment services	A governed, versioned contract between checkout and the order system, owned by a platform team; the regulatory calendar for payment and card rules	Each brand's checkout re-implements VAT and promotion rules; a payment-flow change needs five deployments

Kestrel & Pine is a stage-3 merchant running a stage-2 checkout. Waypoint does not replace it (W4, stay and surround); it changes what the checkout *calls*: from Q4 2027 the availability answer at order creation comes from the availability service (W1), and from 13 October 2027 every storefront order is handed to the OMS, the order management system that becomes the order hub (W2), while the platform keeps the checkout. The chapter's advice holds at every stage; what changes is how many systems stand behind each call.

Eva's Checkout, Step by Step

The recurring order, with every system it touches. Times inside the window are illustrative; the order time, amounts and order number are canonical. Kestrel & Pine runs its platform's hosted checkout with two sanctioned extensions added in 2027, a delivery-date block and a gift-wrap option.

Time	Step	What checkout calls	Answer written into the order
21:09	Add to cart	Nothing beyond the storefront; the cart holds the jacket (women's, size M, moss) and two pairs of socks	Cart, no reservation

Time	Step	What checkout calls	Answer written into the order
21:10	Cart page	Pricing: €199 and 2 × €18; promotions: none; tax: NL 21% included in the consumer price; shipping: free over €75	Total €235 shown before checkout starts
21:11	Identity	Eva signs in to her online account; the address book returns her Nijmegen address	Customer reference, delivery address
21:12	Shipping options	The delivery-date block turns the static rule "ordered before 22:00, delivered tomorrow" into a date: "delivered Wednesday 14 April"; PostNL home delivery, €0	Delivery promise and carrier service
21:13	Price review	Pricing, promotions and tax recomputed for the final cart and address: net €194.21, VAT €40.79, total €235	The tax basis, frozen on the order
21:13	Payment	iDEAL selected; the PSP creates a payment session; Eva is redirected to her bank's app	Payment reference, status pending

Time	Step	What checkout calls	Answer written into the order
21:14	Confirmation	Order #1072455 created with the payment pending and the three units committed in the platform; the PSP's webhook arrives seconds later with the iDEAL result and the order becomes *paid*; the confirmation e-mail is sent	The order the rest of the book follows

Three details carry the design. The order was created at 21:14 **before** the webhook confirmed the payment, so a lost redirect could not have lost the order; it would have sat as *pending* until the webhook or the daily reconciliation job settled it. A second delivery of the webhook would have changed nothing. And the stock figure consulted at 21:14 was the platform's own copy, in April 2027 still fed hourly from the ERP, which is itself fed every 15 minutes by the WMS (the warehouse management system); the same step on Black Friday 2026 sold 1,340 jackets the warehouse no longer had. One promise in the table was not computed at all: the delivery date came from a rule, not from the warehouse's last carrier collection, so the parcel arrived on Thursday 15 April, a day after the date Eva was shown (Getting It There: Carriers, Labels, Tracking and the Delivery Promise and Journey 1: Eva's Jacket, From Search to Doorstep compute the promise instead).

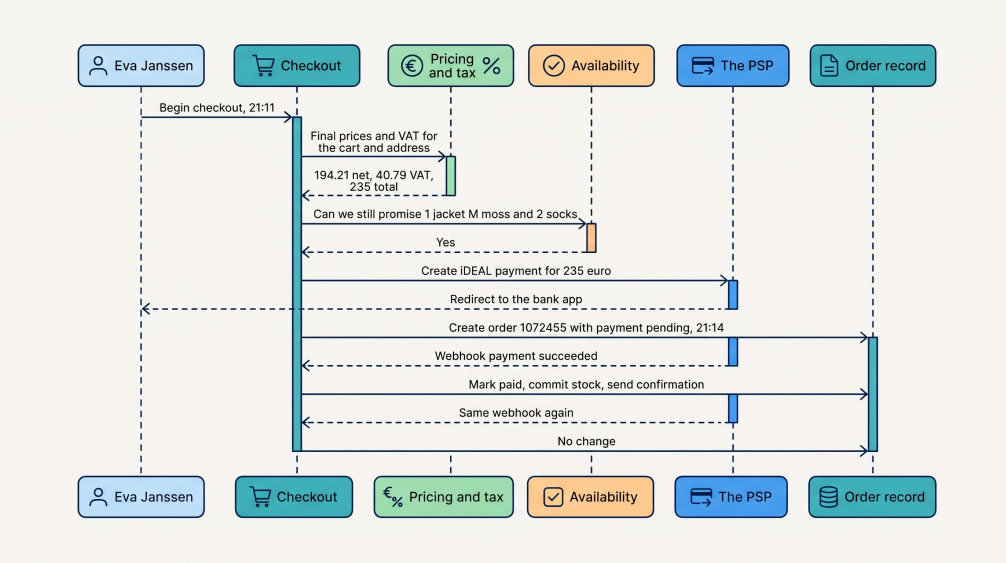


Figure 5: *Eva's checkout creates the order once, before the bank's answer, and lets the PSP's webhook move it to paid; a second delivery of the same webhook changes nothing.*

Daniel's question is now answerable. On 27 November 2026 at 19:40 the WMS knew the Ridgewalker was gone; the availability call was answered by the platform's copy, refreshed up to two hours earlier. Checkout trusted the answer it was given. The fix is not in checkout; it is in who answers the availability call, which is W1.

When It Goes Wrong

Checkout failures cluster into five kinds.

The duplicate order. A double-click, a browser retry after a timeout or a repeated webhook, and two orders exist for one cart, or one order is paid twice. Detect it with a report of orders sharing a cart id or payment reference; recover by canceling the duplicate before fulfillment and releasing the authorization rather than refunding a capture; prevent it with the idempotency key on order creation and a webhook handler that records every event id.

The lost outcome. The shopper paid and never returned, the webhook failed and the PSP stopped retrying, and the order sits pending while the money sits authorized. Detect it with a daily list of orders pending longer than the method's expiry window; recover by querying the PSP and settling the order either way. Mollie's documentation prescribes exactly this fetch-the-status pattern, and makes its webhook carry only an id so that a forged call cannot mark anything paid.

The stale answer. Availability, price or tax came from a copy that was behind. Detect it through oversold lines, price-parity warnings from marketplaces and invoices whose

VAT differs from the checkout figure. For stock, cancel or delay with an apology and a voucher (most of Kestrel & Pine's €96,000 Black Friday cost); for price and tax, honor the promise and correct the source. Prevent it with a current availability call at commitment, prices recomputed at the price review step and tax from the ship-to address.

The slow or failed dependency. An extension exceeds its time budget, a tax engine is unreachable, a shipping-rate call hangs. Watch checkout latency and error rate per step on peak day as closely as orders per minute (Black Friday Every Year: Peak Load, Resilience and Recovery), and decide the degradation ladder in advance: switch off the recommendation block, fall back to the platform's rate table with a flag for finance, offer the default carrier service without a date. The payment step and order creation are never on the ladder.

The attack. The Card-Testing Night: 48,000 bot attempts with stolen card numbers against Kestrel & Pine's guest checkout in one night in August 2026, €2,900 of PSP fees for declined authorizations and a 40-minute account pause. Stripe's documentation on card testing (n.d., accessed November 2026) names the symptoms (a spike in failed payments, nonsensical customer names) and the controls: rate limits per address, velocity rules, a CAPTCHA trigger on guest checkout and the PSP's risk tools fed with every signal the integration can send; Visa's May 2025 monitoring fact sheet tracks these enumeration attacks separately from disputes. The architectural control is a kill switch that disables guest card payments without a deploy (Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers).

What to Take Away

Established.

- The cart is a draft that reserves nothing; checkout is the orchestration that asks every promise-making system for a final answer; the order is the record that results. The three have different lifetimes, owners and rules.
- The payment outcome is external truth that arrives by webhook, late, unordered and possibly twice; an order is created exactly once, keyed on the cart and the payment, and settled by the webhook or a reconciliation job, never by the thank-you page (Stripe's and commercetools' documentation, n.d., accessed November 2026).
- Platforms protect checkout by withdrawing free-form customization in favor of time-boxed extension points; Shopify's dated timeline from February 2023 to August 2026 is the documented case.
- Surprise costs, slow delivery and forced accounts lead Baymard's 2026 list of abandonment reasons; two of the three are answered by calling tax, shipping and the delivery promise on the cart.

Depends on context.

- The hosted checkout as is, or with sanctioned extensions, is the right choice for Starting, Growing and most Omnichannel merchants on a SaaS platform; a custom checkout is justified when several channels must share one checkout logic the platform cannot express, and only with a team that can keep it current and secure.
- Guest checkout should be on by default for consumers; for business customers the account *is* the checkout (contract prices, VAT treatment, terms, approvals).
- Creating the order before the PSP's final answer is the robust pattern where redirects and asynchronous methods dominate (iDEAL, Bancontact, 3-D Secure challenges); where every method answers synchronously it adds a pending state you must still reconcile.

The author's synthesis.

- Treat checkout as a conductor, never a musician: the moment it owns a price list, a stock figure or a payment state of its own, it has become a second system of record for a fact that already has one, and the Truth patterns follow.
- Keep an inventory of every checkout customization with its mechanism and the vendor's support date for that mechanism; the Forked Platform on a SaaS checkout is a deadline you were told about and did not write down.

Questions to Ask

1. What does our checkout call, in which order, and which of those answers come from a copy rather than from the system that owns the fact?
2. At the moment the order is created, how old can the availability figure be? What answered it on our busiest hour last year?
3. If the shopper pays and never returns to our site, what creates or settles the order, and when?
4. What happens if the PSP's webhook for one payment arrives twice? If it never arrives?
5. Which customizations run inside our checkout, through which mechanism, and when does the vendor's support for that mechanism end?
6. Which third-party scripts load on our payment page, and who approved each one?
7. Are tax and shipping shown on the cart page, or only after the shopper has typed an address?
8. Can we disable guest card payments, or a single extension, without a deploy, in the middle of a peak hour?
9. For business customers, which system answers the credit check and the VAT treatment?

Checkout hands the order to two systems at once. The order record goes to order management, where its lifecycle begins; the payment request goes to the PSP, where the money's own lifecycle, authorization, capture, settlement, payout and sometimes refund and dispute, begins. The next chapter follows the money: Taking the Money: Payment Providers, Authorization, Capture and Refunds.

Sources and Evidence

Company, product and investor sources

- The checkout.liquid theme file is being deprecated, Shopify changelog, 2023 — the 13 August 2024 sunset for in-checkout pages and the description of checkout extensibility.
- checkout.liquid layout, Shopify developer documentation, n.d., accessed 2026-11 — the 28 August 2025 and 26 August 2026 sunsets for thank-you and order-status pages.
- Upgrade to checkout extensibility and Checkout extensibility, Shopify Help Center, n.d., accessed 2026-11 — plan gating and deadlines.
- Script Editor, Shopify Help Center, n.d., accessed 2026-11 — Shopify Scripts retired on 30 June 2026.
- Shopify Functions and Function APIs, Shopify developer documentation, API version 2026-10, accessed 2026-11 — sandboxed checkout logic and its limits.
- Black Friday Cyber Monday 2025 data, Shopify, 2025 — Shop Pay's 32% share of orders (vendor-defined).
- Checkout payments lifecycle, commercetools documentation, n.d., accessed 2026-11 — order created with a pending transaction before the PSP's answer.
- API Extensions, commercetools documentation, n.d., accessed 2026-11 — synchronous call-outs and the two-second default timeout.
- Fulfill orders, Receive Stripe events in your webhook endpoint, Place a hold on a payment method, Idempotent requests and Protect yourself from card testing, Stripe documentation, n.d., accessed 2026-11 — fulfillment by webhook, delivery guarantees, authorization windows, idempotency keys, card-testing controls.
- Handling payment status and Webhooks, Mollie documentation, n.d., accessed 2026-11 — session expiry per method and the id-only webhook with ten retries over 26 hours.
- Apple Pay, Apple developer site, n.d., accessed 2026-11, and Google Pay API for web: overview, Google, n.d., accessed 2026-11 — express wallet flows and tokens.
- PaymentTermsType, Shopify Admin API 2026-10, accessed 2026-11 — B2B payment-term types.
- Synchronize customers and companies, Microsoft Learn, 2026 — ownership of payment terms and credit limits between ERP and platform.

Standards, regulation and official documentation

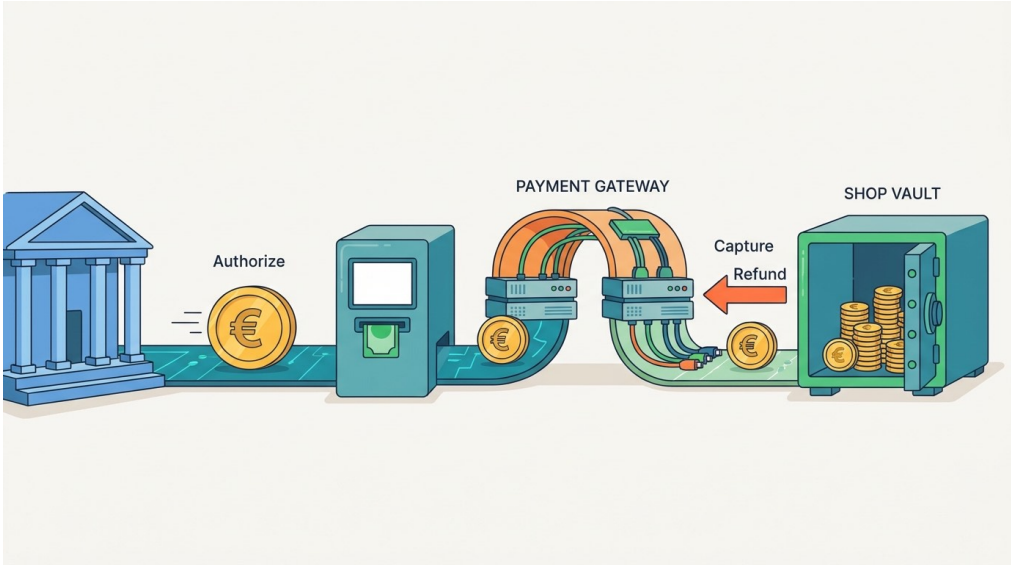
- Important updates announced for merchants validating to SAQ A, PCI Security Standards Council, 2025 — payment-page script requirements 6.4.3 and 11.6.1 and the 31 March 2025 date.
- Visa Acquirer Monitoring Program fact sheet, Visa, 2025 — enumeration monitoring.
- Commission Delegated Regulation (EU) 2018/389, EU Publications Office, 2018 — strong customer authentication and its exemptions, applied from 14 September 2019.

Research and practitioner sources

- 50 Cart Abandonment Rate Statistics, Baymard Institute, 2026 (updated 28 October 2026) — the 70.22% average and the reasons for abandonment, with the methodology caveats noted in the text.
- Checkout flow: average form fields, Baymard Institute, 2024 — 11.3 fields and 5.1 steps on average; eight fields suffice.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

12. Taking the Money: Payment Providers, Authorization, Capture and Refunds



IN THIS CHAPTER, YOU WILL: Follow Eva's €235 from her bank to Kestrel & Pine's account and learn why the order must treat the payment provider's answer as the truth and every refund as a one-owner action.

KEY POINTS:

- **Payment is a state machine whose truth lives outside your systems.** The payment service provider, not the checkout, decides whether money moved, and its answer arrives by webhook: late, unordered and sometimes twice. The order keeps a small vocabulary (pending, authorized, paid, refunded, disputed) with the provider's references beside it, never a flag that says "paid: true".
- **Money moves in four stages, and each payment method bends them differently.** Authorization reserves, capture takes, settlement moves money between banks, payout lands it in the merchant's account.

A card authorization expires in days; iDEAL has no capture and no chargeback; buy-now-pay-later holds for weeks. A design that assumes one method's behavior breaks on the others.

- **Every action that moves money has one writer and one key**. The Double Refund cost Kestrel & Pine €19,400 twice because two systems could refund the same return and neither sent a key the provider could recognize. Idempotency keys, a daily reconciliation against the provider's records and a card number that never touches your servers are the three disciplines this chapter is built on.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Boomerang** — a refund issued from two systems for one return is reconciled by hand after the money has gone, and the second one cannot be pulled back.
- **The Message in a Bottle** — a payment result sent once and trusted, or a refund request retried without a key, produces unpaid shipments and paid-twice customers.
- **The Lynchpin** — one payment provider carries every online sale; when its account is paused for 40 minutes, the checkout is dark and nobody has rehearsed the switch.

Everything a commerce platform does before this point is a promise. **Taking the money is the first moment the merchant receives something back**, and the first moment it depends on a system it does not own, run by a company it does not control, talking to banks it has never heard of. The money moves through that chain in stages, and the merchant's systems learn about each stage by message.

That dependence produces a quiet kind of failure. A stock problem is visible: a shelf is empty. A payment problem is not: a refund goes out twice, an authorization expires before anyone captures it, a webhook never arrives. **Payment mistakes are discovered in the books, weeks later, by the people least able to fix the cause**.

This chapter owns the terms **PSP**, **authorization and capture**, **SCA** (strong customer authentication), **payment webhook** and **refund**. The checkout before the payment belongs to From Cart to Confirmation: Checkout Orchestration; the money in the books to Tax, Invoices and the Books: From Order to Ledger; the deeper payment operations, from interchange to orchestration across providers, to the sibling book RECONCILED in Payments for Finance Architects: Eleven Reasons the Numbers Disagree, Adyen: One Platform from Checkout to Settlement and Stripe: Every Movement Is a Balance Transaction, which this chapter points to rather than repeats.

Where we are in the order's journey. Eva's jacket (the recurring order from Meet Kestrel & Pine) was placed on Tuesday 13 April 2027 at 21:14 and paid by iDEAL. This chapter is the minute before, 21:13, when Eva is redirected to her bank's app, and the

days after, when the money settles, is paid out and is matched in the books. It returns on 22 April for the refund variant of her return, which is where the Double Refund lives.

Setting the Scene at Kestrel & Pine: Three Weeks in January

Kestrel & Pine, the fictional outdoor brand this book follows, has used one **PSP** since 2016: a payment service provider, the company that offers the shopper a choice of payment methods, collects the money through the banks and card networks behind each method, and pays the merchant one net amount a day. It handles iDEAL (the Dutch method that pays by bank transfer from the shopper's banking app), cards, PayPal, Klarna, Bancontact and Apple Pay for the three storefronts; the six stores use a separate **terminal provider** at the till; wholesale customers pay invoices by bank transfer within 30 days.

In January 2027 Sanne de Wit, the Finance Director, closed the month and found orders with **two credits** against them in the ERP (enterprise resource planning system: the back-office system that keeps the books). Over three weeks, **212 returns had been refunded twice**: once by the returns portal through the platform's API, and once by support agents, by hand, in the PSP's own console. **€19,400** had gone out twice; most of it was recovered by asking customers to transfer the second amount back, because a bank-transfer refund cannot be recalled. The mechanism, reconstructed by Noor Visser in February, was ordinary; the days are a Kestrel & Pine illustration.

Day	What happened	Which system acted
0	A returned parcel arrives at Duiven; the customer sees PostNL's "delivered" scan	The carrier
0–3	The customer writes to support: "my return arrived, where is my refund?"	The customer
1	The agent opens the PSP console, sees the payment and no refund, and refunds €199 to be helpful	Support, in the PSP console

Day	What happened	Which system acted
2–5	The warehouse grades the item resellable; the returns portal, which refunds on inspection, calls the platform's API; the platform instructs the PSP, which accepts a request that names only the order and an amount	The returns portal, the platform, the PSP
31	Finance finds two credits per order when matching the PSP's payout report to the ERP	The ERP

Two systems could refund, neither sent a key that named the business event, and the only place the two paths meet is the ledger. Noor's whiteboard question in February is the chapter's: **who may move money, with what key, and how does the order find out what the PSP actually did?**

A second finding sharpened it. On the Card-Testing Night of August 2026, bots fired **48,000** stolen card numbers at the guest checkout, costing **€2,900** in fees for declined authorizations and a **40-minute** pause of the PSP account, and of every online sale. Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers owns the defense; this chapter owns the fact that one provider carried every sale.

What a Payment Service Provider Does

When Eva pays, at least five parties take part.

Party	In plain words	In Eva's iDEAL payment
The shopper	Pays with a card, a bank account or a wallet	Eva, in her bank's app
The issuer	The shopper's bank, which holds her money or her credit	Eva's Dutch bank
The scheme or network	The rules and rails a method runs on: Visa, Mastercard, iDEAL, SEPA, PayPal's own network	The iDEAL scheme

Party	In plain words	In Eva's iDEAL payment
The acquirer	The licensed institution that accepts payments on the merchant's behalf and receives the money from the issuer	Part of the PSP
The PSP	The merchant's single counterparty: one API, many methods, fraud screening, authentication, a balance, payouts and reports	Kestrel & Pine's PSP

A PSP hides the other four behind one contract; most PSPs serving European merchants are also acquirers or partner with one (a reference-architecture description, not a fact about any one company). Adyen's, Stripe's and Mollie's documentation (all n.d., accessed November 2026) describe the same four-stage flow: **authorization**, **capture**, **settlement**, **payout**, with **refund** and **chargeback** flowing backward.

Even a platform that appears to take payments itself works this way: Shopify's processor list (n.d., accessed November 2026) names Stripe, PayPal and Adyen as the processors behind Shopify Payments in the Netherlands, Belgium and Germany, and its European payment terms state that no Shopify entity is "licensed as a bank, payment institution, or money services business". "The platform's own payments" is a commercial wrapper over PSPs, and the four stages apply underneath.

The four stages

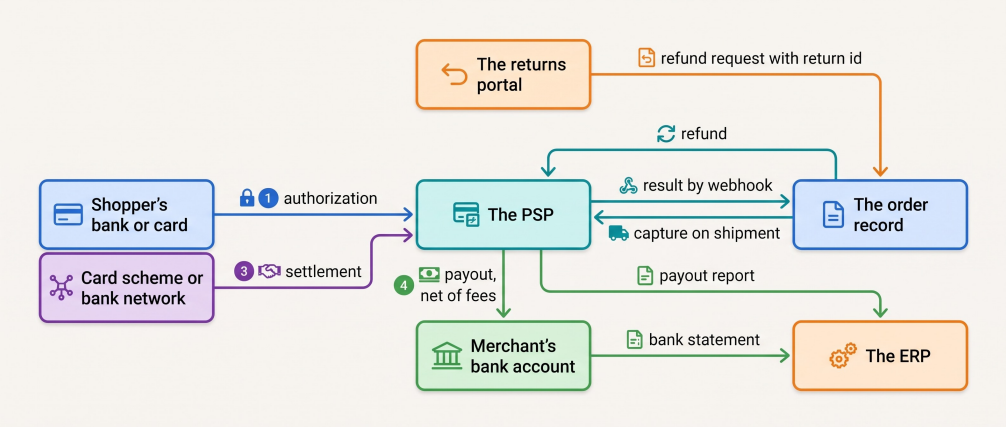


Figure 1: *Money moves in four stages the merchant does not control; the order learns the result by webhook, triggers capture and refunds through one path, and finance matches the payout to the orders it contains.*

Stage	Who acts	In plain words	What the order record keeps
Authorization	The issuer approves; the PSP reserves	"The money is there and held for you"	PSP reference, amount, method, authentication result, expiry
Capture	The merchant, or the PSP automatically	"Now take it"; Adyen: manual capture "lets you settle funds on fulfillment, when an order is shipped"	Captured amount, capture reference, link to the shipment
Settlement	The scheme or bank moves money to the PSP	The merchant's systems do nothing	Nothing
Payout	The PSP pays the merchant's bank	One batch per sales day, net of fees, refunds and chargebacks	Batch id, net amount, fees; matched by finance, not by the order system

The windows matter. Stripe lists card authorizations as valid for seven days for Visa, Mastercard, American Express and Discover when the shopper initiates an online

payment; if the authorization expires before capture "the funds are released and the payment status changes to canceled". Klarna must be captured within 28 days at Stripe and Mollie. **"Capture on shipment" is a promise with a deadline**: if the warehouse ships on day eight, the card money is gone and the parcel is a gift.

Payout timing is separate: Adyen's default is "sales day payout", all funds from one sales day in one batch with a two-business-day delay, and Stripe's initial settlement timing in most EU countries is seven calendar days, moving to three business days. The batch report, listing the payments, refunds, chargebacks and fees it nets, is what finance matches against orders (Tax, Invoices and the Books: From Order to Ledger; in depth, Payments-to-Ledger: Draining the Suspense Swamp).

Methods are not interchangeable

Kestrel & Pine sells in three countries, and each pays differently; across the euro area cards still lead at 48% of online payments by number (the European Central Bank's SPACE 2024 study), so the card path is the one to get right first. The figures are the latest dated public ones; the last column is the author's judgement.

Country	Dominant online method	Evidence	What it means for the design
Netherlands	**iDEAL**, a transfer from the shopper's own bank account	Dutch Payments Association, February 2025: 1.47 billion iDEAL payments worth €141 billion in 2024; the ECB's SPACE 2024 study puts "e-payment solutions" at 76% of Dutch online payments by number	Instant and irrevocable, no separate capture, no chargeback, a 15-minute session; a refund is a new transfer back

Country	Dominant online method	Evidence	What it means for the design
Belgium	**Bancontact**, a card scheme paid mostly through its app	Bancontact Payconiq Company, February 2025: 382 million online payments in 2024, 90% from a smartphone	Card-like: authorized then captured; refunds and disputes follow card rules
Germany	**PayPal**, **purchase on account** (*Kauf auf Rechnung*) and direct debit	EHI's "Online-Payment 2024" study as reported by Ecommerce News, April 2024 (data for 2023): PayPal 27.7%, invoice 26.7%, bank transfer 16.7%	"Payment" may be a 30-day receivable or a buy-now-pay-later loan; the lender captures late and bears the credit risk

As of November 2026 the iDEAL scheme's own site brands itself "iDEAL | Wero", reflecting its move under the European Payments Initiative's wallet; what that changes for integrations is not documented there, so this chapter uses Mollie's documented semantics (instant charge, automatic capture, partial refunds for 365 days, no chargeback, 15-minute session) and flags the name as in motion.

Costs differ too. Adyen's indicative price list (n.d., accessed November 2026) charges "$0.13 + €0.22" for iDEAL, "$0.13 + Interchange++ + 0.60%" for Visa and Mastercard and "$0.13 + 2.99%" for Klarna in Germany; **interchange**, the fee the merchant's side pays the cardholder's bank, is capped in the EU at 0.2% for consumer debit and 0.3% for consumer credit cards (Regulation (EU) 2015/751, in force since 9 December 2015). A buy-now-pay-later sale can cost ten times an iDEAL sale, a pricing decision before it is a technical one.

The Order Depends on a Message It Does Not Control

The rule the rest of the chapter builds on: **nothing in the merchant's own systems knows whether a payment succeeded**. The checkout sent a request. The shopper left for her bank's app or a 3-D Secure challenge (the card networks' extra authentication step, explained below) and may never come back. The PSP learns the outcome from the issuer and tells the merchant by **webhook**: an HTTP call to an address the merchant

registered, saying "something happened to payment X". The order's job is to wait for that message and to be safe when it arrives twice or not at all.

The documentation is blunt about that message. Stripe (n.d., accessed November 2026): "Stripe doesn't guarantee the delivery of events in the order that they're generated"; "Webhook endpoints might occasionally receive the same event more than once"; and every event is signed, because "Without verification, an attacker could send fake webhook events to your endpoint to trigger actions like fulfilling orders." Mollie's webhook carries only an id, so "fake calls to your webhook will never result in orders being processed without being actually paid"; the merchant must fetch the status; and Mollie retries ten times over 26 hours before giving up.

Three consequences follow, and I would write them into the integration contract of any order system:

1. **Persist every event id and treat a repeat as a no-op**. The second "payment succeeded" changes nothing; so does the second "refund succeeded".
2. **When the decision matters, fetch the object rather than trust the payload**. The webhook is the doorbell; the PSP's record is the truth.
3. **Run a daily job for everything still pending**. Webhooks stop retrying, after 26 hours at Mollie and three days at Stripe. A job that lists every order whose payment has not reached a final state within the method's expiry window, and asks the PSP, is the floor under the design. From Cart to Confirmation: Checkout Orchestration placed the same job at the end of checkout; Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation generalizes it.

Figure 2: *The webhook is only the doorbell: the order trusts the PSP's record, ignores a repeated ring, and runs a daily job for every payment still pending.*

The order should keep its **own small payment vocabulary** and map the PSP's statuses (Mollie has seven, Stripe's PaymentIntent six) into it inside the adapter, so that a merchant that changes PSP changes an adapter, not every downstream system.

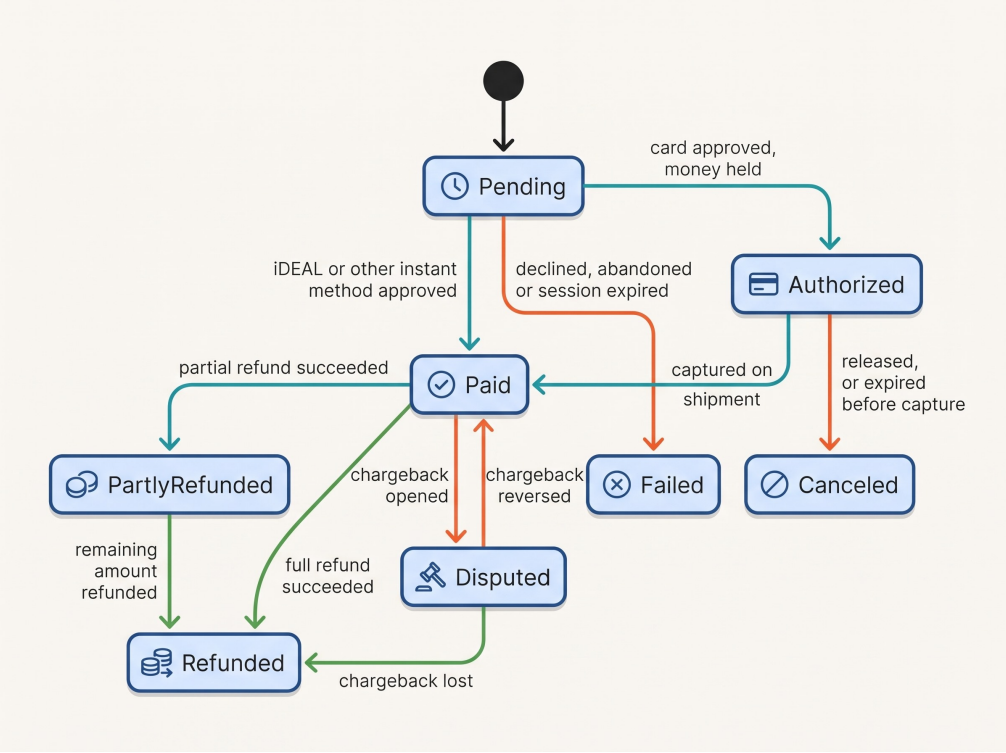


Figure 3: *The order's payment vocabulary: a few states every channel and system understands, moved only by the PSP's confirmed events, with the PSP's raw status and references stored beside them.*

Each flow between the order and the PSP has its own requirements; the latencies are assumptions for a merchant of Kestrel & Pine's shape.

Flow	How fresh	How consistent	How fast	If the other side is down
Payment outcome (webhook in)	As soon as the PSP knows	Applied once; duplicates ignored by event id	Seconds, but tolerate hours *(assumption)*	The daily pending-payments job settles it
Capture (out, on shipment)	Within the authorization window	One capture per fulfillment, keyed by it	Minutes *(assumption)*	Retry with the same key; alert before the window closes
Refund (out, on inspection)	Within the legal 14 days (The Way Back: Returns, Exchanges, Refunds and Reverse Logistics)	Exactly one per return, keyed by return id	Hours *(assumption)*	Never retry blindly; check the PSP's refund list first
Payout report (in)	Daily batch	Every line matched to an order, refund, fee or dispute	A day	Finance waits; nothing else does

Strong Customer Authentication and 3-D Secure

PSD2, the second Payment Services Directive (Directive (EU) 2015/2366, applied from 13 January 2018), defines **strong customer authentication** (SCA) in Article 4(30) as authentication using two or more of knowledge, possession and inherence, and Article 97 requires it when a payer initiates an electronic payment. The detailed rules are in Commission Delegated Regulation (EU) 2018/389, applied from **14 September 2019**, which sets the exemptions: contactless at the terminal up to €50 (Article 11), trusted beneficiaries (Article 13), recurring transactions after the first (Article 14), low-value remote payments up to €30 (Article 16) and transaction risk analysis up to €500 for PSPs with low fraud rates (Article 18).

For cards the protocol that performs SCA is **3-D Secure 2** (3DS): low-risk transactions pass "frictionless" and higher-risk ones require "a challenge, such as a one-time passcode" (EMVCo, n.d., accessed November 2026). Two rules follow from Stripe's SCA

page. Exemptions belong to the PSP and the issuer, not the merchant: "Exemptions aren't guaranteed." And the **liability shift**: "If a cardholder disputes a 3DS payment as fraudulent, the liability typically shifts from you to the card issuer"; where an exemption was applied, "liability shift doesn't apply". An exemption buys conversion at the price of fraud liability, so the order should store the authentication result and the liability flag. A **merchant-initiated transaction** (MIT), one the merchant triggers later without the shopper present under a prior agreement, is what a split shipment captured late, a pre-order or "charge on dispatch" past the authorization window relies on; it needs an SCA-authenticated first interaction flagged for future use.

What comes next is not yet law. **PSD3** and the Payment Services Regulation reached provisional agreement on 27 November 2025 (PSPs liable for customers' losses where fraud prevention was inadequate, mandatory payee-name checks, all charges disclosed before payment, per the European Parliament's press release of 21 November 2025); the Council published compromise texts on 23 April 2026, and both files were still open in its state of play of 29 October 2026. As of 7 November 2026 neither is in the Official Journal. The Instant Payments Regulation (Regulation (EU) 2024/886) is in force: euro-area PSPs had to receive instant credit transfers by 9 January 2025 and send them by 9 October 2025, with payee verification free of charge, which makes instant transfer a plausible checkout and refund rail.

Refunds, Cancels and Chargebacks: Money Flowing Backward

A refund looks like an undo. It is not. **A refund is a new payment in the opposite direction**, with its own state, failure modes and fees.

- **You can only refund what was captured**. Adyen: "You can only refund a payment after it has been captured"; an uncaptured authorization is *canceled* instead. Stripe's cost advice points the same way: businesses that refund soon after the sale should capture manually and cancel before capture "rather than processing a refund", because "Stripe's processing fees from the original transaction aren't returned."
- **A refund is asynchronous and can fail later**. Adyen: a webhook with `success: true` means the request went to the scheme, and "Although rare, a refund can fail after you received" it; for iDEAL, Bancontact and bank transfers a REFUNDED_-REVERSED event means the money came back because the account was invalid; funds can take "up to 40 business days" to reach the shopper. Stripe: the customer "sees the refund as a credit approximately 5-10 business days later".
- **Refunds go only to the original method, and partial refunds are bounded** (Stripe; Adyen). Store credit is a different instrument with its own ledger (Discounts, Coupons and Loyalty: Promotions Without Chaos).

The sequence shows how the Double Refund happened, and where one key would have stopped it.

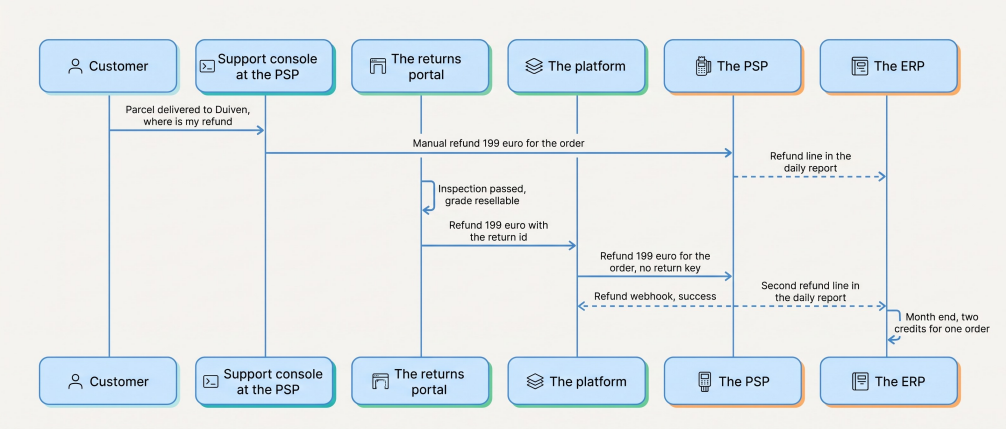


Figure 4: *The Double Refund: two writers, two paths, no shared key; the PSP cannot refuse the second request because nothing in it says it is for the same return, and the two paths meet only in the ledger.*

The fix Waypoint adopted in February 2027, which Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation describes from the integration side, has three parts. **One writer per money action**: refunds are issued by the returns portal through the platform, and support's PSP console access becomes read-only. **One key**: every refund request carries the return id as its **idempotency key**, a unique token sent with a request so that a retry does nothing twice; Stripe's implementation is the public reference (it "saves the status code and body of the first request made with a given idempotency key" and returns it on retries, and may prune keys after 24 hours, so the merchant's own key store must outlive the PSP's). **One daily check**: refunds issued by the PSP against refunds recorded by the platform, by order number and amount, with exceptions to Sanne de Wit's team.

Figure 5: *The fix for the Double Refund: one writer per money action and one key per business event, so the PSP refuses a repeat instead of paying twice.*

Chargebacks are the other backward flow, and not the merchant's decision: a dispute "occurs when a cardholder questions your payment with their card issuer"; the issuer "immediately reverses the payment", and the PSP "debits your balance for the payment amount and dispute fee" (Stripe). Adyen's flow runs from a notification of chargeback that opens the defense period through the chargeback, the supplied defense and a possible reversal, and Adyen auto-defends a dispute "that was already refunded". Cardholders "can dispute a charge up to 120 days after a payment was made", so a month's dispute ratio keeps changing for four months. The ratio has teeth: Visa's Acquirer Monitoring Program (fact sheet, May 2025) counts fraud reports plus disputes over settled transactions, with the excessive-merchant line lowered to 150 basis points on 1 April 2026. A slow refund path pushes customers to dispute instead, a second reason to get refunds right; iDEAL has no chargeback at all.

Fraud and Card Data: Keep the Attack Out and the Card Number Out

Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers owns the defense in depth and the sibling book LISTED covers scoring in Scams, Fakes and Account Takeovers: Fraud Prevention for Listings; this chapter keeps to what shapes the payment design.

Fraud screening is a ratio problem, so it belongs before authorization. Stripe's card-testing page (n.d., accessed November 2026) describes what hit Kestrel & Pine: attackers

"use scripts to test a large amount of card information at once"; the symptoms are "A spike in failed or blocked payments" and small payments "with nonsensical customer names and emails"; the costs are authorization fees, disputes and monitoring programs. Its controls: the PSP's rate limiters and models, CAPTCHA triggers, requiring login ("The easier it's for fraudulent actors to reach your payment form (for example, using guest checkout), the easier it's for them to execute card testing attacks"), per-IP limits and velocity rules. The addition I insist on is a **kill switch**: a setting, not a deploy, that disables guest card payments or one method while leaving iDEAL and logged-in customers alone.

Card data is a scope decision. **PCI DSS** is the card industry's security standard for anyone who stores, processes or transmits card data; version 4.0.1 was published on 11 June 2024, and its future-dated requirements took effect on 31 March 2025. The merchant's obligations depend on which systems can see a card number, and the design goal is that none of the merchant's can. The mechanism is **hosted fields**: the PSP draws the card-number box inside the merchant's page from its own domain, or redirects to its own page, so the number never touches the merchant's code or servers. Stripe's PCI guide maps the integration to the self-assessment questionnaire (SAQ) the merchant signs: hosted checkout or hosted fields to **SAQ A**, a merchant-hosted form posting via the PSP's script to **SAQ A-EP**, direct API with raw card data to **SAQ D**, where a business "might be required to meet more than 300 security controls". Adyen adds the boundary: "Adyen is solely responsible for the security of cardholder data only as soon as Adyen receives the data." Since 30 January 2025 the PCI Security Standards Council has replaced the two payment-page-script requirements in SAQ A with an eligibility statement that the site "is not susceptible to attacks from scripts", so every analytics, chat and testing script on the payment page remains the merchant's risk. The rule: **never store, log or transmit a card number**; **use the PSP's token**; **keep a list of every script on the payment page**.

The Choices: Four Ways to Take Payments

1. **The platform's built-in payments**. A hosted platform offers its own payment service, a wrapper over one or more PSPs, with a hosted checkout, automatic capture, refunds from the admin screen and payouts to the merchant's bank; Shopify Payments is the public example. Nothing to integrate, SAQ A, one payout feed. The limits: the method mix and fees are the platform's, and any second system that must move money goes through the platform's API or not at all.

2. **One external PSP through the platform's connector**. The merchant contracts with a PSP and uses its app or the platform's native integration; the PSP's hosted page or drop-in component renders the methods. Kestrel & Pine's setup since 2016: the PSP's local methods, pricing and risk tools, SAQ A if the page stays hosted, and ownership of the relationship, the webhook endpoint and the question of who else may call the PSP.

3. **A custom integration with hosted fields.** A headless or heavily customized checkout (Delivering the Storefront: Themes, Headless, Rendering and Performance) calls the PSP's API directly and embeds its hosted fields: full control of capture on shipment, partial captures and MITs; SAQ A if the eligibility criteria hold; and a checkout team that handles every property of the webhook itself.

4. **A payment service of the merchant's own, over several PSPs.** An internal service presents one payment API to every channel, routes each payment to one of several PSPs and reconciles across them. Payments for Finance Architects: Eleven Reasons the Numbers Disagree covers the trade-offs; it is worth its complexity only when volume, countries or resilience demand it.

Criterion	1. Platform's own payments	2. One PSP via connector	3. Custom with hosted fields	4. Own payment service, several PSPs
Time to first sale	Hours	Days	Weeks to months	Months
Method mix and pricing	The platform's	The PSP's, negotiable at volume	The PSP's	Best per method and country
PCI scope	SAQ A	SAQ A	SAQ A or A-EP	SAQ A-EP to SAQ D
Capture on shipment, partial capture, MIT	What the platform exposes	Via the connector's settings	Full	Full
Who else may move money	The platform's API only	Whoever holds the PSP's API key	The payment adapter	The payment service, by policy
Resilience to a provider outage	None	None	None without a second PSP	Routing to another PSP
Fits	Starting; one country	Growing; Omnichannel with discipline	Omnichannel with a checkout team	Group, or very high volume

My rule for a merchant of Kestrel & Pine's size: **stay on option 2 and spend the effort on discipline, not orchestration.** One PSP, local methods per country, capture on

shipment for cards, one refund path, verified webhooks handled by an adapter, a daily reconciliation, and a rehearsed fallback for one method rather than an orchestration layer a 14-person team would have to run.

Stores, Wholesale and Marketplaces: Four Feeds, Not One

Online is one of four ways money reaches Kestrel & Pine.

Stores take cards through the terminal provider, where SCA is chip-and-PIN or the contactless limit, the PCI questionnaire is a different one (SAQ C or a validated point-to-point encryption solution), and the payout is a separate daily feed into the ERP. A web order refunded at the till is the store's version of the Double Refund (Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment).

Wholesale customers do not pay at checkout. Bergzicht's spring reorder was invoiced on 9 April 2027 and is due on 9 May: "payment" is a **receivable**, an amount owed and tracked in the ERP, settled by a bank transfer that finance matches to the invoice (Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, Tax, Invoices and the Books: From Order to Ledger).

Marketplaces collect the shopper's money and pay the seller out, net of fees, on their own schedule: a third payout feed (Selling on Someone Else's Platform: Marketplaces and Channel Integration).

Online is the PSP's daily batch. Four feeds, four reconciliations, and the reason Waypoint's W5 says finance moves by daily batch: the books need completeness, not speed. Where four feeds meet, a payments sub-ledger earns its place (Never Update, Only Append: Designing Custom Ledgers).

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Signs the stage is ending
1 — Starting (under about €2M)	The platform's own payments or one PSP on a hosted page; automatic capture; refunds from the admin screen; one country's methods	Two-step authentication on the PSP and admin logins; someone who reads the decline rate	Refunds done by hand in the PSP console; a second country whose shoppers want a method the setup lacks
2 — Growing (about €2M–€20M)	One PSP with local methods per country; the first custom integrations (a returns portal, an ERP connector) that can call the PSP; card testing as the first incident	Capture on shipment for cards; one writer per money action; a webhook endpoint that is owned and verified; an edge rate limit and a kill switch for guest card payments	Two systems can refund; an agency script handles webhooks and nobody knows how; a dispute lost for want of a stored 3DS result
3 — Omnichannel (about €20M–€200M)	Four feeds: online PSP, terminal provider, marketplaces and wholesale receivables; payment states on an order hub	The order hub's payment vocabulary and adapter; refund keys; a rehearsed PSP fallback; a payment-page script inventory; a payments sub-ledger	Payout matching takes days each month; a PSP account pause stops every channel; fee lines nobody can explain per country

Stage	What is typical	What becomes necessary	Signs the stage is ending
4 — Group (over about €200M, or several brands)	Several PSPs or an orchestration layer; network tokens so saved cards are not locked to one PSP; a central regulatory calendar	A payment service with routing and one reconciliation pipeline; a deliberate decision whether any system may touch card numbers	Each brand negotiating its own PSP contract; a tokenized card base that cannot move

The judgement: the payment *mechanism* does not change with scale. What changes is **how many systems can move money and how many feeds finance must match**; stage 2 is where the Double Refund happens, because it is the stage at which the first integration beyond the platform gets a PSP key.

In Practice

Kestrel & Pine: Eva's €235, start to finish. On Tuesday 13 April 2027 at 21:13 the platform created an iDEAL payment for €235 at the PSP and redirected Eva to her bank's app. At 21:14 the platform created order #1072455 with the payment *pending*; seconds later the PSP's webhook reported success and the order became *paid* (From Cart to Confirmation: Checkout Orchestration walks the checkout). There was no capture step: iDEAL is captured at approval, and no chargeback can follow. Assume the PSP charges about €0.30 per iDEAL payment *(assumption; Adyen's indicative price is $0.13 + €0.22)*. Tuesday's sales were paid out as one batch on Thursday 15 April *(assumption, modeled on a two-business-day sales-day payout)*, net of fees and of refunds of earlier orders; Sanne's team matched the batch to the day's orders in the ERP by the PSP's reference, which the order carried from the webhook. Had Eva paid by card, Tuesday's authorization would have been captured on Wednesday 14 April when the label printed, inside Visa's seven-day window, with the fulfillment order id as the capture's key.

The refund variant. Had Eva chosen a refund instead of size L on 19 April, the returns portal would have waited for the inspection on 22 April, then asked the platform to refund €199 with the return id as the key; the platform would have instructed the PSP; the PSP would have made a new iDEAL transfer back to Eva's account and reported it by webhook; the ERP would have booked a credit note against the invoice (Tax, Invoices and the Books: From Order to Ledger). The refund appears as a deduction in a later payout batch, which is why the batch for 22 April does not equal the sales of 22 April,

and why matching is by line, not by total. Support, since February 2027, could have seen all of this in the returns portal and issued nothing in the PSP console.

When It Goes Wrong

The duplicate refund. Two systems, no key. Detection: a daily list of refunds at the PSP against refunds in the platform by order and amount. Recovery: ask for the money back, because bank transfers cannot be recalled, and block the writer. Prevention: one writer, the return id as idempotency key, read-only console access for everyone else.

The lost outcome. The shopper paid and never returned; the webhook failed; the PSP stopped retrying. Detection: the daily job for payments pending beyond their method's expiry (15 minutes for iDEAL, 30 for cards at Mollie). Recovery: query the PSP, settle the order either way.

The expired authorization. The warehouse shipped on day eight and the capture failed. Detection: an alert when an authorized order approaches its window uncaptured. Prevention: capture at label print, and a fulfillment hold when the window is close.

The provider outage or account pause. The Card-Testing Night: 40 minutes with no online sales. Detection: a checkout health check that watches the PSP's error rate, not only its status page. Recovery: a rehearsed fallback for one method, a kill switch for the attacked method, an honest storefront message. Prevention: edge rate limits and velocity rules before authorization, so the PSP never has to pause you.

What to Take Away

Established.

- Money moves in four stages, authorization, capture, settlement and payout, with refunds and chargebacks flowing backward; capture must happen inside an authorization window of days, and a refund is a new payment that can fail later (Adyen, Stripe and Mollie documentation, n.d., accessed November 2026).
- The payment outcome arrives by webhook, at least once, in no guaranteed order, and must be verified; the merchant persists event ids, fetches the object when the decision matters, and runs a daily job for everything still pending (Stripe and Mollie documentation).

Depends on context.

- Capture on shipment is right for cards where cancellations before dispatch are common, because it saves refund fees and ties money to goods; it does not exist for iDEAL and other instant methods, and it is a liability if the warehouse cannot ship inside the window.

- One PSP with disciplined integration is right up to the Omnichannel stage; orchestration across PSPs is right at Group scale or where one provider's outage would stop several brands, and it changes the PCI scope if the merchant's service ever sees a card number. **The author's synthesis**.
- I treat payments as a problem of writers and keys before a problem of providers: name one system that may refund, one that may capture, one that may create a payment, give each action a business key the PSP will remember, and reconcile daily; everything else about the PSP can be changed later.
- The order's payment vocabulary should be the merchant's, not the PSP's: five or six states, moved only by confirmed events, with the provider's raw status beside them, so that changing provider changes one adapter.

Questions to Ask

1. Which systems hold a credential that can refund, capture or create a payment at the PSP, and who approved each one?
2. What is the idempotency key for a refund, a capture and a payment creation, and what does the PSP do when it sees the same key twice?
3. Where does the order store the PSP's reference, the authentication result and the liability-shift flag?
4. How long is the authorization window for each method, and what happens when a shipment approaches it uncaptured?
5. If the PSP's account is paused for an hour, what still sells, and who flips which switch?
6. How many payout feeds does finance match each month, by line or by total, and how long does it take?
7. Which regulatory changes (PSD3, 3-D Secure versions, PCI DSS) are on a calendar with an owner?

The money is in, and the webhook has told the order so. The order now has a life of its own, across the platform, the warehouse, the carrier and, when Waypoint delivers it, an order management system that holds every channel's orders in one record. The Order Record: Lifecycle, State and the Order Management System follows the order from "paid" onward.

Sources and Evidence

Company, product and investor sources

- Capture, Adjust authorisation, Refund, Sales day payout, Dispute process and flow, Webhooks, Batch-level reconciliation, PCI DSS compliance guide and SAQ A

eligibility, Adyen documentation, n.d., accessed 2026-11 — authorization and capture semantics, expiry windows, asynchronous refunds, payout batches, dispute stages, the PCI responsibility boundary.
- Pricing, Adyen, n.d., accessed 2026-11 — indicative per-method fees (volatile).
- Place a hold on a payment method, Refunds, Disputes, Measuring disputes, Monitoring programs, Strong Customer Authentication, CITs and MITs, Idempotent requests, Webhooks, Payouts, Payout reconciliation, Security guide, Card testing and PCI compliance guide, Stripe documentation, n.d., accessed 2026-11 — authorization windows, refund behavior and fees, dispute mechanics and thresholds, SCA exemptions and liability shift, MITs, idempotency keys, webhook delivery, settlement timing, reconciliation reports, SAQ mapping, card-testing controls.
- iDEAL, Place a hold for a payment, Refunds, Webhooks, Handling payment status and Settlement report, Mollie documentation, n.d., accessed 2026-11 — iDEAL semantics, hold periods, duplicate-refund blocking, id-only webhooks with ten retries over 26 hours, status model and expiry per method, settlement contents.
- Shopify Payments Terms of Service (Ireland) and Processor list, Shopify, n.d., accessed 2026-11; Form 10-K for fiscal 2025, Shopify Inc., 2026-02-11; Shopify Payments supported countries, Shopify Help Center, n.d., accessed 2026-11 — a platform's payments as a wrapper over licensed PSPs; Payments penetration; country availability.
- Annual turnover iDEAL and PIN payments, Dutch Payments Association, 2025-02-18 — 2024 iDEAL volumes; iDEAL | Wero, Currence iDEAL, n.d., accessed 2026-11 — the scheme's current branding.
- 2.5 billion payments with Bancontact and Payconiq in 2024, Bancontact Payconiq Company, 2025-02-12 — Belgian online volumes.
- PayPal remains most popular online payment method in Germany, Ecommerce News reporting EHI's "Online-Payment 2024", 2024 — German method shares for 2023 (secondary source).

Standards, regulation and official documentation

- Directive (EU) 2015/2366 (PSD2), Official Journal, 2015-12-23 — the SCA definition (Article 4(30)) and requirement (Article 97); applied from 13 January 2018.
- Commission Delegated Regulation (EU) 2018/389, Official Journal, 2018-03-13 — the RTS on SCA, applied from 14 September 2019, and its exemptions.
- Regulation (EU) 2015/751 on interchange fees, Official Journal, 2015-05-19 — the 0.2% and 0.3% caps (older than three years; still in force).
- Regulation (EU) 2024/886 on instant credit transfers, Official Journal, 2024-03-19 — receive and send deadlines, free payee verification.
- Payment services deal: more protection from online fraud and hidden fees, European Parliament, 2025-11-21, and State of play of legislative files, Council of the EU, 2026-10-29 — PSD3 and PSR content and status.

- EMV 3-D Secure, EMVCo, n.d., accessed 2026-11 — frictionless and challenge flows.
- Just published: PCI DSS v4.0.1, PCI Security Standards Council, 2024-06-11; Important updates announced for merchants validating to SAQ A, PCI SSC, 2025-01-30; PCI DSS, PCI SSC, n.d., accessed 2026-11 — versions, dates, scope, the SAQ A change.
- Visa Acquirer Monitoring Program fact sheet, Visa, 2025-05 — VAMP ratio and thresholds (volatile).
- Study on the payment attitudes of consumers in the euro area (SPACE 2024), European Central Bank, 2024-12 — online payment shares by method and country.

Research and practitioner sources

- Payments for Finance Architects: Eleven Reasons the Numbers Disagree, Adyen: One Platform from Checkout to Settlement, Stripe: Every Movement Is a Balance Transaction, Payments-to-Ledger: Draining the Suspense Swamp, Never Update, Only Append: Designing Custom Ledgers and Reconciled by Design: Traceability and Audit Across Boundaries in RECONCILED — the deeper treatment of parties, pricing, orchestration, tokens, the payments sub-ledger and reconciliation this chapter points to.
- Scams, Fakes and Account Takeovers: Fraud Prevention for Listings in LISTED — scoring and velocity controls beyond the basics given here.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

13. The Order Record: Lifecycle, State and the Order Management System

IN THIS CHAPTER, YOU WILL: Learn what an order record holds, which states it moves through, how it is routed and split, and when an order management system becomes necessary.

KEY POINTS:

- **An order is one record with several state machines, not one status.** Money, goods and returns each move on their own, and the goods state belongs to each shipment, not to the order. A single "status" string is where Kestrel & Pine's Three-Status Order began.
- **Routing and splitting are decisions, and they need a home.** Which location ships which line, whether to split, and what a split may cost are rules over stock, capacity, cut-off times and carrier cost. Platforms route by priority or distance; the rest is order management.

- **An order management system is the order hub once two of four things are true**: several channels create orders, several locations fulfill them, promises depend on location, or service cannot see one order across systems. Kestrel & Pine has all four and buys an OMS (Waypoint W2).

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **Schrödinger's Order** — three systems each held an honest status of a different object and nobody held the order's.
- **The Spaghetti Junction** — four order sources (platform, marketplace connector, ERP, store tills) each with its own connector to the warehouse and the books.
- **The Message in a Bottle** — one shipment confirmation, sent once and trusted, turned a two-parcel order into a delivered one.

The moment a shopper clicks "pay", a promise becomes an obligation. The merchant now owes a parcel, and a record must exist that says what was bought, by whom, for how much, to be delivered where and by when. That record is the **order**. The warehouse, the carrier, the payment provider, the service agent, the finance team and the shopper's account page will all read it, and each will change a part of it. **The order is the one object every commerce system shares, and the one most often held in several places at once**.

A small merchant never notices, because its commerce platform holds the only order there is. A growing merchant adds a marketplace, a warehouse system, an ERP, a store till and a wholesale channel, and each arrives with its own idea of an order. Orders now begin in four systems and are copied into others, with each copy carrying its own status. The question "where is my order?" then has several true answers and no correct one. **This chapter is about giving the order one home**: what the record holds, which states it passes through, how a line is routed to a location, how splits, edits and cancellations are handled, and when an order management system (**OMS**) earns its place beside the platform.

Kestrel & Pine, the fictional outdoor brand this book follows, is the worked example, and its Three-Status Order of December 2026 is the spine.

Where we are in the order's journey. Eva's jacket (the recurring order, see Meet Kestrel & Pine) was confirmed at 21:14 on 13 April 2027 as order #1072455 and paid by iDEAL. From that second until the exchange order ships on 22 April, this chapter is where the order lives: who holds it, what its states are, and who tells the warehouse to pick. From Cart to Confirmation: Checkout Orchestration ends at confirmation; Pick, Pack, Ship: Warehouse Management and Fulfillment takes the pick; The Way Back: Returns, Exchanges, Refunds and Reverse Logistics takes the exchange.

Setting the Scene at Kestrel & Pine: Order #1052877

In the first week of December 2026, a customer ordered a Trailhead fleece (€119) and a pair of merino hiking socks (€18) on the Dutch storefront: order **#1052877**, €137. The fleece was picked and shipped from Duiven on Monday 7 December. The socks were short in their pick location and waited for replenishment from bulk stock, so the warehouse management system (**the WMS**, the software that runs picking, packing and shipping) kept the order open as a second pick.

The flow that carries shipment confirmations from the WMS to **the platform** (the hosted commerce platform behind the storefronts) had been built by the agency and had no notion of a partial shipment. On the first confirmation it marked the whole platform order fulfilled and attached the fleece's tracking number. PostNL delivered the fleece on Tuesday 8 December; the carrier's scan turned the platform order to **Delivered** and the "delivered" e-mail went out. **The ERP** (the enterprise resource planning system: the books, invoicing, purchasing) invoices on an "order complete" message from the WMS, which could not come while the socks were unpicked, so it showed **Not invoiced**. The WMS, honest about its own object, showed **In picking**.

System	What it held	Its status	True of what
The platform	The customer's order, one tracking number	Delivered	The first parcel
The WMS	A warehouse order with two lines, one still open	In picking	The second pick
The ERP	A copy awaiting completion	Not invoiced	The invoice run
The customer	A box with a fleece and no socks	"Where are my socks?"	The order

Figure 1: *Order #1052877: the platform, the WMS and the ERP each held an honest status of a different object, and nobody held the order.*

Eleven tickets followed over six days, until the socks shipped on 14 December (Where Is My Order? Notifications, Service and the Customer Record follows the tickets). When Noor Visser, the new Head of Engineering, reviewed the case in January 2027, she did not find a bug. She found that no system held the order. The platform held the customer-facing record and could show one status word. The WMS held an execution record. The ERP held a financial one. The marketplace connector, the POS (the store till software) and the EDI provider (EDI, electronic data interchange, is the standard by which large trading partners exchange orders and invoices between their systems) held other orders of their own. Her question to Mara Lindqvist, the CEO, became Waypoint decision W2: **which system is the order's home, and what must that record be able to say?**

What an Order Record Holds

An order is more than a list of items. Stripped to what every downstream system needs, the record has six parts.

Part	What it holds	Who reads it most
Header	Order number, channel, customer reference, placed-at time, currency, market, legal seller	Everyone

Part	What it holds	Who reads it most
Lines	Each SKU (a stock-keeping unit, one sellable variant), quantity, frozen unit price, discounts allocated, tax	Warehouse, finance
Money	Payment method, authorization and capture references from the payment service provider (**PSP**, the company that takes the payment), amounts paid, refunded, owed	Finance, service
Promises	Delivery address and method, the promised date, pickup store if any	Warehouse, carrier, service
Goods	Which lines are assigned to which location, in which shipment, with which tracking number	Warehouse, carrier, service, customer
References	The marketplace's order id, the EDI document number, the invoice number, return ids	Integration, finance

Three distinctions trip up newcomers, and Follow the Order: What Happens Between "Buy" and "Delivered" draws them in full. **A cart is not an order**: a cart is a draft that commits nobody. **An order is not a shipment**: one order can leave in two parcels from two locations. **An order is not an invoice**: the invoice is a legal document issued when goods ship or when finance says so (Tax, Invoices and the Books: From Order to Ledger).

Order numbers and identity

The order number is the one thing every system, the customer and the carrier label share. Two rules make it useful. First, **one number family across channels**: whether an order came from the storefront, Amazon.de or an EDI message, it gets a number from the same sequence, and the channel's own id (the marketplace order id, the EDI document number) is kept as a reference, not used as the key. Second, **the order number is the idempotency key for creating the order anywhere else**. Idempotent handling means that receiving the same message twice has the same effect as once; keyed on the order number, a duplicated "order placed" event finds the existing order and does nothing. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation owns the pattern; this chapter needs only the rule, because the ERP, the WMS and the OMS will each receive that event at least once.

Several State Machines on One Record

A **state machine** is a fixed set of states an object can be in and the allowed moves between them. The mistake behind the Three-Status Order was to assume an order has one. It has at least three, and the platforms say so in their own documentation.

Shopify keeps three status families on one order (help center, n.d., accessed November 2026). The *order status* is Open, Archived or Canceled. The *payment status* has eight values, from Authorized and Paid to Partially refunded, Refunded, Voided (which "releases the hold on the customer's payment method", distinct from canceling) and Expired. The *fulfillment status* has six: Unfulfilled, In progress, On hold, Scheduled, Partially fulfilled, Fulfilled. Below the order, each *fulfillment order* (the instruction to one location to ship a group of lines) has seven statuses of its own plus a request status for the handshake with the warehouse (developer docs, n.d.). commercetools keeps an `OrderState` (Open, Confirmed, Complete, Cancelled), a `ShipmentState` with eight values including *Partial* ("items in the Order are shipped in more than one shipment") and *Backorder*, and a `PaymentState`; a fourth field can point to a custom state machine, with transitions left unvalidated when "an external system runs the workflow" (API docs, n.d., accessed November 2026). Its return items carry two further states each.

Machine	Typical states	Lives at	Who moves it
Order lifecycle	Open, confirmed, complete, canceled	The order	The order system
Money	Authorized, paid, partially refunded, refunded, voided	The order	The PSP's events
Goods	Requested, accepted, in progress, shipped, delivered, canceled, incomplete	Each fulfillment order (shipment), not the order	The warehouse and the carrier
Return	Requested, received, graded, refunded or exchanged	Each return	The returns portal and the warehouse

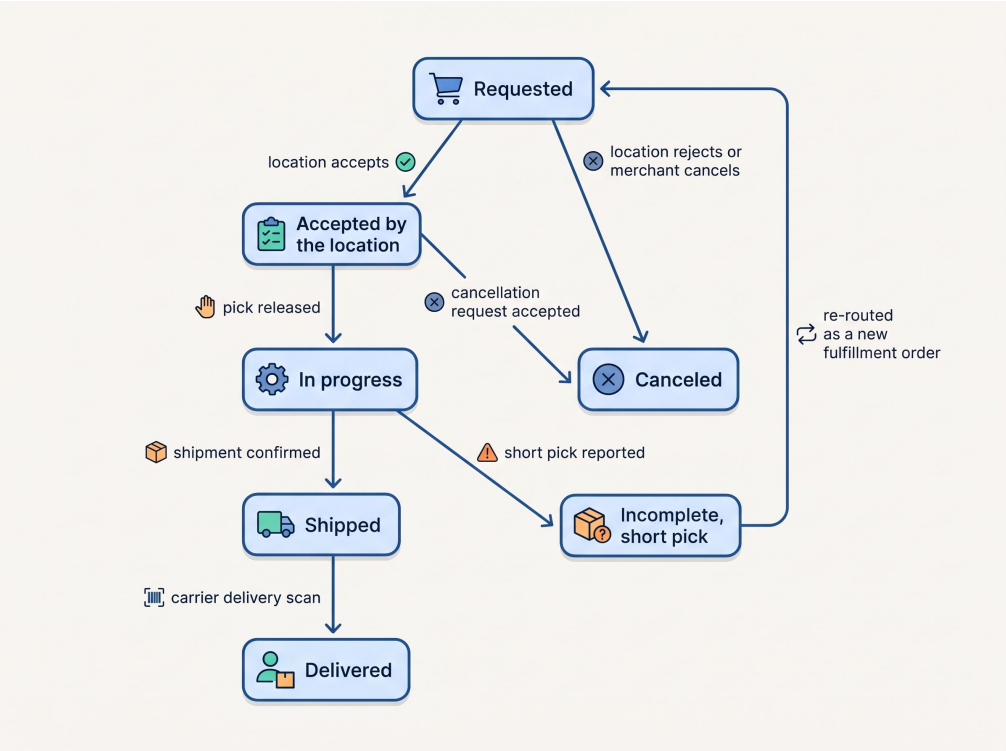


Figure 2: *The goods state machine lives on each fulfillment order, not on the order: a two-parcel order has two of these, and "delivered" is true only when both reach the last state.*

The order-level words a customer sees ("Delivered", "Shipped") are summaries computed from the machines below, and two rules keep the summary honest. **"Fulfilled" is not "delivered"** unless the carrier's events are fed back; both Shopify's and commercetools' models leave that gap for the merchant to close (Getting It There: Carriers, Labels, Tracking and the Delivery Promise). And **the summary of a split order is the state of its least-advanced shipment**, which is the rule the agency's flow lacked: with it, #1052877 would have read "Partially shipped: one parcel delivered, one in preparation".

What "Delivered" on the order says	What it does not say
At least one carrier has scanned at least one parcel as delivered	That every line of the order has been delivered
The platform received a delivery event	That the invoice was issued or the ERP knows

What "Delivered" on the order says	What it does not say
The customer-facing summary has been computed	That a return, if any, has started

Changing a Placed Order

An order is edited, held or canceled far more often than newcomers expect: wrong size, wrong address, a line out of stock, a fraud hold, a customer who changed their mind. Each change is a transaction against the money, goods and stock commitments, not a form update.

Edits. Shopify's order editing (developer docs, n.d., accessed November 2026) begins a *calculated order* that tracks changes, lets a line's quantity be set to zero to remove it, and commits the edit; "You can edit only unfulfilled line items", and "If an edit changes the total order value, then a balance might need to be collected from or refunded to the customer". The constraint is the rule everywhere: **the edit window closes when the warehouse accepts the work.** In the fulfillment-order handshake that Shopify documents, once a warehouse has accepted a request "the merchant can no longer cancel it directly and must submit a cancellation request". Every edit must also reach the systems that already copied the order: the WMS's pick, the ERP's invoice copy, the availability service's commitment. An edit that reaches the platform and not the warehouse ships the wrong size.

Holds. A hold stops fulfillment without canceling: a fraud review, an address check, a customer's request. The stock stays committed; Shopify documents that an order on hold keeps its inventory reserved while fulfillment waits.

Cancellations. Before acceptance, a cancellation releases stock and voids or refunds the payment. After shipment, a cancellation is not possible; what the customer calls "cancel" becomes a return, and the undo of a shipment is a business process (return label, receipt, inspection, refund) rather than a database rollback. This is a **compensation**: a later step that reverses an earlier one when the whole cannot be undone, the vocabulary of the saga pattern that Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation explains. The order system is, in practice, the coordinator of that sequence.

Routing, Sourcing and Splits

Once an order can be fulfilled from more than one location, three decisions appear that a single-warehouse merchant never makes. **Sourcing** is choosing which location fulfills which line. **Routing** is the rule set that makes that choice. A **split shipment** is one order

shipped in more than one parcel, usually from more than one location or at more than one time.

Platforms route by simple rules. Adobe Commerce ships source priority (work through a configured list) and distance priority (the nearest source to the shipping address), per its documentation updated 20 August 2026. Shopify assigns lines to a location through fulfillment orders, and the assigned location may change only while the fulfillment order is still open, scheduled or on hold. These rules know about stock and geography. They do not know about capacity, cut-off times, carrier cost, marketplace fees or a maximum number of parcels.

Order management vendors describe what the fuller decision looks like, and the descriptions are worth reading as a specification rather than as marketing. Kibo's routing API (documentation, n.d., accessed November 2026) returns a *suggestion* built from *candidates*, "a single, possible way to fulfill an item or group of items", logs why locations were considered or rejected, and "does not create shipments or deduct inventory"; when only part of a shipment is in stock, "the remaining unavailable quantity is automatically placed into a new child shipment for re-routing". Fluent Commerce's split logic "analyses inventory and costs across split scenarios, aiming to fulfil the maximum number of items while using the minimum number of shipments", and merchants may "enforce a maximum allowable cost or number of shipments for any given order" (documentation, n.d., accessed November 2026). Manhattan says an OMS assigns each order "to the fulfillment center that can meet the promise most profitably" (n.d., accessed November 2026).

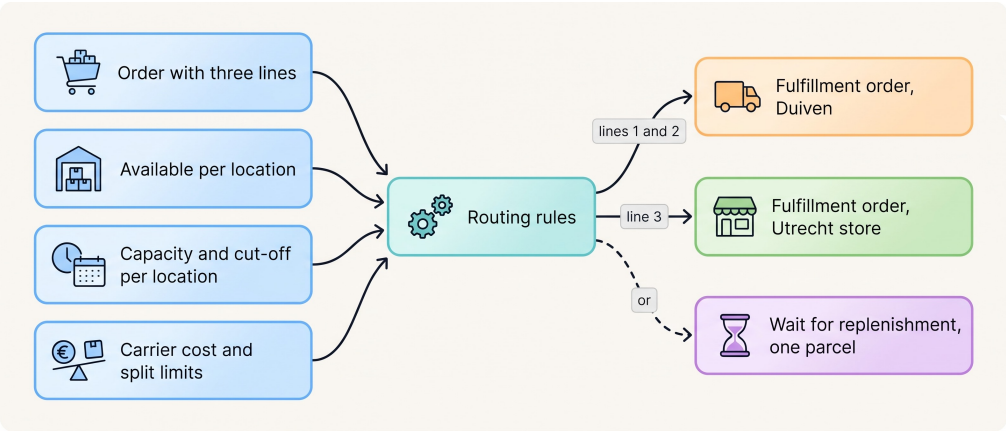


Figure 3: *Routing turns one order into one or more fulfillment orders using stock per location, capacity, cut-off times and cost; the same inputs decide whether to split now or wait and ship once.*

The business trade-off inside a split is concrete. Shipping the fleece now and the socks later keeps the promise on one line and breaks it quietly on the other, and costs a second

parcel. Waiting ships once but late. **A split is a cost decision that needs the carrier rates, the promise and the customer's channel as inputs**, and whichever system makes it must also tell the customer "two parcels", the message nobody at Kestrel & Pine could send in December 2026.

A **backorder**, a line accepted for a unit that is out of stock but expected, is a split in time rather than space, and needs the same honesty: a date from Is It in Stock? Inventory, Availability and the Promise You Can Keep's incoming quantities, and a message.

Many Channels, One Record

Kestrel & Pine's orders begin in four systems. Storefront and B2B (business-to-business) portal orders are created by the platform. Marketplace orders are pulled by the marketplace connector. Bergzicht's EDI orders land in the ERP, and 38% of wholesale orders are keyed into the ERP by hand. Store sales are POS receipts. Each source has its own connector to the warehouse, its own path to the books and its own status vocabulary. That is the Spaghetti Junction, the normal result of adding one channel at a time.

The alternative is one order hub: every channel's order is created in one system, with one state model and one number family, and that system drives the warehouse and informs the books. Which system that should be is the real question, with three honest candidates.

The order lives in	What it is good at	What it lacks	Common at
The platform	The customer-facing record: account history, status page, checkout context, returns through its API	Orders from channels the platform did not sell through; routing beyond priority and distance; service view across systems	Starting and Growing

The order lives in	What it is good at	What it lacks	Common at
The ERP	The financial record: invoice, credit note, receivable, cost; wholesale order entry	Customer-facing state; parcel-level goods state; speed (the ERP runs batches); the ERP partner becomes the integration owner	Growing merchants with strong wholesale
An OMS	The operational record: every channel, one state model, routing and splits, holds and edits, the service view	The checkout (stays on the platform) and the books (stay in the ERP); it is a third system to integrate	Omnichannel and Group

Fluent Commerce states the boundary plainly on its own site: "your ERP stays the system of record" for finance, while the OMS decides where orders go and writes back (n.d., accessed November 2026). The three records are not rivals; they are three truths about the same order, and the design decision is which system owns which truth and how the others are told (Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record owns the general idea). Waypoint W2 is that decision written down: the OMS becomes the order hub, every channel's order is created there, the platform keeps the checkout and the storefront order history, and the ERP keeps the financial record.

The OMS as a Product

An **order management system** takes orders from every channel, decides which location fulfills each line, tracks each order through fulfillment, delivery and returns, and gives service agents one view. **Orchestration** is its job of sequencing those steps across systems. The vendors describe the same shape: Manhattan lists inventory visibility across sources, promise dates at checkout, sourcing, "end-to-end fulfillment execution and tracking" and post-purchase changes (n.d., accessed November 2026); Kibo lists "distributed order orchestration across every store, warehouse, and vendor" with rules such as fewest shipments, lowest cost or closest distance (n.d.); IBM Sterling lists ship from store, in-store pickup and a call-center add-on "for complex order modifications" (n.d.).

What the OMS owns, and what it leaves alone, is the point.

Component	Responsibility	System of record for	Not for
Order intake	Create one order per channel event, keyed on the order number; attach channel references	The operational order	Checkout, pricing, payment
Availability module	Available per channel and location from stock events (Is It in Stock? Inventory, Availability and the Promise You Can Keep)	Available, committed	On hand
Routing	Choose locations, decide splits, create fulfillment orders	The sourcing decision	Picking
Orchestration	Drive the money, goods and return machines from events; request, accept, cancel with the warehouse	The order's state	The warehouse's internal states
Service view and edits	One screen per order across channels; edits validated against state	Edits and holds	Refund execution (the PSP), invoice (the ERP)

Component	Responsibility	System of record for	Not for
Events out	Status events to the platform (customer history), the ERP (invoice triggers), notifications, the data warehouse	Nothing; it publishes	Anything it did not change

Requirements per interaction

The figures are the author's illustrative targets for an Omnichannel merchant of Kestrel & Pine's size, not Kestrel & Pine's records.

Interaction	How fresh	How consistent	How fast	If the OMS is down
Order creation from a channel event	Seconds after confirmation	Exactly one order per order number	Under 2 s end to end *(assumption)*	Events queue; the platform's checkout keeps working; nothing is lost
Fulfillment order to the WMS	Before the next wave release; minutes at peak	Exactly one pick list per fulfillment order	Minutes	The WMS continues on what it has
Status event back to the storefront and service	Seconds to a minute	Per shipment, never collapsed to one word	Under 1 min *(assumption)*	The customer sees the last known state, dated
Edit or cancellation	Now	Validated against the goods state; propagated to WMS and ERP	Under 2 s *(assumption)*	Refuse the edit rather than apply it in one system only

Interaction	How fresh	How consistent	How fast	If the OMS is down
Invoice trigger to the ERP	Per shipment, at dispatch (Tax, Invoices and the Books: From Order to Ledger sets the rule)	Idempotent per shipment id	Daily batch is acceptable	Resume from the last acknowledged shipment

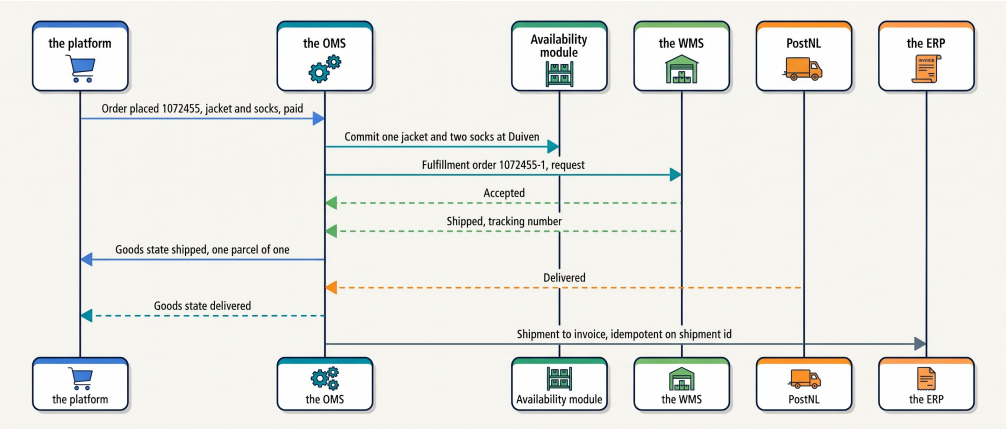


Figure 4: *Eva's order under Waypoint: the platform confirms and pays, the OMS commits stock, requests fulfillment, receives the warehouse's and the carrier's events, computes the goods state per parcel and publishes it to the storefront and the ERP.*

The rule I use for when an OMS is needed: **at least two of these four**. Several fulfillment locations. Several channels creating orders. Promises that depend on location (click-and-collect, ship-from-store). Service agents who cannot see one order across systems. One of them can be handled by the platform plus a connector; two make the joins the problem. Kestrel & Pine has had all four since 2023.

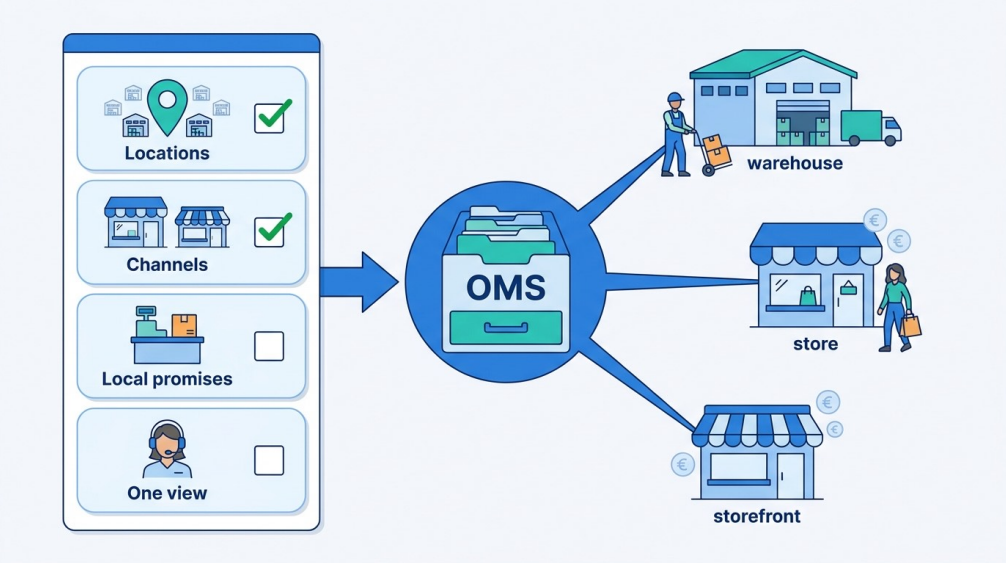


Figure 5: *An OMS becomes the order hub once two of four conditions hold: several locations, several channels, location-based promises, or service that cannot see one order.*

The Choices

1. **The platform's order is the record**. The platform creates and holds every order; marketplace apps and a POS from the same vendor create orders in it; a 3PL (a third-party logistics provider, a warehouse that ships for the merchant) or WMS connector reads them and writes fulfillments back. Shopify's own features, fulfillment orders with a request-and-accept handshake, holds, edits, returns and multi-location assignment, cover this well; what its documentation does not describe is rule-based sourcing on cost, capacity or cut-off, and that absence is the limit (an inference from the documentation, not a vendor statement).

2. **The ERP as the order hub**. Every channel's order is created in the ERP, which drives the warehouse and the books. Attractive where wholesale dominates and the ERP already holds most orders, and where the ERP partner is strong. The costs are speed (ERPs run in batches), a customer-facing state the ERP cannot show, and a hub owned by the finance system's vendor. The Back Office: ERP Integration and Who Is the Hub works through the hub debate.

3. **A bought OMS as the hub**. The platform keeps checkout; an OMS holds every order, routes, orchestrates and serves; the ERP keeps finance. This is W2. The price is a third core system, its subscription (illustratively €4,500 a month for Kestrel & Pine's OMS including its availability module, with a one-time implementation of about €320,000; see Build, Buy or Assemble: Choosing and Changing Platforms), and the integration work to re-point every order source and the warehouse.

4. **A built order service**. A small team writes its own order record, state machines and routing over a queue. Right for a vendor whose product is commerce, or a group with unusual rules and a platform team; wrong for a 14-person technology team with a warehouse to run. The order record is where custom code is most expensive to get wrong, because every other system depends on it.

Trade-offs

Criterion	1. Platform	2. ERP hub	3. Bought OMS	4. Built service
Orders from every channel	Only channels sold through the platform or its apps	Yes, often by hand or batch	Yes, by event	Yes
State model	The platform's; summaries collapse on the way out	The ERP's, financial first	One model, money, goods and return separate	Whatever is designed
Routing and splits	Priority and distance	Basic	Rules over stock, capacity, cut-off, cost	Custom
Service view across channels	The platform's orders only	The ERP's screen	One screen	Must be built
Time to value	Days	Months, with the ERP partner	Months (Kestrel & Pine: August to December 2027)	Quarters
Running cost	Included	ERP licenses and partner	Subscription plus integration	Team
Exit cost	Low	High, entangled with finance	Medium, the order events are reusable	High

Criterion	1. Platform	2. ERP hub	3. Bought OMS	4. Built service
Breaks when	A second channel or location arrives	Peak day; a customer asks the ERP a parcel question	A missed event; a routing rule nobody owns	The team changes

My judgement: stay with the platform's order until the second order source or the second location, then choose between the ERP and an OMS by asking who should answer a customer's parcel question. If the answer is "not the finance system", it is an OMS. Build only when order management *is* the product.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs the approach is outgrown
1 — Starting (Kestrel & Pine 2016–2018)	The platform's order is the only order; a 3PL app reads it; the accounting tool imports a daily summary	Order number continuity; the platform's own states used as they are	Orders keyed by hand into anything
2 — Growing (2019–2022)	The platform's order plus copies in the ERP and the 3PL; a marketplace app creates marketplace orders in the platform; support checks two systems	Idempotent order creation in the ERP keyed on the order number; a daily reconciliation of orders between platform and ERP; one status mapping written down	The ERP's order count differs from the platform's; the first partial shipment that reads "fulfilled"; store orders invisible online

Stage	What is typical	What becomes necessary	Warning signs the approach is outgrown
3 — Omnichannel (2023–, now)	Four order sources, four status vocabularies, connectors from each to the warehouse and the books	An OMS as hub with one number family and separate money, goods and return machines; routing rules with owners; status events per shipment to the storefront, service and ERP; edits validated against state	The Three-Status Order; eleven tickets; routing done by warehouse staff by phone; click-and-collect checked by e-mail
4 — Group	Several brands or countries, several warehouses, perhaps several OMS instances after acquisitions	One state model and one number family per unit, with shared definitions; routing across brands' stock with allocation rules; the OMS as the saga coordinator for every channel	Brands' orders routed to each other's stock without rules; the service tool reads two OMSs

The shape of the record does not change with scale: an order has lines, money, promises, goods per shipment and references at every stage. What changes is how many systems create orders and how many fulfill them, and the moment either count passes one is the moment the order needs a hub.

In Practice

The dated vendor evidence is above: Shopify's three status families and fulfillment-order handshake, commercetools' separate states, Adobe's sourcing algorithms (2026), Kibo's candidates and child shipments, Fluent's split rules and ERP boundary, Manhattan's and IBM's OMS descriptions, all accessed November 2026. One figure frames the volume: Shopify processed 13 billion webhooks in the period its CTO described for 2021

(engineering blog, 20 January 2022; a source more than three years old, so the figure is historical), the scale at which "your app might receive the same webhook more than once" becomes the design assumption behind idempotent order creation.

Kestrel & Pine, worked: W2 and the order that had three statuses

Waypoint W2 is being implemented through 2027, seam by seam (Moving Without Burning the Bridge: Replatforming and Migration describes the method). The OMS shadow-copied all orders in August, ran in parallel from 1 to 14 September (differences on about 1.4% of orders in the first week, mostly status mapping and timing, illustrative), took marketplace orders on 15 September, storefront orders on 13 October, paused for a change freeze around Black Friday, and takes EDI and manual wholesale orders on 8 December; the old order paths retire in January 2028. From then on every order, whatever its channel, is created in the OMS by an event keyed on its order number, gets a number from one family continuing the platform's sequence, and keeps the channel's own id as a reference.

Replay #1052877 through that design. The order is created in the OMS from the platform's "order placed" event. Routing finds the fleece available at Duiven and the socks only in bulk stock; the rule for a two-line D2C (direct-to-consumer) order under €150 is to wait up to one working day for replenishment before splitting (an illustrative rule), and on Monday morning the socks are still short, so the OMS creates two fulfillment orders and emits a "ships in two parcels" status that the status page and the notification service both receive. The fleece's shipment confirmation moves fulfillment order 1052877-1 to shipped; the carrier's scan moves it to delivered; the order's summary reads "Partially delivered: 1 of 2 parcels". The ERP receives a "shipment to invoice" event per shipment, idempotent on the shipment id, and invoices the fleece on 7 December and the socks on 14 December: one invoice per shipment, both carrying the one order number, the rule Sanne de Wit, the Finance Director, adopts for Waypoint (Tax, Invoices and the Books: From Order to Ledger). The agent opens one record and sees both parcels. The customer still waits six days for socks, a stock problem for Is It in Stock? Inventory, Availability and the Promise You Can Keep; what disappears is the lie, and with it most of the eleven tickets.

When It Goes Wrong

Failure	What it looks like	Detection	Recovery
Duplicate order	The "order placed" event arrives twice; the ERP or WMS creates two orders; the customer receives two parcels	Two records with one order number; a daily count of orders per source against the platform's count	Idempotent creation keyed on the order number; cancel the duplicate before it ships; refund if not
Lost confirmation	The parcel left; the confirmation never arrived; the order reads unfulfilled, the ERP never invoices	A daily reconciliation of shipped-in-WMS against shipped-in-OMS	Replay from the WMS; never re-key by hand (Pick, Pack, Ship: Warehouse Management and Fulfillment)
Collapsed status	A partial shipment reads "fulfilled"; a delivery scan reads "delivered" for the whole order	Orders with more than one fulfillment order and a single-word status	Goods state per fulfillment order; summary from the least-advanced shipment
Out-of-order events	"Delivered" arrives before "shipped"; a cancellation arrives after the pick started	A state transition the machine does not allow	Tolerant handlers: store the early event, apply it when its predecessor arrives; carrier events keyed on the tracking number
Edit after acceptance	The platform shows the new size; the box holds the old one	Edits committed while the fulfillment order is accepted or in progress	Lock edits at acceptance; turn later changes into a cancellation request plus a new order

Failure	What it looks like	Detection	Recovery
Cancel after shipment	Support "cancels" a shipped order; stock is released, money refunded, the parcel still arrives	Cancellations on orders with a shipped fulfillment order	Cancel becomes return; refund on receipt (The Way Back: Returns, Exchanges, Refunds and Reverse Logistics)
Routing nobody owns	Lines split across three locations for a €40 order; a store asked to ship what the warehouse has	Parcels per order and split cost per channel, tracked weekly	Named owner for routing rules; a maximum shipments per order; a split cost cap
Stuck invoice trigger	The ERP waits for "order complete" that a short pick prevents; nothing is invoiced for a week	Shipped-not-invoiced older than one day	Invoice per shipment, idempotent on the shipment id; an exception queue with an owner

Most rows end the same way: an identity on every message, a state machine that refuses impossible moves, and a daily comparison between the systems that hold copies. **An order hub does not remove the joins; it gives each join one key, one owner and one check.**

What to Take Away

Established.

- An order carries several state machines: Shopify keeps order, payment and fulfillment statuses on one order plus statuses on each fulfillment order; commercetools keeps order, shipment and payment states plus return states; in both, "fulfilled" means "delivered" only when carrier events are fed back (documentation accessed November 2026).
- The fulfillment order, with a request, accept or reject, in-progress, shipped and cancellation-request lifecycle, is the documented contract between order

management and the warehouse; edits are possible only on unfulfilled lines, and cancellations after acceptance are requests (Shopify documentation, n.d.).
- Order management vendors converge on one boundary: the OMS holds the operational order and the sourcing decision, the ERP the financial record, the warehouse the execution (Fluent, Kibo, Manhattan, IBM, accessed November 2026).

Depends on context.

- The platform's order is enough while one channel creates orders and one location fulfills them; the second of either makes an order hub necessary.
- The ERP can be the hub where wholesale dominates, the ERP partner is strong and no customer ever asks the finance system where a parcel is; an OMS is the hub where consumers, stores and marketplaces do.
- Invoice per shipment, not per completed order, wherever orders can split (Kestrel & Pine's rule); a merchant that never splits may invoice per order, but either trigger must be idempotent on the shipment or order id, or a short pick blocks the books.
- Split now or wait is a cost rule that depends on carrier rates, marketplace fees and the promise; it needs an owner and a cap, not a default.

The author's synthesis.

- The order's home should be chosen by asking who answers the customer's parcel question, because the system that can say "two parcels, one delivered" is the only one that can also route, edit and reconcile honestly; everything else follows from that record.
- Buy the OMS and keep the checkout on the platform and the books in the ERP; the order record is the worst place for a small team's custom code, because every other system fails when it does.

Questions to Ask

1. Which systems create orders today, and which single system is the order's home for operations, for the customer and for finance?
2. Does one order number family cover every channel, and is the order number the idempotency key for creating the order in every other system?
3. How many state machines does the order record keep, and is the goods state held per shipment or collapsed into one word?
4. What computes the customer-facing summary of a split order, and what message goes out when an order ships in two parcels?
5. When does the edit window close, and how does an edit reach the warehouse, the ERP and the stock commitment?

6. What are the routing rules, who owns them, and what is the maximum number of parcels or the maximum split cost per order?
7. What triggers the invoice, and what happens to it when one line is short-picked?
8. Which daily reconciliation compares orders and shipments across the platform, the hub, the warehouse and the ERP, and who reads it?
9. If the order hub is down for an hour, can checkout continue, and how are the missed events replayed?

Eva's order now has one home, one number, three honest state machines and a warehouse that has accepted the work. The next chapter follows the money and the paperwork that this record generates: the tax it carries, the invoice it becomes and the entries it posts in the books. Continue with Tax, Invoices and the Books: From Order to Ledger.

Sources and Evidence

Company, product and investor sources

- Order status, Shopify Help Center, n.d., accessed 2026-11 — order, payment and fulfillment status families; holds keep stock reserved.
- FulfillmentOrder object and Order management apps, Shopify developer documentation, n.d., accessed 2026-11 — fulfillment order statuses, request statuses, assigned location.
- Build for fulfillment services, Shopify developer documentation, n.d., accessed 2026-11 — the request, accept, reject and cancellation-request handshake.
- Edit orders, Shopify developer documentation, n.d., accessed 2026-11 — calculated orders, unfulfilled lines only, balance collection or refund.
- Webhooks, Shopify developer documentation, n.d., accessed 2026-11 — duplicate and unordered deliveries; reconciliation jobs.
- Cloud, Load, and Modular Code: What 2022 Looks Like for Shopify, Shopify Engineering, 2022-01-20 — 13 billion webhooks.
- Orders (OrderState, ShipmentState, PaymentState, return states) and States, commercetools documentation, n.d., accessed 2026-11 — separate state enumerations; custom state machines with optional transitions.
- Source algorithms and reservations, Adobe Commerce documentation, updated 2026-08-20 — source priority and distance priority.
- Order Routing and Fulfillment concept guide, Kibo documentation, n.d., accessed 2026-11; Order Management, Kibo, n.d. — candidates and suggestions, child shipments, routing criteria, edits validated against state.
- Split orders across multiple fulfilment locations, Fluent Commerce documentation, n.d., accessed 2026-11; Order management system, Fluent Commerce, n.d. — split rules, cost and shipment caps, "your ERP stays the system of record".

- What is an OMS? and Manhattan Active Order Management, Manhattan Associates, n.d., accessed 2026-11 — OMS definition and capabilities; Forrester Wave Q1 2025 placement as reported by the vendor.
- IBM Sterling Order Management, IBM, n.d., accessed 2026-11 — omnichannel fulfillment and the call-center add-on.

Standards, regulation and official documentation

- None cited directly; the invoice rules this chapter defers to are in Tax, Invoices and the Books: From Order to Ledger.

Research and practitioner sources

- Saga, Chris Richardson, microservices.io, n.d., accessed 2026-11 — compensation and orchestration vocabulary used for "cancel after shipment".

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

14. Tax, Invoices and the Books: From Order to Ledger

IN THIS CHAPTER, YOU WILL: Follow one order from the tax shown at checkout to the invoice, the credit note and the ledger entries, and learn what finance needs from the shop.

KEY POINTS:

- **Tax is computed from facts the shop owns and rules it should not own.** The ship-to address, the product's tax category and the customer's status (consumer or VAT-registered business) decide the rate; the rates, thresholds and mandates belong to a tax engine or one rate table, maintained once for every channel.
- **The invoice is a legal document, not a receipt.** It carries a gap-free sequential number, cannot be changed once issued, and is corrected only by a credit note that references it; one system per legal entity issues it, and in Belgium since January 2026 a business invoice travels as structured data over Peppol.

- **The books agree with the shop only when someone proves it.** Orders, the PSP's payout reports and the bank statement are three views of the same money; a monthly three-way match, keyed on stable identifiers, turns a commerce platform's promises into accounts an auditor will sign.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Boomerang** — refunds, credit notes and restocking that are matched by hand after the return, and sometimes twice, because no document references the one it corrects.
- **Schrödinger's Order** — the order that is *Delivered* in the platform and *Not invoiced* in the ERP for six days, because the invoice waits for an "order complete" message that a short pick prevents, and nobody watches for it.
- **The Long Fuse** — an e-invoicing mandate with a published date that becomes a project only when the first business customer refuses a PDF.

A commerce platform spends most of its life making promises to customers. Its last obligation on every order is to the state and to the company's own accountants: the right tax, a valid invoice, and books that match the bank. **The shop makes promises**; **the ledger keeps score**, and the handoff between them is where a finance team either trusts the systems or rebuilds everything in spreadsheets.

None of this is hard on one order. All of it is hard on 293,000 online orders, 43,000 marketplace orders, 118,000 store transactions and 9,400 wholesale orders a year, through four channels, three payment providers and three countries.

This chapter follows Eva Janssen's order at Kestrel & Pine, the fictional outdoor brand this book follows, from the VAT inside her €235 to the entries in the ERP, and then a Belgian wholesale order invoiced as structured data over Peppol. Payment mechanics stay in Taking the Money: Payment Providers, Authorization, Capture and Refunds; e-invoicing in depth belongs to Documents Between Companies: EDI, Punchout and E-Invoicing; the ERP as a hub to The Back Office: ERP Integration and Who Is the Hub. For the ledger as a designed thing, the sibling book RECONCILED goes deeper: Payments-to-Ledger: Draining the Suspense Swamp and Never Update, Only Append: Designing Custom Ledgers.

Where we are in the order's journey. This is the last step of Eva's jacket: after checkout, payment, picking, delivery and the exchange, the order becomes an invoice, a payout and ledger entries, and its tax lands in the Dutch VAT return.

Setting the Scene at Kestrel & Pine: Sanne's April Close

Sanne de Wit, the Finance Director, closes April 2027 in the second week of May. Four feeds must each agree with the ERP (the enterprise resource planning system: the books, invoicing, purchasing and the item master) before she can file the VAT returns for three countries: the **online PSP** (payment service provider, the company that takes the shopper's money through many methods and pays the merchant one net amount), which paid out every sales day in one batch net of fees, refunds and chargebacks (payments reversed by the cardholder's bank), about thirty batches of some 800 orders each *(illustrative)*; the **terminal provider**, which settled the six stores' card payments separately; the **three marketplaces**, each paying out on its own rhythm net of commission; and **1,100 wholesale accounts** paying invoices on 30-day terms, some short, some twice, some quoting an invoice number the ERP cannot find because the order was keyed in by hand.

Two things from the winter are still open. The **Double Refund** of January: 212 returns refunded by the returns portal and again by hand in the PSP console, €19,400 out twice; the credit notes in the ERP were right, the refunds were not, and nothing compared them. And the **Three-Status Order**, #1052877, *Delivered* in the platform and *Not invoiced* in the ERP for six days in December, because the ERP invoiced only on the warehouse's "order complete" message, which could not come while the socks waited for replenishment, and no report listed shipped orders without an invoice.

A third thing is new. Since 1 January 2026, businesses established in Belgium must exchange invoices with each other as structured electronic documents over the Peppol network (a European network for structured electronic invoices). Whether the obligation reaches a supplier with no Belgian establishment is not settled by the sources this book relies on (Documents Between Companies: EDI, Punchout and E-Invoicing reads the scope), but Kestrel & Pine's Belgian business customers' bookkeeping expects a Peppol invoice, so its ERP has sent them one since then through the access point of its EDI provider, the service that relays electronic business documents between companies. In April Ardennes Trail Co., a wholesale customer in Namur, rejected an invoice because the buyer reference it had asked for was missing. Sanne asks Noor two questions: **which system is allowed to issue an invoice, and how do we prove, every month, that the orders, the payouts and the bank agree without reading them one by one?**

What the Commerce System Owes Finance

In business terms the shop owes finance three outcomes. **The right tax**: every sale carries the VAT (value-added tax, Europe's consumption tax, charged at every sale and reclaimed by business buyers) of the right country at the right rate; tax authorities

enforce it. **A valid invoice**: numbered in sequence, never edited and kept for years; auditors and business customers, whose VAT deduction depends on it, enforce it. **Books that match the bank**: the ledger, the PSP's reports and the bank statement agree; the auditors and the board enforce it at every close.

A handful of documents carry those outcomes, and each has one owner. This is the chapter's version of the entity–owner table from Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record.

Document	Created when	Owner (system of record)	References
Order	Checkout completes	The platform (from Q4 2027 at Kestrel & Pine, the OMS, the order management system)	Cart, payment reference
Tax calculation	At checkout (estimate) and at invoicing (final)	The tax engine or rate table; recorded on the order and the invoice	Ship-to address, product tax category, customer status
Invoice	At dispatch of each shipment (goods) or at order (digital goods)	The ERP, one numbering sequence per legal entity	Order number, shipment id
Credit note	When a return is accepted or an invoice is wrong	The ERP, same sequence family	The invoice it corrects
Payment event	When the PSP authorizes, captures, refunds	The PSP; mirrored on the order	Order, invoice or credit note
Payout report	When the PSP pays the merchant's bank	The PSP	Every payment, refund, fee and chargeback in the batch
Ledger entries	When documents are posted	The ERP's general ledger	All of the above, through their identifiers

Two rules follow from the table and carry the rest of the chapter. **The shop does not issue invoices**: it issues order confirmations, which look like invoices to a consumer and are not; the ERP issues the legal document. **Nothing posts to the ledger from a webhook** (the HTTP call a PSP makes to the merchant's server when a payment event happens): payment events update the order; the ledger is fed from documents, in batches, with identifiers that let every entry be traced back.

VAT in Plain Words

Five facts about VAT carry the chapter.

Rates differ by country and by product category. EU law sets a minimum standard rate of 15% and lets each Member State apply reduced rates to listed categories (European Commission VAT rates page, n.d., accessed November 2026). Kestrel & Pine's three markets:

Country	Standard rate	Reduced rates	Source
Netherlands	21%	9%; 0% for exports and intra-EU supplies	business.gov.nl, n.d., accessed 2026-11
Germany	19%	7%	§ 12 UStG, consolidated text, accessed 2026-11
Belgium	21%	12%, 6%, 0%	PwC Worldwide Tax Summaries, September 2026 (the official page could not be fetched)

Kestrel & Pine's clothing and equipment carry the standard rate everywhere.

Consumer prices include VAT; business prices exclude it. Stripe Tax's documentation (n.d., accessed November 2026) puts it plainly: "Tax-inclusive prices are common for B2C sales in many markets outside the US", "Tax-exclusive prices are common in the US and Canada and for B2B sales in other countries", and once a price's tax behavior is set it cannot be changed. Eva's €199 jacket is tax-inclusive; the wholesale €95 is tax-exclusive. The pricing model is a property of the market and the customer type, not of the product (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax).

Cross-border consumer sales are taxed where the customer is. Once a merchant's sales to consumers in other EU countries exceed €10,000 a year, it charges the *destination* country's VAT, and since 1 July 2021 the EU's One Stop Shop (OSS) lets it report all of that

VAT in one quarterly return at home (European Commission OSS page, n.d., accessed November 2026). Kestrel & Pine's €8.5 million of Belgian and German online sales passed that threshold years ago, so a Belgian shopper pays 21% Belgian VAT, a German one 19% German VAT, and both are reported through the Dutch OSS return. The shop must know the destination rate per product category *at checkout*.

Business sales to another EU country carry no VAT. With a valid customer VAT number, checked in the EU's VIES service, the seller charges 0% and the buyer accounts for the VAT in its own return, the **reverse charge** (Your Europe cross-border VAT page, n.d., accessed November 2026). The invoice states both VAT numbers and the reverse-charge mention. Ardennes Trail Co.'s order, below, is this case.

The United States is the contrast. There is no VAT; each state levies its own sales tax, and since *South Dakota v. Wayfair* (21 June 2018) a state may require a remote seller to collect it once sales into the state pass a threshold (South Dakota's was $100,000 or 200 transactions), regardless of physical presence. Rates vary by state, county and city, which is why **tax engines** exist: Avalara's product page claims rules across "12,000+ U.S. sales and use tax jurisdictions" (n.d., accessed November 2026). A Dutch merchant that opens a US storefront inherits all of this on day one.

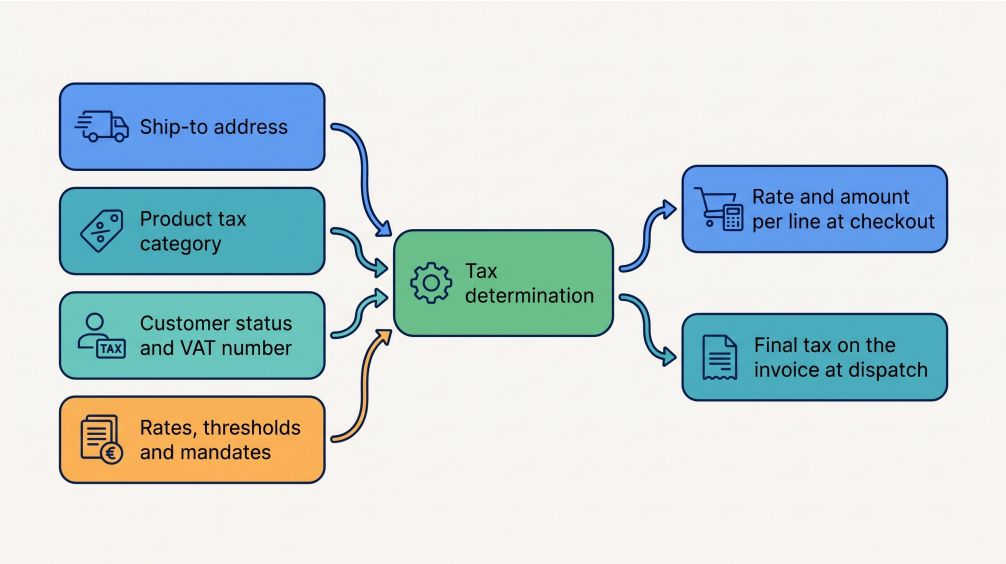


Figure 1: *Three facts the shop owns and one set of rules it should not own decide the tax, once as a promise at checkout and once as the legal amount on the invoice.*

Tax at Checkout and Tax at Invoice

The tax a shopper sees at checkout is a **promise**, computed from the ship-to address, the product category and the customer's status at that moment. The tax that matters legally is the one on the **invoice**, computed when the supply happens, which for goods is dispatch. Between the two, a line may be removed, an order split or a rate changed on 1 January. The order system has two honest options: re-run the determination at invoicing and treat the checkout figure as an estimate, or freeze the checkout result as the tax basis on the order and reconcile any difference explicitly. It must never let the two drift silently. Tax engines are commonly used the same way: a calculation at checkout that records nothing, and a committed document at invoicing.

Flow	How fresh	How consistent	How fast	If it is down
Tax at checkout	Per address and cart, at the price review step	Recorded on the order as the basis	Under 200 ms *(assumption)*	Fall back to the platform's rate table, flagged for finance
Tax at invoicing	At dispatch, from the shipped lines	Final; the invoice is immutable	Batch	Hold the invoice, not the shipment

When a cart mixes rates, the shipping charge and any basket-wide discount must be split across lines; Stripe Tax documents two methods, proportional allocation and the highest rate. Discount allocation is as much a tax problem as a promotions problem (Discounts, Coupons and Loyalty: Promotions Without Chaos): it decides what the credit note for one returned line is worth.

The Invoice as a Legal Document

An **invoice** states what was sold, to whom, when, for how much and with how much VAT, and its content is prescribed. The Dutch tax authority's invoice requirements page (n.d., accessed November 2026) lists both parties' names and addresses, the supplier's VAT identification number (and the customer's for reverse-charge and intra-EU supplies), the invoice number and date, the date of supply, the quantity and nature of the goods, and per rate the net amounts, the rate and the VAT amount. Germany's § 14 UStG (consolidated text, accessed November 2026) requires the same and spells

out the number: "eine fortlaufende Nummer mit einer oder mehreren Zahlenreihen", a consecutive number from one or more number series, unique to the invoice.

Three properties follow, and each is an architectural rule.

The number is sequential and gap-free within its series. So it is assigned by the system of record at the moment the invoice is posted, never pre-allocated by checkout, never reused, and never taken from an order-number sequence that has cancellations in it.

The invoice is immutable. Odoo's accounting documentation (version 18.0, accessed November 2026) describes the standard ERP behavior: an invoice is created in draft, receives its number from a sequence on confirmation, and "once confirmed, an invoice can no longer be updated". Any system that lets a user edit an issued invoice is not issuing invoices.

Corrections are documents. The same documentation states that "issuing a credit/debit note is the only legal method for canceling, refunding, or modifying a validated invoice". A **credit note** is the invoice's mirror image: it references the invoice, reverses some or all of its lines, and carries its own sequential number. A return never edits the invoice; it produces a credit note that references it, and a refund that references both, and the three reconcile to zero. The Double Refund happened because the second refund referenced nothing.

Figure 2: *An issued invoice is immutable: a return never edits it but produces a credit note that references it and carries its own number.*

One invoice per shipment, not per order. When an order ships in two parcels, Kestrel & Pine issues two invoices, each for the lines in its parcel, dated at its dispatch

and carrying the one order number; wholesale follows the same rule, one invoice per delivery. The supply happens per shipment, so an invoice per shipment carries the right date of supply, while an invoice that waits for the whole order waits for the slowest line, which is how #1052877 sat *Not invoiced* for six days. The trigger is the warehouse's shipment confirmation, idempotent on the shipment id. Invoicing per order works only for a merchant that never splits.

Invoices are kept for years: seven for basic records in the Netherlands (Belastingdienst retention page, n.d., accessed November 2026), eight for invoices in Germany (§ 14b UStG) and ten for books (§ 147 AO). The invoice must outlive the commerce platform, one reason the ERP, not the shop, is its system of record (Moving Without Burning the Bridge: Replatforming and Migration).

E-Invoicing at a Glance

An **e-invoice** is an invoice issued, transmitted and received as structured data (XML) that the receiver's software can process; a PDF by e-mail is not one. The European standard EN 16931 defines what it contains; **Peppol** is the open network through which a business reaches any other participant, each side connecting through its own accredited **access point** (OpenPeppol, n.d., accessed November 2026). Credit notes travel the same way.

The dates are the requirement, and they differ per country. Belgium made structured e-invoices mandatory for domestic business-to-business transactions from 1 January 2026, over Peppol (einvoice.belgium.be, 18 September 2024). Germany has required every business to be able to *receive* e-invoices since 1 January 2025 and phases in the obligation to *issue* them from 2027, or 2028 for issuers with prior-year turnover up to €800,000 (Federal Ministry of Finance FAQ, accessed November 2026). The Netherlands has no domestic mandate; the cabinet announced on 11 September 2026 that it intends one from 1 July 2030. The EU's ViDA package, adopted in March 2025, requires digital reporting based on e-invoicing for cross-border business transactions from 1 July 2030 (European Commission, accessed November 2026). The Long Fuse is lit for the German one. Documents Between Companies: EDI, Punchout and E-Invoicing owns the formats and the mandates; Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation owns the calendar.

What matters here is ownership. An e-invoice has a transmission status (sent, delivered, rejected by the receiver's validator) that a PDF never had, so the system that issues the invoice must also hold that status and the archive. At Kestrel & Pine that is the ERP; Ardennes Trail Co.'s rejection was a missing buyer reference (Peppol's BT-10 field) that the B2B portal never asked for, a checkout change rather than a finance one.

From Order to Ledger: What Gets Posted

A **ledger** (the general ledger) is the book of accounts in which every transaction is recorded twice, as a debit and a credit, so that the books always balance. The ERP posts from documents, not from events. For a consumer order paid through a PSP the standard entries are these (account names vary by company):

Event	Debit	Credit
Invoice at dispatch	Accounts receivable, PSP (gross)	Revenue (net), VAT payable
PSP payout received	Bank (net); payment fees (expense)	Accounts receivable, PSP (gross)
Credit note after a return, refund deducted from a later payout	Revenue (reversal), VAT payable (reversal)	Accounts receivable, PSP
Chargeback	Dispute expense (or a receivable while defended), dispute fee	Accounts receivable, PSP
Gift card sold	Bank or PSP receivable	Gift-card liability (no revenue yet)

Three accounting rules sit under the table.

Revenue is recognized when control of the goods passes to the customer, not when the order is placed or paid. IFRS 15, effective from 1 January 2018, recognizes revenue when a performance obligation is satisfied, which for goods is the transfer of control, in practice dispatch or delivery depending on the terms (IFRS Foundation, n.d., accessed November 2026). Kestrel & Pine recognizes consumer revenue at delivery and wholesale revenue at delivery to the customer's shop or distribution center; "shipped" is the operational milestone, not the accounting one. Because the invoice is issued at dispatch, finance makes a cut-off adjustment at month end that moves parcels invoiced but not yet delivered into the next month: an order shipped on 30 April and delivered on 3 May is May revenue on an April invoice. This is why the shop's "sales today" dashboard and the ledger never agree to the euro (Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics).

A right of return is a liability. With a 19% order return rate, Kestrel & Pine expects one order in five to come back in part; under IFRS 15 the expected refunds are a liability estimated at period end (the author's reading of the standard).

Gift cards are liabilities until redeemed. Selling a €50 gift card is €50 owed, not revenue, until the card pays for goods; the part never redeemed (breakage) follows rules

that depend on the jurisdiction (RevenueHub's summary of the US rules, n.d.). The gift-card balance is a finance fact, and only one system may change it (Discounts, Coupons and Loyalty: Promotions Without Chaos).

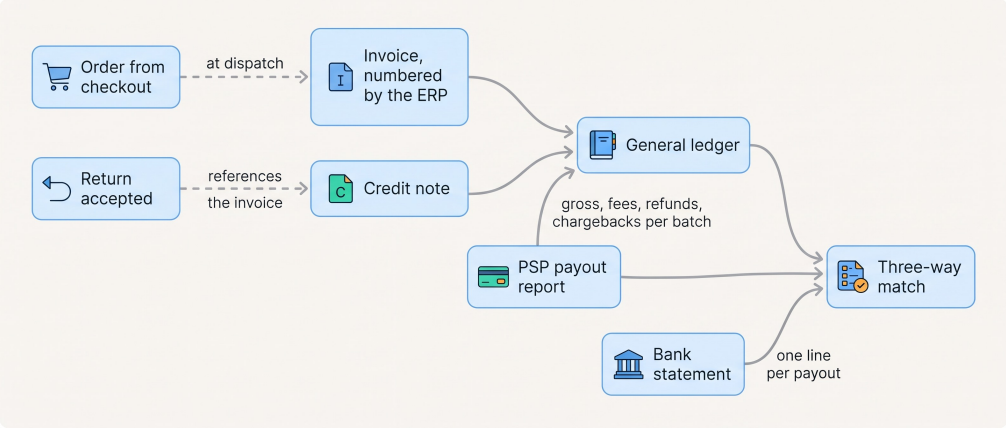


Figure 3: *Documents, not events, feed the ledger: the invoice at dispatch, the credit note on return, the payout report per batch, and the bank line that the monthly match ties them to.*

Reconciliation: Orders, Payouts and the Bank

Reconciliation is proving that two records of the same money agree, line by line, and explaining every difference. Commerce has a three-way version: the ERP's invoices and credit notes, the PSP's payout reports, the bank statement.

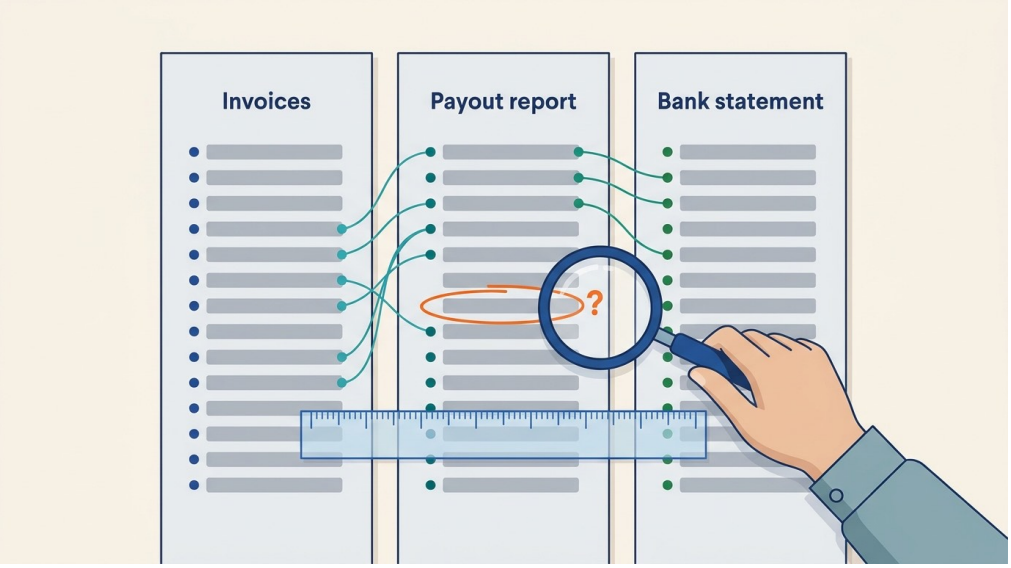


Figure 4: *Reconciliation lines up the ERP's documents, the PSP's payout report and the bank statement on shared keys, and every unmatched line needs an explanation.*

The PSP's side is the best documented. Stripe's payout reconciliation report (n.d., accessed November 2026) lists "every payment, refund, dispute, fee, and other balance transaction included in the payout", can carry the merchant's own order id, and gives a `trace_id`, "a unique identifier generated by bank for the Stripe payouts", that matches the bank line. Adyen's batch-level reconciliation keys everything on a batch number, and Mollie's settlement report does the same per settlement. In all three, the join keys are the merchant's order reference on the way in and the PSP's batch or trace reference on the way out. **A merchant that does not pass its order number to the PSP cannot reconcile at the order level**; it can only reconcile totals.

What does not match	Why	How to catch it
Captured in April, paid out in May	Settlement delay (Adyen's default two business days; Stripe's EU default several days)	Reconcile by sales day, not by calendar month alone
Payout smaller than the day's sales	Fees netted inside the batch; earlier refunds and chargebacks deducted	Post fees from the report, not from a rate card
A refund in the bank with no credit note	Someone refunded in the PSP console (the Double Refund)	A daily job listing PSP refunds without a matching credit note

What does not match	Why	How to catch it
A credit note with no refund	The return was accepted but the refund call failed	The same job, in the other direction
A failed refund returning weeks later	Adyen notes refunds can fail after acceptance; Stripe says the money returns in up to 30 days	A refund state on the order that is not terminal until the PSP says so
Four payout feeds	Online PSP, terminal provider, marketplaces, B2B receivables	One reconciliation job per feed, one owner (W5)

The Double Refund is the table's third row. The fix had two parts: one writer for refunds (the returns portal, through the platform; the ownership rule from The Way Back: Returns, Exchanges, Refunds and Reverse Logistics), and a daily comparison of PSP refunds against credit notes, a reconciliation job in the sense of Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation. RECONCILED's Reconciled by Design: Traceability and Audit Across Boundaries treats the discipline in full.

The Choices

Three decisions shape the handoff, each with three realistic answers.

Where is tax computed? In the platform's rate tables, in a **tax engine** (Avalara, Vertex, TaxJar and others) called from checkout and invoicing, necessary for the United States, or in the ERP at invoicing only, which is wrong for a consumer shop that must show the final price.

Where is the invoice issued? By the platform, by the ERP, or by an invoicing service in front of several ERPs, the Group answer where some countries run **state clearance** (the tax authority's platform must accept an invoice before it is valid, as in Italy and Poland).

How does the ledger learn? From one posting per paid order, from documents in a daily idempotent batch (the W5 rule at Kestrel & Pine), or from summary journals with the detail in a **sub-ledger**, a detailed book of every order whose totals feed the general ledger (Never Update, Only Append: Designing Custom Ledgers).

Decision	Simplest answer and its risk	Usual answer and its risk	Scaled answer
Tax computed	Platform rate tables; a rate maintained in four places	Tax engine or one rate service behind every channel; cost per call	Engine plus nexus tracking per entity
Invoice issued	The platform; a second numbering authority once the ERP exists	The ERP, one sequence per legal entity; connector failures hide invoices	An invoicing service in front of several ERPs
Ledger fed	Posting per order on payment; revenue before dispatch, duplicates on retry	Documents in a daily idempotent batch; a failed batch must be noticed	Summary journals plus a sub-ledger; detail outside the general ledger

My recommendation for a merchant at Kestrel & Pine's stage is the usual answer in each row: destination rates maintained once, invoices and credit notes from the ERP only, documents posted in a daily idempotent batch with a job that lists the orders that should have been invoiced and were not. The Three-Status Order was six days of *Not invoiced* because that last job did not exist.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting (under about €2M)	The platform's tax tables and tax-inclusive home-market prices; the platform's invoice or an accounting package fed by order export; one PSP payout feed matched by the bookkeeper	Gap-free, immutable invoice numbers; the order number passed to the PSP	The €10,000 OSS threshold crossed without noticing; invoices edited after issue
2 — Growing (€2M–€20M)	OSS registration; destination rates per market; an ERP or accounting system as the invoice's system of record, fed by a connector; VIES checks and reverse-charge invoices for business customers	The connector idempotent per shipment and per return; invoices at dispatch, not at payment; a daily job for orders not invoiced; refunds from one writer	Month-end takes a week; refunds reconciled by hand; revenue posted before dispatch; the first Peppol request from a Belgian customer

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
3 — Omnichannel (€20M–€200M)	Four payout feeds (online PSP, terminals, marketplaces, B2B); invoicing out of the shop entirely, one sequence per legal entity across every channel; a tax engine or one rate service behind every channel; Peppol from the ERP	A reconciliation job per feed with a named owner; order-level matching or a sub-ledger so the general ledger stays readable; a regulatory calendar for three countries' mandates	The Double Refund; three definitions of "revenue"; marketplace VAT booked as if the merchant collected it
4 — Group	Several legal entities and countries, some with state clearance systems; US nexus tracking; an invoicing service in front of several ERPs; a finance data pipeline consuming PSP reporting APIs	The order-to-ledger contract versioned and tested like any integration; one chart-of-accounts mapping per entity; intercompany invoices for stock moved between entities	Each brand's shop computes VAT its own way; month-end depends on one spreadsheet; an acquisition brings a second numbering authority

The shape of the handoff does not change with scale: documents, identifiers, one issuer, one match. What changes is the number of feeds and legal entities, and whether anyone can still do the match by eye. Kestrel & Pine's answer under Waypoint is W5: finance moves by daily batch, with a reconciliation job per flow.

Eva's Order in the Books, and Ardennes Trail Co.'s Peppol Invoice

Eva's jacket. Order #1072455, placed Tuesday 13 April 2027 at 21:14 on the Dutch storefront: a Ridgewalker jacket at €199 and two pairs of merino socks at €18, €235 in

all, free shipping, paid by iDEAL (the Dutch bank-transfer method most online shoppers there use). Dutch consumer prices include 21% VAT, so the tax basis frozen on the order at checkout is:

Line	Gross	Net	VAT (21%)
Ridgewalker jacket, women's M, moss	€199.00	€164.46	€34.54
Merino hiking socks × 2	€36.00	€29.75	€6.25
Shipping	€0.00	€0.00	€0.00
Total	**€235.00**	**€194.21**	**€40.79**

The payment: iDEAL has no separate authorization and capture; the money moves when Eva confirms in her bank app at 21:14, and the PSP's webhook marks the order paid. Assume a PSP fee of about €0.30 for an iDEAL payment *(assumption, in the range of published PSP price lists)*.

The invoice: on Wednesday 14 April the WMS (the warehouse management system) confirms the shipment; that evening the ERP connector's daily batch creates invoice 2027-041822 *(illustrative number)* from the order, with the order number on it, and the ERP posts: debit PSP receivable €235.00; credit revenue €194.21; credit VAT payable €40.79. April revenue, because PostNL delivered the parcel in April, on 15 April.

The payout: the PSP pays out each sales day in one batch two business days later *(assumption, matching one PSP's documented default)*. Thursday 15 April's payout for the 13 April sales day contains Eva's €235 among some 800 orders, net of fees. Finance posts it from the PSP's report: debit bank and payment fees, credit PSP receivable. The bank line carries the PSP's batch reference, which is how the May close ties the three together.

The exchange: on 22 April the size M jacket is received and graded resellable and the size L ships. In the books this is a credit note referencing invoice 2027-041822 for the jacket line (net €164.46, VAT €34.54) and a new invoice for the size L at the same amounts; no money moves. In the refund variant the credit note is the same, the returns portal triggers a €199 iDEAL refund through the platform, the PSP deducts it from a later batch, and the daily refund-versus-credit-note job sees one of each.

Ardennes Trail Co.'s invoice. On Wednesday 21 April 2027 Ardennes Trail Co. (fictional), an outdoor shop in Namur, Belgium, orders 40 Ridgewalker jackets at the wholesale price of €95 through the B2B (business-to-business) portal, €3,800 excluding VAT *(illustrative order, at Kestrel & Pine's wholesale price)*; it ships from Duiven by bpost. The portal validates its Belgian VAT number in VIES; the order is an intra-EU business supply, so the VAT line is 0% with the reverse-charge mention, and payment is on 30-day terms, so no PSP is involved. The ERP issues the invoice at dispatch on 23 April from the same Dutch sequence, with both VAT numbers, the buyer reference (BT-10) and the order number, and sends it as a Peppol BIS Billing 3.0 invoice through

the EDI provider's access point, recording the delivery receipt. The entry is debit trade receivable €3,800, credit revenue €3,800 (April revenue once bpost delivers), no VAT; the receivable is due 23 May and is cleared when the bank transfer arrives quoting the invoice number. The Dutch VAT return reports the supply in the intra-EU sales listing.

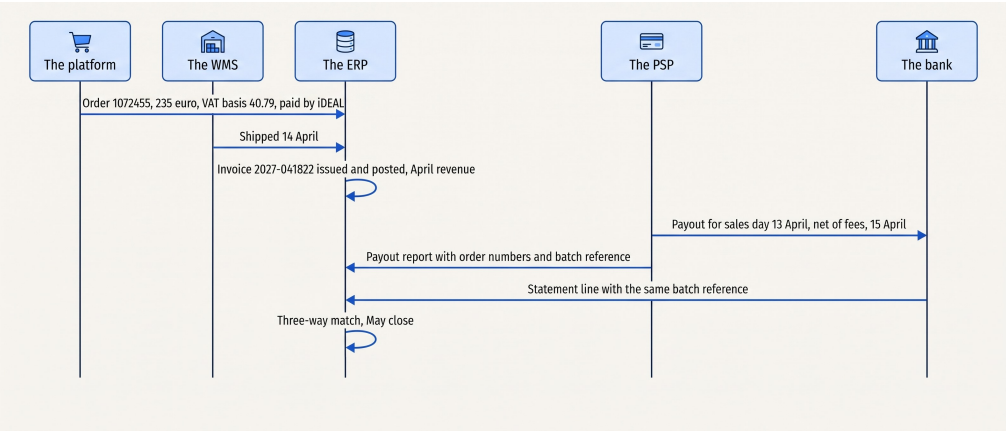


Figure 5: *Eva's order reaches the ledger as an invoice at dispatch and a payout batch two days later; the order number and the batch reference are the keys that let the May close prove the three records agree.*

When It Goes Wrong

The invoice that never came. The invoice waits for a trigger that does not arrive: an "order complete" message that a short pick prevents (the Three-Status Order), or a connector batch that fails on one order and continues with the rest. The order is *Delivered* in the platform and *Not invoiced* in the ERP for days. Detect it with a daily list of shipped orders older than one day with no invoice, the single most valuable report in this chapter; recover by invoicing the listed shipments, which requires the trigger to be idempotent per shipment.

The invoice that came twice. The connector retried after a timeout and the ERP created two invoices for one order. Detect it as two invoices referencing one order number; recover with a credit note against the duplicate, never a deletion, because the sequence must stay gap-free; prevent it with an idempotency key per shipment on the invoice-creation call.

The tax that differs. The invoice VAT differs from the checkout VAT because the address, the lines or the rate changed in between. Compare the order's frozen tax basis with the invoice at posting and list the differences. The invoice is right by definition; if the customer was promised a lower gross price, the difference is a commercial discount, not a tax correction.

The refund that doubled, and the payout that does not match. Both are the reconciliation table above: one writer for refunds, a daily job comparing PSP refunds with credit notes in both directions, and every payout residual investigated in the same month, because a residual carried forward becomes next quarter's write-off.

The mandate discovered late. A Belgian or German customer rejects a PDF, and the ERP partner quotes four months to add Peppol. A regulatory calendar with the mandates per country, their dates and an owner (Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation) is the detection; an access point in front of the existing ERP is usually faster than the partner's quote. The Long Fuse is a governance failure.

What to Take Away

Established.

- VAT follows the destination for cross-border consumer sales above the €10,000 OSS threshold, is reverse-charged for business sales to another EU country with a valid VAT number, and differs by country and category (European Commission and national tax authority pages, accessed November 2026).
- An invoice has prescribed content and a sequential gap-free number, is immutable once issued and is corrected only by a credit note that references it (Belastingdienst; § 14 UStG; Odoo documentation describing standard ERP behavior).
- Revenue for goods is recognized when control transfers, at dispatch or delivery depending on the terms, not at order or payment (IFRS 15; Kestrel & Pine uses delivery); gift cards are liabilities until redeemed.
- Belgium requires structured business-to-business e-invoices over Peppol since 1 January 2026; Germany phases in issuing from 2027; the Netherlands plans 2030; ViDA requires cross-border digital reporting from 1 July 2030.

Depends on context.

- The platform's own tax tables suffice for one or two countries with standard-rate products; a tax engine becomes necessary with many categories, many countries or any US sales, at a cost per call.
- The platform may issue invoices at the Starting stage if numbering is gap-free and immutable; from the Growing stage, one issuer per legal entity (usually the ERP) for every channel, because stores, marketplaces and EDI customers otherwise bring their own numbering.
- Summary journals with a sub-ledger are the right ledger shape at high volume or across several entities; at Kestrel & Pine's volume, order-level postings in a daily batch keep the general ledger readable.

- Recomputing tax at invoicing is right when addresses and baskets change between checkout and dispatch (split shipments, B2B); freezing the checkout basis is right for a consumer shop that ships whole orders within a day.

The author's synthesis.

- The order-to-ledger handoff is a small set of documents with stable identifiers, nothing more: shipment → invoice at dispatch, return → credit note, payout report → bank. Every failure in this chapter is a document issued by the wrong system, twice, or without a reference.
- The report that lists shipped orders with no invoice, and the job that compares PSP refunds with credit notes, are worth more than any finance dashboard; build them in the first month, before the OMS and the PIM.

Questions to Ask

1. Which system issues our invoices, and is there exactly one numbering sequence per legal entity across web, stores, marketplaces and EDI?
2. Is the invoice created at dispatch or at payment? What does our revenue figure mean on the last day of the month?
3. Which facts decide the tax at checkout, and in how many places are the rates maintained?
4. Are we registered for the One Stop Shop, and does the checkout charge the destination country's rate per product category?
5. Do we pass our order number to the PSP, and does its payout report come back with it and with the bank's batch reference?
6. How many payout feeds do we reconcile, who owns each, and how long did the last close take?
7. Which e-invoicing mandates apply to our customers by country, from which dates, and who owns the calendar?

Eva's jacket has now been promised, paid, picked, delivered, exchanged and accounted for. The promise was made in Part III; keeping it is the work of the warehouse, the carriers, the stores and the returns flow, where the next part begins: Part IV — FULFILLMENT: From Warehouse to Doorstep and Back, starting with Pick, Pack, Ship: Warehouse Management and Fulfillment.

Sources and Evidence

Company, product and investor sources

- Specify product tax codes and tax behavior, Stripe Tax documentation, n.d., accessed 2026-11 — tax-inclusive versus tax-exclusive pricing and shipping allocation methods.
- Payout reconciliation report, Stripe documentation, n.d., accessed 2026-11 — report contents, merchant order ids and the bank trace id.
- Refund and cancel payments, Stripe documentation, n.d., accessed 2026-11 — refund timing and failed refunds.
- Batch-level reconciliation and Sales day payout, Adyen documentation, n.d., accessed 2026-11 — batch numbers, net credit and the default payout delay.
- Refund, Adyen documentation, n.d., accessed 2026-11 — refunds that fail after acceptance.
- Settlement report, Mollie documentation, n.d., accessed 2026-11 — settlement totals and line items.
- AvaTax calculations, Avalara, n.d., accessed 2026-11, and TaxJar API reference, TaxJar, n.d., accessed 2026-11 — what a tax engine takes and returns (vendor pages).
- Customer invoices and Credit notes and refunds, Odoo documentation 18.0, 2024–2025 — draft-to-confirmed numbering, immutability and credit notes as the only correction.
- Synchronize orders, Microsoft Learn, 2026 — how a documented ERP connector creates sales orders, invoices and credit memos from shop orders and retries pending refunds.
- ERP integration, Shopify, updated 2026-08-28 — financials in real time or daily batch; name the system of record per field.

Standards, regulation and official documentation

- VAT rates, European Commission, n.d., accessed 2026-11 — the 15% minimum standard rate and reduced-rate rules.
- VAT rates and exemptions, business.gov.nl, n.d., accessed 2026-11 — Dutch rates.
- § 12 UStG, § 14 UStG and § 14b UStG, gesetze-im-internet.de, consolidated text, accessed 2026-11 — German rates, invoice content and numbering, eight-year invoice retention.
- § 147 AO, gesetze-im-internet.de, consolidated text, accessed 2026-11 — German retention of books and records.
- Belgium: other taxes, PwC Worldwide Tax Summaries, 2026 — Belgian VAT rates (secondary source; the official page could not be fetched).
- Cross-border VAT, Your Europe, n.d., accessed 2026-11 — reverse charge and VIES.
- One Stop Shop, European Commission, n.d., accessed 2026-11 — the 1 July 2021 scheme and the €10,000 threshold.
- Invoice requirements, Belastingdienst, n.d., accessed 2026-11 — Dutch invoice content.

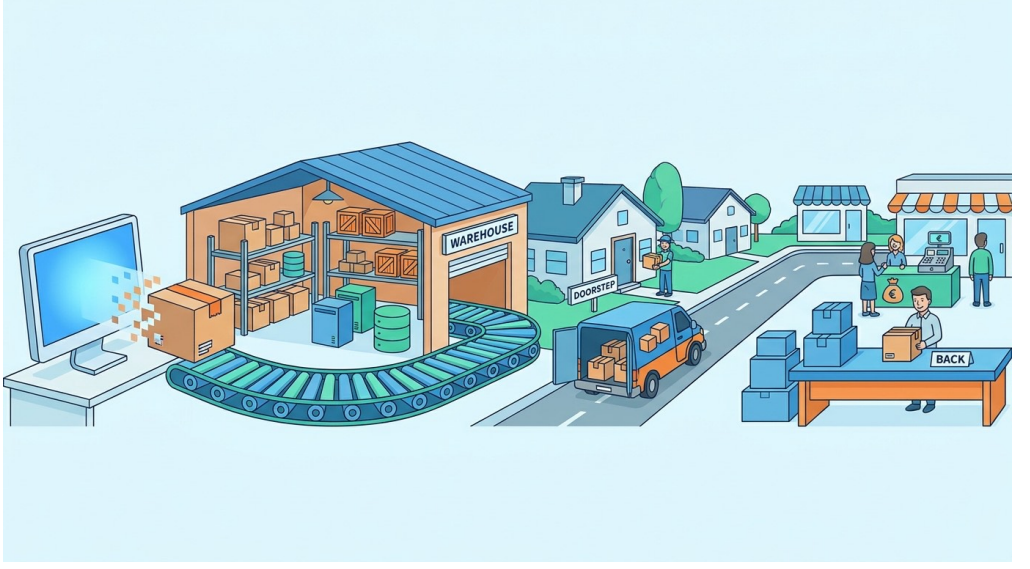
- Hoe lang moet u uw administratie bewaren, Belastingdienst, n.d., accessed 2026-11 — seven-year retention of basic records, ten for immovable property.
- South Dakota v. Wayfair, Inc., US Supreme Court via Cornell LII, 2018 — economic nexus.
- IFRS 15 Revenue from Contracts with Customers, IFRS Foundation, n.d., accessed 2026-11 — the five-step model and effective date.
- About Peppol, OpenPeppol, n.d., accessed 2026-11, and Peppol BIS Billing 3.0, OpenPeppol, 2026 — the four-corner model, document types 380 and 381.
- Structured electronic invoices between companies are compulsory in 2026, FPS Finance Belgium, 2024 — the Belgian mandate.
- FAQ E-Rechnung, German Federal Ministry of Finance, n.d., accessed 2026-11 — receipt since 2025 and the phased issuing obligation.
- Kabinet kiest voor invoering e-facturatie en rapportage voor bedrijven, Rijksoverheid, 2026 — the Dutch plan for 1 July 2030.
- VAT in the Digital Age (ViDA), European Commission, n.d., accessed 2026-11 — adoption in March 2025 and the 2030 reporting date.

Research and practitioner sources

- Unexercised rights (breakage), RevenueHub, n.d., accessed 2026-11 — gift-card breakage under the US standard, used as a proxy for the IFRS wording.
- Payment reconciliation 101, Stripe, 2025 — reconciliation as comparing transaction data with the books and investigating differences.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Part IV — FULFILLMENT: From Warehouse to Doorstep and Back



IN THIS PART, YOU WILL: Follow the order out of the software and into the building, the van, the store counter and the returns bench, and learn which system owns each physical fact, how the others learn of it, and what the customer is told along the way.

KEY POINTS:

- **The physical world has its own systems of record.** The warehouse system is the only one that counts shelves, the carrier's scan is the only proof of delivery, the till keeps selling when the network is down,

and the returns bench is the first to know what actually came back. Everything upstream derives from their events; nothing upstream may overwrite them.

- **A promise about the physical world is a computation, not a sentence**. "Delivered tomorrow" needs the last pick wave, the backlog in the order feed and the carrier's transit on that lane; "ready for collection" needs a store stock event, a buffer and a person's confirmation; "your refund is on its way" needs the warehouse's inspection. Every incident in this part is a promise made without its inputs.
- **One writer per fact, one source per view**. The refund has one writer keyed by the return id; stock rises only when the warehouse's verdict on a returned item (its disposition) says "restocked"; the status page, the notifications and the agent's screen read the same order record. When they read different systems, the customer finds the difference and writes in.

FAILURE PATTERNS THIS PART ADDRESSES:

- **The Phantom Stock** — store stock copied nightly is promised online as if counted this minute, and a returned jacket is restocked before anyone has looked at it.
- **Schrödinger's Order** — the platform says Delivered, the warehouse says In picking, finance says Not invoiced, and the agent opens five tabs to find out which is true.
- **The Boomerang** — a returns portal and a support agent can each refund the same return, and finance finds the second refund at month-end.
- **The First Snowfall** — a static delivery promise stays on the page while the order feed to the warehouse backs up for four hours on the busiest evening of the year.

Parts II and III happened in software. A product was shown, a price promised, money taken, an order record written. None of it moved a jacket one centimeter. **Part IV is where the promise has to be kept by people, buildings and vans**: a picker walking to a shelf, a carrier's depot at midnight, a store assistant looking for a size, two people at a returns bench. Physical work takes time, runs in batches, has capacity, and cannot be undone by deleting a row. The systems that run it are different from the ones before, and the joins to them are where most of a merchant's broken promises live.

Part IV is the second half of Eva's jacket. Her order of Tuesday 13 April 2027 is picked and packed at Duiven on Wednesday morning, collected by PostNL that evening and delivered in Nijmegen on Thursday 15 April, a day later than the storefront promised. On Monday 19 April she asks for size L; the return is received and graded on Thursday 22 April and the exchange ships the same day. In between she receives a string of messages

from four systems. The branch she did not take, collecting or exchanging the jacket at the Nijmegen store, is the part's third chapter.

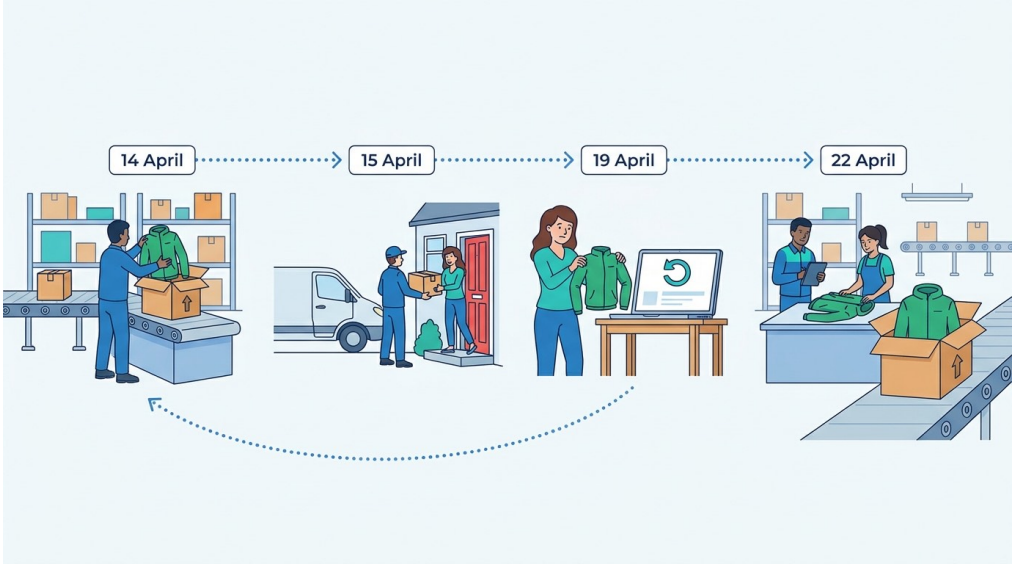


Figure 1: *Eva's jacket in Part IV: packed at Duiven on 14 April, delivered in Nijmegen on 15 April, size L requested on 19 April, and the exchange shipped on 22 April.*

Kestrel & Pine, the fictional outdoor brand this book follows, ships 336,000 parcels and 9,400 wholesale shipments a year from one 8,500-square-meter warehouse, with four carriers, six stores on a separate point-of-sale (POS) system, the till software, and 72,000 return parcels, one item in four. In 2026 the storefront promised "ordered before 22:00, delivered tomorrow" while the warehouse released its last pick wave at 21:00; store stock reached the back office once a night; a returns portal and a support console could both refund; and no system could say "your order ships in two parcels". **The claim of this part is that the physical world has its own systems of record, that every promise about it must be computed from their facts, and that the customer must be shown one status built from their events.** Waypoint's decisions W1 (the warehouse management system, or WMS, and the POS own stock), W2 (one order record), W6 (shipping notices to wholesale customers sent from what was actually packed) and W8 (promise and status by API) are that claim written down.

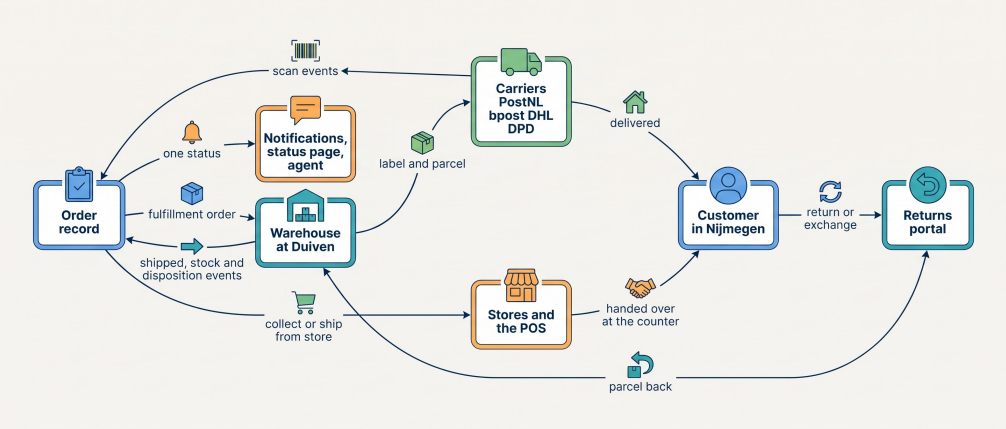


Figure 2: *The systems of Part IV: the order record instructs the warehouse and the stores, the warehouse and the carriers move the parcel there and back, and every physical fact returns as an event to the one record that the notifications, the status page and the agent read.*

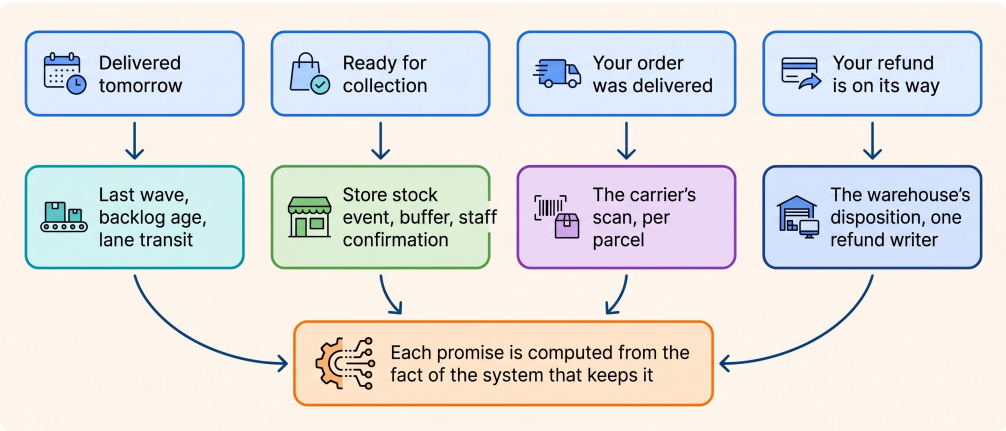


Figure 3: *The four promises of Part IV and the inputs each needs; the Queue That Waited, the Store Stock Mirage, the Three-Status Order and the Double Refund are the four made without them.*

Chapters in this part

Pick, Pack, Ship: Warehouse Management and Fulfillment decides **what the warehouse must be told, what it must own and what it must report back**: a fulfillment order with an accept step as the contract between the order record and the warehouse, the WMS as the only writer of on-hand stock, and the cut-off, capacity and wave schedule as data the delivery promise reads. The case is Duiven's day and the Queue That Waited on Black

Friday 2026, when the order feed backed up for four hours and roughly 8,000 orders missed next-day delivery while the page promised it.

Getting It There: Carriers, Labels, Tracking and the Delivery Promise decides **who talks to the carriers and what a delivery promise is computed from**: a normalizing layer between the merchant and four carriers that retire API versions on their own calendars, tracking scans as events from which the order's status is derived, and a promise built from cut-off, backlog, stock location and lane performance. The case is Eva's PostNL parcel and the sentence in the theme that promised Wednesday when the warehouse could only deliver Thursday.

Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment decides **what a store adds to the architecture and when to change the till**: a store is a second system of record that creates orders, holds stock, knows customers and takes money, offline and under fiscal rules of its own; store stock may be promised only with a buffer, a confirmation or a count behind it; "unified commerce" is a goal state, not a product. The case is the Utrecht click-and-collect pilot, the Store Stock Mirage of 37 items in December 2026, and the point-of-sale decision deferred to 2028.

The Way Back: Returns, Exchanges, Refunds and Reverse Logistics decides **which event releases the money, the stock and the credit note, and which single system acts on it**: a return is two records, the request that owns the money and the reverse fulfillment that owns the work; the warehouse's disposition joins them; EU law sets three 14-day clocks. The case is Eva's exchange of 19 to 22 April 2027 and the Double Refund of January 2027, 212 returns and €19,400 refunded twice because nothing in either request named the return.

Where Is My Order? Notifications, Service and the Customer Record decides **how the customer learns what is happening and what the agent must see**: notifications, the status page and the service view as three read models over one order record, separate state machines for money, goods and returns, and "where is my order" measured per shipped order as a reading of the platform's joins. The case is the Three-Status Order #1052877, eleven tickets in six days answered from five tabs, and the one message that would have ended it on day one: "your order ships in two parcels".

Begin with Pick, Pack, Ship: Warehouse Management and Fulfillment.

Sources and Evidence

Every public claim in this part is sourced and dated in the chapter that makes it, under that chapter's Sources and Evidence. Kestrel & Pine is fictional; recommendations and labeled assumptions are the author's judgement.

15. Pick, Pack, Ship: Warehouse Management and Fulfillment

IN THIS CHAPTER, YOU WILL: Follow an order through receiving, picking, packing and shipping, and learn what the warehouse system must be told, what it must own, and what it must report back.

KEY POINTS:

- **The fulfillment order is the contract between the order world and the warehouse**. It says "ship these lines of this order from this location by this service", carries nothing else, and has its own lifecycle: requested, accepted or rejected, in progress, shipped or closed. Cancellations after acceptance are requests, not commands.
- **The warehouse system owns the count of what is physically there, and nothing upstream should overwrite it**. Receiving, put-away, picking, cycle counts and returns change on-hand stock inside the WMS; the platform and the availability service derive what may still be sold from the WMS's events.

- **Capacity, cut-off times and the wave schedule are architecture inputs, not warehouse trivia.** A promise made at checkout is kept or broken by whether the order reaches the warehouse before the last wave of the day leaves, which is why the queue between the platform and the WMS deserves the same attention as checkout.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The First Snowfall** — the order feed into the warehouse is the integration most likely to meet real load for the first time on the busiest evening of the year.
- **Schrödinger's Order** — when the warehouse's shipment confirmation does not reach the order record, the platform, the WMS and the ERP each tell a different story about the same parcel.
- **The Phantom Stock** — a warehouse that is not the single writer of on-hand stock, or that writes it hours late, lets every channel sell units that are no longer on the shelf.

Everything before this chapter happens in software. A product is shown, a price is promised, a delivery date is displayed, money is taken, an order record is written. None of it moves a jacket one centimeter. **The warehouse is where the promise becomes physical**, and physical work has properties software does not: it takes time, it needs people and space, it runs in batches, and it cannot be rolled back by deleting a row. A picker who has walked two hundred meters to a shelf that is empty has lost those minutes for good.

This is also where upstream mistakes are discovered. A stock count that drifted from reality shows up as a pick that cannot be completed; an order edit that never reached the warehouse shows up as the wrong size in the box; a promise made without looking at the warehouse's day shows up as a parcel that leaves tomorrow. **The warehouse does not create most of the problems it reveals**, which is why this chapter is about the interface between the warehouse and everything else as much as about the warehouse itself: what a WMS does, the **fulfillment order** that crosses into it, what it must own and what it must send back.

Where we are in the order's journey. Eva's jacket (the recurring order from Meet Kestrel & Pine) was placed on Tuesday 13 April 2027 at 21:14 and paid by iDEAL, the Dutch bank-transfer payment method. This chapter is Wednesday 14 April at Duiven: the order reaches the warehouse, is picked, packed and confirmed, and a PostNL label is printed. Getting It There: Carriers, Labels, Tracking and the Delivery Promise takes over the moment the parcel leaves the dock.

Setting the Scene: Duiven's Day

Kestrel & Pine, the fictional outdoor brand this book follows, runs its own warehouse in **Duiven**, near Arnhem: 8,500 square meters, opened in 2023, run on a standalone cloud **WMS** (warehouse management system, the software that runs the physical work: where every unit sits, who picks what, in which order, into which box). Before 2023 a **3PL** in Tiel (third-party logistics provider: a company that stores and ships goods on a merchant's behalf, on its own warehouse software) fulfilled all online orders. Daniel Okafor, Head of Operations, runs Duiven with 88 warehouse staff.

In April 2027 Noor Visser, the new Head of Engineering, spends a day there, because three of the ten incidents she inherited (Meet Kestrel & Pine) have the warehouse at one end. The numbers below are Kestrel & Pine illustrations; the rhythm of the day is an assumption added for this chapter.

Duiven, a normal working day in April 2027	**Figure**
Parcels shipped (D2C and marketplace orders)	about **1,300**
Wholesale shipments (cartons and pallets)	about **35**
Units picked	about **7,500**
Pick waves released	**06:30**, **10:00**, **13:00**, **16:00**, **19:00** and a final wave at **21:00**
Carrier collections	PostNL, bpost, DHL and DPD between **18:00 and 22:30**
Inbound deliveries from suppliers	4 to 8 trucks, received against purchase orders
Returns received and inspected	about **280** parcels

Daniel explains the day in the order the building sees it: trucks received in the morning against purchase orders from the ERP (enterprise resource planning system: finance, purchasing and the item master), goods put away, pickers walking routes the WMS planned so that one walk serves many orders, packers choosing the box and printing the label, two people counting a few dozen locations every afternoon. Then he shows Noor the chart from Friday 27 November 2026, the evening of the Black Friday Oversell. The platform held. The warehouse was staffed for the peak. But the **integration flow that creates orders in the WMS backed up for four hours**: an order placed at 20:00 did not exist in the WMS until after midnight, and the 21:00 wave left without most of the evening's orders. The storefront kept promising "ordered before 22:00, delivered tomorrow" the whole time, because that promise is a line of text in the theme, not something computed from the warehouse's state. Daniel calls it **the Queue That**

Waited. Noor's question is the chapter's question: **what does the warehouse need from the rest of the platform, and what does it owe back?**

What a Warehouse Management System Does

A WMS runs a warehouse the way a checkout runs a purchase: as a sequence of steps, each of which changes the state of something. The vocabulary is small and worth learning once.

Step	What it is, in plain words	What it changes
Receiving	Checking an inbound delivery against the purchase order (or the supplier's advance shipping notice) and booking the units in	On-hand stock goes up
Put-away	Moving received goods to a storage **location** (a shelf, bin or pallet position with its own barcode)	Units get an address
Slotting	Deciding where each SKU (stock-keeping unit: one sellable variant, the thing you count on a shelf) should live: fast movers near the packing stations, bulky items on the floor	Walking distance per pick
Wave or batch release	Grouping orders for one picking run, typically by carrier cut-off, channel or zone, so one walk serves many orders	Which orders ship today
Picking	Walking or driving the planned route, scanning each location and unit	Units leave their locations

Step	What it is, in plain words	What it changes
Packing and cartonization	Choosing the box (cartonization is the software's choice of box size from the items' dimensions), packing, weighing	A shipment exists
Shipping confirmation	Printing the carrier label, closing the pick list and reporting the shipment with its tracking number	The order is fulfilled; on-hand goes down
Cycle counting	Counting a small part of the stock on a rolling schedule instead of one annual full count, correcting the record continuously	Record accuracy

Vendors describe these steps in their own words. Odoo's inventory module (n.d., accessed November 2026) lists wave picking "per aisle, or area, and regroup at packing zone", cluster picking (several orders in one trip) and batch picking (several orders in one picking, consolidated at packing), plus cycle counts and barcode scanning. Picqer, a Dutch WMS for the small and mid-sized segment (n.d., accessed November 2026), processes pick lists "in batches", routes pickers "through the warehouse in sequence" and sends labels and tracking codes back to the online store "in one step"; its customers "range from about 25 to 5,000+ orders per day". Manhattan's warehouse product (n.d., accessed November 2026) describes "Order Streaming" that handles "wholesale, retail, and direct orders at the same time". Infios, formerly Körber Supply Chain Software (n.d., accessed November 2026), frames the market by Gartner's warehouse complexity levels "from 2 (rudimentary locator) to 5 (highly automated warehouse)"; on that scale Duiven is a level-3 building, with locations, waves and scanning but no automation.

Three facts about warehouse work shape every integration decision in this chapter:

- **It runs in batches**. Orders are collected into waves, so "did the order reach the warehouse?" has a deadline: the release time of the last wave that can still make the carrier's collection.
- **It has capacity**. A building with 88 people picks a certain number of units per hour. An order that arrives after the day's capacity is spoken for ships tomorrow, whatever the storefront promised.
- **Its facts are physical**. The WMS's on-hand number is checked against shelves

every day by pickers and counters. No other system has that property, which is why Is It in Stock? Inventory, Availability and the Promise You Can Keep and Waypoint decision W1 make the WMS the **system of record** for physical stock at Duiven (the one system whose value counts as the truth when copies disagree).

How the Warehouse Connects to Everything Else

Seen from outside, a warehouse has four interfaces: instructions in, confirmations out, stock events out, and inbound goods in. The instructions come from whichever system holds the orders: today Kestrel & Pine's platform, and from Q4 2027 the **OMS** (order management system: the one place that holds every channel's orders and steers each through fulfillment; The Order Record: Lifecycle, State and the Order Management System). The figure shows the interfaces around the physical steps.

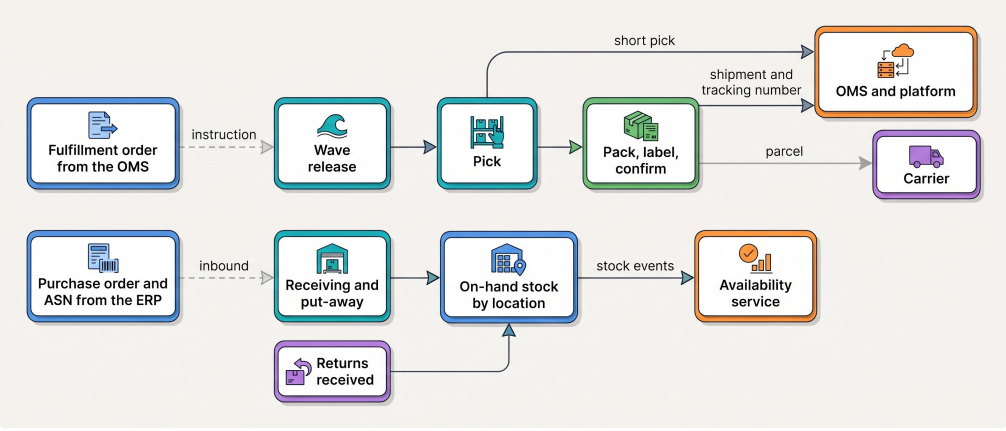


Figure 1: *The warehouse has four interfaces: instructions in as fulfillment orders, confirmations and short picks out to the order record, stock events out to the availability service, and inbound goods in against purchase orders and returns.*

Each interface has different requirements for freshness, consistency, speed and availability. The table states them for a merchant of Kestrel & Pine's shape; the latencies are assumptions, not vendor facts.

Flow	How fresh	How consistent	How fast	How available
Fulfillment order into the WMS	Before the next wave release; at peak, within minutes *(assumption)*	Exactly once; a repeat must not create a second pick list	Must keep pace with the order rate at peak	If the WMS is down, orders queue; nothing is lost
Shipment confirmation out	The moment the label prints	One per shipment, carrying the tracking number	Within a minute, so the "shipped" message and the invoice follow *(assumption)*	Queued if the OMS is down; replayed in order
Stock events out	Each change, not an hourly total	The WMS's count is the truth; consumers derive, never write back	Seconds *(assumption)*	A reconciliation job catches a lost event
Cancellation request in	Before picking starts	A request with an answer, not a command	Minutes	If unanswered, the order is still in progress
Inbound receipt	Same day as the truck	One receipt per purchase-order line	Hours	Can wait

Two rows carry the chapter's argument: the fulfillment order and the stock events. The next two sections take them in turn.

The fulfillment order: the contract between the order world and the warehouse

A **fulfillment order** is the instruction "ship these lines of this order from this location, by this service level, with these notes". It is not the customer's order. The customer's order has prices, discounts, a payment, a customer history and a return policy; the warehouse needs none of that. The fulfillment order has exactly what the warehouse needs and has **its own lifecycle**, separate from the order's: requested, accepted or rejected, in progress,

shipped (possibly in several parcels) or closed incomplete.

Shopify's developer documentation (n.d., accessed November 2026) is the clearest public description of this seam, because Shopify had to formalize it for thousands of third-party warehouses and 3PLs. A fulfillment order is "an item or a group of items in an order that are to be fulfilled from the same location"; each warehouse or 3PL "should have its own fulfillment service" mapped to a location; Shopify sends the service a `FULFILLMENT_REQUEST` or a `CANCELLATION_REQUEST`; the service accepts or rejects; and "fulfillments can't be created for a fulfillment order before the fulfillment service accepts the fulfillment request". The request status moves through `SUBMITTED`, `ACCEPTED` or `REJECTED`, and later `CANCELLATION_REQUESTED`, `CANCELLATION_ACCEPTED` or `CANCELLATION_REJECTED`. The rule that matters most is this one: once accepted, "the merchant can no longer cancel it directly and must submit a cancellation request". Kibo's order management documentation (n.d., accessed November 2026) describes the same shape from the OMS side: shipments move through Accept Shipment, Validate Stock (where substitutions happen), Ready and Fulfilled, and "can be transferred or reassigned between different fulfillment locations".

The sequence shows the handshake as Waypoint designs it for late 2027, when the OMS sends fulfillment orders, using Eva's order as the example. It is carried by the iPaaS (integration platform as a service: the hosted tool that runs the flows between systems) with an idempotency key (a reference that lets the receiver recognize a repeated message), and with a cancellation request arriving too late.

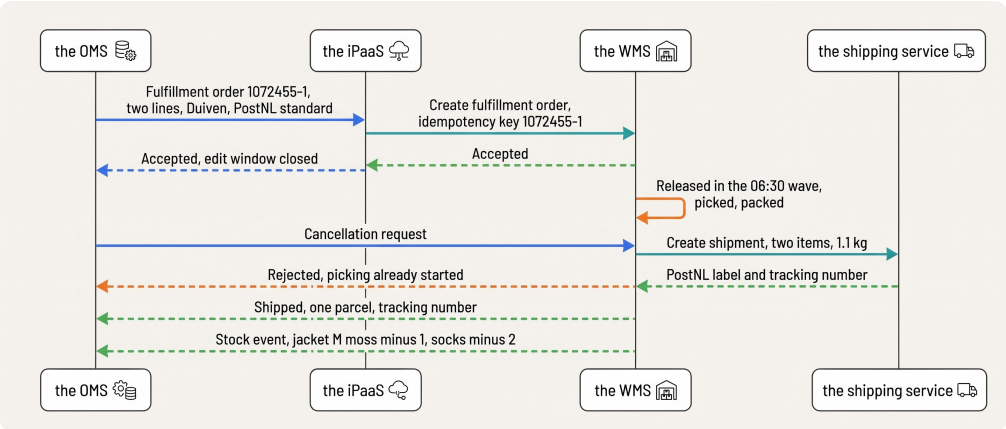


Figure 2: *The fulfillment-order handshake: the warehouse accepts before anyone may count on it, a cancellation after acceptance is a request the warehouse may refuse, and the shipment confirmation and the stock event travel back as separate messages the moment the label prints.*

Why insist on this shape rather than simply "send the order to the WMS"? Four reasons, each tied to a cost.

- **It carries the right data and nothing else.** A WMS that receives prices and payment details holds data it must protect and that changes for reasons it does not care about.
- **It supports partial and split shipments.** One order can have two fulfillment orders (Duiven and the marketplace's fulfillment center), and one fulfillment order can ship in two parcels. The Order Record: Lifecycle, State and the Order Management System owns splits; the warehouse needs each instruction to be complete on its own.
- **The accept step is the lock.** Until the warehouse accepts, the order can be edited and canceled freely. After acceptance, every change is a request with an answer, and that answer is what a service agent needs to tell the customer the truth.
- **It makes retries safe.** Networks and queues guarantee that the same instruction will sometimes arrive twice. If it carries the fulfillment order's identity and the WMS treats a repeat as the same instruction (**idempotent** handling: receiving a message twice has the same effect as once), the retry is harmless. If the WMS creates a pick list per message, the retry ships the jacket twice.

The fulfillment order is also a stable seam when the systems on either side change. Kestrel & Pine's platform sends orders to the WMS today; under Waypoint decision W2 the OMS will send fulfillment orders by Q4 2027, and the WMS need not know which system is on the other side.

Stock: what the warehouse owns and what it reports

The second interface is stock. Is It in Stock? Inventory, Availability and the Promise You Can Keep owns the concepts; this chapter needs three of them. **On hand** is what is physically in the building. **Committed** (or allocated) is the part of on hand that placed orders have already claimed. **Available** (available-to-promise, ATP) is what a channel may still sell: on hand minus committed minus whatever the merchant deliberately holds back. The warehouse's job is the first number only.

The rule I give every merchant: **one writer per stock state.** The WMS writes on hand, because it is the only system that counts shelves. The order system writes committed, because it is the only system that knows which orders exist. The availability service derives available. Nobody else writes any of the three. Kestrel & Pine's "App That Wrote Stock" of March 2026, in which a pre-order app and the WMS feed both adjusted the platform's inventory, and the first reconciliation of May 2027, which found 2.3% of active variants with different quantities in the WMS and the platform, are what happens when the rule is broken.

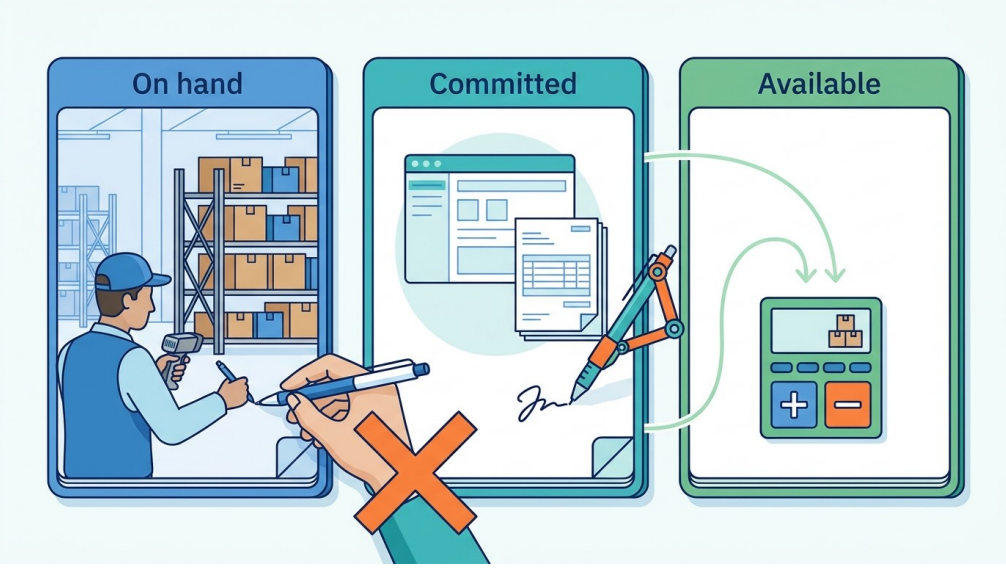


Figure 3: *One writer per stock state: the WMS writes on hand, the order system writes committed, the availability service derives available, and no other hand may write any of the three.*

On hand changes inside the warehouse constantly: a receipt, a pick, a cycle-count correction, a damaged unit moved to quarantine, a return restocked. How the rest of the landscape learns is the architectural question, and two patterns exist.

- **Periodic full sync**. The WMS exports every SKU's on hand on a schedule and consumers overwrite their copy. Shopify's documentation for fulfillment services (n.d., accessed November 2026) describes exactly this: Shopify polls the service's stock endpoint for on hand per SKU hourly. Kestrel & Pine's chain (WMS to ERP every 15 minutes, ERP to platform hourly, platform to the marketplace connector hourly) is three of these in series, which is how a sale took up to two hours to reduce the stock shown elsewhere on Black Friday evening.
- **Stock events**. The WMS emits a message for each change ("SKU X at location Y: minus 1, reason pick, timestamp"). Picqer's API (n.d., accessed November 2026) emits a `products.free_stock_changed` webhook "to support real-time stock updates in a store"; a **webhook** is a call the vendor's system makes to your URL when something happens. Waypoint decision W1 retires the sync chain in favor of this pattern by Q4 2027.

Events are better once order volume makes lag visible, and they are harder: a lost event leaves a stale copy until a **reconciliation** (a job that compares the two systems and lists differences) catches it. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation owns that discipline; the warehouse's part is to emit every change with a reason and to publish a daily full snapshot as the baseline.

Capacity, cut-off times and the wave schedule

The third thing the warehouse owns is its **day**. Every promise the platform makes about timing rests on three warehouse facts that rarely appear in any API:

- **The cut-off** is the latest moment an order can reach the warehouse and still ship today: the release time of the last wave that makes the last carrier collection. At Duiven the last wave is released at 21:00 and PostNL collects at 22:30 (Kestrel & Pine illustrations). The storefront's "ordered before 22:00" line dates from the Tiel 3PL, whose collection was later; nobody updated it in 2023. The promise and the building disagree by an hour on every weekday.
- **Capacity** is units or parcels per hour at the day's staffing. A wave with more work than the time until collection allows leaves orders behind, whatever their order time.
- **The wave policy** decides which orders go first: by carrier collection, by service level, by channel (marketplace orders carry late-shipment penalties).

These three belong in the data the delivery promise is computed from. Until they are, the promise is a sentence; Getting It There: Carriers, Labels, Tracking and the Delivery Promise is about making it a computation. On Black Friday evening 2026 the warehouse's capacity was never the limit; the order feed was.

Wholesale fulfillment: pallets, SSCC labels and the despatch advice

A parcel for Eva and a pallet for Bergzicht Outdoor leave the same building through different processes. Wholesale shipments are picked to cartons and pallets, and a large retail customer expects two things a consumer never sees: a label on each logistics unit and an electronic message announcing what is on it.

The **SSCC** (serial shipping container code) is, in the words of the GS1 General Specifications, Release 26.0 (January 2026), "the GS1 identifier for an individual logistics unit" and "the only GS1 key that SHALL be used as the identifier of a logistic unit": the 18-digit number on the pallet label. The **despatch advice** (DESADV in EDIFACT, the UN message standard most European EDI uses; also called an ASN, advance shipping notice) tells the receiver what is coming, pallet by pallet, so receipt can be scanned instead of counted. Documents Between Companies: EDI, Punchout and E-Invoicing owns the messages; this chapter owns where they are generated.

The retailers' rules are public and strict. bol's supplier page (n.d., accessed November 2026) states that "as of December 31, 2021, all logistic shipping units delivered to bol must be provided with GS1 label(s)" and that "for suppliers with an EDI connection, the SSCC must be included in the DESADV message". Jumbo's 2025 guidelines for its

mechanized distribution center say that "a delivery without an accompanying DESADV message cannot be processed and is therefore refused" and that "the SSCC label forms the link with the DESADV despatch message"; its purchasing conditions of 9 April 2026 add €70 per message that does not comply with GS1 standards.

The architectural consequence is one sentence: **the DESADV must be generated from what the warehouse actually packed, never from the order**. The pallet's SSCC and its contents are known only at pack time, in the WMS. Kestrel & Pine's ASN Chargeback of Q3 2026 (€14,200 deducted by Bergzicht for despatch advices whose SSCC labels did not match what arrived) happened because the DESADV was built from the ERP's view of the order while the WMS packed something slightly different. Waypoint decision W6 moves SSCC generation to the WMS at pack time and sends the DESADV from the WMS confirmation. Bergzicht's spring reorder of April 2027 (240 units across 18 SKUs on three pallets, DESADV at pack confirmation, delivered to Zwolle on 9 April) is the first cycle under the new rule; Journey 3: A Chain Reorders by EDI, From Order to Invoice follows it end to end.

The Choices: Who Runs the Warehouse Software

A merchant has four realistic ways to get warehouse execution, and most use two at once.

The ERP's warehouse module. Mid-market ERPs include receiving, locations, picking and shipping; Odoo's module is the public example. It is one system, one integration and one vendor fewer. It fits one building, modest volume and simple routes; it strains when waves must be planned around carrier cut-offs, when labor needs managing or when the warehouse serves more than one company.

A dedicated WMS. Two segments exist. Small and mid-sized WMS products (Picqer: "about 25 to 5,000+ orders per day") are cloud services integrated by order push and stock webhook, with carrier labels included. Enterprise WMS products (Manhattan, Blue Yonder, Infios) add wave optimization, labor management, slotting, automation interfaces and multi-site operation; Blue Yonder notes (n.d., accessed November 2026) its placement as a Leader in Gartner's 2026 Magic Quadrant for the eighteenth consecutive time, and Infios cites "1,600+ WMS customers". The step between segments is a project measured in quarters.

A 3PL. The provider runs the building on its own WMS, often in a multi-client mode. The merchant integrates with it exactly as with its own WMS: fulfillment orders in, confirmations and stock events out. The difference is who owns the people, the lease and the process. A 3PL is right when volume, geography or season does not justify a building, and wrong when the merchant's process (gift wrapping, kitting, strict brand packaging) is the product.

A marketplace's fulfillment program. Amazon's FBA ("we'll pick, pack, and ship

orders, as well as handle customer service and returns", n.d., accessed November 2026) and Zalando's ZFS, launched in autumn 2017 and used by "more than 100 partner brands" by June 2019, are 3PLs run by the marketplace. The seller sends stock in advance; the marketplace stores, ships and handles returns; the seller pays per unit, per order and per day of storage. Zalando's ZFS invoice structure (updated 16 February 2026) charges outbound "per order, regardless of package count", storage "per item per day" and a peak surcharge of €0.15 per outbound order from 15 October to 31 January; the amounts are in the partner contract. In October 2023 Zalando productized the network as **ZEOS** for brands "whether or not they sell on Zalando"; in November 2024 it announced it would fulfill most of NEXT's continental-European orders "through a single shared stock pool and an order management system", and its letter to shareholders of 12 March 2026 reports ZEOS in "24 markets" serving "18 sales channels". That shared pool is the availability problem of Is It in Stock? Inventory, Availability and the Promise You Can Keep run by the provider instead of the merchant. For the merchant, marketplace-held stock is a **location it can see but not count**: a separate location in the availability model with its own buffer, fed by a purchase-order-like inbound flow. Kestrel & Pine sends about 40% of its German marketplace units to such a program.

Choice	Fits when	Limits when	What the integration looks like
ERP warehouse module	One building, simple routes, modest volume	Waves around cut-offs, labor planning, several clients or sites	Orders already in the ERP; stock is the ERP's own table
Dedicated WMS, small or mid-sized	Own building, up to a few thousand orders a day	Automation, labor management, multiple sites	Order push by API; stock and shipment webhooks back
Dedicated WMS, enterprise	Several sites, automation, wholesale and parcels in one building	Needs its own specialists	Fulfillment-order contract, event streams, EDI from pack confirmation
3PL	No building of your own; seasonal or regional volume; a new country	Brand-specific packing, returns grading, cost at high volume	Same contract as a WMS; the 3PL's portal is not the record

Choice	Fits when	Limits when	What the integration looks like
Marketplace fulfillment program	Marketplace share is large and its delivery promise sells	Stock you cannot count; fees per order and per day; returns graded by the marketplace	An external location with inbound shipments and stock reports

My recommendation follows the requirements, not the product. One location and one channel: use what the platform or the ERP gives you. One building, several channels and a carrier schedule to plan around: a dedicated WMS, behind the contract above. Several buildings, or a wholesale flow with SSCC and DESADV obligations: an enterprise WMS or a 3PL that can meet them, behind the same contract. At no stage let the warehouse system become a second order system that holds prices, customer data and status for support; that is how Kestrel & Pine arrived at three definitions of an order.

Across Scale Stages: When It Fits and When It Limits

Stage	Typical setup	What becomes necessary	Warning signs of outgrowing it
1 — Starting	A back room or a 3PL; the platform prints labels; stock is the platform's own fields	A 3PL with an order feed the day the founders cannot pack everything; tracking numbers back on the order	Orders re-typed into the 3PL's portal; stock counted in a spreadsheet; the first parcel shipped twice after a retry

Stage	Typical setup	What becomes necessary	Warning signs of outgrowing it
2 — Growing	A 3PL's WMS or a small WMS; orders pushed by a connector; stock back by webhook or nightly file	The accept-or-reject handshake; stock events instead of a nightly file; a cut-off the storefront and the warehouse agree on	The 3PL's portal is the only place to see shipment status; next-day promises missed on busy days; the first retail chargeback
3 — Omnichannel	Own building or a 3PL per region; the WMS as system of record for on hand; an OMS sending fulfillment orders; waves by carrier cut-off and channel; wholesale and parcels in one building; marketplace fulfillment stock as an external location	SSCC and DESADV from pack confirmation; a queue between the order hub and the WMS sized for the peak hour; WMS-to-platform reconciliation; capacity and cut-off exposed to the delivery promise	The order feed backing up at peak; WMS and platform status disagreeing for days; one connector per channel into the WMS; a DESADV built from the order
4 — Group	A WMS per site, often from different vendors after acquisitions; routing among sites; automation; shared returns	One fulfillment-order contract for every site, whatever its vendor; a network-level view of capacity	Each brand's warehouse integrated its own way; nobody can say how many parcels the network can ship tomorrow

The judgement: the warehouse software can be bought, rented or outsourced at any stage, and the choice can change. What should not change from Growing onward is the

contract: fulfillment orders in with an accept step, confirmations and stock events out, the WMS as the only writer of on hand. Kestrel & Pine bought a good WMS in 2023 and integrated it as if it were a 3PL portal: orders pushed one by one, status pulled by the ERP, stock copied hourly. The building was Omnichannel; the integration was Starting.

In Practice

Public evidence

Three details from the public record sharpen the argument above. Shopify's fulfillment-service documentation (n.d., accessed November 2026) warns that "SKU strings are case-sensitive", a detail that has produced many a phantom unit at the seam between two catalogs. Picqer's API (n.d., accessed November 2026) publishes a rate limit of about 500 requests per minute, the kind of number an order feed must be designed against before the peak hour arrives. And DeHoratius and Raman's 2008 study in *Management Science* found, "examining nearly 370,000 inventory records from 37 stores of one retailer", that 65% were inaccurate; the study is old, from one US retailer's stores and before RFID, so read it as the field's reminder that a record drifts from the shelf unless someone counts.

Jumbo's conditions also carry delivery-performance penalties (€250 per order below 90% delivered on time and in full on an order line). Chargebacks thus come in two families: delivery performance, a logistics problem, and data and label compliance, an architecture problem.

Kestrel & Pine: Eva's jacket at Duiven, and the Queue That Waited

Eva's order #1072455 is placed on Tuesday 13 April 2027 at 21:14. In April 2027 the platform still sends it to the WMS through the iPaaS; the OMS of Waypoint decision W2 arrives in Q4. The flow creates the order in the WMS at 21:16 (illustrative). The last wave was released at 21:00. The order waits for the 06:30 wave on Wednesday 14 April, is picked with about 180 other orders on one route (illustrative), packed into one box of 1.1 kg with a PostNL label and confirmed at 08:40. The confirmation carries the tracking number; the stock event says jacket M moss minus one and socks minus two. PostNL collects it that evening and delivers on Thursday 15 April. The storefront had promised Wednesday. The warehouse did nothing wrong; the promise was not computed from the warehouse. Getting It There: Carriers, Labels, Tracking and the Delivery Promise starts from that gap.

Black Friday evening, Friday 27 November 2026, is the same gap at scale. In the peak hour, 20:00 to 21:00, the storefront took 2,300 orders, 38 a minute. The iPaaS flow that creates orders in the WMS made several calls per order and was throttled by the WMS

vendor's rate limit (illustrative mechanism); it fell behind, and by 21:00 the backlog was four hours long. Orders placed at 20:00 reached the WMS after midnight; the 21:00 wave contained the orders of the early afternoon. Roughly 8,000 orders placed between 17:00 and midnight (assumption) missed next-day delivery while the storefront promised it. The warehouse's staffing that night was never tested.

What would have prevented it is not a faster warehouse. It is a queue between the platform and the WMS that is **allowed to grow** without failing checkout, a flow that creates orders in **batches** within the WMS's rate limit, a monitor on the age of the oldest unprocessed order, and a cut-off that **moves** when that age grows, so the promise on the page tells the truth. All four are on Waypoint's list under decisions W2 and W5; Black Friday Every Year: Peak Load, Resilience and Recovery and Journey 4: Black Friday, Hour by Hour work the whole evening.

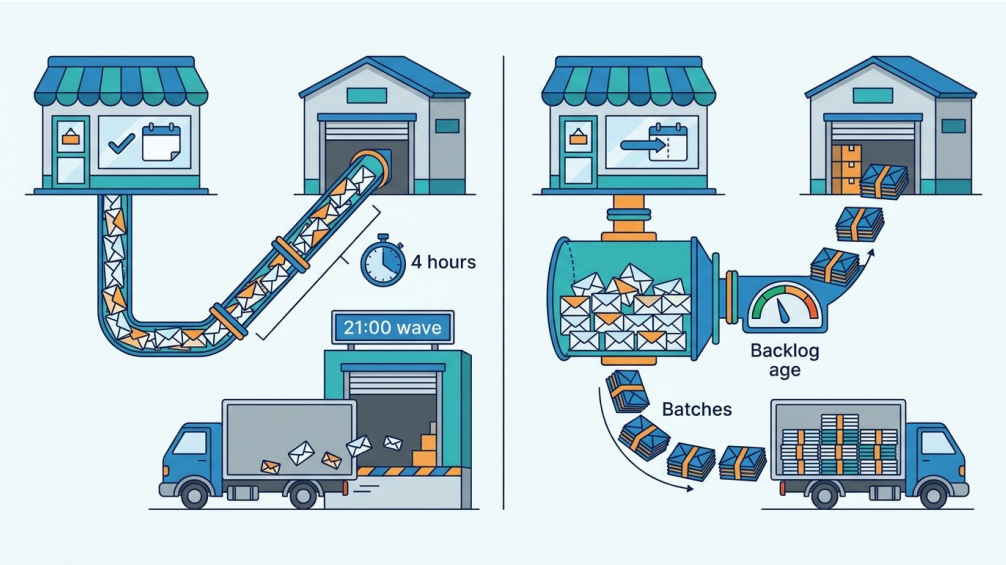


Figure 4: *The Queue That Waited and its cure: on Black Friday 2026 the order feed fell four hours behind while the page still promised next day; a queue allowed to grow, batch creation, a backlog-age monitor and a moving cut-off keep the promise honest.*

When It Goes Wrong

The warehouse interface fails in a small number of recognizable ways. Each has a detection and a recovery.

Duplicate fulfillment orders. A retry after a timeout delivers the same instruction twice and the WMS creates two pick lists; the customer receives two jackets. Detection: a fulfillment-order identity on every message and a daily check for two WMS orders

with the same identity. Recovery: idempotent creation in the WMS (same key, same order) and an alert rather than a second pick list.

The cancellation that arrived after the pick. Support cancels the order in the platform; the parcel ships anyway because the WMS had already accepted it. Detection: the cancellation must be a request with an answer, and the platform must show the answer. Recovery: the order stays in progress, support tells the customer, and the item comes back through The Way Back: Returns, Exchanges, Refunds and Reverse Logistics.

The lost shipment confirmation. The parcel left; the confirmation was lost; the platform says "unfulfilled" and the ERP never invoices. This is Kestrel & Pine's Three-Status Order (#1052877: *Delivered* in the platform, *In picking* in the WMS, *Not invoiced* in the ERP for six days) seen from the warehouse. Detection: a daily reconciliation of shipped-in-WMS against fulfilled-in-platform. Recovery: replay the confirmation from the WMS; never re-key it by hand.

Stock events out of order or missing. A pick event arrives before its receipt event, or not at all, and the available figure is wrong until the next snapshot. Detection: sequence numbers on events, and a weekly reconciliation such as Kestrel & Pine's May 2027 run (2.3% of active variants differing). Recovery: apply the daily snapshot as the baseline and replay events since.

The short pick. The record said one; the shelf has none. Detection: the WMS records a short pick against the fulfillment order. Recovery: the WMS tries another location; if none, it reports a partial shipment or closes the fulfillment order as incomplete, and the order system decides (ship the rest, wait, or cancel the line and refund). A short-pick rate per SKU tells you where to cycle count.

SKU mismatch. The platform's variant identifier and the WMS's differ by a suffix, a case or a leading zero; the fulfillment order is rejected or maps to the wrong product. Detection: rejects per SKU and a daily comparison of the two catalogs. Recovery: one identifier family owned by the product system (What Are We Selling? Products, Variants and the Catalog), published to both.

The WMS outage. The cloud WMS is unavailable for an hour on a peak evening. Detection: the age of the oldest queued order. Recovery: queue, do not fail checkout; when the WMS returns, release in batches within its rate limit; move the cut-off while the backlog is older than the time to the last wave.

What to Take Away

Established.

- A warehouse runs in batches with finite capacity and a daily cut-off set by carrier collections; wave, cluster and batch picking are the standard methods (Odoo, Picqer, Manhattan, n.d., accessed November 2026).

- The fulfillment order, with request, accept or reject, in-progress, shipped and cancellation-request states, is the documented contract between order management and fulfillment in Shopify's and Kibo's models: nothing ships before acceptance, and cancellations after acceptance are requests.
- Stock records drift from the shelf without counting (the classic 2008 study found 65% of records inaccurate in one retailer's stores); cycle counting is the standard remedy.
- Large retail customers require SSCC-labeled logistics units announced by a matching DESADV (GS1 General Specifications 26.0; bol since 31 December 2021; Jumbo 2025–2026), with refusal or fees when they do not match.

Depends on context.

- An ERP's warehouse module is enough for one building with simple routes and modest volume; a dedicated WMS becomes necessary when waves must be planned around cut-offs, labor must be managed, or the building serves several companies or channels.
- A 3PL or a marketplace fulfillment program is right when volume, geography or season does not justify a building, and wrong when packing or returns grading is part of the brand. Marketplace-held stock must be modeled as a location you cannot count.
- Periodic stock sync is acceptable while the lag is shorter than the time in which a popular unit sells out; stock events become necessary the moment that is no longer true, which for a brand with a hero product is the first peak.
- Generating the DESADV from the WMS pack confirmation is mandatory for a merchant with EDI retail customers and irrelevant for one without.

The author's synthesis.

- Design the warehouse integration as a contract that survives every system on either side: fulfillment orders in with an accept step, confirmations and stock events out, the WMS the only writer of on hand. Buy, rent or outsource the WMS freely; never let the contract depend on the vendor.
- Cut-off, capacity and wave schedule are data the delivery promise must read, not operations folklore. A merchant whose storefront states a cut-off its warehouse does not keep has already broken the promise; it just has not measured how often.

Questions to Ask

1. What exactly is sent to the warehouse per order, and does it carry anything the warehouse does not need?

2. Is there an accept-or-reject step before the warehouse counts on an order, and what happens to a cancellation after acceptance?
3. If the same instruction is delivered twice, does the warehouse create one pick list or two?
4. Which system writes on-hand stock, and can anything else (an app, a script, the ERP) also write it?
5. How does the rest of the landscape learn of a stock change, and how long is the lag at peak?
6. When was the last reconciliation between the warehouse's stock and the platform's, and what share of SKUs differed?
7. What is the release time of the last wave, and does the storefront's cut-off agree with it?
8. How many orders per minute can the order feed into the warehouse sustain, and what happens to checkout when it cannot keep up?
9. For wholesale customers with EDI: which system generates the SSCC label and the DESADV, and from what?
10. If the warehouse system is unavailable for an hour on the busiest evening, what do customers see?

The parcel is packed, labeled and confirmed. What happens next is outside the building and outside the merchant's control: a carrier's network, a scan at a depot, a van on Eva's street, and the date she was shown at checkout. Getting It There: Carriers, Labels, Tracking and the Delivery Promise is about choosing the carrier, producing the label, reading the tracking events, and making a delivery promise the warehouse can keep.

Sources and Evidence

Company, product and investor sources

- Build for fulfillment services, Shopify developer documentation, n.d., accessed 2026-11 — the fulfillment request, accept, reject and cancellation-request handshake; hourly stock polling; case-sensitive SKUs.
- FulfillmentOrder object and Order management apps, Shopify, n.d., accessed 2026-11 — fulfillment-order definition, statuses and request statuses; cancellation after acceptance as a request.
- Fulfillment concept guide and Order routing, Kibo Commerce documentation, n.d., accessed 2026-11 — shipment states, reassignment between locations, routing kept apart from execution.
- Picqer product page and Picqer API, Picqer, n.d., accessed 2026-11 — batch pick lists, labels and tracking back in one step, 25 to 5,000+ orders per day, `picklists.closed` and stock-change webhooks, ~500 requests per minute.

- Odoo Inventory, Odoo, n.d., accessed 2026-11 — wave, cluster and batch picking, cycle counts, barcode scanning in an ERP's warehouse module.
- Manhattan Active Warehouse Management, Manhattan Associates, n.d., accessed 2026-11 — order streaming across wholesale, retail and direct orders; wave rebalancing.
- Warehouse Management Systems, Infios (formerly Körber Supply Chain Software), n.d., accessed 2026-11 — receiving to shipping scope, Gartner complexity levels 2 to 5, customer count.
- Warehouse Management, Blue Yonder, n.d., accessed 2026-11 — 2026 Gartner Magic Quadrant Leader claim.
- Fulfillment by Amazon, Amazon, n.d., accessed 2026-11 — what FBA does and its fee structure.
- Zalando Fulfillment Solutions reaches milestone, Zalando release via MarketScreener, 2019 — ZFS launch in autumn 2017 and 100+ partner brands by June 2019 (older source).
- ZFS invoice structure, Zalando Partner University, updated 2026-02-16 — fee categories including per-order outbound, per-item-per-day storage and the peak surcharge.
- Productizing Zalando's capabilities: introducing ZEOS, Zalando SE, 2023 — ZEOS as fulfillment for any channel from 12 warehouses.
- ZEOS becomes fulfilment partner of NEXT in continental Europe, Zalando SE, 2024 — the single shared stock pool and order management system.
- Letter to shareholders 2025, Zalando SE, 2026 — ZEOS in 24 markets and 18 sales channels.
- SSCC for EDI suppliers, bol, n.d., accessed 2026-11 — GS1 labels on all logistic units since 31 December 2021; SSCC in the DESADV.
- Guidelines for the mechanised DC, Jumbo, version 2025-10 — DESADV required at pallet level; refusal without DESADV or GS1 barcode.
- General purchasing conditions, Jumbo, 2026-04-09 — €70 per non-compliant message; refusal rights; delivery-performance penalties.

Standards, regulation and official documentation

- GS1 General Specifications, Release 26.0, GS1, 2026-01 — definitions of GTIN, GLN and SSCC; the SSCC as the only identifier of a logistics unit.

Research and practitioner sources

- Inventory Record Inaccuracy: An Empirical Analysis, DeHoratius and Raman, *Management Science* 54(4), 2008 — 65% of 370,000 records inaccurate in one retailer's stores (older source; pre-RFID).

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

16. Getting It There: Carriers, Labels, Tracking and the Delivery Promise

IN THIS CHAPTER, YOU WILL: Learn how a parcel gets a carrier, a label and a tracking history, and what data a delivery promise needs before you may show it at checkout.

KEY POINTS:

- **Carriers are many, and each speaks its own language**. Every carrier has its own API generation, label format, manifest rule and status codes, and each migrates on its own calendar; a normalizing layer, bought or built, is what keeps carrier change out of the order record.
- **A tracking event is a fact about a parcel; the order's status is derived from it**. Map every carrier's scans onto one small vocabulary, let the order status page and notifications read from that vocabulary, and measure carrier performance per lane from the same events.

- **A delivery promise is a computation, not a sentence.** It needs the stock location, the warehouse's cut-off and capacity, the carrier's transit time for the lane and a margin for the tail; a promise written into the page is kept by luck and broken without anyone noticing.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Pulled Rug** — carriers retire API versions, rename products and change manifest rules on published dates; a label integration that nobody watches breaks on the morning the old version is switched off.
- **The Message in a Bottle** — a tracking number that does not reach the order record, or a scan that arrives twice, leaves the customer without a "shipped" message or with two.
- **The First Snowfall** — a static promise is right on an ordinary Tuesday and wrong on the one evening a year when the queue to the warehouse is four hours long.

A commerce platform makes four kinds of promise: this product, at this price, delivered by then, returnable if it does not fit. The first two are kept by data the merchant controls. **The delivery promise is kept by people the merchant does not employ**: a warehouse crew on a shift, a carrier's sorting center, a van driver, sometimes a customs officer. This chapter is about the systems and data between checkout and the doorstep, and about the difference between writing a promise and computing one.

Shipping also has an unusual property for an integration: there are many parties on the other side and none of them is the merchant's customer. Kestrel & Pine, the fictional outdoor brand this book follows, ships with PostNL in the Netherlands, bpost in Belgium, DHL in Germany and DPD as a backup everywhere. Each has its own API, its own label, its own rules for closing the day and its own vocabulary for what happened to the parcel. **Carrier change is constant and arrives on the carrier's calendar, not yours.** The architecture question is where to absorb it.

Where we are in the order's journey. Eva's jacket (the recurring order from Meet Kestrel & Pine) was packed and labeled at Duiven on Wednesday 14 April 2027 and confirmed at 08:40, as Pick, Pack, Ship: Warehouse Management and Fulfillment described. This chapter is the label itself, the PostNL collection that evening, the scans on Wednesday night and Thursday morning, the delivery on Thursday 15 April, and the promise Eva saw on Tuesday evening before any of it happened.

Setting the Scene: A Sentence in the Theme

In April 2027 Milan Petrov, Ecommerce and Marketplaces Manager, asks Noor Visser, the Head of Engineering, for something that sounds small: the German and Belgian

storefronts should show a delivery date the way the Dutch one does. The Dutch storefront says "ordered before 22:00, delivered tomorrow". Daniel Okafor, Head of Operations, objects before Noor can answer. bpost and DHL collect at different times from PostNL, transit to a Belgian or German address is not one day everywhere, and even the Dutch line is wrong: the warehouse management system (WMS) at Duiven releases its last pick wave at 21:00 (a Kestrel & Pine illustration from Pick, Pack, Ship: Warehouse Management and Fulfillment). Noor asks the question an architect asks: **what is the promise computed from?** The answer is nothing. It is a line of text in the storefront theme, written in the years when the Tiel 3PL (third-party logistics provider: a company that stores and ships on a merchant's behalf) shipped the orders. The agency can change the words in an hour. Nobody can say how often the words are true.

Three facts from Kestrel & Pine's records show what the sentence costs. Eva ordered on Tuesday 13 April at 21:14, was promised Wednesday, and received the parcel on Thursday. On Black Friday evening 2026 the line stayed on the page while the order feed to the warehouse backed up for four hours, so thousands of customers were promised a day the building could not deliver. And Waypoint decision W8 names "availability and delivery promise through APIs" as a prerequisite for anything agent-shaped in 2028: a promise that lives in a theme cannot be offered through an API to a marketplace, a service agent or a shopping agent. Milan's small request is the chapter.

The Vocabulary of Getting It There

Shipping has a small vocabulary a newcomer needs once.

Term	In plain words
Carrier	The company that moves the parcel: PostNL, bpost, DHL, DPD. Each sells several **services** (standard, next-day, evening, pickup point, signature).
Carrier API	The carrier's interface for creating a shipment, getting a label, closing the day and reading tracking.
Label	The sheet on the parcel with the carrier's barcode and routing data; the barcode is the parcel's identity in the carrier's network.

Term	In plain words
Manifest	Closing the day: handing the carrier the list of parcels it is collecting. DHL's Parcel DE Shipping API states that "shipments are transferred to DHL backend systems only when manifested", with an automatic close-out at 17:45 by default (API reference updated 3 November 2026).
Tracking event	One scan with a time and place: accepted, at depot, out for delivery, delivered, attempted, exception.
Pickup point, locker	A delivery address that is not the customer's home: a shop counter, a parcel locker (DHL calls its lockers Packstations and addresses them by a `lockerID`).
Delivery option	What the customer chooses at checkout: home, pickup point, locker, a time window, a date.
Cut-off	The latest order time that still ships today, set by the warehouse's last wave and the carrier's collection (Pick, Pack, Ship: Warehouse Management and Fulfillment).
Lead time, transit time	Days from order to shipment (handling) and from collection to delivery (transit), per lane: an origin, a destination area and a service.
Delivery promise, estimated delivery date (EDD)	The date or window shown before purchase; the EDD is the same thing computed rather than written.
Rate shopping	Choosing the cheapest or fastest carrier service for each parcel, given its size, weight, destination and the promise made.
Multi-carrier platform	A service that sits between the merchant and many carriers: one API for labels and tracking, one event vocabulary, often one contract.

How the Pieces Fit Together

Shipping touches three moments of the order: the promise at checkout, the label at packing, and the tracking after collection. The figure shows the systems around them.

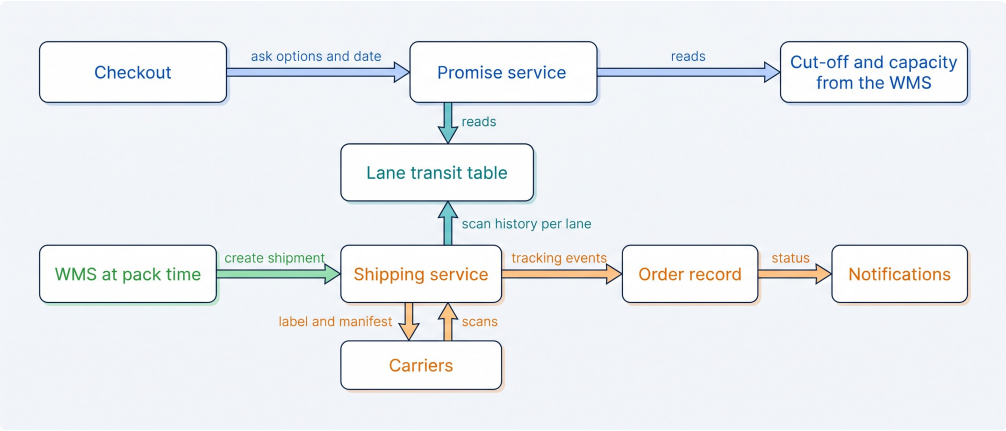


Figure 1: *Three moments, one shipping layer: the promise reads the warehouse's cut-off and the lane table before checkout, the WMS creates the shipment through the shipping service at pack time, and the carriers' scans come back through the same service as normalized events that drive status, notifications and, over time, the lane table itself.*

The requirements differ by moment. The latencies below are assumptions for a merchant of Kestrel & Pine's shape, not vendor figures.

Flow	How fresh	How consistent	How fast	How available
Delivery options and date at checkout	Reflects today's cut-off and the current backlog *(assumption: minutes)*	Same answer for the same address and basket within a session	Under a few hundred milliseconds; checkout waits for it *(assumption)*	Must degrade to a safe default ("2–3 working days") rather than block checkout
Pickup point search	Carrier's list, cached hours	Points that exist and are open	Sub-second	Degrade to home delivery

Flow	How fresh	How consistent	How fast	How available
Label creation at pack	Immediate	Exactly one label per parcel; a retry must not create a second shipment	Seconds per parcel; the packing station waits	If a carrier's API is down, fall back to the backup carrier or queue the parcel
Manifest	Before the carrier's close-out	Every labeled parcel on exactly one manifest	Minutes	Missing the close-out means the carrier does not know the parcels exist
Tracking events in	Within minutes of the scan	Events idempotent by carrier, barcode and event id; order preserved	Minutes	Late is fine; lost must be caught by polling or reconcilia-tion
Tracking number to the order	At label print	One per parcel, linked to the fulfillment order	Within a minute, so the "shipped" message is right	Queued and replayed

Labels and manifests: the carrier's side of the contract

Creating a shipment is a transaction with the carrier: it returns a barcode and a label, and from that moment the carrier expects the parcel. Three details decide whether the integration is reliable.

- **Idempotency** (a retried call must have the same effect as the first). A timeout after the carrier created the shipment, followed by a retry, yields two barcodes for one parcel; one of them will be manifested and never collected, and the customer may receive two tracking numbers. The shipment call must carry the merchant's shipment reference and the integration must look up before it creates.
- **Manifesting**. A label is not a booking. DHL's rule above is explicit; PostNL's current "Warehouse V4" API family includes a separate Confirming step (developer portal, n.d., accessed November 2026). A packing flow that prints labels but

misses the close-out produces parcels that leave the building with no record in the carrier's network.

- **Version drift**. PostNL's developer portal says that "in 2025, we started rolling out the renewed APIs in phases", with Shipment V4, Return V4, Checkout V4 and Warehouse V4 next to the "legacy" Send & Track and Checkout APIs. DHL's Parcel DE Shipping API is at version 2.2.0 (8 October 2026), replaced the Warenpost product with "DHL Kleinpaket (from 1.1.2025)", and scheduled a TLS certificate change for 2–4 November 2026. bpost's Shipping Manager API "moved from v3 to v5" on 17 April 2023. Four carriers, four calendars; a merchant that integrates directly owns four migrations.

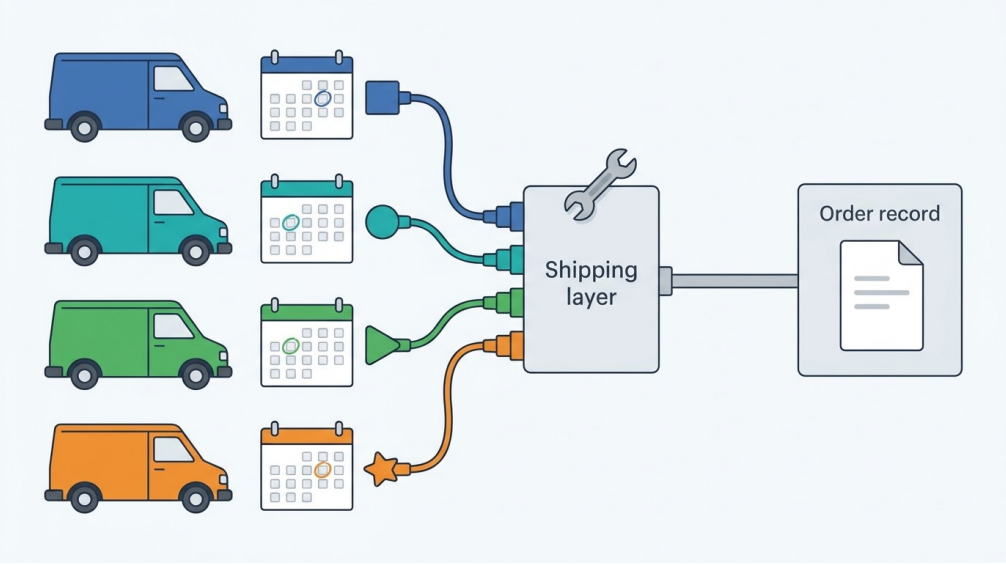


Figure 2: *Four carriers retire API versions on four calendars; a normalizing shipping layer absorbs each migration so the order record never notices.*

Tracking events: one vocabulary for many carriers

Every carrier publishes its own status codes. DHL's Shipment Tracking Unified API (launched 1 October 2018, changelog through September 2026) says that status codes "vary by service" and that clients "should treat unrecognized values as undefined", which is honest and inconvenient. DPD Belgium's Predict notifications fire "when the parcel arrives at the depot, and when it leaves the depot for the receiver" (n.d., accessed November 2026). The merchant needs one vocabulary.

EasyPost's tracker object (n.d., accessed November 2026) is a reasonable public model: ten statuses, `unknown`, `pre_transit`, `in_transit`, `out_for_delivery`, `delivered`, `available_for_pickup`, `return_to_sender`, `failure`, `cancelled` and

`error`, with a scan history of status, message, time and location, and webhooks (calls the vendor's system makes to your URL when something happens) on each update. Shopify's order status page (n.d., accessed November 2026) shows that the customer needs even fewer: "Confirmed, On its way, Out for delivery, Delivered, and Attempted delivery", and only for supported carriers, because "if you fulfill an order without a tracking number, then no shipping updates are received".

Normalized event	What it means	Who consumes it
`pre_transit`	Label created; the carrier has not scanned the parcel	Nobody yet; a parcel stuck here for a day is a missed manifest
`in_transit`	Scanned into the carrier's network, moving between depots	Status page: "On its way"; notification: shipped
`out_for_delivery`	On the van	Status page and notification
`delivered`	Handed over or left as instructed	Status page; the 14-day return clock of The Way Back: Returns, Exchanges, Refunds and Reverse Logistics starts
`available_for_pickup`	At the pickup point or locker	Notification with the location and the holding period
`failure`, attempted	Not delivered: absent, refused, address problem	Service queue; second attempt or re-route
`return_to_sender`	Coming back without a return request	Warehouse receiving; refund or re-ship decision
`exception`	Lost, damaged, customs hold	Service and claims

Two rules follow. **The order's shipping status is derived from these events**, never set by hand, and the status page, the notifications and the service agent read the same derived status (Where Is My Order? Notifications, Service and the Customer Record). And **every event is stored with its lane**, because the scan history is the only data that will ever tell you how long PostNL really takes from Duiven to Nijmegen on a Wednesday in April.

The delivery promise: from sentence to computation

A static promise is a business rule with no data behind it. A computed one is an estimate with inputs, and the inputs are the chapter's argument.

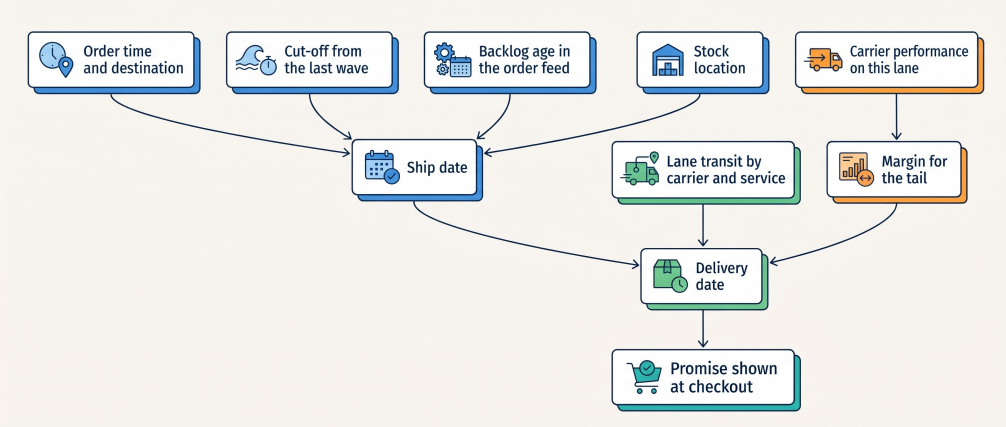


Figure 3: *A computed promise: the ship date comes from the order time, the warehouse's cut-off, the backlog in the order feed and where the stock is; the delivery date adds the carrier's transit for the lane and a margin set by how that lane has actually performed.*

In words, for Eva on Tuesday 13 April at 21:14: the stock is at Duiven; the last wave was 21:00, so the ship date is Wednesday; PostNL's standard service from Duiven to Nijmegen takes one day on that lane; the margin is small because the lane has been reliable; the promise is **Thursday 15 April**. That is the day the parcel arrived. The static sentence said Wednesday. A computed promise would have been honest, and it would have known at 20:00 on Black Friday that the ship date had already slipped by at least a day.

The public evidence shows that even the best-resourced operators treat the promise as a model with a measured effect. Amazon's SPEEDY paper (ACM SIGSPATIAL, 2025) learns from "multiple heterogeneous views of order-to-delivery data" to recommend "dynamic, faster same-day delivery time slots", predicting narrower slots for "60%+ eligible orders" in its test locale, and reports a 21-day online A/B test with +17 basis points in units for the treatment group: a real, small, measured gain from a sharper promise. A 2025 arXiv paper by Ye, Hijazi and Van Hentenryck forecasts order fulfillment time as a distribution rather than a point, with "spatiotemporal features tied to fulfillment location and carrier performance", because "traditional rule-based approaches often fail to capture the inherent uncertainties in delivery operations"; on an industrial dataset it reports "up to 14% higher accuracy and up to 75% better identification of late deliveries" than the rule-based system. The point for a newcomer is not the models. It is that **the promise has a tail**, and a point estimate hides it; the margin in the figure is where the tail lives.

Vendor case studies claim that showing an estimated date lifts conversion by around 10–20% (Maude via Wonderment, Natch via Hive, 2024–2026); they are vendor-reported A/B tests without peer review, so read them as "the effect is positive and worth measuring", not as a number to plan on. Consumer law sets the outer bound: Directive 2011/83/EU (2011), Article 18, requires delivery "without undue delay, but not later than 30 days from the conclusion of the contract" unless otherwise agreed, and Article 20 passes the risk of loss or damage to the consumer only when they have "acquired the physical possession of the goods". Until the scan says delivered, a lost parcel is the merchant's loss.

Delivery options, cross-border and the cost lever

Options at checkout. Pickup points and lockers are a second delivery network with its own lookup: PostNL's Checkout V4 bundles Locations and TimeFrame APIs so a shop can "add delivery options to your checkout"; multi-carrier platforms aggregate them (Sendcloud lists "390K+" service points and nShift "1.2 million pick-up and drop-off points", both n.d., accessed November 2026, both marketing counts). The option chosen must be stored on the order, because it changes the label, the promise, the notifications and, under Article 13(2) of the same directive, the refund: a merchant need not reimburse the extra cost of a delivery "other than the least expensive type of standard delivery".

Cross-border. Kestrel & Pine ships only within the EU, so no customs applies. The moment a merchant ships outside the customs union, three things appear: an import declaration (since 1 July 2021, "required for all goods entering the EU, regardless of their value", with a reduced data set for consignments up to €150, per the European Commission's customs pages, n.d., accessed November 2026), the **IOSS** (Import One-Stop Shop: the scheme that lets a seller charge destination-country VAT at checkout on imported consignments up to €150 and remit it monthly, so the parcel is not stopped for VAT at the border; a 2026 Council document proposes deleting the €150 threshold, so check its status), and customs data per SKU (stock-keeping unit: one sellable variant): a commodity code and a country of origin, which are catalog attributes (What Are We Selling? Products, Variants and the Catalog). DHL's Parcel DE API added paperless customs documents to production on 3 November 2026. Cross-border makes the label call depend on catalog data quality; Tax, Invoices and the Books: From Order to Ledger owns the VAT.

Cost. Shipping is a business lever as much as a logistics fact. Kestrel & Pine's D2C average order is €92 and shipping is free above €75, so most parcels carry a cost the margin absorbs; assume €4.50 to €6.00 per domestic parcel for a merchant of this size (assumption; carrier contracts are confidential). Rate shopping across carriers and services, right-sized boxes from the WMS's cartonization, and steering customers to pickup points (cheaper for most carriers) are the three levers the architecture can offer; the platform's checkout must be able to price options differently and the shipping service must be able to choose per parcel.

The Choices

Who talks to the carriers

Direct integration per carrier. The merchant's WMS or shipping module calls each carrier's API. It gives the best access to carrier-specific features and negotiated rates; it costs one integration per carrier per country, each with its own migration calendar, label format and status codes. It fits a merchant with volume commitments to two or three carriers and an engineer who owns the integrations; it limits a merchant with a small team the first time two carriers migrate in the same quarter.

A multi-carrier platform. A service such as Sendcloud ("30,000+ e-commerce businesses across Europe", "170+" carriers on its marketing page of 6 November 2026; its developer site says "160+", which is the usual inconsistency of marketing counts), nShift ("over 1,000 carrier services and 450+ platforms", n.d.) or ShipStation sits between the merchant and the carriers with one API, one label flow and one tracking vocabulary, and often its own carrier contracts. It absorbs version drift and gives small merchants rates they could not negotiate. It limits a merchant whose carrier contract includes services the platform does not expose, and it is one more vendor between the packing station and the truck.

Through the WMS or the 3PL. Most WMS products and every 3PL bundle carrier integration; Picqer sends "carrier labels and tracking codes" back to the store in one step. The merchant integrates nothing and sees only tracking numbers. It fits Starting and Growing merchants completely; it limits a merchant who wants to offer delivery options at checkout that the WMS cannot price, or who wants the scan history for its own promise.

Through the commerce platform. Hosted platforms sell labels and tracking in their admin for the markets they cover. Simplest of all for a single-country merchant; limited the moment a warehouse system owns the packing station.

Choice	Fits when	Limits when	Who absorbs carrier change
Direct per carrier	Volume contracts, carrier-specific services, an owner for each integration	Small team, many countries, two migrations in one quarter	The merchant

Choice	Fits when	Limits when	Who absorbs carrier change
Multi-carrier platform	Several carriers and countries; no negotiated rates yet; delivery options at checkout wanted	Contract services the platform does not expose; one more vendor in the critical path	The platform vendor
WMS or 3PL bundle	One warehouse owns packing; the merchant wants no shipping code	Checkout options and the promise need data the WMS does not expose	The WMS or 3PL vendor
Commerce platform's labels	One country, one warehouse, no WMS	A WMS or 3PL enters	The platform vendor

The combination Kestrel & Pine runs (a multi-carrier shipping service inside the WMS flow, with carrier contracts of its own for PostNL and DHL) is common at the Omnichannel stage: direct contracts for price, a shipping service for labels and tracking normalization. My rule is that **the normalizing layer is not optional from the second carrier onward**; what is optional is whether you buy it or build it.

How the promise is made

A static rule. "Ships in 1–2 days." "Ordered before 22:00, delivered tomorrow." Costs nothing, needs no data, and is right as often as the warehouse and the carrier happen to agree with it. Fits a Starting merchant with one carrier and one cut-off; limits anyone with two storefronts or one peak.

A transit table. A table of lead days per carrier, service and destination area, plus the warehouse's cut-off, maintained by operations. Honest on an ordinary day, blind to backlog and to the carrier's bad week. Fits a Growing merchant.

A promise service. A component that reads the warehouse's cut-off and backlog, the stock location, the lane table and a margin from recent carrier performance, and answers "options and date for this basket and this address" to the checkout, to marketplaces and, later, to agents. This is what Waypoint W8 means by "delivery promise through APIs". Fits the Omnichannel stage; its cost is that it must be fed, and that a promise service nobody measures is a static rule with more moving parts.

A predictive model. Distributional forecasts of fulfillment time from scan history, as in the 2025 papers above. Right for a Group-stage operator with the data volume to train

and the analysts to watch it; a Giant's Boots for everyone else.

Approach	Inputs	Honest when	Blind to
Static rule	None	Warehouse and carrier happen to agree	Everything
Transit table	Cut-off, lead days per lane	Ordinary days	Backlog, carrier bad weeks, stock location
Promise service	Cut-off, backlog, stock location, lane table, performance margin	Most days including peak, if fed and measured	Events it has no data for yet
Predictive model	Scan history at scale	At volume, with monitoring	Its own drift, without a back-test

Across Scale Stages: When It Fits and When It Limits

Stage	Typical setup	What becomes necessary	Warning signs of outgrowing it
1 — Starting	The platform's labels or a multi-carrier plugin; one carrier; a static promise; a tracking link in the e-mail	A tracking number on every order the moment the label prints; one person who knows the carrier's close-out time	Addresses typed into the carrier's portal; "where is my order" e-mails answered by looking up the carrier site by hand

Stage	Typical setup	What becomes necessary	Warning signs of outgrowing it
2 — Growing	A multi-carrier platform; a second carrier or country; pickup points at checkout; a transit table; the platform's notifications driven by tracking	The delivery option stored on the order; one event vocabulary; the promise text agreed with the warehouse's cut-off; a watch on carrier API changelogs	A different promise on each storefront; labels failing the morning a carrier version is retired; a lost parcel nobody notices until the customer does
3 — Omnichannel	A shipping service in the WMS flow with direct contracts for volume; a normalizing layer for tracking; a promise service reading cut-off, backlog and lane performance; the promise exposed by API to marketplaces; an exceptions queue	Scan history stored per lane; the promise measured (promised vs delivered, per lane, per week); a backup carrier that can be switched on; cross-border data in the catalog if shipping leaves the EU	A static promise during peak; marketplace promise and storefront promise disagreeing; tracking numbers missing on 3PL or marketplace-fulfilled orders; carrier migrations found by an outage
4 — Group	The promise as a shared service with per-brand policies; group-level carrier contracts; carrier invoice audit against scan data; lane performance across brands	One promise API for every brand and channel; an owner for carrier relationships separate from the warehouse	Each brand with its own carrier stack and promise logic; nobody can say the group's on-time rate

The judgement: shipping is the one area where a Growing merchant should buy the normalizing layer early and without regret, because carrier change is external, frequent and uninteresting to the merchant's customers. The promise is the opposite: it should stay simple (a transit table agreed with the warehouse) until the merchant has the cut-off, backlog and lane data to feed a real service, and then it should be built or bought as a component with an API, because by then three other systems need the same answer.

In Practice

Public evidence

The carriers' own developer pages document the drift that this chapter asks the architecture to absorb: PostNL's phased V4 rollout from 2025 with legacy APIs still listed; DHL's Parcel DE Shipping API v2 (current 2.2.0, 8 October 2026), its product rename to DHL Kleinpaket from 1 January 2025 "with automatic correction of old requests", its 17:45 default manifest close-out and its November 2026 certificate change; its tracking API's initial access of 250 calls per day, one every 5 seconds, which rules out polling as the primary tracking source at any real volume; bpost's Shipping Manager API move from v3 to v5 on 17 April 2023 and its bpack integration manual at version 3.3.35 (26 May 2026). Each of these is a Pulled Rug for a merchant that integrated directly and did not subscribe to the changelog.

On what carriers deliver, the public measures are thin. PostNL's Annual Report 2024 (published 2025) reports "delivery quality parcels in NL" of 97% in both 2023 and 2024 and a parcel volume of 371 million items in 2024; that is PostNL's own key performance indicator, and no independent parcel on-time measure for the Netherlands, Belgium or Germany was found. The figures of the Dutch regulator ACM (Authority for Consumers and Markets) concern letters (86% within 24 hours in 2024 against a 95% legal requirement, per DutchNews, 13 June 2025; a fine of nearly €7 million for 2023, reported 12 May 2026). The lesson for an architect is practical: **the only on-time rate you will ever trust is the one you compute from your own scan history**, which is why the events must be stored per lane from the first day.

On the promise, Amazon's 2025 SPEEDY paper and the 2025 conformal-forecasting paper above are the clearest public statements that a promise is a model with a measured conversion effect and a tail; Shopify's five customer-facing states and its "no tracking number, no updates" rule are the clearest statement of what the customer actually needs to see.

Kestrel & Pine: Eva's PostNL parcel, and the promise that should have said Thursday

On Wednesday 14 April 2027 at 08:40 the WMS confirms Eva's shipment and the shipping service creates it with PostNL: one parcel, 1.1 kg, standard service, home address in Nijmegen, with the fulfillment order's reference as the idempotency key. The label prints; the tracking number travels back through the iPaaS (the integration platform that runs the flows between Kestrel & Pine's systems) to the platform within a minute, and the platform sends Eva's "shipped" e-mail with a tracking link. At 18:30 the shipping service manifests the day's PostNL parcels, 1,100 of them (illustrative); PostNL collects at 22:30. The scans arrive as webhooks into the shipping service: accepted at the Nijmegen-area depot at 23:40, out for delivery at 07:50 on Thursday, delivered at 11:20 (times illustrative). Each scan is normalized, stored with its lane (Duiven to the Nijmegen postcode area, PostNL standard, Wednesday collection) and forwarded to the platform, which updates the status page and sends "out for delivery" and "delivered". The whole flow works, and it worked for 336,000 parcels in 2026.

What did not work was Tuesday evening. At 21:14 the storefront showed "ordered before 22:00, delivered tomorrow". A promise service would have read the last wave (21:00, already gone), the backlog (none), the stock location (Duiven), the lane (one day, reliable) and said Thursday. Eva would have seen Thursday and received the parcel on Thursday. The difference between the two is not the parcel; it is one "where is my order" (WISMO) ticket, one disappointed customer and, multiplied by every weekday order between 21:00 and 22:00, a measurable share of the year's complaints that nobody has measured. Under Waypoint W8, the promise becomes an API by 2028; the first version Noor specifies in May 2027 is the transit table plus the WMS's cut-off and backlog, measured weekly as promised-versus-delivered per lane (illustrative plan). Journey 1: Eva's Jacket, From Search to Doorstep follows Eva's order end to end; Journey 4: Black Friday, Hour by Hour shows the same promise on Black Friday evening.

Figure 4: *Eva's promise written versus computed: the theme's sentence said Wednesday, while a promise computed from the 21:00 wave, the stock at Duiven and the one-day lane would have said Thursday 15 April, the day the parcel arrived.*

When It Goes Wrong

The retired API version. A carrier switches off an API version on a published date; the shipping flow fails at the first label of the morning. Detection: subscribe to every carrier's changelog, and test the label call in the carrier's sandbox against the new version before the date. Recovery: switch to the backup carrier for the day (Kestrel & Pine keeps DPD for exactly this), queue the parcels, fix the integration. The multi-carrier platform's job is to make this its problem instead of yours.

Two labels for one parcel. A timeout and a retry create two shipments. Detection: two barcodes against one fulfillment order; a `pre_transit` event that never progresses. Recovery: cancel the second shipment with the carrier before the manifest; make the shipment call idempotent by reference.

The missed manifest. Parcels left on the truck without a close-out; the carrier scans them as unknown or not at all. Detection: parcels in `pre_transit` for more than a few hours after collection. Recovery: manifest late if the carrier allows; otherwise the parcels travel on the barcode alone and the tracking is late or absent. Prevention: the close-out is a scheduled job with an alert, not a button.

The lost or duplicated scan. A webhook delivery fails or arrives twice. Detection: events keyed by carrier, barcode and event id; a daily poll of parcels with no event in 24 hours. Recovery: idempotent event handling; a status derived from the full history, not from the last message received.

The tracking number that never reached the order. The label printed; the confirmation lost its tracking number on the way; the customer sees "unfulfilled" and the status page shows nothing, which is Shopify's "no tracking number, no updates" rule in action. Detection: shipments in the shipping service with no tracking number on the order. Recovery: replay from the shipping service.

The promise at peak. A static promise stays on the page while the order feed backs up (the Queue That Waited of Pick, Pack, Ship: Warehouse Management and Fulfillment). Detection: the age of the oldest unprocessed order, and the promised-versus-delivered measure per day. Recovery: a promise that reads the backlog, or at minimum a peak switch that changes the text; a goodwill voucher policy decided before the day.

Undeliverable, lost, damaged. The parcel comes back (`return_to_sender`), disappears (`exception` with no further scans) or arrives broken. Detection: the normalized events and the absence of a `delivered` within the lane's expected time plus margin. Recovery: a service workflow with the carrier's claims process, and a decision rule (reship from stock, refund) owned by service, with the stock side handled by The Way Back: Returns, Exchanges, Refunds and Reverse Logistics. Under Article 20 of the Consumer Rights Directive the loss is the merchant's until delivery.

The marketplace parcel. Orders fulfilled by the marketplace's program (about 40% of Kestrel & Pine's German marketplace units) produce tracking the merchant never sees unless the connector imports it. Detection: marketplace orders with no tracking in the order record. Recovery: import tracking from the marketplace as events; Selling on Someone Else's Platform: Marketplaces and Channel Integration owns the connector.

What to Take Away

Established.

- Carriers version and retire their APIs, rename services and set manifest rules on their own calendars (PostNL V4 from 2025; DHL Parcel DE v2 and Kleinpaket from 2025; bpost v5 from 2023), and their status codes differ and change (DHL: treat unknown values as undefined).
- A tracking number must reach the order record for any status or notification to exist (Shopify, n.d.); a small normalized event vocabulary such as EasyPost's ten statuses covers every carrier and the customer needs five of them.
- Consumer law bounds the promise: delivery within 30 days absent agreement (Directive 2011/83/EU, Art. 18), risk with the merchant until physical possession (Art. 20), and refund of only the cheapest standard delivery (Art. 13(2)).
- The promise has a tail; published research (Amazon 2025; Ye, Hijazi and Van Hentenryck 2025) treats it as a distribution with a measured conversion effect, not a rule.

Depends on context.

- Buy a multi-carrier platform from the second carrier or country onward unless you have volume contracts, carrier-specific services and an engineer who owns each integration; at Omnichannel scale, direct contracts for price and a normalizing layer for labels and tracking commonly coexist.
- Keep the promise a transit table agreed with the warehouse until you have cut-off, backlog and lane data to feed a service; build or buy the service when marketplaces, service agents or shopping agents need the same answer by API.
- Cross-border customs data (commodity codes, origin, IOSS) matters only if you ship outside the customs union, and then it is a catalog-quality problem before it is a shipping problem.
- Pickup points and lockers are worth offering where the carrier networks are dense (the Benelux and Germany) and are a cost lever as well as a convenience.

The author's synthesis.

- A promise you cannot compute is a promise you cannot measure, and a promise you cannot measure is broken more often than you think. The first step is not a model; it is making the storefront's sentence agree with the warehouse's last wave, and counting how often the parcel arrives on the day shown.
- Store every scan with its lane from the first day. It is the cheapest data a merchant will ever collect and the only source of an on-time rate it can trust.

Questions to Ask

1. What exactly is the delivery promise computed from today, and who would know if it stopped being true?
2. Does the storefront's cut-off agree with the warehouse's last wave and the carriers' collection times, on every storefront?
3. Which system talks to the carriers, and who is notified when a carrier announces an API version retirement?
4. If the label call times out and is retried, can one parcel get two barcodes?
5. When and how is the manifest closed, and what alerts if it is missed?
6. Which event vocabulary does the order status use, and is the status derived from events or set by hand?
7. How long after the label prints does the tracking number reach the order, and what happens to the "shipped" message if it does not?
8. Is the scan history stored per lane, and can you say last month's on-time rate for Duiven to Antwerp by bpost?
9. What does the customer see at checkout when the promise service is down?

10. Which carrier is the backup, and when was it last used?

The parcel is on Eva's doormat in Nijmegen and the status page says delivered. For most orders this is the end. For Kestrel & Pine, with six stores, a click-and-collect pilot and store stock that reaches the ERP (the enterprise resource planning system that keeps the books and the item master) only nightly, the next question is what changes when the delivery address is a shop and the stock is on a store shelf. Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment takes the order into the store.

Sources and Evidence

Company, product and investor sources

- PostNL developer portal and API overview, PostNL, n.d., accessed 2026-11 — the V4 API families (Shipment, Return, Checkout with Locations and TimeFrame, Warehouse with Confirming), the phased rollout from 2025 and the legacy APIs.
- PostNL Annual Report 2024, PostNL N.V., 2025 — delivery quality parcels NL 97% (2023 and 2024); 371 million parcels in 2024.
- Parcel DE Shipping API v2, DHL Group, updated 2026-11-03 — manifesting rule and 17:45 close-out, `lockerID` for Packstations, DHL Kleinpaket from 1 January 2025, paperless customs documents in production from 3 November 2026.
- Shipment Tracking – Unified API, DHL Group, changelog through 2026-09 — status codes vary by service; unrecognized values undefined; initial rate limit of 250 calls per day.
- DHL developer portal, DHL Group, n.d., accessed 2026-11 — the TLS certificate change of 2–4 November 2026.
- Sending tracking notifications (Predict), DPD Belgium, n.d., accessed 2026-11 — depot arrival and departure notifications.
- bpack integration manual and Shipping Manager API downloads, bpost, changelog through 2026-05-26 — the v3 to v5 move of 17 April 2023; manual version 3.3.35.
- Sendcloud, Sendcloud, 2026-11-06, and Sendcloud developer site, n.d., accessed 2026-11 — customer, carrier and service-point counts (marketing figures; inconsistent carrier counts noted).
- nShift, nShift, n.d., accessed 2026-11 — carrier services, pick-up and drop-off point counts (marketing figures).
- Trackers, EasyPost documentation, n.d., accessed 2026-11 — the ten tracker statuses, scan history and webhooks.
- Understanding order status pages, Shopify Help Center, n.d., accessed 2026-11 — the five customer-facing states and the "no tracking number, no updates" rule.
- Picqer, Picqer, n.d., accessed 2026-11 — carrier labels and tracking codes returned to the store in one step.

- How Maude leveraged delivery promise to increase conversion rate by 12%, Wonderment, 2024–2026, and Natch case study, Hive, 2024–2026 — vendor-reported conversion effects of showing a delivery date; no peer review.

Standards, regulation and official documentation

- Directive 2011/83/EU on consumer rights, Publications Office of the EU, 2011 — Articles 13(2), 18 and 20: cheapest standard delivery refund, 30-day delivery, passing of risk.
- Customs formalities for low value consignments, European Commission DG TAXUD, n.d., accessed 2026-11 — import declarations for all goods since 1 July 2021; the reduced data set up to €150.
- The One Stop Shop, European Commission, n.d., accessed 2026-11 — the Import One-Stop Shop and the €150 ceiling.
- Council document ST-15179-2026, Council of the EU, 2026 — the proposal to delete the €150 threshold (status not verified).

Research and practitioner sources

- SPEEDY: Framework for sharpening promise time estimates in sub-same-day delivery, Sarkar and Mandayam Comar, Amazon Science, ACM SIGSPATIAL, 2025 — learned promise windows and the A/B test result.
- Conformal Predictive Distributions for Order Fulfillment Time Forecasting, Ye, Hijazi and Van Hentenryck, arXiv, 2025 — fulfillment time as a distribution; accuracy and late-delivery identification gains over rules.
- PostNL fails to meet next-day delivery targets for sixth year, DutchNews, 2025, and PostNL calls €7m fine for late deliveries disproportionate, DutchNews, 2026 — the regulator's letter-delivery figures and fine, cited as the contrast to the parcel KPI.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

17. Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment

IN THIS CHAPTER, YOU WILL: Learn what a store adds to the architecture, how click-and-collect, ship-from-store, return-in-store and endless aisle work, and when to change the point of sale.

KEY POINTS:

- **A store is a second system of record, not a sales channel.** It creates orders, holds stock, knows customers and takes money, often while offline and under fiscal rules of its own. The architecture has to model it as a stock location, an order source and an identity source, each with a named owner.
- **Store stock may be promised only with a buffer, a confirmation or a count behind it.** The classic field study found about two-thirds of store inventory records wrong, and a nightly copy makes a wrong number older. The Store Stock Mirage is what promising at face value from a nightly copy produces.

- **"Unified commerce" is a goal state, not a product.** One record per order, per stock unit and per customer across channels can be reached with one platform or with an order management system and an identity layer over several. Which path fits depends on the scale stage, which is why Kestrel & Pine fixed the joins first and deferred its point-of-sale decision to 2028.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Phantom Stock** — store stock copied nightly into the back office is promised online as if it were counted this minute.
- **Schrödinger's Order** — a click-and-collect order exists in the web platform, in an e-mail to the store and as a till receipt, and none of the three knows about the others.
- **The Familiar Stranger** — the store's loyalty list, the web shop's accounts and the newsletter each count the same shopper as a different person.

A web shop is a system. A store looks like a building with people in it, which is why it is so often left out of the architecture. But every store runs a **point of sale** (POS, the till: the software and hardware that complete a sale, print the receipt and record what was sold), and that POS creates orders, holds stock, recognizes customers and takes money, as the web platform does. **The store is a second commerce system, with a different vendor, a different data model and different rules**, standing next to the first.

For a merchant with one store, the two systems can live apart: the store sells what is on its shelves, the web shop sells what is in the warehouse, and a nightly report tells finance what both did. The trouble starts when the business asks the two to make promises about each other: order online and collect in the store, ship an online order from a store's shelf, return a web order at the till, or sell a customer in the store a size the store does not have. **Each of those services is a promise made against stock that the promising system does not control**, and each needs the store's stock, orders and customers visible to the other systems quickly and accurately enough to keep it.

This chapter explains what the store adds, how the four omnichannel services work, what store stock is worth as a promise, and how to decide between the platform's own POS and a separate one. It owns the terms **POS**, **store stock**, **click-and-collect**, **ship-from-store** and **unified commerce**, and leans on Is It in Stock? Inventory, Availability and the Promise You Can Keep for the availability calculation, The Order Record: Lifecycle, State and the Order Management System for the order hub and One Customer, One Product: Identity and Master Data Across Systems for who the store customer is.

Where we are in the order's journey. Eva's jacket (the recurring order from Meet Kestrel & Pine) never enters a store: it is ordered online, shipped from the Duiven warehouse and delivered to her door in Nijmegen. But Kestrel & Pine has a store in

Nijmegen, and the services this chapter describes are the alternatives Eva did not take: collecting the jacket there, having it shipped from its shelf, or exchanging size M for L at the counter. The chapter sits beside the fulfillment step, as the branch of the journey that passes through a store.

Setting the Scene at Kestrel & Pine: The Utrecht Pilot and a Deferred Decision

Kestrel & Pine, the fictional outdoor brand this book follows, has six stores: Arnhem, Utrecht, Amsterdam, Nijmegen, Eindhoven and Antwerp. Each holds about **1,800 variants** of stock. Together they took **118,000 transactions** in 2026 and earned **€6.0 million**, an average of **€51** per receipt. The stores run a **cloud POS** from a vendor other than the platform, chosen in 2019 when the first store opened. Card payments go through a separate **terminal provider** (the company whose card readers sit on the counter). Store stock lives in the POS and is copied to the ERP (the enterprise resource planning system: the books, purchasing and the item master) once a night. There is **no link between the POS and the platform** at all.

In 2026 the Utrecht store piloted **click-and-collect**: a shopper orders on the Dutch storefront, chooses "collect in Utrecht", and picks the order up at the counter. With no link between the systems, the pilot ran on people. The platform checked the item against the ERP's copy of Utrecht's stock, which was last night's. It then e-mailed the store. A member of staff walked to the shelf, found the item or did not, and phoned or e-mailed the customer. In **December 2026** the pilot promised **37 items** that the store had already sold earlier that day, before the night's copy could say so. This book calls this **the Store Stock Mirage**: the pattern *The Phantom Stock*, seen from the store side.

When Noor Visser, the new Head of Engineering, reviewed the incident with Daniel Okafor (Head of Operations, who owns the stores) in February 2027, the obvious question was whether to replace the POS with the platform's own, so that store and web would share one stock and order record. The Waypoint program, approved in March, answered no, for now. Decision **W4** keeps Kestrel & Pine on its SaaS (software-as-a-service: hosted and rented) platform, adds the capabilities the platform lacks beside it, and **evaluates the platform's own POS in 2028**. Decision **W1** makes the POS the **system of record** for store stock (the one system whose value counts as the truth when copies disagree) and has an **availability service** compute what each channel may promise from stock **events** (a message sent the moment a sale or a count changes a quantity, instead of a nightly copy). Decision **W2** brings every channel's orders into a bought **order management system** (OMS), "later stores" included. The rest of this chapter is the reasoning behind that order of moves: fix what the store tells the other systems, and how fast, before changing the till.

What a Store Adds to the Architecture

In plain words, a store adds four things the online architecture must now take into account, and two constraints it has never had to live with.

What the store adds	What it means for the architecture	Chapter that owns the concept
A second order source	Every receipt is an order created outside the platform, with its own number, its own line items and its own tax treatment	The Order Record: Lifecycle, State and the Order Management System
A stock location per store	Six stores are six more places stock can be promised from, each counted by hand and sold from by people	Is It in Stock? Inventory, Availability and the Promise You Can Keep
A second customer list	The POS knows a customer by loyalty card, phone number or e-mail captured at the till, if at all; most store sales are anonymous	One Customer, One Product: Identity and Master Data Across Systems
A second payment path	Terminal payments settle through the terminal provider, with their own references and payout files	Taking the Money: Payment Providers, Authorization, Capture and Refunds
Offline operation (constraint)	The store must keep selling when the connection drops; for those hours it is its own system of record, reconciled later	this chapter
Fiscal rules (constraint)	Some countries prescribe how a sale is recorded and what a receipt must contain, down to a signed transaction counter	this chapter, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation

The two constraints make a POS a different kind of product from a web platform. A web checkout can refuse to work when a dependency is down; a till cannot. A web order can be a row in a database; in Germany a store sale must be signed by a certified security device and carry a sequential number that makes gaps detectable. That is why "just use the platform's POS" is sometimes right and sometimes not available at all.

Figure 1: *A store is a second commerce system, not a channel: like the web shop it creates orders, holds stock, knows customers and takes money, and it must also keep selling offline and follow fiscal rules.*

The offline rule

Vendors document what their POS does without a connection. Shopify's help pages (n.d., accessed November 2026) say its POS needs the internet to sync with the admin, e-mail receipts and capture card payments; cash sales work offline, and card payments taken offline are "captured after your device reconnects to the internet", with a merchant-set daily limit per device, a declined payment that surfaces only after reconnection, and advice to reconnect "ideally within 24 hours". Square's help page (n.d., accessed November 2026) goes further: offline payments "expire after 72 hours if you don't reconnect", and "You're responsible for any expired, declined, or disputed payments accepted while taking offline payments."

For the hours a store is offline, it is an independent system of record for sales and stock. Its transactions need locally generated identifiers that never collide; its stock changes arrive late and in a batch; the merchant, not the payment provider, carries the

risk of a declined card. Any online promise made against that store's stock during those hours is a guess.

The fiscal rule

Fiscalization is a legal requirement that every sale be recorded in a way the tax authority can verify. Its form differs by country, and the differences decide which POS products can be used where.

Country	Mandate for retail stores	Mechanism	Dates and source
Germany	Yes, every electronic cash-register-like system	A certified **technical security device** (TSE: security module, storage medium, uniform interface) signs each transaction; a sequential transaction number so that "gaps in transaction records are recognizable"; a receipt obligation; registration with the tax office within one month	§ 146a Fiscal Code; KassenSichV of 26 September 2017 (last amended 14 January 2026); registration via ELSTER (the tax administration's online portal) since 1 January 2025
Belgium	Not for shops; the "white cash register" (GKS) applies only to restaurant and catering services above €25,000 turnover	For hospitality: a certified register, a certified fiscal data module ("black box") and a VAT signing card; GKS 2.0 mandatory for new hospitality businesses from 1 July 2026	FPS Finance GKS site, 2026; records kept seven years

Country	Mandate for retail stores	Mechanism	Dates and source
Italy	Yes	Electronic storage and telematic transmission of daily receipts to the tax agency	From 1 July 2019 (turnover above €400,000) and 1 January 2020 (all others), Agenzia delle Entrate release of 29 June 2019
Netherlands	No certified register	Register data are part of the administration, kept seven years; a voluntary private quality mark	Belastingdienst brochure, n.d., accessed November 2026

The table is my compilation of the sources listed at the end. For Kestrel & Pine the point is specific: its five Dutch stores faced no certified-register mandate when the POS was chosen, and Antwerp's receipts follow Belgian VAT and seven-year record-keeping rules, not the hospitality-only black box. A first **German store** would need a POS with a certified TSE, registered with the tax office, and that requirement alone can rule a product out. A store opening in a new country is a POS decision as much as a logistics one.

How the Pieces Fit Together

The January 2027 landscape at Kestrel & Pine is the shape many Growing-stage merchants have when they open stores: a POS chosen for the stores, an ERP as the meeting point, and the web platform on the other side of a nightly copy.

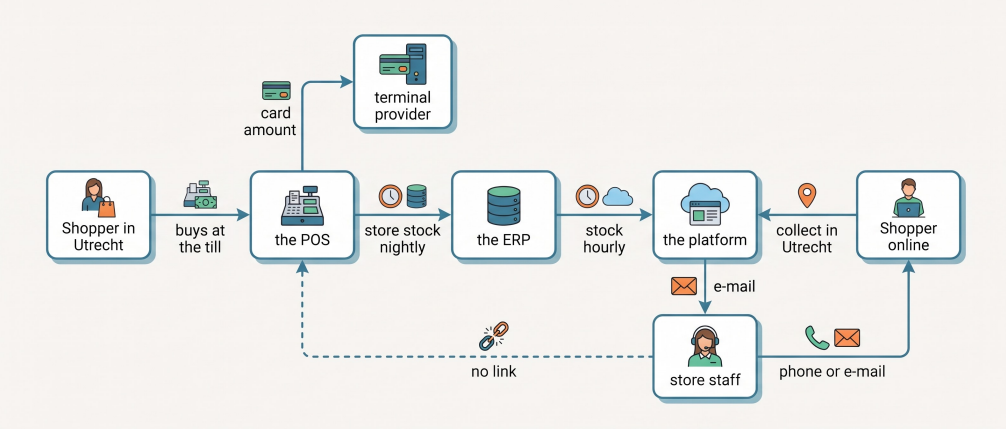


Figure 2: *Kestrel & Pine's stores in January 2027: a sale at the till reaches the platform the next day by way of the ERP, while a click-and-collect order reaches the store as an e-mail and never reaches the POS at all.*

The target shape is different in two places: store stock flows as events into one availability calculation, and store orders of every kind pass through one order hub. The components and their responsibilities:

Component	Responsibility	Owns (system of record for)	At Kestrel & Pine
The POS	Ring up sales, take terminal payments, issue receipts, handle returns at the counter, count stock	Store transactions; store stock (W1)	The cloud POS from 2019
The terminal provider	Card acceptance in the store	The card transaction and its reference	Separate from the online PSP (the payment service provider for online payments)

Component	Responsibility	Owns (system of record for)	At Kestrel & Pine
The availability service	Compute what each channel may promise per location from stock events, with buffers	The available-to-promise number (what a channel may still sell: physical stock minus what is promised, minus buffers)	A module of the OMS, from Q4 2027
The OMS	Hold every channel's orders in one state model; route each order line to a location (**sourcing**); hand work to the warehouse or a store	The operational order record	Bought, Q4 2027; store receipts imported "later"
Store staff tooling	Show staff what to pick, confirm, mark ready and hand over; look up stock elsewhere; start a return	Nothing; a client of the OMS and the POS	E-mail and phone today
The customer record	Recognize the same person across till, web and newsletter	Customer identity	Four unlinked lists; One Customer, One Product: Identity and Master Data Across Systems

The four omnichannel services

The vocabulary first, because vendors use several names for each.

Service	Order placed	Stock used	Handed over	What it needs from the architecture
Click-and-collect (BOPIS, buy online pick up in store)	Online	Store stock, or warehouse stock sent to the store	At the counter	Store stock visible to availability; an order line routed to a store; staff tooling to confirm, hold and hand over; a notification when ready
Ship-from-store	Online	Store stock	Carrier collects at the store	Everything above plus labels, packaging and a carrier pickup at the store; routing rules that weigh store cost and capacity
Return-in-store	Online	None	Return accepted at the till	The till can find an online order, accept the item, trigger one refund through one path, and restock to the store or send it back

Service	Order placed	Stock used	Handed over	What it needs from the architecture
Endless aisle	In store	Warehouse or another store's stock	Carrier to the customer	The till can see availability elsewhere, create an order in the order hub, and take payment for goods that are not in the building

Each row is a promise made against stock somewhere else, and **all four are the same three problems in a different order**: knowing what stock is where (availability), deciding which location serves the order (sourcing), and giving the store the tools to do its part (staff tooling). The requirements differ by service:

Flow	How fresh must store stock be	How consistent	How fast	What happens when the link is down
Store sale reduces online availability	Minutes, as an event; nightly is the Mirage	Eventually consistent is fine; the buffer absorbs the lag	Not latency-critical	The store keeps selling; availability holds the last known number minus a larger buffer

Flow	How fresh must store stock be	How consistent	How fast	What happens when the link is down
Click-and-collect promise	Minutes, plus a per-store buffer	The order must be visible to the store within minutes and confirmed by a person before "ready" is sent	The confirmation, not the promise, is the commitment	Offer collection "within two hours" rather than "now", or route to the warehouse
Ship-from-store routing	Minutes	The chosen store must accept the fulfillment order before the carrier label exists	Routing in seconds at order time	Fall back to the warehouse
Return-in-store	Order lookup in seconds	Exactly one refund, from one system	Seconds at the counter	Accept the item, record the return, refund when the link returns, never twice
Endless aisle	Seconds, from the shared availability number	The order is created in the hub, not re-keyed later	Seconds at the counter	Do not promise; take the customer's details and call back

The target flow for click-and-collect looks like this, with the OMS and the availability service in place:

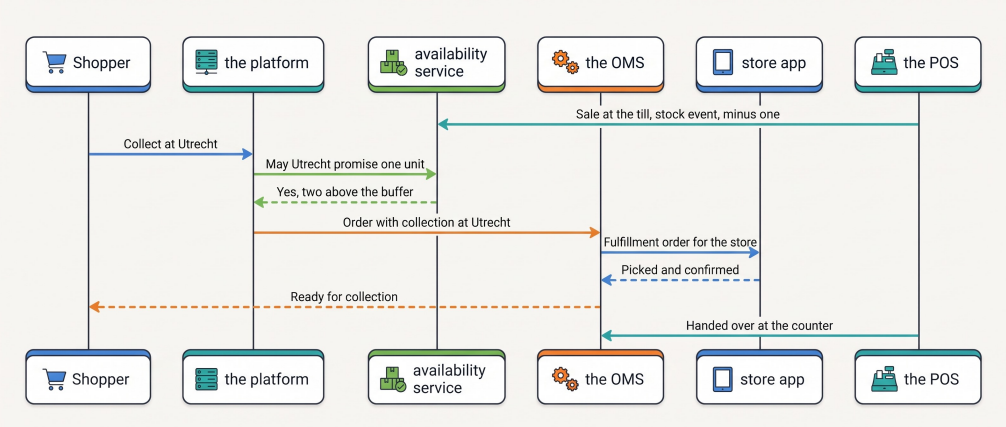


Figure 3: *Click-and-collect done with the joins in place: the till reports every sale as a stock event, the availability service promises only above a buffer, the order hub routes the work to the store, and a person's confirmation, not the stock number, is what turns the promise into "ready".*

Notice where the commitment sits. The storefront's "available for collection" is a **promise with a buffer**; the store's "picked and confirmed" is the **commitment**; the customer hears "ready" only after the second. That ordering is the whole design, and the Utrecht pilot lacked it.

What store stock is actually worth as a promise

The buffer and the confirmation are not caution for its own sake. Store inventory records are wrong far more often than warehouse records, and the research is old but unambiguous. DeHoratius and Raman, in *Management Science* in 2008, examined "nearly 370,000 inventory records from 37 stores of one retailer" and found **65% to be inaccurate**; auditing reduced inaccuracy, store complexity increased it. The data are from one US retailer in the 2000s, before RFID (radio-frequency identification: a tag on each item that a reader counts without line of sight), so take the mechanism rather than the number. Auburn University's RFID Lab reported in 2018 (Project Zipper, eight brands, five retailers) that order audits using barcode data found inaccuracies "in 69% of cases" against "less than 0.01%" with RFID. Both sources are more than three years old; nothing newer with an open methodology turned up in this book's research.

The ways to make a store-stock promise honest, in rising order of cost:

1. **A per-store safety threshold**: never promise the last N units of a variant at a store. Cheap, loses some sales on slow movers, and is the first thing to do.
2. **A confirmation step**: the store confirms the pick before the customer is told to come. Costs a few minutes of staff time per order; removes almost all broken promises.

3. **Exception-driven cycle counting**: when a store cannot find an item the system said it had, that variant is counted that day, and the count corrects the record. Turns every miss into a data fix.
4. **RFID**: tags on every item and readers at the door and the counter. Expensive; the evidence above is why fashion retailers at Group stage do it anyway.

Kestrel & Pine's interim fix for Utrecht, until the availability service arrives, is the first two: a threshold of two units per variant per store and a staff confirmation before the "ready" message. It costs nothing to build and would have prevented most of the 37.

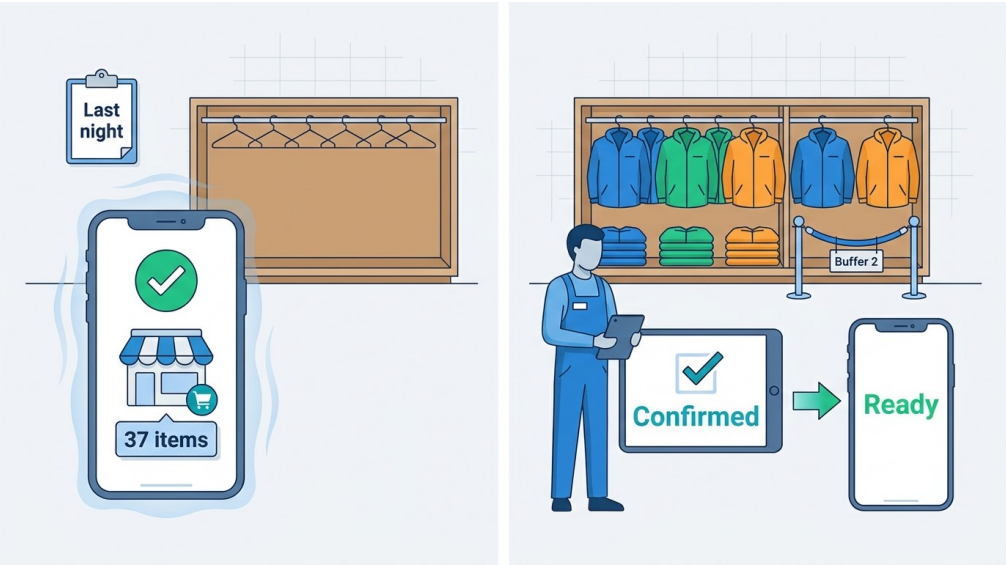


Figure 4: *The Store Stock Mirage and its interim fix: in December 2026 a nightly copy promised 37 items already sold; a buffer of two units and a staff confirmation before "ready" would have prevented most of them.*

The Choices

A merchant with stores has four realistic ways to connect them to the rest of the landscape. Each is a real product category, and each is right somewhere.

1. **The platform's own POS**. The web platform's vendor sells a till on the same back end: one catalog, one stock record per location, one order list, one customer list. Shopify's POS pages (n.d., accessed November 2026) describe "buy online, pickup in store, or buy in store, ship to customer", staff roles and "Returns and exchanges at any location"; POS Pro is €79 per location per month on its euro page as of November 2026. The Dutch vendor Vendit makes the same promise from the POS side: a till "from the same database as

your webshop", so that "if you sell the last item at the till, the article is immediately at zero in your webshop" (n.d., accessed November 2026, translated). The joins disappear. The limits: the platform's POS must be available and fiscally certified in every country you have stores in, it inherits the platform's offline rules, and it couples the stores to the platform contract, so changing the platform later means changing the tills too.

2. **A specialist POS, integrated by events through the order hub**. The stores keep a POS built for stores (fiscal certification, offline operation, loyalty, counts), and three integrations connect it: stock events to the availability service, store receipts imported into the OMS as orders, and fulfillment orders from the OMS to a store app. This is where Waypoint takes Kestrel & Pine: the best POS for the stores and the best platform for the web, at the cost of three contracts and the reconciliation jobs behind them.

3. **A specialist POS, integrated by nightly batch through the ERP**. The January 2027 state. It works until someone promises store stock outside the store; then it produces the Mirage.

4. **The ERP's POS**. Some mid-market ERPs include a till module, so store stock, store sales and the books are one record and the web platform is the odd one out. Attractive for a wholesaler with a few outlet stores; less so for a brand whose stores are a growing channel, because the ERP's till rarely has the customer and omnichannel features of a specialist POS (The Back Office: ERP Integration and Who Is the Hub).

	Platform's POS	**Specialist POS by events through the OMS**	**Specialist POS by nightly batch**	**ERP's POS**
Store stock visible online	Immediately, one record	Within minutes, as events	Next day	Same record as the ERP; the web gets a copy
Store orders in the order record	Native	Imported, with their fiscal identifiers	Only as ERP postings	Native to the ERP, not to the OMS
Fiscal coverage	Where the platform vendor supports it	Chosen per country	Chosen per country	Where the ERP supports it
Offline operation	The platform's rules and limits	The POS's rules	The POS's rules	The ERP's rules

	Platform's POS	Specialist POS by events through the OMS	Specialist POS by nightly batch	ERP's POS
Integrations to own	Few	Three contracts plus reconciliation	One batch job	Web-to-ERP only
Exit cost	The tills change with the platform	Each part replaceable	Low	The tills change with the ERP
Fits best	Starting and Growing; one country; web-first brands	Omnichannel; several countries; stores that matter	Stores with no omnichannel promises	Wholesalers with outlet stores

My rule: **choose by the number of countries and the weight of the stores, not by the vendor's demo**. A web-first brand with two stores in one country should take the platform's POS and enjoy the one record. A merchant whose stores earn a serious share of revenue, in more than one fiscal regime, should keep a store-grade POS and spend the money on the events and the order hub. Kestrel & Pine sits between: six stores in two countries, **12.5% of revenue**, one fiscal regime that is lenient and one that is lenient for shops. That is why W4 defers rather than decides: the 2028 evaluation will have an availability service and an OMS in place, and the question will have shrunk to "which till", not "how do stores join the architecture".

"Unified commerce", and who is speaking

You will hear **unified commerce** in every vendor meeting. Shopify's 2026 blog post defines it as connecting "sales channels, inventory, and customer data through a single back-end platform", against omnichannel, which "doesn't connect back-end systems". Adyen's point-of-sale documentation (n.d., accessed November 2026) says the same from the payments side: in-person payments "processed on the same platform as online payments", with shopper recognition across channels. No analyst definition could be fetched for this book; every definition above comes from a vendor with a platform to sell.

I use the term for the **goal state**: one record per order, per stock unit and per customer, visible to every channel in near real time. It can be reached with one platform (choice 1) or with an OMS and an identity layer over several systems (choice 2). When a vendor

says "unified commerce", ask which they mean, and whether their definition would survive your store in Germany.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
1 — Starting	No store, or a pop-up on the platform's POS; stock is the back room; store sales are a line in the monthly accounts	Nothing beyond the platform's own POS; one location record	Store count and platform count disagree every month-end; staff keep a notebook of "web orders to pick up"
2 — Growing	One to three stores, often on a specialist POS chosen by the store manager; POS to accounting or ERP nightly; no omnichannel promises	A decision on which system owns store stock; a per-store threshold before any click-and-collect pilot; one customer key (loyalty id or e-mail) captured at the till	Click-and-collect run by e-mail and phone; the first Mirage; store managers who cannot say whether online refunds come off their sales

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
3 — Omnichannel	Several stores in more than one country, a visible share of revenue; stock events from the POS; an OMS that routes to stores; staff tooling; return-in-store	The availability service with store buffers; store receipts imported into the order hub with fiscal identifiers; one refund path; reconciliation of POS, OMS and ERP; a fiscal checklist per country	Store-stock promises without a confirmation step; ship-from-store launched before routing knows store capacity; a POS API deprecation nobody owns
4 — Group	Dozens to hundreds of stores across fiscal regimes; several POS products after acquisitions; RFID in fashion; stores as fulfillment nodes with their own cut-offs and carrier pickups	One fulfillment-order contract every POS and WMS honors; a registry of which store does what; accuracy targets; per-brand decisions on shared versus separate POS	A single POS forced on brands with different fiscal needs (One Size Fits None); store fulfillment that undercuts store sales because the economics were never modeled

The judgement: the store joins the architecture at the Growing stage, with the first omnichannel promise, and the cheapest moment to decide who owns store stock is before that promise, not after the first broken one. The POS itself rarely needs to change until the merchant opens stores in a country whose fiscal rules its POS cannot meet, or until the stores' weight justifies a single record. Kestrel & Pine has reached the first point with its joins and not the second with its till.

In Practice

Ship-from-store on someone else's marketplace: Zalando Connected Retail, 2021 to 2027. Zalando's Connected Retail program let physical shops list their store stock on Zalando and ship the parcels from the store; trade press reported more than 3,400

active stores in nine markets in early 2021. In April 2026 TextilWirtschaft reported a Zalando spokesperson confirming that the program ends by the end of June 2027, to bundle all partners in one central model. Two lessons transfer: ship-from-store works at scale when the store's stock feed and the return path are built properly, and programs retire. A merchant that exposed store stock through its own store-stock feed rather than a program-specific tool faces a connector change, not a redesign (Selling on Someone Else's Platform: Marketplaces and Channel Integration).

Store orders in one order model. Microsoft's documentation for its Business Central connector to Shopify (published 14 October 2026) notes that orders created through Shopify POS carry a staff identifier that the connector maps to a salesperson. Small detail, large point: on a platform with its own POS, a store sale and a web order travel to the ERP by the same road. A merchant with a separate POS builds that road itself, and the import must carry the store's own identifiers (receipt number, fiscal transaction number, staff id) rather than re-create the sale as a web order.

Routing to stores. Adobe Commerce's inventory documentation (updated August 2026) ships two source-selection rules, a fixed priority and a distance rule that picks "the closest source". Shopify binds work to a location through a **fulfillment order** whose assigned location may change only while it is open, scheduled or on hold (n.d., accessed November 2026). Kibo's documentation (n.d., accessed November 2026) describes routing that returns a suggestion, logs why locations were rejected, "does not create shipments or deduct inventory", and lets a pickup order trigger a transfer shipment to the store. That is the ladder: simple rules in the platform, then an explainable routing decision separate from the deduction of stock, which is the OMS's job once stores, cost and capacity enter the rule (The Order Record: Lifecycle, State and the Order Management System).

Store payments and PCI scope. With **point-to-point encryption** (P2PE; the PCI Security Standards Council lists validated solutions by exact version), the card number is encrypted inside the terminal and decrypted only at the payment provider, so a **semi-integrated** terminal that receives an amount and returns a result keeps the POS out of the scope of PCI DSS (the Payment Card Industry Data Security Standard) (Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers). Keep the terminal provider's transaction reference on the receipt; it later matches the payout file (Taking the Money: Payment Providers, Authorization, Capture and Refunds).

Adoption, a decade ago. The only open, dated adoption figures found are old and UK-weighted: a 2016 UPS survey of over 700 European retailers put stores used as online fulfillment centers at 27%; a 2018 Savills survey of UK stores found 47% offering click-and-collect, about one in ten in ship-from-store, and nearly one in three store managers in one category who "did not know whether online refunds were deducted from store sales". Treat the percentages as history; keep the last finding. "Whose sale is it" must be answered before the first store ships a web order.

Kestrel & Pine, worked: the Utrecht pilot, before and after

The numbers below are Kestrel & Pine illustrations; where they are not among Kestrel & Pine's recorded figures they are labeled as assumptions.

	December 2026 (the Mirage)	**Interim fix, from April 2027**	**Target, from Q4 2027**
Store stock seen by the platform	ERP copy, up to a day old	Same copy, minus a threshold of 2 units per variant *(assumption)*	POS stock events into the availability service, minutes old, per-store buffer
Promise shown online	"Available for collection today"	"Collection today, we confirm within two hours"	"Ready in two hours" after confirmation, computed per store
How the store learns of the order	E-mail from the platform	E-mail, with a reply template that confirms or declines	Fulfillment order in the store app from the OMS
Commitment to the customer	At order placement	At the store's confirmation	At the store's confirmation
Broken promises	**37 items** in the month	Illustrative target under 5 *(assumption)*	Measured per store as "promised but not found"
Store receipt for the handover	A POS sale at €0 keyed by hand *(local detail)*	Same	Handover recorded against the OMS order; the POS imports it with its receipt number

Two details deserve a sentence. The €0 till sale is how many pilots record a collection, and it is why the order exists three times: in the platform (paid), in an e-mail (picked) and in the POS (a zero sale); that is *Schrödinger's Order* with a store in it. And the interim promise is deliberately weaker than the original: a two-hour window kept every time beats "today" broken once in thirty.

When It Goes Wrong

Stores fail in ways online systems do not, because people and buildings are involved. The ones that matter most, with how to notice them and how to recover:

- **The till and the web sell the same last unit.** A store sale and an online reservation race for the final unit before the store's event reaches availability. *Detection*: "promised but not found" per store, counted daily. *Recovery*: the buffer makes it rare and the confirmation step catches it before the customer travels; when it still happens, the OMS reroutes to the warehouse or cancels with a notification, and the variant is counted.
- **Offline session replay creates duplicates.** A store reconnects after three hours and uploads its sales; the upload is retried after a timeout, and the same receipts arrive twice. *Detection*: daily sales count against the receipt-number range. *Recovery*: every import is **idempotent** on the POS transaction number (receiving the same receipt twice has the same effect as once), which is also what Germany's fiscal transaction counter is for (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation).
- **The nightly job fails silently.** The POS-to-ERP copy stops on a Friday; by Monday the ERP's store stock is three days old and the platform has promised against it all weekend. *Detection*: a monitor that alarms when the copy is older than its schedule, not only when it errors (APIs, Events, Files and Middleware: How Commerce Systems Talk). *Recovery*: rerun and reconcile the weekend's orders by hand; the cure is events with a freshness check.
- **Return-in-store refunds twice.** A web order is returned at the Nijmegen counter; the till refunds to the card through the terminal provider, and the returns portal, seeing the item "received", refunds through the platform's API as well: the store's version of *The Double Refund* (212 returns, €19,400, January 2027). *Recovery*: one refund path per order, decided once; a return recorded at the till closes the portal request before anything is paid (The Way Back: Returns, Exchanges, Refunds and Reverse Logistics).
- **The store charges a different price.** During the Price Drift of October 2026 the German storefront sold the Ridgewalker at €179 while the stores charged €199; the stores were right by accident, because the POS has its own hand-loaded price list. *Recovery*: one pricing source publishing to the POS as to the storefronts (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax).
- **The loyalty list and the web account never meet.** Eva Janssen is a POS loyalty customer and an online account with no shared identifier (the *Familiar Stranger* pattern). Return-in-store still works, because the till finds the order, not the person; store purchases simply never reach her online history. *Recovery*: a declared matching rule with a confidence level (One Customer, One Product: Identity and Master Data Across Systems).

- **A fiscal rule arrives with a store.** A German lease is signed before anyone checks whether the POS has a certified TSE (*The Long Fuse*). *Recovery*: store openings go on the regulatory calendar (Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation); a POS per country is acceptable, a store without a compliant till is not.

What to Take Away

Established.

- A store's POS is a system of record for store transactions and, for the hours it is offline, for store stock; vendors document offline capture, limits and merchant-borne risk (Shopify and Square help pages, accessed November 2026).
- Store inventory records are far less accurate than people assume: 65% inaccurate in the 2008 DeHoratius and Raman study; 69% of audited orders with barcode-data errors against under 0.01% with RFID in the 2018 Auburn study. The numbers are old; the mechanism is not.
- Fiscalization differs by country and constrains POS choice: Germany requires a certified security device and sequential transaction numbers (§ 146a AO, Kassen-SichV 2017); Italy requires telematic transmission (2019–2020); Belgium's certified register applies to hospitality only; the Netherlands requires seven-year records and no certified device.
- Platforms and order management systems separate routing from the deduction of stock and bind work to a location through a fulfillment order that can move only before work starts (Shopify, Kibo and Adobe Commerce documentation, 2026).

Depends on context.

- The platform's own POS is the right choice for web-first brands with a few stores in one lenient fiscal regime; a specialist POS integrated by events is the right choice once stores carry weight or span fiscal regimes. The line moves with the platform vendor's country coverage.
- Click-and-collect against store stock needs at least a threshold and a confirmation step; ship-from-store needs routing rules that know store capacity and cost; both can wait until an availability service exists, and both can run earlier with weaker promises.
- Whether store sales are imported into the OMS from day one or "later" (Kestrel & Pine's W2) depends on whether anyone needs a store receipt in the order hub: returns-in-store and customer history do; finance does not, because the ERP already has the nightly postings.

The author's synthesis.

- Fix what the store tells the other systems, and how fast, before changing the till: a store-stock event feed, a buffer and a confirmation step remove most broken promises, and they are the same work whichever POS you end up with.
- Promise only what a person has confirmed. The storefront's availability number is a forecast; the staff member's "picked" is the commitment; the customer should hear from the second, not the first.

Questions to Ask

1. Which system is the system of record for store stock, and how old is the copy that the storefront promises from?
2. What does the POS do when the connection drops, for how long, and who carries the risk of a declined card taken offline?
3. Which countries will we have stores in within three years, and does the POS meet each country's fiscal rules today?
4. When a click-and-collect order is placed, what exactly is the customer promised, and at what moment does a person confirm it?
5. How does a store receipt reach the order record: imported with its own identifiers, re-keyed, or not at all?
6. When a web order is returned at the till, how many systems can issue the refund?
7. Does the POS receive prices from the same source as the storefronts and the marketplaces?
8. If we replaced the platform in five years, would the tills have to change with it?

The store is where a web order can come back to the counter, and the return is where stock, money and the books have to agree again. The Way Back: Returns, Exchanges, Refunds and Reverse Logistics follows the jacket on its way back.

Sources and Evidence

Company, product and investor sources

- Shopify POS and Shopify POS pricing, Shopify, n.d., accessed 2026-11 — the platform-POS feature set (pickup in store, ship to customer, returns at any location, staff roles) and the €79 per location per month POS Pro price.
- Using Shopify POS offline and Offline payments, Shopify Help Center, n.d., accessed 2026-11 — what works offline, deferred capture, device limits and the 24-hour advice.
- Process offline payments, Square Support, n.d., accessed 2026-11 — the 72-hour expiry and the merchant's responsibility for offline payments.

- What is unified commerce?, Shopify (Roxanne Voidonicolas), 2026 — the vendor definition contrasted with omnichannel.
- Lightspeed Retail POS, Lightspeed, n.d., accessed 2026-11 — stock transfers between stores and customer profiles that follow shoppers between locations.
- Kassasysteem, Vendit, n.d., accessed 2026-11 — a Dutch POS that runs from the same database as the web shop (translated).
- Point of sale documentation, Adyen, n.d., accessed 2026-11 — in-person and online payments on one platform, the Terminal API and shopper recognition.
- Synchronize orders (Shopify connector), Microsoft Learn, 2026 — POS orders carrying a staff id into the ERP connector.
- Source algorithms and reservations, Adobe Commerce, 2026 — source priority and distance priority routing.
- FulfillmentOrder object, Shopify GraphQL Admin API, n.d., accessed 2026-11 — the fulfillment order and when its assigned location may change.
- Order routing and Fulfillment concept guide, Kibo Commerce, n.d., accessed 2026-11 — routing suggestions separate from stock deduction; transfer shipments for pickup orders.
- Zalando stellt Connected Retail ein, TextilWirtschaft, 2026 — the end of Connected Retail by June 2027; and Zalando baut Connected Retail aus, FashionUnited, 2021 — the 3,400 active stores figure (evidence age).

Standards, regulation and official documentation

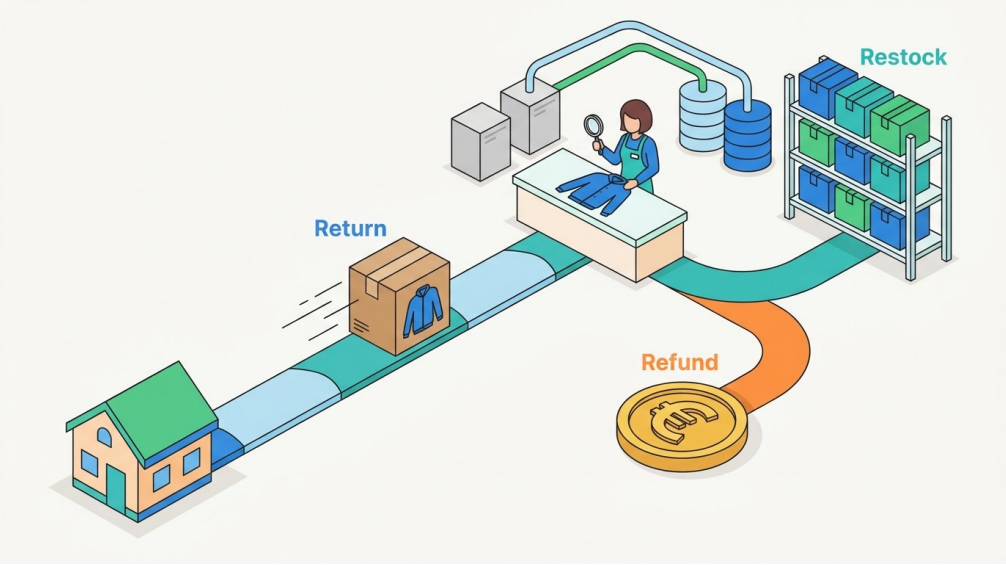
- § 146a Abgabenordnung, German Fiscal Code, current text, accessed 2026-11 — the certified technical security device, the receipt obligation and the registration duty.
- Kassensicherungsverordnung, ordinance of 26 September 2017, amended 2026 — transaction numbering, signature counter and receipt content.
- Mitteilungspflicht für elektronische Kassensysteme, Grant Thornton Germany, 2025 — the ELSTER registration procedure and the 31 July 2025 deadline.
- Geregistreerd kassasysteem and GKS 2.0 mandatory from 1 July 2026, FPS Finance (Belgium), 2026 — the hospitality-only scope, the €25,000 threshold and the GKS 2.0 dates (translated).
- Comunicato stampa: corrispettivi telematici, Agenzia delle Entrate, 2019 — the 2019–2020 start of telematic receipts in Italy (translated; evidence age for the start dates only).
- Het kassasysteem in uw bedrijf, Belastingdienst, n.d., accessed 2026-11 — seven-year retention and the voluntary quality mark (translated); Keurmerk Betrouwbare Afrekensystemen, n.d., accessed 2026-11.
- Point-to-Point Encryption solutions, PCI Security Standards Council, n.d., accessed 2026-11 — the validated P2PE solution listing.

Research and practitioner sources

- Inventory Record Inaccuracy: An Empirical Analysis, Nicole DeHoratius and Ananth Raman, *Management Science* 54(4), 2008 — 65% of 370,000 store inventory records inaccurate (evidence age).
- RFID can lead to 100% order accuracy, Auburn study finds, Retail Dive on the Auburn University RFID Lab's Project Zipper, 2018 — 69% of audited orders with barcode-data errors against under 0.01% with RFID (evidence age).
- UK retailers responding slowly to needs of online shoppers, says UPS, Post & Parcel on the UPS "Pulse of the Omni-channel Retailer" survey, 2016 — 27% of European retailers using stores as online fulfillment centers (evidence age).
- Click & collect sales and in-store online ordering, Savills Research, 2018 — click-and-collect and ship-from-store adoption in UK stores and the store managers who did not know how online refunds were booked (evidence age).

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

18. The Way Back: Returns, Exchanges, Refunds and Reverse Logistics



IN THIS CHAPTER, YOU WILL: Follow Eva's jacket back from Nijmegen to Duiven and learn how a return keeps the stock, the money and the books in agreement when one item in four comes back.

KEY POINTS:

- **A return is two records, not one.** The *return* (what comes back, why, and whether the customer gets a refund or an exchange) belongs to the order side; the *reverse fulfillment* (receiving, inspection, the decision what to do with the item) belongs to the warehouse. The warehouse's decision, not the customer's request, is the event that releases money and stock.
- **Money, stock and books move on different events, each exactly once.** The refund is issued by one system with the return id as its key; stock rises only on a "restocked" decision; the credit note follows the

refund. EU law sets the clocks: 14 days to withdraw, 14 days to refund, and the right to wait for the goods or proof of sending.
- **At a 24% item return rate the return path is a main road, not a side street.** Kestrel & Pine takes back 72,000 parcels a year; 28% of returns are exchanges, which are new orders held against a return; marketplace and store returns arrive as events the merchant did not start. Design for all of them, not for the happy path plus exceptions.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Boomerang** — refunds, restocking and credit notes are reconciled by hand after the return arrives, and sometimes twice, because no single event and no single writer were ever agreed.
- **Schrödinger's Order** — a customer asks "where is my refund?" and support checks the returns portal, the warehouse and the payment console, which disagree about whether the parcel has arrived.
- **The Phantom Stock** — a returned item is booked back into sellable stock before anyone has looked at it, or never booked back at all.

Every chapter so far has moved the order forward: a product shown, a price promised, money taken, a parcel picked and delivered. This chapter runs the film backward. The jacket comes out of the box, is tried on, does not fit, and has to travel back through most of the same systems in the opposite direction, with a refund where the payment was, a credit note where the invoice was, and a restock where the pick was. **The way back is harder than the way out**, because the merchant controls the timing of nothing: the customer decides when to send, the carrier decides when it arrives, and the law decides when the money must be back.

It is also busier than most newcomers expect. In apparel, one item in four or five comes back. That is not an exception path; it is a second logistics operation, with its own stock movements, its own money movements and its own legal clocks, and it touches the returns portal, the platform, the warehouse, the carrier, the payment provider and the books. When those systems disagree about what has happened to a return, the merchant pays twice, restocks what it cannot sell, or misses a legal deadline.

This chapter owns the terms **RMA**, **disposition**, **exchange** and **refund reconciliation**. It builds on the refund mechanics of Taking the Money: Payment Providers, Authorization, Capture and Refunds and the warehouse interface of Pick, Pack, Ship: Warehouse Management and Fulfillment, and hands the credit note to Tax, Invoices and the Books: From Order to Ledger, store returns to Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, marketplace returns to Selling on Someone Else's Platform: Marketplaces and Channel Integration and the customer's messages to Where Is My Order? Notifications, Service and the Customer Record. Journey 2: The Jacket Comes Back later walks the whole path step by step.

Where we are in the order's journey. Eva's jacket (the recurring order from Meet Kestrel & Pine) was delivered on Thursday 15 April 2027. This chapter is Monday 19 April, when Eva asks the returns portal to exchange the women's size M for size L, through Thursday 22 April, when the return is received and inspected at Duiven and the size L ships. The refund variant, a €199 transfer back by iDEAL (the Dutch bank-transfer payment method Eva paid with) and a credit note, is where the Double Refund lives.

Setting the Scene at Kestrel & Pine: One Item in Four

Kestrel & Pine, the fictional outdoor brand this book follows, shipped 336,000 parcels from Duiven in 2026 and took **72,000** back. Its **item return rate** on online sales, the share of shipped items that come back, was **24%**; its **order return rate**, the share of orders with at least one return, **19%**. The two rates differ because customers return part of an order, and because some order two sizes meaning to keep one, a habit the trade calls **bracketing**. **28%** of returns were exchanges, mostly size. **61%** of returned items were resold as new. The returns bench at Duiven, two people on a normal day, handles about **280** parcels; after Black Friday it is five people for three weeks (the daily figures are Kestrel & Pine illustrations).

Since 2025 a **returns portal**, a hosted software service (SaaS) costing €650 a month plus €0.95 per return, is the customer's way back: it checks that the order is eligible, asks for a reason, issues a prepaid PostNL, bpost or DHL label, offers an exchange or a refund, and, when the warehouse has inspected the item, triggers the refund through the platform's API. Kestrel & Pine's commercial policy is free returns within 14 days of delivery in all three countries, which is also the legal minimum for the withdrawal right explained below.

The portal was bought to fix a returns process run by e-mail. It did, and it created the Double Refund. In January 2027 **212 returns** were refunded twice, **€19,400** in total, most of it recovered. Taking the Money: Payment Providers, Authorization, Capture and Refunds told the story from the money side; here is the returns side. Customers could see on PostNL's tracking that their parcel had reached Duiven. Some wrote to support the same day asking for their refund. The agent could see the original payment in the console of the PSP (payment service provider: the company that takes the shopper's money and settles it to the merchant) and no refund, because the portal refunds on inspection, which took two to five days. The agent refunded by hand. Two to five days later the portal refunded again. Nothing in either request named the return, so the PSP accepted both.

When Noor Visser, Head of Engineering since January, reviewed the incident in February, she drew three columns on the whiteboard, money, stock and books, and

asked for each: **on which event does it move, and which system moves it?** Nobody could answer for all three. That is the chapter's question.

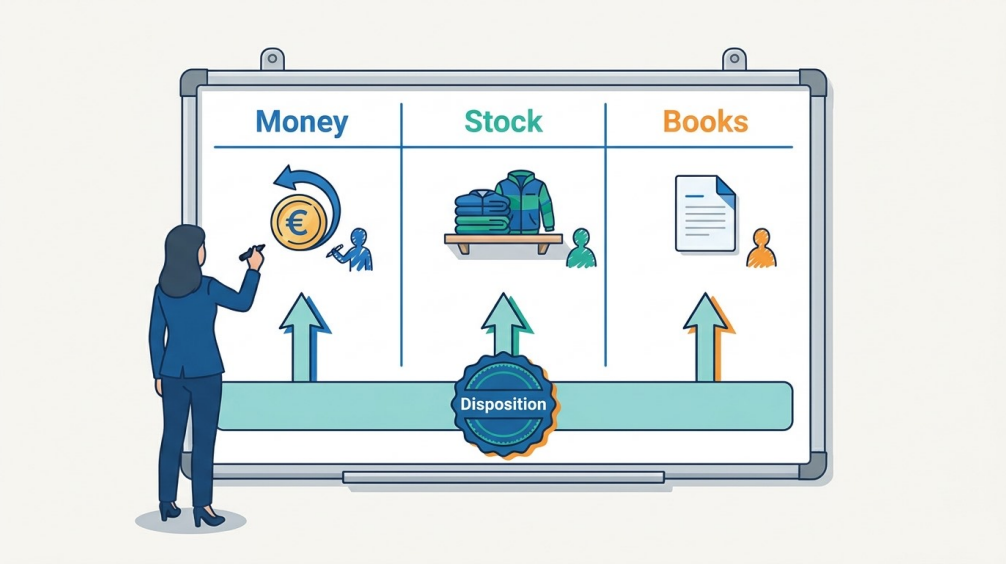


Figure 1: *Noor's question as a whiteboard: for money, stock and books, which event moves each and which one system moves it; the answer is the warehouse's disposition, releasing the refund, the restock and the credit note once each.*

What a Return Is, Step by Step

A return passes through nine steps, and each step has a record, an owner and a way to go wrong. The vocabulary is small and worth learning once.

Step	What happens, in plain words	Record	Who acts	What can go wrong
Request	The customer says which items are coming back and why	A **return request**, or **RMA** (return merchandise authorization: the merchant's record that this return of these items was approved)	The customer, in the portal or with an agent	Eligibility computed from a stale copy of the order
Eligibility	Is the item returnable, is it inside the window, who pays shipping	Rules evaluated against the order and the product	The portal or the order system	A sealed hygiene product accepted; a 15-day-old order refused that the law still covers
Label	A prepaid label or a drop-off code is issued	A return shipment with a tracking number	The portal via the carrier API	A label addressed to the old third-party warehouse in Tiel
Transit	The carrier scans the parcel on its way back	Carrier tracking events	The carrier	"Delivered to Duiven" read as "received"
Receiving	The warehouse opens the parcel against the RMA	A **reverse fulfillment** record: the work to process this return	The warehouse	A parcel with no RMA; an empty box
Inspection	Someone looks at the item and grades it	An inspection result per item	The warehouse	No check at all; a grade nobody can explain

Step	What happens, in plain words	Record	Who acts	What can go wrong
Disposition	The decision what happens to the item: back to stock, needs processing, discard, missing	A **disposition** per item	The warehouse	Restocked but not resellable; restocked to the wrong location
Settlement	Refund, exchange or store credit, and the credit note	A refund with the return id; an exchange order; a credit note	One system for money; the ERP (enterprise resource planning system, the books)	Two writers; a refund before receipt; the wrong amount after a promotion
Stock	The resellable unit becomes available again	A stock event from the WMS (warehouse manage-ment system)	The WMS to the availability service	An increment on request instead of on disposition

Three terms carry the chapter. The **RMA** is the merchant's side of the promise "you may send this back". The **disposition** is the warehouse's verdict, and it is the most important event in the whole flow, because it is the first moment the merchant knows what it actually got back. An **exchange** is a return paired with a new order for a different size, color or item, with any price difference settled as a balance. **Reverse logistics** is the trade's name for the whole backward operation: labels, carriers, receiving, grading, resale, recycling.

Return rates as an economic fact

The rate decides how much of this deserves engineering. The US National Retail Federation (NRF) with Happy Returns put US retail returns at **$890 billion** for 2024 (December 2024) and projected **$849.9 billion** for 2025, with an estimated online return

rate of **19.3%** of online sales and **9%** of all returns judged fraudulent (October 2025); its 2024 survey found that **76%** of consumers "consider free returns a key factor in deciding where to shop". Apparel is higher: Coresight's survey of 100 US apparel decision-makers (April 2023, a small survey and now three years old) estimated a **24.4%** return rate on online apparel orders, with size and fit the reason in **53%** of cases. Kestrel & Pine's 24% is apparel-typical. Definitions matter: NRF's rates are shares of sales value across all US retail; Coresight's is a share of online apparel orders; Kestrel & Pine quotes items and orders separately. Quote the basis with every number.

What a return costs is mostly invisible in the order system: the label, the handling at the bench, the grading, the repackaging, the markdown on the 39% not resold as new, and the capital tied up in a jacket that spends ten days in transit and inspection. Policy is the lever that moves the rate: better size guidance and photos, charging for returns on some items, and steering toward exchanges. All three need data the return record must capture: the reason, the size ordered against the size kept, the disposition.

How the Pieces Fit Together

Seen from outside, a return is a loop through six systems. The figure shows the target design at Kestrel & Pine after Waypoint, with the OMS (order management system, the one record of every channel's orders that Waypoint introduces in Q4 2027) as the order record; in April 2027 the platform still plays that role.

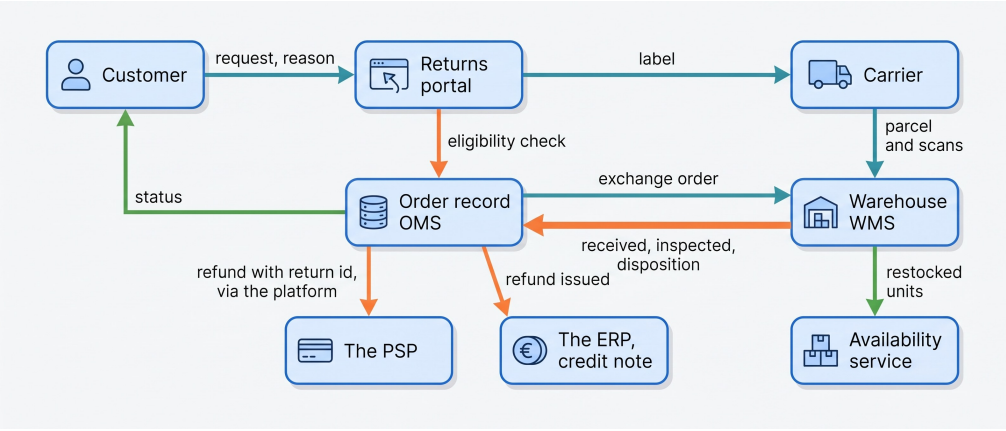


Figure 2: *A return loops through six systems; the warehouse's disposition is the one event that releases the refund, the restock and the credit note, and the order record is the only system that may ask for a refund.*

Two records: the return and the reverse fulfillment

The clearest public model of this split is Shopify's developer documentation (n.d., accessed November 2026). A **return** "represents the intent of a buyer to ship one or more items from an order back to a merchant"; it moves through `REQUESTED`, `OPEN`, `DECLINED`, `CLOSED` ("when all items have been processed and a restock decision has been made") and `CANCELED`. Separately, a **reverse fulfillment order** "represents the work that's required to process a return", and a **reverse delivery** is "a set of items to be packaged together and sent back to the merchant". The dispositions on the reverse fulfillment order are `RESTOCKED`, `PROCESSING_REQUIRED` ("An item that requires further processing before being restocked or discarded"), `NOT_RESTOCKED` and `MISSING` ("An item that was expected but absent"). commercetools' order API (n.d., accessed November 2026) keeps the same two concerns as two states per return item: a shipment state that starts at `Advised` or `Returned` and may move to `BackInStock` or `Unusable`, and a payment state that starts at `NonRefundable` or `Initial` and moves to `Refunded` or `NotRefunded`.

The split is the pattern to teach. The first record is about intent and money and belongs to the order side; the second is about work and stock and belongs to the warehouse. They meet at the disposition.

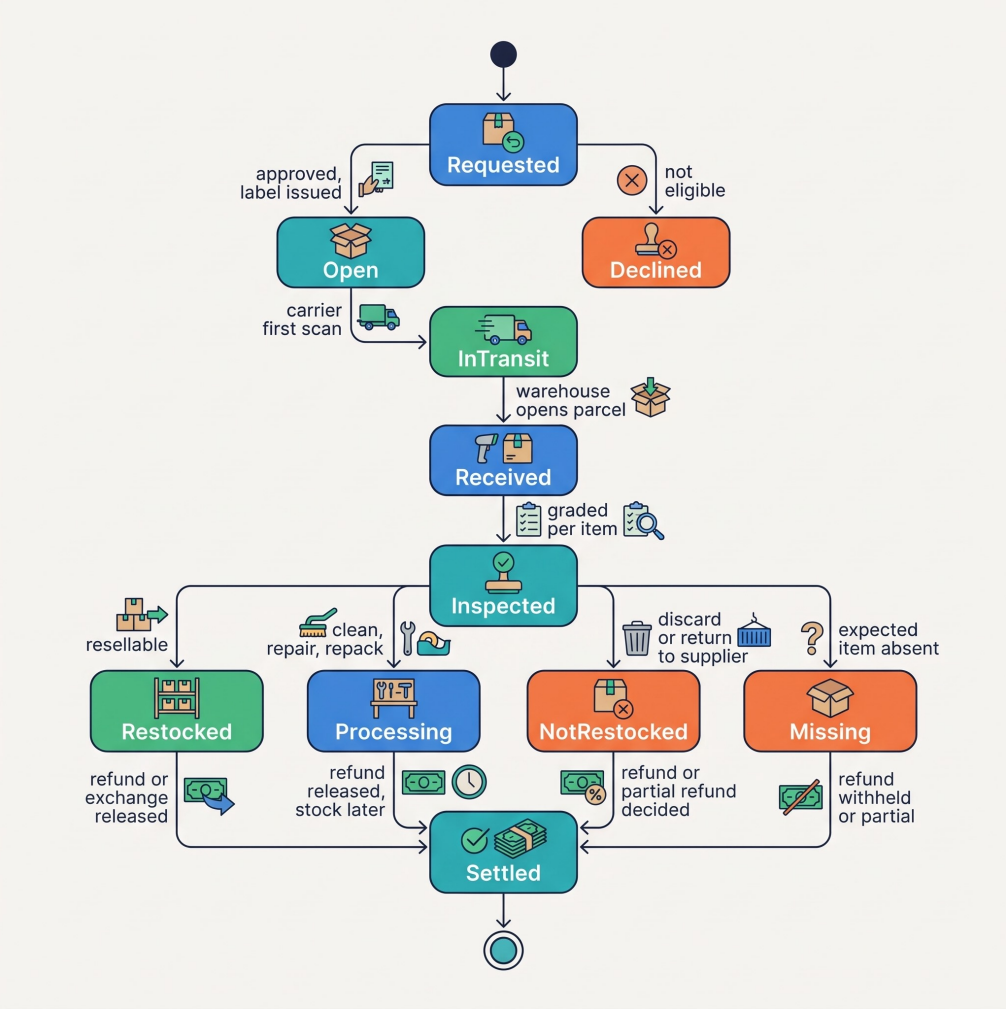


Figure 3: *The return's lifecycle: the request and the label are the customer's side, receiving and inspection are the warehouse's, and settlement waits for a disposition.*

Requirements per flow

Flow	How fresh	How consistent	How fast	If the other side is down
Eligibility check (portal reads the order)	The order as it is now, including earlier returns on the same line	Must see the delivery date and the delivery option chosen	Seconds, the customer is waiting	Refuse politely and retry; never guess
Received and disposition (WMS to order record)	The moment the bench scans the RMA	One event per return item, idempotent by return id and line	Minutes *(assumption)*	Queue; the refund waits with it
Refund (order record to PSP)	On disposition, inside the legal 14 days	Exactly one per return, keyed by return id	Hours *(assumption)*	Queue; check the PSP's refund list before any retry
Restock (WMS to availability)	On a restocked disposition only	Never on request or carrier scan	Minutes *(assumption)*	A weekly reconciliation catches a lost event
Credit note (order record to ERP)	After the refund is confirmed	One credit note per refund, at the allocated line price	Daily batch is enough	Finance waits
Status to the customer	On each state change	The same state the agent sees	Minutes	The status page shows the last known state

The Law: Fourteen Days, Three Times Over

For a consumer buying at a distance in the EU, the right to return is not the merchant's policy; it is law, and the architecture has to compute its clocks. The **Consumer Rights Directive**, Directive 2011/83/EU of 25 October 2011, published in the Official Journal on 22 November 2011, applies to contracts concluded from **13 June 2014**. Its text, as consolidated by the Publications Office, says:

- **Article 9(1):** the consumer "shall have a period of 14 days to withdraw from a distance or off-premises contract, without giving any reason, and without incurring any costs other than those provided for in Article 13(2) and Article 14". For goods the period runs from the day the consumer "acquires physical possession of the goods", and for several goods delivered separately, from possession "of the last good" (Article 9(2)(b)).
- **Article 10:** if the trader did not inform the consumer of the right, the period extends to twelve months.
- **Article 13(1):** the trader "shall reimburse all payments received from the consumer, including, if applicable, the costs of delivery without undue delay and in any event not later than 14 days from the day on which he is informed of the consumer's decision to withdraw", "using the same means of payment as the consumer used". **Article 13(2):** the trader need not refund extra costs "if the consumer has expressly opted for a type of delivery other than the least expensive type of standard delivery". **Article 13(3):** the trader "may withhold the reimbursement until he has received the goods back, or until the consumer has supplied evidence of having sent back the goods, whichever is the earliest".
- **Article 14(1):** the consumer must send the goods back "not later than 14 days from the day on which he has communicated his decision to withdraw" and "shall only bear the direct cost of returning the goods unless the trader has agreed to bear them or the trader failed to inform the consumer that the consumer has to bear them". **Article 14(2):** the consumer is liable only for diminished value "resulting from the handling of the goods other than what is necessary to establish the nature, characteristics and functioning of the goods".
- **Article 6(1)(h) and (i):** before the purchase, the trader must state the conditions and model form for withdrawal and that the consumer bears the return cost where that applies.
- **Article 16:** no withdrawal right for, among others, goods "made to the consumer's specifications or clearly personalised", goods "liable to deteriorate or expire rapidly", and "sealed goods which are not suitable for return due to health protection or hygiene reasons and were unsealed after delivery".

Read as requirements, the directive asks for four dates per order line: the delivery date (possession), the withdrawal notice date, the goods-sent evidence date and the refund date; the three 14-day clocks run from the first three. It asks the order to keep the delivery option chosen at checkout and its standard-rate equivalent, because the refund amount depends on them. It asks for product attributes (personalized, perishable, sealed hygiene) that eligibility rules can read, which is product-information work (What Are We Selling? Products, Variants and the Catalog). And it makes Article 13(3) the legal basis for Kestrel & Pine's "refund on inspection" policy: the trader may wait for the goods, and then has the balance of its 14 days from the notice to pay. The wider regulatory calendar, including the price-reduction rules that decide what a refunded promotion is worth,

is in Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.

The Choices

Four decisions shape a returns architecture; each has a few realistic answers.

1. **Who runs the customer's side**. *Manual*: an e-mail address and a refund button in the platform's admin; the Starting-stage answer. *The platform's own returns*: the platform models the return and the reverse fulfillment, as Shopify does, and may issue labels. *A returns portal*: a specialist SaaS product. Loop (n.d., accessed November 2026) was "Originally built for Shopify", steers shoppers "toward exchanges or store credit instead of refunds" and claims "5,000+ brands"; Narvar sells returns beside tracking and delivery-date promises and claims "1,500+ global brands"; Returnista, from Amsterdam, offers labels, exchanges "for the right size", store credit, rules-based automation and an API with webhooks for "1,000+ brands in Europe"; Happy Returns, "a UPS company", runs "10,000 Return Bar locations" for box-free drop-off and consolidates returns before they reach the merchant. Multi-carrier shipping platforms bundle the same portal with their labels. *The OMS*: order management products such as Manhattan and Fluent list returns orchestration and cross-channel returns as modules (The Order Record: Lifecycle, State and the Order Management System). The vendor claims are undated and will change.

2. **Which event releases the refund**. This is the decision that caused the Double Refund, and it is a policy choice with a legal floor.

Trigger	Customer waits	Merchant's risk	Legal position (Directive 2011/83/EU)	Fits
On request	Not at all	Pays for items that never come back or come back worn	Allowed; the merchant gives up Article 13(3)	Low-value goods; trusted repeat customers
On carrier scan (first scan or "delivered to warehouse")	A day or two	Empty boxes, wrong items, worn goods; the 9% fraud share	Allowed; "evidence of having sent back"	Fast-fashion scale with fraud scoring

Trigger	Customer waits	Merchant's risk	Legal position (Directive 2011/83/EU)	Fits
On receipt	Transit plus a day	Wrong or damaged items refunded before anyone looks	Allowed; goods received	Standardized goods with little grading
On inspection	Transit plus grading, two to five days at Duiven	Lowest; the refund matches the disposition	Allowed; the merchant then has the balance of 14 days from the notice to pay	Apparel and equipment, Kestrel & Pine's choice

3. **Who issues the refund**. The portal through the platform's API (Kestrel & Pine today), the OMS through the same platform refund API once it holds the return records (after Waypoint, Q4 2027), or the ERP as part of the credit note. Any of the three works; **two of them at once is the Boomerang**. Taking the Money: Payment Providers, Authorization, Capture and Refunds gave the money-side rules: one writer, the return id as idempotency key, and a daily comparison of the PSP's refunds with the platform's.

4. **How exchanges work**. *Exchange as a held order*: a new order for the replacement is created at request time, reserving stock, and ships when the return's disposition is resellable; Kestrel & Pine's model, and the right one where the method cannot hold money, as iDEAL cannot. *Instant exchange*: the replacement ships immediately and the merchant holds the customer's card for the original price until the return arrives, which needs a card, an authorization that survives the transit window and a merchant-initiated transaction if it does not (Taking the Money: Payment Providers, Authorization, Capture and Refunds). *Exchange with a balance*: a different item at a different price, settled as a partial refund or an extra charge. Shopify's model lets the merchant "Release the exchange and collect a balance". Loop's and Returnista's pitch, steering to exchanges and store credit, reflects the economics: an exchange keeps the revenue and costs one more parcel.

Choice	Manual	Platform's own returns	Returns portal	OMS returns module
Who computes eligibility	A person	The platform, from its order	The portal, from a copy of the order	The OMS, from the order record
Labels and carriers	By hand	Platform's carriers	Many carriers, drop-off points	Via the shipping service
Exchanges	By hand	Basic	Strong, with balances and store credit	Strong, cross-channel
Who may refund	A person in the admin	The platform	The portal through the platform's API	The OMS
Marketplace and store returns	By hand	Mostly outside	Some connectors	Designed for it
Fits	Starting	Starting to Growing on one platform	Growing to Omnichannel	Omnichannel to Group

My rule: **choose the refund trigger and the refund writer before choosing the product**, write both into the integration contract, and let the warehouse's disposition be the trigger unless the goods are cheap enough that fraud costs less than waiting. At Kestrel & Pine's rate and prices, inspection is the only defensible trigger, and the €199 jacket that comes back worn is the reason.

Returns From Marketplaces and Stores

A return the merchant did not start still has to reconcile. On Amazon, for seller-fulfilled orders, Amazon's seller guidance (April 2025) says most items "can be returned within 30 days of the order's 'estimated delivery date' "; US return requests covered by Amazon's policy are authorized automatically and prepaid labels are issued through Amazon's shipping; sellers should "respond to requests within 24 hours" when their authorization is needed and issue refunds "within two days of receiving returns, or Amazon may issue an automatic refund"; a claim under the A-to-z Guarantee can be charged to the seller. For orders fulfilled by the marketplace's own program, the marketplace receives and

grades the return and charges the seller a returns-processing fee; Zalando's logistics arm ZEOS described "around 20 returns centres" in its November 2024 announcement. The architectural consequence: **the merchant's return record must accept RMAs it did not create and refunds it did not issue**, keyed by the marketplace's return id, and the 24-hour and two-day clocks are service levels on the integration, not on people (Selling on Someone Else's Platform: Marketplaces and Channel Integration).

Store returns are the mirror case. A web order brought to the Nijmegen counter is returned at the till: the till must find the online order, accept the item, record the return and restock to the store or send it to Duiven, and the portal's request, if one exists, must be closed before anything is paid. Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment sets that rule; until the POS (the stores' point-of-sale system) and the platform are linked, Kestrel & Pine's stores refund web orders only after the portal request is closed by a support agent, by hand *(illustrative)*.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Signs the stage is ending
1 — Starting (under about €2M)	Returns by e-mail; a refund button in the platform; the third-party warehouse or the back room restocks by hand; refund on receipt	The withdrawal information in the terms and on the product page (Article 6); one person who owns refunds	Refunds take longer than the legal 14 days in busy weeks; "where is my refund" is the top ticket
2 — Growing (about €2M–€20M)	A returns portal with labels and reasons; refund on inspection; exchanges as held orders; the ERP books credit notes from an export	The refund writer and key decided once; "received" and "disposition" as events from the warehouse; a return-aging report against the 14-day clock	Two systems can refund (the Double Refund); returns restocked without grading; the first marketplace return with no record

Stage	What is typical	What becomes necessary	Signs the stage is ending
3 — Omnichannel (about €20M–€200M)	Returns from storefronts, marketplaces and stores into one record; dispositions feeding availability; returns analytics by reason, size and product; a returns bench that scales for peak	An order hub that accepts external RMAs and external refunds; one refund path per channel; a daily refund reconciliation; returns in the Black Friday plan	Support checks three systems for one return; marketplace refunds counted twice in the books; the bench backed up for weeks in December
4 — Group (over about €200M, or several brands)	Shared reverse logistics (returns centers, consolidation, resale channels) with dispositions per brand; return policy as a per-brand setting on shared infrastructure; return fraud scoring	A returns service with per-brand rules; cross-brand customer return history (One Customer, One Product: Identity and Master Data Across Systems); negotiated carrier return networks	Each brand running its own portal and carrier contracts; dispositions defined differently per warehouse

The judgement: the return's *shape* (request, transit, inspection, disposition, settlement) does not change with scale, and neither does the law. What changes is the number of channels a return can come from and the number of systems that could act on it. Stage 2 is where the refund trigger and writer must be fixed, because it is the stage at which a portal, a warehouse and an ERP first all hold a copy of the return.

In Practice

Public examples. Shopify's return and reverse-fulfillment model, with its four dispositions, is the most complete public description of the two-record pattern (n.d., accessed November 2026). commercetools' per-item shipment and payment states show the same

separation in a headless platform. Amazon's seller rules (April 2025) show a marketplace owning the policy, the authorization and sometimes the refund. NRF's 2024 and 2025 reports put numbers on why it matters: free returns as a deciding factor for 76% of consumers, and 9% of returns fraudulent.

Kestrel & Pine: Eva's exchange, 19 to 22 April 2027. The table says what each system knew and did; the days are canonical, the hours are illustrations.

When	Event	System	Record
Mon 19 Apr, 20:40	Eva opens the portal from her order e-mail, selects the jacket, reason "too small", exchange for size L	The returns portal	Return R-20270419-1, status requested, 4 days after delivery, inside the 14-day window
Mon 19 Apr, 20:41	The portal checks eligibility against order #1072455 (delivered 15 April, standard delivery, apparel, no earlier return on the line) and issues a prepaid PostNL label	The portal, the platform	Return open; a return shipment with a tracking number
Mon 19 Apr, 20:41	The portal asks the platform to create the exchange order for size L, held until the return is graded	The platform	Order #1072455-X1 for one jacket, women's L, moss, €0 balance; one unit of L committed
Tue 20 Apr	Eva drops the parcel at a PostNL point; first scan	PostNL, the shipping service	Return in transit

When	Event	System	Record
Thu 22 Apr, 10:15	The bench scans the RMA, opens the parcel, inspects the jacket	The WMS	Reverse fulfillment received; inspected; disposition restocked
Thu 22 Apr, 10:16	The WMS reports the disposition; the platform closes the return and releases the held order	The WMS, the integration platform, the platform	Return closed; exchange order released; stock of M moss plus one in the WMS
Thu 22 Apr, 13:00 wave	Size L is picked, packed and labeled	The WMS	Exchange shipped; Eva's shipping confirmation
Night of 22 Apr	The ERP receives the return, the credit note for the size M invoice line and the new invoice for size L, netting to zero	The ERP	Books agree (Tax, Invoices and the Books: From Order to Ledger)

The refund variant. Had Eva chosen a refund, step three would not exist. On the disposition at 10:16 on 22 April the portal would have asked the platform to refund €199 with return id R-20270419-1 as the idempotency key; the platform would have instructed the PSP; the PSP would have sent a new iDEAL transfer to Eva's account and reported it by webhook; the ERP would have booked a €199 credit note. Eva's notice was 19 April, so the legal deadline was 3 May; the refund went out on day three. Had support, on 21 April, seen "delivered to Duiven" on the tracking and refunded in the PSP console, that would have been the Double Refund; since February 2027 the console is read-only for support and an agent opens the return in the portal instead, where the pending refund and its trigger are visible.

The amount. A refund is the price the customer paid for that line, after the discounts that applied to it. The promotions chapter records what happens otherwise: in November 2026, 190 returned jackets were refunded at €159.20 instead of €143.28 because the order-level discount had not been allocated to lines in the export. Since Waypoint's W5 the order export to the returns portal, the WMS and the ERP carries the allocated net price per line (Discounts, Coupons and Loyalty: Promotions Without Chaos).

When It Goes Wrong

The double refund. Two writers, no key. Detection: the daily comparison of the PSP's refunds with the platform's by order and amount; two for one return is a hit. Recovery: ask for the money back; block the second writer. Prevention: one writer, the return id as key, read-only console access, and a portal the agent can see.

Figure 4: *The Double Refund in one picture: two writers and no key let the payment provider accept two refunds for one return; one writer using the return id as its key lets it accept exactly one.*

The refund without a return. Refund on request or on scan, and the parcel never arrives or holds a different item. Detection: refunds older than N days with no "received" event; dispositions of `MISSING` or `NOT_RESTOCKED` against refunded returns. Recovery: a customer conversation and, at scale, a fraud score per customer. Prevention: refund on inspection where the goods are worth it.

The phantom restock. Stock is incremented on the request, or on the carrier scan, or on receipt before grading, and a worn jacket is sold again. Detection: the weekly WMS-to-availability reconciliation; complaints about "new" items. Prevention: stock moves only on a `RESTOCKED` disposition from the WMS, which is the single writer of on-hand stock (Is It in Stock? Inventory, Availability and the Promise You Can Keep, Waypoint W1).

The return with no record. A parcel arrives at Duiven with no RMA: a marketplace-authorized return the connector did not import, or a customer who used an old label. Detection: a "no RMA" queue at the bench with a daily count. Recovery: match

by tracking number or order number, create the RMA after the fact, settle under the channel's rules. Prevention: every marketplace return event acknowledged into the order record with the marketplace's return id (Selling on Someone Else's Platform: Marketplaces and Channel Integration).

The exchange that ships before the return exists. An instant exchange, or a held order released by a "delivered to warehouse" scan rather than a disposition; the return never comes. Detection: exchange orders shipped with the paired return not closed after N days. Prevention: release on disposition; for instant exchanges, hold the card and know the window.

The missed deadline. The refund is issued on day sixteen because the bench was three weeks behind in December. Detection: a return-aging report against the Article 13 clock, by notice date. Recovery: refund now, apologize, and staff the bench for peak. Prevention: returns in the Black Friday plan (Journey 4: Black Friday, Hour by Hour).

The three-system status. The customer asks "where is my refund?"; the portal says open, the WMS says received, the PSP says nothing. Detection and recovery: one status, built from the same events, shown to the customer and the agent alike (Where Is My Order? Notifications, Service and the Customer Record).

What to Take Away

Established.

- A return is two records: the return (intent and money) and the reverse fulfillment (work and stock), with the disposition as the event that joins them (Shopify's and commercetools' public data models, n.d., accessed November 2026).
- EU consumers may withdraw within 14 days of receiving the goods, the trader must refund within 14 days of the notice by the same means, and may withhold the refund until the goods or proof of sending arrive (Directive 2011/83/EU, Articles 9, 13 and 14, applicable since 13 June 2014).
- Return rates in apparel run around a quarter of items, free returns drive where consumers shop, and a measurable share of returns is fraudulent (NRF 2024 and 2025; Coresight 2023), so the return path is a first-class flow.
- A marketplace owns the return policy, the authorization and sometimes the refund for orders sold through it, with clocks the seller's integration must meet (Amazon seller guidance, April 2025).

Depends on context.

- Refund on inspection is right for apparel and equipment at Kestrel & Pine's prices; refund on carrier scan is right for low-value goods at high volume with fraud scoring; refund on request is right only where trust is worth more than the goods.

- A returns portal is right from the Growing stage when the platform's own returns lack labels, exchanges or carriers; an OMS returns module is right once returns come from three channels and the OMS is the order record.
- Exchanges as held orders are right where the payment method cannot hold money or the goods are expensive; instant exchanges are right where cards dominate and a slow exchange loses the sale.

The author's synthesis.

- I decide three things before buying any returns product: which event releases the money, which single system may issue the refund, and that stock rises only on a restocked disposition; the product is then a matter of labels, carriers and screens.
- I treat the 14-day clocks as data requirements, not as legal footnotes: four dates per line, the delivery option kept, and an aging report that someone reads every week.

Questions to Ask

1. On which event does the refund go out: request, carrier scan, receipt or inspection, and who decided it?
2. Which single system may ask the PSP for a refund, and what key does the request carry?
3. What does the warehouse send back when a return arrives, and does the restock wait for a disposition?
4. Which four dates does the order keep per line, and can the system compute the 14-day clocks from them?
5. Does the eligibility rule know the delivery option chosen and the product's exemption attributes?
6. How does a marketplace-authorized return, or a store return of a web order, enter the return record?
7. How is an exchange modeled, and what releases the replacement?
8. What is the refund amount on a line that carried an order-level discount?
9. How many returns are older than ten days without a refund, and who sees that list?

The item is back on the shelf, the money is back in Eva's account or a size L is on its way, and the books agree. What Eva saw of all this was a few messages: your return is registered, your parcel is on its way, we have received it, your exchange has shipped. Those messages, the status page behind them and the agent who answers when they are not enough are the subject of Where Is My Order? Notifications, Service and the Customer Record.

Sources and Evidence

Company, product and investor sources

- Build returns apps and ReverseFulfillmentOrderDispositionType, Shopify developer documentation, n.d., accessed 2026-11 — the return, reverse fulfillment order and reverse delivery model, return statuses, dispositions, exchanges with a balance.
- Orders, commercetools API documentation, n.d., accessed 2026-11 — return item shipment and payment states.
- Manage customer returns, Amazon, 2025-04-15 — seller-fulfilled return windows, automatic authorization, prepaid labels, response and refund clocks, A-to-z Guarantee.
- ZEOS becomes fulfilment partner of choice for British retailer NEXT, Zalando, 2024-11-21 — returns centres in a marketplace-run logistics network.
- Loop Returns, Narvar, Returnista and Happy Returns, vendor sites, n.d., accessed 2026-11 — returns-portal features and customer claims (volatile; marketing sources).

Standards, regulation and official documentation

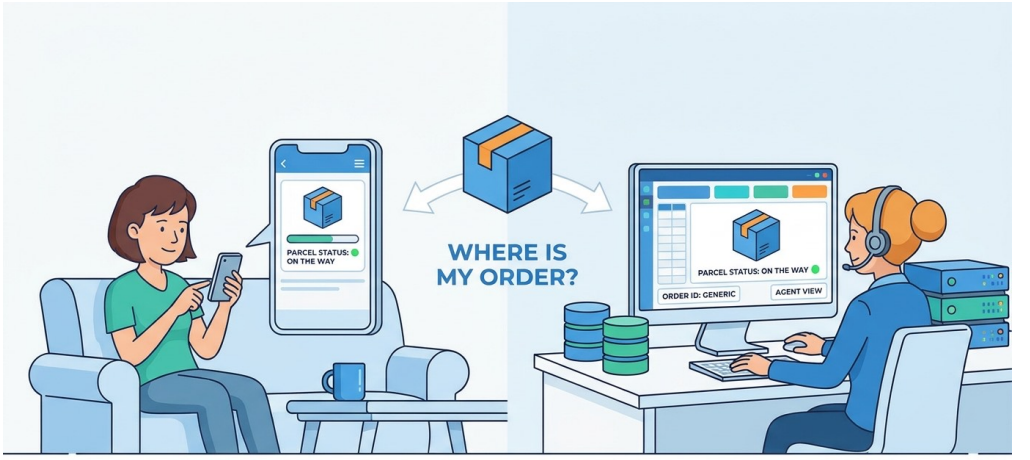
- Directive 2011/83/EU on consumer rights, Official Journal, 2011-11-22 — the right of withdrawal (Article 9), the information duty (Article 6), the twelve-month extension (Article 10), reimbursement (Article 13), the consumer's obligations (Article 14) and the exceptions (Article 16); applicable to contracts concluded from 13 June 2014.

Research and practitioner sources

- 2023 Consumer Returns in the Retail Industry, NRF and Appriss Retail, 2023-12-22; 2024 Consumer Returns in the Retail Industry, NRF and Happy Returns, 2024-12-05; 2025 Retail Returns Landscape, NRF and Happy Returns, 2025-10-15 — US return volumes, online return rates, free returns as a deciding factor, the fraudulent share.
- The True Cost of Apparel Returns, Coresight Research, 2023-04-04 — the online apparel return rate and the size-and-fit reason (small survey; evidence age).
- Payments for Finance Architects: Eleven Reasons the Numbers Disagree in RECONCILED — refunds as new payments and the reconciliation that follows; Scams, Fakes and Account Takeovers: Fraud Prevention for Listings in LISTED — scoring returns abuse at scale.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

19. Where Is My Order? Notifications, Service and the Customer Record



IN THIS CHAPTER, YOU WILL: Learn how order events become notifications, a status page and an agent's view, why the three must share one status, and how to read service load as a signal.

KEY POINTS:

- **Notifications, the status page and the agent's screen are three views of one status, and they must be built from the same events.** The platform emits "confirmed", "canceled" and "refunded"; the warehouse emits "shipped" with a tracking number; the carrier emits "out for delivery" and "delivered". When each view reads a different system, the customer and the agent see different orders.
- **An order has several status machines, not one.** Money, goods and returns each move on their own; a single "status" string collapses them

and loses what the customer is asking about. Kestrel & Pine's Three-Status Order was a two-parcel order whose first parcel turned the whole order "Delivered".

- **"Where is my order" is a measurement, not a cost of doing business**. The one vendor dataset that states its basis puts it at about a fifth of tickets; measured per shipped order, it rises when tracking events go missing, promises slip or shipments split without explanation. Self-service that is gated on the fulfillment state removes the contacts that need no judgement.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **Schrödinger's Order** — the customer, the status page and the agent each read a different system and get a different answer.
- **The Message in a Bottle** — a shipping notification sent once, from a label printed rather than a parcel handed over, is trusted by the customer and wrong.
- **The Familiar Stranger** — the agent searches by e-mail and finds the online account, not the store purchases, the newsletter consent or the open return.

After the order is placed, the customer's relationship with the merchant is a sequence of messages: your order is confirmed; it has shipped; it is out for delivery; it was delivered; your return was received; your refund is on its way. Between those messages sits a page the customer can open at any time, and behind both sits a person the customer can write to when the messages stop making sense. **Notifications, the order status page and the service agent are the merchant's voice after the sale**, and the architecture decides whether that voice is one or three.

The business problem is simple to state. Every message is triggered by an event, and the events come from different systems: the platform knows the order was placed and paid, the warehouse knows it was packed, the carrier knows it was delivered, the returns portal knows it came back. If the status page reads the platform, the notifications read the warehouse and the agent reads all of them in separate tabs, the customer sees "Delivered", receives "Shipped" and is told "In picking" by the person meant to help. **Every contradiction is a ticket, and a ticket costs more than the margin on a €92 order**, Kestrel & Pine's average.

This chapter owns the **transactional notification**, the **order status page** and the **service view**. It builds on the order record and its state machines from The Order Record: Lifecycle, State and the Order Management System, tracking events from Getting It There: Carriers, Labels, Tracking and the Delivery Promise and the return record from The Way Back: Returns, Exchanges, Refunds and Reverse Logistics. Who the customer is, across the online account, the store's loyalty list and the newsletter, is One Customer,

One Product: Identity and Master Data Across Systems's subject; this chapter only shows what the agent cannot see without it.

Where we are in the order's journey. This chapter covers the messages Eva Janssen receives about her jacket (the recurring order from Meet Kestrel & Pine) from the confirmation at 21:14 on 13 April 2027 to the exchange shipped on 22 April, the page she opens in between, and what an agent sees if she writes in. It is the last chapter of Part IV, and the one that looks at the whole journey from the customer's side.

Setting the Scene at Kestrel & Pine: Eleven Tickets, Six Days, One Order

Kestrel & Pine, the fictional outdoor brand this book follows, had an order in December 2026 that this book calls **the Three-Status Order**: order **#1052877** showed *Delivered* in the platform, *In picking* in the WMS (the warehouse management system that runs picking, packing and shipping at Duiven) and *Not invoiced* in the ERP (the enterprise resource planning system that keeps the books and issues invoices) for **six days**, and one customer opened **11 support tickets** about it. The Order Record: Lifecycle, State and the Order Management System explains why the three systems could hold three states. This chapter follows the tickets, because they show what the customer and the agent each could and could not see.

The order was a two-line order: a Trailhead fleece (€119) and a pair of merino hiking socks (€18), €137 in total. The details below are the chapter's reconstruction, and later chapters reuse them. The fleece was picked and shipped on Monday 7 December; the socks were short in the pick location and waited on replenishment from bulk stock. The agency-built flow in the iPaaS (the integration platform that carries orders between systems) had no notion of a partial shipment: on the first shipment confirmation from the WMS it marked the whole platform order fulfilled and attached the fleece's tracking number. PostNL delivered the fleece on Tuesday 8 December, the platform's status page turned the order to **Delivered**, and the "delivered" e-mail went out. The WMS, which tracks the order as a whole until every line ships, still showed **In picking**. The ERP invoices on an "order complete" message from the WMS that never came, so it showed **Not invoiced**. Nobody told the customer there were two parcels.

Day	Channel	What the customer asked	What the agent could see	What the agent did
Tue 8 Dec, evening	E-mail	"Delivered, but the socks are not in the box."	Platform: Delivered, one tracking number	Asked for a photo of the packing slip
Wed 9 Dec	E-mail	Photo sent; "where are my socks?"	Same	Checked PostNL tracking: delivered; replied that it was delivered
Wed 9 Dec	Chat	"I know it was delivered, the socks were not in it."	Same	Opened a "missing item" case; said someone would check with the warehouse
Thu 10 Dec	E-mail	"Any news?"	Same; the warehouse question sat in an inbox	Replied "still checking"
Thu 10 Dec	Phone	Asked for a refund of the socks	Platform: paid €137; refund possible	Offered the refund; the customer said she wanted the socks
Fri 11 Dec	E-mail	"Status still says Delivered. Is this a scam?"	A WMS reply by e-mail: order "In picking", socks on re-plenishment	Explained that the socks would ship; could not say when
Fri 11 Dec	Chat	"Why does the page say Delivered if it is being picked?"	Both answers, in two tabs	Apologized; could not change the status page

Day	Channel	What the customer asked	What the agent could see	What the agent did
Sat 12 Dec	E-mail	Asked whether she had been charged for both	Platform: paid €137; ERP not visible to support	Asked finance by e-mail on Monday
Sun 13 Dec	E-mail	Asked again	Nothing new	No reply until Monday
Mon 14 Dec	Phone	"Just tell me one thing that is true."	Socks shipped that morning; new tracking number	Gave the second tracking number by phone
Mon 14 Dec	E-mail	Asked for the tracking number in writing	Same	Sent it, with a €10 voucher *(local detail)*

Eleven contacts, three channels, three people answering, and one piece of information that would have ended it on day one: "**your order ships in two parcels**; **the second is on its way on the 14th**." Nobody had it, because no system held it. The agent worked from what the support team calls **the five tabs**: the platform's order screen, the WMS, the shipping service's tracking page, the PSP console (the payment service provider's portal, where refunds can be issued by hand) and the returns portal. The ERP was a sixth, visible only to finance. Waypoint's answer (**W2**, one order record in a bought order management system, or OMS, by Q4 2027, and **W8**, order status by API) is the subject of the rest of this chapter; the tickets are its justification.

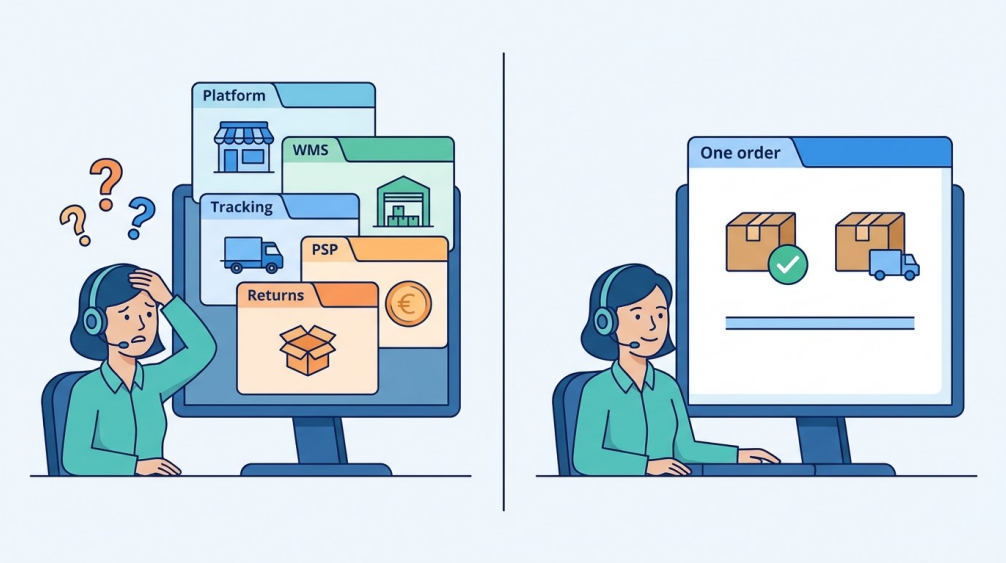


Figure 1: *The five tabs: each system told the agent a different truth about one order, so the message that would have ended the eleven tickets, two parcels, existed on no screen; a service view over one order record shows it.*

What the Customer Needs to Know, and Which System Knows It

A **transactional notification** is a message triggered by an event on the customer's own order, as opposed to marketing: confirmed, shipped, delivered, refunded. The list of notifications a merchant sends is, read the other way, a list of events some system must emit. Shopify's help page on customer notifications (n.d., accessed November 2026) is a usable inventory: order confirmation (which cannot be deactivated), shipping confirmation at fulfillment, shipping update "when tracking information is updated", out for delivery and delivered, each triggered "when a carrier or fulfillment app reports" the event, order refund and order canceled.

Customer-facing message	Event behind it	Which system emits it	Typical lag the customer tolerates
Order confirmed	Order placed and payment authorized	The platform, after the PSP answers	Seconds

Customer-facing message	Event behind it	Which system emits it	Typical lag the customer tolerates
Payment problem	Authorization failed or pending (bank transfer, buy-now-pay-later)	The platform, from the PSP's webhook (a message the PSP pushes when a payment changes state)	Minutes
Shipped, with tracking	Parcel handed to the carrier, label scanned	The WMS or the 3PL (third-party logistics provider), via the shipping service	Minutes to an hour
Ships in two parcels	A fulfillment order split, or one line delayed	The OMS; nothing emits it in a landscape without one	Before the first parcel arrives
Out for delivery; delivered; attempted	Carrier scan events	The carrier, normalized by the shipping service	Minutes
Ready for collection	Store confirmation of a click-and-collect pick	The OMS from the store app (Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment)	Minutes
Return received; refund issued	Return inspected; refund executed	The WMS and the platform or PSP, via the returns portal (The Way Back: Returns, Exchanges, Refunds and Reverse Logistics)	Hours to a day

The middle row is the one a newcomer should stare at. "Ships in two parcels" is the message that would have ended the eleven tickets, and it has no emitter at Kestrel & Pine in 2026, because no system holds the order's lines and their fulfillment state together. It is also the kind of message that an order management system produces as a side effect of doing its job (The Order Record: Lifecycle, State and the Order Management System).

The order status page

The **order status page** is the self-service page where the customer sees where the order is. It is the cheapest answer to "where is my order" the merchant will ever give, and it is a **read model**: a view assembled from the order record and the tracking stream for the purpose of being read, not the place where either is written. Shopify's documentation of its page (n.d., accessed November 2026) shows the design choices in miniature: five customer-facing states, "Confirmed, On its way, Out for delivery, Delivered, and Attempted delivery", updated only for supported carriers; "If you fulfill an order without a tracking number, then no shipping updates are received"; and an access rule under which a visitor who is not logged in sees items, total and status but not the address, the payment method or the tracking number, with link access lasting three weeks in the same browser.

Three rules follow. The page must be fed by the **same** status the notifications and the agent use. It must receive tracking numbers the moment the parcel is handed over, or it goes silent. And it is a privacy boundary: a link in an e-mail is a credential, and the page should show a stranger holding it as little as possible (Whose Data Is It? Privacy, Consent and Retention).

"Where is my order" as a measurement

WISMO ("where is my order") is the service contact that asks for delivery status. Gorgias, a helpdesk vendor, reported from its own customer base in 2023 that WISMO requests "account for 18% of incoming requests on average" (an article updated in November 2023, now approaching three years old, drawn mostly from small Shopify merchants). Other vendor figures range from 10% to 50% and rise at peak; a 2026 critical reading on the Branvas blog points out that these are shares of *tickets*, not of *shipped orders*, and are not comparable across vendors. No independent study was found for this book.

The safe statement: WISMO is consistently reported as the largest single contact category in online retail service, roughly a fifth of tickets in the one dataset that states its basis. The useful statement is architectural: **WISMO per shipped order is a direct measure of the notification and tracking layer**. It rises when tracking numbers arrive late, the promise slips, shipments split without a message, or the status page reads a system that is behind. Measured that way, it is a dashboard for the integrations, not a staffing problem.

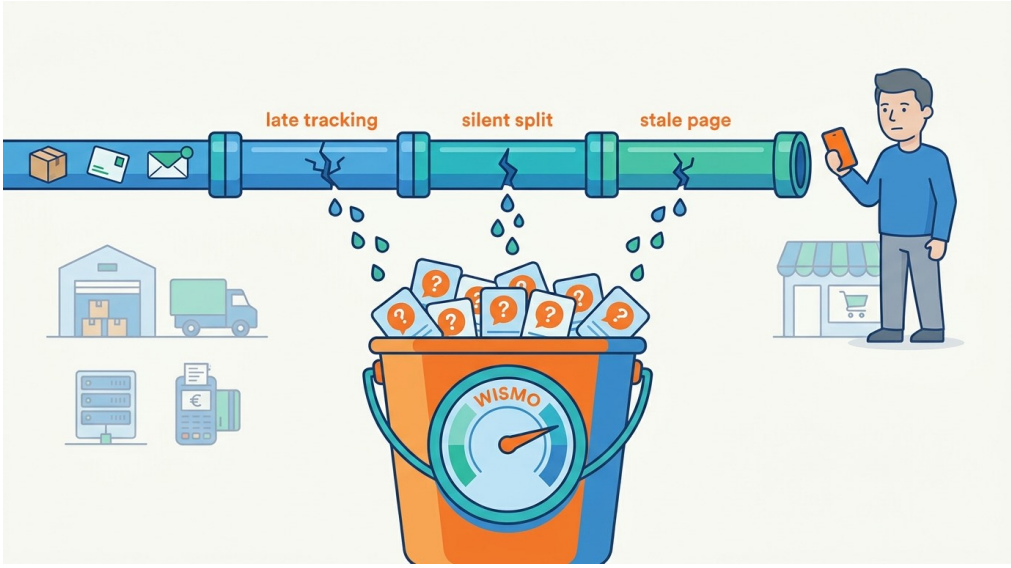


Figure 2: *WISMO per shipped order reads the health of the notification and tracking layer: every late tracking number, silent split or stale status page leaks out as a where-is-my-order ticket.*

How the Pieces Fit Together

The target shape has one order record (the OMS, from Q4 2027 at Kestrel & Pine) emitting events; a notification service that turns events into messages per channel and country; a status page and a service tool that read the same record; and self-service actions that write back through the order record's own rules.

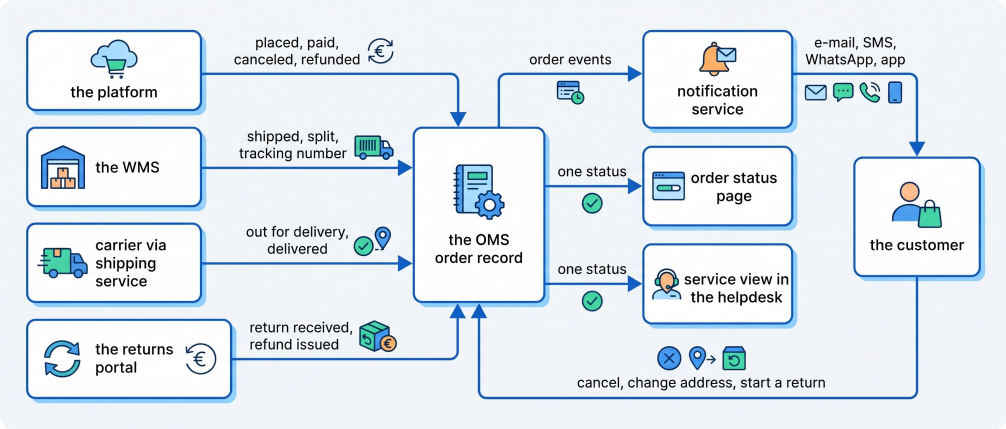


Figure 3: *One order record, fed by the platform, the warehouse, the carrier and the returns portal, drives the notifications, the status page and the agent's view alike; self-service actions go back through the same record and its rules, so the customer and the agent can never read two different orders.*

Component	Responsibility	What it must never do	At Kestrel & Pine, 2027
Order record (OMS)	Hold every channel's order with separate state machines for money, goods and returns; emit an event on every change	Collapse the state machines into one string	Bought; order hub by Q4 2027 (W2)
Notification service	Subscribe to order events; choose channel and language per customer and country; render templates; send through providers; record what was sent	Send from a label printed rather than a parcel handed over; send twice for one event	The platform's built-in e-mails today; a subscriber to the OMS later

Component	Responsibility	What it must never do	At Kestrel & Pine, 2027
Order status page	A read model of the order record plus the tracking stream, with a privacy boundary	Read a different system than the agent's tool	The platform's page today, fed by the iPaaS; W8 adds status by API
Service view	Show the agent one order across systems; offer actions (refund, cancel, change address, reship) gated on the order's state	Issue a refund through a second path	Five tabs and a helpdesk that reads the platform; a helpdesk over the OMS later
Self-service	Let the customer cancel before picking, change the address before the label, and start a return, each gated on the fulfillment state	Allow an edit the warehouse has already acted on	A returns portal; no cancel or address change
Customer record	Tell the agent who this is: online account, store customer, wholesale contact, consent	Guess a match	Four unlinked lists; One Customer, One Product: Identity and Master Data Across Systems

Requirements for the flows

Flow	How fresh	How consistent	How fast	How available
Order confirmation	Seconds after payment	Must match the order the customer sees in their account	Under a minute	Queued if the mail provider is down; never lost
Shipped notification	Minutes after the carrier scan	Tracking number must already resolve at the carrier	Within the hour	Degrades to "shipped, tracking to follow"
Status page	Minutes behind the record	Identical to the agent's view	Under two seconds to load	Serve the last known status with its timestamp when the record is unreachable
Service view	Seconds	Identical to the status page; actions gated on state	Under two seconds per lookup	Read-only fallback when the OMS is down; refunds queued, not re-keyed
Self-service cancel	Immediate check against the fulfillment state	Exactly once; reversed stock commitment	Seconds	Refuse rather than guess when the state is unknown

The consistency column is the point. Freshness can be minutes; two views of one order can never differ, because the difference is what the customer writes in about.

The state machines behind "status"

The Three-Status Order was not a bug in one system; it was three systems each telling the truth about a different thing. Platforms model this openly. Shopify's help page on order status (n.d., accessed November 2026) keeps three status families on one order: an

order status (open, archived, canceled), a payment status (eight values from authorized to refunded) and a fulfillment status (six values including partially fulfilled), with seven more statuses on the fulfillment order below it. commercetools' orders API (n.d., accessed November 2026) keeps an order state, a shipment state (with *Partial* for "items in the Order are shipped in more than one shipment" and *Backorder*) and a payment state, plus return item states. In both, "fulfilled" does not mean "delivered" unless carrier events are fed back.

The lesson for the service layer: the agent's screen and the status page should show the **goods** state per line (the socks are on backorder), the **money** state for the order (paid €137, nothing refunded) and the **return** state when there is one. A single word is a subject line for the customer's inbox, not what the agent or the page should read.

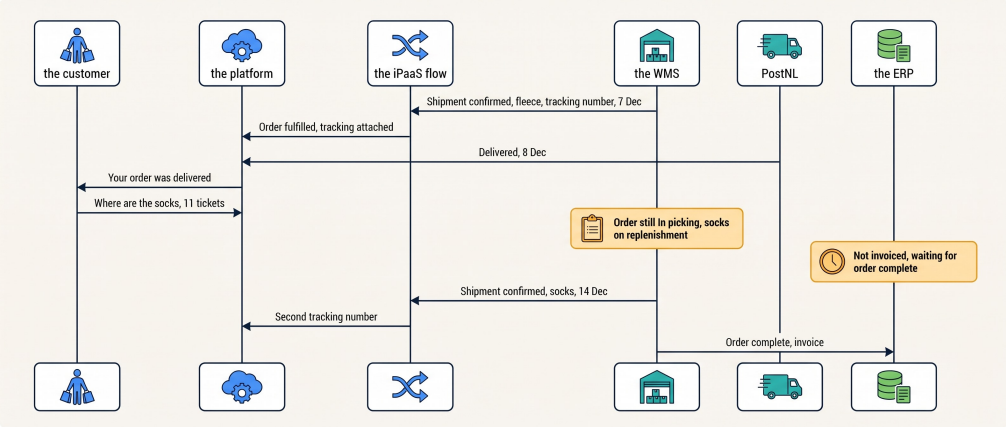


Figure 4: *The Three-Status Order: a shipment confirmation for the first parcel turned the whole order "fulfilled", the carrier's delivery scan turned it "Delivered", and the socks stayed "In picking" and the invoice "Not invoiced" until the second parcel shipped six days later; the one message the customer needed, "two parcels", had no system to come from.*

The Choices

A merchant has four realistic ways to build the service layer, and they line up with the scale stages.

1. **The platform's built-in notifications and a helpdesk that reads the platform.** The platform sends its standard e-mails; a helpdesk such as Zendesk or Gorgias shows the platform's order next to the ticket. Zendesk's documentation of its Shopify integration (n.d., accessed November 2026) describes the shape: orders matched by the requester's e-mail or phone; date, value, payment status, fulfillment status and tracking numbers in the sidebar; agents can refund in any fulfillment state and cancel only unfulfilled orders.

This is the right choice while the platform is the only system that fulfills. It stops being enough the day a warehouse system or an OMS holds a state the platform does not.

2. **A tracking-and-notifications vendor over the carrier stream.** Multi-carrier and post-purchase vendors (Sendcloud, nShift, Narvar and others) sell branded tracking pages and shipping notifications driven by carrier events; Narvar claims its pages "convert 10% higher and reduce support costs" (vendor claim, n.d., accessed November 2026). They solve the carrier half well and the order half not at all: they know where the parcel is, not whether there is a second one or whether the refund went out.

3. **A notification service and a service view over the order management system.** Once an OMS is the order record, the notification service subscribes to its events, the status page reads it, and the helpdesk's order sidebar is backed by it. OMS vendors sell this view: Fluent Commerce says agents "see the same order data the platform used to make its decisions"; Kibo lets representatives edit, credit and refund "without leaving the order screen" (both n.d., accessed November 2026). This is where Waypoint takes Kestrel & Pine.

4. **A built notification and service layer.** A merchant with engineers and unusual needs (many countries, brands and channels) builds the event subscriber, the template engine and the service screens itself. Right at Group stage; the Giant's Boots for anyone smaller (Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says).

	Platform notifications and helpdesk	**Tracking vendor**	**Notification service and service view over the OMS**	**Built layer**
Knows the carrier events	Only for supported carriers with tracking numbers	Yes, normalized	Yes, via the shipping service	Yes
Knows splits, backorders and returns	No	No	Yes	Yes
Agent actions gated on state	Refund any state; cancel unfulfilled	None	Yes, by the OMS's rules	As built

	Platform notifications and helpdesk	Tracking vendor	Notification service and service view over the OMS	Built layer
Channels and languages	The platform's templates	The vendor's	Configurable per country	Anything
Integrations to own	None	One (carrier stream)	OMS events, templates, helpdesk connector	Everything
Risk of two refund paths	Helpdesk and returns portal both refund through the platform	None	Low if the OMS owns the refund command	As built
Fits best	Starting, Growing	Growing, as a complement	Omnichannel	Group

My rule: **the service view must read the system that owns the goods state**. While the platform owns it, choice 1 is right and cheap. The moment an OMS or a WMS owns it, every view that still reads the platform is a Schrödinger's Order waiting to happen, and the helpdesk must be re-pointed at the OMS even if nothing else changes. The notification channels are a second-order question; the source of truth is the first.

Self-service, gated on state

The contacts that need no judgement (cancel before picking, change the address before the label, start a return) should not reach a person. Each is an edit to a placed order, and each must check the fulfillment state first. Shopify's order editing documentation (n.d., accessed November 2026) states the constraint plainly: "You can edit only unfulfilled line items", and its fulfillment order has a request status (unsubmitted, submitted, accepted) after which "the merchant can no longer cancel it directly and must submit a cancellation request". In plain words: once the warehouse has accepted the work, a cancellation is a request to the warehouse, not a change to a record. A self-service cancel button that does not know this will cancel orders that are already in a box.

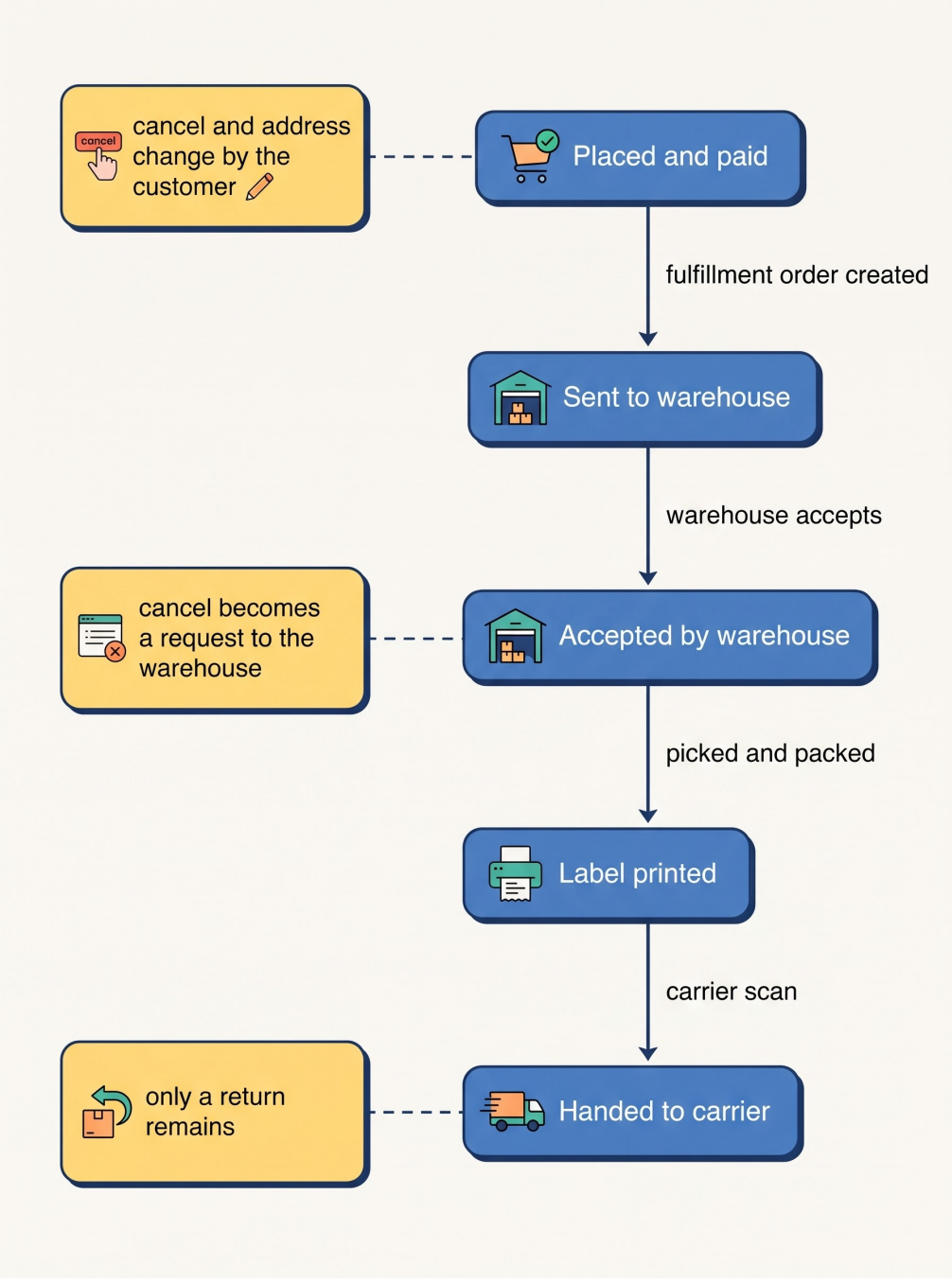


Figure 5: *Which self-service actions are safe depends on the goods state: before the warehouse accepts, the customer may cancel or change the address; afterwards a cancel is a request; once the carrier has the parcel, only a return remains.*

Service for marketplace orders

On a marketplace, the customer conversation belongs to the marketplace, and the merchant answers under its rules and clocks. bol's partner standards (n.d., accessed November 2026) include a per-seller "dynamic standard" for customer questions as a share of orders, a response-time standard and a strike process in which weeks below standard accumulate toward closure; Amazon's order performance policy (European policy document, May 2026) sets the Order Defect Rate, the share of orders with negative feedback, a guarantee claim or a chargeback, at "under 1%" over 60 days. So the helpdesk must ingest marketplace messages with their deadlines, and the service view must find an order by the marketplace's id as well as its own. Personal data is the other difference: Amazon's SP-API (Selling Partner API) returns a buyer's name and address only through a restricted data token, so the service view holds only what fulfillment needed (Selling on Someone Else's Platform: Marketplaces and Channel Integration, Whose Data Is It? Privacy, Consent and Retention).

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
1 — Starting	The platform's standard e-mails; a shared mailbox; the founder answers WISMO by looking at the carrier's site	Tracking numbers on every shipment; the platform's status page turned on; one reply template per common question	The same question asked ten times a week; orders shipped without a tracking number

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
2 — Growing	A helpdesk reading the platform's order; a 3PL or WMS that confirms shipments by integration; the platform's notifications plus a carrier's own messages	The shipment confirmation from the 3PL must carry the tracking number within the hour; a WISMO-per-shipped-order measure; a single refund path between the helpdesk and the returns tool	Agents keep a second tab open for the 3PL; "shipped" e-mails that precede the handover; refunds issued from two consoles
3 — Omnichannel	Orders from web, marketplaces, stores and wholesale; an OMS as order record; a notification service with channels per country; a service view over the OMS; self-service gated on state; marketplace messages in the helpdesk with their deadlines	One status source for the page, the messages and the agent; split-shipment and backorder messages; a customer record that joins the online account to the store customer; service metrics per channel	Five tabs; the Three-Status Order; WISMO rising at peak while order volume is flat; marketplace strikes for late replies

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
4 — Group	Several brands and countries; a shared notification platform with per-brand templates; AI-assisted agents and chat; a customer data layer feeding service	Per-brand service levels on a shared order record; consent and data residency per country in the notification layer; audit of what was sent to whom	A shared template that breaks one brand's tone or one country's law; an assistant answering from a stale copy of the order

The judgement: the layer changes shape exactly once, when the goods state leaves the platform. Everything before that point can stay on the platform's built-in tools; everything after it must read the order hub. Kestrel & Pine is at that point now, and the eleven tickets are what the delay cost.

In Practice

Events, not polling, is also the rule for the messages customers receive. DPD's Predict notifications in Belgium (n.d., accessed November 2026) are sent by the carrier itself when the parcel "arrives at the depot, and when it leaves the depot for the receiver". EasyPost's tracker object (n.d., accessed November 2026) reduces every carrier's codes to ten statuses, from pre-transit to delivered, with webhooks on update. A notification service needs five of the ten for the customer and all ten for the agent, pushed, not polled (APIs, Events, Files and Middleware: How Commerce Systems Talk).

Channels change their economics on published dates. Meta's WhatsApp Business Platform pricing page (accessed November 2026) classifies templates as marketing, utility ("template messages for transactional or informational objectives") and authentication, states that per-message pricing arrived on 1 July 2025, when utility messages sent inside the 24-hour customer service window became free, and that from 1 October 2026 those utility messages are charged as regular messages again, with a free tier of 1,000 a month that applies to service messages only. A notification service that chooses the channel per message, with cost as an input, survives such changes; one that hard-codes "WhatsApp for shipped" does not.

Order status by API is a prerequisite, not a feature. Waypoint's W8 lists order status by API among what an AI shopping agent would need before acting for a customer (Assistants and Agents: AI in Commerce and What Is Real in 2026). The same endpoint

feeds the status page, the helpdesk and the carrier vendor's tracking page: one read model, every consumer.

The service tool's refund button is another refund path. Zendesk's Shopify integration lets an agent refund from the ticket in any fulfillment state; Kestrel & Pine's returns portal refunds through the platform's API; its PSP console allows a refund by hand. In January 2027 the last two produced the Double Refund (**212 returns**, **€19,400** refunded twice, most recovered); the helpdesk's button is a third. The rule: the order record owns one refund command and every tool invokes it, with the order and return ids as the **idempotency key** (a key that makes a repeated request have the same effect as one) (Taking the Money: Payment Providers, Authorization, Capture and Refunds, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation).

Kestrel & Pine, worked: Eva's jacket from the customer's side

The dates are Kestrel & Pine's records; the message times and channels are illustrative details. The "2026 landscape" column is what Eva would have received had she ordered in December 2026; the "target" column is the Q4 2027 design.

When	What happened	2026 landscape: what Eva receives	Target: what Eva receives
Tue 13 Apr 2027, 21:14	Order #1072455 placed, €235 by iDEAL	Confirmation e-mail from the platform within a minute	Same, with "delivery expected Thursday" computed, not static (Getting It There: Carriers, Labels, Tracking and the Delivery Promise)
Wed 14 Apr, 08:40; collected 22:30	Picked, packed, PostNL label; collected that evening	"Shipped" e-mail when the iPaaS flow marks the order fulfilled, sometimes before the parcel leaves	"Shipped" when the carrier scan arrives, with a tracking link that already works

When	What happened	2026 landscape: what Eva receives	Target: what Eva receives
Thu 15 Apr, morning	Out for delivery	A PostNL message, if Eva opted in with PostNL	A message from Kestrel & Pine, same event, her chosen channel
Thu 15 Apr, 11:20	Delivered	Status page: Delivered	Status page: Delivered, per parcel
Mon 19 Apr	Exchange for size L requested in the returns portal	Portal e-mail with the label; the platform's page knows nothing of it	"Return registered, send within 14 days"; the order's return state visible on the page
Thu 22 Apr	Return received, graded resellable; size L ships	Two e-mails from two systems, in two designs	"Received; your size L ships today", one message, one design
If Eva writes in	"Has my return arrived?"	The agent opens the returns portal tab and the WMS tab	The agent's view shows the return received at 10:15 and the exchange's tracking number

Nothing in the target column needs a new channel. It needs one record emitting the events and one read model serving the page, the messages and the agent. The messages that matter most are the ones that pre-empt the ticket: "two parcels", "received", "ships today".

Measuring service as an architecture signal

A small set of measures turns the service desk into the platform's most honest monitor. Targets are illustrative.

Measure	Definition	Why it is an architecture signal	Kestrel & Pine, 2026 *(assumption)*	Illustrative target
Contacts per 100 orders	All tickets, calls and chats divided by shipped orders	The total cost of every gap	9	Under 5
WISMO per 100 shipped orders	Delivery-status contacts only	The notification and tracking layer	3.5, rising past 6 in Black Friday week	Under 1.5, flat at peak
Contacts per split shipment	Tickets on orders shipped in more than one parcel	Whether the split message exists	Not measured	Under 0.2
Status disagreements found	Orders whose page and agent view differ, sampled weekly	Whether the views read one source	Not measured	Zero
Tickets per order (max)	The worst order of the month	The long tail that becomes a social-media story	11	Under 4
First reply with a true answer	Share of WISMO tickets answered with the correct goods state on first reply	Whether the agent can see the order	Unknown	Over 95%

Hanna Berg's data team can produce most of these from the helpdesk and the order record once there is one; the first and the last are the ones to put on Noor's monthly page (Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics).

When It Goes Wrong

- **The notification is sent twice**. Events are delivered at least once; a retried "delivered" event sends a second e-mail, and the customer wonders whether there were two parcels. *Detection*: sent-message log keyed by order and event. *Recovery*: the notification service is idempotent on the event id; it records what it sent before it sends.
- **The notification precedes the event**. "Shipped" goes out when the label prints, and the parcel waits a day for the carrier (Kestrel & Pine's Black Friday queue backed up for four hours); "delivered" goes out when the carrier's scan says so, and the parcel is with a neighbor. *Detection*: WISMO tickets that quote the notification. *Recovery*: trigger "shipped" on the handover scan, and say "delivered to a neighbor" when the carrier says so; the carrier's vocabulary has that state, and the message should too.
- **No tracking number, no updates**. A fulfillment confirmed without a tracking number leaves the status page frozen at "confirmed". *Detection*: shipments without tracking, counted daily. *Recovery*: the WMS integration contract requires the number; the notification service sends "shipped, tracking to follow" and then the number.
- **The split that nobody announced**. The Three-Status Order. *Detection*: contacts per split shipment. *Recovery*: an order record that holds goods state per line, a "ships in two parcels" message, and an agent view that shows both parcels.
- **The carrier is down**. No scans for a day; every parcel looks stuck. *Detection*: a monitor on the carrier stream's freshness. *Recovery*: a banner on the status page ("carrier updates delayed") and a hold on "attempted delivery" messages until the stream resumes (Black Friday Every Year: Peak Load, Resilience and Recovery).
- **The agent and the page disagree**. The helpdesk reads the platform; the page reads the OMS. *Detection*: the weekly status-disagreement sample. *Recovery*: re-point the helpdesk; never run two sources.
- **The template is wrong for the country**. A Belgian customer in French receives a Dutch "shipped" with a Dutch legal footer. *Recovery*: templates selected by storefront, language and country, with the legal text owned by whoever owns the regulatory calendar (Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation).
- **The agent cannot find the person**. Eva writes from the e-mail on her store loyalty account, which differs from her online account's; the agent finds no order (the *Familiar Stranger* pattern). *Recovery*: search by order number first, by any known identifier second, with a matching rule of declared confidence (One Customer, One Product: Identity and Master Data Across Systems).

What to Take Away

Established.

- Platforms model an order with several status machines (Shopify's order, payment and fulfillment statuses plus the fulfillment order's own; commercetools' order, shipment and payment states), and "fulfilled" means "delivered" only when carrier events are fed back (vendor documentation, accessed November 2026).
- Transactional notifications are triggered by events from three different emitters: the platform (placed, canceled, refunded), the warehouse or 3PL (shipped with tracking) and the carrier (out for delivery, delivered, attempted); a status page without a tracking number receives no updates (Shopify documentation, accessed November 2026).
- Order edits and cancellations are safe only before the warehouse accepts the work; afterwards they become requests (Shopify order editing and fulfillment order documentation, accessed November 2026).
- "Where is my order" is consistently reported as the largest single contact category in online retail service; the only figure with a stated basis is 18% of tickets (Gorgias, 2023), and vendor figures are shares of tickets, not of orders.

Depends on context.

- While the platform owns the goods state, its built-in notifications and a helpdesk that reads the platform are the right design; once a WMS or an OMS owns it, every view must read the order hub. The trigger is the ownership of the goods state, not the order volume.
- Whether to add a tracking-and-notifications vendor depends on how many carriers and countries the merchant ships to; it complements the order record and never replaces it.
- The channel mix (e-mail, SMS, app, WhatsApp) depends on country and on pricing that changes on published dates; choose per message with cost as an input.

The author's synthesis.

- Build the status page, the notifications and the agent's screen as three read models over one order record, and treat any difference between them as a defect, because every difference is a ticket with the merchant's name on it.
- Measure WISMO per shipped order and put it next to the integration monitors, not next to the staffing plan: it is the customer's reading of the platform's joins, and the one that arrives first.

Questions to Ask

1. Which system does the order status page read, and which system does the service agent's tool read? Are they the same?
2. Which event triggers the "shipped" message: the label, or the carrier's scan?
3. If an order ships in two parcels, what does the customer receive, and when?
4. How many systems can issue a refund on one order, and which of them is the one that should?
5. What can a customer do without contacting anyone (cancel, change address, start a return), and what state check guards each action?
6. How many WISMO contacts per hundred shipped orders did we have last month, and how did the number move in peak week?
7. How many tabs does an agent open to answer "where is my order"?
8. How do marketplace messages reach the agent, with which deadline, and under which order id?
9. What happens when the customer writes from an e-mail the platform has never seen?

Part IV ends here, with the order delivered, returned if it must be, and explained to the customer. The next part leaves the merchant's own channels: Part V — CHANNELS: Selling Everywhere begins with selling on someone else's platform, in Selling on Someone Else's Platform: Marketplaces and Channel Integration, where the customer conversation, the deadlines and the order id all belong to the marketplace.

Sources and Evidence

Company, product and investor sources

- Customer notifications, Shopify Help Center, n.d., accessed 2026-11 — the notification list and the events and emitters behind each message.
- Understanding order status pages, Shopify Help Center, n.d., accessed 2026-11 — the five customer-facing states, the tracking-number rule and the access rules without login.
- Order status, Shopify Help Center, n.d., accessed 2026-11 — the three status families on one order.
- Edit orders and FulfillmentOrder object, Shopify developer documentation, n.d., accessed 2026-11 — edits only on unfulfilled items; cancellations as requests after acceptance.
- Orders (OrderState, ShipmentState, PaymentState), commercetools, n.d., accessed 2026-11 — separate order, shipment and payment states, including Partial and Backorder.

- Using the Shopify integration in Support and Chat, Zendesk, n.d., accessed 2026-11 — the helpdesk's order sidebar, refund in any state, cancel only unfulfilled.
- What's the secret to reducing WISMO requests?, Gorgias, 2023 — the 18% share of incoming requests (evidence age approaching three years; one vendor's customer base) and the remedies listed.
- Gorgias, Gorgias, n.d., accessed 2026-11 — the one-inbox helpdesk with in-ticket order actions (vendor claims).
- Fluent Commerce order management and Kibo order management, n.d., accessed 2026-11 — the service view sold from the OMS side (vendor claims).
- Narvar, n.d., accessed 2026-11 — branded tracking pages and the vendor's claims on conversion and support cost.
- Trackers, EasyPost, n.d., accessed 2026-11 — the ten tracker statuses and webhooks on update.
- Sending tracking notifications (Predict), DPD Belgium, n.d., accessed 2026-11 — carrier-originated notifications at depot arrival and departure.
- WhatsApp Business Platform pricing, Meta, accessed 2026-11 — template categories, per-message pricing from 1 July 2025 and the 1 October 2026 change for utility messages in the service window.
- Service standards, Customer questions and Strike process, bol partner platform, n.d., accessed 2026-11 — the customer-questions standard, response time and strikes.
- Order Performance program policy (DE), Amazon, 2026 — the Order Defect Rate definition and the under-1% threshold.
- Tokens API use case guide, Amazon SP-API, n.d., accessed 2026-11 — personal data available only through a restricted data token.

Standards, regulation and official documentation

- Directive 2011/83/EU on consumer rights, Publications Office of the EU, 2011 — the 14-day withdrawal and refund clocks that the return messages refer to.

Research and practitioner sources

- WISMO benchmarks, Branvas, 2026 — a critical reading of vendor WISMO figures as shares of tickets rather than of orders.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Part V — CHANNELS: Selling Everywhere

IN THIS PART, YOU WILL: Follow the orders that never start on the merchant's own web page, from marketplaces, wholesale accounts and EDI (electronic data interchange) trading partners, into the one order record and on into the ERP (enterprise resource planning system, the finance back office) that posts them.

KEY POINTS:

- **Every outside channel is a way into the same order record, not a second order system**. A marketplace order, a portal order, an order keyed in from an e-mail and an EDI order should create the same order object, flagged by channel. Kestrel & Pine's orders are born in four systems, and nobody can see one customer's orders in one place.

- **The intermediaries translate; they must not own.** The marketplace connector, the EDI provider and the ERP partner's connector carry facts between the systems that own them. Each fact has one honest source: the stock number from one availability calculation, the credit limit from the ERP, the pallet contents from the warehouse that packed them, the invoice from the ERP that numbers it.
- **Outside parties keep score and set the calendar.** Marketplaces grade cancellations and late shipments against published thresholds, retail chains charge for a despatch advice that does not match the pallets, API versions retire on published dates, and e-invoicing mandates come with legal dates. Finance can run daily and in batch; availability cannot, so the ERP must never sit in the path of a sale.

FAILURE PATTERNS THIS PART ADDRESSES:

- **Schrödinger's Order** — marketplace orders live in the connector, portal orders in the platform, keyed and EDI orders in the ERP, and support opens all three.
- **The Pulled Rug** — a marketplace retires an API version, a chain revises its message guideline, a country sets an e-invoicing date, and the merchant learns from a canceled order or a refused truck.
- **The Lynchpin** — the EDI mappings live in one consultant's head and the ERP connector in one partner's configuration export.

Most of this book follows an order that starts on the merchant's own web page. A growing merchant takes a large share of its revenue from orders that do not. A shopper buys on bol's or Amazon's page and never notices whose parcel arrives. An outdoor shop orders sixty jackets on thirty-day terms at a price agreed for the season. A retail chain's purchasing system sends a purchase order as a standard document and expects a pallet-by-pallet notice and a machine-readable invoice back. **Each of these channels comes with an outside party that sets the rules**: a scorecard, purchasing conditions with penalties, a legal mandate with a date. And every one of these orders ends in the ERP, where finance posts it.

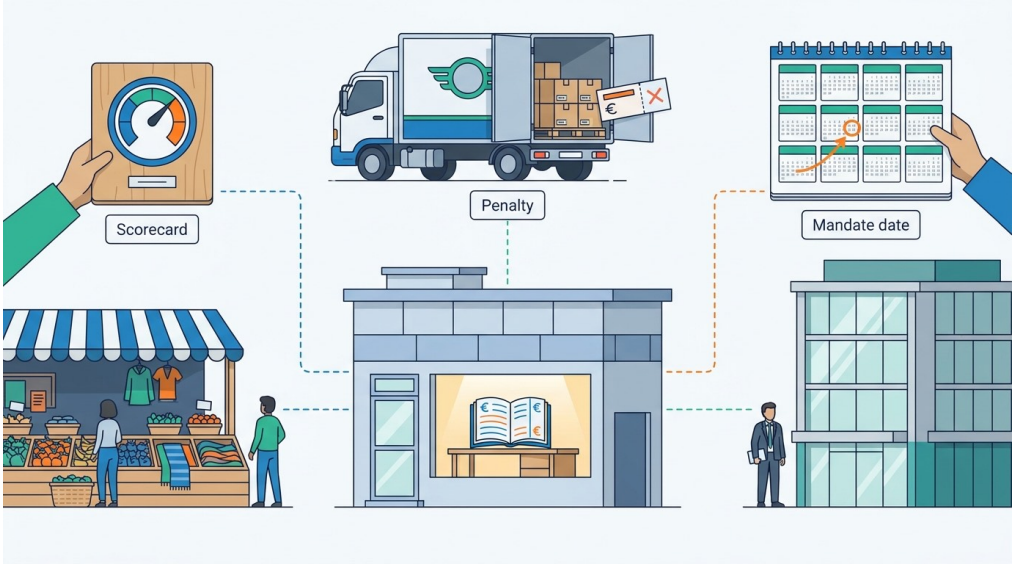


Figure 1: *Each outside channel comes with an outside party that sets the rules: a marketplace keeps a scorecard, a retail chain charges penalties, and a mandate arrives with a legal date.*

Part V sits beside Eva's jacket rather than on it. Her order of 13 April 2027 never touches a marketplace, a wholesale account or an EDI message. Its siblings do: two Amazon.de orders for a Ridgewalker jacket placed the same day, one shipped from Duiven and one by the marketplace; De Kaap Buitensport's pre-season order of 25 February; Bergzicht Outdoor's spring reorder of 6 April, 240 units on three pallets. All of them meet Eva's order in the warehouse and in the ledger, which is where the part ends.

Kestrel & Pine, the fictional outdoor brand this book follows, earned €3.0 million on three marketplaces in 2026 through a connector rented since 2021, and €12.0 million from 1,100 wholesale accounts, 62% through the platform's B2B portal and 38% keyed into the ERP by hand, plus one retail chain, Bergzicht, that orders by EDI. The ERP has been connected to the platform since 2023 by a connector the ERP partner built and nobody else could read. **The claim of this part is that an outside channel is a way into the one order record, and that the systems between the merchant and the outside party must translate without owning anything.** Waypoint's decisions W2 (one order record), W6 (EDI compliance) and the finance-only rule for the ERP are that claim written down.

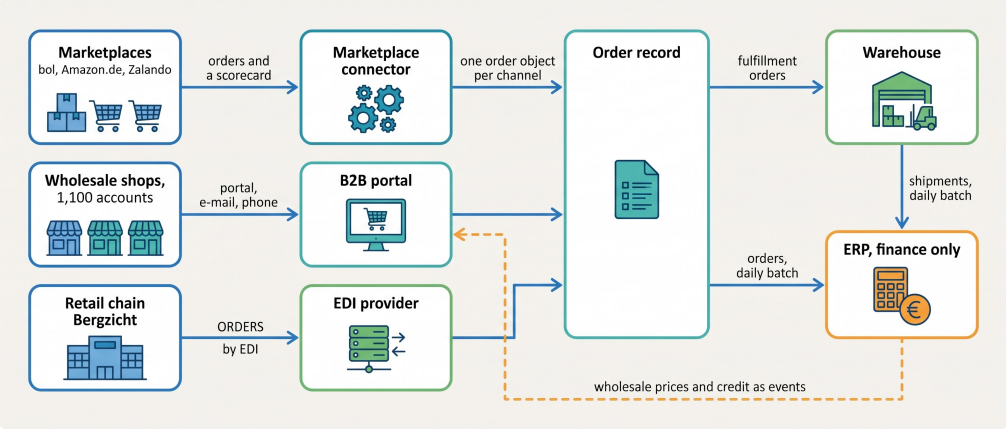


Figure 2: *The outside channels of Part V: three kinds of outside party, three intermediaries that translate, one order record that every channel's order is created in, and an ERP that receives daily batches and feeds no channel directly: its wholesale prices and credit limits leave it only as events.*

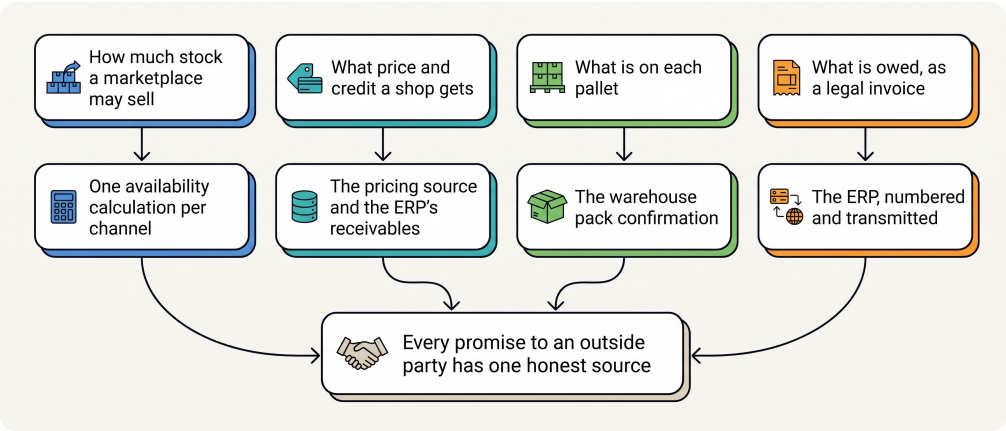


Figure 3: *The four promises Part V makes to outside parties and the one system allowed to make each; every chargeback, strike and keyed-in order in the part is a promise that came from the wrong system.*

Chapters in this part

Selling on Someone Else's Platform: Marketplaces and Channel Integration decides **what it takes to list, sell and ship on a marketplace, and where a marketplace order belongs**: a marketplace is a channel with a scorecard (Amazon's cancellation rate under 2.5%, bol's strikes), the connector translates five flows and must own none of them, the stock number sent is a promise computed once per channel, and every

marketplace API retires on a published date. The case is the three marketplaces behind the connector: the 160 marketplace orders and the bol strike inside the Black Friday Oversell, the Deprecated Endpoint's two days of stale stock, and the two Amazon.de orders of 13 April 2027.

Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals decides **what changes when the buyer is a company, and which system owns the account, the price and the credit limit**: the customer becomes an organization with locations, people and roles; a B2B price is a promise with a validity date that the invoice must match; payment terms and credit limits are finance facts in the ERP that the checkout must be able to read. The case is the 38% of wholesale orders keyed in by hand, and De Kaap Buitensport's pre-season order of 25 February 2027, €6,720 at list, held because it would push the shop past its €15,000 credit limit.

Documents Between Companies: EDI, Punchout and E-Invoicing decides **how large trading partners exchange orders, despatch advices and invoices without a web shop, and what the e-invoicing mandates require**: an EDI order creates the same order record as a web order, each document has its own system of truth, and structured e-invoicing is law with dates, Belgium since 1 January 2026, Germany from 2027, the Netherlands planned for 2030. The case is Bergzicht's ORDERS–DESADV–INVOIC cycle of 6 April 2027 and the €14,200 ASN Chargeback, answered by decision W6.

The Back Office: ERP Integration and Who Is the Hub decides **what the ERP should own in a commerce business, and whether it should be the hub**: four jobs (the ledger, purchasing, inventory valuation, master data), three honest answers to "who is the hub" and one accident, and the rule that finance can be batch but availability cannot. The case is the partner-built connector that took three weeks to reconstruct, the January 2027 month-end that took eleven working days, Eva's order in the books, and the finance-only rule that retires the connector in late 2027.

Begin with Selling on Someone Else's Platform: Marketplaces and Channel Integration.

Sources and Evidence

Every public claim in this part is sourced and dated in the chapter that makes it, under that chapter's Sources and Evidence. Kestrel & Pine is fictional; recommendations and labeled assumptions are the author's judgement.

20. Selling on Someone Else's Platform: Marketplaces and Channel Integration

IN THIS CHAPTER, YOU WILL: Learn how a merchant lists, sells and ships on a marketplace, what a channel connector does and must not do, and how to allocate stock and import orders so that marketplace promises are kept.

KEY POINTS:

- **A marketplace is a channel with a scorecard.** It measures your cancellations, late shipments and tracking against public thresholds (Amazon: cancellation rate under 2.5%, late shipment rate under 4%; bol: at most 2% cancellations a week), so every hour of stock or shipment latency inside your integration is a commercial risk, not a technical detail.
- **The connector translates; it must not own.** Listings should come from the product system of record, the stock number from one availability calculation, prices from the pricing source, and every marketplace

order should land in the merchant's one order record. A connector that keeps its own copies becomes a fourth system of record, which is what Kestrel & Pine has today.
- **The number you send is a promise**. Decide how much stock each channel may sell (a buffer at first, later a computed available-to-promise figure: what can still be sold after existing orders and reservations), model marketplace-held stock as a location you do not control, and own the deprecation calendar of every marketplace API, because versions retire on published dates whether or not anyone is watching.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Phantom Stock** — the stock figure sent to a marketplace is a copy of a copy, hours old, and the last unit sells twice.
- **The Pulled Rug** — marketplaces and connectors retire API versions on published dates; an unowned script finds out two days later.
- **Schrödinger's Order** — a marketplace order that lives only in the connector cannot be found by support, finance or the warehouse report.

A **marketplace** is a shop that sells other merchants' products next to its own: Amazon, bol in the Netherlands and Belgium, Zalando in fashion, and dozens of others. The shopper buys from the marketplace's page, pays the marketplace, and often never notices which merchant the parcel came from. For the merchant it is reach without a storefront, in exchange for a **commission** (the percentage of each sale the marketplace keeps), a set of rules about service, and a loss of control over the customer relationship.

The rules are the part newcomers underestimate. **A marketplace does not just take orders; it grades you.** It measures how often you cancel, how often you ship late and whether your tracking numbers are valid, and it publishes thresholds with consequences: lost visibility, restricted selling, a closed account. Those metrics are computed from data your systems send it, so an hourly stock sync or a warehouse confirmation that arrives a day late is not an integration nuisance; it is a number on your scorecard.

This chapter is about the architecture behind that relationship: how products become **listings** (the marketplace's product page built from your data), how marketplace orders get into your systems and their status back out, how much stock each channel may sell, and how prices stay consistent. It also covers the thing in the middle, the **channel connector**, and the decision that matters most about it: what it is allowed to own.

Where we are in the order's journey. Eva's jacket, the recurring order, was placed on Kestrel & Pine's own storefront and never touches a marketplace. Its sibling does: on the same day, 13 April 2027, two Amazon.de customers order a Ridgewalker jacket. One is shipped by Kestrel & Pine from its Duiven warehouse; the other is shipped by the marketplace from stock Kestrel & Pine sent to its warehouse in advance. The two orders look identical to the customers and nothing alike to the systems, and this chapter

follows both. Follow the Order: What Happens Between "Buy" and "Delivered" gives the full path.

Setting the Scene at Kestrel & Pine: Two Days of Stale Stock

Kestrel & Pine, the fictional outdoor brand this book follows (Meet Kestrel & Pine), sells on **bol** (Netherlands and Belgium), **Amazon.de** and **Zalando** (Germany and the Netherlands) through **the marketplace connector**, a channel integration service it has rented since 2021 for €900 a month. In 2026 the marketplaces brought **43,000 orders** and **€3.0 million**, an average of **€70** an order, against €92 on the brand's own storefronts. About **40%** of marketplace units for Germany are shipped by the marketplace's own fulfillment program from stock Kestrel & Pine sends there in advance. Milan Petrov, the Ecommerce and Marketplaces Manager, owns the connector, and two incidents sit on his desk when Noor Visser, the new Head of Engineering, asks him to walk her through the channel in March 2027.

The first is the **Black Friday Oversell**. On Friday 27 November 2026 the Ridgewalker jacket sold out at Duiven at 19:40, and the marketplaces kept selling it for up to two hours, because stock reached the connector by a chain of copies: the WMS (warehouse management system) told the ERP (enterprise resource planning system, the back office for finance and purchasing) every 15 minutes, the ERP told the platform hourly, the platform told the connector hourly. About **160** of the 1,120 canceled or delayed orders were marketplace orders, **140** of them canceled by Kestrel & Pine as the seller *(illustrative)*. Those seller cancellations counted against the marketplaces' cancellation standards, and bol issued a **strike** (its formal warning for a week below standard) for that week *(illustrative)*. The marketplace penalties are inside the €96,000 the incident cost.

The second is the **Deprecated Endpoint**. In February 2027 the connector retired an older version of its API on a date it had published months earlier. The agency script that pushed the platform's stock figures into the connector still used that version. Nobody had read the notice. For **two days** the connector kept sending bol the last figures it had, and bol sold from them.

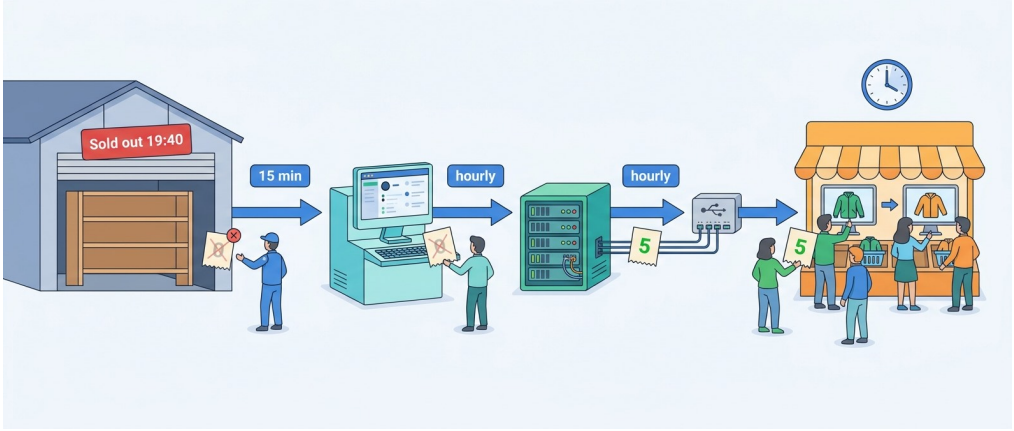


Figure 1: *The Black Friday Oversell: the stock figure reached the marketplaces as a copy of a copy, so the Ridgewalker jacket sold out at Duiven at 19:40 and the marketplaces kept selling it for up to two hours.*

Noor's questions are the chapter's questions. Which stock number should each marketplace be told, and who computes it? Where does a marketplace order live, so that support, the warehouse and finance see the same record? And who owns the API calendar of a vendor Kestrel & Pine pays €900 a month and depends on for €3.0 million?

What a Marketplace Is, and the Two Ways to Sell on One

Selling on a marketplace means publishing an **offer** against a product the marketplace knows: your price, your stock and your delivery promise for an item identified by its **GTIN** (Global Trade Item Number, the number behind the barcode; What Are We Selling? Products, Variants and the Catalog owns it). Several sellers can offer the same product, and the marketplace shows one of them in the **buy box**, the single "add to basket" button on the product page; performance and price decide who wins it.

Every large marketplace offers the same two logistics models under its own names.

Model	Who holds the stock	Who ships	Who handles returns	Amazon name	bol name	Zalando name
Seller-fulfilled	The merchant	The merchant, with its own carriers or the marketplace's labels	The merchant, under marketplace rules	FBM (Fulfilled by Merchant), "Merchant Fulfilled Network"	Own shipping, or "Shipping via bol"	"Own Logistics"
Marketplace-fulfilled	The marketplace's warehouse, stocked in advance	The marketplace	The marketplace	FBA (Fulfillment by Amazon)	"Logistics via bol" (LvB)	ZFS (Zalando Fulfillment Solutions); ZEOS (Zalando E-commerce Operating System) for multi-channel

The names are from the vendors' own pages (bol partner platform and Zalando partner university, n.d., accessed November 2026; Amazon's order performance policy, 2026). bol describes Logistics via bol as "We handle storage, shipping, and returns", counts stock daily and charges for it, with a 40% surcharge on stock costs in November and December, as of November 2026.

The architectural difference is larger than the operational one. In the seller-fulfilled model, a marketplace order must enter your order record and your warehouse flow with the marketplace's deadline attached, and your ship confirmation must travel back in time to count. In the marketplace-fulfilled model, your stock sits in **a location you do not control**: you learn its quantity from the marketplace's reports, its sales from the

marketplace's notifications, and its returns after the marketplace has processed them. Is It in Stock? Inventory, Availability and the Promise You Can Keep treats such stock as an external location with its own feed, and Kestrel & Pine's Waypoint decision W1 says the same.

The marketplace is also a competitor. The German competition authority's 2013 press release on Amazon's price-parity clauses states it plainly: Amazon, "as the largest online retailer, competes directly with" the sellers on its marketplace (Bundeskartellamt, 26 November 2013). Everything you send a marketplace is also market intelligence, and the design question is which attributes and which stock granularity leave the building.

What it costs

Fees differ by marketplace, category, price band and fulfillment model, and change on published dates. As of November 2026, Amazon's US schedule charges $39.99 a month for a professional account plus a referral fee of mostly 8–15% of the price (clothing 5% up to $15, 10% to $20, 17% above); bol charges no subscription but a fixed amount per item plus a percentage of the price including VAT, by category; Zalando publishes no figures. The lesson is not the numbers but their shape: **fees are dated reference data**, not constants. Margin per channel belongs in the analytics layer (Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics) with the fee schedule versioned by effective date, and the channel price floor (below) is computed from it.

What the Marketplace Measures You On

Marketplaces publish the metrics they grade sellers on. Amazon's European order performance policy (PDF dated May 2026) and its seller blog (October 2024) give these thresholds:

Metric	What it counts	Threshold	Window	Applies to
Order Defect Rate	Orders with negative feedback, an A-to-z Guarantee claim or a chargeback	Under 1%	60 days	All orders

Metric	What it counts	Threshold	Window	Applies to
Cancellation rate (pre-fulfillment)	Orders the seller canceled; customer cancellations excluded	Under 2.5%	7 days	Seller-fulfilled
Late shipment rate	Shipping confirmations completed after the expected ship date	Under 4%	10 or 30 days	Seller-fulfilled
Valid tracking rate	Shipments with a valid tracking number	Above 95%	30 days	Seller-fulfilled
On-time delivery rate	Tracked units delivered by the promised date	90% or higher expected on Amazon.de; consistently under 80% after a warning may lead to restrictions	Rolling 14 days	Seller-fulfilled

bol's partner pages (n.d., accessed November 2026) set an "Items on time" minimum of 90%, a cancellation standard of at most 2% a week (from three or more items canceled), and a **strike process**: a strike is a warning for a week below standard on items on time, cancellations or response time; strikes count over 22 weeks; at five the shop is temporarily closed pending an improvement plan; at seven the account is closed for at least a year. bol also cancels an unconfirmed order automatically on the third day after the promised delivery date, and counts that against the seller.

Read the table as an integration specification. **An oversell becomes a seller cancellation**, and a merchant with a few hundred marketplace orders a week needs only a handful to cross 2.5%. **A warehouse confirmation that reaches the marketplace a day late is a late shipment**, however promptly the parcel left. Each row maps to a flow in the next section and to a freshness requirement on it.

How the Pieces Fit Together

A **channel integration platform** (connector, integrator, "marketplace middleware") sits between the merchant's systems and many marketplaces and translates four kinds of data both ways. Vendors describe the same four functions: ChannelEngine calls itself "One platform for listings, orders, inventory, and pricing across 1,300+ channels" (n.d., accessed November 2026); Shopify's own Marketplace Connect app does the same for Amazon, Walmart, Target Plus and eBay from inside the platform (Shopify Help Center, accessed November 2026).

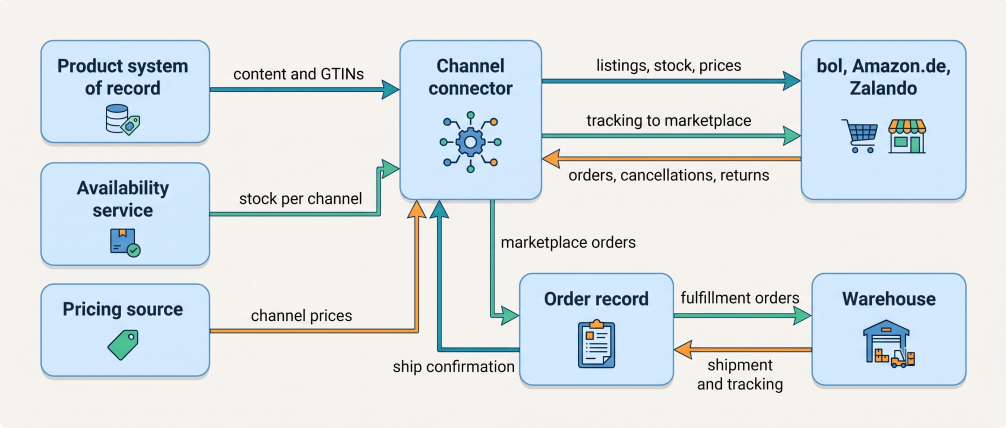


Figure 2: *The connector translates five flows between the merchant's systems of record and the marketplaces; it owns none of the data it carries.*

The five flows and what each needs:

Flow	Direction	How fresh	How consistent	What goes wrong when it is late
Listings (content, images, attributes, GTINs)	Merchant to marketplace	Hours to a day; seasonal	Must match the product system of record; incomplete content is rejected	A listing goes offline or shows last season's text

Flow	Direction	How fresh	How consistent	What goes wrong when it is late
Stock (quantity per offer)	Merchant to marketplace	Minutes; as events happen	One availability figure per channel, computed once	Oversell, seller cancellation, a strike
Prices	Merchant to marketplace	Minutes to hours	Derived from the pricing source, bounded by a floor	Price drift between channels, a parity warning
Orders (import, acknowledge, cancel)	Marketplace to merchant	Minutes; before the ship deadline	One order record, keyed on the marketplace order id, idempotent	Auto-cancellation, a duplicate shipment
Status back-flow (ship confirmation, tracking, returns)	Merchant to marketplace	Within the day of shipment	Carrier and tracking from the shipping service, once	Late shipment rate, invalid tracking rate

The two fast rows, stock and orders, are why Waypoint's W5 moves them by **events** (a message sent the moment something changes) rather than by periodic copies; APIs, Events, Files and Middleware: How Commerce Systems Talk picks a style per flow from requirements like these.

Order import, acknowledgement and back-flow

The marketplace order lifecycle has the same shape everywhere, and the connector's job is to keep two order records in step. The steps below are my synthesis of Amazon's, bol's and ChannelEngine's documented interfaces; marketplace state names differ.

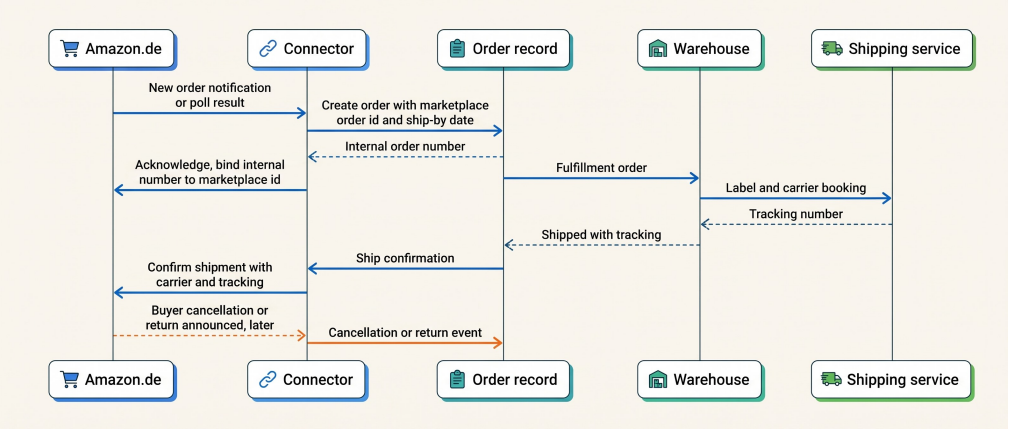


Figure 3: *A seller-fulfilled marketplace order: the order is created once in the merchant's record, acknowledged back with the binding between the two order numbers, and its shipment confirmed with tracking before the marketplace's deadline.*

Three points on this diagram carry the chapter.

Acknowledgement is a binding. ChannelEngine's documentation calls it "the critical step that links the ChannelEngine order to your internal system"; bol auto-cancels unacknowledged orders on day three after the promised delivery date. The acknowledgement ties the marketplace order id to your order number, and that pair is the key for everything after: cancellations, returns, disputes, payouts.

Delivery can be duplicated. Amazon's notifications push to a queue you own; you can also poll; a connector retries. All three can deliver the same order twice, so the receiving side must be **idempotent**: creating the same marketplace order a second time has no effect, because the order record enforces uniqueness on the marketplace order id (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation).

Ship confirmation is a deadline, not a courtesy. The late shipment rate is computed from the confirmation's timestamp, so the path WMS → order record → connector → marketplace must complete on the day of shipment. When Kestrel & Pine's WMS connector queue backed up for four hours on Black Friday 2026, every delayed confirmation was also a late shipment on a scorecard.

What the marketplace APIs actually impose

The marketplaces' own documentation shows that an integration cannot be designed as a tight loop of calls.

Amazon's Selling Partner API (SP-API), the only programmatic route since the older MWS (Marketplace Web Service) was retired on 1 April 2024, uses a token-bucket **rate limit** per operation: each call draws a token, tokens refill at a published rate, and a throttled request returns HTTP 429 and must be retried later. The Orders

API's `getOrders` has a rate of 0.0167 requests a second, one call a minute, with a burst of 20 (SP-API documentation, accessed November 2026). Amazon's guidance is explicit: "Don't hardcode timers" and "Respond to events rather than polling on a loop, for example, update prices on price notifications". Listings, prices and stock go up as **feeds**, asynchronous bulk uploads whose result you fetch later, so a change is *eventually* applied and the connector must track the feed's outcome. Customer names and addresses are **restricted data** that need a separate token, a signal that your order record should hold only the personal data fulfillment needs (Whose Data Is It? Privacy, Consent and Retention).

bol's Retailer API (as of November 2026) warns that "most calls are asynchronous; confirm the result afterwards", publishes its rate limits, and advises sending "only changes that need synchronizing" and "events as they happen instead of large batches". Its lifecycle policy: when a new major version is released, the previous one is deprecated and "scheduled for removal in 12 months". On 3 November 2026 bol's partner news announced that Offers API v10 stops on 1 February 2027.

That last sentence is the Deprecated Endpoint in public form. **A deprecation schedule is a contract clause with a date**, and someone at the merchant must own the calendar even when the connector vendor does most of the work; Promises Between Systems: API Contracts, Webhooks and Versioning owns the general discipline.

Channel Allocation: The Number You Send Is a Promise

The stock figure a connector sends to a marketplace is not a report. It is **the quantity that marketplace is now entitled to sell**, and the marketplace will sell it without asking again. Amazon even publishes a notification (`LISTINGS_ITEM_MFN_QUANTITY_CHANGE`) so a seller can see what Amazon currently believes its quantity to be.

Channel allocation is the decision of how much of your available stock each channel may promise. Connectors support several policies, and the choice follows the merchant's stage:

Policy	How it works	Fits	Breaks when
Shared pool	Every channel is told the full available quantity	One channel, or very deep stock	Two channels sell the last units in the same window; the Oversell

Policy	How it works	Fits	Breaks when
Fixed quantity per channel	Each marketplace gets a set allocation, replenished by hand	A few hero products, a small team	Stock sits unsold in one channel while another runs out
Buffer	Each channel is told available minus a safety margin; zero below it	Hourly or slower sync; the interim fix	Long tails: the last units never sell anywhere
Percentage split	Each channel gets a share of available stock	Channels with predictable demand ratios	Demand shifts (a promotion on one marketplace) and the split is stale
Computed available-to-promise per channel	One availability service computes what each channel may still sell, from stock events, reservations and channel rules, and publishes changes as they happen	Omnichannel and Group; several warehouses and external locations	Only if the events it depends on are late or missing

Figure 4: *The number you send is a promise: several channels draw on one stock, so one availability calculation decides how much each channel may still sell, instead of each channel reaching into the jar.*

The last row is what Is It in Stock? Inventory, Availability and the Promise You Can Keep builds and what Waypoint's W1 buys as a module of the OMS (order management system, the hub that holds every channel's orders). The principle behind it is simple: **compute available-to-promise once, from the system of record for physical stock, and publish it to every channel from that one computation.** A connector that holds its own stock copy and applies its own rules is a fourth system of record, which is what Kestrel & Pine has in January 2027: the connector keeps a copy of product data for listings and a copy of stock that is up to two hours old.

Marketplace-fulfilled stock adds a second rule. Units in a marketplace's warehouse are **an external location** with its own feed (Amazon's `FBA_INVENTORY_AVAILABILITY_CHANGES` notification exists for exactly this), never a copy of your warehouse figure; the availability service tracks the location, and the marketplace promises from it.

Kestrel & Pine after the Oversell

Milan Petrov's first fix, set in December 2026, was a buffer. The connector now publishes the platform's figure **minus ten units per variant** to each marketplace, and zero below ten *(illustrative)*. With a two-hour sync lag, ten units is enough for most variants; for the Ridgewalker on a Black Friday evening it would have held for minutes. The buffer has a cost: across roughly 17,000 active variants, the last few units of each are never offered on marketplaces at all.

The target, under W1, is this: the WMS is the system of record for stock at Duiven; the marketplace's fulfillment stock in Germany is a tracked external location; the

availability service computes available-to-promise per channel from stock events and publishes changes to the connector as they happen; the connector forwards them to each marketplace within minutes, as bol's rate-limit guidance asks. The hourly chain is retired by Q4 2027, and the buffer shrinks to what the event latency justifies.

A channel rule can then say, for example, "hold the last 20 units of each variant for the storefront, where the margin is highest" *(assumption)*: the connector sends the marketplaces zero while the storefront keeps selling. None of that is possible when the connector only knows "the platform said 410 an hour ago".

Prices across channels

The same discipline applies to prices. **Price parity** clauses, which required a seller's marketplace price to be no higher than its price elsewhere, were dropped by Amazon in the EU in 2013 after the German competition authority's proceedings (Bundeskartellamt, 26 November 2013; an old source, but the origin of today's rules). Marketplaces still compare prices to decide visibility: Amazon's `PRICING_HEALTH` notification tells a seller that an offer is "ineligible for the Featured Offer (Buy Box) because its price is uncompetitive", and bol's pages say good performance "improves your chance of appearing in the buy box".

Repricing is an automated rule that moves a listing's price within bounds in reaction to competitors; ChannelEngine sells it as "dynamic pricing, margin guardrails, and cost-based insights to adjust prices per channel" (n.d., accessed November 2026). The architectural rule: repricing acts on a **channel price** derived from the merchant's pricing source, never on the base price list, and it is bounded by a **floor** computed from cost and fees. For the Ridgewalker at €199 including 21% VAT, with an assumed 15% commission (the most common referral rate on Amazon's US pricing page; European rates differ) and €6 of shipping, about €128 remains against a landed cost of €58 *(assumptions)*, so the floor sits well above cost. A connector rule allowed below the floor can undercut the brand's own storefront, which is one way the Price Drift of October 2026 (€179 in Germany for nine days, and a parity warning from a marketplace) happens. What Does It Cost? Price Lists, Contract Prices, Currencies and Tax owns the channel price as a derived price with a source, an effective date and a floor.

The Choices

A merchant has four realistic ways to connect to marketplaces, and the order-record decision cuts across all four.

1. **The marketplace's own seller tools**. Upload listings by spreadsheet, read orders in the seller portal, key them in, confirm shipments by hand. Right at a few orders a day on one marketplace; wrong when a second marketplace arrives or the late-shipment clock starts biting.

2. **The platform's native channel app**. Hosted platforms ship or list channel apps (Shopify's Marketplace Connect and its social and Google channels, for example). Orders arrive in the platform's own order list and stock is the platform's stock. Cheap, quick and adequate at the Growing stage for the marketplaces the app covers; the reviews are the warning (Marketplace Connect's listing shows a 4.1 rating with 15% one-star reviews citing sync and pricing errors, accessed November 2026). The limit is structural: the app treats the platform as the system of record for stock and orders, true at stage 2 and false at stage 3.

3. **A channel integration platform**. A specialist SaaS (software as a service: rented, hosted software) product (ChannelEngine, EffectConnect, Rithum, Linnworks and others) with ready-made marketplace connections, listing mapping, allocation rules, repricing and order import into your platform, OMS or ERP. This is Kestrel & Pine's choice and the common one for Omnichannel merchants in Europe: tracking dozens of marketplaces' content rules and APIs is the vendor's business. The risk is the fourth-copy problem: the connector *can* hold your stock, prices and products, and will if you let it.

4. **Direct integration**. Build against the marketplaces' APIs yourself. Right for Group-stage merchants with the volume to justify engineers on marketplace APIs, or where no connector expresses a rule the business needs. Below that, you inherit every deprecation, feed quirk and rate limit, per marketplace.

Across all four runs the question **where the marketplace order lives**: either the connector (or app) creates the order in your order record and mirrors status back, or the order lives in the connector and your systems read it there. The first gives one order record with one state model; the second is cheaper at the Growing stage and produces the Three-Status Order the moment stores and EDI arrive. Kestrel & Pine's W2 chooses the first.

	Marketplace tools by hand	**Platform channel app**	**Channel integration platform**	**Direct integration**
Marketplaces covered	Any, one at a time	The few the app supports	Dozens to hundreds	Those you build
Who tracks API changes	Nobody	The platform vendor	The connector vendor; you own the connector's own versions	You

	Marketplace tools by hand	Platform channel app	Channel integration platform	Direct integration
Stock allocation	By hand	Platform stock, sometimes per-channel availability	Rules per channel; risk of a private stock copy	From your availability service
Order record	Keyed in	The platform's order list	Configurable: create in the OMS or hold in the connector	Your OMS
Cost	Staff time	Low subscription or per-order fee	Subscription (Kestrel & Pine: €900 a month)	Engineers
Fits	Starting	Growing, D2C-first (direct-to-consumer)	Omnichannel	Group, or a rule no vendor expresses

My recommendation for a merchant at the Omnichannel stage is the third option with two conditions written into the contract and the design: the connector creates orders in the order record and does not keep them; and the stock number it publishes comes from the merchant's availability calculation, not from a rule inside the connector. Both conditions assume the merchant has, or is getting, an order record and an availability service. Without them, the connector will own those facts by default, because something has to.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting	One marketplace, or none; seller portal and spreadsheets; orders keyed into the shop; one fulfillment model	A decision whether the marketplace is a test or a channel; GTINs on every variant before listing	More than an hour a day copying orders; the first late-shipment warning
2 — Growing	Two or three marketplaces through the platform's channel app or a connector; the platform's stock is the figure sent everywhere; orders mirrored into the platform	A buffer per channel; one owner for the seller account and its scorecard; the deprecation calendar on someone's list	A second order source (a store or wholesale) makes the platform's order list incomplete; a strike or a restricted category
3 — Omnichannel	A channel integration platform over several marketplaces and countries; both fulfillment models; marketplace-held stock	Available-to-promise per channel from an availability service; marketplace orders created in the OMS; channel prices with floors; ship confirmation by event from the WMS; reconciliation of connector stock against the system of record	Oversells at peak; the connector holds the only copy of marketplace orders; nobody can say what a marketplace believes the stock is

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
4 — Group	Many marketplaces per brand and country; direct integrations where volume justifies; a marketplace as logistics provider for the brand's own shop; perhaps operating a marketplace	A channel syndication service over the PIM (product information management system) with reduced, channel-specific views; fee schedules and performance data in the data platform; seller-identity obligations if operating a marketplace	Each brand runs its own connector with its own rules; margin per channel unknown; a marketplace program change forces a re-architecture

The judgement: the mechanism scales with the number of *order sources*, not with order volume. One marketplace feeding the platform's order list works at stage 2 at any volume the platform handles. The moment a store, a wholesale portal or a second marketplace also creates orders, the platform's order list is no longer the order record, and the marketplace integration must be re-pointed at whatever is. Kestrel & Pine crossed that line in 2023 and noticed in December 2026.

In Practice

Marketplaces retire interfaces on their own calendar. Amazon's older MWS interface was no longer available from 1 April 2024, after a migration schedule that had already moved once (SP-API changelog, 2023–2024); bol's Offers API v10 stops on 1 February 2027. Programs retire too: Zalando announced in April 2026 that Connected Retail, under which "stationary retailers sold branch stock directly through the platform and shipped orders from their stores", ends by the end of June 2027 (TextilWirtschaft, 24 April 2026). A merchant whose store-stock exposure was built on Connected Retail's tool rebuilds it; one whose store stock is a location in its own availability service changes a connector (Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment).

The marketplace is becoming infrastructure. Zalando's ZEOS, announced on 17 October 2023 as "one stock pool, one carrier integration and one user interface", was by December 2025 available in 24 markets and serving 18 sales channels, including brands'

own web shops (Zalando corporate releases, 2023 and 2025). For a merchant that means a third kind of stock location, marketplace-held but multi-channel, and a dependency on the marketplace's APIs for its *own* storefront's orders.

Social channels switch between feed and order source. Meta moved Facebook and Instagram Shops from in-app checkout to website checkout in 2025 (trade press, June 2025), so for most merchants it is again a **shopping feed** destination: a structured product file with title, price, availability, GTIN and image. TikTok Shop went the other way, opening in-app checkout in Germany, France and Italy on 31 March 2025 (widely reported) and in the Netherlands, Belgium, Austria and Poland from 15 June 2026 (TikTok newsroom, 28 May 2026). Treat every channel as potentially both: syndicate the catalog from the product system of record, and route any orders through the same import as marketplace orders. **Retail media** (sponsored listings on a marketplace) is a per-channel cost line and a data stream for the data platform; it does not touch the order flow.

Operating a marketplace is a different book. Mirakl sells the operator side (seller onboarding, per-seller offers, order splitting, payouts) to over 450 companies (Mirakl and Wikipedia, accessed November 2026). The Digital Services Act adds the legal core: since 17 February 2024, Article 30 requires platforms that let consumers buy from traders to obtain and verify the trader's identity, payment account and register details before they can sell, and to suspend traders who do not supply them (Regulation (EU) 2022/2065); that is why every marketplace onboarding asks a merchant for identity documents and bank details. The operator's architecture is the subject of LISTED; start with Rules of the Marketplace: DSA, DAC7 and Vertical Regulation and Liquidity Is the Product: Network Effects, Local Markets and Marketplace Metrics.

Kestrel & Pine: the marketplace variant, 13 April 2027

Two Amazon.de customers order a Ridgewalker jacket on the day of Eva's order. Here is each order's path in April 2027, before W1 and W2 land, and what changes after.

Step	Seller-fulfilled order (shipped from Duiven)	Marketplace-fulfilled order (shipped by Amazon)
Where the order appears	The connector pulls it from Amazon and creates a connector order; the iPaaS (integration platform as a service, which runs Kestrel & Pine's data flows) passes it to the WMS; it reaches the ERP only in the daily finance batch	The connector sees it in Amazon's reports; no warehouse work; it reaches the ERP only in the finance batch
Stock effect	Duiven stock falls when the WMS picks; the ERP learns within 15 minutes, the platform within an hour, the connector within two	The German external-location quantity falls by one; Kestrel & Pine learns it from the marketplace's inventory feed
Shipment	Picked and packed 14 April, DHL label from the shipping service; tracking flows WMS → iPaaS → connector → Amazon the same day	Amazon ships and sends tracking to the customer; Kestrel & Pine is told after the fact
Scorecard	Cancellation, late shipment, valid tracking and on-time delivery all apply	Only the order defect rate applies
Return	Announced by Amazon, acknowledged by the connector, received at Duiven	Handled by Amazon from the external location
After W1 and W2	Created in the OMS by the connector with the Amazon order id and ship-by date; available-to-promise reduced by event; ship confirmation by event from the WMS	Created in the OMS as a marketplace-fulfilled order for finance and analytics; the external location adjusted by event; no fulfillment order

The second column is the one most merchants get wrong: because the marketplace does all the work, the order never enters the order record, and revenue, returns and the external location's stock are reconstructed from the marketplace's statements at month-end. Hanna Berg's three definitions of "order" (W7) are partly this.

When It Goes Wrong

Failure	How it shows	Detection	Recovery
Duplicate order delivery (notification and poll both deliver it; the connector retries)	Two internal orders, two parcels, one payment	Uniqueness on the marketplace order id at creation; a daily count of marketplace orders by id against internal orders	Cancel the duplicate before pick; if shipped, treat as a return; make the import idempotent
Stale stock (sync failure, a retired API version, a feed rejected)	The marketplace keeps selling a sold-out variant; the cancellation rate rises	An **age monitor** per channel: alert when the last accepted stock update is older than the agreed interval; read every feed result (Amazon keeps them 28 days); compare what the marketplace believes with available-to-promise	Push a full stock snapshot; fix source data in the product system of record, not in the connector; cancel oversold orders early
Late ship confirmation (warehouse queue, iPaaS outage)	The late shipment rate climbs though parcels left on time	Compare carrier scan time with confirmation time daily	Send confirmations by event from the WMS; backfill from the shipping service's records

Failure	How it shows	Detection	Recovery
External-location drift (marketplace-fulfilled stock)	Your figure and the marketplace's differ; replenishment is planned on the wrong number	Weekly reconciliation of the external location against the marketplace's inventory report	Adopt the marketplace's figure as the location's truth and investigate the difference
Return announced by the marketplace, unknown to the returns portal	The unit arrives at Duiven with no record; the marketplace refunds while a second refund is prepared internally	Every marketplace return acknowledged into the order record with the marketplace return id	One refund path per channel; The Way Back: Returns, Exchanges, Refunds and Reverse Logistics
A retired API version	Everything stops, quietly, on a published date	The deprecation calendar in the integration inventory with an owner per entry; the age monitor above	Upgrade; meanwhile a manual stock snapshot through the seller portal

The **age monitor** is the cheapest control in the chapter: for each channel, the time since the marketplace last accepted a stock update. It would have caught the Deprecated Endpoint within the hour instead of after two days. **Early cancellation** runs against instinct: once an oversell is certain, canceling at once costs a cancellation-rate point, while waiting for the stock to "turn up" costs a late shipment and often a defect as well.

What to Take Away

Established.

- Marketplaces grade sellers on published performance metrics with consequences; Amazon's thresholds (cancellation rate under 2.5% over 7 days, late shipment rate under 4%, valid tracking above 95%, order defect rate under 1%) and bol's standards and strike process (at most 2% cancellations a week; five strikes close the shop) are documented as of 2026.
- Marketplace APIs are rate-limited and asynchronous, and versions retire on published dates: Amazon's `getOrders` allows one call a minute and Amazon

recommends notifications over polling; bol removes deprecated major versions after 12 months. An integration must be event-driven, idempotent and owned.
- The two fulfillment models are architecturally different: seller-fulfilled orders enter the merchant's order and warehouse flow with a deadline; marketplace-fulfilled stock is a location the merchant does not control and learns about from the marketplace's feeds.

Depends on context.

- A platform's native channel app is enough while the platform is the only source of orders and holds the only stock figure (Starting and Growing stages). Once a store, wholesale or a second system creates orders, marketplace orders must be created in whatever the order record is.
- A stock buffer per channel is the right interim allocation when sync is hourly; computed available-to-promise per channel is right when stock moves by events and several channels and locations compete. The buffer's cost (units never offered) grows with the catalog's long tail.
- Direct marketplace integrations are right only with the volume to keep engineers on the marketplaces' APIs, or a rule no connector expresses; below that, rent the connector and own the contract.

The author's synthesis.

- The connector's job is translation, and the one design rule I insist on is that it owns nothing: not the product content, not the stock figure, not the order. Every incident in this chapter is a connector that was allowed to own a copy because nothing else did.
- Treat the marketplace scorecard as an integration SLA (service-level agreement) you did not negotiate: derive a freshness requirement per flow from the metrics, and monitor the age of the last accepted update per channel before anything else.

Questions to Ask

1. Which system computes the stock figure each marketplace is told, and does the connector apply any rule of its own on top of it?
2. Where is a marketplace order created, and can support, the warehouse and finance find it by the marketplace order id?
3. Who owns the seller account's scorecard, and who sees the cancellation and late shipment rates weekly?
4. How long does a ship confirmation take to reach the marketplace, and what happens when the iPaaS is down?

5. For marketplace-fulfilled stock, which system holds the external location's quantity, and how often is it reconciled against the marketplace's report?
6. Who holds the deprecation calendar for the connector's own API and for each marketplace's API, and when is the next retirement date?
7. Is the price sent to each marketplace derived from the pricing source with a floor, and can a repricing rule go below it?
8. If a marketplace closed our account tomorrow, what revenue and which stock would be stranded?

Marketplaces are one kind of order that arrives from outside the shop. The next chapter turns to the other kind Kestrel & Pine lives on: the 1,100 wholesale accounts whose buyers are companies, with contract prices, payment terms and credit limits that no consumer checkout knows how to express. Continue with Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.

Sources and Evidence

Company, product and investor sources

- Usage plans and rate limits, Amazon SP-API documentation, n.d., accessed 2026-11 — token-bucket rate limiting, HTTP 429, "Respond to events rather than polling".
- Orders API rate limits and Orders API, Amazon, n.d., accessed 2026-11 — `getOrders` at 0.0167 requests a second; v0 deprecated, v2026-01-01 current.
- Notifications API and Notification type values, Amazon, n.d., accessed 2026-11 — order change, pricing health, listing quantity and FBA inventory notifications.
- Feeds API and Tokens API use case guide, Amazon, n.d., accessed 2026-11 — asynchronous feeds, 28-day result retention, Restricted Data Tokens for personal data.
- Changes to Order Performance Program Policy (DE), Amazon, 2026 — performance metric definitions, thresholds and windows.
- Amazon seller policies to know, Amazon, 2024 — ODR under 1%, late shipment under 4%, cancellation under 2.5%.
- Pricing (US), Amazon, n.d., accessed 2026-11 — subscription and referral fees by category and price band.
- bol partner platform news, bol, 2026-11-03 — Offers API v10 stops on 1 February 2027.
- API lifecycle and Rate limiting, bol Retailer API, n.d., accessed 2026-11 — 12-month deprecation, asynchronous calls, "events as they happen".
- Commission, Items on time, Cancellations, Strike process and Conditions and rates LvB, bol, n.d., accessed 2026-11 — fee model, service standards, strikes, fulfillment surcharges.

- Zalando logistics solutions: an overview, Zalando Partner University, n.d., accessed 2026-11 — ZFS, Own Logistics and ZEOS terms.
- Productizing Zalando's capabilities: introducing ZEOS, Zalando SE, 2023, and The evolution of our B2B growth engine, Zalando SE, 2025 — ZEOS launch; 24 markets, 18 channels, over 60 merchants.
- Zalando stellt Connected Retail ein, TextilWirtschaft, 2026 — Connected Retail ends by June 2027.
- ChannelEngine, EffectConnect, Rithum and Linnworks, vendor pages, n.d., accessed 2026-11 — the four connector functions and channel counts; Merchant API: orders, ChannelEngine support, n.d. — acknowledgement "links the ChannelEngine order to your internal system".
- Marketplace Connect and app listing, Shopify, n.d., accessed 2026-11 — supported marketplaces, pricing, review profile.
- Kaufland Global Marketplace conditions and OTTO Market, vendor pages, n.d., accessed 2026-11 — subscription and commission figures.
- TikTok Shop expands across Europe, TikTok, 2026 — launch in the Netherlands, Belgium, Austria and Poland from 15 June 2026.
- Meta phases out Facebook and Instagram Shops checkout, PPC Land, 2025 — move to website checkout (trade press).
- Mirakl, vendor page, n.d., accessed 2026-11, and Mirakl, Wikipedia, accessed 2026-11 — marketplace operator platform.

Standards, regulation and official documentation

- Regulation (EU) 2022/2065 (Digital Services Act), Publications Office of the EU, 2022 — Article 30 trader traceability; application from 17 February 2024.
- Amazon abandons price parity clauses for good, Bundeskartellamt, 2013 — end of EU price-parity clauses; Amazon as competitor to its sellers (evidence age).

Research and practitioner sources

- Important: Amazon MWS removal dates, Amazon SP-API changelog, 2023–2024 — MWS no longer available from 1 April 2024 (page has since moved).

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

21. Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals

IN THIS CHAPTER, YOU WILL: Learn what is different when the buyer is a company, the objects every B2B platform shares, who should own accounts, prices and credit, and the choices for building a wholesale channel.

KEY POINTS:

- **The customer stops being flat.** A business buyer is a company with locations, several people and roles, an assortment, a price agreement, payment terms and a credit limit. Shopify, commercetools, SAP, Adobe and BigCommerce all model it that way, under different names, and the model must have one owner.
- **A B2B price is a promise with a validity date.** List, tiered, contract and quoted prices coexist and must say which wins; a quote freezes a price, and the invoice must match what the portal showed, or the wholesale team keys orders in by hand to make it so.

- **Credit and terms are finance facts.** Payment terms and credit limits usually belong to the ERP (enterprise resource planning system, the back office for finance, purchasing and invoicing), which is why a portal that cannot read them at checkout cannot replace e-mail and phone; the order record, not the ERP's order entry, is where a wholesale order should live.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Price Nobody Can Explain** — the portal shows one wholesale price, the ERP invoices another, and the negotiated pre-season discount lives in an e-mail.
- **Schrödinger's Order** — portal orders live in the platform, keyed orders in the ERP, EDI (electronic data interchange) orders in the EDI provider's mailbox; one customer, three order lists.
- **The Spaghetti Junction** — each large buyer asks for its own way in (a portal login, a punchout session, an EDI mailbox, a spreadsheet by e-mail), each with its own connector.

When Eva Janssen buys a jacket, she is one person with one address who pays before the parcel leaves. When an outdoor shop in Groningen buys sixty of them, the buyer is a company: an owner who negotiates the season's terms, a buyer who places the order, a bookkeeper who pays the invoice thirty days later, two shops that each want their own delivery, and a seller that has, in effect, lent the shop the money in between. **B2B commerce** (business-to-business: selling to companies that resell or use what they buy) is the same products sold under a different set of promises.

The promises are richer. A consumer sees one price; a business customer sees *its* price, agreed for the season, with quantity breaks, and expects the invoice to match it to the cent. A consumer pays at checkout; a business customer buys on **payment terms** (an agreed number of days to pay after the invoice) within a **credit limit** (the most the seller will let it owe at once). A consumer buys a jacket; a business buys cartons, in size runs, for delivery before the season starts, and may need a manager's approval above an amount.

Most commerce platforms were built for the consumer case and have added the business case since, with varying depth. The question this chapter answers is not which platform has the longest B2B feature list but **which system owns each of these new facts** (the company, its people, its prices, its terms, its credit, its orders) and how the systems that do not own them read them in time to make a promise at checkout.

Where we are in the order's journey. Eva's jacket is a consumer order and never passes through the wholesale channel. Its wholesale sibling, Bergzicht Outdoor's spring reorder of 6 April 2027, arrives by EDI and is worked in Journey 3: A Chain Reorders by EDI, From Order to Invoice and Documents Between Companies: EDI, Punchout and

E-Invoicing. This chapter covers the orders in between: the independent shops that buy through the B2B portal or, still, by e-mail, and the account, price and credit facts those orders depend on.

Setting the Scene at Kestrel & Pine: The 38%

Kestrel & Pine, the fictional outdoor brand this book follows (Meet Kestrel & Pine), sells to **1,100 active wholesale accounts** (640 in the Netherlands, 210 in Belgium, 250 in Germany), mostly independent outdoor shops, plus one retail chain, Bergzicht Outdoor, that orders by EDI. Wholesale brought **€12.0 million** in 2026, a quarter of revenue, in **9,400 orders** averaging **€1,277**, on 30-day terms. Lea Moreau, Head of Wholesale, runs it.

Since 2023 the wholesale shops have had a **B2B portal**: the platform's own business features, with company accounts, price lists and net terms, switched on in the same tenant (the one platform account that hosts the storefronts) as the consumer storefronts. **62%** of wholesale orders arrive through it. The other **38%** arrive by e-mail or phone and are keyed into **the ERP** by the wholesale team, because the ERP, not the platform, is where Sanne de Wit's finance team manages customer terms, credit limits and invoices.

When Noor Visser asks Lea in February 2027 why the 38% has not fallen since 2024, the answer is not that shops dislike the portal. It is that the portal cannot express what those shops have agreed *(illustrative)*:

- The portal shows one **wholesale price list per country**. The accounts that order by e-mail are mostly the larger ones, whose **pre-season terms** (an extra discount on orders placed before the season, agreed per account) live in the ERP's customer price lists and in Lea's e-mail, not in the platform.
- The portal cannot see **credit**. A shop near its limit can place a portal order that finance then holds, by hand, a day later; the team keys larger orders into the ERP because the ERP checks the limit at entry.
- A portal order and a keyed order are **two different records**: one in the platform, one only in the ERP. To see one shop's orders, Lea opens both.

That is the chapter's problem in one company. The portal is good at what the platform owns (the catalog, the cart, the checkout) and weak at what the ERP owns (terms, credit, negotiated prices, invoices), and neither holds the order record for the whole channel. Waypoint's answer, W2, is that every wholesale order, portal, keyed or EDI, is created in the OMS (order management system, the one record of every channel's orders) from Q4 2027, and the ERP keeps finance.

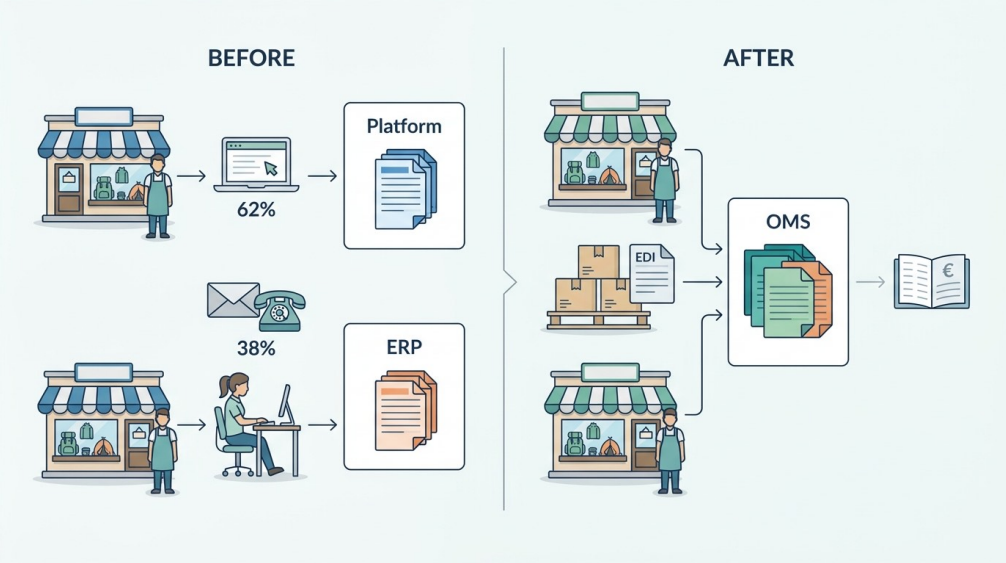


Figure 1: *Today 62% of Kestrel & Pine's wholesale orders arrive through the portal and 38% are keyed into the ERP, so one shop's orders live in two records; under W2 every wholesale order is created once in the OMS.*

What Is Different When the Buyer Is a Company

The contrasts below are industry practice; the vocabulary is defined as it appears.

	Consumer (B2C)	Business (B2B)
Who buys	One person, one account	A **company account**: an organization with several locations, several users and roles (who may order, who may approve, who may see invoices)
Price	One list price, promotions on top	**Contract pricing** (a price agreed with one customer or group), **tiered pricing** (price breaks by quantity), customer-specific assortments; prices quoted **excluding VAT**

	Consumer (B2C)	Business (B2B)
How the price is agreed	Fixed	Often negotiated; a **quote** (the seller's offer at a stated price with a validity date) that the buyer accepts into an order
Payment	At checkout, before shipment	On **payment terms** (net 30: due 30 days after the invoice) within a **credit limit**; sometimes card or direct debit
Order size and units	One or two items	Dozens to hundreds of units in cartons, inner packs and size runs; sometimes pallets
When orders are placed	Any time	In apparel, a **pre-season order** months ahead for delivery in a window, then **in-season reorders** of what sells
Who may commit	The buyer	The buyer, subject to the buyer's own **approval workflow** (a manager must approve above a limit)
Document that matters	The order confirmation	The **invoice**, matched by the buyer against its purchase order and the delivery

Each row introduces a fact that has to live somewhere and be readable by the checkout at the moment of the promise. Three of them, **the company account**, **the price and the credit limit**, are the ones the ERP and the platform fight over.

The vocabulary the platforms share

Whatever the vendor, the documented object models converge on the same shapes. The table is my synthesis of the vendors' documentation as of November 2026; a blank cell means "not verified", not "absent".

Concept	Shopify	commercetools	SAP Commerce (Spartacus docs)	Adobe Commerce B2B	BigCommerce B2B Edition	OroCommerce
The buying organization	Company, Company-Location, Company-Contact	Business Unit (Company and Divisions, up to five levels) with Associates	B2BUnit tree with B2BUser	Company account, company hierarchies	Company account with users	Customer, customer group, customer user
Who may do what	Contact permissions (ordering only, location admin)	Associate Roles	Roles including approver and administrator; purchase limits	Company roles; administrators bypass rules	Roles controlling Buyer Portal access	Customer user roles
Customer-specific prices and assortment	Catalogs (price lists plus product sets) assigned to company locations	Stores and prices per Business Unit	Cost centers, budgets	Shared catalogs with custom pricing	Price lists per company	Price lists with scheduled adjustments

Concept	Shopify	commercetools	SAP Commerce (Spartacus docs)	Adobe Commerce B2B	BigCommerce B2B Edition	OroCommerce
Negotiation	Draft orders for review and approval	Quote Request, Staged Quote, Quote	Quotes	Quotes	Sales quotes	RFQ (request for quote) to Quote, with approval workflow
Spending control	— (apps)	Approval Rules and Approval Flows	Order approval, budgets, purchase limits	Purchase orders with approval rules	—	Quote approval workflows
Pay later	Payment terms per company location (net, fixed date, on receipt, on fulfillment)	— (payment integrations)	—	Payment on Account	Net terms and available credit	—

Shopify's developer documentation defines a Company as "Information about the business entity that makes a B2B purchase", a CompanyLocation as "A single location or branch of the company" to which catalogs, tax exemptions and payment terms are assigned, and a CompanyContact as "A person that acts on behalf of the company"

(shopify.dev, accessed November 2026). commercetools says "Business Units are representations of Companies or Divisions", nesting up to five levels, with Associates whose roles "grant Associates different permissions" (commercetools documentation, accessed November 2026). The structure is the same: **organization**, **places**, **people**, **permissions**, with prices and terms attached to the organization or the place, not to the person.

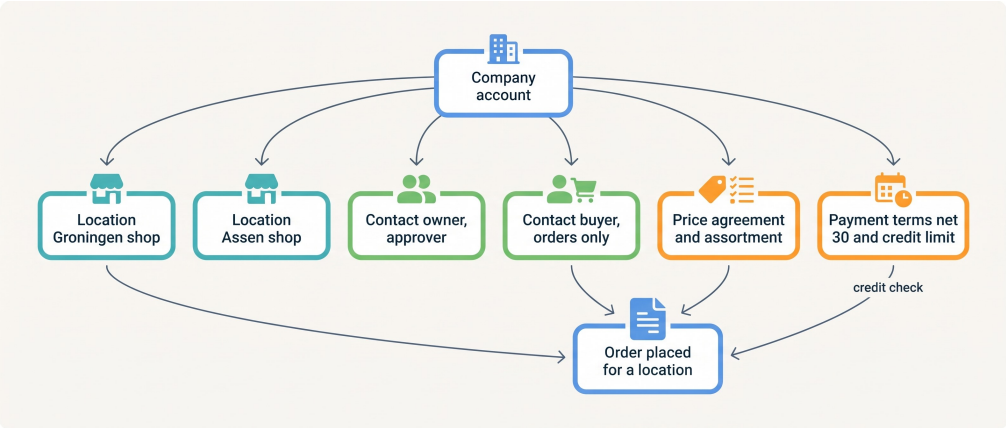


Figure 2: *The B2B object model every platform shares: an organization with locations and people, with prices and terms attached to the organization, and an order that draws on all of them at checkout.*

Who Owns What

The company account is the first place the customer record stops being flat, and the first place two systems claim it. The commerce platform needs the company, its contacts and its catalog to run the portal. The ERP needs the customer, its terms and its credit to invoice and to manage receivables. Both can hold all of it; neither should.

Microsoft's documentation for its Business Central to Shopify connector states the problem and the usual answer without hedging: merchants must "decide which system is the system of record for customer data. Most businesses manage customer records, payment terms, and credit limits in Business Central and export them to Shopify … Mixing both directions without a clear ownership rule can lead to conflicting records" (Microsoft Learn, 14 October 2026). The connector imports company locations with payment terms and tax IDs and can create a B2B catalog per exported company so that portal prices follow the ERP customer's price group. Shopify's own 2026 guidance on ERP integration says the same in general: "Document every field that moves between systems, and name the system of record for each" (Shopify enterprise blog, updated 28 August 2026).

My ownership table for a merchant with an ERP, a commerce platform with B2B features and, at the Omnichannel stage, an OMS:

Fact	System of record	Readers	Why there
Company identity, legal name, VAT number, locations	The ERP (or a master-data system at Group stage)	Platform, OMS, invoicing, EDI	The ERP must know the customer before it can invoice; the VAT number drives tax treatment
Contacts, logins, roles, permissions	The platform (or an identity service)	Portal, service tools	Logins and sessions are the platform's job
List and tiered wholesale prices	The pricing source: ERP price lists or a pricing engine (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax)	Platform, OMS, EDI	One source, published to channels; the platform's catalogs are a copy with an effective date
Contract and pre-season agreements	The same pricing source, as customer-specific price lists	Platform, OMS	If they live only in e-mail and the ERP's order entry, the portal cannot show them, and the 38% stays
Quotes	The platform or OMS, as an explicit object with a validity date	Portal, sales reps, OMS	A quote is a frozen price; the order is created from it
Payment terms, credit limit, open balance	The ERP (receivables)	Platform or OMS at checkout, by a credit check	Finance facts; the ERP knows what is owed

Fact	System of record	Readers	Why there
The customer's own approval rules	The platform or OMS	Portal	The buyer's internal control; invisible to the ERP
Orders (portal, keyed, EDI, punchout)	The OMS at stage 3 and beyond; the platform at stages 1 and 2; never only the ERP's order entry	ERP (for posting), WMS (warehouse management system), service	One order record with one state model (The Order Record: Lifecycle, State and the Order Management System)
Invoices, credit notes	The ERP	Peppol (the European e-invoice network) or EDI gateway, portal (for display)	Numbering, archive and legal weight live here (Tax, Invoices and the Books: From Order to Ledger)

The pattern is that **the ERP owns the money facts and the platform owns the session facts**, with prices sourced once and published. For wholesale and contract prices that source is the ERP: it publishes them as price events with effective dates to the OMS and the platform's B2B catalogs, and feeds no channel directly, so no storefront reads the ERP live. It is the split SAP describes for its own suite: master data replicated from the ERP to the commerce system, storefront orders sent to the ERP (SAP Learning, accessed November 2026). The Back Office: ERP Integration and Who Is the Hub works out when the ERP should also be the live source at checkout and when it should not.

What the flows need

Interaction	Freshness	Consistency	Latency at checkout	When it fails
Price lookup for a company	Effective-dated; changes published hours ahead	The portal price must equal the invoice price for the same date and quantity	Precomputed per catalog; millisec-onds	Recompute at invoicing and hope: The Price Nobody Can Explain
Credit check at order placement	Open balance as of today, including unposted shipments	One answer from the receivables system	Under a second if syn-chronous *(assump-tion)*; otherwise place the order and hold it	Nightly copy of the balance: an order accepted at 09:00 against yesterday's balance
Order creation	Immediate	One record, one number, idempotent on the buyer's purchase order number	Seconds	Keyed duplicates of e-mailed orders
Invoice on shipment	Same day as shipment	Generated from what shipped at the price the order carried	Batch	Invoice from the order rather than the shipment: short shipments invoiced in full

The credit-check row decides whether the portal can replace e-mail. If the ERP's open balance is copied to the platform nightly, the portal is a day behind and finance keeps holding orders by hand. If the platform or OMS asks the ERP at order time, or receives balance changes as events, the portal can say "this order needs approval from Kestrel & Pine" at once. Stock for wholesale follows the same logic: it is promised from available-to-promise (stock not yet promised to another order) with a wholesale allocation, so that wholesale and consumer channels do not sell the same units (Is It in Stock? Inventory,

Availability and the Promise You Can Keep).

How a Price Becomes a Promise

B2B pricing is layered, and every layer must say whether it beats or adds to the one below.

1. **List wholesale price**: the price in the seasonal wholesale price list, excluding VAT, per country or currency. Kestrel & Pine's Ridgewalker is €95, the Trailhead fleece €57, merino socks €8.50.
2. **Tiered or volume price**: breaks by quantity. Shopify's catalogs support "discounted price breaks when a customer buys a set quantity" and quantity rules per order (Shopify Help Center, accessed November 2026).
3. **Contract or customer-group price**: a price agreed with one account or group, usually a percentage off list or a fixed price per product, with an effective period. Shopify expresses it as catalogs assigned to company locations; Adobe as shared catalogs "with custom pricing for different companies" (Adobe B2B guide, 1 October 2026); OroCommerce as price lists with scheduled adjustments (OroCommerce documentation, version 7.0).
4. **Quote**: a one-off price for a specific request, with a validity date.

The rule that matters is which layer wins. Commonly the most specific wins (quote over contract over tier over list), and discounts do not stack unless the agreement says so. Write it down and implement it once in the pricing source, so the portal, the OMS and the invoice compute from the same rule; a portal that applies the rule its own way produces the invoice dispute.

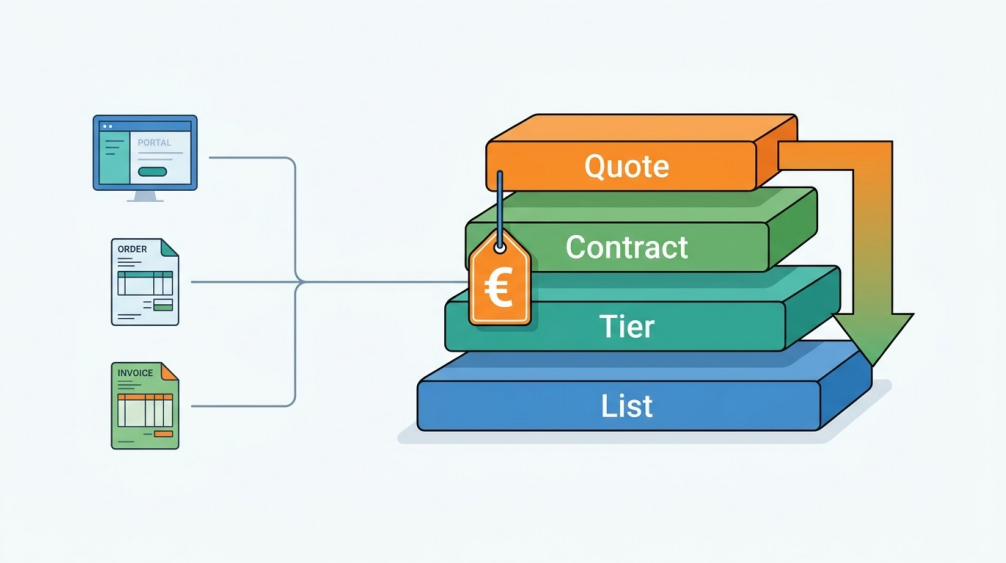


Figure 3: *B2B prices are layered: the most specific layer wins, and the portal, the order and the invoice must all compute the price from that one rule.*

A quote freezes the price. commercetools models negotiation as three objects, Quote Request, Staged Quote (with a `validTo` date) and Quote, and says "Quotes are legally binding custom offers presented to a Buyer" (commercetools documentation, accessed November 2026). Adobe's purchase-order flow states the consequence for the order built from one: "If the price of an item changes after the PO is created, the original price is used" (Adobe Commerce documentation, accessed November 2026). On Shopify, no native "request a quote" page was found in November 2026; the documented mechanism is the **draft order** a merchant creates "for contacts to review and approve", into which third-party quote apps convert accepted quotes. Under every name the decision is the same: the moment a seller states a price to a named buyer with a validity date, that price is stored as an object the order is created from, not recomputed later from whatever the price list then says.

VAT follows the buyer. B2B prices are shown and invoiced excluding VAT, and the treatment depends on who the buyer is and where: a Dutch shop is invoiced with Dutch VAT; a German or Belgian business customer with a valid VAT identification number is commonly invoiced without Dutch VAT under the EU rules for cross-border business supplies, and accounts for the tax itself. Tax, Invoices and the Books: From Order to Ledger owns the rules; this chapter only insists that the VAT number is master data with an owner and a validation date, not a free-text field on the portal signup form.

Credit, Terms and the Buyer's Own Controls

Payment terms are typed, not free text. Shopify's Admin API (version 2026-10) models them as `NET` ("Payment is due a set number of days after the invoice is issued"), `FIXED` (a calendar date), `RECEIPT`, `FULFILLMENT` and `UNKNOWN`, with templates carrying the number of days. The due date is then computed from the invoice date, and the invoice date is the ERP's. This is why terms belong to the ERP and are *mirrored* to the platform: the platform needs them to show "net 30" and skip card payment at checkout; the ERP needs them to age receivables and chase payment.

The credit limit is the seller's control: the most a customer may owe. A credit check at order time compares the open balance (unpaid invoices, plus shipped orders not yet invoiced, plus accepted orders not yet shipped, depending on policy) with the limit, and accepts, holds for approval or rejects. BigCommerce's B2B Edition documents "available credit, and net terms" in payment settings (accessed November 2026); Microsoft's connector can carry credit limits to Shopify as metafields (custom fields), which is a mirror, not a check.

The buyer's approval workflow is the opposite control: the buyer's own rule that an order above an amount needs a manager. commercetools' Approval Rules are tiered ("Each tier contains one or more roles that are required to approve an Order before it can proceed to the next tier"), inherited down the Business Unit hierarchy, and "a single instance of rejection will lead to the rejection of the entire Approval Flow". SAP Commerce's organization model adds cost centers, budgets and purchase limits (Spartacus documentation, which describes an unmaintained storefront version); Adobe lets company administrators "bypass all approval rules". For a brand selling to independent shops, approval workflows are rarely needed; for one selling to a chain's 38 branches, or to corporate buyers, they are the difference between an order and a request.

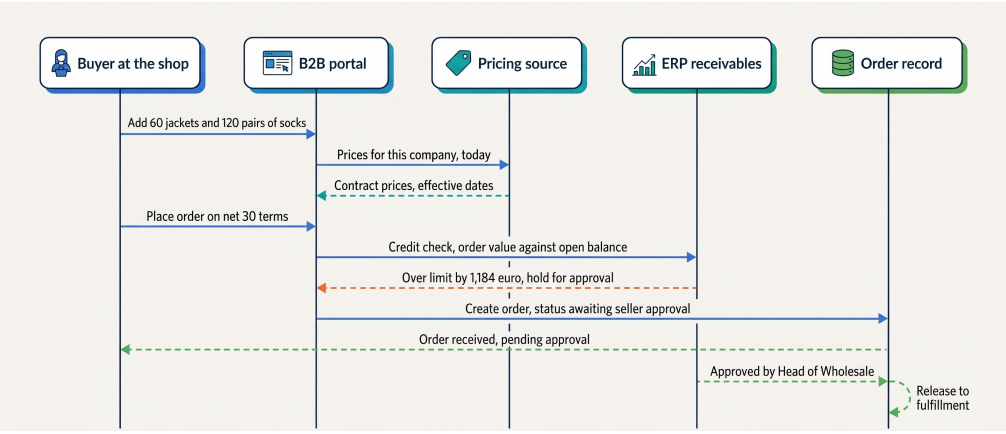


Figure 4: *A portal order at the Omnichannel stage: prices come from the pricing source,*

the credit decision from the ERP, and the order is created once in the order record, where the approval and the release are states, not e-mails.

The Choices

A merchant that sells to businesses has four realistic ways to give them a channel, and the choice depends less on features than on who the buyers are and which system is fast enough to answer at checkout.

1. **The B2C platform's B2B features.** Shopify's B2B runs "in one store or use a separate B2B-only store", on the Basic, Grow, Advanced and Plus plans (Shopify Help Center, accessed November 2026; the expansion beyond Plus is recent and undated, and plans below Plus allow at most three active catalog assignments). Adobe Commerce ships company accounts, shared catalogs, quick order, purchase orders, quotes and requisition lists as a B2B module (Adobe guide, 1 October 2026); BigCommerce's B2B Edition serves its APIs from a separate domain, the trace of a partner-built origin. This is Kestrel & Pine's choice: one tenant, one catalog, one checkout, with company accounts and price lists added. It fits a brand whose wholesale customers are many small shops buying the same products as consumers, and it limits when terms, credit and negotiated prices must come from the ERP in real time and the platform's model cannot express them.

2. **A dedicated B2B platform.** OroCommerce (open source, for "distributors, wholesalers, and manufacturers") models requests for quote, quotes with approval workflows, price lists with scheduled adjustments and order entry by matrix form (OroCommerce documentation, version 7.0); SAP Commerce carries budgets and cost centers from its procurement heritage. These fit a wholesaler or manufacturer whose whole business is B2B, with quotes as the normal path and buyers with their own controls; they are a second platform to run beside a consumer storefront, with a second catalog copy to govern.

3. **An ERP-driven web store.** Sana Commerce's pitch is "built on your SAP or Dynamics ERP", so that "your buyers order online with the prices you agreed and the stock you really have" (sana-commerce.com, accessed November 2026). The store is a window on the ERP: no price copy, no terms copy, no separate order list. It fits a B2B-first company with one ERP and modest traffic; it limits when consumer traffic arrives, when the ERP's response time becomes the storefront's, and when the business adds channels the ERP cannot model.

4. **The buyer's system as the channel.** Large buyers do not log into a supplier's portal. They send EDI orders (Documents Between Companies: EDI, Punchout and E-Invoicing), or they **punch out**: their procurement system opens the supplier's catalog in a session, the buyer fills a cart, and the cart returns to the procurement system as a requisition that later comes back as a purchase order. The cXML (commerce XML, a procurement message standard) User's Guide (version 1.2.037, dated 2018 and still

the "current" link) describes it: "procurement applications display a button instead of product or pricing details … When users are done selecting items, they click a button that returns the order information to the procurement application." SAP's OCI (Open Catalog Interface) is the equivalent for SAP procurement. The rule for the supplier is that **a punchout cart is not an order**; the order arrives afterwards, as a cXML OrderRequest or an EDI ORDERS, and must enter the same order record as a portal order. Punchout is worth building for the few buyers who require it and nobody else.

	B2C platform's B2B features	**Dedicated B2B platform**	**ERP-driven web store**	**EDI and punchout**
Fits	Brands selling to many small resellers alongside consumers	B2B-first wholesalers and manufacturers with quotes and buyer controls	B2B-first companies with one ERP and modest traffic	A few large buyers who dictate the channel
Where prices live	Platform catalogs, mirrored from the pricing source	The platform's price lists; ERP mirrored	The ERP, read live	The buyer's contract; the supplier confirms
Credit check	Mirror of the limit; a live check needs integration	Built in or integrated	Live, in the ERP	In the ERP on order receipt
Order record	The platform (stages 1–2); the OMS (stage 3)	The B2B platform	The ERP	The OMS or ERP

	B2C platform's B2B features	Dedicated B2B platform	ERP-driven web store	EDI and punchout
Limits	Negotiated terms, approvals and credit beyond the platform's model; catalog caps on lower plans	A second platform and catalog copy; weak consumer features	ERP latency becomes storefront latency; no multichannel model	Per-buyer mapping cost; no self-service
Kestrel & Pine	Today, for 1,100 shops	Not needed	Not chosen: three consumer storefronts share the tenant	Bergzicht by EDI

My recommendation for a brand like Kestrel & Pine is to stay with the platform's B2B features and fix the three gaps that keep the 38% alive: publish contract and pre-season price lists from the ERP into the platform's catalogs as price events with effective dates; add a credit check at order placement against the ERP's open balance (synchronous if the ERP answers in under a second, otherwise accept-and-hold with an event-driven release); and create every wholesale order, however it arrives, in the OMS. That assumes the ERP exposes prices and balances by API and the platform's catalogs can carry per-account lists; where either fails, the ERP-driven store is the better answer for the wholesale channel alone.

Sales reps and B2B2C. The rep's tooling is the seller-side portal: create an order or a draft on behalf of a shop (Shopify's draft orders; BigCommerce's "Masquerade" lets staff act as a buyer), see terms and open balance, convert a quote. A rep's order is the same order object as the shop's own, with a different creator; a rep tool that writes straight into the ERP is another unowned writer. **B2B2C** names a flow, not a product: either a brand feeds product content and stock to a retailer who sells to consumers (a data and document problem, What Are We Selling? Products, Variants and the Catalog), or a business customer's own end users buy under the business's terms, as in a dealer portal, which is the model above with the company holding the prices and terms and its end users as contacts.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting	A few resellers served by e-mail and a spreadsheet price list; orders keyed into the shop or the accounting package; payment by invoice at the founder's discretion	Wholesale prices excluding VAT in one list; a written rule for which price wins; GTINs on every variant for the first chain	More than a few wholesale orders a week keyed by hand; a reseller's invoice disputed over a price
2 — Growing	The platform's B2B features or a simple portal: company accounts, one price list per country, net terms; the ERP or accounting system holds customers and invoices; a partner-built connector between them	One system of record for the company account (usually the ERP), mirrored to the portal; typed payment terms; a credit limit per account in the ERP, checked by hand	Negotiated accounts bypass the portal; the portal's and the ERP's prices drift; two order lists for one customer

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
3 — Omnichannel	Hundreds to thousands of accounts in several countries; contract and pre-season price lists; the first EDI buyer; wholesale competing with consumer and marketplace channels for the same stock	Prices published from one source with effective dates; a credit check at order placement; quotes as objects; wholesale orders in the OMS; wholesale allocation in the availability service; punchout for the buyers who require it	The share of keyed orders stops falling; credit holds by hand; a chain's chargeback for a document the ERP generated; the wholesale team cannot see a customer's orders in one place
4 — Group	Business units per brand or country; buyers with their own hierarchies and approval rules; key accounts on EDI and punchout; distributors and dealers as B2B2C	Business units with inherited rules; per-brand price lists under one pricing service; an ERP-fed credit check shared across brands; a master-data service for company identity and VAT numbers	Each brand runs its own portal and terms; the same retailer is three customers in three ERPs; approvals re-implemented per platform

The judgement: the B2B channel outgrows the platform's features not when the account count grows but when **the facts the checkout needs stop living in the platform**: negotiated prices, credit, approvals. Kestrel & Pine's 38% is that signal in numeric form, and it has held steady since 2024 because the facts behind it never moved.

In Practice

The platforms converge on one model, and none resolves ownership for you. Shopify, commercetools, Adobe and BigCommerce (whose older B2B authentication

token was deprecated on 30 September 2025) added B2B objects to consumer platforms; OroCommerce and SAP Commerce came from the B2B-first side with RFQs, budgets and cost centers. Every one of them can hold terms and credit, and the advice to pick one owner appears in the connector documentation (Microsoft's, above), not in the platforms'.

The market is large by any definition, and the definitions differ. Forrester's June 2022 release put US B2B e-commerce sales at $1.7 trillion in 2021, forecast to exceed $3 trillion by 2027, when it would be 24% of US B2B sales (a four-year-old forecast counting manufacturer and distributor sales to businesses). Digital Commerce 360 counted digital B2B sales "on ecommerce sites, password-protected sites, and apps", excluding EDI, at about $2.1 trillion in 2023 (7 February 2024), against total US manufacturing and wholesale sales of $15.12 trillion in 2025 (26 January 2026). No European figure meeting this book's sourcing standard was found. The point for an architect is the share that is *not* on a portal yet.

Kestrel & Pine: De Kaap Buitensport's pre-season order

Let me work one order through the portal as it is in spring 2027 and as it should be after Waypoint. **De Kaap Buitensport** is a fictional independent outdoor retailer with two shops, in Groningen and Assen, one of Kestrel & Pine's larger Dutch accounts, on net 30 terms with a **credit limit of €15,000** and pre-season terms of **5% off list** on orders placed before 1 March *(illustrative).* On **25 February 2027** its buyer wants 60 Ridgewalker jackets (a size run across women's and men's) and 120 pairs of merino socks for delivery in the first week of April.

	What happens in February 2027	What happens after Waypoint
Price	The portal shows list: 60 × €95 = €5,700 and 120 × €8.50 = €1,020, total €6,720 excl. VAT. The 5% pre-season discount exists only in the ERP's customer price list and Lea's e-mail, so the buyer e-mails the order instead	The pre-season list is published from the ERP as a price event into the account's portal catalog, effective to 1 March; the portal shows €6,384

	What happens in February 2027	What happens after Waypoint
Credit	The wholesale team keys the order into the ERP, which flags that open invoices of €9,800 plus this order exceed the €15,000 limit by €1,520 (at list) and holds it for Lea	The portal asks the ERP at placement; the order is created in the OMS as *awaiting seller approval*, and the buyer sees that status at once
Approval	Lea approves by e-mail on 27 February because a €4,100 invoice is due on 2 March; the team releases it in the ERP	Lea approves in the OMS; the release is an event; the ERP posts the order
Order record	The order exists in the ERP only; the portal history for De Kaap shows nothing for February	One order, one number family, visible to the shop, Lea, finance and the warehouse
Stock	The ERP reserves 60 jackets from a figure 15 minutes behind the WMS and an hour ahead of the platform	The availability service reserves against wholesale allocation; consumer channels see the reduced figure within minutes
Invoice	On shipment in April, from the ERP, at the price keyed in; if it differs from the e-mail, a credit note follows	From the ERP, from what shipped, at the price the order carried

The after-Waypoint column assumes the ERP can answer a credit check in under a second at the portal's volume and that the platform's catalogs can carry per-account, effective-dated lists; both should be tested before the 38% is promised to fall.

When It Goes Wrong

Failure	How it shows	Detection	Recovery
Portal price differs from invoice (the ERP recomputes at invoicing; a contract list expired)	Credit notes, disputes, shops asking for the "real" price	Compare order-line price with invoice-line price daily; alert on any difference	Invoice at the order's price; publish price lists with effective dates and recalculate only on explicit change
Stale credit balance (nightly copy)	Orders accepted that finance later holds, or rejected against an outdated balance	Age of the balance the portal used, logged per order	Credit check against the ERP at placement, or accept-and-hold with an event-driven release
Duplicate orders (the shop e-mails and then uses the portal; the rep keys an order the buyer already placed)	Double shipments, double invoices	Uniqueness on the buyer's purchase order number per company; a same-day duplicate check on lines	One order record for every entry path; a rep tool that creates the same order object
Wrong unit of measure (12 pieces ordered as 12 cartons)	A twelve-fold shipment and invoice	Order-line quantity against pack sizes and the account's history	Units as master data on the product; the portal confirms pack sizes
Expired or invalid VAT number	The wrong tax treatment on an invoice; a correction months later	Validate at account creation and periodically; store the validation date	The VAT number as master data with an owner
Quote accepted after it expired, or at a changed price	A dispute over which price was agreed	Quote state and `validTo` enforced when the order is created	Create orders only from valid quotes; expire the rest

The first two rows are the Kestrel & Pine story. The others arrive with scale, and all are cheaper to prevent in the object model than to fix at month-end.

What to Take Away

Established.

- Every major platform's B2B model has the same shape: an organization with locations, people and roles; customer-specific prices and assortments; quotes; approval rules; and payment terms, documented by Shopify, commercetools, SAP, Adobe, BigCommerce and OroCommerce as of 2026.
- A quote freezes a price. commercetools' "legally binding custom offers" with a validity date and Adobe's "the original price is used" are the same design decision, and the order must be created from the quote.
- Payment terms and credit limits are receivables facts; connector vendors' own documentation (Microsoft, 2026) says to manage them in the ERP and name one system of record for customer data.

Depends on context.

- The B2C platform's B2B features are the right channel for a brand selling to many small resellers, provided prices are published from one source and a credit check reaches the ERP at order time; once negotiated terms, approvals and credit live outside the platform's model, an ERP-driven store or a dedicated B2B platform fits better.
- A synchronous credit check is right when the ERP answers in under a second at the portal's volume; otherwise accept the order and hold it, with the hold released by an event, rather than copy the balance nightly.
- Punchout and EDI are worth building only for the buyers who require them; for everyone else they are cost without benefit, but the orders they produce must still enter the same order record.

The author's synthesis.

- The share of wholesale orders keyed in by hand is the best single measure of a B2B architecture, because it counts the facts the portal cannot see; Kestrel & Pine's 38% is a list of missing integrations (contract prices, credit, one order record), not a list of missing features.
- Decide ownership of the company account, the price and the credit limit before choosing or configuring any B2B platform; every platform can hold all three, and the one that does by default is the one nobody decided on.

Questions to Ask

1. Which system is the system of record for the company account, and which fields are mirrored to the portal, in which direction?
2. Where do contract and pre-season prices live, and can the portal show them to the right account on the right dates?
3. Which price wins when a quote, a contract price, a volume break and a list price all apply, and is that rule implemented once?
4. Is the credit limit checked at order placement against today's balance, or against a copy, and how old is the copy?
5. What share of wholesale orders arrive outside the portal, and for each reason, which fact is the portal missing?
6. Are quotes objects with a validity date, and is an order created from the quote or recomputed from the price list?
7. Is the invoice generated from the shipment at the order's price, and how often do portal price and invoice price differ?
8. Do punchout and EDI orders enter the same order record as portal orders, with the buyer's purchase order number as a uniqueness key?

Portal orders are the wholesale channel's self-service path. The largest buyers do not use it: they send documents. The next chapter opens the EDI envelope, from the ORDERS message to the despatch advice whose pallet labels cost Kestrel & Pine €14,200, and the e-invoicing mandates that now reach every business customer in Belgium. Continue with Documents Between Companies: EDI, Punchout and E-Invoicing.

Sources and Evidence

Company, product and investor sources

- Shopify B2B and Catalogs in B2B, Shopify Help Center, n.d., accessed 2026-11 — plan availability, catalogs, volume pricing, the three-catalog cap below Plus.
- B2B app development overview, Shopify, n.d., accessed 2026-11 — Company, CompanyLocation and CompanyContact definitions; draft orders for review and approval.
- PaymentTermsType and PaymentTermsTemplate, Shopify Admin API 2026-10 — typed payment terms.
- ERP Integration: Methods, Benefits, and Examples, Shopify, 2026 — "name the system of record for each" field.
- Business Units, Quote Requests, Staged Quotes, Quotes, Approval Rules and Approval Flows, commercetools, n.d., accessed 2026-11 — the B2B object model, quoting and approval.

- B2B Commerce Organization and B2B Order Approval, SAP Spartacus documentation, n.d., accessed 2026-11 — units, budgets, cost centers, purchase limits, approval (documents an unmaintained storefront version).
- Implement an Integration of SAP S/4HANA Cloud with SAP Commerce Cloud, SAP Learning, n.d., accessed 2026-11 — master data to commerce, orders to ERP.
- B2B for Adobe Commerce guide overview, Adobe, 2026, and Purchase order flow, Adobe, n.d., accessed 2026-11 — company accounts, shared catalogs, purchase orders, "the original price is used".
- About B2B Edition APIs, BigCommerce, n.d., accessed 2026-11 — company hierarchies, net terms and available credit, the separate API domain, the 30 September 2025 token deprecation.
- Sana Commerce, vendor page, n.d., accessed 2026-11 — the ERP-driven web store.
- OroCommerce README, Oro Inc., 2026, and Back-office Sales documentation, version 7.0, accessed 2026-11 — RFQs, quotes, price lists, order entry.
- Synchronize customers and companies, Microsoft Learn, 2026 — the system-of-record advice for customers, payment terms and credit limits.
- Forrester: US B2B e-commerce will reach an estimated $3 trillion by 2027, Forrester, 2022 — market size and definition (evidence age).
- B2B ecommerce market (US), Digital Commerce 360, 2024, and US B2B sales exceed $15 trillion in 2025, Digital Commerce 360, 2026 — digital B2B sales and total sales, with their definitions.

Standards, regulation and official documentation

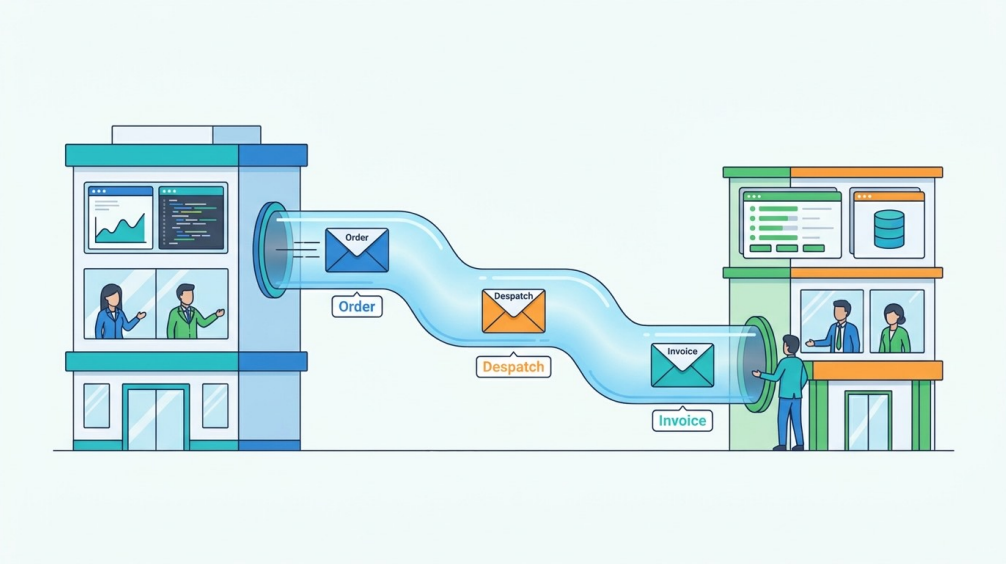
- cXML User's Guide 1.2.037, cXML.org (SAP Ariba), 2018 — the punchout protocol and messages (evidence age; still the current link).

Research and practitioner sources

- SAP SRM OCI PunchOut, TradeCentric, 2026 — OCI's fields and limits compared with cXML (integrator source).

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

22. Documents Between Companies: EDI, Punchout and E-Invoicing



IN THIS CHAPTER, YOU WILL: Learn how companies trade by exchanging standard documents, why each document must come from a different system, and which e-invoicing deadlines already apply.

KEY POINTS:

- **EDI is a channel into the order record, not a separate order system.** An EDI order should create the same order object as a web order, flagged by channel, so allocation, invoicing and the books see one shape.
- **Each document has its own system of truth.** The order comes from the buyer, the despatch advice must come from what the warehouse packed and labeled, and the invoice must come from what was shipped and priced. Generating any of them from the wrong system is how chargebacks happen.

- **E-invoicing is a legal calendar, not a feature.** Belgium has required structured e-invoices over Peppol for domestic business-to-business invoices since 1 January 2026, Germany's issuing obligation starts in 2027, and the Netherlands plans 2030. The invoice becomes a transmitted document with a delivery status the ERP must hold.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Pulled Rug** — a trading partner changes its message guideline, a label rule or a legal format, and the merchant learns from a rejected delivery.
- **The Lynchpin** — the EDI mappings live in one consultant's head or one provider's portal, and nobody inside the company can read them.
- **The Long Fuse** — an e-invoicing mandate with an official date is discovered when the first customer's system refuses a PDF.

Most of this book follows orders that start on a web page. This chapter is about orders that never do. When a retail chain buys from a brand, nobody logs in to a storefront. The chain's purchasing system writes a purchase order, sends it to the brand's system in a format both agreed years ago, and expects two documents back: a notice saying exactly what is on each pallet before the truck arrives, and an invoice its accounts-payable software can read without a person retyping it. **The buyer, not the seller, chooses this channel**, and writes it into its purchasing conditions with penalties.

Electronic data interchange, **EDI**, is the name for this exchange of standard business documents between two companies' systems: no e-mail, no PDF, no retyping. It is older than the web, slow to change, and a brand that wants to sell to chains has to speak it.

A second kind of document has joined it. Since 1 January 2026, a business in Belgium that invoices another Belgian business must send a **structured electronic invoice** over a public network called Peppol; a PDF by e-mail no longer counts. Germany, France, Italy and Poland have their own rules and dates, and the Netherlands has announced 2030. **The invoice is becoming a transmitted legal document with a delivery receipt**, and the system that issues invoices has to know whether each one arrived.

Where we are in the order's journey. Eva's jacket, the recurring consumer order in this book, never touches EDI. Its wholesale sibling does: Bergzicht's spring reorder of 6 April 2027, which Journey 3: A Chain Reorders by EDI, From Order to Invoice follows end to end. The two orders meet in the warehouse, where the same Duiven stock and the same WMS (warehouse management system: the software that runs receiving, picking, packing and shipping) serve both, and in the ledger, where both become postings.

Setting the Scene at Kestrel & Pine: A Deduction Instead of a Payment

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), has one customer that orders by EDI: **Bergzicht Outdoor**, a fictional Dutch outdoor retail chain with 38 shops and a distribution center in Zwolle, worth €2.1 million a year. Bergzicht has sent its orders electronically since 2025 through **the EDI provider**, a value-added network (VAN) that relays documents between trading partners. Three message types travel: Bergzicht's order (ORDERS), Kestrel & Pine's despatch advice (DESADV) and its invoice (INVOIC).

In October 2026, Lea Moreau, Head of Wholesale, received a remittance from Bergzicht that was **€14,200 short**. The attached statement listed nine third-quarter deliveries whose pallet labels had not matched the despatch advice announcing them. Bergzicht's supplier conditions let it charge a fixed compliance fee per non-compliant delivery plus the cost of counting the pallets by hand. The book calls this **the ASN Chargeback**, after ASN (advance shipping notice), the general name for a despatch advice.

The mechanism took Lea and Ruben Alves, the integration engineer, two weeks to reconstruct, because the only person who knew the EDI mappings was an external consultant engaged in 2025. **The ERP** (enterprise resource planning system: the back-office system that keeps the books, purchasing and invoicing) generated the despatch advice the moment a wholesale order was released to the warehouse, before anyone had packed anything, with pallet numbers from an ERP number range and contents as the order lines suggested. At Duiven, the WMS printed its own pallet labels at pack time. When pickers merged two half-pallets or split a heavy one, the labels on the truck and the message in Bergzicht's system described different pallets, and Bergzicht's scanners rejected them.

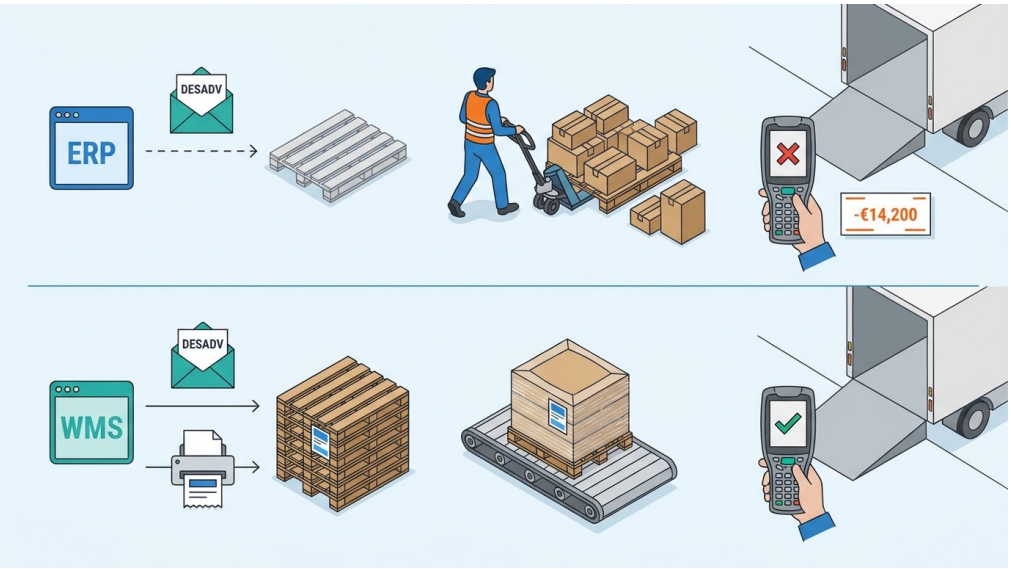


Figure 1: *The ASN Chargeback: the ERP announced pallets before anyone had packed them, the WMS labeled the pallets it actually built, and the mismatches on nine deliveries cost €14,200.*

Noor Visser's question in the Waypoint proposal is the one this chapter answers: **which system knows the truth each document claims to carry?** The ERP knew the order. Only the WMS knew the pallets. Waypoint decision **W6** fixed it: pallet labels generated by the WMS at pack time, the despatch advice sent from the WMS pack confirmation, and one EDI partner card per trading partner so that no mapping lives in one person's head again.

What EDI Is and Why It Has Not Gone Away

In plain words, EDI is a set of agreed message formats plus a way of delivering them. A **message** is a document such as an order; a **standard** says which fields it has and in which order; a **transport** is how the file travels; a **mapping** is the translation between the standard's fields and the fields in your own system.

Two standards dominate. **UN/EDIFACT** is the United Nations standard, maintained by UN/CEFACT, whose syntax became ISO 9735 in 1987; European retail uses the **GS1 EANCOM** subset, which GS1's manual (2002 edition, last updated 2021) describes as the UN/EDIFACT messages and fields relevant to retail. **ANSI X12** is the North American standard. A European brand selling to a Dutch chain and a US chain needs both; the message pairs are functional equivalents, not field-for-field translations.

GS1's 2021 implementation survey, the most recent public one, reported ORDERS, INVOIC and DESADV as the three most implemented EANCOM messages and AS2

as the most used transport. The survey is five years old; the standards it describes have barely changed since, which is the point.

Why does a format from the 1980s still carry wholesale orders in 2027? Four reasons, stated as industry practice. **The buyer chooses**: Jumbo, the Dutch supermarket chain, writes in its purchasing conditions of 9 April 2026 that suppliers "undertake to conduct all communication concerning the flow of goods (being order, packing slip (DESADV) and invoice) via EDI", forfeit €70 per message otherwise, and may have deliveries without DESADV and SSCC (the pallet's serial number, defined below) refused. **The documents carry meaning**: an order is a purchase commitment, a despatch advice a receiving plan, an invoice a payment claim, and their stability is a feature. **Network effects**: once a chain has mapped hundreds of suppliers, changing format costs more than keeping it. **Regulators accommodate it**: Germany lets an EDI procedure that does not yet meet the e-invoice requirements run to the end of 2027 rather than banning it.

The Documents: Order, Despatch Advice, Invoice

Document	EDIFACT	X12	Sender	What it says	Internal system of truth
Order	ORDERS	850	Buyer	Items, quantities, delivery date and address	The buyer's purchasing system; it creates an order record on the seller's side
Order response	ORDRSP	855	Seller	What will be delivered, line by line, including shortages	The seller's order record after allocation
Despatch advice (**ASN**)	DESADV	856	Seller	What is physically coming, pallet by pallet	The WMS pack confirmation

Document	EDIFACT	X12	Sender	What it says	Internal system of truth
Invoice	INVOIC	810	Seller	What is owed for what was shipped, at agreed prices, with VAT (value-added tax)	The ERP, from shipment and price, never from the order
Syntax acknowl-edgment	CONTRL	997	Receiver	"I could parse your file", not "I accept it"	The EDI translator

The **ASN** (advance shipping notice) deserves its own sentence. It is a message sent at despatch telling the receiver what is coming, pallet by pallet, so receipt can be scanned instead of counted. Jumbo's guidelines for its mechanized distribution center (October 2025) state the consequence: every delivery "must be preceded by a DESADV message", completed to pallet level, "a delivery without an accompanying DESADV message cannot be processed and is therefore refused", and "the SSCC label forms the link with the DESADV despatch message". The ASN is the receiving plan; the pallet label is the key into it.

A CONTRL or 997 says only that the file was syntactically readable, not that the order was accepted, the delivery received or the invoice will be paid; those answers come in ORDRSP, a receiving advice and, eventually, payment. **Treating a syntax acknowledgment as acceptance is a common and expensive mistake**.

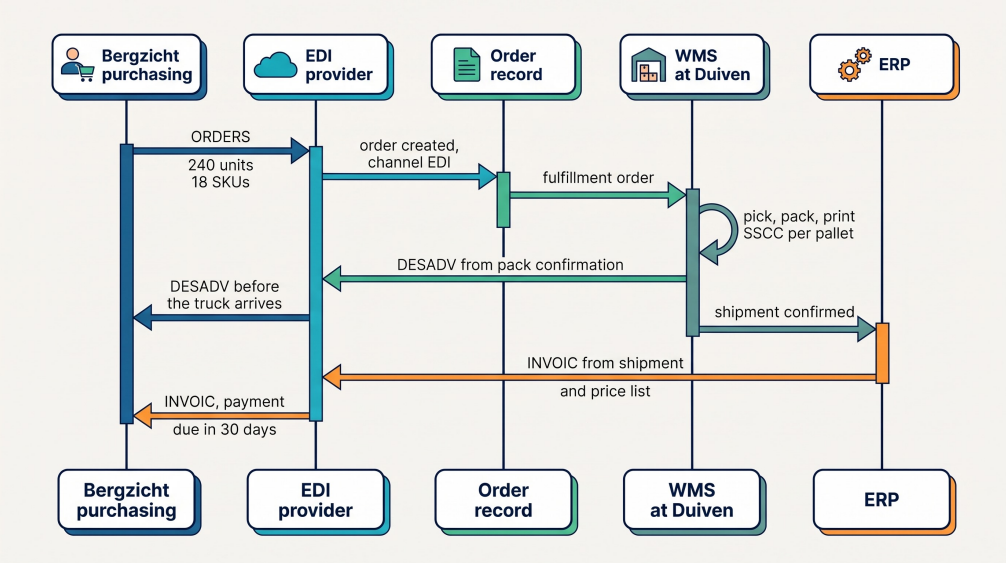


Figure 2: *Bergzicht's spring reorder as a document cycle: the order creates a normal order record, the despatch advice comes from the WMS once the pallets are labeled, and the invoice comes from the ERP once the shipment is confirmed. Three documents, three systems of truth.*

Identifiers: What, Where and This Pallet

EDI works because both sides use the same numbers for things, places and shipments. GS1's General Specifications (Release 26.0, ratified January 2026) define all three:

- A **GTIN** (Global Trade Item Number) identifies *what*: a sellable product, the number in the barcode on a jacket's hang tag; What Are We Selling? Products, Variants and the Catalog owns the term.
- A **GLN** (Global Location Number) identifies *who and where*: a legal entity or a physical place, such as Bergzicht's Zwolle distribution center. Messages address parties by GLN; Jumbo requires that the supplier GLN in the DESADV equal the GLN the order was sent to.
- An **SSCC** (Serial Shipping Container Code) identifies *this pallet*: an 18-digit serial number on a logistics unit's label. GS1 states that the SSCC "is the only GS1 key that SHALL be used as the identifier of a logistic unit" and "is used with EDI communications to enable identification and traceability". bol has required SSCC labels on every logistics unit since 31 December 2021 and requires EDI suppliers to include the SSCC in the DESADV.

The ASN ties the three together: each SSCC lists the GTINs and quantities it holds, addressed from one GLN to another. Hence the rule Kestrel & Pine paid for: **the SSCC**

must be generated by the system that physically builds the pallet. An ERP that invents pallet numbers before packing produces an ASN that is wrong whenever the warehouse deviates from the plan.

Transport: Point-to-Point or Network

Transport	What it is	You connect to	Typical use
AS2	A protocol (RFC 4130, 2005) for signed, encrypted documents over HTTP with a signed receipt proving delivery	Each partner: one certificate and endpoint per partner	Large retailers; the most used channel in GS1's survey
SFTP	Secure file transfer to a server, polled on a schedule	Each partner	Smaller partners, batch flows
VAN (value-added network)	A provider-operated mailbox network that relays to any partner on it and often translates formats	The provider, once	The common choice for a mid-size brand; Kestrel & Pine's EDI provider is one

Transport	What it is	You connect to	Typical use
Peppol	An open network run by OpenPeppol, a Brussels non-profit, in which parties connect through any accredited **access point** and reach any participant; it mandates document standards and has a public directory	Your access point, once	E-invoices where governments mandate them; Belgian business invoices since 2026

AS2 and SFTP are point-to-point: every partner is a relationship to manage. A VAN and Peppol are networks: connect once, reach many. Peppol alone has a public directory and mandates what the documents look like. OpenPeppol calls it the **four-corner model**: the sender hands the document to its access point, which delivers it to the receiver's access point, which delivers it to the receiver, with no bilateral agreement.

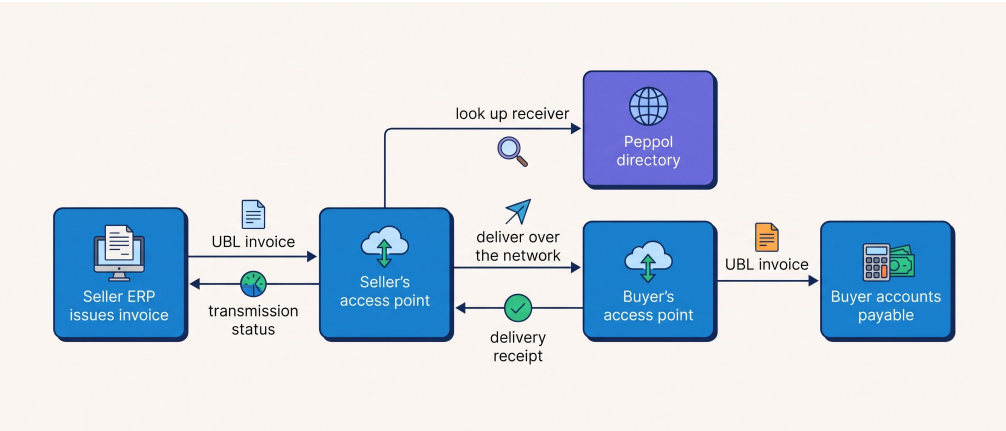


Figure 3: *Peppol's four-corner model: each party connects once to an access point of its choice; the sender's access point finds the receiver in a public directory and delivers; the transmission status flows back to the system that issued the invoice, which must store it.*

Mapping, Onboarding and the Chargeback Economy

The standards are public; the cost is per partner. Every chain publishes its own **message implementation guideline**: which optional fields it requires, which codes it accepts, how deep the DESADV must go. **Mapping** is the translation between a partner's guideline and your own order, shipment and invoice records, done and maintained per partner. That is the business model of the EDI providers: SPS Commerce's annual report for 2025, filed in February 2026, describes a network that lets companies "connect once and immediately transact with thousands of trading partners without negotiating standards, building integrations, or maintaining" them, and reported $751.5 million of revenue, 96% of it recurring. Jumbo tells small suppliers that its WebEDI portal is "the simplest, and cheapest in the short term", while an integrated connection "requires translating EDI messages to and from your internal system, with effort from IT, a solution provider, and system users".

Chargebacks are how the buyer enforces the guideline. They come in two families. **Delivery performance**: Walmart's on-time-in-full program charged a 3% fee on orders not delivered in full or on time when it launched in 2017 (the thresholds have risen since; no official Walmart page states them), and Jumbo's 2026 conditions add €250 per order when less than 90% of an order line arrives as agreed. **Data and label compliance**: a missing ASN, an unscannable label, an ASN quantity that does not match receipt; Jumbo's €70 per message is this family. The first family is a logistics problem. **The second is an architecture problem**: the ASN is only as right as the system that generated it.

How EDI Documents Reach the Order Record and the Ledger

Here is the rule this chapter exists to state. **An EDI order is a channel into the order record, not a separate order system.** An ORDERS message should create the same order object a web order creates, flagged by channel, so that stock allocation, invoicing and the books see one shape. The common failure is the opposite: EDI orders sit in the ERP or the provider's portal, web orders in the platform, marketplace orders in the connector, and nobody has one list. That was Kestrel & Pine's January 2027 state, and The Order Record: Lifecycle, State and the Order Management System explains why Waypoint's OMS (order management system: the one place every channel's order is created, with one state model) takes EDI orders from the ERP in Q4 2027.

Each document then has a different internal source:

Document	Produced from	Never from	Why
Order record	The ORDERS message after mapping and validation: item codes resolved, prices checked against the contract price list, delivery date feasible	A retyped e-mail	The buyer's commitment enters the system of record once, unchanged
Order response	The order record after allocation (stock set aside for this order; Is It in Stock? Inventory, Availability and the Promise You Can Keep)	The ORDERS echoed back	A response that promises everything promises nothing
Despatch advice	The WMS pack confirmation: which SSCC holds which GTINs in which quantity	The order or an ERP number range	The receiver scans what is physically on the truck
Invoice	The ERP, from the confirmed shipment and the contract price, with VAT, in the legal numbering sequence	The order (quantities may have changed) or the WMS (prices and VAT live elsewhere)	The invoice is a payment claim for what was delivered; Tax, Invoices and the Books: From Order to Ledger owns its legal shape

The flows have different requirements, which is the argument against forcing them through one system:

Flow	How fresh	How consistent	How fast	How available
ORDERS in → order record	Minutes; chains commonly expect a response within hours *(assumption)*	Exactly once: a duplicate ORDERS must not create two orders	Batch acceptable	A provider outage must queue, not lose
DESADV out	Must leave before the truck arrives	Must match the labels exactly	Minutes after pack confirmation	A late ASN is a refused delivery
Invoice out	Same day as shipment, or per month-end rules	One invoice per shipment; immutable once numbered	Daily batch acceptable	Rejection must be a tracked state
Transmission status back	Within hours	Per document	Not critical	Needed as dispute evidence

Punchout: When the Buyer's System Owns the Order

Some corporate buyers send no ORDERS at all; they shop in your web shop from inside their own procurement system. This is **punchout**. The cXML User's Guide, the protocol behind SAP Ariba's punchout, describes it plainly: procurement applications "display a button instead of product or pricing details", the buyer's browser then shows "pages from the supplier's local website", and when the user is done, a button "returns the order information to the procurement application". SAP's OCI (Open Catalog Interface) is the equivalent for SAP procurement; a 2026 integrator's description notes that OCI posts cart lines back to a hook URL and lacks native purchase-order and invoice automation.

Punchout **inverts the usual flow**. The buyer's procurement system owns the requisition and the purchase order; the supplier's shop is only a catalog session. The real order arrives later as a cXML order request or an EDI ORDERS, so **the shop must never treat a punchout cart as an order**. Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals covers punchout as a feature; here it matters as another channel into the same order record.

E-Invoicing: The Invoice Becomes a Transmitted Legal Document

The European Commission defines an **electronic invoice** as one "issued, transmitted and received in a structured data format which allows for its automatic and electronic processing"; a PDF attached to an e-mail is not one. Three layers to keep apart:

1. **The semantic model.** **EN 16931** is the European standard listing what a core invoice contains: seller and buyer with VAT identifiers, a document type (380 for an invoice, 381 for a credit note), lines each with a VAT category, a VAT breakdown per category and rate, payment information.
2. **The syntax.** **UBL** (Universal Business Language, an XML format) is the one most e-invoices use; national formats such as Germany's XRechnung are profiles of the same model.
3. **The network and profile.** **Peppol BIS Billing 3.0** is a usage specification of EN 16931 for the Peppol network. Its current release, 3.0.21, was published on 20 May 2026 and became mandatory on 17 August 2026: even the invoice format has a version calendar.

An EDI INVOIC and a Peppol invoice carry the same claim and differ in almost everything else:

	EDI INVOIC (EANCOM or X12 810)	**Peppol / EN 16931 invoice**
Who defines the content	A bilateral guideline per retailer, on top of the standard	The European standard, with a published usage specification and validator
Purpose	Supply-chain settlement: match to order and receipt	A tax-compliant legal invoice any receiver's accounts-payable system can read
Addressing	GLNs agreed per partner	A Peppol participant identifier in the public directory
Transport	AS2, VAN or SFTP, per partner	Any accredited access point

	EDI INVOIC (EANCOM or X12 810)	Peppol / EN 16931 invoice
Validation	Syntax acknowledgment and partner-side rejection	Validation rules at the access point; state clearance in Italy and Poland
Legal weight in 2026	Tolerated in Germany through 2027; in Belgium only if it meets the structured-invoice requirement	The format the mandates require

Three things change for the architecture. First, **the system that numbers and issues invoices, usually the ERP, must also hold the transmission status, rejections and archive**. Second, in **clearance** countries (Italy's SdI, Poland's KSeF) the state platform sits in the invoice path and the invoice is valid only once accepted, so rejection becomes a first-class state. Third, **commerce platforms do not issue tax invoices in these regimes**; they produce orders and confirmations, and the storefront's "invoice" PDF is a courtesy copy. Tax, Invoices and the Books: From Order to Ledger explains the invoice as a legal document; The Back Office: ERP Integration and Who Is the Hub explains why its home is the ERP.

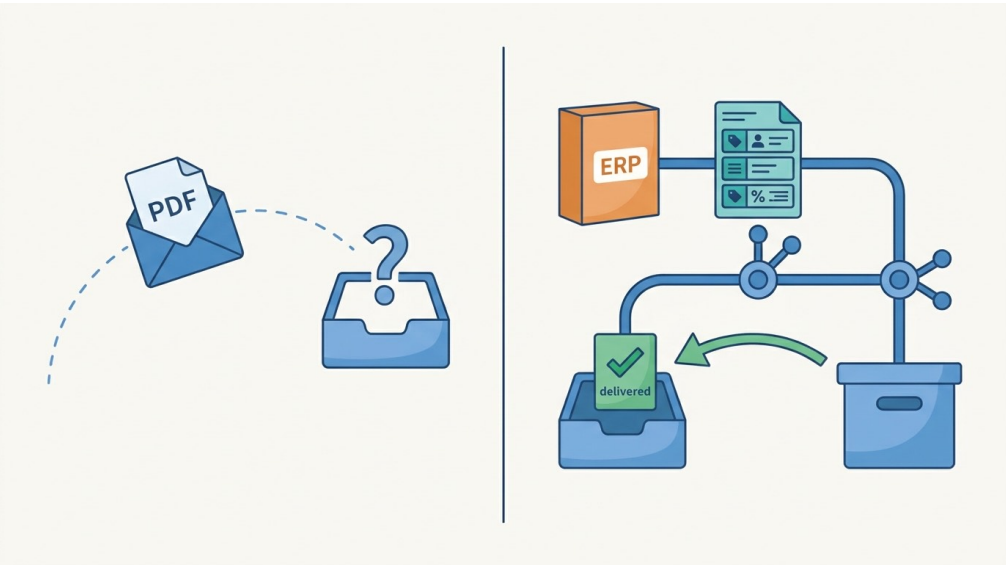


Figure 4: *An e-invoice is not a PDF in an e-mail but a structured, transmitted document with a delivery state, and the system that issues invoices must hold that state.*

The Mandate Calendar

The dates below are the official ones as of November 2026, from national tax administrations and the Commission. They are the most volatile facts in this chapter: check the source before relying on any of them, and read Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation for how to keep a regulatory calendar.

Jurisdiction	What is mandatory	Since or from	Format and channel	Touches Kestrel & Pine?
Belgium	Structured e-invoices for domestic business-to-business invoices between VAT-registered businesses	**1 January 2026**; tolerance on penalties in the first three months of 2026 only for businesses that can show timely steps; "under no circumstances" a general postponement	Peppol network, Peppol BIS at minimum, credit notes included; fines of €1,500, €3,000 and €5,000 for lacking the technical means	Indirectly: the bookkeeping of its 210 Belgian wholesale accounts expects Peppol invoices; whether the obligation reaches a Dutch supplier is not settled (see below)

Jurisdiction	What is mandatory	Since or from	Format and channel	Touches Kestrel & Pine?
Germany	Receive e-invoices (all domestic businesses); issue after transition periods	Receive since **1 January 2025** ("an e-mail inbox is already sufficient"); issue from **1 January 2027**, or **1 January 2028** if prior-year turnover was at most €800,000; EDI procedures tolerated to end 2027	EN 16931 formats such as XRechnung and ZUGFeRD from 2.0.1; no network mandated	Indirectly: covers transactions between businesses established in Germany
France	Receive (all); issue plus transaction reporting (large and mid-size)	**1 September 2026**; small and medium-sized enterprises (SMEs) and micro-enterprises issue from **1 September 2027**	Accredited platforms	No sales in France
Italy	Business-to-business and business-to-consumer e-invoicing via the state platform	Since **1 January 2019**; size exemptions removed **1 January 2024**	FatturaPA XML via the Sistema di Interscambio; EU derogation to 31 December 2027	No

Jurisdiction	What is mandatory	Since or from	Format and channel	Touches Kestrel & Pine?
Poland	Structured invoices via the national system KSeF	**1 February 2026** (2024 sales above PLN 200 million), **1 April 2026** (others), **1 January 2027** (smallest)	KSeF 2.0 central platform	No
Netherlands	None for business-to-business today; e-invoices to central government required	Cabinet plan of 11 September 2026: invoicing from **1 July 2030**, reporting from **1 July 2031**; bill before summer 2027	EN 16931; network not yet decided	Yes, from 2030: 640 Dutch accounts
EU (ViDA, "VAT in the Digital Age")	Digital reporting based on e-invoicing for cross-border business-to-business supplies	Adopted 11 March 2025; in force 14 April 2025; reporting from **1 July 2030**; national systems aligned by 1 January 2035	EN 16931	Yes, from 2030: every Belgian and German wholesale invoice

Two readings for a newcomer. The mandates are **domestic** in scope: Belgium's FAQ states that VAT-registered persons not established in Belgium and without a permanent establishment there are not subject to the obligation (the FAQ adds that the law is still being amended on this point), and Germany's covers transactions between domestic companies. A Dutch brand invoicing a Belgian shop is not automatically obliged, but the Belgian customer's bookkeeping expects a Peppol invoice, and many customers ask for

one; whether a foreign supplier with a Belgian store counts as established is a question for a tax adviser. The second reading: **the cross-border case arrives in 2030 with ViDA**, so a brand selling into several EU countries will soon send every business invoice as structured data. Building the capability once, for the first country that requires it, is cheaper than building it per deadline.

The Choices

A provider's web portal. You log in to the provider's site, read the chain's order, key the shipment and the invoice. No integration. Jumbo recommends it for small suppliers. It is also retyping, which EDI was meant to remove, and it does not scale past a few orders a day.

A provider integrated with your systems. The provider's translator maps the chain's guideline to your order record, WMS and ERP, over the provider's network, often through an iPaaS (integration platform as a service: a hosted tool for building and running the flows between systems). The provider owns the mappings and the partner connections; you own the contract for what each document means inside your company. This is the common choice at Kestrel & Pine's size.

In-house EDI. Your own translator software and AS2 endpoints, your own mappings under version control. Full control, full cost: certificates, onboarding, guideline changes and a person who understands EDIFACT. It makes sense when EDI is the business, as for a distributor with hundreds of partners.

An e-document gateway. One internal component, bought or assembled, that produces and receives every structured document (EDI, Peppol, punchout, clearance formats) from the same internal order, shipment and invoice records, with per-partner and per-country rules as configuration. The integrated provider grows into this as partners and countries multiply.

	Provider portal	**Provider, integrated**	**In-house**	**E-document gateway**
Time to first partner	Days	Weeks to months	Months	Months
Running cost	Lowest sub-scription; staff time per order	Subscription plus mapping per partner	Staff and software	Highest fixed cost

	Provider portal	Provider, integrated	In-house	E-document gateway
Who owns the mappings	Provider	Provider; you hold a copy and a partner card	You	You, under version control
Chargeback exposure	High: retyping; ASN rarely from the WMS	Low if the ASN comes from the WMS	Low if staffed	Lowest
Right when	A handful of orders a week, one partner	One to a dozen partners; a WMS exists	EDI is core to the business	Several countries, many partners, e-invoicing in more than one regime

My recommendation for a brand at the Omnichannel stage is the integrated provider, with two conditions that are not optional: the ASN is generated from the WMS pack confirmation, and the company holds a **partner card** per trading partner in its own repository, listing the partner's GLNs, transport, message versions, guideline version, mapping version, contacts, compliance rules and open disputes. The provider may hold the mapping; the company must hold the knowledge. That is W6's third clause.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting	No EDI; business customers order by e-mail; invoices as PDF from an accounting package	A provider portal if the first chain demands EDI; a Peppol-capable accounting package if any customer sits in a mandate country	The first purchasing conditions with an EDI clause arrive
2 — Growing	First chain forces EDI; a provider connector into the ERP; invoices numbered in the ERP and, where required, sent through a Peppol access point	A decision on which system produces the ASN (if a 3PL, a third-party logistics provider, packs, its system must)	Orders retyped from the portal into the ERP; the first chargeback; a second chain with a different guideline
3 — Omnichannel	Integrated EDI through the provider into the OMS via the iPaaS; SSCC labels from the WMS; Peppol invoices from the ERP to customers in mandate countries	A partner card per partner; a compliance scorecard (ASN match rate, on-time-in-full); disputes backed by pack and label evidence; a regulatory calendar	Mappings one person understands; EDI orders in the ERP while web orders live in the platform; a deduction nobody can explain

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
4 — Group	An e-document gateway shared across brands and countries; mappings under version control; clearance integrations where required	One invoicing service behind which country entities handle different clearance systems; mandate monitoring as a standing task	Each brand with its own provider contract; the same chain onboarded three times

Kestrel & Pine sits at stage 3 with a stage 2 setup: one chain, one provider, mappings built by one consultant, the ASN from the wrong system. The deduction was the warning sign.

In Practice

Belgium as the first mandate the reader will meet. The Belgian government's e-invoicing site announced on 18 September 2024 that from 1 January 2026 "all Belgian enterprises liable to VAT will have to use structured electronic invoices" for transactions with each other, that "sending a PDF invoice by e-mail or via a platform will no longer be enough", and that invoices travel "via a decentralised network called Peppol". On 18 December 2025 it confirmed a three-month tolerance on penalties for businesses that could show timely steps and refused any general postponement.

Kestrel & Pine's spring reorder, done right. On Tuesday 6 April 2027 Bergzicht's ORDERS arrives through the EDI provider: 240 units across 18 SKUs (sellable variants: one jacket in one size and color), Ridgewalker jackets and Trailhead fleeces, €19,860 excluding VAT. Under the W6 flow it creates an order in the ERP (the OMS is not live until Q4 2027), flagged as channel EDI, with Bergzicht's item codes resolved to Kestrel & Pine's SKUs by GTIN and prices checked against Bergzicht's seasonal contract price list, which the ERP holds. Allocation sets the 240 units aside at Duiven and an ORDRSP confirms all lines. The WMS picks and packs to three pallets, prints an SSCC label per pallet at pack time and records which GTINs each pallet holds. The DESADV is generated from that pack confirmation and reaches Zwolle before the truck. Delivery is 9 April; Bergzicht scans three labels against three pallets in the message. The WMS confirms the shipment to the ERP, which issues INVOIC the same day for the quantities shipped at contract prices, payment due 9 May. The moment the Q3 2026 flow would have failed is the pack step, where labels and message came from different systems.

The Belgian variant. Ardennes Trail Co., a fictional outdoor shop in Namur and one of the 210 Belgian accounts, orders 40 Ridgewalker jackets at €95, **€3,800** excluding VAT, through the platform's business-customer (B2B) portal on Wednesday 21 April 2027. The jackets leave Duiven by bpost, and at dispatch on 23 April the ERP issues the invoice in the same numbering sequence: an intra-EU supply at 0% VAT with the reverse-charge mention, payment due 23 May, rendered as a UBL (Universal Business Language, the XML invoice format Peppol carries) document under Peppol BIS Billing 3.0 and sent through the EDI provider, which is also Kestrel & Pine's Peppol access point; the ERP stores the transmission status. Kestrel & Pine has sent Peppol invoices to all Belgian business customers since January 2026 by choice, not because the mandate is known to bind a Dutch supplier (see above): its larger Belgian customers' bookkeeping expects Peppol, and one rule was cheaper than exceptions.

When It Goes Wrong

A duplicate ORDERS. The provider re-sends a file after a timeout that was actually a success. Without idempotent handling (receiving a message twice has the same effect as once; Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation owns the term), two orders for 240 units are allocated. Detection: the buyer's order number plus GLN is the natural key; a second ORDERS with the same key is a duplicate or a change, never a new order. Recovery: cancel before allocation; if shipped, a return and a credit note.

A changed guideline. The chain moves to a new DESADV level or SSCC rule and notifies the address on file, which is the consultant's. The first the merchant hears is a refused truck: the Pulled Rug. Detection: the partner card names an internal owner who receives partner notices; the scorecard shows the match rate dropping. Recovery: version the mapping, test against the chain's test environment, keep the old version until the switch date.

The ASN from the wrong system. The Kestrel & Pine case. Detection: compare every DESADV against the WMS pack confirmation before sending and alert on any difference; a match rate below 100% is a defect, not a statistic. Recovery: re-send a corrected DESADV before arrival where the chain allows it; otherwise dispute the deduction with the pack record and the label print log as evidence, which is why both must be kept.

A rejected e-invoice. The receiver's access point rejects a UBL file because a VAT category code is missing on one line, or because Peppol's validation rules changed with a release. The invoice is numbered in the ERP but was never received. Detection: the transmission status is a field on the invoice, and an "issued but not delivered" report runs daily. Recovery: correct and re-send, or a credit note and a new invoice; which one depends on the country and belongs in the ERP's configuration, not in someone's memory.

A provider outage, or the consultant leaves. The VAN is down for a morning: nothing is lost if every document is persisted before sending and carries a reference the provider can deduplicate, and a heartbeat per partner turns silence from a chain that orders daily into an alert. The consultant leaves: the mappings are in the provider's portal and the test cases in an inbox, which is why the partner card and a mapping export belong in the company's own repository.

What to Take Away

Established.

- EDI is a buyer-driven channel with its own standards (UN/EDIFACT and GS1 EANCOM in Europe, ANSI X12 in North America), identifiers (GTIN, GLN and SSCC, defined in GS1's General Specifications) and transports (AS2 per RFC 4130, VANs, SFTP, Peppol); the order, despatch advice and invoice are its most implemented documents, per GS1's survey.
- The ASN is a receiving plan keyed by the SSCC label, and large receivers refuse deliveries without it, as Jumbo's 2025 guidelines and 2026 purchasing conditions and bol's supplier rules document.
- Structured e-invoicing is law with dates: Belgium since 1 January 2026 over Peppol, Germany's issuing obligation from 2027, Italy since 2019 and Poland since 2026 through state clearance, the Netherlands planned for 2030, and ViDA's cross-border reporting from 1 July 2030.

Depends on context.

- A provider's portal fits a handful of orders a week with one partner; an integrated provider fits from the first chain with a WMS onward; in-house EDI fits only where EDI is the business; a shared e-document gateway is a Group-stage answer.
- Whether a foreign supplier is obliged under a domestic mandate depends on establishment rules; the decision to send structured invoices anyway depends on what customers can receive and on the cost of exceptions.
- EDI orders belong in the OMS where one exists; at the Growing stage, before an OMS, the ERP is an acceptable home, provided the ASN still comes from the system that packs.

The author's synthesis.

- Each document has exactly one honest source, and the architecture should make it impossible to generate it anywhere else: the order from the buyer's message, the ASN from the pack confirmation, the invoice from the ERP's confirmed shipment. Every compliance chargeback in the public rule sets above is a document that came from the wrong place.

- The e-invoice turns the invoice into a message with a delivery state, so the ERP needs the discipline the order channel needs: persistence before sending, idempotent re-sends, a status per document and a daily report of what was issued but not received.

Questions to Ask

1. Which of our customers require EDI, under which purchasing conditions, and what do those conditions charge for a missing or wrong ASN?
2. Which internal system generates the DESADV, and is it the system that physically packs and labels the pallets?
3. Does an EDI order create the same order record as a web order, or does it live in the ERP or the provider's portal?
4. For each trading partner, where is the partner card: GLNs, transport, guideline version, mapping version, internal owner, open disputes?
5. What is our ASN match rate per partner, who sees it weekly, and who receives the chain's notices of guideline changes?
6. In which countries do we invoice business customers, which mandate applies to each, and from what date?
7. Which system numbers and issues invoices, and does it store the Peppol transmission status, rejections and archive?
8. Who inside the company can read our EDI mappings without calling a consultant?

The documents in this chapter end in one place: the ledger. The next chapter asks what the system that keeps the ledger, the ERP, should own in a commerce business, and whether it should be the hub everything else connects through. Continue with The Back Office: ERP Integration and Who Is the Hub.

Sources and Evidence

Company, product and investor sources

- SPS Commerce Form 10-K for 2025, SPS Commerce via SEC EDGAR, 2026: revenue, recurring share and the "connect once" description.
- EDI Jumbo Supermarkten information, Jumbo, 2026 profile (accessed 2026-11): WebEDI vs integrated routes, document sets, the GLN rule.
- Algemene inkoopvoorwaarden Jumbo Supermarkten, Jumbo, 2026: EDI obligation, €70 per message, €250 per order, refusal rights (translated from Dutch).
- Richtlijnen gemechaniseerd DC, versie 2025-10, Jumbo, 2025: DESADV before every delivery, refusal without it, the SSCC label as the link (translated from Dutch).

- SSCC for EDI suppliers, bol, n.d. (accessed 2026-11): SSCC labels mandatory since 31 December 2021 and required in the DESADV.
- Walmart U.S. gives suppliers new order fulfillment guidelines, Talk Business & Politics, 2017: the 3% on-time-in-full fee at launch (older than three years; thresholds have changed).
- How to dispute Walmart OTIF chargebacks, SPS Commerce, 2024: current targets as a provider reports them; no official Walmart page found.
- cXML User's Guide 1.2.037, cXML.org (SAP Ariba), 2018: the punchout description (older than three years; still the "current" link).
- SAP SRM OCI PunchOut, TradeCentric, 2026: OCI's mechanism and limits (integrator source).

Standards, regulation and official documentation

- GS1 General Specifications, Release 26.0, GS1, 2026: definitions of GTIN, GLN and SSCC and the SSCC's role in EDI.
- GS1 EANCOM 2002, Syntax 4, Part 1, Edition 2016 Upd. 2021, GS1, 2021: EANCOM as a subset of UN/EDIFACT (standard older than three years).
- Implementation of GS1 EDI standards in 2020, GS1, 2021: most-implemented messages and transports (older than three years).
- X12 Transaction Sets, X12, n.d. (accessed 2026-11): the 850, 855, 856, 810, 846 and 997 descriptions.
- EDIFACT, Wikipedia, 2026 (accessed 2026-11): UN/EDIFACT's origin and ISO 9735 (encyclopedia source; unece.org refused fetches).
- RFC 4130: Applicability Statement 2 (AS2), IETF, 2005: what AS2 is (older than three years; still in production use).
- About Peppol, OpenPeppol, n.d. (accessed 2026-11): the four-corner model and governance.
- Peppol BIS Billing 3.0 specification and release notes, OpenPeppol, 2026: the usage specification of EN 16931, document type codes, release 3.0.21 dates.
- What is eInvoicing, European Commission, n.d. (accessed 2026-11): the definition of an electronic invoice.
- Structured electronic invoices between companies are compulsory in 2026, FPS Finance, Belgium, 2024; Period of tolerance during the first three months of 2026, 2025; FAQ: general questions B2B and FAQ: specific questions about e-invoicing, n.d. (accessed 2026-11): the Belgian mandate, scope, format, fines and tolerance.
- FAQ E-Rechnung, Bundesministerium der Finanzen, Germany, n.d. (accessed 2026-11): receiving since 2025, issuing transition periods, EDI tolerated to end 2027, accepted formats (translated from German).
- À partir de quand suis-je concerné par la réforme de la facturation électronique ?, DGFiP, France, 2026: the French dates (translated from French).

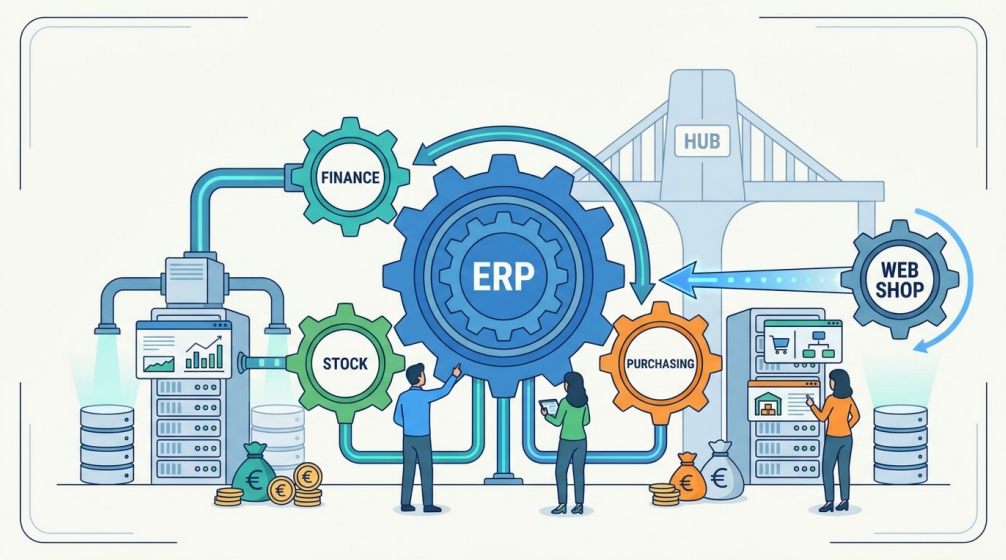
- eInvoicing in Italy, European Commission, 2026: SdI since 2019, exemption removal 2024, derogation to 2027.
- KSeF priority page, Ministry of Finance, Poland, 2026: the KSeF dates (translated from Polish).
- Kabinet kiest voor invoering e-facturatie en rapportage voor bedrijven, Rijksoverheid, Netherlands, 2026: the 2030 and 2031 plan (translated from Dutch).
- E-invoicing, business.gov.nl, n.d. (accessed 2026-11): the existing Dutch obligation toward central government.
- VAT in the Digital Age (ViDA), European Commission, DG TAXUD, n.d. (accessed 2026-11): adoption, entry into force, the 2030 and 2035 dates.

Research and practitioner sources

- The scale-stage table, the "one honest source per document" rule and the partner-card recommendation are the author's synthesis of the sources above and of common practice; the Kestrel & Pine latencies and Bergzicht's compliance clause are illustrative.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

23. The Back Office: ERP Integration and Who Is the Hub



IN THIS CHAPTER, YOU WILL: Learn what an ERP owns in a commerce business, what flows between it and the commerce systems, and how to decide which system is the hub.

KEY POINTS:

- **The ERP keeps the books, not the shop.** Its four jobs are the ledger, purchasing, inventory valuation and master data. The WMS (warehouse management system) knows what is on the shelf; only the ERP knows what it is worth.
- **"Who is the hub" is a scale-stage decision.** ERP-as-hub fits one channel and one warehouse, commerce-as-hub fits a direct-to-consumer (D2C) brand, OMS-as-hub (an order management system at the center) fits omnichannel, and an integration platform with no hub at all is Kestrel & Pine's thirty-four integrations and three definitions of "order".
- **Finance can be batch; availability cannot.** Push orders, shipments and payments as events from the systems that own them and let the

ERP post them daily, but never let the ERP's cycle time become the storefront's stock latency. The partner-built connector that carries these flows is a contract to own, not a black box to rent.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Spaghetti Junction** — every system was connected to the ERP by whoever implemented it, and the ERP became a hub by accident, with no owner for any flow.
- **Schrödinger's Order** — the ERP's copy of the order lags the platform's and the warehouse's, so finance, support and the customer see three statuses.
- **The Lynchpin** — the platform-to-ERP connector was built by the ERP partner, is documented nowhere, and only the partner can change it.

Every commerce business has a system that keeps the books. In a company of three people it is an accounting package; in a company of three hundred it is an **ERP**, an enterprise resource planning system: the back-office software that records every sale, payment, fee and refund as a posting in the general ledger, buys stock from suppliers, says what that stock is worth on the balance sheet, and holds the master records of items, customers and suppliers that everything else copies. Finance lives in it. Month-end closes in it. The auditor reads it.

Because the ERP holds the master records and because it arrived early, it tends to become the system everything else connects to. The web shop sends it orders; the warehouse sends it stock; the stores send it sales; the EDI provider (electronic data interchange: standard documents exchanged between companies' systems, see Documents Between Companies: EDI, Punchout and E-Invoicing) sends it wholesale orders; the storefront reads its stock and prices back. **The ERP becomes the hub without anyone deciding it should be**, and then its rhythm, which is the rhythm of accounting (hourly, nightly, monthly), becomes the rhythm of the shop, which is the rhythm of a customer deciding in the next four seconds whether the jacket is in stock.

This chapter asks what the ERP should own, what should flow between it and the commerce systems, in which direction and how often, and whether it, the commerce platform, an order management system or nothing at all should be the hub. The answer is not "get rid of the ERP"; it is to give the ERP a smaller, clearer job.

Where we are in the order's journey. This chapter is the last step of Eva's jacket: accounting. Her €235 order of 13 April 2027, the iDEAL payment, the fee of the PSP (payment service provider: the company that takes the shopper's payment and settles the money to the merchant), the shipment and, in the refund variant, the €199 credit note all end as postings in the ERP. Invisible to Eva; decisive for Sanne de Wit, the Finance Director, who closes April's books from it.

Setting the Scene at Kestrel & Pine: The Connector Nobody Could Read

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), has run a cloud mid-market ERP since 2023. It does what the definition above says: finance, purchasing, inventory valuation, the item master, invoicing, and the manual entry of the 38% of wholesale orders that still arrive by e-mail or phone. The connector between the platform and the ERP was built by **the ERP partner**, the external firm that implemented the ERP.

In her first month, January 2027, Noor Visser asked the partner for the connector's specification: which fields move, in which direction, how often, and which system wins when they disagree. The answer was a configuration export and an invoice for the hours to explain it. Reconstructing it with Ruben Alves, the integration engineer, took three weeks. The connector copied platform orders into the ERP hourly as sales orders, which the ERP invoiced only on a WMS "order complete" message, and pushed ERP stock to the platform hourly from a figure the ERP received from the WMS every fifteen minutes; German storefront prices came from an ERP price list through the same connector. Nothing was documented, nothing was idempotent (receiving a message twice did not have the same effect as once), and the retry on failure was "run again tomorrow".

Three of 2026's incidents ran through this connector. The **Black Friday Oversell** happened because the ERP sat between the WMS (warehouse management system: the software that runs receiving, picking, packing and shipping at Duiven) and the platform in the stock chain, adding up to an hour of delay on top of the WMS's fifteen minutes and the connector's hour. The **Three-Status Order** (#1052877, December 2026) showed "Not invoiced" in the ERP for six days because the ERP invoices only on a WMS "order complete" message, and the second line of a two-line order shipped a week after the first. The **Price Drift** of October 2026 was a price-list sync through the same connector that failed silently for nine days.

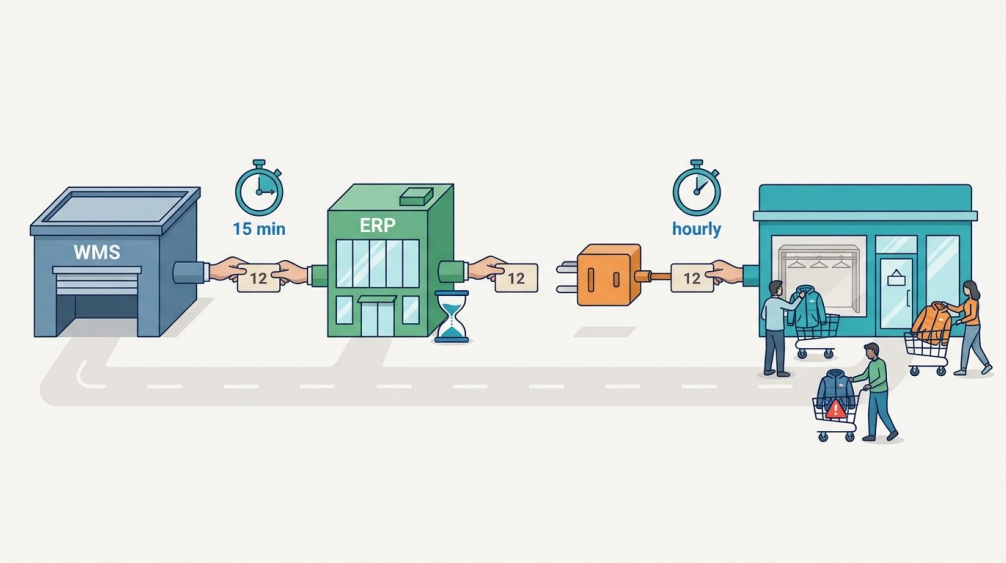


Figure 1: *With the ERP in the stock chain, a stock figure from Duiven waited on the WMS's fifteen minutes, the ERP and the connector's hour before a storefront saw it, which is how the Black Friday Oversell happened.*

Sanne's month-end problem was quieter and larger. The ERP held platform, EDI and manual wholesale orders, but marketplace orders arrived as a weekly summary from the marketplace connector and store sales as a nightly total from the POS (point of sale: the stores' till software), each in a different shape; January's close took eleven working days. Her question to Noor was the chapter's question from the finance side: **what should the ERP be the truth for, and what should it just receive?**

Waypoint answered it as an ownership rule that this chapter calls **the finance-only rule**: the ERP is the system of record for finance (the ledger, invoices and credit notes, receivables and payables, purchasing, inventory valuation and the cost side of the item master) and for nothing else. Orders move to the OMS (order management system: the one place every channel's order is created, with one state model; W2), product content to the PIM (product information management system: the owner of product descriptions, attributes and media; W3), physical stock to the WMS and availability to the availability service (W1); the ERP consumes orders, shipments, payments and refunds by daily batch (W5) and feeds no channel directly: wholesale and contract prices, business terms and credit limits leave it only as events that the OMS and the platform's B2B catalogs consume, and no storefront reads the ERP live. The partner-built connector is retired as the OMS takes over the order paths in late 2027.

What an ERP Does in a Commerce Business

In plain words, the ERP has four jobs, and a commerce architect should be able to name them:

Job	What it means	What it needs from commerce	What commerce needs from it
The ledger	Every order, payment, payment fee, refund, VAT (value-added tax) amount and shipping charge becomes a posting; month-end, VAT returns and the annual accounts are produced from it	Orders, payments, refunds and shipments, complete and once	Nothing in real time
Purchasing	Supplier orders, goods receipts, supplier invoices, landed cost (purchase price plus freight and duty)	Sales history and forecasts	Expected stock (what is on order) for pre-orders and planning
Inventory valuation	What the stock is worth: a cost per unit under a costing method, and the balance-sheet value	Stock movements and quantities, in batch	Cost per SKU (a sellable variant, such as one jacket in one size and color), for margin reports

Job	What it means	What it needs from commerce	What commerce needs from it
Master data	Items (identity, cost, supplier, purchasing units), customers (terms, credit limits), suppliers	New customers from the storefront; new items from merchandising	The item must exist here before it can be bought or valued; business-customer (B2B) terms and credit limits

The Dutch mid-market ERPs describe themselves in exactly these terms. Exact's trade edition page (accessed November 2026) promises to "manage stock, orders and accounting in one system", with purchasing, multi-warehouse stock, sales orders, returns and "(e-)invoicing" included; AFAS describes one integrated system with one database in which "data only needs to be entered once, and processes run automatically" (both translated from Dutch). The pitch is true at the size the pitch is written for, which is the size at which the ERP *is* the commerce back end.

Inventory valuation is the job newcomers underrate. IAS 2, the international accounting standard for inventories, measures them "at the lower of cost and net realisable value" under a cost formula such as first-in, first-out or weighted average. Microsoft's Business Central documentation (March 2026) lists five costing methods and warns that "you can't change an item's costing method if item ledger entries exist for the item". None of this has anything to do with whether Eva can buy a jacket tonight. **The commerce platform knows stock quantities; only the ERP knows what the stock is worth.** Keeping the two apart, quantity in the WMS and value in the ERP, is the first boundary this chapter draws. Is It in Stock? Inventory, Availability and the Promise You Can Keep owns the quantity side.

What Flows Between Commerce and the ERP

Six kinds of data cross the boundary. The direction and the cadence are the design, not details:

Flow	Direction	System of record	Cadence that fits	Why
Orders	Commerce → ERP	The OMS (or the platform before an OMS exists)	Event on creation, consumed by the ERP in a daily batch	The ERP needs every order once; it does not need it within the minute
Shipments	WMS → ERP	The WMS	Event, consumed daily	Triggers the invoice; revenue is recognized at delivery, not shipment (Tax, Invoices and the Books: From Order to Ledger)
Payments, fees, refunds, payouts	PSP and platform → ERP	The PSP for money movements; the platform for refund decisions	Daily batch, reconciled against PSP payouts	Finance works in periods
Invoices and credit notes	ERP → customer, EDI, Peppol (the European e-invoice network)	The ERP	On shipment; daily	Legal numbering and archive live here; see Documents Between Companies: EDI, Punchout and E-Invoicing

Flow	Direction	System of record	Cadence that fits	Why
Products	ERP ↔ PIM ↔ commerce	ERP for identity and cost; PIM for content	Scheduled publication; nightly full sync as safety net	Content tolerates minutes of lag; the item must exist in the ERP first
Stock quantity	WMS → ERP	The WMS	Batch, for valuation; **never the ERP → storefront** at the Omnichannel stage	Availability needs events; valuation needs totals
Customers and terms	ERP → commerce (B2B); commerce → ERP (D2C)	The ERP for business accounts, terms and credit limits; the platform for consumer accounts	On change	Payment terms and credit limits are finance facts

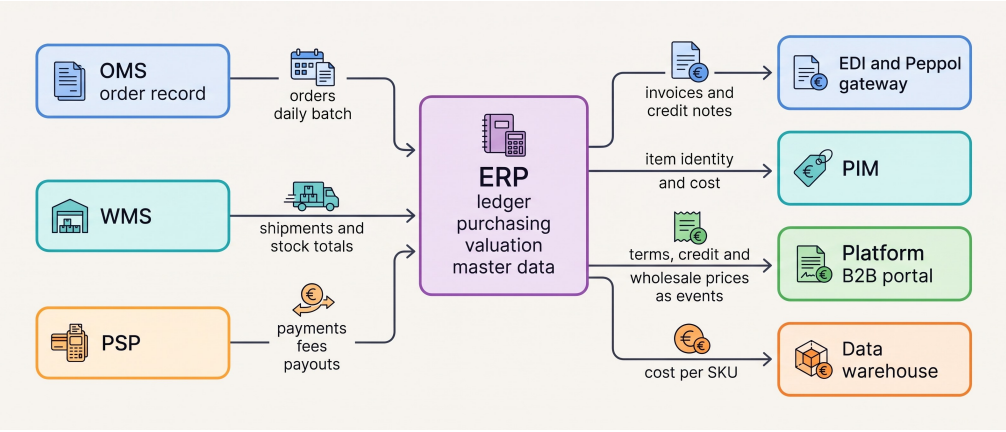


Figure 2: *The finance-only ERP: orders, shipments and money flow in from the systems that own them, in daily batches; invoices, item identity and cost, business terms and cost per SKU flow out, as events or batches. No channel reads stock, availability or prices*

from the ERP live.

The flows have different requirements, which is why they should not share one connector and one schedule:

Flow	How fresh	How consistent	How fast	How available
Orders to ERP	Same day	Exactly once per order; one ERP document per order number	Batch window of minutes	A missed night must be replayable
Shipments to ERP	Same day	Matches the WMS pack record	Batch	Replayable
Payments and payouts to ERP	Daily, aligned to the PSP's payout cycle	Every payout reconciled to its orders; Payments-to-Ledger: Draining the Suspense Swamp covers the method	Batch	Replayable
Business terms to the portal	Minutes after a credit limit changes *(assumption)*	The ERP's value wins	Event or short poll	The portal must degrade to "order pending review", not block
Availability to the storefront	Seconds to a minute	Per SKU and location	Event	**Not from the ERP**

The last row is the whole argument in one line.

Who Is the Hub? Three Answers and a Non-Answer

An **integration hub** is the one system every other system connects through and that owns the shared records. There are three honest answers to which system that should be, and one accident.

ERP as hub. Every system integrates with the ERP; the ERP holds stock, prices and customers, routes orders to the warehouse and feeds the storefront. The purest product form is a web store that reads the ERP directly: Sana Commerce positions itself (page accessed November 2026) as "the B2B e-commerce platform for manufacturers, built on your SAP or Dynamics ERP", where buyers "order online with the prices you agreed and the stock you really have". ERP-centered suites such as Odoo and JTL do the same from the ERP side. This fits one channel, one warehouse, a B2B-heavy business and a small team, because it avoids the integration problem by not having separate systems.

Commerce platform as hub. The platform is the system of record for orders, customers and stock; the ERP is a finance subscriber that receives what happened. Microsoft's Business Central Shopify connector documentation (October 2026) describes exactly this division for a D2C merchant: orders flow from Shopify into Business Central as sales orders, with the "Shopify Order Id" kept on every document. It fits a D2C brand with one or two channels. It breaks when stores, wholesale and marketplaces each have their own order source, because a storefront platform's order model holds EDI and POS orders badly.

OMS as hub. An order management system (the one place every channel's order is created, with one state model, which The Order Record: Lifecycle, State and the Order Management System explains) owns the order record, allocation and availability across channels; the ERP owns finance and valuation; the platform owns the storefront. The OMS vendor OneStock argues the case in a 2026 blog post, which favors its own category and should be read that way: "ERPs update inventory in batch cycles, hourly or nightly in many implementations", the ERP "was designed for process consistency, not fulfillment optimization", and "ERP isn't broken. It's just not built for everything." It fits the Omnichannel and Group stages, and it is wrong at the Starting and Growing stages, where it is a second integration surface nobody needs.

No hub: an iPaaS. An **iPaaS** (integration platform as a service: a hosted tool in which you configure the flows between systems instead of coding them) carries point-to-point flows, and no system owns the cross-channel order record. This is not a design; it is what happens when each system is connected as it arrives. Kestrel & Pine's thirty-four integrations and three definitions of "order" are the result; APIs, Events, Files and Middleware: How Commerce Systems Talk explains the iPaaS as a tool, not an architecture.

Hub	What it means	Fits	Breaks when
ERP as hub	Every system integrates with the ERP; it holds stock, prices, customers and routes orders	One channel, one warehouse, B2B-heavy, small team	Peak traffic and real-time availability meet batch cycles; several channels write into one monolith; change is slow
Commerce platform as hub	The platform owns orders, customers, stock; the ERP subscribes for finance	D2C-first brands on a hosted platform, one or two channels	Stores, wholesale and marketplaces each bring their own order source
OMS as hub	The OMS owns orders, allocation and availability; ERP owns finance; platform owns the storefront	Omnichannel and Group	Cost and a second integration surface; wrong at stages 1–2
iPaaS, no hub	Point-to-point flows through an integration platform; nobody owns the cross-channel record	Growing stage with a few flows	Thirty-four integrations and three definitions of "order"

Waypoint's finance-only rule is the OMS-as-hub position written as an ownership rule; it does not move the ledger anywhere. Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record explains why ownership has to be decided per fact, not per system; this chapter applies that rule to the back office.

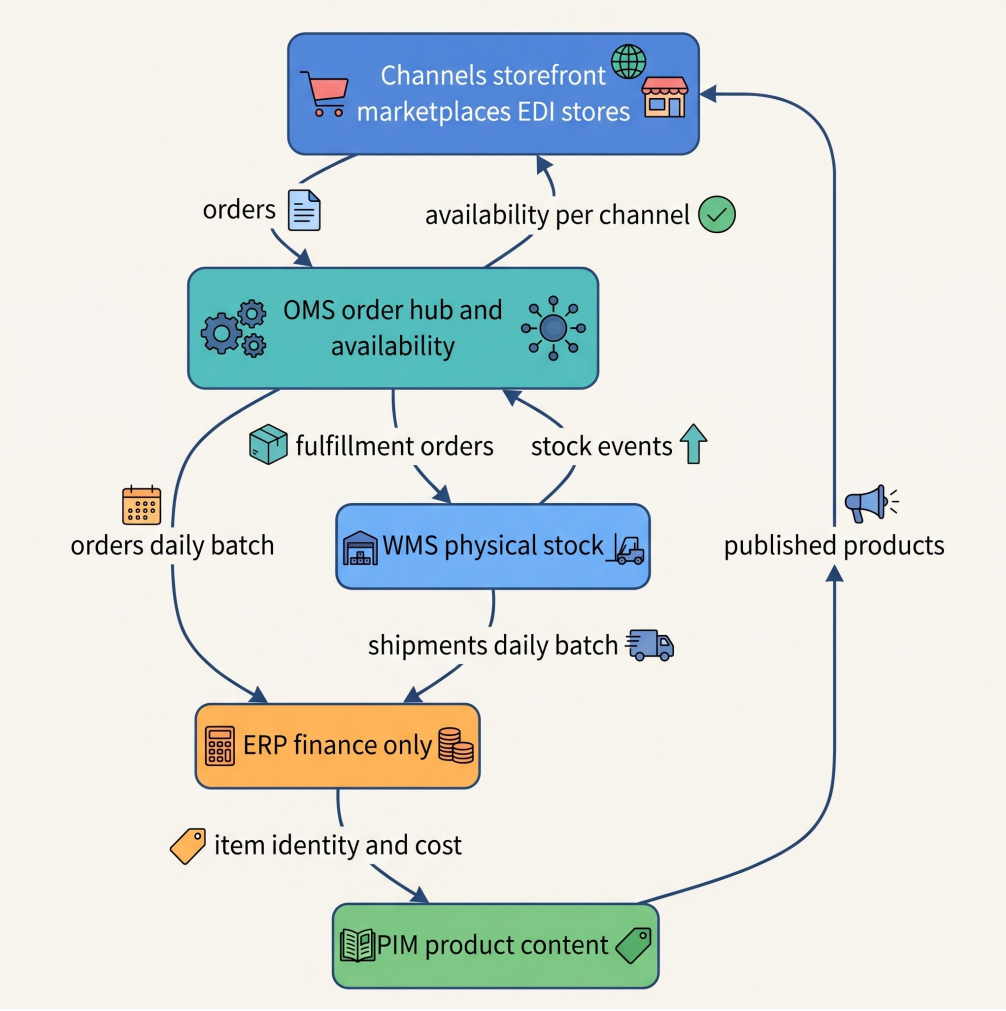


Figure 3: *Kestrel & Pine's target shape after Waypoint: the OMS is the hub for orders and availability, the WMS owns physical stock, the PIM owns content, and the ERP receives daily batches and feeds no channel directly. The ERP is no less important; it is no longer in the path of a sale.*

Batch or Events: The ERP's Rhythm

ERP integration is batch by default, and the vendors say so. Shopify's enterprise guidance on ERP integration (updated August 2026) says financials "sync in real time or through a daily batch". Business Central's connector runs its customer, company, order and inventory synchronizations as scheduled job-queue entries and recommends "every 15-30 minutes" for inventory. OneStock's "hourly or nightly" is the same observation from a competitor's side.

Batch is the right rhythm for finance. Postings, payouts, invoices and valuation are period-based; a daily cycle that is complete and reconciled beats a real-time feed that drops one order in a thousand. The rule I use is: **finance can be batch, availability cannot**. Push events from the system that owns each fact; let the ERP consume them in its own rhythm, daily, with a reconciliation that proves completeness; and never let the ERP's cycle time become the storefront's availability latency. Kestrel & Pine's 2026 chain (WMS → ERP every fifteen minutes → platform hourly → marketplace connector hourly) broke this rule by routing a real-time fact through a batch system, and the Black Friday Oversell was the bill. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation owns the mechanics of events, idempotency and reconciliation.

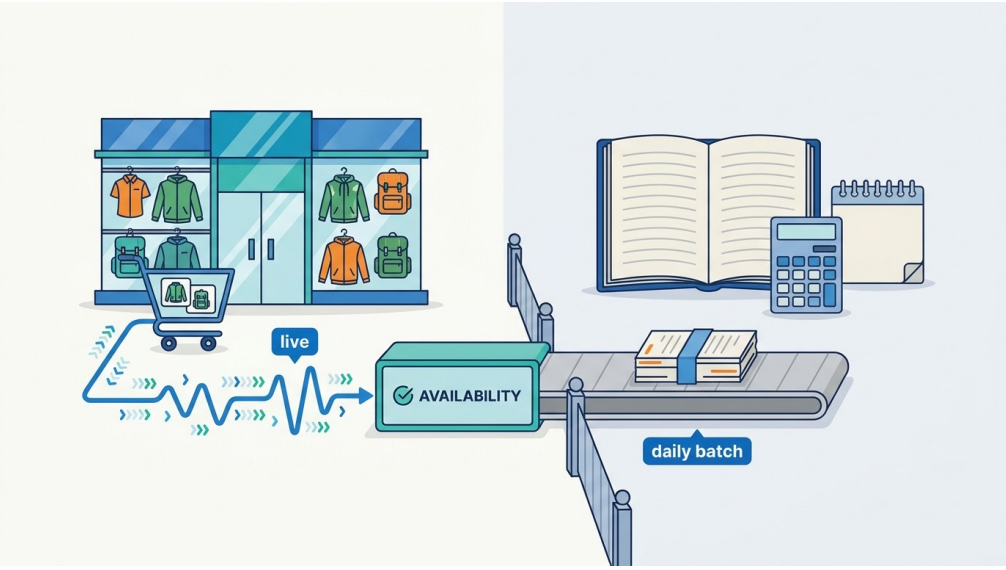


Figure 4: *Finance can be batch, availability cannot: the ERP consumes what happened in its own daily rhythm and is never the system a storefront waits on.*

The business reason to keep the ERP out of the real-time path is not performance but **change**: a finance system is changed carefully, in releases, with auditors watching, while a storefront's availability logic changes weekly, and coupling them means the slow one sets the pace.

Connectors as Products

Between a hand-built integration and a general iPaaS sits a third option: a **connector** built and maintained by the vendor of one of the two systems. Shopify's Global ERP Program, announced on 14 October 2021, let selected ERP vendors build direct

integrations into its app store; the announcement is five years old and the program's shape has changed since. The best-documented result is Microsoft's **Business Central Shopify Connector**, whose documentation (dated September and October 2026) is unusually explicit about ownership and worth reading as a dated example of a vendor encoding the entity–owner table into configuration:

- It is "preinstalled for all new sign-ups and trials" of Business Central.
- For products, the merchant must "decide which system is the system of record for product data" before configuring the direction.
- For inventory, "Business Central acts as the system of record for inventory"; stock is pushed one way, as projected available balance or free inventory, every 15 to 30 minutes.
- For customers and B2B companies, "most businesses manage customer records, payment terms, and credit limits in Business Central and export them to Shopify", and "mixing both directions without a clear ownership rule can lead to conflicting records".
- For orders, a real-time notification from Shopify triggers a batch import job; each sales document keeps the Shopify order id; refunds create credit memos and a refund "still pending in Shopify" is retried later.
- The connector "uses expiring offline access tokens" that it refreshes automatically, with a refresh token valid for up to 90 days without a successful refresh.

The last bullet is the small print of what "a connector as a product" actually maintains: tokens, retries, version changes, mapping options. Shopify's 2026 guidance adds that certified ERP apps "undergo a 100-checkpoint review" and that "custom integrations introduce more risk and maintenance overhead".

SAP's learning material for integrating S/4HANA Cloud with SAP Commerce Cloud describes the suite-era division from the ERP side: master data replicated from the ERP to the commerce platform, storefront orders sent automatically to the ERP.

> **INFERENCE, NOT DOCUMENTED:** The Business Central inventory rule ("Business Central acts as the system of record for inventory") is the ERP-as-hub position for stock, and it is correct for the merchant the connector is built for: one warehouse, one storefront, no WMS. The same rule applied at Kestrel & Pine's size, with a WMS, six stores and three marketplaces, is the stock chain that oversold. Which rule is right is a property of the merchant, not of the connector.

Master Data: Who May Create and Change What

Master data is the slow-changing reference data (items, customers, prices, locations) that many systems copy; **master data ownership** says which system may create and

change each record. The table below is my recommendation for a merchant at the Omnichannel stage; at the Starting and Growing stages, the platform takes the PIM's and OMS's rows.

Data	Owner	Readers	Note
Item identity (SKU, GTIN barcode number), cost, supplier, purchasing units	ERP	PIM, platform, WMS, EDI	The item exists in the ERP before it can be bought or valued
Product content (names, descriptions, attributes, media, translations)	PIM	Platform, marketplaces	Kestrel & Pine's "three places" problem was the absence of this owner; What Are We Selling? Products, Variants and the Catalog
Wholesale list and contract prices	ERP price lists, published as price events with effective dates	OMS, platform B2B catalogs, EDI	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
Consumer prices	The platform's market price lists	POS, marketplace connector	The Price Drift was a consumer price copied through the ERP connector
Business accounts, payment terms, credit limits	ERP	Platform B2B portal, EDI, invoicing	Business Central's advice; Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals

Data	Owner	Readers	Note
Consumer accounts	Platform	ERP (for invoicing), service tools	One Customer, One Product: Identity and Master Data Across Systems for the matching problem
Stock quantity and availability	WMS for physical stock; OMS or availability service for available-to-promise (what a channel may still sell)	Platform, marketplaces, EDI	Never the ERP at the Omnichannel stage
Stock value	ERP	Finance, data warehouse	IAS 2 and the costing method
Orders	OMS; the platform before an OMS; never the EDI portal	ERP for posting, WMS	One order record
Invoices, credit notes, the ledger	ERP	EDI and Peppol gateway, finance	Numbering and archive live here

The product and pricing vendor Akeneo describes a PIM (product information management system) as "a central source of truth" for product content that "collects data from your ERP, MDM, PLM, and other systems" (MDM and PLM are master data management and product lifecycle management systems); OneStock's post says "ERP is the master for product data (if no dedicated PIM system exists)". Both are vendor sources, and both say the same thing from different sides: the ERP owns the item's identity and cost; something else owns what the shopper reads.

Trade-offs

Criterion	ERP as hub	Commerce as hub	OMS as hub	iPaaS, no hub
Integration surfaces to maintain	One, the ERP's	One, the platform's	Two: OMS and ERP	One per pair of systems
Availability latency	The ERP's batch cycle	The platform's	Seconds, by events	Whatever the slowest hop is
Order record across channels	The ERP's, if every channel feeds it	The platform's; weak for EDI and POS	The OMS's, by design	None
Cost	Lowest licenses; highest customization when channels multiply	Platform plus connector	Highest licenses; lowest custom code	Low licenses; high and hidden labor
Right when	One channel, one warehouse, B2B-heavy, under about €20M *(illustrative)*	D2C-first, one or two channels	Stores, wholesale, marketplaces and several countries	Never as a destination; acceptable as a transition

My recommendation follows the requirements table: if any channel needs availability in seconds and more than one channel creates orders, the ERP cannot be the hub, whatever the ERP vendor's connector assumes. If one channel creates orders and one warehouse holds stock, the ERP-centered suite is the cheapest honest architecture and an OMS would be the Giant's Boots.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting	An accounting package or a small ERP connected by a vendor connector; the platform owns orders and products; the ERP owns the ledger and purchasing	Nothing more; a preinstalled connector (Business Central's is the dated example) is enough if the entity–owner choices in its configuration are made consciously	The founder posts marketplace sales by hand; the first "which number is right" argument
2 — Growing	The ERP owns items, business customers and terms; the platform owns consumer customers; a connector or iPaaS carries orders in and stock out; the ERP partner builds the first integration and becomes a dependency	A written field-level contract for the connector; a daily reconciliation of orders between platform and ERP	Stock to the storefront routed through the ERP; a second order source (wholesale, marketplace) living only in the ERP

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
3 — Omnichannel	The ERP retreats to finance, purchasing, valuation and the cost side of the item master; an OMS owns orders and availability; a PIM owns content; the partner's connector is replaced by owned contracts	The finance-only rule; events from the owners, daily batches into the ERP; an e-document gateway for invoices; the ERP out of every real-time path	A peak day where the ERP's batch is the bottleneck; month-end over ten days; an order status that finance, support and the warehouse disagree on
4 — Group	A finance ERP per legal entity, or one ERP with company codes; shared OMS and PIM across brands; a shared integration platform with ownership rules	A single posting interface the OMS and WMS write to, with per-entity routing; master data governance as a function	Each brand with its own ERP connector to the same OMS; intercompany flows done by hand

Kestrel & Pine is at stage 3 with a stage 2 back office: the ERP owns too much, the connector is the partner's, and the ERP sits in the stock path. The month-end that takes eleven days is the finance-side warning sign; the Oversell is the commerce-side one.

In Practice

Microsoft's connector as a written entity–owner table. The Business Central Shopify documentation (September and October 2026) is the clearest public example of a vendor making the merchant answer the ownership question in configuration: products either direction, inventory in Business Central, business terms in Business Central, orders from Shopify. Read it as a template for the contract your own connector should have, not as the right answer for your merchant.

Shopify's two sentences. On ownership, Shopify's enterprise guidance (updated 28 August 2026) gives two sentences I would put on the wall of any integration team: "Document every field that moves between systems, and name the system of record for each" and "Decide in advance what happens when the two systems disagree." The second is the one most integration contracts lack.

Kestrel & Pine: Eva's order in the books. Eva Janssen's order #1072455 of 13 April 2027 is €235 including Dutch VAT at 21%: a Ridgewalker jacket at €199 and two pairs of merino socks at €18. Under Waypoint's target flow, the OMS sends the order to the ERP in the night batch of 13–14 April; the WMS sends the shipment confirmation of 14 April in the next batch; the PSP's payout file, received a few days later, carries the €235 iDEAL payment and the PSP's fee (assume an illustrative fee of about €0.30 for an iDEAL transaction). The ERP posts revenue of €194.21, recognized at delivery on 15 April, and VAT payable of €40.79, relieves stock at the jacket's landed cost of €58 plus the socks' cost, records the PSP fee as an expense, and reconciles the payout to the order through the PSP's reference, which Payments-to-Ledger: Draining the Suspense Swamp in the finance book explains in depth. When Eva exchanges the jacket for size L on 19 April, the exchange is a return and a new order in the OMS; the ERP receives a credit note for €199 and a new sale for €199 in the batch of 22 April, and the two net to zero in revenue while stock moves twice. Nothing in this paragraph needed the ERP within the minute, and nothing in it is visible to Eva.

Kestrel & Pine: the wholesale side. Bergzicht's spring reorder of 6 April 2027 (240 units, €19,860 excluding VAT) enters the ERP as an EDI order today and the OMS from Q4 2027. Either way, the ERP issues the INVOIC (the EDI invoice message) on 9 April from the WMS's shipment confirmation, posts a receivable due 9 May, and runs dunning (payment reminders) if payment is late; Bergzicht's credit limit and 30-day terms live in the ERP and are read at order acceptance. The ERP's job did not shrink here.

Kestrel & Pine: retiring the connector. The partner-built connector is replaced in two steps. In Q3 2027 Ruben writes the field-level contract it should have had, as the specification for the OMS-to-ERP batch: one row per field, with direction, cadence, system of record and the rule when the two disagree. Then the connector is switched off flow by flow as each order path moves through the OMS (marketplace orders in September, storefront orders in October, EDI and manual wholesale in December 2027), with the old paths retired in January 2028 and stock already out of the ERP's path under W1. The partner remains the ERP's implementation partner; it stops being the owner of an integration nobody else could read.

What Never Belongs in the ERP, and the Partner as a Dependency

Some things should never run in the ERP, because they are in the path of a sale and change at the storefront's pace: carts, search, promotion logic, real-time availability

for checkout, marketplace listings, customer-facing order status. Some things always belong there: the posting of every order, payment, fee, refund and VAT amount; the cost of stock; the legal invoice and credit note; receivables and dunning; supplier orders and landed cost. The grey area is master data, and the ownership table above settles it.

The **ERP implementation partner** is an architectural dependency most merchants do not list. The connector is usually theirs, undocumented, and priced into the ERP relationship. That is the Lynchpin: a capability customers depend on, running through one firm, with no exit plan. The remedy is not to fire the partner but to treat the connector as a contract the merchant owns: a field-level specification, a named internal owner, idempotent handling, a reconciliation job and a monitored failure alert, which is Waypoint's W5 applied to one integration. Who Builds and Who Owns: Teams, Vendors, Agencies and Governance covers the partner relationship as a management question.

When It Goes Wrong

Duplicate sales orders. The platform's webhook fires twice, or the nightly batch is re-run after a partial failure, and the ERP holds two sales orders for #1072455. Finance sees double revenue until month-end. Detection: one ERP document per order number is a rule the batch enforces, and a daily count of orders in the OMS against sales orders in the ERP. Recovery: cancel the duplicate before posting; after posting, a reversing entry with a reference to the original.

Posted before paid. The order reaches the ERP as a sale before the payment is captured, and the customer's bank declines. Detection: the order event carries payment status, and the batch posts only captured orders. Recovery: a credit note for the uncaptured sale, and a rule that the PSP's capture event, not the order event, is what makes a sale postable.

Refunded twice. Kestrel & Pine's Double Refund of January 2027: 212 returns refunded once by the returns portal through the platform's API and once by support in the PSP console, €19,400 twice. The ERP received and posted both, which is how finance found it. Detection: a daily reconciliation of refunds against PSP refund records by return number; the ERP is where the two paths meet. Recovery: The Way Back: Returns, Exchanges, Refunds and Reverse Logistics owns the fix; the ERP's role is to be the reconciliation point.

The item that does not exist. Merchandising creates a variant in the platform; the ERP has no item for it; the order batch fails on that line and, because nothing is idempotent, the whole night's batch. Detection: the batch reports per line, not per file. Recovery: create the item in the ERP, replay the line; and fix the cause, which is an item created outside its owner.

The batch window meets the peak. The nightly order batch, sized for about 800 orders, meets Black Friday's 11,500 (31,000 in the week) and runs into the morning while the stock sync waits behind it. Detection: a batch duration alert. Recovery: split the batch

by flow so a slow one does not block a fast one, and take the ERP out of the stock path for good; Black Friday Every Year: Peak Load, Resilience and Recovery covers the peak plan.

The partner changes the connector. The ERP's new release changes a field; the partner updates the mapping to keep the connector running and tells nobody, and a flow starts failing quietly. The Price Drift shows the cost: one failed push of the German price list was reported as a success, and the storefront sold at €179 for nine days. Detection: a reconciliation that compares prices across systems daily. Recovery: the contract, under version control, with a change notice as a condition of the partner's work.

What to Take Away

Established.

- An ERP's jobs in a commerce business are the ledger, purchasing, inventory valuation and master data; inventory valuation is an accounting function governed by IAS 2 and the costing method, distinct from stock availability.
- ERP integration is batch by default: the vendors' own documentation (Business Central's job queues, Shopify's "real time or through a daily batch", OneStock's "hourly or nightly") says so, and batch is the right rhythm for finance.
- Ownership must be named per field, with a rule for disagreement, as Microsoft's connector documentation and Shopify's 2026 guidance both state.

Depends on context.

- ERP-as-hub is right for one channel, one warehouse and a B2B-heavy business; it becomes the bottleneck when stores, wholesale and marketplaces each need stock in seconds, because the ERP's batch cycle becomes the availability latency.
- An OMS as hub is right at the Omnichannel and Group stages and wrong before them, where it is cost and a second integration surface nobody needs.
- A vendor connector is right when the merchant matches the merchant the connector was built for (its documented ownership rules are the test); otherwise its configuration encodes someone else's architecture.

The author's synthesis.

- The finance-only rule: the ERP is the system of record for finance and nothing else, consumes everything by daily batch from the systems that own the facts, and feeds no channel directly: what it owns for channels (wholesale prices, terms, credit) leaves it as events. It gives the ERP a smaller job and a more important one, because a ledger that is complete and reconciled is worth more than one that is fast.

- The connector your ERP partner built is the most important undocumented integration you own; write its contract before you replace it, because the contract is what you are really buying.

Questions to Ask

1. For each of orders, shipments, payments, products, stock and customers, which system is the system of record, and is that written down with a rule for disagreement?
2. Does any channel read stock or availability from the ERP? If so, what is the end-to-end delay from a sale to the storefront's number?
3. Who built the platform-to-ERP connector, who can change it, and where is the field-level specification?
4. Is the ERP in the path of any real-time decision (checkout, availability, order status to the customer)?
5. How does the ERP learn about marketplace orders and store sales, and in what shape?
6. What reconciles orders in the commerce systems against sales orders in the ERP, how often, and who reads the result?
7. Is a sale posted on the order event or on the payment capture, and what happens to a declined payment?
8. If the ERP partner ended the relationship tomorrow, which integrations would you be unable to change?

The ERP is one of a dozen systems that must behave like one. The next part is about how they talk: the styles of integration, the events and reconciliations that let separate systems disagree and recover, the contracts between them, and the identity and data questions that follow. Continue with Part VI — INTEGRATION: Making Separate Systems Behave Like One.

Sources and Evidence

Company, product and investor sources

- Get started with the Shopify Connector, Microsoft Learn, 2026: preinstalled connector and token mechanics.
- Synchronize items and inventory (items), Microsoft Learn, 2026: "decide which system is the system of record for product data" and the SKU mapping options.
- Synchronize inventory, Microsoft Learn, 2026: "Business Central acts as the system of record for inventory" and the 15–30 minute cadence.

- Synchronize customers and companies, Microsoft Learn, 2026: customer, terms and credit-limit ownership advice.
- Synchronize and fulfill sales orders, Microsoft Learn, 2026: order import, the Shopify order id, refunds and credit memos.
- Design details: costing methods, Microsoft Learn, 2026: the five costing methods and the no-change rule.
- Shopify launches Global ERP Program, Shopify, 2021: the program and its launch partners (older than three years).
- ERP Integration: Methods, Benefits, and Examples, Shopify Enterprise blog, 2026: "real time or through a daily batch", the review checkpoints and the two ownership sentences.
- Implement an Integration of SAP S/4HANA Cloud with SAP Commerce Cloud, SAP Learning, n.d. (accessed 2026-11): master data to commerce, orders to the ERP.
- OMS vs ERP: the architecture shift reshaping commerce, OneStock, 2026: the OMS-as-hub argument and the batch-cycle observation (vendor source).
- Sana Commerce, Sana Commerce, n.d. (accessed 2026-11): the "built on your SAP or Dynamics ERP" positioning.
- Handel: voorraad, orders en boekhouding, Exact, n.d. (accessed 2026-11): the mid-market ERP's self-description (translated from Dutch).
- ERP software, AFAS, n.d. (accessed 2026-11): one system, one database (translated from Dutch).
- What is a PIM?, Akeneo, 2026 (accessed 2026-11): the PIM as source of truth for content, collecting from the ERP (vendor source).

Standards, regulation and official documentation

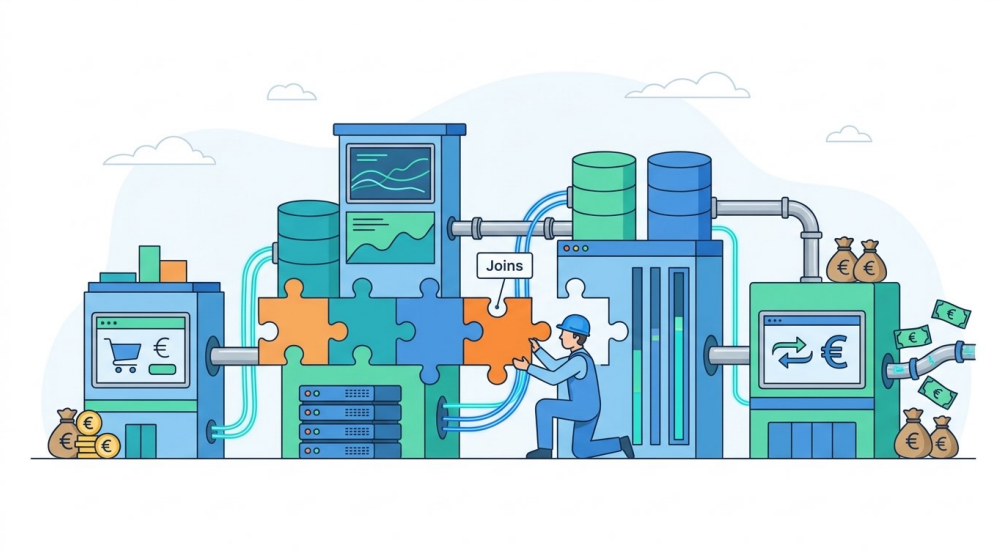
- IAS 2 Inventories, IFRS Foundation, n.d. (accessed 2026-11): measurement at the lower of cost and net realisable value; FIFO and weighted average.

Research and practitioner sources

- The hub comparison, the finance-only rule, the master data ownership table and the scale-stage table are the author's synthesis of the sources above and of common practice; Kestrel & Pine's connector behavior, month-end duration and the iDEAL fee are illustrative.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Part VI — INTEGRATION: Making Separate Systems Behave Like One



IN THIS PART, YOU WILL: Follow the facts that cross between Kestrel & Pine's thirteen systems and learn the five questions every join must answer: how the systems talk, how they recover, what they promise, whom they mean and what they count.

KEY POINTS:

- **Integration is decided per flow, and every flow needs an owner**. Stock, orders and prices want events with a safety net, products want scheduled publication, finance wants a daily batch. Kestrel & Pine's

inventory found 34 integrations, 13 with nobody to call in January 2027, and none that compared stock or orders between two systems.

- **Separate systems are built to disagree; design them to recover and to prove it.** Every delivery arrives at least once and in any order, so a receiver needs an idempotency key, a sender an outbox, a partial failure a compensating action, and every flow a reconciliation job that compares the two ends. Every contract has a version and a sunset date, and the consumer's job is to write that date down.
- **Moving the data is half the work; meaning the same thing by it is the other half.** One customer became four records and one month's orders became three counts because nobody had named the master record for a customer or the definition of an order. One owner per entity and per metric, with the links and the bridge published, is what makes joined systems behave like one.

FAILURE PATTERNS THIS PART ADDRESSES:

- **The Spaghetti Junction** — 34 connectors built one need at a time, the mapping between systems living inside each connector, and 13 flows with no owner in January 2027.
- **The Message in a Bottle** — a webhook or a scheduled sync sent once and trusted: the Double Refund, the Price Drift and 390 variants whose stock drifted while nothing compared the two ends.
- **The Pulled Rug** — the marketplace connector retired an API version on a published date, and the only person who had read the notice had left the agency.
- **The Familiar Stranger** — Eva Janssen is an account, a loyalty entry, a subscriber and a returns contact in four systems with no shared identifier.

A growing merchant does not run one system. It runs a dozen or more, each bought because it was good at one thing: the platform sells, the warehouse system picks, the ERP keeps the books, the connector talks to the marketplaces. **The trouble is that an order, a stock level, a price or a customer has to pass between them, and every handoff is a place where it can arrive late, twice, in the wrong shape or not at all.** This part is about the joins that carry the facts between them, where most of the incidents this book opened with happened.

Figure 1: *Every handoff between systems is a place where an order, a stock level or a price can arrive late, twice, in the wrong shape or not at all.*

Part VI sits underneath every step of Eva's jacket. Her order of 13 April 2027 is created in the platform at 21:14 and crosses to the integration platform (the iPaaS, a hosted tool that runs flows between systems) and on to the warehouse; her payment confirmation crosses from the PSP (payment service provider) to the platform; the jacket's stock reduction travels back through the ERP to the storefronts and the marketplaces; her exchange crosses from the returns portal to the platform and the PSP. At the end of the month she and her order are counted, and whether she is one customer and her order one order is decided in this part too.

Kestrel & Pine, the fictional outdoor brand this book follows, runs thirteen systems joined by 34 integrations in four styles. **The claim of this part is that making separate systems behave like one is not a tool you buy but five decisions you make, flow by flow: how it moves, how it recovers, what it promises, whom it means and what it counts.** Waypoint's decisions W5 (integration standards) and W7 (one definition of an order) and the June 2027 customer-identity rule are those decisions written down.

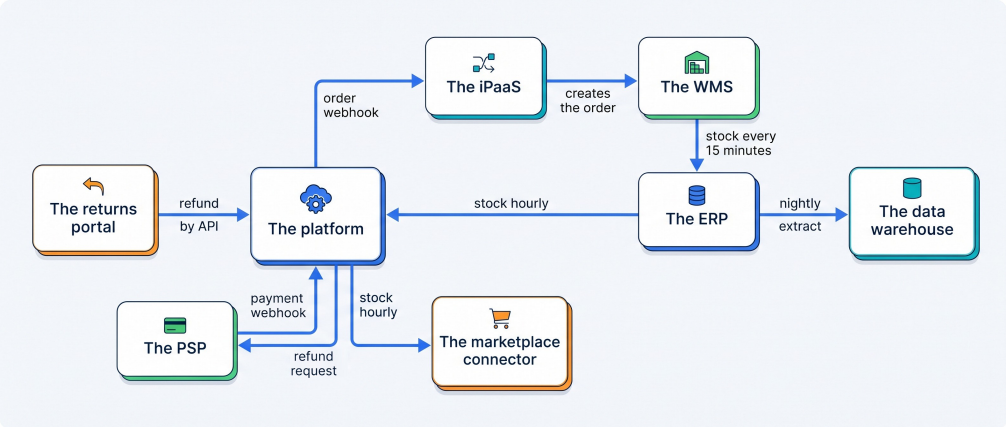


Figure 2: *The joins Eva's order crosses in April 2027: eight systems, four styles, and a stock figure that takes up to two hours to travel from the warehouse to the marketplaces.*

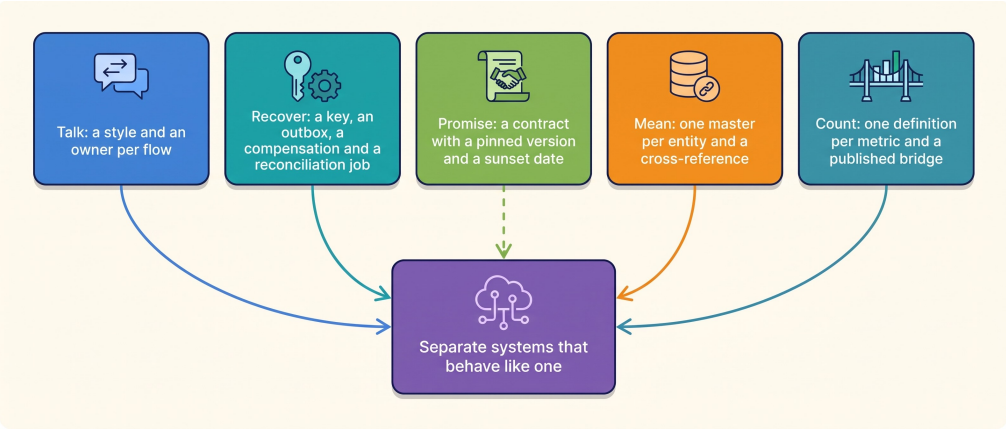


Figure 3: *The five questions of Part VI and the answer each chapter gives; every incident in the part is a flow that had no answer to one of them.*

Chapters in this part

APIs, Events, Files and Middleware: How Commerce Systems Talk decides **how commerce systems should exchange data, and how to choose per flow**: four styles (file, shared database, direct call, message) and five triggers, a direct call only where the next step needs the answer, a webhook as a notification and polling as the truth, a hub only with ownership rules. The case is the integration inventory Noor Visser built in February 2027: 34 flows, 13 with no owner as of January, 3 with a reconciliation job and none on stock or orders, and the rule that no flow goes live without a row and no row without an owner.

Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation decides **how systems that can only be eventually consistent are kept from drifting apart for good**: the receiver builds exactly-once with an idempotency key that names the business action, the outbox makes "saved" and "announced" one act, a cancellation after shipment is a compensation rather than a rollback, and reconciliation is a component with a card, a key, a tolerance and a named recipient. The cases are the Double Refund (212 returns, €19,400, two writers and no shared key) and the first stock reconciliation of May 2027, which found 2.3% of active variants, about 390, with the wrong figure.

Promises Between Systems: API Contracts, Webhooks and Versioning decides **how a system that other systems depend on can change without breaking them**: a contract is a promise with a shape, a meaning, a timing and a lifetime; change by adding; a deprecation is a date pair plus monitoring; a webhook is a signed hint to verify, answer fast, deduplicate and sweep. The case is the Deprecated Endpoint: a 2021 agency script calling a version the marketplace connector retired on Monday 8 February 2027, two days of stale stock on bol, and a fix of two inventory columns: the pinned version and its sunset date.

One Customer, One Product: Identity and Master Data Across Systems decides **how a merchant knows that two records in two systems are the same customer or product, and who owns the answer**: name a master per entity, keep a cross-reference of every other system's key, match exact keys automatically and send the uncertain band to a person, keep every merge reversible, and let consent travel with the record. The case is the four Evas: 410,000 platform records, 38,000 loyalty entries, 165,000 subscribers and 61,000 returns contacts with no shared field, and the June 2027 identity rule whose first run linked about 21,000 loyalty entries and 96,000 subscribers to platform customers.

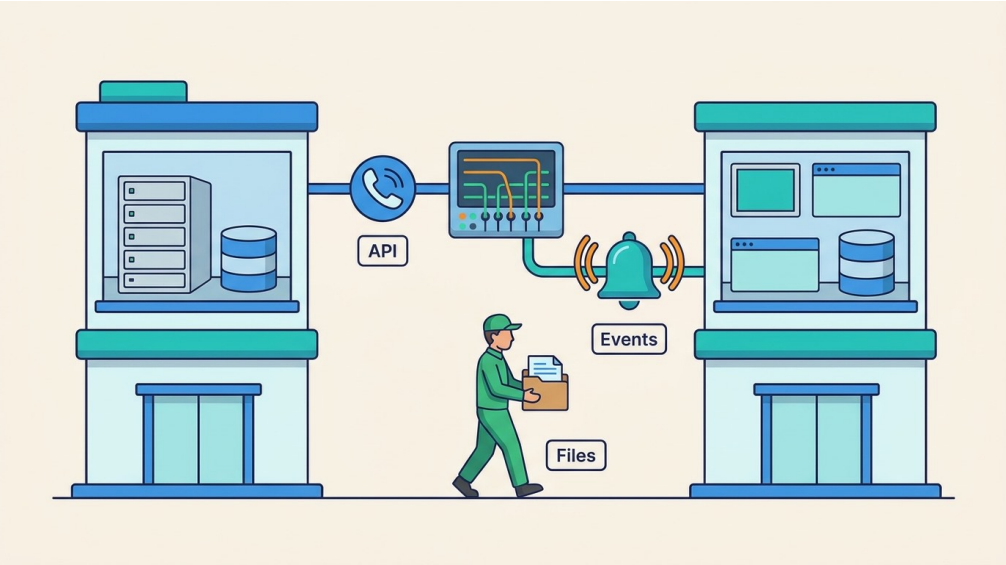
Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics decides **why three reports show three order counts, and how to build analytics the business trusts**: keep operational and analytical data apart, copy every system into one warehouse, define each metric once in a metrics layer with an owner, and publish the bridge between the counts next to the number. The case is February 2027: 21,050 orders in the platform, 22,710 in the ERP and 24,140 in the warehouse report, each right about what it counted; under decision W7 they became 23,150 orders with a bridge, and the first unexplained difference, in August 2027, was 23 marketplace orders a connector had dropped.

Begin with APIs, Events, Files and Middleware: How Commerce Systems Talk.

Sources and Evidence

Every public claim in this part is sourced and dated in the chapter that makes it, under that chapter's Sources and Evidence. Kestrel & Pine is fictional; recommendations and labeled assumptions are the author's judgement.

24. APIs, Events, Files and Middleware: How Commerce Systems Talk



IN THIS CHAPTER, YOU WILL: Learn the four ways commerce systems exchange data, the mechanisms that trigger each exchange, how to choose a style per flow, and why an inventory of integrations is the first management tool.

KEY POINTS:

- **There are four styles and five triggers, and the right one is chosen per flow, not per company.** A file, a shared database, a direct call or a message moves the data; a call, a webhook, a poll, a schedule or a database log decides when. Stock and orders want events plus a safety net, products want scheduled publication, finance wants a daily batch.
- **Point-to-point connectors multiply with every system; a hub only helps when it comes with ownership rules.** An integration platform (iPaaS) centralizes mapping, retries and monitoring, but a hub whose

flows have no owner, contract or reconciliation recreates the same tangle in one place.
- **The integration inventory turns integration debt into a list.** One row per flow, with source, target, style, owner, contract version, failure handling and reconciliation. Kestrel & Pine's inventory found 34 flows, 13 of them without an owner in January 2027, and none with a check on stock or orders.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Spaghetti Junction** — every pair of systems has grown its own connector, with its own owner or none, because each was built for the need of the moment.
- **The Message in a Bottle** — a scheduled sync or a webhook is sent once and trusted, so a silent failure stays silent until a customer finds it.
- **The Lynchpin** — a flow that only an agency script, one consultant or one vendor connector understands is a single point of failure the business does not know it has.

A commerce business at the Omnichannel stage (the third of this book's four scale stages: stores, wholesale, marketplaces and several countries) runs a dozen or more systems, each bought because it was good at one thing. The hosted platform (software as a service, SaaS: rented by the month and run by the vendor) sells, the warehouse management system (WMS) picks and packs, the ERP (enterprise resource planning system) keeps the books and the item master, the marketplace connector talks to bol and Amazon.de. None of them is wrong. **The trouble is that an order, a stock level or a price has to pass between them, and every handoff is a place where it can arrive late, twice or not at all.**

That handoff is an **integration**: the arrangement by which two systems exchange data or trigger each other's work. The choice of *how* two systems talk decides how fresh a stock figure is in the channel that sells it, what happens when one side is down, and who has to understand both ends when something breaks. Those are business properties. The two-hour oversell window in Kestrel & Pine's Black Friday was not a bug in any system; it was the sum of three scheduled syncs, each reasonable on its own.

This chapter names the four basic styles of exchange, the mechanisms that trigger them, the three ways to organize many integrations (hand-built connectors, a hub, the vendor's own connector), and the rule I use to choose per flow. It ends with the one management tool that makes the whole thing visible: an inventory of every flow, with an owner on each row. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation takes the hardest style, events, and shows how to make it safe; Promises Between Systems: API Contracts, Webhooks and Versioning covers changing an integration without breaking it.

Where we are in the order's journey. Eva's jacket (the recurring order this book follows; see Meet Kestrel & Pine) is placed on the Dutch storefront on Tuesday 13 April 2027 at 21:14 and picked at Duiven the next morning. In between, the order crosses from the platform to the integration platform to the WMS, its payment confirmation crosses from the PSP (payment service provider: the company that takes the shopper's payment) to the platform, and the jacket's stock reduction starts a journey back through the ERP to the storefronts and marketplaces. This chapter is about those crossings: not what happens inside each system, but how facts get from one to the next.

Setting the Scene at Kestrel & Pine: Thirty-Four Lines Nobody Had Drawn

Kestrel & Pine, the fictional outdoor brand this book follows, runs thirteen systems and, in January 2027, **34 integrations** between them. When Noor Visser started as Head of Engineering that month, she asked Ruben Alves, the lead integration engineer, for the list. There was one for the fourteen flows that ran on **the iPaaS** (integration platform as a service: a hosted tool in which you configure flows that move data between applications, paid by usage). There was none for the rest.

Noor spent most of February building the **integration inventory**: one row per flow, with the source, the target, what moves, how, how often, who owns it, what happens when it fails and whether anything checks the result. Three flows were found only because data arrived that no listed flow sent. The finished table had 34 rows:

What the inventory found (February 2027, illustrative)	Count
Flows that move data by **files or batch exports** (EDI documents, that is electronic data interchange, the standard exchange of business documents between companies' systems; Peppol e-invoices on the European e-invoicing network; the nightly stock export from the POS, the store till system; data-warehouse extracts; spreadsheet imports)	9
Flows that **read another system's database directly** (the BI tool, for business intelligence reporting, on the ERP's reporting copy)	1

What the inventory found (February 2027, illustrative)	Count
Flows that are **direct API calls** (one system calling an operation another exposes through its application programming interface: checkout to the PSP, the returns portal to the platform's refund API, the WMS to the shipping service, the connector to three marketplaces, the stock pulls)	11
Flows that are **events, webhooks or queues** (the platform's order webhooks into the iPaaS, PSP payment webhooks, WMS shipment events, the connector's order imports)	13
Running on the iPaaS / as agency scripts / as vendor-built connectors or plugins	14 / 9 / 11
Flows with **no named owner** (as of January 2027; 5 still unowned in April 2027, after the ownership map)	13
Flows with a **reconciliation job** (something that compares both ends): PSP payouts, EDI invoices with Bergzicht, store cash	3
Flows that **write stock into the platform**	4 (the hourly ERP sync and three apps); with hand edits in the ERP and the platform admin, six writers of Duiven stock in all

Two numbers made the business case for Waypoint's integration standards (decision W5, below). In January 2027, thirteen flows had nobody who would be called when they stopped, and nothing compared stock or orders between any two systems. The Black Friday Oversell, the Price Drift and the Deprecated Endpoint had all happened in flows with no owner and no check. Noor's summary to Mara Lindqvist, the CEO: "We have 34 integrations and 34 different answers to the question 'how do we know it worked?' "

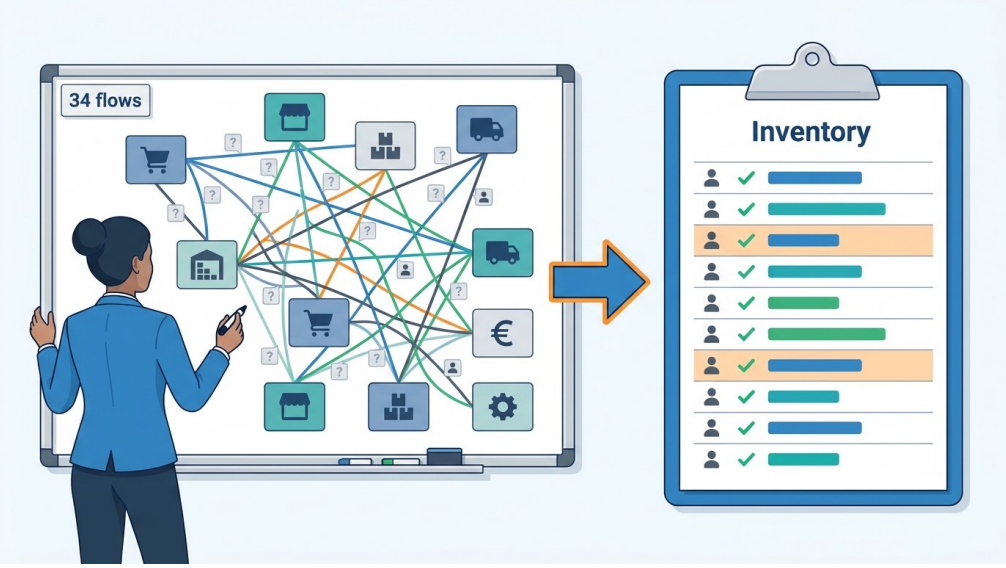


Figure 1: *Before the inventory, 34 integrations ran between thirteen systems, and 13 of them had nobody who would be called when they stopped; the inventory turned the tangle into a list.*

What an Integration Is, and Why the Joins Cost Money

Two systems need to exchange data when a fact one of them owns is needed by the other. The WMS knows how many Ridgewalker jackets in size M are on the shelf; the storefront needs that number to show "in stock". The platform knows Eva ordered one; the warehouse needs the order to pick it. The fact has a **system of record** (the one system whose value counts as the truth when copies disagree; Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record owns the idea), and every other system holds a **copy**.

An integration moves the fact from the owner to the copies, or sends a request the other way ("refund €199 on order #1072455"). Four properties of that movement matter to the business:

- **Freshness**. How old is the copy? A stock copy an hour old sells units that are gone; a product description an hour old is fine.
- **Reliability**. When the move fails, is the fact sent again, lost, or sent twice?
- **Coupling**. How much does each side need to know about the other? A flow that depends on the exact field names of the WMS's interface breaks when the WMS vendor changes them.

- **Ownership**. Who understands this flow, is told when it fails, and decides what to change?

The style is the main lever on the first three; the inventory is the only lever on the fourth.

The Four Styles: File, Shared Database, Direct Call, Message

The vocabulary most practitioners still use comes from Gregor Hohpe and Bobby Woolf's *Enterprise Integration Patterns* (2003; the book is old, the styles have not changed, the technology under them has). It names four **integration styles**:

1. **File transfer**. One system writes a file; the other reads it later. The file is the contract. Commerce examples: the nightly product feed to a marketplace, the EDI order Bergzicht's system sends through the EDI provider, the POS export of store stock to the ERP.
2. **Shared database**. Two systems read and write the same tables. Common inside a suite where the web shop and the ERP are one product; risky across vendors, because any schema change breaks the other side and there is no record of who changed what.
3. **Direct call** (Hohpe and Woolf say *remote procedure invocation*). System A calls an operation System B exposes over the network, usually through an **API** (application programming interface: the set of operations a system lets other software perform), and waits for the answer. Commerce examples: the checkout asks the PSP to authorize €235; the returns portal asks the platform to refund €199.
4. **Messaging**. System A puts a message on a shared channel (a **queue** or **broker**: software that holds messages until the receivers take them) and continues without waiting; B, C and D read it when they are ready. Commerce example: "order #1072455 placed" is published once and consumed by the WMS, the ERP connector and the e-mail service.

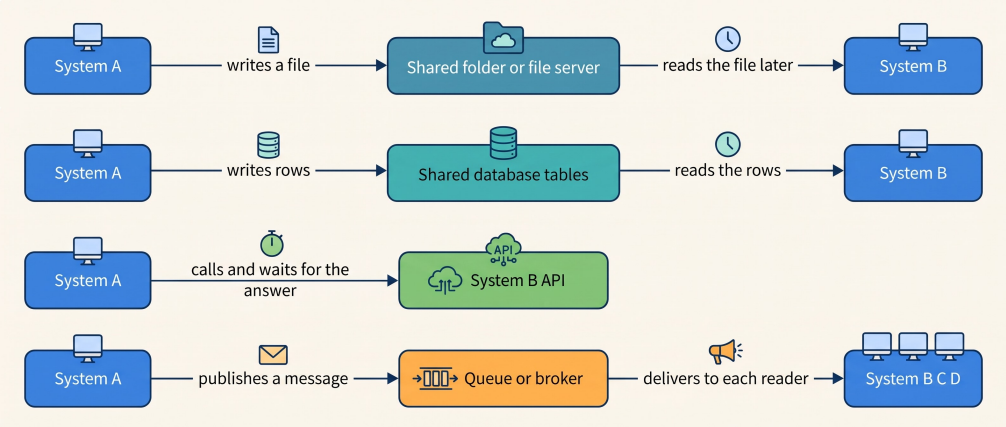


Figure 2: *The four integration styles. Only the direct call makes A wait for B; the other three let the two systems work at different times, which is what makes them resilient and what makes their copies stale.*

Two words recur. A **synchronous** exchange makes the caller wait for the answer (the direct call): the checkout cannot continue until the PSP says yes or no. An **asynchronous** exchange lets the sender continue (files, messages): the storefront does not wait for the warehouse to confirm it received the order. Synchronous is simple to reason about and fragile under load: if B is slow, A is slow; if B is down, A fails. Asynchronous is resilient and introduces a delay during which the two sides disagree. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation is about living with that delay.

Hohpe and Woolf's warning is the one to remember: "Tightly coupled applications make numerous assumptions about how the other applications work." A direct call to the WMS assumes the WMS is up, fast and unchanged; a message assumes only that the broker will hold it.

Five Ways to Trigger an Exchange

The style says how the data moves. A second choice says **when**: who notices that something changed and starts the exchange. Five mechanisms do that work, and most merchants run all of them.

Mechanism	Who starts it	Typical delay	What the receiver must handle	Typical commerce use
Direct API call	The caller, when it needs the answer	Milliseconds	Timeouts, partial success, retry storms under load	Price, tax or stock check at checkout; payment authorization; a refund request
Webhook	The source, when something happens: one HTTP request to a URL you registered	Seconds	Missed, duplicated and out-of-order deliveries; must answer fast	Order placed, payment captured, shipment created
Polling	The receiver, on a timer: "what changed since my last visit?"	Minutes	Rate limits, large pages, clock differences	Catching up after an outage; the safety net under webhooks
Scheduled batch or file	Either side, on a clock	Hours to a day	Late files, partial files, stale by design	Product feeds, EDI documents, nightly stock and finance exports
Queue or event stream	The source publishes; each consumer pulls at its own pace	Seconds	Duplicates (delivery is "at least once"); ordering only per key	Stock and order events with several consumers

Mechanism	Who starts it	Typical delay	What the receiver must handle	Typical commerce use
Change data capture (CDC)	The database's own change log, read by a connector	Seconds	A row change is not a business event; someone must translate it	Turning an old ERP or order management system (OMS) into an event source without rewriting it

(The author's synthesis of the vendor documentation cited under "In Practice" and in the next chapter.)

Two of these need plain words. A **webhook** is a notification, not a guarantee: the source makes one HTTP request to your address when something happens, and if your server was down, slow or busy, the request is retried for a while and then dropped. Shopify's developer documentation (n.d., accessed November 2026) says its HTTPS webhooks are retried eight times over four hours, that a subscription created through the API is deleted after eight consecutive failures, and that delivery "isn't always guaranteed." Stripe's (n.d., accessed November 2026) retries for up to three days. Both tell integrators the same thing: run a periodic **reconciliation job** that asks the source for everything changed since the last run. The webhook is a speed-up on top of polling, never a replacement for it.

Change data capture reads a database's transaction log, the record every database keeps of each change, and turns it into a stream of "row inserted, updated, deleted" events. Debezium, the most used open-source tool for this (documentation for version 3.7, accessed November 2026), describes itself as a set of source connectors that use each database's own change-capture features. CDC is the honest way to get events out of a system never built to publish them. Its limit is meaning: "row updated in the item ledger table" is not "stock changed for SKU X at location Y" (a SKU being one sellable variant, one size in one color), and someone has to write that translation and keep it current.

"Event-driven" itself means several things. Martin Fowler's 2017 article separates **event notification** (the source says "order #1072455 changed" and receivers call back for details; low coupling, hidden flow) from **event-carried state transfer** (the event carries the data; receivers keep their own copy and never call back; faster, but every receiver now holds a copy that can go stale). A **stock event** in this book's sense (Is It in Stock? Inventory, Availability and the Promise You Can Keep) is the second kind: "SKU 2113-M-moss at Duiven: 14 on hand, 3 reserved, as of 21:14:07." Fowler's other two, event

sourcing and CQRS (command query responsibility segregation), are internal design choices of a platform or an OMS, not integration styles.

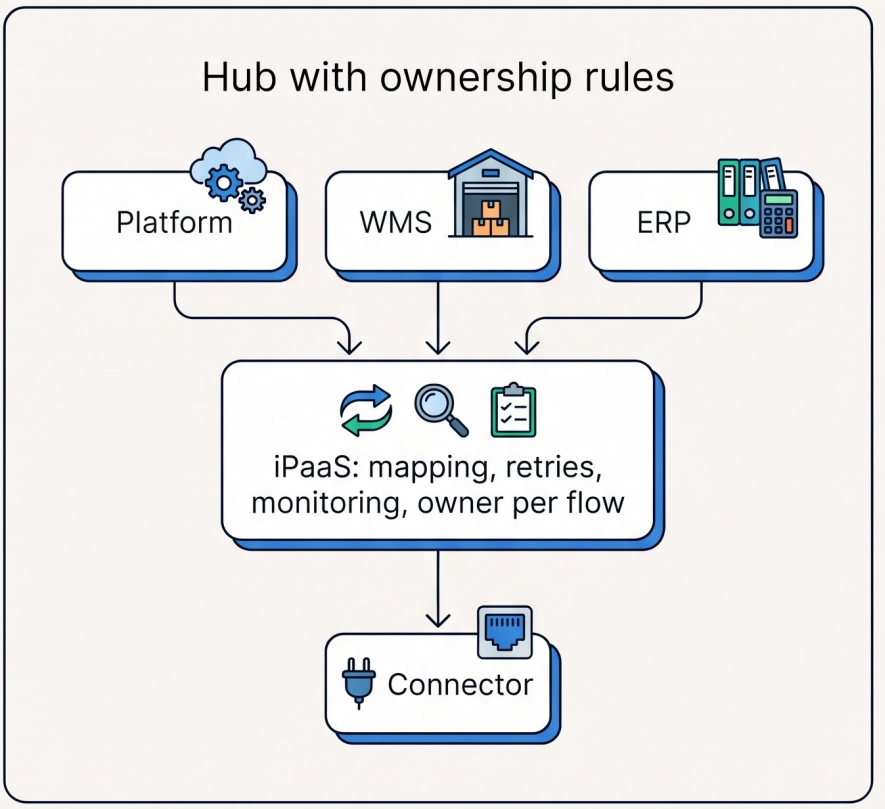
Point to Point, Hub, or the Vendor's Connector

With thirteen systems there is a second question: how the whole landscape is wired. There are three answers, and most Omnichannel merchants run all three at once.

Point to point. Each pair of systems that needs to talk gets its own connector, built by whoever needed it: the agency, the ERP partner, an engineer who has since left. The first three are cheap and fast. By the tenth, every change to one system means finding and testing every connector that touches it, and the mapping between two data models (which WMS field means which platform field) lives in code one person understands. This is The Spaghetti Junction, and it is what buying one good system per need, one need at a time, produces.

A hub. An integration platform sits in the middle; every system talks to the hub, which holds the mapping, the schedule, the retries and the monitoring in one place. The hosted form is the iPaaS. Gartner's definition, as relayed by an encyclopedia entry (2015 retrieval; the Gartner page itself could not be fetched), is "a suite of cloud services enabling customers to develop, execute and govern integration flows between disparate applications," with prebuilt connectors so that "no software development" is needed for common pairs. The catch is that a hub does not create ownership. Kestrel & Pine's fourteen iPaaS flows were the best-documented of the 34 and still had three with no owner.

The vendor's connector. Between the two sits a connector built and maintained by the vendor of one of the systems. Microsoft's documentation for Dynamics 365 Business Central's Shopify Connector (published October 2026) shows what such a connector decides for you. It tells the merchant to "decide which system is the system of record for product data" before configuring, states that for inventory "Business Central acts as the system of record" and pushes stock one way at a recommended interval of 15 to 30 minutes, and for orders runs a batch import that a real-time notification from Shopify merely *triggers*. That last detail is the webhook-plus-polling pattern built into a product. A vendor connector encodes an ownership model; if it matches yours, it is the cheapest reliable option, and if not, no configuration will fix it.


Hub with ownership rules
Platform
WMS
ERP
iPaaS: mapping, retries, monitoring, owner per flow
Connector

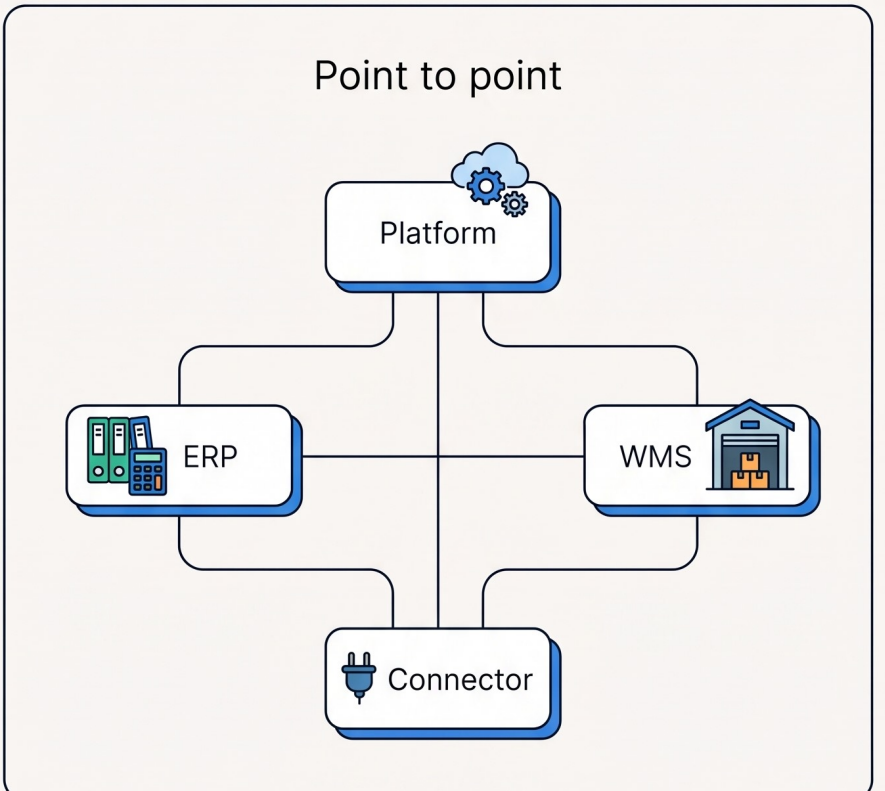

Point to point
Platform
ERP
WMS
Connector

Figure 3: *Four systems need six point-to-point links; a hub needs four. The saving grows with every system added, but only if each flow through the hub still has an owner, a contract and a check.*

The arithmetic: n systems that all need to talk need up to $n(n-1)/2$ point-to-point links, 78 for thirteen systems; through a hub they need 13. The real saving is one place to find a flow's mapping, schedule, failure handling and log.

Hub costs are hard to compare. Fetched in November 2026: MuleSoft packages "start at $2,000 per month, billed annually" on usage-based credits, with a $50,000 minimum for its pre-commit model; Boomi's pay-as-you-go starts at $99 a month plus usage; Workato and Celigo publish no prices; Make bills in credits with a free plan of 1,000 a month; n8n charges per workflow execution, from €20 a month for 2,500 to €667 for 40,000, billed annually. The lesson is that the metering unit (credits, executions, connections, concurrency) differs per vendor, so the only honest comparison is per flow at your volume: Kestrel & Pine's roughly 950 orders a day (D2C, marketplace and wholesale, the 2026 average), each triggering five or six flow executions, costs very differently under per-execution and per-connection pricing. The low end is priced for a Growing-stage merchant's first three integrations; a $50,000 pre-commitment signals a Group-stage buyer.

> **INFERENCE, NOT DOCUMENTED:** Every iPaaS pricing page in November 2026 sells "AI agents" beside integration. What a merchant buys underneath is the same four functions as in 2020: connectors, mapping, scheduling and monitoring. I would evaluate on those and treat the AI framing as a feature that may not survive the next pricing page.

Choosing Per Flow

The mistake I see most often is choosing a strategy for the company ("we are event-driven now"). Flows differ in what they need, and the style follows from the needs. "Consistency" below means how closely the copies must agree with the system of record; "freshness" is how old a copy may be.

Flow	System of record (typical)	Freshness	Consistency	Latency of the exchange	Availability
Stock	WMS, POS, 3PL (a third-party warehouse); an availability service for what may be sold	Seconds to minutes; the oversell window is the sync interval	High per SKU and location; a wrong copy sells a unit that is gone	Seconds	Must survive a consumer being down: queue it
Orders	Platform or OMS	Seconds for the warehouse, minutes for finance	Never lost, never doubled; one order number family	Seconds	Must survive the WMS being down: queue it
Prices	The platform, the ERP or a pricing service	Minutes; promotions with start times want the minute	A stale price is a broken promise with legal weight	Minutes	Fail loudly, never silently
Products and content	PIM, the product information management system (or the ERP item master before one exists)	Hours; a season is published, not streamed	Completeness over speed	Hours	Can wait a day

Flow	System of record (typical)	Freshness	Consistency	Latency of the exchange	Availability
Customers	Platform, CRM (customer relationship management system) or an identity service	Hours	Matching is the hard part, not transport (One Customer, One Product: Identity and Master Data Across Systems)	Hours	Can wait
Finance (invoices, payments, payouts)	ERP and the PSP	Daily; finance works in periods	Exact to the cent; reconciled, not trusted	Hours	Complete, not fast

(Typical values; the author's recommendation, not a standard.)

From those needs, the rule I use:

Flow	At the Growing stage	At Omnichannel and beyond	Why
Stock	The inventory owner pushes to the channels every 15–30 minutes (the interval Business Central's connector recommends, October 2026)	Stock events per SKU and location into an availability service, plus a daily reconciliation and a safety buffer	The oversell window equals the sync interval; at 38 orders a minute in the peak hour, fifteen minutes is 570 orders

Flow	At the Growing stage	At Omnichannel and beyond	Why
Orders	A webhook starts the import; a polling sweep catches what it missed	Order events to the OMS, WMS and ERP connector; every consumer idempotent (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation)	Orders must never be lost or doubled; a webhook alone cannot promise that
Prices	Scheduled sync, hourly or faster, with an alert when it fails	Events for promotion start and end; scheduled sync as the net	The Price Drift was a scheduled sync that failed without an alert
Products	Scheduled publication from the system of record	Event on change for the storefront, plus a nightly full publication	A nightly full copy heals any missed change
Customers	Copied with each order; periodic merge	An identity service or customer data platform	The problem is deciding whether two records are one person
Finance	Nightly batch to the ERP, reconciled against the PSP's payout report	Same; reconciliation becomes a data-platform job across brands	A daily batch that is checked beats an event stream nobody balances

This is what Waypoint's decision **W5** says for Kestrel & Pine: stock, orders and prices move by events; products by scheduled publication; finance by daily batch; and every flow has an owner, a contract, idempotent handling and a reconciliation job. The next chapter explains the last two terms; the reason the rule is per flow is this table.

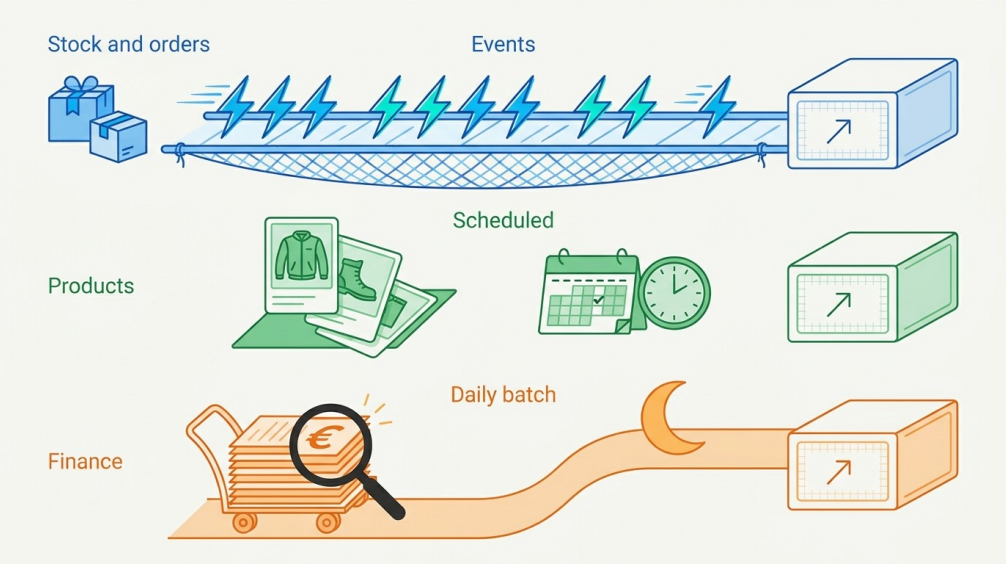


Figure 4: *The style follows the flow: stock and orders move as events with a reconciliation net beneath them, products by scheduled publication, finance by a daily batch that is checked.*

The Trade-Offs

	Direct API call	Webhook plus polling	Queue or event stream	Scheduled file or batch	Shared database
Freshness	Immediate	Seconds, gaps healed by the poll	Seconds	Hours to a day	Immediate
When the other side is down	The caller fails or waits; under load, a retry storm	Deliveries are retried for hours, then dropped	Messages wait in the queue	The file is late	Both sides fail

	Direct API call	Webhook plus polling	Queue or event stream	Scheduled file or batch	Shared database
Coupling	High: on the callee's availability and interface	Medium: on the payload format and the callback address	Low: on the message format	Low: on the file format	Highest: on the schema, with no record of who changed what
Duplicates and order	None if the call is idempotent	Expected; the receiver deduplicates	Expected; delivery at least once, order per key	A rerun file is the risk	None
Who owns the mapping	The caller	The receiver	Each consumer, or the hub	Whoever writes the file spec	Nobody, which is the problem
Fits	Decisions that need an answer now: payment, tax, a stock check at checkout	Receiving changes from a SaaS vendor: the platform, the PSP, the returns portal	Facts with several consumers that must not be lost: stock, orders	Facts that change by season or by day: products, finance, EDI	A suite from one vendor; nothing across vendors
Limits	Every dependency's outage becomes yours	A deleted subscription stops the flow silently	Needs the discipline of the next chapter	Stale by design; a late file is a quiet failure	Breaks on the first upgrade

Three judgements follow. **Use a direct call only where the answer is needed before the next step can happen**: whether the card is authorized, what the tax is, whether this unit may be promised; everything else should be asynchronous, so one system's outage does not become every system's outage. **Treat a webhook as a notification and polling as the truth**; the vendors say so in their own documentation. **A scheduled batch is not old-fashioned**; it is right for finance and product publication, provided a missed run

raises an alert.

Across Scale Stages: When It Fits and When It Limits

The four stages are the book's (see What Changes at Each Scale: From First Store to Group).

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting	The platform's apps and one or two vendor connectors (shop to accounting, shop to the 3PL); the merchant owns no integration code	An inventory anyway, even with four rows, and a weekly look at each app's failure log	A second channel whose orders are keyed in by hand; a stock number maintained in two places
2 — Growing	The first point-to-point scripts and the first iPaaS; an agency builds most flows; stock and orders move by scheduled sync every 15–60 minutes	Choose the style per flow; a reconciliation job for stock and for orders; a named owner per flow, even when the agency built it	An oversell in the first busy week; a flow only the agency can explain; a sync interval that is "fine" until the first promotion

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
3 — Omnichannel	A dozen or more systems, 20–40 flows in three or four styles, several builders; the joins are the incident source	The integration layer becomes a product with an owner, standards (idempotency, contract versions, monitoring, reconciliation) and a queue or event stream for stock and orders; an OMS as the order hub	Nobody can draw the landscape from memory; the same fact is written by more than one flow; a vendor's API retirement is discovered by a customer
4 — Group	Several brands or countries, each with its own landscape, or a shared platform with a central team	A governance decision: shared hub with service levels, or per-brand autonomy with shared standards; a schema registry for events; integration as a platform team's product	Every brand queues behind one central hub; a brand builds its own connectors because the hub is too slow; hub flows with no owner, The Spaghetti Junction inside one tool

The judgement: the styles do not change with scale; the **discipline around them** does. A Starting-stage merchant and a Group should both choose events for stock and batch for finance. What a Group needs on top is ownership at team level, contracts enforced by tooling, and a hub run as a product, not bought as a box. Kestrel & Pine is at the point where that discipline becomes necessary, which is what W5 is.

In Practice

Dated public evidence. Shopify's webhook documentation (n.d., accessed November 2026) is the clearest public statement of the webhook-plus-reconciliation rule: it does not guarantee ordering within a topic, warns that an app "can miss or mishandle events," and tells apps to "use reconciliation jobs to periodically fetch data from Shopify" with

`updated_at` filters. Stripe's webhook documentation (n.d., accessed November 2026) says the same about ordering and duplicates and asks the receiver to answer with a success code "before any complex logic" and to process events from a queue, because spikes "might overwhelm your endpoint hosts." Microsoft's Business Central connector (October 2026) shows a vendor connector encoding ownership into configuration; Debezium (3.7) shows CDC as a product category.

Kestrel & Pine's order flow, before and after. Eva's order on 13 April 2027 travels like this today. At 21:14:03 the platform creates order #1072455 and, within seconds, sends an "order created" webhook to the iPaaS. The iPaaS flow maps the order to the WMS's format and creates it there by a direct API call; if the WMS does not answer, the flow retries three times at one-minute intervals and then stops *(illustrative).* The PSP sends a "payment succeeded" webhook to the platform. The ERP partner's connector copies the order to the ERP in an hourly batch, which is where the invoice and the books come from. The jacket's stock reduction leaves the WMS fifteen minutes later, reaches the platform within the hour and the marketplace connector within the hour after that. Four styles, five mechanisms, one order, and no row anywhere saying who is called if the 21:14 webhook is the one that is dropped.

Under Waypoint, the same order in 2028 goes: platform webhook to the iPaaS, which publishes an "order placed" event; the OMS consumes it and becomes the order's home; the WMS, the ERP connector and the notification service consume it from the OMS; the WMS publishes a stock event through the iPaaS when the pick is confirmed, and the availability service recomputes what each channel may still promise and publishes it, again through the iPaaS, to the platform and the marketplace connector. The ERP is no longer on the stock path. A nightly sweep compares the platform's orders of the day with the OMS's; a weekly one, daily in November, compares the WMS's stock with the availability service's. The styles are the same four. What changed is that each flow was chosen for its needs and has an owner.

Kestrel & Pine flow (inventory excerpt, illustrative)	**Style and trigger**	**Owner (Feb 2027 / under W5)**	**Reconciliation (Feb 2027 / under W5)**
Platform → iPaaS → WMS: orders	Webhook, then direct call	Ruben / Ruben	None / nightly order count and list

Kestrel & Pine flow (inventory excerpt, illustrative)	Style and trigger	Owner (Feb 2027 / under W5)	Reconciliation (Feb 2027 / under W5)
WMS → ERP → platform → connector: stock	Scheduled pulls, 15 min then hourly	Ruben, the ERP partner, Milan / retired; WMS stock events through the iPaaS to the availability service, which publishes to the platform and the connector	None / weekly per SKU and location, daily in November
PSP → platform: payment status	Webhook	Nobody named / Milan	Monthly payout matching by finance / daily
Returns portal → platform: refunds	Direct API call	Nobody named / Sanne's team	None / daily refund matching against the PSP
Bergzicht → EDI provider → ERP: order, despatch advice and invoice messages (ORDERS, DESADV, INVOIC)	Files in EDIFACT, the UN EDI message standard	The EDI consultant / Lea, with the DESADV sent from the WMS (W6)	Invoice matching by Bergzicht / outbound document log checked weekly
POS → ERP: store stock	Nightly file	The store team / Daniel; an event feed once the POS link exists	None / daily
ERP → data warehouse: orders and invoices	Nightly extract	Hanna / Hanna	Three order counts / one definition (W7)

The Integration Inventory and Integration Debt

No vendor-neutral source defines "integration debt," so this book defines it operationally: **the set of integrations whose owner, contract version, failure alert and reconciliation job are unknown**. Each unowned flow is cheap today and charges interest as the next silent failure, the next vendor retirement, the next person who leaves with the only understanding of a script.

The interest is real, and the vendors document it. Shopify deletes an API-created webhook subscription after eight consecutive failed deliveries, so an unmonitored integration silently stops receiving events. Shopify declared its REST Admin API legacy as of 1 October 2024 and removes deprecated fields on a published schedule; Stripe and Shopify both retire API versions on dates they choose (Promises Between Systems: API Contracts, Webhooks and Versioning). An unmaintained integration breaks on the vendor's schedule, not yours.

The inventory makes the debt a list. One row per flow:

Column	What goes in it	Why it matters
Source → target, and what moves	"WMS → ERP: stock on hand per SKU"	The flow's identity
Style and trigger	Scheduled pull, every 15 min	Decides freshness and failure mode
System of record	The WMS	Which side wins when they disagree
Owner	A named person, not a team or a vendor	Who is called at 20:30 on Black Friday
Contract version	The API version or file format, and the vendor's retirement date	The Pulled Rug has a date; put it in the row
Volume and expected rate	About 950 orders a day; 38 a minute at peak	Sizes the queue and the rate-limit budget
Failure handling	Retries, dead-letter queue, alert	Whether a failure is loud or silent
Reconciliation	What is compared, how often, who gets the exceptions	Whether anyone knows it worked
Last incident	Date and one line	The row's history

Noor's rule, which I would adopt anywhere: **no flow goes live without a row, and no row without an owner**. By April 2027 her ownership map (Who Builds and Who

Owns: Teams, Vendors, Agencies and Governance) had named owners for 8 of the 13; 5 were still unowned. The inventory is reviewed quarterly, and a flow whose owner has left is an incident, not a note. Apps inside the platform are inventoried separately (Apps, Plugins and Functions: Extensibility and Ecosystem Governance); the integration inventory counts flows between two systems, and an app that moves data to another system appears in both lists on purpose.

When It Goes Wrong

The silent stop. A scheduled sync fails and nothing says so. The Price Drift (October 2026) was a price-list sync to the German storefront that stopped; the storefront kept €179 for nine days while the stores and the marketplace charged €199: 410 orders, about €8,200 of margin and a price-parity warning. The fix is not a different style; it is an alert when a scheduled run does not happen, or produces no changes when changes were expected.

The dropped notification. A webhook is retried for hours, then dropped, or the subscription is deleted after repeated failures. The receiver believes nothing happened. Detection is the polling sweep; recovery is the reconciliation job that lists orders present in the source and absent in the target.

The backed-up queue. On Friday 27 November 2026 the WMS connector queue backed up for four hours. No order was lost, but orders reached the warehouse late. A queue's depth and its oldest message's age are health metrics; unwatched, a resilience feature becomes a delay nobody can explain (Black Friday Every Year: Peak Load, Resilience and Recovery).

The retired contract. In February 2027 the marketplace connector retired an API version on a published date; the agency script that read it had not been updated, and stock to bol was stale for two days. A contract-version column with the retirement date puts it on someone's calendar (Promises Between Systems: API Contracts, Webhooks and Versioning).

The second writer and the vendor outage. The App That Wrote Stock (March 2026) adjusted platform inventory while the hourly ERP sync, carrying the WMS's figure, also wrote it: two weeks of phantom reservations on 600 variants. Two flows writing one fact is a failure of the entity-owner table (Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record), visible in the inventory's "system of record" column. And when the PSP paused Kestrel & Pine's account for 40 minutes during the Card-Testing Night, every direct call from the checkout failed; that is what a synchronous dependency means, and the only responses are a short call list, quick timeouts and a decision made in advance about what the checkout does when no answer comes (Taking the Money: Payment Providers, Authorization, Capture and Refunds).

What to Take Away

Established.

- The four integration styles (file transfer, shared database, remote call, messaging) and the criteria for choosing between them have been stable since Hohpe and Woolf named them in 2003; the technology changes, the trade-offs do not.
- Webhooks are a notification, not a guarantee. Shopify and Stripe both state in their current documentation that deliveries can be missed, duplicated and reordered, and both tell integrators to run periodic reconciliation.
- A vendor's connector encodes an ownership model (Business Central's Shopify connector, documented October 2026, has the ERP own inventory and the shop own orders); it fits when that model matches yours.

Depends on context.

- At the Starting and Growing stages a scheduled stock sync every 15–30 minutes is normal and acceptable; it stops being acceptable when the sync interval times the peak order rate exceeds the stock buffer you can afford to oversell. At 38 orders a minute, fifteen minutes is 570 orders.
- An iPaaS pays for itself when the number of flows and builders makes point-to-point unmanageable, usually at the Omnichannel stage; below that, two vendor connectors and a short inventory are cheaper. Compare vendors per flow at your volume, because each meters a different unit.
- Whether the hub is the iPaaS, the OMS or the ERP depends on which system owns the order (The Back Office: ERP Integration and Who Is the Hub argues this out); the style per flow does not change with that choice.

The author's synthesis.

- Choose the style per flow from its needs, never for the company: stock and orders by events with a reconciliation net, products by scheduled publication, finance by daily batch, direct calls only where the next step needs the answer.
- The integration inventory with a named owner per row is worth more than any integration tool, because every incident in this chapter happened in a flow that had no row.

Questions to Ask

1. How many integrations do we have, and can anyone produce the list with an owner on every row?

2. For each flow that moves stock: how old can the copy be, who writes it, and what checks that the two ends agree?
3. Which flows are synchronous calls, and what does the caller do when the other side does not answer within two seconds?
4. For each webhook we receive, what polls the source to catch what it missed; for each scheduled sync, what alerts when it does not run?
5. Which vendor connector are we relying on, and which ownership model does it encode for products, stock and orders?
6. What does our iPaaS meter, and what would our peak-day volume cost under it?
7. Which flows does only the agency, one consultant or one engineer understand, and which API versions retire when?

Choosing the style is half the work. The other half is living with what every asynchronous style introduces: a window in which two systems disagree, messages that arrive twice or out of order, and failures that leave one side updated and the other not. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation shows how to make that window safe, with idempotency, the outbox, retries, compensation and reconciliation as a first-class component, and Kestrel & Pine's first stock reconciliation as the worked example.

Sources and Evidence

Company, product and investor sources

- Webhooks, Shopify developer documentation, n.d., accessed 2026-11 — ordering not guaranteed, deliveries can be missed, reconciliation jobs with `updated_at` filters recommended.
- HTTPS webhook delivery, Shopify, n.d., accessed 2026-11 — eight retries over four hours, subscription deletion after eight consecutive failures, duplicate-detection headers.
- REST Admin API (legacy notice) and API versioning, Shopify, n.d., accessed 2026-11 — REST Admin API legacy as of 1 October 2024; deprecated fields removed on a schedule.
- Receive Stripe events in your webhook endpoint, Stripe, n.d., accessed 2026-11 — retries up to three days, no ordering guarantee, duplicates, answer fast and process from a queue.
- Synchronize items with Shopify (2026-10-16), Synchronize inventory with Shopify (2026-10-14) and Synchronize and fulfill sales orders (2026-10-14), Microsoft Learn — a vendor connector's ownership model, the 15–30 minute inventory interval, notification-triggered order import.
- Debezium features, Debezium project, n.d. (version 3.7), accessed 2026-11 — change data capture as a product category.

- MuleSoft Anypoint pricing, Boomi pricing, Workato pricing, Celigo pricing, Make pricing, n8n pricing, all n.d., accessed 2026-11 — iPaaS metering units and the public price points quoted.

Standards, regulation and official documentation

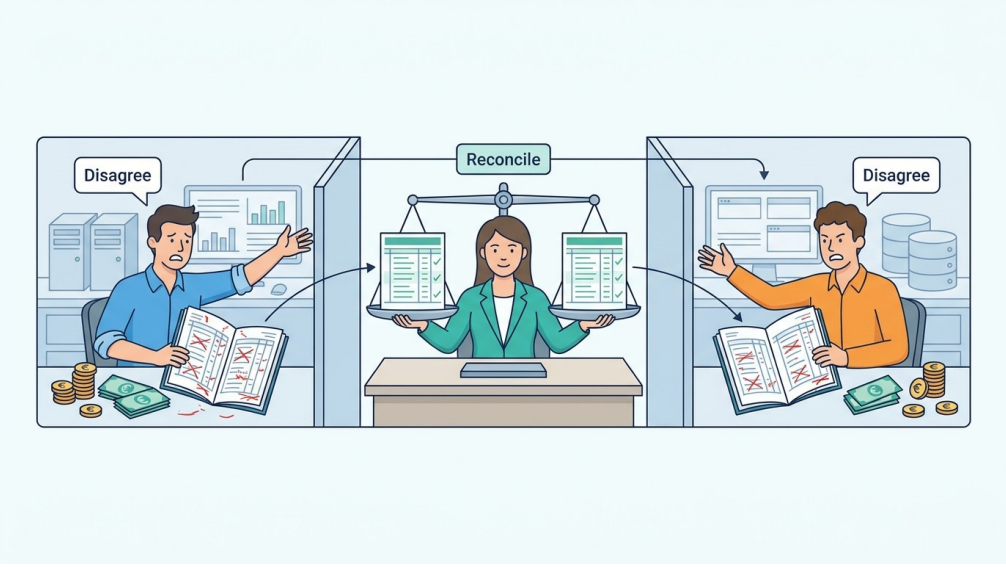
- Cloud-based integration (iPaaS), Wikipedia, relaying Gartner's definition retrieved 2015, accessed 2026-11 — the iPaaS definition; the Gartner page itself was not reachable.

Research and practitioner sources

- Integration Styles, Gregor Hohpe and Bobby Woolf, *Enterprise Integration Patterns*, 2003 (site updated 2023) — the four styles and the criteria for choosing; the source is more than twenty years old and the styles are unchanged.
- What do you mean by "Event-Driven"?, Martin Fowler, 2017 — event notification, event-carried state transfer, event sourcing and CQRS.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

25. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation



IN THIS CHAPTER, YOU WILL: Learn why copies of a fact always disagree for a while, and the five mechanisms that make the disagreement safe: idempotency, the outbox, retries, compensation and reconciliation.

KEY POINTS:

- **Every delivery you will meet is "at least once"; "exactly once" is something the receiver builds.** Webhooks, queues and integration-platform retries all deliver duplicates and reorder messages. An idempotency key, a business identifier such as the order number, makes a repeat harmless, and the receiver must be able to say what its key is.
- **The outbox makes "the order was saved" and "the event was sent" one act; compensation undoes a step that cannot be rolled back.** A cancellation after shipment is not an undo but a return and a refund: a business process with its own owner.

- **Reconciliation is a component, not a cleanup.** A job that compares two systems' records on a schedule, with a key, a tolerance and a named recipient for the exceptions, is the only way to know that the other mechanisms worked. Kestrel & Pine's first one found 2.3% of active variants with the wrong stock figure.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Message in a Bottle** — a message sent once and trusted produces exactly the duplicates and gaps that idempotency and reconciliation exist to catch.
- **The Boomerang** — the Double Refund is what happens when a money action has two writers and no shared key.
- **The Phantom Stock** — a stock copy that drifts from the warehouse's truth, with nothing comparing the two, sells units that are gone.

The previous chapter, APIs, Events, Files and Middleware: How Commerce Systems Talk, ended on a promise: once you choose an asynchronous style, two systems will disagree for a while, and that is the price of not having one system's outage become every system's. This chapter is about paying that price safely.

The first instinct is to make the disagreement go away with one database and one transaction. Inside one system that works. Across a hosted platform (software as a service, SaaS: rented by the month and run by the vendor), a warehouse system from another vendor, an ERP (enterprise resource planning system: the books and the item master) from a third and a payment provider in another country, it cannot. **The systems are built to disagree. The design question is how they recover.**

The mechanisms are few, old and documented by the vendors a merchant already uses. A receiver that can tell a repeat from a new message. A sender that cannot lose an event between saving the fact and publishing it. A retry that does not become a stampede. A way to undo a step that cannot be rolled back. And a job that compares the two ends and tells someone when they differ. This chapter owns four of their names: idempotency, the outbox, compensation and reconciliation.

Where we are in the order's journey. When Eva's jacket (the recurring order; see Meet Kestrel & Pine) is placed at 21:14 on 13 April 2027, the platform tells the warehouse, the ERP and the e-mail service, and each hears it a little later than the others. When her exchange is received on 22 April, the returns portal tells the platform, which tells the PSP (payment service provider: the company that takes the shopper's payment and settles the money) to refund, and the WMS (warehouse management system) puts the size M jacket back on the shelf. Every one of those messages can arrive twice or not at all. This chapter is about what happens then.

Setting the Scene at Kestrel & Pine: The First Reconciliation

Kestrel & Pine, the fictional outdoor brand this book follows, had never compared its warehouse's stock figures with its storefront's. The WMS, which runs receiving, picking and shipping at Duiven, sent stock to the ERP every fifteen minutes, the ERP sent it to the platform hourly, and everyone assumed the numbers matched.

In **May 2027**, as part of Waypoint's integration standards (decision W5: every integration gets an owner, a contract, idempotent handling and a reconciliation job), Ruben Alves ran the first weekly **reconciliation**: a job that reads the WMS's on-hand quantity and the platform's available quantity for every active variant and lists the ones that differ. Of about 17,000 active variants, **2.3%**, about **390**, had a different figure in the two systems *(illustrative)*. The breakdown, over the following week:

Cause of the difference (May 2027, illustrative)	Variants (about)	What it was
Three platform apps that wrote inventory	240	The pre-order app from the App That Wrote Stock incident of March 2026 and two others each adjusted platform stock on their own, on top of the ERP sync
In-flight units	90	Returns the WMS had received and restocked that the hourly chain had not yet carried to the platform when the job ran
Manual edits	40	Corrections typed into the platform's admin by the e-commerce team after customer complaints, never passed back to the WMS
Unexplained	20	No writer found; overwritten from the WMS and watched

Noor's note to Milan Petrov and Daniel Okafor the next morning made three points. The number was not the news; the news was that nobody had known it. Most of the drift

had a writer that should not have been writing, which is a question of ownership (W1: the WMS is the system of record for physical stock), not of integration technology. And the job is a net, not the fix: the availability service that W1 brings in Q4 2027 closes the gap by consuming stock events, and the job then proves it. The job stayed weekly, with a daily run through November, the peak month; the three apps lost their permission to write stock, so that after W1 none of the apps can, and the pre-order app gets a pre-order allocation in the availability service instead. I would have run the job daily all year, 390 wrong variants for a week being 390 chances to repeat Black Friday in miniature; Noor judged the event feed, not the job, to be where the effort belonged.

The second scene is older. In **January 2027**, over three weeks, **212 returns** were refunded twice: once by the returns portal through the platform's API (application programming interface: the operations a system lets other software perform), which instructed the PSP, and once by support agents in the PSP's own console, who refunded by hand when customers whose parcels had reached Duiven asked where their money was. **€19,400** went out twice, most of it recovered. Two writers, no shared key, no job comparing refunds issued with refunds due. The rest of this chapter is the toolkit that stops both scenes from recurring.

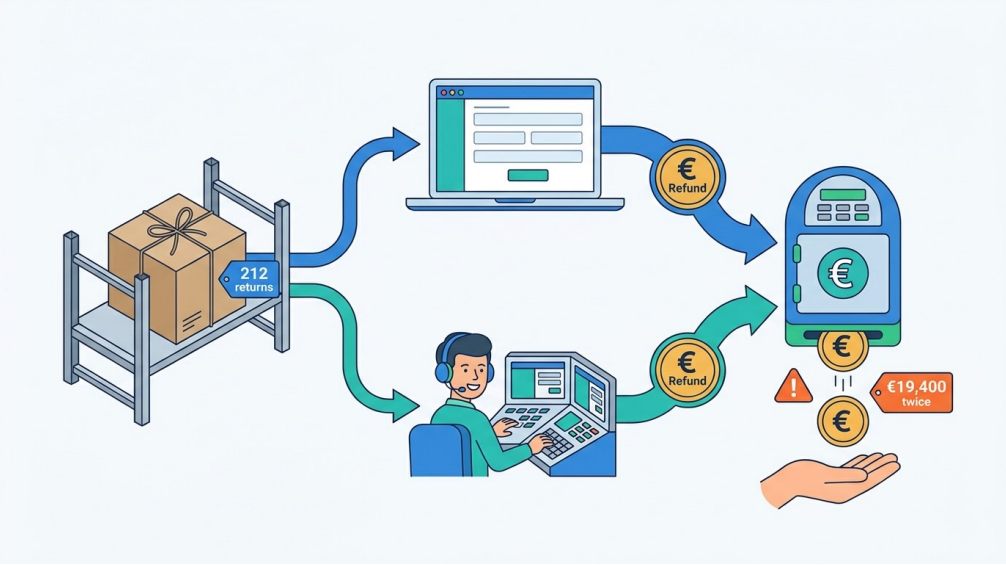


Figure 1: *Two writers and no shared key: 212 returns were refunded twice, once by the returns portal and once by hand in the payment provider's console, and €19,400 went out twice.*

Eventual Consistency in Plain Words

A **copy** of a fact is consistent with its **system of record** (the system whose value counts when they disagree; see Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record) when both show the same value. Across separate systems joined by messages, they cannot always show the same value, because the message takes time. **Eventual consistency** is the promise that is possible: if updates stop, all copies will eventually show the last value. Werner Vogels, then Amazon's CTO, defined it that way in a December 2008 article (old, but the definition has not changed), together with the smaller promises a designer can add on top, such as "a reader always sees its own writes."

Gregor Hohpe's 2004 essay "Starbucks Does Not Use Two-Phase Commit" made the point most practitioners still quote: the cashier takes the order and the money while baristas make drinks asynchronously; drinks finish out of order, so the name on the cup matches drink to customer; and when something goes wrong the shop does not freeze the queue; it writes off the drink, remakes it or refunds the money. Starbucks, he wrote, "is primarily interested in maximizing throughput of orders." So is a commerce platform at 38 orders a minute. The three recoveries he named, **write-off**, **retry and compensating action**, are still the whole menu.

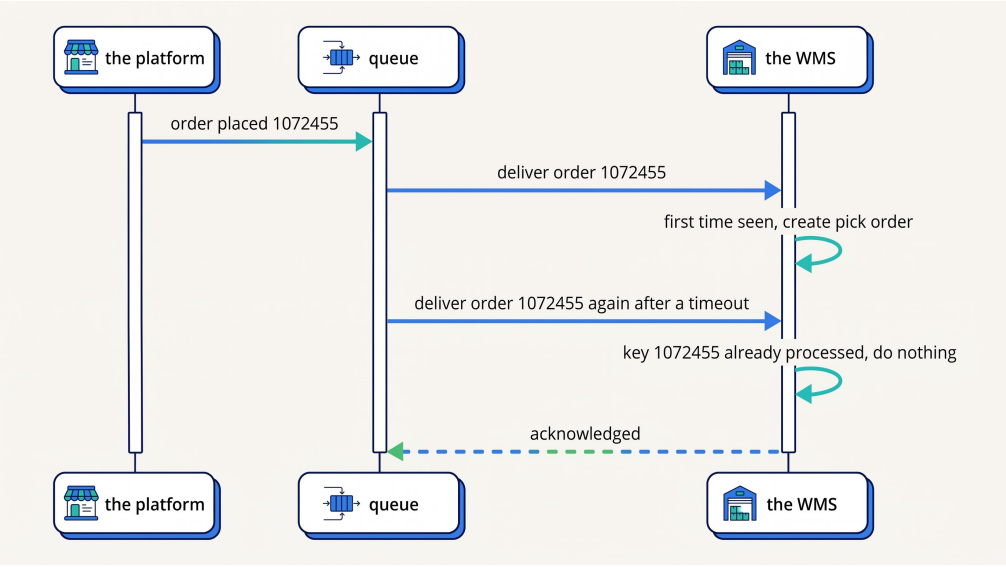


Figure 2: *The same order event delivered twice. The queue is doing its job (it would rather repeat than lose); the WMS is safe only because it remembers the order number it has already processed.*

The window of disagreement is normal; a window that never closes, or that nobody can see, is not.

At Least Once: Why Duplicates Are the Default

Every delivery mechanism a merchant meets prefers repeating a message to losing it. Apache Kafka's design documentation (version 4.4, accessed November 2026) states that Kafka "guarantees at-least-once delivery by default"; exactly-once is available inside Kafka's own processing and with transactions, but destinations outside Kafka "need cooperation from those systems." Shopify's webhook documentation (n.d., accessed November 2026) says it "minimizes duplicate deliveries, but your app might receive the same webhook more than once." Stripe's (n.d., accessed November 2026) says endpoints "might occasionally receive the same event more than once." Mollie, a Dutch PSP, calls a webhook up to ten times over 26 hours until it gets a success response (documentation accessed November 2026).

At-least-once delivery means exactly what it says: the message will arrive one or more times. The alternative, at-most-once, means it may not arrive at all, which for an order or a refund is worse. "Exactly once" is not a property you receive from a queue or a vendor; it is a property the receiver builds by recognizing repeats. The CloudEvents specification (a standard of the Cloud Native Computing Foundation, CNCF, for the envelope around an event; graduated January 2024) builds this in: every event carries a `source` and an `id`, a re-sent duplicate keeps the same `id`, and consumers may treat events with the same `source` and `id` as duplicates.

Ordering is the second thing not to expect. Shopify "doesn't guarantee ordering within a topic," so a product update can arrive before the product's creation; Stripe "doesn't guarantee the delivery of events in the order that they're generated" and warns against using the event's timestamp to decide order, because distinct events can share one. Kafka orders messages only within a partition, one slice of a stream. A design that assumes "shipped" arrives after "allocated" will one day process them the other way round.

Idempotency: Making a Repeat Harmless

An operation is **idempotent** when doing it twice has the same effect as doing it once. Reading a stock level is; "create an order", "refund €199" and "subtract one from stock" are not, unless you make them so. The way to make them so is an **idempotency key**: a unique token for the business action, sent with the request and remembered by the receiver, so that a second request with the same key is recognized and answered without doing the work again.

The PSPs document it best, because money is where duplicates hurt most. Stripe's API (documentation accessed November 2026) accepts an `Idempotency-Key` header on every request that creates or changes something; it "saves the status code and body of the first request made with a given idempotency key" and returns the saved response to retries, errors if the retry's parameters differ from the original's, suggests random unique

identifiers (UUIDs) as keys, and may prune keys after 24 hours. Shopify's GraphQL Admin API (accessed November 2026) takes an `idempotencyKey` on mutations that are not naturally idempotent, so that "repeated requests with the same parameters are executed only once, no matter how many times the request is retried." On the receiving side, Shopify gives each webhook delivery an `X-Shopify-Webhook-Id` and Stripe each event an `id`, and both tell you to store the ids you have processed and skip repeats.

The rule I use for choosing the key is that **it must name the business action, not the attempt**. A timestamp or a random number per attempt defeats the purpose, because the retry gets a new one. The table gives the keys for the actions every merchant performs:

Action	Idempotency key	Why that key
Create the order in the WMS, the OMS (order management system, the order hub) or the ERP	The order number (#1072455)	One order, one record, however many times the event arrives
Refund a return	The return id, or (order, line, quantity, reason)	The same return refunded twice is the Double Refund
Capture a payment	The order number plus "capture"	A capture retried after a timeout must not capture twice
Apply a stock event	(SKU, location, event id); a SKU is one sellable variant, one size in one color	The same movement applied twice doubles the change
Send a shipping confirmation e-mail	(order, shipment id)	The customer should get one
Create a customer	E-mail or external id, with care (One Customer, One Product: Identity and Master Data Across Systems)	Matching is harder than deduplication

The Double Refund fails this test twice over. The returns portal and the support console were two writers with no shared key; the PSP would have declined the second refund had the second request carried the return id the first one used. Waypoint's fix was two rules: **one writer per money action** (refunds go through the returns portal and the platform; support no longer refunds in the PSP console) and the return id as the key on every refund request. A receiver that cannot say what its key is cannot be made safe; that is the first question to ask of any integration that creates, moves or pays.

Out-of-order events need a second defense, because a stale update is not a duplicate. Three responses, in rising cost: make the handler tolerant (a "shipped" event that arrives before "allocated" is stored and re-evaluated when the missing state arrives); carry a version or `updated_at` per entity and drop updates older than what is stored; or route all events for one entity through one consumer, as Walmart's inventory design described on Confluent's blog in May 2020 does, one partition per item and store so that "only one consumer processes it." Stripe's own advice is the simplest of all: when an event arrives early, fetch the current object from the API rather than trusting the payload.

The Outbox: Never Losing the Event Between the Database and the Queue

There is a quiet way to lose an event. A service saves the order to its database, then publishes "order placed" to the queue. If the process dies between the two steps, or the queue is briefly unreachable, the order exists and nobody is told. Doing it the other way round (publish first, then save) risks telling everyone about an order that was never saved. This is the **dual-write problem**, and no amount of retry fixes it, because the sender does not know which half failed.

The **transactional outbox**, as cataloged by Chris Richardson (microservices.io, accessed November 2026), solves it by making the two writes one. The service writes the business change *and* a row in an **outbox table** ("event to send: order placed, #1072455") in the same database transaction, so either both exist or neither. A separate relay reads the outbox and publishes to the queue, either by polling the table or by tailing the database's transaction log (the change data capture of APIs, Events, Files and Middleware: How Commerce Systems Talk). The relay "might publish a message more than once," Richardson notes, which is why "a message consumer must be idempotent": the outbox and idempotency are two halves of one design.

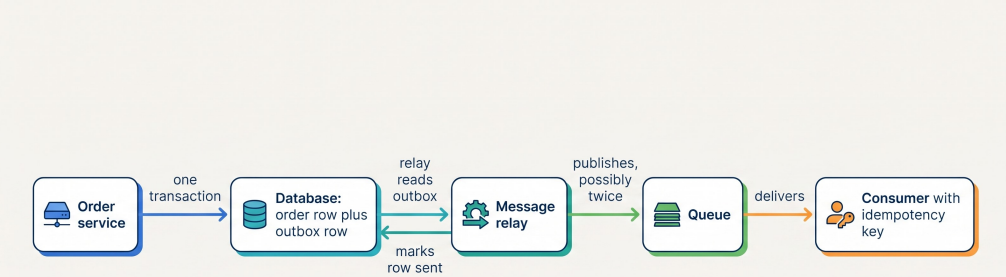


Figure 3: *The outbox: the order and the promise to announce it are saved together, so a crash between saving and publishing loses nothing; the relay may repeat, and the consumer's key makes the repeat harmless.*

A merchant on a SaaS platform cannot build or inspect the platform's outbox. What plays the same role from the outside is the pair the previous chapter described: the platform's webhook plus a polling sweep of everything changed since the last run. The sweep is the merchant's substitute for a relay it cannot inspect.

Retries, Backoff and the Dead-Letter Queue

When a call or a delivery fails, the obvious response is to try again. Done naively, retries make an outage worse: a thousand integrations all retrying at the same second hit the recovering system together. **Exponential backoff** waits longer after each failure (one second, two, four, eight). Marc Brooker of AWS showed by simulation in March 2015 that backoff alone is not enough, because clients still cluster: "The solution isn't to remove backoff. It's to add jitter," meaning randomness in the wait, which in his simulation with 100 contending clients cut total calls by more than half. Stripe's rate-limit guidance (accessed November 2026) says the same for its `429` responses: back off exponentially and "add randomness to the backoff schedule to avoid a thundering herd effect." Shopify's recommended backoff after a throttle is one second.

Retries must end somewhere. A **dead-letter queue** (DLQ) is where a message goes after it has failed more than a set number of times, so that it is neither lost nor retried forever. The documentation of Amazon SQS (Simple Queue Service; accessed November 2026) describes the `maxReceiveCount` after which a message moves, advises setting it "high enough to allow for sufficient retries," and recommends giving the dead-letter queue a longer retention than the source, because a moved message keeps its original timestamp. Messages can be moved back ("redrive") once the cause is fixed.

A dead-letter queue with nobody watching it is a silent order-loss mechanism. The operational rule: **every DLQ has an alarm, an owner and a written replay procedure**, and the number of messages sitting in dead-letter queues is a health metric beside orders per minute (Black Friday Every Year: Peak Load, Resilience and Recovery). On Black Friday 2026 no order was lost, but an order that had exhausted the integration platform's (the iPaaS's) three retries would have gone nowhere a person would look: the inventory of APIs, Events, Files and Middleware: How Commerce Systems Talk found that no flow had a DLQ at all.

Sagas and Compensation: Undoing What Cannot Be Rolled Back

Placing an order touches several systems: reserve the stock, authorize the payment, create the order, confirm to the shopper. Inside one database you would wrap those in a **transaction**, a unit of work that either completes entirely or leaves no trace. Across the availability service, the PSP and the OMS there is no such thing. If the payment is declined after the stock was reserved, the reservation must be released by a second, deliberate action.

The name for this is a **saga**. Hector Garcia-Molina and Kenneth Salem introduced it in a 1987 paper (old, but it is still the definition in use): a long-lived transaction written as a sequence of smaller transactions, where the system guarantees "that either all the transactions in a saga are successfully completed or compensating transactions are run to amend a partial execution." A **compensating action** is the business undo: release the reservation, void the authorization, cancel the pick. Richardson's catalog adds the two ways to coordinate one: **choreography**, where each step publishes an event that triggers the next (no central brain, hard to see the whole flow), and **orchestration**, where one component tells each participant what to do and tracks where the saga is.

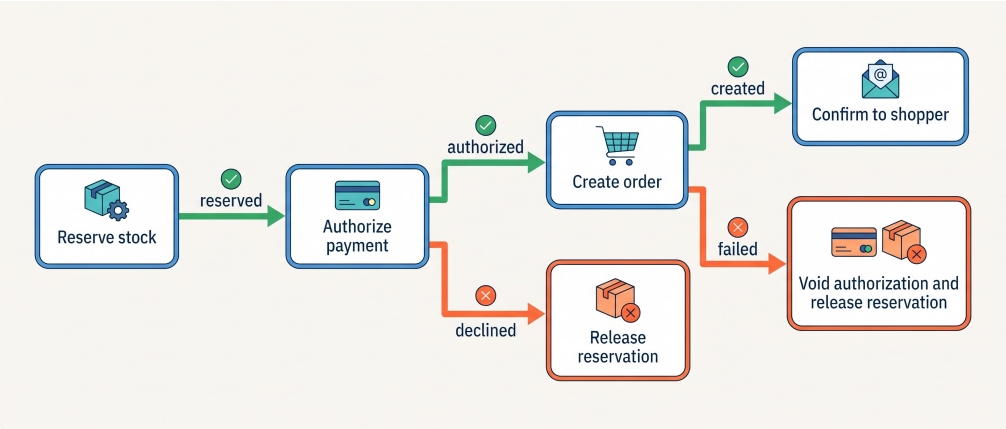


Figure 4: *The place-order saga: each step that fails triggers the compensations for the steps before it; nothing is rolled back, everything is undone on purpose.*

The commerce sagas worth naming are three. **Place order**: reserve, authorize, create, confirm, with the compensations in the figure. **Cancel after shipment**: there is no undo for a parcel already with PostNL; the compensation is a return and a refund on receipt, which is a business process with a parcel, a warehouse grade and a credit note, not a database operation (The Way Back: Returns, Exchanges, Refunds and Reverse Logistics). **Split shipment**: each location's allocation is its own step with its own release. The drawback Richardson names, "lack of isolation," is real: between steps, another order can

see stock that is reserved-but-not-yet-paid, which is exactly why an availability service counts reservations, not just physical units (Is It in Stock? Inventory, Availability and the Promise You Can Keep).

In practice the saga orchestrator for a merchant's order is the **OMS**, the one place that holds every order and steers it through payment, fulfillment, delivery and return (The Order Record: Lifecycle, State and the Order Management System). That is a large part of what Waypoint's decision W2 buys: a component whose job is to know which step of which order has happened, and what to do when the next one fails.

Reconciliation as a First-Class Component

Everything above reduces the chance of disagreement. None of it proves there is none. **Reconciliation** is the job that compares two systems' records on a schedule and reports or repairs the differences. In finance it is centuries old; in commerce integration it is usually an afterthought.

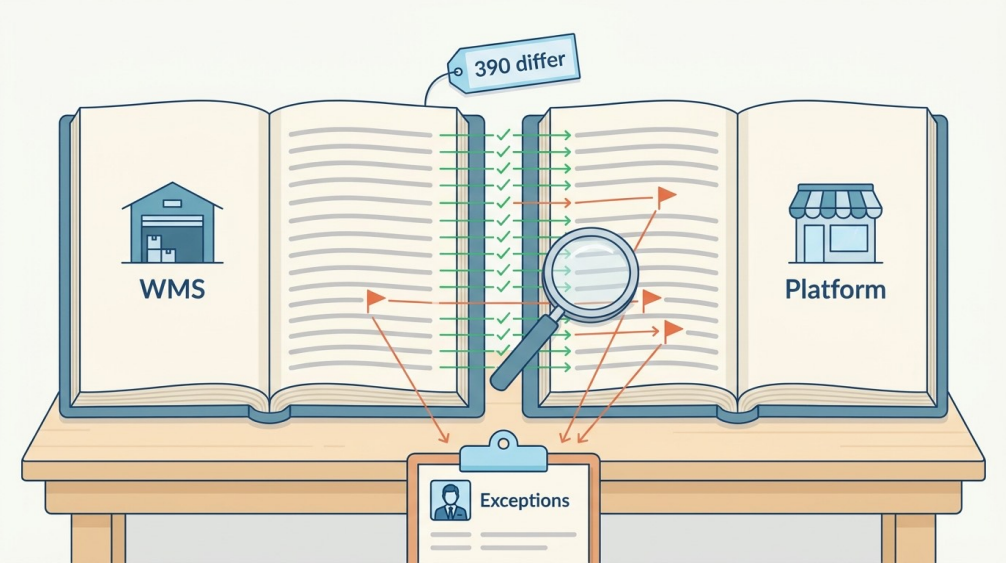


Figure 5: *Reconciliation is two ledgers matched line by line: in the first stock run, about 390 of some 17,000 variants differed between the WMS and the platform, and each difference went on an exception list to a named owner.*

The vendors give it precise shape. Stripe's payout reconciliation report (accessed November 2026) helps "match the payouts you receive in your bank account with the batches of payments and other transactions that they relate to," itemizing "every payment, refund, dispute, fee, and other balance transaction included in the payout" with an id per line and an `order_id` for matching, and it warns that "a payout might

arrive before its reconciliation data is ready." Shopify tells apps to run periodic sweeps with `updated_at` filters so they stay consistent with Shopify's data. Microsoft's Business Central Shopify connector (documentation of October 2026) reconciles stock continuously: it compares "Shopify Stock" with "Last Calculated Stock" per variant and location and sends an update only when they differ, keeping "Last Synced On" and "Last Calculated On" timestamps for troubleshooting. That compare-and-send loop is a reconciliation job built into a product.

Every reconciliation job can be written on one card, and I would require the card before the job goes live:

Card field	Stock (Kestrel & Pine, from May 2027)	Refunds (from February 2027)	Orders (from Q4 2027)	PSP payouts
What two sets are compared	WMS on-hand minus reservations vs platform available, per active variant	Refunds issued by the PSP vs refunds recorded by the platform, per day	Platform orders of the day vs OMS orders of the day	PSP payout lines vs ERP receivables
Matching key	SKU and location	Order number and amount	Order number	Balance transaction id or order id
Tolerance	Zero per variant	Zero	Zero	Zero to the cent
Cadence	Weekly, Sunday 03:00; daily in November; the availability service closes the gap by events in between	Daily	Nightly	Per payout, and monthly close
Who gets the exceptions	Ruben's team and Milan	Sanne's team	Ruben's team	Finance

Card field	Stock (Kestrel & Pine, from May 2027)	Refunds (from February 2027)	Orders (from Q4 2027)	PSP payouts
The fix	Overwrite the platform from the WMS (W1), find and remove the writer	Reverse the duplicate, block the writer	Replay the missing event	Book the difference, open a dispute

Two design points matter. **The system of record decides the direction of the fix.** Because W1 names the WMS for physical stock, the platform's figure is overwritten and the platform-side writer is hunted, never the reverse. And **a zero tolerance is a choice, not a default**: for stock it is zero because one wrong variant at peak is one Phantom Stock incident; for a copy of product descriptions it could be a count threshold. No public source gives a typical disagreement rate between a WMS and a shop; the 2.3% is Kestrel & Pine's illustration and has no benchmark behind it.

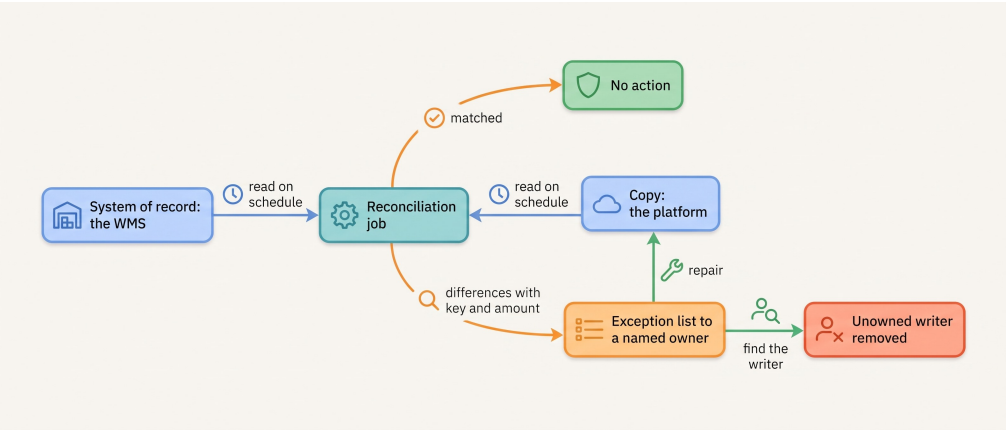


Figure 6: *A reconciliation job is a loop with a recipient: the copy is corrected from the system of record, and the exception list leads to the writer that should not have been writing.*

Reconciled by Design: Traceability and Audit Across Boundaries in the sibling book RECONCILED covers the finance version for ledgers; Taking the Money: Payment Providers, Authorization, Capture and Refunds covers the PSP's reports.

What Each Flow Needs

The mechanisms are chosen per flow, like the styles. The table states, for the flows this chapter touches, how long disagreement is tolerable and which mechanism closes the gap.

Flow	Tolerable window of disagreement	Freshness needed	Consistency on arrival	Availability	Mechanisms that earn their place
Stock events (WMS to availability, availability to channels)	Seconds; minutes at most	Seconds	Per SKU and location, latest version wins	Queue; channels keep selling from the last value	Idempotency on (SKU, location, event id); version per entity; reconciliation with zero tolerance, daily in peak months
Order placed (platform to OMS, WMS, ERP)	Minutes for the warehouse; an hour for finance	Seconds	Never lost, never doubled	Queue; the warehouse works from what it has	Order number as key; outbox or webhook plus sweep; nightly order reconciliation; DLQ with owner

Flow	Tolerable window of disagreement	Freshness needed	Consistency on arrival	Availability	Mechanisms that earn their place
Refunds (returns portal, platform, PSP)	Hours	Minutes	Exactly once, to the cent	Can queue; a late refund is a complaint, a double one is a loss	One writer; return id as key; daily refund reconciliation
Shipment confirmed (WMS to platform, OMS, notifications)	Minutes	Minutes	One confirmation per shipment	Can queue	(order, shipment) as key; tolerant handler for early arrival
Prices and promotions	Minutes; a promotion start to the minute	Minutes	Latest version wins	Fail loudly	Version per price list; alert on a missed run (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax)

Flow	Tolerable window of disagreement	Freshness needed	Consistency on arrival	Availability	Mechanisms that earn their place
Payment status (PSP to platform)	Minutes; method expiry windows for the long tail	Seconds to minutes	Terminal state must be reached	Webhook retried by the PSP for a day or three	Event id as key; fetch the status from the PSP when it matters; daily sweep of orders not in a terminal state

(The author's recommendation; the Kestrel & Pine cadences are illustrative.)

The Choices and Their Trade-Offs

There are four ways to keep two systems in agreement, and a merchant will use more than one.

	Distributed transaction (two-phase commit)	Events with idempotent consumers and reconciliation	Scheduled compare-and-send	Manual correction
How it works	Both systems commit together or not at all	Each change is published; receivers deduplicate; a job checks the ends	On a timer, read both sides, send what differs	A person fixes what a customer reported

	Distributed transaction (two-phase commit)	Events with idempotent consumers and reconciliation	Scheduled compare-and-send	Manual correction
Window of disagreement	None	Seconds, closed by the job	The timer interval	Days
Coupling and availability	Both must be up; one locks the other	Each works alone; the queue holds the rest	Each works alone	None
Cost to build	High; needs both vendors' cooperation, which SaaS vendors do not give	Medium: keys, outbox or sweep, DLQ, card per job	Low	None, then everything
Fits	One database, one vendor	Stock, orders, refunds, shipments across vendors	Products, prices, finance, and as the safety net under events	Never as the design; always as the last resort
Limits	Hohpe's warning: it "can also hurt the free flow of messages"; unavailable across SaaS	Needs the discipline of this chapter	Stale by design; the Price Drift was a compare-and-send that stopped	The Double Refund was manual correction without a key

And between the two ways to run a saga:

	Choreography	Orchestration
Who decides the next step	Each participant, by reacting to events	One orchestrator, usually the OMS
Visibility	The whole flow is in nobody's code	One place shows where every order is

	Choreography	Orchestration
Coupling	Lowest	The orchestrator knows every participant
Fits	Two or three steps, few participants	The order lifecycle at Omnichannel scale, where support must answer "where is my order"

My judgement: for a merchant, orchestration by the OMS wins once there are more than three systems in the order's path, because the question support asks ("where is this order and what happens next?") has to be answerable from one screen. Choreography is right for the edges: notifications, analytics, anything that listens and never has to be undone.

Across Scale Stages: When It Fits and When It Limits

The four stages are the book's (see What Changes at Each Scale: From First Store to Group).

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting	The platform and its apps handle retries and duplicates; the merchant never sees a queue	Check the apps' failure logs weekly; reconcile PSP payouts to the books monthly	A refund issued twice "by accident"; an order the 3PL (the third-party warehouse) never received, found by the customer

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
2 — Growing	The first custom integration (ERP connector, returns portal, marketplace sync) built by an agency or a partner, usually without keys or a DLQ	Idempotent writes on every create, refund and stock change; retry with backoff; a daily reconciliation for orders and for stock	An oversell in the first busy week; a connector that "sometimes" doubles orders; nobody can say what a flow's key is
3 — Omnichannel	A dozen systems, 20–40 flows; a double-refund or a stock-drift incident has already happened	Written standards: a key per entity, an event envelope with id, source and type, a DLQ with an owner per flow, a reconciliation card per flow, four signals per flow; an OMS orchestrating the order saga	Reconciliation findings that nobody owns; a DLQ nobody has opened; sagas that live in three people's heads
4 — Group	A shared event bus and schema registry across brands; a platform team owns the standards	Reconciliation as a data-platform job across brands (Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics); replay tooling; service levels for the bus	One brand's bad event schema stalling another's consumers; exceptions routed to a central team that does not know the business

The judgement: idempotency and reconciliation do not change with scale; a Starting-stage merchant needs them from its PSP and its 3PL, a Group needs them from its own bus. What changes is who builds them. Below the Omnichannel stage they are bought

inside products (the PSP's idempotency keys, the connector's compare-and-send); from the Omnichannel stage the merchant has enough flows of its own that the discipline must be written down and owned, which is what W5 is.

In Practice

Dated public evidence. Stripe's idempotent-requests documentation (accessed November 2026) is the clearest public model of a key-based design: one key per request, the first response saved and replayed, a mismatch in parameters rejected. Mollie's webhook design (accessed November 2026) shows a different defense against lies and duplicates: the webhook carries only an `id`, never the status, so "fake calls to your webhook will never result in orders being processed without being actually paid," and the receiver must fetch the current status. Microsoft's Business Central connector (October 2026) ships a compare-and-send reconciliation for stock. Stripe's payout reconciliation report (accessed November 2026) is the finance-side template for the reconciliation card. Walmart's 2020 inventory design, described on Confluent's blog (six years old; it may describe a replaced system), shows one consumer per item and location at a scale no merchant in this book needs; the idea is the same at any size. Google's Site Reliability Engineering book (2017) names the four golden signals, latency, traffic, errors and saturation, that the monitoring section below applies to flows.

Kestrel & Pine's refund path after the fix. From February 2027 a refund for Eva's exchange, had she chosen a refund instead of size L, would travel like this. The returns portal receives the jacket's inspection result on 22 April and calls the platform's refund API with the return id as the idempotency key *(illustrative design)*; the platform asks the PSP to reverse €199 by iDEAL (the Dutch bank-transfer payment method), with the same key; the PSP's "refund succeeded" webhook, carrying the event id, is stored and skipped if it arrives twice; the ERP books a credit note from the nightly batch. Support agents can see the refund in the platform and can no longer issue one in the PSP console. Every morning a job lists the PSP's refunds of the previous day and the platform's, keyed on order number and amount, and sends any unmatched line to Sanne de Wit's team. The 212 duplicates of January would have appeared on the first morning.

Monitoring the flow. The SRE book's four signals become, per integration: end-to-end lag (order placed to order in the WMS), events per minute against the expected rate, failed deliveries and DLQ depth, and rate-limit headroom. The reconciliation job's variance is the black-box check, "what's broken"; the connector's own log is the white-box check, "why." The alert most teams forget is traffic at zero while the shop is busy: a deleted webhook subscription produces no errors at all.

When It Goes Wrong

Duplicates. The Double Refund: two writers, no key. Detection is the daily refund reconciliation; recovery is a reversal and a block on the second writer. The same shape appears as a double pick when a WMS keys on its own sequence number instead of the order number.

Gaps. A webhook retried for hours and then dropped; a subscription deleted after repeated failures; an event stuck in a DLQ nobody opens. Detection is the polling sweep and the DLQ alarm; recovery is replay. Stripe allows manual resend of events for 15 days from its dashboard, 30 by its command-line tool (documentation accessed November 2026); replay is safe only because of idempotency.

Stale copies. The 390 variants of May 2027, and the Price Drift of October 2026. Detection is the reconciliation job and the alert on a missed run; recovery is overwriting from the system of record and finding the writer. A stale copy with no job is found by a customer.

Retry storms. A recovering ERP hit by every connector at once. Backoff with jitter on every caller, and a circuit breaker (stop calling after repeated failures, try again later) on the ones that matter at peak (Black Friday Every Year: Peak Load, Resilience and Recovery).

Partial failures. The order exists in the WMS and not in the ERP, or the reservation exists and the payment was declined. The saga's compensations handle the second; the nightly order reconciliation finds the first. A partial failure nobody designed for becomes the Three-Status Order (The Order Record: Lifecycle, State and the Order Management System).

Vendor outages. When the PSP paused Kestrel & Pine's account for 40 minutes in August 2026, webhooks stopped too. Webhooks resume with retries, but only within the vendor's window (26 hours at Mollie, three days at Stripe); a daily sweep of orders whose payment has not reached a terminal state, querying the PSP directly, is the net under the net.

What to Take Away

Established.

- Delivery across systems is at least once and unordered: Kafka's design documentation, Shopify's and Stripe's webhook documentation and Mollie's retry schedule all say so in their own words. Exactly once is built by the receiver.
- An idempotency key must identify the business action; Stripe's and Shopify's API documentation describe the mechanism, and CloudEvents standardizes `source` plus `id` as the duplicate key for events.

- The transactional outbox (Richardson) and sagas with compensating transactions (Garcia-Molina and Salem, 1987) are the standard answers to losing events and to undoing partial work; backoff with jitter (Brooker, 2015) is the standard retry.

Depends on context.

- Below the Omnichannel stage the merchant buys these mechanisms inside products (the PSP's keys, a connector's compare-and-send) and needs only to check payouts monthly and failure logs weekly; from the Omnichannel stage, with its own flows, it must write the standards down and own them.
- Orchestration by an OMS is right when more than three systems sit in the order's path and support must answer "where is it" from one screen; choreography is right for listeners that never have to be undone.
- Reconciliation cadence follows the cost of a wrong copy: daily for refunds, nightly for orders, per payout for money, weekly for content; for stock, weekly is tolerable only while an event feed closes the gap in between, and daily in the peak months.

The author's synthesis.

- Treat the reconciliation card as part of the integration, not as its audit: no flow that creates, moves or pays goes live without one, because the other mechanisms reduce disagreement and only reconciliation proves it.
- Most drift has a writer that should not have been writing; when a reconciliation finds differences, the first question is "who else writes this?", not "which queue dropped it."

Questions to Ask

1. For each integration that creates, refunds, captures or adjusts stock: what is its idempotency key, and where are processed keys stored?
2. What happens when the same order event arrives twice, and when "shipped" arrives before "allocated"?
3. Which flows have a dead-letter queue, who is alerted when it is not empty, and how is a message replayed?
4. For each money action: how many systems can perform it, and is it one?
5. Which reconciliation jobs run, on what key, with what tolerance, and who receives the exceptions?
6. When a reconciliation finds a difference, which side is corrected, and is that written down as the system of record?
7. If the platform's webhooks stopped for a day, what would notice?

8. Which component orchestrates the order saga, and what compensations exist for a declined payment after a reservation and for a cancellation after shipment?

The mechanisms in this chapter protect a flow against the messages it receives. They do not protect it against the other side changing the shape of those messages: a field renamed, an API version retired, a webhook format revised on a date the vendor chose. Promises Between Systems: API Contracts, Webhooks and Versioning is about those promises between systems, with Kestrel & Pine's Deprecated Endpoint as the worked example.

Sources and Evidence

Company, product and investor sources

- Idempotent requests, Stripe API reference, n.d., accessed 2026-11 — key-based idempotency: saved first response, parameter comparison, 24-hour pruning.
- Receive Stripe events in your webhook endpoint, Stripe, n.d., accessed 2026-11 — retries up to three days, no ordering guarantee, duplicates, event ids, manual resend windows.
- Rate limits, Stripe, n.d., accessed 2026-11 — exponential backoff with randomness on `429`.
- Payout reconciliation report, Stripe, n.d., accessed 2026-11 — the finance-side reconciliation model: itemized payout lines, matching ids, timing caveat.
- Idempotent requests, Shopify developer documentation, n.d., accessed 2026-11 — `idempotencyKey` on GraphQL Admin API mutations.
- Webhooks and HTTPS webhook delivery, Shopify, n.d., accessed 2026-11 — ordering not guaranteed, duplicates possible, `X-Shopify-Webhook-Id`, reconciliation jobs; Rate limits — one-second backoff.
- Webhooks, Mollie, n.d., accessed 2026-11 — ten retries over 26 hours; the id-only payload that makes forged calls harmless.
- Synchronize inventory with Shopify, Microsoft Learn, 2026-10-14 — compare-and-send stock reconciliation with "Last Synced On" and "Last Calculated On".
- Walmart's Real-Time Inventory System using Kafka, Suman Pattnaik, Confluent blog, 2020-05-04 — partition per item-store pair so one consumer processes each; the author's views, not Walmart's endorsement; six years old.

Standards, regulation and official documentation

- CloudEvents specification, CNCF, version 1.0, accessed 2026-11, and the CNCF project page — `source` plus `id` as the duplicate key; graduated 2024-01-25.

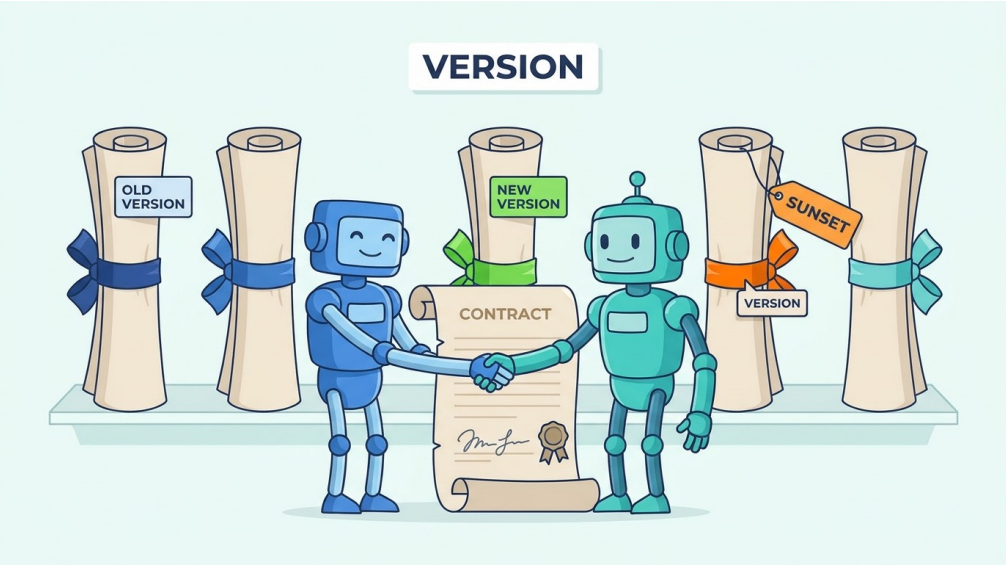
- Kafka design, Apache Software Foundation, Kafka 4.4 documentation, accessed 2026-11 — at-least-once by default; exactly-once within Kafka; ordering per partition.
- Using dead-letter queues in Amazon SQS, AWS, n.d., accessed 2026-11 — `maxReceiveCount`, retention advice, redrive.

Research and practitioner sources

- Eventually Consistent, Werner Vogels, 2008 — the definition of eventual consistency and its variants; eighteen years old, definitions unchanged.
- Starbucks Does Not Use Two-Phase Commit, Gregor Hohpe, 2004 — asynchronous processing, correlation, write-off, retry and compensation; twenty-two years old, still the clearest analogy.
- Transactional outbox and Saga, Chris Richardson, microservices.io, n.d., accessed 2026-11 — the outbox and relay; choreography vs orchestration; compensating transactions; lack of isolation.
- Sagas, Hector Garcia-Molina and Kenneth Salem, Proc. ACM SIGMOD 1987 — the original definition; thirty-nine years old.
- Exponential Backoff And Jitter, Marc Brooker, AWS Architecture Blog, 2015 — the simulation showing jitter halves contending calls.
- Monitoring Distributed Systems, Google, *Site Reliability Engineering*, 2017 — the four golden signals; symptoms vs causes; black-box vs white-box.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

26. Promises Between Systems: API Contracts, Webhooks and Versioning



IN THIS CHAPTER, YOU WILL: Learn what an API contract promises, how dated versions and deprecation dates let a system change without breaking its consumers, and how to receive webhooks safely.

KEY POINTS:

- **A contract is a written promise with a lifetime.** It says what a system accepts and returns, how fast and how often you may call it, and until when. Shopify's quarterly dated versions and Stripe's dated versions are two public models; both make the consumer responsible for pinning a version and scheduling the upgrade.
- **Change by adding; retire by date, with consent and monitoring.** Google's and Zalando's published rules agree that adding optional fields is safe and removing or renaming anything is not. A deprecation is a date pair, announced and removed, plus a check of who still calls the

old version; a "deprecated" label without a date is a warning, not a deprecation.
- **A webhook is a signed hint, not a ledger.** Shopify and Stripe both document that deliveries can be missed, duplicated and reordered. The receiver verifies the signature, answers fast, deduplicates on the delivery id and runs a periodic sweep for what it missed.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Pulled Rug** — a vendor's API version or a partner's document format changes on a date nobody at the merchant had written down.
- **The Message in a Bottle** — a webhook is received once and trusted, so a missed, duplicated or reordered delivery becomes a missing or doubled order.
- **The Lynchpin** — the only person who knew which version a script called, and where the vendor's notices went, was one developer at the agency.

A commerce platform keeps its promises through other systems. The storefront promises Eva Janssen a jacket at €199 delivered tomorrow; the warehouse, the carrier, the payment provider and the marketplace connector have to make it true, by talking to each other through **APIs**, the documented ways one program asks another to do or tell it something. **Every one of those conversations rests on a second promise: that the API will still accept tomorrow what it accepted today.** That promise is the contract.

The problem is easy to state and hard to live with. The providing system has to change: new features, a security fix, a new generation of its API. The consuming systems were written against the old shape and keep sending it, because nobody rewrites a working integration for fun. Somewhere between those two facts an order goes missing or a stock level stops updating. **The architecture question is how a provider changes without breaking its consumers, and how a consumer survives a provider that changes anyway.**

For Kestrel & Pine, the fictional outdoor brand this book follows, almost all of its 34 integrations consume contracts a vendor owns: the platform, the marketplace connector, the PSP, the carriers, the returns portal. The merchant cannot stop a vendor changing; it can only know when, and be ready. A few contracts it provides itself, to the agency, to the returns portal and, by EDI, to Bergzicht Outdoor, and there it is the one who must not pull the rug.

Where we are in the order's journey. This chapter sits beside every step of Eva's jacket rather than on one of them. The order reached the warehouse through a webhook from the platform; the exchange came back through the returns portal's call to the platform's API; the stock that let bol sell the jacket arrived through the marketplace connector's API. Each arrow is a contract with a version and an expiry date. APIs, Events, Files and

Middleware: How Commerce Systems Talk explained how systems talk and Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation how to survive duplicates and gaps; this chapter is about the promise itself.

Setting the Scene: The Deprecated Endpoint

On Wednesday 10 February 2027, Milan Petrov, Kestrel & Pine's Ecommerce and Marketplaces Manager, got an escalation from bol's seller support: a cluster of orders for Ridgewalker jackets and Trailhead fleeces, all canceled by Kestrel & Pine for lack of stock, all placed in the previous two days. Ruben Alves, the Lead Integration Engineer, found the cause in an afternoon. The hourly job that pushed stock levels from the platform to the marketplace connector for the bol listings had stopped updating anything on Monday morning. It was an agency script from 2021, when bol was the first marketplace and the connector did not yet have the native stock feature that Amazon.de and Zalando used later, and it called the connector's API with the 2021 version string written into its code. The connector had retired that version on Monday 8 February, a date published in its developer changelog the previous November and e-mailed to the technical contact on file, a developer at the agency who had left in 2025. Since Monday every call had returned an error; the script wrote it to a log file and ran again an hour later. For two days bol showed Monday morning's stock. The dates and the script's history are local to this chapter and fictional.

Nothing had failed. The connector had done what its documentation said; the script had done what it was written to do; the notice had been sent. **A contract had reached the end of its published life, and nobody on the consuming side had written the date down.** When Noor Visser put the incident on the list behind Waypoint, it became the argument for the "contract" column of decision W5: every integration gets a named owner and a contract, and, after this chapter, a pinned version and a sunset date.

What a Contract Promises

An **API contract** is the written description of what a system promises to accept and return: the operations you may call, the fields each one takes and gives back, the errors it can answer with, the limits on how often you may call it, and the period for which all of that stays true. In plain words, it is what lets two teams, or a merchant and a vendor, build against each other without reading each other's code. It has four parts, and each breaks in its own way.

Part of the contract	What it covers	How it breaks	Example in this chapter
Shape	The operations, fields, types and required values	A field is removed, renamed or made required	A connector drops a field the script sends
Meaning	What a value means and what the call does	The same field starts meaning something else	"available" changes from physical stock to available-to-promise
Timing	Rate limits, timeouts, retries, delivery guarantees	A limit is lowered; a retry window changes; ordering was never promised	A stock sync is throttled at peak because another app used the budget
Lifetime	Version, support window, deprecation and sunset dates	The version is retired on a date the consumer did not track	The Deprecated Endpoint

Shape is the part people write down; the other three are where incidents come from. A contract that documents only the fields is a lease that names the apartment but not the rent or the end date.

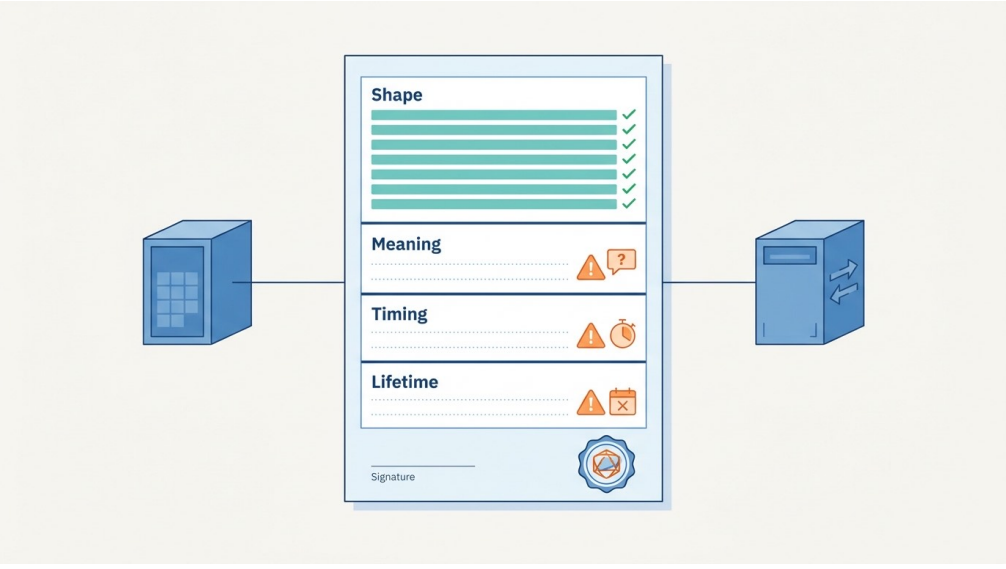


Figure 1: *A contract has four parts: shape is the part people write down, while meaning, timing and lifetime are where incidents come from.*

Contracts are written in machine-readable formats so that tools can check them: **OpenAPI** for HTTP APIs (version 3.2.1 as of September 2026), **AsyncAPI** for message-driven APIs where one system publishes events and others subscribe (version 3.0.0, December 2023), **JSON Schema** (2020-12) for the shape of a single document, and **CloudEvents**, a vendor-neutral envelope that gives every event an id, a source, a type and a time so that a consumer can deduplicate and route it without reading its body. A merchant writes none of these from scratch; it reads its vendors' documents and, when it becomes a provider, publishes its own. The reason to write the contract down is not ceremony: **a written contract can be diffed**, and a diff between two versions is the only reliable way to know whether your integration is affected.

A merchant is on both sides of the table, and the obligations differ. As a **consumer**, the merchant cannot prevent change; its obligations are to know which version each integration uses, when that version expires, and to test against the next version before the old one disappears. As a **provider**, the merchant owes its consumers what it wants from its vendors: a published document, additive change, and a dated, monitored deprecation. At Omnichannel scale the provider role grows: once Kestrel & Pine's OMS (order management system, the single record of every order, bought under Waypoint decision W2) exposes order status by API, the agency, the returns portal and later a shopper-facing agent all consume a contract that Kestrel & Pine owns.

Requirements differ by flow

Not every contract needs the same guarantees. The figures below are illustrative.

Flow	Direction	How fresh	How consis-tent	How fast	What the contract must say
Order placed, platform to WMS	Webhook, then queue	Seconds to minutes	Exactly one order in the WMS per order number	2xx (an HTTP success code) within the sender's timeout	Signature, delivery id, retry schedule, payload version

Flow	Direction	How fresh	How consistent	How fast	What the contract must say
Stock level, platform to marketplace connector	API call, hourly today; events under W1	Minutes	Last write wins per SKU and location	Must finish within the rate budget	Version, sunset date, batch size, cost per call
Availability check at checkout	Synchronous API	Live	Read of the system of record	Under a few hundred milliseconds *(assumption)*	Timeout, degraded answer, version
Order status, OMS to agency and portal	API provided by the merchant	Minutes	Read from the order hub	Under a second *(assumption)*	Published document, additive change, deprecation policy

Two rows carry the chapter. The order webhook needs the receiving discipline of the webhook section; the stock push needs the version discipline of the next two sections. The Deprecated Endpoint was the second row with no contract card.

Changing Without Breaking

The cheapest way to version an API is not to. If every change a provider makes is one a consumer written against the old shape can survive, there is nothing to retire and nothing to pull. Two published rule sets define "survivable", and they agree.

Google's API Improvement Proposal 180 on backwards compatibility (created July 2019, updated October 2025) states that existing client code "must not be broken" by a minor or patch release. It allows adding fields, enumeration values and methods, provided new fields are optional and default to the old behavior, and classes as breaking: removing or renaming anything, changing a field's type "even to a wire-compatible type", changing default values or visible behavior.

Zalando's RESTful API Guidelines, published on GitHub and owned by its API Guild, with changelog entries to 16 March 2026, make the same rules mandatory: rule #106, **MUST** not break backward compatibility ("Change APIs, but keep all consumers running"); #107, **SHOULD** prefer compatible extensions; #108, **MUST** prepare clients to accept compatible extensions; #113, **SHOULD** avoid versioning. Rule #108 is the consumer's half of the bargain: a **tolerant reader** is a client that ignores fields it does not know, so the provider can add fields without asking anyone. A client that rejects a payload because an unexpected field appeared has made itself impossible to extend.

The combination is the workhorse of API change in commerce: **the provider only adds**; **the consumer ignores what it does not understand**. Most quarterly change in a platform's API is of this kind, and a tolerant consumer never notices it.

Additive change has limits. You cannot add your way out of a field whose meaning was wrong or a whole generation of the API built on a different model; Shopify's REST Admin API, "a legacy API as of October 1, 2024", with all new public apps on GraphQL since 1 April 2025, is change of that size. For it, you need versions.

Versioning: Dated, Semantic or in the Media Type

Versioning is naming generations of a contract so that a consumer can say which one it relies on. Three strategies are in common use.

Dated versions. The version is a date, the consumer states it on every call, and the provider supports each date for a published window. Shopify's documentation (undated, accessed November 2026) is the clearest public example: Shopify "releases a new API version every three months at 5pm UTC on the first day of the quarter", named by date (the latest stable version at writing is `2026-10`, accessible until 16 October 2027); "Each stable version is supported for a minimum of 12 months", with at least nine months of overlap between consecutive versions; the version is written in the request URL. Deprecations are announced in the developer changelog with migration guidance, and "Deprecated fields or types are removed in a subsequent release".

Stripe uses dates with a different support promise. Its 2017 engineering post by Brandur Leach (nine years old; the mechanism is still the documented one) explained that each account is pinned to the version current at its first request, that a caller can override it per request with the `Stripe-Version` header, and that breaking changes are implemented as "version change modules" applied in reverse until the caller's version is reached; Stripe had then "maintained compatibility with every version of our API since the company's inception in 2011". Since October 2024 Stripe releases monthly versions with no breaking changes and two named major releases a year; the current version at writing is `2026-10-28.endive`.

Semantic versions. A number in three parts, `major.minor.patch`, where only a major change may break consumers: the convention for software libraries, which Stripe

applies to the software development kits it ships with each major release.

Versioning in the media type. The version travels in the `Accept` header, as part of the content type the client asks for. Zalando's guidelines require it (#114) and forbid versions in the URL (#115), because a resource's address should not change when its representation does.

Strategy	Where the version lives	Who moves	Public example	Fits when
Additive only, no versions	Nowhere; one contract that only grows	Nobody	Zalando's preferred path (#113)	Changes are small and the provider controls meaning
Dated versions, forced upgrade	URL; supported for a window, then removed	Every consumer, at least yearly	Shopify, quarterly, 12-month minimum	Many consumers; a provider that must keep its surface small
Dated versions, consumers may stay	Account default or header; old versions kept alive by transformation code	Only consumers who want new features	Stripe, every version since 2011	Payments, where silent change is unacceptable
Media type versioning	`Accept` header	Consumers, per representation	Zalando (#114)	Internal and partner APIs where URL stability matters
Semantic versions	Package or URL prefix such as `v2`	Consumers at each major	SDKs; many internal APIs	Libraries; coarse-grained APIs with few consumers

Two details of Shopify's design decide what an expired version does to you. First, the **fall-forward rule**: "If your app targets an inaccessible version, Shopify falls forward

and responds using the oldest accessible stable version", and a response header says which one answered. The call does not fail; it is answered by a newer contract that may have removed or changed what you asked for. That is kinder than an error and more dangerous, because it is silent. Second, **webhook payloads follow the version of the subscription** that created them, so a subscription made in 2025 keeps sending 2025-shaped payloads while the app's other calls have moved on. The marketplace connector in the Deprecated Endpoint answered with an error, the loud failure; a fall-forward would have kept the script running against a changed contract, the quiet version of the same incident.

My reading is that Shopify and Stripe offer the same contract with a different default: Shopify makes consumers move every year; Stripe lets them stay but ships breaking changes only behind opt-in. For a merchant, both mean two obligations: **pin the version explicitly** (in the URL for Shopify; in the header and on each webhook endpoint for Stripe) and **put the upgrade on a calendar**. A merchant that pins nothing is on the vendor's default, which changes when the vendor says so.

Deprecation: A Date Pair, Monitoring and Consent

Deprecation is the announced, dated withdrawal of part of a contract; the **sunset** is the date it stops working. The two dates are the whole mechanism, and the web has standard headers for them. RFC 9745, a Proposed Standard of the Internet Engineering Task Force (IETF, the body that publishes the web's protocol standards as numbered RFCs) from 2025, defines the `Deprecation` HTTP response header, with a date that may lie in the future or the past; the older `Sunset` header (RFC 8594) says when the resource is expected to stop responding, and its date "MUST NOT be earlier than the one given in the Deprecation header field". A deprecation with no Sunset "doesn't imply the resource will go offline". In plain words: **"deprecated" alone is a warning**; **"deprecated on this date, gone on that date" is a deprecation**.

Zalando's guidelines show what a provider owes beyond the headers: #185, **MUST** obtain approval of clients before API shut down ("all clients have given their consent on a sunset date"); #187, **MUST** reflect deprecation in the specification with the sunset date and migration guidance; #188, **MUST** monitor usage of a deprecated API scheduled for sunset; #189, **SHOULD** add `Deprecation` and `Sunset` headers; and, for the consumer, #191, **MUST** not start using deprecated APIs. The organization behind the rules matters as much as the text: Zalando's September 2015 engineering post (eleven years old; the guild may have changed since) described an API Guild of about twenty people spending a tenth of their week on it, "a sounding board and review resource, not an approval board", with the business reason in one sentence: "We cannot afford to break our APIs".

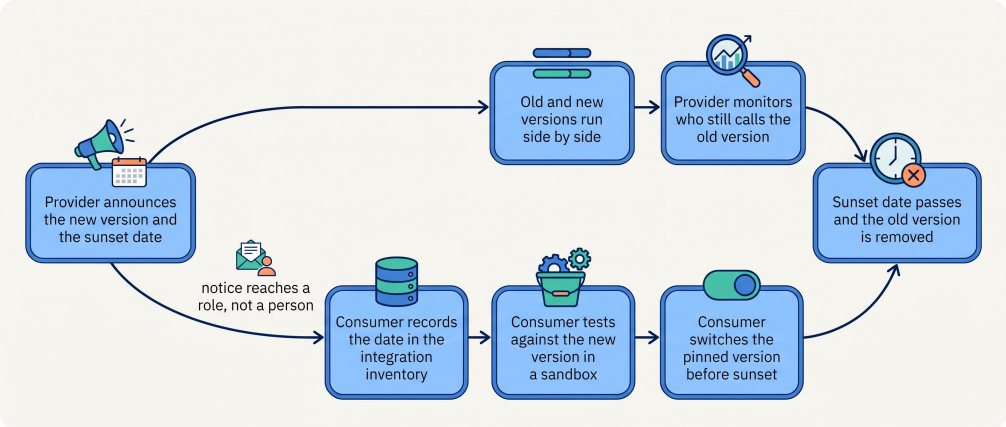


Figure 2: *A deprecation is two timelines that must meet before the sunset date: the provider announces, overlaps, monitors and removes; the consumer records, tests and switches. The Deprecated Endpoint is the bottom line with nobody on it.*

The consumer's half is the half a merchant controls, and it is small. The **integration inventory** that APIs, Events, Files and Middleware: How Commerce Systems Talk introduced (one row per flow: source, target, style, owner, volume, reconciliation) gains two columns: the pinned version and its sunset date. The vendor's changelog is read monthly from a role mailbox, not a person's. Each sunset date becomes a calendar entry three months ahead, assigned to the flow's owner. That is the whole fix for The Pulled Rug on the consuming side; it costs an afternoon to set up and an hour a month to keep. Vendors signal the same obligation from their side: Shopify's app requirements state that "apps using APIs that will be deprecated within 90 days can't be submitted" to its App Store.

Webhooks Done Well

A **webhook** is an HTTP request the source system sends to a URL you registered when something happens: an order is placed, a payment is captured, a shipment is created. It is the most common way a hosted platform tells the rest of the landscape that the world changed, and the mechanism most often trusted beyond what it promises. The two best-documented providers say the same things about their own webhooks.

Shopify's documentation (undated, accessed November 2026) states that "Webhook delivery isn't always guaranteed, and your app can miss or mishandle events for other reasons", that Shopify "doesn't guarantee ordering within a topic, or across different topics for the same resource", and that apps should "use reconciliation jobs to periodically fetch data from Shopify so that your app stays consistent with Shopify's data". "If Shopify receives no response or an error, it retries 8 times over the next 4 hours"; and "After 8 consecutive failures, the subscription is automatically deleted if it was configured using the Admin API". Every delivery carries a signature computed from

the raw body and the app's secret and a webhook id unique per delivery, because "your app might receive the same webhook more than once".

Stripe's documentation (undated, accessed November 2026) says Stripe "attempts to deliver events to your destination for up to three days with an exponential back off in live mode", "doesn't guarantee the delivery of events in the order that they're generated", that endpoints "might occasionally receive the same event more than once", and that handlers must "quickly return a successful status code (2xx) before any complex logic". Its signature is an HMAC (a keyed hash that proves who sent the message and that it was not altered) over a timestamp and the payload, with a five-minute tolerance against replay.

The Standard Webhooks specification (version 1.0.0; its implementations include Svix, Kong, Twilio, OpenAI and Anthropic) generalizes the pattern into three headers: `webhook-id`, unchanged across retries; `webhook-timestamp`; and `webhook-signature` over `id.timestamp.body`. It tells consumers to "Use the `webhook-id` header as an idempotency key", tells senders to retry over "multiple days, with an exponential backoff", and does not address ordering.

Property	Shopify	Stripe	Standard Webhooks
Signature	HMAC-SHA256 of the raw body	HMAC-SHA256 of `timestamp.body`	HMAC-SHA256 of `id.timestamp.body`
Replay protection	Triggered-at header (advisory)	5-minute tolerance by default	Timestamp tolerance, value unspecified
Duplicate key	`X-Shopify-Webhook-Id`	`event.id`	`webhook-id`
Retries	8 over 4 hours; subscription deleted after 8 consecutive failures when API-created	Up to 3 days, exponential backoff	Multiple days, exponential backoff with jitter
Ordering	Not guaranteed	Not guaranteed	Not addressed
Response expected	2xx within 5 seconds in total	2xx quickly, before business logic	Not specified

Read the table as a contract: **delivery is at least once at best**; **order is not promised**; **the signature is the only proof of origin**; **the receiver owns deduplication and a periodic sweep**. The receiving design that follows is the same for every provider.

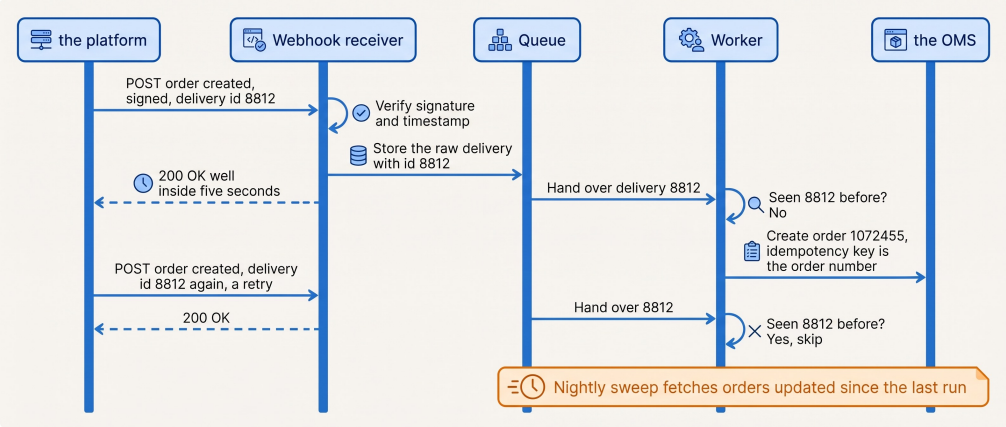


Figure 3: *A safe webhook receiver verifies, stores, answers fast and processes later; a duplicate delivery is recognized by its id, a duplicate order by its number, and a nightly sweep catches whatever was never delivered.*

Four rules, in the order they are usually skipped. **Verify the signature** before you parse anything; an unsigned webhook endpoint is a public URL that will create orders for anyone who finds it (Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers). **Answer first, work later**: store the raw delivery, return 2xx, process from a queue, so that a slow ERP does not make the platform give up and, in Shopify's case, delete the subscription. **Deduplicate twice**: on the delivery id, so a retried delivery is skipped, and on the business key (the order number, the return line), so two deliveries describing the same change produce one effect; Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation owns the idempotency key, the value that makes a repeated request have the effect of a single one. **Sweep periodically**: fetch everything updated since the last run, because the provider has told you deliveries will be missed. The sweep is the real integration; the webhook is the optimization that makes it feel immediate.

Rate Limits and Partner APIs

A **rate limit** is the maximum number of calls, or the maximum "cost" of calls, a consumer may make against an API in a period; above it, the provider answers with an error instead of data. It is a timing clause of the contract, and it decides whether an integration can finish before its next run starts.

Shopify's GraphQL Admin API meters calls in cost points: a bucket per app and per store refills at 100 points per second on the Standard plan, 200 on Advanced, 1,000 on Plus and 2,000 on its enterprise offering; an object costs 1 point, a mutation (a write) 10, and "A single query may not exceed a cost of 1,000 points, regardless of plan limits". Stripe's live-mode limits are 100 requests per second per account and 25 per endpoint, answered with `429 Too Many Requests`.

Two consequences follow, and the first is arithmetic. Kestrel & Pine's platform is not named, so apply Shopify's published numbers to its volumes purely as an illustration: a stock push that updates each of the roughly 17,000 active variants with one mutation at 10 points needs 170,000 points, about 28 minutes at 100 points per second and under three minutes at 1,000. An hourly job that takes 28 minutes and shares its bucket with 40 other apps will sometimes not finish, which is why batch operations are a necessity rather than an optimization, and why a flow's contract card records its expected call rate. The second consequence is that **limits are per store and shared by every app**: at peak, a chatty analytics app can starve the WMS connector (Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Black Friday Every Year: Peak Load, Resilience and Recovery).

Partner APIs, the ones a marketplace, a carrier or a wholesale customer exposes to the merchant, carry the same four parts and usually a thinner document. The EDI relationship with Bergzicht Outdoor is a contract too, and the ASN Chargeback of 2026 was a shape-and-meaning failure of exactly this kind: the despatch advice said one thing and the pallets said another (Documents Between Companies: EDI, Punchout and E-Invoicing).

Who Owns a Contract

Inside a company, the provider team owns the contract: it publishes the document, keeps the compatibility rules, announces deprecations and monitors usage. Zalando's account in InfoQ (December 2016; ten years old) put it as "We see API definitions as a source truth, part of a contract between a service provider and consumers", written in OpenAPI before any code ("API First"). The often-told origin story, an Amazon mandate from around 2002 that teams communicate only through service interfaces with "no direct reads of another team's data store", exists in public only as Steve Yegge's 2011 recollection, never confirmed by Amazon in any page this book cites; its rule is the useful part: **the contract is the only door**.

For a merchant of Kestrel & Pine's size, the rule applies to vendors before teams: integrations go through a vendor's documented API, never through a direct read of the ERP's database or a scrape of an admin page, because an undocumented path has no contract and therefore no deprecation notice. With vendors, ownership has three parts. The **commercial contract** names a minimum deprecation notice period, in months, and a technical contact that is a role. The **integration inventory** records the pinned version and the sunset date. And a **consumer test** runs against the vendor's sandbox on each new version before it becomes the default, checking that the fields the integration relies on are still there and still mean the same. That is as far as **consumer-driven contracts** can go with a vendor. The idea, coined by Ian Robinson in June 2006 (twenty years old and unchanged), says a provider's contract should be shaped by the union of what its consumers actually use; the Pact tool (documentation updated August 2026) generates

the contract from the consumer's tests and verifies the provider against it. Between services inside one company it works well; with a SaaS vendor, the vendor's schema is the contract.

The Choices and Their Trade-offs

As a consumer, the merchant has three postures per integration; the agency script took the second one, with a version string nobody treated as a pin.

Posture as consumer	What it means	Maintenance cost	Failure mode	Right when
Pin and schedule	State the version, record its sunset date, test the next version in a sandbox, switch before sunset	Low, continuous	A missed calendar entry	The flow carries orders, stock or money
Float on the default	State no version; take whatever the vendor serves	None, then an incident	Silent behavior change or sudden error	A read-only report; never orders, stock or money
Delegate to a connector	The vendor's own integration or an iPaaS connector tracks versions for you	Connector fee; its own calendar	Connector lags the vendor; mapping changes unnoticed	The merchant lacks engineers to own the flow

As a provider, the choices are the versioning strategies above, with one judgement: a merchant providing its first APIs should take Zalando's compatibility rules wholesale (additive change, tolerant readers, avoid versions) and defer the full deprecation machinery until it has consumers it cannot call on the phone.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting	The platform's apps and a vendor connector are the integrations; the merchant owns no code	A list of which apps and connectors exist and which version each pins	An app stops working after a platform release and nobody knows why
2 — Growing	First custom scripts and an iPaaS; webhooks from the platform to an ERP or 3PL; an agency writes most of it	Pinned versions with sunset dates in the inventory; webhook handlers that verify, deduplicate and sweep; a role mailbox for vendor notices	A script with a version string from the year it was written; a webhook subscription that vanished after an outage
3 — Omnichannel (Kestrel & Pine)	A dozen systems and dozens of flows; the merchant becomes a provider too (order status to agencies, portals, marketplaces)	A published document for every merchant-owned API, additive change, dated deprecation with monitoring, signed webhooks, a sandbox test per vendor version, notice periods in vendor contracts	The Deprecated Endpoint; one app's calls starving another at peak; a partner asking which version of "your" API to use

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
4 — Group	Several brands or units share platforms and partners; external partners depend on the group's APIs	API guidelines in the Zalando mold, a schema registry for events, contract tests in the delivery pipeline, a guild that reviews APIs	Two brands publishing incompatible versions of the same concept; a partner broken by a change one unit made

The topic never disappears; who owns it moves. At Starting, the vendor owns everything and the merchant's duty is to read. At Growing, the merchant's first scripts create its first consumer obligations, and this is where most Pulled Rugs are planted. At Omnichannel the merchant becomes a provider; at Group the duties become a function with a name. My judgement is that the Growing-stage habit pays most: a merchant that pins and schedules from its first script never has to learn the rest in an incident.

In Practice: The Contract Card for the bol Stock Flow

After February 2027, Ruben rewrote the inventory row for the flow that failed. Under Waypoint decision W5, every integration gets a contract; here is the card for the stock push to bol, in the form Kestrel & Pine uses from March 2027. All details are fictional.

Field	Value
Flow	Stock levels, platform to marketplace connector, bol listings
Owner	Milan Petrov (business), Ruben Alves (technical)
Style	Scheduled API push, hourly, until the availability service's events replace it in Q4 2027 under W1
Contract	The connector's stock API, pinned to the connector's current 2026 release; its changelog read monthly from the integrations role mailbox

Field	Value
Sunset of the pinned version	Published by the connector; recorded in the inventory; calendar entry three months ahead, assigned to the owner
System of record	The WMS for Duiven stock (W1); the push carries available-to-promise per channel once the availability service exists
Idempotency	Last write wins per SKU and location; each push carries its snapshot time so an older push cannot overwrite a newer one
Failure handling	An error raises an alert within fifteen minutes; two consecutive failed runs page the owner; the job never exits silently
Reconciliation	A daily comparison of the connector's stock per SKU against the platform's, with the variance published
Expected rate	About 17,000 variants per run, in batches within the connector's limit

The card is ordinary, and that is the point. Nothing on it required a new system; the flow now has a person, a version, a date, an alarm and a check. When the connector publishes its next retirement date, it lands on Milan's calendar and Ruben's test plan, not in the mailbox of someone who has left. By the end of 2027 the inventory shows the pinned version and sunset date of all 34 integrations on one page *(illustrative)*, which is what Noor shows the board when asked what the integration standards bought: not a platform, a calendar.

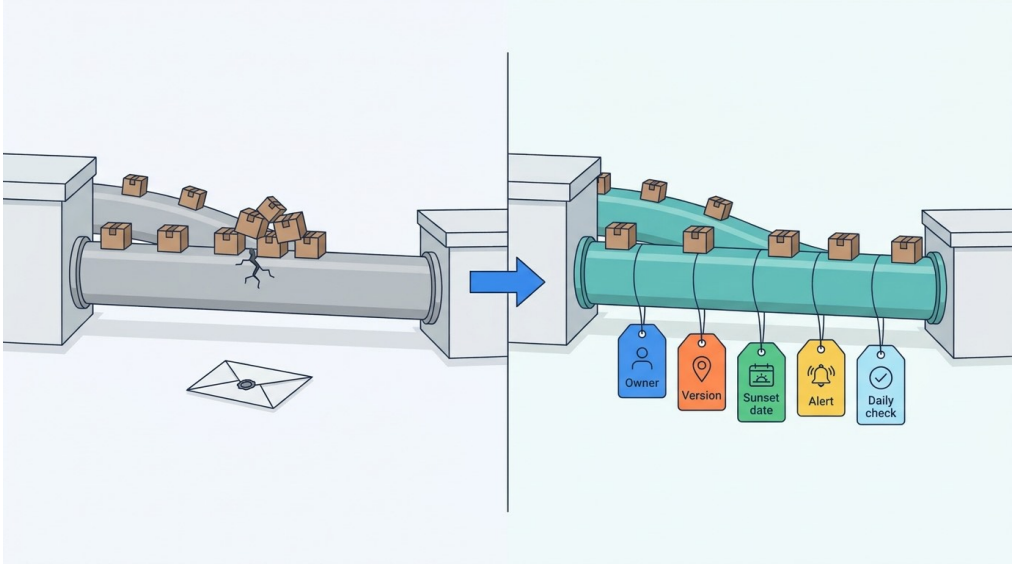


Figure 4: *The contract card adds no system: the stock flow now has a person, a version, a date, an alarm and a check.*

When It Goes Wrong

Failure	How you notice	What to do
The version expires (the Deprecated Endpoint)	Errors in a log nobody reads; a partner's complaint; a zero-traffic alert on the flow	Alert on errors and on silence; pin and calendar every version; test the next version early
Silent fall-forward	Nothing fails; a field is missing or means something else; a reconciliation variance grows	Watch the version header the provider returns; validate responses against the schema you expect
Webhook subscription deleted after an outage on your side	Orders stop arriving while the shop is busy; the sweep finds a backlog	Alert on traffic dropping to zero; re-create subscriptions automatically; rely on the sweep

Failure	How you notice	What to do
Duplicate or reordered deliveries	Two orders in the WMS with one order number; a cancellation processed before its order	Deduplicate on delivery id and business key; order by the provider's timestamp; apply updates as state, not as steps
Meaning changes without a version	Numbers shift on a date; no error anywhere	Read the changelog; keep a reconciliation that would notice; raise it with the vendor as a contract matter

Recovery is the same in every row: the sweep and the reconciliation job find what the contract failure hid, and the inventory says who fixes it.

What to Take Away

Established.

- Additive change is safe and removal is not: adding optional fields keeps existing clients working; removing, renaming, retyping or changing meaning breaks them (Google AIP-180, 2019, updated 2025; Zalando rules #106 to #108).
- A deprecation is a date pair plus monitoring: the `Deprecation` and `Sunset` headers (RFC 9745, 2025; RFC 8594) and Zalando's rules #185 to #189 make the announced date, the removal date, usage monitoring and consumer consent the standard parts.
- Webhooks are at least once, unordered and signed, and the receiver owns deduplication and a periodic sweep: Shopify's and Stripe's documentation say so in their own words, and Standard Webhooks 1.0.0 standardizes the headers.

Depends on context.

- Whether to force consumers to upgrade (Shopify) or carry old versions indefinitely (Stripe) depends on how many consumers you have and what a silent change costs: for payments, carrying old versions is worth the transformation code; for a catalog API with many apps, a forced annual upgrade keeps the surface small.
- Consumer-driven contract tests pay between services one organization owns; with a SaaS vendor, a sandbox validation per version is the practical substitute.

- Delegating version tracking to an iPaaS or a vendor connector is right for a merchant without integration engineers; it moves the obligation rather than removing it.

The author's synthesis.

- The cheapest fix for The Pulled Rug is a calendar: two columns in the integration inventory, a role mailbox for vendor notices and a rule that no flow goes live against a version with less than six months of support left. I have not seen a merchant need more until it becomes a provider.
- Treat your first self-provided API as if a stranger would call it, because within a year one will: publish the document, change only by adding, and deprecate with dates, before anyone asks.

Questions to Ask

1. For each integration that moves orders, stock or money: which contract version does it pin, when does that version expire, and who is on the calendar for the upgrade?
2. Where do the vendor's deprecation notices go: a role mailbox someone reads monthly, or a person who may leave?
3. Does our webhook receiver verify the signature, answer within the provider's timeout, deduplicate on the delivery id, and run a periodic sweep for what it missed?
4. If the provider answers a retired version by falling forward instead of failing, would we notice?
5. Which APIs do we provide to agencies, portals, partners or stores, and is each described in a published document with a stated compatibility and deprecation policy?
6. When a flow goes silent, who is alerted, within how long, and by what signal other than a partner's complaint?

Contracts decide how systems speak to each other; the next question is whether they are speaking about the same thing. When the platform, the POS and the returns portal each hold a record called "Eva Janssen" with no shared identifier, no contract can tell you they are one customer. That is the subject of One Customer, One Product: Identity and Master Data Across Systems.

Sources and Evidence

Company, product and investor sources

- API versioning, Shopify developer documentation, n.d., accessed 2026-11: quarterly dated versions, 12-month minimum support, nine-month overlap, fall-forward, release candidates, deprecation in the changelog; latest stable 2026-10.
- REST Admin API reference, Shopify, n.d., accessed 2026-11: REST Admin API legacy as of 1 October 2024; GraphQL-only for new public apps from 1 April 2025.
- Webhooks and HTTPS webhook delivery, Shopify, n.d., accessed 2026-11: delivery not guaranteed, no ordering, reconciliation jobs, timeouts, eight retries over four hours, subscription deletion, signature and id headers.
- GraphQL Admin API rate limits, Shopify, n.d., accessed 2026-11: cost points by plan, mutation cost, query maximum.
- App requirements checklist, Shopify, n.d., accessed 2026-11: apps using APIs deprecated within 90 days cannot be submitted.
- Versioning, Stripe documentation, n.d., accessed 2026-11: account pinning, `Stripe-Version` header, webhook endpoint versions, current version 2026-10-28.endive.
- APIs as infrastructure: future-proofing Stripe with versioning, Brandur Leach, Stripe, 2017: version change modules, compatibility with every version since 2011 (evidence age: nine years).
- Introducing Stripe's new API release process, Stripe, 2024: monthly non-breaking releases and two major releases a year.
- Receive Stripe events in your webhook endpoint, Stripe, n.d., accessed 2026-11: three-day retries, no ordering, duplicates, fast 2xx, signature scheme and tolerance.
- Rate limits, Stripe, n.d., accessed 2026-11: per-account and per-endpoint limits and the 429 response.
- RESTful API Guidelines, Zalando SE, changelog to 2026-03-16, accessed 2026-11: API First, compatibility rules #106 to #115, deprecation rules #185 to #191.
- On APIs and the Zalando API Guild, Zalando Engineering, 2015: the guild's size, time budget and review-not-approval role; "We cannot afford to break our APIs".
- Zalando's APIs and delivery under Radical Agility, InfoQ, 2016: API definitions as source of truth and contract.
- Stevey's Google Platforms Rant, Steve Yegge, 2011: the recollected Amazon service-interface mandate; widely reported, not an Amazon document.

Standards, regulation and official documentation

- RFC 9745: The Deprecation HTTP Response Header Field, IETF, Dalal and Wilde, 2025: the Deprecation header and its relation to Sunset (RFC 8594).
- AIP-180: Backwards compatibility, Google, created 2019, updated 2025: what counts as a compatible and a breaking change.
- Standard Webhooks specification 1.0.0 and project site, n.d., accessed 2026-11: headers, signed content, replay tolerance, retry guidance, steering committee and implementations.

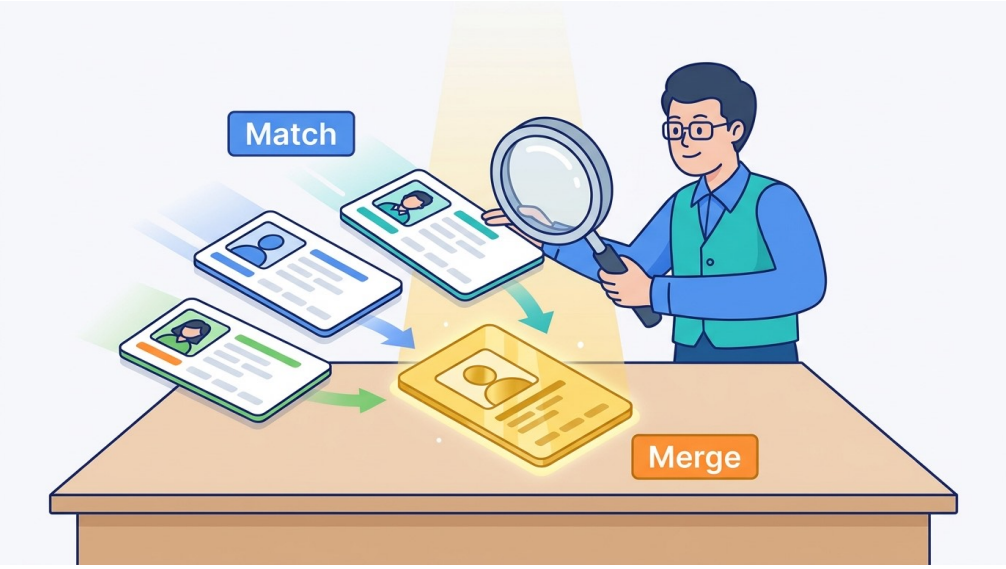
- OpenAPI Specification v3.2.1, OpenAPI Initiative, 2026: the HTTP API description standard.
- AsyncAPI Specification 3.0.0 and release notes, AsyncAPI Initiative, 2023: message-driven API descriptions.
- CloudEvents specification, CNCF, accessed 2026-11, and project page: the vendor-neutral event envelope.
- JSON Schema specification, JSON Schema, n.d., accessed 2026-11: version 2020-12.

Research and practitioner sources

- Consumer-Driven Contracts: A Service Evolution Pattern, Ian Robinson, 2006: provider, consumer and consumer-driven contracts (evidence age: twenty years; the idea is unchanged).
- Pact documentation, Pact Foundation, updated 2026-08-25: code-first consumer-driven contract testing.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

27. One Customer, One Product: Identity and Master Data Across Systems



IN THIS CHAPTER, YOU WILL: Learn why one person or product becomes four records, how matching and merging decide that they are one, and who should own the result.

KEY POINTS:

- **Every system keys the same person or product its own way, so duplicates are the normal case.** An online account is keyed by e-mail, a store customer by phone or loyalty card, a wholesale contact by company plus person, a product by SKU in the warehouse and by GTIN on the marketplace. Nothing links them unless you build the link.
- **Identity is a decision before it is software.** For each master entity, name the system of record, keep a cross-reference of every other system's identifier for it, write the matching rules and the survivorship rules down, and make every merge reversible. A merge that joins two different people is a privacy incident, not a data-quality blemish.

- **The marketing copy is not the master.** A customer data platform builds a fuzzy, useful profile for audiences; the record that drives orders, refunds, invoices and deletion requests must be exact and owned. Keep the two apart and let the exact one feed the fuzzy one.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Familiar Stranger** — each system counts customers its own way, so nobody can say whether the store customer and the online customer are the same person.
- **The Spaghetti Junction** — every connector maps product and customer identifiers in its own way, so the cross-reference between systems lives inside the connectors, with no owner.

Ask Kestrel & Pine, the fictional outdoor brand this book follows, how many customers it has, and you get four honest numbers. The platform counts accounts and guest checkouts. The point-of-sale system in the stores counts a loyalty list. The newsletter tool counts subscribers. The returns portal counts the people who have sent something back. Each number is right for its system, and none of them is the number of people who buy from Kestrel & Pine.

The same happens to products, more quietly. The Ridgewalker jacket in women's size M, color moss, is one physical thing with one barcode. In January 2027 it was also a platform variant, an ERP item with a variant code, a WMS item, a connector listing and three marketplace offers, each with its own identifier, and each connector held its own translation table. **When the joins between systems live inside the connectors, nobody owns them, and the day a connector changes, the joins go with it.**

This chapter is about the records that transactions point to, and the identifiers that name them. It owns four terms: **customer identity**, **master data**, the **golden record** and **matching**. It leans on Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record for the idea of a system of record, hands the legal side of personal data to Whose Data Is It? Privacy, Consent and Retention, and hands the product model itself to What Are We Selling? Products, Variants and the Catalog.

Where we are in the order's journey. Eva's jacket, the recurring order (see Meet Kestrel & Pine), is placed on 13 April 2027 by a logged-in platform account. Before that, Eva bought socks in the Nijmegen store and joined its loyalty list. After it, on 19 April, the returns portal creates a contact for her exchange. This chapter sits underneath the whole journey: it decides whether those records are one customer.

Setting the Scene at Kestrel & Pine: Four Evas

In **March 2027**, Milan Petrov, the ecommerce and marketplaces manager, proposed an online loyalty program for the autumn. Noor Visser, the Head of Engineering, asked

a question with no answer: how many customers would be eligible, and how would the program know that a shopper in the Nijmegen store is an account on the Dutch storefront?

Ruben Alves, the lead integration engineer, pulled the counts. The figures are illustrative.

List	System	Records (illustrative)	Key the system uses
Customer records	The platform (storefront, cart, checkout)	about 410,000, including guest checkouts	Customer id; e-mail address
Loyalty list	The POS (point of sale: the till software in the six stores)	about 38,000	Phone number; loyalty number
Subscribers	The newsletter tool (one of the 41 platform apps)	about 165,000	E-mail address
Contacts	The returns portal	about 61,000	E-mail address plus order number

The four lists add up to about 674,000 records. Nobody believes Kestrel & Pine has 674,000 customers; nobody can say how many it has.

Eva Janssen shows why. She is one person in Nijmegen. In Kestrel & Pine's systems she is four:

Where	What the record says	What it does not know
The platform	An account from 2024 under her personal e-mail; three orders, the latest #1072455 of 13 April 2027; an address in Nijmegen	That she shops in the Nijmegen store
The POS loyalty list	"E. Janssen", joined at the Nijmegen store in 2023 with her mobile number; four purchases, the latest socks in February 2027	Her e-mail; her online orders

Where	What the record says	What it does not know
The newsletter tool	A subscriber since 2022 under an older e-mail address	That she is a customer at all
The returns portal	A contact created on 19 April 2027 from order #1072455, with her personal e-mail	Nothing else about her

No field is shared by all four. **The Familiar Stranger is not a data-quality accident. It is what four systems with four keys produce when nobody has decided that they describe one person.** Milan and Noor deferred the loyalty program until the customer record was unified (Discounts, Coupons and Loyalty: Promotions Without Chaos); this chapter is what "unified" means.

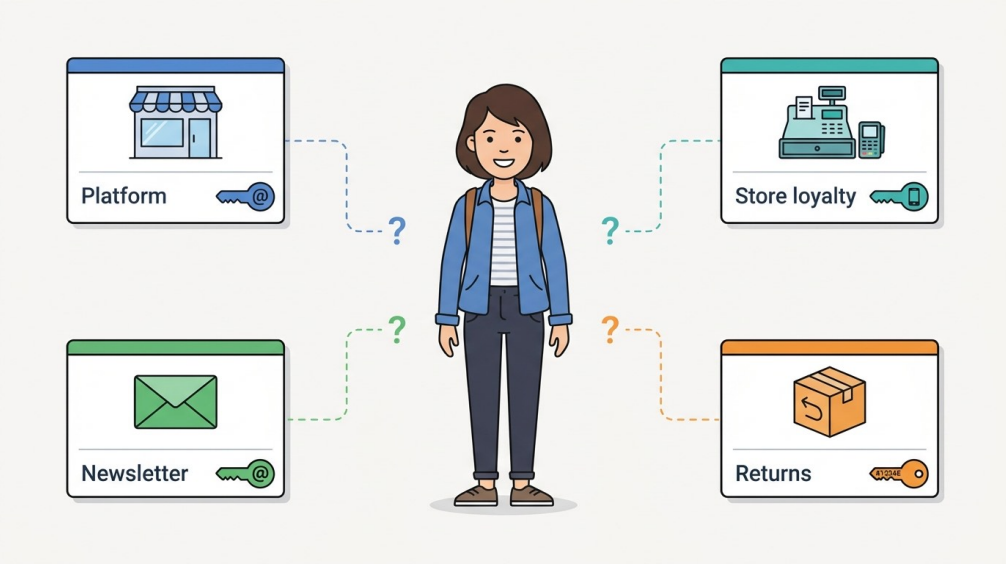


Figure 1: *One shopper, four records, four keys: no field is shared by all four, so the four lists add up to about 674,000 records and nobody can say how many customers there are.*

Why One Customer and One Product Matter

The vocabulary first, gently. An **identifier** is a value a system uses to tell one record from another: a customer id, an e-mail address, an order number, a barcode number. **Customer identity** is the set of identifiers that different systems use for the same person, plus the rules that link them. **Master data** is the slowly changing reference data that

transactions point to: customers, products, suppliers, locations. A **system of record** is the one system whose version of a fact is the truth (the idea comes from Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record); for master data it is usually called the **master**.

Why does a merchant lose money or trust when identity is undecided?

- **Marketing**. Eva gets the new-subscriber offer at her older e-mail while her account is three orders old; a store customer who opted out is e-mailed because the opt-out lives only in the POS. Consent belongs to a person, and a person the systems cannot see cannot be respected (Whose Data Is It? Privacy, Consent and Retention).
- **Service**. The agent answering Eva's exchange question cannot see her store purchases. Where Is My Order? Notifications, Service and the Customer Record describes the five tabs an agent works from; identity is why there is no sixth that joins them.
- **Loyalty and promotions**. "First order", "members only" and "once per customer" need to know who the customer is (Discounts, Coupons and Loyalty: Promotions Without Chaos); a program built on one list rewards that list and ignores the rest.
- **Returns and refunds**. A refund goes back to the order's payment method, which is safe; a store credit or an exchange raised against the wrong record goes to the wrong person.
- **Analytics**. "Customers", "new customers" and "repeat rate" are undefined while the person is (Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics).
- **Privacy**. A person who asks to be forgotten must be found in every system within one month; without an index of where Eva is, the request cannot be fulfilled (Whose Data Is It? Privacy, Consent and Retention).

Products have their own version of each loss: a marketplace listing without the GTIN of its warehouse item becomes a second product, and an ERP item whose variant code maps to no platform variant is a sale the ERP cannot cost.

The identifiers a commerce estate uses

For customers:

Identifier	Who assigns it	Stable?	Unique to one person?	Who else understands it
Customer id	The platform, the POS, the ERP, each its own	Yes, inside that system	Yes, inside that system	Nobody else, unless carried in a cross-reference
E-mail address	The person	Changes with jobs and providers	Often shared by a household; one person has several	Every system, which is why it is the usual match key
Phone number	The person	Changes rarely	Usually one person, sometimes a family	Stores and carriers
Loyalty number	The POS or a loyalty program	Yes	Yes, but one person may hold two cards	Only the program
Postal address	The person	Changes with moves	Shared by a household	Carriers; useful for matching, never alone

Identifier	Who assigns it	Stable?	Unique to one person?	Who else understands it
Company id plus contact	The ERP or the B2B portal	Yes	A company has many contacts; a contact moves between companies	EDI (electronic data interchange) partners by GLN (the Global Location Number that GS1, the barcode standards body, assigns to companies and sites; see Documents Between Companies: EDI, Punchout and E-Invoicing)

For products, the lesson of What Are We Selling? Products, Variants and the Catalog is that a SKU (the merchant's own code for a sellable variant) and a GTIN (the Global Trade Item Number under the barcode, licensed from GS1 so that every company means the same item by it) are assigned once and never change. Here is what each identifier is good for, and what it is not:

Identifier	Who assigns	Scope	Good for	Not
GTIN (EAN-13, the 13-digit barcode number, in Europe)	The brand owner, through GS1	Global, one per sellable item	Marketplace listings, retail scanning, matching across companies	An internal key: resellers of the same item share it, and packaging variants differ
SKU	The merchant	Internal, one per variant	Warehouse, stock, merchandising, the join key between the merchant's own systems	Stable across systems unless governed: connectors overload it
Internal id	Each system	That system only	Joins inside that system	Meaningful anywhere else
Marketplace id	The marketplace (an ASIN, Amazon's own product number; an offer id on bol)	That marketplace	That channel's listings and orders	Yours; the marketplace can change what it refers to

The table is the author's synthesis of the GS1 US definition of the GTIN (n.d., accessed November 2026) and of Microsoft's documentation for its Business Central connector for Shopify (October 2026), which matches items on SKU or barcode and, when it finds an existing variant, links rather than creates. That one sentence of documentation is a product-identity policy: **match on the governed identifiers, link, never duplicate**.

How the Pieces Fit Together

Identity is not one system but a set of responsibilities spread across the systems a merchant already has.

Responsibility	What it does	Where it usually lives	Kestrel & Pine, January 2027
The customer master	The one record per consumer the business treats as true: name, contact details, addresses, consent	The platform for consumer merchants; the ERP or a CRM (customer relationship management system) for business customers	Undecided; four lists
The product master	Holds the one record per product and variant, with its SKU and GTIN	The PIM (product information management system) where there is one; the ERP's item master where there is not	Three copies plus the connector's; the PIM arrives in Q3 2027 (W3)
The cross-reference and merge log	For each master record, every other system's identifier for it, and every link or merge made, reversibly	A table owned by the master's owner; often in the integration platform or the data warehouse	Inside each connector's mapping
Matching	Decides whether two records are the same entity when they share no identifier	A batch job, a platform feature, a customer data platform (CDP) or a master data management (MDM) tool	Nothing
Survivorship	When two records merge, which value wins per field	Rules written down; enforced by the merge tool	Nothing
Consent	The person's marketing choices, with when, where and what they agreed to	On the customer master, synchronized outward	In the newsletter tool and the POS, separately

Responsibility	What it does	Where it usually lives	Kestrel & Pine, January 2027
The customer data platform (CDP)	A unified profile per person built from every touchpoint, for audiences and personalization	A separate product, fed by the master and the event streams	None

The design rule I use: **the master owns the record, the cross-reference owns the links, and every other system keeps its own record but carries the master's id once matched.** The POS still needs its loyalty list and the returns portal its contacts; what changes is that each record points at one master, and the master knows where its copies are.

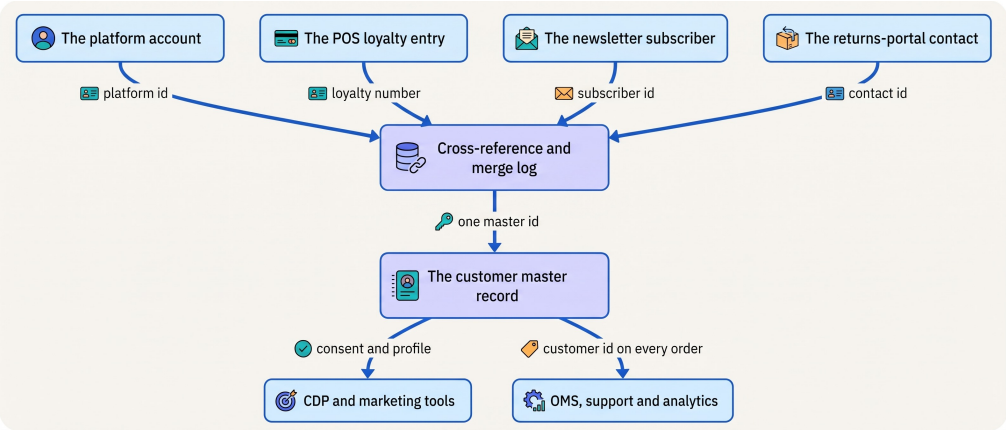


Figure 2: *Four systems keep their own records of Eva; a cross-reference links each to one master record, which feeds marketing copies and the operational systems.*

How a new record finds its master

The sequence below shows what should happen when Eva buys socks in the Nijmegen store in February 2027, in the design Kestrel & Pine adopts later in the chapter.

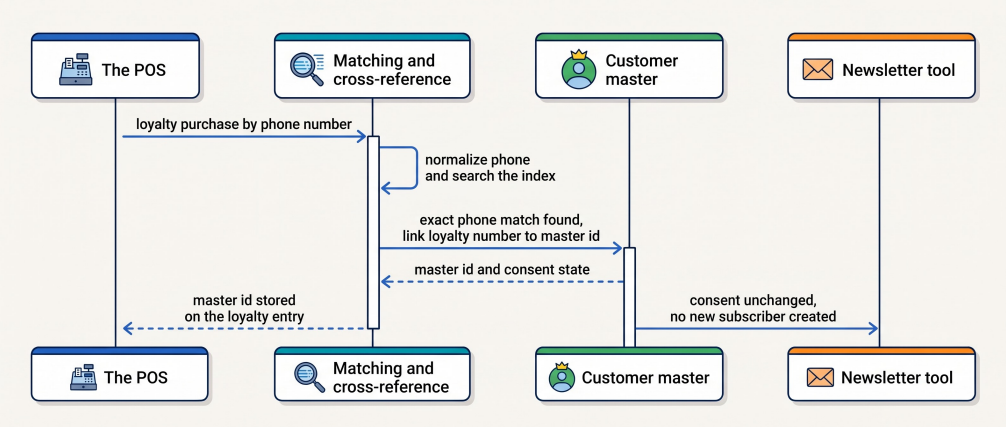


Figure 3: *A store purchase is matched on a normalized phone number; the loyalty entry is linked to the master, and consent is read from the master rather than recreated.*

Three things carry the design: the POS keeps its own entry and gains a master id; the match is on an exact, normalized key, so no human is needed; and consent is read from the master, which is why the newsletter tool does not quietly gain a subscriber from a store sign-up.

Requirements per flow

Identity flows differ in how fast and how sure they must be.

Flow	How fresh	How consistent	How fast	How available
Online checkout, logged in	Immediate	The account id is the key; no matching needed	Part of checkout	Must not block checkout
Guest checkout	Immediate	Creates a record keyed by e-mail; matched to an account later, never during checkout	Part of checkout	Must not block checkout

Flow	How fresh	How consistent	How fast	How available
Store loyalty sign-up or purchase	Hours	Exact match on phone or e-mail; otherwise a new master record	Batch is fine	Can wait; the store never waits on it
Newsletter sign-up	Hours	Exact match on e-mail; consent recorded on the master with time, place and wording	Batch is fine	Can wait
Returns-portal contact	Minutes	Keyed by order number, so the match is exact	Part of the return start	Can degrade to "unlinked"
New product variant	Weeks before launch	One SKU and one GTIN per variant, created once in the master, published outward	Batch	Office hours

The pattern: **the flows that face a customer are exact or deferred, never fuzzy**. Nothing in the checkout path waits for a matching job; probabilistic matching, explained next, runs in the background and sends its uncertain cases to a person.

Matching and Merging

Matching is deciding that two records with no shared identifier refer to the same real-world thing. The field is called record linkage, and it is older than commerce computing: Dunn described it in 1946, Newcombe and colleagues showed in 1959 that agreeing on a rare value (an unusual surname) is stronger evidence than agreeing on a common one, and Fellegi and Sunter gave the model most tools still use in 1969. In plain words: for each candidate pair, add up evidence field by field (the same e-mail is strong, the same

first name weak), and compare the total with two thresholds. Above the upper one, a match; below the lower one, not; between them, a person decides. The 1969 paper is paywalled, so the citation is secondary.

There are two families of rule. **Deterministic matching** uses exact rules: the same normalized e-mail or phone means the same person. It is cheap, explainable and safe, and it misses every person who used two e-mail addresses. **Probabilistic matching** scores partial agreement (name, address, postal code) and tolerates typos. It finds more matches and makes more mistakes, which is why it needs thresholds and a review queue.

Before either, **normalization**: lower-case the e-mail and strip spaces; write phone numbers in the international format; write postal codes in the national format. Most "duplicates" in a commerce estate are the same key spelled two ways. And for any estate larger than a few thousand records, **blocking**: compare only pairs that already agree on something (the same postal code, the same surname), because comparing every record with every other grows with the square of the count; two lists of a million records would make a million million pairs.

The rules I use for a merchant's customer matching:

1. Normalize first; match on exact e-mail or exact phone automatically.
2. Score name plus address plus postal code; auto-link above a high threshold, queue the middle band for a person, and never link on name plus city alone.
3. Never merge during a customer-facing flow; link in the background.
4. Keep every merge reversible, in a log that records which records became one, when, by which rule and who approved it.
5. Treat a wrong merge as a privacy incident and rehearse the unmerge: it shows one person the other's orders and addresses, a personal-data breach (Whose Data Is It? Privacy, Consent and Retention).

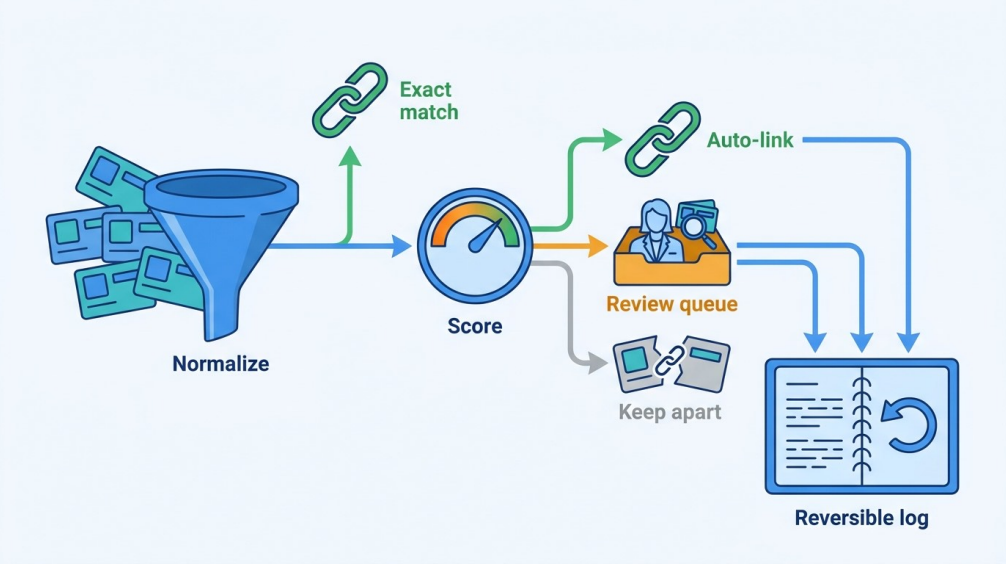


Figure 4: *Customer matching in the order the chapter recommends: normalize, link exact e-mail or phone matches automatically, score the rest, send the middle band to a person, and keep every merge reversible.*

Survivorship is the rule that decides which value wins when two merging records disagree: two names, two addresses, two consent states. The usual rule types are source priority (the platform's address beats the newsletter tool's), recency (the latest address wins) and completeness (the record with a phone number wins), as the MDM vendor Profisee describes them (August 2025). The rule is per field, not per record: the newest address, the strictest consent, the earliest created-date.

Shopify documents its own survivorship, which makes it the clearest dated example. Its `customerMerge` mutation (API version 2026-10) says the two ids "don't guarantee which customer is kept": the caller may name the customer whose e-mail survives; otherwise the one with an e-mail wins, then an enabled account, then the marketing-consent state; first and last name can be overridden per field; the merge runs as a job whose resulting id must be treated "as authoritative", and a preview shows the outcome first. Most platforms have a comparable rule and do not publish it. **That is why the entity–owner table must name the customer master: if the platform merges by its rules and the ERP by others, the two masters drift apart on their own.**

For products, matching is simpler and stricter. The GTIN is the match key across companies, the SKU inside one. A product record with no GTIN is matched by hand or not at all. GS1's rule, relayed by secondary sources, is that every distinct trade item and packaging configuration gets its own GTIN and a GTIN is never reused for a different item; What Are We Selling? Products, Variants and the Catalog carries the detail. The product cross-reference (GTIN, SKU, each system's internal id, each marketplace's id) is the product master's companion table. It belongs to the PIM where there is one, and

to the ERP's item master where there is not.

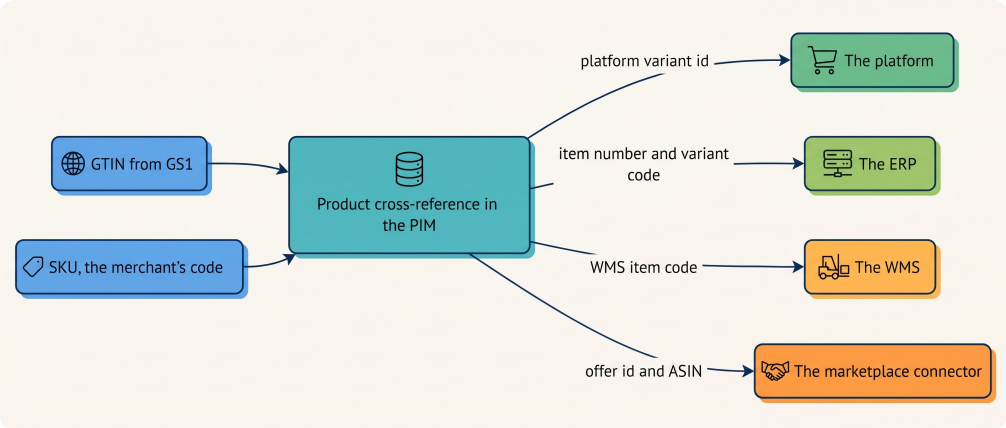


Figure 5: *One product, two governed identifiers and four internal ones; the cross-reference in the PIM is where every other system's key for the Ridgewalker women's M moss is recorded.*

Consent travels with the identity

The General Data Protection Regulation (GDPR), the EU's personal-data law, defines consent as a freely given, specific, informed and unambiguous indication of the person's wishes (Article 4(11)) and requires the controller to be able to demonstrate that consent was given (Article 7(1)). "Demonstrate" is a data requirement: the record must hold when, where and to what the person agreed, which is why Shopify's consent mutation records "when and where the customer gave or withdrew consent". A merge must carry that evidence forward, with its source, rather than recompute consent from whichever record survived. The legal side belongs to Whose Data Is It? Privacy, Consent and Retention.

The Choices

A merchant has four realistic ways to own customer identity, and the product side has two.

The platform's customer record is the master, with periodic deduplication. Every other system's record is linked to a platform customer id; matching is the platform's own merge feature plus a scheduled job for the other lists. This fits a consumer merchant whose platform already holds most customers, which is most merchants below the Group stage.

The ERP or a CRM is the master. The finance or sales system holds the customer; the platform's accounts are copies. This fits wholesale-heavy businesses, where the customer is a company with terms and a credit limit that the ERP already owns (Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals). For consumers it usually fails: the ERP's customer record was designed for invoicing, not for consent, addresses and preferences.

A cross-reference and matching service beside the existing systems. Nothing moves; a small service, or tables in the integration platform or the data warehouse, holds the master id, the links and the merge log, and runs matching. It is the lightest way to make several masters behave as one, and the design Kestrel & Pine adopts. Its weakness: it is "just tables" until a wrong merge happens.

A customer data platform as the master. The CDP Institute (2026 wording) defines a CDP as software that "creates and maintains a persistent, unified customer record that is accessible to other systems" and that "assumes primary responsibility for defining and maintaining customer identity". Twilio Segment's identity resolution (documentation of June 2026) merges each person's history across web, mobile, server and third-party sources into one profile through an identity graph of cookie ids, device ids, e-mails and external ids, with merge protection against non-unique anonymous ids. These are strong marketing tools. My judgement, labeled as mine: do not make the CDP the master that service agents, the OMS and the ERP read. Its profile is built for audiences and may be fuzzier than a record that drives invoices, refunds and deletion requests. Let the exact master feed the CDP, and let the CDP's matches come back as suggestions, not merges.

For products: **the ERP item master or the PIM.** Until product content needs enrichment, translation and syndication, the ERP's item master with a SKU and a GTIN per variant is enough, and the cross-reference to channel ids lives beside it. From that point the PIM is the product master and the ERP keeps costs and purchasing, which is Waypoint's W3 (What Are We Selling? Products, Variants and the Catalog).

A fifth option exists for the Group stage: a **master data management (MDM)** platform. Informatica describes MDM as creating "a single master record for each person, place, or thing in a business", the record "also known as a 'golden record' or 'best version of the truth' ", with software that matches and merges across systems and keeps an audit trail (n.d., accessed November 2026). A **golden record** is that one assembled record per customer or product, built by merging sources under survivorship rules. Below the Group stage I keep MDM small: a decision per entity (owner, survivorship rule per field, merge process), not a product. Buying the platform is a move driven by several brands, acquisitions or countries, each with a master to reconcile.

Trade-offs

Option	Fits when	Strengths	Costs and limits	Warning sign
Platform as customer master	Consumer merchant; most customers come through the storefront	Already exists; merge and consent features; one id on every online order	Store and wholesale records are second-class; survivorship rules are the vendor's	The POS list is never matched because "the platform is the master"
ERP or CRM as master	Wholesale-heavy; company customers with terms	Terms, credit and invoices already there	Consumer fields fit badly; marketing tools cannot read it	Consumer records keyed by invoice address
Cross-reference and matching service	Several systems holding customers; no replat-forming	Nothing moves; reversible; cheap to start	Needs a named owner and a review queue; easy to leave untended	A wrong merge nobody can undo
CDP as master	Marketing needs audiences across many touchpoints	Identity graph across anonymous and known; activation to advertising and e-mail	Probabilistic; not built for refunds, invoices or deletion; cost scales with events	Service agents working from the CDP's view
MDM platform	Group stage: several brands, countries or acquisitions	Survivorship per field across masters; audit trail; stewardship workflow	License plus a data-governance function; a year to implement	Bought before the second brand exists

The product side has the same shape in miniature: the ERP item master until enrichment and syndication need a PIM, and never two masters for the same identifier.

Across Scale Stages: When It Fits and When It Limits

The four stages are the book's (see What Changes at Each Scale: From First Store to Group).

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting	One platform; its customer record is the only one; SKUs typed by hand, GTINs bought in a block	One e-mail per customer; a periodic duplicate merge in the platform; one SKU and one GTIN per variant, never reused	Guest checkouts outnumber accounts and nobody merges them; a SKU corrected in place after the first sale
2 — Growing	The first store, wholesale account and marketplace each bring a new key: loyalty number, company plus contact, marketplace id	Decide the customer master (platform, ERP or CRM) and the product master (ERP or PIM); a product cross-reference before the first marketplace; consent on the master	A loyalty list nobody can join to online accounts; a marketplace listing created as a second product for want of a GTIN

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
3 — Omnichannel	A dozen systems holding customers; four product copies; a mapping inside each connector; marketing asking for a CDP	Matching with thresholds, a review queue and a merge log; consent with evidence on the master, synchronized outward; a PIM as product master; a CDP, if needed, fed from the master	A wrong merge found by a customer; "how many customers" answered with four numbers; a deletion request that cannot find every copy (Whose Data Is It? Privacy, Consent and Retention)
4 — Group	Several brands or countries, each with its own masters; acquisitions bring more	An MDM function or platform across brands; survivorship per field; a merge log privacy can audit; one consent model with national differences as configuration	One brand's customers e-mailed under another's consent; two GTINs for one item in two catalogs

The judgement: the identifiers do not change with scale, and the cheapest decisions are the earliest: one SKU and one GTIN per variant from the first day, one e-mail per account, a named customer master before the first store. What changes is how many systems hold a copy, and so how much linking there is to own. Kestrel & Pine is a stage-3 merchant that skipped the stage-2 decision, which is why it has four lists and no answer.

In Practice

The public evidence for this chapter is documentation, not case studies: vendors describe their identity features, and no merchant publishes its duplicate rate.

- **Shopify (2026)** documents what a platform-as-master design gives a merchant: `customerMerge` with published survivorship rules and a preview, and a consent

mutation that records when and where consent was given. Both operate only on the platform's own records. Orders from Shopify's own POS land in the same customer and order model (Microsoft's connector documentation of October 2026 maps their staff id to a salesperson): the suite answer to the Familiar Stranger is one vendor for till and storefront. Kestrel & Pine's POS is from another vendor; the platform's own POS is evaluated in 2028 (W4).

- **Microsoft's Business Central connector (October 2026)** matches items on SKU or barcode and links rather than creates; its SKU field may hold the ERP item number, the item number plus a variant code, or the vendor's item number, depending on a setting. A merchant with two such connectors and two settings has two cross-references.
- **The CDP category (2013–2026)**: named by David Raab in 2013, defined by the CDP Institute (founded 2016), implemented with identity graphs by products such as Twilio Segment (June 2026).
- **The MDM vocabulary**: Informatica's and Profisee's pages (2025–2026) define the golden record and survivorship; DAMA's Data Management Body of Knowledge (revised 2024) is the formal reference.

Kestrel & Pine's identity decision (2027, illustrative)

In **June 2027**, at the weekly architecture hour, Noor Visser, Milan Petrov and Ruben Alves adopted a customer-identity rule, recorded as an addition to Waypoint rather than a new W-decision. The figures are illustrative.

- **The platform's customer record is the master for consumers**. The POS loyalty list, the newsletter tool and the returns portal keep their own records and carry the platform customer id once matched.
- **The ERP is the master for business customers and contacts**, as it already was for terms and credit (Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals).
- **The cross-reference and merge log live in the data warehouse**, owned by Hanna Berg's data pair, because the warehouse already receives every list nightly and W7's one definition of a customer needs the same table.
- **Matching runs weekly**. Exact matches on normalized e-mail or phone link automatically; name plus address plus postal code goes to a review queue worked by the application support specialists; the platform's own merge is used only after review, with its preview.
- **Consent is read from the master**. The newsletter tool's subscription state and the POS's opt-in flag are copied to the platform's consent record with source and date; the strictest state wins where they disagree.

The first run, in **July 2027**, linked about **21,000** of the 38,000 loyalty entries to a platform customer by exact phone or e-mail, sent about **4,100** to the review queue and left the

rest unlinked. Of the 165,000 newsletter subscribers, about **96,000** matched a platform customer. Eva Janssen's four records became one master with three links: her loyalty entry by phone, her returns contact by order number, and her older newsletter address by a reviewer who saw the same name, street and postal code. The product side waited for the PIM: its July 2027 load from the four copies of product data found 1,900 variants with conflicting attributes (Moving Without Burning the Bridge: Replatforming and Migration), and the cross-reference of GTIN, SKU, platform, ERP, WMS and marketplace ids was built in the PIM at the same time (W3).

What the decision bought: one answer to "how many customers" (the master count, with unlinked store entries reported separately), a loyalty program that could start in 2028 on one list, a service screen that could show Eva's store purchases next to her online orders once the OMS arrived (W2), and an index of where every person's data lives, which Whose Data Is It? Privacy, Consent and Retention needs.

When It Goes Wrong

Failure	How it shows	Detection	Recovery
The wrong merge	A customer logs in and sees another person's orders or address	A complaint; a review of recent merges near the threshold	Unmerge from the log; notify the affected people; assess it as a personal-data breach (Whose Data Is It? Privacy, Consent and Retention); raise the threshold
Duplicates from guest checkout	One person, six platform records, six "first order" discounts (Discounts, Coupons and Loyalty: Promotions Without Chaos)	Duplicate rate on normalized e-mail, reported monthly	Scheduled exact-match merge; a "sign in to use your discount" rule

Failure	How it shows	Detection	Recovery
Consent lost in a merge	A person who opted out is e-mailed because the surviving record said "subscribed"	A consent reconciliation between the master and each marketing tool	Strictest state wins; re-sync outward; keep the evidence of the original opt-out
The overloaded SKU	A connector writes the ERP's item-plus-variant code into the platform's SKU field; the WMS expects the plain SKU; pick lists fail	A cross-reference reconciliation: one SKU and one GTIN per variant in every system	Fix the connector setting; rebuild the cross-reference from the master
The missing GTIN	A marketplace creates a second listing for the same item; stock and reviews split between them	A listing-to-GTIN reconciliation per marketplace	Merge the listings where the marketplace allows; never list without a GTIN
The connector that owned the mapping	A connector is replaced (Promises Between Systems: API Contracts, Webhooks and Versioning) and the cross-reference goes with it	Discovered at migration	Keep the cross-reference in the master's companion table, never only in a connector; export it before any connector change

The wrong merge is what makes matching dangerous; the safeguard is not a better algorithm but a reversible log and a person in the middle band. The overloaded SKU is the product-side Spaghetti Junction: the estate's identifier policy expressed as a drop-down in one connector's settings.

What to Take Away

Established.

- Record linkage is a mature field: exact rules for shared keys, probabilistic scoring with two thresholds and a review band (Fellegi and Sunter, 1969), normalization and blocking to make it feasible.
- Platforms and connectors implement survivorship and cross-reference policies their own way and sometimes publish them: Shopify's `customerMerge` rules (API 2026-10) and Business Central's item matching on SKU or barcode (October 2026).
- The GTIN is the only product identifier every external party understands; the SKU is the only one the merchant controls; both are assigned once and never reused (GS1; What Are We Selling? Products, Variants and the Catalog).
- Consent must be demonstrable (GDPR Article 7(1)), so it is a record with time, place and wording that travels with the identity through every merge.

Depends on context.

- The platform is the right customer master for a consumer merchant below the Group stage and the ERP for business customers; a merchant with both has two masters and one cross-reference, which is fine as long as both are named.
- A CDP earns its place when marketing needs audiences across anonymous and known touchpoints; it does not earn the role of master for service, orders and deletion, whatever the vendor's definition says.
- Probabilistic matching is worth its review queue from the Omnichannel stage; below it, exact matching on normalized e-mail and phone catches most duplicates at no risk.
- An MDM platform is a Group-stage purchase, triggered by a second brand or an acquisition, not by the first duplicate list.

The author's synthesis.

- Identity is the box the capability map leaves blank the longest, because every system works without it; make the decision before the first store, and if you did not, make it the way Kestrel & Pine did: nothing moves, the links are owned, every merge can be undone.
- Keep the exact record and the fuzzy profile apart and let the exact one feed the fuzzy one; the day they are the same table, a marketing match becomes an invoice error.

Questions to Ask

1. For each master entity (consumer, business customer, product, supplier, location): which system is its master, and is that written down?
2. Where is the cross-reference between the master's id and every other system's id kept, and is it anywhere other than inside a connector?
3. What are the matching rules, what is matched automatically, and who works the review queue?
4. Is every merge logged and reversible, and when was an unmerge last rehearsed?
5. Which value wins, per field, when two records merge, and does consent carry its evidence?
6. What is the duplicate rate on normalized e-mail, and who reports it?
7. Does every sellable variant have one SKU and one GTIN in every system, and which job checks?
8. Could you list, today, every system that holds a given person's data?

The last question is where this chapter hands over. Once a merchant can say who its customers are and where their records live, it can count them. Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics takes the master and the cross-reference built here and asks why three reports still show three different order counts, and how to build analytics the business trusts. The question of what the merchant may do with the records, and how it forgets them, waits for Whose Data Is It? Privacy, Consent and Retention.

Sources and Evidence

Company, product and investor sources

- customerMerge mutation, Shopify developer documentation, API version 2026-10, accessed 2026-11 — the published survivorship rules, asynchronous job and preview.
- customerEmailMarketingConsentUpdate mutation, Shopify developer documentation, n.d., accessed 2026-11 — consent state, opt-in level, and "when and where" consent was given or withdrawn.
- Customer accounts, Shopify developer documentation, n.d., accessed 2026-11 — accounts, pre-authenticated order status pages, deprecation of legacy accounts.
- Synchronize items with Shopify, Microsoft Learn, 2026-10-16 — SKU and barcode matching, the SKU field's three possible contents, link rather than create.
- Synchronize and fulfill sales orders, Microsoft Learn, 2026-10-14 — POS and draft orders carrying a staff id into one order model.

- What is a CDP?, CDP Institute, 2026 wording, accessed 2026-11 — the definition and the "primary responsibility" for customer identity.
- Identity Resolution, Twilio Segment documentation, last modified 2026-06-04 — identity graph, external ids, merge protection.
- What is master data management?, Informatica, n.d., accessed 2026-11 — the golden record and MDM domains.
- What Is a Golden Record in MDM?, Benjamin Bourgeois, Profisee, 2025-08-26 — survivorship rule types: source priority, recency, completeness.

Standards, regulation and official documentation

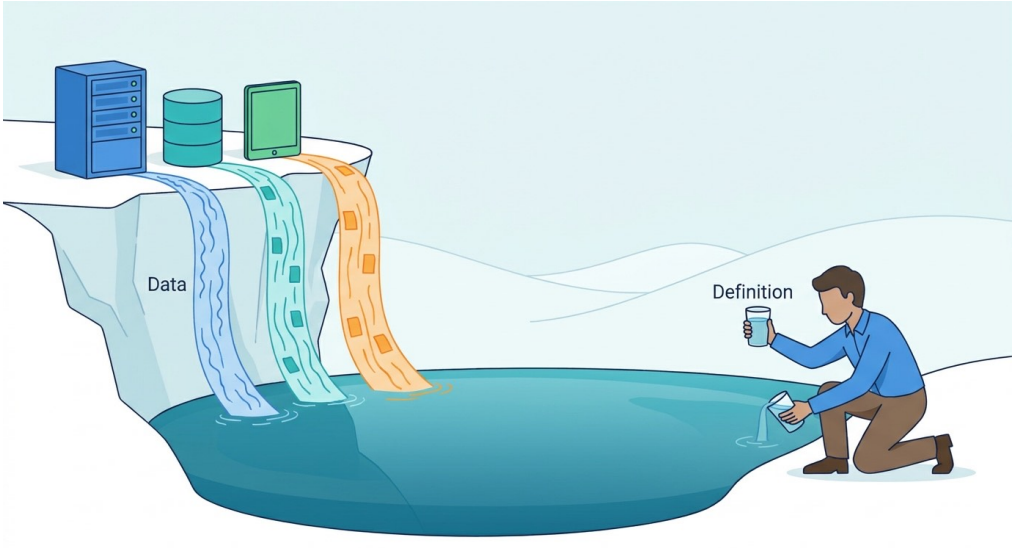
- What is a GTIN?, GS1 US, n.d., accessed 2026-11 — GTIN-8, -12, -13 and -14 and their uses.
- Regulation (EU) 2016/679 (GDPR), Official Journal text, 2016 — Article 4(11) on consent and Article 7(1) on demonstrating it.
- DAMA Body of Knowledge, DAMA International, n.d., accessed 2026-11 — the practitioner framework for master data governance, revised 2024.

Research and practitioner sources

- Record linkage, Wikipedia, accessed 2026-11 — the history (Dunn 1946, Newcombe 1959, Fellegi and Sunter 1969), thresholds, blocking; a secondary source, used because the 1969 paper is paywalled.
- One Property, Three Listings: Duplicates and Entity Resolution in the sibling book LISTED — the full treatment of matching, blocking and merge reversibility for marketplaces, for readers who need depth.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

28. Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics



IN THIS CHAPTER, YOU WILL: Learn why honest systems count orders differently, how a data platform collects every system's data, and how one owned definition per metric makes a number trustworthy.

KEY POINTS:

- **Three order counts have three honest causes: definition, collection and time.** Each system counts what it exists to handle (orders placed, orders invoiced, parcels shipped), each loses or gains records in the copying, and each uses its own clock. None is wrong; none is the company's number until someone defines it.
- **Keep operational and analytical data apart, and copy everything into one place.** Operational stores answer "where is this order" for one order; analytical stores answer "how many orders by channel" over all of them. A warehouse or lakehouse fed by extract-and-load jobs and event streams is where the counts can be compared.

- **Define each metric once, with an owner, and publish the bridge.** A metrics layer holds "order", "net sales" and "return" as code so every dashboard computes them the same way. Waypoint decision W7 gives Kestrel & Pine's definitions to the Data Lead and puts the reconciliation between the platform's count and the warehouse's count next to the reported number.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **Schrödinger's Order** — when the operational systems disagree about an order, every report built on them disagrees too, and the dashboard becomes a fourth opinion.
- **The Message in a Bottle** — an order edit or refund that one connector missed is a permanent difference between the warehouse and the platform until a reconciliation finds it.
- **The Familiar Stranger** — "customers" cannot be counted while four systems each count their own, so the customer metrics inherit the identity problem of the previous chapter.

Every chapter so far has been about keeping a promise to one customer: this jacket, this price, this delivery, this refund. This chapter is about a different kind of promise, the one the company makes to itself when it says "we took 21,000 orders last month". **A business runs on a few numbers: orders, revenue, margin, returns, stock. If those numbers are not the same in every room, the rooms make different decisions.** Merchandising buys for a season from the sell-through numbers; finance closes the month from the invoiced numbers; operations plans the warehouse from the shipped numbers; the board judges all three from a dashboard.

The numbers come from the same systems the rest of the book describes, and those systems were built to run the business, not to count it. The platform knows orders placed; the ERP knows orders invoiced; the warehouse knows parcels shipped. Each is right about its own job. **The failure is not that they differ; it is that nobody has said which difference is the definition and which is an error.** This chapter is about building the place where the counts can be compared, and the discipline that turns three honest numbers into one trusted one.

For Kestrel & Pine, the fictional outdoor brand this book follows, that discipline is Waypoint decision W7: one definition of an order, of revenue and of a return, owned by the Data Lead. It is the cheapest of the eight Waypoint decisions and, in my experience of merchants at this stage, the one that most changes how the leadership meeting goes.

Where we are in the order's journey. Eva's jacket is one order in the platform (#1072455, €235, 13 April 2027), a set of invoice lines in the ERP, one parcel shipped from Duiven on 14 April, and, after her exchange, a second parcel on 22 April and a credit note against the first. Whether that is one order, two shipments or one order and

one return depends on who is counting. This chapter sits at the end of the journey, where every step is counted, and looks back along it.

Setting the Scene: Three Answers to One Question

On the first Monday of March 2027, Mara Lindqvist, the CEO, asked her leadership meeting a question she expected to be simple: how many orders did we take in February? Milan Petrov had the platform's number: **21,050**. Sanne de Wit had the ERP's number: **22,710**. Daniel Okafor had the warehouse report the BI (business intelligence) tool builds from the WMS's (warehouse management system's) data: **24,140**. All figures in this scene are illustrative and fictional. Nobody had made a mistake, and the next forty minutes of the meeting went to finding out why.

Hanna Berg, the Data Lead, had the explanation by Wednesday, and it is worth reading slowly because every merchant has a version of it.

What each count includes	Platform: 21,050	ERP: 22,710	Warehouse report: 24,140
What is counted	Orders created in the platform in February	Sales orders invoiced in February	Parcels and shipments leaving Duiven in February
Date used	Order creation, Amsterdam time	Invoice date	Ship date
D2C (direct-to-consumer: the brand's own storefronts) orders	18,900 created, including 290 later canceled and 40 test or draft orders	18,420 invoiced (canceled and test orders excluded; 150 fewer from month-end timing)	19,100 parcels, including 640 second parcels of orders split across two boxes
Marketplace orders	Not here; they live in the marketplace connector	3,310, from the connector's weekly summary	2,520 shipped from Duiven; the 790 fulfilled by the marketplace's program never pass through the warehouse

What each count includes	Platform: 21,050	ERP: 22,710	Warehouse report: 24,140
Wholesale orders	610 placed through the B2B portal	980: the 610 portal orders plus 370 keyed in by the wholesale team or received by EDI	980 shipments
Exchange orders from the returns portal	1,540, created through the platform's API	None; an exchange is a credit note and a replacement line	1,540 exchange parcels
Store transactions	Not here	A nightly sales total, not orders	Not here

Three kinds of difference hide in the table. The first is **definition**: is a canceled order an order? Is an exchange a new order? Is a parcel an order? Is a store receipt? The second is **collection**: marketplace orders reach the ERP as a weekly summary and the platform not at all, so each system sees a different population. The third is **time**: an order placed on Sunday 28 February at 23:40 is a February order to the platform, a March invoice to the ERP and a March parcel to the warehouse.

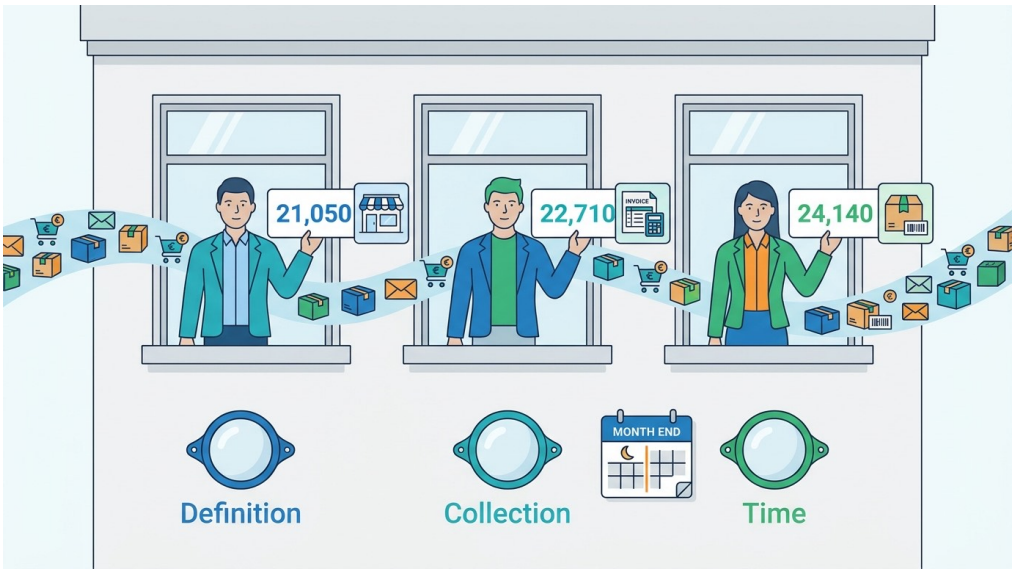


Figure 1: *The platform's 21,050, the ERP's 22,710 and the warehouse report's 24,140 are honest counts that differ by definition, collection and time; none is the company's number*

until someone owns the definition.

Mara's reaction was the right one. She did not ask which number was correct. She asked who owned the definition, and the answer, in March 2027, was nobody.

Operational Data and Analytical Data

The first idea a newcomer needs is that the same facts are used in two different ways, and that the two ways want different stores.

Operational data is the live state of the business: this order's status, this SKU's stock at Duiven, this customer's address. Operational systems (the platform, the OMS, the WMS, the ERP, the POS) read and write it one record at a time, thousands of times a minute, and must answer fast. **Analytical data** is the history of the business laid out for questions across all records: orders by channel by week, return rate by product by size, stock cover by category. Analytical questions read millions of rows at once and can tolerate hours of lag.

The two must be kept apart for a practical reason. A report that scans every order of the year, run against the platform's own database on a Friday evening in November, competes with the checkout for the same machine. The rule is simple: **never run an analytical question against an operational store, and never trust an analytical store for an operational answer.** "The dashboard says it shipped" is not a shipment (Black Friday Every Year: Peak Load, Resilience and Recovery has the Black Friday version).

The analytical store has had two names. A **data warehouse** is a database built for large analytical queries, into which copies of every system's data are loaded and modeled into tables analysts can use. Snowflake describes its architecture as separating storage from compute so that one team's queries do not slow another's; Google's BigQuery calls itself "a fully managed serverless data warehouse" with the same separation (both from current documentation, accessed November 2026). A **lakehouse** keeps the raw copies in open file formats in cloud storage and runs warehouse-style queries over them; the term comes from a 2021 paper by Databricks' founders (Armbrust, Ghodsi, Xin and Zaharia, CIDR 2021) that argued the two-tier design of a data lake feeding a separate warehouse suffered from staleness, duplication and cost. For a merchant of Kestrel & Pine's size the difference is a vendor choice, not an architecture choice; what matters is that one analytical store exists and that every system lands in it.

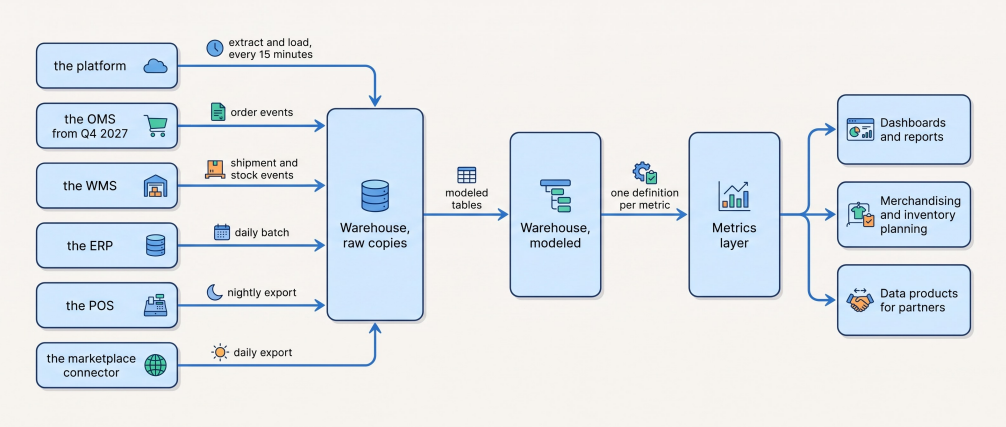


Figure 2: *Every operational system is copied into one analytical store, modeled once, and served through a metrics layer, so that every dashboard, plan and partner feed computes "order" the same way.*

Getting the Data Out

Three mechanisms carry data from the operational systems into the analytical store, and a merchant usually uses all three.

Extract-and-load (ELT: extract, load, then transform inside the warehouse) copies a system's tables on a schedule through a connector. Hosted tools sell this by volume: Fivetran prices by "monthly active rows", the rows inserted, updated or deleted in a month, with a free tier of 500,000 and 15-minute syncs on its Standard plan (pricing page, accessed November 2026; prices change). The connectors do what the previous chapters taught: Airbyte's Shopify source (version 4.3.5, changelog entry of 3 November 2026) reads orders, refunds, products, variants, customers, transactions, fulfillments and inventory levels, syncs incrementally using each record's update time as a cursor, offers a "lookback window" to catch late updates, captures deleted products in a separate stream, and backs off on rate-limit errors. Read that list as a reconciliation sweep (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation) applied to analytics: an order edited after the connector's last run is picked up by the lookback window or not at all, and a missed edit becomes a permanent difference between the warehouse and the platform.

Event streams carry each change as it happens. A system publishes an event (order placed, stock changed), a durable log keeps it, and the warehouse reads the log at its own pace alongside the OMS and the WMS. The best-known public account is from a Walmart engineer on Confluent's blog (May 2020; six years old), describing more than ten event sources normalized into one inventory event feed, with each item and store pair always handled by one consumer so that no two updates to the same stock figure race. APIs, Events, Files and Middleware: How Commerce Systems Talk owns the style;

for analytics the point is that an event stream gives the warehouse the same facts the operational systems act on, at the same moment, with no second extraction.

Platform exports exist to spare the API. Shopify's bulk operations run a query asynchronously and deliver a file; Stripe recommends its Data Pipeline "for a complete export of API data" rather than read-heavy integrations (both from documentation accessed November 2026). A merchant that pulls a year of orders through the same API its checkout uses, one page at a time, is spending the rate budget Promises Between Systems: API Contracts, Webhooks and Versioning described on a report.

The requirements differ by what the data is for; the targets below are illustrative.

Data flow	How fresh	How complete	Who may disagree with it	Typical mechanism
Orders and shipments into the warehouse	Minutes to an hour	Every order, including edits and cancella-tions after the fact	The platform's own count, within a published tolerance	Events from the OMS; ELT with a lookback window
Stock snapshots for planning	Hourly, or a daily snapshot	Every SKU and location, including the marketplace program's stock	The WMS, which is the system of record	Stock events; a nightly snapshot
Finance figures	Daily	To the cent; reconciled with the ERP and the payouts of the PSP (payment service provider)	Nothing; the ERP wins	Daily batch from the ERP; PSP settlement reports

Data flow	How fresh	How complete	Who may disagree with it	Typical mechanism
Operational dashboards (orders per minute, checkout errors, queue depth)	Seconds	Approximate is fine	Nobody; they are rates for action, not records	Direct from the operational systems' monitoring
Board and management reports	Monthly, revised	Defined and reconciled	The ERP for revenue	The metrics layer

The last two rows are the ones that get confused. An orders-per-minute dashboard and a monthly order count are different objects: one is a rate used to act now, the other a record that will be revised and reconciled. Showing both on one screen with one label is how a dashboard becomes the fourth opinion.

One Definition: Metrics and the Metrics Layer

A **metric definition** is the written rule for computing a business number: what is counted, which records are included or excluded, which date decides the period, which currency and tax treatment apply, and what happens when the underlying record changes later. The public definitions of large companies show how many decisions hide in a word like "order".

Source	What it defines	What it says, in short
Shopify's ShopifyQL sales schema, API version 2026-10	`orders`, `gross_sales`, `net_sales`, `total_sales`	An order is counted once "even when the order contains multiple items"; gross sales exclude "taxes, shipping, duties, and fees"; net sales are gross sales minus discounts minus "sales reversals" (refunds, returns, cancellations, edits); "a single order can produce multiple rows if refunds or adjustments follow"
Shopify's FY2025 annual report (February 2026)	GMV (gross merchandise volume)	"the total dollar value of orders facilitated through our platform … net of refunds"
Zalando's 2025 annual report (March 2026)	GMV, average basket	GMV after cancellations and returns, including VAT (value-added tax); the average basket of €62.8 is GMV per order on that basis
Google Analytics 4 e-commerce reference (accessed November 2026)	the `purchase` event	Requires a `transaction_id` to "avoid getting duplicate events for a purchase"; `value` is the sum of price times quantity, excluding shipping and tax; a `refund` is a separate event

Four sources, four units: an order net of reversals as later rows, an order value net of refunds, an order value after cancellations and returns including tax, and a browser event with its own value rule. Each is internally consistent; none is interchangeable with another. This is why analytics vendors who reconcile a platform's sales against web analytics publish lists of the usual gaps (orders created by apps or subscriptions that never fire a browser event, point-of-sale and draft orders, ad blockers and refused consent, refunds netted by one side and not the other, currency and tax inclusion,

attribution windows); those lists are widely reported in vendor help pages rather than in a neutral source, and the direction of each gap depends on the merchant.

The architectural answer to definitional drift is the **metrics layer**: one place where each metric is defined as code, from which every dashboard, report and tool computes it. Benn Stancil's 2021 essay named the gap: "there's no central repository for defining a metric", so metric logic lives in individual dashboards, is re-derived by each analyst, and drifts. The products that followed implement the idea in different ways: dbt's Semantic Layer lets a team define metrics on top of its models so that "If a metric definition changes in dbt, it's refreshed everywhere it's invoked" (documentation updated September 2026); LookML, Looker's modeling language, describes "dimensions, aggregates, calculations, and data relationships" once; Cube's semantic layer adds access control, caching and APIs so that applications and AI tools query governed metrics (all accessed November 2026; product scope changes often). The tool matters less than two properties: **the definition lives outside any one dashboard**, **and it has an owner**.

What a definition should contain, in plain words, is a table like the one Hanna wrote for W7 (the In Practice section below). The hard rows are always the same: what to do with cancellations, returns, exchanges, test orders and split shipments; which date; which currency and tax basis; and how a late change (an order edited in March) is reported against February.

What the Data Is For

The same warehouse serves four audiences with different tolerances, and a design should name them.

Reporting is the monthly and weekly record: orders, net sales, margin, return rate, by channel, country and category. It must be defined, reconciled and revised with a stated policy.

Experimentation is the comparison of two versions of something (a checkout step, a delivery promise, a price) on randomly split traffic. It needs event-level data with a stable customer or session key, and it needs the same metric definitions as reporting, or the winning variant will be judged by a number nobody else recognizes. Finding the Product: Search, Navigation and Merchandising and Delivering the Storefront: Themes, Headless, Rendering and Performance use it; this chapter only insists on the shared definitions.

Data science builds forecasts and models: demand per SKU and week, return probability per product, customer value. It reads the modeled tables and writes back predictions; it is the main reason a merchant keeps raw history rather than only aggregates.

Merchandising and inventory planning is the most commercially important use at Kestrel & Pine's stage. Two numbers drive a season. **Sell-through** is the share of the units bought for a season that have been sold at a point in time (units sold divided by

units received); it tells Teun Baas whether the moss Ridgewalker is a hit or a markdown waiting to happen. **Weeks of cover** is the stock on hand divided by the average weekly sales rate; it tells Daniel Okafor and the buyers when to reorder and when to stop. Both depend on one definition of a sale and one view of stock across Duiven, the six stores and the marketplace program's location, which is why Is It in Stock? Inventory, Availability and the Promise You Can Keep and W1 are prerequisites for trustworthy planning, not just for selling. What Are We Selling? Products, Variants and the Catalog supplies the attributes (category, season, size) the numbers are grouped by.

Data products for partners appear at the Omnichannel stage: sell-through of a third-party brand's products to that brand, listing performance to a marketplace team, a stock feed to a wholesale customer such as Bergzicht. A **data product** is a dataset with an owner, a definition, a contract and a consumer, and the warehouses now make delivery cheap: Snowflake's Secure Data Sharing lets a consumer query a provider's data in place with "no actual data … copied or transferred between accounts", and Databricks documents an open sharing protocol that works "regardless of the computing platforms" the other organization uses (both accessed November 2026; Databricks' product naming for this was changing at the time of writing). Cheap delivery moves the cost to definitions and consent: a sell-through feed to a brand is also a disclosure of the merchant's sales, and a customer-level feed is personal data (Whose Data Is It? Privacy, Consent and Retention).

Ownership follows from the audiences. Zhamak Dehghani's 2019 data mesh article argued that domains should "host and serve their own datasets" and treat them as products, with a shared self-serve platform and global standards so that cross-domain data fits together. The commerce reading is modest: the order domain (the OMS team after W2) owns the order dataset and its definitions; the inventory domain owns stock snapshots; finance owns revenue recognition; the data team owns the platform, the keys, the time zones and the currency rules that make the domains' data joinable. A central data team that owns every definition becomes a bottleneck; domain teams with no shared standards produce four incompatible "orders" again, inside the warehouse this time.

Real-Time Dashboards and When They Mislead

A real-time dashboard shows a number that will be revised. Google Analytics' export documentation says its intraday data is best-effort and may have gaps and that some attribution lags "by up to 24 hours"; Stripe's reconciliation documentation says a payout "might arrive before its reconciliation data is ready"; Shopify's sales schema records a refund as a later row against an earlier order. None of these is a defect. They are what it means to count while the facts are still changing.

The rule I use separates two kinds of number by what they are for. **Operational signals** (orders per minute, checkout error rate, queue depth, the integration lag from order

placed to order in the WMS) are rates used to act in the next ten minutes; show them live, approximately, from the operational systems' monitoring (Black Friday Every Year: Peak Load, Resilience and Recovery). **Business metrics** (revenue, conversion, return rate, margin) are records; show them on a lag, with a definition, a revision policy and the reconciliation beside them. A screen that mixes the two with one label invites the question "why does the live dashboard say 2,300 orders and the report say 2,210", to which the answer is always "because 90 of them were canceled, refunded, edited or duplicated after the dashboard counted them".

Figure 3: *Operational signals are live rates for acting in the next ten minutes; business metrics are records shown on a lag with a definition and a revision policy, which is why a live 2,300 orders becomes a reported 2,210.*

The Choices and Their Trade-offs

Four realistic designs exist for a merchant's analytics, and the right one is mostly a function of stage.

The platform's reports plus web analytics. The commerce platform's own sales reports and a web analytics tool such as GA4. No warehouse, no engineering. Two definitions of "order" from day one, and no way to count marketplaces, wholesale or stores alongside.

A BI tool on operational copies. A business intelligence tool pointed at exports or read replicas of the ERP, the WMS and the platform. Cheap and quick; much of Kestrel & Pine's reporting in early 2027 still worked this way, beside a warehouse with no owned

definitions. Every report embeds its own definition, and a report that is wrong is wrong invisibly.

A warehouse with extract-and-load, a modeling tool and a metrics layer. Every system copied into one store on a schedule, modeled into shared tables, with metrics defined once and reconciled against the source systems. The standard Omnichannel design; it needs one or two data engineers and a named owner per domain.

An event stream feeding a lakehouse and the operational systems. Order and stock events carried once, to the OMS, the WMS, the ERP connector and the warehouse alike, with the lakehouse keeping full history for data science and partner products. The Group-stage design, and Walmart's 2020 account is its large-scale version.

Design	Cost and skills	Definitions	Freshness	Right when
Platform reports plus web analytics	None beyond subscriptions	Two, both the vendors'	Live, unreconciled	One channel; Starting stage
BI on operational copies	A BI tool; an analyst	One per report; drift is the norm	Hours; risks loading operational systems	A stopgap; never for a multi-channel merchant
Warehouse, ELT, metrics layer	ELT fees by volume; one or two data engineers; domain owners	One per metric, owned, reconciled	15 minutes to a day by flow	Two or more channels; Omnichannel stage; Kestrel & Pine from 2027
Event stream and lakehouse	Streaming platform; a data platform team	As above, plus schemas on events	Seconds for events; history for models	Several brands or units; data science and partner products at scale

My recommendation for a merchant at Kestrel & Pine's stage is the third design, with one addition the vendors do not sell: a **reconciliation published with the number**. Each monthly report shows the platform's count, the OMS's count and the warehouse's count and the bridge between them. A number with its bridge is believed; a number

alone is argued with.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting	The platform's reports and web analytics; the founder knows the numbers by heart	Knowing that the two tools define "order" differently and which one the accountant uses	Marketing's conversion rate and the platform's order count stop agreeing and nobody can say why
2 — Growing	A BI tool on exports; spreadsheets for the season; a first ERP report	A warehouse fed by an ELT connector, three or four agreed metric definitions, and a monthly check of warehouse orders against platform orders	Three order counts in the leadership meeting; a buying decision made on a report nobody can reproduce
3 — Omnichannel (Kestrel & Pine)	Many systems, several channels, a warehouse that exists but has no owned definitions	An event stream or OMS feed into the warehouse, a metrics layer, domain ownership of definitions, data products for key partners, operational dashboards separated from business metrics	Reports revised after the board saw them; partners asking for feeds built by hand; the warehouse used as a system of record for stock

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
4 — Group	Several brands or units, each with its own warehouse or definitions	A shared data platform with federated ownership, warehouse-native sharing with partners, a governance function that also serves privacy	Two brands reporting "net sales" on different bases to one board; group-level forecasts that cannot be reconciled to any unit's books

The subject changes shape with scale more than most. At Starting there is nothing to build and one thing to know. At Growing, the warehouse is worth building before the second channel, not after, because the second channel is what creates the second count. At Omnichannel the problem is ownership, not technology: Kestrel & Pine had a warehouse for three years before it had a definition. At Group the problem becomes federation across units that each solved it their own way.

In Practice: Waypoint W7 at Kestrel & Pine

Under W7, Hanna Berg published Kestrel & Pine's metric definitions in June 2027 and the warehouse's monthly report adopted them from the July close. The definitions and the bridge below are fictional and illustrative.

Metric	Definition	Decisions it settles
Order	A confirmed customer purchase with one order number, in any sales channel: D2C storefronts, the B2B portal, manual and EDI wholesale, and marketplaces including orders fulfilled by the marketplace's program. Counted at the moment of confirmation in Amsterdam time. Test and draft orders are excluded. Canceled orders are counted as orders and reported with a cancellation status; "orders net of cancellations" is a second, named metric. An exchange order created by the returns portal is a return outcome, not an order. An order split into two parcels is one order.	Exchanges, cancellations, test orders, split shipments, the marketplace program, the period
Store transaction	A POS receipt. Reported beside orders, never added to them, until the POS joins the OMS (evaluated in 2028 under W4).	Whether a €51 receipt is an order

Metric	Definition	Decisions it settles
Net sales	Merchandise value excluding VAT, after discounts, minus the value of returns and cancellations in the month they are refunded; shipping charges and payment fees excluded. Recognized when control passes to the customer under IFRS 15 (the international accounting standard for revenue): at delivery for D2C and marketplace orders, and at delivery to the customer's distribution center for wholesale; "shipped" is the operational milestone, not the accounting one. This is the ERP's basis, so the warehouse's net sales reconcile to the ERP to the cent each month.	Tax, shipping, returns timing, recognition
Return	A returned item received and accepted at Duiven. Item return rate is returned items over shipped items (24% in 2026); order return rate is orders with at least one return over orders (19%). Exchanges are returns with an exchange outcome (28% of returns).	What a return is, and which rate is meant

Metric	Definition	Decisions it settles
Customer	Deferred to the identity work of One Customer, One Product: Identity and Master Data Across Systems; until then, customer counts are reported per channel and labeled as such.	Whether Eva is one customer

With the definitions, the February numbers became one number and a bridge. Hanna's report for February 2027, recomputed in July, reads: **orders 23,150** (18,860 D2C after excluding the 40 test orders and keeping the 290 canceled, 3,310 marketplace, 980 wholesale), **22,860 net of cancellations**, and 9,200 store transactions reported beside them.

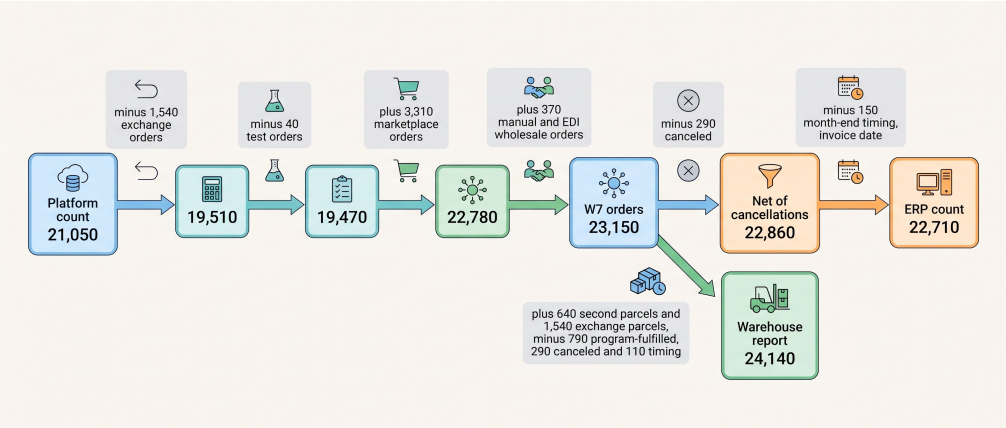


Figure 4: *The bridge from the platform's February count to the W7 definition, and from there to the ERP's and the warehouse's counts: each count is right about what it counts, and the bridge is what makes the company's number believable.*

The bridge, not the number, is what changed the meeting. From the July close, the monthly report shows the W7 order count, the platform's own count, the ERP's invoiced count and the warehouse's shipment count on one page, with the bridge between them; a difference that the bridge does not explain is an exception for Hanna's team, which is how a missed order edit or a connector gap is found. The first such exception, in August 2027, was 23 marketplace orders the connector's weekly summary had dropped during a vendor outage *(illustrative)*.

Two things made W7 cheap. The definitions took Hanna three weeks of conversations, not three months of engineering; the warehouse and the BI tool already existed. And

the OMS, from Q4 2027, makes the order count's collection problem smaller, because every channel's order is created in one place with one number family (W2); W7 is still needed, because one record does not settle whether an exchange is an order.

When It Goes Wrong

Failure	How you notice	What to do
A late edit or cancellation is missed by the extract (the Message in a Bottle, in the warehouse)	The bridge shows an unexplained difference; a refund in the PSP has no match	A lookback window on incremental loads; a monthly full compare of order ids and amounts against the source
Two clocks (store time versus UTC; order date versus invoice or settlement date)	Month-end orders move between months; the ERP and the warehouse disagree by a day's volume	One period rule per metric, written in the definition; store the source timestamp and the time zone
Refunds netted on one side only	Net sales in the warehouse exceed the ERP's by the month's refunds	Define the refund month in the metric; reconcile net sales to the ERP every close
A report built straight on an operational database	The platform or ERP slows during reporting; worst on peak days	Route every analytical query to the warehouse; revoke reporting access to operational stores
Browser analytics as the order count	Marketing's count is lower by the ad-blocked and consent-refused share, and misses app-created orders	Treat web analytics as a behavior source, never as the order record; reconcile it against the platform monthly
A metric redefined in one dashboard	Two dashboards with the same title show different figures	Definitions only in the metrics layer; dashboards consume, never compute
The warehouse used as a system of record	Someone adjusts stock or an order "because the dashboard said"	The warehouse is a copy; corrections go to the owning system and flow back

Failure	How you notice	What to do
A partner feed without a contract	A brand or wholesale customer disputes its sell-through figure	A data product with an owner, a definition, a schedule and a version, like any API (Promises Between Systems: API Contracts, Webhooks and Versioning)

What to Take Away

Established.

- Operational and analytical workloads belong in different stores: the warehouse and lakehouse vendors' own architecture descriptions (Snowflake, BigQuery; the 2021 lakehouse paper) rest on separating the two, and a report run against a checkout database is a known peak-day failure.
- Public metric definitions differ and are explicit about it: Shopify's sales schema (2026-10), Shopify's and Zalando's annual-report GMV definitions and GA4's purchase event each count an order on a different basis, which is why a merchant's own definition must be written down.
- Incremental extraction from a SaaS platform is a reconciliation sweep: connectors use update timestamps and lookback windows, and a missed late edit is a permanent discrepancy until a compare finds it (Airbyte's Shopify source documentation, 2026).

Depends on context.

- Whether to feed the warehouse by scheduled extract-and-load or by an event stream depends on whether the operational systems already emit events: once an OMS and an availability service publish order and stock events (W1, W2), the warehouse should read the same stream; before that, ELT with a lookback window is enough.
- Whether definitions are owned centrally or by domains depends on team size: under about five data people, one owner (the Data Lead) with domain reviewers; beyond that, domain ownership with central standards for keys, time and currency.
- A metrics layer product is worth its cost when more than one tool consumes the metrics; a merchant with one BI tool can keep the definitions as versioned models in the modeling layer and add the product later.

The author's synthesis.

- The number that gets believed is the number published with its bridge: show the platform's, the ERP's and the warehouse's counts beside the defined one every month, and treat an unexplained difference as an incident, not a footnote.
- Define before you build. Kestrel & Pine's warehouse was three years old when W7 was decided; the definitions took three weeks and did more for trust than the warehouse had.

Questions to Ask

1. If I ask three people how many orders we took last month, do I get one number, and if not, who owns the definition?
2. Where is each metric defined: in a metrics layer with an owner, or inside the dashboards that show it?
3. What happens to February's numbers when an order is canceled, refunded or edited in March, and does the report say so?
4. Which systems are not in the warehouse at all (marketplaces, stores, the returns portal), and whose count is missing because of it?
5. Does any report run against an operational database, and what happens to it at 20:00 on Black Friday?
6. How does the warehouse learn about orders edited after the extract ran, and when was the last full compare against the platform?
7. Which numbers on the live dashboard are rates for action, and which are records that will be revised, and are they labeled?
8. Which partners receive data from us, under what definition, on what schedule, and who signed off on what it discloses?

Part VI has been about making separate systems behave like one: how they talk, how they recover, what they promise, who they mean, and what they count. The next part turns from the joins to the platforms themselves, and asks how commerce platforms are built, why their vendors made the choices they made, and what a merchant should copy and what it should not. Continue with Part VII — PLATFORM: How Commerce Platforms Are Built.

Sources and Evidence

Company, product and investor sources

- Key Concepts and Architecture, Snowflake documentation, n.d., accessed 2026-11: separation of storage, compute and services.
- What is BigQuery?, Google Cloud documentation, n.d., accessed 2026-11: serverless warehouse with separated compute and storage.

- Fivetran pricing, Fivetran, n.d., accessed 2026-11: monthly active rows, free tier, sync frequency by plan.
- Shopify source connector, Airbyte documentation, changelog to 2026-11-03: streams, incremental sync by update time, lookback window, deleted-product stream, rate-limit backoff.
- Bulk operations: queries, Shopify developer documentation, n.d., accessed 2026-11: asynchronous bulk export.
- Rate limits and Payout reconciliation report, Stripe documentation, n.d., accessed 2026-11: Data Pipeline recommended for exports; reconciliation data may lag the payout.
- ShopifyQL sales schema, API version 2026-10, Shopify developer documentation, 2026: definitions of orders, gross sales, discounts, sales reversals, net sales and total sales.
- Shopify Inc. Form 10-K for fiscal 2025, SEC EDGAR, filed 2026-02-11: the GMV definition.
- Zalando SE Annual Report 2025, Zalando, 2026: GMV after cancellations and returns including VAT; average basket.
- Walmart's real-time inventory management using Kafka, Suman Pattnaik on Confluent's blog, 2020 (evidence age: six years; the author's views, not Walmart's endorsement): normalized inventory events, one consumer per item and store.
- dbt Semantic Layer, dbt Labs documentation, updated 2026-09-29: metrics defined once and refreshed everywhere.
- What is LookML?, Google Cloud documentation, n.d., accessed 2026-11: a modeling language for dimensions and aggregates.
- Cube introduction, Cube documentation, n.d., accessed 2026-11: modeling, access control, caching and APIs.
- Introduction to Secure Data Sharing, Snowflake documentation, n.d., accessed 2026-11: sharing without copying.
- Share data securely across organizations, Databricks documentation, updated 2026-10-27: an open sharing protocol across platforms (product naming in flux).

Standards, regulation and official documentation

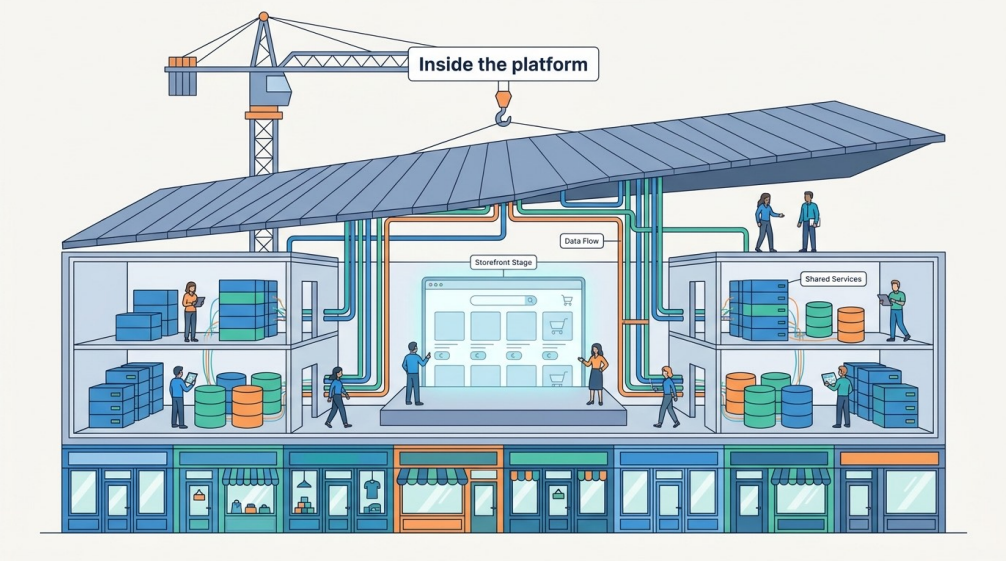
- GA4 recommended events reference and Measure ecommerce, Google Analytics developer documentation, n.d., accessed 2026-11: the purchase event, transaction id, value and refund.
- BigQuery Export schema and limits, Google Analytics Help, n.d., accessed 2026-11: raw export differs from reports; intraday export is best-effort.

Research and practitioner sources

- Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics, Armbrust, Ghodsi, Xin and Zaharia, CIDR 2021: the two-tier problem and the lakehouse proposal.
- The missing piece of the modern data stack, Benn Stancil, 2021: the case for a metrics layer.
- How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh, Zhamak Dehghani, 2019: domain ownership of data as a product.
- Why is GA4 under- or over-reporting?, Elevar help center, n.d., accessed 2026-11 (vendor help page; widely reported causes of the platform-versus-analytics gap).

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Part VII — PLATFORM: How Commerce Platforms Are Built



IN THIS PART, YOU WILL: Open the platform a merchant rents: how it is structured inside, how it is shared among thousands of merchants, what apps may touch, how a page reaches the shopper's phone, and how the chain behind it survives forty times the normal load.

KEY POINTS:

- **The platform's internals are the vendor's; the code around it is yours.** Amazon split at a scaling wall in 2001, Zalando for team autonomy in 2015, and Shopify refused to split; every account describes thousands of engineers. A merchant's own code needs one deployment unit per team that must release independently, and for most merchants that is one.

- **Renting a shared system means governing your share of it.** A hosted platform keeps its tenants apart with identifiers, pods and rate limits; the limit is what the merchant meets, a budget of calls per app per store that every installed app spends from. Every app is an integration with an owner, a scope and at most one writer per fact.
- **The storefront renders facts it does not own, and the peak is where that shows.** Cache the page, fetch the price and the stock live, and go headless only for a need the theme cannot meet. The platform smooths its peak across tenants; the merchant's connectors see the full spike, so measure the chain in orders per minute, queue everything behind the order, and reconcile the morning after.

FAILURE PATTERNS THIS PART ADDRESSES:

- **The Giant's Boots** — a microservices rebuild or a headless storefront copied from a company with thousands of engineers, by a merchant with nine.
- **The Forked Platform** — a core or a theme customized so deeply that every vendor release is a project.
- **The Secret Agent** — an app that writes stock, prices or orders without a named owner, because app review protects the platform, not the merchant's facts.
- **The First Snowfall** — the API budget, the vendor's shared capacity and the hand-built order flow meet real load for the first time on the busiest evening of the year.

Every part so far has treated the platform as a box with a name on it. **This part opens the box.** A merchant paying €2,300 a month for an enterprise subscription rents a slice of a system that one vendor runs, on one version, for thousands or millions of shops at once. That is why the subscription is cheap and why the checkout holds on Black Friday. It is also why the merchant cannot change the platform's structure, cannot see which pod (isolated group of databases) its shop lives on, and gets a budget of API calls rather than a server. What the merchant owns is everything around the box: the integration code, the apps, the theme and its scripts, and the chain that carries each order to the warehouse and the stock back.

Part VII sits under the order's path rather than at one step of it. Its storefront chapter is the first step of Eva Janssen's jacket, the product page she opened at 21:14 on 13 April 2027; its peak chapter asks what happens to that order when two thousand other shoppers place theirs in the same hour. The three chapters between them are the machinery the order runs on: how the platform is structured, how it is shared, and how other people's code changes it.

Kestrel & Pine, the fictional outdoor brand this book follows, met four platform questions in Noor Visser's first months: an outside consultancy's composable rebuild of about €3.2

million, the agency's headless storefront (a separate front-end application fed by the platform's APIs) of about €240,000, a board member's question whether it was "behind" for keeping a monolith (one program, one database), and forty-one installed apps with nobody's name on them. **The claim of this part is that the rebuild proposals would have replaced the part that held on 27 November 2026 and left the joins that failed to "phase two".** The platform served 2,300 orders in the peak hour; the hand-built order flow behind it carried about seven a minute, and the stock copied back around the loop was up to two hours old. With nine engineers, the attention belonged on the merchant's own layer, and the five chapters say how.

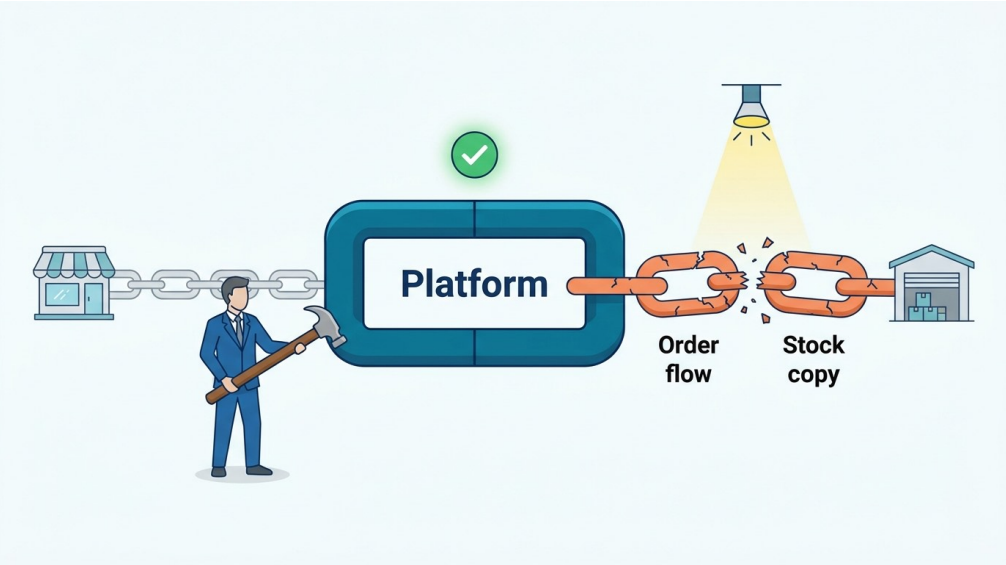


Figure 1: *On 27 November 2026 the rented platform held at 2,300 orders in the peak hour; the merchant's own joins, an order flow of about seven orders a minute and stock up to two hours old, were the weak links the rebuild proposals left for phase two.*

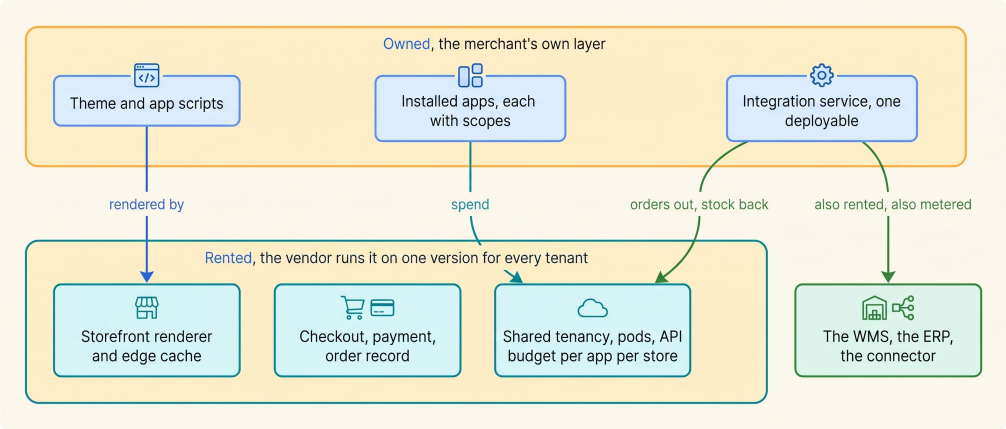


Figure 2: *A merchant on a hosted platform rents the renderer, the checkout and a share of a multi-tenant system, and owns the theme, the apps and the integration code around them; the owned layer and the budget it spends are where the peak day is won or lost.*

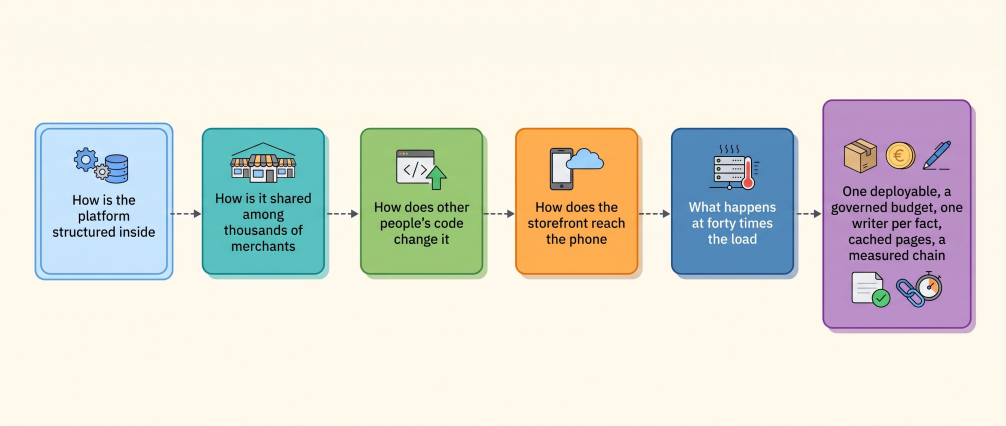


Figure 3: *The five questions of Part VII in reading order, from the inside of the platform outward to the day it is tested; each answer narrows what a merchant should treat as its own.*

Chapters in this part

Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says decides **how a commerce system should be structured inside, and what the famous examples really show**: the four styles differ in how many separately deployed pieces exist and who owns the joins, and the rule is one deployment unit per team that must release without waiting for another. The case is the April 2027 board question "did we choose right?" and Noor's May memo: one storefront codebase and one integration

service with a module per flow, against a rebuild of about a dozen services for nine engineers.

Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation decides **how a hosted platform serves thousands of merchants on shared systems, and why a merchant should care**: pool, silo and bridge (shared, dedicated or mixed arrangements of tenants), pods and regions, and the rate limit, a budget per app per store that every app spends from and the vendor may lower under load. The case is Ruben Alves's June 2027 reading of the Black Friday logs: nine spenders of one budget, the stock-writing apps and the marketplace sync taking about 55%, the order flow 20% and throttled about 1,900 times, against a WMS limit of about 30 calls a minute.

Apps, Plugins and Functions: Extensibility and Ecosystem Governance decides **how platforms let other people's code change them, and how a merchant should govern the apps it installs**: five mechanisms differ by where the code runs and what it may touch, and app review never tells a merchant which two of its apps write the same fact. The case is the April 2027 spreadsheet, 41 apps, 6 unused, 3 writing stock, and the governance rule (an owner per app, scopes on a card, one writer per fact, a quarterly uninstall review) that left 34 apps by July.

Delivering the Storefront: Themes, Headless, Rendering and Performance decides **how the storefront reaches the shopper's phone, and when headless is worth it**: a storefront renders facts other systems own, the checkout stays on the platform under every rendering model, and headless is justified by a shared front end, a content-led site or a B2B portal, never by speed alone. The case is the agency's February 2027 headless proposal, declined after measurement: the page's main content loaded in 2.1 seconds and its layout barely shifted, both already good by Google's thresholds, while its response to a tap (310 milliseconds) was slowed by scripts from eleven apps, down to 180 by June after a €38,000 theme pass.

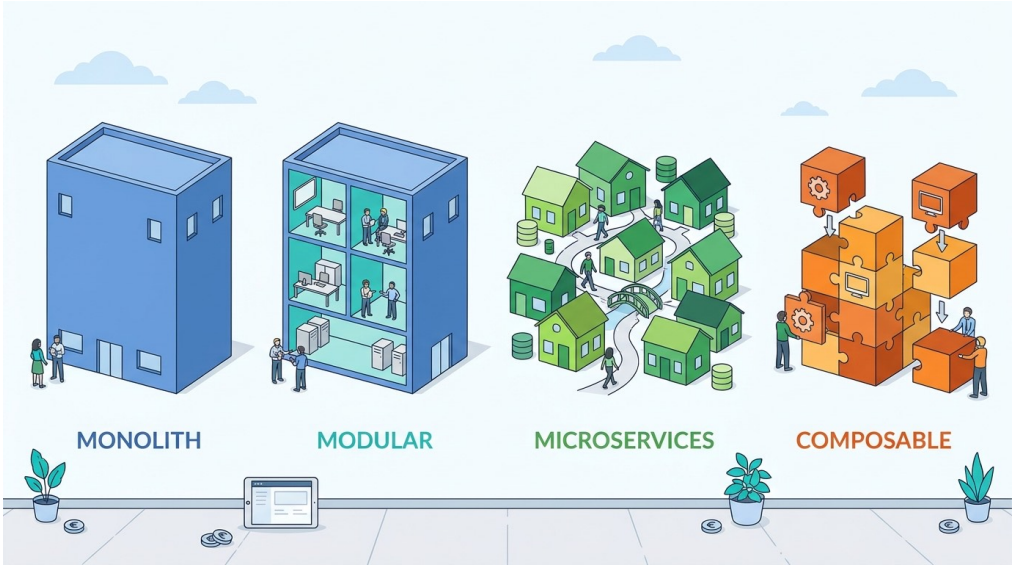
Black Friday Every Year: Peak Load, Resilience and Recovery decides **how to prepare a platform and its integrations for a day with forty times the normal traffic**: measure every link in orders per minute, load-test the connectors with marked orders, queue everything behind the order, write a switch-off ladder on which checkout, payment and the order record never appear, and name who reconciles what the morning after. The case is the post-mortem of 27 November 2026 (31,000 orders in the week, the order flow four hours behind, 1,340 units oversold) and the 2027 plan: 60 orders a minute forecast, every link tested at 80, eight rehearsed switches, a freeze from 15 November, a situation room and a reconciliation report at 08:00 on Saturday.

Begin with Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says.

Sources and Evidence

Every public claim in this part is sourced and dated in the chapter that makes it, under that chapter's Sources and Evidence. Kestrel & Pine is fictional; recommendations and labeled assumptions are the author's judgement.

29. Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says



IN THIS CHAPTER, YOU WILL: Learn the four ways a commerce system can be structured inside, what Amazon, Zalando and Shopify documented, and how to size modularity to your team.

KEY POINTS:

- **A style is a choice about deployment units and boundaries, not about quality.** Monolith, modular monolith, microservices and composable differ in how many separately deployed pieces exist, who owns which data, and how the boundaries between them are enforced. The same commerce logic can live in any of them; what changes is where the cost lands.
- **The famous examples document reasons and costs, not a recipe.** Amazon split at a scaling wall in 2001; Zalando split for team autonomy in 2015 and spent a decade re-standardizing; Shopify refused to split

and spent seven years learning that boundaries drawn on an org chart do not match how code runs. Every account is dated and describes a company with thousands of engineers.
- **For a merchant, the style is mostly the vendor's; what is yours is the integration layer.** Size modularity to the number of teams that must release without blocking each other, decide who owns each fact before splitting anything, and treat a microservices or all-custom composable rebuild by nine engineers as The Giant's Boots.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Giant's Boots** — a microservices or all-custom composable rebuild copied from a company with thousands of engineers, without its reasons.
- **The Spaghetti Junction** — composable moves the distribution costs the big companies paid into the merchant's own integration layer, where every join needs an owner.
- **The Burned Bridge** — a rebuild that replaces the platform on one date, when every documented migration took years and was done piece by piece.

Every few years a new word for "how software should be built inside" arrives in commerce. Around 2015 it was **microservices**; in 2020, **composable** and the acronym MACH (originally Microservices, API-first, Cloud-native, Headless); in 2026 the same vendors sell "agentic". Each word came with case studies from very large companies, and the case studies are real: Amazon, Zalando and Shopify have described their internal architecture in their own posts, with dates. The question is not whether those companies did what they say. The question is **what their accounts show, for whom, and at what size.**

This chapter reads the evidence as evidence: what each company wrote, when, how old that is, and what the account does not say. Then it turns to the question a merchant actually faces: how much of this is yours to decide at all? On a hosted platform the answer is "less than the pitch suggests". The platform's internal style is the vendor's problem. What is yours is the code around it (connectors, custom apps, data pipelines), and that code should be split into **exactly as many pieces as you have teams that must release independently.**

Two terms carry the chapter. An **architecture style** is the shape of a system as deployed and run: how many separately running programs there are, which data each owns, and how they talk. A **deployment unit** (or deployable) is one program that is built, tested, released and run as a whole; splitting a system means creating more of them.

Where we are in the order's journey. This chapter sits off the order's path and bears on every step of it. When Eva Janssen orders her Ridgewalker jacket at 21:14 on 13

April 2027, her order crosses some number of separately running programs inside the platform (the vendor's decision, invisible to her) and inside Kestrel & Pine's own code (Noor Visser's decision). Every crossing is a network call that can be slow, a copy that can be stale, or a failure that can be partial. The style decides how many crossings there are.

"Are We Behind?" at Kestrel & Pine

Kestrel & Pine, the fictional outdoor brand this book follows, had already made its platform decision when the question arrived. Two pitches had arrived in the second week of February 2027 (Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026): an outside consultancy's **composable rebuild** (an API-only commerce engine, a custom storefront, a custom order service and "a cloud-native microservices foundation", about **€3.2 million over eighteen months**) and the agency's headless rebuild of the storefronts on the current platform (Delivering the Storefront: Themes, Headless, Rendering and Performance). The platform review that followed (Build, Buy or Assemble: Choosing and Changing Platforms) costed the composable rebuild beside an enterprise-suite option and **stay and surround**, which became Waypoint decision W4. Waypoint launched in March with **€1.1 million over 2027–2028**.

In **April 2027**, the month the board asked for a one-page stage map (What Changes at Each Scale: From First Store to Group), a board member from Veluwe Capital forwarded Mara Lindqvist, the CEO, an article about a fashion retailer with about three thousand people in technology that had "left the monolith behind", and asked whether Kestrel & Pine was behind. Mara sent it to Noor Visser, Head of Engineering since January, with one line: "Did we choose right?"

Noor had fourteen people in technology, nine of them engineers. On Black Friday 2026 the platform itself had held at 2,300 orders in the peak hour; what failed was the chain of connectors between the platform, the ERP (enterprise resource planning system: finance, purchasing, invoicing), the WMS (warehouse management system) and the marketplace connector, which let the Ridgewalker jacket sell for two hours after the warehouse ran out. The rebuild option would have rebuilt the part that worked while leaving the joins that failed to "phase two". Noor's memo to the board is the rest of this chapter.

Figure 1: *The famous accounts describe companies with thousands of engineers; a microservices rebuild copied from them by nine engineers is The Giant's Boots, paying the costs without the reasons.*

Four Styles in Plain Words

Three ideas separate the styles: the **deployment unit**, defined above; **data ownership**, which program may read and write a given table or record directly; and the **boundary**, the rule that one part of the system may only reach another through a defined interface, and how that rule is enforced (by a tool, by a review, or not at all).

Style	What is deployed	Who owns the data	How boundaries are enforced	Documented example
Monolith	One program holding every domain: catalog, cart, orders, payments, admin	One shared database; any code can read any table	Usually not enforced; conventions and reviews	Zalando's Java shop "Jimmy" 2010–2015; Shopify's Rails core before 2017

Style	What is deployed	Who owns the data	How boundaries are enforced	Documented example
Modular monolith	Still one program, divided into named modules with a public interface each	One database, but each module is meant to own its tables	A static check in the build that blocks new cross-module references	Shopify 2019–2024 (Packwerk); Medusa's module layout (n.d., accessed 2026-11)
Microservices	Many small programs, each built, released and scaled on its own	Each service owns its own database; nobody else reads it	The network: a service can only be reached through its API	Amazon from about 2001 (described 2006); Netflix 2008–2016; Zalando from 2015
Composable (MACH)	Separately *bought* API-first SaaS services plus a storefront and an integration layer the merchant owns	Each vendor owns its data; the merchant owns the copies between them	Vendor APIs and contracts	commercetools and the MACH Alliance vendors (2020–2026)

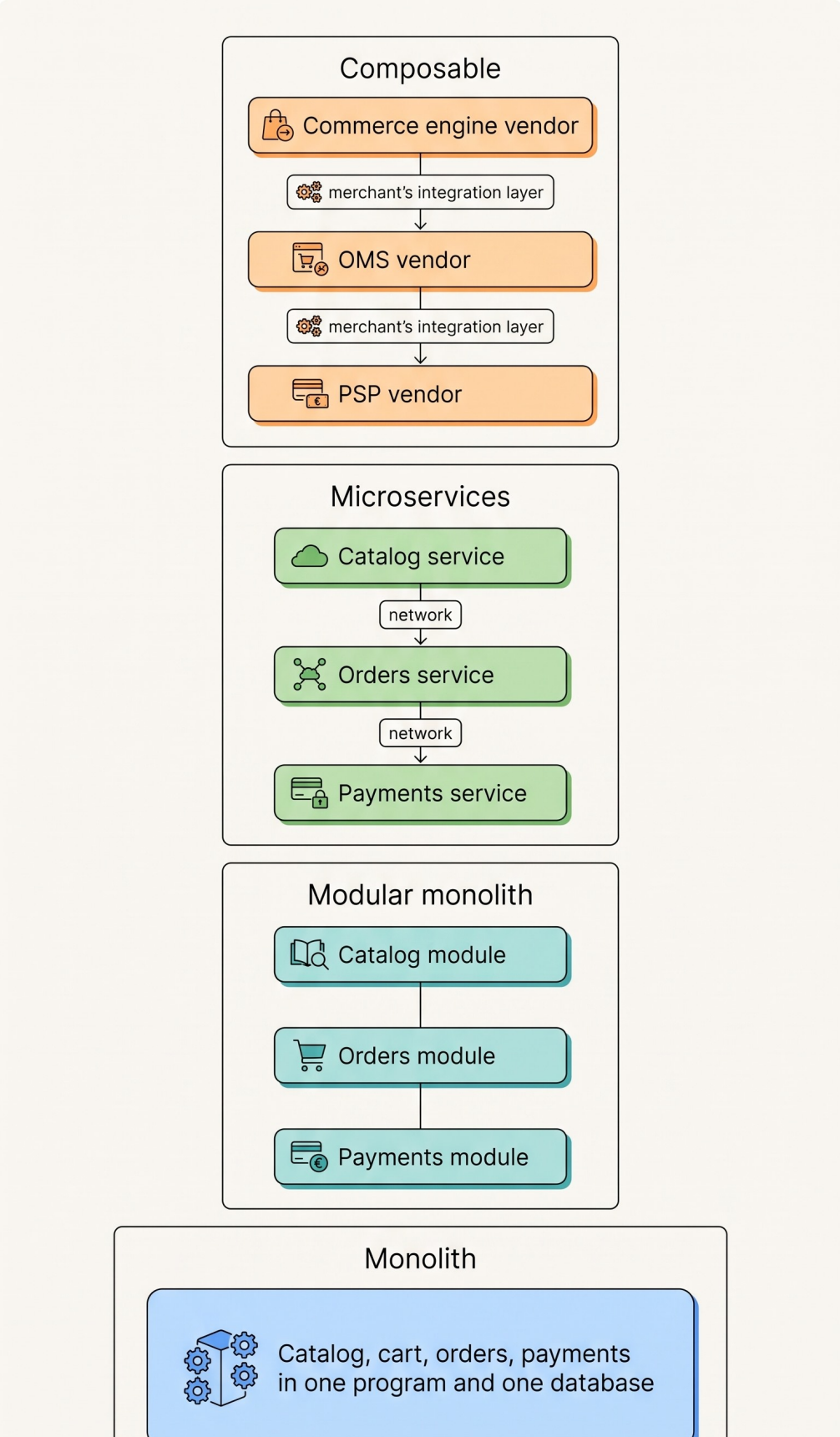

Composable
Commerce engine vendor
merchant's integration layer
OMS vendor
merchant's integration layer
PSP vendor
Microservices
Catalog service
network
Orders service
network
Payments service
Modular monolith
Catalog module
Orders module
Payments module
Monolith
Catalog, cart, orders, payments
in one program and one database

Figure 2: *The four styles differ in how many separately deployed pieces exist and who owns the joins between them; in a modular monolith the joins are function calls inside one program, in microservices they are network calls between a company's own services, and in composable they are integrations between vendors that the merchant must own.*

Two clarifications a newcomer needs. First, "**monolith" is not an insult**. It is one program with one database, which is also the shape in which every transaction is cheapest and every fact is consistent. Second, **composable is a different kind of thing from the other three**. They describe how one organization structures code it writes. Composable describes how a merchant *buys*: several API-first products (an engine, a search service, a PIM, an OMS, a payment provider) assembled behind a storefront the merchant builds or buys. The merchant does not write the services; it writes the joins, and the distribution costs the big companies document for microservices transfer to that integration layer almost unchanged.

Two terms from earlier chapters recur. A **system of record** (Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record) is the one system allowed to be right about a fact: the WMS for physical stock, the OMS (order management system) for the order. **Eventual consistency** (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation) means a copy of a fact in another system may be out of date for a while, and will catch up. Every style with more than one deployment unit lives with it, and with **idempotent** handling: processing the same message twice must have the same effect as processing it once.

What the Famous Examples Actually Show

The accounts below are listed with their dates because the architecture they describe has certainly changed since. Treat each as evidence that a pattern existed and worked at that date in that company, not as a description of today's systems.

Amazon: a scaling wall, then hundreds of services (2001–2007)

The primary account is a 2006 conversation between Jim Gray and Werner Vogels, Amazon's chief technology officer, in *ACM Queue*. As digested by High Scalability in 2007, Amazon's front end was a "two-tier monolith" and "in 2001 it became clear that the front-end application couldn't scale anymore"; Amazon moved to "a fully-distributed, decentralized, services platform" that grew into hundreds of services, with a single page request touching between 100 and 150 of them. The same source carries the two cultural rules the industry still quotes: "you build it, you run it" (the team that writes a service also operates it, confirmed in AWS's own 2006 relay of the interview) and the "two-pizza team", a team small enough to be fed by two pizzas. The 2007 Dynamo paper gives the scale: the shopping cart service served "well over 3 million checkouts in a single day".

What this shows: a company that split because one program could no longer serve its traffic, and organized the split around small teams owning services end to end. What it does not show: a timeline a smaller company could copy, or anything about Amazon's retail architecture today. The evidence is two decades old.

Zalando: a split for autonomy, then a decade of discipline (2015–2025)

Zalando's record is unusually complete and unusually honest. Henning Jacobs's 2020 retrospective of a decade in Zalando's technology gives the sequence: a PHP shop on the Magento stack, rewritten in Java in 2010 because the stack hit scaling limits; in-house order and payment services in 2012; and in March 2015, under the banner "Radical Agility", a move to Amazon Web Services (AWS), Amazon's cloud, with containerized microservices owned by autonomous teams, each with its own AWS account. Zalando's 2015 post on the shop rebuild gives the motive: the monolith, "Jimmy", meant that "all of the Shop teams shared the same code base" and teams "couldn't move quickly because of Jimmy's slow startup time and our overly complicated deployment processes". The trigger was not traffic. It was too many teams in one program.

By 2020 Zalando had, per Jacobs, more than 260 engineering teams, more than 1,300 developers and more than 4,000 applications. The same post lists the costs: team autonomy over technology produced "similar CRUD REST services written in six languages" (simple create-read-update-delete web services), which "hurts productivity at scale"; a standard stack would have been "a major win"; and in 2020 Zalando "still dealt with monoliths from 2010", with some functionality implemented twice. The 2015 engineering principles repository is now marked deprecated, noting that they "no longer reflect the current reality (2020) in all aspects".

The discipline that followed is documented step by step. In July 2020 Zalando narrowed its technology radar, dropping Clojure, Haskell and Rust for new applications because "a broad freedom of choice leads to increased difficulties in supporting software long-term". In 2023 one developer portal replaced "more than 100 disconnected interfaces". In March 2025 it reported that its event-driven pipeline for price and stock updates "became a bottleneck": more than 90% of each payload was unchanged, during Cyber Week stock and price events "could be delayed by up to 30 minutes", and teams that consumed the stream adapted product data for their own needs, producing what the post calls "competing sources of truth"; it was replaced by a synchronous read API. Zalando's 2025 annual report (published March 2026) puts about 3,000 employees in its technology departments.

What this shows: microservices gave Zalando team autonomy at hundreds of teams, and the bill came as technology sprawl, tooling sprawl and, in one critical path, a boundary that had to be redrawn. What it does not show: that microservices "failed". Each later fix narrowed a choice; none reversed the architecture.

Shopify: the monolith that refused to split, and what it learned (2019–2024)

Shopify is the counter-example, and its own account runs from 2019 to 2024. In February 2019 it described its Rails codebase as one of the largest in existence, worked on for over a decade by more than a thousand developers, and said that around 2016 "the downsides of the monolith were outweighing the benefits". It rejected microservices explicitly: they would add test and deployment pipelines and infrastructure per service, network calls that add latency and reduce reliability, and coordinated deploys for large refactors; the team "wanted more modularity without more deployment units". Instead it regrouped code into business components (orders, shipping, inventory, billing), each meant to have a public interface and "exclusive ownership of its associated data", and built a tool, Wedge, to score each component's isolation. The payoff it claimed: the isolation work let it replace a legacy tax engine.

By September 2020 the core had "over 2.8 million lines of Ruby code", 37 components and an Architecture Guild of about 400 members, and Shopify said it split out services "only for good reasons", naming storefront rendering and credit-card vaulting. The same post is candid: "every component depended on over half of all the other components", with "lots of circular dependencies". Packwerk, open-sourced that month, is a static check that records existing boundary violations in a list so that new ones are blocked without fixing everything at once. Shopify's FY2025 annual report (filed February 2026) confirms the broad shape today: a multi-tenant platform, Ruby on Rails, "single branch" software on which all merchants run the latest version, and about 7,600 employees.

The February 2024 Packwerk retrospective is the latest first-party account and the most useful. Privacy checks "were removed from Packwerk with the release of version 3.0" because they pushed teams into API design instead of dependency management. Several common ways of loading code are "untracked and invisible to the tool", so "a package can show zero violations and still fail at runtime". Isolating one base package with zero violations and zero dependencies "took many months of hard work". The rate at which the tool found problems "vastly outpaced" teams' capacity to fix them; "we have discussed removing Packwerk from our monolith", and it stayed because it "has provided value in holding the line against new dependencies at the base layer". Most important: "domains and the boundaries between them do not reflect the way Shopify's code actually functions in practice", so components became organizational groupings rather than enforced units. Two quoted lessons: "Packwerk is a sharp knife, and it must be wielded with care" and "running code, more than any metric, will always be the best indicator of real progress".

The one documented case of Shopify carving a hot path out of the monolith is the Storefront Renderer, rewritten from 2019 because storefront code "has much stricter performance requirements" than, say, payment processing; by August 2020 it reported server response times four to six times faster. It was carved for latency, behind a verifier that compared old and new responses on live traffic, not for team autonomy.

What this shows: a company with a thousand developers kept one deployment unit, got real value from ownership and from one cheap gate at the base layer, and found that enforced domain boundaries did not match runtime coupling. The honest reading is not "Shopify modularized its monolith" but "Shopify spent seven years learning which modularity was worth paying for".

The consolidations: Segment (2018) and Prime Video (2023)

Two accounts are often quoted as "microservices failed" and support something narrower. Segment wrote in 2018 that it had grown to over 140 services, each with its own queue, that three full-time engineers spent most of their time keeping them running, and that shared-library versions diverged; it consolidated them into one service and wrote "the solution for us was a monolith", while listing the trade-offs (weaker fault isolation; a shared upgrade can break several destinations). In 2023 Prime Video's video-quality team replaced a serverless, orchestrated pipeline (small functions run on demand by the cloud provider, chained by a workflow service) that "supported only around 5% of the expected load" with a single process, cutting operational cost by 90%; the original post has been taken down, and InfoQ's contemporaneous summary is the surviving source. Both are consolidations of *one subsystem* that had been cut too fine. The transferable claim is "right-size the unit of deployment to the data flow across it", not "reverse the architecture".

Composable and MACH: a style whose definition moved (2020–2026)

The MACH Alliance was announced in June 2020 by commercetools, Contentstack, EPAM and Valtech, with the acronym spelled out as "Microservices, API-first, Cloud-Native, and Headless". It passed 100 members in December 2023. In August 2025 it listed five guiding principles without restating the acronym, and as of November 2026 its principles page states three (Open, Composable, Connected) and no longer defines microservices or headless at all. Its own February 2026 survey of 600 enterprise technology decision-makers reports that 37% cite integration complexity as a primary concern, a finding against interest from a vendor alliance whose respondents are its adopters. In the same period commercetools, the alliance's flagship vendor, cut about 10% of staff in February 2025 after missing growth targets (widely reported), and in January 2026 launched a product it describes as layering agentic capabilities "on top of your existing commerce solution". The composable champions now sell composable incrementally.

What a composable vendor sells as a unit is visible in its documentation. A commercetools Project is "the top-level container" for a merchant's data, isolated from other Projects, with published limits (25 API Extensions per Project as of November 2026). Its API Extensions, synchronous call-outs to the merchant's own code before a cart or order

is saved, carry a plain warning: "if it fails or takes a second longer to return a result, the whole API call fails or takes a second longer." That sentence is the distribution cost of microservices, delivered to the merchant's doorstep. And none of the accounts above describes a company with fourteen people in technology.

What Each Style Costs

Martin Fowler's 2015 "Microservice Trade-Offs" names the benefits as strong module boundaries, independent deployment and technology diversity, and the costs as **distribution** (remote calls are slower and can fail), **eventual consistency** ("because of their laudable insistence on decentralized data management") and **operational complexity** ("there's just no way to handle dozens of services without the automation and collaboration that continuous delivery fosters"). His "MonolithFirst" of the same year adds the point that decides most cases: "even experienced architects working in familiar domains have great difficulty getting boundaries right at the beginning", so "the premium of microservices is a drag you should do without" early on.

In commerce those costs land on specific promises. The table takes three steps of Eva's order and shows what changes when the pieces sit inside one deployment unit versus across several.

Promise in Eva's order	Inside one deployment unit	Across deployment units (microservices or composable)	Who pays
The price at checkout matches the product page	One function call; one transaction; always the same number	A network call to a pricing service, or a copy that can be stale (the Price Drift of October 2026 was a stale copy between systems)	The merchant, in margin and in a reconciliation job

Promise in Eva's order	Inside one deployment unit	Across deployment units (microservices or composable)	Who pays
The last unit is sold once	One row, one lock, one decrement	A reservation message; two channels can both read "one left" before either decrement lands (the Black Friday Oversell)	The customer, in cancellations; the merchant, in goodwill
The order reaches the warehouse	Not applicable; the WMS is always a separate system	A message that can be delayed, duplicated or lost; needs idempotent handling, retries and reconciliation	The operations team, in late orders

The third row is the honest one: some boundaries exist whatever style you choose, because the WMS, the PSP (payment service provider) and the carriers are other companies' systems. The question a style answers is how many *additional* boundaries of that kind you create inside your own code.

The people cost is the one the pitch decks omit. Netflix's own account of its move to cloud microservices runs from August 2008 to January 2016: seven years. Uber reported in 2020 that with about 2,200 critical microservices a root-cause investigation could run through about 50 services owned by 12 teams, a simple feature needed 10 touchpoints, services had a half-life of about 1.5 years, and ownership had blurred into "networked monoliths" that must be deployed together; its remedy grouped services into about 70 domains behind gateways, the modular-monolith discipline rebuilt on top of microservices. Segment's three engineers keeping 140 services alive is the same cost at a company a hundredth of Uber's size.

	Monolith	Modular monolith	Microservices	Composable
Teams it serves well	One, up to roughly ten engineers	A few teams on one release train; Shopify ran it with a thousand developers	Many teams releasing on their own schedules	A merchant with an integration team that can own every join
Consistency of price, stock, order	Transactional	Transactional	Eventual; needs idempotent handling, reliable publishing and reconciliation	Eventual, between vendors; reconciliation is the merchant's
Release independence	None	None by default; ownership per module	Full	Per vendor, on the vendor's schedule
Operating cost	One pipeline, one runtime	Plus one boundary check	A pipeline, monitoring and on-call per service; a platform team	An integration platform, contracts and reconciliation per vendor pair
Typical failure	Nothing changes without touching everything	Boundaries drift; the tool reports more than teams can fix	The distributed monolith; sprawl of languages and tools	The Spaghetti Junction; a vendor's limit becomes the merchant's outage
Right when	One team, one product, early	Several teams, one product, a decade of change ahead	Hundreds of engineers; the organization is the bottleneck	Capabilities differ in pace and the merchant can staff the joins

Modularity Sized to the Team

Melvin Conway wrote in 1968 that organizations "are constrained to produce designs which are copies of the communication structures of these organizations". Every account above is Conway's observation at work: Amazon's two-pizza teams produced services; Zalando's 260 teams produced 4,000 applications; Shopify's one release train produced one deployment unit with internal ownership. Team Topologies, the 2019 book by Matthew Skelton and Manuel Pais, rests on the premise that "teams can only handle so much complexity before breaking down".

The rule I use follows from that. **Create one deployment unit per team that must release without waiting for another team.** Not per domain, flow or capability: per team with its own release need. A four-person integration team gets one deployable with modules inside it; a storefront team gets the theme and its apps. When those two teams block each other's releases, that is the evidence for a second unit, and not before.

Figure 3: *One deployment unit per team that must release without waiting for another: a small team gets one deployable with modules inside, and a second unit only when two teams block each other's releases.*

Three disciplines make a single unit work for years, all in the public record.

Decide who owns each fact before you split anything. Fowler's warning about boundaries and Shopify's finding that domains and their boundaries "do not reflect the way Shopify's code actually functions in practice" are the same warning. In commerce the boundaries that matter are the systems of record: who owns stock, the order, the

product. Kestrel & Pine's Waypoint decisions W1 to W3 are exactly that, made before a line of custom code was moved.

Enforce boundaries cheaply and gate new violations rather than chase zero. Packwerk's design, a list of existing violations and a block on new ones, is the transferable idea, and Shopify's retrospective says where it earned its keep: holding one base layer clean. A merchant's version is a dependency check in the build of its integration code that says, for example, the stock module may not import from the orders module.

Split a hot path only on measured evidence, behind a verifier. Shopify's Storefront Renderer is the model: a measured latency need, a rewrite, a comparison of old and new responses on live traffic, a fallback. If you cannot name the measurement, you do not have the evidence for the split.

The vendor's view and the merchant's view differ, and the pitch deck blurs them.

Question	A platform vendor	A merchant on a hosted platform
Who must be modular?	Thousands of engineers shipping one product to millions of tenants	A dozen engineers shipping connectors, apps and pipelines around a bought core
What is the unit of change?	A component or service inside the platform	A connector, an app, a theme, a data pipeline
Where does distribution cost land?	In the vendor's engineering budget	In the merchant's integration layer, as integration debt (APIs, Events, Files and Middleware: How Commerce Systems Talk)
What does "microservices" buy?	Release independence for many teams	Nothing directly; the merchant does not deploy the platform
What does "composable" buy?	A product category to sell	Replacing one capability (search, OMS, PIM) without replacing the core, at the price of owning the joins

A vendor's internal style is also not something a merchant can verify. A WMS vendor's page saying it is "built on microservices" (Manhattan's does, as of November 2026) says nothing about uptime, release cadence or the API contract, which are the things to ask

about (Promises Between Systems: API Contracts, Webhooks and Versioning).

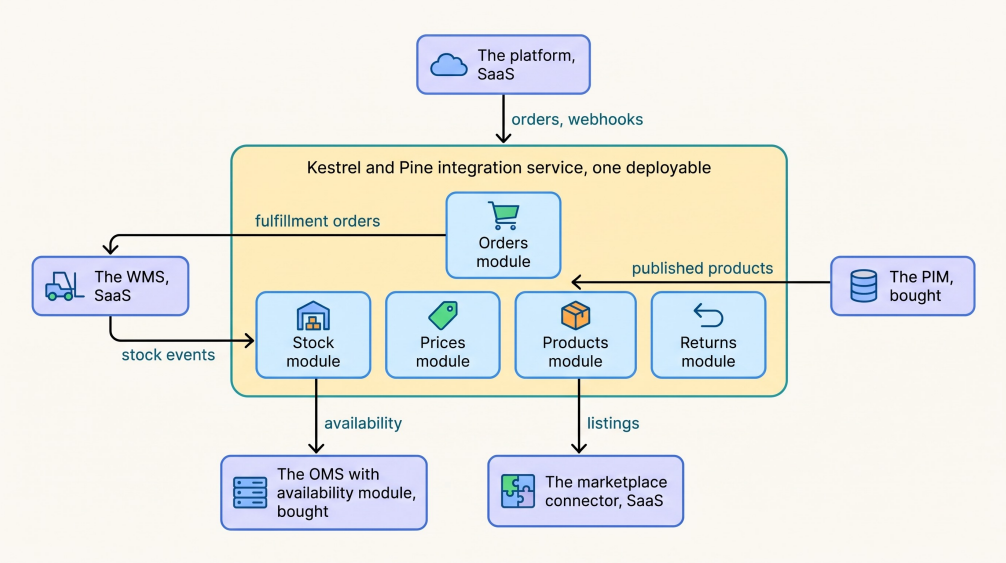


Figure 4: *Kestrel & Pine's own code in 2027 is one deployment unit with a module per flow, surrounded by bought systems; the boundaries that matter are the systems of record on the outside, not service boundaries on the inside.*

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting (under about €2M)	A hosted platform and a handful of apps; no custom code beyond the theme; the style is the vendor's	An inventory of apps with their owners; nothing more	A custom integration with no owner; an agency proposes "headless" before the first store or wholesale account

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
2 — Growing (about €2M–€20M)	A first ERP or accounting connector, a 3PL feed, a marketplace tool; custom code as agency scripts	One repository and one deployable for the merchant's own integration code; boundaries drawn around systems of record	Scripts in three languages on three hosts that nobody can list; a connector fails and its author has left
3 — Omnichannel (about €20M–€200M)	A dozen systems; an OMS and PIM beside the platform; an integration team of two to five; a modular monolith for own code	Modules per flow with a dependency check; OMS and PIM bought rather than built; a second deployable only when two teams block each other	A rebuild proposal sized for a company a hundred times larger; a "service" per flow with one engineer each; releases that need three teams to coordinate
4 — Group (over about €200M, or several brands)	Platform teams; shared storefront, integration and data services across brands; some services the group writes itself	Standard stacks, API guidelines and a paved road, a supported default toolset (Zalando's 2020 and 2023 lessons); a deployable per stream-aligned team (a team owning one flow of business change end to end); a boundary gate in every shared codebase	Six languages for similar services; a service half-life under two years; a feature that needs ten teams; functionality built twice

The judgement: through stage 3 the right number of deployment units for a merchant's own code is one, perhaps two, and the energy goes into boundaries around systems of record. Microservices become a real question only at stage 4, when the organization rather than the traffic is the bottleneck, and even then the documented path is incremental and takes years. Composable read as "replace one capability at a time around a kept core" fits stage 3; read as "an API-only core and a custom storefront" it is a stage 4 choice that needs a platform team.

In Practice: Noor's Memo

Noor's memo to the board (May 2027) ran to two pages. The numbers below are Kestrel & Pine illustrations.

She started with the headcount. Nine engineers, shaped from April 2027 as a "Buy" team of three storefront and apps engineers, a "Fulfil and Return" team of two back-end engineers, an integration pair under Ruben Alves and a data pair (Who Builds and Who Owns: Teams, Vendors, Agencies and Governance). Three of those groups write code, and none has yet waited on another to release. By the rule above, Kestrel & Pine's own code needs one storefront codebase and one integration service, shared by the integration pair and the Fulfil and Return team until the day they block each other. The rebuild option had described about a dozen services for nine engineers.

She then matched the rebuild against the incidents. The Black Friday Oversell, the Three-Status Order, the Price Drift and the Store Stock Mirage were all failures of ownership and of the joins, not of the platform's internal structure; an API-only engine would have recreated every one of those joins in new code. Waypoint decides ownership (W1 to W3), buys the missing hubs and makes the copies trustworthy (W5); none of it depends on the platform's internals.

	The composable rebuild option *(illustrative)*	**Stay and surround as approved**
Cost over 2027–2029	About €3.2M, 18 months	About €1.33M including subscriptions; the €1.1M Waypoint budget
What is rebuilt	Storefront, commerce engine, order service, every integration to them	Nothing; the platform stays (W4)

	The composable rebuild option *(illustrative)*	**Stay and surround as approved**
What is bought	An API-only engine; the rest custom	OMS with availability module, PIM (product information management system); standards on the existing iPaaS (integration platform as a service)
Deployment units the team must run	About a dozen services plus a storefront	One integration service; vendors run the rest
Incidents addressed in 2027	None until "phase two"	Oversell (W1), Three-Status Order (W2), Price Drift and Double Refund (W5 contracts and reconciliation)
Reversibility	One-date cutover of storefront and checkout: The Burned Bridge	Each hub added beside the platform; decision revisited in 2029

The one piece of the rebuild option Noor kept was the vocabulary. She wrote the integration rule into the engineering handbook: one repository, one deployable, a module per flow (stock, orders, prices, products, returns), a dependency check in the build that blocks new cross-module imports, and a measured latency problem as the only accepted reason to split anything. The agency scripts that move in-house during 2027, and every new connector outside the iPaaS, land in that one service. The answer to "are we behind?" was no: Kestrel & Pine stood where every documented company stood before it split, and those companies split for reasons Kestrel & Pine did not have.

When It Goes Wrong

The distributed monolith. Services that must be deployed, tested and rolled back together, with the network added for nothing; Uber named it "networked monoliths" in 2020. Detection: a release that needs three teams in one meeting; a change to one service that always needs a change to another. Recovery: merge the pair, or redraw the boundary around the data that actually flows, as Uber's domains did.

The half-finished migration. Zalando in 2020 "still dealt with monoliths from 2010" and had some functionality implemented twice. Detection: two systems that both claim

to own an order or a price; a legacy program nobody dares to switch off. Recovery: the strangler approach (Moving Without Burning the Bridge: Replatforming and Migration), replacing the old system one capability at a time with a retirement date for each; never the one-date cutover of The Burned Bridge.

The boundary in the wrong place. Zalando's price-and-stock pipeline (described in 2025) reprocessed payloads that were more than 90% unchanged and fell up to 30 minutes behind during Cyber Week, and consumers' adapted copies became competing sources of truth; Prime Video's pipeline paid for every step between stages that belonged in one process. Detection: consumers behind at peak; copies that disagree; a bill that scales with hops, not with orders. Recovery: a synchronous read API beside the events for "what is the price now", and a merged boundary where the data flow is tight.

The over-decomposed integration layer. Segment's 140 services for 140 destinations is the shape a merchant's 34 integrations take when each is its own deployable with its own queue. Detection: engineers who spend their weeks keeping connectors alive; library versions that differ per connector. Recovery: one integration service with a module per flow and recorded traffic replayed as tests, as Segment did.

Composable without a platform team. The MACH Alliance's own 2026 survey puts integration complexity first among its adopters' concerns. Detection: every vendor pair joined by a different script with a different owner; a vendor's call-out timeout failing your checkout. Recovery: an integration inventory (APIs, Events, Files and Middleware: How Commerce Systems Talk) and a contract per join (W5).

What to Take Away

Established.

- The costs of splitting a system into separately deployed pieces are distribution, eventual consistency and operational complexity, and boundaries are hard to get right at the start (Fowler, 2015); every documented commerce case confirms both.
- The documented splits were driven by a traffic wall (Amazon, 2001) or by too many teams in one codebase (Zalando, 2015), and each took years; the documented refusal to split (Shopify, 2019–2024) served a thousand developers on one deployment unit.
- System structure follows team structure (Conway, 1968); teams can only carry so much complexity (Skelton and Pais, 2019).
- Composable is a buying style, not an internal architecture; its own alliance dropped "microservices" from its principles (2026) and its own survey ranks integration complexity as the top concern (2026).

Depends on context.

- One deployment unit for a merchant's own code is right through the Omnichannel stage; a second when two teams measurably block each other's releases; a service per team at Group stage, with a platform team and a standard stack to carry it.
- Composable per capability (replace search, add an OMS, add a PIM) fits a merchant that can staff the joins; an API-only core with a custom storefront fits a group with platform teams and a differentiated front end, and not before.
- Splitting a hot path out of a single unit is right when a measured latency or load requirement demands it, behind a verifier and fallback; it is wrong as a reorganization tool.

The author's synthesis.

- The number of deployment units in a merchant's own code should equal the number of teams that must release independently, and for most merchants below group scale that is one; the modularity that matters is the ownership of facts across bought systems, not service boundaries inside custom code.
- Every famous architecture account should be read with three questions: what size was the company, what was the trigger, and what did they pay; a proposal that cannot answer those three for your company is The Giant's Boots.

Questions to Ask

1. How many teams do we have that must release without waiting for another team? That is our ceiling for deployment units.
2. Which facts (stock, order, price, product, customer) have a named system of record, and which do not? Boundaries come from that list, not from a diagram.
3. For any proposed split: what is the measured problem, in latency, load or blocked releases, that the split solves?
4. For any proposed rebuild: which of our incidents of the last year would it have prevented, and by when?
5. What is the reference company's size in engineers, what triggered its split, and what did it report paying?
6. Where will the distribution costs (retries, duplicates, stale copies, reconciliation) land, and who owns them?
7. If we adopt composable for one capability, who owns the join to the core, and what are the vendor's call-out timeout and failure rule?
8. What is the rollback if the new piece is wrong: a feature switch, a parallel run, or The Burned Bridge?

On a hosted platform the internal structure is the vendor's problem; what a merchant shares with the vendor, every day and especially on its peak day, is the platform itself.

The next chapter asks how one platform serves thousands of merchants at once, and why rate limits, noisy neighbors and pods are a merchant's concern: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation.

Sources and Evidence

Company, product and investor sources

- Deconstructing the Monolith: Designing Software that Maximizes Developer Productivity, Shopify Engineering, 2019 — the componentization decision, the reasons for rejecting microservices, Wedge and the tax-engine payoff.
- Under Deconstruction: The State of Shopify's Monolith, Shopify Engineering, 2020 — 2.8 million lines, 37 components, circular dependencies, services split "only for good reasons".
- Enforcing Modularity in Rails Apps with Packwerk, Shopify Engineering, 2020 — the violation list that blocks new boundary breaks.
- A Packwerk Retrospective, Shopify Engineering, 2024 — what worked and what did not after four years; the quoted lessons.
- How Shopify Reduced Storefront Response Times with a Rewrite, Shopify Engineering, 2020 — the one hot path carved out for latency, with verifier and fallback.
- Cloud, Load, and Modular Code: What 2022 Looks Like for Shopify, Shopify Engineering, 2022 — the CTO's "majestic, modular monolith".
- Shopify Inc. Form 10-K for fiscal 2025, U.S. SEC, 2026 — multi-tenant, Rails, "single branch", employee count.
- One Decade in Zalando Tech, Henning Jacobs, 2020 — the 2010–2020 sequence, team and application counts, "six languages", the costs.
- From Jimmy to Microservices: Rebuilding Zalando's Fashion Store, Zalando Engineering, 2015 — the monolith's name and the motive for the split.
- Zalando Engineering Principles (deprecated repository), Zalando, n.d., accessed 2026-11 — the 2015 principles and the 2020 deprecation note.
- Technology Choices at Zalando: Tech Radar Update, Zalando Engineering, 2020 — the narrowed radar and the quoted reason.
- Sunrise: Zalando's Developer Platform Based on Backstage, Zalando Engineering, 2023 — one portal replacing more than 100 interfaces.
- From Event-Driven to API: Rethinking Price and Stock Updates, Zalando Engineering, 2025 — the pipeline bottleneck, the 30-minute Cyber Week delay, the read API.
- Zalando SE Annual Report 2025, Zalando, 2026 — about 3,000 technology employees.
- Amazon Architecture (digest of "A Conversation with Werner Vogels", ACM Queue 2006), High Scalability, 2007 — the 2001 scaling wall, hundreds of services, 100–150 services per page; the primary interview text was not fetchable.

- ACM Queue Interview with Werner Vogels, AWS News Blog, 2006 — "you build it, you run it".
- Two-Pizza Teams, AWS whitepaper, n.d., accessed 2026-11 — the team-size rule and its ownership rationale.
- Dynamo: Amazon's Highly Available Key-value Store, DeCandia et al., 2007 — the shopping cart's daily checkout volume.
- Completing the Netflix Cloud Migration, Netflix, 2016 — the 2008–2016 timeline and "hundreds of microservices".
- Introducing Domain-Oriented Microservice Architecture, Uber Engineering, 2020 — 2,200 services, "networked monoliths", the domain and gateway remedy.
- Goodbye Microservices: From 100s of Problem Children to 1 Superstar, Segment (Twilio), 2018 — 140 services consolidated, with the stated trade-offs.
- Prime Video Switched from Serverless to EC2 and ECS to Save Costs, InfoQ, 2023 — the surviving summary of the removed Prime Video post.
- EPAM Joins Newly Formed MACH Alliance as Founding Member, EPAM Systems, 2020 — the founding and the acronym's original expansion.
- MACH Alliance Surpasses 100 Members, MACH Alliance, 2023 — the 2023 restatement.
- MACH Alliance Advances End-User Leadership Model, MACH Alliance via Silicon UK, 2025 — five principles without the acronym.
- MACH Principles, MACH Alliance, n.d., accessed 2026-11 — the three current principles.
- MACH Alliance Research: 6x More Organizations Achieve AI ROI with a Composable Foundation, MACH Alliance, 2026 — the 37% integration-complexity figure.
- commercetools General Concepts and API Limits, commercetools, n.d., accessed 2026-11 — the Project as the unit and its limits.
- commercetools API Extensions, commercetools, n.d., accessed 2026-11 — the synchronous call-out and its failure rule.
- Medusa Architecture Overview, Medusa, n.d., accessed 2026-11 — a layered single application with modules.
- Manhattan Active Warehouse Management, Manhattan Associates, n.d., accessed 2026-11 — a vendor describing its product as built on microservices.

Standards, regulation and official documentation

- None; this chapter cites no standard or regulation.

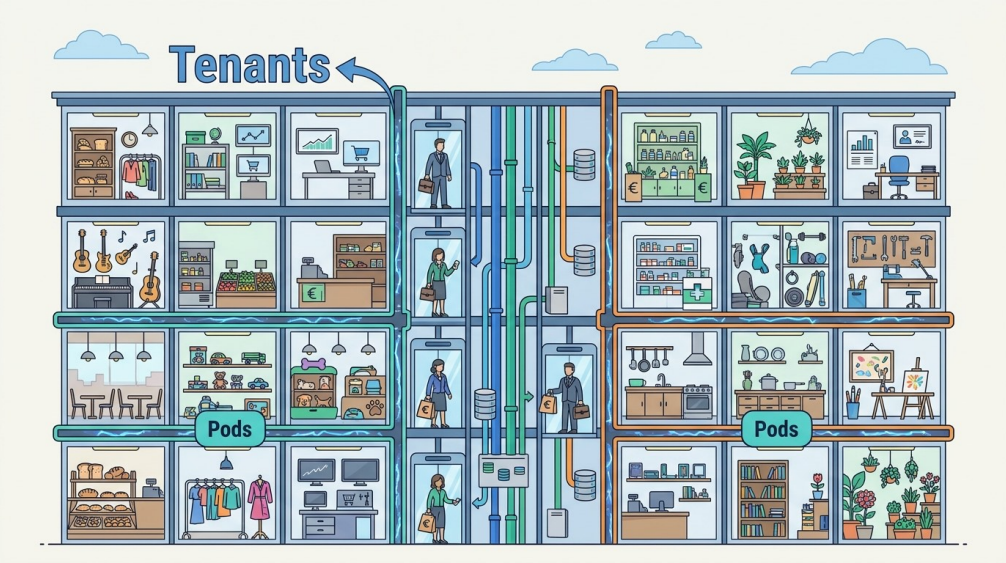
Research and practitioner sources

- How Do Committees Invent?, Melvin Conway, *Datamation*, 1968 — the quoted thesis.

- Microservices, James Lewis and Martin Fowler, 2014 — the definition, decentralized data, complexity moving into the connections.
- MonolithFirst, Martin Fowler, 2015 — boundaries are hard to get right at the start.
- Microservice Trade-Offs, Martin Fowler, 2015 — the three costs quoted in this chapter.
- Team Topologies: Key Concepts, Matthew Skelton and Manuel Pais, n.d., accessed 2026-11 — the team types and the complexity premise.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

30. Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation



IN THIS CHAPTER, YOU WILL: Understand how a SaaS (software-as-a-service) platform serves thousands of merchants on shared systems, and what that sharing means for your rate limits, neighbors and peak day.

KEY POINTS:

- **Tenancy is the vendor's design; its consequences are yours**. A hosted platform pools merchants on shared servers and databases, keeps them apart with a tenant identifier, a routing layer and, at the largest scale, isolated groups of databases; the unit of isolation is invisible to you and the vendor may move you between units.
- **The rate limit is the tenancy mechanism you meet**. A platform protects shared capacity by giving each app a budget of calls per store, tiered by plan. Every connector and app you install spends from that budget, so your noisiest neighbor on a peak day is often your own stock-sync app, and the vendor may lower the budget "to protect platform stability".

- **Every SaaS you rent is a tenancy arrangement, and the peak day is shared.** The WMS, the returns portal, the PIM and the marketplace connector pool you with strangers whose Black Friday is the same Friday; one tenant or several for three countries is a real decision with costs both ways.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The First Snowfall** — the API budget, the checkout throttle and the vendor's shared capacity are met for the first time on the busiest day of the year.
- **The Spaghetti Junction** — every connector and app draws on the same per-store budget, and nobody has added up what they spend.
- **One Size Fits None** — three countries or several brands are forced into one tenant, or split across several, before anyone has measured what the sharing costs.

A merchant on a hosted commerce platform does not own servers, databases or a copy of the software. It rents a slice of a system that serves, in Shopify's case, "millions" of merchants on one codebase (Shopify's FY2025 annual report, filed February 2026). That is what makes a €2,300-a-month enterprise subscription possible: one platform, one version, one operations team, the cost divided among everyone. **The same arrangement means that every merchant's traffic, data and API calls share something with every other merchant's**, and the vendor has to decide how far that sharing goes and how to keep one tenant's bad day from becoming everyone's.

Those decisions have names: **multi-tenancy** (one running system serving many customers, each called a tenant), **isolation** (how far one tenant's load, data and failures are kept from another's), **sharding** (splitting one database into many slices), **pods** (isolated groups of tenants with their own databases) and **rate limits** (a budget of API calls per tenant). Vendors write engineering posts about them. Merchants meet them as an error code on the busiest evening of the year.

This chapter explains how the sharing works, with dated evidence from Shopify, Salesforce, commercetools and Amazon's seller API, and then turns to the merchant's side: what sharing means on a peak day, how to budget the API calls a store is allowed, whether three countries should be one tenant or several, and what to ask every SaaS vendor you rent from, because the WMS and the returns portal are multi-tenant too.

Where we are in the order's journey. When Eva Janssen orders her Ridgewalker jacket at 21:14 on 13 April 2027, her browsing, her cart and her checkout run on servers and databases that Kestrel & Pine shares with thousands of other shops; the checkout has its own throttle, separate from everything else. The moment the order exists, it has to travel to Kestrel & Pine's warehouse system through an API call that spends from Kestrel & Pine's budget on the platform and from its budget at the WMS vendor. On 13 April the budget is ample. On 27 November it was not.

Two Rate Limits on One Friday Evening

Kestrel & Pine, the fictional outdoor brand this book follows, learned about tenancy in the 2027 analysis of Black Friday 2026. The facts are in Black Friday Every Year: Peak Load, Resilience and Recovery: the platform held at 38 orders a minute in the peak hour; the order flow to the WMS (warehouse management system, the software that runs the Duiven warehouse) created orders one at a time, several API calls each, and was throttled by the WMS vendor's rate limit until the backlog reached four hours by 21:00.

In **June 2027**, preparing the peak plan, Ruben Alves (Lead Integration Engineer) pulled the API logs from both ends of that flow. Two findings shaped this chapter; the numbers are Kestrel & Pine illustrations.

On the **WMS side**, the vendor's API allows each customer about 30 calls a minute. The order flow made four calls per order, one order at a time, so it could carry about seven orders a minute on any day, and 38 were arriving. The WMS is a multi-tenant SaaS like the platform; the limit is how its vendor keeps one customer's peak from slowing the warehouses of its other customers, several of which were also having their Black Friday.

On the **platform side**, the API budget is granted per app per store. Kestrel & Pine's one tenant had, that evening, nine consumers of the same budget: the iPaaS order flow (the integration platform that carries orders to the WMS), the ERP connector (the link to the enterprise resource planning system that runs finance), the marketplace connector's hourly stock and order sync, the returns portal, the three apps that could write inventory, a reviews app and an e-mail app. Between 20:00 and 22:00 the three stock-writing apps and the marketplace connector's full-catalog sync consumed about 55% of the budget, the order flow about 20%, and the order flow was throttled about 1,900 times, each throttle adding a one-second wait before retry. Nobody had decided that orders should come first, so nothing did.

Neither finding was a bug; both vendors did what their documentation said. **The rate limit is the shared platform's way of being fair**; **the merchant's job is to decide how its own share is spent**. The rest of the chapter explains the machinery behind the error code.

Why Platforms Share: Tenancy in Plain Words

A **tenant** is one customer of a shared software platform, with its own data and configuration: at a commerce platform, one shop. **Multi-tenancy** is the design in which one running system serves many tenants. Vendors choose it for two reasons a merchant benefits from every day. The first is cost: one fleet of servers, one database estate and one operations team serve everyone, which is how a hosted platform can be rented by the month. The second is a single version: Shopify's FY2025 annual report calls its platform

"single branch" software, meaning "all of our merchants use the latest version"; there is no upgrade project because there is nothing to upgrade.

The Amazon Web Services (AWS) SaaS Lens, a published reference for SaaS vendors (n.d., accessed November 2026), names three ways to arrange tenants. In the **silo** model "tenants are provided dedicated resources", such as a separate database or a separate stack per tenant, while identity, onboarding and operations stay shared. In the **pool** model "tenants share resources": "the more classic notion of multi-tenancy", one set of servers and tables holding everyone's rows with a tenant identifier on each. The **bridge** model mixes them: "some microservices in your architecture might be implemented with silo and others might use pool", chosen by regulatory profile, noisy-neighbor risk, agility and cost.

The term **noisy neighbor** is also the SaaS Lens's: "a user of a system could place load on the system's resources that could have an adverse effect on other users". In a pool, one tenant's flash sale is every tenant's load unless the vendor meters it. In a silo, the noisy tenant only hurts itself, at a cost per tenant that few commerce subscriptions can carry.

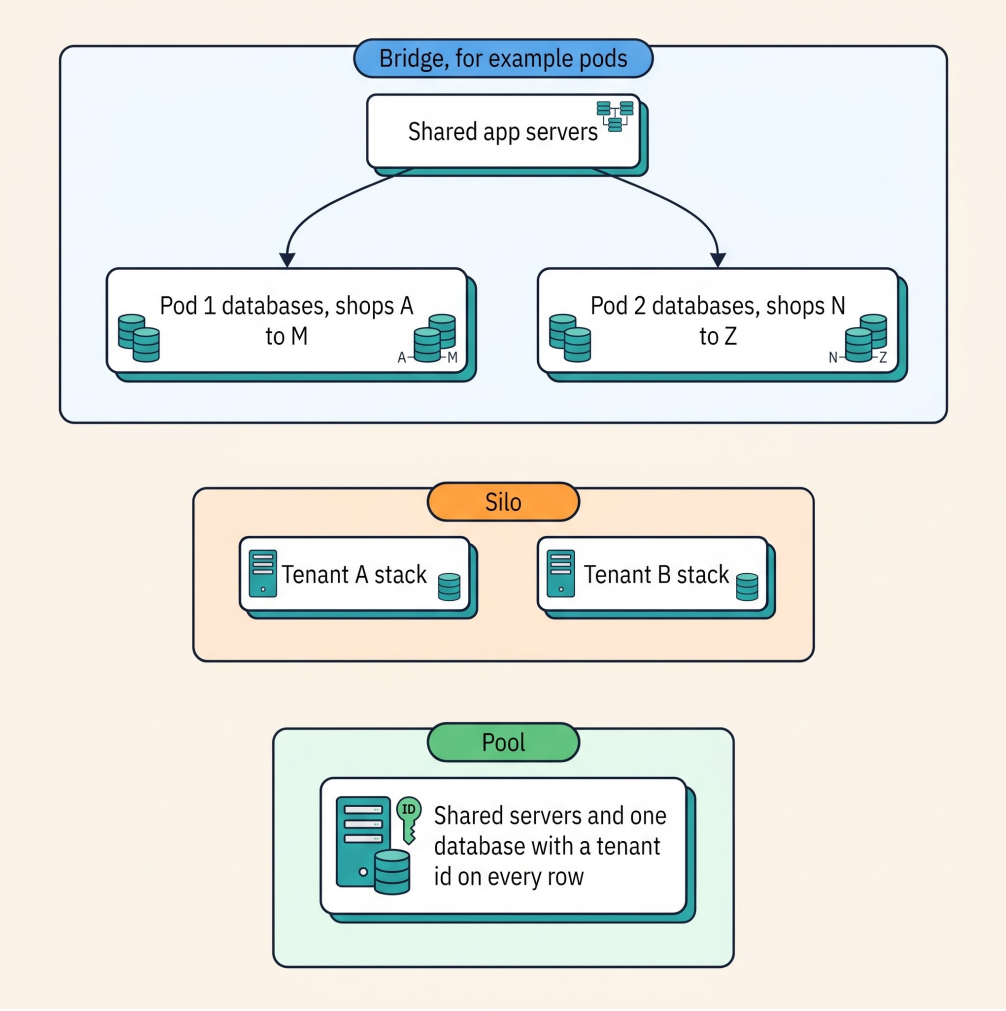


Figure 1: *Three ways to arrange tenants: a pool shares everything and relies on identifiers and limits, a silo dedicates a stack per tenant at a cost per tenant, and a bridge pools the compute while grouping tenants into isolated sets of databases.*

A merchant can see the spectrum in what vendors sell. Adobe's information center for Adobe Commerce (updated October 2026) lists three deployment options: on-premises, where the merchant or its host runs everything (a silo the merchant operates); "on cloud infrastructure", where Adobe runs the platform and the merchant maintains its own code (a silo Adobe operates); and "as a Cloud Service", a fully managed SaaS (a pool). Shopware draws the same line from the extension side: plugins that run inside the core process "have direct access to the database" and therefore "are not supported by Shopware cloud", while apps run outside and talk through APIs. In a pool, nobody's code may touch the shared database, which is why hosted platforms extend by API and not by plugin (Apps, Plugins and Functions: Extensibility and Ecosystem Governance).

How a Shared Platform Keeps Tenants Apart

The public evidence comes from three vendors who have described their mechanisms, each at a date.

Shopify: a tenant identifier, a router and pods (2018–2021). By 2015, per Shopify's 2018 engineering post, its database "could no longer be scaled by buying a bigger server", so it **sharded**: split the one database into many, each holding a set of shops. Sharding alone reduced resilience, because one failed shard could take an action down for everyone. In 2016 Shopify reorganized into **pods**: "a pod consists of a set of shops that live on a fully isolated set of datastores". Application servers, job workers and load balancers stay shared, but "all shared resources can only ever communicate to a single pod at a time"; every web request or background job is assigned to one pod, and a routing component ("Sorting Hat") matches each request to its pod and adds a header the application uses to pick the right datastore. Each pod has an active and a recovery data center, and a "Pod Mover" could move a pod between them in about a minute without dropping requests. That post is eight years old and describes a pre-cloud topology.

The 2021 post on shard balancing fills in the tenant mechanics. Each shop's data lives in one pod with its own MySQL database shard plus Redis and Memcached caches; the shop is the sharding unit, with a `shop_id` on every shop-owned table; a routing table mapping domains to pods "lives in a separate database that isn't sharded". Shops land on shards arbitrarily and grow at different rates, so the least- and most-loaded shards differed by "almost four times" before rebalancing. Moving a shop uses Ghostferry (open-source): rows are copied and changes tailed while the shop stays live, a short application-level write lock covers the cutover, the routing table is updated and stale data pruned; "the only opportunity for downtime is during the cutover". The post's worked example is two flash-sale merchants on the same pod whose bursts coincided, one of which was moved; placement uses measured utilization, not shop counts.

> **INFERENCE, NOT DOCUMENTED:** In SaaS Lens terms a Shopify pod is a bridge: siloed datastores under pooled compute. Shopify has not described the pod topology after its move to Google Cloud; the 2019 backup post and the 2021 balancing post both describe shards on Google Cloud, so pods and the cloud coexisted at least to 2021. Nothing public after 2021 gives pod or shard counts.

Salesforce: one database, one identifier, governor limits (n.d., accessed November 2026). Salesforce's architecture documentation describes the purest pool in commercial use: a "multitenant and metadata-driven" platform in which tenants' definitions and data sit in shared tables, each organization identified by an OrgID; "when the kernel accesses the database, it uses this unique identifier to ensure that each org's activities are private". Because everything is shared, the platform protects itself with **governor**

limits, published caps on what one tenant's code may consume. In a pool, limits are the isolation mechanism.

commercetools: projects and regions (n.d., accessed November 2026). commercetools documents a Project as "the top-level container" for a merchant's data and configuration, isolated from other Projects, with published per-Project limits (for example 25 API Extensions and 50 Subscriptions). It hosts in five Regions (two in North America, two in Europe, one in Australia) that "are completely isolated from each other, and no data is transferred between them". **Data residency**, the question of which country's servers hold your data, is therefore decided at sign-up by choosing a region, and changing it later is a migration (Whose Data Is It? Privacy, Consent and Retention).

Mechanism	What it keeps apart	What the merchant sees	Evidence
Tenant identifier on every row	One tenant's data from another's in a shared table	Nothing, unless it fails; the design rule to copy in any shared system you build	Shopify 2021; Salesforce n.d.
Router in front of the application	Requests, so that each one touches only its tenant's datastore	Nothing	Shopify 2018
Pod or shard	One group's database load and failures from another's	A move: a short write lock at cutover; a neighbor's flash sale, if you share a pod	Shopify 2018, 2021
Governor and resource limits	Shared compute from any one tenant's code	Published caps on scripts, queries and extensions	Salesforce n.d.; commercetools n.d.
Region	One jurisdiction's data from another's	The residency choice at sign-up; a migration to change it	commercetools n.d.

Mechanism	What it keeps apart	What the merchant sees	Evidence
Rate limit	Shared API capacity from any one app on any one store	The budget, the HTTP 429 (“too many requests”) or throttled response and the retry	All platforms (next section)

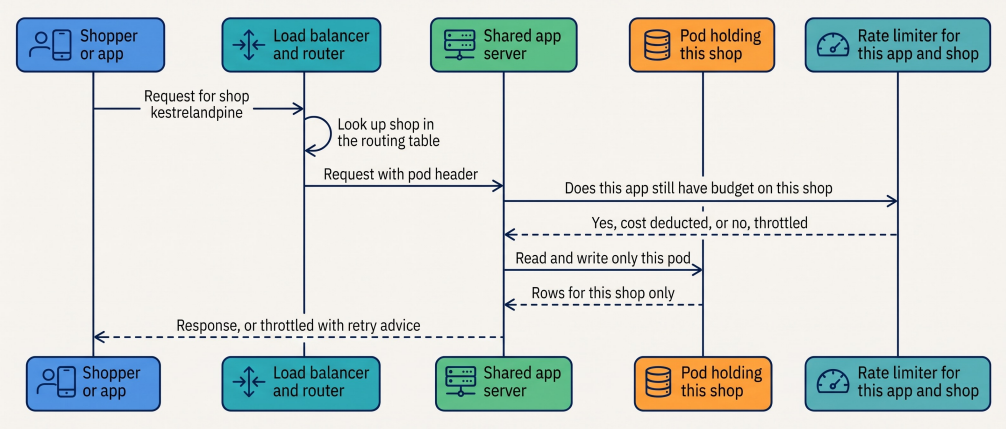


Figure 2: *A request on a pooled platform is routed to the one group of databases that holds the shop, and an API request is first checked against the budget of the app that made it for that shop; the merchant never sees the pod, and always sees the budget.*

Rate Limits: The Tenancy Tool You Meet

A **rate limit** is the maximum number of API calls, or units of query cost, that a platform lets one caller make in a time window. On a hosted commerce platform the caller is usually an **app** (any program that talks to your store through its API, whether installed from an app store or built for you) and the window is per second or per minute. Three vendors’ published models, all volatile and all as of November 2026, show three philosophies.

Platform	Model	Published figures (November 2026, subject to change)
Shopify GraphQL Admin API	A "leaky bucket" of query cost per app and store, refilling at a plan-dependent rate; a write costs ten times a simple read; no single query may exceed 1,000 points	100 points per second on the standard plan, 200 on Advanced, 1,000 on Plus, 2,000 for enterprise; bulk operations exempt from the per-query cost
Amazon Selling Partner API	A token bucket per operation with a rate and a burst, looked up from the caller's credentials; "standard" plans are static, "dynamic" plans are "automatically adjusted to each selling partner based on the current and historical business needs"	Example on the page: one request per second with a burst of two
BigCommerce	A request quota per time window, by plan, reported in response headers	60,000 requests an hour on the Scale plan; 20,000 on Growth and Core
commercetools	No published request rate; instead caps on data and query shape per Project	500 elements per query, 500 update actions per request, a GraphQL complexity ceiling, 25 API Extensions

Shopify charges by the shape of the query; Amazon charges by operation and adapts to the seller's history; BigCommerce counts requests; commercetools caps capacity. For a merchant the consequence is the same across all four: **the budget belongs to a tenant, and everything you install spends from it**. Three rules from Shopify's documentation apply, in spirit, to every hosted platform:

- The budget is per app per store, "both the bucket size and the restore rate vary by API and by plan", and the recommended wait after a throttle is one second. A merchant that buys a higher plan buys a larger bucket; a merchant that installs another app adds another spender.

- "Shopify may temporarily reduce API rate limits to protect platform stability." The budget is not a contract; on the platform's own worst day it can shrink.
- Buyer traffic is treated differently from app traffic: "requests from real buyers aren't subject to a fixed request-per-minute limit", bots are limited, and checkout creation is "throttled per minute", returning a throttled response that the caller should queue and retry. The platform protects the shopper and the checkout first, and the merchant's integrations second.

Webhooks, the notifications a platform sends when something changes, are the other half of the budget story, because they are how a merchant avoids spending it on polling. They are also a tenancy mechanism in reverse: the platform spends its capacity to call you. Shopify's documentation promises eight retries over four hours if your endpoint fails, and warns that after eight consecutive failures a subscription created through the Admin API "is automatically deleted"; an integration that was down for an evening can wake up unsubscribed (Promises Between Systems: API Contracts, Webhooks and Versioning).

Flow in Eva's order	Who owns the capacity	What is metered	What the merchant controls	On a peak day
Browsing and search on the storefront	The platform; served largely from caches and the content delivery network	Not rate-limited for real buyers; bots are	Theme weight, app scripts	The platform's peak, not yours; it is smoothed across tenants
Checkout and payment	The platform and the PSP (payment service provider)	Checkout creation throttled per minute; the PSP has its own limits	Nothing but plan tier and warning the PSP of a flash sale	Shoppers queue briefly; payment is separated from order creation under load (Shopify, 2018)

Flow in Eva's order	Who owns the capacity	What is metered	What the merchant controls	On a peak day
Order to the warehouse through a connector	Your share of the platform's API budget and of the WMS vendor's	Calls per app per store on one end; calls per customer on the other	Which apps share the budget, batching, webhooks instead of polling	The first link to fall over, and the one you can govern
Stock sync to channels	The same two budgets plus the market-place's	Writes cost more than reads	Events instead of full-catalog polls; bulk operations	The loudest spender of the budget unless redesigned

The table shows where tenancy bites. The platform's own peak is shared across every tenant and smoothed, which is why the storefront held; the merchant's integrations draw on budgets that are per tenant and fixed, which is why the order flow did not (Black Friday Every Year: Peak Load, Resilience and Recovery).

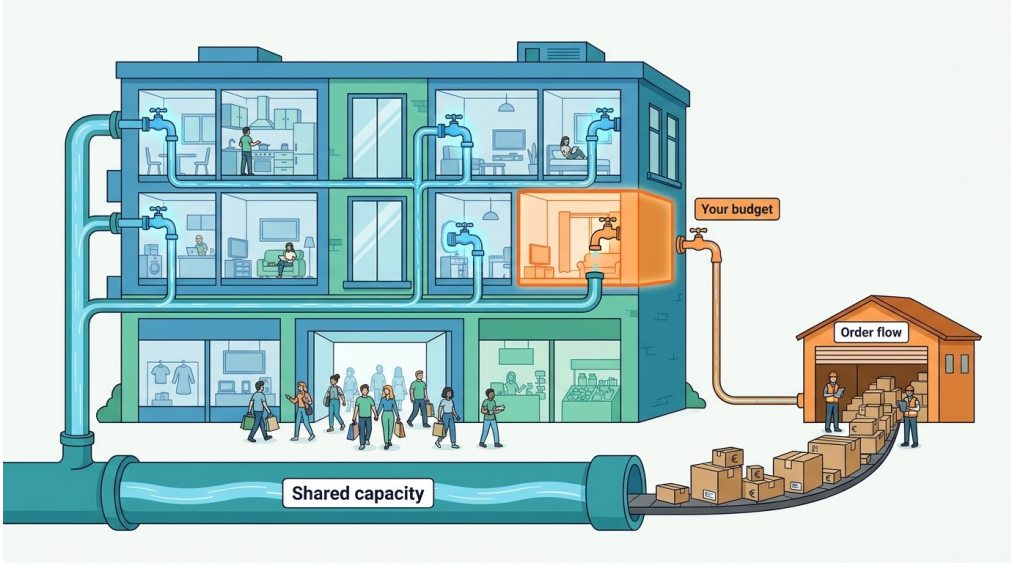


Figure 3: *The platform's own peak is shared across every tenant and smoothed, which is why the storefront held; the merchant's integrations draw on a fixed budget per tenant, which is why the order flow did not.*

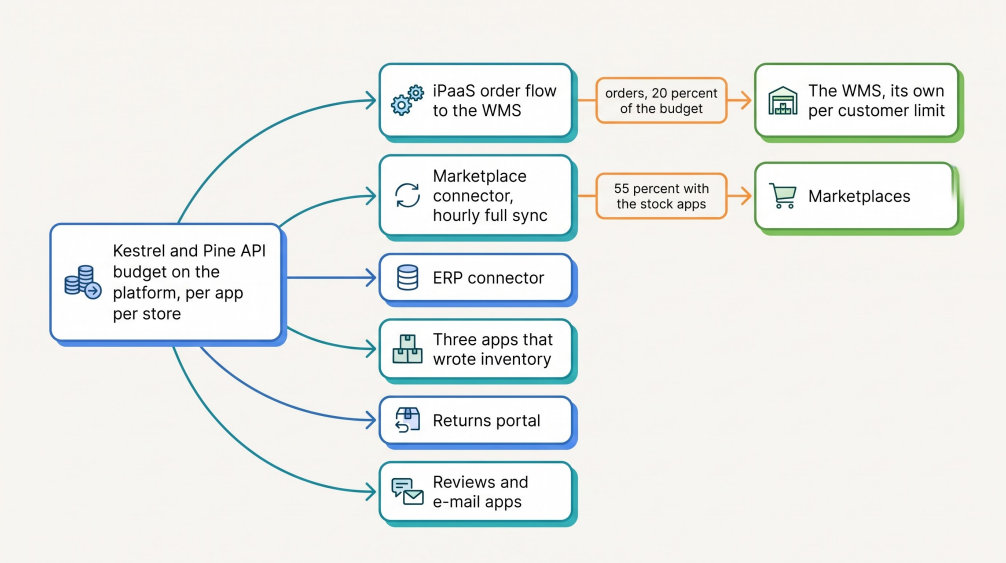


Figure 4: *On Black Friday 2026 nine consumers spent one store's API budget with no order of priority; the order flow, the only one with a customer waiting on it, got a fifth of the budget on one end and met the WMS vendor's own limit on the other (illustrative shares).*

One Tenant or Many

A merchant with several countries or brands faces a tenancy decision of its own: one tenant for everything, or one per country, brand or business unit. Kestrel & Pine runs three storefronts (Netherlands, Belgium in Dutch and French, Germany) in **one tenant**. The choice has costs both ways.

Hosted platforms have built features to keep multi-country merchants in one tenant. Shopify's Markets (n.d., accessed November 2026) lets one shop tailor "currency, language, pricing, and theme content" by customer location, with per-market settings for domains, product availability and taxes; its Managed Markets variant makes a partner the merchant of record (the legal seller on the invoice) but, as of November 2026, is offered to merchants in the United States, Canada and the United Kingdom, not the EU.

Option	What is shared	What multiplies	Fits when	Breaks when
One tenant, several markets (Kestrel & Pine)	Catalog, customers, apps, integrations, the API budget, the blast radius	Nothing	One brand, one assortment, one operations team; the platform supports markets	One country's flash sale starves another's order export; one country needs a legal entity, catalog or checkout the platform cannot vary per market
One tenant per country	Nothing but the vendor contract	Catalogs, apps, integrations, reconciliation, reporting	Separate legal entities, assortments or fulfillment per country; regulatory separation	Product data drifts between tenants (The Price Nobody Can Explain across countries); the integration count triples

Option	What is shared	What multiplies	Fits when	Breaks when
One tenant per brand (Group stage)	Possibly a shared OMS, PIM and integration layer beneath	Storefronts and apps per brand	Brands with different assortments, audiences and teams	A group forces shared apps and release dates on brands that do not share customers (One Size Fits None); or keeps brands apart that share a warehouse and a customer

My rule: **stay in one tenant as long as the platform can vary what differs by market and the shared budget can carry every market's peak with the integrations you have**. The first condition is a product question (markets, catalogs per market, per-market taxes); the second is the budget analysis above, repeated per market. Split tenants when a legal entity, a separate assortment or a separate operations team makes the sharing a fiction, and when you do, put the PIM and the OMS beneath all tenants so product and order truth is not multiplied (What Are We Selling? Products, Variants and the Catalog, The Order Record: Lifecycle, State and the Order Management System). Group-stage merchants with several brands become a tenant of tenants and should ask the vendor what they ask of themselves: which unit can be isolated, moved and restored alone (What Changes at Each Scale: From First Store to Group).

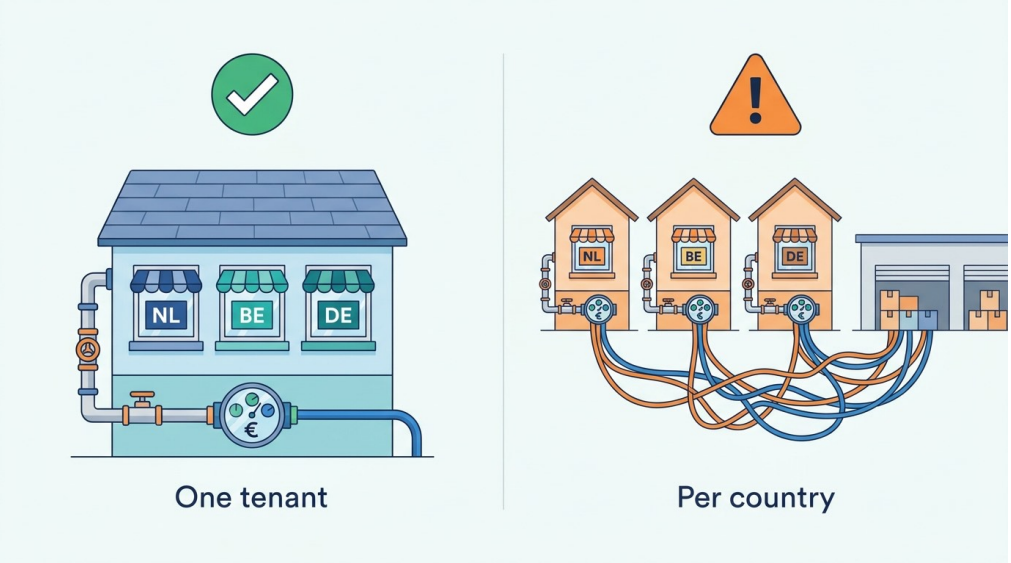


Figure 5: *Kestrel & Pine keeps its three storefronts in one tenant, sharing one catalog, one set of integrations and one API budget; one tenant per country would multiply catalogs, apps and integrations.*

Tenancy in the SaaS You Also Rent

The platform is the most visible tenancy arrangement but not the only one. Kestrel & Pine's WMS, returns portal, marketplace connector, iPaaS, shipping service and, from 2027, OMS (order management system) and PIM (product information management system) are each a multi-tenant SaaS in which Kestrel & Pine is one tenant among the vendor's other customers. Three consequences follow.

The peak day is shared. A WMS vendor's other customers are retailers whose Black Friday is the same Friday. The vendor's capacity planning is your risk, and its per-customer rate limit is its tool for being fair when every customer peaks at once. The WMS limit that throttled Kestrel & Pine's order flow was doing for that vendor's other tenants what the platform's budget does for its own.

The pricing unit tells you what the vendor meters. Operational SaaS commonly prices by volume: Picqer, a Dutch WMS, publishes plans from €379 a month including 3,000 orders and €0.28 per extra order (n.d., accessed November 2026). A per-order price means the vendor's capacity model is per order, and a peak day costs money as well as calls (Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026).

Residency is per component. Each SaaS was provisioned in a region at sign-up. A European merchant should record which region holds the data at the platform, the PIM, the OMS, the returns portal and the search service; the answer differs per vendor and moving is a migration (Whose Data Is It? Privacy, Consent and Retention).

The questions for every vendor are the same four: what is your unit of isolation, and can you move or restore my tenant alone; what are my limits, per what, and do they change under load; which region holds my data; and what happened on your busiest day last year.

The Merchant's Choices and Their Trade-Offs

A merchant cannot change how its vendor pools tenants, but it has four realistic responses to sharing.

Accept the pool and govern the budget. Keep the tenant and the plan; inventory every spender of the API budget; decide an order of priority; replace polling with webhooks and events; use bulk operations for catalog work; remove what nobody uses. It is cheap, and it is where Kestrel & Pine starts.

Buy a larger share. Move up a plan tier for a bigger bucket, or negotiate an enterprise agreement with higher or, as some vendors advertise for their largest customers, no fixed limits. It is a subscription line, not an engineering project, and it does nothing for the WMS vendor's limit or the marketplace's.

Rent or run a silo. A platform-as-a-service or on-premises edition (Adobe Commerce's two non-SaaS options are the public example) gives dedicated capacity and your own database at the price of running upgrades, capacity and security yourself (Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says).

Split tenants. One tenant per country or brand, as in the table above.

	Govern the budget	Buy a larger share	Rent or run a silo	Split tenants
Cost	Engineering days; an inventory and some redesign	A plan tier or enterprise contract	An operations capability and a larger team	Integrations, catalogs and reconcilia-tion per tenant
Protects against	Your own apps starving your order flow	Running out of budget at your own peak	Other tenants' peaks; vendor-side limit changes	One market's peak starving another's

	Govern the budget	Buy a larger share	Rent or run a silo	Split tenants
Does not protect against	Other tenants' load; the vendor lowering limits	The WMS or marketplace limits; the vendor lowering limits	Your own capacity mistakes; it is now your problem	Shared vendors beneath the tenants
Right when	Always first; every stage	The budget is measured and still short	Group stage with platform teams, or a regulatory reason	Legal entities, assortments or teams genuinely differ

The recommendation is the order of the table: govern first, buy second, and treat silos and splits as decisions with their own chapters' costs. It assumes the merchant can measure its budget consumption; a merchant that cannot see the logs should get them first.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting (under about €2M)	A pooled tenant on a standard plan; a handful of apps; the vendor's region chosen by default	A list of installed apps with their permissions; knowing which region holds the data	A 429 or throttled response in a connector log that nobody reads; an app that polls the whole catalog every hour

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
2 — Growing (about €2M–€20M)	A first ERP connector, a 3PL (third-party logistics provider) feed, a marketplace tool, all spending the same budget; the first peak that throttles	An inventory of budget spenders; webhooks and events for stock and orders; one owner per connector; a question to each vendor about its limits	The order export falls behind on a promotion; two apps write stock; the agency cannot say how many calls a flow makes
3 — Omnichannel (about €20M–€200M)	One tenant for several countries; a dozen multi-tenant SaaS products; the budget is the peak-day bottleneck	A budget allocation with the order flow first; batching within the WMS limit; bulk operations for catalog; a decision on one tenant or several per market; residency recorded per component	A market's peak starving another's; a vendor's limit change breaking a flow; a country that needs what the platform cannot vary per market
4 — Group (over about €200M, or several brands)	Several tenants or brands; enterprise agreements; some dedicated capacity; shared OMS and PIM beneath	Vendor-side isolation in the contract (pods, dedicated instances, restore of one tenant); the group's own shared services built with a tenant identifier on every row; peak capacity reserved per brand	One brand's sale starving another's on a shared connector; a group service that cannot restore one brand alone; a vendor that cannot say what its isolation unit is

The judgement: tenancy does not change what a merchant must do so much as how much of it. At every stage the first move is the same, know who spends your budget and put the order flow first; what grows is the number of budgets, the number of markets sharing them and, at Group stage, the merchant's own duty to isolate its brands the way its vendors isolate their tenants.

In Practice: Kestrel & Pine's API Budget for 2027

The peak plan in Black Friday Every Year: Peak Load, Resilience and Recovery forecasts 60 orders a minute for Black Friday 2027 and tests every link at 80. Its tenancy decisions, from Ruben's June 2027 analysis (all figures illustrative):

Spender of the platform's API budget	Black Friday 2026	Black Friday 2027 plan
Order flow to the warehouse	One order at a time through the iPaaS, about 20% of the budget, throttled about 1,900 times in two hours	Orders created in the OMS (W2), which sends fulfillment orders to the WMS in batches within the WMS limit; webhooks from the platform instead of polling; first call on the budget
Marketplace connector stock sync	Hourly full-catalog write of about 17,000 active variants, the largest single spender	Stock by events from the availability service (W1), changed variants only; if the service misses its 15 October go/no-go, the fallback is the old chain with the ten-unit marketplace buffer
Three apps writing inventory	Each polled and wrote stock on its own schedule	None may write stock after W1; the pre-order app uses an allocation in the availability service; six unused apps removed in the audit

Spender of the platform's API budget	Black Friday 2026	Black Friday 2027 plan
ERP connector	Hourly	Daily batch for finance (W5); nothing in the peak hours
Returns portal, reviews, e-mail	Small, unmanaged	Small, listed, each with an owner and a scope
WMS vendor's own limit	About 30 calls a minute per customer; four calls per order, one at a time, about seven orders a minute	Fulfillment orders in batches within the limit; tested at 120 orders a minute with marked orders the WMS skips

Two decisions carry over. The three storefronts stay in **one tenant**: the platform varies price, language and tax per market, the catalog is one assortment, and the budget analysis showed that governing the spenders, not splitting the markets, was what the order flow needed. And the plan records the **region** of every SaaS component Kestrel & Pine rents, the platform's EU region among them, so that the privacy work in Whose Data Is It? Privacy, Consent and Retention starts from a list rather than a search.

When It Goes Wrong

A neighbor's flash sale on your pod. The documented case is Shopify's 2021 example of two flash-sale merchants on one pod with synchronized bursts; the vendor's remedy was to move one. Detection: slow responses on a day that is not your peak, with your own logs quiet. Recovery: there is none on your side; report it, and ask the vendor what its isolation unit is and whether it rebalances.

A noisy neighbor inside your tenant. Your own stock app consumes the budget your order flow needs. Detection: throttled responses in the order flow's logs while another app's call count climbs; the gauge that was missing in 2026, queue depth with rate-limit headroom. Recovery: pause the app (a switch on the degradation ladder), then redesign it to events and bulk operations.

The vendor lowers the limit. "Shopify may temporarily reduce API rate limits to protect platform stability" is a published possibility, and other vendors have the same right in their terms. Detection: throttles at call rates that worked yesterday. Recovery: a flow that reads the budget headers and slows itself, rather than one that assumes a fixed rate.

The webhook that unsubscribed itself. Your endpoint failed eight times in a row

during an outage and the platform deleted the subscription. Detection: silence; a reconciliation job that finds orders in the platform and none in your system. Recovery: re-subscribe, replay from the platform's API, and keep the reconciliation job (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation).

The shop move. A vendor moves your shop between shards; the documented mechanism takes a short write lock at cutover and fails the move if the lock cannot be acquired in time. Detection: a brief burst of write failures with no change on your side. Recovery: idempotent retries; it should pass in seconds, and the vendor's status page should say so.

The shared peak at a specialist vendor. Your WMS, returns portal or connector slows because every tenant peaks at once. Detection: throttles and latency at the vendor's end, correlated with the calendar rather than with your load. Recovery: batch within the limit, queue behind the order, and put the vendor's busiest-day record into next year's contract talk.

What to Take Away

Established.

- Multi-tenant SaaS arranges tenants as pool, silo or bridge, and the noisy-neighbor problem is inherent to pooling (AWS SaaS Lens, n.d.); the documented commerce mechanisms are a tenant identifier on every row, a router, isolated database groups, resource limits and regions (Shopify 2018 and 2021, Salesforce n.d., commercetools n.d.).
- Rate limits on commerce platforms are granted per app and per store, tiered by plan, and can be reduced by the vendor under load (Shopify documentation, 2026); buyer traffic and checkout are protected ahead of integrations.
- Webhook delivery is not guaranteed, is unordered, and a subscription can be deleted after repeated failures; consumers must be idempotent and reconcile (Shopify documentation, 2026).

Depends on context.

- One tenant for several countries is right while the platform can vary what differs by market and one budget can carry every market's peak; it stops being right when a legal entity, assortment or operations team genuinely differs.
- Buying a larger budget is right when consumption is measured and still short; it is wrong as a substitute for removing the spenders, because it does nothing for the WMS's or the marketplace's limits.
- A silo (platform-as-a-service or self-hosted) is right at Group stage with a platform team or under a regulatory requirement; below that it trades a vendor's shared operations for your own.

The author's synthesis.

- Tenancy is the vendor's architecture, but the budget is the merchant's: treat the API budget per store as a capacity you plan, with the order flow first and a named owner for every spender, because on a pooled platform that budget is the only isolation you control.
- Every SaaS you rent should be asked the same four questions (unit of isolation, limits and their changes, region, busiest day), and the answers belong in the vendor card next to the price, because a vendor's tenancy is a promise about your peak day.

Questions to Ask

1. Which apps and connectors spend our platform's API budget, how much each, and in what order of priority on a peak day?
2. What is our plan's budget per app per store, and may the vendor lower it under load? Do our flows read the budget headers and slow down, or assume a fixed rate?
3. For each multi-tenant SaaS we rent (WMS, OMS, PIM, returns, connector, iPaaS): what is its limit per customer, and how did it behave on its busiest day last year?
4. What is each vendor's unit of isolation? Can it move, restore or rebalance our tenant alone, and how would we know it happened?
5. Which region holds our data at each vendor, and what would it take to change it?
6. Are our markets or brands one tenant or several, and which condition (legal entity, assortment, operations team, budget) would change that answer?
7. Which flows poll where they could subscribe, and which write the whole catalog where they could write only the changes?
8. If a webhook subscription were silently deleted tonight, which reconciliation job would notice by morning, and is there a tenant identifier on every row of every shared service we have built ourselves?

The budget a merchant spends on a shared platform is spent mostly by apps, and apps are the next chapter's subject: how platforms let merchants and partners extend them without breaking them, what an app may touch, and how a merchant with forty-one apps, six unused and three writing stock, should govern them: Apps, Plugins and Functions: Extensibility and Ecosystem Governance.

Sources and Evidence

Company, product and investor sources

- A Pods Architecture to Allow Shopify to Scale, Shopify Engineering, 2018 — sharding in 2015, pods in 2016, the router and the Pod Mover; eight years old, pre-cloud.
- MySQL Database Shard Balancing at Terabyte Scale, Shopify Engineering, 2021 — the shop as sharding unit, the routing table, Ghostferry moves, the four-times imbalance and the two flash-sale merchants.
- Shopify Inc. Form 10-K for fiscal 2025, U.S. SEC, 2026 — "multi-tenant", "single branch", the definition of a merchant as a paying shop.
- GraphQL Admin API rate limits, API limits and REST Admin API rate limits, Shopify, n.d., accessed 2026-11 — the cost model, plan tiers, per-app-per-store scope, the one-second backoff and the stability clause.
- Storefront API, Shopify, n.d., accessed 2026-11 — buyer traffic without a fixed limit, bot limits, the per-minute checkout throttle.
- Faster checkout for customers during flash sales, Shopify changelog, 2018 — payment processing separated from order creation under load; eight years old.
- Webhooks and HTTPS webhook delivery, Shopify, n.d., accessed 2026-11 — no delivery guarantee, eight retries over four hours, subscription deletion after eight failures.
- Shopify Markets and Managed Markets, Shopify Help Center, n.d., accessed 2026-11 — one shop with several markets; the merchant-of-record variant and its geography.
- Platform Multitenant Architecture, Salesforce Architects, n.d., accessed 2026-11 — the metadata-driven pool, the OrgID and governor limits.
- commercetools General Concepts and API Limits, commercetools, n.d., accessed 2026-11 — Projects, Regions and per-Project limits.
- Usage Plans and Rate Limits, Amazon Selling Partner API, n.d., accessed 2026-11 — token buckets per operation, standard and dynamic plans.
- API rate limits, BigCommerce, n.d., accessed 2026-11 — request quotas by plan and the response headers.
- Adobe Commerce platforms and services, Adobe Experience League, 2026 (updated 2026-10-21) — on-premises, cloud infrastructure and Cloud Service as three answers to who runs the servers.
- Extensions concept, Shopware, n.d., accessed 2026-11 — plugins with database access unsupported on the cloud edition; apps over APIs.
- Picqer pricing, Picqer, n.d., accessed 2026-11 — per-order pricing as an example of a volume-metered operational SaaS.

Standards, regulation and official documentation

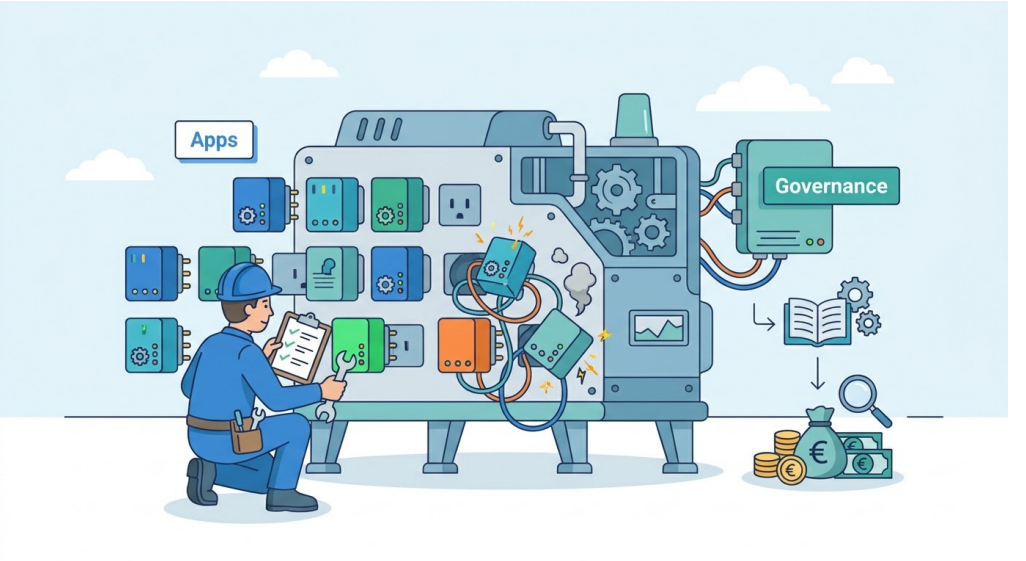
- Silo, pool, and bridge models and Noisy neighbor, AWS Well-Architected SaaS Lens, n.d., accessed 2026-11 — the tenancy models and the noisy-neighbor definition.

Research and practitioner sources

- Ghostferry, Shopify (GitHub), n.d., accessed 2026-11 — the open-source tool behind shop moves, with its own caveat that the model is not a proof of the implementation.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

31. Apps, Plugins and Functions: Extensibility and Ecosystem Governance



IN THIS CHAPTER, YOU WILL: Learn how platforms let other people's code change them, what each mechanism may touch, why installed apps sprawl, and how to govern the apps a merchant runs.

KEY POINTS:

- **Extension mechanisms differ by where the code runs and what it may touch.** A plugin inside the platform's process can reach anything, including the database; an app outside it may do only what its granted permissions allow; a function at a named extension point gets a fixed input, a fixed output and a time budget. The deeper the code sits, the more every vendor release costs.
- **The app store is the vendor's business model and the merchant's governance problem.** Scopes, app review and dated API versions protect the platform and the shopper; none of them tells the merchant

which of its installed apps writes stock, prices or orders. That inventory is the merchant's own work, and the App That Wrote Stock is what happens when nobody does it.
- **Govern by owner, scope and writer, and keep custom behavior out of the core.** One named owner per app, its scopes on a card, one writer per business fact, a quarterly uninstall review, and custom logic in the platform's bounded extension points rather than in a modified core. A platform you cannot upgrade is a platform you have already left.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Secret Agent** — an app that writes stock, prices or orders without a named owner, its own identity or anyone's approval is this pattern in its most common form.
- **The Forked Platform** — in-process customization of a platform's core is the mechanism that makes a platform un-upgradable, and the vendors describe the cost in their own documentation.
- **The Spaghetti Junction** — forty-one apps, each with its own path into the platform's data, behave like forty-one hand-built connectors, many of them without an owner.

No commerce platform does everything a merchant needs. A hosted platform serves millions of shops from one codebase; a suite serves thousands of enterprises with one product; neither can build the size guide, the pre-order flow, the loyalty program and the Belgian fiscal receipt that one merchant wants this quarter. So every platform lets other people's code change it, and sells that ability as a **marketplace of apps, plugins or extensions. For the vendor, extensibility is how one product fits a million businesses. For the merchant, it is how one product quietly becomes forty.**

The arrangement works until the extensions touch the same facts. An app that shows "only 3 left" reads stock. An app that takes pre-orders may write it. An app that syncs the warehouse writes it too. Each was installed for a good reason by someone who did not know about the others, and the platform runs all three. The failures look like inventory, pricing or order failures; the cause is a governance gap that this chapter owns.

Kestrel & Pine, the fictional outdoor brand this book follows, supplies the worked example: 41 installed apps, 6 unused, 3 writing stock, and the rule Noor Visser set in April 2027.

Where we are in the order's journey. Apps sit beside the path of Eva's jacket rather than on it. The size guide she opened on the product page, the "only a few left" badge, the review widget and the notification app that mailed her tracking link are all extensions; each reads order or stock data, and the ones that write it decide whether the stock Eva saw was real. This chapter is about the code beside the journey, and about keeping it from rewriting the journey's facts.

Setting the Scene at Kestrel & Pine: Forty-One Apps and One Spreadsheet

In the second week of April 2027, Noor Visser (Head of Engineering since January) sat down with Milan Petrov, who manages the storefronts, the apps and the marketplace connector, to list every app installed in the platform. The admin showed 41. Nobody had a list of why. Milan knew about twenty of them well; 22 had been installed by the agency over the years, many for theme features; the rest came from campaigns, pilots and former colleagues. The subscriptions added up to about €2,900 a month (a Kestrel & Pine illustration), spread over the platform invoice, the agency's invoice and three company cards. The list they produced is the table every merchant should be able to show in an afternoon (figures illustrative).

Category	Apps	Unused	Write platform data	Example
Storefront features (size guide, reviews, badges, pop-ups, bundles)	14	3	0	A "low stock" badge that reads inventory 40 times a minute
Marketing and analytics (pixels, e-mail, referral, A/B testing)	11	2	1 (customer tags)	An e-mail app writing customer tags for segmentation
Operations (pre-orders, returns, shipping rates, invoices, stock sync)	9	0	5 (stock, orders, fulfillment)	The pre-order app and an inventory-sync app the agency installed
Finance and compliance (tax, cookie consent, fiscal receipts)	4	0	1 (order metadata)	The Belgian cookie-consent banner
Agency utilities and pilots	3	1	0	A search pilot nobody switched off
Total	**41**	**6**	**7**	**3 of the 7 write inventory**

The three that write inventory are the ones every later chapter refers to, and none of them is the main stock feed. That feed carries the WMS's (warehouse management system's) counts into the platform every hour through the integration platform (the **iPaaS**, integration platform as a service: a hosted tool for building the flows that move data between systems; APIs, Events, Files and Middleware: How Commerce Systems Talk). One app is an inventory-sync app the agency installed to copy stock adjustments between the platform's locations on its own schedule. One is the pre-order app behind **the App That Wrote Stock** of March 2026: it adjusted inventory to reserve units for pre-orders while the WMS feed overwrote the same numbers an hour later, and for two weeks about 600 variants carried phantom reservations that the storefront reported as sold out. The third once pushed stock to a channel the company no longer uses and still holds write access.

Noor's question to Milan was not "which app did it". It was: **who in this company is allowed to let a program write stock, and how would we know it had?** Nobody was, and nobody would. That is the question this chapter answers.

Five Ways to Extend a Platform

A commerce platform is a promise-keeping machine that many businesses share. Shopify's annual report for the year ended 31 December 2025 (filed 11 February 2026) describes its platform as " 'single branch' software, which means that all of our merchants use the latest version", serving "millions" of merchants with "more than 21,000 apps" in its App Store at year end. One codebase cannot carry a million businesses' particular needs, so the long tail of behavior lives in apps; at a different scale, that is why SAP Commerce describes itself as "a flexible, modular architecture based on extensions". Three things decide how dangerous an extension is: **where it runs** (inside the platform's process or outside it), **what it may touch** (everything the platform can reach, or only the data its permissions cover) and **how tightly it is coupled to the platform's version** (whether a release can break it silently, or whether it talks to a versioned interface with a stated support window).

In plain words, an **extension point** is a named place in the platform's flow where the platform agrees to run, or call, someone else's code. Platforms offer five kinds.

Mechanism	Runs where	Can touch	Coupling to the platform's version	Dated examples
In-process plugin, **module**, **extension**, **cartridge**	Inside the platform's program, in its language	Anything the code can reach, including the database	Tight: re-tested on every release	Adobe Commerce plugins; SAP Commerce extensions; WordPress hooks under WooCommerce; Shopware plugins; Salesforce B2C Commerce cartridges
Theme code	Inside the storefront renderer (templates and scripts)	Presentation, and through scripts the whole page	Medium: theme updates can conflict with edits	Liquid themes (Shopify); Stencil themes (BigCommerce); SFRA, the Storefront Reference Architecture (Salesforce B2C Commerce)
App over APIs and webhooks	Outside, hosted by the app's vendor	Only what its granted scopes permit	Loose: versioned APIs with a support window	Shopify, BigCommerce, Shopware and Saleor apps; Adobe App Builder

Mechanism	Runs where	Can touch	Coupling to the platform's version	Dated examples
Serverless function at an extension point	Inside the platform's request path, in a sandbox	A fixed input and a fixed output	Loose, and time-boxed	Shopify Functions
Synchronous call-out	The platform calls the merchant's endpoint before saving	Validate or amend one resource	Loose; latency becomes the merchant's problem	commercetools API Extensions

In-process: the plugin can do anything, which is the problem

An **in-process plugin** runs inside the platform program itself, in the same language, and can therefore do anything the platform can. Adobe Commerce's developer documentation describes its plugins (also called interceptors) as code that changes a public method's behavior by running *before*, *after* or *around* it without modifying the class; an "around" plugin that never calls the original method skips it entirely, several plugins on one method are ordered by a `sortOrder` setting or else by module load order, and around plugins "should be avoided unless they're required, because they add to stack traces and can hurt performance" (n.d., accessed November 2026). Read that as a newcomer: a plugin can replace the platform's own logic for computing a price or saving an order, and two plugins can disagree about who goes first.

WooCommerce's extension guide adds the warning that matters most: "backwards compatibility between WooCommerce releases is not guaranteed" (n.d., accessed November 2026). Salesforce B2C Commerce layers **cartridges** (packages of templates and code) over a base storefront and tells developers "you never edit `app_storefront_base` directly", which "keeps the backward compatibility guarantee intact" (Trailhead, n.d., accessed November 2026): extend by layering over the core, never by editing it. Shopware draws the line most explicitly, because it sells both kinds: plugins "are executed within the Shopware Core process", "have direct access to the database", and therefore "are not supported by Shopware cloud"; apps "are not executed within the process of the Shopware Core" and interact through webhooks and the admin API (n.d., accessed November 2026). A vendor that runs the platform for you cannot let your code run inside it.

Theme code: the storefront's own extension point

Theme code is the set of templates and scripts the platform renders into pages; Delivering the Storefront: Themes, Headless, Rendering and Performance owns it. It is a second place where apps land: before Shopify's Online Store 2.0 (announced 29 June 2021), apps typically injected code into theme files and uninstalling could leave "ghost code" behind, which theme app extensions were introduced to end. Agency-edited theme files are the storefront's version of the Forked Platform: the more templates edited by hand, the more a theme update becomes a merge.

Apps over APIs: outside the process, limited by scopes

An **app** in the hosted-platform sense is a separate program, run elsewhere, that talks to the platform through its APIs and receives **webhooks** (push notifications the platform sends to the app's address when something changes; Promises Between Systems: API Contracts, Webhooks and Versioning). It may only do what its granted **scopes** allow: a scope is one named permission the app asks for at install, such as "read orders" or "write inventory", shown to the merchant on the consent screen. Shopify's documentation gives `read_inventory` and `write_inventory`, `read_orders` and `write_orders` as examples and notes that order scopes "cover orders created within the last 60 days" unless a broader scope is approved (n.d., accessed November 2026). Saleor describes its apps the same way, as external services over webhooks and the API.

Apps are loosely coupled because the API is versioned. Shopify "releases a new API version every three months", supports each stable version "for a minimum of 12 months" and, if an app targets a retired version, "falls forward and responds using the oldest accessible stable version" (n.d., accessed November 2026). The price of looseness is that every app carries an upgrade obligation of its own.

Functions and call-outs: a fixed input, a fixed output, a time budget

The newest mechanism runs the merchant's small program at a named moment in the platform's own flow, with a fixed input, a fixed output and a limit on how long it may take. Shopify Functions plug into targets such as discounts, delivery and payment customization, cart and checkout validation and order routing. The function is compiled to WebAssembly (a portable binary format that runs in a sandbox), receives JSON the platform produces, returns a list of operations, cannot be called by URL and has no network access; the documented limits for API version 2026-10 are 11 million instructions per run for carts of up to 200 lines, 10 MiB of memory and a 256 KiB module. The earlier mechanism, Scripts (Ruby code running inside checkout for Plus merchants), was retired on 30 June 2026 (help center, accessed November 2026).

commercetools' **API Extensions** are the synchronous call-out variant: after a create or update on a cart, order, payment or customer is processed but before it is saved, the platform calls the merchant's endpoint, which may reject the change or return update actions. The documentation is candid: "if it fails or takes a second longer to return a result, the whole API call fails or takes a second longer." The default budget is 2 seconds to connect and 2 seconds to respond, configurable to 10 with a support request; the recommended target is about 50 milliseconds; a project may have 25 extensions; a failure fails the API call and is not retried (n.d., accessed November 2026).

The common design is the lesson. **An extension point is a named moment with a fixed input, a fixed output, a time budget and a failure rule.** The time budget is the vendor protecting checkout from the merchant's code, which is the architectural fact behind the claim in From Cart to Confirmation: Checkout Orchestration that checkout is the most protected part of a platform.

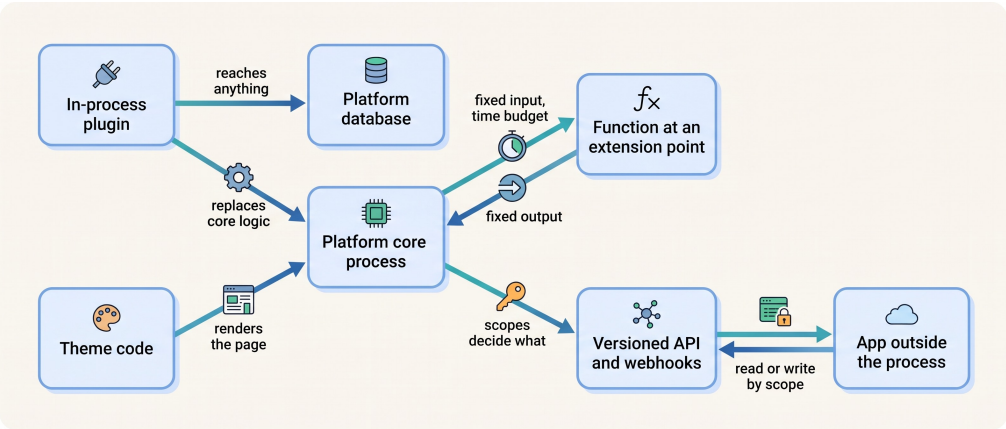


Figure 1: *Where an extension runs decides what it can touch: a plugin inside the process reaches the database and can replace core logic; a function gets a fixed input and a time budget; an app outside the process reads and writes only what its scopes allow.*

What each interaction needs from the platform

The requirements differ by interaction; time budgets are the vendors' documented ones where given and assumptions otherwise.

Interaction	Mechanism	Time budget	On failure	Who owns the outcome
Checkout validation or discount	Function or call-out	Milliseconds (Shopify: an instruction count; commercetools: 2 s default)	The platform fails the call or skips the function by its documented rule	The merchant; the shopper waits
Stock write from the warehouse	App over API	Seconds, asynchronous *(assumption)*	Retried; stale stock until it succeeds	The integration owner
Order created notification to an app	Webhook	Shopify: 1 s to connect, 5 s total; 8 retries over 4 hours	After 8 consecutive failures the subscription is deleted	The app's owner, and the merchant
Core method replaced (price, tax, save order)	In-process plugin	Whatever the plugin takes	The platform does what the plugin does, right or wrong	The merchant, entirely

Every row but the last has a platform rule for failure. The in-process row has none, because the platform no longer knows what it is doing.

The App Store: Business Model and Governance

An **app store** is where a platform vendor lists third-party apps, reviews them, bills for them and takes a share. It is a business and a governance mechanism, and the two are not the same.

As a business. Shopify's revenue-share page states that developers keep all of the first $1,000,000 of lifetime gross app revenue (counting revenue earned on or after 1 January 2025) and pay 15% above it (n.d., accessed November 2026). The app store aligns the vendor's and the developers' interests around installs, which is why it is full, and why nothing in it is designed to reduce the number of apps a merchant runs.

As governance. The review requirements are the platform's and the shopper's protection, and read like a list of the harms the vendor has seen:

- **Least scope**. Shopify asks that an app request only the scopes it needs. BigCommerce states that "Apps configured with all scopes aren't approved", that a checkout-content scope triggers a security assessment, and that apps handling card data need a PCI review (against the card industry's Payment Card Industry security standard) (both n.d., accessed November 2026).
- **Performance and currency**. A Shopify public app "shouldn't lower a store's Lighthouse performance score by more than 10 points" (Lighthouse is Google's page-speed audit tool), and apps using APIs deprecated within 90 days cannot be submitted.
- **Clean uninstall**. BigCommerce requires that uninstalling "return the control panel and storefront to their pre-install state" and remove storefront scripts.
- **Privacy**. Every Shopify App Store app must implement three compliance webhooks (customer data request, customer redaction, shop redaction) and complete the request "within 30 days". The deletion request that touches nine of Kestrel & Pine's systems in Whose Data Is It? Privacy, Consent and Retention reaches its apps this way.
- **Billing through the platform**, so the merchant sees charges on one invoice.

Now read what the list does not contain. **Nothing in app review tells the merchant that two of its installed apps both write inventory**. Each app passed review on its own; each asked for `write_inventory` for a reason; the merchant approved each consent screen months apart. The platform cannot know that, in your business, the warehouse feed is the system of record for stock and the pre-order app is not, because that is a decision about your business (Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record). The App That Wrote Stock is a governance gap on the merchant's side, and no vendor can close it.

What an App May Touch: The Writer Problem

The most important question about any extension is not what it does but **what it writes**. A reader can be slow, wrong or expensive. A writer can change a fact that other systems trust. In March 2026 Kestrel & Pine's platform had two writers of one fact: the WMS feed **set** stock levels every hour to what the warehouse counted, and the pre-order app **adjusted** stock by the quantity pre-ordered. Each operation was correct alone. Together they fought, and the number the storefront showed drifted away from both.

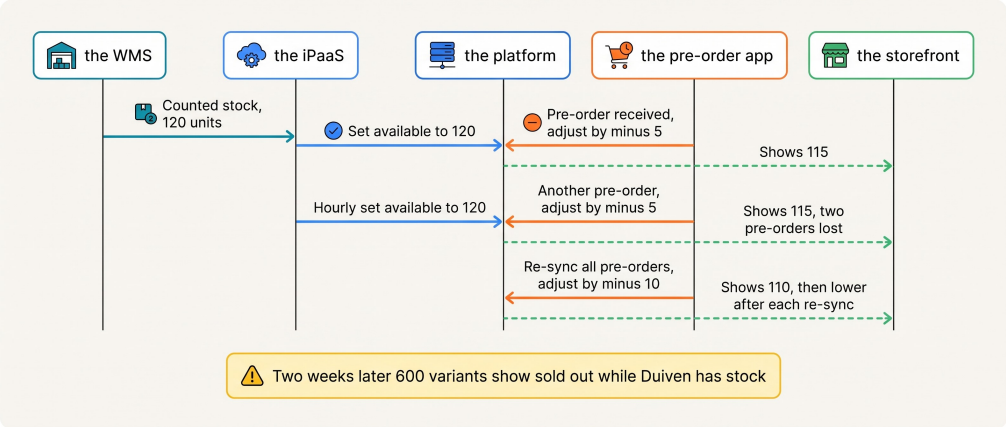


Figure 2: *Two writers of one fact: the warehouse feed sets stock to the counted quantity, the pre-order app adjusts it by reservations, and the number the storefront shows drifts away from both.*

Shopify's inventory documentation (n.d., accessed November 2026) shows the distinction cleanly: its API offers `inventoryAdjustQuantities` (a change per state) and `inventorySetQuantities` (an absolute quantity), and models stock per location in states such as *available*, *committed* (set aside for placed orders) and *on hand*. Kestrel & Pine's platform is not named and the mechanism is general: **set and adjust do not mix on the same number**, and a reservation belongs in its own state, not subtracted from the physical count. Is It in Stock? Inventory, Availability and the Promise You Can Keep owns the states and the availability service that Waypoint's decision W1 introduces; this chapter owns the rule that got Kestrel & Pine there: **one writer per fact**. For each fact the platform holds there is one system of record and at most one path by which it is written into the platform; every app that holds the write scope on that fact is either that path or a mistake.

Fact in the platform	System of record (Kestrel & Pine, after Waypoint)	The one writer into the platform	Apps that may only read
Physical stock and availability	The WMS for physical stock; the availability service for what each channel may promise (W1)	The WMS feed through the iPaaS, until the availability service replaces it (Q4 2027)	Badges, feeds, the pre-order app

Fact in the platform	System of record (Kestrel & Pine, after Waypoint)	The one writer into the platform	Apps that may only read
Prices	The platform's market price lists for consumer prices; the ERP's (enterprise resource planning system's) for wholesale (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax)	Price events with effective dates from the price lists	Repricing and marketplace apps
Product content	The PIM, product information management system (W3)	The PIM's publication job	Search-engine optimization (SEO) and feed apps
Orders	The OMS, order management system (W2); the platform keeps checkout and order history	The checkout, and the OMS for status	Notification, invoice and analytics apps

Writing is not forbidden (the returns portal creates exchange orders through the platform's API, with an owner and a reason); every writer is named.

The Forked Platform: When the Core Has Been Changed

The Forked Platform is a platform whose core or theme has been customized so deeply that it can no longer be upgraded, so every vendor release is a project and, eventually, no release is taken. The vendors who offer in-process mechanisms describe it best.

Adobe Commerce's upgrade guidance says: "The amount of custom work and the number of installed third-party modules strongly affects the complexity of an upgrade", "The quality of the custom work and modules can determine whether an upgrade goes smoothly", skipping releases makes the next upgrade harder, and if upgrades cost more than expected, audit the project (n.d., accessed November 2026). Adobe ships an Upgrade Compatibility Tool that checks a customized instance's modules and core code against a target version and reports "critical issues, errors, and warnings" (updated 8 October 2026). Its reasoning for its newer out-of-process model is the same lesson from the vendor's side: in-process code "must stay compatible with each Commerce version and its runtime", while App Builder apps can be "developed, deployed, and operated on their own schedule"; the trade-off is that in-process extensibility offers "deep, synchronous control of the Commerce runtime" that out-of-process does not (n.d., accessed November 2026).

The clock that turns a postponed upgrade into a forced migration is the support policy. Adobe's lifecycle page (accessed November 2026) gives each release line roughly three years of standard support from general availability. Version 2.4.4 reached general availability on 12 April 2022 and left standard support on 12 April 2025; 2.4.7 (general availability 9 April 2024) is supported to 31 May 2027. From 1 June 2027 Adobe says it will no longer maintain cloud environments running unsupported versions, including "suspending traffic to the affected infrastructure", so the storefront "will go offline". A merchant still on 2.4.4 has a dated deadline and a bill that grows with every module.

Hosted platforms cannot be forked, but they withdraw extension points, which is the same risk from the other direction. Shopify's checkout template (`checkout.liquid`) stopped working for in-checkout pages on 13 August 2024, by a notice of 13 February 2023; non-upgraded checkouts were "automatically upgraded to an un-customized, out-of-the-box Shopify Checkout"; Scripts followed on 30 June 2026 (From Cart to Confirmation: Checkout Orchestration tells the story). **An extension point is a promise with a date**, and the merchant's inventory must record which date each app depends on.

So open source and suites put the upgrade risk in the merchant's hands (the merchant may change the core, and must manage module count, module quality and an upgrade calendar under the vendor's support clock), while hosted SaaS puts the deprecation risk there instead (the merchant cannot change the core, and must manage app scopes, API versions per app and the vendor's deprecation calendar). Neither is safe, and both are managed by the same inventory.

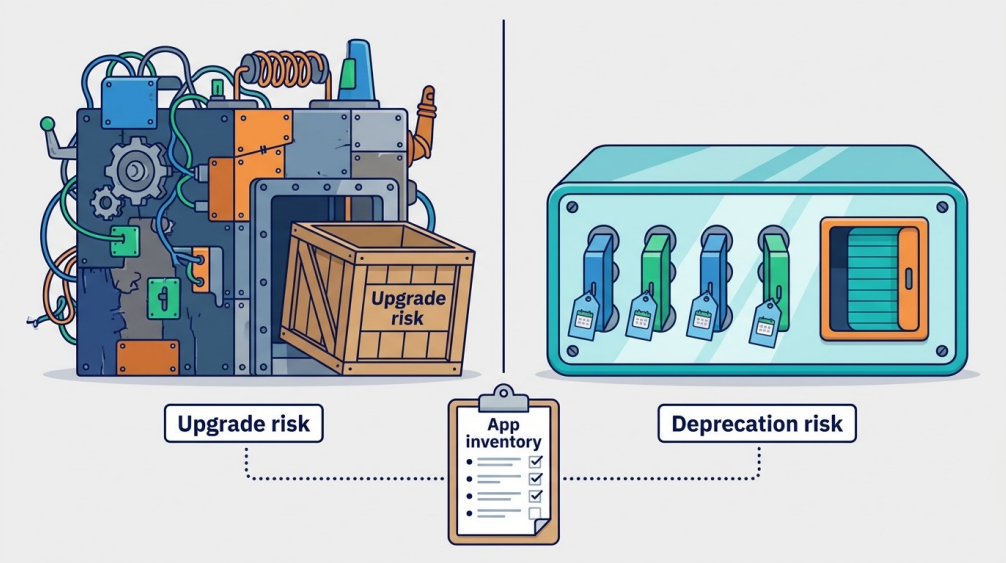


Figure 3: *Open-source and suite platforms put the upgrade risk in the merchant's hands and hosted platforms put the deprecation risk there instead; an extension point is a promise with a date, and one app inventory manages both.*

The Choices: Where Custom Behavior Should Live

When a merchant needs the platform to behave differently, there is a ladder of places to put the change. I climb it from the top and stop at the first rung that works: **configuration** (most "customizations" are settings the platform already has; Pete Hodgson's 2017 toggle taxonomy helps decide whether a switch is a release, ops or permissioning toggle, and so whether it belongs in configuration at all); **a platform-native extension point** (a function, a call-out or a workflow tool such as Shopify Flow, which "monitors your store for events" and runs actions); **a public app** for commodity capabilities; **a custom app over the API** for integrations and special behavior, with an owner and an API-version pin; **an in-process plugin** only where the platform allows it and the behavior needs synchronous control of the core; and **editing the core**, never.

Choice	Upgrade coupling	What it may touch	Latency risk to checkout	Who maintains it	When it is right
Configuration	None	What the vendor exposes	None	The merchant's admin	Always try first

Choice	Upgrade coupling	What it may touch	Latency risk to checkout	Who maintains it	When it is right
Native extension point (function, call-out, workflow)	Loose, versioned	Fixed input and output	Bounded by the vendor	The merchant or agency	Checkout, discount and routing rules
Public app	Loose	Its scopes	Theme scripts can slow pages	The app vendor	Commodity capabilities
Custom app	Loose	Its scopes	None, if asynchronous	The merchant (named owner)	Integrations; special behavior
In-process plugin	Tight	Anything	Whatever it takes	The merchant, on every release	Rarely; synchronous core control
Core edit	Total	Anything	Whatever it takes	Nobody can	Never

The recommendation assumes a merchant with a small team that does not sell software; a Group-stage merchant with a platform team on a suite can afford a disciplined, budgeted in-process layer, and Kestrel & Pine, with three storefront-and-apps engineers and an agency, cannot.

Across Scale Stages: When It Fits and When It Limits

The mechanisms do not change with scale; the governance does, and the stage decides which failure arrives first.

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
1 — Starting (Kestrel & Pine 2016–2018)	A hosted platform and ten to twenty public apps installed by the founder or the agency; no custom code beyond the theme	A list of installed apps with their scopes, kept from day one; one person who approves installs	Two apps for the same job; an app billed to a card nobody checks; theme edits by hand
2 — Growing (2019–2022)	The first custom app or connector (ERP, 3PL or third-party logistics provider, marketplace); thirty apps; agency-built theme features; on open-source platforms, the first in-process modules	A named owner per app and connector; the API version each targets, written down; one writer each for stock and prices; out-of-process mechanisms preferred even where in-process ones are allowed	Stock or price disagreements traced to an app; an upgrade postponed because "the modules might break"; the agency as the only one who knows what an app does
3 — Omnichannel (2023–now)	Forty or more apps across storefront, marketing, operations and compliance; apps that write stock, orders and customers; several countries in one tenant	A governance card per app; a quarterly uninstall review; scopes revoked when a system of record moves (W1, W2, W3); on in-process platforms, an upgrade calendar set by the vendor's support clock and a module budget	A phantom-stock or price incident caused by two writers; app subscriptions nobody can account for; a deprecation discovered after its date; a platform release more than a year behind

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
4 — Group	Several brands, each with its own app list; platform teams; possibly in-process customization on a suite	A shared extension catalog across brands; one upgrade calendar for every vendor; app security review inside vendor management (Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers); the same contract discipline for the group's own APIs	Each brand solving the same need with a different app; a module written for one brand blocking the group's upgrade; **One Size Fits None** when brands are forced to share apps that fit none of them

The judgement: extensibility is cheap to adopt and expensive to govern, and the bill arrives at the Omnichannel stage as two writers of one fact. A merchant that keeps the list from the Starting stage pays almost nothing for it; one that starts at forty apps pays a week of archaeology, as Kestrel & Pine did.

In Practice

Dated public examples

- **A hosted platform withdrawing in-process code (2021–2026).** Shopify announced WebAssembly-based Functions in beta on 29 June 2021, deprecated its checkout template by a notice of 13 February 2023 effective 13 August 2024, and retired Scripts on 30 June 2026. Over five years the platform replaced free-form code inside checkout with sandboxed, time-budgeted extension points; read that as the direction every hosted platform takes as it grows.
- **A suite vendor moving extensibility out of process (2025–2026).** Adobe's Commerce as a Cloud Service, whose release notes begin in November 2025,

extends through App Builder, API Mesh, events and webhooks rather than in-process modules, with updates delivered continuously and 30 days to evaluate them in a sandbox (Adobe, 20 October 2026). The vendor that documented the upgrade problem most candidly built its SaaS edition without the mechanism that causes it.

Kestrel & Pine: the app governance rule

Noor and Milan turned the April 2027 spreadsheet into a rule, approved by Mara Lindqvist alongside Waypoint and recorded on one page per app (the **app governance card**, reproduced in the Toolkit: Cards and Templates for Commerce Architecture Decisions). The rule has five lines.

1. **Every app has a named owner** in the business (Milan for storefront and marketing apps, Daniel Okafor's team for fulfillment apps, Sanne de Wit's team for tax and invoicing apps, Ruben Alves for connectors); the agency installs nothing without an owner's approval.
2. **Every app's scopes are recorded** on its card, with the API version it targets and the date that version leaves support.
3. **One writer per fact.** The WMS feed through the iPaaS is the only writer of stock into the platform until the availability service takes over in Q4 2027 (W1); consumer prices change only through the platform's market price lists and wholesale prices only through the ERP's (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax); the OMS will be the only writer of order status (W2). Every other write scope on those facts is revoked.
4. **A quarterly uninstall review.** Any app without an owner, without use in the quarter, or duplicating another is removed.
5. **Custom behavior goes to the lowest rung that works**: configuration, then the platform's functions and workflows, then a public app, then a custom app with an owner.

The results: the six unused apps were uninstalled in April 2027; inventory write access was withdrawn from all three stock-writing apps by June 2027, after the first weekly reconciliation in May had measured the damage (2.3% of active variants with a different quantity in the WMS and the platform, most from those apps); the pre-order app kept read access and records pre-orders without adjusting stock until the availability service handles pre-order quantities. Thirty-four apps remained in July 2027, at about €2,400 a month (illustrative). The saving is not the point. The point is that Noor can now answer, on one page, the question nobody could answer in March 2026: who is allowed to let a program write stock.

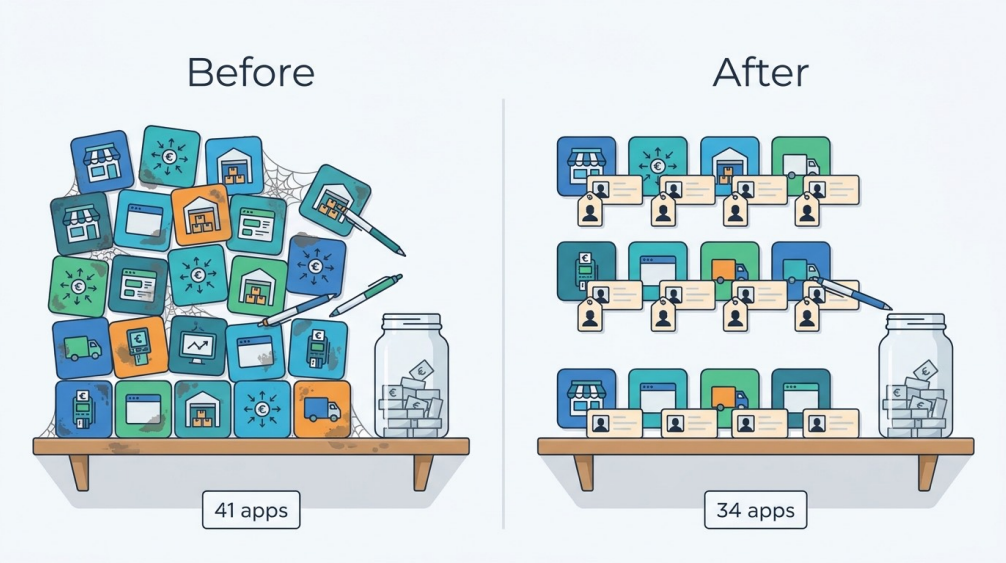


Figure 4: *The April 2027 spreadsheet found 41 apps with nobody's name on them; an owner per app, scopes on a card, one writer per fact and a quarterly uninstall review left 34 by July.*

App governance card	Kestrel & Pine example: the pre-order app
App, vendor, owner	Pre-order app, public, installed 2025; owner Milan Petrov
Purpose	Take pre-orders for next season's Ridgewalker colors
Scopes granted	read_products, read_inventory, write_orders (pre-order tags)
Scopes revoked and why	write_inventory, June 2027: the WMS feed is the only writer of stock (W1)
API version and support end	2027-01; supported at least to January 2028 *(illustrative; mirrors the 12-month rule of hosted platforms)*
Review date	October 2027
Exit plan	Pre-orders handled by the availability service and the OMS from Q4 2027; app removed after the spring 2028 season

When It Goes Wrong

Extensions fail the way integrations fail, with one difference: the merchant often does not know the extension exists.

Failure	What it looks like	Detection	Recovery
Two writers of one fact (the App That Wrote Stock)	Stock, price or status drifts between the platform and its system of record	The reconciliation job (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation) lists variants whose quantity differs; the platform's audit log, where it exists, names the last writer	Revoke the extra write scope; re-set the fact from the system of record; keep the reconciliation weekly
An app lost its webhook	An app stops receiving orders after an outage on its own side; Shopify, for example, deletes a subscription "After 8 consecutive failures" over 4 hours	Orders missing in the app; the platform's webhook delivery log	Re-subscribe; backfill from the API; webhooks need a reconciliation job, not trust

Failure	What it looks like	Detection	Recovery
The withdrawn extension point (the Pulled Rug)	Checkout customizations stop on the vendor's announced date; an app on a retired API version "falls forward" to a newer one with different fields	The deprecation calendar on the cards; the vendor's changelog	Move the logic to the replacement mechanism before the date; after it, the platform's default behavior runs, usually safe and always surprising
The un-upgradable core (the Forked Platform)	Each release is a project; releases are skipped; the support end approaches	The upgrade compatibility report; the module count; the version gap	Audit modules; move what can move out of process; plan the upgrade as a migration (Moving Without Burning the Bridge: Replatforming and Migration)
An app as the attacker's path, or an app nobody maintains	An app with broad scopes, or its vendor, is compromised (Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers); or a capability customers use runs through an app whose vendor has gone quiet (the Lynchpin)	Scope review; vendor assessments; the review date and API-version lag on the card	Revoke and rotate credentials; the exit plan on the card

Every row is found faster, and fixed faster, by a merchant that can list its apps, their owners, their scopes and the facts they write.

What to Take Away

Established.

- Where an extension runs decides what it can touch and what it costs at upgrade time: in-process plugins can reach anything (Adobe, Shopware and WordPress documentation); apps over APIs only what their scopes allow (Shopify, BigCommerce, Saleor); functions and call-outs get a fixed input, a fixed output and a time budget (Shopify Functions, commercetools API Extensions).
- Upgrade cost scales with the number and quality of in-process customizations, and the vendor's support clock turns a postponed upgrade into a forced migration (Adobe Commerce's upgrade guidance and lifecycle policy).
- Hosted platforms withdraw extension points on published dates; Shopify's checkout template and Scripts timeline (2023–2026) is the reference case.
- App review protects the platform and the shopper (least scope, performance, clean uninstall, privacy webhooks); it does not tell a merchant which installed apps write the same fact.

Depends on context.

- Whether in-process extension is acceptable depends on who runs the platform and the size of the team: a merchant on a hosted platform has no choice; one on a suite with a platform team can run a disciplined, budgeted in-process layer; one with a small team and an agency should treat every in-process module as debt.
- How formal app governance must be depends on stage: a list with owners and scopes at Starting; a card per app, a quarterly review and a one-writer rule at Omnichannel; a shared extension catalog and an upgrade calendar at Group.
- Whether to build a custom app or buy a public one depends on whether the need is special: commodity capabilities belong to the store; integrations and business rules belong to a custom app or a function with a named owner.

The author's synthesis.

- I treat every installed app as an integration and give it what Waypoint's W5 gives every integration: an owner, a contract (its scopes and API version), idempotent handling where it writes, and a reconciliation job for the facts it touches. Forty apps are forty integrations.
- I would rather lose a feature than keep a second writer. When a business need seems to require a second app writing stock or prices, the need is real and the mechanism is wrong: the answer is a reservation state, a price list or an availability service, not another writer.

Questions to Ask

1. How many apps, plugins or extensions are installed, and can someone produce the list with an owner per app in an afternoon?
2. For each fact the platform holds (stock, price, product content, order status, customer), which single path writes it, and which apps hold a write scope on it anyway?
3. Which mechanism does each custom behavior use: configuration, a function or call-out, a public app, a custom app, an in-process module, or an edit to the core?
4. On an open-source or suite platform: how many in-process modules are installed, which version is running, and when does standard support for it end?
5. On a hosted platform: which API version does each app and connector target, and which deprecations fall in the next twelve months?
6. What time budget does the platform give our checkout customizations, and what happens to the shopper when they time out?
7. Who approves a new app, and if its vendor stopped maintaining it tomorrow, what is the exit plan on its card?

The apps that extend a platform mostly land on one surface: the storefront, where theme code, app blocks and scripts meet the shopper's browser. The next chapter follows that surface from the platform to the phone, compares themes with headless storefronts, and works through the headless proposal Noor declined in February 2027. Continue with Delivering the Storefront: Themes, Headless, Rendering and Performance.

Sources and Evidence

Company, product and investor sources

- Shopify Inc., Annual Report on Form 10-K for the fiscal year ended December 31, 2025, U.S. Securities and Exchange Commission, filed 11 February 2026 — "single branch" software, "millions" of merchants and "more than 21,000 apps" at year end.
- Shopify Unite 2021 announcements, Shopify Partners blog, 29 June 2021 — theme app extensions and app blocks, uninstalls removing ghost code, Functions in beta.
- The checkout.liquid theme file is being deprecated, Shopify changelog, 13 February 2023 — the 13 August 2024 date and the automatic upgrade to an uncustomized checkout.
- Script Editor, Shopify Help Center, n.d., accessed 2026-11 — Scripts retired on 30 June 2026.
- Shopify Functions and Function APIs, shopify.dev, n.d. (API version 2026-10), accessed 2026-11 — targets, WebAssembly, no network, instruction and memory limits.

- Access scopes, shopify.dev, n.d., accessed 2026-11 — scope examples, the 60-day order window, protected customer data.
- App requirements checklist, shopify.dev, n.d., accessed 2026-11 — least scope, the 10-point Lighthouse rule, the 90-day deprecation rule, billing.
- Revenue share, shopify.dev, n.d., accessed 2026-11 — the 0% to $1,000,000 lifetime threshold and 15% above.
- Privacy law compliance, shopify.dev, n.d., accessed 2026-11 — the three mandatory compliance webhooks and the 30-day completion rule.
- HTTPS webhook delivery, shopify.dev, n.d., accessed 2026-11 — timeouts, eight retries over four hours, subscription deletion after eight consecutive failures.
- API versioning, shopify.dev, n.d., accessed 2026-11 — quarterly versions, the 12-month minimum support and the fall-forward rule.
- Inventory management apps, shopify.dev, n.d., accessed 2026-11 — inventory states and the adjust-versus-set mutations.
- Shopify Flow, Shopify Help Center, n.d., accessed 2026-11 — the workflow tool and its plan gating.
- Web performance, Shopify Help Center, n.d., accessed 2026-11 — theme code and apps named as what can slow a site.
- App marketplace requirements, BigCommerce developer documentation, n.d., accessed 2026-11 — "Apps configured with all scopes aren't approved", checkout and PCI reviews, clean uninstall.
- Extensions: plugins and apps, Shopware developer documentation, n.d., accessed 2026-11 — plugins in process with database access and unsupported in the cloud; apps out of process.
- Plugins (interceptors), Adobe Commerce PHP developer guide, n.d., accessed 2026-11 — before, after and around plugins, ordering and the performance warning.
- Out-of-process extensibility, Adobe Commerce developer documentation, n.d., accessed 2026-11 — the in-process versus App Builder trade-off in Adobe's words.
- Upgrade best practices, Adobe Experience League, n.d., accessed 2026-11 — custom work and modules as the drivers of upgrade complexity.
- Upgrade Compatibility Tool overview, Adobe Experience League, updated 8 October 2026 — what the tool checks and reports.
- Adobe Commerce lifecycle policy, Adobe Experience League, n.d., accessed 2026-11 — about three years of standard support, release dates, the 1 June 2027 cloud enforcement.
- Adobe Commerce as a Cloud Service overview, Adobe Experience League, 20 October 2026 — continuous updates with a 30-day sandbox window; extensibility through App Builder, API Mesh, events and webhooks.
- Extension concept, SAP Help Portal (SAP Commerce 2205), n.d., accessed 2026-11 — "a flexible, modular architecture based on extensions".
- Customize with cartridges, Salesforce Trailhead, n.d., accessed 2026-11 — the cartridge path and the rule never to edit the base storefront.

- API Extensions, commercetools documentation, n.d., accessed 2026-11 — synchronous call-outs, time budgets, limits and failure behavior.
- Apps overview, Saleor documentation, n.d., accessed 2026-11 — apps as external services over webhooks and the API.
- Getting started building extensions, WooCommerce developer documentation, n.d., accessed 2026-11 — "backwards compatibility between WooCommerce releases is not guaranteed"; the 3.6M+ active stores reach figure.

Standards, regulation and official documentation

- Hooks, WordPress Plugin Handbook, n.d., accessed 2026-11 — actions and filters as the in-process mechanism under WooCommerce. No cross-vendor standard governs commerce extensibility; the privacy obligations that reach apps are covered in Whose Data Is It? Privacy, Consent and Retention.

Research and practitioner sources

- Feature Toggles (aka Feature Flags), Pete Hodgson, martinfowler.com, 9 October 2017 — the toggle taxonomy used to decide whether a change belongs in configuration.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

32. Delivering the Storefront: Themes, Headless, Rendering and Performance

IN THIS CHAPTER, YOU WILL: Follow a product page from the platform to the shopper's phone, compare themes with headless storefronts, and learn when owning the storefront is worth it.

KEY POINTS:

- **A storefront is a rendering of facts other systems own.** It shows the catalog, prices, promotions, availability, search results and the cart, and hands the shopper to checkout; it owns none of them. Three rendering models exist (a theme rendered by the platform, a headless storefront on the vendor's runtime, a headless storefront anywhere else), and under all three the checkout stays on the platform.
- **Cache the shell, fetch the promise.** Product and category pages can be cached per market and language, which is what survives a peak day; the price for a logged-in account, live stock, the cart and the checkout

cannot, and are fetched after the cached page renders. The platform's full-page cache and a headless framework's regeneration are the same idea at two layers.

- **Go headless for a need the theme cannot meet, never for speed alone.** The performance evidence is directional, dated and self-selected; a themed storefront with disciplined app scripts can pass Google's thresholds. Headless is justified by a shared front end across brands, a content-led site or an account-priced B2B portal, and it costs a second deployable that a small team must run.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Giant's Boots** — headless storefront architectures are copied from brands with front-end teams of fifty, by merchants with three.
- **The Forked Platform** — a theme edited by hand for years is the storefront's version of a core that cannot take the vendor's updates.
- **The First Snowfall** — the storefront meets forty times its normal traffic once a year, and whether its pages are cacheable decides whether it stands.

Everything else in this book happens behind a screen. The shopper sees one thing: a page, on a phone, that shows a jacket, a price, "in stock", a delivery promise and a button. The **storefront** is the software that produces that page, and the architectural question is not what it looks like but **where it is rendered, from which data, and who owns the machinery**. A storefront rendered by the platform from a theme is one answer; a separate application that fetches everything through APIs (a **headless** storefront, so called because the "head" the shopper sees has been cut off from the commerce engine behind it) is another.

The question matters because the storefront is where the rest of the architecture becomes visible. It reads the catalog, prices, promotions, search, availability and the cart, and if any of those is slow, stale or wrong, the page is. It also carries the merchant's brand, which is why agencies and designers want to own it, and it is where most installed apps land their scripts, which is why it gets slow. **The storefront is the one component every stakeholder wants to change and nobody wants to operate.**

Kestrel & Pine, the fictional outdoor brand this book follows, supplies the worked example: three localized storefronts rendered from one agency-customized theme, 41 apps' worth of scripts, a peak hour of 41 times normal traffic, and the headless rebuild Noor Visser was asked to approve in February 2027 and declined.

Where we are in the order's journey. This chapter is the first step of Eva's jacket: the product page she opened on her phone at 21:14 on 13 April 2027, with its price, its "in stock" badge and its "ordered before 22:00, delivered tomorrow" promise. Every later step (From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment

Providers, Authorization, Capture and Refunds, The Order Record: Lifecycle, State and the Order Management System) begins with what that page showed her and whether it was true.

Setting the Scene at Kestrel & Pine: The Headless Proposal

In February 2027, the agency that has built and maintained Kestrel & Pine's theme since 2017 presented a proposal to Noor Visser, in her second month in the job: rebuild the three storefronts as a headless application on a modern front-end framework, hosted on the platform vendor's own edge runtime (servers spread close to shoppers that run the merchant's front-end code), with a headless content system behind it. The pitch had three arguments: the storefront would be faster, the brand team would get a content model the theme could not give them, and the company would be "ready for the future". The figure, a Kestrel & Pine illustration, was about €240,000 and nine months, plus a second runtime to operate.

Noor did what the next sections of this chapter do. She measured the storefront, listed what each rendering model would change, and asked which of the agency's three arguments described a need the theme could not meet. The product pages' field measurements (the ones Google collects from real visitors, at the 75th percentile on mobile, February 2027, all illustrative) were: largest contentful paint 2.1 seconds, interaction to next paint 310 milliseconds, cumulative layout shift 0.08. Two of the three were already in Google's "good" range. The third, responsiveness, was not, and the agency's own audit traced it to scripts from eleven apps, not to the theme or the platform.

She declined the rebuild. The reasons form the rest of this chapter.

What a Storefront Does, and What It Does Not Own

A storefront produces pages: home, category (also called collection) pages, product pages, search results, content pages, the cart, the account pages and, at the end, the checkout. To produce them it needs facts from almost every other part of the platform. **It owns none of those facts.** It owns the templates, the layout, the brand, and the scripts that run in the browser.

What the page shows	Who owns the fact	Chapter
Product name, description, images, attributes	The catalog, or the PIM (product information management system) once there is one	What Are We Selling? Products, Variants and the Catalog
Price, and the tax-inclusive display for EU shoppers	The pricing engine or price list	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
Discounts and badges	The promotion engine	Discounts, Coupons and Loyalty: Promotions Without Chaos
Search results, facets, sort order	The search index	Finding the Product: Search, Navigation and Merchandising
"In stock", "only 3 left", pre-order	The availability service, or the platform's stock until there is one	Is It in Stock? Inventory, Availability and the Promise You Can Keep
The delivery promise	The shipping service, or a static rule	Getting It There: Carriers, Labels, Tracking and the Delivery Promise
The cart and the checkout	The platform's checkout	From Cart to Confirmation: Checkout Orchestration
Editorial content (stories, guides, campaign pages)	A content management system (CMS), the tool editors use to write and publish pages without engineers	this chapter
Who the shopper is, and which prices they see	Identity and the customer record	One Customer, One Product: Identity and Master Data Across Systems

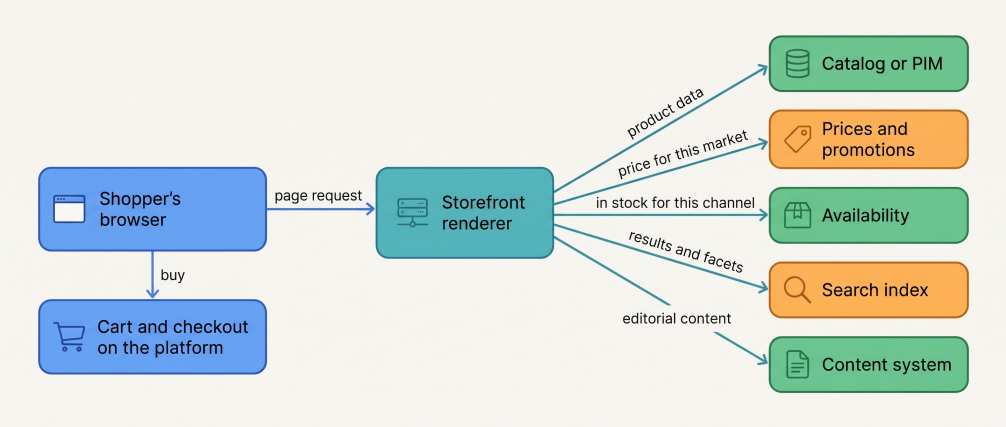


Figure 1: *The storefront renders facts it does not own: product data, prices, availability, search results and content come from other systems, and the buy button hands the shopper to a checkout the platform runs.*

That table is the reason the storefront question is an architecture question. A headless storefront does not move any of those facts; it moves the **rendering** and the **glue** between them into the merchant's hands. Whether that is a gain depends on whether the merchant wanted either.

Three Rendering Models

Two words first. **Server-side rendering** means a server builds the complete page as HTML and sends it; the browser shows it at once. **Client-side rendering** means the server sends a mostly empty page plus JavaScript, and the browser fetches data and builds the page itself. Early headless storefronts were client-rendered single-page applications, which is where their reputation for slow first loads comes from; today's headless frameworks render on the server by default, so the distinction between the models is no longer about rendering technique but about **who runs the renderer and where the templates live**.

Model 1: the theme, rendered by the platform

A **theme** is a set of templates (and, on hosted platforms, the settings and scripts around them) that the platform's own renderer turns into pages. Shopify's Liquid themes, BigCommerce's Stencil ("BigCommerce's Handlebars-based theme engine") and the templates of Salesforce's Storefront Reference Architecture (SFRA) are the dated examples. The platform owns rendering, caching, delivery, security and scale; the merchant owns the templates.

Shopify described its renderer in August 2020 (evidence now six years old): the Storefront Renderer is "a server-side application that loads a Shopify merchant's storefront

Liquid theme" and returns HTML, rewritten from scratch from January 2019 because storefront code "has much stricter performance requirements" than, say, payment processing. It reads from dedicated database replicas and caches at three layers: a query cache in a key-value store, an in-memory cache of the most frequent queries, and a full-page cache. Reported results: server response times "4x to 6x faster than the legacy implementation". In January 2022 the CTO added that the separate renderer could "deploy in regions worldwide without deploying the full Rails monolith".

The theme model's ceiling is the template language and the platform's page types. Its floor is high: a merchant inherits caching, edge delivery and a renderer tuned by a vendor whose business depends on it. Shopify's Online Store 2.0 (announced 29 June 2021) moved theme content into JSON templates with sections on every page, added theme app extensions (so apps add blocks instead of editing templates) and set a performance bar for themes.

Model 2: headless on the vendor's runtime

The platform vendor supplies a storefront **API** and a hosting runtime, and the merchant builds a separate front-end application on it. Shopify's Hydrogen is "Shopify's headless toolkit", not a framework: it adds Shopify-specific clients for "the Storefront API and Customer Account API only" to a framework the merchant chooses (React Router 7 is the starter), deployed to Oxygen, Shopify's hosting (documentation version 2026-10, accessed November 2026); Shopify's February 2023 Editions post made Oxygen available on its main plans "at no extra cost". Salesforce's PWA Kit is "a storefront technology for headless commerce using Salesforce Commerce APIs and React", deployed to a Managed Runtime. Adobe's Commerce as a Cloud Service uses a headless storefront on Edge Delivery Services as its only storefront (Adobe, 20 October 2026).

What stays on the platform in this model is instructive. Hydrogen's documentation says it covers the Storefront and Customer Account APIs and "doesn't include checkout UI, because checkout stays on Shopify"; the headless storefront builds the cart and hands off to the platform's checkout. The vendor keeps the part it is paid for and the part it must keep secure.

Model 3: headless anywhere

The merchant builds the storefront on a general-purpose framework and hosts it where it likes, against the platform's storefront API, possibly against several platforms. Vercel's Next.js Commerce template, "A high-performance, server-rendered Next.js App Router ecommerce application" with Shopify as reference and forks for BigCommerce, Medusa, Saleor and Shopware, was archived and moved to a successor repository (accessed November 2026), a reminder that this layer churns. Alokai (formerly Vue Storefront) sells a **unified data layer** that normalizes "product data from each platform into this common format" through a middleware, for BigCommerce, commercetools, Magento,

SAP and Salesforce (documentation, accessed November 2026): the merchant's front end talks to one shape of product, whichever engine is behind it, which is the Group-stage reason to go headless.

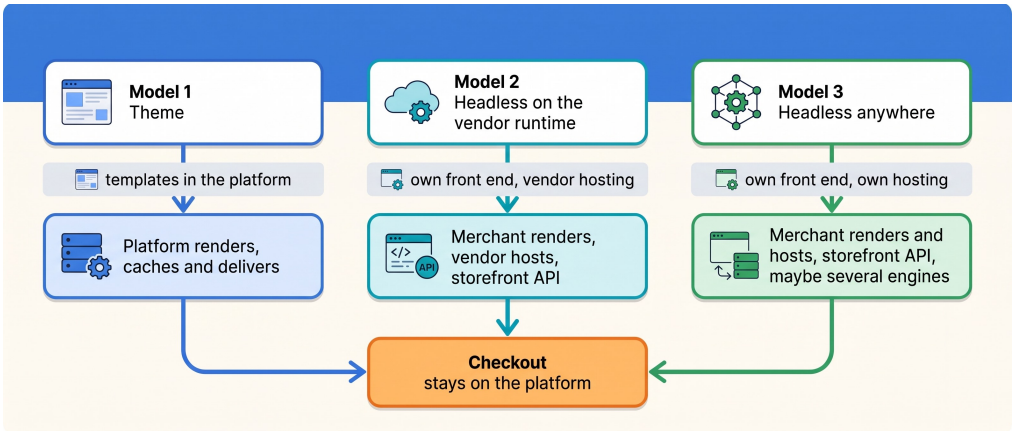


Figure 2: *Three rendering models differ in who renders, who hosts and how many engines the front end can face; under all three the checkout stays on the platform.*

Cache the Shell, Fetch the Promise

A storefront's performance on an ordinary day and its survival on a peak day both come from one split. Some of a page can be computed once and served many times; some must be computed for this shopper, now.

Cacheable: the product page's content, images, attributes and layout, per market and language; category pages; editorial pages; search facets to a degree. These change when the catalog or the campaign changes, not when a shopper arrives.

Not cacheable: the price for a logged-in wholesale account; live stock and the "only 3 left" badge; the cart; the checkout; anything personalized. These are **promises**, in this book's vocabulary, and a cached promise is a lie waiting for its moment, which is exactly what the Price Drift of October 2026 and the Black Friday Oversell were from the page's side.

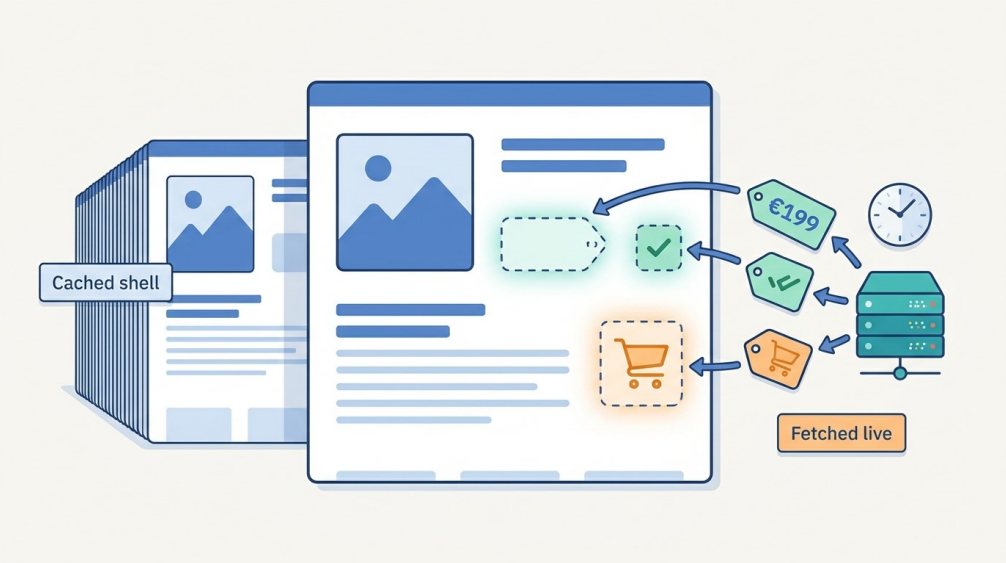


Figure 3: *Cache the shell, fetch the promise: the page content is computed once and served many times, while the price, the stock badge and the cart are fetched for this shopper, now.*

The pattern every model implements is **cache the shell, fetch the promise**: serve the cached page immediately, then fetch the live facts into it. Shopify's full-page cache is this at the platform layer. At the headless layer the same idea appears as **incremental static regeneration**: Vercel's documentation (updated 28 August 2026) describes serving the cached page while regenerating it in the background on a timer or on demand, names e-commerce catalogs "that need current pricing and availability without rebuilding the entire site" as the first use case, collapses concurrent requests for an uncached page "into one function invocation per region, protecting your backend during traffic spikes", and serves stale content with a retry if regeneration fails. Request collapsing is the peak-day feature: when 41 times the normal traffic hits one hero product at once, the backend sees one request, not thousands.

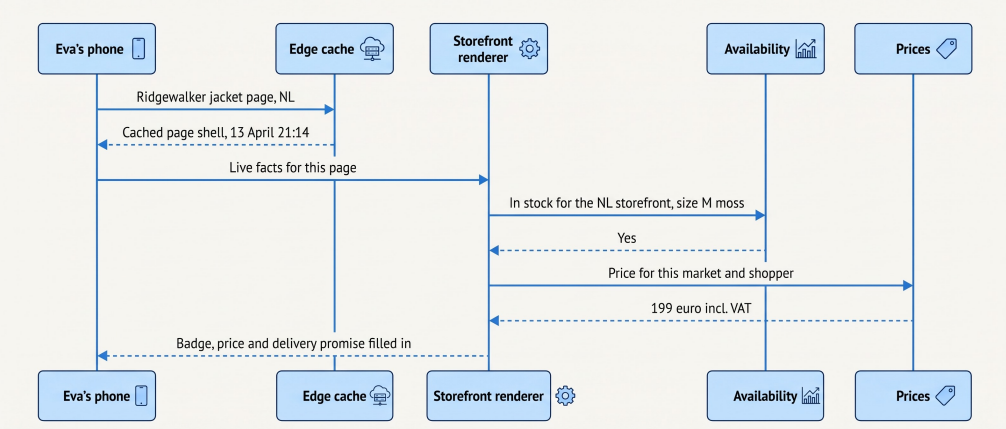


Figure 4: *Eva's product page arrives from the edge cache first; the storefront then fetches the facts that must be true now, availability and price, and fills them in.*

Requirements per page type

Page	How fresh	How consistent	How fast	On a peak day
Product and category content	Minutes after a catalog publish	Same content in every channel (the PIM's job after W3)	Cached at the edge; server response in tens of milliseconds *(assumption)*	Must be fully cacheable; request collapsing on misses
Price shown	Within the price change's effective time; What Does It Cost? Price Lists, Contract Prices, Currencies and Tax	One price per market and channel	Fetched live or cached for seconds, by market	Cache per market, never per shopper, for anonymous visitors

Page	How fresh	How consistent	How fast	On a peak day
Stock badge and delivery promise	Seconds to a minute after a stock event *(assumption)*	Same answer the checkout will give	Fetched after the shell	Degrade to "in stock / out of stock" or hide the count (Black Friday Every Year: Peak Load, Resilience and Recovery)
Search results	Minutes after a catalog publish	Index matches the catalog	Under 300 ms *(assumption)*	Fewer facets, simpler sorts
Logged-in B2B (business-to-business, wholesale) pages	Live	Contract price per account	Never cached across accounts	Rate-limited by account
Cart and checkout	Live	One cart per session; the checkout's rules win	Platform-owned	Never on the switch-off list

Performance: What the Evidence Says, With Dates

Core Web Vitals are Google's three field metrics for page experience, measured from real visitors and assessed at the 75th percentile of page loads, mobile and desktop separately (web.dev, updated 31 October 2024): **largest contentful paint** (LCP, when the main content has loaded; "good" is 2.5 seconds or less), **interaction to next paint** (INP, how quickly the page responds to a tap or click; good is 200 milliseconds or less) and **cumulative layout shift** (CLS, how much the page jumps while loading; good is 0.1 or less). INP replaced the older first-input-delay metric on 12 March 2024. Google's Search documentation says it "highly recommend[s]" good Core Web Vitals "for success with Search" and that they "align with what our core ranking systems seek to reward", without stating a weight (accessed November 2026).

The evidence that speed moves money is consistent in direction and weak in precision:

- **Deloitte Ireland for Google (24 March 2020; six years old)**: across mobile sites in retail, travel, luxury and lead generation, a 0.1-second improvement in site speed went with +8.4% retail conversions and +9.2% retail average order value. Caveats the study carries: commissioned by Google, the number of sites not disclosed, and a correlation, not an experiment.
- **web.dev's case collection (undated, accessed November 2026)**: Vodafone Italy improved LCP 31% and saw sales up 8%; Lazada tripled LCP speed and saw mobile conversion up 16.9%; Cdiscount improved all three metrics and reported +6% revenue in a Black Friday sale. Every case is a self-selected success story, and none separates speed from the redesign that came with it.
- **A vendor's admission (Shopify, accessed November 2026)**: the theme-performance page says performance "directly influences conversion rates, repeat business, and search engine rankings" without citing a figure; the Theme Store requires a minimum average Lighthouse performance score of 60; public apps "shouldn't reduce Lighthouse performance scores by more than 10 points"; and the admin's performance report warns that theme code and apps "can slow down your site".

The honest reading: **faster is better, nobody can tell you by how much, and the usual culprit is not the rendering model but the scripts**. A rule that caps each app at ten points is a vendor admitting that apps are where storefront speed goes. Kestrel & Pine's INP of 310 milliseconds was eleven apps' scripts; the rendering model was innocent.

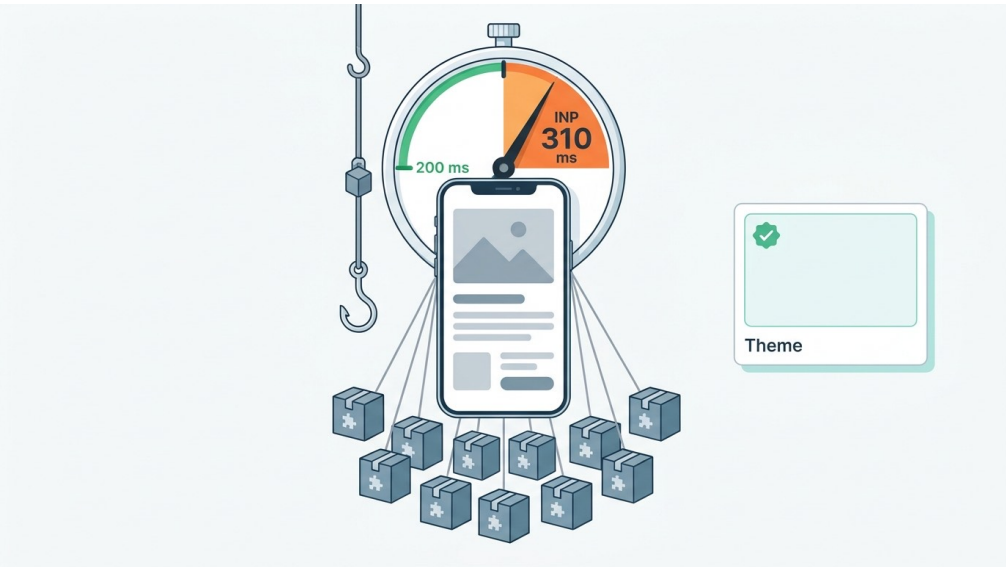


Figure 5: *Kestrel & Pine's interaction to next paint of 310 milliseconds was eleven apps' scripts, not the rendering model; good is 200 milliseconds or less.*

Accessibility is a requirement, not a performance option

WCAG 2.2, the Web Content Accessibility Guidelines of the World Wide Web Consortium (W3C), is a Recommendation dated 12 December 2024, with three conformance levels (A, AA, AAA) and four principles (perceivable, operable, understandable, robust). The **European Accessibility Act** (Directive (EU) 2019/882) applies to services "provided to consumers after 28 June 2025" and lists "e-commerce services" explicitly, defined as a service provided at a distance "with a view to concluding a consumer contract"; microenterprises (under 10 people and up to €2 million turnover) are exempt. Kestrel & Pine, with 310 employees and three consumer storefronts, has been in scope since June 2025; its B2B portal, serving businesses, is not a consumer service. National enforcement differs; Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation owns the calendar. For the storefront the consequence is that accessibility criteria belong in the theme and in every app block, and that the agency's and the apps' output must be tested against them, whichever rendering model is chosen.

Mobile apps and progressive web apps

A **progressive web app** (PWA) is a website "built and enhanced with modern APIs" that can be installed like an app, work offline and send push notifications (web.dev, accessed November 2026). For a merchant it is mostly the headless question in disguise: the same server-rendered storefront can be made installable. A **native app** adds a second storefront that must show the same prices, stock and promises as the first, and is justified by loyalty, push and in-store features, not by speed.

The Choices and Their Trade-Offs

The realistic options for a merchant on a hosted platform are four.

1. **The theme, disciplined.** Keep the platform's renderer; use sections and app blocks instead of edited templates; police app scripts with a performance budget; let the platform's cache and edge do the peak-day work.
2. **The theme plus a content system.** Keep the theme for commerce pages and feed editorial content from a CMS through the platform's content features or a small integration; the brand team gets its content model without a new renderer.
3. **Headless on the vendor's runtime.** Own the front end; let the vendor host it, serve the storefront API and run checkout. The team now owns rendering, caching rules and the glue to search, prices and availability.
4. **Headless anywhere, possibly over several engines.** Own everything in front of the APIs; add a unified data layer if several engines or brands sit behind one front end.

Option	Who runs rendering and caching	Flexibility	Peak-day posture	Team it needs	When it is right
Theme, disciplined	The vendor	The template language and page types	The vendor's cache and edge; apps are the risk	One or two front-end engineers plus an agency	Starting through Omnichannel, unless a need below applies
Theme plus CMS	The vendor, plus a content feed	Content model of the CMS; commerce pages as the theme allows	As above	The same, plus editors	A content-led brand on a themed platform
Headless on the vendor runtime	The merchant renders, the vendor hosts	Full control of the front end; checkout stays the vendor's	The merchant's caching rules; request collapsing if used	Three or more front-end engineers who can own a production application	A design system shared across brands or countries; a B2B portal with account pricing that the theme cannot express
Headless anywhere	The merchant	Full, including several engines behind one front end	Entirely the merchant's	A front-end platform team	Group stage: several brands or engines behind one storefront

The recommendation is tied to assumptions. Options 1 and 2 assume the platform's page types and checkout fit the business, which for a direct-to-consumer (D2C) brand with a B2B portal on the same platform they usually do. Options 3 and 4 assume a team that can run a production application on its own: on-call, caching bugs, framework upgrades every year, and the storefront API's versioning calendar (Promises Between Systems: API Contracts, Webhooks and Versioning). The agency can build it; the question is who runs it in year two.

Three needs justify headless, in my experience: a **shared front end across brands or countries** that the theme model cannot express because each tenant has its own theme; a **content-led site** where the editorial model, not the catalog, drives the page structure; and a **B2B portal** with account pricing, assortments and approval flows that the consumer theme cannot represent (Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals). Speed is not on the list, because a disciplined theme reaches the thresholds and an undisciplined headless storefront misses them.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
1 — Starting (Kestrel & Pine 2016–2018)	A purchased theme with light customization; a handful of apps; the platform's defaults for caching and delivery	Field measurements at the 75th percentile, read monthly; a rule that every app's script is justified	A theme edited by hand "just this once"; LCP drifting past 2.5 seconds after each app install

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
2 — Growing (2019–2022)	An agency-customized theme; two or three storefronts for countries; thirty apps; the first content pages fought over by brand and engineering	A performance budget per page type; sections and app blocks instead of edited templates; a content model decision (theme sections or a CMS)	Theme updates skipped because the merge is painful; brand asking for page types the theme lacks; a headless proposal arriving before any need is measured
3 — Omnichannel (2023–now)	Several localized storefronts in one tenant; a B2B portal; forty apps; a peak hour many times normal; accessibility obligations in force since June 2025	Cacheability per page type as a design rule; the stock badge and the delivery promise fed by the availability service (W1) and the shipping service rather than static text; an app-script audit each quarter; accessibility testing on theme and app blocks; a headless decision made against the three needs, not the pitch	Peak-day slowness traced to uncacheable pages; the B2B portal needing account logic the theme cannot express; a second brand or a native app that must share the design system

Stage	What is typical	What becomes necessary	Warning signs of outgrowing the approach
4 — Group	Several brands or engines; a front-end platform team; design systems; possibly a unified data layer over several commerce engines	One front-end platform (framework, design system, normalization layer) across brands; headless as the shared surface, with each brand's engine behind it; a performance and accessibility standard enforced across brands	Each brand running its own storefront stack; a group-wide campaign needing five implementations; the One Size Fits None pattern when a brand with a different model is forced onto the shared front end

The judgement: at Starting and Growing stages the theme is the right answer and the work is discipline; at the Omnichannel stage headless becomes a real question only when one of the three needs appears; at the Group stage headless becomes the usual answer, because the thing being shared is the front end itself.

In Practice

Dated public examples

- **A platform carving out its renderer for latency (Shopify, 2020–2022).** The Storefront Renderer rewrite is the one documented case of Shopify separating a hot path from its monolith, and it was done for storefront latency, not for team autonomy. The rollout pattern transfers to any merchant replacing a storefront: a verifier compared "status code, headers, and body" of old and new responses on sampled production traffic, with fallbacks to the legacy path. Replace a storefront behind a comparison, not on a date.
- **Checkout stays home under every model (Shopify Hydrogen, docs 2026-10; Adobe Cloud Service, October 2026).** Hydrogen "doesn't include checkout UI"; Adobe's SaaS edition ships a headless storefront but keeps the commerce engine and checkout. A merchant's headless storefront is a new front door, not a new shop.

- **Regeneration as the peak-day mechanism (Vercel, updated 28 August 2026).** Request collapsing to "one function invocation per region" and serving stale content on failed regeneration are the documented behaviors that make a headless storefront survivable; a merchant choosing model 3 should ask its hosting for the equivalent and test it before November.
- **Accessibility as a dated obligation (EU, 28 June 2025; W3C, 12 December 2024).** The European Accessibility Act's application date and WCAG 2.2 are the storefront's two external deadlines, and they apply to the theme and to every app's block alike.

Kestrel & Pine: why Noor said no, and what she said yes to

Noor's answer to the agency in late February 2027 had four parts, recorded as a short decision note that later folded into Waypoint's W4 (stay and surround; no replatforming, with the platform's own point-of-sale (POS) system evaluated in 2028 and the whole decision revisited in 2029).

1. **The speed argument did not survive measurement.** LCP 2.1 seconds and CLS 0.08 were already "good"; INP of 310 milliseconds was the problem, and it was app scripts. The agency's retainer (€14,000 a month) was redirected for one quarter to a theme performance pass and an app-script audit, run alongside the app governance review of April 2027 (Apps, Plugins and Functions: Extensibility and Ecosystem Governance). By June 2027 INP on product pages measured 180 milliseconds (illustrative), inside the threshold, after six unused apps and four redundant scripts were removed.
2. **The content argument was real, and headless was the wrong remedy.** Teun Baas's brand team wanted seasonal guides and stories that the theme's page types handled badly. The remedy was option 2: theme sections for commerce pages, and a content model that waits for the PIM (W3, Q3 2027) so product stories draw on one product record rather than a fifth copy.
3. **"Ready for the future" named no need.** None of the three needs applied: one brand, three countries sharing one design, a B2B portal that the platform's B2B features serve for 62% of wholesale orders, and no native app. A headless rebuild would have been the Giant's Boots: an architecture borrowed from brands with front-end teams of fifty, run by three engineers who also own the apps.
4. **The peak-day argument pointed the other way.** The 2026 peak hour (2,300 orders, 41 times an average hour) was survived by the platform's cache; what failed was the stock sync chain behind the badge. The fix for the badge is the availability service (W1), not a new renderer. Until W1 lands in Q4 2027, the 2027 peak plan hides the stock count on product pages and shows "in stock / low stock" from a per-channel buffer (Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour).

What she said yes to cost about €38,000 of agency time over the quarter (illustrative) instead of €240,000 and nine months, and left the storefront on the one rendering model her team can run. The decision is revisited at the 2029 review together with the POS question, because the one need that could change the answer, a second brand or a store-mode app sharing the design system, is a Group-stage question Kestrel & Pine has not yet reached.

When It Goes Wrong

Failure	What it looks like	Detection	Recovery
A cached promise	A price change or a sell-out shows on the page minutes or hours after the fact; a shopper buys at the old price or orders the last unit twice	Compare what the page showed with what checkout computed; the Price Drift and the Oversell are both this	Cache content, never promises; purge on publish; fetch price and stock after the shell
The uncacheable peak	Pages that depend on a per-shopper fact are rendered fresh for every visitor; the renderer or the APIs behind it saturate at 41 times normal	Cache hit rate per page type in the load test (Black Friday Every Year: Peak Load, Resilience and Recovery)	Make product and category pages cacheable per market; collapse requests on misses; degrade the badge
Script sprawl	INP and LCP drift with every app install; checkout steps slow or break when a script errors	Field metrics at the 75th percentile, monthly; the platform's app performance rule	A performance budget; the quarterly app-script audit; app blocks instead of injected scripts

Failure	What it looks like	Detection	Recovery
The forked theme	Theme updates skipped for years; security and feature updates unavailable; every change a merge	The gap between the installed and the current theme version; the count of hand-edited templates	Rebuild the customizations as sections and app blocks on a current theme; treat the theme like code, with versions
The headless storefront that drifted	A front end pinned to an old storefront API version breaks on the vendor's retirement date; a framework upgrade stalls for a year	The API version on the integration card (Promises Between Systems: API Contracts, Webhooks and Versioning); the framework's support calendar	Treat the storefront as the integration it is: an owner, a version pin, an upgrade calendar
Bots at the front door	Crawlers and AI agents read the catalog at a rate that costs money or starves real shoppers; Commerce.com's (formerly BigCommerce) second-quarter 2026 filing reported lower gross margin, and an earnings-call summary attributed part of it to hosting costs from AI crawlers and agents (6 August 2026)	Traffic by source; the storefront API's bot limits (Shopify limits bots "most strictly when unsigned")	Bot rules and signed agent access; a catalog feed for agents instead of page scraping (Assistants and Agents: AI in Commerce and What Is Real in 2026)

Failure	What it looks like	Detection	Recovery
An inaccessible storefront	A complaint or an enforcement notice under the Accessibility Act; checkout unusable with a screen reader	Automated and manual WCAG 2.2 testing of theme and app blocks	Fix in the theme and in every app block; add accessibility to the app review on the governance card

What to Take Away

Established.

- Core Web Vitals are defined and thresholded by Google (LCP 2.5 s, INP 200 ms, CLS 0.1, at the 75th percentile; INP since 12 March 2024) and are recommended for Search success; the business evidence (Deloitte 2020; web.dev cases) is consistent in direction and imprecise in size.
- Under every rendering model the vendor keeps checkout: Hydrogen ships no checkout UI; Adobe's Cloud Service keeps the engine behind its headless storefront.
- Cacheable content and live promises must be split; the platform's full-page cache (Shopify, 2020) and incremental regeneration with request collapsing (Vercel, 2026) are the two documented mechanisms.
- The European Accessibility Act applies to consumer e-commerce services since 28 June 2025, with WCAG 2.2 (December 2024) as the technical reference; app scripts are the usual cause of storefront slowness (Shopify's ten-point rule).

Depends on context.

- Whether to go headless depends on one of three needs being present (a shared front end across brands or countries, a content-led site, a B2B portal the theme cannot express) and on a team that can run a production application; absent both, the theme is right at every stage up to Group.
- Where editorial content lives depends on how much of the site the brand team drives: theme sections suffice for a catalog-led brand; a CMS feeding the theme suits a content-led one; a headless front end is the answer only when content drives the page structure of the whole site.
- How aggressively to cache depends on the business model: an anonymous D2C storefront can cache nearly everything per market; a B2B portal with account prices can cache almost nothing across accounts.

The author's synthesis.

- I treat the storefront as an integration consumer with a brand on top: it owns no fact, so the architectural work is deciding which facts are fetched live, which are cached, and who runs the renderer. Decide that before the framework.
- I decline headless proposals that lead with speed. The speed is in the scripts and the cache, which the theme already has; the real reasons for headless are organizational (brands, content, B2B), and a merchant that does not have one should keep the renderer the vendor runs and spend the money on the availability service that makes the badge true.

Questions to Ask

1. What do the field Core Web Vitals say at the 75th percentile on mobile for the product and category pages, and when were they last read?
2. Which rendering model is in use, who runs the renderer, and who is on call when it fails on a Friday evening in November?
3. Which facts on the product page are cached, for how long, and which are fetched live; is the stock badge one of the live ones?
4. How many hand-edited theme templates exist, and when was the theme last updated from the vendor's version?
5. How many apps add scripts to the storefront, and what is the performance budget each must meet?
6. If a headless storefront is proposed, which of the three needs (shared front end, content-led site, B2B portal) does it serve, and who will run it in year two?
7. Which storefront API version does the front end target, and when does that version leave support?
8. Has the storefront, including every app block and the checkout, been tested against WCAG 2.2, and who owns the result?
9. What does the page show for stock and delivery when the availability service or the shipping service is down?

The storefront is where the peak arrives first: forty-one times the usual traffic hits the pages this chapter described, and whether they are cacheable decides whether the shop stands. The next chapter takes that day apart, from load testing and backpressure to the order in which features are switched off, and returns to Kestrel & Pine's Black Friday hour by hour. Continue with Black Friday Every Year: Peak Load, Resilience and Recovery.

Sources and Evidence

Company, product and investor sources

- How Shopify Reduced Storefront Response Times with a Rewrite, Shopify Engineering (Maxime Vaillancourt), 20 August 2020 — the Storefront Renderer, its three cache layers, the 4x–6x result and the verifier-and-fallback rollout; six years old.
- Cloud, Load, and Modular Code: What 2022 Looks Like for Shopify, Shopify Engineering, 20 January 2022 — the renderer deployed in regions without the monolith.
- Shopify Unite 2021 announcements, Shopify Partners blog, 29 June 2021 — Online Store 2.0, JSON templates and sections, theme app extensions, Hydrogen and Oxygen previewed.
- Shopify Editions, Shopify Partners blog, 9 February 2023 — Oxygen hosting on the Basic, Shopify, Advanced and Plus plans at no extra cost.
- Hydrogen, shopify.dev, documentation version 2026-10, accessed 2026-11 — Hydrogen as a toolkit, the APIs it covers, no checkout UI, the React Router 7 starter and Oxygen.
- Storefront API, shopify.dev, n.d., accessed 2026-11 — access modes, no fixed limit for real buyers, bots limited most strictly when unsigned, versioned paths.
- Theme performance best practices, shopify.dev, n.d., accessed 2026-11 — Core Web Vitals as a ranking signal, the Lighthouse 60 minimum, app scripts.
- App requirements checklist, shopify.dev, n.d., accessed 2026-11 — the ten-point Lighthouse rule for public apps.
- Web performance, Shopify Help Center, n.d., accessed 2026-11 — theme code and apps as what can slow a site.
- Stencil, BigCommerce developer documentation, n.d., accessed 2026-11 — the Handlebars-based theme engine.
- PWA Kit, Salesforce Commerce Cloud, GitHub, n.d., accessed 2026-11 — the React storefront on a Managed Runtime.
- Adobe Commerce as a Cloud Service overview, Adobe Experience League, 20 October 2026 — the headless Commerce Storefront on Edge Delivery Services as the only storefront.
- Adobe Commerce Storefront, Adobe Experience League, n.d., accessed 2026-11 — drop-in components and document-based authoring.
- Next.js Commerce, Vercel, GitHub, n.d., accessed 2026-11 — the server-rendered template, its platform forks, and its archiving.
- Incremental Static Regeneration, Vercel documentation, updated 28 August 2026 — stale-while-revalidate, the e-commerce use case, request collapsing and failure behavior.
- Unified Data Layer, Alokai documentation, n.d., accessed 2026-11 — normalizing product data across platforms.

- Commerce.com, Form 8-K exhibit 99.1, second quarter 2026 results, U.S. Securities and Exchange Commission, 6 August 2026 — revenue and gross-margin figures; the attribution of hosting costs to AI crawlers comes from an earnings-call summary (Nasdaq, 2026), not from the exhibit.

Standards, regulation and official documentation

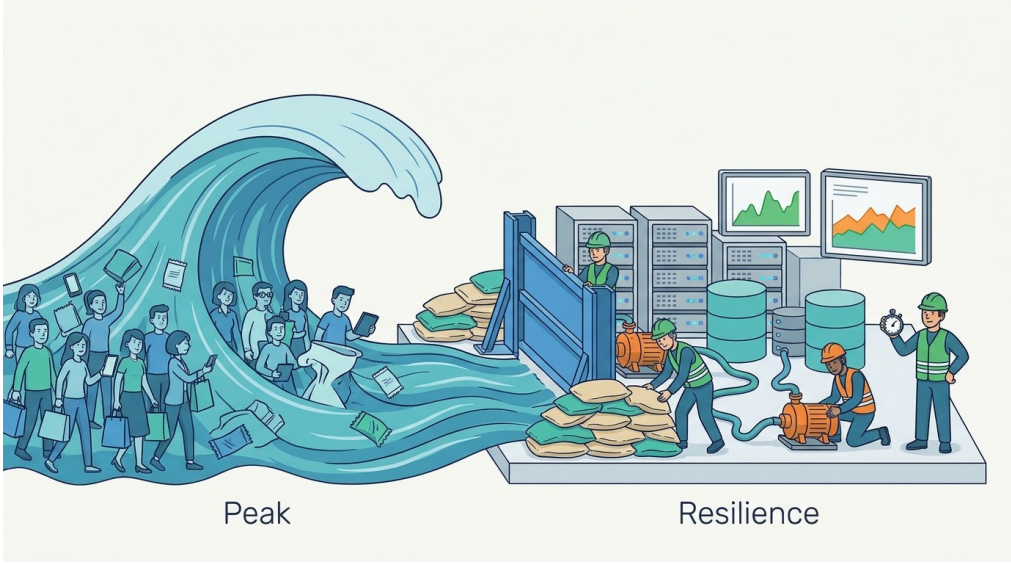
- Web Vitals, web.dev (Google), updated 31 October 2024 — the three metrics, their thresholds and the 75th-percentile rule.
- Interaction to Next Paint becomes a Core Web Vital on March 12, web.dev, 2024 — INP replacing first input delay.
- Understanding Core Web Vitals and Google search results, Google Search Central, n.d., accessed 2026-11 — "highly recommend" and alignment with ranking systems, without a weight.
- What are Progressive Web Apps?, web.dev, n.d., accessed 2026-11 — the PWA definition.
- Web Content Accessibility Guidelines (WCAG) 2.2, W3C Recommendation, 12 December 2024 — levels, principles and the current edition.
- Directive (EU) 2019/882, the European Accessibility Act, Publications Office of the European Union, 7 June 2019 — e-commerce services in scope, application from 28 June 2025, the microenterprise exemption.

Research and practitioner sources

- Milliseconds Make Millions, Deloitte Ireland for Google, 24 March 2020 — the 0.1-second study and its caveats; six years old.
- The business impact of Core Web Vitals, web.dev, n.d., accessed 2026-11 — the self-reported case studies quoted as illustrations.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

33. Black Friday Every Year: Peak Load, Resilience and Recovery



IN THIS CHAPTER, YOU WILL: Learn how a peak day breaks a commerce chain, how to size, test, protect and observe that chain in orders per minute, and how to recover afterwards.

KEY POINTS:

- **The platform's peak is not your peak.** A hosted platform smooths thousands of merchants' spikes into a gentle curve; your own connectors, warehouse system and ERP see your spike alone. The storefront usually holds; the chain behind it is where the day is lost.
- **Measure the whole chain in one unit, orders per minute, and protect it with queues and switches.** Every link gets a capacity figure, a load test and a place in a written switch-off order. Checkout, payment and the order record are never on that list; everything behind them may wait.
- **Recovery is reconciliation.** A peak leaves orders accepted but not exported, stock sold twice and messages dropped in retries. The plan

names, before the day, who compares which two systems the morning after and how the differences get fixed.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The First Snowfall** — the peak is the first time the integrations, not just the storefront, meet real load, and nobody has measured what they can carry.
- **The Phantom Stock** — every sync interval that is harmless on a quiet day becomes, at forty times the pace, a window in which every channel sells the same last unit.
- **The Message in a Bottle** — retries, throttles and timeouts at peak produce the duplicates and gaps that quiet days never show.

Most of the year, a commerce platform is bored. Kestrel & Pine's three storefronts take about **800 orders on an ordinary day**, around 33 an hour, and every system in the chain has hours of slack. Then, one Friday evening in November, the same chain is asked to take **2,300 orders in an hour**. Nothing in it was bought for that hour, and most of it was never tested at it.

A peak is not a bigger ordinary day. It is a different day, on which the slow links of the chain stop hiding behind the fast ones. The vendor-hosted storefront, built to serve millions of merchants, usually holds. The hand-built flow that carries orders to the warehouse, the hourly job that copies stock back and the people picking parcels were sized for 33 orders an hour. At 38 a minute each becomes a bottleneck in turn, and each bottleneck is a promise broken to a customer: a jacket sold that was already gone, a confirmation that arrives late, a parcel that misses the date on the page. This chapter is written for a merchant that does not own its platform, which is most merchants.

Where we are in the order's journey. Eva's jacket, the order this book follows, was placed on an ordinary Tuesday evening in April 2027 and reached the warehouse within minutes. This chapter asks what happens to Eva's order when two thousand other Evas place theirs in the same hour: the same steps, at a pace none of the handoffs was designed for.

Setting the Scene at Kestrel & Pine: Reading the Post-Mortem

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), had its First Snowfall on **Friday 27 November 2026**. In March 2027, Noor Visser, Head of Engineering since January, is reading the post-mortem the team wrote in December, because Mara Lindqvist, the CEO, has asked a one-line question: *will it happen again on 26 November 2027?*

The facts are the ones the whole book reuses. In the week of **23–29 November 2026** the storefronts took **31,000 orders**, 10.6% of the year's. The peak hour, **Friday 20:00–21:00**, brought **2,300 orders**, **38 a minute**, with storefront traffic about **41 times** an average hour. **The platform held**: no page failed, no checkout timed out, every payment was authorized. The chain behind it did not:

- The **stock sync chain** ran as it always ran. The WMS (warehouse management system, the software that runs the Duiven warehouse) reported stock to the ERP (enterprise resource planning system: the books, purchasing and the item master) every 15 minutes; the ERP copied it to the platform hourly; the platform copied it to the marketplace connector hourly. A sale at Duiven took **up to two hours** to reduce the stock shown anywhere else. The Ridgewalker jacket, the hero of the week's 20% outerwear promotion, sold out in the warehouse at **19:40**; the storefronts and marketplaces kept selling it for up to two hours. **1,340 units** were oversold and **1,120 orders** canceled or delayed, at a cost of about **€96,000** including goodwill vouchers and marketplace penalties.
- The **order flow to the warehouse**, a flow on the iPaaS (integration platform as a service, the hosted tool that runs the data flows between systems), created orders in the WMS one at a time, several API calls each, and was throttled by the WMS vendor's **rate limit** (the maximum number of calls a vendor lets one customer make in a time window). By 21:00 the backlog was **four hours** long; orders placed at 20:00 reached the warehouse after midnight, and the "ordered before 22:00, delivered tomorrow" promise on the Dutch storefront, a sentence on a page rather than a computed figure, was broken for roughly 8,000 orders (a Kestrel & Pine figure from Pick, Pack, Ship: Warehouse Management and Fulfillment).

Nothing broke. Every system did what it was configured to do, at the rate it was configured to do it. **The chain had never been measured in one unit, so nobody knew which link would be slowest at 38 orders a minute, and nobody had the switches to protect it when it was**. Journey 4: Black Friday, Hour by Hour tells the day hour by hour and lays out the 2027 plan in full; this chapter explains the ideas the plan is built from.

What a Peak Is, and Why the Merchant's Is Worse Than the Platform's

A **peak** is a period, planned or not, in which traffic and orders are many times their normal level. Peaks have different shapes, and the shape decides what breaks.

Peak profile	Shape	What it stresses most
Seasonal peak (Black Friday, Singles' Day, pre-Christmas)	Days of elevated load with hour-long spikes; a known date	Everything in sequence: storefront, checkout, order export, warehouse, carriers, returns three weeks later
Launch or drop	Minutes of extreme load on one product at a known moment	Stock accuracy and fairness: one SKU (a sellable variant), thousands of buyers, a few hundred units
Flash sale	A short promotion concentrating demand on a few products	Checkout throughput, promotion rules, stock on the discounted items
Unplanned moment (a TV mention, a viral post)	No warning	Whatever has no headroom; usually the first integration

The figures that circulate about peaks come from the largest operators, and they teach one lesson above all: **each reports in its own unit**. Alibaba's cloud arm reported **583,000 orders per second** 26 seconds after midnight on 11 November 2020, without defining "order" (Alibaba Cloud, 2020). Zalando reported **7,200 orders per minute** at its 2019 Cyber Week peak (Zalando Engineering, 2020; six years old, so historical). Shopify reported **$5.1 million of sales per minute** at 12:01 p.m. EST on Black Friday 2025 against a weekend total of **$14.6 billion**, with **489 million requests per minute at the edge** and **117 million on its app servers**, all unaudited (Shopify, December 2025). Adobe put US Cyber Monday 2024 at **$13.3 billion**, with **$15.8 million per minute** in the peak hours (Adobe, December 2024).

Two things in Shopify's figures matter more than their size. First, the ratio of edge to app-server requests, about **four to one**, means roughly three quarters of peak traffic was absorbed by the content delivery network (**CDN**, the layer of servers near the shopper that serves cached pages and images) before it reached application code. Second, the peak minute ($5.1 million) against the weekend average (about $2.7 million a minute over roughly 5,400 minutes) is a **peak-to-average ratio of about two**. Kestrel & Pine's ratio in orders was about **seventy** (2,300 in the peak hour against 33 in an average one). That is the central fact of this chapter:

> **The platform's peak is not the merchant's peak.** A hosted platform serving a million merchants smooths their spikes into a curve. Each merchant's own connectors, ERP, WMS and warehouse staff see that merchant's spike, alone and in full.

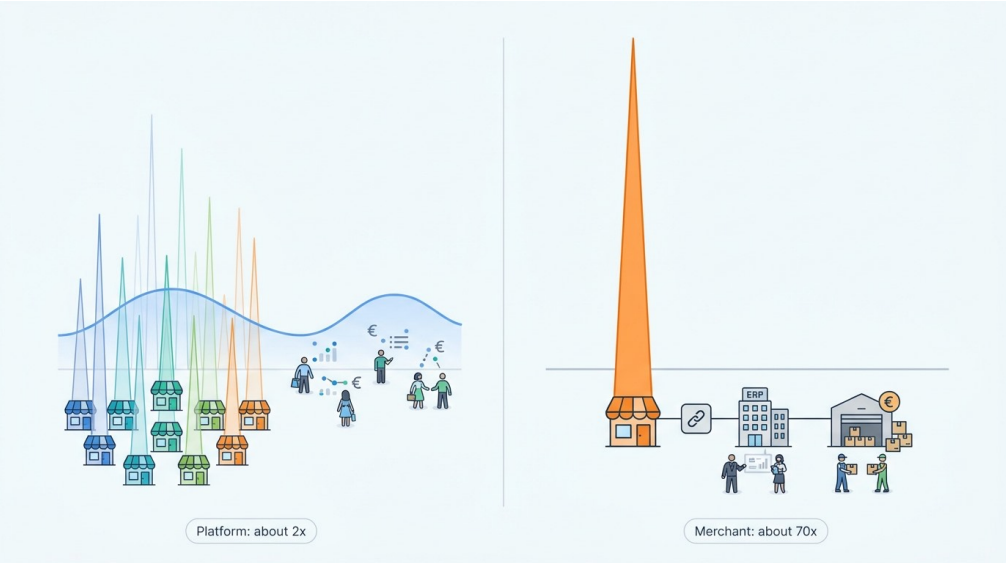


Figure 1: *The platform's peak is not the merchant's peak: a hosted platform smooths a million merchants' spikes to a peak about twice its average, while Kestrel & Pine's own chain saw about seventy times an average hour.*

Multi-tenancy (Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation) gives the platform its smoothing; the merchant's chain gets none, and it is what this chapter sizes.

The Chain Under Load: Where a Peak Day Breaks

A commerce chain is a sequence of handoffs, and a peak day runs down it from the front. The figure shows Kestrel & Pine's chain in November 2026 with the capacity each link turned out to have; the per-link figures are illustrative reconstructions consistent with Kestrel & Pine's recorded totals.

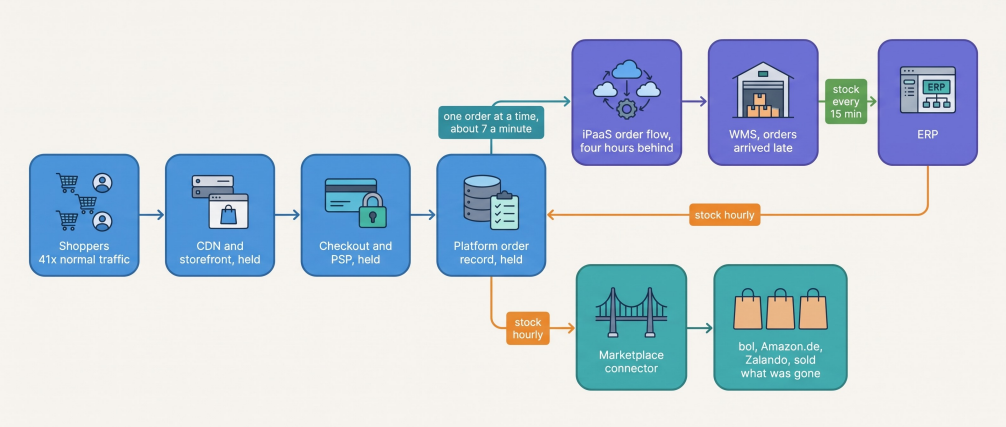


Figure 2: *Kestrel & Pine's chain on 27 November 2026: the vendor-hosted links at the front held at 38 orders a minute, the hand-built order flow behind them carried about seven, and the stock copied back around the loop was up to two hours old by the time the marketplaces saw it.*

Reading the chain from the front:

The storefront. Catalog pages, images and scripts are mostly cacheable, and a hosted platform's CDN serves them without touching the application. This is why "the platform held" is the normal outcome at Omnichannel scale and why the storefront is the wrong thing to worry about first. Delivering the Storefront: Themes, Headless, Rendering and Performance covers what can be cached: a product page, yes; the stock badge and a wholesale account's price, only with care.

Checkout and payment. Checkout is the most protected part of any platform, because it is where the money is. The **PSP** (payment service provider, the company that takes the shopper's payment) has its own rate limits: Stripe's documentation asks merchants planning flash sales that may exceed them to tell support in advance (Stripe, n.d., accessed November 2026). Shopify described in a 2018 changelog that under high sales volume "processing payments at checkout happens separately from creating orders", so the shopper finishes checkout at once and the order record and confirmation e-mail catch up afterwards (Shopify changelog, November 2018; eight years old, and today's "checkout throttle" is not publicly described). The idea transfers whatever platform you use: **take the money and the commitment fast; let the record-keeping queue.** From Cart to Confirmation: Checkout Orchestration and Taking the Money: Payment Providers, Authorization, Capture and Refunds own the details.

The order export. Here the vendor-hosted part ends and the merchant's part begins. The order has to reach the WMS, usually through a connector or an iPaaS flow the merchant or its agency built. This link is almost always the first to fall over, for three reasons that compound: it processes orders one at a time with several API calls each; it runs under rate limits at both ends, the platform's (shared with every installed app, which are busiest when the store is) and the WMS vendor's; and it was tested, if at all,

at the rate of an ordinary afternoon. A flow carrying seven orders a minute against 38 arriving builds a queue of 31 a minute, invisible unless someone built a gauge for it.

The WMS and the people. The warehouse system is usually a hosted product with capacity to spare; the constraint is pickers and carrier collections. Pick, Pack, Ship: Warehouse Management and Fulfillment owns this, and Getting It There: Carriers, Labels, Tracking and the Delivery Promise owns the delivery promise, which in 2026 no system could withdraw when the warehouse fell behind.

The stock loop. Stock has to come back around the loop to every channel before the next sale. On a quiet day a two-hour loop sells nothing twice, because nothing sells out in two hours. On a peak day the loop is the oversell. Is It in Stock? Inventory, Availability and the Promise You Can Keep owns **available-to-promise** (ATP: physical stock minus what is already promised) and the fix; here the point is that *every* sync interval is a peak-day risk in proportion to the sales rate.

The ERP, the data warehouse and the notifications come last and can wait longest. An invoice posted at 03:00 instead of 21:00 costs nothing; a confirmation e-mail an hour late costs a support ticket. They are where queues should grow.

The requirements table makes the ordering explicit; the latencies are the author's illustrative targets for an Omnichannel merchant.

Flow	Freshness needed at peak	Consistency	Latency target *(assumption)*	If it cannot keep up
Product page and search	Minutes (cached)	Eventual	Under 1 s	Serve cached; drop personalization
Stock shown on the product page	Minutes, with a buffer	Eventual, but bounded	Under 2 min from the WMS event	Show "low stock" from the buffer; never "in stock" from a stale copy
Checkout, payment authorization, order record	Immediate	Strong (one order, one payment)	Under 3 s	**Never degrade**; queue everything after it

Flow	Freshness needed at peak	Consistency	Latency target *(assumption)*	If it cannot keep up
Order to the WMS	Minutes	At-least-once, idempotent	Under 10 min from order to warehouse	Let the queue grow; widen the delivery promise
Stock from the WMS to every channel	Minutes	Eventual with a per-channel buffer	Under 2 min	Pause marketplace listings of low-stock SKUs
Notifications, ERP import, invoicing	Minutes to hours	At-least-once; daily batch for finance	Under 15 min for e-mails; overnight for the ERP	Delay; never duplicate

Preparing: Capacity, Forecasts and Load Tests

Capacity planning is deciding, before the day, how many orders a minute each link must carry and proving it can. The input is a forecast; the proof is a **load test**, deliberately sending realistic artificial traffic to find where the chain breaks before customers do.

The two best-documented programs are a platform's and a retailer's. Shopify's 2025 readiness post describes preparation that "started in March with capacity planning", five full-scale tests from April to October on production infrastructure at around **150% of the previous year's load**, scripted browse, cart and checkout flows with flash-sale bursts layered on top, and **Game Days** that injected faults into "critical journeys" (checkout, payment processing, order creation, fulfillment, search) and recorded the findings in a Resiliency Matrix of vulnerabilities, runbooks and recovery objectives. Early tests saw "checkout queues backed up", and adding authenticated checkout "exposed rate-limit paths that anonymous browsing never touches" (Shopify Engineering, November 2025). Its 2023 post explains why tests "bypass the CDN for a portion of the traffic": so that the origin, not the cache, is measured (Shopify Engineering, January 2023).

Zalando load-tested its production website end to end in 2019 because team-level tests could not show whether the whole system held together: recorded shopper journeys were replayed to about **130,000 requests per second**, above the real Cyber Week peak, and broke on replay because sessions expired, items sold out and rate limiting blocked requests, which is itself a finding (Zalando Engineering, April 2019; seven years old). Its 2020 retrospective adds the practice a merchant can copy most directly: **order-flow**

simulators that place test orders which fulfillment skips, exercising inventory and payments without shipping anything (Zalando Engineering, 2020).

For a merchant on a hosted platform, the translation is short:

1. **Forecast in orders per minute**, from last year's peak hour and this year's promotion plan. One and a half times last year's peak is a defensible figure for a growing merchant; the forecast's owner is whoever owns the promotion calendar, because the promotion makes the peak.
2. **Test the journeys, not the servers**. Place marked orders at the forecast rate and watch them reach the WMS, the stock come back, the e-mails go out and the ERP import run. Mark the orders so the warehouse skips them and finance excludes them.
3. **Test the authenticated and integrated paths**. Logged-in wholesale accounts, the marketplace connector's order pull and every app that writes stock take API budget that anonymous browsing does not.
4. **Rehearse failure on the money paths**. Switch off the WMS connector for thirty minutes in a test and see whether anyone notices and whether replay works; have the PSP decline every tenth test payment; zero the stock feed. Each exercise finds a gap or proves a runbook.
5. **Fix the slowest link and repeat** until every link carries the forecast with headroom.

A **freeze** (no new apps, theme edits, connector or configuration changes after a set date) is the cheapest reliability measure available, because the most common cause of a peak-day incident is a change made the week before.

Running the Peak: Queues, Backpressure, Caches and Switches

Preparation removes most surprises; it does not remove all of them. Four mechanisms let a chain survive the surprises that are left.

Queues and backpressure

A **queue** is a buffer in which work waits until the next system can take it. **Backpressure** is the discipline of letting a slow downstream system slow the producers, or letting work pile up in a queue, instead of failing the customer-facing step. The AWS Builders' Library states the principle: a system given more work than its resources support becomes slow and should "start rejecting excess requests so that it can focus on the requests it decides to let in"; what matters is **goodput**, "the subset of the throughput that is handled without errors" and fast enough to be useful (AWS, December 2019).

In a commerce chain the principle has a specific shape. **Accept and persist the order fast**; **queue everything downstream**: the confirmation e-mail, the ERP export, the WMS release, the marketplace stock push. Zalando's 2015 practice of pausing non-essential asynchronous processing to protect order acceptance is this rule in production (Zalando Engineering, 2020). Kestrel & Pine's order flow *had* a queue in 2026, on the iPaaS; what it lacked was a gauge, an alarm, an owner and a plan for when the gauge rose. A queue nobody watches is a delay nobody announces.

Queues also create the duplicates and gaps of **The Message in a Bottle**. A flow that times out and retries delivers the same order twice unless the receiver is **idempotent** (handling a repeated message has the same effect as handling it once), with the order number as the key. A message that fails every retry lands in a **dead-letter queue** (a holding queue for messages that could not be processed, so a person can look at them), which needs an alarm and an owner or it is a silent order-loss mechanism. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation owns these mechanics.

Caching

A **cache** is a stored copy served instead of recomputing or refetching. On a peak day, what not to cache is the question: a cached product page is a gift; a cached stock figure is a Phantom Stock waiting to happen. My rule: a cached stock badge may be minutes old only if it reads "low stock" under a threshold, never "in stock" from a copy older than the sales rate allows.

Waiting rooms

A **waiting room** is a virtual queue in front of the storefront that admits shoppers at a set rate when traffic exceeds a threshold. Cloudflare launched its Waiting Room in January 2021 after vaccine registration sites failed under demand; it is configured with a maximum number of active users, a rate of new users per minute and a session duration (Cloudflare, January 2021; configuration reference accessed November 2026). Queue-it, a specialist vendor, redirects visitors only when traffic exceeds a set capacity and lets operators adjust the outflow during the event (Queue-it, n.d., accessed November 2026).

A waiting room converts an unknown spike into a known outflow rate. That is useful only if the merchant knows its end-to-end capacity in orders per minute, and it protects the storefront, not the chain: **it does nothing for an order export that falls over at 2,000 orders an hour**. For a hosted-platform merchant it is a launch-day tool more than a Black Friday one, and its admission rate should be set from the slowest tested link, usually behind the platform.

Graceful degradation and the switch-off order

Graceful degradation is switching off non-essential features in a planned order so the essential ones keep working. The mechanism is a **feature switch** (also called a feature flag or toggle: a setting that turns a behavior on or off at runtime without a deployment). Pete Hodgson's taxonomy calls the relevant kind **ops toggles** and notes that long-lived kill switches let operators "gracefully degrade non-vital system functionality" under high load (Hodgson, martinfowler.com, October 2017).

The documented case of degradation under fire is Amazon's Prime Day 2018, as reported by CNBC from internal documents: within fifteen minutes of the noon launch the site switched to a simplified fallback front page, at 12:15 international traffic was cut off to relieve a failing internal system, and at 12:37 the default page was restored for a quarter of traffic (CNBC, July 2018; a press account, eight years old). The fallback page and the traffic cut were switches that existed before the day. The team had somewhere to go.

A merchant's switch-off order is shorter. The figure gives the ladder I recommend for an Omnichannel merchant on a hosted platform; Journey 4: Black Friday, Hour by Hour gives Kestrel & Pine's 2027 version with owners and triggers.

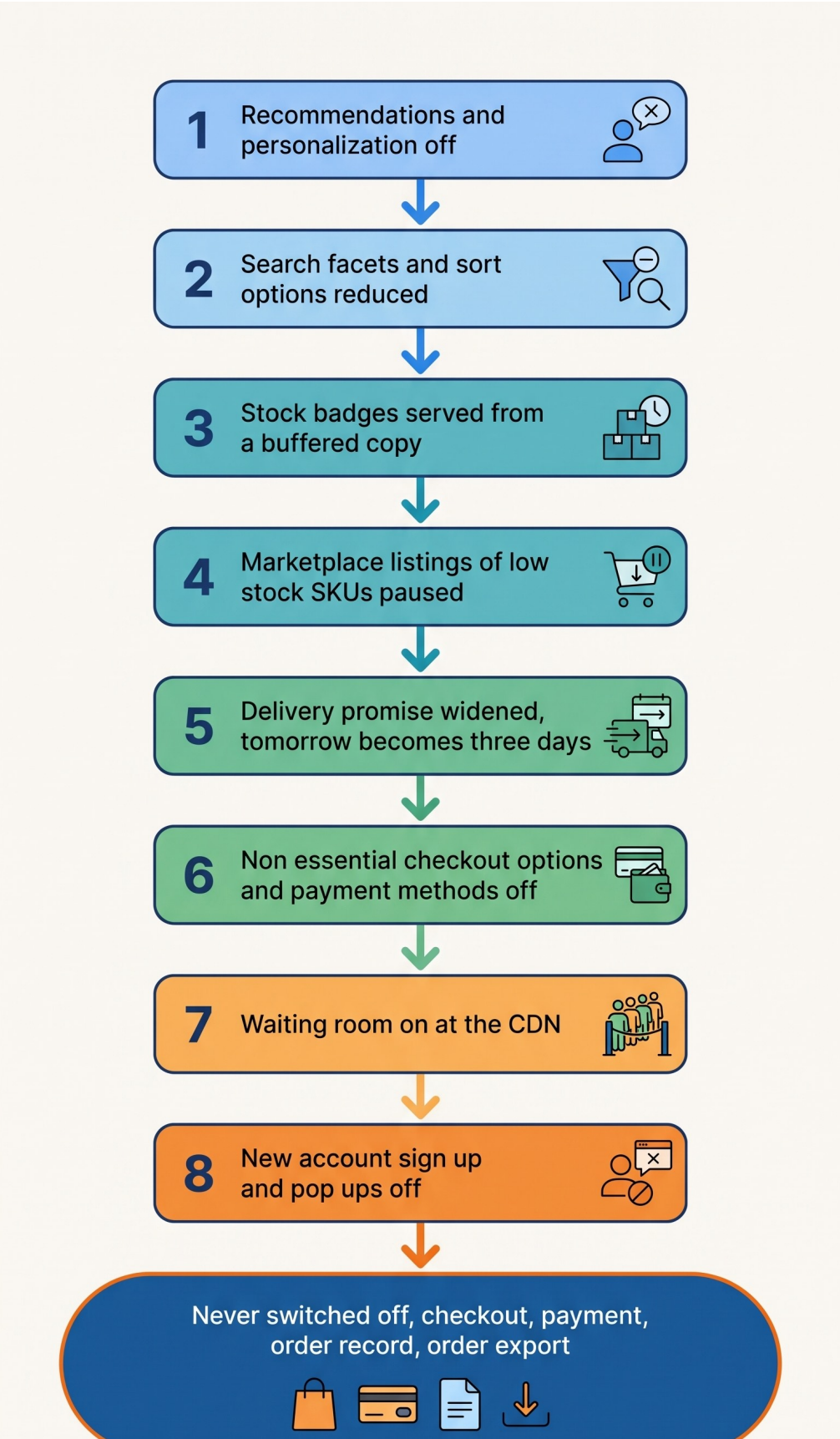

1
Recommendations and personalization off
2
Search facets and sort options reduced
3
Stock badges served from a buffered copy
4
Marketplace listings of low stock SKUs paused
5
Delivery promise widened, tomorrow becomes three days
6
Non essential checkout options and payment methods off
7
Waiting room on at the CDN
8
New account sign up and pop ups off
Never switched off, checkout, payment, order record, order export

Figure 3: *A degradation ladder for a hosted-platform merchant: each rung removes load or risk while keeping the promise that matters, and the bottom rung names what is never switched off.*

Three rules make a ladder work: every switch has been flipped in a test before November, because a switch that has never been used is a hope; every switch has one named owner who may flip it without a meeting; and the order is decided in a calm month by the people who own the revenue, not in the peak hour by whoever is on call.

The Choices and Their Trade-offs

There are four realistic postures toward a peak, and they are cumulative. **Trust the platform** means doing nothing beyond what the vendor does; it is honest for a Starting merchant with one channel and a 3PL (a third-party logistics company that stores and ships on the merchant's behalf), until the first connector is built. **Protect the front door** adds a CDN, a waiting room and a higher platform tier; it protects the link that was already safest and suits launches on one product. **Protect the chain** measures every link in orders per minute, load-tests the integrations with marked orders, puts gauged queues behind the platform, writes the switch-off ladder, runs a situation room and names a reconciliation owner; it is the Omnichannel posture, and the one Kestrel & Pine adopts for 2027. **Build a peak-grade stack** means owning the platform and running the full readiness calendar that Shopify and Zalando document.

	Trust the platform	**Protect the front door**	**Protect the chain**	**Build a peak-grade stack**
What it protects	Nothing beyond the vendor's own uptime	The storefront and checkout	Every link from order to warehouse and back	Everything, including the platform itself
Cost	None	Low: CDN, waiting room, a tier upgrade	Months of integration work, test tooling, one owner's time	A platform team and a year of calendar

	Trust the platform	Protect the front door	Protect the chain	Build a peak-grade stack
Fails when	The first connector exists	The order export is the bottleneck	Nobody owns it after the first year	Copied by a merchant a hundred times smaller (The Giant's Boots)
Right for	Starting	Growing with launch-driven peaks	Omnichannel	Group, or a platform vendor

My recommendation for a merchant with its own warehouse and more than one channel is the third posture, with one addition: **the peak plan has a named owner all year**, not a project team that disbands in December. It does not hold for a merchant whose peak is under five times its normal hour (ordinary reliability work is enough) or whose integrations are all vendor-built and vendor-tested, which is rare.

Observing the Day: Orders per Minute as the Health Signal

Server metrics tell you that the servers are fine; in 2026 every server in Kestrel & Pine's chain was fine while the business lost €96,000. **The health signal for a commerce platform is orders per minute**, plotted against the same weekday and hour of recent weeks, with a sustained drop as the alarm. The operators of the largest peaks report their health in business units (orders per second at Alibaba, orders per minute at Zalando, checkouts and sales per minute at Shopify), and a merchant should do the same.

Figure 4: *The health signal is orders per minute against the same weekday and hour of recent weeks: in 2026 every server in the chain was fine while the business lost €96,000.*

Around that one line, I recommend a peak dashboard built on the **four golden signals** from Google's site reliability engineering (SRE) book, traffic, latency, errors and saturation (Google, 2017), applied per flow rather than per server: for the order export, orders per minute entering the flow, the age of the oldest order not yet in the WMS, failed deliveries and dead-letter depth, and queue depth with rate-limit headroom; for the stock loop, events per minute and lag in minutes; for payment, authorizations per minute, authorization time and the decline rate against normal. Two of these deserve emphasis. **Queue depth with rate-limit headroom** is the gauge Kestrel & Pine lacked: it would have shown the order flow falling behind in the afternoon, hours before anyone noticed. And **decline rate against normal** catches what no server metric can: a payment provider refusing a method, a card-testing wave or a promotion rule rejecting carts. The missing-event case, traffic falling to zero while the shop is busy, is the one most teams forget to alert on.

Targets belong in business units too. Google's SRE book defines a **service level objective** (SLO) as a target value for a measured aspect of service (Google, 2017); a merchant's peak SLOs are checkout success rate, order-confirmation latency, time from order to WMS release and stock lag to marketplaces. The vendors' service agreements cover their own uptime; **the merchant's integration SLOs are nobody else's**.

Both documented programs staff a room through the peak: Zalando's Situation Room with incident commanders (2020), Shopify's round-the-clock incident coverage (2025). A merchant's version fits six people in one room, but needs the same elements: an **incident commander** who decides, one person per link, the vendors' emergency contacts, a **runbook** (a written list of symptoms, checks and actions, including which switch to flip

and who may flip it) and a communication plan. The runbook is written in September and rehearsed in November; a runbook written during the incident is a diary.

Across Scale Stages: When It Fits and When It Limits

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting (one channel, hosted platform, a 3PL)	The platform's peak is not yours; the 3PL's cut-off and the PSP's limits are the real risks	Know the 3PL's peak capacity and cut-off; confirm the PSP's limits; a promotion calendar	The first hand-built connector; a launch that sells more than the 3PL can ship in a day
2 — Growing (two or three channels, an ERP link, a marketplace)	The first integrations are the first bottlenecks; peaks found by surprise; the agency on the phone	Test the order export at ten times normal; a queue between the platform and the ERP; a written switch-off list; marked test orders	A sync interval that oversells; a queue nobody can see; a peak that the agency, not the merchant, runs

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
3 — Omnichannel (stores, wholesale, marketplaces, own warehouse)	Kestrel & Pine in 2026: the storefront holds and the chain behind it fails in sequence; marketplaces and store stock are in the blast radius	A peak plan with a named owner; capacity per link in orders per minute; integration load tests; gauged queues with dead-letter alarms; the degradation ladder with owners; a situation room; a reconciliation owner	Vendors' service level agreements (SLAs) quoted as the merchant's SLOs; the same post-mortem two years running; an OMS (order management system) or availability service bought but not load-tested before November
4 — Group (several brands or countries, platform teams)	Shared peak tooling across brands; some brands on different platforms; a group calendar	Shared load tests, dashboards and runbooks; vendor SLAs read against group SLOs; failover decisions; a platform team that owns the year-round plan	One brand's flash sale starving another's order export on a shared connector; peak readiness that depends on one engineer

The judgement: the practices do not change across stages, the ownership does. At Starting and Growing a peak plan is a checklist the founder or the agency runs; at Omnichannel it is a program with a name, a budget line and an owner who does nothing else in October and November; at Group it is a platform team's product. Kestrel & Pine ran its 2026 peak with a Growing-stage plan (the agency watching the storefront) for an Omnichannel-stage chain.

In Practice: Kestrel & Pine's 2027 Peak Plan

Noor's answer to Mara's question was a plan built from the dated practices above, approved with Waypoint in March 2027 and worked out in full in Journey 4: Black Friday, Hour by Hour. Its spine, with the figures the plan commits to (all fictional):

- **One unit and one forecast**. Every link is measured in orders per minute. The forecast for Black Friday 2027 is **60 orders a minute** in the peak hour, about one and a half times 2026, and every link is tested at **80**.
- **The order path rebuilt**. By the peak, marketplace orders (since 15 September 2027) and direct-to-consumer (D2C) orders (since 13 October) are created in the new OMS (Waypoint W2), which sends fulfillment orders to the WMS in batches within its rate limit, with the order number as the idempotency key, a gauge on the age of the oldest unsent order, an alarm at ten minutes and a dead-letter queue with an owner. Tested at 120 orders a minute with marked orders that the WMS skips.
- **The stock loop replaced**: the availability service, a module of the OMS (W1), computes available-to-promise per channel from WMS stock events within a minute, with a buffer per channel. Go/no-go for the peak: **15 October 2027**; if it is not ready, the old chain runs with the ten-unit marketplace buffer in force since December 2026.
- **A degradation ladder** of eight switches, each with an owner and a trigger, rehearsed in October.
- **A freeze** on apps, theme, connector and OMS changes from **15 November to 5 December 2027**, after an app audit removed the six unused apps and reduced the systems writing stock to one.
- **A situation room** on Friday 26 November from 16:00 to midnight with Noor as incident commander, and a **reconciliation report** at 08:00 on Saturday comparing orders in the platform, the OMS and the WMS, stock in the WMS and the availability service, and payments captured against orders.
- **Deliberately after the peak**: wholesale orders by EDI (electronic data interchange, the structured order messages a retail chain sends) and by hand move to the OMS on 8 December. Nothing that has not run through a load test goes live in November.

The plan costs about four months of one integration engineer's time, a load-testing tool and Noor's October and November. Against €96,000 in one evening, Mara approved it in twenty minutes.

When It Goes Wrong: Recovery and Reconciliation

A peak plan reduces incidents; it does not eliminate them. What distinguishes a prepared merchant is that it knows, before the day, what each failure leaves behind and how to clean it up.

Failure at peak	What it leaves behind	Detection	Recovery
Order export falls behind	Orders in the platform not yet in the WMS; a broken delivery promise	Queue depth; order-to-WMS lag alarm	Let the queue drain; widen the promise; notify customers once
Connector retries after timeouts	The same order twice in the WMS; two parcels	Duplicate check on the order number; the morning reconciliation	An idempotent receiver prevents it; otherwise cancel the duplicate before picking
Messages exhaust their retries	Orders in the dead-letter queue, invisible to the warehouse	Dead-letter alarm	Replay after the cause is fixed; confirm each order arrived once
Stock copy stale	Units sold in several channels	Orders for a SKU above available stock	Pause listings; allocate what exists to the earliest orders; cancel or delay the rest with a voucher
PSP degraded or one method failing	Declines rising; carts abandoned	Decline rate vs. normal	Switch the method off; confirm every captured payment has an order

Recovery after a peak is **reconciliation**: comparing two systems' records and fixing the differences. The platform's own documentation makes the case: Shopify tells apps that "webhook delivery isn't always guaranteed" and to run reconciliation jobs that periodically fetch changed data (Shopify developer docs, accessed November 2026). The

morning-after report compares orders across systems by order number, stock per SKU between the WMS and the availability service, and payments captured against orders; the owner of each difference is named in the plan, not found on the day. Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation owns the reconciliation card.

The last step is the **post-mortem**, written within two weeks, blameless, with the timeline in orders per minute and every action assigned. Kestrel & Pine's became the Waypoint proposal; a post-mortem that becomes nothing is the surest predictor of the same incident next year.

What to Take Away

Established.

- A hosted platform's multi-tenancy smooths its peak to roughly twice its average (Shopify's 2025 figures), while a single merchant's chain sees that merchant's full spike; the storefront is the link most likely to hold and the integrations the links most likely to fail (Shopify 2025, Zalando 2020).
- Preparation at the documented operators follows one shape: forecast, scale tests at forecast load that bypass caches and include checkout and authenticated paths, fault injection on the money paths, a freeze, a staffed room and a runbook (Shopify 2023 and 2025, Zalando 2019 and 2020).
- Under overload, persisting the customer-facing commitment fast and queuing everything behind it preserves goodput (AWS 2019); planned degradation through ops toggles keeps the essential path alive (Hodgson 2017); and because delivery between systems is at-least-once, a peak's retries produce duplicates and gaps unless receivers are idempotent and a reconciliation runs afterwards (Shopify and Stripe documentation, 2026).

Depends on context.

- A waiting room pays off for launches and flash sales where the storefront is the bottleneck and the admission rate can be set from a known end-to-end capacity; for a hosted-platform merchant whose bottleneck is the order export it adds little.
- One and a half times last year's peak is a reasonable forecast for a growing merchant with a stable promotion plan; a merchant adding a marketplace, a country or a television campaign should forecast from the plan, not the history.
- The full readiness calendar belongs to platform owners and Group-stage merchants; an Omnichannel merchant that copies it wears the Giant's Boots, and one that skips its integration half meets the First Snowfall.

The author's synthesis.

- Measure the whole chain in orders per minute, because it is the only unit that every link, every vendor and every executive can read, and the only one in which the slowest link is visible before the day.
- The peak plan is a year-round product with one owner, not a November project; the switch-off ladder is written by the people who own the revenue, in a calm month, and the engineers' job is to make sure every rung exists and has been flipped once before it matters.

Questions to Ask

1. What was our peak hour last year, in orders per minute, and what is the forecast for this year? Who owns that forecast?
2. For each link from checkout to warehouse and back, what is its tested capacity in orders per minute, and when was it last tested at that rate?
3. Which apps and connectors share the platform's API budget, and what are the WMS's and the PSP's rate limits?
4. Where are the queues in our chain, who can see their depth, and what happens to a message that fails every retry?
5. What is our switch-off order, who owns each switch, and when was each last flipped?
6. Can we withdraw the delivery promise on the page when the warehouse falls behind, and who decides?
7. Does the dashboard show orders per minute against last week, order-to-warehouse lag and payment decline rate, and who is the incident commander on the day?
8. The morning after, which two systems do we compare, by which key, and who fixes the differences? When is this year's freeze?

The peak is the day the chain is honest. The next part turns to the people who exploit that honesty on purpose, the card testers, account thieves and bots that arrive with the shoppers, and to the rules that govern what a commerce platform may do with the data it holds. Continue with Part VIII — TRUST: Security, Privacy and Regulation.

Sources and Evidence

Company, product and investor sources

- Shopify merchants achieve record-breaking $14.6 billion in Black Friday Cyber Monday sales, Shopify investor press release, 2025 — BFCM 2025 sales, peak sales per minute, edge, app-server and API requests per minute, and the unaudited caveat.

- Black Friday Cyber Monday 2025 data, Shopify News, 2025 — the same figures with the GMV definition and the methodology caveat.
- Black Friday Cyber Monday 2024 data, Shopify News, 2024 — BFCM 2024 load figures for comparison.
- How we prepare Shopify for BFCM, Shopify Engineering, 2025 — capacity planning from March, five scale tests, Game Days, the Resiliency Matrix, checkout queues backing up in early tests, rate-limit paths found by authenticated checkout.
- Scale and performance testing at Shopify, Shopify Engineering, 2023 — full-scale testing with CDN bypass and cache-busting, and the abort switch.
- Faster checkout for customers during flash sales, Shopify Changelog, 2018 — payment processing separated from order creation under high volume (eight years old).
- Storefront API reference, Shopify developer documentation, n.d., accessed 2026-11 — the present-day checkout throttle.
- Webhooks, Shopify developer documentation, n.d., accessed 2026-11 — delivery not guaranteed; reconciliation jobs recommended.
- Rate limits, Stripe documentation, n.d., accessed 2026-11 — per-account limits, backoff with jitter, and the advice to announce flash sales.
- End-to-end load testing Zalando's production website, Zalando Engineering, 2019 — recorded journeys replayed at 130,000 requests per second; situation rooms; what broke on replay (seven years old).
- How Zalando prepares for Cyber Week, Zalando Engineering, 2020 — 7,200 orders per minute in 2019, production-readiness reviews, test orders that fulfillment skips, pausing non-essential processing, the Situation Room (six years old).
- The magic behind the world's largest online shopping festival, Alibaba Cloud, 2020 — 583,000 orders per second at the 2020 peak, "orders" undefined.
- Adobe: Cyber Monday 2024 recap, Adobe, 2024 — $13.3 billion and $15.8 million per minute in the peak hours.
- Amazon internal documents: what caused the Prime Day crash, CNBC, 2018 — the fallback front page and the international traffic cut (a press account, eight years old).
- Cloudflare Waiting Room, Cloudflare, 2021, and Waiting Room configuration settings, Cloudflare Docs, n.d., accessed 2026-11 — what a waiting room is and how it is configured.
- How does Queue-it work, Queue-it, n.d., accessed 2026-11 — threshold-triggered virtual queues with adjustable outflow.

Standards, regulation and official documentation

- Using load shedding to avoid overload, AWS Builders' Library, 2019 — overload, goodput and prioritizing requests.
- Service Level Objectives and Monitoring Distributed Systems, Google SRE book, 2017 — SLI, SLO, error budget and the four golden signals.

- Amazon SQS dead-letter queues, AWS documentation, n.d., accessed 2026-11 — what a dead-letter queue is and how messages are redriven.
- Exponential backoff and jitter, AWS Architecture Blog, 2015 — why retries need randomness.

Research and practitioner sources

- Feature Toggles (aka Feature Flags), Pete Hodgson, martinfowler.com, 2017 — the toggle taxonomy and ops toggles as kill switches for graceful degradation.
- Starbucks Does Not Use Two-Phase Commit, Gregor Hohpe, 2004 — accept the order fast and let the work queue; write-off, retry and compensation as the error strategies.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Part VIII — TRUST: Security, Privacy and Regulation

IN THIS PART, YOU WILL: Learn what attackers take from a shop and which controls stop them, where personal data lives and how to design consent, retention and deletion across many systems, and how to read a law as a dated data requirement with an owner.

KEY POINTS:

- **Attackers do not break in; they use the shop as built.** Card testing, credential stuffing and skimming are the payment form, the login and the checkout page used at machine speed. The hosted platform secures the service and the card-data scope; the merchant secures every identity, key, app, script and endpoint it adds, and watches the decline rate as closely as the order rate.

- **The order and the person are two different things in law**. The merchant is the controller of its customers' data; its platform, warehouse system and apps are processors; carriers, marketplaces and the payment provider decide for themselves. The order rests on the contract and on tax law and stays for years; the profile rests on consent and goes when the person says so, so the data model must let the transaction outlive the person.
- **Most rules are data requirements with a date**. A 30-day price history, a 14-day refund clock, a manufacturer field on every listing, a structured invoice over a network: each belongs in a system of record, with a test and a calendar row that names an owner. The platform absorbs some of it, never all of it.

FAILURE PATTERNS THIS PART ADDRESSES:

- **The Secret Agent** — an app, script or integration that acts without its own identity cannot be traced when its key leaks, and it copies customer data as a processor nobody appointed.
- **The Lynchpin** — one shared admin password, one payment account and one person who knows where the keys are is a security architecture that fails in a single step.
- **The Familiar Stranger** — a person the systems cannot recognize as one cannot be deleted, told the truth about or respected when they opt out.
- **The Long Fuse** — a deletion request has a one-month clock and every rule in this part was published years before it applied; both are discovered late by a merchant with no map.

Every part so far has asked how a shop works when everyone means well. **This part asks what happens when someone does not, and what the law requires regardless**. A shop is three public forms: a payment form that takes money from anyone with a valid card number, a login that lets anyone with the right password act as a customer, and an admin that lets them act as the merchant. None of it has to be broken to be abused. The same shop holds a name, an address and a purchase history in a dozen systems from a dozen vendors, each copy made for a reason and each reason with its own rule for how long it may stay. And much of what the shop must show, record and refund is written in the Official Journal of the European Union, with a date.

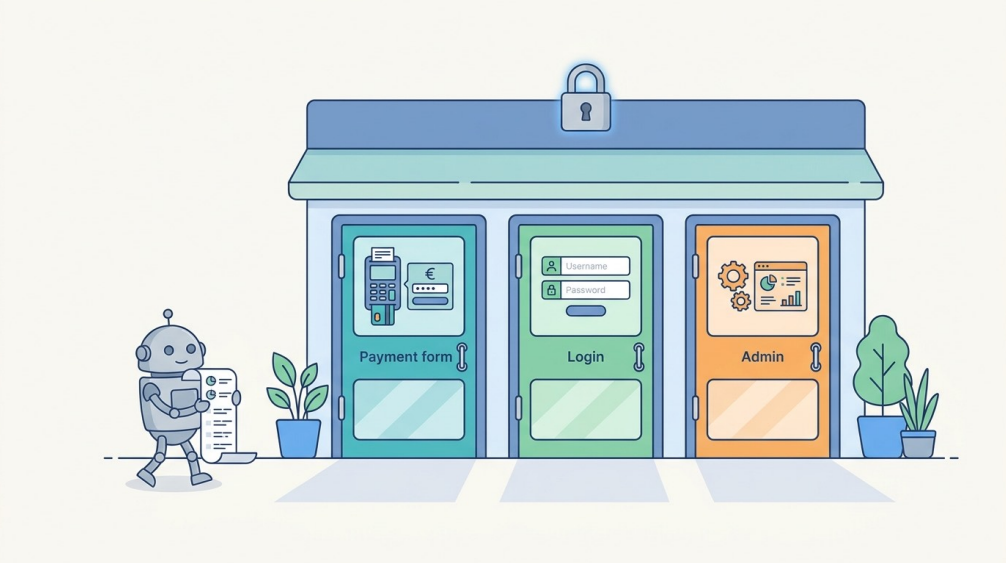


Figure 1: *A shop is three public forms, a payment form, a login and an admin, and none of them has to be broken to be abused.*

Part VIII sits at the checkout and payment steps of the order's journey, then alongside every step after them. Eva Janssen's jacket, the order this book follows, met the law four times without anyone noticing: the price she saw had to include VAT and sit in a price history that any later reduction is measured against, the checkout button carried prescribed words ("order with obligation to pay" or an equivalent), the product page carried a manufacturer's address, and her exchange request on 19 April 2027 started two 14-day clocks, one for sending the goods back and one for any refund. Her name went to the platform, the warehouse system, the carrier, the payment provider and the ERP (the enterprise resource planning system that keeps the books and issues invoices), each under its own retention rule. And the 48,000 requests that hit the same checkout one night in August 2026 looked like hers to every system except the one that counted them.

Kestrel & Pine, the fictional outdoor brand this book follows, learned the three lessons of this part one incident at a time: the Card-Testing Night of 18–19 August 2026, the Request From Ghent in May 2027, and four rules that arrived through a marketplace's error log, an agency audit, a customer's refused PDF and an auditor's newsletter. **The claim of this part is that the three have one remedy: the ownership map built from Part I's system-of-record map and Part VI's integration inventory, one row per system, integration and app with its owner, gains three more columns.** Who holds the key to each integration. Which legal role and retention clock each system has. Which rule lands in which system of record, by when, and whose name is on the row.

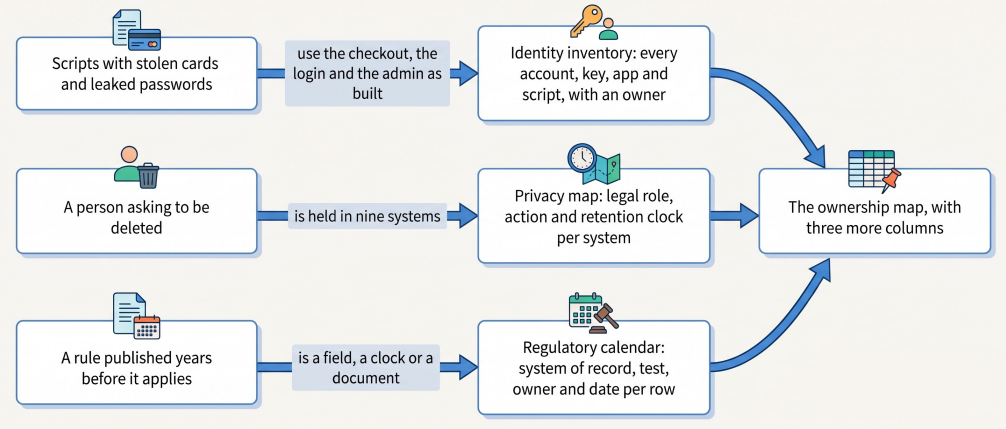


Figure 2: *Three things reach a shop from outside, a script, a person with rights and a rule with a date, and each is answered by a list with owners; the three lists are columns of the same ownership map, not a separate discipline.*

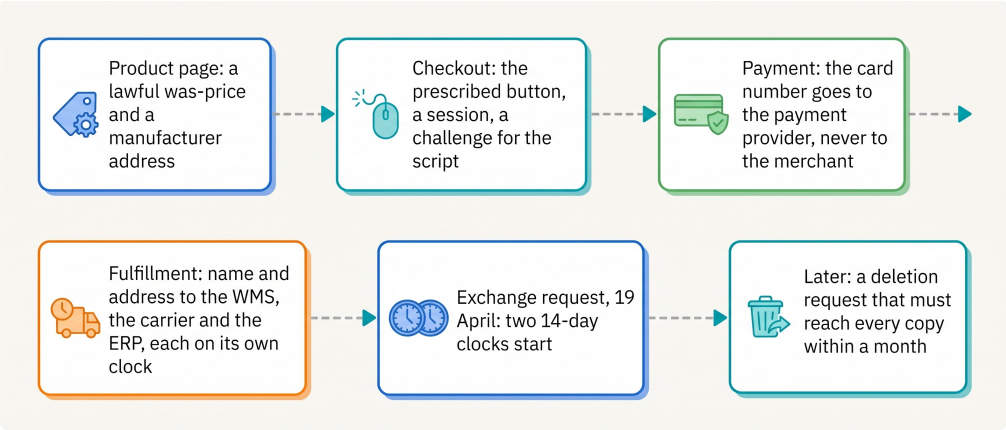


Figure 3: *Where Part VIII meets Eva's jacket: security at the checkout and the payment, regulation at the price, the button, the product page and the clocks, privacy in every system her name reaches and in the request that one day asks for it back.*

Chapters in this part

Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers decides **what attackers want from a shop and which controls matter most**: keep the card number out of your systems with a hosted checkout or hosted fields (card boxes served by the payment provider inside your page), challenge and rate-limit every endpoint that can trigger a payment or a login, own every script on the checkout page, and give every

person, app and integration its own identity and restricted key. The case is the Card-Testing Night: 48,000 attempts in one night, about 260 live cards and 21 disputes; then the April 2027 baseline, with a decline-rate alert at three times the 30-day median, a kill switch, multi-factor authentication (MFA) enforced and no shared accounts, one key per integration in a vault, a script inventory and three runbooks.

Whose Data Is It? Privacy, Consent and Retention decides **where personal data lives, under which rules, and how to design consent, retention and deletion**: every integration is a data flow with a legal role, consent is recorded on the customer master with its evidence, and a deletion request is a fan-out with a one-month deadline that needs an identity index, an orchestrator and a decided action per system, erase, pseudonymize, retain or refer. The case is the Request From Ghent of Monday 10 May 2027: a customer found in nine of the thirteen systems and seventeen working days before the reply on 4 June; from October 2027 an orchestrator on the integration platform, and a second request answered in two working days.

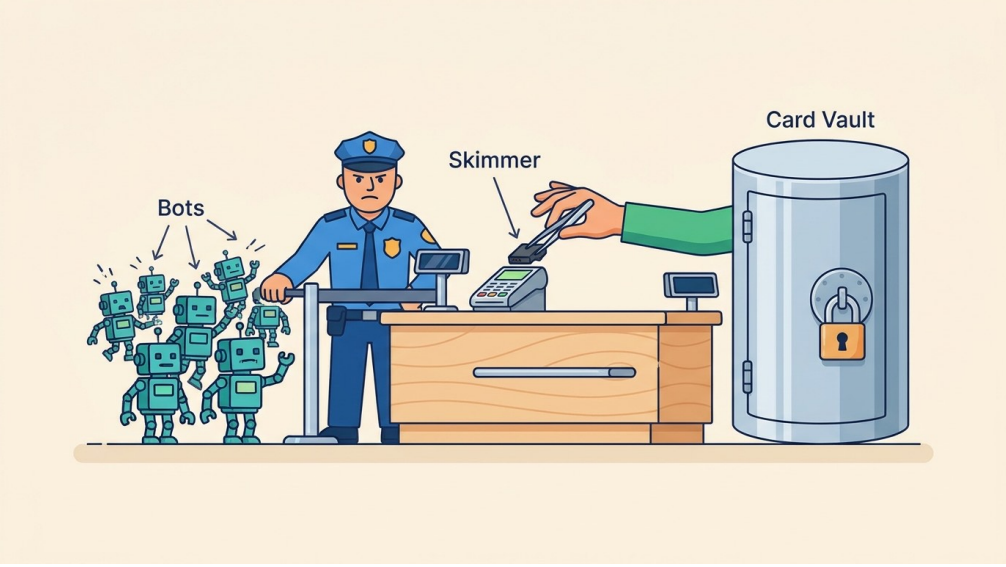
Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation decides **which laws change what a platform must do, by when, and how to read them as requirements**: scope first, then the field, history, document or clock the article demands, the system of record that owns it, the channels that show it, a test in the release process and a calendar row with an owner. The case is four rules through four doors, product-safety listing fields rejected by a marketplace in December 2024, an accessibility audit of May 2025 with about 140 findings, Belgian e-invoicing through a refused PDF in December 2025 and the ban on destroying unsold apparel from 19 July 2026, and then the regulatory calendar Noor Visser and Sanne de Wit write in March 2027, where most rows are maintenance, the owners are mostly not engineers, and the few projects land in systems Waypoint builds anyway.

Begin with Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.

Sources and Evidence

Every public claim in this part is sourced and dated in the chapter that makes it, under that chapter's Sources and Evidence. Kestrel & Pine is fictional; recommendations and labeled assumptions are the author's judgement.

34. Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers



IN THIS CHAPTER, YOU WILL: Learn what attackers take from a shop, where the card-data standard draws its line, which controls stop card testing, takeover and skimming, and what stays your job on a hosted platform.

KEY POINTS:

- **Attackers do not break in; they use the shop as built.** Card testing, credential stuffing, coupon abuse and inventory hoarding are the payment form, the login, the promotion engine and the cart used at machine speed. The control is to make every endpoint that can trigger a payment or a login authenticated, rate-limited and challengeable, and to watch the decline rate as closely as the order rate.
- **Keep the card number out of your systems, and own everything around the box that takes it.** With a hosted checkout or hosted fields the card number never touches your code and the lightest PCI

DSS self-assessment applies. Since 31 March 2025 the standard's script requirements (6.4.3 and 11.6.1) make the scripts on the payment page your inventory to keep, because skimmers run in the shopper's browser on a page you serve.

- **The platform secures the platform; you secure the identities.** Every vendor's shared-responsibility statement draws the same line: the vendor patches and certifies the service; the merchant owns admin accounts, API keys, apps, scripts and integrations. An integration without its own identity and a least-privilege key is the Secret Agent waiting for a leak.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Secret Agent** — a script, app or integration that can change orders, prices, stock or money without its own identity cannot be traced when its key leaks, and a leaked key is the attacker's favorite entry.
- **The Lynchpin** — one shared admin password, one payment account and one person who knows where the keys are is a security architecture that fails in a single step.
- **The Spaghetti Junction** — thirty-four integrations (iPaaS flows, agency scripts and vendor connectors), each holding a credential nobody inventoried, are thirty-four doors with no record of who holds the key.

Every shop is three public forms and a catalog. The payment form takes money from anyone who types a valid card; the login lets anyone with the right password act as a customer; the admin login lets them act as the merchant; the catalog, cart and promotion engine answer any request at any speed. **None of this has to be broken to be abused; it only has to be used faster and in worse faith than the people who built it imagined.** Most commerce attacks are the shop as built, driven by a script.

The merchant notices the abuse as money and trust: fees for declined payments, a payment account paused in the middle of the night, customers whose saved cards were used by someone else, stock held in bot carts, and in the worst case a few lines of script on the checkout page copying every card number typed into it, which is how British Airways came to a £20 million penalty in 2020 for a 2018 breach.

This chapter is written for a merchant that rents its platform, which is most merchants: what the card-data standard asks and where it stops, how card testing and takeover work and what stops them, why the scripts on your checkout page are your problem, and why identities and keys, not firewalls, are the center of a merchant's security. It leaves personal data and consent to Whose Data Is It? Privacy, Consent and Retention, the law to Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, and fraud models to LISTED's Scams, Fakes and Account Takeovers: Fraud Prevention for Listings.

Where we are in the order's journey. Eva's jacket, the order this book follows, was placed at 21:14 on a Tuesday in April 2027 and paid by iDEAL in under a minute. This chapter sits in the same two steps, checkout and payment, seen from the other side: the 48,000 requests that hit the same checkout one night in August 2026 looked like Eva's to every system except the one that counted them.

Setting the Scene at Kestrel & Pine: The Card-Testing Night

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), had its security incident of 2026 on the night of **Tuesday 18 to Wednesday 19 August**. The numbers below are Kestrel & Pine's records; the mechanism is reconstructed as an illustration.

Between about 23:40 and 04:10 a script ran **48,000 checkout attempts** against the Dutch storefront's guest checkout, each with a different stolen card number and a nonsense name, each buying the cheapest item in the catalog. It was asking Kestrel & Pine's payment provider, card by card, which numbers still worked. This is **card testing**, also called carding: using a merchant's payment form to sort a stolen list into live and dead cards, which are then sold or used elsewhere. The OWASP (Open Worldwide Application Security Project) Automated Threats project, the shared vocabulary for this kind of abuse, lists it as OAT-001 (n.d., accessed November 2026).

Three things happened that the shop was not built to notice.

- The **PSP** (payment service provider: the company that takes the shopper's payment and settles the money to the merchant) charged a fee for every authorization request, declined or not: **€2,900** on the September statement for sales that never happened.
- At **02:05** the PSP's own monitoring saw an authorization rate and a decline rate it had never seen from this account and **paused the account for 40 minutes**. For 40 minutes nobody could pay by card on any storefront, and the e-mail that said so went to a shared mailbox nobody read until 07:30.
- About **260** of the 48,000 cards were live (illustrative). The platform created real orders for them, which Milan Petrov, who owns the storefronts, canceled and refunded by hand; **21** came back as disputes, and the share of genuine card payments declined by issuers rose from about 4% to about 7% for two weeks, because issuers had seen a burst of fraud from this merchant (illustrative; Stripe's card-testing documentation describes the mechanism, n.d., accessed November 2026).

Most attempts used cards issued outside Europe (illustrative), where **strong customer authentication** (SCA, the European rule that a shopper must prove their identity at

payment; Taking the Money: Payment Providers, Authorization, Capture and Refunds) does not bind the issuer, so the usual challenge never fired. The agency switched on the platform's bot protection and the PSP's velocity rules that week; no runbook was written. In March 2027 Noor Visser, Head of Engineering since January, reads the incident report for the Waypoint program and asks a different question: **not "how do we stop card testing", but "which of our doors are open to a script, and who holds the key to each".**

What Attackers Want From a Shop

The table lists the threats in roughly the attacker's order of preference, with the OWASP Automated Threat identifier where one exists (OWASP, n.d., accessed November 2026).

Threat	What it is, in plain words	Who loses	Where the control lives
Card testing (OAT-001; OAT-010 when the expiry or security code is guessed)	Many small payments or card-save attempts to find which stolen cards work	The merchant: fees, disputes, a paused account, a worse decline rate for months	Checkout, PSP rules, rate limits, challenges
Credential stuffing (OAT-008) and **account takeover**	Password pairs leaked from one site tried on yours at machine speed; any route that ends with an attacker logged in as a customer or employee	The customer: saved addresses and cards; the merchant: refunds and trust	Login, multi-factor authentication (MFA), bot management
Web skimming (Magecart)	Malicious script on the checkout page that copies what the shopper types into the card fields	Every shopper who paid; then the merchant, through penalties and reputation	The scripts on the payment page: inventory, policy, tamper detection

Threat	What it is, in plain words	Who loses	Where the control lives
Coupon abuse (OAT-002)	Guessing or sharing codes; stacking; one-time codes used many times	Margin	The promotion engine (Discounts, Coupons and Loyalty: Promotions Without Chaos)
Refund and returns abuse	Refunds to a different card, empty-box returns, serial "not received" claims	Margin and stock	The returns and order systems (The Way Back: Returns, Exchanges, Refunds and Reverse Logistics)
Inventory hoarding and scalping (OAT-021, OAT-005)	Bots fill carts to hold stock during a launch, or buy it all to resell	Real customers, then the brand	Cart reservation rules (Is It in Stock? Inventory, Availability and the Promise You Can Keep), bot management
Admin compromise	An attacker with a staff login or a leaked API key: payout account changed, scripts added, customers exported	Everything	Identity: MFA, least privilege, key management
Supply-chain risk	An app, plugin, theme or hosted script that is compromised and runs with your privileges	Everything the app could touch	App governance (Apps, Plugins and Functions: Extensibility and Ecosystem Governance), script policy, patch cadence

The last two rows bypass every other control: a skimmer does not need to beat the PSP, and an attacker with the admin password does not need to guess. And half of the rows

are not intrusions but **policy abuse**, whose controls live in the order, promotion and returns systems; a bot-management product covers the top of the table and nothing else. Vendor reports put automated traffic at about half of all web requests (Imperva, 2025, measuring its own customers); whatever the true share, **on most shops a large part of the requests are not people**.

Shared Responsibility: What the Platform Does and What You Must Do

The line between what the vendor does and what the merchant does is published. Adobe's shared-responsibility model for Adobe Commerce on cloud infrastructure (n.d., accessed November 2026) is the most explicit: Adobe maintains the core code, the operating system, global firewall rules and PCI certification for the infrastructure; the merchant secures "custom code and integrations with third-party applications", applies patches, manages user access and handles its own incidents. Shopify states that it is "certified Level 1 PCI DSS compliant" and that the certification "extends by default to all stores powered by Shopify", while adding that "there are also steps you will need to take on your own" (Shopify, n.d., accessed November 2026). Stripe says the same about payments: "PCI compliance is a shared responsibility and applies to both Stripe and your business" (Stripe, n.d., accessed November 2026).

Read together: the vendor covers the service's code, hosting, patching and certification and the built-in tools with their defaults; the merchant covers the code, scripts and apps it adds, who holds an admin account and whether MFA is enforced, which API keys exist, and incident response for the shop. **The hosted platform removes the patching burden. It never removes the identity, script and endpoint burden**, which is the rest of this chapter.

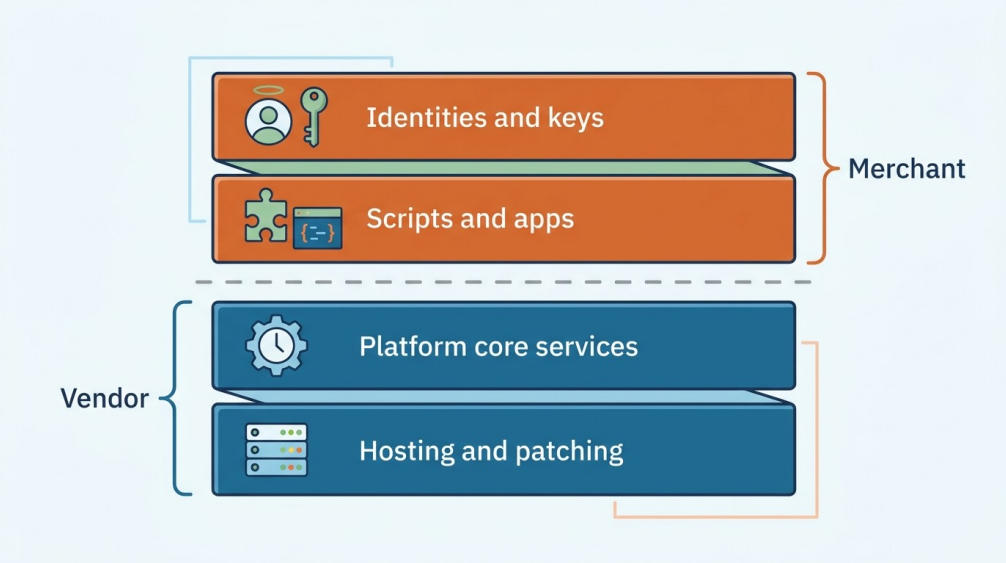


Figure 1: *The hosted platform removes the patching burden; the identities, keys, scripts and apps the merchant adds stay the merchant's to secure.*

Card Data: Where PCI DSS Draws Its Line

PCI DSS (Payment Card Industry Data Security Standard) is the card brands' security standard for anyone who stores, processes or transmits card data, a contractual obligation through the merchant's acquiring bank (the acquirer: the bank that accepts card payments on the merchant's behalf) rather than a law. Its **scope** is the set of systems that touch card data, and the design goal of a modern checkout is to make that set as small as possible: ideally, none of the merchant's own.

The dates matter because the standard changed recently: PCI DSS v4.0 was published in March 2022; v3.2.1 was retired on 31 March 2024; a limited revision, **v4.0.1**, was published on 11 June 2024, and v4.0 was retired on 31 December 2024. The requirements that v4.0 introduced as "best practice" became mandatory on **31 March 2025**, and v4.0.1 did not move that date (PCI Security Standards Council blog, 11 June 2024).

How much of the standard applies to you

The answer depends on how the card number, the **PAN** (primary account number), reaches the PSP. Smaller merchants attest compliance through a **self-assessment questionnaire** (SAQ), and which SAQ applies follows the integration pattern. The table is the author's synthesis of Stripe's and Adyen's guides (n.d., accessed November 2026); the acquirer, not a vendor's documentation, sets the SAQ.

Integration pattern	Who sees the card number	Typical SAQ	What the merchant still owns
PSP-hosted checkout page (the shopper is redirected to the PSP's page)	The PSP only	**SAQ A**, the lightest	Site integrity (the script statement below), admin access, order data
Hosted fields (card input boxes served by the PSP inside an iframe on the merchant's page)	The PSP only	SAQ A, if the eligibility criteria are met	Everything on the page around the iframe: scripts, headers, policy
Merchant-hosted form posting through the PSP's JavaScript	The merchant's page collects, the PSP tokenizes	SAQ A-EP, considerably heavier	Page security, scripts, tamper detection
Direct API with raw card data	The merchant's servers	SAQ D or a full assessment; "the most challenging SAQ there is" (Stripe)	The full cardholder-data environment: more than 300 controls

Hosted platforms with their own checkout put every merchant in the first two rows. **This is the largest security decision a merchant makes, and most make it by default.** A custom checkout that collects card numbers itself, as some headless projects do for design reasons (Delivering the Storefront: Themes, Headless, Rendering and Performance), moves the merchant down the table and multiplies the controls it must prove.

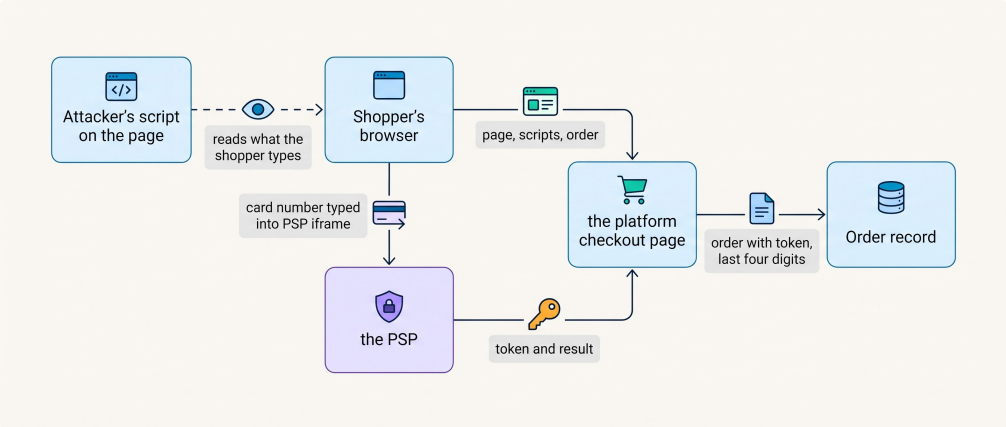


Figure 2: *With hosted fields the card number travels from the shopper's browser straight to the PSP and the merchant's systems hold only a token; the remaining exposure is any script on the page that can watch the shopper type, which is why the page's scripts are the merchant's responsibility.*

The card brand, last four digits and expiry returned by the PSP are out of scope and may be stored (Stripe security guide, n.d.); everything else is replaced by a **token**, a reference the PSP issues that can charge or refund that card again without the merchant ever holding the number.

The two script requirements and the SAQ A statement

Two requirements exist because of skimmers. **Requirement 6.4.3**: every script on a payment page is authorized, integrity-checked and inventoried with a written business justification. **Requirement 11.6.1**: a mechanism detects and alerts on unauthorized changes to the payment page's content and security-relevant HTTP headers. Both were best practice from March 2022 and are required since **31 March 2025** (PCI SSC, 26 March 2025).

The Council then **removed 6.4.3 and 11.6.1 from SAQ A** and added an eligibility criterion instead: an SAQ A merchant must confirm that its site "is not susceptible to attacks from scripts that could affect the merchant's e-commerce systems", which it can do by applying the techniques itself, through a third party, or by obtaining its PCI-compliant payment processor's confirmation that the hosted solution includes them when implemented as instructed. As the Council wrote, "these requirements were only removed from SAQ A, and they are still in the standard" (PCI SSC, 26 March 2025). In plain words: **a merchant on a hosted checkout is no longer asked to prove a script inventory, but it is asked to state that its page cannot be skimmed, and it had better know why that statement is true**.

Adyen's guidance describes the work: deploy a **Content Security Policy** (CSP, an HTTP header that tells the browser which script sources a page may load), first in report-

only mode, then enforcing; keep the inventory with business reasons; use **Subresource Integrity** (SRI, a hash on a script tag so the browser refuses a changed file) on the PSP's scripts; and treat CSP violation reports as "valid alerts and 'indicators of compromise' " (Adyen docs, n.d., accessed November 2026).

Two other requirements recur. **Requirement 12.8** makes the merchant responsible for its service providers and for holding each one's attestation of compliance ("Adyen is solely responsible for the security of cardholder data only as soon as Adyen receives the data", Adyen docs, n.d.). **Requirement 8.4.2**, also mandatory since 31 March 2025, requires multi-factor authentication for all access into the cardholder-data environment: for a hosted merchant, the PSP console and the platform admin.

Card Testing: The Attack and the Controls

Stripe's documentation is the clearest public description of the attack (Stripe, "Protect yourself from card testing", n.d., accessed November 2026). Attackers prefer **saving a card** to paying with it, because card-setup authorizations "don't typically show up on cardholder statements". The symptoms are a spike in failed payments, a spike in HTTP 402 responses (the status code a payment API returns for a declined charge) and low-value payments "often with nonsensical customer names and emails". The consequences outlast the night: in Stripe's words, "a high decline rate might damage the reputation of your business with card issuers and card networks".

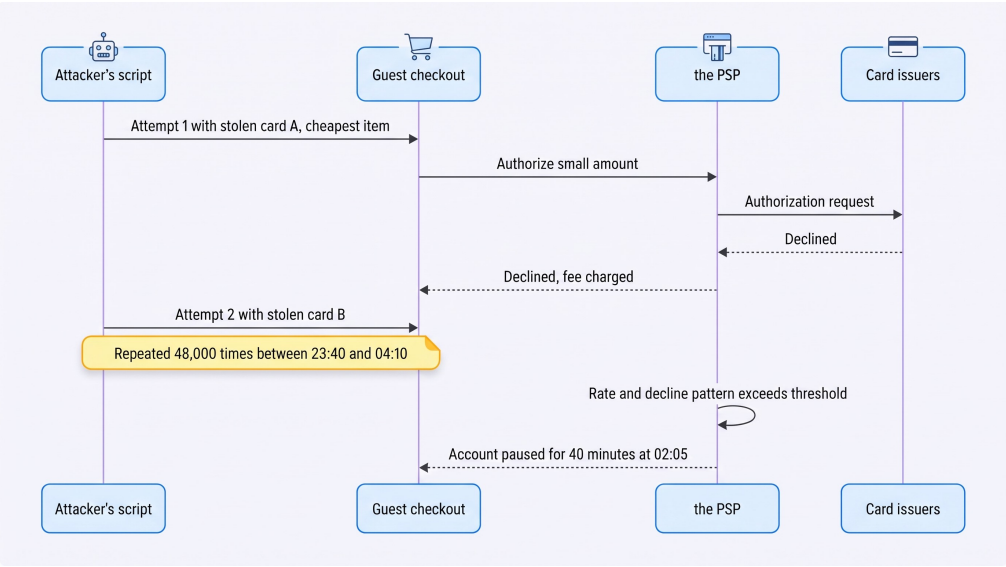


Figure 3: *Card testing on the Card-Testing Night: every attempt is a normal checkout to the platform and a normal authorization to the PSP; only the count, the pace and the decline rate reveal it, and the first system to react was the PSP, by pausing the account.*

Stripe states that "simple firewall rules or filters based on a single heuristic such as IP addresses are usually not sufficient", because attackers rotate addresses. The **leaked secret key** is the worst version: "card testers can create payments and set up cards using your secret key", with no checkout involved at all.

The control ladder

The controls form a ladder; the right rung depends on how much of the checkout the merchant built.

1. **Use the hosted checkout or hosted fields and accept the PSP's defaults**; do not bypass them with a custom flow.
2. **Switch on what you already pay for**: the platform's bot protection on checkout, the PSP's velocity rules (attempts per card, address and hour), a CAPTCHA validated on the server.
3. **Require a session before a payment**. Stripe recommends login or session validation before payment, limits on cards per account and rate limits such as new customers per address per day. A guest still has a session that loaded a product page and a cart; a request that reaches the payment step with no history is the script.
4. **Alert on the decline rate, not only the order rate**. Black Friday Every Year: Peak Load, Resilience and Recovery makes orders per minute the health signal; the security twin is **declines against normal**. Kestrel & Pine's night was visible at 23:55 as a 99% decline rate against a usual 4%.
5. **Have a kill switch**: a setting, not a deploy, that disables guest card payments while iDEAL, Bancontact and PayPal keep running (From Cart to Confirmation: Checkout Orchestration).
6. **Never retry the cards the attacker saved**. Stripe warns that retrying saved cards "repeats the original attack"; a dunning job (an automatic retry of failed payments) that retries every failed card is a card-testing machine with your name on it.

Requirements per control point

Controls sit on the request path and have budgets like any other call (figures are assumptions).

Control point	Latency budget	False-positive tolerance	Consistency	When the control itself fails
Challenge on guest checkout	Under 300 ms at the payment step *(assumption)*	Low: each false challenge costs a conversion	Stateless	Fail open for methods with their own authentication (iDEAL, cards verified by 3-D Secure, the issuer's own confirmation step), fail closed for plain card
Login rate limit and MFA	Under 100 ms *(assumption)*	Low for customers; zero for staff	Per-account and per-address counters	Fail closed for staff, degrade to a challenge for customers
Webhook signature check	Microseconds	Zero: an unsigned webhook is never processed	Idempotent handling by event id (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation)	Queue and alert, never process

A challenge that fires on 2% of genuine shoppers costs more than most card-testing nights, which is why challenges are triggered by risk signals (no session history, velocity, reputation) rather than shown to everyone.

Account Takeover, Bots and Scraping

Credential stuffing is "the automated injection of stolen username and password pairs" to gain access to accounts, and OWASP names multi-factor authentication as a primary countermeasure (OWASP, n.d., accessed November 2026). In commerce: a shopper reuses a password that leaked elsewhere; a script tries it; the attacker changes the delivery address and orders with the saved card; the merchant sees refunds weeks later.

Three controls carry the weight: **rate limits and bot management on the login**, an edge service that classifies requests as human, good bot or bad bot from device, behavior and reputation signals; **a challenge for new devices and risky logins**, with passkeys (a phishing-resistant login using the device's own unlock) as the platforms' direction; and **notification of change**, an e-mail when an address or payment method changes, which catches the takeover after the fact and costs nothing. Customer MFA is rarely enforced because it costs conversion; staff MFA is never optional.

Inventory hoarding turns the cart into an attack surface: if a cart holds stock for 30 minutes, a bot can hold a launch's units for 30 minutes at no cost. The control is the reservation model, not the edge: a short hold, extended only by progress through checkout, and a limit per account during launches (Is It in Stock? Inventory, Availability and the Promise You Can Keep). **Scraping** is tolerated selectively: comparison and search engines scrape, so the typical policy allows product pages, challenges cart and checkout, and rate-limits the storefront API.

Skimmers: The Attack in the Shopper's Browser

The British Airways breach of 2018 remains the clearest public record of web skimming, though it is eight years old and predates PCI DSS v4. The attacker entered BA's network on 22 June 2018 with a stolen third-party login that had no multi-factor authentication, found a saved privileged credential, and between 14 and 25 August 2018 added 22 lines of JavaScript to the payment pages that sent card details to an attacker-controlled site; roughly 429,612 people's data was potentially accessible, and the UK Information Commissioner's Office issued a £20 million penalty on 16 October 2020 (Wikipedia, citing the ICO penalty notice, 2020).

Two lessons survive. **The skimmer did not break the PSP; it ran in the shopper's browser, on a page the merchant served, loaded by a script the merchant or a vendor put there.** And the entry was identity: a third-party login without MFA.

The supply-chain version is newer. On 25 June 2024 Sansec reported that `cdn.polyfill.io`, a script embedded by more than 100,000 sites, had been sold in February 2024 and was injecting code that redirected mobile users to a betting site, skipping admin users to evade detection (Sansec, 2024). Nobody who embedded that script had changed anything. For self-hosted platforms the equivalent is the patch:

Adobe fixed CosmicSting (CVE-2024-34102) on 11 June 2024, and Sansec counted thousands of unpatched stores breached that October and November (Sansec, 2024). Stripe's guide puts the principle in one sentence: "Including JavaScript from other sites makes your security dependent on theirs."

The controls are the ones 6.4.3 and 11.6.1 encode: a **script inventory** with an owner and a reason per entry, a **CSP** that reports violations, **SRI** on scripts that should never change, and **tamper detection** against the known page. On a hosted platform the checkout page is the vendor's; the merchant's exposure is the scripts it adds through the theme, apps and tag managers, and the inventory is still the merchant's.

Admin Accounts, Keys and the Apps

The commerce admin is the most valuable login on the estate: it can change the payout bank account, add a script to the checkout, install an app, export every customer and issue refunds. **An attacker with it needs no other technique**, which is why identity is the center of a merchant's security program.

Shopify's documentation shows how platforms think about it (Help Center, n.d., accessed November 2026): staff "can set up two-step authentication for their own individual accounts", the store owner "can't activate two-step authentication for any other staff member" on standard plans, Plus organizations can require it for all users, and without it on the Payments account "your payouts might be placed on hold". The platform makes MFA available; **enforcing it is the merchant's decision**, on many plans a policy to police rather than a switch to flip.

Keys are the other half. Stripe's key guidance (n.d., accessed November 2026) applies to every system: never put secret keys in source code, because "bad actors continuously scan public repositories" for them; use a secrets vault; create **restricted keys** with the least privilege the job needs (a refund-only key for the returns portal); rotate; audit; treat any exposure as compromise. A **webhook** (a message the PSP or platform sends to your endpoint when something happens) is an unauthenticated request until its signature is verified; an unsigned "payment succeeded" accepted by an order system is a free order.

This is where the Secret Agent and the Spaghetti Junction meet security. Kestrel & Pine's integration inventory (APIs, Events, Files and Middleware: How Commerce Systems Talk) found **34 integrations**, **13 without a named owner** as of January 2027 (5 were still unowned in April 2027, after the ownership map), and **41 apps**, three of which could write inventory (Who Builds and Who Owns: Teams, Vendors, Agencies and Governance). Every one holds a credential. A credential without an owner cannot be rotated when the agency's engineer leaves, cannot be scoped, and cannot be traced when the audit log shows an order changed "by API". The rule I use: **one identity per integration**, **one key per identity**, **the narrowest scope that works**, **an owner who can rotate it**, **and an entry in the inventory**. An app's requested scopes are its

security review: an app that asks to write inventory in order to send e-mails is declined, in procurement, not after installation (Apps, Plugins and Functions: Extensibility and Ecosystem Governance).

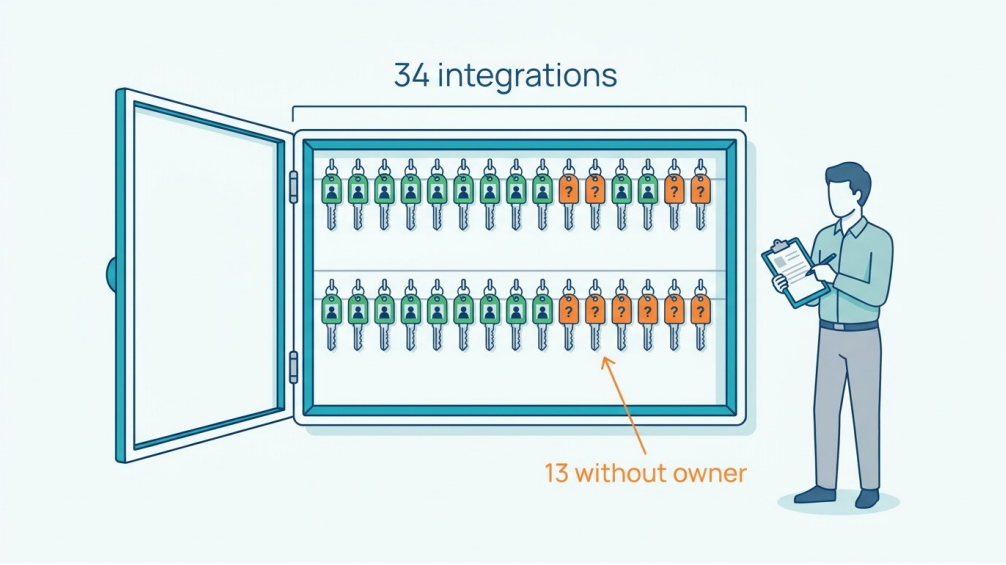


Figure 4: *Kestrel & Pine's inventory found 34 integrations, 13 without a named owner: every one holds a credential, and a key without an owner cannot be rotated, scoped or traced.*

The Choices: Four Security Postures

The four are **platform defaults**, **defaults plus the tools you already pay for**, **edge policy and identity management** (a CDN, the content delivery network in front of the storefront, with bot management, per-endpoint policies, a CSP, a vault, keys with owners, attestations on file, app review in procurement) and **a security function** (central identity, maintained inventories, monitoring with on-call). The last is right for a group or a vendor and the Giant's Boots for a merchant with fourteen engineers.

Posture	Cost (illustrative)	Closes	Leaves open	Right when
Platform defaults	Nothing	Card-data scope (SAQ A); the service itself	Card testing on guest checkout, takeover, keys, scripts, apps	One channel, no integrations, few staff
Defaults plus paid tools	Days of configuration; attention	Card testing, most takeover, the admin	Integrations' keys, scripts, apps, terminals	Two or three channels, first integrations
Edge policy and identity	A CDN tier, a vault, a quarter of an engineer *(assumption)*	Keys, scripts, apps, per-endpoint abuse	Monitoring depth, cross-brand consistency	Stores, wholesale, marketplaces, many integrations
Security function	Headcount; tooling	Nearly everything in the table	Cost, and controls nobody can explain	Group or vendor scale

The recommendation is dull and I hold it firmly: **move one rung at the moment the landscape gains the thing the rung protects**. The first integration with a key is the moment for the vault and the inventory; the first custom checkout is the moment for the CSP and script inventory; the first store terminal is the moment for the attestation file. Earlier buys controls nobody operates; later buys an incident.

Across Scale Stages: When It Fits and When It Limits

Stage	Typical posture	What becomes necessary	Warning signs of outgrowing it
1 — Starting (under about €2M)	Hosted checkout, SAQ A, the PSP's defaults, the founder's admin with MFA	Removing unused apps; no shared admin account; MFA on the PSP console	A custom checkout "for the brand"; a developer's key in a repository; the first decline spike
2 — Growing (about €2M–€20M)	Defaults plus paid tools; an agency with admin access; first vendor connectors	Challenges on guest checkout and login; velocity rules; MFA enforced for staff and agency; a decline-rate alert; a kill switch; a key per integration	Keys shared between agency scripts; nobody can list the apps; a PSP pause noticed at 07:30 (Kestrel & Pine, 2026)
3 — Omnichannel (about €20M–€200M)	Edge policy and identity management; store terminals; dozens of integrations	Per-endpoint bot policy; CSP with reporting; a vault and rotation; attestations for every provider; app review in procurement; three runbooks	Integrations without owners (13 of 34 at Kestrel & Pine in January 2027); a leaked key that reaches every channel; vendor logs the contract does not grant
4 — Group	A security function; central identity; one PCI program or a token vault across brands	Vendor allow-lists; a shared CSP policy; supplier clauses with incident clocks; monitoring with on-call	Each brand with its own PSP keys and no central revocation

The shape of the control barely changes with scale: identities, keys, scripts and endpoints are the same problem at every size. What changes is **the number of them, and whether anyone has a list**. A Starting-stage merchant holds the list in its head; an Omnichannel merchant with 34 integrations and 41 apps cannot, and that is the moment the Lynchpin

applies.

In Practice: Kestrel & Pine's 2027 Security Baseline

In April 2027 Noor Visser and Milan Petrov write a one-page security baseline: the second posture, plus the parts of the third the integration inventory demands. All figures are illustrative.

Item	What was true in August 2026	What is true from Q2 2027
Card-data scope	Hosted checkout, SAQ A; store terminals through the terminal provider; nobody held the providers' attestations	SAQ A confirmed with the acquirer; the script statement answered with the platform's confirmation; attestations for every payment provider on file, reviewed yearly
Guest checkout	No challenge, no velocity rule, no alert	Platform bot protection on checkout; PSP velocity rules per card, address and hour; a server-validated challenge when a payment request has no session history
Decline-rate alert and kill switch	Neither	Alert at three times the 30-day median to the on-call phone; a platform setting that disables guest card payments, tested quarterly

Item	What was true in August 2026	What is true from Q2 2027
Staff and vendor identity	31 platform admin accounts, 4 shared, MFA optional; the agency on one shared login	MFA enforced on the platform, the PSP console, the ERP (finance and invoicing), the WMS (warehouse management system) and the iPaaS (the integration platform); no shared accounts; agency engineers named; leavers removed within a day
Integration identities	34 integrations, 13 without owner in January 2027 (5 still in April), keys in scripts and the iPaaS, never rotated	One identity and one restricted key per integration, owner in the inventory, stored in a vault, rotated yearly or on exposure
Apps	41, of which 6 unused and 3 writing stock	6 removed; scopes reviewed; every new app through a procurement check of scopes and data access; a quarterly review
Checkout scripts	Unknown; the agency's theme loaded a tag manager	A script inventory with an owner and reason per script; CSP in report-only mode from May, enforcing from September
Runbooks	None	Three: a card-testing wave, a skimmer found on checkout, an admin or key compromised; each names who is called, the first three actions and the vendor contacts

The baseline reads the Card-Testing Night backwards: the alert fires at 23:55, the

challenge on session-less payment requests stops most of the 48,000, the kill switch ends the rest in a minute, and the runbook makes Milan's phone ring instead of a shared mailbox. The €2,900 was never the cost that mattered; the paused account and the two weeks of raised declines were.

When It Goes Wrong

A card-testing wave. Detection: the decline-rate alert, a 402 spike, orders with nonsense names, a PSP e-mail. Response, in order: kill switch for guest card payments; cancel and refund the orders that succeeded before they reach the warehouse; rotate every key if the attempts did not come through the checkout, because then they came through a leaked credential; afterwards watch the decline rate for weeks. Never retry the cards the attacker saved. If the PSP pauses the account, it is doing its job; have the contact and the evidence ready.

A skimmer found on checkout. Detection: a CSP violation report, a tamper alert, a shopper's bank, or a researcher's e-mail, which is how many are found. Response: remove the script through whatever loaded it (theme, app, tag manager) or switch to the PSP's redirect page so no script of yours is on the page where the card is typed; preserve the evidence; find how it got there, which is usually an identity rather than a vulnerability; then the legal clock of Whose Data Is It? Privacy, Consent and Retention starts, because card numbers are personal data.

An admin account or key compromised. Detection: a payout account changed, a script added, a bulk export in the audit log, a vendor's notice that a key was seen in public. Response: revoke, then investigate; rotate every key the account could see; check the payout account and the staff list; read the audit log from the first suspicious login. A merchant that cannot say which integration a leaked key belonged to must rotate everything.

Refund and coupon abuse. Not intrusions, so no edge control helps. Refund only to the original payment method; inspect before refunding above a threshold; bind one-time codes to a customer; give the returns portal a refund-only key. The Double Refund of January 2027 (The Way Back: Returns, Exchanges, Refunds and Reverse Logistics) is the same lesson from the inside: one writer per money movement.

What to Take Away

Established.

- Keep the card number out of your systems: a hosted checkout or hosted fields puts the merchant in PCI DSS SAQ A with tokens and the last four digits as the only card data held (Stripe and Adyen guidance, n.d.; PCI DSS v4.0.1, 2024).

- The scripts on the payment page are the merchant's responsibility: requirements 6.4.3 and 11.6.1 are mandatory since 31 March 2025, and SAQ A merchants must state that their page is not susceptible to script attacks (PCI SSC, 2024 and 2025); British Airways and Polyfill are why.
- Card testing and credential stuffing are defeated by layered controls, not single filters (Stripe documentation; OWASP, n.d.).
- Shared responsibility is published and consistent: the platform secures the service; the merchant owns identities, keys, apps, scripts and integrations (Adobe, Shopify, Stripe, n.d.).

Depends on context.

- How far down the posture ladder to go depends on what the landscape contains; the trigger is the first key, the first custom checkout, the first terminal.
- Whether to challenge guest checkout depends on the payment mix: where most payments carry their own authentication (iDEAL, 3-D Secure cards), a challenge on card-only, session-less requests costs little; where plain card dominates, the false-positive cost is real and the PSP's velocity rules should carry more of the load.
- On a self-hosted platform the patch cadence is a security control in its own right (CosmicSting, 2024); on SaaS the saved effort should go into identities and scripts.

The author's synthesis.

- For a merchant that rents its platform, security is an **inventory problem before it is a technology problem**: a list of every identity, key, app, script and endpoint, with an owner, is worth more than any product, because every incident in this chapter started with something nobody had on a list.
- The decline rate belongs next to orders per minute on the dashboard: the two signals catch the two ways a checkout is used in bad faith.

Questions to Ask

1. Which SAQ does our acquirer say applies to us, and who holds the attestation of compliance for every provider that touches payments?
2. Can anyone list the scripts that load on our checkout page, with an owner and a reason for each, and would a CSP tell us if a new one appeared?
3. Which endpoints can trigger a payment authorization or a login, and which has no challenge, rate limit or session requirement?
4. Is multi-factor authentication enforced, not merely available, on every admin console, including the agency's and the partner's accounts?

5. Does every integration and app have its own identity and a scoped key, with an owner who can rotate it today?
6. If the PSP paused our account at two in the morning, who would know, and what would they do first?
7. What is our normal decline rate, would anyone be alerted if it tripled, and can we switch off guest card payments without a deploy?
8. Which apps can write inventory, prices, orders or refunds, and did anyone review those scopes before installation?

Security decides who may act on the shop. The next chapter asks the mirror question about the data the shop holds on people: who may use it, for what, for how long. Continue with Whose Data Is It? Privacy, Consent and Retention.

Sources and Evidence

Company, product and investor sources

- Protect yourself from card testing — Stripe Docs, n.d., accessed 2026-11; the attack's mechanism, symptoms, consequences for the decline rate, controls, the retry warning and the leaked-key vector.
- Integration security guide — Stripe Docs, n.d., accessed 2026-11; shared responsibility, what card data may be stored, third-party JavaScript, webhook signatures.
- Best practices for managing secret API keys — Stripe Docs, n.d., accessed 2026-11; restricted keys, vaults, rotation, exposure handling.
- PCI compliance guide — Stripe, n.d. (Dutch-language edition), accessed 2026-11; merchant levels and the SAQ by integration pattern.
- PCI DSS compliance guide — Adyen Docs, n.d., accessed 2026-11; where the PSP's responsibility starts, attestations and requirement 12.8.
- Script security for ecommerce — Adyen Docs, n.d., accessed 2026-11; CSP, script inventory, SRI and violation reports as the 11.6.1 alert source.
- Security and PCI compliance — Shopify, n.d., accessed 2026-11; Level 1 certification extending to all stores, and the merchant's own steps.
- Two-step authentication — Shopify Help Center, n.d., accessed 2026-11; who can enforce MFA, SMS retirement, payouts on hold.
- Shared responsibility — Adobe Experience League, n.d., accessed 2026-11; the merchant's and Adobe's duties on Adobe Commerce cloud infrastructure.
- Polyfill supply chain attack hits 100K+ sites — Sansec, 2024-06-25; the compromise, its evasion logic and the CSP advice.
- CosmicSting — Sansec, 2024, accessed 2026-11; the Adobe Commerce flaw, its fix dates and the breach campaigns.

- 2025 Bad Bot Report — Imperva, 2025; vendor figures on automated traffic, quoted as such.

Standards, regulation and official documentation

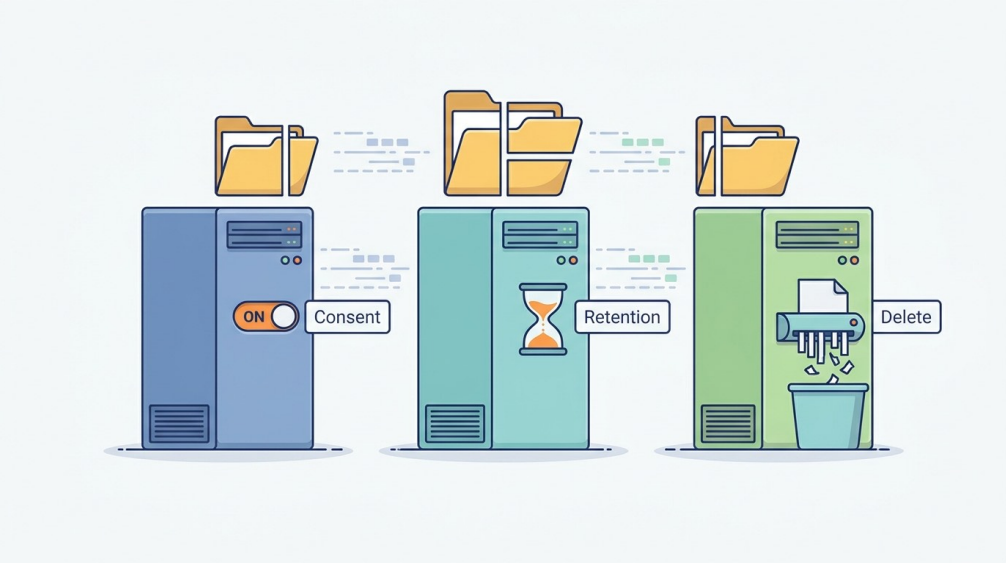
- Just Published: PCI DSS v4.0.1 — PCI Security Standards Council, 2024-06-11; publication of v4.0.1, retirement of v4.0 on 31 December 2024, v4.0's March 2022 publication, the unchanged 31 March 2025 effective date.
- Guidance for PCI DSS e-commerce requirements effective after 31 March 2025 — PCI Security Standards Council, 2025-03-26; requirements 6.4.3 and 11.6.1, their removal from SAQ A and the eligibility statement.
- OAT-001 Carding — OWASP Automated Threats to Web Applications, n.d., accessed 2026-11; the threat identifiers used in this chapter.
- Credential stuffing — OWASP, n.d., accessed 2026-11; definition and MFA as countermeasure.

Research and practitioner sources

- British Airways data breach — Wikipedia, citing the ICO penalty notice of 16 October 2020, accessed 2026-11; the entry point, the script, the numbers and the penalty. Eight years after the incident; the ICO's own pages could not be fetched.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

35. Whose Data Is It? Privacy, Consent and Retention



IN THIS CHAPTER, YOU WILL: Learn where personal data lives in a commerce estate, which rules govern each use of it, and how to design consent, retention and deletion across many systems.

KEY POINTS:

- **Every integration is also a data flow with a legal role.** The merchant decides why and how customer data is used, so it is the controller; the platform, the warehouse system and most apps act on its instructions as processors; carriers, marketplaces and payment providers usually decide for themselves. The system-of-record map needs that column.
- **The order and the marketing profile are two different things in law, even when they describe one person.** The order rests on the contract and on tax law and must be kept for years; the profile rests on consent and must go when the person says so. Design them to separate: pseudonymize the transaction, erase the person.

- **A deletion request is a fan-out with a deadline**. One month, every system that holds the person, a different right answer in each, and a record of what was done. Without an identity index and an orchestrator it is a seventeen-day hunt through nine systems; with them it is a job.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Familiar Stranger** — a person the systems cannot recognize as one cannot be deleted, told the truth about, or respected when they opt out.
- **The Secret Agent** — an app, script or export that copies customer data without a named owner or a contract is a processor nobody appointed.
- **The Long Fuse** — a data subject request has a legal clock of one month, and a cookie or transfer rule can change with a court decision; both are discovered late by merchants with no map.

A commerce platform is, among other things, a machine for collecting personal data: a name, an address, an e-mail, what someone bought, what they returned and why, which pages they looked at. Every one of those is needed by someone, and every one is, in European law, personal data about a person who has rights over it.

The trouble is not that a merchant holds personal data but that it holds it in a dozen systems from a dozen vendors, copied by integrations nobody mapped for this purpose, under rules that differ by system and by country. **Privacy in commerce is less a question of policy than of architecture: where is the person, who may hold them, for how long, and how do you reach every copy when they ask?** This chapter is not legal advice; it is an architect's reading of the rules.

This chapter owns four terms: **controller and processor**, **consent**, **retention** and the **data subject request**. It builds on One Customer, One Product: Identity and Master Data Across Systems, which decided which system knows who a customer is, and hands the wider regulatory calendar to Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.

Where we are in the order's journey. When Eva Janssen orders her jacket on 13 April 2027 (the recurring order; see Meet Kestrel & Pine), her name and address go to the platform, the warehouse management system (WMS), the carrier and the payment service provider (PSP), her invoice to the ERP (the enterprise resource planning system that keeps the books), and her e-mail to the notification service and, if she ticked the box, the newsletter tool. When she exchanges the jacket on 19 April, the returns portal joins the list. Each system received the data for a reason, and each reason has its own rule for how long it may stay.

Setting the Scene at Kestrel & Pine: The Request From Ghent

Kestrel & Pine, the fictional outdoor brand this book follows, received a message through the Belgian storefront's contact form on **Monday 10 May 2027**. A customer in Ghent, call her Lotte Claes, wrote in Dutch: please delete all my data and stop e-mailing me. The details are illustrative.

Lotte was a modest customer: two orders on the Belgian storefront (October 2025 and November 2026), one bol order (March 2026), one return, a newsletter subscription since 2025 and a loyalty sign-up at the Antwerp store's till, the POS (point-of-sale system), in December 2026. The request reached Noor Visser, who asked the question this chapter is about: **which systems hold Lotte, and what should each of them do?**

Ruben Alves and the application support team took seventeen working days to find out. She was in nine of the thirteen systems:

System	What it held	What they did	What they could not do
The platform	Account, addresses, two orders, consent flag, the apps' copies	Deleted the account; the platform anonymized the orders and notified its apps	Confirm that every installed app complied
The marketplace connector	A copy of the bol order with name and address	Anonymized the copy	Touch bol's own record; bol is a separate controller
The ERP	Two consumer invoices and a credit note	Nothing: invoices are tax records	Erase; retention rules apply
The WMS	Three shipments with name and address	Deleted addresses from shipments older than a year	Erase recent shipments while a claim is possible
The shipping service	Labels and tracking for three parcels	Deleted after the claims window	Reach bpost's own delivery record
The PSP	Payment records for two orders	Removed shopper details from the console view	Erase the PSP's own records

System	What it held	What they did	What they could not do
The POS	A loyalty entry with phone number and five purchases	Deleted the entry; receipts kept as fiscal records	Unlink receipts already exported to the ERP
The returns portal	A contact and one return	Deleted the contact; anonymized the return	Nothing further
The data warehouse	Rows keyed by e-mail in four tables and a BI extract	Replaced identifiers with a hash	Reach a spreadsheet export made in 2026

The newsletter tool, one of the platform's apps, unsubscribed her through the platform's notification; the iPaaS (the integration platform) held her in three days of logs that expired on their own; nobody checked the backups. The reply went out on **4 June 2027**, inside the one-month limit with six days to spare. It had taken three people and a shared spreadsheet. **The request was answered. The next one would arrive before anyone had written down how.**

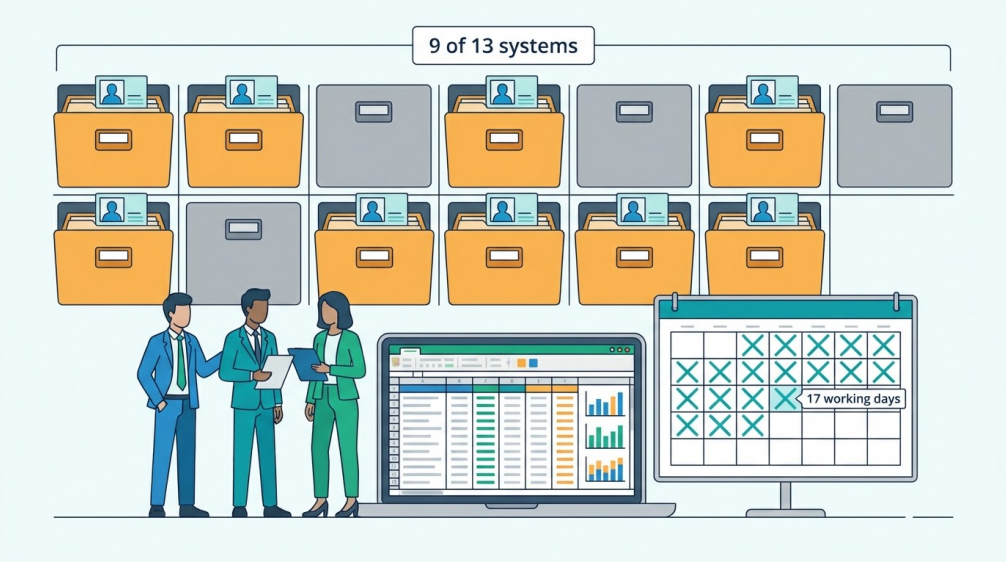


Figure 1: *One customer's deletion request found her in nine of the thirteen systems and took seventeen working days, three people and a shared spreadsheet to answer.*

What Personal Data a Commerce Platform Holds, and Under Which Rules

The vocabulary, because the words carry the design. **Personal data** is any information relating to an identified or identifiable person: a name, an e-mail, an order with an address, a cookie id tied to a browser. The **General Data Protection Regulation (GDPR)**, Regulation (EU) 2016/679, has applied across the European Union since 25 May 2018; it gives the person rights (access, rectification, erasure, objection, portability) and the organizations that handle the data duties.

A **controller** is the organization that decides why and how personal data is used; a **processor** handles data only on the controller's instructions (Article 4(7) and 4(8)). A **data processing agreement (DPA)** is the contract Article 28 requires between them. A **lawful basis** is the legal reason a controller may use the data at all; Article 6 lists six, of which four matter here: consent, performance of a contract, a legal obligation and the controller's legitimate interests. A **data subject request (DSR)** is a person exercising a right: send me my data, correct it, delete it. **Retention** is how long data is kept, and the law pulls two ways: no longer than the purpose needs, and at least as long as tax law demands.

The fine ceiling for the core infringements is €20 million or 4% of worldwide annual turnover (Article 83(5)). For Kestrel & Pine, with €48.0 million of 2026 revenue, 4% would be €1.9 million; commoner sanctions are orders to stop processing and the public finding.

What a merchant like Kestrel & Pine holds, by purpose:

Data	Where it lives	Purpose	Usual lawful basis	How long it is needed
Name, delivery and billing address	Platform, OMS (order management system), WMS, shipping service, ERP, carriers	Deliver and invoice	Contract; legal obligation for the invoice	Order: the fiscal period; shipment: the claims window

Data	Where it lives	Purpose	Usual lawful basis	How long it is needed
E-mail and phone	Platform, notification service, returns portal, POS, newsletter tool	Order messages; service; marketing	Contract for order messages; consent or the soft opt-in for marketing	Service: while the account exists; marketing: until withdrawn
Order history, returns, refunds	Platform, OMS, ERP, returns portal, PSP, data warehouse	Fulfill, account, support, warranty	Contract; legal obligation; legitimate interests for fraud	Fiscal period; then pseudonymized
Browsing and behavior	Analytics, advertising tags, CDP (customer data platform), personalization	Measurement; marketing	Consent (the ePrivacy Directive for the device; GDPR for the profile)	Until withdrawn or aggregated

The right-hand columns are the design. The same person appears in every row, but each row has its own basis and its own clock. **The order record and the marketing profile need separate lawful bases, separate retention clocks and separate deletion behavior, even though they refer to the same person.**

Controller, Processor and the Commerce Supply Chain

Who is the controller for Eva's data? The merchant: Kestrel & Pine decides that orders are taken, delivered and invoiced, that a newsletter exists, that a warehouse system is used. The guidelines of the European Data Protection Board (EDPB) on the two roles (version 2.0, July 2021) draw the line: decisions on the purpose of processing "are clearly always for the controller to make"; the "essential means" (which data, for how long, who receives it, whose data) are the controller's; the "non-essential means" (the choice of hardware or software, the detailed security measures) may be left to the processor.

Applied to the commerce supply chain, the usual outcome of that test, as the author

reads it and not as a rule:

System	Role for the merchant's customer data	Why
The hosted platform, the WMS, the OMS, the iPaaS, the returns portal, the data warehouse	Processor	Tools the merchant chose, used for the merchant's purposes under its instructions and a DPA
Most apps	Processor	Installed by the merchant to act on its data; each needs a DPA
The PSP	Processor for the merchant's order data; usually an independent controller for the payment itself	Payment providers have their own legal duties (anti-money-laundering, fraud) that they decide on
The carrier; the marketplace	Independent controllers	The carrier decides how it delivers and keeps its own records; the shopper is the marketplace's customer
Advertising platforms	Often joint or independent controllers	They use the data for their own purposes too

Platforms encode this in their app rules. Shopify requires every App Store app to subscribe to three compliance webhooks: `customers/data_request`, `customers/redact` (sent "10 days after the deletion request") and `shop/redact` (48 hours after a store owner uninstalls the app); apps must complete the action within 30 days and may decline redaction where the law requires them to retain data (developer documentation, n.d., accessed November 2026). Its protected-customer-data program gates name, address, phone and e-mail behind approval and requires minimum data, stated purposes, respect for consent, retention periods and encryption. That is a platform acting as the merchant's processor and policing the sub-processors on its behalf.

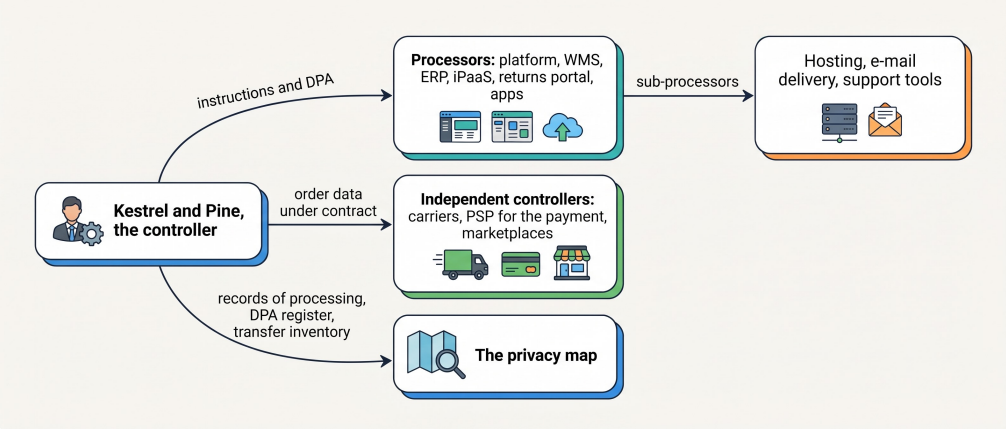


Figure 2: *The merchant is the controller; most of its systems are processors it must instruct and contract; carriers, payment providers and marketplaces decide for themselves.*

The consequence: **the system-of-record map from Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record needs a column for each system's legal role, and the integration inventory from APIs, Events, Files and Middleware: How Commerce Systems Talk is also the inventory of data flows.** The thirteen integrations with no named owner in the integration inventory as of January 2027 were thirteen data flows with nobody accountable for them; five were still unowned in April 2027, after the ownership map.

Lawful Bases: The Order Versus the Marketing

In commerce terms, the four bases divide the work like this:

Use	Basis	In practice
Taking, delivering, invoicing and supporting the order; warranty; returns	**Contract**	Needed to do what the customer asked; no consent needed; cannot be withdrawn while the contract runs
Keeping invoices, accounting records, VAT evidence	**Legal obligation**	Kept for the statutory period even if the customer objects

Use	Basis	In practice
Fraud screening; security; basic analytics; defending claims	**Legitimate interests**	Allowed where the merchant's interest is not overridden by the person's; documented; can be objected to
Newsletters; profiling for personalized marketing; cross-site tracking	**Consent**, or for e-mail to existing customers about similar products, the ePrivacy **soft opt-in**	Freely given, specific, informed, recorded, and as easy to withdraw as to give

The soft opt-in deserves a sentence because every merchant asks about it. The ePrivacy Directive (Article 13(2), 2002 text consolidated in 2009) lets a business that obtained a customer's e-mail "in the context of the sale of a product or a service" use it "for direct marketing of its own similar products or services", provided the customer can object, free of charge, at collection and in every message. National transpositions differ; the lawyer confirms the local reading. The architectural point holds either way: **the order's e-mail address and the marketing permission are two facts, and the second must be recorded with its basis**.

Consent, Cookies and Tracking

Consent lives in two laws. The GDPR defines it (Article 4(11)) as a freely given, specific, informed and unambiguous indication of the person's wishes by a statement or a clear affirmative action, and requires the controller to be able to demonstrate it (Article 7(1)). The ePrivacy Directive (Article 5(3), as amended in 2009) adds a rule about the device: storing information on, or reading information from, a user's device is allowed only with consent after clear information, except for what is strictly necessary to transmit a communication or to provide a service the user explicitly asked for.

In plain words: the cart cookie and the language choice need no consent; the analytics tag, the advertising pixel and the personalization script do, before they fire. The regulators Kestrel & Pine answers to agree on the substance:

- The **EDPB's consent guidelines** (May 2020) forbid cookie walls and say scrolling or swiping "will not under any circumstances" count as consent; its technical-scope guidelines (version 2.0, October 2024) bring tracking pixels, URL tracking and IP-only tracking within the device rule.
- The **Dutch regulator** (Autoriteit Persoonsgegevens, n.d., accessed November 2026) exempts functional cookies and "limited analytical cookies" with little privacy impact, requires consent for tracking cookies, and calls cookie walls prohibited.

- The **Belgian regulator** (Gegevensbeschermingsautoriteit, November 2023) exempts only strictly necessary functional and technical cookies (naming the language choice, cookie preferences and the shopping basket), rejects pre-ticked boxes and continued browsing, and requires cookies to be grouped by purpose, with own and third-party advertising as separate purposes.

A **consent management platform (CMP)** is the banner and the record behind it: it stores each visitor's choices per purpose and tells each tag whether it may run. **Server-side tagging**, in which the merchant's own server receives the browser's events and forwards them to vendors, changes who receives the data first, not whether consent is needed: the browser is still told to emit identifiers, so the device rule still applies. Present it as a governance improvement, never as a consent bypass; that reading follows the EDPB's technical-scope guidelines and is the author's.

Two moving parts. The Commission's **Digital Omnibus** proposal of November 2025 would move the device rule into the GDPR with four no-consent purposes (transmission, requested service, audience measurement, security), require a one-click refusal and oblige controllers to honor machine-readable browser signals; one law firm's reading (Osborne Clarke, December 2025) is that the end of cookie banners "is not yet in sight". It is a proposal, not law, as of November 2026. In the United States the model is opt-out: California's law lets consumers opt out of the "sale or sharing" of their data, including through the Global Privacy Control browser signal, and the 2025 chart of the IAPP (International Association of Privacy Professionals) lists nineteen state laws of that family.

The design consequence joins this chapter to the previous one. **Consent is state that must travel**: from the CMP to the tag layer, to the newsletter tool and the CDP, to the POS where staff ask for a loyalty sign-up, and to every app that sends a message. One Customer, One Product: Identity and Master Data Across Systems put it on the customer master with its time, place, wording and source; here is why: the merchant must demonstrate it per person and per purpose, and withdraw it everywhere at once.

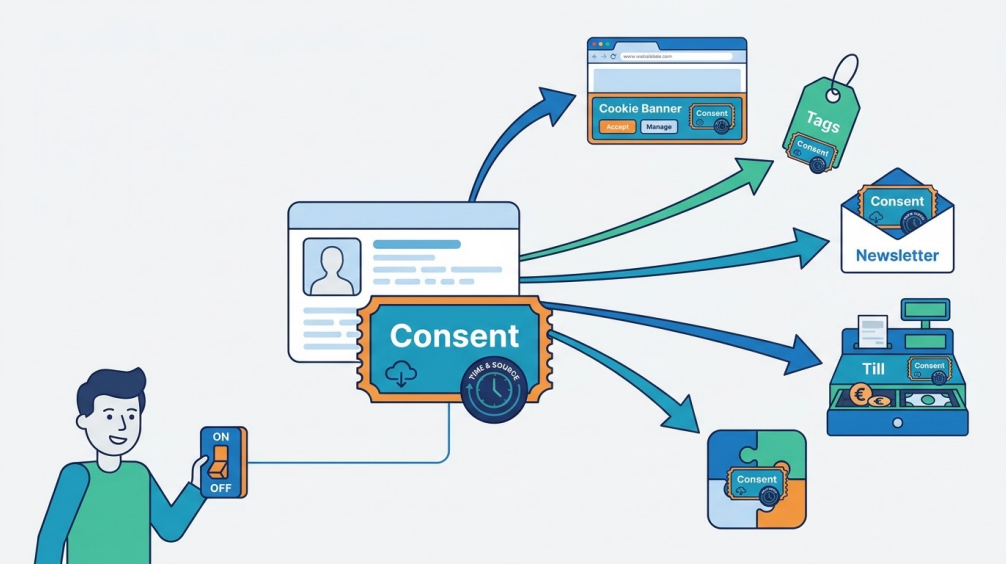


Figure 3: *Consent is state that must travel: recorded once with its evidence on the customer master, it must reach every tag, tool, till and app, and a withdrawal must reach them all at once.*

Requirements per flow

Flow	How fresh	How consistent	How fast	How available
Consent capture (banner, checkout box, store sign-up)	Immediate for the device; hours to reach every outbound tool	One record per person per purpose, with evidence	Must not slow the page or the till	Degrade to "no consent" when the CMP is down, never to "consent"
Consent withdrawal	Immediate for the channel used; same day everywhere	Strictest state wins across tools	Minutes	Works even if the master is unreach-able; reconcile later

Flow	How fresh	How consistent	How fast	How available
Data subject request	One month, extendable by two	Every system that holds the person answers; answers recorded	Days	Batch; the orchestrator can queue
Retention purge	Daily or weekly	Pseudonymize transac-tions; erase profiles; log what ran	Hours	Batch; a failed run must be visible

Data Subject Requests Across Many Systems

The GDPR requires the controller to act on a request "without undue delay and in any event within one month", extendable by two months for complex or numerous requests (Article 12(3)). What makes it hard is not the law but the fan-out: one person, many systems, a different right answer in each. Three parts make it a job.

An identity index: which systems hold this person, under which identifiers. This is the cross-reference One Customer, One Product: Identity and Master Data Across Systems built for another reason; a deletion request is its second customer. Without it, every request starts with a search of every system by e-mail, phone and name, which is where Lotte's seventeen days went.

A request orchestrator: a small workflow that verifies the requester (asking for "all my data" with someone else's e-mail is a way to steal it), fans the request out, records each answer and produces the reply. Shopify's compliance webhooks are the platform-side half of one; for the systems outside the platform, the merchant builds or buys the other half.

A per-system action table, decided in advance:

Answer	When	Example at Kestrel & Pine
Erase	The purpose has ended and no law requires the data	The platform account, the loyalty entry, the newsletter subscription, the returns contact, old shipment addresses

Answer	When	Example at Kestrel & Pine
Pseudonymize	The record must stay for accounting, statistics or evidence, but the person need not be identifiable from it	Orders in the platform and the OMS, warehouse rows, data-warehouse rows: name and address replaced by a token the merchant cannot reverse
Retain	A legal obligation applies or a claim is still possible	Invoices and credit notes in the ERP; recent shipments during the carrier's claims window; fraud evidence

Two answers lie outside the merchant's reach: **refer** (the carrier, the marketplace and the PSP are told, and the person must ask them too, because they are controllers in their own right) and **except** (backups, under a documented, bounded exception: erased data is not restored, and if a backup is restored the deletion is replayed).

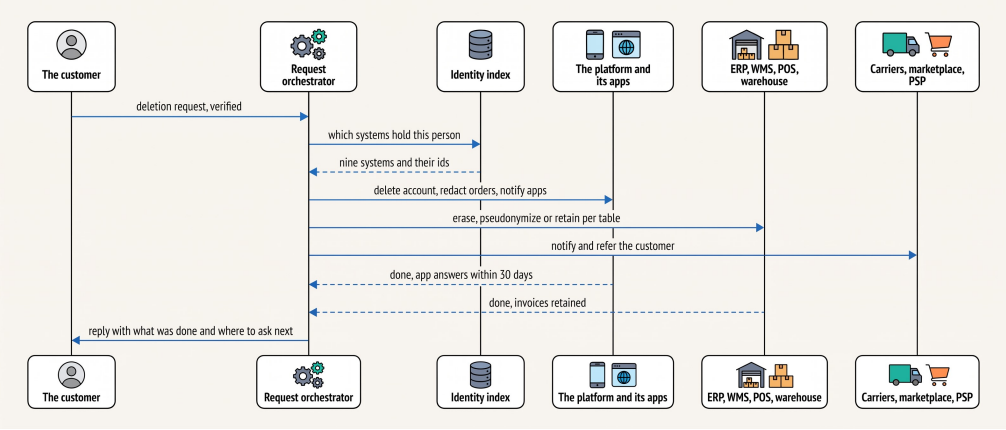


Figure 4: *A deletion request is verified once, resolved against the identity index, fanned out with a predetermined action per system, and answered with a record of what each system did.*

The reply should say, in plain words, what was erased, what was kept and why, and whom else to ask; the record behind it is what a regulator asks for.

Retention: The Person Versus the Transaction

Tax law makes the merchant keep what privacy law makes it release. One design move reconciles them: **split the person from the transaction**. The retention facts, with their

status:

- **Netherlands**. The government's business portal states that businesses "must keep your business records for at least 7 years", naming the general ledger, stock records, purchase and sales records and debtor and creditor accounts, with at least 10 years for data on immovable property, and requires digital files to remain accessible for inspection (business.gov.nl, n.d., accessed November 2026).
- **Germany**. §147 of the Abgabenordnung requires books, records, inventories and annual accounts to be kept for ten years, accounting vouchers for eight, and other records including commercial letters for six, from the end of the calendar year of the record (gesetze-im-internet.de, current text, accessed November 2026).
- **Belgium**. Ten years for accounting records and invoices is widely cited; the official text was not verified for this chapter, and Sanne de Wit treats ten years as the Belgian rule pending confirmation.

So Lotte's invoice from November 2026 stays until at least the end of 2036 under the Belgian reading, whatever she asks; "legal obligation" is a lawful basis. The conflict arises only when the invoice *is* the customer record, as it is in many ERPs and platforms: the order carries name, address and e-mail, and the customer master is the list of people with orders. Deleting the person then means deleting the order, which tax law forbids, so nothing is deleted.

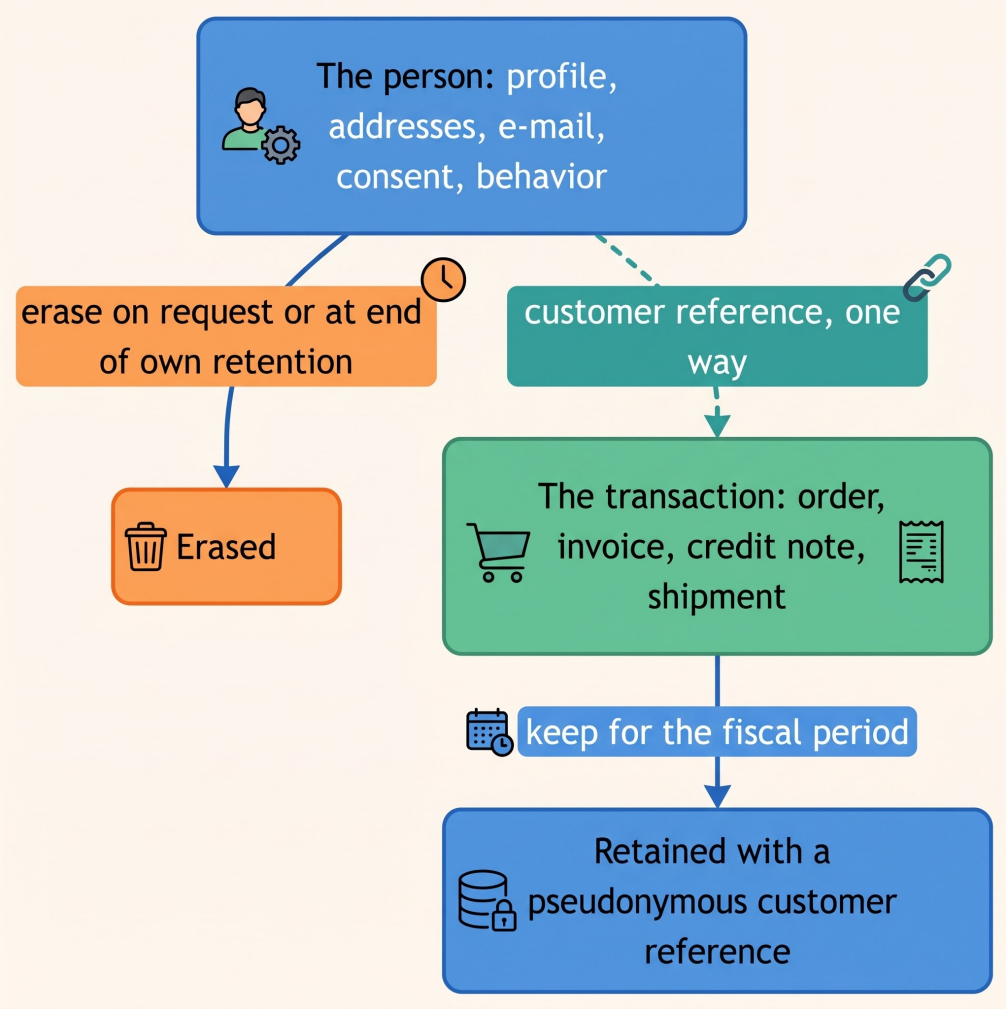


Figure 5: *Profiles are erasable; transactions are kept with a reference that no longer points to a person once the profile is gone.*

In practice: the order and the invoice carry a customer reference and the delivery address (part of the transaction, kept for the fiscal period); the profile (e-mail, phone, preferences, consent, browsing, loyalty) is a separate, erasable record; and the analytics tables carry a hash of the customer reference, not the e-mail. **Retention periods become a property of each record type, not of the person**, and a nightly job purges what has expired without anyone deciding per customer.

Backups need a sentence. A deletion cannot reach into last month's backup; the accepted practice is a bounded, documented exception: backups are kept for a stated period, never used to resurrect erased data, and if one is restored the deletion log is replayed against it.

Sharing, Transfers and the Store

Sharing with partners. Every order sends personal data to a carrier, the PSP, sometimes a marketplace, and to apps and connectors. The rules are minimization (the warehouse does not need the e-mail, the carrier does not need the order history), a recorded role per recipient, and a DPA or terms that cover the transfer. Kestrel & Pine's marketplace connector copied everything the marketplace sent; after Lotte's request it copies what the warehouse needs.

International transfers. Most commerce stacks send data to the United States somewhere: analytics, e-mail delivery, support tooling, hosting. The lawful routes are an adequacy decision, standard contractual clauses (SCCs) or binding corporate rules. The EU-US Data Privacy Framework (DPF), Commission Implementing Decision (EU) 2023/1795 of 10 July 2023, is the adequacy decision for certified US companies; the modern SCCs are Decision (EU) 2021/914. The General Court dismissed Philippe Latombe's challenge to the DPF on 3 September 2025; an appeal (Case C-703/25 P) is pending as of November 2026; law firms advise keeping SCCs as a fallback. The architectural asset is a **transfer inventory** keyed to the integration map: per vendor, where the data goes and under which mechanism, so that a court decision becomes a filter on a list rather than a discovery project.

The store. The POS holds a loyalty list and receipts; a store may also run cameras, Wi-Fi analytics and tablet sign-ups. Consent asked at a till by a member of staff is easy to ask badly and leaves no evidence. Kestrel & Pine's rule from mid-2027: the POS loyalty sign-up shows the same purposes and wording as the web, records the time and the store, and carries the master's consent once matched (One Customer, One Product: Identity and Master Data Across Systems).

The vendor's side. For a commerce software vendor, the obligations are a processor's: a DPA you can sign, sub-processors listed and announced, data kept in the regions you promise, the ability to export and erase one customer of one merchant, security you can evidence, and breach notification to the merchant without undue delay (Article 33(2)) so that it can meet its own 72-hour deadline.

The Choices

For the data subject request, four realistic designs:

By hand, with a runbook: a written list of systems, the action per system and the person responsible. Fits a Starting or Growing merchant with few systems and few requests; Lotte's seventeen days were this, without the runbook. **Platform-centered**: the platform's own erasure plus its app fan-out handles the storefront side; a runbook covers the rest. Fits a merchant whose customer data mostly lives in the platform and its apps. **An orchestrator on the integration platform, with the identity index**: the

request becomes a flow (verify, resolve, fan out, record, reply). Fits the Omnichannel stage, where a dozen systems hold the person and requests arrive weekly; it is Kestrel & Pine's design from autumn 2027. **A privacy management product**: software that automates intake, verification, discovery across connected systems, consent records and the records of processing. Fits the Group stage; my judgement is that below it the product is bought to avoid a map that still has to be made.

Option	Fits when	Strengths	Costs and limits	Warning sign
Runbook by hand	Few systems, few requests	No build; forces the map to be written	Slow; no record unless disciplined	The runbook is a person's memory
Platform-centered	Most data in the platform and its apps	Erasure and app fan-out come with the platform	ERP, WMS, POS and warehouse are outside it	"The platform handles GDPR"
Orchestrator plus identity index	A dozen systems; weekly requests	Repeatable; a record per request; reuses the identity index	A flow to own; per-system steps; a human verification step	Requests queued behind other integration work
Privacy management product	Several brands, countries, entities	Intake, verification, discovery, consent and records in one place	License and configuration; connectors still map to your systems	Bought before the map exists

For consent the ladder is shorter: the platform's built-in banner and checkout box (Starting), a CMP with per-purpose consent synchronized to the customer record (Growing), and consent as an attribute of the master with evidence, synchronized to every outbound tool (Omnichannel onward).

Across Scale Stages: When It Fits and When It Limits

The four stages are the book's (see What Changes at Each Scale: From First Store to Group).

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
1 — Starting	The platform's standard DPA, cookie banner, checkout consent box and app vetting; requests handled in the platform admin	A privacy notice that matches the stack; a DPA for the 3PL and the e-mail tool; one person who answers requests	An app with no DPA; a newsletter list built from every order without a soft opt-in notice
2 — Growing	The first store, ERP and marketplace bring data outside the platform; a CMP with per-purpose consent	A DPA register and the rule that nothing goes in without one; a runbook for requests; consent on the customer record; a retention rule per record type	A request that takes two weeks; POS consent that never reaches the newsletter tool; tags firing before the banner answers

Stage	What is typical	What becomes necessary	Warning signs of outgrowing it
3 — Omnichannel	A dozen systems, 20–40 flows, requests weekly; a data warehouse keyed by e-mail	A data-flow map kept with the integration inventory; an identity index and a request orchestrator; the person split from the transaction; a transfer inventory; a bounded backup exception	A request that touches nine systems by hand; an erased customer resurrected by a backup; a court decision that forces a vendor-by-vendor investigation
4 — Group	Several brands, countries and legal entities, each a controller; a privacy function	One consent model with national differences as configuration; a privacy management product; a data protection officer where required; group-wide records of processing	One brand's consent used by another; two brands answering the same person differently

The judgement: the duties do not change with scale; a Starting-stage merchant is as much a controller as a Group. What changes is where the data is. Below the Omnichannel stage the platform holds most of it and its DPA, banner and app rules do most of the work; from the Omnichannel stage the person is in a dozen systems and the merchant must own the map, the index and the orchestrator itself. Kestrel & Pine learned the stage-3 lesson from one request from Ghent.

In Practice

Shopify's compliance webhooks and protected customer data (accessed November 2026) are the clearest public example of a platform acting as the merchant's processor

and policing its sub-processors: three mandatory webhooks, a 30-day completion window, and a gated program for the four identifying fields.

Sephora, California, August 2022. A case more than three years old, but still the reference example: the Attorney General's settlement (US$1.2 million) found that third-party trackers on Sephora's site and app amounted to a sale of personal information without notice, and that Global Privacy Control signals were not honored. The tag layer is a legal surface, and a browser signal must be read as well as a banner.

The DPF and its appeal (2023–2026). A merchant whose transfer inventory names each US vendor's mechanism can respond to the outcome in an afternoon.

Kestrel & Pine's deletion design (autumn 2027, illustrative)

After Lotte's request, two changes. In **June 2027** the identity rule linked every system's record to the customer master (One Customer, One Product: Identity and Master Data Across Systems), and the July matching run produced the index. In **October 2027** Ruben Alves built the request orchestrator as an iPaaS flow with a human verification step: the request is logged, the requester verified through the platform's account or a signed e-mail reply, the index queried and the action table applied. The platform deletes the account, anonymizes the orders and notifies its apps; the OMS (live for marketplace orders from 15 September) pseudonymizes its order copies; the WMS and the shipping service purge addresses older than their claims window of **twelve months** *(assumption)*; the ERP keeps invoices and credit notes and marks the customer reference as erased; the POS deletes the loyalty entry; the returns portal deletes the contact; the data warehouse replaces e-mail and name with a hash. The carrier, the marketplace and the PSP are notified and the customer told to ask them directly. Backups are kept for **35 days** *(assumption)* and the deletion log is replayed after any restore.

The second request through the flow, in November 2027, took **two working days** *(illustrative)*. The map that made it possible, one row per system with its legal role, data, action and retention clock, is now a column of Noor's ownership map, reviewed quarterly.

When It Goes Wrong

Failure	How it shows	Detection	Recovery
The missed copy	An erased customer receives a newsletter because nobody reached the newsletter tool	A complaint; a reconciliation of erased master ids against each tool's list	Erase; add the tool to the index and the action table; ask who else was missed
The resurrected customer	A restore from backup brings back erased records	The deletion log replayed after a restore	Replay the log; document the backup exception
The impostor	Someone requests "all my data" with another person's e-mail	A verification step that fails	Never answer an unverified request with data; verify through the account or a signed reply
The over-deletion	An invoice or a fraud record is erased with the profile	The ERP's retention check; a tax audit	Restore from the retained copy; fix the action table: transactions retained, the person erased
Consent without evidence	Marketing cannot show when or how a person consented	A consent audit sampling records	Treat unevidenced consent as absent; re-ask where the soft opt-in does not apply
The invalidated transfer	A court or a decision removes a transfer mechanism	A transfer inventory naming the mechanism per vendor	Switch to SCCs or an EU region, vendor by vendor from the list

The missed copy is the one that recurs, and its cause is a system added after the map was made; that is why the privacy map lives with the integration inventory and is reviewed when a row is added.

What to Take Away

Established.

- The merchant is the controller of its customers' data; its platform, warehouse system, integration platform and most apps are processors that need a contract; carriers, payment providers for the payment itself, and marketplaces usually decide for themselves (GDPR Articles 4 and 28; EDPB Guidelines 07/2020).
- Orders rest on the contract and on tax law; marketing rests on consent or the soft opt-in; consent must be demonstrable, and cookie walls, scrolling and pre-ticked boxes do not produce it (GDPR Articles 6 and 7; EDPB Guidelines 05/2020; the Dutch and Belgian regulators).
- Requests must be answered within one month, extendable by two (Article 12(3)); business records must be kept at least seven years in the Netherlands and ten, eight or six years in Germany depending on the record (business.gov.nl; §147 AO).
- Platforms have operationalized the fan-out: Shopify's three compliance webhooks and 30-day window are a documented, dated example.

Depends on context.

- Below the Omnichannel stage the platform's DPA, banner, consent box and app rules do most of the work; from the Omnichannel stage the merchant must own a data-flow map, an identity index and an orchestrator, because the platform's reach ends at its apps.
- The soft opt-in depends on national transposition and on the "similar products" test; take the lawyer's reading per country.
- The retention period per record type is a legal and finance decision per country; the architecture needs only that it be a property of the record type, not of the person.

The author's synthesis.

- Privacy in a commerce estate is an integration problem wearing a legal coat: the map, inventory and identity index that fix the Phantom Stock and the Familiar Stranger make a deletion request a job rather than a hunt.
- Split the person from the transaction in the data model before the first request, because every system that stores the name on the order otherwise forces a choice between the tax law and the person.

Questions to Ask

1. Which systems hold personal data, and does the system-of-record map say, per system, whether it is a processor or a controller?
2. Is there a signed DPA for every processor, including every app, and who keeps the register?
3. Where is consent recorded, with what evidence, and how long does a withdrawal take to reach every outbound tool?
4. Do tags fire before the banner is answered, and what happens when the CMP is down?
5. How would you find every record of one person today, and how long did the last request take?
6. Can an order outlive its customer in your data model, or does deleting the person delete the invoice?
7. Which vendors receive data outside the EU, under which mechanism, and is that a list or a memory?
8. What happens to an erased person when a backup is restored?

A deletion request is one legal clock among many. Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation lays out the others, from the right of withdrawal and the Omnibus price rules to e-invoicing, accessibility, product safety and the platform rules for marketplaces, and shows how Kestrel & Pine turned them into a regulatory calendar for 2027 so that no deadline is discovered only months before it applies.

Sources and Evidence

Company, product and investor sources

- Privacy law compliance, Shopify developer documentation, n.d., accessed 2026-11 — the three compliance webhooks, the 10-day and 48-hour delays, the 30-day completion window.
- Protected customer data, Shopify developer documentation, n.d., accessed 2026-11 — minimum data, purpose limitation, consent, retention, encryption and level-2 controls for apps.
- Attorney General Bonta announces settlement with Sephora, California Department of Justice, 2022-08-24 — trackers as a sale, Global Privacy Control, the US$1.2 million settlement.
- Latombe's challenge to EU-US data transfers fails, Lewis Silkin, 2025-09-03 — the General Court judgment and the advice to keep SCCs as a fallback.

- Digital Omnibus reshapes EU cookie rules, leaves banner fatigue largely intact, Osborne Clarke, 2025-12-10 — a reading of the November 2025 proposal.

Standards, regulation and official documentation

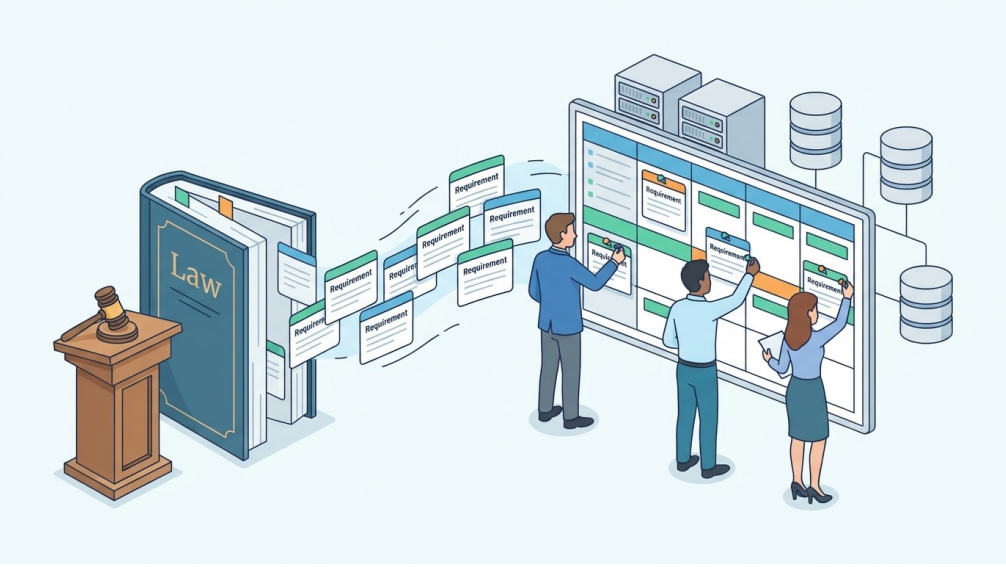
- Regulation (EU) 2016/679 (GDPR), Official Journal text, 2016 — Articles 4, 6, 7, 12(3), 28, 33 and 83(5); application from 25 May 2018.
- Directive 2009/136/EC, Official Journal text, 2009 — the amended Article 5(3) device rule of the ePrivacy Directive.
- Directive 2002/58/EC (ePrivacy), consolidated 2009, EUR-Lex — Article 13(2), the soft opt-in; text more than three years old, national transpositions differ.
- Guidelines 07/2020 on the concepts of controller and processor, European Data Protection Board, version 2.0, 2021-07-07 — purposes, essential and non-essential means.
- Guidelines 05/2020 on consent, European Data Protection Board, 2020-05-04 — cookie walls, scrolling.
- Guidelines 2/2023 on the technical scope of Art. 5(3) of the ePrivacy Directive, European Data Protection Board, version 2.0, 2024-10-07 — pixels, URL tracking, IP-only tracking.
- Cookies, Autoriteit Persoonsgegevens (Dutch), n.d., accessed 2026-11 — functional and limited analytical cookies, tracking cookies, cookie walls.
- Cookies en andere traceringsmiddelen, Gegevensbeschermingsautoriteit (Belgium, Dutch), 2023-11-17 — exempt categories, pre-ticked boxes, purpose grouping.
- Keeping business records, business.gov.nl (Netherlands Enterprise Agency), n.d., accessed 2026-11 — at least seven years; ten for immovable property; digital files accessible.
- §147 Abgabenordnung, gesetze-im-internet.de, current text, accessed 2026-11 — ten, eight and six years.
- Commission Implementing Decision (EU) 2023/1795 (EU-US Data Privacy Framework), Official Journal text, 2023 — the adequacy decision.
- Commission Implementing Decision (EU) 2021/914 (standard contractual clauses), Official Journal text, 2021.
- Case C-703/25 P, Latombe v Commission, Official Journal C/2025/6610, 2025-12-22 — the pending appeal.
- California Consumer Privacy Act, California Department of Justice, n.d., accessed 2026-11 — thresholds, rights, Global Privacy Control; US State Privacy Legislation Tracker, IAPP, updated 2026-09-08, and its 2025 chart — nineteen state laws.

Research and practitioner sources

- One Customer, One Product: Identity and Master Data Across Systems — the identity index this chapter's orchestrator depends on; Scams, Fakes and Account Takeovers: Fraud Prevention for Listings in the sibling book LISTED for the legitimate-interest side of risk screening.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

36. Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation



IN THIS CHAPTER, YOU WILL: Learn which EU and US rules shape a commerce platform, what each requires in data terms and by which date, and how to keep a regulatory calendar that stops deadlines arriving as surprises.

KEY POINTS:

- **Most rules are data requirements, not page requirements.** The prior-price rule needs 30 days of price history per SKU (stock-keeping unit: one sellable variant) and country; the withdrawal right needs refund clocks started by the right events; product-safety and marketplace rules need manufacturer and seller fields on every listing; e-invoicing needs a structured invoice over a network. Each belongs in a system of record, not in a template.

- **Dates slip, scope does not; transposition multiplies the calendar.** The AI Act's high-risk dates moved by more than a year, the Dutch e-invoicing mandate moved to 2030 before it was a bill, and one directive becomes 27 national laws with different dates. Plan to the scope, schedule to the date, and keep the calendar live.
- **The platform absorbs some of it, never all of it.** A hosted platform brings a compliant checkout button, cookie banner and card-data scope; price history, product-safety fields, packaging reports, deletion fan-out and every vendor contract clause stay with the merchant. The practical artifact is one calendar with an owner, the systems touched and a date per row.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Long Fuse** — every rule in this chapter was published years before it applied, and the merchant that discovers it months away pays for speed instead of planning.
- **The Pulled Rug** — a marketplace turns a regulation into a mandatory listing field on a date of its own, and the connector starts rejecting products before anyone reads the notice.
- **The Price Nobody Can Explain** — a discount claim is lawful only against a 30-day price history, and a shop with seven copies of a price cannot produce one.

Nobody joins a commerce team to read the Official Journal of the European Union. Yet a surprising share of what a commerce platform must do is written there, with a date: the button that says "order with obligation to pay", the 14 days a shopper has to change their mind, the "was" price under a discount, the manufacturer's address on a product page, the structured invoice a Belgian business customer may demand, the ban on destroying unsold jackets. **Each started as a proposal, became law years before it applied, and arrived in some merchant's backlog weeks before the date, as an emergency.** That is the pattern this book calls the Long Fuse.

The chapter's claim is that regulation is easier to handle than it looks if it is read the way an architect reads anything else: **as requirements on data, with a system of record, a test and a date.** It is not legal advice, and national rules differ in detail; it aims to make you the person who can read a legal text, say which field in which system it touches, and put it on a calendar with a name next to it. Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers and Whose Data Is It? Privacy, Consent and Retention cover the other two parts of trust; this chapter covers the rest of the law and the calendar that holds all three.

Where we are in the order's journey. Eva's jacket, the order this book follows, met the law at four steps without anyone noticing: the price she saw had to include VAT and

sit in a price history that any later reduction is measured against, the checkout button carried prescribed words, the product page carried a manufacturer's address, and her exchange request on 19 April 2027 started two 14-day clocks.

Setting the Scene at Kestrel & Pine: Four Rules Through Four Doors

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), is a Dutch company with 310 employees selling in three countries. In February 2027, while building the integration inventory (APIs, Events, Files and Middleware: How Commerce Systems Talk), Noor Visser, Head of Engineering since January, notices that four regulations reached engineering in two years, each through a different door and none through a plan. The rules' dates are real; how they arrived is an illustration.

- **The listing rejections (December 2024).** The **General Product Safety Regulation** (GPSR, Regulation (EU) 2023/988) applied from 13 December 2024 and requires every online offer to show the manufacturer's name and address, a responsible person in the EU where the manufacturer is outside it, product identification and warnings. Amazon.de and Zalando turned those into mandatory listing fields, the connector began rejecting third-party equipment products, and Milan Petrov, who owns the marketplaces, filled the gaps from a spreadsheet over three weeks.
- **The accessibility audit (summer 2025).** The **European Accessibility Act** applied from 28 June 2025 to e-commerce services, exempting only microenterprises. The agency audited the three storefronts in May 2025 (illustrative: about 140 findings, mostly contrast, labels and keyboard navigation in the customized theme) and fixed them over the summer. The checkout page was the platform's problem; the theme was Kestrel & Pine's.
- **The refused PDF (December 2025).** Belgium made structured electronic invoices mandatory between Belgian VAT-registered businesses from 1 January 2026. Whether that binds a Dutch supplier without a Belgian establishment is not settled by the sources; what mattered was that the Belgian customer's bookkeeping now expected a Peppol invoice. Kestrel & Pine learned of it when a larger Belgian account announced it would no longer process PDFs (Tax, Invoices and the Books: From Order to Ledger tells it from finance's side); the EDI (electronic data interchange) provider, already an access point to Peppol, the European network for exchanging structured business documents, was connected to the ERP (the finance and invoicing system) in six weeks, and Peppol invoices have gone to all Belgian business customers since January 2026.
- **The newsletter (June 2026).** The **Ecodesign for Sustainable Products Regulation** (ESPR, Regulation (EU) 2024/1781) prohibits, from 19 July 2026, the

destruction of unsold apparel and footwear by large enterprises, and its definition of "unsold" includes goods returned under the right of withdrawal. Teun Baas, who owns the seasons, had an August routine for end-of-line stock and returns not resold as new (about 39% of returned items); it now needs a documented disposition per unit and a yearly disclosure. Sanne de Wit, the Finance Director, found the rule in an auditor's newsletter in June 2026, weeks before it applied.

Four rules, four doors: an error log, an agency, a customer and a newsletter. None was a secret: the GPSR was published in May 2023, the Accessibility Act in June 2019, the ESPR in June 2024. **The company had no door of its own.** So Noor adds a regulatory calendar for 2027, with an owner per row, to Waypoint's working documents.

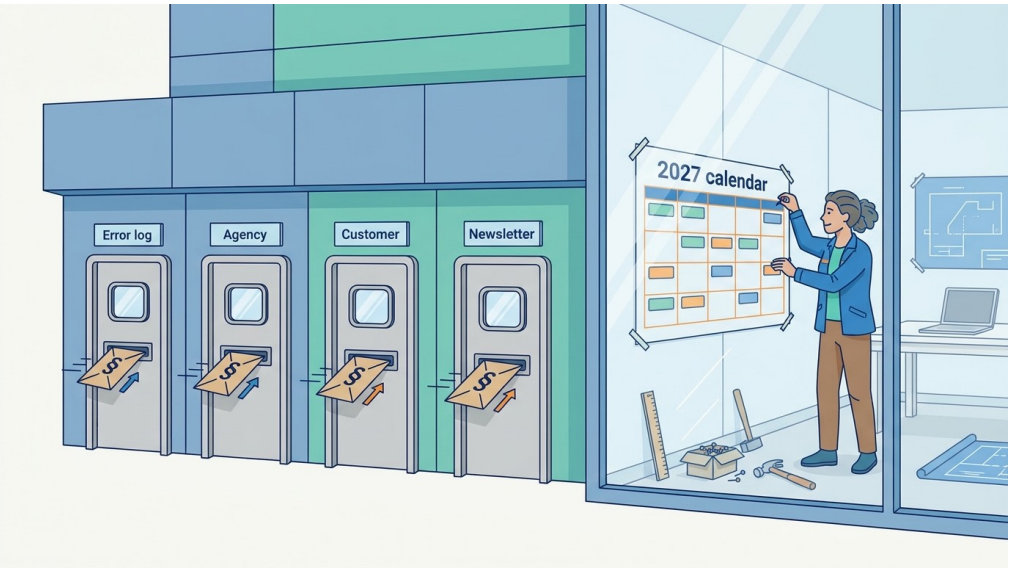


Figure 1: *Four rules, four doors: an error log, an agency, a customer and a newsletter brought the law in, and none of them was a door the company owned; the 2027 calendar is the door of its own.*

How to Read a Law as a Requirement

A legal text is written for courts, not engineers; the translation follows four questions.

1. **Who is in scope, and are we?** By role (trader, marketplace, producer, payment provider), by size (microenterprises are exempt from the Accessibility Act; small and medium enterprises are exempt or deferred under the ESPR) and by place (established in a member state, selling into it, or both).

2. **What does it require, in data terms?** Nearly every obligation is a field, a history, a document or a clock: the lowest price in the last 30 days; a manufacturer's address per product; a structured invoice; a refund within 14 days of a dated notice.
3. **Which system of record holds that, and which channels must show it?** The field belongs where the fact is owned (Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record); a manufacturer field kept only in the marketplace connector is a copy with no owner.
4. **When does it apply, and how will we know we comply?** The application date, the national transposition date for a directive, and a test: a query, a report or a check in the release process.

In the table, the OMS is the order management system, the PIM the product information management system and the WMS the warehouse management system. ViDA is the EU's "VAT in the Digital Age" package; the PPWR is the Packaging and Packaging Waste Regulation; WCAG are the Web Content Accessibility Guidelines.

Rule	Data requirement	System of record	Channels that must show it	Test
Prior price (Omnibus, Art. 6a)	Lowest price per SKU per country in the 30 days before a reduction	Pricing engine or price history (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax)	Storefronts, marketplaces, stores	Every strike-through price equals the 30-day minimum
Right of withdrawal (Consumer Rights Directive)	Delivery, notice and goods-received dates; refund within 14 days of the notice	OMS and returns portal (The Way Back: Returns, Exchanges, Refunds and Reverse Logistics)	Returns portal, service tool	No refund older than 14 days from notice

Rule	Data requirement	System of record	Channels that must show it	Test
Product safety (GPSR, Art. 19)	Manufacturer and responsible-person name and address, identification, warnings in the shopper's language	PIM (What Are We Selling? Products, Variants and the Catalog)	Storefronts, marketplaces	No product published without the fields
E-invoicing (Belgium, Germany, ViDA)	EN 16931 invoice over Peppol or an accredited platform, with transmission status	ERP and access point (Documents Between Companies: EDI, Punchout and E-Invoicing)	Business customers	Every in-scope invoice has a delivered status
Packaging and producer responsibility (PPWR, national registers)	Packaging weight by material per unit sold, per country	PIM and WMS	Reporting only	A yearly report per country reconciles with units shipped
Unsold-goods destruction (ESPR, Art. 24–25)	Disposition per unsold or returned unit, reasons, yearly disclosure	WMS and returns disposition	Reporting only	No "destroy" disposition for apparel without a derogation

Rule	Data requirement	System of record	Channels that must show it	Test
Accessibility (European Accessibility Act)	Storefront, checkout, identification and payment meet WCAG 2.1 AA	Theme and embedded widgets (Delivering the Storefront: Themes, Headless, Rendering and Performance)	Storefronts	An audit per release; vendor statements on file
AI transparency (AI Act, Art. 50)	Disclosure that a shopper talks to an AI; machine-readable marking of generated content	Any assistant or content tool (Assistants and Agents: AI in Commerce and What Is Real in 2026)	Storefronts, service	No undisclosed assistant; generated images marked

The pattern to notice is the third column: **most rows land in a system Kestrel & Pine did not have or did not own in 2026** (the PIM, the OMS, the price history, a disposition record). Regulation is one more reason the Waypoint hubs exist.

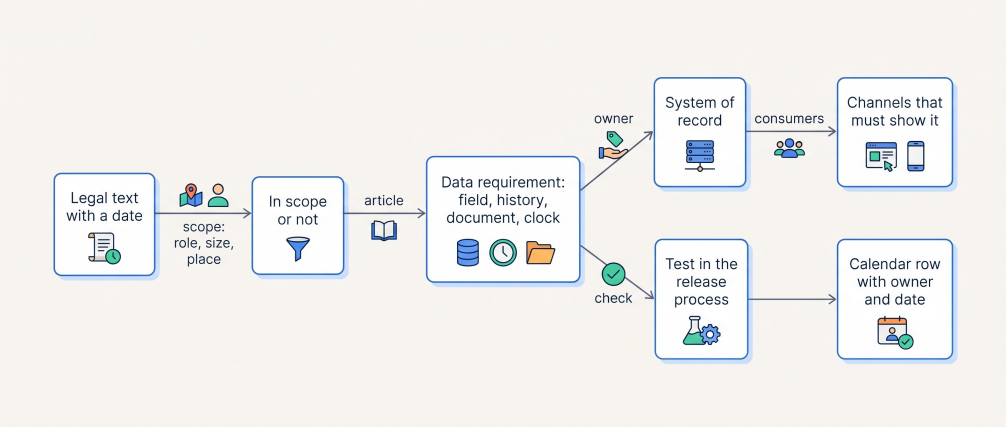


Figure 2: *Reading a law as a requirement: scope first, then the field or clock it demands,*

the system that owns it, the channels that show it, a test, and a dated row with a name on it.

The Rules, by What They Touch

Consumer law: withdrawal, the button, the prior price, reviews

The **Consumer Rights Directive** (2011/83/EU, applying to contracts from 13 June 2014) gives a consumer "a period of 14 days to withdraw from a distance or off-premises contract, without giving any reason", extended to 12 months if the trader fails to inform them; the trader must refund "not later than 14 days from the day on which he is informed of the consumer's decision to withdraw", and may withhold it until the goods are back; and an order button must be labeled "only with the words 'order with obligation to pay' or a corresponding unambiguous formulation" (Directive 2011/83/EU, 2011). Three clocks and one string. The clocks need the right events recorded: delivery (the carrier's scan), the withdrawal notice (the returns portal) and receipt of the goods (the WMS). Eva's exchange request of 19 April 2027 started the clock that day, not on 22 April when the jacket reached Duiven.

The **Omnibus Directive** (2019/2161, applied from 28 May 2022) added the **prior-price rule**: "any announcement of a price reduction shall indicate the prior price", meaning "the lowest price applied by the trader during a period of time not shorter than 30 days prior to the application of the price reduction" (Directive (EU) 2019/2161, 2019). It also made ranking parameters and review provenance material information, blacklisted claiming reviews are genuine "without taking reasonable and proportionate steps to check", and required disclosure of prices personalized by automated decision-making. What Does It Cost? Price Lists, Contract Prices, Currencies and Tax works the Black Friday case where an autumn campaign at €179 makes the lawful "was" price €179, not €199; Finding the Product: Search, Navigation and Merchandising covers reviews. **A 30-day price history per SKU per country is a legal record**, and a merchant whose prices live in seven copies, as Kestrel & Pine's did in 2026, cannot produce it.

Payments: strong customer authentication and what is coming

The second **Payment Services Directive**, **PSD2** (Directive (EU) 2015/2366, applying from 13 January 2018) requires strong customer authentication when a payer initiates an electronic payment; the technical standards (Delegated Regulation (EU) 2018/389, from 14 September 2019) allow exemptions for remote transactions up to €30 and for low-risk transactions below thresholds. For the checkout this means a challenge in the middle of paying by card and an exemption strategy for saved cards (Taking the Money: Payment Providers, Authorization, Capture and Refunds).

Since 9 October 2025 euro-area banks also apply **verification of payee** (Regulation (EU) 2024/886): the name on a transfer must match the account, so the company name on a B2B invoice must match the bank account.

PSD3 and the Payment Services Regulation are pending as of November 2026: provisionally agreed on 27 November 2025, confirmed by member states' representatives on 22 April 2026, and still open in the Council's state of play of 29 October 2026 (Council of the EU, 2026). Providers that fail to implement fraud prevention "will be liable for covering customers' losses" (European Parliament, 21 November 2025). The dates will follow adoption; the scope is known.

Invoicing: the mandates and ViDA

Documents Between Companies: EDI, Punchout and E-Invoicing owns the mandate calendar in full. For a Dutch brand selling to businesses in three countries, as of November 2026: **Belgium** requires structured e-invoices over Peppol between VAT-registered businesses since 1 January 2026, with fines of €1,500 to €5,000 for lacking the technical means (FPS Finance, 2024 and 2025); **Germany** requires receiving since 1 January 2025 and issuing from 1 January 2027 for German-established issuers above €800,000 prior-year turnover, from 2028 for all (Federal Ministry of Finance FAQ, n.d.); **the Netherlands** plans a domestic mandate from 1 July 2030 and reporting from 1 July 2031 (Rijksoverheid, 11 September 2026); and **ViDA** requires e-invoicing-based reporting for cross-border business-to-business supplies from 1 July 2030 (Council Directive (EU) 2025/516). The mandates are domestic in scope: a Dutch supplier is not automatically obliged in Belgium or Germany, but its customers there can demand the format, and many do.

Platforms and marketplaces: the DSA and product safety

The **Digital Services Act** (Regulation (EU) 2022/2065, applying from 17 February 2024) requires a marketplace, before a trader may sell, to obtain the trader's identity and contact details, an identification document, "the payment account details of the trader", the trade-register number and a self-certification to offer only compliant products, and to verify them (Art. 30); it bans deceptive interfaces (Art. 25) and requires the interface to let traders provide compliance and product-safety information (Art. 31). For a merchant selling on three marketplaces this is an onboarding data set to keep current three times; for one operating a marketplace or dropship program, a seller-onboarding workflow with verification (LISTED's Rules of the Marketplace: DSA, DAC7 and Vertical Regulation treats the operator's side).

The **GPSR** (Regulation (EU) 2023/988, applying from 13 December 2024) treats a product offered online as made available in the EU if the offer targets EU consumers, and requires every distance offer to show the manufacturer's name and address, the responsible person's where the manufacturer is outside the EU, identification including a picture,

and warnings in a language the consumer understands (Art. 19); marketplaces must let traders provide this per product (Art. 22(9)). In data terms: **four mandatory attributes per product**, **owned by the PIM and syndicated everywhere**, with publication blocked when they are missing.

Product and packaging: destruction bans, producer responsibility, the passport

The **ESPR** (Regulation (EU) 2024/1781) prohibits, "from 19 July 2026, the destruction of unsold consumer products as listed in Annex VII", which covers apparel, clothing accessories and footwear; the prohibition "shall not apply to micro and small enterprises" and applies to medium-sized enterprises from 19 July 2030 (Art. 25(1)); and economic operators that discard unsold products must disclose yearly the number and weight discarded by category, the reasons and the share sent to reuse, recycling, recovery or disposal (Art. 24). The regulation also defines the **digital product passport**, product data reachable through a data carrier on the item, with a registry of identifiers due by 19 July 2026 (Art. 13); which product groups come first depends on delegated acts not yet adopted, and the expectation that textiles come early (around 2027) is an assumption. For an apparel brand: **the disposition of every unsold or returned unit is a record with legal weight**, and the PIM becomes the passport's source.

The **Packaging and Packaging Waste Regulation** (Regulation (EU) 2025/40) applies from 12 August 2026 and makes producers responsible for packaging they first place on a member state's market (Art. 45), with harmonized labels from 12 August 2028 or later (Art. 12). National registers already collect the fees (Germany's LUCID: "if you are distributing packaged goods in Germany, you have to be registered", n.d., accessed November 2026; Dutch and Belgian schemes not verified here). The fee is computed from packaging weight by material per country of sale: the product needs its sales packaging, the shipment its carton, the order its destination.

Accessibility

The **European Accessibility Act** (Directive (EU) 2019/882) covers e-commerce services (websites and apps through which consumers conclude contracts) from **28 June 2025**, exempts microenterprises providing services (fewer than 10 persons and at most €2 million turnover), lets products lawfully in use before that date continue until 28 June 2030, and requires "identification methods, electronic signatures, security, and payment services which are perceivable, operable, understandable and robust" (Directive (EU) 2019/882, 2019). The technical content in practice is WCAG 2.1 AA. Because the obligation covers identification and payment, **every embedded widget on the path to the order is in scope**: the hosted checkout, the PSP's iframe, the consent banner. Vendor conformance statements become a procurement item, and the theme is audited per release.

Cyber and AI: NIS2, the Cyber Resilience Act and the AI Act

NIS2 (Directive (EU) 2022/2555, applied nationally from 18 October 2024) covers listed entity types that are medium-sized or larger, among them "providers of online marketplaces"; significant incidents require an early warning within 24 hours, a notification within 72 hours and a final report within a month (Art. 23(4)). The Netherlands brought it into force as the **Cyberbeveiligingswet** on 15 August 2026 (Rijksoverheid, 7 July 2026). An ordinary retailer is usually out of scope; a marketplace operator or commerce SaaS vendor is in. The merchant meets NIS2 through its suppliers, as incident-notification clauses in the platform's, the WMS vendor's and the iPaaS vendor's contracts, so the 24-hour and 72-hour clocks become terms the merchant's incident process (Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers) must meet. The **Cyber Resilience Act** (Regulation (EU) 2024/2847) adds vendors' duties from 11 December 2027.

The **AI Act** (Regulation (EU) 2024/1689) applied generally from 2 August 2026. Its live obligations for commerce are transparency: a system "intended to interact directly with natural persons" must say it is an AI unless that is obvious, and generated content must be "marked in a machine-readable format and detectable as artificially generated" (Art. 50). The **Digital Omnibus on AI** (Regulation (EU) 2026/1744, July 2026) moved the high-risk obligations to 2 December 2027 (Annex III) and 2 August 2028 (Annex I). A search or fraud model is not high-risk; worker monitoring in warehouse software is the case to watch. Kestrel & Pine's W8 decision, agent readiness without agents, keeps it outside most of this in 2027 (Assistants and Agents: AI in Commerce and What Is Real in 2026).

Data: the Data Act and transfers

The **Data Act** (Regulation (EU) 2023/2854) applies from 12 September 2025; its switching chapter covers cloud contracts concluded after that date and, from 12 September 2027, older ones (Art. 29). For a merchant it is an exit right: the OMS and PIM contracts Kestrel & Pine signs in 2027 must allow switching and data retrieval, the legal floor under Moving Without Burning the Bridge: Replatforming and Migration. Transfers belong to Whose Data Is It? Privacy, Consent and Retention.

The Regulatory Calendar for 2027

Kestrel & Pine's calendar as Noor and Sanne write it in March 2027, with application dates as of November 2026 from the official sources in this chapter. *Maintain* rows are in force and need a test; *prepare* rows have a date ahead; *watch* rows are pending. Owners and statuses are illustrative; dates are documented.

Date	Rule (official source)	What it requires of Kestrel & Pine	Owner	2027 status
13 Jun 2014	Consumer Rights Directive 2011/83/EU	14-day withdrawal and refund clocks; the "order with obligation to pay" button	Daniel Okafor	*Maintain*: refund-age report monthly
14 Sep 2019	PSD2 RTS on SCA (2018/389)	Challenge or exemption on every card payment	Milan Petrov	*Maintain*: exemption rate from the PSP
28 May 2022	Omnibus Directive 2019/2161	30-day prior price per SKU and country; review provenance; ranking disclosure	Milan Petrov	*Maintain*: strike-through check before every campaign
17 Feb 2024	Digital Services Act, Art. 30	Trader data and self-certification current on bol, Amazon.de, Zalando	Milan Petrov	*Maintain*: yearly review
13 Dec 2024	GPSR 2023/988, Art. 19	Manufacturer, responsible person, identification and warnings on every offer	Teun Baas	*Maintain*: publication blocked without the fields (PIM rule from Q3)

Date	Rule (official source)	What it requires of Kestrel & Pine	Owner	2027 status
1 Jan 2025 / 1 Jan 2027 / 1 Jan 2028	Germany e-invoicing (BMF FAQ)	Receive since 2025; German-established issuers above €800k issue from 2027, all from 2028; EDI tolerated to end 2027	Sanne de Wit	*Prepare*: offer XRechnung (Germany's structured invoice format) over Peppol to German accounts that ask; not obliged as a Dutch issuer
28 Jun 2025	European Accessibility Act 2019/882	Storefronts, identification and payment steps accessible; not a microenterprise	Milan Petrov	*Maintain*: audit per theme release; vendor statements on file
12 Sep 2025	Data Act 2023/2854	Switching and data-retrieval terms in cloud contracts	Noor Visser	*Prepare*: clause check before signature

Date	Rule (official source)	What it requires of Kestrel & Pine	Owner	2027 status
1 Jan 2026	Belgium e-invoicing (FPS Finance)	Structured invoices over Peppol to Belgian business customers, as they expect (whether the mandate binds a Dutch supplier is unsettled); fines of €1,500–5,000 for Belgian businesses lacking the means	Sanne de Wit	*Maintain*: delivered-status report; 210 accounts
19 Jul 2026	ESPR 2024/1781, Art. 24–25	No destruction of unsold or returned apparel (large enterprise); yearly disclosure	Daniel Okafor	*Maintain*: disposition codes; first disclosure for 2026
2 Aug 2026	AI Act 2024/1689, Art. 50	Disclose AI assistants; mark generated content	Noor Visser	*Watch*: applies the day an assistant or image generator is used

Date	Rule (official source)	What it requires of Kestrel & Pine	Owner	2027 status
12 Aug 2026	PPWR 2025/40; national registers (LUCID)	Producer responsibility per country; packaging data by material	Sanne de Wit	*Maintain*: yearly reports NL, BE, DE
15 Aug 2026	Dutch Cyberbeveiligingswet (NIS2)	Out of direct scope; vendor clauses with 24/72-hour clocks	Noor Visser	*Maintain*: clause inventory
2 Dec 2027	AI Act high-risk, Annex III	Worker monitoring in warehouse software would be in scope	Daniel Okafor	*Watch*: ask the WMS vendor
11 Dec 2027	Cyber Resilience Act 2024/2847	Vendors' product security duties (reporting since 11 Sep 2026)	Noor Visser	*Watch*
Pending	PSD3 and PSR; Digital Omnibus (data); Digital Fairness Act; DPF appeal C-703/25 P	Fraud liability and payee checks; cookie rules; dark patterns; transfers	Noor Visser	*Watch*: quarterly review of status

Date	Rule (official source)	What it requires of Kestrel & Pine	Owner	2027 status
1 Jul 2030 / 1 Jul 2031	ViDA 2025/516; Netherlands e-invoicing and reporting (planned)	Cross-border e-invoicing and reporting; domestic Dutch mandate for 640 accounts	Sanne de Wit	*Prepare*: the Peppol path already exists; watch the Dutch bill in 2027
Not yet set	ESPR delegated act for textiles; digital product passport	Resolvable identifier and passport data per product *(assumption on timing)*	Teun Baas	*Watch*: verify the Commission's working plan each quarter

Three readings. **Most rows are maintenance, not projects. The 2027 projects are few**: Data Act clauses in two contracts, the GPSR publication rule when the PIM arrives, the first ESPR disclosure. And **the owners are mostly not engineers**, because they own the facts; Noor owns the calendar and the vendor rows. The toolkit card is in Toolkit: Cards and Templates for Commerce Architecture Decisions.

Requirements for the compliance data flows

The rows become flows between systems, with requirements like any other (APIs, Events, Files and Middleware: How Commerce Systems Talk).

Flow	Freshness	Consistency	Latency	Availability
Price history for the prior price	Every price change, with effective dates	One record per SKU, market and channel; the strike-through computed from it	Batch; the check runs before a campaign	The campaign does not launch without it
Withdrawal and refund clocks	Each event as it happens (delivery scan, notice, receipt)	One order record with dated events (the OMS)	Minutes	A missed event is a missed deadline; reconcile daily

The Long Fuse: Why Deadlines Arrive Late

Every row had the same life: a proposal, an adopted text, often a national transposition, an application date, then enforcement. Publication to application is usually two to four years.

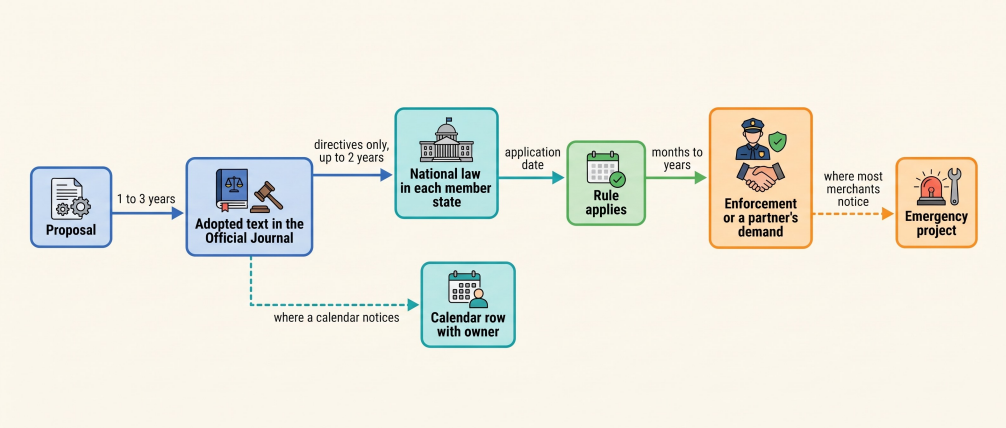


Figure 3: *The Long Fuse: a rule is public years before it applies, a calendar notices it at adoption, and a merchant without one notices it when a customer, a marketplace or a regulator enforces it.*

Five lessons the evidence supports.

Dates slip; scope does not. The AI Act's high-risk dates moved by more than a year (Regulation (EU) 2026/1744); the PCI Council softened its script requirements weeks

before they took effect (Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers); the Dutch e-invoicing mandate moved to 2030 before it was a bill. Plan the capability to the scope and schedule it to the date.

Transposition multiplies the calendar. A directive becomes 27 laws with 27 dates: e-invoicing in Belgium (2026), Germany (2025–2028) and the Netherlands (planned 2030) under one EU framework; NIS2 in the Netherlands in August 2026. A merchant selling in three countries keeps three calendars.

Partners enforce before regulators do. Marketplaces turn the GPSR and the DSA into listing fields with their own deadlines; business customers turn a mandate into a refused PDF; retail chains write label standards into purchasing conditions with penalties (Documents Between Companies: EDI, Punchout and E-Invoicing). The Pulled Rug and the Long Fuse are often one event seen from two sides.

Courts and appeals are calendar events too. *South Dakota v. Wayfair* created sales-tax obligations for remote sellers in a day (21 June 2018); the Eighth Circuit vacated the FTC's click-to-cancel rule in July 2025; the pending appeal against the EU–US Data Privacy Framework (C-703/25 P) could change every US transfer. A "watch" row is a real row.

The platform absorbs some of it, never all of it. A hosted platform ships the button text, the cookie banner, the withdrawal flow, the card-data scope of its checkout and the accessibility of the pages it renders. It does not ship the merchant's price history, product-safety fields, packaging reports, disposition records or deletion fan-out. The rows that remain are the data rows.

The Choices: Four Ways to Run Compliance

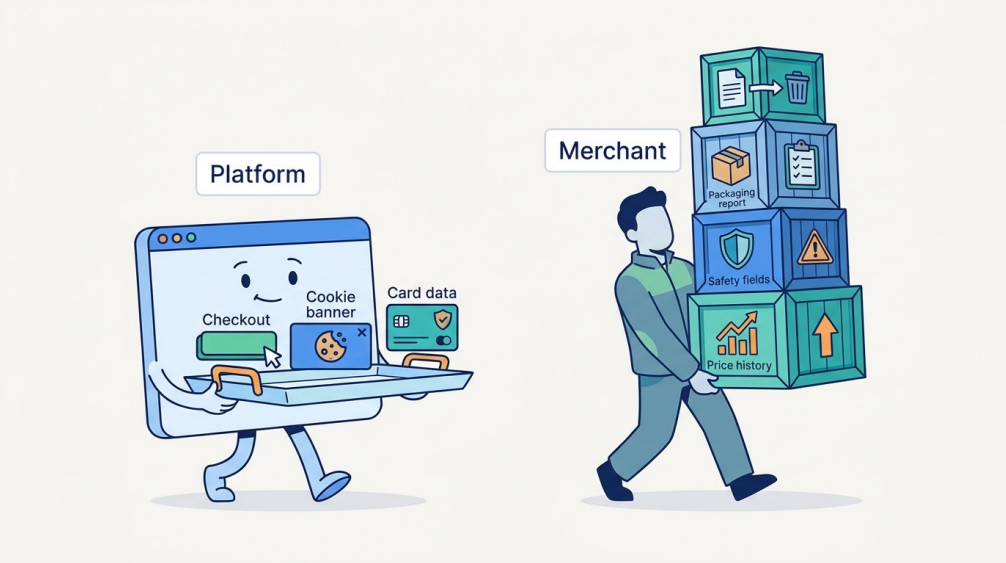


Figure 4: *The platform absorbs some of it, never all of it: the button, the banner and the checkout's card scope travel with the platform, while the data rows stay with the merchant.*

React when someone complains. No calendar; rules arrive through customers, marketplaces, auditors and fines. The default, and nearly adequate for a Starting-stage shop on a hosted platform in one country, because the platform carries most of what applies.

One owner, one calendar, reviewed quarterly. A named person keeps the table, reads the official source per row and brings new instruments to a quarterly review. Cheap; it removes the surprise but does not put the fields in the systems.

Calendar plus requirement cards tied to the system-of-record map. Each row becomes a card (scope, data requirement, system of record, channels, test, date, owner, as in Figure 2), attached to the entity–owner table of Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, with the tests in the release process. Where an Omnichannel merchant should be, because that is when rules start landing in systems it owns.

A compliance function with counsel and tooling. Legal and compliance staff, tracking subscriptions, controls testing, local advisers. Right for a group or a vendor; at Kestrel & Pine's size the Giant's Boots.

Approach	Cost (illustrative)	Catches	Misses	Right when
React	Nothing, then emergencies	Whatever a partner or regulator enforces	Every deadline's lead time	One country, one channel, hosted platform
One owner, one calendar	Days a quarter	Published dates; pending files	Whether the data actually exists in the systems	Two or three channels, first mandates
Calendar plus requirement cards	A card per row; tests in releases	Dates and the data behind them	Deep national detail; litigation risk	Omnichannel; several countries; owned hubs
Compliance function	Headcount; subscriptions	Nearly everything	Cost; distance from engineering	Group, marketplace operator, vendor

My recommendation: **the third approach, built from the second, the quarter the merchant first owns a PIM, an OMS or a price history**, because from then on the rules are its fields. The calendar is a management artifact; the cards are what engineering can test.

Across Scale Stages: When It Fits and When It Limits

Stage	Typical approach	What becomes necessary	Warning signs of outgrowing it
1 — Starting (under about €2M)	React; the hosted platform's defaults; one country	Checking the defaults against home-country consumer law; reading a marketplace's listing requirements before listing	A second country; the first business customer who wants a structured invoice; a microenterprise exemption that stops applying
2 — Growing (about €2M–€20M)	One owner, one calendar; the agency fixes what the platform does not cover	Price history as a record; GPSR fields in the catalog; a refund-age report; the first e-invoicing route; an accessibility audit of the theme	Rules arriving through marketplaces' error logs (December 2024); a customer's refused PDF (December 2025)
3 — Omnichannel (about €20M–€200M)	Calendar plus requirement cards on the system-of-record map; owners in finance, operations and merchandising	Fields owned by the PIM and OMS; tests in releases; vendor clauses with incident clocks; packaging and disposition reporting; a Data Act check on new contracts	Three national calendars in people's heads; a rule found in a newsletter (June 2026); a report rebuilt by hand each year
4 — Group	A compliance function tied to the map; per-country entities	Country clearance systems behind one invoicing service; one consent model across brands; a regulatory watch with counsel	Each brand reading the law separately; a marketplace business bringing DSA and NIS2 duties of its own

Here **scope follows size**: a microenterprise is exempt from the Accessibility Act, a small enterprise from the ESPR ban, a Dutch company below the small-business threshold from the planned e-invoicing mandate, and a retailer that becomes a marketplace operator picks up the DSA's and NIS2's duties that day. Growth is a change of scope, and a row should say which size or role triggers it.

In Practice

The United States, as the contrast

A European merchant selling into the United States meets a different shape of law: few federal rules, many state ones, and lawsuits where the EU has regulations. **Sales tax**: since *South Dakota v. Wayfair* (21 June 2018) a state may tax remote sellers without physical presence (thresholds of $100,000 of sales or 200 transactions were upheld), and marketplace-facilitator laws make the marketplace collect, so US sales tax is a tax-engine problem (Tax, Invoices and the Books: From Order to Ledger). **Marketplace sellers**: the INFORM Consumers Act (effective 27 June 2023) requires marketplaces to collect and verify bank, tax and contact data from high-volume sellers and to disclose sellers above $20,000 on listings, with penalties of $53,088 per violation (Federal Trade Commission, FTC, n.d., accessed November 2026), a mirror of DSA Art. 30. **Reviews**: the FTC rule of 14 August 2024 bans fake or AI-generated reviews. **Accessibility**: no federal rule for private sites; exposure is litigation (a vendor counted 5,114 suits in 2025, 70% against e-commerce; UsableNet, 2025). **Privacy**: an opt-out model across at least 19 state laws (IAPP, 2025) against the EU's opt-in. A platform serving both regions needs a tax engine, a consent record that expresses both models, and the same onboarding data twice.

Kestrel & Pine's 2027 worked example

Take one row through the method. **GPSR, Art. 19**. Scope: a trader offering products to EU consumers at a distance, so every product, own-brand and third-party. Data: manufacturer name and address (Kestrel & Pine B.V. for own brand; the supplier for third-party equipment), a responsible person where the manufacturer is outside the EU, a picture and an identifier (the GTIN, the global trade item number behind the barcode, that every own-brand SKU has carried since 2021), warnings in Dutch, French and German. System of record: the PIM from Q3 2027 (W3); until then the platform and the connector, which is why the 2024 fix lived in a spreadsheet. Channels: three storefronts, three marketplaces. Test: a PIM rule that blocks publication without the four fields, run on the 1,200 new variants each season. Owner: Teun Baas. Cost (illustrative): two days of PIM configuration and a clean-up of about 850 third-party variants. Three weeks of spreadsheet work in 2024 becomes a validation rule in 2027, because the system that should own the field now exists.

When It Goes Wrong

A deadline discovered late. Detection: a customer, marketplace or auditor says so. Recovery: the capability first, the evidence second; Belgium's tolerance of December 2025 spared only businesses that could show timely steps ("under no circumstances" a general postponement), and a dated plan, a signed provider contract and a test log are that evidence. Then ask which other rows have no owner.

A clock that nobody started. A withdrawal notice received by e-mail and typed into the returns portal four days later; the clock ran from the e-mail. Detection: a refund-age report using the notice date. Recovery: every notice channel creates the event with its real date.

A partner's deadline before the law's. A marketplace requires a field months before a regulation applies, or a chain demands a label standard the law never mentions. Detection: partner notices read by the row's owner. Recovery: the partner's date becomes the row's date, because the partner is the enforcer that matters.

What to Take Away

Established.

- The core EU rules for a shop are documented facts with dates: 14-day withdrawal and refund (2014); the 30-day prior price (28 May 2022); strong customer authentication (14 September 2019); DSA trader traceability (17 February 2024); GPSR product information (13 December 2024); accessibility (28 June 2025); the ESPR destruction ban for large enterprises (19 July 2026); e-invoicing in Belgium (1 January 2026) and Germany (2025–2028); ViDA from 1 July 2030.
- Most of these are requirements on data held in a system of record: price history, dated order events, product attributes, structured invoices, disposition and packaging records.
- Directives are transposed with national differences; regulations apply directly; scope depends on role, size and place; pending files (PSD3/PSR, the Digital Omnibus, the Digital Fairness Act, the DPF appeal) are rows with no date yet.

Depends on context.

- Whether a mandate applies depends on establishment and size, not on where your customers are: a Dutch issuer is not bound by the German e-invoicing mandate, the sources do not settle whether the Belgian one reaches it, and customers in both countries will ask for the format; a microenterprise is exempt from the Accessibility Act; a small or medium enterprise is exempt or deferred under the ESPR ban.

- A US presence changes the model from regulation to litigation and from opt-in to opt-out, and adds a tax engine and seller disclosures; a merchant without US sales leaves those rows out.

The author's synthesis.

- **Regulation is a second source of the system-of-record map**, not a separate discipline: most rules in the method table land in a PIM, an OMS or a price history, so the Waypoint hubs are compliance infrastructure too.
- The right unit of work is the calendar row with an owner who owns the fact, and the right cadence is a quarterly read of the official source per row; the rest is ordinary engineering.

Questions to Ask

1. Do we have a regulatory calendar, who owns it, and when was each row's source last read?
2. For each row, which system of record holds the data, and which channels consume it?
3. Can we produce the lowest price of the last 30 days for any SKU in any country, and does the strike-through on the storefront compute from it?
4. Which event starts our 14-day refund clock, and is it the customer's notice or the day someone typed it in?
5. Are manufacturer, responsible-person and warning fields mandatory in our catalog, and is publication blocked without them?
6. Which of our size or role thresholds (microenterprise, small or medium enterprise, marketplace operator) are close, and which rows switch on when we cross them?
7. Which vendors pass NIS2 or Cyber Resilience Act clauses to us, and do this year's contracts carry Data Act switching terms?
8. Which marketplaces or customers enforce a rule earlier than the law does, and who reads their notices?

With security, privacy and regulation in place, Part VIII is complete, and the book turns from the parts to the whole: four journeys that follow one order, one return, one wholesale order and one peak day through every system at once. Continue with Part IX — JOURNEYS: One Order, End to End.

Sources and Evidence

Company, product and investor sources

- Sephora settlement — California Attorney General, 2022-08-24; the CCPA settlement and the Global Privacy Control requirement.
- 2025 Year-End Digital Accessibility Lawsuit Report — UsableNet, 2025-12; US lawsuit counts, quoted as a vendor figure.
- US State Privacy Legislation Tracker chart — IAPP, 2025; the count of comprehensive state privacy laws.

Standards, regulation and official documentation

- Directive 2011/83/EU (Consumer Rights) — Publications Office of the EU, 2011; withdrawal, refund, the button label.
- Directive (EU) 2019/2161 (Omnibus) — Publications Office of the EU, 2019; the prior price, reviews, ranking, personalized prices.
- Directive (EU) 2015/2366 (PSD2) and Delegated Regulation (EU) 2018/389 (RTS on SCA) — Publications Office of the EU, 2015 and 2018; strong customer authentication and exemptions.
- Regulation (EU) 2024/886 (Instant Payments) — Publications Office of the EU, 2024; verification of payee by 9 October 2025.
- Council document 15005/26, state of play for ECOFIN — Council of the EU, 2026-10-29; PSD3/PSR still open. Payment services deal — European Parliament, 2025-11-21; the agreement's content.
- Structured electronic invoices between companies compulsory in 2026 (2024-09-18), FAQ on fines (n.d., accessed 2026-11) and Tolerance during the first three months of 2026 (2025-12-18) — FPS Finance Belgium.
- FAQ E-Rechnung — Bundesministerium der Finanzen, n.d., accessed 2026-11; German dates and thresholds (translated).
- Kabinet kiest voor invoering e-facturatie en rapportage — Rijksoverheid, 2026-09-11; the Dutch 2030 and 2031 decision (translated).
- Council Directive (EU) 2025/516 (ViDA) — Publications Office of the EU, 2025; application from 1 July 2030.
- Regulation (EU) 2022/2065 (Digital Services Act) — Publications Office of the EU, 2022; Art. 25, 30 and 31.
- Regulation (EU) 2023/988 (General Product Safety) — Publications Office of the EU, 2023; Art. 4, 19 and 22(9).
- Regulation (EU) 2024/1781 (Ecodesign for Sustainable Products) — Publications Office of the EU, 2024; Art. 13, 24 and 25 and Annex VII (text verified 2026-11).
- Regulation (EU) 2025/40 (Packaging and Packaging Waste) — Publications Office of the EU, 2025; application date, Art. 12 and 45. LUCID Packaging Register — Zentrale Stelle Verpackungsregister, n.d., accessed 2026-11.
- Directive (EU) 2019/882 (European Accessibility Act) — Publications Office of the EU, 2019; scope, dates, the microenterprise exemption, Annex I.

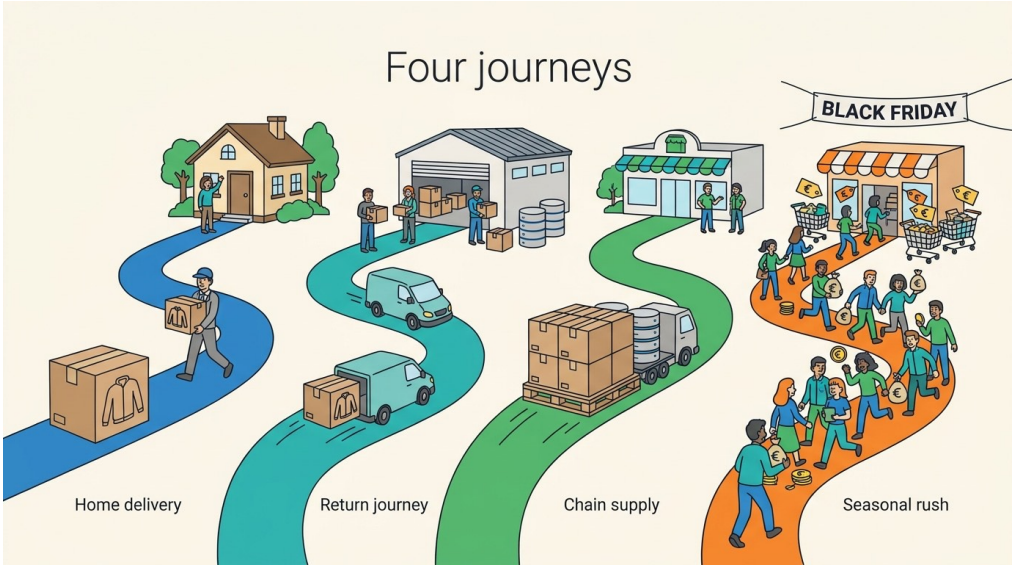
- Directive (EU) 2022/2555 (NIS2) — Publications Office of the EU, 2022; scope and reporting clocks. Cyberbeveiligingswet in force 15 August 2026 — Rijksoverheid, 2026-07-07. Regulation (EU) 2024/2847 (Cyber Resilience Act) — Publications Office of the EU, 2024.
- Regulation (EU) 2024/1689 (AI Act) and Regulation (EU) 2026/1744 (Digital Omnibus on AI) — Publications Office of the EU, 2024 and 2026; Art. 50 and the revised dates.
- Regulation (EU) 2023/2854 (Data Act) — Publications Office of the EU, 2023; switching rights and dates.
- Appeal C-703/25 P, Latombe v Commission — Official Journal C/2025/6610, 2025; the pending DPF appeal.
- Guidance for PCI DSS e-commerce requirements effective after 31 March 2025 — PCI Security Standards Council, 2025-03-26; the SAQ A change.
- South Dakota v. Wayfair, Inc. — Supreme Court of the United States via Cornell LII, 2018-06-21; economic nexus.
- Informing Businesses about the INFORM Consumers Act — FTC, n.d., accessed 2026-11; seller thresholds and penalties. Final rule banning fake reviews — FTC, 2024-08-14.
- ADA Title II web rule — US Department of Justice, 2024, updated 2026; scope limited to governments.

Research and practitioner sources

- FTC Relaunches Negative Option Rulemaking — Kelley Drye via JD Supra, 2026-03-13; the vacated click-to-cancel rule.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Part IX — JOURNEYS: One Order, End to End



IN THIS PART, YOU WILL: Follow four orders through every system of Kestrel & Pine's late-2027 architecture, a consumer order, its return, a chain's EDI reorder and a Black Friday evening, and see which system keeps each promise and where the same order broke a year earlier.

KEY POINTS:

- **An order is a chain of promises, each with one owner.** Eva's jacket crosses fifteen handoffs on the way out and twelve on the way back; Bergzicht's pallets cross fifteen more. The joins between rows are where most failure patterns live: eight systems holding a piece of one order is normal; a fact with two writers is not.

- **The same four stories run on two architectures.** Each journey keeps the ledger's April 2027 dates and amounts exactly, runs them through the Waypoint architecture of late 2027, and then through the January 2027 landscape. The dates do not change; what changes is whether the promises held.
- **Operations by event, finance by batch, and every join with a key and a net.** Stock and order handoffs cannot wait an hour; invoices and payouts can wait a day. Every message may arrive twice or not at all, so each carries an idempotency key, and a daily reconciliation is the net under every webhook.

FAILURE PATTERNS THIS PART ADDRESSES:

- **Schrödinger's Order** — an order or a return whose status lives in three systems has no status; the journeys show where one record replaces three.
- **The Phantom Stock** — the storefront's "in stock" is a copy of the warehouse's count, and the journeys show how old that copy may be on an ordinary Tuesday and on a Friday in November.
- **The First Snowfall** — the order flow, the stock chain and the evening shift met forty times their normal load for the first time on the day itself.

The eight parts before this one took the commerce platform apart, one capability at a time. **This part puts it back together.** Four stories each follow one order through every system it touches, so the chapters you have read become steps on one path. Kestrel & Pine, the fictional outdoor brand this book follows, is the setting throughout; its figures, dates and systems are invented for teaching.

Figure 1: *An order is a chain of promises: each system carries the order for one leg, hands it on in a marked zone, and owns only its own leg.*

Part IX covers the whole of the order's journey: out, back, sideways and at speed. Journey 1 walks Eva Janssen's jacket from the search box to the ledger; Journey 2 brings it back as an exchange; Journey 3 follows a retail chain's reorder that competes with Eva for the same shelf; Journey 4 runs Eva's order 2,300 times in one hour on Black Friday 2026.

How to read the journeys. The first three chapters have the same shape. An entity–owner table says which single system is the system of record for each fact about the order. A step table lists each step's system, data, promise, likely failure and owning chapter. A sequence diagram draws the handoffs, a requirements table says how fresh, consistent, fast and available each join must be, and the failures get their mechanism, detection and recovery. The fourth journey is told twice, hour by hour: as it happened in 2026, and as it is planned for 2027 with a forecast, switches and a runbook.

The setting is the late-2027 architecture. The first three journeys run through the landscape after Waypoint has landed: the availability service computing what each channel may promise (W1), the OMS as the one order record (W2), the PIM as the source of product content (W3) and, for wholesale, the despatch advice (the message telling a buyer what is on each pallet) written by the warehouse system (W6). The ledger's dates are April 2027, when none of this was live yet; running an April order through the December architecture is a teaching device, and every date and amount is kept. Each journey ends with a **"before Waypoint" comparison**: the same order run through the January 2027 landscape, with its hourly stock copies, three order records and two refund writers.

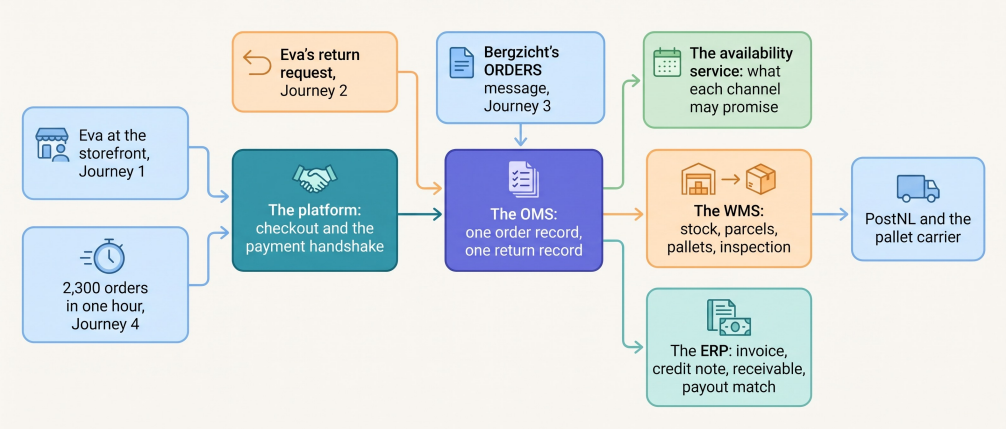


Figure 2: *Four journeys, one chain: a consumer, a return, a retail chain and a peak evening enter the same systems through different doors, and every arrow is a handoff with one sender and one owner.*

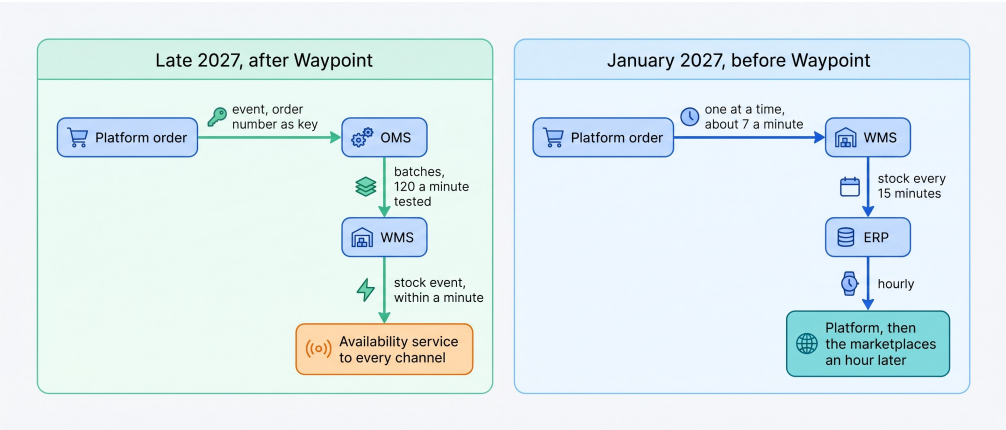


Figure 3: *What the "before Waypoint" comparison in each journey shows: in January 2027 orders crawled to the warehouse one at a time and stock came back through three scheduled copies up to two hours late; in late 2027 orders move as keyed events in batches and stock returns as events within a minute.*

Chapters in this part

Journey 1: Eva's Jacket, From Search to Doorstep decides **which single system owns each promise an ordinary order makes, and how the next system learns whether it was kept**: the PIM owns what the jacket is, the availability service whether it can be sold, the platform the checkout and the payment handshake, the OMS the order from the moment it exists, the WMS the stock and the parcel, the ERP the invoice and the books.

The case is order #1072455: on Tuesday 13 April 2027 at 21:14 Eva orders a Ridgewalker shell jacket in women's M moss and two pairs of merino socks, €235 paid by iDEAL, delivered by PostNL on Thursday 15 April. In January 2027 the same order got a static delivery promise a day wrong and three order records.

Journey 2: The Jacket Comes Back decides **which event may move the goods, the money and the books when the order comes back**: the warehouse's disposition from inspection, not the request and not the carrier scan, releases the restock, the exchange or the refund, and the credit note; exactly one system calls the payment provider, with the return id as the key. The case is Eva's exchange from M to L, requested on Monday 19 April and shipped on Thursday 22 April once the M was graded resellable, with a credit note and a new invoice that net to zero. The failure path is the Double Refund of January 2027: 212 returns refunded twice, €19,400.

Journey 3: A Chain Reorders by EDI, From Order to Invoice decides **which system writes each wholesale document, and why the despatch advice, not the order, is where the money is lost**: the order record belongs to the OMS, the despatch advice to the WMS that packed the pallets, the invoice to the ERP, and the money arrives as a bank transfer to be matched thirty days later. The case is Bergzicht Outdoor's spring reorder: an EDI order message on Tuesday 6 April 2027 for 240 units, €19,860 excluding VAT, three pallets delivered and invoiced on Friday 9 April, payment due 9 May. The failure path is the ASN Chargeback of 2026, €14,200 deducted because the ERP wrote the despatch advice before the warehouse had packed anything. A Belgian shop's Peppol invoice is the variant.

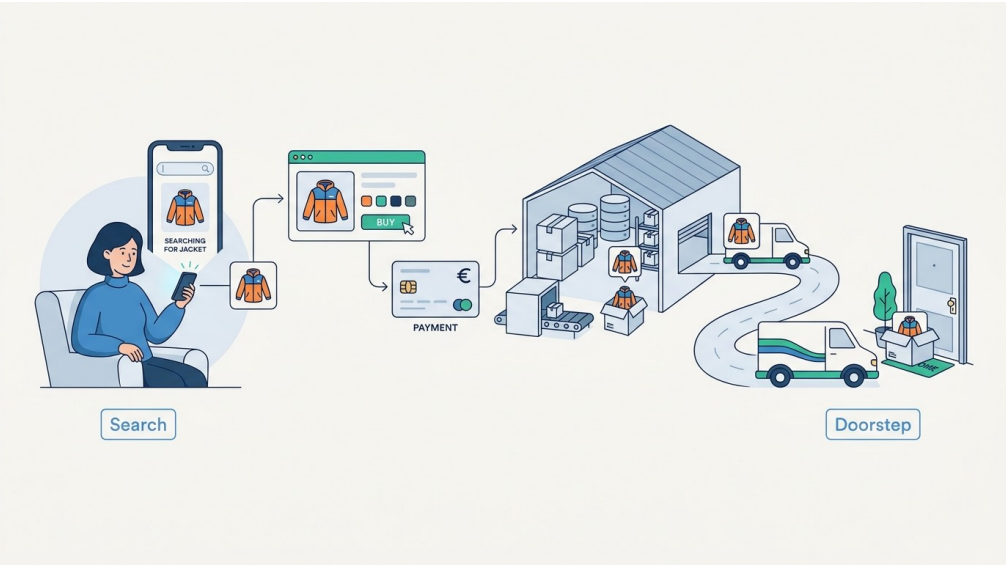
Journey 4: Black Friday, Hour by Hour decides **whether the peak is a campaign the marketing team owns or an operation the whole chain rehearses**. The case is Black Friday week 2026, 23–29 November: 31,000 orders in the week and 2,300 in the hour from 20:00 on Friday 27 November; an order flow to the warehouse that fell behind at about 13:00 and was four hours behind by 21:00 with no gauge to show it; a sell-out at 19:40 that reached the storefronts at about 20:30 and the marketplaces at about 21:35; 1,340 units sold that did not exist, 1,120 affected orders and about €96,000. Then the 2027 plan for Friday 26 November: a forecast of 60 orders a minute, every link tested at 80, stock events within a minute, eight switches with owners, a runbook and a reconciliation report at 08:00 on Saturday.

Begin with Journey 1: Eva's Jacket, From Search to Doorstep.

Sources and Evidence

Every public claim in this part is sourced and dated in the chapter that makes it, under that chapter's Sources and Evidence. Kestrel & Pine and Bergzicht Outdoor are fictional; recommendations and labeled assumptions are the author's judgement.

37. Journey 1: Eva's Jacket, From Search to Doorstep



IN THIS CHAPTER, YOU WILL: Follow one consumer order through every system it touches, from search to the ledger, and see which system makes each promise and what breaks when it cannot keep it.

KEY POINTS:

- **One order is fifteen handoffs, and each handoff is a promise.** The storefront promises a product, a price, a stock level and a delivery day; the payment provider promises money; the warehouse promises a parcel; the carrier promises a doorstep; the ERP promises a correct invoice. Nearly everything that goes wrong is a promise one system made and another could not keep.
- **Each promise has exactly one owner in the target architecture.** The PIM owns what the jacket is, the availability service owns whether it can be sold, the platform owns the checkout and the payment handshake, the OMS owns the order from the moment it exists, the WMS owns the physical stock and the parcel, the ERP owns the invoice and the books. Copies are allowed; a second writer is not.

- **Every join delivers at least once and sometimes twice, late or not at all.** The order number is the key that makes a repeated message harmless, and a daily reconciliation is the net under every webhook. A design that assumes each message arrives exactly once is the one that produces the duplicates and the gaps customers find.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **Schrödinger's Order** — an order whose status lives in three systems has no status; this journey shows where one record replaces three.
- **The Phantom Stock** — the storefront's "in stock" is a copy of the warehouse's count, and the journey shows how long that copy may be allowed to lag.
- **The Message in a Bottle** — the order crosses six joins, and the chapter names, at each, what happens when the message arrives twice or not at all.

Most of this book takes the order apart, one capability at a time. This chapter puts it back together. It follows **one real-shaped order through every system in sequence**, so that the chapters you have read become steps on one path and the chapters you have not yet read become visible as the places where this path can break.

The order is a plain one on purpose: a jacket and two pairs of socks, paid by the most ordinary Dutch payment method, shipped by the most ordinary Dutch carrier, delivered in two days. **That is the point**: at 293,000 orders a year, a path that fails once in a thousand times fails almost every day, so the plain path is where architecture earns its keep.

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), is used throughout; its figures, dates and systems are invented for teaching.

Where we are in the order's journey. All of it. This chapter walks Eva's jacket from the search box to the ledger; Journey 2: The Jacket Comes Back continues with the exchange six days later; Follow the Order: What Happens Between "Buy" and "Delivered" introduced the same path at an overview level in Part I.

The Order, the Setting and the Rules of This Walk

The order. On **Tuesday 13 April 2027 at 21:14**, Eva Janssen in Nijmegen orders on the Dutch storefront: one **Ridgewalker shell jacket**, **women's**, **size M**, **color moss (€199)** and **two pairs of merino hiking socks (€18 each)**. Total **€235** including VAT, free shipping (the threshold is €75), paid by **iDEAL**. Order number **#1072455**. The order is picked and packed at the Duiven warehouse on **Wednesday 14 April**, shipped with PostNL and delivered on **Thursday 15 April**.

The setting. This chapter runs the order through Kestrel & Pine's **target architecture of late 2027**: the landscape after the first three Waypoint decisions have landed, with the **availability service** live (W1), the **OMS** acting as order hub (W2; storefront orders route through it from 13 October 2027) and the **PIM** live (W3). In April 2027 the order in fact ran through the January landscape, which had none of these; a short section near the end shows how. Running an April order through the December architecture is a teaching device; the ledger's dates and amounts are kept exactly.

The systems. Each is named by its role, as the whole book does. The table is also the **entity–owner table** for this one order: for each fact, the single system whose value is the truth when copies disagree, which Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record calls the **system of record**.

Fact about the order	System of record (late 2027)	Who else holds a copy
What the Ridgewalker is: name, attributes, images, sizes, GTIN	**the PIM** (product information management system)	the platform, the search index, the marketplace connector
Consumer price €199	**the platform's** price list for the NL storefront	the search index, the OMS (as the price sold at)
Physical stock at Duiven	**the WMS** (warehouse management system)	the availability service, the ERP (valuation only)
What may still be sold to the NL storefront	**the availability service** (a module of the OMS)	the platform, the search index
The cart and the checkout	**the platform**	nobody
The payment	**the PSP** (payment service provider), with its reference stored on the order	the platform, the OMS, the ERP
The order: lines, state, customer reference, promises made	**the OMS** (order management system)	the platform (customer-facing history), the ERP, the data warehouse
The fulfillment order and the parcel	**the WMS**, with the shipping service for the label	the OMS
Tracking events	**PostNL**, normalized by the shipping service	the OMS, the platform's order status page

Fact about the order	System of record (late 2027)	Who else holds a copy
The invoice, VAT and the ledger entries	**the ERP**	the data warehouse

Eight systems hold a piece of one order, and that is the normal condition, not a problem to fix. The problem is a fact with two writers, and the table has none.

Fifteen Steps in One Table

Here is the whole journey at a glance. Each row names the step, the system that performs it, the data that moves, the promise made to Eva or to another system, the failure that belongs to this step, and the chapter that owns it. Times within a day are illustrative; the ledger fixes the dates.

#	Step	System	Data	Promise made	What could fail here	Chapter
1	Search "waterproof jacket"	the platform's search index	query, ranked products	"These products exist and match"	Stale index: a discontinued or renamed product shows	Finding the Product: Search, Navigation and Merchandising

#	Step	System	Data	Promise made	What could fail here	Chapter
2	Product page	the platform, reading PIM content	attributes, sizes, images, price €199	"This is the jacket, at this price"	Wrong attribute; price differs from another channel (The Price Drift)	What Are We Selling? Products, Variants and the Catalog, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
3	Availability check, size M moss	the availability service	available-to-promise for channel NL	"You can buy this one"	Stale copy, two writers, wrong buffer (The Phantom Stock)	Is It in Stock? Inventory, Availability and the Promise You Can Keep
4	Add to cart, 21:09	the platform	cart lines, quantities	none: a cart is a draft	Cart lost on login; abandonment	From Cart to Confirmation: Checkout Orchestration

#	Step	System	Data	Promise made	What could fail here	Chapter
5	Checkout: identity, address, delivery option	the platform, calling the delivery-promise service	address, shipping method, promised date	"Delivered Thursday 15 April"	Static promise contradicts the warehouse cut-off	From Cart to Confirmation: Checkout Orchestration, Getting It There: Carriers, Labels, Tracking and the Delivery Promise
6	Price review and VAT	the platform	€235 incl. 21% VAT, free shipping	"You pay exactly this"	Promotion stacking error; tax basis differs from invoice	Discounts, Coupons and Loyalty: Promotions Without Chaos, Tax, Invoices and the Books: From Order to Ledger
7	Payment by iDEAL, 21:13–21:14	the PSP, Eva's bank	payment reference, amount, status	"The money is paid"	Eva never returns from the bank app; webhook late or duplicated	Taking the Money: Payment Providers, Authorization, Capture and Refunds

#	Step	System	Data	Promise made	What could fail here	Chapter
8	Order created and confirmed, 21:14	the platform, then the OMS	order #1072455, lines, customer and payment references	"The order exists and will be kept"	Created twice; created in the platform but never in the OMS	The Order Record: Lifecycle, State and the Order Management System
9	Stock committed	the availability service	committed +1 jacket, +2 socks	"Nobody else can buy these units"	Commit event lost; oversold on a hot SKU	Is It in Stock? Inventory, Availability and the Promise You Can Keep
10	Fulfillment order sent and accepted	the OMS → the WMS via the iPaaS	lines, destination, service level, promised date	"Duiven will ship this"	Request duplicated or never acknowledged; WMS queue backs up	Pick, Pack, Ship: Warehouse Management and Fulfillment, APIs, Events, Files and Middleware: How Commerce Systems Talk

#	Step	System	Data	Promise made	What could fail here	Chapter
11	Pick and pack, Wednesday morning	the WMS	pick list, packed units, carton	"These exact units are in this box"	Short pick (shelf count wrong); wrong size scanned	Pick, Pack, Ship: Warehouse Management and Fulfillment
12	Label, manifest, handover to PostNL	the shipping service, PostNL	tracking number, label, manifest	"The parcel is with the carrier"	Label call fails; manifest closed before the parcel	Getting It There: Carriers, Labels, Tracking and the Delivery Promise
13	Shipped event, confirmation and tracking	the WMS → the OMS → the platform	shipment, tracking number, carrier events	"You can see where it is"	Tracking number not passed on: no updates, a WISMO ticket	Where Is My Order? Notifications, Service and the Customer Record
14	Delivered, Thursday 15 April	PostNL → the OMS	delivered event, time	"It is at your door"	Delivered to a neighbor; lost; damaged	Getting It There: Carriers, Labels, Tracking and the Delivery Promise
15	Invoice, revenue and payout	the ERP, the PSP	invoice, VAT €40.79, PSP payout line	"The books agree with the bank"	Order posted twice or not at all; payout unmatched	Tax, Invoices and the Books: From Order to Ledger

Every "promise made" column entry has a different owner from the row above it. That is why the joins between rows are where Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong locates most of its sixteen patterns.

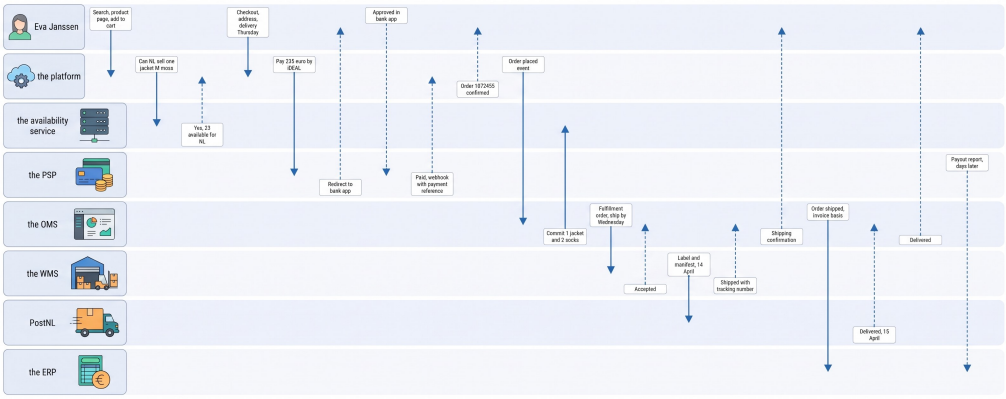


Figure 1: *Eva's jacket in the late-2027 architecture: the platform sells and takes the money, the OMS owns the order from the moment it exists, the WMS and PostNL move the parcel, and the ERP books the invoice and matches the payout; every arrow is a handoff with one sender and one owner.*

Before the Buy: Search, Product Page and "In Stock"

At 21:02 Eva types "waterproof jacket" into the Dutch storefront. The storefront does not search the catalog; it searches a **search index**, a copy of the catalog rebuilt from the PIM whenever product content is published. A copy is fast and a copy goes stale, so the first promise of the journey ("these products exist and match") belongs to the publication schedule: under W5, **products move by scheduled publication**, the PIM publishes and the index consumes. Finding the Product: Search, Navigation and Merchandising owns the index; What Are We Selling? Products, Variants and the Catalog owns the PIM.

The product page makes the second and third promises, and they have different owners. **What the jacket is** (the name, the fabric attributes, the sizes, the moss color, the GTIN, the Global Trade Item Number behind the barcode on every own-brand SKU since 2021) comes from the PIM. **What it costs** (€199, including 21% Dutch VAT, as every consumer price in the EU is shown) comes from the platform's price list for the Dutch storefront. Before the PIM the same jacket was described in four places, and the Price Drift of October 2026, nine days of the German storefront selling it at €179 while every other channel charged €199, is what a silent copy costs (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax).

"In stock" is the most expensive words on the page. When Eva selects size M in moss, the storefront asks the availability service for **available-to-promise** for the NL channel: in plain words, the units that may still be promised to a new order. The formula the industry uses, as practice rather than as any one vendor's definition, is on-hand stock minus units already committed to orders minus any buffer the merchant holds back, per location and per channel. Every platform exposes some version of it: Shopify's inventory documentation (n.d., accessed 2026-11) separates `on_hand`, `committed` and `available` per location; commercetools documents that its `availableQuantity` is "eventually consistent" with the quantity used for reservations "with a delay of up to 10 seconds" (n.d., accessed 2026-11). Ten seconds is small; on a hot SKU during a launch it is still enough to sell the last unit twice, which is why Is It in Stock? Inventory, Availability and the Promise You Can Keep treats the buffer and the reservation path as design decisions.

In the target architecture the answer comes from one place. The WMS is the system of record for physical stock at Duiven (W1) and emits a **stock event**, a message sent the moment a count changes, for every receipt, pick and adjustment. The availability service consumes those events, subtracts commitments and the per-channel buffer (one unit per SKU for the storefronts), and publishes an available number per channel; the storefront reads it and never computes its own. Assume the moss jacket in size M shows 23 available for NL at 21:05 *(assumption)*. "In stock" is then a copy seconds old rather than up to two hours old, the January 2027 figure behind the Black Friday Oversell.

Checkout: Six Promises Under a Clock

Eva adds the jacket and two pairs of socks at 21:09. A **cart** is a draft: it reserves nothing and promises nothing, and most carts never become orders; the Baymard Institute's running average of fifty published studies puts abandonment at 70.22% (updated 28 October 2026). Kestrel & Pine's platform, like most, reserves stock at order creation rather than at add-to-cart, so an abandoned cart costs nothing but a lost sale.

Checkout is the ninety seconds in which the platform turns the draft into promises it can keep. It is an orchestration: a sequence of calls to systems that own the facts, run under the clock of Eva's patience, and it owns none of those facts itself (From Cart to Confirmation: Checkout Orchestration). In order:

1. **Identity.** Eva signs in to her account. An order needs a customer *reference*, not an account; forcing one is the fourth-largest stated reason to abandon in Baymard's list. Her account is still one of four records of her, which is One Customer, One Product: Identity and Master Data Across Systems's problem, not this chapter's.
2. **Address.** Nijmegen, the Netherlands. The address decides the tax: a Dutch consumer pays Dutch VAT at 21%; a Belgian one would pay Belgian VAT under the EU's One Stop Shop rules once the merchant's cross-border consumer sales pass €10,000 a year (Tax, Invoices and the Books: From Order to Ledger).

3. **Delivery option and promise.** The Dutch storefront's standing promise is "ordered before 22:00, delivered tomorrow". In January 2027 that was a sentence in the page template with no data behind it. In the target architecture the checkout asks a **delivery-promise service**, which W8 lists as a prerequisite, and that service knows the warehouse's real cut-off. Duiven releases its last pick wave at 21:00 and PostNL collects at 22:30 (Kestrel & Pine illustrations from Pick, Pack, Ship: Warehouse Management and Fulfillment): an order placed at 21:14 misses the last wave, is picked Wednesday and delivered Thursday, so the checkout shows **Thursday 15 April**, and that is the day the parcel arrives. The dates are the same under both architectures; what changes is whether the promise matched them. Getting It There: Carriers, Labels, Tracking and the Delivery Promise owns the computed promise.
4. **Price review.** €199 plus 2 × €18 is €235; shipping is free above €75; no coupon applies. The platform freezes these amounts on the order. The price is now a promise, and the invoice in step 15 must match it.
5. **Payment.** See the next section.
6. **Confirmation.** The order exists.

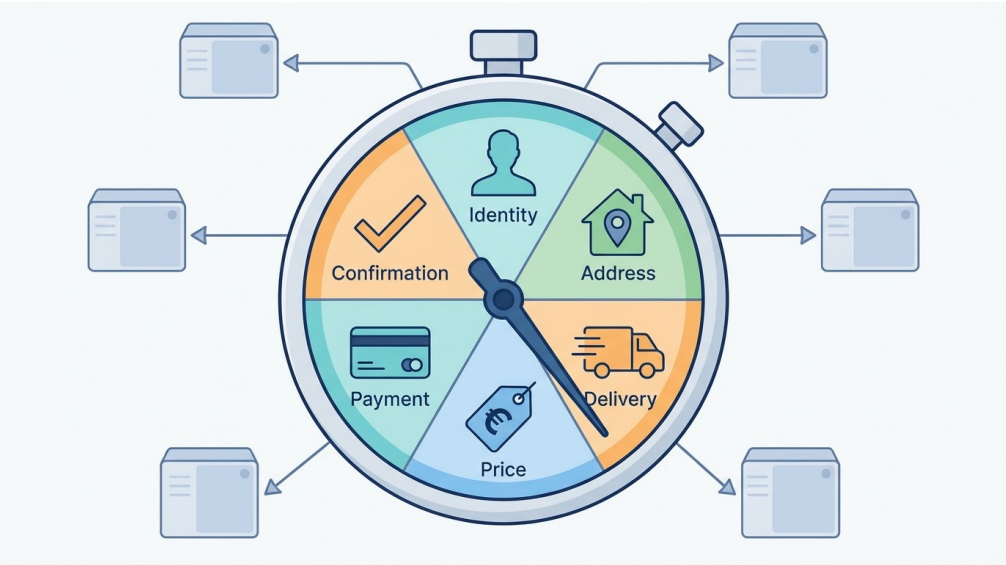


Figure 2: *Checkout is six promises under a clock: the platform orchestrates them in about ninety seconds, but every fact it promises is owned by another system.*

None of these calls should be patched into the checkout's markup. Shopify's dated history is the public lesson: in February 2023 it announced that the `checkout.liquid` file merchants had edited would stop working for the main checkout pages on 13 August 2024, replaced by sandboxed extensions. Kestrel & Pine's agency customized the theme,

not the checkout, and the decision to stay on the platform (W4) depends on keeping it that way.

The payment: iDEAL, a detour through the bank

Eva chooses iDEAL. It matters which method she chose, because the money's state machine differs by method, and the order system must model the one it is given rather than a generic "paid: true" (Taking the Money: Payment Providers, Authorization, Capture and Refunds).

iDEAL is a bank-initiated transfer: the shopper is redirected to her own bank's app, approves the amount there, and the bank pushes the money. The Dutch Payments Association reported 1.47 billion iDEAL payments worth €141 billion in 2024 (18 February 2025). Mollie's method documentation (n.d., accessed 2026-11) states the properties that shape the architecture: an "instant charge", **auto-capture only**, no chargeback risk, a session that expires after 15 minutes. Three consequences follow.

- **There is no separate capture.** For a card, the PSP first *authorizes* (reserves the money) and the merchant later *captures* (takes it), often when the parcel ships. For iDEAL the money moves at approval; "capture on shipment" does not apply, and a refund is a new transfer back, not the release of a hold.
- **The shopper leaves and may not come back.** Between 21:13 and 21:14 Eva is in her bank's app. Stripe's fulfillment guide (n.d., accessed 2026-11) states the rule for every PSP: "You can't rely on triggering fulfillment only from your checkout landing page, because it's not guaranteed customers visit that page." The platform learns the outcome from the PSP's **webhook**, an HTTP call the PSP makes to the merchant's system when the status changes, not from Eva's browser.
- **The webhook is a notification, not a fact.** Mollie's webhook carries only an identifier, "the status is not transmitted", so a forged call cannot mark an order paid; the receiver fetches the status from the API. Mollie retries an unanswered webhook ten times over 26 hours; Stripe retries for up to three days and does not guarantee order. The platform records the payment reference and the fetched status on the order, never the webhook's claim.

At 21:14 the platform creates order **#1072455** with the payment *pending* and shows Eva the confirmation; seconds later the PSP's webhook arrives, the platform fetches the status, and the order is marked paid. The order confirmation e-mail, the one notification Shopify's documentation says cannot be switched off (n.d., accessed 2026-11), goes out from the platform in the same minute.

The Order Becomes a Record: Platform to OMS to WMS

Until 21:14 the order was the platform's. From 21:14 it belongs to the OMS, and this handoff is the one Waypoint (W2) was created to fix.

The platform emits an "order placed" event carrying the order number, the lines, the frozen prices, the customer reference, the delivery promise and the PSP's payment reference. The iPaaS, the integration platform through which Kestrel & Pine's flows run, delivers it to the OMS. The OMS creates the order with **one state model** for every channel: the money state (paid, by iDEAL, with its reference), the goods state (unfulfilled) and later the return state are three separate machines on one record, as Shopify's and commercetools' order models keep them (both n.d., accessed 2026-11), because "fulfilled" never means "delivered" and "paid" never means "invoiced". The platform keeps the order history Eva sees in her account; the OMS keeps the operational truth; the ERP will keep the financial one.

The order number is the idempotency key. The event may arrive twice: Shopify's documentation says plainly that "your app might receive the same webhook more than once" and that ordering is not guaranteed (n.d., accessed 2026-11), and every queue and iPaaS retry behaves the same way. **Idempotency** means that handling a message twice has the same effect as handling it once; the OMS keys order creation on #1072455, so the second copy finds the order and does nothing. Stripe's API applies the same principle in the other direction, saving the first response to a client-supplied idempotency key so a retry returns it instead of charging again (n.d., accessed 2026-11). W5 makes this a standard for every Kestrel & Pine integration.

Stock is committed. The OMS tells the availability service to move one jacket and two pairs of socks from available to committed for the Duiven location. On-hand does not change; the shelf still holds the units. Available drops from 23 to 22 for the moss jacket in M, and the next shopper's product page says so within seconds.

The fulfillment order is the contract with the warehouse. The OMS now creates a **fulfillment order**, an instruction to one location to ship these lines to this address by this service level, and sends it to the WMS through the iPaaS. It carries what the warehouse needs (lines, destination, carrier service, promised date) and nothing it does not (prices, payment, customer history). The handshake follows the best-documented public model: in Shopify's fulfillment-order design the merchant *requests* fulfillment, the warehouse *accepts* or *rejects*, and once accepted the merchant must *request* cancellation rather than cancel (shopify.dev, n.d., accessed 2026-11). The WMS accepts at 21:15; the order is now **in progress**, the state the Three-Status Order of December 2026 could never agree on.

Wednesday morning at Duiven. The order joins a **pick wave**, a group of orders released together so one walk through the warehouse serves many of them, at the first release of the day, 06:30 (illustrative). A picker scans the jacket's and the socks' GTINs

into a tote; the scan is the moment the WMS confirms that the units it promised exist. Packing chooses the carton and closes the lines. Two things then leave the WMS for different places: a **stock event** (on-hand minus three units) to the availability service, which also clears the commitment, and a **shipped event** to the OMS once the label exists.

The label comes from the **shipping service**, the multi-carrier layer inside the WMS flow. It asks PostNL's API for a label and a tracking number, prints it, and at the end of the day **manifests**: closes the day's list of parcels and hands it to the carrier. Manifesting is a carrier-specific rule integrations forget; DHL's German parcel API transfers shipments "only when manifested", with an automatic close-out at 17:45 (developer.dhl.com, updated 2026-11), and PostNL's own APIs are mid-migration to a renewed generation rolled out from 2025 (n.d., accessed 2026-11). The shipping service absorbs these differences so the WMS sees one label call and one tracking vocabulary (Promises Between Systems: API Contracts, Webhooks and Versioning).

At 18:30 the shipping service closes Wednesday's PostNL manifest, and at 22:30 the PostNL truck collects (times illustrative). Eva's jacket has left the building.

In Transit: Tracking, Notifications and "Where Is My Order"

From the collection scan onward the parcel's state is PostNL's to know and Kestrel & Pine's to relay. A **tracking event** is one carrier scan with a time and a place: accepted, in transit, out for delivery, delivered, attempted. Carriers each have their own codes; a multi-carrier layer maps them to one small vocabulary, such as the ten statuses EasyPost documents for its tracker object (n.d., accessed 2026-11). The shipping service feeds normalized events to the OMS, which updates the order's goods state and raises the notifications.

Eva receives three messages with three origins: the **shipping confirmation** with the tracking link when the WMS reports the label (Wednesday afternoon), **out for delivery** when PostNL's morning scan arrives (Thursday 07:50), and **delivered** at 11:20 on Thursday (times illustrative). The order status page in her account is a read model over the OMS's order and the tracking stream, the same record a service agent sees; built from different sources, the two would disagree on the day it matters.

The measure of this part of the journey is a service metric. "Where is my order" (WISMO) is consistently the largest single category of e-commerce support contacts; Gorgias's helpdesk data put it at 18% of incoming requests (updated 13 November 2023). The number is an architecture signal: a tracking number that never left the WMS, a static promise, a split shipment without an explanation. Shopify's help documentation states the failure in one sentence: "If you fulfill an order without a tracking number, then no shipping updates are received" (n.d., accessed 2026-11). Where Is My Order? Notifications, Service and the Customer Record owns the service view.

Delivery is also a legal event. Under the EU Consumer Rights Directive (2011/83/EU), the risk of loss passes to the consumer when she "has acquired the physical possession of the goods"; until 11:20 on Thursday the parcel is Kestrel & Pine's risk, which is why the OMS needs the delivered event and not only the shipped one.

The Money and the Books

The parcel's journey ends at the door. The order's does not. Three documents still have to agree: the invoice, the payment and the bank.

The invoice comes from the ERP, not the shop. The OMS sends a "shipped" event to the ERP on Wednesday, and the ERP issues the invoice from it. The reason is numbering. A Dutch invoice must carry a unique, sequential invoice number and date, the parties, the goods, the net amount per VAT rate and the VAT amount in euros (Belastingdienst, n.d., accessed 2026-11); "sequential" means gap-free within a series, so the number is assigned by one system at the moment of posting, never pre-allocated by a checkout. Once issued, an invoice is immutable: Odoo's documentation, representative of any ERP, states that "Once confirmed, an invoice can no longer be updated" and that a credit note "is the only legal method for canceling, refunding, or modifying a validated invoice" (18.0 documentation, accessed 2026-11). Eva's exchange next week will produce a credit note, not an edit (Journey 2: The Jacket Comes Back).

For #1072455 the invoice reads, net of 21% VAT: jacket €164.46, two pairs of socks €29.75, total net **€194.21**, VAT **€40.79**, gross **€235.00**, arithmetic on the ledger's prices. The amount matches the checkout because nothing changed between the two; when something does (a short pick, a canceled line) the invoice follows what shipped, not what was ordered (Tax, Invoices and the Books: From Order to Ledger).

Revenue is recognized at delivery, not at payment. Under IFRS 15 (issued 2014, effective from 2018), revenue is recognized when control of the goods passes to the customer, which for a parcel is delivery. Kestrel & Pine was paid on Tuesday, invoiced on Wednesday and earned the revenue on Thursday, and a 24% item return rate means a refund liability sits beside the revenue until the withdrawal period runs out. Sanne de Wit's month-end depends on these being three different fields.

The payout arrives days later and is matched by finance, not by the order system. The PSP pays out in batches: Adyen's default "sales day payout" groups a day's funds into one batch with a delay of two business days (n.d., accessed 2026-11); Stripe's default settlement in most EU countries moves from seven calendar days to three business days once an account is established (n.d., accessed 2026-11). One bank deposit therefore contains hundreds of orders, minus fees, refunds and chargebacks. The join is a **payout reconciliation report**: Stripe's lists "every payment, refund, dispute, fee, and other balance transaction included in the payout" with the order id and a `trace_id` the bank also sees (n.d., accessed 2026-11). The ERP matches Eva's €235 inside Tuesday's batch to

the invoice, posts the PSP's fee (for iDEAL a fixed amount per transaction on Adyen's indicative price list, n.d., accessed 2026-11), and the three-way match of invoices, PSP report and bank statement closes. The deeper mechanics belong to the sibling book RECONCILED (Payments-to-Ledger: Draining the Suspense Swamp, Reconciled by Design: Traceability and Audit Across Boundaries).

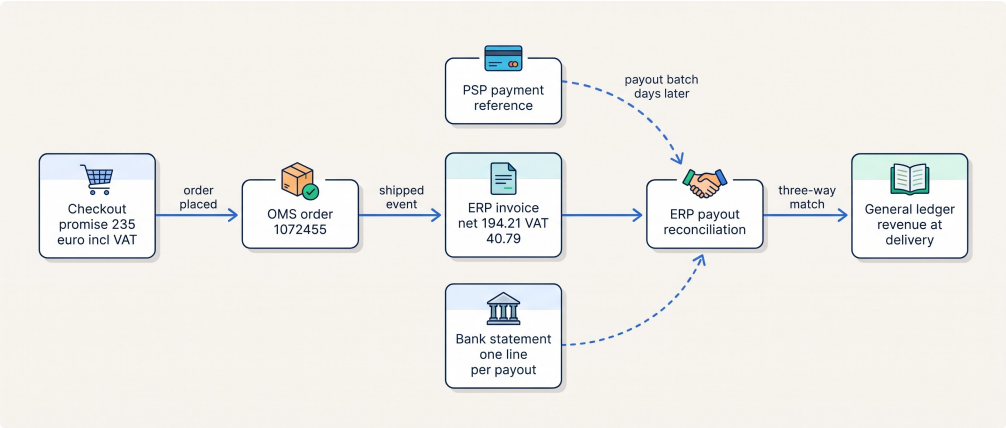


Figure 3: *One order, three financial records: the checkout's promise, the ERP's invoice issued on dispatch, and the PSP's payout matched against the bank statement; the ledger closes only when all three agree.*

What Each Handoff Requires

The style of every join follows from four questions: how fresh the data must be, how consistent, how fast, and how available. The latency figures are the author's illustrative targets for a merchant of Kestrel & Pine's size, not ledger facts.

Handoff	Freshness	Consistency	Latency	Availability	Style (W5)
WMS → availability service → storefront	Seconds	Eventually consistent, one writer per quantity	Under 10 s *(assumption)*	Storefront keeps the last value, with a buffer	Events

Handoff	Freshness	Consistency	Latency	Availability	Style (W5)
Checkout → PSP → platform	Redirect immediate; webhook within minutes	Status fetched, never assumed	Under 2 s per call *(assumption)*	Fails closed: no order without a fetched status	Call plus webhook
Platform → OMS	Seconds	One order per order number	Under 30 s *(assumption)*	OMS catches up by replay	Events, idempotent
OMS → WMS (fulfillment order)	Minutes	Request, accept, reject, cancel-request	Before the next pick wave	Queue absorbs an outage; the cut-off is the deadline	Events with acknowledgement
WMS → shipping service → carrier	At pack	One label per parcel; manifest once	Under 5 s per label *(assumption)*	Fallback carrier (DPD) if the label call fails	Synchronous call
OMS → ERP (invoice, posting)	Same day	One invoice per shipment, one posting per invoice	Daily batch	Reconciliation count per day	Daily batch
PSP → ERP (payout)	Days	Batch total equals bank line	Days	Monthly close	File or reporting API

Two rows cannot be batch, availability and the fulfillment order, because an hour's delay on the first is the Black Friday Oversell and on the second a missed cut-off. Two rows should be batch, invoice posting and payouts, because finance tolerates a day and gains a reconciliation count from it. "Finance can be batch; availability cannot" is the rule The Back Office: ERP Integration and Who Is the Hub states, and this journey demonstrates it.

The Same Order in January 2027

Run #1072455 through the landscape Noor Visser found in January, and four things change while the dates stay the same.

Step	Late 2027	January 2027
"In stock" on the product page	Seconds old, from the availability service on WMS events	Up to two hours old: WMS → ERP every 15 minutes, ERP → platform hourly (The Phantom Stock)
Delivery promise at checkout	Computed: Thursday 15 April	Static "delivered tomorrow": Wednesday 14 April, which the parcel missed by a day
The order record	One, in the OMS, with one state model	Three: the platform's order, the WMS's pick, the ERP's posting, each with its own status (Schrödinger's Order)
The invoice	From the ERP on the shipped event	From the ERP through the partner-built connector, only on the WMS's "order complete" message, so a split order showed "Not invoiced" for days

The parcel still shipped on Wednesday and arrived on Thursday; Eva would have noticed only that the promised day was wrong. The Three-Status Order of December 2026, which showed *Delivered*, *In picking* and *Not invoiced* for six days and cost eleven tickets, is this January path with one split shipment: the iPaaS flow marked the whole two-line order fulfilled at the first parcel, the WMS still showed the second line in picking, and the ERP waited for an "order complete" message that could not come until the second parcel shipped. Nothing in that landscape was broken; the joins had no owner, no idempotency and no reconciliation, and W5 is the sentence that changed that.

When It Goes Wrong Along This Path

The step table names a failure per step. Five of them deserve the mechanism, the detection and the recovery, because they recur at every merchant.

The duplicate order. The "order placed" event arrives at the OMS twice, or the iPaaS replays a day after an outage. Detection: by design, the OMS keys on the order number and the second copy is a no-op; a count of "orders received" against "orders created" per hour shows the duplicates without harm. Without the key, the WMS ships two parcels and the customer keeps one: the Message in a Bottle in its most expensive form.

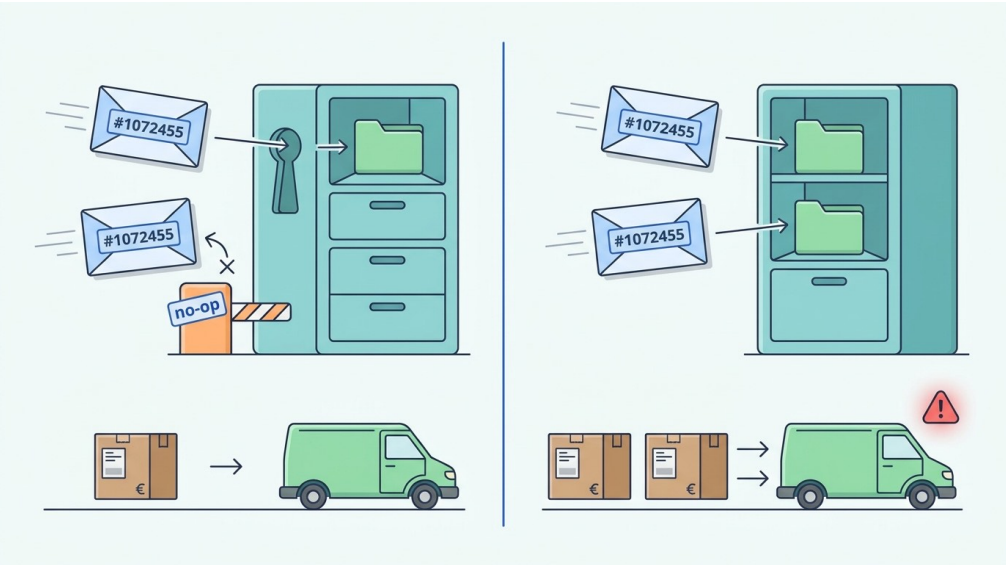


Figure 4: *The order number is the key that makes a repeated message harmless: the second copy of #1072455 is a no-op, and without the key the warehouse ships two parcels.*

The order that was paid and never confirmed. The PSP's webhook fails for 26 hours and then stops; the shopper paid and the order still says *pending*. Detection: a daily sweep that lists checkouts whose payment reached no terminal state within the method's expiry window and queries the PSP, the job Stripe's and Mollie's documentation both ask merchants to run. Recovery: mark the order paid from the fetched status, late but once, and release it to the OMS. Webhooks are a latency optimization over a reconciliation, never a replacement (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation).

The short pick. The picker finds no moss jacket in M where the WMS says there are 22. The classic study of inventory-record accuracy found 65% of about 370,000 records inaccurate at one US retailer's stores (DeHoratius and Raman, *Management Science*, 2008, dated but still the reference point); cycle counting keeps the number low, not zero.

Detection: at the scan. Recovery: the WMS reports the line short, the OMS splits or holds it, the availability service corrects on-hand, and the customer is told before the promise date. A WMS that silently ships two of three items produces a wrong invoice and a WISMO ticket. The queue that backed up for four hours on Black Friday 2026 is the same path under load (Journey 4: Black Friday, Hour by Hour).

The lost parcel. No scan for 48 hours after collection. Detection: a rule on the tracking stream, not a customer e-mail. Recovery: re-ship from stock (the risk was still the merchant's) and claim against the carrier; the OMS records a second shipment on the same order, and the invoice does not change. PostNL reports a parcel "delivery quality" of 97% in 2023 and 2024 (annual report, 2025); at Kestrel & Pine's volumes the remaining 3% is about ten thousand parcels a year that need this path.

The payout that does not match. Tuesday's batch is short by €235. Causes, in the order finance checks them: a refund deducted from a later batch, a fee netted differently, a failed refund returned days later, or an order refunded twice, the Double Refund of January 2027 (€19,400, twice). Detection: the three-way match. Recovery: a journal correction with the PSP's line as evidence and, for the double refund, a rule that only one system may call the PSP's refund API (Taking the Money: Payment Providers, Authorization, Capture and Refunds).

What to Take Away

Established.

- The order's journey crosses systems that each own one fact, and the handoffs between them are where commerce failures concentrate; Shopify, commercetools, Stripe and Mollie document the same model from four sides: separate state machines for money and goods, order creation driven by fetched payment status, request-and-accept handshakes with the warehouse.
- Every delivery mechanism between those systems is at-least-once in practice: Shopify, Stripe and Mollie all document duplicate and out-of-order deliveries and ask the receiver to deduplicate and reconcile. Exactly-once is a property the receiver builds with an idempotency key, not one it receives.
- The invoice is issued by one numbering authority at dispatch, is immutable once issued, and is corrected only by credit note; revenue is recognized at delivery under IFRS 15. The order system makes promises; the ERP keeps the books.

Depends on context.

- Whether the OMS should exist at all. At Kestrel & Pine's stage (four channels, one warehouse, stores, marketplaces, EDI) one order record is necessary; a Starting or Growing merchant with one channel and a 3PL gets the same journey from

the platform's own order and fulfillment-order model, and an OMS would be the Giant's Boots.
- Whether the delivery promise should be computed. A static rule is honest at one warehouse with one carrier and a cut-off the warehouse actually meets; compute the promise when the warehouse's collection time and the page's cut-off differ, as they did here, or when there is more than one lane.
- Whether capture should follow shipment. For cards, capturing at dispatch saves refund fees and ties money to goods; for iDEAL and other instant methods there is no capture step, and the design must not assume one.

The author's synthesis.

- I read any commerce architecture by walking one order through it with this chapter's step table, and I stop at every row where the promise and the system of record belong to different systems with no event, no key and no reconciliation between them. Those rows are next year's incidents.
- I would rather have eight systems with one owner per fact than three systems with two owners for stock. Count writers, not boxes.

Questions to Ask

1. For each fact on an order (product, price, stock, promise, payment, status, invoice), which single system is the system of record, and which systems hold copies?
2. What is the idempotency key for "create order", "create fulfillment order", "refund" and "post invoice", and what happens when the same message arrives twice?
3. When the PSP's webhook never arrives, which job notices, how long after, and who owns it?
4. Where does the delivery promise shown at checkout come from, and is the warehouse's real cut-off one of its inputs?
5. Does the storefront's "in stock" come from an availability service fed by stock events, or from a periodic copy, and how old can the copy be at peak?
6. When the warehouse picks short, who tells the customer, when, and does the invoice follow what shipped?
7. Does the order status page read the same record the service agent reads?
8. Which system issues the invoice number, and can anyone edit an issued invoice?
9. How is one PSP payout matched to the orders it contains, and what were last month's unexplained differences?

Eva's jacket arrived on Thursday, and on Monday she will ask for a size L. The way back crosses many of the same systems in the opposite direction, with a refund and a credit note where the payment and the invoice were, and it is where Kestrel & Pine paid €19,400 twice. Journey 2: The Jacket Comes Back follows it.

Sources and Evidence

Company, product and investor sources

- Inventory management apps: inventory quantities — Shopify developer documentation, n.d., accessed 2026-11; the named inventory states (`on_hand`, `committed`, `available`).
- Draft order and transfer shipment inventory is moving from reserved to committed — Shopify changelog, 2026-08-05; a stock state's meaning changing under its readers.
- Build for fulfillment services and FulfillmentOrder object — Shopify developer documentation, n.d., accessed 2026-11; the request, accept, reject and cancellation-request handshake.
- Order status — Shopify Help Center, n.d., accessed 2026-11; separate payment and fulfillment status families on one order.
- Webhooks and HTTPS delivery — Shopify developer documentation, n.d., accessed 2026-11; duplicate and unordered deliveries, retries, reconciliation jobs.
- Customer notifications and Understanding order status pages — Shopify Help Center, n.d., accessed 2026-11; the notification list, the order confirmation that cannot be disabled, and "no tracking number, no updates".
- The checkout.liquid theme file is being deprecated — Shopify changelog, 2023-02-13, and checkout.liquid, n.d., accessed 2026-11; the dated removal of checkout markup customization.
- Orders and Inventory — commercetools API documentation, n.d., accessed 2026-11; order, shipment and payment states; the 10-second availability consistency window.
- Fulfill orders with Checkout, Webhooks, Idempotent requests, Payouts and Payout reconciliation report — Stripe documentation, n.d., accessed 2026-11; fulfillment from webhooks, retries for three days, idempotency keys, settlement timing, payout matching.
- iDEAL, Webhooks and Handling payment status — Mollie documentation, n.d., accessed 2026-11; iDEAL's instant charge and auto-capture, the id-only webhook, ten retries over 26 hours.
- Capture, Sales day payout and Pricing — Adyen documentation and price list, n.d., accessed 2026-11; authorization versus capture, payout batching and delay, indicative iDEAL fee.
- Annual turnover iDEAL and PIN payments 2024 — Dutch Payments Association, 2025-02-18; iDEAL volumes.
- Parcel DE Shipping API v2 — DHL developer portal, updated 2026-11; manifesting as a carrier rule.
- PostNL developer portal and API overview — PostNL, n.d., accessed 2026-11; the phased V4 migration and checkout delivery options.

- Trackers — EasyPost documentation, n.d., accessed 2026-11; a canonical tracking-status vocabulary.
- PostNL Annual Report 2024 — PostNL, 2025; the self-reported 97% parcel delivery quality.
- How to automate WISMO requests — Gorgias, updated 2023-11-13; WISMO as 18% of requests in one helpdesk's data.
- Customer invoices and Credit notes — Odoo 18.0 documentation, accessed 2026-11; sequential numbering, immutability, credit notes as the only correction.

Standards, regulation and official documentation

- Directive 2011/83/EU on consumer rights — Official Journal, 2011-11-22; delivery within 30 days (Art. 18) and passing of risk (Art. 20).
- Invoice requirements — Belastingdienst, n.d., accessed 2026-11; mandatory invoice contents in the Netherlands.
- VAT rates and exemptions — business.gov.nl, n.d., accessed 2026-11; the 21% Dutch standard rate.
- One Stop Shop — European Commission, n.d., accessed 2026-11; the €10,000 cross-border threshold.
- IFRS 15 Revenue from Contracts with Customers — IFRS Foundation, issued 2014, effective 2018; revenue when control transfers.
- Study on the payment attitudes of consumers in the euro area (SPACE 2024) — European Central Bank, 2024-12; e-payment solutions at 76% of Dutch online payments.

Research and practitioner sources

- Cart abandonment rate statistics — Baymard Institute, updated 2026-10-28; the 70.22% average and the reasons list.
- Are inventory records inaccurate? Evidence from retail — DeHoratius and Raman, *Management Science* 54(4), 2008; the 65% record-inaccuracy finding (dated).
- Starbucks does not use two-phase commit — Gregor Hohpe, 2004; write-off, retry and compensation as the three error strategies behind every handoff in this chapter (dated, still the clearest analogy).

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

38. Journey 2: The Jacket Comes Back

IN THIS CHAPTER, YOU WILL: Follow one returned jacket from the request to the restock, the exchange or the refund, and the credit note, and see which system owns each step and where the money goes out twice.

KEY POINTS:

- **A return is three flows that must end in agreement, not one.** The goods come back through the carrier and the warehouse, the money through the payment provider, and the books record a credit note. Each flow has its own owner and clock, and the return is finished only when stock, money and books agree.
- **The refund is triggered by what the warehouse found, and it has one writer and one key.** The decision to refund belongs to the disposition event from inspection, not to the request or the carrier scan; exactly one system may call the payment provider, with the return id as the key that makes a repeated call harmless. The Double Refund of January 2027 is what two writers and no key cost.

- **An exchange is a return plus a new order, never an edit.** The exchange order carries the original frozen price, commits stock in the availability service and ships through the normal path; the books record a credit note and a new invoice that net to zero. Marketplace and store returns are the same flow with a different first step.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Boomerang** — refunds, restocking and credit notes reconciled by hand after the return arrives, and sometimes twice; this journey shows the one order of events that makes the three agree.
- **The Message in a Bottle** — the refund request, the disposition event and the refund confirmation each cross a join that can deliver twice or not at all, and the chapter names the key at each.
- **Schrödinger's Order** — a return whose state lives in the portal, the warehouse and the payment console has no state; the chapter shows where one return record replaces three.

Journey 1: Eva's Jacket, From Search to Doorstep followed Eva's jacket from the search box to the ledger. This chapter follows it back. **Most merchants design the way out with care and let the way back design itself**: a returns portal with a refund button, a warehouse that restocks what it receives, credit notes booked at month-end from a spreadsheet. Each piece works. Together they produce money refunded twice, stock restocked that cannot be sold, and a customer who asks three times where her refund is.

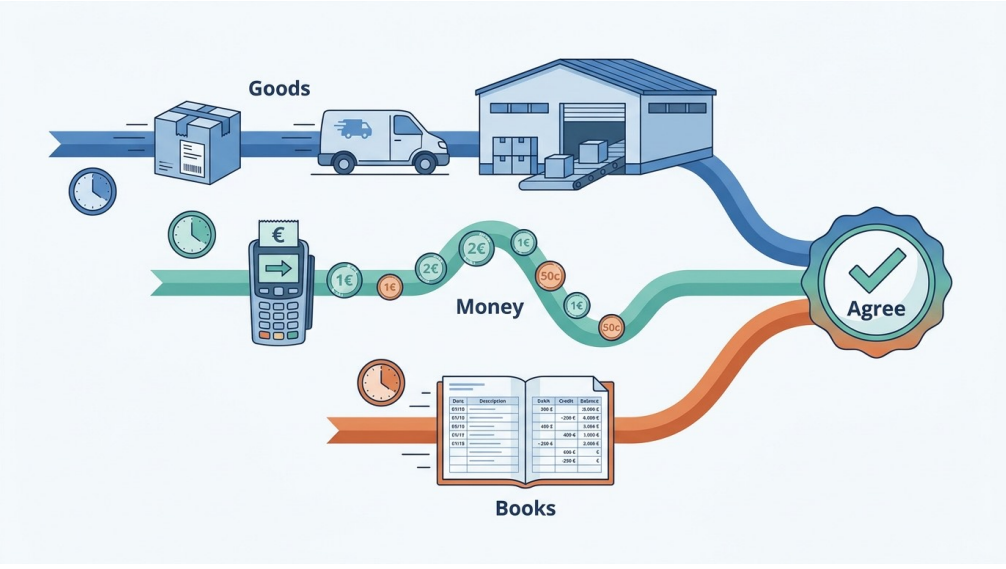


Figure 1: *A return is three flows, not one: the goods, the money and the books each have their own owner and clock, and the return is finished only when the three agree.*

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), shipped 336,000 parcels in 2026 and took back 72,000; its item return rate is 24% and its order return rate 19% on D2C (direct-to-consumer) orders; 28% of returns are exchanges and 61% of returned items are resold as new. Public figures sit in the same range: the US National Retail Federation (NRF) put the 2025 online return rate at 19.3%, with 9% of returns fraudulent (October 2025). **One parcel in four or five comes back.**

Where we are in the order's journey. Day six. Eva's jacket was delivered on Thursday 15 April 2027; on Monday 19 April she asks for a size L. This chapter covers the request, the way back to the warehouse, the exchange or the refund, and the credit note; The Way Back: Returns, Exchanges, Refunds and Reverse Logistics owns each step in depth, and this chapter puts them in sequence.

The Return, the Setting and the Rules of This Walk

The return. On **Monday 19 April 2027** Eva Janssen in Nijmegen opens the returns portal from her order status page and requests an **exchange of the Ridgewalker shell jacket, women's, size M, color moss (€199), for size L**, reason "too small". The two pairs of merino socks (€18 each) stay. The return is received and inspected at the Duiven warehouse on **Thursday 22 April**, graded resellable, and the exchange order ships the same day. If Eva had asked for her money back instead, the refund variant is **€199**, returned by iDEAL and booked as a credit note. Times within a day, ids and carrier scans below are illustrative.

The setting. As in Journey 1, the return runs through Kestrel & Pine's **target architecture of late 2027**: the availability service live (W1), the OMS acting as order hub with its own return state machine (W2), the PIM live (W3). In April 2027 the return in fact ran through the January landscape, where the returns portal, the platform and the WMS each held a piece of it; a section near the end shows how, and why that landscape produced the Double Refund.

The systems. Each is named by its role. The table is the **entity–owner table** for this one return: for each fact, the single system whose value is the truth when copies disagree, the **system of record** of Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record. The return record, the disposition and the refund are new since Journey 1, and each needs an owner.

Fact about the return	System of record (late 2027)	Who else holds a copy
The original order, its lines, frozen prices and payment reference	**the OMS** (order management system)	the platform, the returns portal
Whether this line may be returned, by when, at what cost	**the OMS**, from the delivery date, the policy and the product's attributes in the PIM	the returns portal
The return record: lines, reason, requested outcome, state	**the OMS**	the returns portal, the WMS, the ERP
What arrived and what it is worth: inspection and **disposition**	**the WMS** (warehouse management system)	the OMS, the data warehouse
Physical stock at Duiven, including returned units	**the WMS**	the availability service
What may be sold again, per channel	**the availability service** (a module of the OMS)	the platform, the marketplace connector
The refund: amount, method, state	**the PSP** (payment service provider), with its reference on the return record	the OMS, the platform, the ERP
The credit note and the new invoice	**the ERP**	the data warehouse

A fact with two writers is the problem, and in January 2027 the refund had two.

Twelve Steps in One Table

The whole way back at a glance: the step, the system, the data that moves, the promise made, the failure that belongs to the step, and the chapter that owns it.

#	Step	System	Data	Promise made	What could fail here	Chapter
1	Return request, Monday 19 April	the returns portal, reading the OMS	order #1072455, jacket line, reason, outcome "exchange for L"	"You may return this, free, until 29 April"	Eligibility from a stale order copy	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation

#	Step	System	Data	Promise made	What could fail here	Chapter
2	Held exchange order created	the OMS, asking the availability service	order #1072455-X1 for one L moss at €0 balance, held; one unit of L committed for NL	"Size L is yours when the M is back"	Size L promised with no stock; unit committed and never released	Is It in Stock? Inventory, Availability and the Promise You Can Keep, The Order Record: Lifecycle, State and the Order Management System
3	Return record created	the OMS	return R-20270419-1, requested → approved, linked to #1072455 and to the held order	"This return exists and will be tracked"	Created twice from a retried request; held only in the portal	The Order Record: Lifecycle, State and the Order Management System

#	Step	System	Data	Promise made	What could fail here	Chapter
4	Label issued and return announced	the returns portal via the shipping service; the OMS → the WMS	PostNL label with the return reference; reverse fulfillment order	"This label brings it to Duiven, and Duiven expects it"	Label without a reference; announcement lost	Getting It There: Carriers, Labels, Tracking and the Delivery Promise, Pick, Pack, Ship: Warehouse Management and Fulfillment
5	Drop-off and transit, 20–22 April	PostNL → the shipping service → the OMS	accepted, in transit, delivered-to-warehouse scans	"It is on its way" and, legally, "evidence of having sent back"	No scan: a return cannot be told from a lost parcel	Getting It There: Carriers, Labels, Tracking and the Delivery Promise
6	Receiving and inspection, Thursday 22 April	the WMS	return reference, GTIN, grade "resellable"	"This is the jacket, and it can be sold again"	Restocked without inspection; wrong item scanned	Pick, Pack, Ship: Warehouse Management and Fulfillment

#	Step	System	Data	Promise made	What could fail here	Chapter
7	Disposition and stock event	the WMS → the availability service	disposition restock; on-hand +1 for M moss	"One more M moss can be sold"	Stock raised for an unsellable unit; event delivered twice	Is It in Stock? Inventory, Availability and the Promise You Can Keep
8	Disposition event to the order	the WMS → the OMS	return R-20270419-1 received, line disposition	"The merchant may now settle"	Event lost: nothing released; duplicated: settled twice	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation
9a	Held exchange order released and shipped	the OMS, then the WMS and PostNL	#1072455-X1 released, L moss at the frozen €199, fulfillment order	"Size L ships today"	Priced at today's price; released by a scan rather than a disposition	The Order Record: Lifecycle, State and the Order Management System, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax

#	Step	System	Data	Promise made	What could fail here	Chapter
9b	Refund requested (the alternative)	the OMS → the platform's refund API → the PSP	one request keyed on R-20270419-1, €199, original method	"Your €199 is on its way"	Two writers; a retry without the key; a refund that fails days later	Taking the Money: Payment Providers, Authorization, Capture and Refunds
10	Notification	the OMS → e-mail	registered, received, exchange shipped, refund issued	"You know what is happening"	"Refund issued" before the PSP confirmed; silence at receipt	Where Is My Order? Notifications, Service and the Customer Record
11	Credit note and new invoice	the ERP	credit note –€199 against the invoice of #1072455; invoice +€199 for #1072455-X1	"The books show what happened"	Invoice edited instead of credited; refund without credit note	Tax, Invoices and the Books: From Order to Ledger

#	Step	System	Data	Promise made	What could fail here	Chapter
12	Reconciliation	the ERP, the PSP's reports, a daily job	PSP refunds matched to OMS refunds and to credit notes	"Money, stock and books agree"	The match runs monthly, and 212 double refunds wait three weeks	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation

Rows 7, 9 and 11 are stock, money and books; one event, row 8, releases all three.

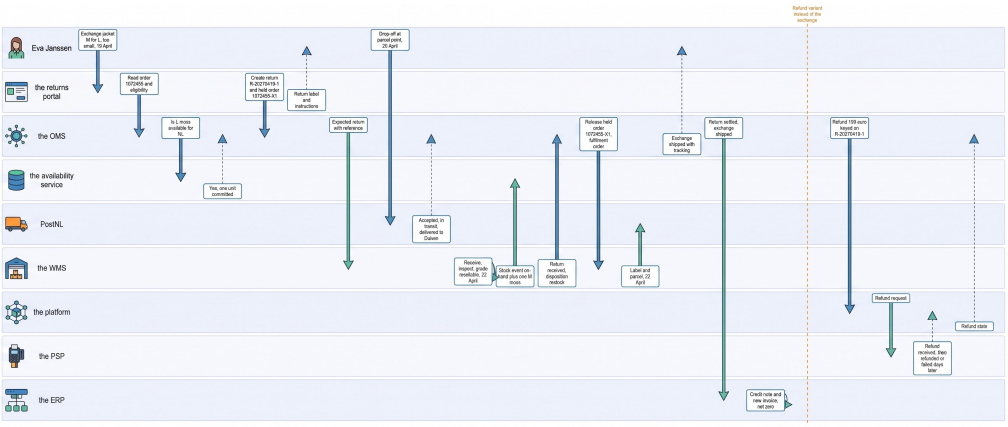


Figure 2: *The way back in the late-2027 architecture: the returns portal creates the return in the OMS, PostNL and the WMS bring the jacket to inspection, and one disposition event releases the stock, the exchange or the refund, and the credit note; the refund, when there is one, has a single writer and a single key.*

The Request: Eligibility, Reason and the Choice

A **returns portal** is the customer-facing page where a return is requested, a reason chosen, a label issued, and an exchange or a refund selected. Kestrel & Pine has rented one since 2025 (€650 a month plus €0.95 per return). In the target architecture the portal owns the conversation with Eva and nothing else: the facts it shows come from the OMS, and the return it creates lives there. The industry calls the merchant's record of

an approved return an **RMA** (return merchandise authorization); this book says "return record".

Eligibility is computed from three facts, each with an owner. The delivery date (15 April) is on the OMS's order, from PostNL's delivered scan in Journey 1. The policy is the merchant's: Kestrel & Pine offers free returns within 14 days of delivery in all three countries, with a prepaid label from the portal (illustrative). The product's attributes come from the PIM: EU law lets a merchant refuse withdrawal for goods made to the consumer's specification, perishable goods and sealed hygiene goods once unsealed, so "returnable" is a product attribute, not a portal setting (What Are We Selling? Products, Variants and the Catalog).

Under the policy sits the law. The EU Consumer Rights Directive (Directive 2011/83/EU, 2011) gives a consumer **14 days** from taking physical possession of the goods to withdraw from a distance contract without giving a reason (Article 9); Eva's window runs to 29 April. She must send the goods back within 14 days of telling the merchant (Article 14). The merchant must refund within 14 days of being told, but may withhold the refund until it has the goods back **or evidence that they were sent**, whichever comes first (Article 13). The consumer bears the cost of return only if the merchant said so before the sale (Article 14). Four dates must therefore be computable per order line: delivery, notice, evidence of sending, refund. Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation owns the law; this chapter needs the clocks. The reason, "too small", is data too: collected per SKU, it feeds size guidance.

The choice: exchange or refund. Eva chooses an exchange, as 28% of Kestrel & Pine's returning customers do. Portals are built to steer that way, because a refund gives back €199 and an exchange keeps it. An exchange needs stock. Kestrel & Pine models an exchange as a **held order**: at request time the OMS asks the availability service for one L moss for the NL channel, creates order **#1072455-X1** for it at a €0 balance, commits the unit and holds the order until the return is graded. The size is Eva's from the moment she asks; the parcel waits for the inspection.

When the held order is released is a policy with three settings. *Release on disposition* (Kestrel & Pine's: the L ships on 22 April, after the M is inspected) is cheapest and slowest. *Release on scan* ships the exchange when the carrier accepts the return parcel, a day or two earlier, at the risk of an empty box. *Instant exchange* ships at once and holds a payment authorization until the return arrives; Shopify's returns model calls the mechanics "release the exchange and collect a balance" (n.d., accessed 2026-11). The rule I use: release on disposition until inspection results are measured per SKU, then move the hero products to release on scan.

At 20:40 on 19 April the portal creates return **R-20270419-1** in the OMS, state *requested*, linked to #1072455, its jacket line and the held order, with the reason, the outcome and the four dates; the policy approves it automatically. The return state machine is the third on the OMS's order record, beside money and goods (The Order Record: Lifecycle, State and the Order Management System).

The Way Back: Label, Drop-off and the Scan That Ends a Right

The portal asks the shipping service, the multi-carrier layer in Kestrel & Pine's WMS flow, for a **return label**: a PostNL label addressed to Duiven, with the return reference printed as a barcode. The reference is the one thing the label must carry; a parcel that arrives without it is a jacket from nobody.

In the same minute the OMS tells the WMS to **expect** the return: a **reverse fulfillment order**, the mirror of Journey 1's fulfillment order, with the lines, the return reference and the tracking number. Shopify's model names the split this chapter teaches: a *return* "represents the intent of a buyer to ship one or more items from an order back", a *reverse fulfillment order* "represents the work that's required to process a return" (n.d., accessed 2026-11). The first is the OMS's, the second the warehouse's. The message may arrive twice; the return reference makes the second copy a no-op (W5).

Eva drops the parcel at a PostNL point in Nijmegen on **Tuesday 20 April** at 17:50 *(illustrative)*. The acceptance scan reaches the OMS within the hour, and the return moves to *in transit*. **That scan is a legal event**. It is "evidence of having sent back the goods" under Article 13(3): from here the merchant may no longer withhold the refund for lack of proof. The refund deadline, 14 days from Eva's notice on 19 April, does not move, so a week in a depot eats the merchant's margin. The OMS computes the deadline from the notice and the end of the withholding right from the scan.

PostNL delivers the parcel to Duiven on **Thursday 22 April**. A parcel on the dock is not yet a return.

At the Dock: Receiving, Inspection and Disposition

Returns receiving is a separate flow in the warehouse, with its own dock, its own bench and one rule: **nothing is restocked that has not been looked at**. A receiver scans the return reference, which finds the reverse fulfillment order, then the jacket's GTIN (Global Trade Item Number, the number behind the barcode on every own-brand SKU since 2021), which confirms the right item came back. Then an inspection: the jacket that was sent, unworn, tags on? Eva tried it on, which Article 14(2) allows without liability.

The result is a **disposition**, the warehouse's decision about the returned unit. Shopify's reverse-fulfillment model names four, and they are the industry's four under any name: *restocked*, *processing required* (clean, repair, re-label before a decision), *not restocked* (discard, donate, return to the supplier) and *missing* (n.d., accessed 2026-11). Kestrel & Pine resold 61% of returned items as new in 2026, 27% after processing at a reduced price, 9% were recycled or donated and 3% were missing or the wrong item (illustrative

split), and **none of the 39% that were not graded new may increase sellable stock at full price**. A WMS that increments on-hand at the dock scan, before the grade, is the mechanism behind "restocked but not resellable": the availability service promises a jacket that is in a repair bin.

At 10:15 on 22 April the inspector at the returns bench, which handles about 280 parcels a day with two people (illustrative), grades Eva's jacket resellable, and the WMS emits two events. A **stock event** goes to the availability service (on-hand +1 for M moss at Duiven), and the next shopper sees one more M within seconds. The **disposition event** goes to the OMS: return R-20270419-1, jacket line, received, restock. That second event is the hinge of the whole journey.

Figure 3: *Nothing is restocked that has not been looked at: the inspector's disposition sorts every returned unit, and only the 61% graded new goes back to sellable stock at full price.*

The Exchange Order

On the disposition event the OMS's return moves to *received* and releases the outcome Eva asked for. **An exchange is a return plus a new order**. The held order **#1072455-X1**, created on Monday, has one line, Ridgewalker women's L moss, at **€199**: the price frozen on the original order, not today's list price, the rule Kestrel & Pine adopted in April 2027 for invoices and exchanges (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax). It references the return, carries Eva's address and the original payment reference, and shows a balance of **€0**. Had Eva exchanged the jacket for a €119

Trailhead fleece, the balance would be –€80 and the refund path below would carry it; a dearer item would have been paid for through the platform's checkout before approval.

From the release the exchange is Journey 1 from step 10: the OMS creates a fulfillment order for the already committed unit; the jacket joins the 13:00 pick wave; the shipping service prints a PostNL label; the parcel leaves with the evening collection on 22 April. Eva gets the shipping confirmation that afternoon and the jacket on **Friday 23 April** *(illustrative).* The original order's goods state stays *delivered*, its return state reads *received, exchanged*, and the exchange order has its own goods state: three state machines, one customer-facing line, "Your exchange has shipped."

Why not edit the line from M to L? Because the M was picked, invoiced and paid for; an edit would leave an invoice for a jacket Eva no longer has.

The Refund Alternative: One Writer, One Key

Suppose instead that Eva asks for her money. The return record carries outcome *refund*; everything up to the disposition event is identical; what the event releases is a refund of €199.

The refund is triggered by the disposition, not by the request and not by the scan. Refund on request is the most generous policy and the most exposed to fraud; refund on scan meets the 14-day clock early; refund on inspection, Kestrel & Pine's policy, keeps money and goods in one causal chain: no inspection, no refund. The deadline from the notice (19 April) runs to 3 May; the inspection on 22 April leaves eleven days of margin, enough for a week in transit and a dock that inspects within a day. A merchant whose dock is three days behind in January must know that number, because the law does not.

One writer. On the disposition event the OMS raises exactly one refund request, through the platform's refund API, because the platform holds the PSP integration and the payment reference from the checkout, and the PSP is the only system that can move money. Support cannot refund, the returns portal cannot, the ERP cannot. In February 2027, after the Double Refund, Kestrel & Pine made the returns portal the single writer through the same platform API and made support's PSP console access read-only; when the OMS took over return records in Q4 2027 the writer moved with the record. What never changed is the count.

One key. The request carries the return id, R-20270419-1, as its **idempotency key**: a token sent with a request so that a retry does nothing twice. Stripe's API saves the first response to a key and returns it to any retry (n.d., accessed 2026-11); Mollie's documentation says the PSP "blocks duplicate refunds made in a short time frame" and asks that after a connection error the merchant "do not retry automatically" but check the payment's refunds through the API first (n.d., accessed 2026-11). The PSP's protection is a backstop; the merchant's key is the design.

Refunds are asynchronous, and they fail late. Adyen's documentation states that "the refund process is asynchronous", that "although rare, a refund can fail after you received" the success webhook, sometimes days later, and that for iDEAL a *refunded-reversed* event means the funds came back because the shopper's account was invalid (n.d., accessed 2026-11). Stripe's says the customer sees a refund "approximately 5-10 business days later" and that the original processing fees are not returned (n.d., accessed 2026-11). For iDEAL there was no capture step in Journey 1 and there is no hold to release now: the refund is a new transfer to Eva's bank account.

So the return's money state is a small machine of its own, *requested* → *accepted by the PSP* → *refunded* or *failed*, driven by the PSP's webhooks and confirmed by fetching the refund's status, as the payment was in Journey 1. "Refund issued" is Eva's message when the PSP accepts the request, with "it can take a few business days to show"; "refund failed" is a case for an agent, not a silent retry. Taking the Money: Payment Providers, Authorization, Capture and Refunds owns the mechanics.

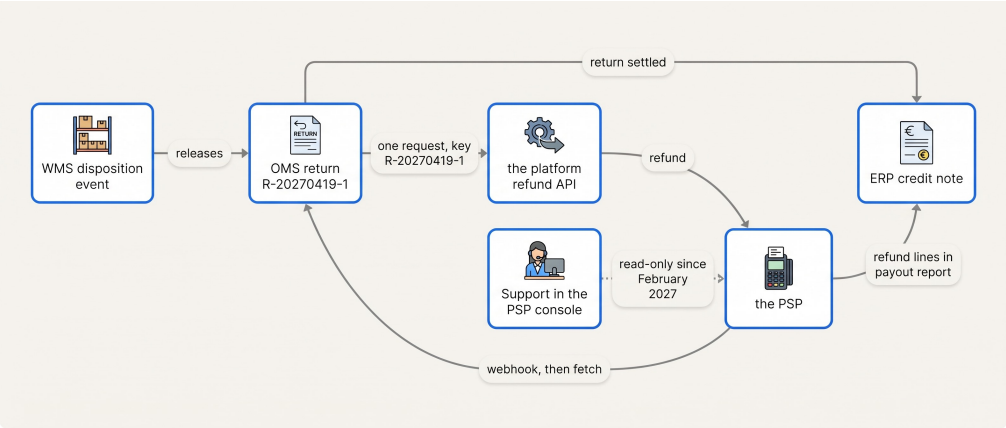


Figure 4: *One writer and one key: the disposition event releases a single refund request keyed on the return id, the PSP's answer comes back by webhook and is confirmed by a fetch, and the ERP matches the credit note to the PSP's payout lines; the dotted path is the second writer the Double Refund closed.*

The Books: Credit Note, New Invoice, Reconciliation

An invoice is never edited. Journey 1 left an invoice for #1072455 in the ERP: net €194.21, VAT €40.79, gross €235.00. As Journey 1 showed, a credit note is the only way to correct an issued invoice. On the "return settled" message the ERP issues a **credit note** against the original invoice for the jacket line: gross **–€199.00**, net –€164.46, VAT –€34.54, referencing the invoice number and the return id. For a Belgian business customer the credit note would travel through Peppol like the invoice (Documents

Between Companies: EDI, Punchout and E-Invoicing); for Eva it is a PDF and a ledger entry (Tax, Invoices and the Books: From Order to Ledger).

An exchange is two documents that net to zero. The exchange order produces a new invoice on dispatch, gross €199.00 with the same VAT split. Credit note minus €199, invoice plus €199, no refund: Eva's bank statement shows nothing, and the ledger shows the full history.

The refund variant pairs the credit note with a refund at the PSP; the credit note reverses the revenue recognized at delivery in Journey 1.

Reconciliation is daily, not monthly. Since February 2027 a daily job matches the PSP's refunds to the merchant's on order number and amount and sends exceptions to the finance team; the monthly payout reconciliation of Journey 1: Eva's Jacket, From Search to Doorstep remains the net under everything. In the target architecture the match has three legs: every refund at the PSP has a return in the OMS with state *refunded*, every *refunded* return has a credit note, every credit note traces to an invoice. A return that fails a leg is an exception by name, the same day. The sibling book RECONCILED works the ledger side (Payments-to-Ledger: Draining the Suspense Swamp, Reconciled by Design: Traceability and Audit Across Boundaries).

Two Variants: The Marketplace Return and the Store Return

The same jacket can come back through two other doors. The first step changes; the rest does not.

The marketplace return. On 13 April an Amazon.de customer bought a Ridgewalker that Kestrel & Pine shipped itself from Duiven, and another bought one the marketplace's fulfillment program shipped from stock sent there in advance. On a marketplace, **the return policy is the marketplace's, not the seller's.** Amazon's 2025 guidance for seller-fulfilled orders, written for the US, says most items can be returned within 30 days, that sellers should respond within 24 hours where their authorization is needed, and that they should refund "within two days of receiving returns, or Amazon may issue an automatic refund" (15 April 2025; European terms sit behind Seller Central and are not verified here). For the marketplace-fulfilled jacket the marketplace receives, grades and refunds, and charges the seller: Zalando lists "return shipping" and "return processing" among its fulfillment fees, Amazon a "returns processing" fee for FBA (partner and seller pages, 2026, subject to change).

The consequence for the OMS: **it must accept returns it did not create and refunds it did not issue.** The marketplace connector creates the return in the OMS from the marketplace's authorized-return event, keyed on the marketplace's return id, and marks the money state *refunded externally* when the marketplace reports its refund; the 24-hour and two-day clocks are service levels on that integration. The seller-fulfilled jacket

then follows steps 4 to 8 unchanged. The marketplace-fulfilled jacket never comes to Duiven; its disposition arrives as a report line that adjusts the external location's stock, and the money as a deduction in the marketplace payout (Selling on Someone Else's Platform: Marketplaces and Channel Integration).

The store return. Eva lives in Nijmegen, which has had a Kestrel & Pine store since 2023. In January 2027 a store could only take the jacket and phone head office, because the POS had no link to the platform. In the target architecture the store's tooling looks the order up in the OMS, the POS records the return against #1072455, and three things differ. The **inspection** happens at the counter, with the same four dispositions. The **stock** lands in the Nijmegen store location, whose system of record is the POS (W1), so the availability service learns of one more M moss in Nijmegen, not Duiven. And the **refund** still goes to Eva's original iDEAL payment through the online PSP, raised by the OMS exactly as above, never as cash from the till, because the directive's "same means of payment" rule and the reconciliation both depend on it. From 2028 store receipts are imported into the OMS with their numbers (Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment); matching Eva's store and online records is One Customer, One Product: Identity and Master Data Across Systems's problem.

What Each Handoff Requires

How fresh, how consistent, how fast and how available each join must be; the figures are the author's illustrative targets, not ledger facts.

Handoff	Freshness	Consistency	Latency	Availability	Style (W5)
Returns portal → OMS (create return)	Immediate	One return per request; return id as key	Under 2 s *(assumption)*	Portal fails closed: no label without a return record	Synchronous call, idempotent
OMS → WMS (expected return)	Minutes	Request and acknowledgement; return reference as key	Before the parcel arrives	Queue absorbs a WMS outage	Events with acknowledgement

Handoff	Freshness	Consistency	Latency	Availability	Style (W5)
WMS → availability service (stock event)	Seconds	Single writer per quantity; restock only on disposition	Under 10 s *(assumption)*	Weekly reconciliation as the net	Events
WMS → OMS (disposition)	Seconds	One settlement per return line	Under 30 s *(assumption)*	Daily job lists returns received but not settled	Events, idempotent
OMS → platform → PSP (refund)	Immediate	One writer, one key; state fetched, never assumed	Under 5 s *(assumption)*	Queued if the PSP is down; never retried blindly	Synchronous call plus webhook
OMS → ERP (credit note, invoice)	Same day	One credit note per settled return line	Daily batch is acceptable	Reconciliation count per day	Daily batch

The stock event and the disposition event cannot be batch, because a returned unit and Eva's exchange both wait on them; the credit note can be. **Operations by event**, **finance by batch**, as in Journey 1.

The Same Return in January 2027

Run R-20270419-1 through the January 2027 landscape, and the dates stay the same while four things change.

Step	Late 2027	January 2027
The return record	One, in the OMS, with its own state	Three: the portal's return, the platform's order note, the WMS's receipt, each with its own status
The exchange	A held order in the OMS, committing a unit in the availability service, released by the disposition	A held order in the platform, committing a unit in the platform's stock copy, which could be two hours stale
The refund	One writer (the OMS), one key (the return id), released by the disposition	Two writers, the portal through the platform's API and support in the PSP console, and no key naming the return
Stock from returns	WMS stock event to the availability service in seconds	WMS restock → ERP every 15 minutes → platform hourly; about 90 of the in-flight units in the May 2027 reconciliation were returns on their way through this chain

The jacket still came back on 22 April and the L still shipped the same day. What the January landscape could not do was tell one agent, in one screen, that the refund had already been sent.

When It Goes Wrong Along This Path

The step table names a failure per step. Four deserve the mechanism, the detection and the recovery.

The Double Refund. Over three weeks in January 2027, 212 returns were refunded twice, €19,400 in total, most of it recovered. The mechanism (illustrative): customers who saw PostNL's "delivered to Duiven" scan asked support for their refund; the agent could see the payment in the PSP console but no refund, because the portal refunds on inspection, two to five days after receipt in a January backlog, and refunded €199 by hand; the portal then refunded again on inspection, and the PSP accepted both because neither request named the return. Two writers, no key, no single view. Sanne de Wit found it at month-end from two credits per order in the ERP. Recovery: asking customers to transfer the second refund back, since a bank transfer cannot be recalled. The fix from

February 2027 is the one writer and one key described above, plus a daily match of PSP refunds to the platform's. The Boomerang and The Message in a Bottle in one incident.

The refund that failed days later. The PSP accepted the request, Eva received "refund issued", and three days later a *refund failed* or *refunded-reversed* webhook arrived because the bank account could not take the transfer. Detection: the webhook, if the receiver handles it; a daily job that lists refunds not in a terminal state after the method's typical window, if it does not. Recovery: a service case and a new refund to a confirmed account, never a blind retry. The return's money state must be able to go backwards.

The restock that was not resellable. On-hand went up at the dock scan; the grade "processing required" came an hour later and did not reverse it; the availability service promised a jacket in a repair bin, and a shopper's order was short-picked. Detection: restocked units reconciled against dispositions, and the short-pick rate on returned SKUs. Recovery: the stock event is emitted on the disposition, never on the scan, and the count is corrected from the bin.

The wrong refund amount. During Black Friday 2026 an automatic 20% outerwear discount stacked with a 10% welcome code, and 190 returned jackets were refunded at €159.20 instead of the €143.28 actually paid, about €3,000, because the order-level discount had never been allocated to lines in the order export. Detection: a refund that exceeds the line's paid amount, which the PSP's ceiling on partial refunds catches only at order level. Recovery: the export to the returns portal, the WMS and the ERP now carries the allocated net price per line, a W5 contract field since 2027, and the refund is computed from it (Discounts, Coupons and Loyalty: Promotions Without Chaos).

What to Take Away

Established.

- A return is a stock flow, a money flow and a book-keeping flow with three owners, and the public models agree on the split: Shopify separates the return (intent and money) from the reverse fulfillment order (work and stock) and names four dispositions; commercetools keeps goods-side and money-side return states apart; every ERP corrects an invoice only by credit note.
- Refunds are asynchronous, can fail days after acceptance and reach the customer days to weeks later; Adyen, Stripe and Mollie document this from three sides and ask the merchant to use idempotency keys, fetch status rather than assume it, and never retry blindly.
- EU consumer law fixes the clocks: 14 days to withdraw from possession, 14 days to refund from notice, with the right to withhold until the goods or proof of sending arrive.

Depends on context.

- Whether the refund waits for inspection. Refund on inspection is right when the dock inspects within a day or two and the return rate makes fraud and unsellable returns matter; refund on scan is right for slow transit lanes and a tight 14-day margin; refund on request is a marketing choice that needs a fraud model behind it.
- Whether the OMS owns the return record. With returns arriving from a portal, three marketplaces and six stores, one return record is necessary; a Starting or Growing merchant with one channel gets the same journey from the platform's own return model, and the portal may be the record.
- Whether stores take online returns at all. The flow needs the POS to look up the order and the refund to go to the original payment method; a merchant whose POS can do neither should route the refund through the online path by hand, and know that it is doing so.

The author's synthesis.

- I count the systems that can call the PSP's refund API, and if the answer is not one, I stop the review there; nothing else about the return path matters until it is.
- The disposition event is the most important message in the return path and the least often designed; I would make it the only trigger for stock, settlement and the credit note, and measure the time from dock scan to disposition every day, because that number is both the customer's wait and the merchant's legal margin.

Questions to Ask

1. Which systems can issue a refund, and what is the idempotency key on the request?
2. What triggers a refund: the request, the carrier's acceptance scan or the warehouse's disposition, and who chose that?
3. Does a returned unit increase sellable stock at the dock scan or at the disposition?
4. How are the four legal dates (delivery, notice, evidence of sending, refund) computed per order line, and what is the median time from drop-off scan to refund?
5. Is an exchange a new order at the original frozen price, with its own invoice, or an edit?
6. Can the OMS accept a return created by a marketplace, a store or the warehouse, and a refund issued by someone else?
7. What matched last month's PSP refunds to credit notes, and how many exceptions did it find?

Eva has her size L, the M is back on the shelf, and the books show a credit note and an invoice that net to zero. The next journey leaves the consumer behind: on 6 April, a

week before Eva's order, a retail chain's computer sent Kestrel & Pine an order for 240 units, and nobody typed anything. Journey 3: A Chain Reorders by EDI, From Order to Invoice follows it from the EDI message to the invoice.

Sources and Evidence

Company, product and investor sources

- Returns apps and ReverseFulfillmentOrderDispositionType — Shopify developer documentation, n.d., accessed 2026-11; return versus reverse fulfillment order, return statuses, the four dispositions, exchanges with a balance.
- Orders — commercetools API documentation, n.d., accessed 2026-11; goods-side and money-side return item states.
- Refund — Adyen documentation, n.d., accessed 2026-11; asynchronous refunds, late failures, refunded-reversed for iDEAL, up to 40 business days.
- Refunds and Idempotent requests — Stripe documentation, n.d., accessed 2026-11; fees not returned, 5–10 business days, double-refund risk on bank debits, idempotency keys.
- Refunds — Mollie documentation, n.d., accessed 2026-11; duplicate blocking and "do not retry automatically".
- Credit notes — Odoo 18.0 documentation, accessed 2026-11; invoice immutability and the credit note as the only correction.
- How to manage Amazon customer returns for seller-fulfilled orders — Amazon, 2025-04-15; the 30-day window, 24-hour response, two-day refund and automatic refunds (US guidance).
- Fulfillment by Amazon — Amazon, n.d., accessed 2026-11; returns processing as an FBA fee category.

Standards, regulation and official documentation

- Directive 2011/83/EU on consumer rights — Official Journal, 2011-11-22; Articles 9, 13, 14 and 16: the 14-day clocks, withholding until goods or proof, return costs, diminished value, exemptions.
- IFRS 15 Revenue from Contracts with Customers — IFRS Foundation, issued 2014, effective 2018; revenue reversal and the refund liability.

Research and practitioner sources

- 2025 Retail Returns Landscape — NRF with Happy Returns, 2025-10-15; 19.3% online return rate, 9% of returns fraudulent.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

39. Journey 3: A Chain Reorders by EDI, From Order to Invoice

IN THIS CHAPTER, YOU WILL: Follow Bergzicht's 240-unit spring reorder from the EDI order message to the paid invoice, and see why the despatch advice, not the order, is where wholesale money is lost.

KEY POINTS:

- **A wholesale order is a conversation of documents with a contract behind it.** An order message comes in, a despatch advice goes out with every pallet, an invoice follows, and a bank transfer arrives thirty days later. Each document is a promise written into the buyer's purchasing conditions, usually with a penalty attached.
- **Each document has its own system of truth.** The order record belongs to the OMS, the despatch advice to the WMS (what was actually packed on which pallet), the invoice to the ERP. Kestrel & Pine lost €14,200 in one quarter because its despatch advices were generated by the wrong system.

- **The money arrives as a bank transfer that must be matched, not as a payment-provider event.** Payment terms turn a sale into a receivable; credit limits, cash application and reminders are ERP work. E-invoicing mandates such as Belgium's change the invoice's format and transport, not its owner.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Pulled Rug** — the buyer's message guideline is a contract that changes on the buyer's calendar, and a supplier that does not track it finds out from a deduction.
- **The Lynchpin** — Kestrel & Pine's EDI mappings were understood by one external consultant; the chapter shows what that costs and what replaces it.
- **The Message in a Bottle** — an order message or a despatch advice sent once and trusted is the mechanism behind duplicate orders and refused deliveries.

A consumer order is made by a person at a screen. A wholesale order is made by a system at another company, in a format its purchasing department chose, under conditions its lawyers wrote. **The buyer, not the seller, decides how the conversation works**, and for a large retail chain that conversation is EDI: electronic data interchange, the long-established practice of exchanging standard business documents (orders, despatch advices, invoices) directly between two companies' systems without e-mail, PDFs or retyping.

Figure 1: *A wholesale order is a conversation of documents with a contract behind it: the order comes in, a despatch advice goes out with the pallets, an invoice follows and the money arrives weeks later.*

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), earns a quarter of its revenue from 1,100 wholesale accounts, and its largest account orders this way. The documents are few and old, the volumes small next to the web shop, the money per order fourteen times larger. **Wholesale is where a single wrong document costs more than a day of web orders**, because the penalty is written into the contract.

This chapter is the companion of Journey 1: Eva's Jacket, From Search to Doorstep: the same jacket from the same shelf, a week earlier, bought by a company instead of a person. What the two journeys share is said once there; what differs is here.

Where we are in the order's journey. Off Eva's path, but on the same shelf. Bergzicht's 240 units are committed from Duiven on 6 April 2027; the number of moss jackets the Dutch storefront may promise Eva on 13 April is smaller because of it. This is the order that competes with hers for stock.

The Order, the Setting and the Rules of This Walk

The buyer. Bergzicht Outdoor (fictional) is a Dutch outdoor retail chain with **38 shops** and a distribution center in **Zwolle**, Kestrel & Pine's largest wholesale customer at **€2.1 million** a year, ordering by EDI since 2025. Its buyer is Pieter Smit; on Kestrel & Pine's side the relationship belongs to Lea Moreau, Head of Wholesale.

The order. On **Tuesday 6 April 2027** an EDI order message (ORDERS) arrives for **240 units across 18 SKUs**, Ridgewalker jackets and Trailhead fleeces, worth **€19,860 excluding VAT** at Bergzicht's contract prices. The goods are picked to **three pallets** with SSCC labels; the despatch advice (DESADV) is sent at pack confirmation; the pallets reach the Zwolle DC on **Friday 9 April**; the invoice (INVOIC) is sent the same day; payment is due **9 May**, on 30-day terms. Times within a day and the credit figures are illustrative; the ledger fixes the dates and amounts.

The setting. As in Journey 1: Eva's Jacket, From Search to Doorstep, the order is run through Kestrel & Pine's **target architecture of late 2027**: the availability service live (W1), the OMS as order hub (W2; EDI and manual wholesale orders route through it from 8 December 2027), the PIM live (W3) and, specific to this chapter, **W6 in force**: SSCC labels generated by the WMS at pack time, the DESADV sent from the WMS's confirmation, and one EDI partner card per trading partner. In April 2027 the order ran through the January landscape; a section near the end shows the difference. The ASN Chargeback of Q3 2026 is walked as it happened, on that older landscape.

The documents and the identifiers

EDI has a vocabulary of its own, and newcomers meet it as a wall of capital letters. The table gives the six messages this journey uses, in the UN/EDIFACT standard most European retail uses (as a GS1 subset called EANCOM) and in the ANSI X12 standard of North American retail, with the system each is generated from. GS1's implementation survey (2021) found ORDERS, INVOIC and DESADV the three most implemented EANCOM messages (Documents Between Companies: EDI, Punchout and E-Invoicing owns this in depth).

Message	X12 twin	In plain words	Sent by	Generated from
ORDERS	850	The purchase order: what, how many, at which agreed price, delivered where and when	Bergzicht	Bergzicht's purchasing system
CONTRL	997	A syntax receipt: "your message parsed", nothing about its meaning	Either side	The EDI provider's translator
ORDRSP	855	The order response: accepted, accepted with changes, or rejected	Kestrel & Pine	The OMS

Message	X12 twin	In plain words	Sent by	Generated from
DESADV	856	The despatch advice, also called the ASN (advance shipping notice): what is on its way, pallet by pallet, so the receiver can scan instead of count	Kestrel & Pine	**The WMS's pack confirmation**
INVOIC	810	The invoice: a legal payment claim for what was delivered, at the agreed prices, with VAT	Kestrel & Pine	**The ERP**
RECADV	861	The receiving advice: what the buyer counted in; optional, and Bergzicht does not send one	Bergzicht	Bergzicht's warehouse system

Three GS1 identifiers tie the documents together, all defined in the GS1 General Specifications (Release 26.0, January 2026): the **GTIN** (Global Trade Item Number) says *what* a unit is, the number behind the barcode on every own-brand SKU; the **GLN** (Global Location Number) says *who and where*, the key for Bergzicht's Zwolle DC and Kestrel & Pine's Duiven warehouse in every address segment; the **SSCC** (Serial Shipping Container Code) says *this pallet*, an 18-digit serial number the specification calls "the only GS1 key that SHALL be used as the identifier of a logistic unit", one that "is used with EDI communications to enable identification and traceability". A despatch advice is the document in which each SSCC lists the GTINs and quantities it carries. **A warehouse system that cannot print an SSCC per pallet and report that pallet's contents cannot produce a compliant despatch advice**, and that sentence is the whole of the ASN Chargeback.

Fifteen Steps in One Table

#	Step	System	Data	Promise made	What could fail here	Chapter
1	ORDERS received, Tuesday 6 April 09:12	the EDI provider (a value-added network: a provider-run mailbox between trading partners)	PO number, GLNs, 18 lines of GTIN, quantity, price, delivery date	"Bergzicht has ordered this"	Delivered twice; sent to a retired mailbox	Documents Between Companies: EDI, Punchout and E-Invoicing
2	CONTRL returned	the EDI provider	syntax acknowledgement	"We received a well-formed message"	Negative acknowledgement nobody reads	Documents Between Companies: EDI, Punchout and E-Invoicing

#	Step	System	Data	Promise made	What could fail here	Chapter
3	Mapping to the order record	the iPaaS, per the Bergzicht partner card	PO fields → order fields; GTIN → SKU; GLN → address	"The message means this order"	Guideline changed (The Pulled Rug); one person can fix it (The Lynchpin)	Documents Between Companies: EDI, Punchout and E-Invoicing, Promises Between Systems: API Contracts, Webhooks and Versioning
4	Validation: items, contract prices, credit	the OMS, reading the ERP's price list and credit limit	18 SKUs resolved; prices equal the contract; receivables within limit	"We accept this at these prices"	Price differs from the contract; limit exceeded	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax

#	Step	System	Data	Promise made	What could fail here	Chapter
5	Order created, channel EDI	the OMS	order in the shared number family, PO reference, NET 30	"The order exists, once"	Created twice from a resent ORDERS; created in the ERP or the EDI portal instead	The Order Record: Lifecycle, State and the Order Management System
6	ORDRSP sent	the OMS → the EDI provider	accepted lines, confirmed dates	"You will get this, then"	Response promises what stock cannot keep	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals
7	Allocation of 240 units	the availability service	committed quantities per SKU at Duiven	"These units are Bergzicht's"	Commit invisible to the storefront (The Phantom Stock)	Is It in Stock? Inventory, Availability and the Promise You Can Keep
8	Fulfillment order, pallet service level	the OMS → the WMS via the iPaaS	lines, ship-to GLN, packing rules, delivery date	"Duiven ships this by Thursday"	Request lost; parcel rules applied to a pallet order	Pick, Pack, Ship: Warehouse Management and Fulfillment

#	Step	System	Data	Promise made	What could fail here	Chapter
9	Pick to pallet, Wednesday and Thursday	the WMS	cartons picked and built onto three pallets	"These units are on these pallets"	Short pick; mixed pallets not recorded	Pick, Pack, Ship: Warehouse Management and Fulfillment
10	SSCC labels printed at pack	the WMS	one SSCC per pallet, contents recorded	"This barcode is this pallet"	Pallet numbers taken from the order system, not the pack (the ASN Chargeback)	Documents Between Companies: EDI, Punchout and E-Invoicing
11	DESADV at pack confirmation, Thursday 8 April	the WMS → the iPaaS → the EDI provider	pallet SSCCs, cartons, GTINs and quantities, PO reference	"This is exactly what arrives"	Sent after the truck; rejected on syntax; edited after sending	Documents Between Companies: EDI, Punchout and E-Invoicing
12	Transport and delivery, Friday 9 April	the shipping service, a pallet carrier	booking, delivery slot at Zwolle	"It arrives Friday"	Slot missed; delivery refused without a DESADV	Getting It There: Carriers, Labels, Tracking and the Delivery Promise

#	Step	System	Data	Promise made	What could fail here	Chapter
13	Receipt at Zwolle	Bergzicht's warehouse system	SSCC scans matched to the DESADV	"What we scanned is what you said"	Scan does not match: manual count, deduction	Documents Between Companies: EDI, Punchout and E-Invoicing
14	INVOIC, Friday 9 April	the ERP → the EDI provider	invoice number, lines as shipped, €19,860 net, €4,170.60 VAT, due 9 May	"You owe this, by then"	Invoiced from the order, not the shipment	Tax, Invoices and the Books: From Order to Ledger
15	Payment and cash application, by 9 May	Bergzicht's bank → the ERP	a SEPA transfer with remittance information	"The receivable is settled"	Short payment; remittance without invoice numbers	Tax, Invoices and the Books: From Order to Ledger, The Back Office: ERP Integration and Who Is the Hub

Rows 10 and 11 are the ones the chapter is about: the two steps where a document is written about physical goods by a system that may or may not have touched them.

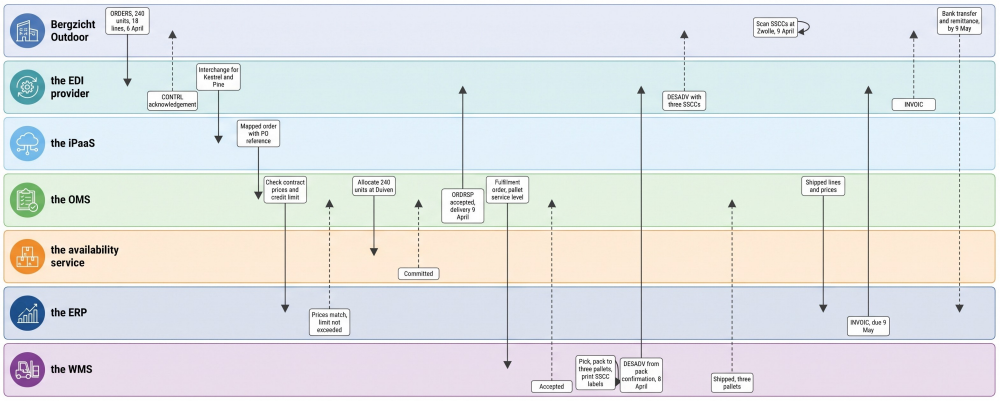


Figure 2: *Bergzicht's reorder in the late-2027 architecture: the order message becomes an OMS order, the availability service commits the stock, the WMS packs and writes the despatch advice from what it packed, the ERP invoices what shipped, and the money comes back thirty days later as a bank transfer to be matched.*

ORDERS In: From Message to Order Record

At 09:12 on Tuesday an interchange lands in Kestrel & Pine's mailbox at the EDI provider. A **VAN** (value-added network) is a provider-run mailbox: each trading partner connects once and the provider relays to everyone. The alternative is a direct point-to-point link, most often **AS2**, an internet protocol for signed and encrypted business documents defined in 2005 (RFC 4130); a direct link means a certificate and an endpoint per partner, which is why mid-sized suppliers buy the network. The provider's translator checks the syntax and returns a **CONTRL** acknowledgement within minutes. It says "parsed", not "agreed"; a supplier that treats CONTRL as acceptance has made its first mistake.

Mapping is the translation between Bergzicht's message implementation guideline, the document that says which of EDIFACT's many segments Bergzicht uses and how, and Kestrel & Pine's order record. Every retailer has its own guideline, so the cost of EDI is per partner, not per standard, which is the business model of every EDI provider: SPS Commerce's annual report for 2025 (filed 19 February 2026) sells a network that lets suppliers "connect once and immediately transact with thousands of trading partners". In the target architecture the mapping runs in the iPaaS and is described on the **EDI partner card** that W6 requires: GLNs, guideline version, messages in each direction, test status, contacts on both sides, and the owner at Kestrel & Pine, Ruben Alves. In 2026 all of that lived in one external consultant's head.

Validation turns a message into an acceptable order. Three checks run in the OMS against data the ERP owns:

- **Items.** Each of the 18 GTINs resolves to a SKU in the item master; a retired or

mistyped GTIN rejects the line in the order response rather than creating a line for nothing.

- **Prices**. An ORDERS message carries the price the buyer expects. Bergzicht's contract prices differ from the list wholesale prices (€95 for the Ridgewalker, €57 for the Trailhead) and live in the ERP's price list for this customer, because the ERP owns B2B terms: Microsoft's documentation for its Business Central-to-Shopify connector (October 2026) states that "Most businesses manage customer records, payment terms, and credit limits in Business Central and export them" and warns that mixing directions "can lead to conflicting records". A price mismatch is answered in the ORDRSP, never silently accepted, because an unexpected invoice price is a short payment four weeks later (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax).
- **Credit**. The ERP holds Bergzicht's credit limit and open receivables. Assume a limit of €450,000 and €188,000 open on 6 April *(assumption)*; the order passes. A credit hold is a visible state of the order, not a silent queue.

The order is created in the OMS, with channel EDI, Bergzicht's PO number as a reference, NET 30 terms, and a number from the same family as every web and marketplace order (W2). It is not created in the ERP, and not in the EDI provider's web portal, where EDI orders commonly end up and "no single order record" quietly begins. The ERP receives the order for posting; the OMS owns it. Shopify's Admin API (version 2026-10) shows how a platform types such terms, defining `NET` as "due a set number of days after the invoice is issued".

Idempotency protects this step: an idempotent operation has the same effect whether performed once or twice. The VAN may redeliver an interchange, and Bergzicht's system may resend an ORDERS after a timeout; the OMS keys creation on Bergzicht's GLN plus its PO number, so the second arrival finds the order and does nothing. Stripe's API documents the same discipline for payments (n.d., accessed 2026-11). Without the key, Kestrel & Pine ships 480 units and invoices twice: the Message in a Bottle at pallet scale.

The **ORDRSP** goes back on Tuesday afternoon: all 18 lines accepted, delivery 9 April. It is the first promise Kestrel & Pine makes in this journey, and the availability service is what lets the OMS make it honestly.

Allocation: Whose Stock Is It?

A wholesale order takes stock differently from a web order. Eva's jacket is one unit committed at order creation and picked the next morning. Bergzicht's 240 units are **allocated** three days before they ship, while the storefronts, the marketplaces and the B2B portal keep selling from the same Duiven shelves.

In the target architecture there is one answer: the **availability service** computes available-to-promise per channel (what a channel may still promise: on-hand stock,

minus units committed to orders, minus a buffer) from the WMS's stock events (W1). The OMS commits Bergzicht's 240 units on Tuesday morning and every channel's "in stock" drops within seconds. Assume the order includes 12 women's moss Ridgewalkers in size M, on-hand at Duiven was 36 and the storefront buffer is one unit *(assumption)*: the NL storefront's available number for that SKU falls to 23 on 6 April, the number Eva sees on 13 April in Journey 1: Eva's Jacket, From Search to Doorstep. In January 2027 the wholesale order sat in the ERP and the platform learned of it an hour later at best; one such hour during a season launch is an oversell with a chain's pallet behind it.

Two B2B rules follow. **Large accounts get allocations, not promises**: some merchants reserve a share of a season's stock for key accounts, so a web surge cannot consume a chain's reorder (Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals). And **shortfalls are negotiated, not canceled**: a missing line is answered in the ORDRSP with a changed quantity or a later date, because a chain plans its shelves on the response.

Figure 3: *One shelf, two buyers: Bergzicht's 12 moss jackets and Eva's storefront draw on the same 36 at Duiven, so the storefront may promise 23 once the allocation and the buffer are taken.*

Pick to Pallet, Label and Despatch Advice: The Document That Costs Money

On Tuesday afternoon the OMS sends the WMS a **fulfillment order**, the instruction to ship these lines to this address by this date, with the wholesale service level: cartons

of fixed quantities, built onto pallets, delivered to a DC with a booked slot. Warehouse systems run parcel and pallet work side by side (Manhattan describes its warehouse product as handling "wholesale, retail, and direct orders at the same time", n.d., accessed 2026-11); at Duiven the wholesale wave runs in the afternoon after the morning's parcel waves *(a chapter-local detail)*.

Packing is the moment of truth. Over Wednesday and Thursday the pickers build three pallets. As each pallet is closed, the WMS prints its **SSCC label** and records which cartons, and therefore which GTINs in which quantities, are on it. This is W6's first rule: the SSCC is generated by the WMS at pack time, from the pallet as built, not from the order, which says only what should ship. If a carton of fleeces ended up on pallet two, the label and the record say so.

The DESADV is written from that record. At pack confirmation on Thursday 8 April *(assumption; the ledger says "at pack confirmation")*, the WMS produces the despatch advice: the shipment, then each pallet by SSCC, then the cartons and the GTIN quantities inside, with ship-from and ship-to GLNs and Bergzicht's PO number. The iPaaS hands it to the EDI provider, which sends it to Bergzicht before the pallets leave Duiven. This is W6's second rule: the DESADV is sent from the WMS's confirmation, not from the ERP.

Why the buyer cares is best read in a buyer's own words. Jumbo, a Dutch supermarket chain, publishes its receiving rules: "Every delivery to our mechanized DC must be preceded by a DESADV message", "A delivery without an accompanying DESADV message cannot be processed and is therefore refused!", and "The SSCC label forms the link with the DESADV despatch message" (guidelines version October 2025, translated from Dutch). Its purchasing conditions of 9 April 2026 add the price: a supplier that fails to use EDI with GS1-compliant content "forfeits €70 … per message", Jumbo may refuse a delivery without DESADV and SSCC, and €250 applies per order line delivered less than 90% on time and in full. bol has required SSCC labels on every logistic unit since 31 December 2021 (n.d., accessed 2026-11); Walmart's on-time-in-full program carried a 3% non-compliance fee at its 2017 introduction (trade press, dated). Bergzicht's conditions are fictional; their shape is this.

The pallets travel on Thursday night with a pallet carrier booked through the shipping service and reach Zwolle on **Friday 9 April**. Receiving scans each SSCC; the scan finds the pallet in the DESADV and books its contents in without counting. Bergzicht does not send a RECADV, so Kestrel & Pine's only evidence of what was received is what was not deducted a month later; the partner card records that a receiving advice has not been agreed.

Invoice, Terms and the Money Thirty Days Later

The invoice is the ERP's, and it is written from what shipped. On Friday the OMS passes the shipped lines and their contract prices to the ERP, and the ERP issues the

INVOIC: a gap-free sequential number, the dates of issue and supply, both parties with VAT numbers, 18 lines at contract prices, **€19,860** net, **€4,170.60** VAT at the Dutch 21% rate, **€24,030.60** gross, due **9 May**. The Dutch tax authority's invoice requirements list exactly these elements (Belastingdienst, n.d., accessed 2026-11), and an issued invoice is immutable: a credit note "is the only legal method for canceling, refunding, or modifying a validated invoice", as Odoo's documentation puts it for any ERP (18.0, accessed 2026-11). Had pallet two shipped a carton short, the invoice would show the shipped quantity, because an invoice that exceeds a receipt is a short payment by design (Tax, Invoices and the Books: From Order to Ledger).

On Bergzicht's side the invoice meets the four-way match: purchase order, despatch advice, receipt scan, invoice. A line that agrees on all four is paid; one that does not is paid short, and the difference arrives as a deduction with a code, not as a conversation.

Revenue and the receivable. The ERP posts accounts receivable of €24,030.60 and revenue of €19,860; under IFRS 15 (effective from 2018) revenue is recognized when control passes, which for a delivered pallet is 9 April. For thirty days the sale is a receivable, an amount owed, and the risk Kestrel & Pine carries is Bergzicht's credit, not a card network's.

The money arrives as a bank transfer. On or before 9 May Bergzicht pays by SEPA credit transfer, with remittance information listing the invoice numbers it settles, often several at once. Since October 2025 euro-area banks must be able to send instant credit transfers (Regulation (EU) 2024/886), so the money can land in seconds; the work is **cash application**, matching the transfer to open invoices in the ERP, and a remittance that names no invoice numbers turns a ten-second transfer into an afternoon's e-mail. A late payment enters a reminder cycle, commonly called dunning; a typical pattern is a reminder at the due date, a second after a week or two, then a hold on new orders and a conversation between Lea Moreau and Pieter Smit. The order hold is a state the OMS must be able to show.

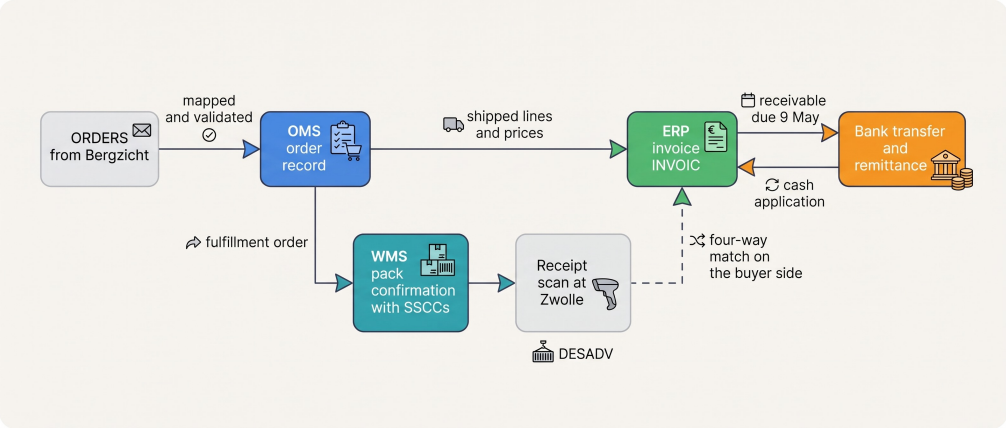


Figure 4: *Each wholesale document has its own system of truth: the order belongs to the OMS, the despatch advice to the WMS's pack confirmation, the invoice to the ERP, and the money to the bank; a document written by any other system is the one the buyer deducts for.*

What Each Handoff Requires

The four questions the book asks of every join (how fresh, how consistent, how fast, how available) give wholesale a different profile from the consumer journey: slower almost everywhere, stricter on consistency, with one hard deadline, the truck. Latencies are the author's illustrative targets.

Handoff	Freshness	Consistency	Latency	Availability	Style (W5)
EDI provider → iPaaS → OMS (ORDERS)	Minutes	One order per GLN plus PO number	Under 15 min *(assumption)*	Mailbox queues; replay after an outage	Events, idempotent
OMS → ERP (prices, credit)	At validation	Price list and credit limit owned by the ERP	Seconds	Hold the order on ERP outage, never guess	Synchronous call
OMS → availability service (allocation)	Seconds	One writer per committed quantity	Under 10 s *(assumption)*	Storefront keeps last value, with buffer	Events
OMS → WMS (fulfillment order)	Minutes	Request, accept, reject, cancel-request	Before the wholesale wave	Queue absorbs an outage; the slot is the deadline	Events with acknowledgement

Handoff	Freshness	Consistency	Latency	Availability	Style (W5)
WMS → EDI provider → Bergzicht (DESADV)	At pack confirmation	Matches the pallets exactly; corrections reference the original	Before the truck leaves	Hold the truck rather than ship without it	Event, acknowledged by CONTRL
OMS → ERP → Bergzicht (INVOIC)	Day of delivery	One invoice per shipment, from shipped lines	Daily batch	Reconciliation count per day	Daily batch
Bank → ERP (payment)	Days	Matched to invoices by remittance	Days	Monthly close	File or bank feed

The DESADV row is the one with no tolerance: freshness, consistency and the deadline all bind at once, and the only safe failure mode is to hold the truck.

The Failure Path: The ASN Chargeback

This is what happened in the third quarter of 2026, on the January landscape, and why it cost **€14,200**.

The mechanism. Bergzicht's ORDERS arrived at the EDI provider and went into the ERP, where wholesale orders were keyed and kept. The ERP generated the DESADV, through the consultant's mapping, the moment it released the order to the warehouse, before anyone had packed anything: pallet numbers came from an ERP number range and contents as the order lines suggested. The WMS then packed as the pickers found best, merging or splitting pallets, and printed its own pallet labels at pack time. When Bergzicht's receivers scanned a pallet, one of three things happened: the SSCC was in the DESADV but the contents differed; the SSCC was not in the DESADV at all; or the DESADV arrived after the truck. Each triggered a manual count and a fixed compliance fee per non-compliant delivery under Bergzicht's supplier conditions: nine Q3 deliveries, deducted from an October 2026 remittance (Documents Between Companies: EDI, Punchout and E-Invoicing owns the reconstruction).

Why nobody saw it for a quarter. The deductions arrived as unexplained short payments on invoices that were themselves correct. Sanne de Wit's team saw receivables that did not close; Lea Moreau heard of "label problems" from Pieter Smit; Daniel Okafor's warehouse had shipped every unit. The three views were never joined, and only the consultant could read the mapping. **The Lynchpin** was the consultant who built the mappings in 2025; **The Pulled Rug** was the contract itself: supplier conditions that demand pallet-level DESADV content (the level Jumbo's public rules also demand), which nobody at Kestrel & Pine owned or checked against what the systems sent.

Detection, in the target architecture. Three signals replace the silence: a per-partner **compliance scorecard** on the partner card (DESADVs sent before departure, CONTRL rejections, deductions per delivery), an alarm on any negative acknowledgement, and a rule in cash application that a short payment opens a case naming the invoice, the DESADV and the deduction code.

Recovery. A deduction can be disputed only with evidence that the DESADV described the pallets as shipped, which is what W6 produces: the SSCC from the WMS at pack, the DESADV from the pack confirmation, the partner card with the guideline version. The €14,200 was not disputed because the evidence did not exist.

What can still go wrong after W6. A DESADV sent after the truck leaves because the manifest closed first (fix: the DESADV precedes the manifest, and the truck waits). A negative CONTRL that stays in a mailbox (fix: the alarm). A quantity changed after the DESADV was sent (fix: a corrected DESADV that references the original, never a silent edit). An SSCC serial reused because two systems could print labels (fix: one serial source, the WMS). And the next guideline revision, which the partner card's change contact now receives.

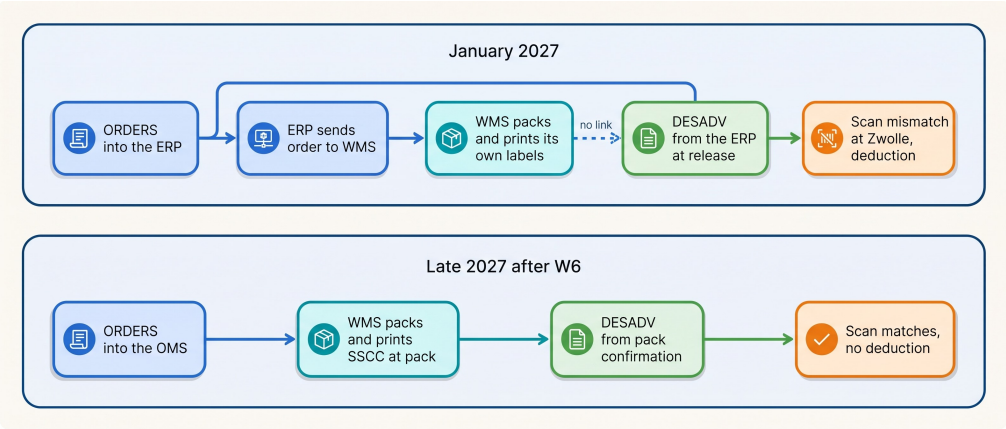


Figure 5: *The ASN Chargeback was an architecture failure, not a warehouse one: in January 2027 the despatch advice described the order, while the pallets described the pack; after W6 one system writes both.*

The Variant: A Belgian Shop's Peppol Invoice

Not every wholesale customer is a chain with an EDI mailbox. Two weeks later, on Wednesday 21 April, Ardennes Trail Co. (fictional), an outdoor shop in Namur, orders 40 Ridgewalker jackets at €95 through the B2B portal, €3,800 excluding VAT *(illustrative; the same order Tax, Invoices and the Books: From Order to Ledger books)*. The order enters the OMS with channel "B2B portal" and ships from Duiven by bpost on 23 April, invoiced that day and due on 23 May; up to the invoice it is Eva's journey with a company account and payment terms.

Belgium mandates structured e-invoices. From **1 January 2026**, Belgian businesses liable to VAT must issue invoices to each other as structured electronic invoices over the **Peppol** network; "Sending a PDF invoice by e-mail or via a platform will no longer be enough" (einvoice.belgium.be, 18 September 2024). A business that lacks the technical means to issue or receive them faces fines of €1,500, €3,000 and €5,000 for successive offenses; the tax administration announced tolerance for the first three months of 2026 but "under no circumstances" a general postponement (einvoice.belgium.be, December 2025). Whether the obligation reaches a Dutch supplier without a Belgian establishment is not settled by the sources this book has read. Kestrel & Pine sends Peppol e-invoices to all its Belgian business customers since January 2026 either way, because their bookkeeping now expects invoices in their accounts-payable system, not in an inbox.

Peppol in plain words. Peppol is an open network and a document standard. Each company connects through an **access point**, a service provider, and reaches any other participant through theirs: the four-corner model (supplier, supplier's access point, customer's access point, customer), as OpenPeppol describes it (n.d., accessed 2026-11). The document is a **Peppol BIS Billing 3.0** invoice: XML in the UBL syntax implementing the European semantic standard **EN 16931**, which lists what any invoice must contain. Release 3.0.21 was published on 20 May 2026 and became mandatory on 17 August 2026; it uses type code 380 for an invoice and 381 for a credit note and requires a VAT category code on every line.

What the ERP produces for Ardennes Trail is the same invoice as Bergzicht's in substance and different in two ways. First, the VAT: the goods leave the Netherlands for a business customer in Belgium, an intra-EU supply at 0% with the reverse-charge mention, valid only if the customer's VAT number was validated in VIES, the EU's registry; the buyer accounts for Belgian VAT itself (europa.eu, n.d., accessed 2026-11). Second, the transport: the ERP hands the document to Kestrel & Pine's access point, which delivers it to the customer's access point and so to the customer's software. **Transmission becomes a state of the invoice**: sent, delivered, or rejected by the network's validation rules. A rejected e-invoice is not an invoice; the ERP must hold that state and retry or correct, a new failure mode for a finance system that used to

attach a PDF.

	EDI INVOIC to Bergzicht	Peppol invoice to Ardennes Trail
Who defines the content	Bergzicht's guideline, on top of the EANCOM standard	EN 16931, with a published validator
Purpose	Settlement matched to order and receipt	A tax-compliant invoice any receiver's software can read
Addressing	GLNs agreed per partner	A Peppol participant identifier in a public directory
Transport	The EDI provider's network	Any accredited access point, four corners
Validation	CONTRL and the partner's rejection	Schematron rules at the access point
Legal position, November 2026	Accepted in the Netherlands, where e-invoicing is planned for 1 July 2030 (cabinet announcement, 11 September 2026)	The format Belgium requires between Belgian businesses

Germany, Kestrel & Pine's third market, sits between the two: every German business must be able to receive e-invoices since 1 January 2025, and the Finance Ministry's FAQ tolerates existing EDI procedures through 2027 (accessed 2026-11). EU-wide, the ViDA package adopted on 11 March 2025 brings digital reporting for cross-border B2B from 1 July 2030. Documents Between Companies: EDI, Punchout and E-Invoicing and Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation keep the calendar; the point here is that the invoice's owner did not change, only its format and its delivery receipt.

The Same Order in January 2027

Step	Late 2027	January 2027
Where the order lives	The OMS, channel EDI, one number family	The ERP's wholesale order entry, beside the platform's web orders and the connector's marketplace orders
Who can read the mapping	Ruben Alves, from the partner card	The external consultant (The Lynchpin)
Stock	Committed in the availability service within seconds	Visible to the platform an hour later at best, through the sync chain
SSCC and DESADV	From the WMS at pack	DESADV from the ERP at release, with ERP pallet numbers; labels printed separately by the WMS (the ASN Chargeback)
Invoice	From the ERP, from shipped lines	From the ERP, from order lines
Deductions	A case per short payment, with DESADV evidence	Unexplained short payments; €14,200 in Q3 2026

When It Goes Wrong Along This Path

Beyond the ASN path, five failures recur in wholesale, each with a detection and a recovery.

The duplicate ORDERS. Detected by the GLN plus PO key; recovered by doing nothing. **The order change after allocation.** Bergzicht sends a change (an ORDCHG message, or a phone call to Lea); the OMS re-validates and re-allocates, and the ORDRSP restates the whole order, not only the changed line. **The short pick.** A carton missing at pack; the DESADV and the invoice show the shipped quantity, and Lea tells Pieter before the truck arrives. **The remittance that names nothing.** A transfer for €23,214.80 against three invoices and two deductions; recovered by a cash-application rule that matches on amount patterns and opens a case rather than parking the money. **The guideline revision.** Detected by the partner card's change contact and a test cycle with the EDI provider before the effective date; the alternative is the Pulled Rug, discovered by deduction.

Gregor Hohpe's 2004 Starbucks essay names the three recoveries all of these reduce to: write-off, retry, or compensating action. In wholesale the compensating action is a credit

note, and the write-off is a deduction nobody disputed.

What to Take Away

Established.

- EDI between retailers and suppliers is a buyer-defined document exchange with penalties in the contract; the public conditions of Jumbo (2026) and bol (2021) and Walmart's program (2017) all tie deliveries to a despatch advice and SSCC labels, and refuse or charge for non-compliance.
- The GS1 identifiers divide the work: GTIN for the item, GLN for the location, SSCC for the logistic unit; the GS1 General Specifications (Release 26.0, 2026) make the SSCC the only identifier of a pallet and the one the DESADV refers to.
- An invoice is issued once, by one numbering authority, from what was delivered, and corrected only by credit note; revenue is recognized at delivery (IFRS 15). Structured e-invoicing is mandatory in Belgium since 2026, receivable in Germany since 2025, planned in the Netherlands for 2030, and EU-wide for cross-border B2B from July 2030 under ViDA.

Depends on context.

- Whether EDI orders should enter an OMS or the ERP. At Kestrel & Pine's stage, with four channels and one warehouse, one order record in the OMS is right; a Growing merchant with one EDI partner and no OMS does better putting the order in the ERP and the DESADV in whichever system packs than adding a hub for one partner.
- Whether to run a direct AS2 link or a provider's network. A provider is right while partners are few and mappings change rarely; a direct link pays off only for a supplier with many partners and an EDI team.
- Whether to send Peppol invoices to customers who are not obliged to receive them. Yes where their bookkeeping already expects it, as in Belgium; elsewhere, have the ERP's access point ready before the local mandate makes it the only route.

The author's synthesis.

- I treat the despatch advice as the most dangerous document a merchant sends, because it is a claim about physical goods that only one system can verify. Whatever system packs the pallet writes the DESADV; no exceptions, no mapping that "knows" how the warehouse packs.
- I would put every trading partner on a one-page partner card before touching a message, and I would consider an EDI relationship that only a contractor can explain to be an outage that has not happened yet.

Questions to Ask

1. Which system creates the order record for an EDI order, and does it carry the same number family and state model as a web order?
2. What is the duplicate key for an incoming ORDERS message, and what happens when the same interchange arrives twice?
3. Which system prints the SSCC label, and which system generates the DESADV: the one that packed the pallet or the one that holds the order?
4. Is the DESADV sent before the truck leaves, and what happens to the truck if it is not?
5. For each trading partner, where are the guideline version, the GLNs, the test status and the change contact written down, and who at the company can change the mapping?
6. How is a short payment or deduction detected, which documents are attached to the dispute, and how many were disputed last quarter?
7. Does the invoice come from the shipped lines or from the order lines, and who owns the contract price it uses?
8. Which countries' e-invoicing obligations apply to the company's customers today and in 2027 and 2030, and does the ERP hold the transmission status of each e-invoice?

Bergzicht's pallets and Eva's parcel left the same warehouse in the same April week, on a landscape built for an ordinary day. The next journey is the day that is not ordinary: Friday 27 November 2026, when the Ridgewalker sold out at 19:40 and the storefronts kept selling it for two hours. Journey 4: Black Friday, Hour by Hour walks it hour by hour.

Sources and Evidence

Company, product and investor sources

- Handout richtlijnen gemechaniseerd DC — Jumbo, version 2025-10 (translated from Dutch); DESADV required before every delivery, the SSCC label as the link, refusal without it.
- Algemene inkoopvoorwaarden — Jumbo, 2026-04-09 (translated from Dutch); the €70 per-message forfeit, refusal at receipt, the €250 per-order-line penalty.
- EDI — Jumbo supplier pages, accessed 2026-11; a retailer's VAN-only channel and the GLN rule for DESADV.
- SSCC for EDI suppliers — bol, n.d., accessed 2026-11; SSCC labels mandatory since 31 December 2021 and required inside the DESADV.

- Walmart U.S. gives suppliers new order fulfillment guidelines — Talk Business & Politics, 2017-02-23 (dated); the 3% on-time-in-full fee at introduction.
- How to dispute Walmart OTIF chargebacks — SPS Commerce, updated 2024-05-06; ASN quantity mismatch as a disputable reason.
- SPS Commerce Form 10-K for 2025 — SEC filing, 2026-02-19; the "connect once" network model and customer count.
- Synchronize customers and companies — Microsoft Learn, 2026-10-14; customer records, payment terms and credit limits owned in the ERP.
- PaymentTermsType — Shopify Admin API 2026-10, accessed 2026-11; how a platform types NET and other B2B terms.
- Warehouse management — Manhattan Associates, n.d., accessed 2026-11; wholesale, retail and direct orders handled in one warehouse system.
- Idempotent requests — Stripe documentation, n.d., accessed 2026-11; the idempotency-key discipline applied here to order messages.
- Customer invoices and Credit notes — Odoo 18.0 documentation, accessed 2026-11; immutable invoices and credit notes as the only correction.

Standards, regulation and official documentation

- GS1 General Specifications, Release 26.0 — GS1, 2026-01; definitions of GTIN, GLN and SSCC and the SSCC's role in EDI.
- GS1 EANCOM 2002, Edition 2016 Upd. 2021, Part I — GS1, 2021 (the standard dates from 2002); EANCOM as a subset of UN/EDIFACT.
- EDI implementation survey — GS1, 2021 (dated); ORDERS, INVOIC and DESADV as the most implemented messages, AS2 the most used channel.
- X12 transaction sets — X12, n.d., accessed 2026-11; the 850, 855, 856, 810 and 997 equivalents.
- RFC 4130: MIME-based secure peer-to-peer business data interchange using HTTP, Applicability Statement 2 (AS2) — IETF, 2005 (dated, still in production use); the AS2 protocol.
- About Peppol — OpenPeppol, n.d., accessed 2026-11; the four-corner model and the role of access points.
- Peppol BIS Billing 3.0 and release notes — OpenPeppol, version 3.0.21, 2026-05-20 (mandatory from 2026-08-17); the invoice specification, type codes 380 and 381, VAT category per line.
- Structured electronic invoices between companies are compulsory in 2026 — FPS Finance (Belgium), 2024-09-18; the mandate and the Peppol network.
- General questions B2B and Specific questions about e-invoicing — FPS Finance (Belgium), n.d., accessed 2026-11; non-established suppliers out of scope, fines of €1,500, €3,000 and €5,000.
- Period of tolerance during the first three months of 2026 — FPS Finance (Belgium), 2025-12-18; tolerance without postponement.

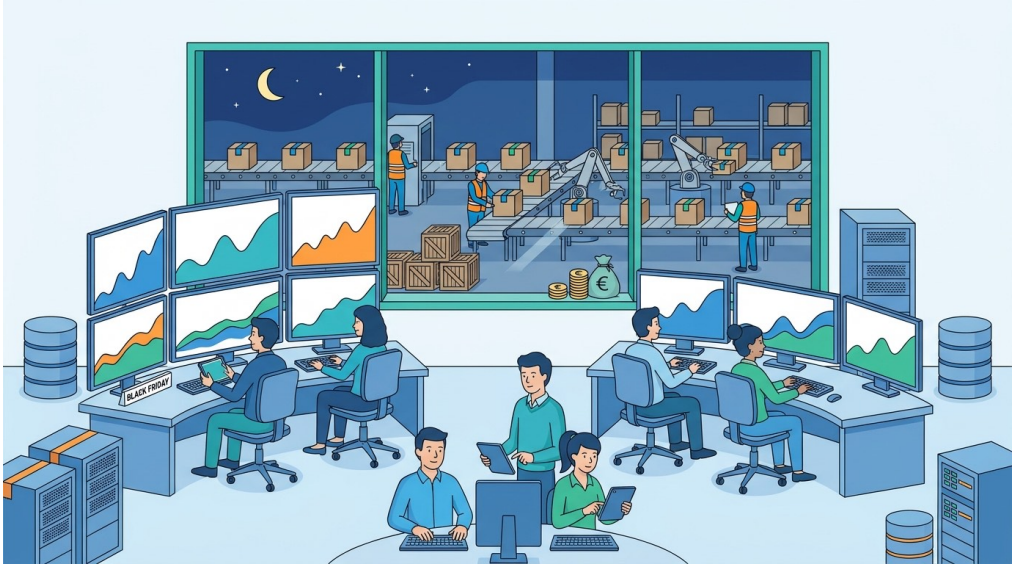
- FAQ E-Rechnung — German Federal Ministry of Finance, n.d., accessed 2026-11 (translated from German); receiving obligation since 2025 and EDI tolerated through 2027.
- Kabinet kiest voor invoering e-facturatie en rapportage voor bedrijven — Dutch government, 2026-09-11 (translated from Dutch); the planned 2030 and 2031 dates.
- VAT in the Digital Age (ViDA) — European Commission, n.d., accessed 2026-11; adoption on 11 March 2025 and the 1 July 2030 reporting date.
- Cross-border VAT — European Commission, n.d., accessed 2026-11; 0% intra-EU B2B supplies with a valid VAT number and VIES.
- Invoice requirements and VAT rates and exemptions — Belastingdienst and business.gov.nl, n.d., accessed 2026-11; mandatory invoice contents and the 21% rate.
- Regulation (EU) 2024/886 on instant credit transfers — Official Journal, 2024-03-19; sending instant euro transfers mandatory from 9 October 2025.
- IFRS 15 Revenue from Contracts with Customers — IFRS Foundation, issued 2014, effective 2018; revenue when control transfers.

Research and practitioner sources

- OMS vs ERP: the architecture shift reshaping commerce — OneStock, 2026 (vendor source); the argument for the OMS owning the order and the ERP owning finance.
- Starbucks does not use two-phase commit — Gregor Hohpe, 2004 (dated); write-off, retry and compensation as the three recoveries.

Kestrel & Pine, Bergzicht Outdoor and Ardennes Trail Co. are fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

40. Journey 4: Black Friday, Hour by Hour



IN THIS CHAPTER, YOU WILL: Follow Kestrel & Pine's Black Friday week of 2026 hour by hour, see what each system and person did, and read the 2027 plan built from it.

KEY POINTS:

- **The evening was lost in the afternoon.** The order flow to the warehouse fell behind at about 13:00, when nobody was watching, because it created orders one at a time under the WMS vendor's rate limit. By the peak hour it was four hours behind, and no gauge showed it.
- **The sell-out at 19:40 took two hours to reach the shelves that were still selling.** The warehouse knew at 19:40, the ERP at 19:45, the storefronts at about 20:30 and the marketplaces at about 21:35, because stock traveled by three scheduled copies. 1,340 units were sold that did not exist.

- **The 2027 plan changes the chain, not the platform.** A forecast of 60 orders a minute, an order path through the OMS tested at 120, stock events to an availability service within a minute, eight switches with owners and triggers, a freeze, a situation room and a reconciliation report at 08:00 on Saturday.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The First Snowfall** — the order flow, the stock chain and the warehouse's evening shift met forty times their normal load for the first time on the day itself.
- **The Phantom Stock** — three channels sold the same last jacket for up to two hours, because each learned of the warehouse's zero by a scheduled copy.
- **The Message in a Bottle** — a queue with no gauge, no alarm and no owner turned a resilience feature into a four-hour delay nobody could explain.

The three journeys before this one follow a single order: Eva's jacket, Eva's exchange, Bergzicht's pallets. This one follows **31,000 orders in a week**, and a Friday evening on which **2,300 of them arrived in one hour**. It is the same chain of systems the earlier journeys crossed, the platform, the iPaaS, the WMS, the ERP, the marketplace connector, running at a pace none of its handoffs was designed for.

The journey is told twice. First **as it happened** in November 2026, with what each system did and what each person saw, in the order they saw it, which is not the order the problems began. Then **as it is planned** for November 2027, with the forecast, the rebuilt order path, the switches, the runbook and the morning-after report. The figures that the book's ledger fixes (31,000 orders, 2,300 in the peak hour, the 19:40 sell-out, 1,340 units, 1,120 orders, €96,000, the four-hour backlog) are canonical; every hour-by-hour figure is an **illustrative** reconstruction consistent with those totals, and is marked so. Black Friday Every Year: Peak Load, Resilience and Recovery explains the ideas this journey applies: orders per minute as the unit, backpressure, the degradation ladder, the situation room, reconciliation.

Where we are in the order's journey. Every step of Eva's jacket happens here, from product page to confirmation to warehouse release to stock update, except that it happens 2,300 times an hour and the steps that took minutes in April took hours in November.

The Week Before the Evening

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), planned Black Friday week 2026 as a campaign. Milan Petrov, Ecommerce and

Marketplaces Manager, configured an automatic **20% discount on all outerwear** for the week of 23–29 November, which put the Ridgewalker shell jacket, the hero product, at **€159.20** instead of €199 (Discounts, Coupons and Loyalty: Promotions Without Chaos tells what else the promotion settings did that week). The agency built the campaign pages. Daniel Okafor, Head of Operations, booked temporary pickers for Duiven and an extra PostNL collection on Friday night. Stock for the German marketplace fulfillment program had been sent in advance in October.

Nobody planned the systems. There was no forecast in orders per minute, no load test, no list of what to switch off, no one on call for the iPaaS, and no dashboard that showed orders against the warehouse's intake. The platform's status page was the only thing anyone intended to watch. That is not negligence; it is what a Growing-stage plan looks like when the business has become Omnichannel without noticing (What Changes at Each Scale: From First Store to Group).

The week built as the table shows. The daily figures are illustrative; the weekly total is the ledger's.

Day	D2C orders *(illustrative)*	What happened
Monday 23 Nov	2,100	Promotion live at 00:00; the storefront's busiest Monday of the year; nothing strained
Tuesday 24 Nov	2,300	Marketplace listings carry the discount; the connector's hourly stock push is unchanged
Wednesday 25 Nov	2,700	Duiven ships Monday's and Tuesday's orders on schedule; the first "where is my order" tickets arrive
Thursday 26 Nov	3,900	Evening traffic double Wednesday's; the order flow to the WMS keeps up, with no slack left
Friday 27 Nov	**11,500**	The day this chapter tells
Saturday 28 Nov	3,900	The queue drains overnight; support finds the oversell; cancellations begin

Day	D2C orders *(illustrative)*	What happened
Sunday 29 Nov	4,600	Reconciliation by hand; the marketplace cancellations are filed; Cyber Monday is tomorrow
Week	**31,000**	10.6% of the year's D2C orders

Thursday evening is the warning nobody read: the order flow kept up with Thursday's evening rate only because that rate was below its ceiling. Nobody knew what the ceiling was.

Friday 27 November 2026, Hour by Hour

The platform took **11,500 orders** on Friday *(illustrative)*. Three clocks matter in what follows: the **order flow** from the platform to the WMS, which ran on the iPaaS and created orders in the WMS one at a time, several API calls each; the **stock chain**, WMS to ERP every 15 minutes, ERP to platform hourly, platform to marketplace connector hourly; and the **warehouse's waves** (a wave is a batch of orders released to pickers together), the last of which is normally released at 21:00 for PostNL's 22:30 collection, an hour before the storefront's "ordered before 22:00" promise expires (Pick, Pack, Ship: Warehouse Management and Fulfillment); for Black Friday Daniel had added a 23:00 collection. The capacity of the order flow, about **seven orders a minute**, is an illustrative reconstruction; it was never measured before the day.

Time	Orders *(illustrative)*	What the systems did	What the team saw and did
00:00–12:00	900	Everything on schedule; the order flow idles between orders	Nothing; a normal busy morning

Time	Orders *(illustrative)*	What the systems did	What the team saw and did
12:00–15:00	1,400 (about 8 a minute)	At about 13:00 the arrival rate passes the flow's ceiling of about 7 a minute; the WMS vendor's rate limit throttles the calls; the queue on the iPaaS begins to grow and nothing shows it	Nothing. The platform admin shows orders arriving; the WMS shows orders arriving; nobody compares the two
15:00–17:00	1,200 (10 a minute)	The queue reaches about 500 orders by 17:00; the 15:00 and 16:00 waves at Duiven are thinner than the storefront's day suggests	The warehouse lead notices quiet pick lists on a day the radio says is Black Friday, and assumes the evening will bring the rush
17:00–18:00	900 (15 a minute)	Queue about 1,000; Ridgewalker stock at Duiven falling fast as the afternoon's orders reach the WMS	Milan watches the campaign dashboard: revenue ahead of plan
18:00–19:00	1,000 (17 a minute)	Queue about 1,600; the ERP's 18:45 stock snapshot still shows Ridgewalkers in most sizes	Daniel's evening shift starts; temporary pickers are briefed

Time	Orders *(illustrative)*	What the systems did	What the team saw and did
19:00–20:00	1,400 (23 a minute)	**19:40: the WMS picks the last Ridgewalker units; physical stock zero.** 19:45: the WMS tells the ERP. Queue about 2,500 by 20:00	19:50: the warehouse lead sees the Ridgewalker pick location empty with orders for it still in the wave, and calls Daniel
20:00–21:00	**2,300 (38 a minute)**	The peak hour. The storefronts and marketplaces still show the Ridgewalker in stock. About 20:30: the ERP's hourly push sets the platform's Ridgewalker quantities to zero; the storefronts stop selling it. Queue about 4,400 by 21:00; the oldest unprocessed order is four hours old	20:20: Daniel calls Milan: "we are out of Ridgewalker and the orders keep coming." 20:35: Milan calls Ruben Alves. 20:50: Ruben opens the iPaaS and sees the queue for the first time

Time	Orders *(illustrative)*	What the systems did	What the team saw and did
21:00–22:00	1,500 (25 a minute)	About 21:15: the platform's hourly push sends zero to the marketplace connector; about 21:35 the marketplaces show sold out. The queue keeps growing	21:05: Milan sets the Ridgewalker to zero by hand in the connector, minutes before the push would have. 21:30: Ruben raises an emergency ticket with the WMS vendor; nobody answers on a Friday night until 22:40
22:00–24:00	900	The storefront's 22:00 promise expires; a late wave releases for the extra 23:00 collection Daniel booked, with thousands of orders placed before 22:00 still on the iPaaS	Daniel decides the night shift will keep picking whatever arrives; Mara Lindqvist is told at 22:15
Saturday 00:00–03:00	—	About 00:00: with the vendor's limit raised, Ruben switches the flow to the WMS's batch import; throughput about 40 a minute; the queue is empty by about 02:30	Orders placed at 20:00 reach the WMS after midnight. Saturday's shift picks through the day; PostNL's Saturday collection carries them; most are delivered Monday

Three things in the table deserve a closer look, because they are the mechanisms the 2027 plan is built against.

The queue nobody could see

The order flow was a resilience feature. Because it queued orders rather than failing checkout, no shopper saw an error and no order was lost. The platform was right to say it held. But the queue had no gauge, no alarm and no owner, so it did what unwatched queues do: it absorbed the problem silently until the warehouse, the last system in the chain, noticed the absence of work. Ruben's 20:50 discovery was the first time anyone had looked at the queue on a peak day. The four-hour backlog at 21:00 and the midnight arrival of the 20:00 orders are the ledger's facts; the throughput of about seven a minute (four calls per order against a WMS limit of about 30 calls a minute) and the shape of the curve are illustrative, reconstructed in Ruben's June 2027 analysis (Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation), which also found that on the platform's side nine apps and connectors shared one API budget that evening and throttled the order flow about 1,900 times. The WMS itself had capacity to spare.

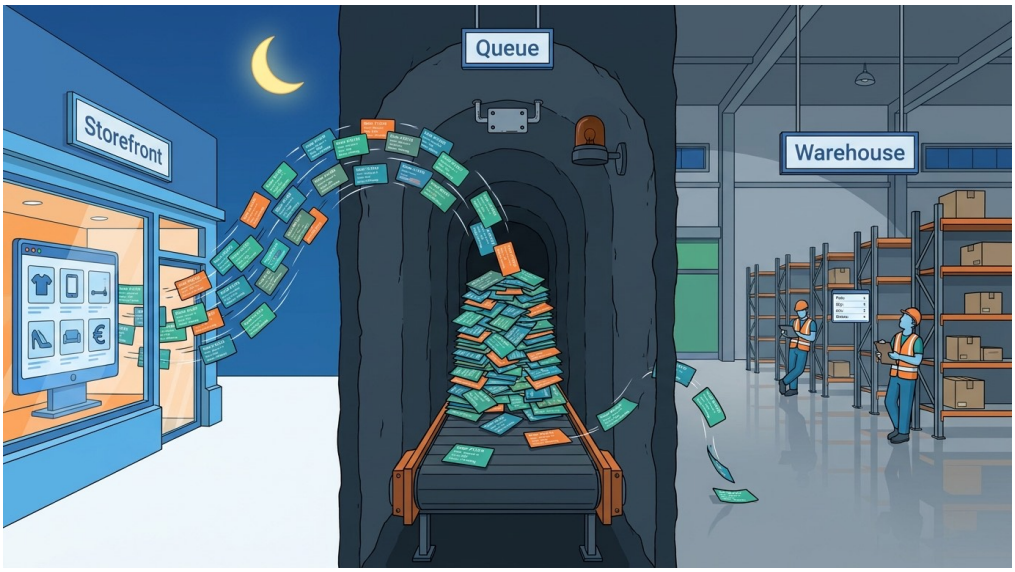


Figure 1: *On Black Friday 2026 the queue did its job and hid the problem: orders piled up between the storefront and the warehouse with no gauge, no alarm and no owner, until the warehouse noticed it had too little work.*

The two-hour sell-out

The sequence diagram shows the two hours that cost €96,000. Each arrow is a scheduled copy, and each copy is late by design.

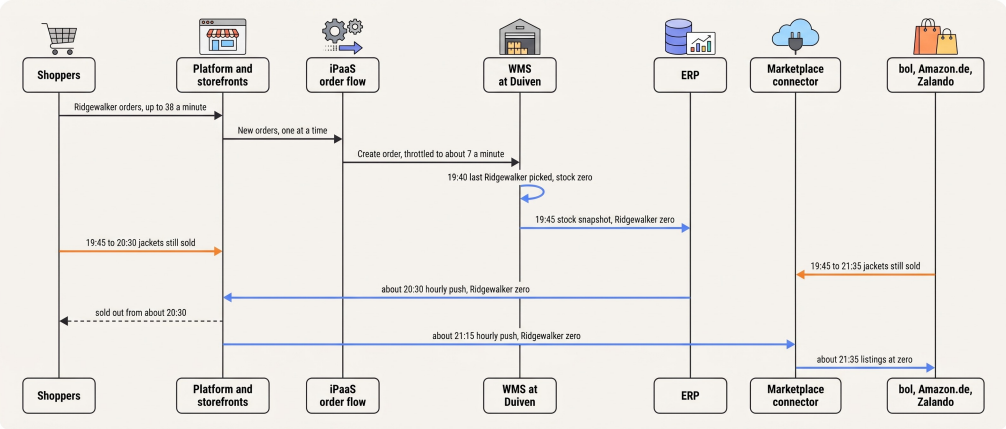


Figure 2: *The sell-out at 19:40 reached the ERP in five minutes, the storefronts in about fifty and the marketplaces in about two hours; in that window the channels sold 1,340 units the warehouse did not have, because stock traveled by three scheduled copies rather than by the event that emptied the shelf.*

The two-hour window was not a fault. It was the chain's design: 15 minutes plus up to an hour plus up to an hour. On every other day of the year the window was harmless, because nothing sold out inside it. Is It in Stock? Inventory, Availability and the Promise You Can Keep explains why the number copied was also the wrong number (physical stock, not available-to-promise) and why the three apps that could write inventory made it worse; this journey only needs the clock.

Three channels sold against the same zero. Of the 1,340 oversold units, about 790 were storefront orders and 550 marketplace orders *(illustrative split from Is It in Stock? Inventory, Availability and the Promise You Can Keep)*. The marketplace share was smaller and cost more, because a seller who cancels a confirmed marketplace order is measured for it: about 160 of the 1,120 affected orders were marketplace orders, 140 of them canceled by Kestrel & Pine, and bol issued a strike, its formal warning for a week below its cancellation standard *(illustrative, from Selling on Someone Else's Platform: Marketplaces and Channel Integration)*.

What the team could and could not do

By 20:50 four people knew something was wrong, and the table of what they could reach is the most instructive part of the evening.

What someone wanted to do	Was it possible?	What happened
Stop selling the Ridgewalker on the storefronts	Yes, by setting quantities to zero by hand in the platform, one variant at a time (24 variants)	Milan was preparing to when the 20:30 push did it
Stop selling it on the marketplaces	Only by editing listings in the connector by hand; no "pause SKU" switch	Done at 21:05, minutes before the hourly push would have
Speed up the order flow	Only with the WMS vendor's help (raise the limit) or a rewrite to batch calls	Ticket at 21:30; answered 22:40; limit raised and flow switched to batch import at about 00:00
Tell customers their delivery would be late	Only by a mass e-mail from the marketing tool, with no list of which orders were affected	Sent Saturday at 10:00 to everyone who ordered after 17:00 Friday
Change "delivered tomorrow" on the Dutch storefront	Yes, by editing the theme text; it was a sentence, not a computed promise	Nobody thought of it until Saturday; the page promised next day until 23:30
Switch off recommendations, search facets, the newsletter pop-up	Not needed; the platform held	—
See orders per minute against the warehouse's intake	No such view existed	Reconstructed on Monday from two exports

The first and third rows say the most. The sell-out could have been stopped on the storefronts at 19:50 by hand, if anyone had known to; the order flow could not have been fixed by anyone at Kestrel & Pine before the vendor woke up. The 2027 plan is largely the table's "No" and "Only" cells turned into switches, gauges and contracts.

Saturday to Monday: Finding Out What Was Sold

Saturday morning the queue was empty and the damage was still invisible. The platform showed 11,500 paid orders. The WMS showed 11,500 orders received, about 1,100 of them with at least one Ridgewalker line it could not allocate. Nothing connected the two lists

to the shoppers waiting for a jacket. Support found the oversell the way support always finds it: from the first "my order says processing" tickets.

The clean-up took three people most of the weekend and finance the week after. Daniel's team produced the shortage list from the WMS. Milan matched it to orders in the platform and in the connector by hand, by SKU, because the two systems had no shared order identifier for marketplace orders. Sanne de Wit's team handled the money. The outcome, in the ledger's totals with an illustrative breakdown:

Outcome	Orders	Units	Treatment
Canceled	760	900	Refund by the original method; a €15 goodwill voucher; marketplace cancellations filed as seller cancellations
Delayed	360	440	Held for a supplier delivery due Tuesday 1 December; shipped that week with the voucher
Total affected	**1,120**	**1,340**	

The **€96,000** the ledger records for the incident breaks down, illustratively, into lost gross margin on the canceled units (about €65,700: 900 units at roughly €73 of margin each at the promotion price), the goodwill vouchers (€16,800: 1,120 at €15), marketplace penalties and the cost of the strike (about €6,500), and support overtime and re-shipping (about €7,000). The roughly 8,000 orders that missed next-day delivery *(assumption, from Pick, Pack, Ship: Warehouse Management and Fulfillment)* are not in that figure; their cost was a week of tickets and an unmeasured amount of trust.

Three reconciliations ran by hand that weekend that should have been jobs: platform orders against WMS orders (to find the stuck and the duplicated), WMS shortages against orders (to find the oversold), and refunds issued against cancellations (to make sure each canceled order was refunded once; the Double Refund of January 2027 shows what happens when that one is skipped, in The Way Back: Returns, Exchanges, Refunds and Reverse Logistics). Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation turns each into a reconciliation card.

The post-mortem, written in December, found five things: no forecast; no capacity figure for any link; a queue with no gauge; stock copied on a schedule rather than sent on change; and no one with the authority to flip anything. It also recorded Milan's first fix, in place by mid-December: the connector publishes the platform's figure minus ten units per variant to each marketplace, and zero below ten *(illustrative, from Selling on Someone Else's Platform: Marketplaces and Channel Integration)*, a buffer that buys minutes on a Black Friday evening and costs a few unsold units on every other day. Noor Visser, who arrived in January 2027, built the Waypoint proposal on that post-mortem

(Meet Kestrel & Pine).

The 2027 Plan: The Same Evening, Rehearsed

Black Friday 2027 falls on **Friday 26 November**; the peak week is 22–28 November and Cyber Monday is the 29th. The plan below was approved with Waypoint in March 2027 and is owned by Noor as incident commander. Every figure is fictional; the dated public practices it borrows from are cited in Black Friday Every Year: Peak Load, Resilience and Recovery.

The forecast and the capacity table

The forecast is **60 orders a minute** in the peak hour, about one and a half times 2026, a convention Shopify's 2025 readiness program also used (roughly 150% of the previous year's load; Shopify Engineering, November 2025). Every link is tested at **80**, and the slowest link sets the number the waiting room and the promise are tuned to.

Link	2026 capacity *(illustrative)*	2027 target	Proof
Storefront and checkout (the platform)	Held at 38 a minute; buyer traffic is not rate-limited	Vendor's responsibility; status page watched	Vendor's own program; no merchant test
Payment authorization (the PSP)	Held	200 a minute confirmed with the PSP in August, with notice of the flash-sale evening as its documentation asks	PSP confirmation in writing
Order into the OMS (platform webhook to OMS)	Not applicable in 2026	150 a minute measured	Load test, September

Link	2026 capacity *(illustrative)*	2027 target	Proof
OMS to WMS (fulfillment orders in batches within the WMS's limit)	About 7 a minute, one at a time	120 a minute tested; alarm at 10 minutes of lag	Load test with marked orders, September and October
WMS stock event to availability service to channels	15 min plus hourly plus hourly	Under 1 minute end to end; 300 events a minute tested	Load test, October
Marketplace stock push (connector to marketplaces)	Hourly	Within 5 minutes, per marketplace rate limit	Connector vendor's stated rate; observed in October
Picking at Duiven (Friday evening and Saturday)	Never the limit in 2026	9,000 parcels a day with the peak roster *(illustrative)*	Daniel's staffing plan; the delivery promise is set from it
ERP import and invoicing	Hourly	Daily batch, overnight, off the peak path (W5)	—

The order path is new. Since 15 September 2027 marketplace orders, and since 13 October D2C orders, are created in the **OMS** (order management system, the one order record of Waypoint W2), which sends fulfillment orders to the WMS in batches sized to the WMS's rate limit, with the order number as the **idempotency key** (a repeated message has the same effect as the first) so that a retry can never create a second pick list. The queue between the OMS and the WMS has a gauge (the age of the oldest unsent order), an alarm at ten minutes, a dead-letter queue (where messages go after they have failed every retry) with Ruben as its owner, and a replay procedure that was exercised in October. Wholesale orders by EDI and by hand move to the OMS on 8 December, after the peak, and nothing else changes during the freeze from 15 November to 5 December (Moving Without Burning the Bridge: Replatforming and Migration gives the cutover sequence).

The stock loop is replaced for the channels that oversold. The WMS emits a **stock event** on every pick, receipt and adjustment; the **availability service** (a module of the OMS, Waypoint W1) recomputes **available-to-promise** per channel (physical stock minus what is already promised, minus a per-channel buffer of three units per SKU for marketplaces and one for the storefronts) and publishes the change to the platform and

the connector within a minute. The go/no-go for using it in the peak was **15 October**. Had it failed, the fallback was the 2026 chain with the ten-unit marketplace buffer and a person pausing listings by hand; that fallback was rehearsed too.

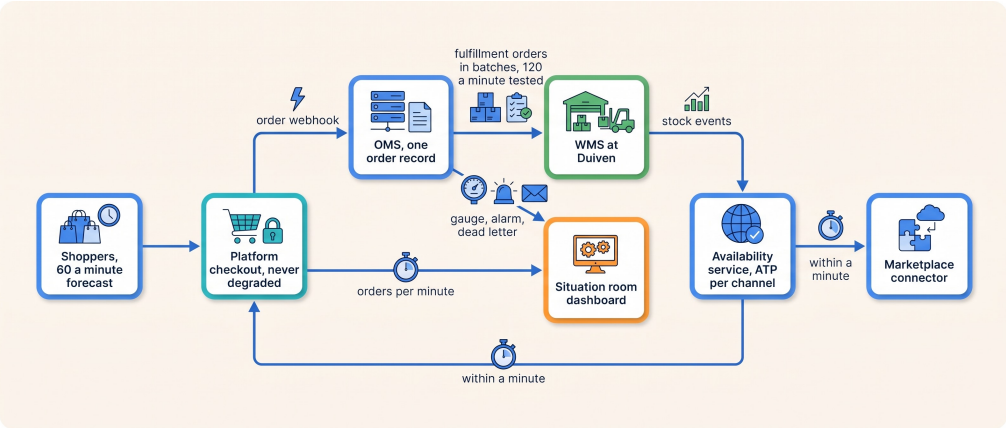


Figure 3: *The 2027 chain: orders flow through the OMS to the WMS in batches with a gauged queue, stock returns as events through the availability service within a minute, and both feed one dashboard in orders per minute; the platform's checkout is the one link that is never switched off.*

The switches

The **degradation ladder** (switching off non-essential features in a planned order so the essential ones keep working) has eight rungs. Each is a feature switch that exists today, was flipped once in the October rehearsal, and has one owner who may flip it without a meeting. The triggers are the plan's; the thresholds are illustrative.

#	Switch	Owner	Trigger	What it does
1	Recommendations and "recently viewed" off	Milan	Storefront p95 response over 2 s, or a platform status notice	Removes the personalized calls from every page
2	Search facets reduced to size, color and price; two sort options	Milan	Same as 1	Cuts the search service's load per query

#	Switch	Owner	Trigger	What it does
3	Stock badges from the buffered copy	Ruben	Availability event lag over 2 minutes	Shows “low stock” under 10 units; never “in stock” from a stale figure
4	Marketplace buffer raised to 25 units for hero SKUs; listings paused below it	Milan	Lag over 2 minutes, or orders for a SKU exceeding its ATP	Protects the channel that is measured for cancellations
5	Delivery promise widened: “tomorrow” becomes “within three working days”	Daniel	Oldest unsent order over 10 minutes, or the Friday pick plan over 80% consumed	The page tells the truth the warehouse can keep
6	Non-essential checkout options off (gift wrap, gift message); a single payment method off if the PSP reports it degraded	Sanne’s team with Milan	Checkout error rate over 1%, or a PSP notice	Shortens the checkout; removes a failing method
7	Waiting room at the CDN	Ruben with the agency	Storefront error rate over 1% or the platform’s notice; admission rate set to 75% of the slowest tested link	Converts the spike into a known inflow
8	New-account creation and the newsletter pop-up off	Milan	Only on Noor’s call	The last load removed before the money path is touched

#	Switch	Owner	Trigger	What it does
—	**Never:** checkout, payment authorization, order creation in the OMS, the order path to the WMS	—	—	Queues grow instead

Rung 5 is the one 2026 lacked most. The Dutch storefront's "ordered before 22:00, delivered tomorrow" was a sentence in a theme file. Since May 2027 the first promise service of Getting It There: Carriers, Labels, Tracking and the Delivery Promise computes the promise from a transit table plus the WMS's cut-off and backlog, so a growing queue already widens it on its own; rung 5 is the override on top, with an owner and a trigger, and for peak week the service is set to "within three working days" from the start, because Daniel's pick plan, not the theme, decides what can be promised.

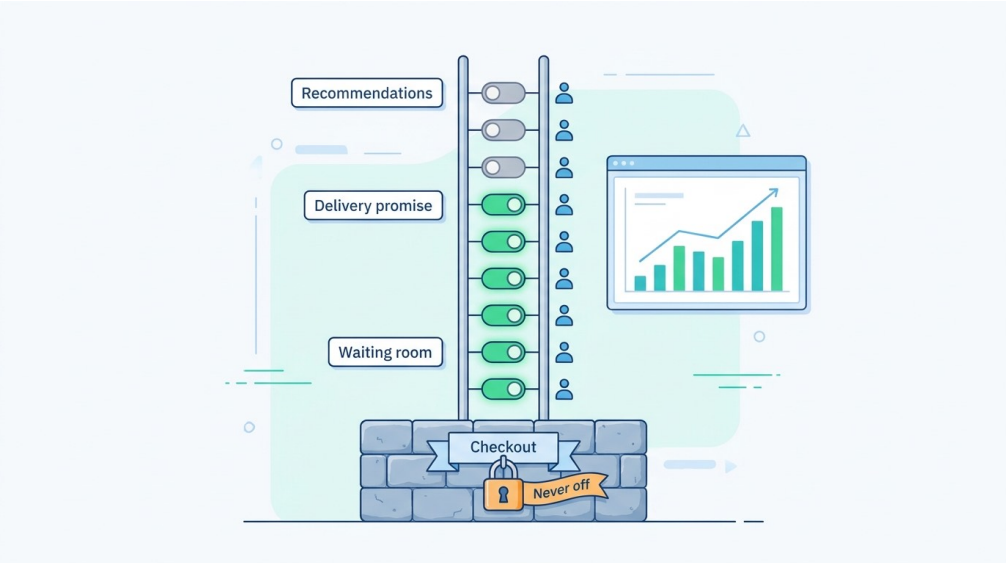


Figure 4: *The 2027 plan's degradation ladder has eight rungs, each a rehearsed switch with one owner; checkout, payment and the order path sit below it and are never switched off.*

The runbook

The runbook is a document of four pages, written in September, rehearsed on 15 November and kept open on the situation-room screen. Its parts:

Roles. Noor is incident commander and decides; nobody else flips rung 8. Ruben owns the OMS-to-WMS path, the availability service and the dead-letter queue. Milan owns the storefronts, the apps and the marketplace listings. Daniel's shift lead owns the warehouse and the delivery promise. Sanne's team owns payment methods and refunds. An agency engineer is on call for the theme and the CDN. The vendors' emergency contacts (platform, OMS, WMS, PSP, connector) are on the first page with ticket priority codes and the hours they are staffed; the WMS vendor's Friday-night gap of 2026 is closed by a contract clause agreed in June.

The dashboard. One screen, refreshed every minute, in business units: orders per minute per channel against the same hour last week; the age of the oldest order not yet in the WMS; stock event lag; dead-letter depth; checkout error rate; PSP authorization rate and decline rate against normal; the WMS's and the PSP's rate-limit headroom. The missing-event alarm fires when orders per minute fall to zero while storefront sessions stay high.

Cadence. The situation room runs on Friday from 16:00 to midnight, with a check-in every 30 minutes until 19:00 and every 15 minutes from 19:00 to 22:00. Each check-in reads the dashboard aloud against the thresholds in the switch table. Saturday and Cyber Monday run on call with the same dashboard.

Communication. Three templates, written in advance: a customer e-mail for a delayed delivery with the order numbers filled from the OMS, not from "everyone who ordered after 17:00"; a note to each marketplace's seller support when listings are paused; and a one-line status to the company every hour. Mara is told at the first switch, not the last.

The morning after. At 08:00 on Saturday the reconciliation report compares orders in the platform, the OMS and the WMS by order number (stuck, duplicated, missing); stock per SKU in the WMS against the availability service; payments captured in the PSP against orders in the OMS; and refunds issued against cancellations. Each difference has a named owner and a fix: replay, manual correction or write-off. Hanna Berg's team produces it from the data warehouse, under the one definition of an order that Waypoint W7 adopted.

The rehearsal

Two load tests and a game day preceded the freeze. In September the team placed marked test orders at 60 a minute through the whole path, platform to OMS to WMS, with the WMS configured to skip them and finance to exclude them, the practice Zalando described for its order-flow simulators in 2020. In October the test ran at 80, then 120 on the OMS-to-WMS link alone, and found the first real defect: the batch size that kept the WMS inside its rate limit at 80 orders a minute exceeded it at 120, and the retries that followed doubled the load, the thundering-herd effect that backoff with jitter exists to prevent. The fix was a smaller batch and a jittered retry, and the test was repeated.

The game day in October rehearsed three failures: the WMS unreachable for thirty minutes (the queue grew to about 1,800 orders, the gauge and the alarm worked, the delivery-promise switch was flipped at minute 11, and replay drained the queue in 22 minutes); the availability service stopped (the stock-badge switch and the marketplace-buffer switch were flipped, and the fallback chain ran for an hour); and the PSP declining one payment method (the method was switched off within eight minutes). Each exercise either proved a runbook page or rewrote it.

When It Goes Wrong in 2027

The plan does not promise a quiet evening. It promises that the failures it expects leave known traces and have known fixes.

If, on 26 November 2027	The plan says
The OMS-to-WMS queue grows past ten minutes	The alarm fires; rung 5 widens the promise; the queue is allowed to grow; replay after the cause is found; no order is lost
The availability service lags past two minutes	Rungs 3 and 4; listings of hero SKUs pause below the raised buffer; the fallback chain is one switch away
The WMS vendor throttles again	The batch size is already inside the limit; the June contract names a Friday-night contact; the queue absorbs the rest
A retry creates a duplicate fulfillment order	It cannot: the order number is the idempotency key; the Saturday report would show it if it did
The PSP degrades	Rung 6 removes the failing method; every captured payment is matched to an order on Saturday
The platform itself has an incident	The vendor's problem to fix and the merchant's to communicate; the waiting room and the status page are the levers; the queues behind the platform are empty, which is the one good thing about it
The forecast was wrong by two	The chain was tested at 80 and the OMS-to-WMS link at 120; above that, the ladder runs from the top and the promise widens

What the plan deliberately leaves slow is the ERP: its daily batch absorbs the week's volume overnight, and if it falls behind, invoices are late, which is the one delay the chain can afford. The 2028 peak will be the first with the promise offered through an API and, if the 2028 evaluation says so, the platform's own POS in the stores.

What to Take Away

Established.

- A queue between a platform and its warehouse system is the right design under peak load, and it is only safe with a gauge, an alarm, an owner and a replay procedure (AWS Builders' Library 2019; Amazon SQS dead-letter documentation; Zalando's 2015 practice of pausing non-essential processing, reported 2020).
- Stock copied on a schedule oversells in proportion to the sales rate and the copy interval; a fifteen-minute plus hourly plus hourly chain is harmless on an ordinary day and a two-hour blind spot at 38 orders a minute.
- A load test of the merchant's own order path, with marked orders that fulfillment skips, is the only test that finds the merchant's bottleneck; the platform vendor's tests find the vendor's (Zalando 2019 and 2020, Shopify 2023 and 2025).

Depends on context.

- The switch-off ladder's order depends on what the business can afford to lose: a brand whose peak is one hero product pauses marketplace listings early; a marketplace-first seller, measured on cancellation rate, protects those listings and widens the storefront's promise first.
- A static but honest peak-week delivery promise is the right interim for a merchant without a computed promise; a merchant with one should let the warehouse's plan set the page automatically and keep the switch as the override.
- Running the OMS cutover before the peak was right for Kestrel & Pine because the parallel run finished in September and the link was load-tested in October; a merchant whose cutover slips into November should run the old path and keep the new one dark until December.

The author's synthesis.

- The lesson of the 2026 evening is not that the systems were too slow; it is that nobody could see them and nobody could touch them. Most of the 2027 plan is visibility and switches, which cost far less than the OMS and would have saved most of the €96,000 on their own.
- A peak plan should be written as a journey, hour by hour, with a name in every row; a plan written as a list of systems leaves the 20:50 moment, when someone finally looks, to chance.

Questions to Ask

1. At what time of day last year did our order flow first fall behind, and who would have seen it?
2. How long does a sell-out at the warehouse take to reach each channel, by the clock, and who tested it?
3. Which of our switches have been flipped in a rehearsal, and who may flip each one without a meeting?
4. Can we change the delivery promise on the page in one minute, and who decides?
5. What are the vendors' Friday-night contacts, and what does the contract say about response time?
6. Which marked test orders have we sent through the whole path at this year's forecast, and what did the warehouse do with them?
7. Which three reconciliations run on Saturday morning, by which key, and who owns each difference?
8. Who is in the room on the evening, and what does the first row of the runbook say?

The journeys end here. Part X asks the questions Kestrel & Pine's 2027 answered in one particular way (buy or build, stay or replatform, how to organize fourteen people around a dozen systems) as questions every merchant faces, and when each answer changes. Continue with Part X — EVOLUTION: Decisions That Change With Scale.

Sources and Evidence

Company, product and investor sources

- How we prepare Shopify for BFCM, Shopify Engineering, 2025 — forecasting at roughly 150% of the previous year's load, scale tests from April to October, Game Days on the money paths.
- Scale and performance testing at Shopify, Shopify Engineering, 2023 — testing with cache bypass so that origins are measured.
- Faster checkout for customers during flash sales, Shopify Changelog, 2018 — payment processing separated from order creation under high volume (eight years old).
- Webhooks, Shopify developer documentation, n.d., accessed 2026-11 — delivery not guaranteed; reconciliation jobs recommended.
- How Zalando prepares for Cyber Week, Zalando Engineering, 2020 — order-flow simulators that fulfillment skips, pausing non-essential asynchronous processing, the Situation Room (six years old).

- End-to-end load testing Zalando's production website, Zalando Engineering, 2019 — production load tests of recorded shopper journeys (seven years old).
- Rate limits, Stripe documentation, n.d., accessed 2026-11 — the advice to tell the PSP about a flash sale in advance, and backoff with jitter.
- Cloudflare Waiting Room, Cloudflare, 2021 — what a waiting room is and how its admission rate is set.

Standards, regulation and official documentation

- Using load shedding to avoid overload, AWS Builders' Library, 2019 — overload, goodput and prioritizing the requests that matter.
- Exponential backoff and jitter, AWS Architecture Blog, 2015 — why synchronized retries double the load and jitter spreads them.
- Amazon SQS dead-letter queues, AWS documentation, n.d., accessed 2026-11 — dead-letter queues and redrive.
- Monitoring Distributed Systems, Google SRE book, 2017 — the four golden signals behind the situation-room dashboard.

Research and practitioner sources

- Feature Toggles (aka Feature Flags), Pete Hodgson, martinfowler.com, 2017 — ops toggles as the mechanism of the switch-off ladder.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Part X — EVOLUTION: Decisions That Change With Scale

IN THIS PART, YOU WILL: Learn to choose and price a platform strategy, move systems without burning the bridge, read your scale stage and its signals, draw an ownership map that fourteen people will keep, and separate what AI does in commerce today from what agents will need tomorrow.

KEY POINTS:

- **At a growing merchant the problems live in the joins, so the platform decision is mostly about what to leave where it is**. Build only what customers choose you for, buy what every merchant needs, own

the joins yourself, and price every option over three years, including the lines the quote leaves out.
- **A stage is a shape, and a migration is how you cross from one to the next**. Most Omnichannel incidents are Growing-stage designs under Omnichannel load. The move is made one seam at a time, old and new side by side, with the old owner kept operable through one peak.
- **Ownership is what makes any of it hold**. One row per system, integration and app with a named owner inside the company; one writer per fact; teams shaped around the order's journeys rather than around vendors; and an AI agent treated as one more integration, with an identity, an owner and no write scope on stock, prices or orders.

FAILURE PATTERNS THIS PART ADDRESSES:

- **The Giant's Boots** — a composable rebuild, a microservices program or a shopping agent copied from a company a hundred times larger, into a fourteen-person team that cannot own it.
- **The Burned Bridge** — a platform or a hub switched over on one date, into or near a peak, with no rehearsed way back.
- **The Secret Agent** — a script, an app or an AI assistant that writes stock, prices or orders with no named owner, which the ownership map and the agent policy exist to prevent.

Every part so far has described a commerce platform as it stands on one day. **This part is about the decisions a merchant has to make again each time the business changes shape**. A merchant does not grow smoothly. It grows in jumps: the first store, the first retail chain that orders by EDI, the first marketplace, the first warehouse of its own, one day a second brand. Each jump adds a place where stock sits or a channel where it sells, and the money is lost at those transitions, not between them.

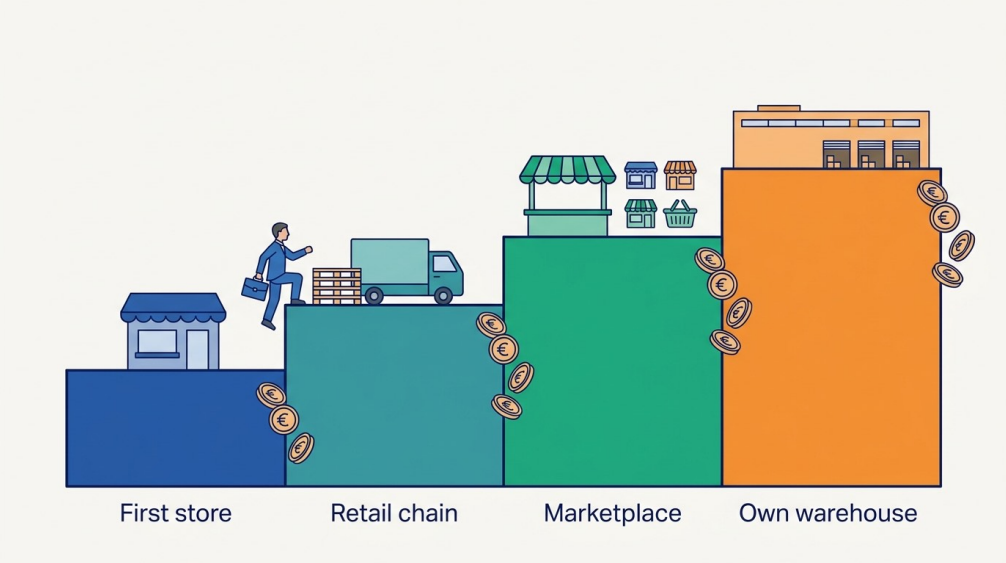


Figure 1: *A merchant grows in jumps, not smoothly: each jump adds a place where stock sits or a channel where it sells, and the money is lost at the jumps rather than between them.*

Part X sits off the path of the order's journey, but every system Eva Janssen's jacket crosses on 13 April 2027 was bought at a different time for a different company: the platform in 2016, the marketplace connector in 2021, the warehouse system and the finance system in 2023, the returns portal in 2025. This part asks when each should change, how to change it without stopping the shop, who answers for it, and which new kind of actor may write to it next.

Kestrel & Pine, the fictional outdoor brand this book follows, met all five questions in 2027. Two proposals on the CEO's desk would replace the one part that held on Black Friday. Two hubs had to be introduced beside a shop selling 800 orders a day. The board asked which fixes were for the company it is and which for the company it was. Thirteen of its 34 integrations had no named owner in January, and three of its 41 apps wrote stock. And an agency offered to build a shopping assistant. **The claim of this part is that these are decisions about ownership and timing, not about tools**: what to own and what to rent, when and how to move a fact's owner, which leftovers to retire, who answers for each row, and which actor may write.

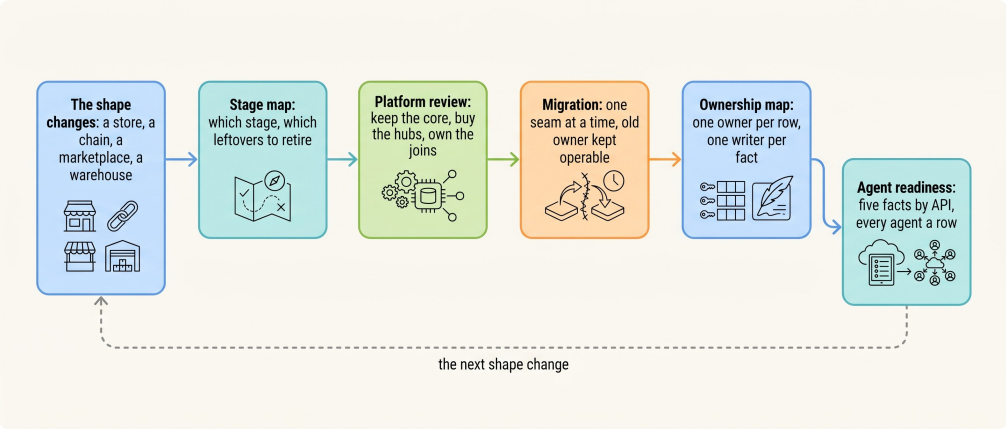


Figure 2: *The five decisions of Part X as one loop: a change of shape is read as a stage, answered by a platform review, carried out as a migration, held in place by an ownership map, and extended to the newest writer, until the shape changes again.*

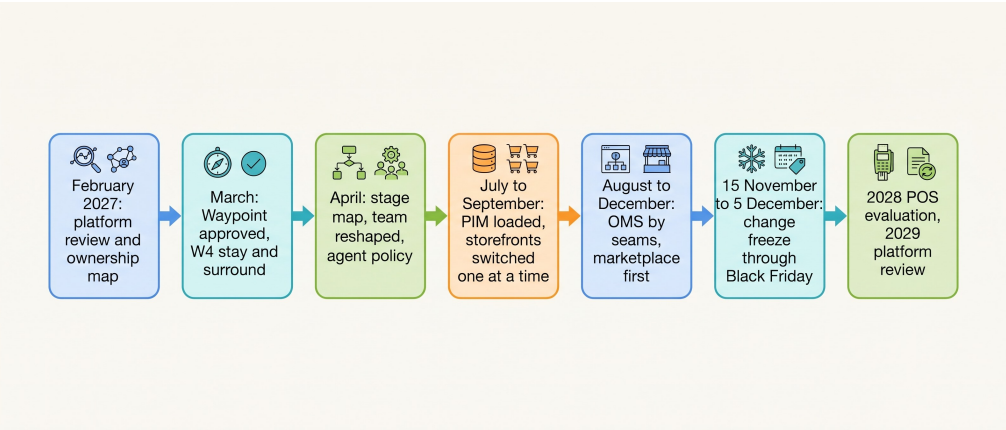


Figure 3: *Kestrel & Pine's 2027 as this part sees it: the decisions are made in the first quarter, carried out seam by seam from July, frozen around the peak, and dated for review rather than treated as final.*

Chapters in this part

Build, Buy or Assemble: Choosing and Changing Platforms decides **how to choose between a suite, a hosted platform with apps, a composable assembly and custom build, and how to price each one honestly**: build what customers choose you for and nobody sells, buy what every merchant needs, assemble when the joins are what you must own, and compare the options over three years on people, agency, migration and exit before the subscription. The case is the February 2027 platform review: a

consultancy's composable rebuild and an enterprise suite at about €3.2 million each over three years, against stay and surround at about €1.33 million, written up as decision W4 and reopened in 2029.

Moving Without Burning the Bridge: Replatforming and Migration decides **how to move a platform or a hub without one cutover date and no way back**: along seams (a channel, a capability, a slice of traffic), with old and new running side by side and a reconciliation that proves they agree, the old owner kept operable through one peak, and products, customers, orders, gift cards and URLs treated as five migrations. The public cases are Target (2011), Finish Line (2012) and Marks & Spencer (2014); the Kestrel & Pine case is the PIM, loaded in July 2027, and the OMS, given marketplace orders on 15 September, storefront orders on 13 October and wholesale orders on 8 December, with a freeze through Black Friday in between.

What Changes at Each Scale: From First Store to Group decides **which architecture decisions change as a business grows, and how to tell that a stage is ending**: a stage is a shape made of channels and stock locations, not a revenue band; each stage leaves behind systems bought for the one before; two transition signals in one year mean the stage has changed, whatever the revenue says. The case is the stage map Noor Visser drew for the board in April 2027: Omnichannel since 2023, with eight of the ten incidents traced to Growing-stage designs.

Who Builds and Who Owns: Teams, Vendors, Agencies and Governance decides **who owns each system, integration and app, and how a small team works with an agency and a dozen vendors**: an ownership map with a technical owner inside the company on every row and one writer per fact, journey teams plus an integration pair rather than one person per vendor, the accounts, repository and decision records in the merchant's hands, and governance light enough to survive a peak season. The case is the ownership map of February 2027 (thirteen systems, 34 integrations, 41 apps; thirteen integrations without a named owner in January and five still unowned in April) and the team shape of April: Buy, Fulfil and Return, an integration pair, a data pair and a 45-minute architecture hour.

Assistants and Agents: AI in Commerce and What Is Real in 2026 decides **what AI already does in commerce, what agentic commerce is, and what a merchant must have before an agent can buy**: established AI reads and proposes while an agent acts, and an agent is a new writer; between September 2025 and November 2026 the flagship in-chat checkout launched and retreated, three protocols appeared without settled governance, and no consumer checkout program reached Europe, where the shopper must still authenticate the exact cart. The case is Noor's agent test of February 2027, five questions an agent asks about the Ridgewalker jacket, which became decision W8: agent readiness, not agents.

Begin with Build, Buy or Assemble: Choosing and Changing Platforms.

Sources and Evidence

Every public claim in this part is sourced and dated in the chapter that makes it, under that chapter's Sources and Evidence. Kestrel & Pine is fictional; recommendations and labeled assumptions are the author's judgement.

41. Build, Buy or Assemble: Choosing and Changing Platforms

IN THIS CHAPTER, YOU WILL: Learn to choose between a suite, a hosted platform with apps, a composable assembly and custom build, and to price each one honestly, including the cost of leaving.

KEY POINTS:

- **The choice is not one choice.** A merchant covers the commerce capability map with four kinds of parts (a suite, a hosted platform plus apps, assembled services, custom code), and the real decision is which parts to buy, which to assemble and what, if anything, to build. The answer changes with business model, scale stage and team.
- **The subscription is the smallest line.** Total cost of ownership is dominated by payment fees, agency and partner work, the people who own integrations after launch, and the cost of leaving. A three-year comparison that prices only licenses is wrong by a multiple.
- **Stay and surround is a legitimate architecture, not a failure to decide.** Kestrel & Pine's 2027 review kept its hosted platform and added

an order hub, a product hub and an availability service beside it, because nothing in its checkout differentiates it and everything in its joins does.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Lynchpin** — a platform decision that leaves one vendor, one agency or one connector holding a capability customers depend on, with no exit plan.
- **The Giant's Boots** — a composable or custom architecture copied from a company a hundred times larger, without its team, its traffic or its reasons.
- **The Forked Platform** — a bought platform customized so deeply that it can no longer be upgraded, which turns "buy" into "build" without anyone deciding it.

Every merchant eventually faces the same meeting. Sales are up, the systems creak, an agency or a vendor has a proposal, and someone asks: **should we move to a new platform?** The question sounds like one decision. It is really four: which capabilities to buy as one bundle, which to buy as separate services and connect, which to build, and which to leave exactly where they are. The answers differ for a founder with one web shop and for a group with six brands, which is why this chapter belongs in a part about decisions that change with scale.

The decision is also the most expensive one in commerce architecture to get wrong, and the hardest to price. A vendor quote lists a subscription. The merchant pays a subscription, plus a share of every payment, plus apps, plus an agency, plus the salaries of the people who keep thirty integrations alive, plus, one day, the cost of leaving. **The visible line is usually the smallest**. This chapter gives you a way to put every line on one page, and a rule for deciding what to build at all.

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), ran exactly this meeting in early 2027. The chapter uses its platform review as the worked example, and its decision, Waypoint W4, "stay and surround", as the pattern to understand.

Where we are in the order's journey. This chapter sits off the order's path. Every step of Eva's jacket, from the product page to the refund, runs on a system somebody chose: the platform for checkout, the WMS for picking, the returns portal for the exchange. The choices in this chapter decide which systems those are, who runs them, and what it costs to change them later.

The 2027 Platform Review at Kestrel & Pine

In February 2027, five weeks into her job, Noor Visser (Head of Engineering) was asked by Mara Lindqvist (CEO) for a recommendation on the platform. The Black Friday

Oversell had cost about €96,000. The Three-Status Order, the Double Refund and the Price Drift had followed within three months. Two pitches were already on Mara's desk, both citing the Oversell.

An outside consultancy proposed a **composable rebuild**: replace the hosted platform with an API-only commerce engine, build a headless storefront (a storefront that is a separate application talking to the commerce engine only through APIs), and add an order management system, a product information system, a search service and an integration layer around it, over eighteen months. The agency, retained since 2017, proposed something smaller: a **headless rebuild of the three storefronts on the current platform**, about €240,000 over nine months plus a second runtime to operate (Delivering the Storefront: Themes, Headless, Rendering and Performance tells how Noor measured that one and declined it).

Noor added a third option of her own to the comparison, so that the review would not be a choice between two pitches: an **enterprise suite**, one vendor's bundle of storefront, B2B portal, order management and promotions, priced on a share of online sales and implemented by a partner over about twelve months. Nobody had pitched it; she priced it because a suite is the usual alternative a board asks about.

Noor's review, which the rest of this chapter follows, asked three questions in order. **What is actually broken?** Not the checkout: the platform held on Black Friday at 38 orders a minute. What broke was the joins: stock copied through four systems with up to two hours of delay, orders living in four systems, product data in four. **What differentiates Kestrel & Pine?** The products, the brand, the stores, the wholesale relationships, and a next-day delivery promise in the Netherlands. Not the cart, not the checkout, not the storefront's rendering. **What would each option cost over three years, including the people and the exit?** That table is in the TCO section below. The outcome, approved in March 2027 as Waypoint W4, was to **stay on the hosted platform and surround it**: an OMS with an availability module and a PIM beside it, no replatforming, the platform's own POS evaluated in 2028, the whole decision revisited at a 2029 review.

Four Ways to Cover the Map

The Map of a Commerce Platform: Capabilities and Layers lays out everything a commerce platform has to do: storefront, catalog, pricing, promotions, search, inventory, cart and checkout, payments, orders, tax and invoicing, warehouse, shipping, returns, customer service, stores, marketplaces, B2B, EDI, ERP, data. No vendor covers all of it well. The question is how you cover it. There are four basic shapes, and most real landscapes mix them.

Shape	What you buy	Who runs it	Where the integration burden lands	Typical pricing	Examples (dated in Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026)
Suite	One vendor's bundle of many capabilities designed to work together	The vendor's cloud, implemented by a partner	Inside the vendor's data model; the joins to the ERP, WMS and carriers remain yours	Negotiated, often linked to GMV or order volume	SAP Commerce Cloud, Salesforce Commerce Cloud, Adobe Commerce
Hosted platform plus apps	A multi-tenant storefront, cart, checkout and admin, extended through an app store	The vendor	Pushed to apps and webhooks; the merchant owns what the apps do	Subscription tier plus payment and transaction fees	Shopify, Commerce.com (BigCommerce), Lightspeed eCom

Shape	What you buy	Who runs it	Where the integration burden lands	Typical pricing	Examples (dated in Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026)
Composable assembly	API-only services (catalog, cart, checkout, search, content) from one or several vendors; the merchant builds or buys the storefront	Vendor clouds; merchant-built front end	On the merchant: it assembles and owns every join	Subscription with GMV or order caps	commercetools, Elastic Path, Medusa, Saleor
Custom build	Nothing; the merchant writes the capability	The merchant	Entirely on the merchant	Salaries and hosting	Large retailers' in-house platforms

Three terms carry this table. **SaaS** (software as a service) is software rented by the month and run by the vendor for many customers at once. **GMV** (gross merchandise value) is the total value of orders through a platform before refunds; several vendors price on it. **Composable** means buying commerce as separate services and assembling them rather than taking one suite; the 2020 acronym **MACH** (microservices, API-first, cloud-native, headless) named the style, though the alliance behind it had reframed its principles as Open, Composable and Connected by 2026 (Case: commercetools, Shopware and the Composable Movement).

Two mixed shapes matter for a merchant of Kestrel & Pine's size. The first is **ERP-**

centered commerce, where the ERP (enterprise resource planning: the system that keeps the books, purchasing and stock valuation) is the hub and the web shop is a front end; Odoo, JTL and PlentyONE sell this, and Microsoft documented in September 2026 that its Shopify connector is preinstalled for new Business Central sign-ups (The Back Office: ERP Integration and Who Is the Hub). The second is the **specialist group**: several specialist products under one owner, pre-integrated with one another. Katapull, a Dutch group formed on 31 March 2025 from five companies and grown to eight labels by its acquisition of the warehouse software Picqer on 7 October 2026, is the public example; its own pages claim "4,500+ customers" and "140+ marketplaces" (vendor figures, accessed November 2026). For a Growing or Omnichannel merchant such a group reduces the joins it has to build, at the price of concentrating several layers under one owner.

The fifth shape is the one most merchants actually have, and the one Kestrel & Pine chose deliberately: **a hosted platform surrounded by specialist layers**. The platform keeps storefront, cart and checkout; an OMS, a PIM, a WMS, a returns portal and a channel connector sit beside it; an integration platform joins them. The composable vendors themselves moved toward this shape in 2025–2026: commercetools launched AgenticLift on 21 January 2026 as a way to "layer agentic commerce capabilities on top of your existing commerce solution", and Adobe's documentation of 21 October 2026 describes Commerce Optimizer as a service that "can work alongside an existing Adobe Commerce or third-party commerce backend". When the champions of replacement start selling layers, the layer is the market.

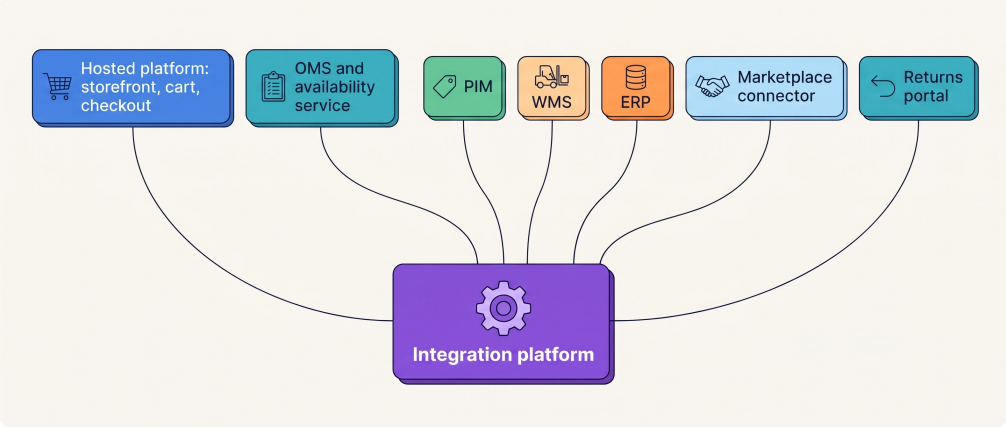


Figure 1: *Stay and surround: the hosted platform keeps the shopper-facing core, specialist systems own the facts it cannot own well, and the integration platform joins them; the merchant's job is the joins, not the core.*

What to Build: Differentiation Against Commodity

The oldest rule in this field is also the most ignored: **build what customers choose you for and nobody sells**; **buy what every merchant needs**; **assemble when the joins between systems are themselves the thing you must own**. In plain words, ask of every capability: if we did this better than anyone, would a customer notice? If not, it is a commodity, and somebody rents it cheaper than you can build it.

For Kestrel & Pine the test was short. Shoppers choose it for jackets, for the brand and for the stores. They do not choose it for its cart. A rebuilt checkout would be a large cost to reproduce a thing the platform already does at 38 orders a minute. What a shopper *does* notice is whether the jacket shown as available is actually there, whether the order's status is the same in the e-mail, the app and the store, and whether the exchange arrives in three days. Those are the joins: availability, order state, product data. That is why W1, W2 and W3 buy an availability service, an OMS and a PIM rather than a new storefront, and why W5 makes integration standards the one thing the team builds itself.

Capability at Kestrel & Pine	Differentiates?	Decision in 2027	Reasoning
Storefront, cart, checkout	No	Keep the hosted platform	Commodity at this scale; held on Black Friday
Available-to-promise per channel	Yes, to the customer	Buy as a module of the OMS; own the rules	The promise is the product; the computation is commodity
One order record across channels	Yes, to the customer and to support	Buy an OMS	A solved category; the ownership decision is the hard part
Product content in one place	Yes, to every channel and to agent readiness	Buy a PIM	A solved category
Integration contracts and reconciliation	Yes, invisibly	Build the standards; run them on the bought iPaaS	Nobody sells your ownership rules
Delivery promise computed from stock and cut-offs	Yes	Compute in the OMS from WMS cut-offs, later	Today a static promise on the page

Figure 2: *Shoppers do not choose Kestrel & Pine for its cart, so checkout is bought; they do notice whether the jacket shown as available is there, so the joins are what it owns.*

The two traps are symmetrical. **Building a commodity** is the Giant's Boots: a merchant copies the architecture of a company with a thousand engineers and spends its fourteen on rendering product pages. **Customizing a bought core until it is a build** is the Forked Platform: every vendor release becomes a project, and the merchant has the costs of building with none of the control (Apps, Plugins and Functions: Extensibility and Ecosystem Governance owns the app side of this).

Total Cost of Ownership: The Lines the Quote Leaves Out

Total cost of ownership (TCO) is everything a choice costs over its life, not what the contract says per month. Shopify's own replatforming guide (published 18 March 2026, vendor content) lists migration costs ("data migration, platform licensing, agency or partner fees, staff retraining, and downtime during transition"), ongoing costs ("hosting, security patches, maintenance, custom development, performance testing") and hidden costs ("peak-traffic surcharges, custom checkout development, additional feature licenses, content management overhead, and app or integration fees not in the original quote"). That is a good list, and it is a vendor's list: it omits the two lines a merchant pays most for. **The people who own the integrations after the agency leaves**, and **the exit**: re-exporting data, redirecting URLs, re-integrating everything that pointed at the old system.

The pricing models themselves shape the cost line (Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026 has the dated figures). A flat tier is cheap to start and forces upgrades on product or variant limits. A subscription with GMV thresholds grows with sales: Commerce.com's Scale plan charges 0.9% above $33,333 of monthly GMV, Shopware requires a paid plan above €1 million of GMV in a rolling year and prices "based on your gross merchandise value", Saleor Cloud charges 0.8% above $200,000 a month (all pricing pages accessed November 2026 and subject to change). An enterprise contract may add "a variable platform fee based on your revenue and business model", as Shopify's Plus documentation puts it. Payment fees are the platform's largest revenue line at Shopify and Lightspeed (Lightspeed reported transaction revenue of $815 million against subscription revenue of $371 million for the year to March 2026), and a merchant who brings its own payment provider pays a transaction fee of 0.2% to 2% on Shopify's tiers. **Pricing encodes the vendor's architecture bet**: GMV pricing assumes the vendor owns checkout; per-order pricing (warehouse and channel tools) assumes it sees every order; per-request pricing (search) assumes it sits in the request path. Peak days cost money on all three.

The lines, and who usually forgets them

TCO line	What it is	Who forgets it
Subscription or license	The monthly fee per tier or contract	Nobody; it is the only line everyone prices
GMV share and transaction fees	A percentage of sales, or a fee for using another PSP	The merchant, until sales double
Payment processing	PSP fees per transaction; separate from the platform's cut	Rarely forgotten, often assumed to be the platform's problem
Apps and specialist layers	Each app or service with its own fee, often per order or per request	The merchant: 41 apps at Kestrel & Pine, 6 unused
Agency or implementation partner	Build, then retainer	Priced for the build, not for the years after
Integration	Building and running every join; the iPaaS, the connectors, the scripts	Almost always: it is "included" in the build and then owned by nobody

TCO line	What it is	Who forgets it
People	The engineers and application specialists who own the systems and the joins	Treated as a fixed overhead rather than a cost of the option
Migration	Data, URLs, integrations, retraining, and the revenue at risk during cutover (Moving Without Burning the Bridge: Replatforming and Migration)	Underpriced by the vendor, who does not pay it
Exit	What it costs to leave this option in five years	Nearly always; it is nobody's line item until it is everyone's

Kestrel & Pine's three options over three years

The table prices the three options of the February 2027 review (the consultancy's rebuild, the suite Noor costed herself, and stay and surround) over 2027–2029. Every figure is a Kestrel & Pine illustration, built from the ledger's prices where they exist (the platform at €2,300 a month; the agency at €14,000 a month; technology spend of about €2.6 million in 2026) and from labeled assumptions where they do not. Costs common to all three options (the ERP at €3,900, the WMS at €2,100, the marketplace connector at €900, the returns portal at €650 plus €0.95 a return, the iPaaS at €1,200 a month, payment processing, and the existing team) are left out, because they do not change the comparison. The table shows **incremental cost over today's run-rate**, in thousands of euros.

Line (€ thousands, 2027–2029, illustrative)	**A. Composable rebuild**	**B. Enterprise suite**	**C. Stay and surround (W4)**
Platform and new service subscriptions	480 *(assumption: commerce engine with a GMV cap, OMS, PIM, search; after go-live)*	1,350 *(assumption: 1.5% of about €30M online and marketplace sales; the trade describes suite pricing as 1–3% of GMV, which no suite vendor states publicly)*	190 *(assumption: OMS with availability module €4,500 and PIM €1,800 a month from mid-2027; the platform's €2,300 continues unchanged)*
One-time build, implementation and migration	1,200 *(assumption: storefront and B2B portal rebuilt, 26,000 variants and order history migrated, 34 integrations rewired)*	900 *(assumption: partner implementation, data migration, integrations)*	580 *(OMS €320, PIM €110, integration standards and reconciliation €150; inside the €1.1M Waypoint budget)*
Agency or partner retainer, incremental	216 *(assumption: retainer rises from €14,000 to €20,000 a month, because a headless storefront needs continuous development)*	144 *(assumption: an implementation partner's retainer of €18,000 a month replaces the agency's €14,000)*	0 *(the agency's €14,000 continues)*
Added people	765 *(assumption: three engineers for storefront and platform at €85,000 loaded)*	255 *(assumption: one engineer)*	510 *(two integration engineers, the rest of the Waypoint budget)*

Line (€ thousands, 2027–2029, illustrative)	A. Composable rebuild	B. Enterprise suite	C. Stay and surround (W4)
Revenue at risk during cutover	540 *(assumption: an 8% fall in online sales for one quarter, the Marks & Spencer figure of 2014, on €27M of D2C revenue)*	540 *(same assumption)*	about 50 *(phased introductions beside a running platform; each step reversible)*
Three-year incremental total	**about 3,200**	**about 3,190**	**about 1,330**
Exit cost at the 2029 review	High: a merchant-built storefront and every join are bespoke	High: GMV contract, suite data model, partner dependency	Moderate: the platform's export plus the OMS and PIM, each with its own export, remain

Three things stand out, and they generalize beyond the fictional numbers. **The subscription is the smallest line in every column** except the suite's, where GMV pricing turns it into the largest. **People and the agency are the largest lines** in the rebuild, and they persist after the build, which is why an eighteen-month project is a seven-year commitment. **Revenue at risk is a cost**, and the only option that avoids it is the one that never cuts over.

Today's €2.6 million makes the same point from the other side. The subscriptions in the ledger add up to roughly €200,000 a year and the agency to €168,000; the rest is people, hardware, hosting and smaller tools. A platform decision that argues about €2,300 a month is arguing about a few percent of the spend.

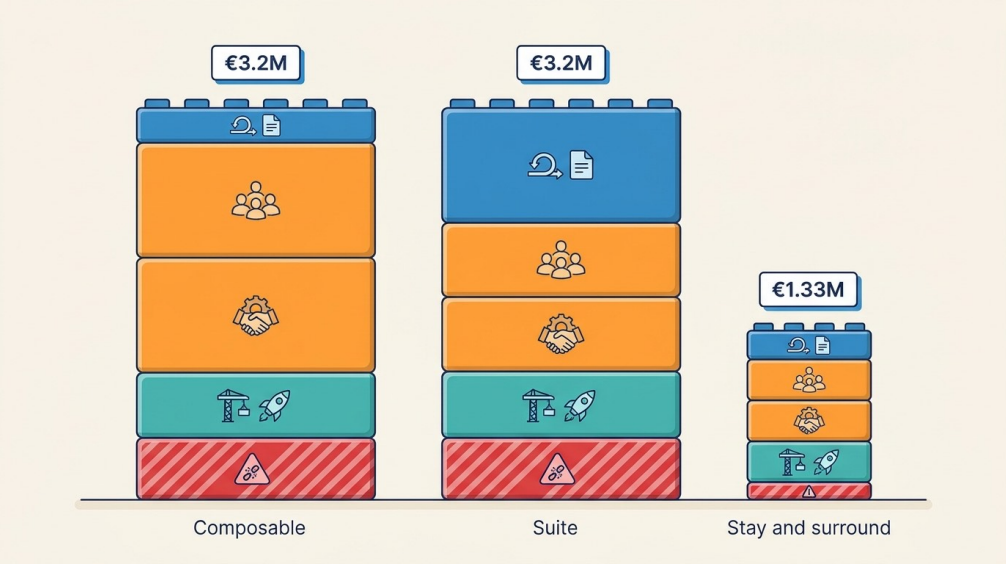


Figure 3: *Over three years the composable rebuild and the enterprise suite each add about €3.2 million against about €1.33 million for stay and surround; the gap is people, agency and revenue at risk, not subscriptions.*

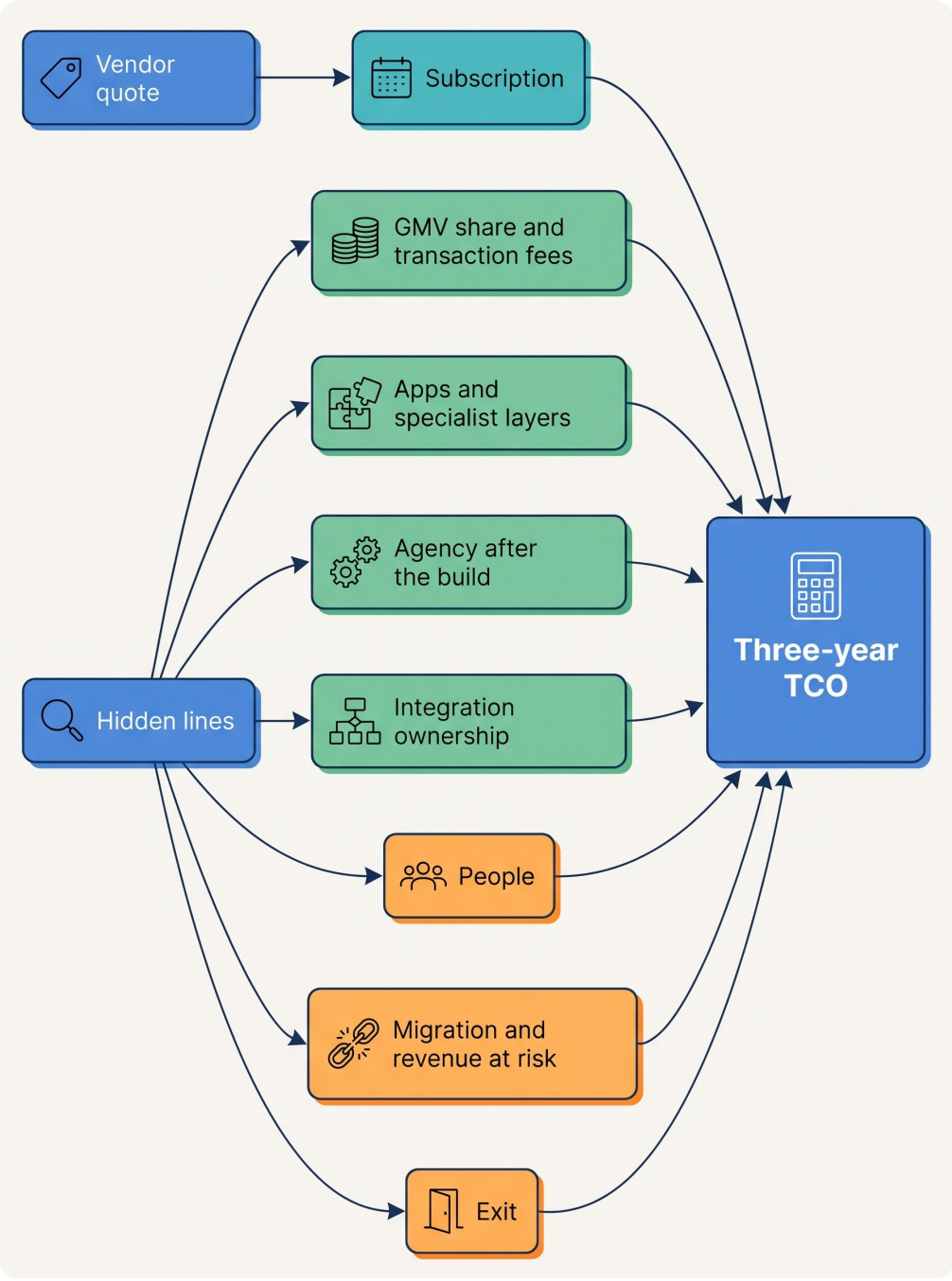


Figure 4: *Total cost of ownership: the quote prices one line; seven others, led by people, agency and exit, decide the total.*

Lock-In and the Cost of Leaving

Vendor lock-in is the condition in which leaving a vendor costs more than staying, whatever the vendor does. It is not a moral failing of vendors; it is the natural result of data, URLs, customizations and integrations accumulating in one place. **Exit cost** is the price of leaving, and it is designed in at purchase or never.

What the platform promises matters here. Shopify's standard Terms of Service (updated 20 October 2026) provide the services "on an 'as is' and 'as available' basis without any warranty", state that Shopify "does not warrant that the Services will be uninterrupted, timely, secure, or error-free", and reserve "the right to modify the Services or any part thereof for any reason, without notice and at any time". Enterprise agreements may add service levels; none is public. Read your own contract, not the marketing uptime.

Regulation now reaches part of this. The EU **Data Act** (Regulation (EU) 2023/2854, applicable from 12 September 2025) requires providers of "data processing services" to enable customers to switch to another provider or to their own infrastructure, to set out switching rights in the written contract, to allow at least 30 days to retrieve data, and, from 12 January 2027, to charge no switching fees at all. Its definition of a data processing service is broad enough to cover SaaS on its face, but whether a hosted commerce platform must therefore offer free switching from 2027 is an open legal question with no guidance or case found as of November 2026; services custom-built for one customer are exempt. The rules point at SaaS platforms; the first cases will tell (Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation).

Whatever the law settles, the exit questions are the same, and you ask them before signing:

- **Data**. Can products, variants, prices, customers, orders, gift-card balances and store credit be exported in full, in a documented format, by you, without a ticket? Password hashes usually cannot move; plan for a reset.
- **URLs**. Do you control the domain and can you emit redirects from every old product and category URL? Marks & Spencer's 2014 relaunch is the cautionary case in Moving Without Burning the Bridge: Replatforming and Migration.
- **Customizations**. Who owns the theme code, the functions, the scripts? If the agency does, the agency is part of the exit.
- **Apps**. Each app holds data (subscriptions, loyalty points, reviews) with its own export, or none.
- **Integrations**. Every connector that points at the platform must be re-pointed; the integration inventory (APIs, Events, Files and Middleware: How Commerce Systems Talk) is the exit's work list.
- **Contract**. Term, notice, GMV floor, what happens to the fee if you move half your traffic away during a strangler migration.

The Lynchpin is lock-in by another name: a capability customers depend on, running through one vendor, one connector, one agency or one person, with no exit plan. Kestrel & Pine had three in January 2027: the EDI consultant, the agency's scripts, and the ERP partner's platform-to-ERP connector. None was a platform decision; all were consequences of one.

Decision Criteria

Five criteria decide more than any feature list. They are the first page of the platform decision card in the Toolkit: Cards and Templates for Commerce Architecture Decisions.

Criterion	What to ask	What it pushes toward
Business model	D2C only, or wholesale, stores and marketplaces too? (Who Sells What to Whom: Business Models and the Economics That Shape Architecture)	A single channel fits a hosted platform; several channels with several stock locations need an OMS whatever the core
Scale stage	Starting, Growing, Omnichannel or Group? (What Changes at Each Scale: From First Store to Group)	Starting and Growing: hosted or ERP-centered; Omnichannel: surround the core; Group: suite versus composable becomes a real question
Team	How many engineers, and do they own integrations today?	A composable assembly needs a team that owns every join; fourteen people with an agency do not have one
Differentiation	What would a customer notice if you did it better than anyone?	Build only that; buy the rest
Time and risk	When is the next peak, and what is the cost of a failed cutover?	Phased introductions over big-bang replacements

Trade-Offs

	Suite	**Hosted platform plus apps**	**Composable assembly**	**Custom build**	**Specialist group**
Time to live	9–18 months with a partner *(industry practice)*	Weeks to months	12–24 months *(industry practice)*	Years	Months per label
Integration burden on the merchant	The joins to ERP, WMS, carriers	The joins, plus governance of apps	All of it	All of it	Reduced inside the group; the joins to everything else remain
Upgrade burden	Vendor, if the merchant has not forked the core	Vendor	Per service, continuous	Merchant	Vendor per label
Flexibility	High within the suite's model	Bounded by the platform's extension points	Very high	Unlimited	Bounded per label
Cost shape	GMV-linked, large partner line	Subscription plus payment share plus apps	Subscriptions with GMV caps, large people line	People	Several subscriptions
Exit	Hard: data model and partner	Moderate: export plus repointing apps and integrations	Hard: everything bespoke	Hardest	Per label, but one owner across layers

	Suite	Hosted platform plus apps	Composable assembly	Custom build	Specialist group
Fits when	Group stage with several brands and a platform team; B2B-heavy models	Starting through Omnichannel when checkout is not the differentiator	Group stage with a team that owns joins, or a vendor	The platform is the product	Growing and Omnichannel merchants in the group's market

My recommendation follows the requirements, so it comes with conditions. **If checkout and storefront are commodities for you, keep a hosted platform and surround it**, buying an OMS, a PIM and a WMS when the business model demands them. **If you have several brands, a platform team and a B2B-heavy model, evaluate a suite and a composable assembly against each other on the people line, not the feature list. Build only a capability that customers choose you for and nobody sells**, and even then, build it beside the bought core. The condition that changes all of this is the team: an assembly is a commitment to own every join forever, and a merchant that cannot name the owner of its integrations today should not sign up to own thirty more.

Across Scale Stages: When It Fits and When It Limits

Stage	Typical choice	What becomes necessary	Warning signs the choice is outgrown
1 — Starting	A hosted platform or a website builder with commerce; a 3PL; the platform's payments	Nothing beyond the platform's export and your own domain	Variant or product limits on the tier; a second channel keyed in by hand

Stage	Typical choice	What becomes necessary	Warning signs the choice is outgrown
2 — Growing	Hosted platform or ERP-centered package; an agency; first marketplace tool, accounting or ERP link, 3PL feed	The integration inventory; a named owner for each connector; the exit questions asked at every new contract	Apps writing stock; the agency owns every script; pricing tier thresholds crossed (platform GMV, PSP volume, per-order quotas) at the same time
3 — Omnichannel	A hosted platform or suite surrounded by OMS, PIM, WMS, returns and channel tools; an iPaaS; in-house integration engineers	Ownership per fact (Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record); the hubs bought, the joins owned; a TCO with the people and exit lines	Replatforming proposed to fix problems that live in the joins; the same incident recurring across channels
4 — Group	Suite or composable assembly with a platform team; shared services decided brand by brand	The multi-brand platform decision (What Changes at Each Scale: From First Store to Group); vendor-scale concerns such as tenancy and extensibility	One stack forced on brands before the benefit is measured, or brands kept apart that should share

The judgement across the stages: **the right time to replatform is almost never the time the problems appear, because the problems appear in the joins**. Kestrel & Pine's stage-3 incidents (stock, orders, prices, products) were all ownership problems,

and ownership is decided by contracts and hubs, not by the storefront. A platform change becomes rational at stage 4, when several brands or a vendor-scale concern (tenancy, extensibility, a team that can own an assembly) makes the core itself the limit.

In Practice

The vendors' own turn, 2025–2026. The clearest public evidence that layers beat replacement for mid-sized merchants comes from the vendors who sell replacement. commercetools, the founding MACH vendor, cut about 10% of staff on 26 February 2025 after missing growth targets, then launched AgenticLift on 21 January 2026 as a layer for "your existing commerce solution". Adobe sells Commerce Optimizer beside a kept backend (documentation of 21 October 2026). Commerce.com reported on 6 August 2026 that revenue was up 0.1% and that B2C replatforming "remains softer than historically". The MACH Alliance's own survey of 600 technology decision-makers (18 February 2026) found 37% citing integration complexity as a primary concern, a finding against the surveyor's interest and the cost line the composable promise omitted.

Vendors retreat from what they do not do well. Shopify completed the sale of the majority of Shopify Logistics to Flexport on 6 June 2023 for a 13% equity stake, more than three years ago but still the point: expect your platform vendor to be a payments company first and to push logistics, returns and EDI to partners, which keeps those layers' contracts in your hands at every stage above Starting. Lightspeed's strategic review, closed on 6 February 2025, named North American retail and European hospitality as its growth engines; read a vendor's strategic review as a signal about which products will be funded.

Reading a vendor claim. Shopify's enterprise page (accessed November 2026) claims a "33% lower average total cost of ownership" for a surveyed sample of organizations with GMV from $50 million to $500 million. It may be true for the sample; it says nothing about your people line or your exit. Ask for the unit, the date, the definition and the baseline before comparing any two vendors (Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026).

A build decision from the public record. On 7 August 2009 Target announced it would "build and manage its own online platform", taking back e-commerce technology, fulfillment and guest services from Amazon, which had run Target.com since 2001. The platform launched in 2011 and crashed on the Missoni collection launch of 13 September 2011; the cause was never confirmed publicly, and the sources are fifteen years old. Moving Without Burning the Bridge: Replatforming and Migration takes the migration side; the build-or-buy lesson is narrower: a retailer of Target's size had reasons to own its platform (a competitor ran it), and the cost of the decision was paid not in the build but in the first peak.

Kestrel & Pine's decision record. Noor wrote W4 as an architecture decision record (ADR), the one-page format Michael Nygard described in November 2011: context as

neutral facts in tension, the decision as "we will", status, consequences. Hers read, in short: *Context: the platform held at peak; the joins failed; the team is fourteen with an agency; nothing in checkout differentiates us; a composable rebuild and a suite each cost about €3.2 million over three years with a quarter of online revenue at risk. Decision: we will stay on the hosted platform and add an OMS with an availability module, a PIM and integration standards beside it; we will evaluate the platform's POS in 2028 and revisit this decision in 2029. Consequences: we accept the platform's extension limits and its transaction fees; we own the joins; the agency's scope narrows to the storefront; the exit from the platform remains possible because the hubs we add are systems of record with their own exports.* The record is the artifact: in 2029 the review starts from it rather than from memory.

When It Goes Wrong

The platform decision fails in slow ways, and the detection is organizational before it is technical.

- **The Lynchpin surfaces at renewal**. The agency owns the theme, the scripts and the knowledge; the renewal arrives with a price. *Detect:* no engineer at the merchant can deploy the storefront. *Recover:* ownership of code and credentials in the contract; a second party able to deploy; the ownership map from Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **The Forked Platform turns buy into build**. An upgrade is skipped because a customization would break; two years later the platform is three versions behind and the vendor's support has lapsed. *Detect:* the last upgrade date, and the list of core files touched. *Recover:* move customizations to supported extension points, one per release; if that is impossible, the real decision is whether to rebuild on the current version or leave.
- **The Giant's Boots are bought with a brochure**. A composable assembly is signed because a far larger retailer has one. *Detect:* the proposal names no owner for the joins after go-live. *Recover:* price the people line; if the merchant cannot staff it, do not sign.
- **The pricing cliff**. Growth crosses a GMV threshold, a per-order quota and a PSP volume tier in the same year; the subscription that was the smallest line doubles. *Detect:* the thresholds written next to the forecast. *Recover:* negotiate at the stage change, not after the invoice.
- **The vendor changes direction**. A strategic review, an acquisition, a retired product (Zalando's Connected Retail technology will be discontinued by 30 June 2027, announced April 2026, with fourteen months' notice; Magento 1's end of life on 30 June 2020 forced merchants to move or run unsupported software). *Detect:* read the vendor's investor and press pages twice a year. *Recover:* the exit questions above, answered before they are urgent.

- **The quote was right and the TCO was wrong.** Integration is "included" in the build and then owned by nobody; incidents appear in the joins a year later. *Detect:* the integration inventory has owners missing. *Recover:* W5: every integration has an owner, a contract, idempotent handling and a reconciliation job.

What to Take Away

Established.

- The visible subscription is a minority of total cost; vendors' own guides (Shopify, March 2026) list migration, ongoing and hidden costs, and pricing models with GMV thresholds and transaction fees are documented on vendor pages as of November 2026.
- The composable vendors and suites themselves sell layers beside an existing core as of 2026 (commercetools AgenticLift, January 2026; Adobe Commerce Optimizer, documented October 2026), and the MACH Alliance's own 2026 survey names integration complexity as the top concern.
- The EU Data Act (Regulation (EU) 2023/2854) applies from 12 September 2025 and prohibits switching charges from 12 January 2027; whether it covers hosted commerce platforms is unsettled.
- Decision records (Nygard, 2011) are the lightweight way to keep a platform decision reviewable.

Depends on context.

- Stay and surround is right when checkout and storefront are commodities for the merchant and the problems live in the joins; it is wrong when the core itself limits the business (several brands, vendor-scale tenancy or extensibility needs, a B2B model the platform cannot express).
- A composable assembly fits a team that owns every join and a business whose differentiation lives in the assembly; it is the Giant's Boots for a fourteen-person team with an agency.
- GMV-based pricing aligns the vendor with growth and raises exit cost; it favors high-margin D2C and punishes large, low-margin B2B orders, which is why B2B terms are negotiated separately.
- The specialist-group model reduces joins for merchants in its market and concentrates several layers under one owner; whether that is a benefit depends on the exit terms per label.

The author's synthesis.

- I price every platform option on three lines first, people, agency and exit, and only then on the subscription, because the first three decide the total and the last one decides the meeting.
- I treat "build" as a decision about ownership rather than code: the thing worth building at a mid-sized merchant is the rules for who owns which fact and how the copies are reconciled, which no vendor sells.

Questions to Ask

1. What exactly is broken, and does it live in the core or in the joins between systems?
2. Which capability would a customer notice if we did it better than anyone? Is anyone proposing to build something else?
3. What does each option cost over three years including agency, people, migration, revenue at risk and exit?
4. Who owns each integration the day after go-live, by name?
5. Which pricing thresholds (GMV tier, per-order quota, PSP volume) will growth cross in the next two years, and what happens to the fee?
6. Can we export products, orders, customers and gift-card balances ourselves, in a documented format, today?
7. Who owns the theme code, the scripts and the credentials, and could a second party deploy them?
8. What does the vendor's contract actually promise about availability, deprecation notice and modification without notice?
9. Is there a decision record, and does it name the date and the conditions under which the decision is revisited?

The decision to stay is only half a decision. Kestrel & Pine still has to introduce an OMS and a PIM beside a running platform without creating the very duplicates and gaps the hubs exist to remove. How to move systems without a single cutover date is the subject of Moving Without Burning the Bridge: Replatforming and Migration.

Sources and Evidence

Company, product and investor sources

- Ecommerce Replatforming and Migration Guide for 2026, Shopify Enterprise blog (Michael Metcalf), 2026-03-18; the vendor's list of migration, ongoing and hidden costs (vendor content).
- Shopify for Enterprise, Shopify, n.d., accessed 2026-11; the "33% lower TCO" and "20% faster migrations" claims and the $50M–$500M GMV survey sample.

- Shopify Plus plan and Shopify pricing (EUR), Shopify, n.d., accessed 2026-11; Plus terms, the variable platform fee, third-party transaction fees of 0.2–2%.
- Shopify Terms of Service, Shopify, 2026-10-20; "as is", no warranty of uninterrupted service, modification without notice.
- Shopify completes sale of Shopify Logistics to Flexport, Shopify, 2023-06-06; the 13% equity stake and the retreat from logistics.
- BigCommerce pricing, Commerce.com, n.d., accessed 2026-11; the 0.9% overage above $33,333 monthly GMV.
- Commerce.com Q2 2026 results, exhibit 99.1, SEC EDGAR, 2026-08-06; revenue up 0.1% and the softer B2C replatforming remark.
- Shopware pricing, Shopware, n.d., accessed 2026-11; GMV-based plans and the €1 million fair-use line.
- Saleor pricing, Saleor, n.d., accessed 2026-11; the 0.8% charge above $200,000 monthly GMV.
- Lightspeed Q4 and FY2026 results, Lightspeed, 2026-05-21, and results of strategic review, 2025-02-06; transaction versus subscription revenue, and the chosen growth engines.
- commercetools launches AgenticLift, commercetools, 2026-01-21; a layer "on top of your existing commerce solution".
- commercetools, a pioneer in headless commerce, lays off dozens of staff, TechCrunch, 2025-02-26; the February 2025 cut and the missed growth targets.
- Adobe Commerce platforms and services, Adobe Experience League, 2026-10-21; Commerce Optimizer "alongside an existing Adobe Commerce or third-party commerce backend".
- MACH Alliance research: 6x more organizations achieve AI ROI with a composable foundation, MACH Alliance, 2026-02-18; 37% cite integration complexity (commissioned survey).
- Katapull and Katapull acquires Picqer, Katapull, accessed 2026-11 and 2026-10-07; the eight-label group and its vendor-stated customer counts; Vijf softwarebedrijven bundelen krachten in Katapull, Dutch IT Channel, 2025; the 31 March 2025 formation.
- Get started with the Shopify Connector, Microsoft Learn, 2026-09-22; the connector preinstalled for new Business Central sign-ups.
- Target parts ways with Amazon.com, InternetNews, 2009-08-07, and Target's website crashes again, MPR News (AP), 2011-10-25; the build decision and the 2011 outages (evidence older than three years).
- Zalando consolidates partnership models, Connected Retail to be discontinued, FashionUnited, 2026-04-23; the 30 June 2027 discontinuation with notice.
- Magento 1 reaches EOL, Help Net Security, 2020; the forced migration (widely reported).

Standards, regulation and official documentation

- Regulation (EU) 2023/2854 (Data Act), EU Publications Office, 2023-12-22; switching rights, the 30-day retrieval period, switching charges prohibited from 12 January 2027, application from 12 September 2025, the custom-built exemption.

Research and practitioner sources

- Documenting Architecture Decisions, Michael Nygard, 2011-11-15; the ADR format used for W4.
- EPAM joins newly formed MACH Alliance, EPAM, 2020-06-24, and MACH principles, MACH Alliance, n.d., accessed 2026-11; the 2020 acronym and the 2026 three-principle reframing.
- Marks & Spencer Q1: general merchandise sales down, CNBC (Reuters), 2014-07-08; the 8.1% fall in M&S.com sales used as the revenue-at-risk assumption (evidence older than three years).

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

42. Moving Without Burning the Bridge: Replatforming and Migration

IN THIS CHAPTER, YOU WILL: Learn to plan a platform or hub migration as a sequence of reversible steps rather than one cutover date, with the data, integrations and rollback each handled on purpose.

KEY POINTS:

- **The documented failures were not feature failures.** Target's 2011 in-house platform went down at its first peak (the cause was never confirmed), Finish Line reverted in 2012 after seventeen days, Marks & Spencer lost 8.1% of online sales in the quarter after its 2014 relaunch. Load, performance, timing and customer habit broke them, not missing features, and one recovered only because the old platform was still running.
- **Migrate along seams, with a bridge that stays up.** The strangler approach moves one channel, one capability or one slice of traffic at

a time behind a router or a copy of the flow, runs old and new side by side with a reconciliation that proves they agree, and keeps the old system operable until the new one has met real load.
- **Data migration is five migrations**. Products (identifiers must not change), customers (consent moves, passwords do not), orders (history is read, not re-created), gift cards and credit (liabilities that must balance to the cent) and URLs (every old address redirected) each need their own plan and check.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Burned Bridge** — everything moves to a new platform on one date, with no way back; this chapter exists to prevent it.
- **The First Snowfall** — a migrated platform, or a newly introduced hub, meets real load for the first time on the busiest day of the year.
- **The Spaghetti Junction** — thirty-four integrations, each with its own owner or none, that all have to be re-pointed when the system in the middle changes.

Every platform is eventually replaced, and so is every hub beside it. The question is never *whether* a merchant will migrate something; it is whether the migration is a **sequence of steps that can each be undone** or **a date after which there is no way back**. The first kind is boring, which is the point. The second kind is the one that gets written up in the trade press, and this chapter opens with three of those write-ups.

A replatforming is the most visible migration, but it is not the most common. More often a merchant adds an order management system beside its platform, introduces a product information system, swaps a warehouse system, or changes payment provider. Each of these moves a fact's ownership from one system to another, and every integration that read the old owner has to learn about the new one. **Those are migrations too**, and they fail the same ways: a date, a big bang, a bridge burned behind.

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), decided in March 2027 not to replatform (Build, Buy or Assemble: Choosing and Changing Platforms). It still has two migrations in 2027: a PIM in the third quarter and an OMS by the fourth, each introduced beside a platform that must keep selling 800 orders a day while the ground moves under it. Those two are the worked example.

Where we are in the order's journey. A migration touches every step of Eva's jacket at once. During the OMS introduction, order #1072455 would be created in the platform, copied to the new hub, compared, and only later routed from the hub to the WMS; during the PIM introduction, the jacket's name, sizes and colors are published from a new owner. This chapter is about keeping her order whole while the systems around it change.

Setting the Scene: Two Hubs Beside a Running Shop

In May 2027 the first weekly stock reconciliation between the WMS and the platform found 2.3% of active variants with a different quantity, most left behind by the three apps that wrote stock. Noor Visser took it as the baseline for the migrations ahead: if two systems that have run side by side for four years disagree on one variant in forty, a new hub loaded from them will inherit every disagreement unless somebody reconciles first.

The agency's plan for the OMS, offered in April, was one paragraph: load the historical orders in a weekend, switch the platform's order export to the OMS on Monday, switch the WMS feed on Tuesday. The ERP partner's plan for the PIM was similar: export the platform's catalog, import it, re-point the storefronts. Both were cutover dates. Neither said what would happen if, on Tuesday, the WMS received an order twice, or no order at all.

Noor's plan, which the rest of this chapter follows, had three rules. **Every step is reversible for at least one peak**. **Old and new run side by side with a job that proves they agree before new takes over**. **The smallest channel moves first**. The marketplace channel, 43,000 orders a year, about 120 a day, was the first to be routed through the OMS; the D2C storefronts with 293,000 a year came after; the EDI and manual wholesale orders last, in December, with the stores deferred to the 2028 POS evaluation. Black Friday 2027 sat in a change freeze in the middle of it.

Reasons to Replatform, and Reasons That Are Not

A platform change is justified when the core itself limits the business and no layer beside it can fix that. It is not justified when the problems live in the joins, which, as Build, Buy or Assemble: Choosing and Changing Platforms argues, is where most Omnichannel-stage problems live.

Reason given	Is it a reason?	What to check first
"Stock is wrong across channels"	No: an ownership and availability problem (Is It in Stock? Inventory, Availability and the Promise You Can Keep)	Who is the system of record for stock, and how do the others learn about changes

Reason given	Is it a reason?	What to check first
"Support can't find an order"	No: an order-hub problem (The Order Record: Lifecycle, State and the Order Management System)	Whether an OMS beside the platform solves it
"The site was slow on Black Friday"	Maybe: find the bottleneck	Usually the integrations, not the platform (Black Friday Every Year: Peak Load, Resilience and Recovery)
"The vendor is ending the product"	Yes, on the vendor's date	The notice period; the export; Magento 1's end of life on 30 June 2020 forced merchants to move or run unsupported, non-compliant software
"We have customized the core until it cannot upgrade"	Yes, but the choice is rebuild-on-current-version versus leave	The Forked Platform (Apps, Plugins and Functions: Extensibility and Ecosystem Governance)
"The platform cannot express our B2B model"	Yes, if a B2B layer beside it cannot either	Price lists, contract prices, punchout, approvals (Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals)
"A competitor runs our platform"	Yes, strategically	Target, 2009: see below
"The agency says so"	No, by itself	Who is paid for the build, and who owns the joins after it (Who Builds and Who Owns: Teams, Vendors, Agencies and Governance)

Three Documented Failures, and What They Share

The public record of replatforming is thin, because merchants do not publish post-mortems. Three cases are documented well enough to use. All three are older than a decade, so they describe replaced systems; the lesson is in the shape, not the software.

Target, 2009–2011. Amazon had run Target.com since 2001. On 7 August 2009 Target announced it would "build and manage its own online platform", taking over its "e-commerce technology platform, fulfillment and guest services operations", to launch before the 2011 holiday season. The new site launched in August 2011. On 13 September 2011, weeks later, the launch of the Missoni for Target collection crashed Target.com; Target's own banner read "we are suddenly extremely popular", and the site was down most of the day. A second outage of about two and a half hours followed on 25 October 2011, for which Target gave no reason. Press at the time disagreed on whether the in-house platform or the Missoni demand was the cause, and Target never confirmed it, so the verified chain is the decision, the launch and the two outages, not the root cause. The shape is the First Snowfall on a new platform: the first real peak was the first real test.

Finish Line, 2012. The American footwear retailer switched to new e-commerce site technology on 19 November 2012, the week before Thanksgiving. Seventeen days later it reverted to its former platform, "which it had kept operable", according to Retail Dive's report of 8 January 2013. The switch cost about $3 million in quarterly sales, "despite a 25% increase in online sales for the quarter". Finish Line is the one case in this list with a happy ending, and the reason is in one clause: the old platform had been kept operable. The bridge was not burned.

Marks & Spencer, 2014. M&S launched a new website in early 2014. Its first-quarter trading statement of 8 July 2014 reported that M&S.com sales fell 8.1% in the quarter; the company had warned in May that the new site would take four to six months to "settle in", and said it had since seen "a gradual improvement in sales performance". Retail Week's commentary of 10 July 2014 called it "a timely reminder of the risks involved in major replatforming programmes". The shape here is different: nothing crashed. Customers had to re-register, the search engines had to re-learn the site, and habit broke. A migration can succeed technically and still cost a quarter.

The one that avoided it. Hobbycraft, a British crafts retailer, delayed its website relaunch past Christmas 2012 because it was "too risky to launch the website in peak trading" (Retail Week, 19 February 2013). That is a sourced example of the simplest rule in this chapter: never cut over into a peak.

Case	Date	What failed	What it cost	Was there a way back?
Target (US)	Launch August 2011; outages 13 September and 25 October 2011	Capacity at the first peak (cause not confirmed publicly)	A day of lost sales at a high-profile launch; a second outage; the Target.com president left weeks later	No public evidence of one
Finish Line (US)	Switched 19 November 2012; reverted 17 days later	Performance of the new site in the holiday run-up	About $3 million of quarterly sales	**Yes**: the former platform had been kept operable
Marks & Spencer (UK)	Relaunch early 2014; results reported 8 July 2014	Customer habit, search visibility, a four-to-six-month "settling in"	M&S.com sales down 8.1% in the quarter	Not reported; the company chose to settle in rather than revert
Hobbycraft (UK)	Relaunch deferred past Christmas 2012	Nothing: the launch was moved off the peak	A delay	Not needed

What the cases share is the pattern this book calls **The Burned Bridge**: everything moves on one date, into or near a peak, with the new system meeting real load for the first time in public, and with no rehearsed way back. Finish Line is the Burned Bridge without the burning, and it recovered. The rest of this chapter is the engineering of that clause, "kept operable", generalized.

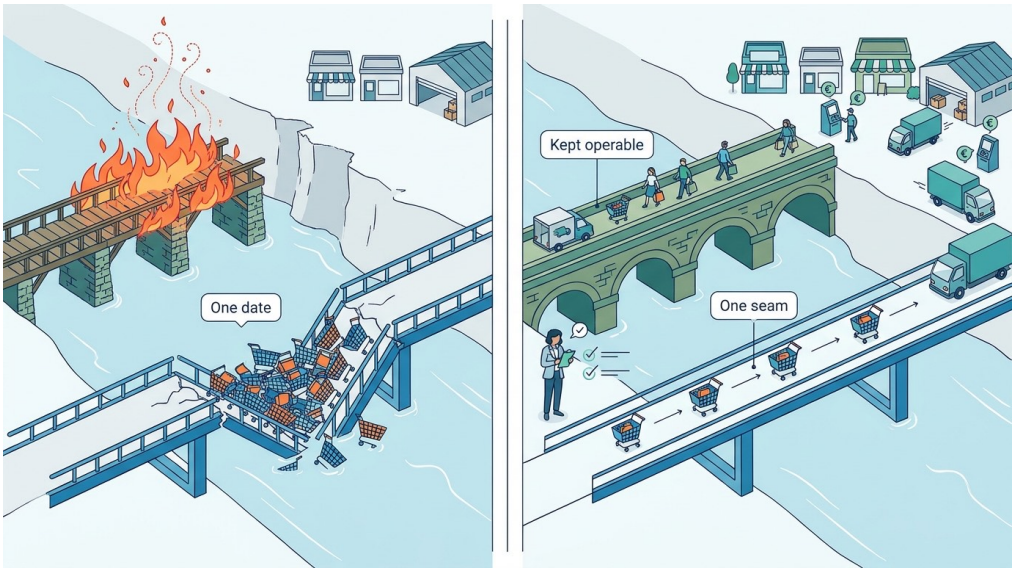


Figure 1: *Finish Line could turn back in 2012 because its old platform had been kept operable: a migration that keeps the old bridge standing until the new one has carried real load always has a way back.*

The Strangler Approach for Commerce

Martin Fowler's note on the **strangler fig** (first written in 2004, restated 22 August 2024) gives the alternative in one sentence: a gradual approach to modernizing a serious system, because plans to replace one wholesale tend to "go down in flames", replacement takes long, users cannot wait for features, and existing behavior is hard to pin down and partly unwanted. The name is from a fig that grows around a host tree until the tree is gone. In plain words: **put something in front of the old system, route one piece of its work to the new system, check, route the next piece, and switch the old system off only when nothing is routed to it any more**.

Thoughtworks' *Patterns of Legacy Displacement* (20 July 2021, revised 5 March 2024) names the moves. **Transitional architecture**: temporary components built only to make the migration safe, and removed afterwards. **Event interception**: capture the changes flowing into the old system and send a copy to the new one. **Divert the flow**: route a slice of traffic or a business flow to the new system. **Legacy mimic**: make the new system look like the old one to the integrations that still expect it. **Dark launching**: run the new path without showing its result. **Canary release**: a small share of traffic first. The authors warn that defining "as is" functionality "was a huge effort" and that the alternatives to seam-based decomposition "all too often result in Feature Parity and Big Bang releases". **Feature parity**, rebuilding everything the old system did before switching, is the plan that produces the Burned Bridge, because it postpones every risk

to one date.

In commerce the seams are visible and there are three kinds.

- **By channel.** Move the B2B portal first, or the German storefront, or the marketplace orders. Channels are natural slices because each has its own entry point and its own volume, and a merchant can choose the smallest.
- **By capability.** Move search first, or returns, or product content, or availability. This is the stay-and-surround program: each hub is a capability moved from the platform (or from spreadsheets and the ERP) to a new owner.
- **By traffic slice.** A share of sessions, a region, a customer segment. This is the canary, and it needs a router in front of the storefront.

The public example of all three is Zalando's rebuild of its Fashion Store from the monolith it called "Jimmy", described in its engineering posts of 16 October 2015 and 5 October 2016. A team put a Go-based router, Skipper, in front of the shop and migrated the page fragment by fragment: header and footer first, then product pages and wish lists, each served by a new team behind the router while the monolith served the rest. The evidence is ten years old and describes a system that has changed since, but the shape is the strangler in production at a large retailer. The same company's wind-down of its Connected Retail technology, announced 23 April 2026 for discontinuation by 30 June 2027, shows the other end: a channel retired with fourteen months' notice rather than switched off.

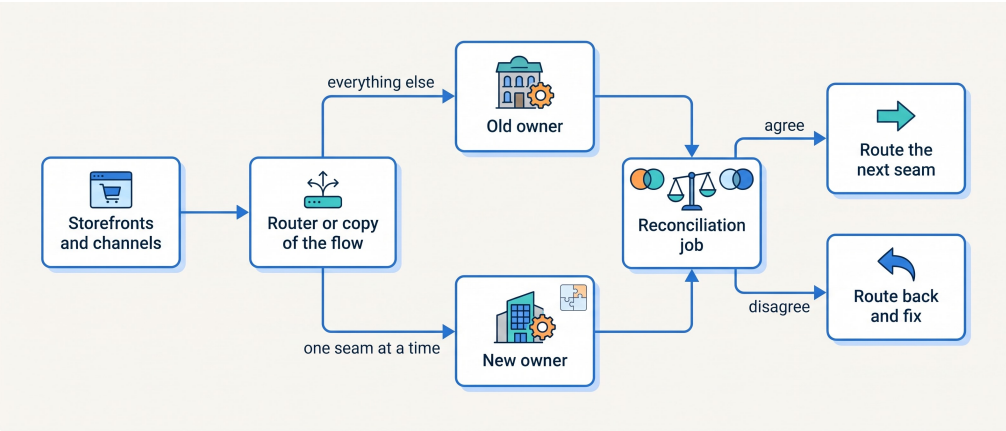


Figure 2: *The strangler in commerce: a router or a copied flow sends one seam at a time to the new owner, a reconciliation job proves old and new agree, and disagreement routes back instead of forward.*

How the Pieces Fit During a Migration

A migration is a period in which **two systems hold the same fact and only one of them is allowed to be right**. The design question is which one, for which fact, for how long, and how the others find out. The requirements change by phase.

Phase	Who owns the fact	Freshness	Consistency check	Latency allowed	What must stay available
Shadow (new system receives a copy; nobody reads it)	Old	Copy within minutes *(assumption)*	Daily reconciliation of counts and samples	Any	Everything on the old path, untouched
Parallel run (both compute; old result is used)	Old	Both within the same minute	Every record compared; differences listed by cause	Any for the new path	Old path; new path may fail silently
Divert one seam (new result used for one channel or capability)	New, for that seam only	Live	Reconciliation per seam; the old path still receives the copy	Production latency for that seam	The old path can take the seam back within an hour *(assumption)*

Phase	Who owns the fact	Freshness	Consistency check	Latency allowed	What must stay available
Divert all	New	Live	Reconciliation continues for one peak	Production	The old system stays operable, read-only or idle, until the peak has passed
Decommission	New	—	Final reconciliation; archive	—	Exports from the old system kept

Two mechanisms carry these phases, and they are easy to confuse.

A parallel run means both systems process the same input and their results are compared, while only one result is acted on. It is the safest way to learn whether the new system agrees with the old one, and it is cheap when the input can be copied (an order created, a product changed). Its weakness is that it finds disagreements, not their causes; the reconciliation job must say *which* records differ and *why* (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation).

Dual writes mean one actor writes the same change to both systems. They sound like a parallel run and are the opposite in risk: two writers to two stores, with no transaction across them, produce exactly the partial failures this book spends Part VI on. If the write to the new system fails and the write to the old one succeeds, the systems diverge and nobody is told. Kestrel & Pine already lived this in March 2026, when a pre-order app and the WMS feed both wrote stock in the platform and left two weeks of phantom reservations on 600 variants. **Prefer one writer and a copied flow** (event interception: the change goes to the owner, and an event carries a copy to the other) over two writers, and where dual writes are unavoidable make them idempotent (a repeated write has the same effect as one) and reconciled (APIs, Events, Files and Middleware: How Commerce Systems Talk).

Data Migration Is Five Migrations

A platform's data is not one export. Each kind of data has its own owner, its own identifiers, its own legal constraints and its own way of going wrong. Plan them

separately.

Data	What moves	The trap	The check
Products and variants	3,400 products, 26,000 variants, attributes, media, translations, categories	Identifiers that change: a SKU or GTIN re-keyed in the new system breaks every integration, feed and barcode (What Are We Selling? Products, Variants and the Catalog)	Every variant in the new system carries the same SKU and GTIN; counts per status match; a sample of fifty variants compared field by field
Prices and promotions	Price lists per market and customer group, active promotions, effective dates	A price-list sync that fails silently, as it did for nine days in October 2026 (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax)	Every active SKU's consumer and wholesale price equal in old and new; a report of differences before go-live
Customers	Accounts, addresses, consent, company accounts and price lists for B2B	Passwords are stored as one-way hashes and usually cannot move; consent must move with its proof; a deletion request mid-migration must reach both systems (Whose Data Is It? Privacy, Consent and Retention)	A password reset flow tested before go-live; consent flags counted; the identity map kept (One Customer, One Product: Identity and Master Data Across Systems)

Data	What moves	The trap	The check
Orders	History for support and returns; open orders in flight	Re-creating history fires every side effect again: notifications, stock reservations, invoices. Orders are read, not replayed (The Order Record: Lifecycle, State and the Order Management System)	History imported read-only with its original numbers; open orders listed and tracked by hand through the cutover window
Gift cards, store credit, loyalty points	Balances, which are liabilities on the balance sheet (Discounts, Coupons and Loyalty: Promotions Without Chaos)	A balance lost is money owed to a customer with no record; a balance duplicated is money given away twice	Sum of balances in old equals sum in new to the cent, signed off by finance
URLs and search presence	Every product, category and content address	A redirect map that is incomplete loses rankings and customers who have bookmarked a page; the M&S quarter	Every old URL returns a redirect to its new equivalent; crawl the old sitemap after go-live

Data	What moves	The trap	The check
Integrations and apps	The 34 flows that read or write the old owner	Each one points at the old endpoint until somebody changes it: the Deprecated Endpoint of February 2027 in reverse (Promises Between Systems: API Contracts, Webhooks and Versioning)	The integration inventory with an owner and a cutover task per flow; a legacy mimic where a flow cannot change in time

Figure 3: *A platform's data is not one export: products, customers, orders, gift cards and URLs each move separately, with their own trap and their own check.*

The order history deserves one more sentence. It is tempting to "migrate the orders" by creating them in the new system. Every order created in a commerce system fires something: a confirmation e-mail, a reservation, a fulfillment request, an invoice. Importing 293,000 D2C orders as new orders is 293,000 e-mails. **History is imported as history**, flagged, read-only, with its original numbers and dates, or it is left in the old system behind a read-only link for support.

Cutover and Rollback

A **cutover** is the moment a seam's traffic is routed to the new owner. Even in a strangler migration there are cutovers; they are just small. Each one needs the same five things.

1. **A window and a freeze.** A time with low volume (not a Friday evening in November), a change freeze around it, and a stop on competing changes to the integrations.
2. **Go and no-go criteria written down beforehand.** The reconciliation shows fewer than an agreed number of differences; the parallel run has lasted an agreed period; the rollback has been rehearsed.
3. **A rehearsed rollback.** Not a plan: a rehearsal, timed, in which the seam is routed back and the records created in the new system during the window are reconciled into the old one. Finish Line's "kept operable" is the minimum; a rehearsal is the standard.
4. **Continuity of identifiers.** The order number family does not restart; the last order number in the old system and the first in the new are recorded; a SKU is the same SKU.
5. **A watch.** Orders per minute, failed messages, reconciliation differences and support tickets watched for the first hours and the first peak (Black Friday Every Year: Peak Load, Resilience and Recovery).

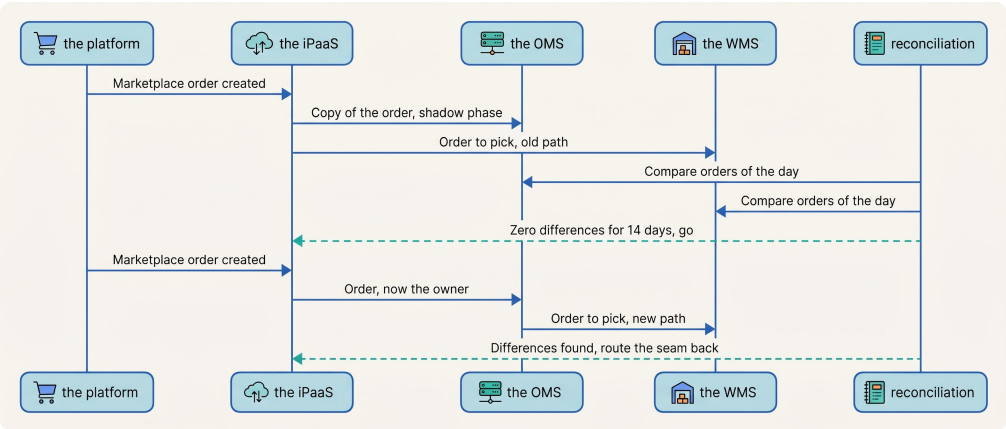


Figure 4: *One seam's cutover at Kestrel & Pine: marketplace orders are copied to the OMS in shadow, reconciled against the WMS for two weeks, then routed through the OMS; a difference routes the seam back rather than forward.*

The rollback has a limit that must be stated: **the longer the new system owns a seam, the more records exist only there.** After a week of the OMS owning marketplace

orders, routing back means reconciling a week of orders into the platform's view. The parallel run narrows that gap, because the old path still receives the copy, but it does not close it. That is why the old system stays operable through one peak and not forever: the window for a cheap rollback closes on its own.

Migrating the Integrations

The system in the middle changes; the thirty-four flows around it do not know. At Kestrel & Pine fourteen of the thirty-four touch orders, stock or product data and have to be re-pointed when the OMS and the PIM arrive (an illustrative count). Three rules, all from Waypoint W5, make that a list of tasks rather than a season of incidents.

- **The inventory first.** Every integration listed with its owner, its direction, its fields, its frequency and the system of record it reads or writes (APIs, Events, Files and Middleware: How Commerce Systems Talk). A flow with no owner gets one before the migration starts, or it gets switched off.
- **A legacy mimic where a flow cannot change in time.** If the agency's script that reads orders from the platform cannot be rewritten before the OMS takes over, the OMS (or the iPaaS) exposes the same shape at the old address for one more season. The mimic is transitional architecture: it has a removal date.
- **Deprecation with notice, in both directions.** The merchant retires its own old endpoints with a published date, as the marketplace connector did in February 2027, and this time the agency script is on the list (Promises Between Systems: API Contracts, Webhooks and Versioning).

Across Scale Stages: When It Fits and When It Limits

Stage	What a migration usually is	What becomes necessary	Warning signs
1 — Starting	Moving from a website builder to a hosted platform; changing 3PL	The platform's export and import; the domain in the merchant's name; a redirect map	The old URLs die; the customer list moves without its consent proof

Stage	What a migration usually is	What becomes necessary	Warning signs
2 — Growing	A first ERP or accounting link; a marketplace tool; a new PSP	A cutover window outside the peak; the integration inventory, because the first connectors are the ones that break	A cutover planned for November; connectors re-pointed by an agency with no owner afterwards
3 — Omnichannel	Hubs introduced beside the platform: OMS, PIM, WMS, availability; a channel re-routed	Strangler seams by channel and capability; parallel runs with reconciliation; the old owner kept operable through one peak; data migrated as five migrations	One weekend planned for an OMS; dual writes to stock; order history "migrated" by re-creating orders
4 — Group	Replatforming a brand; consolidating brands onto one stack; an acquisition's systems	Brand-by-brand migration order (What Changes at Each Scale: From First Store to Group); a platform team that owns the router and the transitional architecture; feature parity resisted per brand	Every brand moved on one date; one brand's customizations rebuilt for all

The judgement: **the migration technique does not change with scale, but the thing being migrated does**. A Starting-stage merchant moves a shop; an Omnichannel merchant moves ownership of a fact; a Group moves brands. The strangler, the parallel run and the kept-operable old system apply at every stage; what grows is the number of seams and the number of integrations each seam drags along.

In Practice: The OMS and PIM Introductions at Kestrel & Pine

The figures below are Kestrel & Pine illustrations; the dates fit the ledger's Waypoint timeline (PIM in Q3 2027, OMS by Q4 2027).

The PIM, July to September 2027. In July the PIM was loaded from the three existing copies of product data (the platform, the ERP item master and the shared spreadsheets) plus the marketplace connector's fourth copy. The load was the reconciliation: it found 1,900 variants whose attributes disagreed across copies (sizes named differently, a color "moss" spelled three ways, translations missing in German), and Teun Baas's catalog team resolved them in the PIM before anything was published. The SKU and GTIN were carried over unchanged for every variant. In August the German storefront, the smallest, became the first consumer: the PIM published to it on a schedule while the platform kept its own copy as a fallback; a nightly job compared the two. In September the NL and BE storefronts and the connector's listings followed; the spreadsheets were made read-only. Rollback at every step was to stop publishing and let the platform keep its last copy. Nothing was cut over in a weekend, because nothing needed to be.

The OMS, August to December 2027. The phases follow the table in the previous sections.

Phase	Dates	What happened	Reversible by
Shadow	August	Every order from every channel copied to the OMS through the iPaaS; nobody reads it; the WMS feed unchanged	Switching the copy off

Phase	Dates	What happened	Reversible by
Parallel run	1–14 September	The OMS computes routing and status for every order; the reconciliation compares its view with the platform's, the connector's and the WMS's each night; differences listed by cause (a timing gap, a status mapping, a true error)	Nothing to reverse
Marketplace seam	15 September	Marketplace orders (about 120 a day) routed through the OMS to the WMS; the old path kept as the copy; OMS order numbers continue the platform's sequence, with the marketplace's own order id kept as a reference (The Order Record: Lifecycle, State and the Order Management System)	Routing the seam back; a rehearsed one-hour procedure

Phase	Dates	What happened	Reversible by
D2C seam	13 October	Storefront orders routed through the OMS to the WMS; the platform keeps checkout and the storefront order history, as W2 says	Routing the seam back within the day
Change freeze	15 November – 5 December	Nothing changes; the OMS meets Black Friday week (26 November 2027) with the old path still receiving copies and the WMS integration able to take the old path back	The freeze itself
Wholesale seam	8 December	EDI orders from Bergzicht and manual wholesale orders created in the OMS; the ERP receives them from there for invoicing	Routing back to the ERP's order entry
Decommission the old order paths	January 2028	The platform-to-WMS and connector-to-WMS flows retired; exports archived; the stores deferred to the 2028 POS evaluation	Not needed; the peak has passed

The reconciliation's first fortnight is the part worth reading twice. The parallel run

found differences on about 1.4% of orders in its first week (illustrative), nearly all from two causes: a status mapping (the WMS's "packed" had no equivalent in the platform) and a timing gap (orders created in the last minutes before the nightly comparison). The one true error was an order the connector had delivered twice to the iPaaS, which the old path had fulfilled twice in the past and the OMS, with idempotent handling, created once. **The migration found an existing defect before it caused a new one**, which is the parallel run's quiet payoff.

The order history. The 2026 orders were imported into the OMS as history: original numbers, dates and statuses, flagged as imported, with no side effects fired. The platform kept its own order history for the storefront's account pages, as W2 requires. Support looked in one place from October.

When It Goes Wrong

- **Duplicates across the seam**. During the window, an order is created on the old path and its copy on the new, and both reach the WMS. *Detect:* the reconciliation's "in both" list; a customer with two parcels. *Recover:* idempotency on the WMS inbound keyed on the order number; the duplicate canceled before picking; the cause (a retry at the router) fixed before the next seam.
- **The gap**. An order created in the minutes between the last copy and the cutover reaches neither system. *Detect:* the count of orders in the window against the platform's own count. *Recover:* a replay from the platform's order export for the window; one more reason the old owner must stay readable.
- **Stale copies downstream**. The search index, a cache or the data warehouse still reads the old owner and shows the old product name for a day. *Detect:* the consumers listed in the inventory with their refresh cadence. *Recover:* every consumer re-pointed and refreshed as a cutover task, not an afterthought (Finding the Product: Search, Navigation and Merchandising, Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics).
- **The rollback that cannot roll back**. A week of orders exists only in the new system when a defect appears. *Detect:* the rollback rehearsal's timing against the records created since. *Recover:* reconcile forward rather than back if the gap is large: fix the defect in the new system and keep going; a rollback after a week is often the riskier move.
- **The vendor outage mid-cutover**. The new hub's SaaS has an incident during the window. *Detect:* the watch. *Recover:* the seam routed back; the window rescheduled; the status page's history read before the next one.
- **The redirect map with holes**. Old product URLs return "not found"; rankings fall; the M&S quarter. *Detect:* crawl the old sitemap after go-live; watch zero-result searches and organic traffic by landing page. *Recover:* redirects added in order of traffic; the old sitemap kept live until the crawl is clean.

- **Gift cards that do not balance**. The sum of balances differs by a few hundred euros after the import. *Detect:* the finance sign-off, before go-live. *Recover:* the import re-run from the source; no go-live until the sums match to the cent.
- **The agency script still pointing at the old endpoint**. The Deprecated Endpoint in reverse: two days of stale stock to a marketplace because one flow was not on the list. *Detect:* the integration inventory, complete, with every flow's cutover task ticked. *Recover:* the legacy mimic at the old address for one more season while the script is rewritten.

What to Take Away

Established.

- The documented replatforming failures (Target, 2011; Finish Line, 2012; Marks & Spencer, 2014) broke on capacity, performance, timing and customer habit, not on features, and the one documented recovery depended on the old platform having been kept operable.
- The strangler fig (Fowler, 2004, restated 2024) and the patterns of legacy displacement (Thoughtworks, 2021, revised 2024) are the established alternative to the big bang: seams, transitional architecture, event interception, diverted flows, and resistance to feature parity. Zalando's 2015–2016 fragment-by-fragment shop rebuild is the public commerce example.
- Order history is imported as history, not re-created as orders; gift-card and credit balances are liabilities that must balance to the cent; identifiers (SKU, GTIN, order numbers) do not change in a migration.

Depends on context.

- A parallel run is worth its cost when the input can be copied cheaply (orders, product changes) and the reconciliation can explain differences; for a storefront rendering migration, a traffic canary behind a router is the better seam.
- How long the old system stays operable depends on the next peak: through one peak for an order or availability hub; until the crawl is clean for a storefront; until the final reconciliation for a PIM.
- Which seam moves first depends on volume and blast radius: the smallest channel for an order hub, the smallest storefront for a product hub, a region or a segment for a storefront.
- At the Group stage, brand order and feature-parity pressure dominate; at the Starting stage, the URL map and the customer list are most of the risk.

The author's synthesis.

- I plan every migration backwards from the first peak the new owner must survive, and I do not let a seam move unless the previous one has been reconciled to zero known causes and the route back has been rehearsed with a stopwatch.
- I treat the reconciliation job as the migration's main deliverable rather than its test: it is the thing that stays in production afterwards, and it is what makes the next migration cheaper than this one.

Questions to Ask

1. What is the seam, and what is the smallest one we could move first?
2. Is there a way back after the cutover, how long does it take, and has anyone rehearsed it?
3. What does the old system do while the new one owns the seam: receive a copy, sit idle, or get switched off?
4. Which reconciliation proves that old and new agree, and does it explain differences by cause?
5. Are there two writers for any fact during the migration, and if so, why?
6. How are orders imported: as history with their original numbers, or re-created with their side effects?
7. Do the gift-card and store-credit balances sum to the same number in both systems, and who signed that off?
8. Does every old URL redirect, and who crawled the old sitemap afterwards?
9. Which of the thirty-four integrations reads or writes the thing that is moving, who owns each, and which need a mimic?
10. When is the next peak, and what is frozen around it?

Migrations are the moments when a merchant moves from one scale stage to the next and feels it. The next chapter steps back to the stages themselves: which decisions change at each one, what the signals of a stage ending are, and how Kestrel & Pine's own path from a web shop to an omnichannel brand maps onto them. Continue with What Changes at Each Scale: From First Store to Group.

Sources and Evidence

Company, product and investor sources

- Target parts ways with Amazon.com, InternetNews, 2009-08-07; the decision to build and manage its own platform (evidence older than three years).

- Target's website crashes under demand for Italian knitwear, MPR News (AP), 2011-09-13, and Target's website crashes again, MPR News (AP), 2011-10-25; the Missoni outage, Target's "extremely popular" notice, and the second outage (evidence older than three years).
- Finish Line pulls a U-turn online after losing $3 million, Retail Dive, 2013-01-08; the 19 November 2012 switch, the reversion after 17 days to a platform "kept operable", the $3 million (evidence older than three years).
- Marks & Spencer Q1: general merchandise sales down 0.8%, like-for-like down 1.5%, CNBC (Reuters), 2014-07-08; M&S.com sales down 8.1%, the May warning of four to six months to settle in (evidence older than three years).
- Comment: M&S fall in online sales highlights the risks of replatforming, Retail Week, 2014-07-10; the trade commentary (paywalled beyond the lead).
- Crafts retailer Hobbycraft to complete delayed digital roll-out by summer, Retail Week, 2013-02-19; "too risky to launch the website in peak trading" (evidence older than three years).
- Magento 1 reaches EOL, Help Net Security, 2020; the 30 June 2020 end of life and the compliance warning (widely reported).
- From Jimmy to microservices: rebuilding Zalando's Fashion Store, Zalando Engineering, 2015-10-16, and Jimmy to microservices: the journey one year later, Zalando Engineering, 2016-10-05; the router-fronted, fragment-by-fragment migration (evidence older than three years; the system has changed since).
- Zalando consolidates partnership models, Connected Retail to be discontinued, FashionUnited, 2026-04-23; a channel technology retired with notice to 30 June 2027.
- Ecommerce Replatforming and Migration Guide for 2026, Shopify Enterprise blog, 2026-03-18; a vendor's list of migration costs including "downtime during transition" (vendor content).

Standards, regulation and official documentation

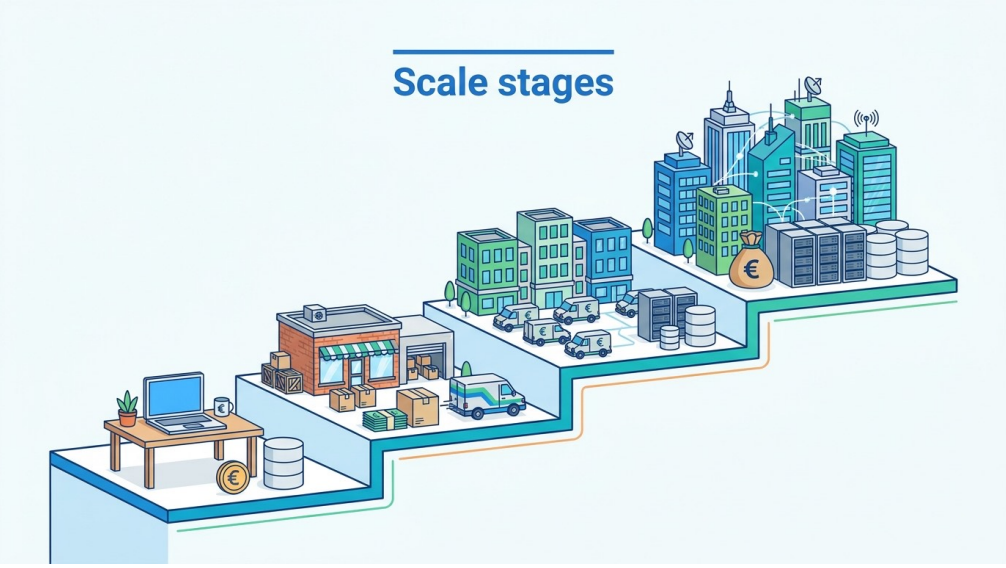
- Regulation (EU) 2023/2854 (Data Act), EU Publications Office, 2023-12-22; the definitions of switching and exportable data that frame the data-export side of a migration.

Research and practitioner sources

- Strangler Fig Application, Martin Fowler, 2024-08-22 (restating a 2004 note); the gradual approach and why wholesale replacement plans fail.
- Patterns of Legacy Displacement, Ian Cartwright, Rob Horn and James Lewis (martinfowler.com), 2021-07-20, revised 2024-03-05; transitional architecture, event interception, divert the flow, legacy mimic, dark launching, canary release, parallel run, and the warning about feature parity and big-bang releases.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

43. What Changes at Each Scale: From First Store to Group



IN THIS CHAPTER, YOU WILL: Read one map of the four scale stages, what each demands in systems, teams and decisions, and the signals that a stage is ending.

KEY POINTS:

- **A stage is a shape, not a revenue number.** The stage changes when the business adds a channel or a stock location that the current systems were not built to keep in step; revenue bands are only rough markers. Kestrel & Pine became Omnichannel in 2023, when stores, wholesale, marketplaces and an own warehouse ran on one stock at the same time.
- **Each stage leaves systems behind, bought for the stage before.** Most incidents at the Omnichannel stage are Growing-stage designs (hourly syncs, agency scripts, the finance system as the hub) running under Omnichannel load. The decision at each stage is which leftovers to retire, not which new tool to add.

- **The signals arrive before the architecture does.** Vendor fee cliffs and plan thresholds, a second stock location, a retail chain asking for orders by EDI (electronic data interchange), a connector nobody owns, the first "which number is right" meeting and the first peak-day incident each mark a transition months before it is named.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Giant's Boots** — an architecture copied from a Group-stage company, or from a trend, into a Growing-stage business that cannot staff it.
- **The Spaghetti Junction** — the Growing stage's one-off connectors, each built for one new channel, that nobody retires when the Omnichannel stage arrives.
- **One Size Fits None** — the Group stage's question answered before it is measured: several brands or countries forced onto one stack, or kept apart when they should share.

A merchant does not grow smoothly. It grows in jumps: the first store, the first wholesale account, the first marketplace, the first warehouse of its own, the first second brand. Each jump adds a place where stock sits or a channel where it sells, and each one asks the existing systems to keep in step with something they were never told about. **The architecture problems of a growing merchant are mostly the problems of systems that were right at the last stage.** The web shop that ran the whole business in 2017 is still there in 2027, with eleven other systems around it, each bought when a jump demanded one.

This book calls the shapes between those jumps **scale stages**, and uses four of them: Starting, Growing, Omnichannel and Group. Every chapter from Part II to Part VIII ends with a table that says what its topic looks like at each stage: what is typical, what becomes necessary and what the warning signs of outgrowing an approach are. This chapter gathers those tables into one map, walks through the four stages in order, and spends most of its attention on the transitions, because **the transition is where the money is lost and where the decisions are made.**

One warning before the map. No public survey of commerce system counts or team sizes by revenue could be verified, so the shapes below are a composite of common practice, dated vendor evidence and the fictional merchant's own path, labeled accordingly.

Where we are in the order's journey. This chapter sits off the path of Eva's jacket, but every system her order crosses on 13 April 2027 (the platform from 2016, the marketplace connector from 2021, the warehouse system and the finance system from 2023, the returns portal from 2025) was bought at a different stage, for a different shape of the company. The order's journey is a walk through Kestrel & Pine's history.

Setting the Scene at Kestrel & Pine: The Stage Map

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), is in its fourth year of the Omnichannel stage when Noor Visser, Head of Engineering since January 2027, is asked by the board in **April 2027** for a **stage map**. Mara Lindqvist, the CEO, put the question plainly: "Waypoint fixes what broke in 2026. Which of those fixes are for the company we are, and which are for the company we were?"

Noor's one-page answer placed Kestrel & Pine in stage 3 since 2023, listed which of its thirteen systems came from which stage, and named the signals that would mean stage 4: a second brand, a fourth country, or revenue past about €200 million. None is planned for 2027–2028, which is why Waypoint buys an order management system (OMS), a product information system (PIM) and an availability service for the Omnichannel stage and defers the Group-stage questions to the 2029 review.

The map had one uncomfortable row. Of the ten incidents of 2026 and early 2027, Noor could attribute eight to a system or integration designed for the Growing stage (2019–2022) and still running in 2026: the hourly stock sync built when marketplace orders were a few a day; agency scripts written for one channel at a time; the ERP (the enterprise resource planning system that keeps the books, purchasing and stock valuation) as the hub for stock it did not physically hold; the POS (the store till system) synced nightly because, in 2019, there was one store next to head office. The Black Friday Oversell (1,340 units, 1,120 orders, about €96,000) was the most expensive. **The company had moved stage; the architecture had not noticed.** That row is this chapter's argument.

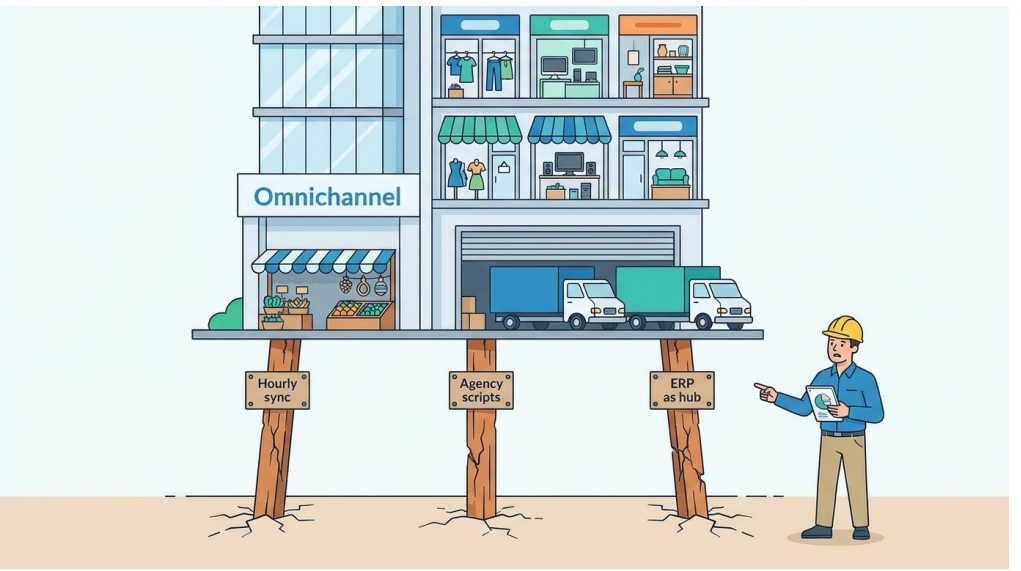


Figure 1: *Eight of Kestrel & Pine's ten incidents traced to Growing-stage designs still*

carrying Omnichannel load: the company had moved stage; the architecture had not noticed.

What a Scale Stage Is, and Is Not

A **scale stage** is a shape of business defined by how many channels sell and how many locations hold stock, because those two numbers decide how many systems must agree about the same facts. A **channel** is any place a customer can buy: a web shop, a store, a wholesale portal, a marketplace, an EDI link (electronic data interchange: orders and invoices exchanged as structured messages between companies' systems). A **stock location** is any place stock physically sits and can be sold from: a back room, a 3PL (a third-party logistics company that stores and ships on the merchant's behalf), an own warehouse, a store, a marketplace's fulfillment center. One channel and one location need no integration. Four channels and four locations need every system to learn, quickly and reliably, what every other system has sold or moved.

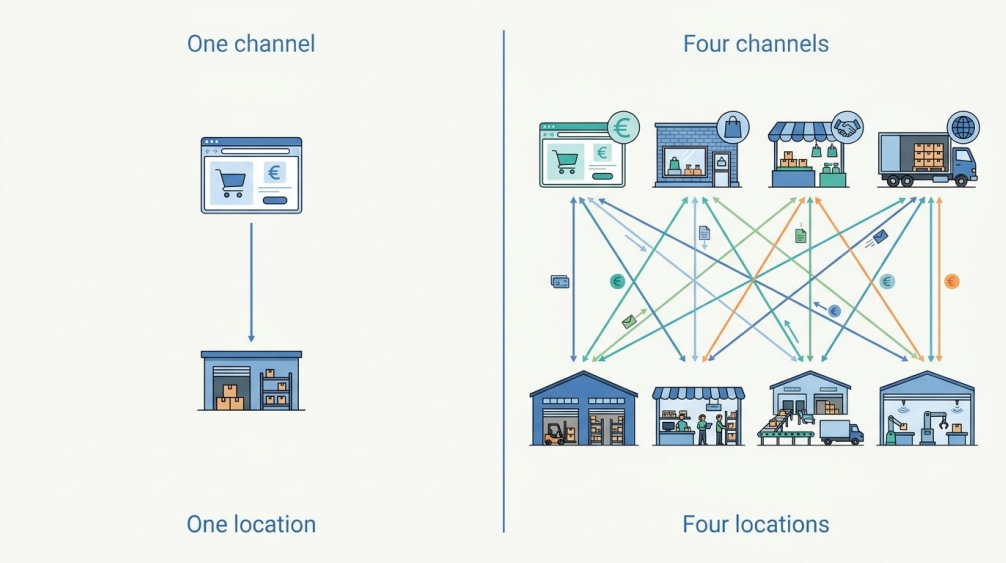


Figure 2: *One channel and one stock location need no integration; four channels and four locations need every system to learn, quickly and reliably, what every other has sold or moved.*

The ledger defines the stages like this, with revenue bands as rough markers:

Stage	Typical size	Shape	Kestrel & Pine
1 — Starting	under about €2M revenue	One channel, a hosted platform, founder-run operations, a 3PL or a back room	2016–2018
2 — Growing	about €2M–€20M	Two or three channels, first store or wholesale, first integrations (ERP or accounting, 3PL, marketplace), a small team and an agency	2019–2022
3 — Omnichannel	about €20M–€200M	Stores, wholesale, marketplaces and several countries; own warehouse; a dozen or more systems; the joins become the problem; OMS, PIM and integration standards become necessary	2023– (now)
4 — Group	over about €200M, or several brands or business units	Multiple brands, countries or business units; platform teams; shared services vs autonomy; acquisitions; vendor-scale concerns (tenancy, extensibility)	not yet

The vendors' own price lists confirm that shape and volume, not headcount, are what the

market tiers on. Shopify's euro pricing page (accessed November 2026) runs from Basic at €21 a month paid yearly to Plus from €2,100 a month on one- or three-year terms, and its fee for using a payment provider other than its own falls from 2% of each order on Basic to 0.2% on Plus. Shopware's pricing page (accessed November 2026) says its plans are "based on your gross merchandise value (GMV) and further individual factors", with a paid plan required above €1 million GMV in a rolling year. BigCommerce's plan page (accessed November 2026) assigns tiers by trailing-twelve-month GMV and auto-upgrades merchants at $2 million. **GMV** is gross merchandise value, the total value of orders through a platform before refunds and before the vendor's cut. The numbers change often; the mechanism does not: a vendor's tiers are a map of which decisions it expects you to make at which size.

What a stage is **not**: a target, a maturity score or a verdict. A merchant can stay at the Growing stage profitably forever; the stage only tells you which problems are now yours.

The Map: Four Stages Side by Side

This is the chapter's centerpiece. Each row is a question the other chapters answer in depth; the cells say what is typical and what becomes necessary at each stage. Everything in it is industry practice, dated vendor evidence or Kestrel & Pine; where a cell makes a judgement, it is mine. The abbreviations are the book's: a **WMS** (warehouse management system) runs the warehouse; a **PSP** (payment service provider) takes the money; an **iPaaS** (integration platform as a service) is a hosted tool for building flows between systems; an **SSCC** is the serial shipping container code on a pallet label; **B2B2C** is selling through a partner's consumer channel.

	1 — Starting	2 — Growing	3 — Omnichannel	4 — Group
Shape	One channel, one stock location, founder-run	Two or three channels, a first store or wholesale line, a 3PL or small warehouse	Stores, wholesale, marketplaces, several countries, an own warehouse; a dozen or more systems	Several brands, countries or business units; acquisitions; platform teams

	1 — Starting	2 — Growing	3 — Omnichannel	4 — Group
Channels (Who Sells What to Whom: Business Models and the Economics That Shape Architecture)	Web shop; perhaps one marketplace	Web shop plus a store, a marketplace or wholesale by e-mail	Four or more, including EDI with a retail chain and marketplace-fulfilled stock	Channels per brand and per country; marketplace *operating* or B2B2C may appear
Storefront and checkout (Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Delivering the Storefront: Themes, Headless, Rendering and Performance)	Hosted SaaS platform on a monthly plan; a theme; public apps	Same platform, higher tier; an agency customizes the theme; first custom app	Enterprise tier or contract; multi-market in one tenant or several; headless considered and usually declined	Shared front end or per brand; suite, composable or platform-plus-layers becomes a real choice
Where the order lives (The Order Record: Lifecycle, State and the Order Management System)	In the platform	In the platform, with copies in the ERP and the 3PL	Several order records per channel until an **OMS** is introduced	OMS as the hub for all brands, or one per business unit

	1 — Starting	2 — Growing	3 — Omnichannel	4 — Group
Where stock truth lives (Is It in Stock? Inventory, Availability and the Promise You Can Keep)	The platform's stock count	The ERP or the 3PL feed, copied to the platform on a schedule	A **WMS** for the warehouse, the POS for stores, an external location for marketplace fulfillment; an availability service computes what each channel may sell	Stock per brand and region; shared availability rules; allocation between business units
Product data (What Are We Selling? Products, Variants and the Catalog)	The platform's catalog	Platform plus ERP item master plus spreadsheets	A **PIM** as the system of record; platform, connector and marketplaces consume it	PIM per brand or shared, with a shared attribute model
Fulfillment (Pick, Pack, Ship: Warehouse Management and Fulfillment)	Back room or 3PL	3PL or a small warehouse run from the ERP's stock module	Own warehouse on a WMS; pallets and SSCC labels for wholesale; marketplace fulfillment programs	Several warehouses, a network, logistics as a shared service

	1 — Starting	2 — Growing	3 — Omnichannel	4 — Group
Finance and ERP (The Back Office: ERP Integration and Who Is the Hub)	Accounting package; the platform exports	A mid-market ERP, often as the integration hub	The ERP kept for finance, purchasing and valuation only; the hub role moves to the OMS and the integration layer	Group ERP with per-entity books; intercompany flows
Integration style (APIs, Events, Files and Middleware: How Commerce Systems Talk)	None; the platform's own connectors	Point-to-point connectors, agency scripts, vendor connectors; the first iPaaS flows	Events for stock, orders and prices; an owned integration layer; contracts, idempotency and reconciliation per flow	A platform team runs the integration layer as an internal product; API guidelines; shared event bus
Who builds and runs (Who Builds and Who Owns: Teams, Vendors, Agencies and Governance)	The founder and the platform's support; an agency for the theme	The agency builds; a first engineer; nobody owns integrations after launch	A 10–20-person team that owns the joins; the agency as a contributor; named owner per integration and app	Stream-aligned teams per brand or journey; a platform team; vendor management as a function

	1 — Starting	2 — Growing	3 — Omnichannel	4 — Group
Pricing and fees you meet	Monthly plan; blended card rates	Plan upgrades; GMV thresholds; the third-party gateway fee; per-order quotas on tools	Negotiated contracts (platform, PSP, WMS, OMS); interchange-plus pricing; several thresholds crossed at once	Group contracts; volume commitments; exit and switching terms
Peak day (Black Friday Every Year: Peak Load, Resilience and Recovery)	The platform's problem, not yours	The first integration bottleneck: the order export, the stock sync	A peak plan with orders-per-minute capacity per system, a switch-off order and a reconciliation owner	Shared peak tooling and runbooks across brands
Decisions that become necessary	Which platform; which PSP; which 3PL	Who owns stock and price truth; the first connector's contract; one tenant or many	OMS, PIM, availability service; integration standards; stay-and-surround or replatform; one definition of an order	Share or separate per brand; platform team or not; acquisition integration playbook; tenancy and residency
Failure patterns that strike	The Lynchpin (one agency, one person)	The Spaghetti Junction; The Price Nobody Can Explain; The First Snowfall	The Phantom Stock; Schrödinger's Order; The Boomerang; The Familiar Stranger; The Pulled Rug; The Secret Agent	One Size Fits None; The Giant's Boots; The Burned Bridge

	1 — Starting	2 — Growing	3 — Omnichannel	4 — Group
Signals the stage is ending	A second channel or a first store; the 3PL cannot say stock per location	A retailer asks for EDI; a second stock location; a fee cliff; the first "which number is right" meeting	A second brand or business unit; an acquisition; a fourth country with its own books; the platform's tenancy or rate limits bind	(The stage has no end; its question is what to share)
Kestrel & Pine	2016–2018: platform, PSP, 3PL in Tiel, agency from 2017	2019–2022: POS, Belgium, connector and iPaaS, GTINs, Germany, Ruben Alves	2023–2027: Duiven on a WMS, ERP, enterprise tier, stores, EDI, returns portal, Peppol; Waypoint	Not yet; signals named in the April 2027 stage map

Three things to notice. **The order record and the stock record move in the same direction**: from "the platform knows" through "the ERP knows, mostly" to "a hub knows, and everything else is a copy." **The integration style changes faster than the systems do**: a merchant can keep its platform for all four stages and still rebuild the joins twice. And **the failure patterns are stage-specific**: the Truth patterns of Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong cluster in the Omnichannel column because that is where the same fact first lives in four systems.

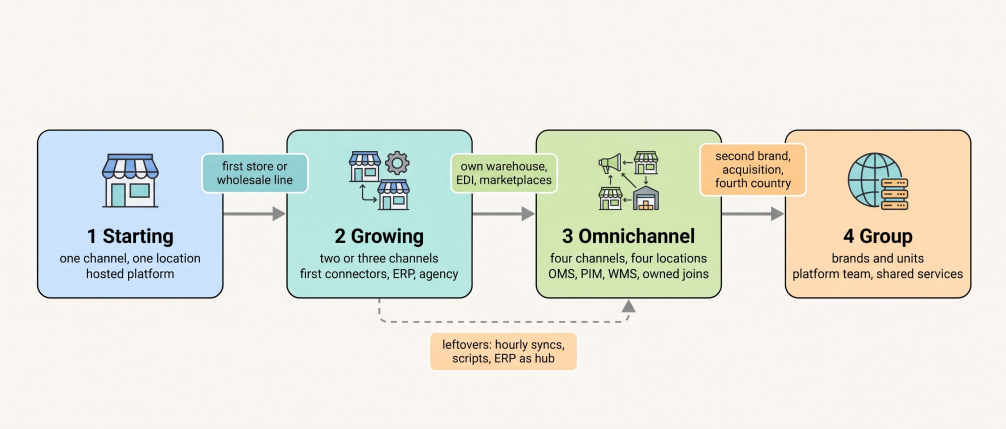


Figure 3: *Each stage is entered by adding a channel or a stock location, and each carries the previous stage's systems with it; the dotted arrow is where most Omnichannel incidents come from.*

How Fresh Must a Fact Be? Requirements by Stage

The reason the joins must be rebuilt is that the same fact needs to travel faster and more reliably at each stage. The table below states the requirement for the three facts that cause most incidents. The latencies are illustrative targets, not benchmarks *(assumption)*; the Kestrel & Pine figures are from the ledger.

Fact	Starting	Growing	Omnichannel	Group
Stock a channel may sell	The platform's own count; freshness is instant because there is one writer	A scheduled copy (hourly is common) from the ERP or 3PL; tolerable while one channel dominates	Seconds to a few minutes, by events, because four channels sell the same unit; Kestrel & Pine's two-hour chain oversold 1,340 units	As Omnichannel, plus allocation rules between brands and regions

Fact	Starting	Growing	Omnichannel	Group
Order status	One record; the customer and support read the same screen	Copies in the ERP and the 3PL; support checks two systems	One state model in an OMS; status events out to the storefront, service tool and market-places within minutes; the Three-Status Order (11 tickets, six days) is the failure	One state model across brands, with per-unit order number families
Price	Set in the platform	Price lists in the platform and the ERP; a sync that can fail silently (the Price Drift: €179 for nine days, about €8,200 lost)	One price source with effective dates; channels consume; parity rules for market-places	Price books per brand and market with shared governance

The point is not the numbers. It is that **a requirement met for free at the Starting stage (one writer, one reader) becomes an engineering problem at the Omnichannel stage (four writers, four readers), and the Growing stage's scheduled copies are the bridge that collapses between them.** Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation owns the mechanics; Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record owns the ownership decision that must come first.

Stage by Stage

Starting: one channel, rented everything

At the Starting stage the architecture is a purchase, not a design. A hosted SaaS platform (software as a service: rented by the month and run by the vendor) provides storefront,

cart, checkout, stock count and order record in one tenant; a payment service provider (PSP) takes the money; a 3PL or a back room ships. Shopify's annual report for 2025, filed in February 2026, says most of its merchants subscribe to its Basic and Grow plans while the majority of its GMV comes from Plus and enterprise merchants: the Starting stage is where most merchants are and most volume is not.

What becomes necessary here is small: a platform whose export you could leave with, a PSP whose fees you understand, an app inventory with each app's permissions written down from day one (Apps, Plugins and Functions: Extensibility and Ecosystem Governance), and the founder, not the agency, holding the vendor logins. The failure pattern that strikes is **The Lynchpin**: one agency or one person through whom everything runs. Kestrel & Pine retained its agency in 2017; ten years later it still holds the theme, the apps and several scripts, at €14,000 a month.

The signal that the stage is ending is a second channel or a first store. The day the business has stock in two places that both sell, it has an integration problem, whether or not it has an integration.

Growing: the first joins, built one at a time

The Growing stage adds a store on a point-of-sale system (POS, the till software that records store sales), a marketplace through a connector, a wholesale line run by e-mail and keyed into an ERP (enterprise resource planning: the finance, purchasing and inventory-valuation system), and often an ERP-centered model where the ERP becomes the hub that every other system syncs with. Microsoft's documentation of 22 September 2026 says its Shopify connector is preinstalled for all new Business Central sign-ups, the clearest public signal that "a hosted storefront with the ERP as hub" is the mainstream Growing-stage shape in at least one vendor channel.

Each join is built when the channel arrives, by whoever is available: the agency, the ERP partner, the connector vendor, a contractor. Each is correct for the volume of the day. Kestrel & Pine's marketplace connector arrived in 2021 with an hourly stock sync, when marketplace orders were a few a day *(illustrative)*; by 2026 the marketplaces took 43,000 orders a year and the hourly sync was one link in a two-hour chain nobody had decided was the architecture. **The Spaghetti Junction is not built; it accretes.**

This is also the stage where fees change character. A merchant that brings its own PSP to a hosted platform pays the platform's third-party gateway fee (2% of each order on Shopify's Basic plan, falling by tier); operational tools price per order (Picqer, a Dutch WMS, lists €379 a month including 3,000 orders, then €0.28 an order, accessed November 2026); and open-source platforms stop being free above Shopware's €1 million GMV line. A merchant moving from Growing to Omnichannel typically crosses several thresholds at once, which is why price lists are a transition signal, not just a cost.

The signals that the stage is ending are concrete. A retail chain asks for EDI (Eurostat's enterprise statistics for 2024, extracted February 2026, show EDI-type sales at 11.07%

of EU enterprise turnover against 8.39% for web sales; by value, European business-to-business electronic ordering is still mostly EDI, so a wholesale line meets EDI before it meets a B2B web shop). A second stock location appears that must be sold from, which vendor guidance converges on as the trigger for an OMS. The first meeting happens in which two reports show two order counts and nobody can say which is right. And the first peak day arrives on which an integration, not the platform, is what fails (**The First Snowfall**).

Omnichannel: the joins become the architecture

At the Omnichannel stage the merchant runs three or four business models on one stock: direct-to-consumer web shops in several countries, stores, wholesale by portal and by EDI, and marketplaces with some stock held by the marketplace itself. Target's annual report for 2025, filed in March 2026, states that its stores fulfilled 97.6% of its merchandise sales, including shipment from stores and store pickup; for a retailer with stores, omnichannel is the business, and an order management system is what makes store stock sellable online. Target does not name its OMS; the mechanism is what matters.

What becomes necessary here is the set of decisions Waypoint makes: a **system of record** per fact (the one system whose value is the truth when copies disagree) for stock, orders and product content (Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record); an OMS as the order hub (The Order Record: Lifecycle, State and the Order Management System); a PIM for product content, which Akeneo's guidance (accessed November 2026) says is signaled by content scattered across teams, manual channel launches and inconsistent storefront data (What Are We Selling? Products, Variants and the Catalog); an availability service per channel (Is It in Stock? Inventory, Availability and the Promise You Can Keep); and integration standards with a named owner, a contract, idempotent handling and a reconciliation job per flow (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation). The ERP is demoted from hub to one system among many (The Back Office: ERP Integration and Who Is the Hub).

The temptation at this stage is **The Giant's Boots**. The composable and microservices vocabulary is everywhere, and the public examples are companies with hundreds of engineers. Fowler's "MonolithFirst" note of June 2015 argued that even experienced architects "have great difficulty getting boundaries right at the beginning"; Shopify's engineering post of February 2019 explained why a company with over a thousand developers chose a modular monolith over microservices. The MACH Alliance's own survey of 600 decision-makers, published in February 2026, reports that 37% cite integration complexity as a primary concern, a figure against the interest of the alliance that commissioned it. For a 14-person team the honest reading is: adopt composable per capability (an OMS here, a PIM there) around a kept core, and own the joins. Monolith,

Modular Monolith, Microservices, Composable: What the Evidence Says and Build, Buy or Assemble: Choosing and Changing Platforms work this through.

The signals that the Omnichannel stage is ending are a second brand or business unit, an acquisition, a fourth country with separate books, or the platform's tenancy binding (one tenant's rate limits shared by every app and connector; Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation). Shopify's enterprise page (accessed November 2026) cites organizations with GMV of $50 million to $500 million and more as its enterprise band, the nearest public hint of where a vendor sees the top of this stage.

Group: what to share and what to keep apart

The Group stage is defined by plurality: several brands, business units or countries, each of which could run its own stack. The question is no longer "which systems" but "**which systems are shared, and who owns the shared ones?**" VF Corporation's annual report for the fiscal year to March 2026 describes a direct-to-consumer business of brand-operated stores, brand e-commerce sites and concession locations across brands such as The North Face and Timberland, at 44% of revenue: one group, many brand storefronts, and a shared question about what sits behind them. Zalando's 2025 annual report, published March 2026, shows what an acquisition does first: consolidating ABOUT YOU changed the group's reported figures (the partner-business share of GMV fell to 32.1% from 34.2% by dilution) before any system was touched. An acquisition changes the metrics on day one and the architecture over years.

The failure pattern is **One Size Fits None**: forcing brands onto one stack before anyone has measured the benefit, or keeping them apart when they should share. The honest default, in my judgement, is to share what is invisible to the customer and expensive to duplicate (identity, the payments contract, the integration layer, data definitions, peak tooling, security) and to let brands own what the customer sees (storefront, content, promotions, assortment), with a platform team running the shared parts as an internal product. The public cautions against too much autonomy are Zalando's and Uber's. Henning Jacobs' 2020 retrospective on a decade at Zalando (more than 260 engineering teams by 2020) reports that team autonomy over technology produced similar services "written in six languages", which "hurts productivity at scale"; Uber's 2020 account reports about 2,200 microservices needing domains, gateways and layers before a root-cause investigation stopped running through fifty services owned by twelve teams. Both sources are six years old and describe companies far larger than any merchant; the lesson transfers: **autonomy without shared standards produces the Spaghetti Junction at group scale**.

The Group stage's remaining questions are vendor-scale ones: tenancy (one tenant per brand or a tenant of tenants), extensibility (a shared app catalog and an upgrade calendar aligned to vendors' support clocks), data residency, and exit terms. The EU Data Act (Regulation 2023/2854) prohibits switching charges for data processing services from 12

January 2027; whether hosted commerce platforms fall under it is an open legal question as of November 2026, but a group negotiating several contracts should read the switching chapter before signing (Build, Buy or Assemble: Choosing and Changing Platforms).

Multi-Brand and Multi-Country: The Group's Decision Table

Because the Group stage's question is "what to share", it deserves its own table. The cells are my recommendations with their conditions.

Capability	Share across brands or units when…	Keep separate when…
Storefront and theme	Brands are positioned alike and share a design system	Brand identity is the product; customers must not see the group
Checkout and PSP contract	Always share the contract and the fraud rules; volume pricing rewards it	Legal entities or countries require separate merchant accounts (share the provider, not the account)
Catalog and PIM	Attribute model and data-quality rules; a shared PIM with brand workspaces	Assortments never overlap and teams are in different countries with different seasons
Stock and availability	Warehouses serve several brands; allocation rules must be one	Brands have separate logistics and no cross-selling
Order hub (OMS)	One state model, one service tool; customers buy across brands	Business units have incompatible order types (B2B contract orders vs D2C) with no shared fulfillment
Integration layer and data definitions	Always; this is the platform team's product	Never, in my judgement; separate definitions of "order" per brand is the Group-stage Schrödinger's Order
Peak tooling, security, compliance	Always	Never

Two conditions govern the table. If the group acquires rather than builds, every acquired brand arrives with its own stack and the question becomes sequencing, which is a migration (Moving Without Burning the Bridge: Replatforming and Migration). And if the group operates a marketplace or a partner program, it has become a platform vendor to other sellers, and the vendor-side concerns of Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation apply to it directly.

The Transition Signals

A transition is easier to see than to name. Every signal below was visible at Kestrel & Pine before the incident that priced it.

Signal	What it usually means	Pattern it precedes	Kestrel & Pine
A vendor fee cliff or plan threshold is crossed	The vendor expects a different kind of customer; read its tier as a map	—	Enterprise tier in 2023
A second stock location that must be sold from	An availability computation is needed, not a copy	The Phantom Stock	Duiven 2023; stores selling online stock from 2026
A retail chain asks for EDI	Order capture moves into the buyer's systems; chargebacks become possible	The Pulled Rug (contract), The Lynchpin (one consultant)	Bergzicht 2025; the ASN Chargeback 2026
Two reports, two order counts	Three definitions of the same fact; a data-definition owner is needed	Schrödinger's Order (in the data)	The three definitions of "order"
A connector nobody can name an owner for	The Growing stage's build-it-when-it-arrives has left orphans	The Spaghetti Junction; The Pulled Rug	The Deprecated Endpoint, February 2027
Support opens several systems to answer one question	No order hub	Schrödinger's Order	The Three-Status Order, 11 tickets

Signal	What it usually means	Pattern it precedes	Kestrel & Pine
An app or script writes a fact another system owns	No ownership rule per fact	The Secret Agent	The App That Wrote Stock, March 2026
The first peak on which an integration fails	The platform scales; the joins do not	The First Snowfall	Black Friday 2026, four-hour WMS queue
A second brand, an acquisition, a fourth country	The Group stage's share-or-separate question has arrived	One Size Fits None	Not yet; on the stage map

The rule I use: **two signals in one year mean the stage has changed, whatever the revenue says.** Kestrel & Pine had six in 2026.

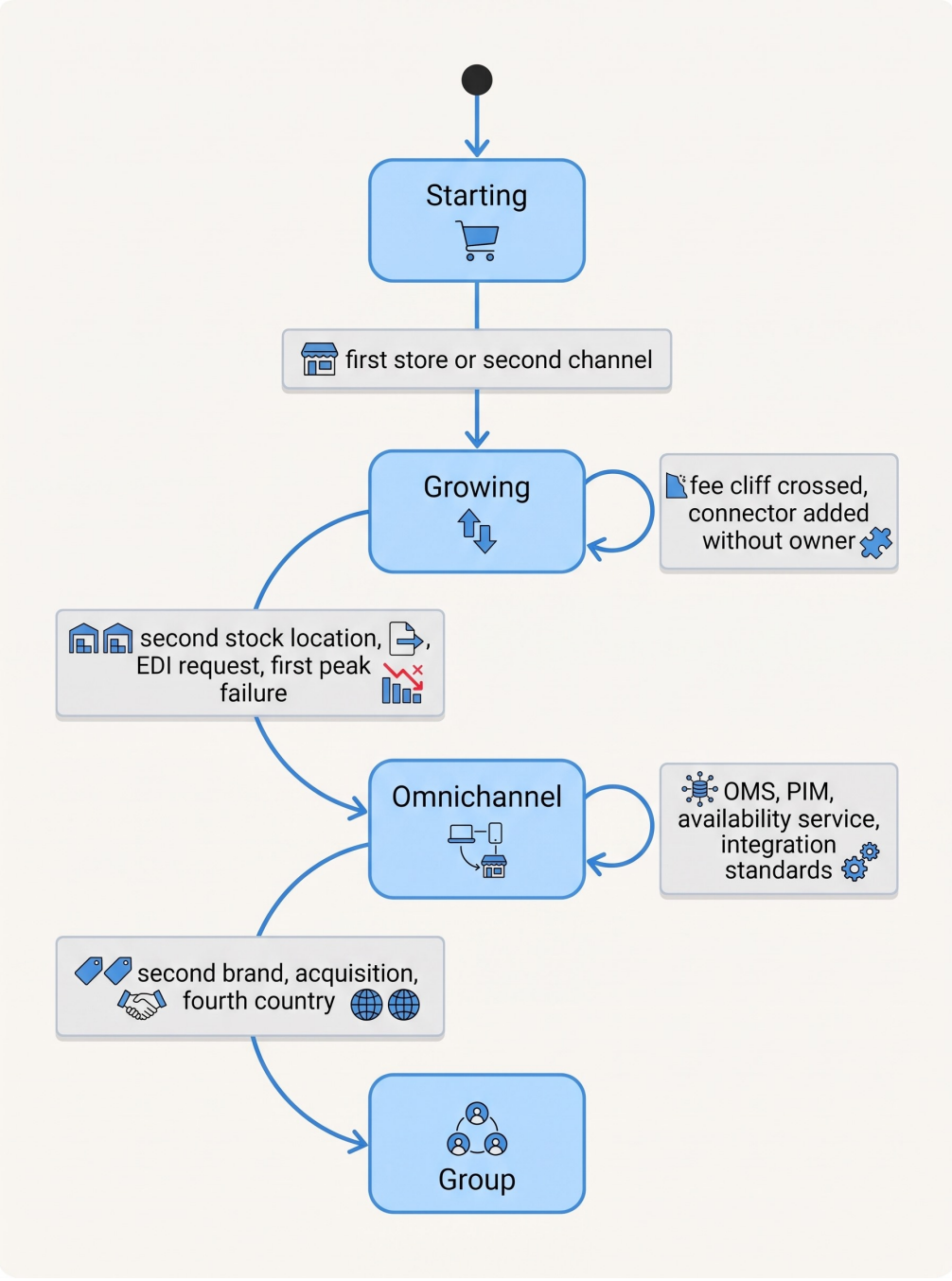


Figure 4: *The transitions are triggered by shape changes, not revenue; the self-loops are where a stage does its work, and where a merchant can stay for years.*

In Practice: Kestrel & Pine's Path, Priced

The stage map is more persuasive with the costs attached. The figures below are Kestrel & Pine's, from the ledger, with one illustrative calculation.

2016–2018, Starting. One Dutch web shop on a hosted platform, one PSP, a 3PL in Tiel, an agency from 2017. The architecture was the vendor's. Nothing from this stage caused a 2026 incident, except the habit it formed: the agency owns the storefront.

2019–2022, Growing. The Arnhem store on a cloud POS from a different vendor (2019), synced nightly. Belgium (2020). The Utrecht store, the marketplace connector with its hourly sync, the iPaaS and GTINs on every own-brand item (2021). Germany, the investor and Ruben Alves as lead integration engineer (2022). Every join was built when its channel arrived. Eight of the ten incidents descend from designs of this era; the clearest are the Store Stock Mirage (37 items promised that Utrecht had sold), the Deprecated Endpoint (an unowned agency script) and, in its cadence, the Black Friday Oversell.

2023–2026, Omnichannel. Duiven on its own WMS, the ERP and the platform's enterprise tier (2023); three more stores and the data warehouse (2023–24); Antwerp, EDI with Bergzicht and the returns portal (2025); Peppol e-invoicing and the Utrecht click-and-collect pilot (2026). The company now had thirteen systems, four order records, four copies of stock and three definitions of "order", and technology spend of about €2.6 million, 5.4% of revenue, high against the 2–4% that secondary sources suggest for store-heavy retailers (no primary benchmark could be verified; a judgement). The incidents of 2026 and January 2027 cost, in round figures, €96,000 (Oversell), €19,400 refunded twice (most recovered), €8,200 of margin (Price Drift) and €14,200 (ASN Chargeback): about €138,000 of direct cost, before support time and goodwill.

The fee example. The fee cliff that matters most to Kestrel & Pine is not the platform subscription (€2,300 a month on the enterprise tier) but payments. On €27.0 million of online D2C revenue, each half a percentage point of blended payment cost is about €135,000 a year *(illustrative arithmetic on ledger figures)*, more than the direct cost of all four priced incidents. That is why Taking the Money: Payment Providers, Authorization, Capture and Refunds and Build, Buy or Assemble: Choosing and Changing Platforms treat the PSP contract as an architecture decision, and why a merchant that never renegotiated its Starting-stage card rate pays a transition tax without knowing it.

2027, the stage map. Waypoint's €1.1 million over two years buys the Omnichannel stage's hubs and standards, retires the Growing stage's chain, and explicitly defers the Group stage. That is the whole use of the stages: not to predict revenue, but to sort decisions by the shape they belong to.

When It Goes Wrong: Too Early, Too Late, Sideways

A stage mismatch fails in three directions, and each has a detection and a recovery.

Too early (The Giant's Boots). A Growing-stage merchant adopts an API-only composable core, or a microservices rebuild, on the strength of case studies from companies with platform teams. Detection: the integration layer is the largest line of engineering effort and nobody can say who is on call for it. Recovery: consolidate the unit of deployment, as Segment did in 2018 when it folded more than 140 services back into one (its own account, now eight years old); keep the capabilities that earn their place and return the rest to the platform.

Too late (The Phantom Stock, Schrödinger's Order). An Omnichannel-stage merchant runs Growing-stage copies: scheduled syncs, the ERP as the stock hub, order records per channel. Detection is cheap: count the systems that hold stock, count the writers, and run one reconciliation between the warehouse and the platform. Kestrel & Pine's first weekly reconciliation in May 2027 found 2.3% of active variants with a different quantity, most from the apps that wrote stock. Recovery is Waypoint: decide the system of record per fact, add the hubs, convert the chain to events, and keep the reconciliation job forever (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation).

Sideways (One Size Fits None). A group forces brands onto one stack by decree, or lets each keep everything, without measuring what sharing would save or cost. Detection: a shared-platform program whose business case is "consolidation" with no per-capability benefit, or as many definitions of "customer" as brands. Recovery: the decision table above, applied per capability, with the shared capabilities run by a team that treats the brands as its customers.

In every direction the recovery runs through the same two tools: an entity–owner table (Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record) and a reconciliation job per copy. The stage decides how many rows those tools have, not whether you need them.

What to Take Away

Established.

- Vendors tier on volume and shape, not headcount: Shopify's plans and third-party gateway fee (2% to 0.2% by tier), Shopware's €1 million GMV fair-use line and BigCommerce's GMV-assigned plans (all accessed November 2026) make a merchant's growth visible as fee cliffs and plan thresholds.
- By value, European business-to-business electronic ordering is still mostly EDI (11.07% of EU enterprise turnover against 8.39% for web sales, Eurostat 2024 data),

so a wholesale line meets EDI before it meets a B2B web shop, usually at the Growing-to-Omnichannel transition.
- For a retailer with stores, omnichannel fulfillment is the business, not an add-on (Target's stores fulfilled 97.6% of merchandise sales in 2025), and an order hub is what makes store stock sellable online.
- Autonomy without shared standards is costly at group scale: Zalando's 2020 retrospective (services in six languages) and Uber's 2020 domain-oriented redesign (about 2,200 services) are the dated public evidence.

Depends on context.

- Whether an OMS, a PIM or a WMS is needed depends on channel and location counts, not order volume: an OMS when a second stock location must be sold from; a PIM when the catalog is published to a third channel in a third format; a WMS when physical operations outgrow the ERP's stock module. A single-channel merchant with high volume may need none of them.
- The revenue bands are markers. A €15 million wholesaler with one warehouse is a simpler shape than a €6 million brand with stores, marketplaces and a 3PL; design for the shape.
- The ERP as integration hub is right at the Growing stage and becomes the bottleneck at the Omnichannel stage, when stores, marketplaces and a WMS need stock faster than the ERP's batch cadence.
- At the Group stage, share what the customer cannot see and what is expensive to duplicate; keep brand-visible capabilities separate until a measured benefit says otherwise.

The author's synthesis.

- Most Omnichannel-stage incidents are Growing-stage designs running under Omnichannel load; the first decision at each stage is which leftovers to retire, because the leftover, not the missing tool, is what fails.
- Two transition signals in one year mean the stage has changed, whatever the revenue says; a stage map that lists the signals is worth more to a board than a roadmap of tools.

Questions to Ask

1. Which stage is the business in by shape (channels and stock locations), and which stage was each of its systems bought for?
2. Which vendor thresholds (plan tier, GMV cap, per-order quota, PSP volume pricing) will the business cross in the next eighteen months, and what does each vendor expect of a customer on the other side?

3. How many systems hold stock today, how many write it, and how often is the warehouse's number compared with the storefront's?
4. Is there one order record, or one per channel? How many systems does support open to answer "where is my order"?
5. Which integrations were built when a channel arrived, by whom, and who owns each one now?
6. Which transition signals from this chapter's table appeared in the last twelve months, and which incident followed each?
7. If a second brand, an acquisition or a fourth country arrived next year, which capabilities would be shared on day one, and who would run them?
8. Which decisions in the current program are for the stage the business is in, which for the stage it was in, and which copy a company much larger, and why?

The stages sort the decisions; people make them. Who owns each system, integration and app, how a 14-person team works with an agency and a dozen vendors, and what governance fits a merchant rather than a software company is the subject of Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.

Sources and Evidence

Company, product and investor sources

- Shopify pricing (EUR), Shopify, n.d., accessed 2026-11 — plan prices from Basic to Plus and the third-party transaction fee by tier.
- Shopify Plus plan, Shopify Help Center, n.d., accessed 2026-11 — Plus base price on one- and three-year terms and the variable platform fee for larger merchants.
- Shopify for Enterprise, Shopify, n.d., accessed 2026-11 — the $50M–$500M+ GMV band cited for enterprise merchants.
- Shopify Inc. Form 10-K, fiscal 2025, SEC filing, 2026 — most merchants on Basic and Grow; most GMV from Plus and enterprise; the GMV definition.
- Shopware pricing, Shopware, n.d., accessed 2026-11 — GMV-based plans and the €1 million fair-use threshold.
- BigCommerce pricing, Commerce.com, n.d., accessed 2026-11 — GMV-assigned plan tiers, the 0.9% overage and the $2 million auto-upgrade.
- Picqer pricing, Picqer, n.d., accessed 2026-11 — per-order pricing of a WMS as an example of operational-layer fees.
- Target Corporation Form 10-K, fiscal 2025, SEC filing, 2026 — stores fulfilled 97.6% of merchandise sales.
- VF Corporation Form 10-K, fiscal 2026, SEC filing, 2026 — a multi-brand group's direct-to-consumer definition and share.

- Zalando SE Annual Report 2025, Zalando SE, 2026 — the effect of consolidating ABOUT YOU on group metrics.
- Get started with the Shopify Connector, Microsoft Learn, 2026 — the connector preinstalled for new Business Central sign-ups.
- Top 3 signs you need a PIM, Akeneo (guest post), n.d., accessed 2026-11 — vendor guidance on PIM triggers; vendor content.
- Deconstructing the Monolith, Shopify Engineering (Kirsten Westeinde), 2019 — why a large vendor chose a modular monolith; older than three years.
- One decade in Zalando tech, Henning Jacobs, 2020 — team counts and the cost of autonomy without standards; older than three years.
- Introducing Domain-Oriented Microservice Architecture, Uber Engineering, 2020 — about 2,200 services and the cost at the far end of scale; older than three years.
- Goodbye Microservices, Twilio Segment (Alexandra Noonan), 2018 — consolidating 140+ services into one; older than three years.
- MACH Alliance research: 6x more organizations achieve AI ROI with a composable foundation, MACH Alliance, 2026 — 37% cite integration complexity as a primary concern; commissioned survey.

Standards, regulation and official documentation

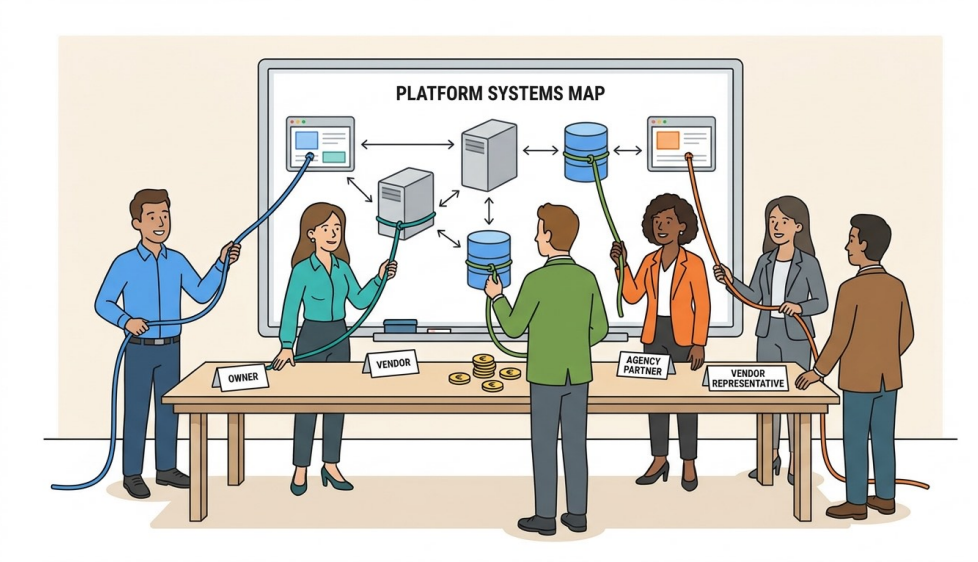
- E-commerce statistics (enterprises), Eurostat, 2026 (2024 reference year) — EDI-type sales 11.07% and web sales 8.39% of EU enterprise turnover.
- Regulation (EU) 2023/2854 (Data Act), EU Publications Office, 2023 — switching charges prohibited from 12 January 2027; application from 12 September 2025.

Research and practitioner sources

- MonolithFirst, Martin Fowler, 2015 — the difficulty of getting boundaries right early; older than three years.
- IT Spending and Staffing Benchmarks 2025/2026, Chapter 8: Retail, Avasant, 2025 — the retail IT-spend benchmark exists but is paywalled; the 2–4% range in the text is judgement.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

44. Who Builds and Who Owns: Teams, Vendors, Agencies and Governance



IN THIS CHAPTER, YOU WILL: Learn who should own each system, integration and app, how to shape a small team around journeys, and how to work with agencies and vendors.

KEY POINTS:

- **Ownership follows money unless it is designed.** An agency is paid to build and a vendor to keep you subscribed; neither is paid to own a connector after launch. The ownership map, one row per system, integration and app with a named owner, is the merchant's first governance artifact, and most incidents at Kestrel & Pine trace to a blank cell in it.
- **Shape teams around journeys, not vendors.** A team per system reproduces the integration problem in the organization chart. For ten to twenty people the honest shape is one or two stream-aligned teams

by journey (buy; fulfill and return), one pair that owns the integration layer as an internal product, and vendors and agencies as services with explicit contracts.
- **Govern lightly, but govern the facts.** Decision records for ownership, vendor and app decisions; one writer per fact; a contract card per integration; a quarterly app review; API versions pinned and recorded. Heavier process will not be followed by fourteen people, and lighter process lets the Secret Agent back in.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Lynchpin** — a capability customers depend on runs through one agency, one consultant or one engineer, with no exit plan.
- **The Secret Agent** — an app, script or integration changes stock, prices or money with no named owner and no record of what it may touch.
- **The Spaghetti Junction** — connectors built by whoever was available when a channel arrived, each with its own owner or none, and no inventory.

A merchant that buys every system still has to own the joins between them, and the joins are where commerce fails. The platform vendor runs the storefront; the partner that implemented the ERP (the enterprise resource planning system that keeps the books, purchasing and stock valuation) built its connector; the agency wrote the scripts; the marketplace connector is a subscription; the WMS (the warehouse management system) came with its own integration. Each party is responsible for its piece. **Nobody is responsible for the fact that the stock number in the warehouse and the stock number on the marketplace are the same number**, unless the merchant decides that someone is.

This chapter is about that decision and the organization that makes it stick. It covers the ownership map, team shapes for a small commerce team, how to work with agencies and implementation partners, what to put in vendor contracts, governance light enough to survive contact with a fourteen-person team, the product manager's role, and the engineering practices that a merchant on rented software can still practice. It ends with how the team shape changes across the four scale stages.

The frame is a merchant's, not a software vendor's. Shopify and commercetools must organize thousands of engineers to ship a platform; the cases in Part XI — PRACTICE: How Others Built It cover that. A merchant must organize a dozen people, an agency and a dozen vendors to keep promises its systems make across each other's boundaries. The two problems share vocabulary and little else.

Where we are in the order's journey. Eva's jacket crosses seven systems and about a dozen handoffs between 13 and 22 April 2027. This chapter asks, of each handoff, who owns it: who decided its contract, who watches it, who is called when it fails, and who

may change it. Where the answer is "the agency, probably", the handoff is where the next incident will be.

Setting the Scene at Kestrel & Pine: The Ownership Map

Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), had thirteen systems, 34 integrations and 41 installed apps when Noor Visser became Head of Engineering in January 2027. In her sixth week, on Monday 8 February, the **Deprecated Endpoint** happened: the marketplace connector retired an API version on a date it had published months earlier, an agency script that used that version failed, and stock to bol was stale for two days before a bol seller-support escalation about canceled orders exposed it. The agency said the script had been written in 2021 for a one-off and was not in its retainer. Milan Petrov, who owns the marketplaces, had not known the script existed.

Noor spent February drawing an **ownership map**: one row per system, integration and app, with a business owner, a technical owner, the vendor or builder, the contract and its renewal date, the API version it depends on, and what it may write. Most flows run through the **iPaaS** (integration platform as a service, a hosted tool for building integrations from prebuilt connectors) that Ruben Alves' team owns; the order management system (**OMS**) and product information system (**PIM**) that Waypoint adds in 2027 get rows of their own. Her January inventory had found **13 of the 34 integrations with no named owner**; the map assigned owners to eight of them. The tallies below are the map as it stood in **April 2027**:

What	Count	Owned by the integration team	Owned by the agency	Owned by a vendor or partner	No named owner
Integrations	34	14 (flows in the iPaaS, Ruben Alves' team)	9 (scripts, mostly from 2019–2022)	6 (vendor connectors, including the ERP partner's platform↔ERP connector)	**5**

What	Count	Owned by the integration team	Owned by the agency	Owned by a vendor or partner	No named owner
Installed apps	41	—	22 installed by the agency	—	19 installed by various people; 6 unused; **3 write inventory**
Systems	13	Business owner known for all; technical owner known for 8	—	—	5 systems with no technical owner inside the company

The five integrations still unowned in April included the bol stock script, a price-list export to the German storefront (the one behind the **Price Drift** of October 2026: €179 instead of €199 for nine days, about €8,200 of margin) and the pre-order app's inventory write (the **App That Wrote Stock**, March 2026). Three of the ten incidents in the ledger sat in the blank cells of one spreadsheet. That is the whole case for the map: **the incidents were not caused by bad code; they were caused by code with no owner.**

What Owning Means

Before team shapes, a definition, because "owner" is used loosely. To **own** a system, integration or app is to carry five duties: to **decide** its contract (what it does, which fields, which direction, how often, which system is the source of truth); to **change** it, or approve changes; to **watch** it, with a monitor and a reconciliation that says when it is wrong; to **fix** it, which means being the one called at 20:40 on Black Friday; and to **retire** it when it is no longer needed or when the vendor retires what it depends on. An agency can do the second and, under contract, the fourth; it will rarely do the first, the third or the fifth, because it is not paid to.

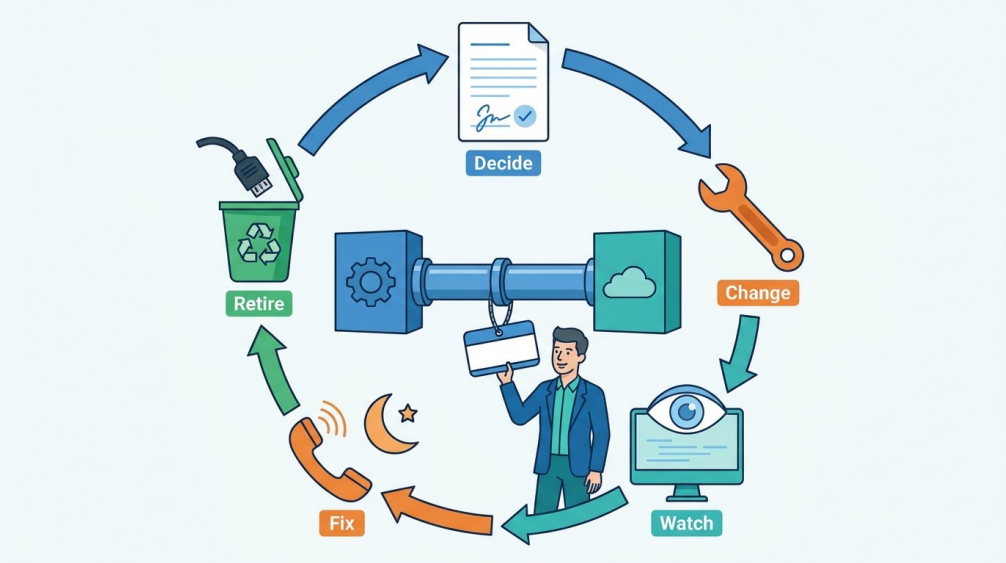


Figure 1: *To own a system, integration or app is to carry five duties: decide its contract, change it, watch it, fix it and retire it.*

Ownership follows money unless it is designed. The vendor-side economics are public. Shopify's partner page (undated, accessed November 2026) pays a partner that builds or transfers a store 20% of the monthly platform fee plus 0.1% of eligible GMV (gross merchandise value: the order value that passes through the platform) for four years, and a one-time $2,500 for a Plus or enterprise referral; partners are tiered Silver to Diamond on attributed GMV and attributed shops, and the older tiers retire on 1 January 2027. An agency on such a program is paid by the platform for keeping the merchant on it and by the merchant for building. Nothing in that arrangement pays for owning a connector after launch, for a reconciliation job, or for an exit. This is not a criticism of agencies; it is the reason the merchant must design ownership rather than inherit it.

A few terms the rest of the chapter uses. A **system of record** is the one system whose value is the truth when copies disagree (Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record). An **integration contract** is a short written agreement for one flow: fields, direction, frequency, source of truth, what happens on a repeat or a failure (APIs, Events, Files and Middleware: How Commerce Systems Talk). An **ADR** (architecture decision record) is a one-page note of a decision, its context and its consequences. A **sandbox** is a copy of a service where nothing real happens. An **API version** is a dated edition of a vendor's interface that the vendor promises to keep working for a stated period. A **stream-aligned team** owns one flow of work end to end; a **platform team** provides internal services to other teams.

The Ownership Map

The map is a table, kept where the integration inventory of APIs, Events, Files and Middleware: How Commerce Systems Talk is kept, and reviewed quarterly. Its columns:

Column	What goes in it	Why
Item	System, integration or app, by role name	One row per thing that can fail
Business owner	The manager accountable for the outcome (Milan for the storefront, Sanne de Wit for the ERP, Daniel Okafor for the WMS)	Decides priorities and accepts risk
Technical owner	The named engineer, team or partner who decides, changes, watches, fixes and retires	The person called
Builder and vendor	Who built it and who runs it	Separates "who wrote it" from "who answers for it"
What it may write	The facts it changes (stock, price, order status, refunds) and in which system	The Secret Agent check: one writer per fact
Contract	Renewal date, notice period, exit terms, support hours	Vendor management lives here
Depends on	API versions pinned, platform release tested against, vendor deprecation calendar	The Pulled Rug check
Watched by	The monitor and the reconciliation job, and who reads them	Ownership without watching is a name on a list

Two rules make the map more than documentation. **Every row has a technical owner inside the company**, even when the builder is a partner; the owner may be the person who manages the partner, but the row is never blank. And **no fact has two writers**: if two rows claim to write stock in the platform, one of them is wrong, and the map says which. At Kestrel & Pine the second rule removed the pre-order app's inventory

permission and became Waypoint decision W1's organizational half.

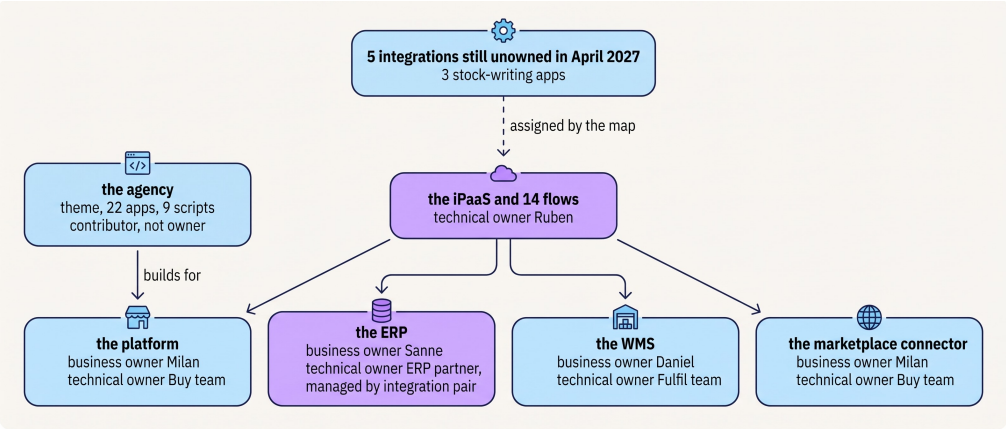


Figure 2: *Kestrel & Pine's ownership map in outline: every system has a business owner and a technical owner inside the company, the agency is a contributor, and the five integrations still unowned in April and the three stock-writing apps are assigned to the integration pair until retired or re-homed.*

Team Shapes for a Commerce Merchant

Conway's 1968 paper observed that organizations "are constrained to produce designs which are copies of the communication structures of these organizations." In a merchant this works in reverse: a landscape of a dozen vendors pushes the organization into one team, or one person, per vendor, and the handoffs between systems become handoffs between people who report to different managers. **The organization copies the vendor landscape, and the vendor landscape's gaps become the organization's.**

Team Topologies (Skelton and Pais, 2019) gives the vocabulary that fits a small team. Four team types: **stream-aligned** (aligned to a flow of work from a segment of the business domain), **platform** (an internal product that accelerates stream-aligned teams), **enabling** (helps a team overcome obstacles and detects missing capabilities) and **complicated-subsystem** (where deep technical expertise is needed). Three interaction modes: collaboration, X-as-a-service, facilitation. One premise: teams can only handle so much complexity before breaking down. The vocabulary was written for software companies; the mapping to a merchant is mine.

The options

By system (one team or person per vendor). The default a vendor landscape produces: the platform person, the ERP person, the WMS person, the agency for the storefront.

Cheap to set up, matches support contracts, and reproduces every integration gap as an organizational gap. Nobody owns the order's journey.

By journey (stream-aligned). One team owns "buy" (storefront, catalog publication, checkout, marketplaces listings, promotions) and one owns "fulfill and return" (order hub, warehouse integration, shipping, returns, EDI, the electronic exchange of orders and invoices with trading partners). Each team touches several vendors; each handoff inside a journey has one owner. The integrations between the two journeys (the order leaving checkout for the OMS) need a contract between the teams.

By layer (front end, integration, data). A storefront team, an integration team, a data team. Fits the skills most merchants hire (front-end, back-end, data) and makes the integration layer a product with an owner. The risk is that the integration team becomes the bottleneck for every change and nobody owns a journey end to end.

Agency-led. The agency owns the storefront and the apps; the merchant owns the accounts and a backlog. Right at the Starting stage, workable at the Growing stage, and the origin of most Omnichannel-stage incidents when the merchant has not taken the joins in-house.

	By system	By journey	By layer	Agency-led
Who owns the order's journey	Nobody	A team	Nobody; the layers do	The agency, implicitly
Who owns the integration layer	The vendor connector or nobody	Must be assigned separately	The integration team	The agency or nobody
Fits team size	Any	8 and up (two teams of 3–5 plus a pair)	10 and up	1–5
Handoffs become	Tickets between people	Contracts between two teams	Queues into the integration team	Change requests
Main risk	The Spaghetti Junction in the org chart	Both teams need the same integration pair	The bottleneck	The Lynchpin

	By system	By journey	By layer	Agency-led
When it fits	Starting; a Group's shared-service vendor manage-ment	Omnichannel	Omnichannel with a strong integration lead	Starting, early Growing

My recommendation for ten to twenty people combines the second and third: one or two stream-aligned teams by journey, one pair (not a team) that owns the integration layer, the ownership map and the reconciliation jobs as an internal product, the data engineers as a second small platform function under the data lead, and the vendors and agency as X-as-a-service providers with written contracts. The pair, not a team, because at this size a full integration team becomes the bottleneck the by-layer option warns of. The condition under which this does not hold: fewer than eight engineers, where one journey team plus one integration engineer is all the structure the people can carry.

Two public cautions bound the autonomy a merchant gives its teams. Henning Jacobs' retrospective of March 2020 on a decade at Zalando reports that team autonomy over technology choice produced similar services "written in six languages", which "hurts productivity at scale", and that a standard stack would have been "a major win". Uber's July 2020 account of its domain-oriented architecture describes about 2,200 microservices in which a root-cause investigation could run through fifty services owned by twelve teams. Both are six years old and describe organizations a hundred times larger than any merchant; the lesson scales down without change: **journey teams need a shared integration standard, or each will build its own connector to the same ERP**. Werner Vogels' 2006 line "you build it, you run it", relayed by AWS's blog at the time, is the other half: the team that changes a flow is the team paged when it fails.

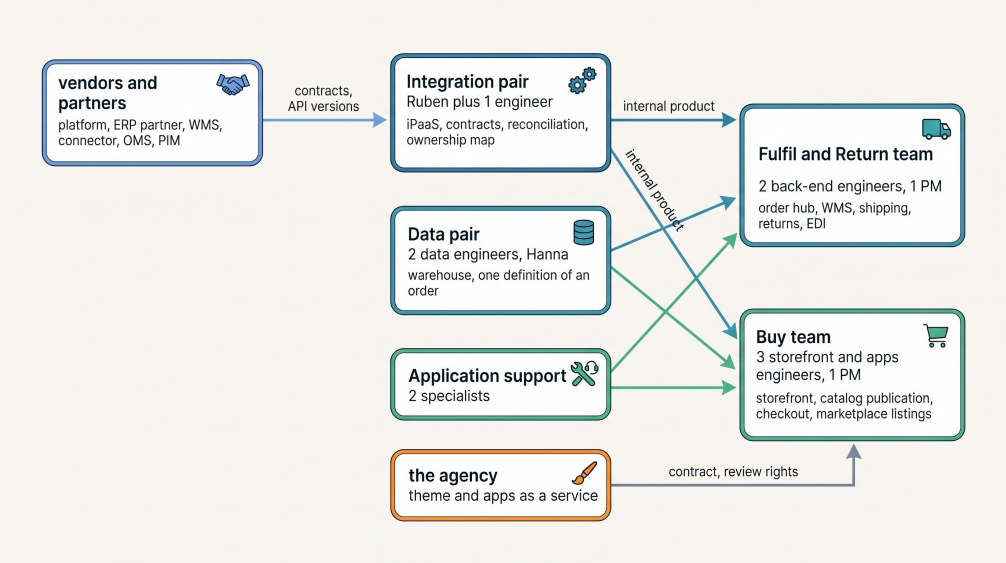


Figure 3: *Kestrel & Pine's 2027 team shape: two journey teams, an integration pair that serves both as an internal product, a data pair, shared application support, and the agency and vendors as services under contract.*

Working With Agencies and Implementation Partners

An agency or implementation partner builds what the merchant cannot staff: the theme, the first apps, a replatforming, an ERP implementation. Hobbycraft's 2022 move to Salesforce Commerce Cloud, reported by a trade site in June 2022, was built by an agency after a competitive pitch, using the agency's accelerator and a stack of six third-party services; the article reports the build, not the outcome, which is typical of agency-sourced evidence. The vendor-side mechanisms for agency work are structural: Shopify's development stores exist so a partner can build a store and transfer it to the merchant, with real transactions disabled until transfer (undated, accessed November 2026); Salesforce's cartridge path layers partner-built code over the vendor's base.

The rules I use, stated as judgement because no dated study of merchant–agency practice in the Netherlands, Belgium or Germany could be found:

- **The merchant owns the repository, the vendor accounts, the API credentials, the ownership map and the decision records.** The agency commits to the merchant's repository and holds review rights, not ownership. If the agency holds the platform login, the merchant has a Lynchpin.
- **Every custom module, app or script the agency builds is a row in the map** with its scopes (what it may read and write), its pinned API version, the platform

release it was tested against, and an exit plan: who could maintain it if the agency left.

- **The statement of work names the API versions and platform releases the deliverable is tested against**, and a deprecation clause: when a vendor announces a retirement, who updates what by when.
- **Ownership after launch is written down before launch.** Either the agency owns the row under a support contract with a named engineer, or the merchant's team does, from the day of go-live. "We'll see" means nobody.
- **Separate building from keeping.** Pay the agency for the first; staff the second.

Kestrel & Pine's agency has been retained since 2017 at €14,000 a month for the theme, apps and small integrations. The 2027 statement of work, after the map, lists each of its nine scripts with an owner and a pinned version; three scripts that write to stock or prices moved in-house to the integration pair, and the rest stay with the agency under a named engineer *(illustrative).* The ERP partner's platform↔ERP connector, built in 2023, gets the same treatment: the partner maintains it; Ruben's pair owns the contract, the monitor and the reconciliation.

Vendor Management: What the Contract Must Say

A merchant on rented software is governed by its vendors' terms more than by its own policies. Shopify's Terms of Service, last updated 20 October 2026, provide the services "as is" and "as available", do not warrant that they "will be uninterrupted, timely, secure, or error-free", and reserve the right to modify the services "for any reason, without notice and at any time". Plus and enterprise agreements may add service levels; none was available to verify, so ask for the actual agreement rather than a marketing uptime figure.

What a merchant can govern is its own dependence. The **vendor card**, one per vendor, sits beside the ownership map:

Field	What to record	Dated example
Term and notice	Contract length, renewal, notice period	Shopify Plus is sold on one- or three-year terms (pricing page, accessed 2026-11)

Field	What to record	Dated example
Versioning and deprecation	How the vendor versions its API and how long versions live	Shopify releases an API version quarterly, supports each for at least twelve months, and "falls forward" retired versions to the oldest supported one (developer documentation, accessed 2026-11); Adobe Commerce gives each release three years of standard support and will stop maintaining cloud environments on unsupported versions from 1 June 2027 (lifecycle policy, accessed 2026-11); WooCommerce states that backward compatibility between releases "is not guaranteed"
Rate limits and quotas	What the tenant may call, shared by every app and connector	Shopify's GraphQL Admin API bucket refills at a plan-dependent rate, from 100 to 2,000 points a second (accessed 2026-11); every app on the store draws from it (Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation)

Field	What to record	Dated example
Export and exit	What data can be exported, in what form, at what cost	The EU Data Act (Regulation 2023/2854) prohibits switching charges for data processing services from 12 January 2027; whether hosted commerce platforms fall under it is an open legal question as of November 2026
Extension retirement	Which mechanisms the vendor has retired and on what notice	Shopify Scripts and the Script Editor were retired on 30 June 2026; scripts no longer run on any store (help documentation, accessed 2026-11)
Service levels in your units	Not the vendor's uptime but your checkout success rate, order-to-WMS latency, stock-sync lag	Google's SRE book (2017) defines an SLO as a target for a measured service level and an SLA as the contract with consequences; the vendor's SLA covers its uptime, and the merchant's integration SLOs are nobody else's
Residency and tenancy	Which region each SaaS system was provisioned in	commercetools' regions are isolated and moving is a migration, not a setting (documentation, accessed 2026-11)

The versioning row deserves emphasis because it is the Pulled Rug's calendar. Kestrel & Pine's Deprecated Endpoint was a published date that nobody had put on a calendar. The vendor card turns each vendor's deprecation policy into a dated list, and the ownership map says who updates which row by when. Promises Between Systems: API Contracts, Webhooks and Versioning owns the mechanics.

Governance That Fourteen People Will Follow

Governance in a merchant has one purpose: to keep the ownership decisions true after the meeting ends. Heavy process (architecture boards, review gates, templates that take an afternoon) will not survive a team of fourteen with a peak season. Light process can, if it is specific.

Decision records. Michael Nygard's note of November 2011 defined the architecture decision record: a short text file with title, context (the forces at play, "described as neutral facts, often in tension"), decision ("We will…"), status (proposed, accepted, deprecated, superseded) and consequences. Without one, a newcomer can only "blindly accept the decision" or "blindly change it". In a merchant the decisions worth a record are the ownership ones (which system is the system of record for stock, price, product, order), the vendor ones (why this OMS, with what exit) and the app ones (which apps may write stock). Kestrel & Pine's Waypoint decisions W1–W8 are ADR-shaped; Noor numbered them as the first eight records *(illustrative)*, and the 2029 review is the date on which W4 is re-opened.

Standing rules. Rules that need no meeting, because they are checkable from the map:

Rule	Check	Pattern it prevents
One writer per fact	The map's "what it may write" column has no duplicates	The Secret Agent, The Phantom Stock
Every integration has an owner, a contract card, idempotent handling and a reconciliation job (Waypoint W5)	No blank cells; each row names its reconciliation	The Spaghetti Junction, The Message in a Bottle
Every app's scopes are recorded and reviewed quarterly; unused apps are removed	41 apps, 6 unused, 3 writing stock becomes 34 apps by July 2027, none unused and none writing stock	The Secret Agent
Every integration pins an API version, recorded with the vendor's retirement date	The vendor card's calendar has no past-due rows	The Pulled Rug

Rule	Check	Pattern it prevents
No change to stock, price or order flows in the four weeks before peak without the peak owner's approval	A dated freeze in the calendar	The First Snowfall

A lightweight review. One 45-minute **architecture hour** a week, open to both journey teams, the integration pair and the data pair, with a standing agenda: new rows in the map, proposed ADRs, deprecation dates in the next ninety days, and one incident replayed against the map. No slides. The output is edits to the map and to ADRs. At Kestrel & Pine this hour replaced three vendor-specific status meetings *(illustrative)*.

The Product Manager in a Merchant

A merchant that buys everything still needs its own product managers, for a reason of asymmetry. The vendor's product manager optimizes the platform's product: GMV through it, tenancy, the app ecosystem. The merchant's product manager's product is **the order's journey**: the share of visits that become orders, the average order value, the share of orders delivered when promised, the return rate and the cost of a return, the time from "where is my order?" to an answer. Those are the metrics of Who Sells What to Whom: Business Models and the Economics That Shape Architecture, and no vendor optimizes them for a particular merchant. Kestrel & Pine's two product managers sit one per journey team; their backlogs are written in those metrics, and a vendor's roadmap is an input, not a plan.

Engineering Practices on Rented Software

Environments and sandboxes. On SaaS the merchant never gets a full copy of production. Shopify's development stores are for development and testing only, cannot process real transactions and are capped at 250 per organization (accessed November 2026); Stripe's sandboxes are isolated test environments where "card networks and payment providers don't process payments" (accessed November 2026); Adobe Commerce Cloud Pro provides integration, staging and production environments, with staging on dedicated hardware matching production (accessed November 2026). The gap is the chain: no vendor offers a sandbox of platform plus payment provider (PSP) plus ERP plus WMS together. Zalando's 2020 account of its Cyber Week preparation describes order-flow simulators that place marked test orders which fulfillment skips, exercising inventory and payments in production. A small team can copy the pattern at small scale: a test customer, a test SKU, an order flag the WMS and the ERP recognize, and a weekly run through the whole chain.

Release management. The release calendar is set by the vendors: quarterly API versions on hosted platforms, yearly release lines on suites, connector updates whenever a marketplace or carrier changes its interface. The cheapest control is to pin the version in every connector and record it in the map.

Feature flags. Pete Hodgson's taxonomy of October 2017 (release, experiment, ops and permissioning toggles) gives the team a vocabulary: release toggles to ship unfinished checkout changes dark, ops toggles as the peak-day switch-off list of Black Friday Every Year: Peak Load, Resilience and Recovery, permissioning toggles for staff-only or B2B-only features.

AI coding tools. The dated evidence is mixed and worth stating plainly. DORA's 2024 report (October 2024) found more than 75% of respondents relying on AI for at least one daily task, with a 25% increase in adoption associated with small gains in documentation and code quality but an estimated 1.5% lower delivery throughput and 7.2% lower delivery stability. DORA's 2025 report calls AI "an amplifier, magnifying an organization's existing strengths and weaknesses". METR's randomized trial of July 2025, with sixteen experienced developers on 246 tasks in mature repositories, measured a 19% *increase* in completion time with early-2025 tools, while the developers believed they had been 20% faster. Shopify's CEO wrote to staff in April 2025 that "reflexive AI usage" is a baseline expectation and that teams must show why AI cannot do a task before asking for headcount. For a commerce team the stability finding is the one that matters: AI tools raise output on exactly the code that touches stock and prices (connectors, theme code, app glue), so tests against sandboxes, small batches, review and the one-writer rule are preconditions, not afterthoughts. AI shifts the bottleneck from writing a connector to owning it.

Across Scale Stages: When It Fits and When It Limits

	Starting	Growing	Omnichannel	Group
Typical shape	Founder plus the platform's support; an agency for the theme	The agency builds; a first engineer; the ERP partner	10–20 people: journey teams, an integration pair, a data function	Stream-aligned teams per brand or journey; a platform team; vendor management as a function
Who owns integrations	The platform's own connectors; nobody else needed	The agency or the partner, then nobody after launch	The integration pair, by map	The platform team, as an internal product with API guidelines
What becomes necessary	The founder holds every account; an app list with scopes	The first engineer owns the map, the repository and the API pins; a staging store with marked test orders	Named owner per row; ADRs; the architecture hour; quarterly app review; a peak owner	Shared standards across teams; an acquisition integration playbook; group vendor contracts
Warning signs of outgrowing it	A second channel arrives and the agency builds the connector with no owner	Support tickets route to the agency; an incident reveals a script nobody knew	Both journey teams build their own connector to the same ERP; the integration pair is the bottleneck for every change	Teams choose their own stacks and build the same service twice (Zalando's six languages)
Kestrel & Pine	2016–2018	2019–2022: Ruben Alves from 2022	2027 shape above	Deferred to the 2029 review

The judgement: the team shape should change one stage *before* the system landscape does, because the owner has to exist before the connector is built. Kestrel & Pine hired its lead integration engineer in 2022, a year after the connector and the iPaaS arrived; the thirteen integrations without a named owner in January 2027 are the cost of that year.

In Practice: Kestrel & Pine, Before and After

Before (January 2027). Thirteen systems; 34 integrations of which thirteen had no named owner; 41 apps of which three wrote inventory; the agency holding the theme, 22 apps and nine scripts on a €14,000 monthly retainer; the ERP partner holding the platform↔ERP connector; one engineer (Ruben) owning the iPaaS and, informally, everything nobody else owned; technology spend of about €2.6 million a year. Support answered "where is my order?" from five tabs. Nobody had a calendar of vendor deprecation dates.

After (from April 2027). The ownership map with no blank cells. Two journey teams (Buy: three storefront and apps engineers and a product manager, with Milan as business owner; Fulfil and Return: two back-end engineers and a product manager, with Daniel and Lea as business owners), the integration pair (Ruben and one back-end engineer) owning the iPaaS, the contracts, the reconciliation jobs and the map, the data pair under Hanna Berg, and two application support specialists serving both journeys. ADRs KP-001 to KP-008 are Waypoint W1–W8. The two integration engineers Waypoint adds from mid-2027 join one journey team each, so the integration pair stays a pair. A vendor card per vendor with a deprecation calendar. The agency under a statement of work that lists each script with an owner and a pinned version. The architecture hour on Tuesdays.

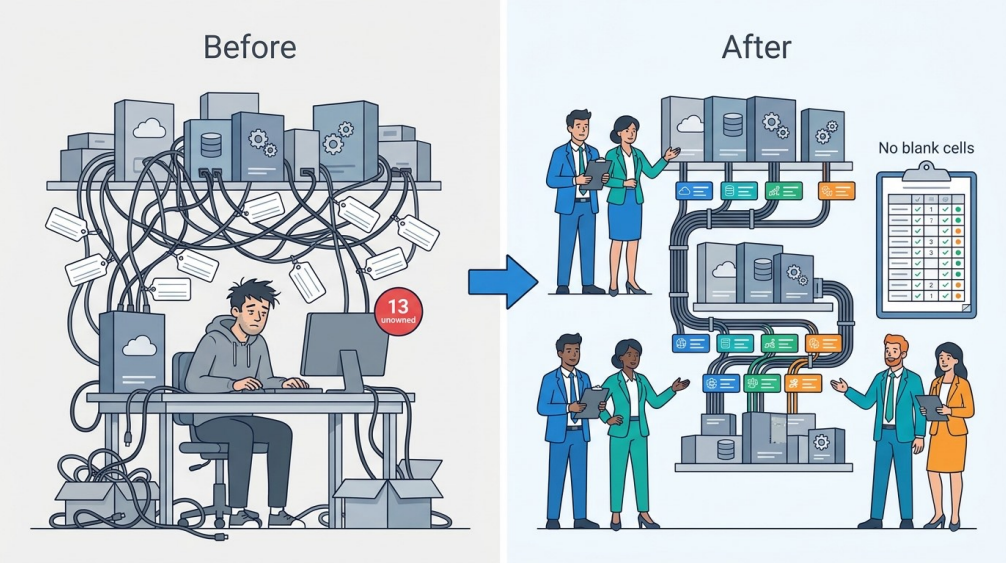


Figure 4: *Before, 13 of 34 integrations had no named owner and one engineer held whatever nobody else owned; after, the ownership map has no blank cells.*

The Deprecated Endpoint, replayed. With the map, the bol stock script has a row: technical owner the integration pair, pinned version recorded, the connector's retirement date on the vendor card ninety days ahead, and a monitor on stock-sync lag to bol. The script is updated in the week the retirement is announced, or retired in favor of the connector's native stock flow. The two stale days do not happen. The cost of the control is one row and one calendar entry.

When It Goes Wrong

The Lynchpin leaves. The one EDI consultant who built Bergzicht's connection, or the one engineer who knows the iPaaS, resigns. Detection: the map shows one name on many rows. Recovery: pair every critical row with a second name, document the contract card, and treat a single-name row as a risk register entry with a date.

The agency contract ends. On exit the agency holds the theme repository, the app credentials and the scripts. Detection: the merchant cannot deploy a theme change without the agency. Recovery: repository and credentials transfer as a condition of every statement of work, from the first one; a handover run through the map, row by row.

The vendor retires the mechanism. Shopify's retirement of Scripts on 30 June 2026 is the dated example of a platform ending an extension mechanism on a published date; Adobe's June 2027 enforcement of supported versions on its cloud is another. Detection: the vendor card's calendar. Recovery: migration to the replacement mechanism

(Functions, in Shopify's case) planned a year ahead, owned by the row's owner (Apps, Plugins and Functions: Extensibility and Ecosystem Governance).

The Secret Agent returns. A new app, installed by a marketer to run a pre-order campaign, asks for an inventory-write scope and gets it. Detection: the quarterly app review, or better, an install rule that routes any write scope on stock, price or orders through the integration pair. Recovery: revoke the scope, reconcile the stock the app touched (the May 2027 reconciliation found 2.3% of active variants different, most from such apps), and record the decision.

Two teams, two connectors. Both journey teams integrate with the ERP, each its own way. Detection: two rows in the map with the same target and different contracts. Recovery: the integration pair owns one flow and the other team consumes it; this is what the pair is for.

What to Take Away

Established.

- Organizations produce designs that copy their communication structures (Conway, 1968); a merchant organized one team per vendor reproduces its integration gaps in the organization chart.
- Vendor partner programs pay agencies to build and to keep merchants on the platform (Shopify: 20% of the platform fee plus 0.1% of GMV for four years, accessed 2026-11), not to own integrations after launch; ownership must be designed by the merchant.
- Decision records with context, decision, status and consequences (Nygard, 2011) are the lightest durable form of architecture governance and fit a team of any size.
- Autonomy without shared standards is costly at scale (Zalando's 2020 "six languages"; Uber's 2020 fifty-service investigations), and the lesson holds for two journey teams sharing one ERP.

Depends on context.

- Journey teams plus an integration pair fit ten to twenty people at the Omnichannel stage; below eight engineers, one journey team and one integration owner is all the structure the people can carry; at the Group stage the pair becomes a platform team.
- Agency-led engineering is right at the Starting stage and workable at the Growing stage, provided the merchant holds the accounts, the repository and the map from the first contract.

- How much to formalize vendor management depends on the vendors' versioning policies: quarterly dated API versions with twelve-month support (Shopify) need a calendar; three-year release lines (Adobe Commerce) need an upgrade plan; "not guaranteed" compatibility (WooCommerce) needs a test suite.
- AI coding tools raise output where the evidence says stability falls (DORA 2024); their value in a commerce team depends on the tests, review and scope governance already in place.

The author's synthesis.

- The ownership map is the merchant's first architecture artifact and the only one that prevents incidents on its own: at Kestrel & Pine three of ten incidents sat in its blank cells.
- Change the team shape one stage before the system landscape changes; the owner must exist before the connector does.

Questions to Ask

1. Is there a map with one row per system, integration and app, and does every row name a technical owner inside the company?
2. Which facts (stock, price, order status, refunds) have more than one writer, and which row says so?
3. Who holds the platform, PSP, ERP and connector logins, the repository and the API credentials: the merchant or the agency?
4. Which vendor deprecation dates fall in the next twelve months, and who updates which integration by when?
5. Does the agency's statement of work name the API versions and releases the deliverable is tested against, and who owns each deliverable after go-live?
6. Does any team own the order's journey end to end, or only systems?
7. Is there a written decision for which system is the system of record for stock, orders and product content, and when it will be reviewed?
8. How does a change to a stock, price or order flow reach production: through which review, which sandbox, which flag, and under which freeze before peak?
9. Which rows in the map carry one name only, and what happens if that person leaves?

Ownership answers "who"; the last chapter of this part asks what the next kind of actor, an AI agent that discovers, compares and buys, will need from the systems and the owners described here. Continue with Assistants and Agents: AI in Commerce and What Is Real in 2026.

Sources and Evidence

Company, product and investor sources

- How to earn as a Shopify Partner, Shopify Help Center, n.d., accessed 2026-11 — partner revenue share and referral payouts.
- Shopify Plus Partner Program, Shopify Help Center, n.d., accessed 2026-11 — partner tiers by attributed GMV and shops; older tiers retiring 1 January 2027.
- Development stores, Shopify developer documentation, n.d., accessed 2026-11 — test stores cannot process real transactions; cap of 250.
- API versioning, Shopify developer documentation, n.d., accessed 2026-11 — quarterly versions, twelve-month support, fall-forward behavior.
- GraphQL Admin API rate limits, Shopify developer documentation, n.d., accessed 2026-11 — plan-dependent bucket shared by every app on a store.
- Script Editor, Shopify Help Center, n.d., accessed 2026-11 — Scripts retired on 30 June 2026.
- Shopify Terms of Service, Shopify, 2026 — services provided "as is" and modifiable without notice.
- Shopify CEO: prove AI can't do the job before asking for headcount, CNBC, 2025 — the April 2025 memo.
- Adobe Commerce lifecycle policy, Adobe Experience League, n.d., accessed 2026-11 — three-year standard support; cloud enforcement from 1 June 2027.
- Adobe Commerce Cloud Pro architecture, Adobe Experience League, n.d., accessed 2026-11 — integration, staging and production environments.
- Getting started with extensions, WooCommerce developer documentation, n.d., accessed 2026-11 — backward compatibility not guaranteed.
- Stripe sandboxes and test mode, Stripe documentation, n.d., accessed 2026-11 — isolated test environments where no payments are processed.
- General concepts, commercetools documentation, n.d., accessed 2026-11 — isolated regions; data residency chosen at provisioning.
- Astound Commerce enlists Hobbycraft for e-commerce replatforming project, Retail Technology Innovation Hub, 2022 — an agency-led replatforming, agency-sourced.
- How Zalando prepares for Cyber Week, Zalando Engineering, 2020 — order-flow simulators with marked test orders; production-readiness reviews; older than three years.
- One decade in Zalando tech, Henning Jacobs, 2020 — the cost of autonomy without a standard stack; older than three years.
- Introducing Domain-Oriented Microservice Architecture, Uber Engineering, 2020 — ownership blur across about 2,200 services; older than three years.
- Werner Vogels interview relayed, AWS News Blog, 2006 — "you build it, you run it"; twenty years old.

Standards, regulation and official documentation

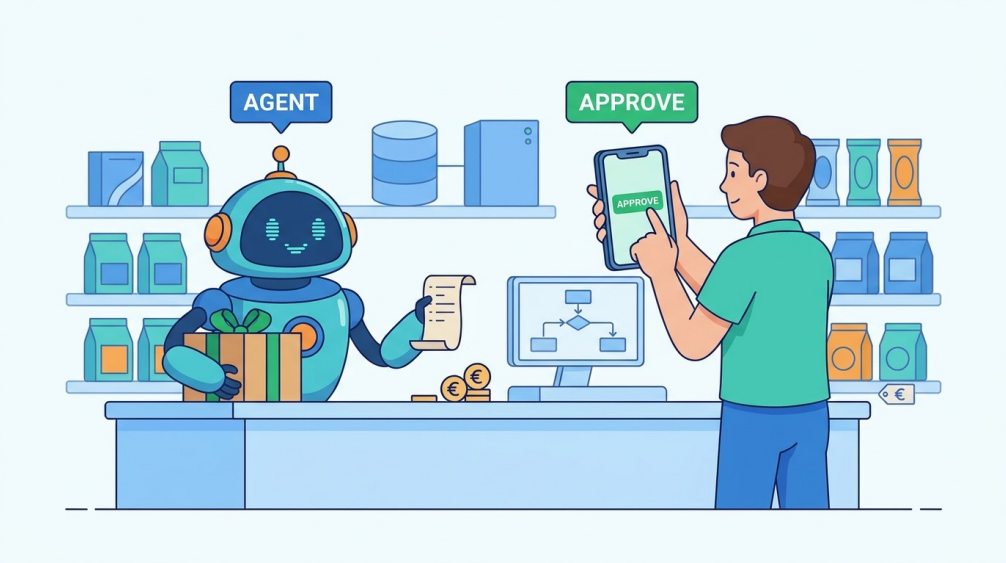
- Regulation (EU) 2023/2854 (Data Act), EU Publications Office, 2023 — switching charges prohibited from 12 January 2027; applicability to hosted commerce platforms open.
- Service Level Objectives, Google SRE book, 2017 — SLI, SLO and SLA definitions; older than three years.

Research and practitioner sources

- How Do Committees Invent?, Melvin Conway, 1968 — organizations produce designs that copy their communication structures.
- Team Topologies: key concepts, Matthew Skelton and Manuel Pais, n.d., accessed 2026-11 (book 2019) — the four team types and three interaction modes.
- Documenting Architecture Decisions, Michael Nygard, 2011 — the architecture decision record; older than three years.
- Feature Toggles, Pete Hodgson, 2017 — release, experiment, ops and permissioning toggles; older than three years.
- Announcing the 2024 DORA report, Google Cloud, 2024 — AI adoption associated with lower delivery throughput and stability.
- State of AI-assisted Software Development 2025, DORA, 2025 — AI as an amplifier of existing strengths and weaknesses.
- Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity, METR (arXiv), 2025 — the randomized trial showing a 19% slowdown.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

45. Assistants and Agents: AI in Commerce and What Is Real in 2026



IN THIS CHAPTER, YOU WILL: Separate what AI already does in commerce from what agents are promised to do, read the protocol landscape as of November 2026, and build the readiness list a merchant needs before any agent may buy.

KEY POINTS:

- **Established AI in commerce reads and proposes; an agent acts.** Search, recommendations, content, service bots, fraud scoring and forecasting are mature and measurable, and every published lift is a vendor figure. An agent that can place an order is a new kind of writer in your landscape, and it raises the question the whole book asks: who may change which fact.
- **Agentic commerce is real, early, and not yet in Europe.** Between September 2025 and November 2026 the flagship in-chat checkout launched and was pulled back, three protocols (ACP, UCP, AP2) were

published with no settled governance, and every live consumer checkout program is limited to the United States, Canada or Australia. European payment rules require the shopper to authenticate the exact cart.
- **Agent readiness is a data and API program, not an AI program.** Product data from one owner, availability and a delivery promise by API, order status by API, structured policies and an identity for every agent make a merchant ready for any protocol. Kestrel & Pine's Waypoint decision W8 buys exactly that in 2027, and no shopper-facing agent.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Secret Agent** — an AI agent or admin assistant that changes orders, stock, prices or money without its own identity, a named owner and a person's approval is this pattern's newest form.
- **The Phantom Stock** — an agent channel is the harshest consumer of availability, because a stale copy becomes a promise a third party makes for you.
- **The Giant's Boots** — building your own shopping agent, or implementing three competing protocols in the storefront, copies the vendors' problem into a fourteen-person team.

Two different things are called "AI in commerce", and a merchant who confuses them spends money on the wrong one. The first is **software that reads your data and proposes something**: a ranking, a recommended fleece, a draft description, a chat answer, a fraud score, a forecast. It is twenty years old, measurable and mostly bought, and it changes nothing about who owns your facts. The second is **software that acts**: it adds to a cart, places an order, changes a price, moves stock. It is new, sold hard, and a new writer of your facts.

A shopping assistant that reads a stale availability figure promises a jacket the warehouse sold two hours ago, and the cancellation carries your name; an admin assistant that can "fix" inventory is a script with no owner. **The failure modes of AI in commerce are the failure modes this book has described all along**, reached faster and through a new door.

This chapter owns **agentic commerce**, the **commerce protocol** and **agent readiness**. Agent internals belong to the sibling book *Architecting Agentic Systems*: Tool Ecosystems, MCP and the Tool Broker for how agents call tools, Side Effects: When Agents Touch the World for how an agent may act on the world, and Agent Identity, Delegation and Least Privilege for who the agent is.

Where we are in the order's journey. At both ends of Eva's jacket: an assistant may be where she starts ("a waterproof women's shell jacket under €200, delivered by

Thursday"), an agent could pay, and after delivery a bot asks "where is my order" and "can I exchange it for a size L".

Setting the Scene at Kestrel & Pine: The Agent Test

Kestrel & Pine, the fictional outdoor brand this book follows, met the question in February 2027, three times in one week. The agency proposed a "Kestrel & Pine shopping assistant" built on a large language model, at an illustrative €60,000 to build and €2,500 a month to run *(assumption)*. Veluwe Capital's board observer asked Mara Lindqvist whether the brand "shows up in ChatGPT". And Milan Petrov found that the platform's vendor had switched on an "AI channel" by default, with checkout limited to shoppers in the United States (an assumption about the fictional platform that mirrors the public programs below).

Noor Visser, Head of Engineering since January, answered with a test rather than an opinion. An agent, whatever protocol it speaks, asks a merchant five questions about a product. She asked them about the Ridgewalker shell jacket, women's, size M, color moss, at two moments: 20:00 on Friday 27 November 2026, the hour of the Black Friday Oversell, and 21:14 on Tuesday 13 April 2027, the minute of Eva's order. The systems are those of Meet Kestrel & Pine: the platform, the WMS (warehouse management system) at Duiven, the iPaaS (the integration platform between them) and the returns portal; Waypoint adds the PIM (product information management system, the one owner of product content) and the OMS (order management system, the one order record).

What an agent asks	Which system answers today	True on 27 Nov 2026, 20:00?	True on 13 Apr 2027, 21:14?
What is it?	The platform's product record; the waterproof rating only in a spreadsheet	Partly: no rating, three copies of the description	Same
Is it in stock?	The platform's stock copy, up to two hours behind the WMS	**No**: sold out at 19:40; the copies said yes until about 20:30 (storefronts) and 21:35 (marketplaces)	Yes, by luck

What an agent asks	Which system answers today	True on 27 Nov 2026, 20:00?	True on 13 Apr 2027, 21:14?
When will it arrive?	A sentence in the theme: "ordered before 22:00, delivered tomorrow"	**No**: the WMS queue was four hours behind	**No**: the 21:00 wave had gone; delivered Thursday, not Wednesday
What does it cost delivered, can I return it?	Price in the platform; policies in page text	Price yes; policies not machine-readable	Same
Where is my order, can I exchange it?	Platform, WMS via the iPaaS, returns portal: no single answer	The Three-Status Order was two weeks away	Mostly, for one parcel

Two of five answers were wrong on the Oversell night, one on an ordinary evening, and none came from a system that owned the fact. **An agent would not have fixed any of this; it would have broadcast it.** Noor's memo ended with one line, which became Waypoint decision **W8**: agent readiness, not agents. The agency's assistant was declined; the vendor's AI channel stayed on for discovery only.

Two Kinds of AI, and Why the Difference Is Architectural

In plain words: A **model** is software trained on data to produce an output from an input: a ranking, a sentence, a score. A **large language model** (LLM) is a model trained on text that can read and write natural language, which is what makes chat assistants possible. An **assistant** is an LLM wrapped in a product that helps a person. An **agent** is an assistant that has been given tools and permission to use them: it can call an API that creates a cart, places an order or changes a stock figure. **Agentic commerce** is the business term for agents that discover, compare and buy on a shopper's behalf, inside a chat product or a marketplace app.

The line between assistant and agent is the line this book draws everywhere: **reading a fact versus writing one**. Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record asks which system may change which fact; Who Builds and Who Owns: Teams, Vendors, Agencies and Governance insists on one writer per fact with a named owner. An assistant that proposes a description is a reader. An agent that adjusts inventory is a writer, and the moment it exists your landscape has a new entry in its writer list, whether or not anyone put it there. For a proposal, the person who acts on it is accountable; for an action, nobody is, unless you decided in advance.

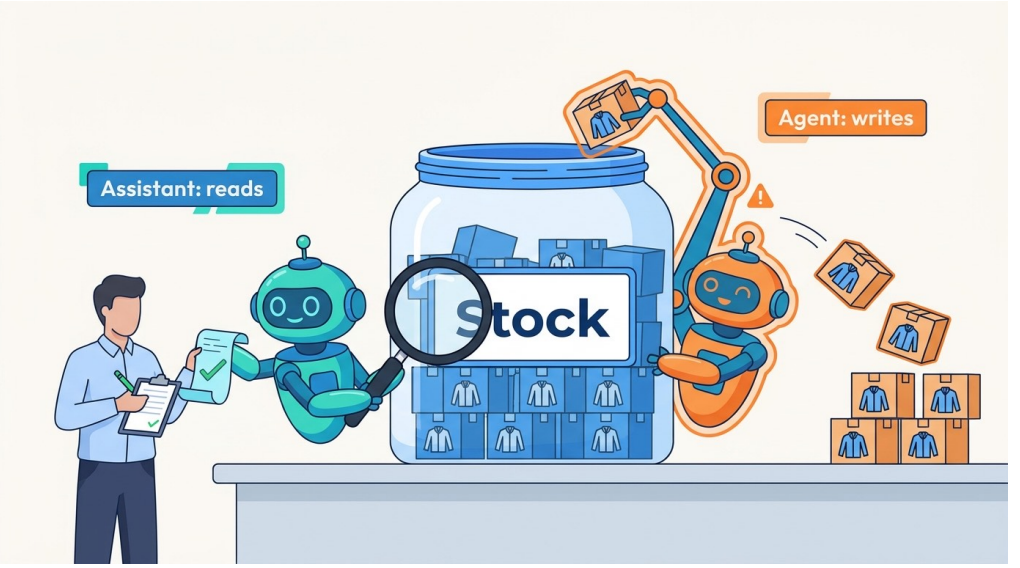


Figure 1: *The line between assistant and agent is reading a fact versus writing one: the assistant reads and proposes, the agent changes the fact.*

What AI Already Does in Commerce

Six uses are established. Each is worth having at the right stage, and each has a public record thinner than the marketing.

Search and recommendations. Amazon's item-to-item collaborative filtering (recommending from what other shoppers bought together) was described in a 2003 paper; its 2019 history records that a deep-learning model first lost to the old method and a plain bestseller list: **a strong baseline is hard to beat**. Hosted "AI search" adds vector search, which matches meaning so that "waterproof jacket" finds "rain shell"; every published lift is a customer case without a control (Finding the Product: Search, Navigation and Merchandising). Zalando's assistant, a ChatGPT-based beta in April 2023, had "more than 22 million unique users" in 2026 by its Q3 2026 release (3 November 2026).

Content. Generated listings are common (Amazon reported more than 100,000 sellers using its tools in September 2023). The architectural point is where generated text lands: **in the system of record for product content, with a review state, or the channels drift apart** (What Are We Selling? Products, Variants and the Catalog).

Service bots. Klarna said on 27 February 2024 that its assistant handled 2.3 million conversations in its first month, two thirds of its chats, the work of 700 agents. In May 2025 its CEO told Bloomberg it was rehiring humans because the bots produced "lower quality" output: measure resolution, not **deflection** (the share of contacts a bot absorbs).

A bot is only as good as the APIs behind it: order status, shipment events, return eligibility and refund state, the same APIs an external agent needs, and the ones Kestrel & Pine's "five tabs" could not provide (Where Is My Order? Notifications, Service and the Customer Record).

Fraud. Scoring belongs in the PSP (the payment service provider that takes the shopper's money) or a specialist; Stripe's Radar page claims a "32% average reduction in fraud" with no accuracy or false-positive rate published. Agents break the premise of bot defense, that a non-human session is hostile (Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers).

Forecasting. Deep demand forecasting (Amazon's DeepAR paper, 2017) produces a purchase proposal a buyer approves, and it needs clean sales and stock-out history, which a merchant with six writers of stock does not have.

AI traffic. Adobe Analytics reported in January 2026 that visits to US retail sites referred by generative-AI products rose 693% year on year in the 2025 holiday season, growth from an unpublished, small base.

What the six have in common: each reads the merchant's data, and each writes either a derived copy that goes stale or a proposal a person approves. None of them owns a fact.

Assistants in the Admin: The Secret Agent Wears a Badge

The second place AI arrived is the back office. Shopify announced Sidekick in July 2023 and said on 21 May 2025 that it could run reports, create segments, filter orders and set up discount codes "with the merchant's approval"; its help page (accessed November 2026) describes it editing orders and presenting changes for review before applying them, and its Fall 2026 edition claims daily sessions up 4.2 times year on year, a vendor figure. Shopware (June 2026) and Salesforce (July 2026) followed with their own admin agents.

Three things are the same across all of them: **the assistant acts through the platform's own APIs and permission model**; **every vendor describes a draft-and-review step**; and **the assistant's reach ends where the platform's data ends** (Sidekick cannot see Kestrel & Pine's WMS unless an app exposes it). The review step is a vendor default today and a setting someone can switch off; a merchandiser who lets an assistant "clean up" inventory has recreated the App That Wrote Stock. One writer per fact, and an assistant is not it (Human Oversight and Bounded Autonomy).

Agentic Commerce: Fourteen Months, Told in Order

On **16 September 2025** Google announced the **Agent Payments Protocol** (AP2) with more than 60 organizations, built on signed **mandates**: an intent mandate (what the

user asked for, with price limits) and a cart mandate (an unchangeable record of the exact items and price). On **29 September 2025** Stripe and OpenAI launched **Instant Checkout** in ChatGPT: US users could buy from US Etsy sellers inside the chat, under the **Agentic Commerce Protocol** (ACP), with a payment token scoped to one merchant and one cart. On **11 January 2026** Google and Shopify announced the **Universal Commerce Protocol** (UCP), co-developed with Etsy, Wayfair, Target and Walmart, with checkout in Google's AI Mode and Gemini app for US shoppers. On **6 March 2026** OpenAI said it was "evolving how we approach commerce in ChatGPT", and on 12 March that Instant Checkout "is moving to Apps"; ChatGPT became a discovery channel with checkout on the merchant's own storefront. On **3 November 2026** Zalando announced Zalando Extended, with its own agent planned to check out at a pilot partner in 2027.

In fourteen months the flagship in-chat checkout launched and retreated, three protocols appeared, Shopify replaced its own agent interface (Storefront MCP, May 2025) with UCP endpoints, and no party published a transaction count. **This is the most volatile part of the book, and the reason the chapter is dated to the day.**

Two shapes exist as of November 2026, and the difference decides what a merchant needs.

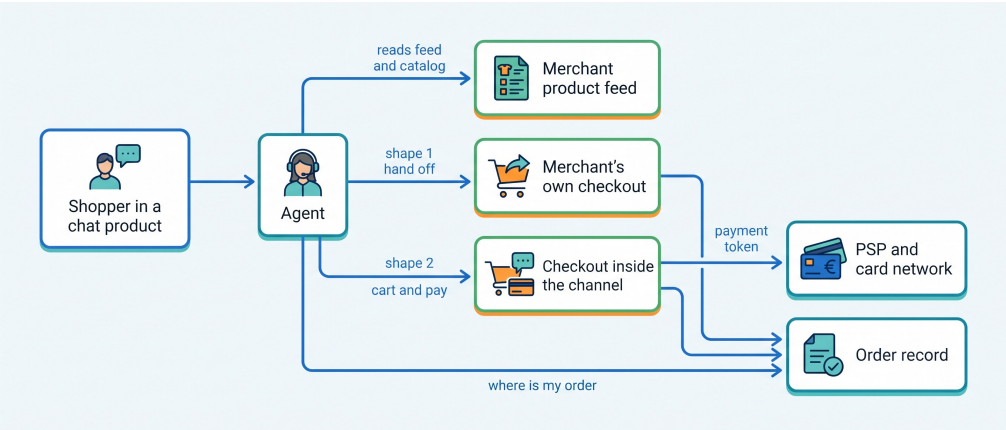


Figure 2: *Discovery with hand-off needs only a correct feed and a stable product URL; checkout inside the channel needs cart, checkout, order and payment-token APIs and an identity for the agent.*

Discovery with hand-off. The agent finds and shows the product; the shopper pays on the merchant's checkout, in an in-app browser or a new tab. The merchant needs a correct, fresh **product feed** (a machine-readable export of the catalog: identifier, title, description, URL, image, brand, price, availability, shipping, returns) and nothing else new. This is where a European merchant can be in 2026.

Checkout inside the channel. The agent builds the cart and pays, with the merchant remaining the **seller of record** (the party that legally sells, collects VAT and owes the

withdrawal right). Google's AI Mode and Gemini, Copilot and Meta's Muse work this way, for shoppers in the United States, with Canada and Australia announced. The merchant needs cart, checkout, order and payment APIs, plus a way to know which agent is calling.

The Protocols in Plain Words

A **commerce protocol** is a published contract (feed, cart, checkout, order, payment token) that lets any agent talk to any merchant without a bespoke integration, as EDI lets any retailer send any supplier an order (Documents Between Companies: EDI, Punchout and E-Invoicing). Four names recur, in layers.

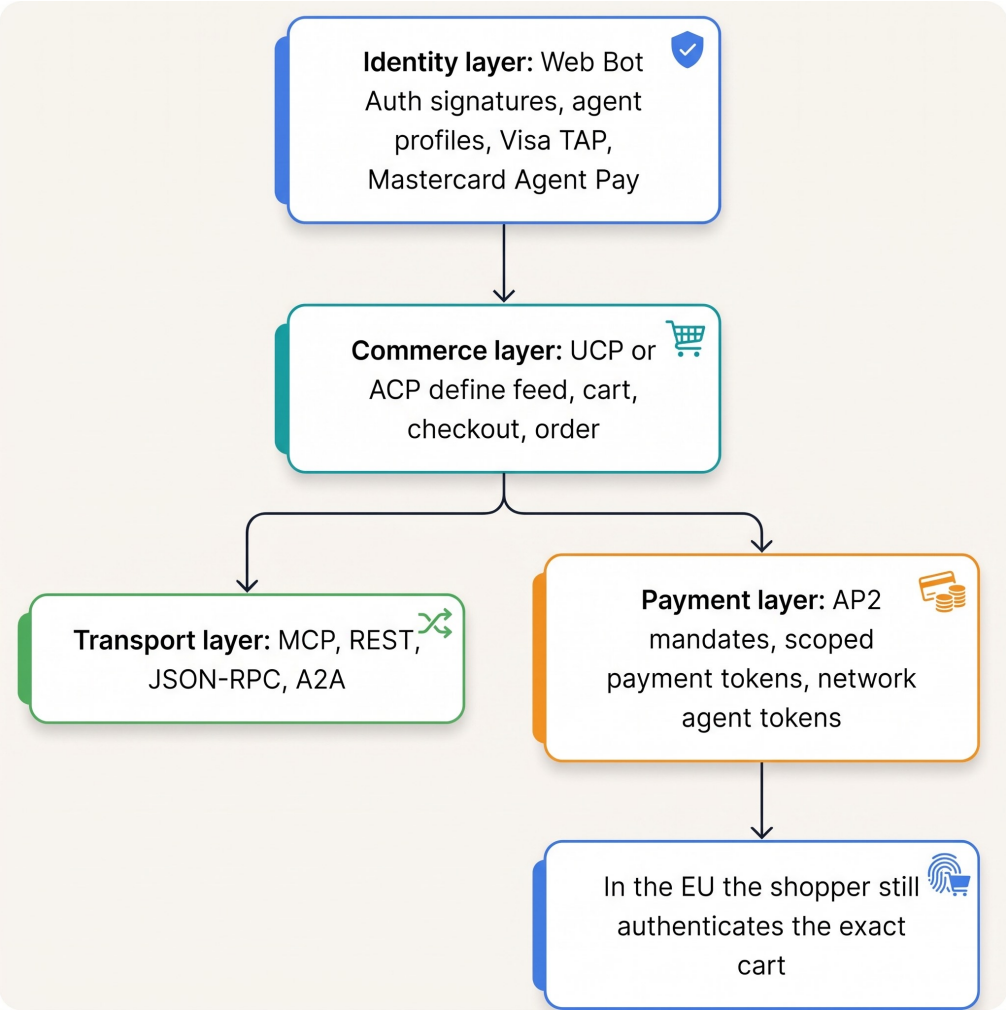


Figure 3: *MCP carries the calls, UCP or ACP say what the calls mean, AP2 and the card networks carry the money, and a signature says who is calling; a European checkout adds the shopper's own authentication at the end.*

MCP (Model Context Protocol) is the transport: how an agent discovers and calls tools such as "search catalog" or "create cart". It says nothing about what a cart is (Tool Ecosystems, MCP and the Tool Broker).

ACP is OpenAI and Stripe's specification: feed, checkout, delegated payment, and since April 2026 cart, orders and authentication. Its repository calls it "in beta", governed by the two founders; its feed upload "currently targets the US".

UCP is the Google-led consortium standard: catalog, cart, checkout, identity linking and order, over REST or JSON-RPC with MCP "built in" and AP2 for payment. Its governance council is Google, Shopify and Stripe; its repository shows no version tags. Google said on 16 September 2026 that UCP "enables direct checkout for hundreds of thousands of brands and retailers", which counts enabled merchants, not transacting ones.

AP2 is the payment layer under UCP: signed mandates that give a "non-repudiable audit trail" of what the shopper authorized. Its repository holds samples and demos.

Agent identity is the layer the others assume. Cloudflare proposed **Web Bot Auth** in May 2025: an agent signs each HTTP request and publishes its keys at a well-known address, because "user agent headers alone are easily spoofed". It is now an IETF working-group draft (29 October 2026) and underpins Visa's Trusted Agent Protocol and Mastercard's Agent Pay; Shopify's agent profiles work the same way, where "higher trust tiers unlock broader access, including direct checkout completion". Amazon, which sued Perplexity in November 2025 over its shopping agent, insists third-party agents "operate openly" (Case: Amazon, From Monolith to Services to the Everything Store; the identity model is in Agent Identity, Delegation and Least Privilege).

Status as of 7 November 2026

Every row is volatile; re-check the primary source before relying on it.

Protocol or program (owner)	Launched	Status on 7 November 2026	Buyer markets
Instant Checkout (OpenAI, Stripe)	29 Sep 2025	Pulled back March 2026; "moving to Apps"; discovery with hand-off	US

Protocol or program (owner)	Launched	Status on 7 November 2026	Buyer markets
ACP (OpenAI and Stripe as "Founding Maintainers"; Apache 2.0)	29 Sep 2025	Beta; latest spec 17 Apr 2026	Feed "currently targets the US"
AP2 (Google with 60+ organizations)	16 Sep 2025	Specification and samples; used inside UCP	Payment layer
UCP (council: Google, Shopify, Stripe, two seats open)	11 Jan 2026	Shopping spec "latest", no version tags; "hundreds of thousands" of merchants enabled	US; Canada and Australia "early next year"; UK later; EU not mentioned
Shopify Agentic Storefronts	10 Dec 2025	Live; ChatGPT discovery only; direct checkout in Google, Copilot, Meta; Storefront MCP replaced by UCP endpoints; checkout needs signed requests	ChatGPT: must sell to US; Google: US, Canada, Australia
Visa Intelligent Commerce and TAP; Mastercard Agent Pay	Apr and Oct 2025	EU pilots with the shopper authenticating each purchase (Visa July 2026; Mastercard June 2026)	Pilot
Worldline, ING, Mastercard	2 Jun 2026	One production pilot transaction (Netherlands)	Pilot
Zalando Extended	3 Nov 2026	Partner products in the app (Westwing pilot); agent checkout planned 2027	Zalando markets

Protocol or program (owner)	Launched	Status on 7 November 2026	Buyer markets
Web Bot Auth (IETF working group)	May 2025; draft -01 29 Oct 2026	Internet-Draft; deployed by Cloudflare, Visa, Mastercard	n/a

Read the table for its shape: **no protocol is governed by a standards body, no checkout program has published volumes, and the only live European transactions are pilots with the shopper present.**

Europe: The Shopper Still Authenticates the Cart

Strong customer authentication. Under PSD2's technical standards (Delegated Regulation 2018/389, in force since September 2019), a remote electronic payment needs **SCA**: two of knowledge, possession and inherence (something you know, have, are), **dynamically linked** to one amount and one payee: "any change to the amount or the payee results in the invalidation of the authentication code". None of the exemptions was written for a delegated purchase whose amount is unknown when the shopper delegates. That is why the European pilots authenticate at the moment of purchase: Worldline and ING's June 2026 transaction completed "only after the consumer explicitly approved it", and Visa's July 2026 pilot authenticated every purchase with a **passkey** (a device-bound credential, usually unlocked by a fingerprint or face, that counts as possession plus inherence). A March 2026 law-firm note (Osborne Clarke) is plain: agent payments "remain subject to PSD2 and the RTS", with "no special 'extra-regulatory' regime"; the PSD3 texts read for this book do not address agent-initiated payments.

The consequence for architecture is small and strict. **The cart the agent builds must be identical to the cart the shopper authenticates**: price, delivery promise and tax final before the mandate is signed, the no-drift rule of From Cart to Confirmation: Checkout Orchestration. Unattended delegated purchases in the EU are unproven in public evidence as of November 2026.

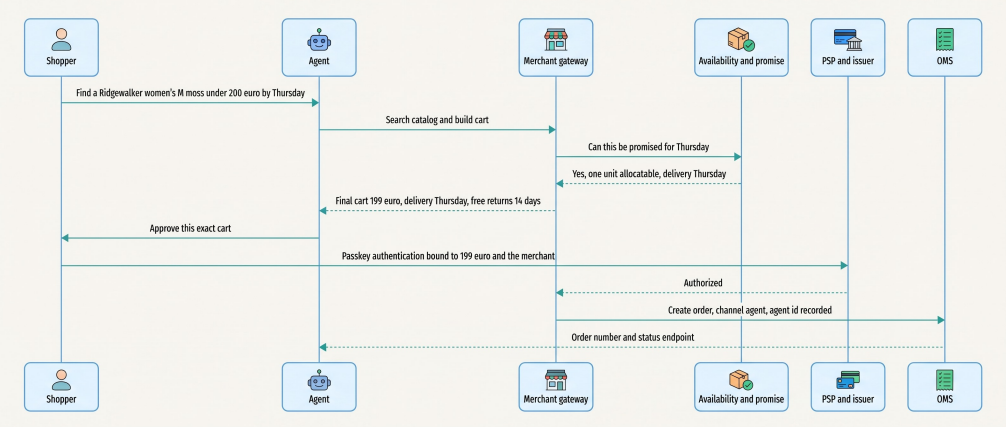


Figure 4: *A European agent purchase as the 2026 pilots shaped it: the agent builds the cart, the merchant finalizes price and promise, the shopper authenticates that exact cart, and the order carries the agent's identity for service and disputes.*

The AI Act. Regulation 2024/1689 applies generally from 2 August 2026. Its Article 50 is the live obligation for commerce: an AI system that interacts with people must tell them so "unless this is obvious", and generated content must be marked as artificial in a machine-readable way. Search and recommendations are not high-risk uses (Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation).

Consumer law. A CERRE issue paper of July 2026 argues that EU consumer law "implicitly assumes" a human makes the purchase decision; every protocol treats the agent as the consumer's instrument, a working assumption, not settled law.

For a Dutch, Belgian or German merchant in November 2026, no live consumer channel lets an agent complete a checkout: ChatGPT has no in-chat checkout anywhere; Google's covers the US, Canada and Australia; Copilot, Meta, Perplexity and Amazon are US-centered; the card-network pilots authenticate the shopper each time; Zalando's agent checkout is planned for 2027. The realistic 2026–2027 work is feed quality, discovery presence, and payment-side readiness through the PSP and the card networks.

What a Merchant Must Have Before an Agent Can Buy

Agent readiness means the five facts an agent asks for are correct, owned by one system and served by API, work most of this book has already described.

Fact the agent needs	**Owner after Waypoint**	**Freshness**	**Consistency**	**Latency *(assumptions)***	**Availability**
Product: title, attributes, GTIN (the barcode number), images, policies	The PIM (W3), published to platform, connector and feed	Hours; feed rebuilt at every publication	One version everywhere; a review state for generated text	Lookup under about 300 ms	Feed served from a cache, never from the PIM
Availability per channel	The availability service (W1), from WMS stock events	Minutes (Shopify lets agents cache price and availability for at most 15 minutes)	Available-to-promise with a buffer, not a copy of on-hand stock	Under about 200 ms	Degrade to "check at checkout", never to a stale yes
Delivery promise	The promise service (planned 2028)	Per request	The same answer to storefront, marketplaces and agent	Under about 300 ms	Fall back to a conservative lane table

Fact the agent needs	Owner after Waypoint	Freshness	Consistency	Latency *(assumptions)*	Availability
Price and policy: price, VAT, shipping, returns	The platform's price list (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax); policies as structured fields	Minutes; ACP asks for a feed update "when a sale starts or ends"	Checkout price equals feed price	Feed	Feed
Order status, returns, refunds	The OMS (W2), with the returns portal	Seconds after a shipment event	One state model for every channel	Under about 500 ms	Read model served from the OMS, not the WMS

Three observations carry the design. **The feed is most of the work, and you may already have it**. The schema.org `Product` vocabulary, Google Merchant Center's specification, OpenAI's ACP feed (nine required fields) and Shopify's catalog ask for the same fields: identifier, GTIN, title, description, URL, image, brand, price, availability, shipping, returns. The new requirement is **freshness for price and availability in minutes, not nightly**. **Availability and the promise are the hard part, and they are not AI work**. An agent cannot negotiate; it reads a figure and either shows the product or does not, so an agent channel is the harshest consumer of the available-to-promise of Is It in Stock? Inventory, Availability and the Promise You Can Keep and the computed promise of Getting It There: Carriers, Labels, Tracking and the Delivery Promise. **Order status is the same API three times**, for the service bot, the order page and the external agent; build it once, on the OMS.

To the list add **an identity and a policy for every agent**: verified agents distinguished from scrapers at the CDN (content delivery network, the edge layer in front of the storefront) or platform level, an allowed set of actions per agent (read the catalog; build a cart; complete checkout only at a trust tier the merchant chose), and no agent writing

stock, prices or orders directly. And add **an "agentic" marker on the order** so service, fraud review and consumer-law disclosure can treat the channel on its own terms.

The Choices and Their Trade-offs

Four realistic postures exist, and they are cumulative: a correct feed; the platform's agent channel, where the vendor carries the protocol churn but sells the platform's view of availability; an agent gateway with adapters over the merchant's own services, so a protocol change is a mapping change; and your own shopping agent, which competes with the largest technology companies for attention with none of their reach. If OpenAI found owning the checkout hard enough to retreat, a fourteen-person team should not volunteer.

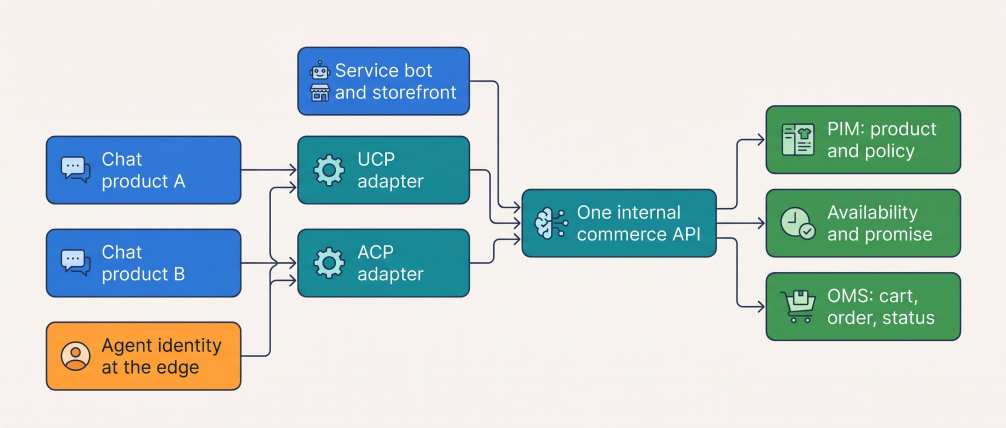


Figure 5: *The low-regret design: adapters per channel over one internal commerce API, so a protocol that changes or disappears costs a mapping, not a rebuild, and every channel reads the same availability and promise.*

Posture	What you build	What you depend on	When it fits	When it limits
Correct feed only	One feed from the product owner, refreshed in minutes	Feed consumers' rules	Every stage; the floor	No order inside a channel

Posture	What you build	What you depend on	When it fits	When it limits
Platform's agent channel	Nothing; switch on, review terms	The vendor's channels, countries, exclusions and availability copy	Starting and Growing; Omnichannel while the platform's copy is good enough	Sells stale availability if the platform is not the stock owner; no EU checkout today
Agent gateway over own services	Adapters per protocol over one internal API; identity at the edge	Your OMS, PIM, availability and promise being right	Omnichannel with Waypoint-grade services; Group	You carry protocol churn; worthless over copies
Own shopping agent	An LLM product with tools and a checkout	Model vendors, every fact above, and shoppers choosing you	Rarely; perhaps a Group with a destination app	The Giant's Boots

I would hold a merchant at the first two postures until a protocol has stable governance, a European checkout program with published volumes exists, and the merchant's own availability and order truth are better than the platform's copy; revisit in mid-2027. **What not to build yet**: a consumer-facing shopping agent; three protocols natively in the storefront; agent authentication from scratch; any agent or assistant that writes stock, prices or orders directly; any promise through an agent that the storefront cannot keep.

Across Scale Stages: When It Fits and When It Limits

Stage	Typical AI use	What becomes necessary	Warning signs
Starting (under about €2M)	The platform's built-in search and assistant; one feed to Google Merchant Center; the PSP's fraud scoring	A correct feed with GTINs and fresh price and stock; multi-factor authentication on the admin; the AI channel for discovery only	Product text edited in three places; the assistant given write permissions "to save time"
Growing (€2M–€20M)	A hosted search service; reviewed generated descriptions; the platform's agent channel; a bot on the platform's inbox	Feeds from one product owner, refreshed in minutes; structured return and shipping policies	The feed publishes the ERP's hourly stock copy; a repricing app with no floor; the bot cannot read shipment events
Omnichannel (€20M–€200M; Kestrel & Pine)	AI search shared by storefront and store staff; a bot in front of the OMS and returns portal; forecasting fed by stock events; admin assistants with review	Availability and the computed promise by API; order status by API on the OMS; an agent policy (identity, allowed actions, no writes); an "agentic" order attribute; AI Act disclosure on every bot	Agent and storefront show different availability; agent orders canceled for stock the warehouse never had; "auto-apply" switched on by a team
Group (over about €200M or several brands)	A shared discovery platform with per-brand indexes; one order-status contract; an agent gateway per brand on one implementation	Per-brand feeds, identities and markets; one payment and disclosure policy per market; a key inventory for every agent credential	Brands implementing protocols separately; agent orders unattributable to a brand; one credential shared across brands

The readiness list does not change with scale; who assembles it does: the platform at the Starting stage, the merchant across PIM, OMS, WMS and PSP at the Omnichannel stage, a shared service at the Group stage.

In Practice: Waypoint's W8 at Kestrel & Pine

W8 is written as prerequisites with owners and dates, and a sentence about what will not be built. The figures are Kestrel & Pine illustrations.

Prerequisite	Owner	Delivered by	How an agent sees it
Product data from one owner, including the attributes shoppers search for (the Ridgewalker's 20,000 mm waterproof rating lived only in a spreadsheet)	Teun Baas, through the PIM (W3)	Q3 2027: the DE storefront in August, NL/BE and the connector in September	One feed per market from the PIM, with structured shipping and return policies
Availability per channel from WMS stock events, with buffers	Daniel Okafor and Ruben Alves, through the availability service (W1)	Q4 2027	An availability endpoint; feed availability refreshed every 10 minutes *(assumption)*
A computed delivery promise	Noor Visser; the May 2027 promise service (transit table plus WMS cut-off and backlog), the promise API in 2028	2028	The same date the storefront shows

Prerequisite	Owner	Delivered by	How an agent sees it
Order status, returns and refunds by API	The Fulfil and Return team, on the OMS (W2)	Q4 2027, after D2C orders route through the OMS on 13 October	One customer-scoped endpoint, used first by the service tool and the order page
An agent policy	Milan Petrov (business), Ruben Alves (technical)	April 2027	Verified agents may read the catalog; checkout only on the storefront in 2027; no agent or assistant holds a write scope on stock, prices or orders; every agent credential on the ownership map
AI disclosure	The Buy team	April 2027 (Article 50 has applied since 2 August 2026)	The service chat says it is automated; generated images are marked

The policy line cost nothing and mattered most: W8 extends the ownership map of Who Builds and Who Owns: Teams, Vendors, Agencies and Governance to agents by a single rule: **an agent is an integration**, with a row, an owner, a contract, a pinned API version and a reconciliation. The agency's admin-assistant trial in June 2027 ran under that rule, with auto-apply disabled and every change reviewed; it drafted 1,400 product descriptions for the autumn season, about 60% accepted unchanged, and could not touch stock *(illustrative)*. What W8 does **not** do is also recorded: no storefront shopping agent, no in-channel checkout and no custom protocol work in 2027; the gateway posture is the 2028 plan if a European checkout program exists by then. Mara's answer to the board observer: "We will be the merchant whose answers are right when the agents arrive."

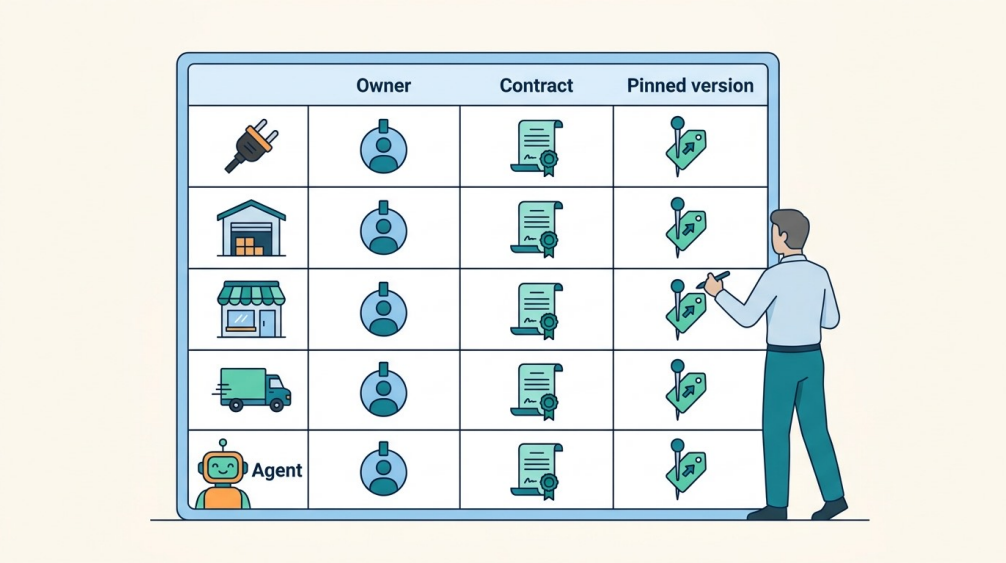


Figure 6: *Waypoint's W8 rule: an agent is an integration, with a row on the ownership map, an owner, a contract, a pinned API version and a reconciliation.*

When It Goes Wrong

The assistant with a write scope. A merchandiser enables auto-apply on an admin assistant to "fix negative stock" before a campaign; the assistant sets quantities in the platform while the WMS feed also sets them. This is the App That Wrote Stock, and the detection is the same: the weekly WMS-to-platform reconciliation (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation) and a rule that no assistant credential holds an inventory write scope. Recovery: revoke, overwrite from the system of record, add the credential to the ownership map.

The stale feed. The feed is rebuilt nightly from the platform's stock copy; a product sells out at 19:40 and every agent shows it in stock until morning. Detection: a feed-freshness monitor (the age of the last price and stock update per market) and rising agent cancellations. Recovery: availability in the feed from the availability service, refreshed in minutes, and "check at checkout" when the service is down.

Duplicate orders from agent retries. An agent's checkout call times out, the agent retries, two orders exist. The remedy is an **idempotency key** (a unique identifier sent with the request, so a repeat has the same effect as the first) on every cart and checkout call, and a daily match of agent orders to PSP authorizations.

Cart drift before authentication. The agent shows €199 with Thursday delivery; by the time the shopper authenticates, the promotion has ended, and under dynamic linking the authentication is void. Remedy: hold price, promise and tax for the authentication

window, re-quote rather than silently charge, and store the mandate reference and agent identity with the order.

The protocol pulled back. A channel you built against disappears or changes its terms within a year, as Instant Checkout did in six months and Shopify's Storefront MCP within twelve: the Pulled Rug of Promises Between Systems: API Contracts, Webhooks and Versioning at protocol scale. Remedy: adapters over one internal API and a contract card per channel.

What to Take Away

Established.

- AI in commerce has two kinds: assistants that read and propose (Amazon 2003 and 2019, Zalando 2023 and 2026, Klarna 2024 and 2025) and agents that act. The second adds a writer, and every writer needs an owner.
- As of 7 November 2026, agentic commerce is three protocols without standards-body governance, one flagship checkout pulled back within six months, and no published transaction volumes; every live consumer checkout is limited to the United States, Canada or Australia.
- In the EU, strong customer authentication and dynamic linking apply to agent payments without a special regime (Osborne Clarke, March 2026); the 2026 pilots all authenticate the shopper on the exact cart; the AI Act's Article 50 requires bots to say they are AI from 2 August 2026.
- Every feed specification asks for the same dozen fields; agent identity is converging on signed HTTP requests with a published key directory (Web Bot Auth, an IETF draft).

Depends on context.

- Switching on the platform's agent channel depends on whether the platform's availability copy is the stock owner's truth: for a merchant whose platform holds stock, yes, now; for one whose stock lives in a WMS and syncs hourly, for discovery only until the availability service exists.
- Building an agent gateway depends on owning better availability, promise and order truth than the platform, and on an EU checkout program existing; before both it is machinery with no traffic.
- How much an admin assistant may apply without review depends on the fact: generated descriptions with a review state, freely; discounts and segments, with approval; stock, prices and refunds, never.

The author's synthesis.

- Agent readiness is the same program as fixing the Phantom Stock and Schrödinger's Order: one owner per fact, availability and promise computed rather than copied, order status from one record. A merchant that does that work is ready for whichever protocol survives; one that chases protocols without it broadcasts its drift.
- Treat every agent as an integration: a row on the ownership map, an identity, a contract, a pinned version, a reconciliation; the Secret Agent is a governance failure first.
- For a European merchant in 2027 the honest plan is feed, discovery, payment-side readiness through the PSP, and a review in mid-2027; anything more is the Giant's Boots, worn early.

Questions to Ask

1. Which system owns each of the five facts an agent asks for (product, availability, promise, price and policy, order status), and can each be read by API from that system today?
2. How old is the stock figure in our product feed when an agent reads it, and what does the agent see when the availability service is down?
3. Which agent channels has our platform switched on by default, under what terms, for which countries, and which of our products are excluded?
4. Does any assistant, app or agent hold a write scope on stock, prices or orders, and is each agent credential on the ownership map with an owner, a contract and a deprecation date?
5. If an agent's checkout call is retried, do we create one order or two?
6. Is the cart the shopper authenticates identical to the cart the agent built, and how long do we hold price and availability for the authentication window?
7. Do our bots say they are AI, and are generated product images and text marked?
8. Which vendor has published a transaction volume for its agent channel, and when will we re-check the protocol table?

Part X ends here, with the newest kind of actor asking the oldest question in the book: who may make which promise, and which system has to keep it. Part XI tests the argument against the public record of real companies, starting with the platform that serves millions of merchants on one codebase. Continue with Part XI — PRACTICE: How Others Built It.

Sources and Evidence

Company, product and investor sources

- Stripe and OpenAI launch Instant Checkout, Stripe newsroom, 2025 — the launch, US Etsy sellers, the scoped payment token.
- Agentic Commerce Protocol repository, OpenAI and Stripe, 2025–2026 — beta status, governance, release dates; Product Feed Spec, OpenAI developers, n.d., accessed 2026-11 — required fields, US targeting.
- OpenAI shifts checkout plans, Digital Commerce 360, 2026, and Shopify says purchases are coming inside ChatGPT, Modern Retail, 2026 — the March 2026 retreat and the "integrity of transactions" argument.
- Announcing Agent Payments Protocol (AP2), Google Cloud, 2025 — mandates and partners; AP2 repository, n.d., accessed 2026-11.
- Agentic commerce: new AI tools and a protocol, Google, 2026; Universal Cart at Google Marketing Live, Google, 2026; Google Shopping holiday updates, Google, 2026 — UCP launch, geography, "hundreds of thousands" enabled.
- ucp.dev, UCP Authors, 2026; UCP MAINTAINERS and repository, n.d., accessed 2026-11 — capabilities, transports, councils, absence of version tags.
- The agentic commerce platform, Shopify, 2026; Winter '26 Edition, Shopify, 2025; Fall '26 Edition, Shopify, 2026 — Agentic Storefronts, Sidekick app extensions, the 4.2x figure.
- Agentic storefronts, ChatGPT channel and Google channel, Shopify Help Center, n.d., accessed 2026-11 — buyer geography and product exclusions.
- Build for agents, Catalog and Storefront MCP, shopify.dev, n.d., accessed 2026-11 — agent profiles, trust tiers, cache limits, the MCP-to-UCP replacement.
- Expanding your AI horizons, Summer '25, Shopify, 2025; Sidekick, Shopify Help Center, n.d., accessed 2026-11; Summer '23 Edition, Shopify, 2023 — Sidekick's capabilities and review step.
- Shopware Community Day 2026, Shopware, 2026, and MCP Server, Shopware developer docs, n.d., accessed 2026-11 — Copilot's agentic mode and the experimental MCP server.
- Salesforce Agentforce Commerce release, Salesforce, 2026; commercetools launches AgenticLift, commercetools, 2026; Adobe Commerce commits to agentic standards, Glenbrook (relay), 2026; Adyen Agentic, Adyen, 2026; Klarna Agentic Product Protocol, Klarna, 2025; PayPal and Perplexity launch Instant Buy, PayPal, 2025 — vendor programs one layer down.
- Amazon Rufus / Alexa for Shopping, Amazon, n.d., accessed 2026-11 — the 13 May 2026 rename and Auto-Buy; Buy for Me, Retail Dive, 2025; Amazon and Perplexity, Engadget, 2025; Amazon blocks Meta's Muse, The Star (Bloomberg), 2026; Perplexity wins the appeal, EcomCrew, 2026.
- Zalando Q3 2026 results, Zalando, 2026 — Assistant users and Zalando Extended; Zalando fashion assistant, Zalando, 2023.
- Klarna AI assistant first month, Klarna, 2024, and Klarna rehires humans, Entrepreneur (Bloomberg), 2025.
- Amazon generative AI listings, Amazon, n.d., accessed 2026-11; Stripe Radar, Stripe, n.d., accessed 2026-11 — vendor claims, labeled as such.

- Visa: live agentic transactions in Europe, Visa, 2026; Visa, BILL and Billtrust B2B pilot, Visa, 2026; Cloudflare and payments companies on Trusted Agent Protocol, Cloudflare, 2025.
- Mastercard unveils Agent Pay, Mastercard, 2025; Europe is building the foundations for trusted agentic commerce, Mastercard, 2026.
- Worldline and ING complete Europe's first agentic payment in production, Worldline, 2026.
- Generative AI holiday shopping traffic 2025, Digital Commerce 360 (Adobe data), 2026.

Standards, regulation and official documentation

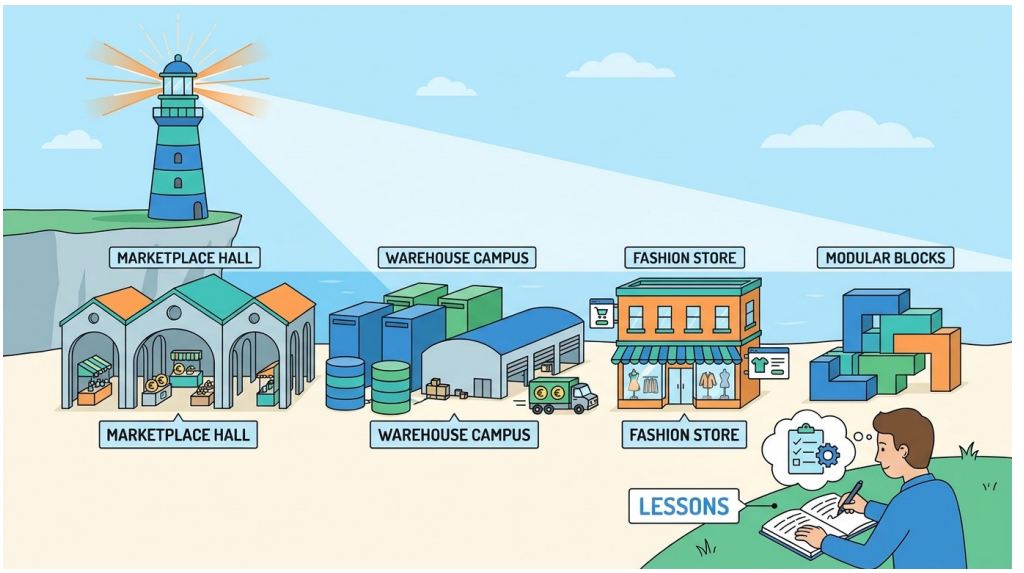
- Commission Delegated Regulation (EU) 2018/389, 2018 — SCA, dynamic linking and the exemptions.
- Regulation (EU) 2024/1689 (AI Act), 2024, and Regulation (EU) 2026/1744 (Digital Omnibus on AI), 2026 — Article 50 and the amended dates.
- PSD3/PSR final compromise texts are published, William Fry, 2026 — status of the files.
- HTTP Message Signatures for automated traffic (draft-ietf-webbotauth-httpsig-protocol-01), IETF Web Bot Auth working group, 2026; Forget IPs: using cryptography to verify bot and agent traffic, Cloudflare, 2025.
- schema.org Product, schema.org, 2026, and Product data specification, Google Merchant Center, n.d., accessed 2026-11 — feed fields.

Research and practitioner sources

- Agentic payments: a new challenge for Europe's payments ecosystem, Osborne Clarke, 2026 — no special regime under PSD2.
- Agentic AI and consumer protection, CERRE (C. Busch), 2026 — the consumer-law questions.
- The history of Amazon's recommendation algorithm, Amazon Science, 2019 — history, more than three years old.
- DeepAR, Salinas, Flunkert and Gasthaus, 2017, and Deep learning based forecasting: a case study from the online fashion industry, Kunz et al., 2023 — forecasting methods, not current systems.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Part XI — PRACTICE: How Others Built It



IN THIS PART, YOU WILL: Read four real companies' dated public records, separate what they document from the folklore told about them, and decide which of their lessons fit a merchant of your size.

KEY POINTS:

- **The documented record is narrower than the folklore, and the narrow claims are the useful ones.** In the companies' own dated words, the slogans shrink to rules a small team can use: the contract is the only door, the shop is the unit of isolation, a copy is not a system of record.

- **Every answer was sized to its company.** Amazon split after a scaling wall, Zalando for 85-plus teams, and the composable vendors designed for global enterprises; in a fourteen-person team the same shapes are copies without their reasons.
- **What transfers is discipline, not machinery.** Dated API versions and deprecations, reconciled integrations, one owner per fact and a kept core with services beside it transfer; pods, a hundred services and a delivery network do not.

FAILURE PATTERNS THIS PART ADDRESSES:

- **The Giant's Boots** — each case is an architecture copied by merchants a hundred times smaller; each chapter names what only the giant needs.
- **The Pulled Rug** — Shopify and Zalando publish how they withdraw APIs, which tells a merchant what to demand of every vendor.
- **The Spaghetti Junction** — Amazon's no-direct-reads rule and the composable movement's own survey both show what unowned joins cost.

Every part so far has used real companies as dated examples inside a chapter about one capability. **This part reads four whole companies against the book's argument.** Shopify is a growing merchant's most likely landlord; Amazon and Zalando are its marketplaces and competitors; the composable vendors sell it the next rebuild. Their stories are retold in design reviews, usually without dates.

So the cases are read by strict rules. **Every claim comes from public evidence and carries its year in the sentence:** filings, papers, engineering posts and documentation, researched on 7 November 2026. Sources older than about three years are flagged because they may describe a replaced system; Amazon's architecture story is from 2006 and 2007. Undocumented claims about a company's internals are labeled **INFERENCE, NOT DOCUMENTED.** And **each case ends where the evidence stops**, in a section listing what could not be established, such as Amazon's current order systems or Shopify's pod count after 2021.

Remember The Giant's Boots while you read. **These companies' answers depend on their scale and their business model:** Shopify is a payments business serving millions of shops, Amazon a retailer that opened its warehouses to sellers, Zalando a platform of 62 million customers. The cases sit beside the order's journey, touching Eva Janssen's jacket where it is bought, listed and priced. Each ends with what Kestrel & Pine, the fictional outdoor brand this book follows, takes from it: always a Waypoint decision already made.

Figure 1: *The Giant's Boots: each case is an architecture copied by merchants a hundred times smaller, so each chapter names what only the giant needs.*

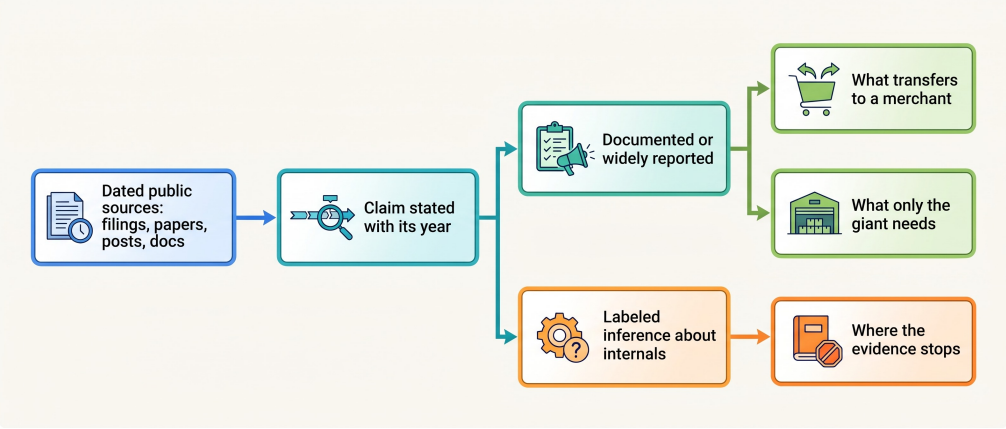


Figure 2: *How each case is read: every claim is dated and typed, documented claims are split into lessons that transfer and machinery that does not, and inferences lead to a list of what the record does not show.*

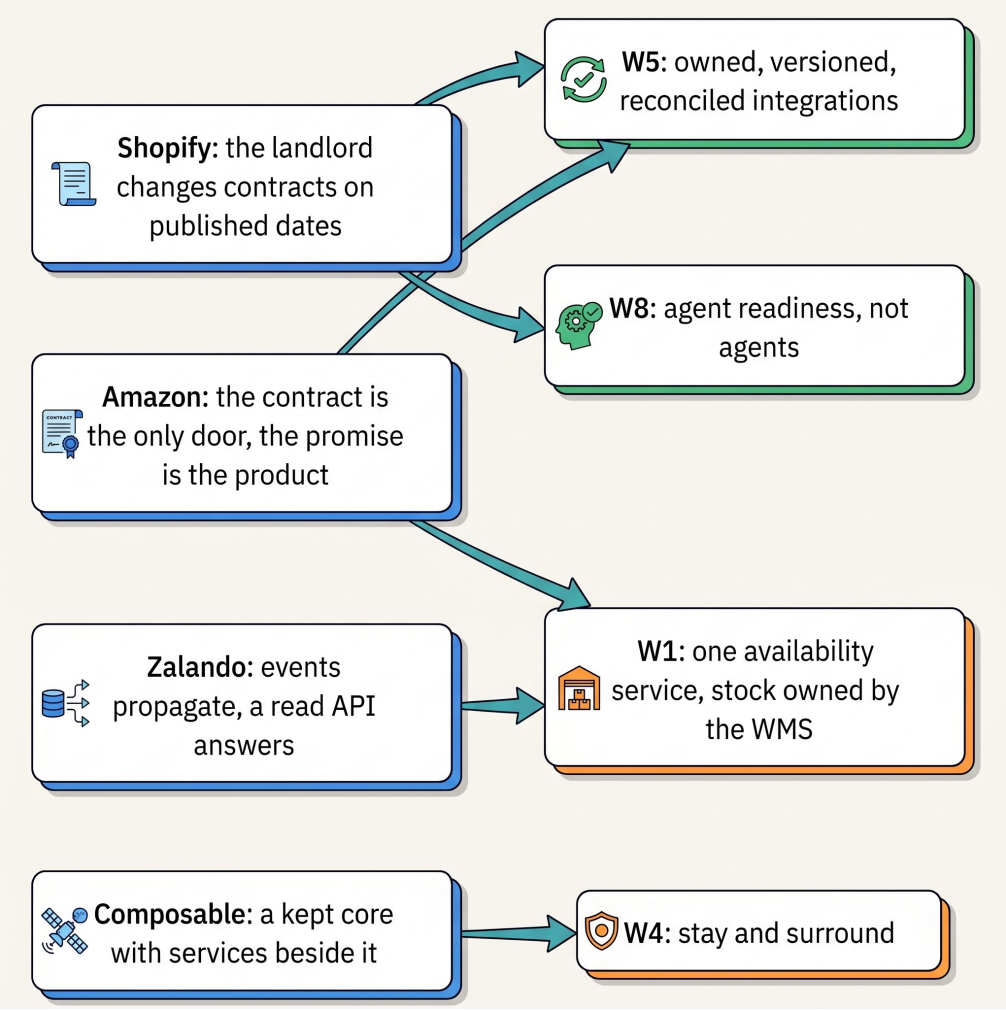


Figure 3: *What Kestrel & Pine takes from the four cases: each dated lesson confirms a Waypoint decision already made, and none changes its architecture.*

Chapters in this part

Case: Shopify, the Modular Monolith That Serves Millions of Merchants asks **which parts of Shopify's public record apply to a merchant that rents one of its millions of shops**: a payments business whose architecture follows the money, a 2.8-million-line monolith that learned which boundary gate to keep, the shop as the unit of isolation, and extension points withdrawn on published dates. Kestrel & Pine takes four disciplines (reconciled integrations, connectors pinned to dated API versions, nothing custom in checkout, agent readiness by API) and rents the rest.

Case: Amazon, From Monolith to Services to the Everything Store asks **what Amazon's**

dated accounts say about its services, its cart and its marketplace, and what a merchant can learn: a split forced by a scaling wall in 2001, a cart built never to refuse an add, and third-party sellers grown from 3% of physical sales in 1999 to over 60% through Prime and Fulfillment by Amazon. Kestrel & Pine takes the contract rule, keeps its stock in Amazon's program as a separate location, and puts an engine behind "ordered before 22:00, delivered tomorrow".

Case: Zalando, Microservices, Platform Strategy and the Return to Discipline asks **what ten years of Zalando's published decisions teach about team-sized modularity, API contracts and becoming a platform**: microservices in 2015 for 85-plus teams, API guidelines that allow a shutdown only with consent and a date, a decade of corrections, and a 2025 read API replacing a price-and-stock stream that ran 30 minutes behind at Cyber Week. Kestrel & Pine writes the deprecation rules into its vendor standards and answers "what is available now" through an API, not a faster chain of copies.

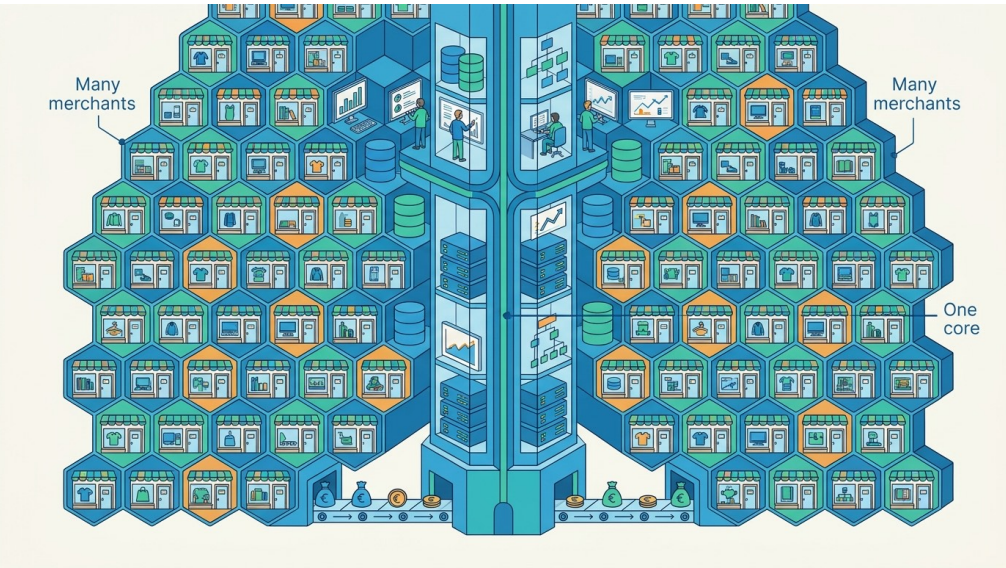
Case: commercetools, Shopware and the Composable Movement asks **when assembling commerce from parts is right, now that the composable vendors have changed what they sell**: commercetools' 2026 modules sold "without replacing their existing platforms", Shopware pulling integration and payments inside the platform, and an alliance whose own 2026 survey found 37% naming integration complexity. The Kestrel & Pine case is the composable rebuild an outside consultancy pitched in February 2027, answered by Waypoint with the movement's own 2026 practice: a kept core with services beside it.

Begin with Case: Shopify, the Modular Monolith That Serves Millions of Merchants.

Sources and Evidence

Every public claim in this part is sourced and dated in the chapter that makes it, under that chapter's Sources and Evidence. Kestrel & Pine is fictional; recommendations, labeled inferences and labeled assumptions are the author's judgement.

46. Case: Shopify, the Modular Monolith That Serves Millions of Merchants



IN THIS CHAPTER, YOU WILL: Read Shopify's own dated accounts of its business and architecture, see what each supports and what it does not, and decide which lessons transfer to a merchant who rents a platform rather than builds one.

KEY POINTS:

- **Shopify is economically a payments business with a subscription as the entry ticket, and its architecture follows the money.** In FY2025 three quarters of its $11.6 billion revenue moved with merchandise volume and two thirds of that volume ran through Shopify Payments, which Shopify resells from licensed processors. Checkout is the most protected path because it is the revenue path.
- **The monolith stayed, and the boundary lessons are the honest part.** Shopify's 2019 to 2024 posts record one codebase of 2.8 million lines

kept as one deployment unit, a tool that gates new dependencies, and the admission that domain boundaries drawn on paper "do not reflect the way Shopify's code actually functions". The shop, not the service, is the unit of isolation, failure and recovery.
- **The platform changes under everyone at once, on published dates.** Free-form checkout customization was withdrawn over three and a half years and replaced by sandboxed extension points; API versions last at least twelve months; the agent interface was replaced within a year. A merchant on any hosted platform inherits that cadence and must budget for it.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Giant's Boots** — pods, shard moves and a custom observability stack are scale-specific machinery that a merchant mistakes for architecture to copy.
- **The Pulled Rug** — checkout.liquid, Shopify Scripts, the REST Admin API and the Storefront MCP endpoints were all withdrawn on published dates; the merchant who did not read the changelog found out on the day.
- **The Secret Agent** — the scope model, the writer problem among 21,000 apps and the trust tiers for AI agents are Shopify's answer to automation that changes facts without an owner.

Shopify is two things to a merchant at once. It is **the most likely landlord**: the fictional Kestrel & Pine's "platform" is a hosted, multi-tenant commerce service of this kind, and so is the platform of most brands of its size. And it is **the most quoted example** in commerce architecture after Amazon: "Shopify kept a monolith" is repeated in design reviews as often as "Amazon moved to services". This chapter goes back to the **dated public record**: Shopify's engineering blog, its help and developer documentation, its product announcements, and its filings with the US Securities and Exchange Commission (the annual **Form 10-K** and the quarterly **10-Q**).

The record has a shape worth stating before the detail. The business figures are current (the 10-K for 2025, filed February 2026; the 10-Q for the quarter ended 30 September 2026, filed 3 November 2026). The architecture posts are old: pods in 2018, componentization in 2019, the monolith's state in 2020, shard balancing in 2021. **Every load-bearing architecture post is more than three years old at the research cut-off**, and the most recent first-party retrospective is from February 2024. Nothing after 2021 describes the pod topology or the component count. Where this chapter describes how Shopify works today, it says so, and where it only knows how Shopify worked in 2020, it says that too.

Where we are in the order's journey. Every step of Eva's jacket that happens inside "the platform" has a documented Shopify counterpart: the product record and

its stock states (Is It in Stock? Inventory, Availability and the Promise You Can Keep), the storefront that renders the product page (Delivering the Storefront: Themes, Headless, Rendering and Performance), the checkout and the payment (From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds), the webhooks that tell the WMS (warehouse management system) an order exists (Promises Between Systems: API Contracts, Webhooks and Versioning), and the apps that write to all of them (Apps, Plugins and Functions: Extensibility and Ecosystem Governance). This case gathers those threads, which the owning chapters use as dated examples, into one story.

How to Read This Case

Every statement about Shopify below is **documented** (the source and its year are in the sentence), **widely reported** (a secondary source, and the text says so) or an **inference**, labeled `INFERENCE, NOT DOCUMENTED` with its reasoning. Sources older than about three years are flagged as **evidence age**, because they may describe a replaced system.

The research was done on 7 November 2026. Several things could not be verified then and are listed under "Where the evidence stops": there is no official merchant count beyond "millions", no first-party description of the checkout queue, no published method behind the Shop Pay conversion claims, no general-availability date for Sidekick, and no first-party confirmation of a vendor's account of a new database. Shopify is a dated example here, never the answer; the book is platform-neutral.

The Business: Where the Money Comes From

Start with the definitions, because Shopify's headline numbers mean less than they seem until you know them. **GMV** (gross merchandise volume) is, in the FY2025 10-K's words, "the total dollar value of orders facilitated through our platform … net of refunds, and inclusive of shipping and handling, duty and value-added taxes. GMV does not represent revenue earned by us." A **merchant** "means the total number of unique shops that are paying for a subscription", so one brand with three regional stores counts three times. **MRR** (monthly recurring revenue) is the subscription run-rate on the last day of a period.

Metric	FY2025	FY2024	Q3 2026
GMV	$378.4 billion (+29%)	$292.3 billion	$119.1 billion (+29%)
Revenue	$11.56 billion (+30%)	$8.88 billion	$3.74 billion (+32%)
Subscription solutions	$2.8 billion (24%)	$2.4 billion	$836 million
Merchant solutions	$8.8 billion (76%)	$6.5 billion	$2.9 billion
Shopify Payments penetration (share of GMV)	65.6%	61.9%	68%
MRR	$205 million	$178 million	$230 million
Merchants	"millions", 175+ countries; 44% US, 31% EMEA	—	—
Apps in the App Store	"more than 21,000" (31 December 2025)	—	—
Employees	about 7,600	—	—

Sources: the FY2025 10-K (filed 11 February 2026) and the Q3 2026 8-K and 10-Q (3 November 2026).

Two revenue lines carry the story. **Subscription solutions** are plans, platform fees, point-of-sale subscriptions, apps, themes and domains. **Merchant solutions** are Shopify Payments processing fees, currency conversion, lending, shipping labels, hardware and advertising, which the 10-K says are "directionally correlated with the level of GMV". Three quarters of revenue therefore moves with what merchants sell, and two thirds of what they sell is paid through Shopify Payments. The 10-K adds the plan mix ("most merchants subscribe to our Basic and Grow plans", while "the majority of our GMV has been generated from merchants subscribing to our Shopify Plus plan and enterprise offerings") and a concentration: "we currently rely on two suppliers to provide the technology we offer through Shopify Payments", Stripe and PayPal, on agreements that renew every twelve months.

> **INFERENCE, NOT DOCUMENTED:** Read together, these facts make Shopify economically a payments-and-platform business whose subscription is the entry ticket. Its architecture priorities follow: the checkout and the payment flow are the most protected paths because they are the revenue paths, and every product that raises Payments penetration (Shop Pay, point of sale, B2B, agent channels paying with Shop Pay) has a direct revenue line. The plan mix means one "single branch" codebase must serve millions

> of small shops that need safe defaults and a few thousand large ones that need extension points, which is why advanced checkout customization, custom Functions and the richer B2B features are gated to the Plus plan.

The 10-K also describes the technology in one paragraph: "a multi-tenant cloud-based system", with Ruby on Rails as "the technology that powers much of the Shopify platform", run as "'single branch' software, which means that all of our merchants use the latest version of Shopify", on "third-party cloud computing services, such as Google Cloud Platform". **Multi-tenant** means many merchants share one running system and one database fleet, isolated by data rather than by separate installations (Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation). **Single branch** means there is no version a merchant can stay on; every change ships to everyone.

The Monolith That Stayed: 2019 to 2024

A **monolith** is one deployable application that holds all of the shop's logic; a **modular monolith** keeps that single deployment but divides the code into named modules with enforced boundaries. Shopify's account of choosing the second over splitting into services runs from 2019 to 2024, and it is more candid than the slogan.

In February 2019 Shopify described its codebase as "one of the largest Ruby on Rails codebases in existence", worked on for over a decade by more than a thousand developers, and said that around 2016 "the downsides of the monolith were outweighing the benefits". It rejected microservices explicitly, listing the costs: pipelines and infrastructure per service, "not always having the data you need when you need it", network latency and reduced reliability, and coordinated deploys for large refactors. Instead, a small team labeled about 6,000 classes by business domain (orders, shipping, inventory, billing) and moved them in one automated pull request, intending each component to have "a clean, dedicated interface" and "exclusive ownership of its associated data". The payoff claimed was concrete: the isolation work let Shopify replace a legacy tax engine. **(Evidence age: seven years.)**

By September 2020 the core had "over 2.8 million lines of Ruby code", 37 components and an Architecture Guild of about 400 members; "every file has a stewardship team assigned". The same post is honest about what had not worked: "every component depended on over half of all the other components", with "lots of circular dependencies". What did pay off was ownership: exceptions were triaged automatically to the owning component, and framework upgrades could be distributed across owning teams. **(Evidence age: six years.)**

Packwerk, open-sourced the same month, is the tool that survived: it checks references between packages and records existing violations in a list so that teams "stop new ones without fixing everything at once". The February 2024 retrospective, the latest first-party architecture account, is the most useful document in the record. Privacy

checks "were removed from Packwerk with the release of version 3.0" because they pushed teams into premature API design. Several common ways of loading code are "untracked and invisible to the tool". Isolating one base package with zero dependencies "took many months of hard work". The rate at which the tool found problems "vastly outpaced" teams' capacity to fix them; "we have discussed removing Packwerk from our monolith", and it stayed only because it "has provided value in holding the line against new dependencies at the base layer". And the sentence that matters most: "domains and the boundaries between them do not reflect the way Shopify's code actually functions in practice", so components were demoted to "top-level organizational tools for grouping one or more packages".

> **INFERENCE, NOT DOCUMENTED:** The honest reading of 2019 to 2024 is not "Shopify modularized its monolith". It is that a company with a thousand developers spent seven years learning that domain boundaries drawn on an organization chart do not match runtime coupling, got durable value from file ownership and from one cheap gate against new base-layer dependencies, and kept one deployment unit throughout. Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says draws the general lesson; what this case adds is that even the best-resourced practitioner of the modular monolith could not make enforced domain boundaries stick, and said so.

The one documented case of carving a hot path out of the monolith is the **Storefront Renderer**, the server-side application that renders a merchant's theme into HTML. Its rewrite began in January 2019 because storefront code "has much stricter performance requirements" than, say, payment processing. It "always reads from dedicated read replicas" behind three cache layers (a query cache, an in-memory cache of the most frequent queries, a full-page cache); the August 2020 post reported server responses "4x to 6x faster" and a rollout in which a verifier compared legacy and new responses on live traffic, with circuit breakers falling back to the old path. **(Evidence age: six years.)** The split was made for latency, behind a verifier, not for team autonomy.

The Shop as the Unit of Isolation: Pods and Moves

A **shard** is one of several databases that each hold a slice of the data; the **sharding key** is the field that decides which shard a row lives in. Shopify's 2018 post describes the reason and the consequence. By 2015 the database "could no longer be scaled by buying a bigger server", so Shopify sharded by shop. Sharding reduced resilience: shared code running across all shards let one failed shard take an action down platform-wide. In 2016 Shopify reorganized into **pods**: "A pod consists of a set of shops that live on a fully isolated set of datastores." Web servers and job workers are shared, but "all shared resources can only ever communicate to a single pod at a time"; a routing layer assigns

each request to one pod, and a "Pod Mover" could move a pod to its recovery data center "in a minute". **(Evidence age: eight years; the pre-cloud era.)**

The 2021 post on shard balancing shows the mechanism that persisted. "A pod is an isolated instance of Shopify" with one MySQL shard plus its caches; "we can attach a `shop_id` to all shop-owned tables and have it serve as a sharding key". A routing table mapping domains to pods lives in a separate, unsharded database. Moving a shop has three phases: Ghostferry copies rows and replays the change log while the store stays live; a cutover takes a brief writer lock, and fails if the lock cannot be acquired in time; then the routing table is updated. "The only opportunity for downtime is during the cutover." Placement uses measured load, not shop counts: before rebalancing the least- and most-used shards differed by "almost four times", and simulation brought the spread "closer to two times". **(Evidence age: five years.)**

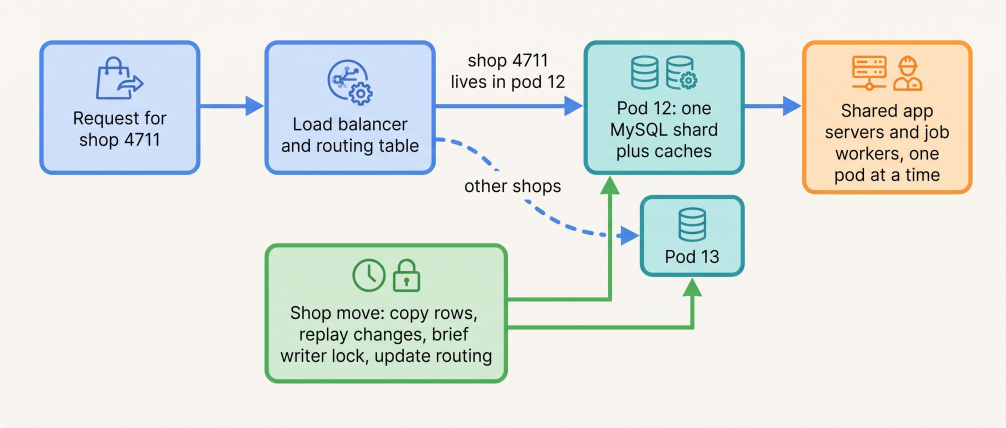


Figure 1: *The shop is the sharding key and the pod is the unit of isolation: a request is routed to one pod, a failing pod is contained, and a shop is moved as a whole with downtime only during a brief cutover (Shopify, 2018 and 2021).*

Two later datastore facts are often misquoted. In January 2024 Shopify described scaling the **Shop app**'s backend with Vitess and said plainly that "Shop app has different properties than the rest of Shopify"; the post does not say the core platform uses Vitess. In February 2026 a database vendor's blog, recapping a Shopify engineer's talk at the vendor's conference, said Shopify runs one of the largest clusters of that vendor's database; no Shopify page confirms it.

> **INFERENCE, NOT DOCUMENTED:** The pod is Shopify's unit of isolation, failure and recovery, and the shop is the sharding key throughout the record: a noisy shop is contained to its pod, and a shop is moved as a whole. Whether the pod topology survived the move to Google Cloud unchanged is not documented; the 2019 backup post and the 2021 balancing post both describe

shards on Google Cloud, so pods and the cloud coexisted at least to 2021. The platform-wide scale is reported only for peak weekends: 14.8 trillion database queries and 1.75 trillion writes over the 2025 Black Friday weekend.

For a merchant none of this is machinery to copy. It is a question to ask any SaaS vendor, at the Growing stage already: **what is your unit of isolation, and can you move or restore my tenant alone?** Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation works that question out.

Storefront Delivery

Shopify offers three rendering models side by side. The default is a **theme** (files in Shopify's template language, Liquid) rendered by the Storefront Renderer; at Unite in June 2021, Online Store 2.0 made themes "fully modular" with sections on every page and let apps add blocks the merchant places in the theme editor. The second is **Hydrogen on Oxygen**: Hydrogen, introduced in 2022 as "our modern React-based framework", is described in the 2026 documentation as "Shopify's headless toolkit", supplying clients for the Storefront and Customer Account APIs and nothing for the admin or the checkout; Oxygen hosts it on Shopify's edge. The third is any framework against the **Storefront API**, which "is available only in GraphQL", has three access modes (tokenless with a complexity cap, a public token, a private server-side token), and promises that "requests from real buyers aren't subject to a fixed request-per-minute limit" while bots are limited "most strictly when unsigned".

The performance gates are specific. Themes need "a minimum average Lighthouse performance score of 60" for Theme Store acceptance; public apps "shouldn't reduce Lighthouse performance scores by more than 10 points"; the admin's performance report warns that "some features can slow down your site", naming theme code and apps.

INFERENCE, NOT DOCUMENTED: Under all three rendering models the cart, the checkout and the Storefront API are the shared substrate, which is why a headless storefront on Shopify still hands off to Shopify's checkout. The "ten Lighthouse points per app" rule is also a tell: apps are the usual cause of slow stores, and the platform polices them at submission because it cannot police them after installation. Delivering the Storefront: Themes, Headless, Rendering and Performance treats the headless decision; for Kestrel & Pine, with a themed storefront and 41 apps, the app gate is the more relevant fact.

Checkout: Freedom Withdrawn on a Schedule

Checkout is where Shopify's "single branch" rule and its revenue model meet, and the history is a timeline of withdrawn freedoms with published dates.

Date	Event
29 June 2021	Unite: first checkout extension (post-purchase) in beta; Scripts rebuilt on WebAssembly promised for Plus
9 February 2023	Winter '23: checkout extensibility "now available"
13 February 2023	Changelog: `checkout.liquid` customizations "will no longer work for in-checkout pages starting August 13, 2024"; non-upgraded checkouts "will be automatically upgraded to an un-customized, out-of-the-box Shopify Checkout"
13 August 2024	`checkout.liquid` stops working for the information, shipping and payment pages (Plus)
28 August 2025	`checkout.liquid` and additional scripts sunset for the Thank you and Order status pages (Plus)
15 April 2026	"Editing and publishing new Shopify Scripts will no longer be possible"
30 June 2026	"All Shopify Scripts will cease to execute entirely"
26 August 2026	Script tags on the Thank you and Order status pages sunset for non-Plus stores

Sources: Shopify's changelog (2023, 2026), the Liquid reference and Help Center (accessed November 2026), the Editions pages (2023).

What replaced the freedom is **checkout extensibility**: a fixed set of slots where apps may render user-interface extensions, a branding API, web pixels for tracking, and **Shopify Functions**, which "allow developers to customize the backend logic of Shopify" in any language that compiles to WebAssembly, invoked by the platform during the purchase flow, never by URL. Functions run in checkout order (cart transform, discount, fulfillment constraints, order routing, delivery and payment customization, validation) under hard limits: a 256 KiB module, 10 MiB of memory, and for carts up to 200 lines about 11 million instructions. Public apps with Functions work on any plan; custom Functions and "advanced checkout customization features are available only on the Shopify Plus plan".

Shop Pay is the other half of the story: "an accelerated checkout option" that stores a buyer's details, "included as part of Shopify Payments". Its conversion claims ("as much as 50% better conversion compared to guest checkout") rest on a 2023 study Shopify has not published; they are vendor claims. The documented reach figure is from Shopify's

own peak reporting: 32% of orders over the 2025 Black Friday weekend.

> **INFERENCE, NOT DOCUMENTED:** The arc is free-form template and script access to checkout, withdrawn over three and a half years with dated deadlines, replaced by sandboxed extension points with hard budgets. Shopify traded merchant freedom for an upgrade-safe checkout contained within the card-data security standard's scope (PCI DSS) that it can change under everyone at once, and the sandbox limits are what let it run merchant logic inside its own transaction at peak. For a merchant this is the reference case for "the platform will withdraw an extension point: plan for it", worked in From Cart to Confirmation: Checkout Orchestration and Apps, Plugins and Functions: Extensibility and Ecosystem Governance. The Forked Platform pattern is what Shopify made impossible by design; the Pulled Rug is what it did instead, with notice.

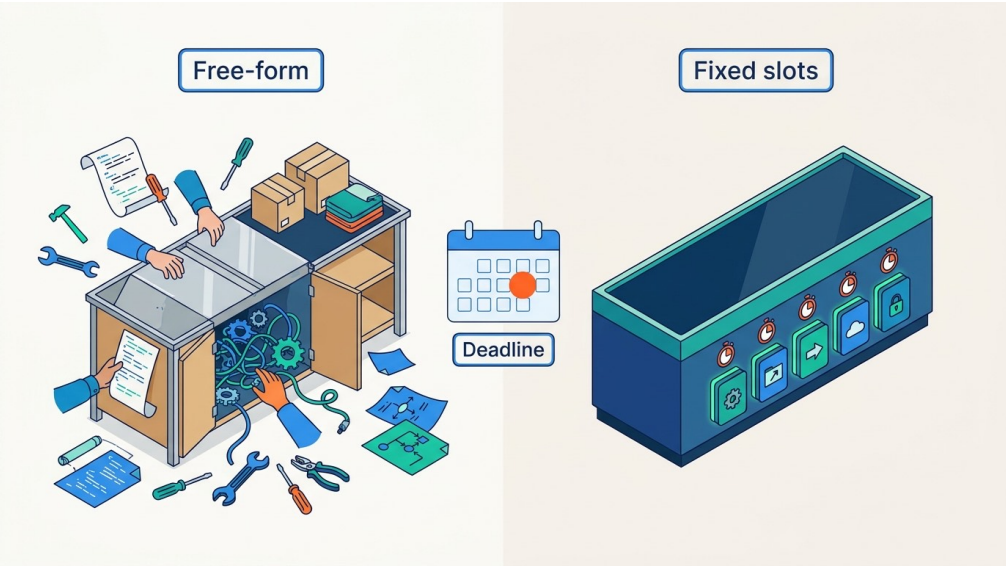


Figure 2: *Over three and a half years and on dated deadlines, free-form checkout access was withdrawn and replaced by sandboxed extension slots with hard budgets.*

Payments: Orchestration, Not a License

Shopify Payments is the clearest public example of a platform that earns from payments without being a payment institution. Its European terms (accessed November 2026) name Shopify International Limited in Ireland as the contracting entity and state: "None of the Shopify entities listed above are licensed as a bank, payment institution, or

money services business." The licensed **PSP** (payment service provider, the company that actually moves the money) is a separate party under separate terms; the processor list names, for the Netherlands, Belgium and Germany alike, Stripe Payments Europe, PayPal (Europe) and Adyen N.V., without saying which handles which transaction. Shopify Payments is available in 41 countries, including the three Kestrel & Pine sells in.

> **INFERENCE, NOT DOCUMENTED:** Shopify owns the checkout, the buyer wallet and the merchant contract, and resells licensed processing from Stripe, PayPal and Adyen; "Shopify Payments" is commerce orchestration over a white-labeled processor stack, which is why the 10-K reports two suppliers as a concentration risk. Taking the Money: Payment Providers, Authorization, Capture and Refunds owns the mechanics and hands the ledger side to the sibling book's Payments for Finance Architects: Eleven Reasons the Numbers Disagree; what this case adds is the business shape: whoever owns the checkout owns the payment revenue, whether or not they hold the license.

Inventory States, Briefly

Shopify's inventory model is a documented worked example of the distinction Is It in Stock? Inventory, Availability and the Promise You Can Keep owns. Stock is held per location in exclusive states: *available* ("the inventory that a merchant can sell"), *committed* ("set aside" by placed but unfulfilled orders), *reserved*, *damaged*, *safety_stock* ("to help guard against overselling"), *quality_control* and *on_hand* ("the total number of units that are physically at a location", the sum of the others), with *incoming* outside on-hand. The states arrived in steps: committed and incoming before February 2023, safety stock, damaged and quality control in August 2023, and on 5 August 2026 holds for draft orders and transfers moved from reserved to committed. Shopify holds no per-customer allocation for B2B and computes no cross-location available-to-promise; those belong to an order management system (OMS), which is why Kestrel & Pine's W1 buys an availability service beside the platform.

The Ecosystem: Apps, Scopes, Webhooks and Versions

The App Store is both a business model and a governance problem. **Business**: "more than 21,000 apps" at the end of 2025; developers pay 0% on the first $1,000,000 of lifetime app revenue and 15% above it. **Governance**: an app must "immediately authorize using OAuth before any other steps occur" (the standard login-and-consent

flow); apps "using APIs that will be deprecated within 90 days can't be submitted"; every app must implement three privacy compliance webhooks and return `401` on a bad signature, or "your app will be rejected". **Scopes** control "which store data your app can read and write", declared by the app and approved by the merchant at install: `read_inventory` and `write_inventory`, `read_orders` (covering "orders created within the last 60 days"), with `read_all_orders` and protected customer data needing separate approval.

Webhooks are the integration contract, and Shopify's own words set the baseline every integrator should assume: "Webhook delivery isn't always guaranteed"; "Shopify doesn't guarantee ordering within a topic, or across different topics for the same resource"; "ignore duplicate deliveries using `X-Shopify-Webhook-Id`"; "use reconciliation jobs to periodically fetch data from Shopify so that your app stays consistent". Delivery allows five seconds; "if Shopify receives no response or an error, it retries 8 times over the next 4 hours"; and "after 8 consecutive failures, the subscription is automatically deleted". Each delivery carries a signature (an HMAC, a keyed hash the receiver can verify). In 2021 the platform sent 13 billion webhooks.

API versioning is dated and guaranteed: "Shopify releases a new API version every three months", each "supported for a minimum of 12 months", and "if your app targets an inaccessible version, Shopify falls forward and responds using the oldest accessible stable version". The REST Admin API "is a legacy API as of October 1, 2024". Rate limits are per app per store and scale with the plan: the GraphQL Admin API allows 100 points a second on standard plans, 1,000 on Plus and 2,000 for enterprise, with no single query over 1,000 points, and "Shopify might temporarily reduce API rate limits to protect platform stability".

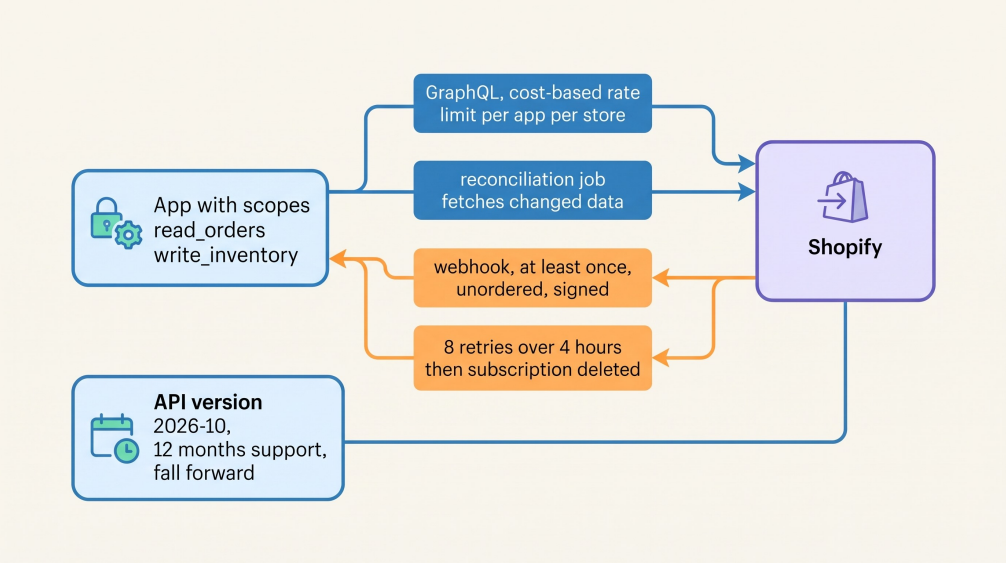


Figure 3: *Shopify's documented integration contract: scoped access, cost-based rate limits, at-least-once unordered webhooks that stop after eight consecutive failures, a reconciliation job the consumer must run, and dated API versions with a guaranteed window.*

INFERENCE, NOT DOCUMENTED: Three rules follow. Webhooks are at best at-least-once and unordered, so every consumer needs idempotent handling and a periodic reconciliation, which is Waypoint W5 in Shopify's own words. The "subscription deleted after eight consecutive failures" rule is the one that silently breaks an integration after an outage on the consumer's side; a stock-sync connector that was down for five hours on a Friday night has no subscription on Saturday morning. And rate limits are a tenancy tool: an integration's throughput on a peak day is bounded by plan tier and by how many apps share the store, so a polling connector can starve another app. Promises Between Systems: API Contracts, Webhooks and Versioning generalizes the versioning rule; Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation the rate limits.

Peak Days With Definitions

Shopify reports its Black Friday weekend every year, and the figures are useful only with their definitions.

Metric	BFCM 2023	BFCM 2024	BFCM 2025
Sales (GMV, unaudited)	$9.3 billion (+24%)	$11.5 billion (+24%)	$14.6 billion (+27%)
Peak sales per minute	$4.2 million	$4.6 million	$5.1 million (all at 12:01 p.m. EST on Black Friday)
Peak load	967,000 requests per second	284 million edge requests per minute; 80 million+ app-server requests per minute	489 million edge requests per minute; 117 million+ app-server; 31.8 million API
Database over the weekend	—	10.5 trillion queries, 1.17 trillion writes	14.8 trillion queries, 1.75 trillion writes
Shop Pay	sales +60%	sales +58%	sales +39%; 32% of orders
Merchants with their best day ever	55,000+	67,000+	94,900+

Sources: Shopify's announcements of 28 November 2023, 2 December 2024 and 2 December 2025. Every year carries the caveat that the data are "unaudited and subject to adjustment" and that prior years "may not be directly comparable" because methodology changes. The 2023 load figure is per second and does not say edge or app server; no year reports an order count.

How Shopify prepares is documented twice. In 2023: a "full-scale approach, where we scale the systems to their expected BFCM profile", with scripted browse, buy and flash-sale flows that "bypass the CDN for a portion of the traffic" (the content delivery network, the cache layer in front of the platform) so the origins are exercised, and an abort switch that stops every test at the first sign of degradation. In 2025: preparation "started in March with capacity planning", game days on "critical journeys" (checkout, payment processing, order creation, fulfillment, search) with injected network faults, and five scale tests from April to October reaching "146 million requests per minute and 80,000+ checkouts per minute" by the fourth; in early tests "core operations threw errors and checkout queues backed up", and authenticated checkout "exposed rate-limit paths that anonymous browsing never touches". The checkout queue itself is documented only in a 2018 changelog: during flash sales "processing payments at checkout happens separately from creating orders", so the buyer finishes at once while the order record, and the confirmation e-mail, follow after a delay. **(Evidence age: eight years for the queue; the Storefront API reference still documents a per-minute checkout throttle.)**

> **INFERENCE, NOT DOCUMENTED:** Two ratios matter more than the absolutes. Edge requests are about four times app-server requests (489 million against 117 million in 2025), so roughly three quarters of peak traffic is absorbed by the content delivery network and caches before it reaches application code. And the peak minute ($5.1 million) against the weekend average (about $2.7 million a minute over roughly 5,400 minutes) is a platform-level peak-to-average ratio of only about two, because millions of merchants' peaks do not coincide. A single merchant sees its own 41-times spike, and so do its connectors, its ERP (the finance and back-office system) and its WMS. The transferable practices are the shape of the program, scaled down: test at forecast load with caches bypassed, rehearse failure on the money paths, decouple payment from order creation so the money step stays fast while the record catches up. Black Friday Every Year: Peak Load, Resilience and Recovery and Journey 4: Black Friday, Hour by Hour apply them.

AI and the Agent Surface: Rebuilt Within a Year

Shopify's AI story has a merchant-facing side and an agent-facing side, and both moved fast. **Sidekick**, the assistant in the admin, was announced in July 2023; on 21 May 2025 Shopify said it could run reports, create segments, filter orders and set up discounts "with the merchant's approval", and the help page (accessed November 2026) describes it editing orders and presenting changes for review. No page states its general-availability date; the 4.2-times growth in daily sessions claimed in the Fall 2026 edition is a vendor figure.

The agent-facing side is the clearest dated example of protocol churn in the book. In May 2025 Shopify built a **Storefront MCP** server on the Model Context Protocol, the tool-calling transport that lets an AI agent discover and call operations such as "search catalog" (Tool Ecosystems, MCP and the Tool Broker). On 10 December 2025 **Agentic Storefronts** became "available to every merchant", syndicating products to AI chat products through a **Catalog**. On 11 January 2026 Shopify co-announced the Universal Commerce Protocol with Google and an **Agentic plan** for brands not on Shopify, at "no monthly fee" and card rates "from 1.9% + €0.25". By November 2026 the developer documentation states that the Storefront MCP catalog and cart tools "were removed" and that UCP "now provides these capabilities" at a new endpoint; every agent request must carry a profile hosted at a well-known URL, Checkout MCP "requires authenticated or signed requests", and "higher trust tiers unlock broader access, including direct checkout completion". The Catalog documentation lets agents cache price and availability for at most fifteen minutes, labels some fields as AI-inferred, prohibits bulk harvesting, and warns that "endpoint URLs might change".

The geography is the fact a European merchant needs. The ChatGPT channel requires that "your store must sell to customers in the United States", with purchases completed

on the merchant's own checkout since OpenAI's March 2026 retreat from in-chat checkout. The Google channel offers direct checkout only to customers in the United States, Canada and Australia, excludes subscriptions, bundles and B2B-only products, and does not fire the merchant's analytics pixels. **No EU buyer market is in scope on any agentic channel page read at the cut-off.** Assistants and Agents: AI in Commerce and What Is Real in 2026 owns the subject and the readiness list; what the case adds is a platform's own record of replacing its agent interface within twelve months, which is the strongest argument in the book for adapters over one internal API rather than channel-specific builds.

Shopify's use of AI in its own engineering is self-reported: a chief-executive memo of April 2025 making "reflexive AI usage" a baseline expectation; an agent that "only works in the open" in Slack and co-authors "one in eight merged pull requests" by Shopify's own count (May 2026); a rebuilt native Shop app in twelve weeks, gated by tests, "two adversarial code reviews, and human approval" (September 2026). Who Builds and Who Owns: Teams, Vendors, Agencies and Governance treats these as a prompt for a merchant's own baseline, with the metrics as the vendor's.

What Transfers to a Merchant, and What Does Not

The table is the author's synthesis; each row names the dated evidence behind it.

Shopify practice	Evidence	Transfers?	How
The shop as the unit of isolation, move and recovery	Pods 2018; balancing 2021	As a question to vendors; as a rule for a Group's own services	Ask every SaaS vendor; enforce tenant identifiers in shared services
Pods, Ghostferry, a custom observability platform	2018–2026	No	Scale-specific machinery; use managed services
Gate new boundary violations in code; do not chase zero	Packwerk 2020 and 2024	Yes, for in-house code	A boundary check in the build pipeline with modest targets

Shopify practice	Evidence	Transfers?	How
Carve out a hot path only on measured latency, behind a verifier with fallback	Storefront Renderer 2020	Yes, the rollout pattern	Shadow-compare before any cutover
Dated API versions with a guaranteed window; deprecate with dates and enforce	Versioning docs; checkout timeline 2023–2026	Yes, as a maintenance budget for every connector	Inventory connectors by API version and support end
Webhooks as at-least-once, unordered, signed, reconciled	Webhook docs	Yes, everywhere	Idempotent consumers and a reconciliation job per flow (W5)
Sandboxed extension points instead of in-process scripts	Functions docs; Scripts sunset 2026	As a governance idea	Rules in bounded places; never in a forked core
Rate limits per app per store, lowered under stress	Rate-limit docs	As a peak-day constraint	Know the budget before Black Friday; one writer per fact
Full-scale load tests with cache bypass; game days on money paths	2023 and 2025 posts	Yes, scaled down	Test connectors, ERP and WMS at forecast load

Only Shopify needs the rest: pods and shards balanced by simulation, a renderer deployed at the edge worldwide, an observability platform ingesting 100 million events a second at peak (a vendor's account of a 2026 talk), a readiness program that starts in March. A merchant who copies that list wears the Giant's Boots; a merchant who copies the disciplines in the table, and reads the vendor's changelog as carefully as Shopify's integrators must, gets the useful part for free.

What Kestrel & Pine takes from the case. Four things, each already in Waypoint. From the webhook contract: every integration idempotent, with a reconciliation job, because

the platform itself says delivery is not guaranteed (W5). From the versioning rule: every connector and app pinned to a dated API version with its support end on the ownership map, after the Deprecated Endpoint cost two days of stale stock on bol (W5, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance). From the checkout timeline: nothing customized in the checkout that the platform could withdraw, and a rule that custom behavior goes to the lowest rung that works (the app governance rule of Apps, Plugins and Functions: Extensibility and Ecosystem Governance). From the agent surface: readiness through product data, availability and order status by API, and no channel-specific build until a protocol has held still for a year (W8). What it does not take is the architecture: it rents the pods, the renderer and the checkout queue, which is the point of W4, stay and surround.

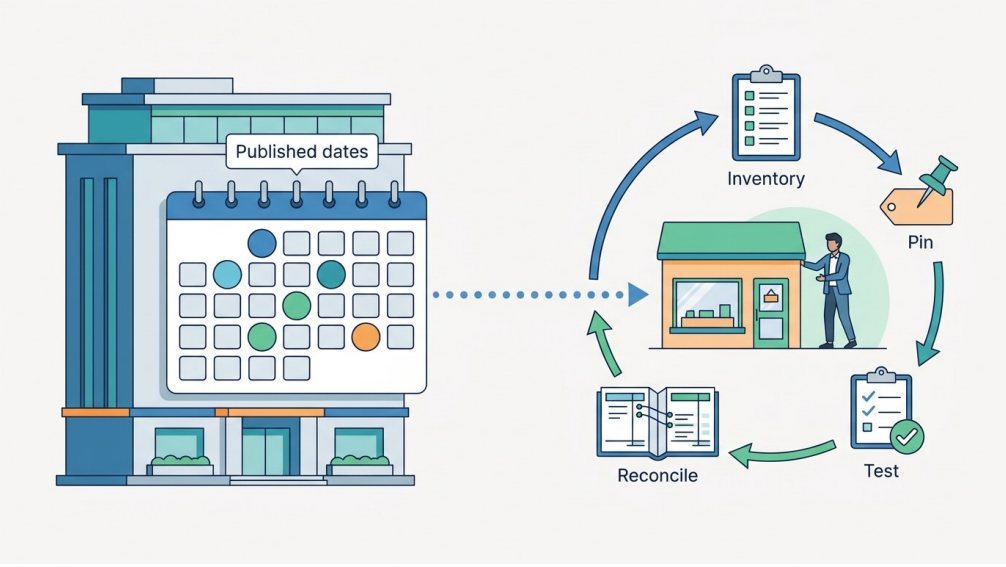


Figure 4: *The landlord changes the platform under everyone on dates it publishes; the merchant's job is to inventory, pin, test and reconcile against that cadence.*

Where the Evidence Stops

Be clear about what this chapter could not establish.

- **Today's topology.** The pod and shard counts, and whether pods survived the Google Cloud migration unchanged, are undocumented after 2021. The component count and Packwerk coverage after February 2024, and deploy frequency after the 2023 figure of "approximately every 30 minutes", are unknown.

- **The checkout queue**. Only a 2018 changelog and a one-line reference to a per-minute throttle exist; community descriptions of a "checkout queue" are not cited here.
- **Merchant and app counts**. Shopify says "millions" of merchants and "more than 21,000" apps; third-party trackers disagree widely and are not used.
- **Shop Pay**. The "150M+ shoppers" figure is reach, not active users; the conversion claims rest on an unpublished 2023 study.
- **Sidekick's general-availability date**, the Agentic plan's country eligibility, the Catalog API's commercial terms (its reference page returned 404), and whether any EU buyer market has a date for agent checkout.
- **The new database**. A vendor's 2026 blog is the only source; Shopify has not published it, and which system it backs is unknown.
- **Scripts' earlier deadline**. An August 2025 date and its extension were seen only in search snippets; the changelog posts opened state only the 2026 dates.

What to Take Away

Established.

- Shopify's revenue is three quarters merchant solutions, correlated with GMV, and two thirds of GMV runs through Shopify Payments, which is orchestrated over licensed processors (10-K FY2025; payment terms, 2026).
- The core platform is one Rails codebase, multi-tenant and single-branch, sharded by shop into isolated pods, with a modularity tool kept as a gate on new base-layer dependencies rather than as enforced domain boundaries (Shopify 2018, 2020, 2021, 2024; 10-K FY2025).
- The integration contract is explicit: at-least-once unordered webhooks that stop after eight consecutive failures, reconciliation jobs expected of the consumer, dated API versions supported for at least twelve months, rate limits per app per store (Shopify developer documentation, 2026).
- Extension points are withdrawn on published dates: `checkout.liquid` (announced February 2023, enforced August 2024 and August 2025), Shopify Scripts (June 2026), the REST Admin API (legacy since October 2024), the Storefront MCP catalog and cart tools (replaced by UCP endpoints in 2026).

Depends on context.

- Whether the modular-monolith lesson applies depends on owning code: a merchant on a hosted platform has nothing to modularize; a vendor or a Group with in-house services should gate new dependencies and not chase zero violations.

- Whether to run a Shopify-style readiness calendar depends on where the merchant's peak breaks: for Kestrel & Pine the platform held and the connectors failed, so the calendar belongs to the integrations, not the storefront.
- Whether the Plus plan's extension points are worth their price depends on needing in-checkout logic at all; a merchant that keeps custom behavior out of checkout avoids both the fee and the next deprecation.
- Whether the agent channels are worth switching on depends on the buyer geography, which excludes the EU at the cut-off; for a European merchant they are discovery and nothing more in 2026.

The author's synthesis.

- The useful Shopify is not the architecture but the landlord's behavior: it changes the platform under everyone on dates it publishes, and the merchant's job is to inventory, pin, test and reconcile against that cadence.
- Shopify's own record undermines the slogan made from it. It did not modularize its monolith into clean domains; it learned which gate was worth keeping, and that honesty is the lesson to copy.
- A platform that rebuilt its agent interface within a year is the best argument I know for a merchant to put nothing channel-specific below an adapter.

Questions to Ask

1. Which of our platform's extension points have a published sunset date, and which of our apps and scripts depend on them?
2. Which API version does each of our connectors target, when does that version leave support, and who owns the upgrade?
3. What does our platform promise about webhook delivery, ordering and retries, and which of our flows runs a reconciliation job to cover the gap?
4. What is our platform's unit of isolation, and can the vendor move or restore our tenant alone?
5. What rate-limit budget does each app and connector have on our plan, and which two of them write the same fact?
6. Which of the vendor's figures are audited, which are defined (GMV, merchants, penetration), and which are claims from unpublished studies?
7. Which checkout behavior have we customized that the platform could withdraw, and what is the un-customized fallback?
8. For every agent channel the platform has switched on by default, which countries, which products and whose checkout?

Shopify is the landlord's record; the next case is the retailer that built the everything store and now sells its machinery to others. Continue with Case: Amazon, From Monolith to Services to the Everything Store.

Sources and Evidence

Company, product and investor sources

- Form 10-K, fiscal year ended 31 December 2025, Shopify Inc. via SEC EDGAR, 2026 — definitions, revenue mix, Payments penetration, suppliers, technology description, app count.
- Form 8-K exhibit 99.1, Q3 2026 results and Form 10-Q, quarter ended 30 September 2026, Shopify Inc. via SEC EDGAR, 2026 — latest quarter.
- Deconstructing the Monolith, Shopify Engineering (K. Westeinde), 2019 — componentization and the case against microservices.
- Under Deconstruction: The State of Shopify's Monolith, Shopify Engineering (P. Müller), 2020 — size, components, dependencies.
- Enforcing Modularity in Rails Apps with Packwerk, Shopify Engineering (M. Ong), 2020, and A Packwerk Retrospective, Shopify Engineering, 2024 — the boundary tool and what it taught.
- A Pods Architecture To Allow Shopify To Scale, Shopify Engineering (X. Denis), 2018; Shard Balancing, Shopify Engineering (P. Madan), 2021; Ghostferry, Shopify, n.d., accessed 2026-11 — pods, shop moves.
- Horizontally scaling the Rails backend of Shop app with Vitess, Shopify Engineering, 2024 — Shop app only; How Shopify is re-architecting with YugabyteDB, Yugabyte (vendor blog), 2026 — unconfirmed vendor account.
- How Shopify Reduced Storefront Response Times with a Rewrite, Shopify Engineering (M. Vaillancourt), 2020, and Cloud, Load, and Modular Code, Shopify Engineering (A. Leinwand), 2022 — the Storefront Renderer and 2021 volumes.
- Adventures in Garbage Collection, Shopify Engineering, 2023 — deploy cadence; Shopify: observability at global scale, ClickHouse (vendor write-up), 2026.
- Shopify Unite 2021 announcements, Shopify Partners Blog, 2021; Developer highlights from our Winter '23 Edition, 2023; Summer '23 Edition, 2023 — Online Store 2.0, checkout extensibility, Sidekick.
- Hydrogen, Storefront API, Theme performance, App requirements checklist, shopify.dev, n.d., accessed 2026-11 — rendering models, access modes, performance gates.
- The checkout.liquid theme file is being deprecated, Shopify Changelog, 2023; Shopify Scripts will be deprecated on June 30, 2026, shopify.dev changelog, 2026; checkout.liquid layout reference and Checkout extensibility, n.d., accessed 2026-11 — the timeline and plan gating.
- Shopify Functions and Function APIs, shopify.dev, n.d., accessed 2026-11 — sandbox and limits.
- Shop Pay, Shopify, n.d., accessed 2026-11 — vendor claims; Shop Pay help, n.d.
- Shopify Payments Terms of Service (Ireland), Payment processors, Supported countries, Shopify, n.d., accessed 2026-11 — orchestration over licensed processors.

- Inventory management apps, shopify.dev, n.d.; On hand and Unavailable states in POS, 2023; Reserved to committed, 2026 — inventory states.
- Revenue share, Privacy law compliance, Access scopes, Webhooks, Webhooks: HTTPS delivery, API versioning, REST Admin API, GraphQL rate limits, REST rate limits, shopify.dev, n.d., accessed 2026-11 — the ecosystem contract.
- BFCM 2023, 2023; BFCM data 2024, 2024; BFCM data 2025 and record $14.6 billion release, 2025, Shopify News and Investors — peak figures and caveats.
- Performance Testing At Scale, Shopify Engineering (J. Neufeld), 2023, and How we prepare Shopify for BFCM, Shopify Engineering, 2025 — readiness programs; Faster checkout during flash sales, Shopify Changelog, 2018 — the decoupled checkout.
- Expanding your AI horizons, Summer '25, 2025; Sidekick, n.d.; Winter '26 Edition: Renaissance, 2025; The agentic commerce platform, 2026; Fall '26 Edition, 2026; Agentic plan, n.d., Shopify — Sidekick, Agentic Storefronts, UCP, the Agentic plan.
- Agentic storefronts, ChatGPT, Google AI Mode and Gemini, Shopify Help Center, n.d., accessed 2026-11 — buyer geography; Build for agents, Catalog, Storefront MCP, shopify.dev, n.d., accessed 2026-11 — profiles, trust tiers, the replaced endpoints.
- Internal memo: Shopify CEO declares AI non-optional, Digital Commerce 360, 2025; Under the River, 2026; Back to native, 2026, Shopify Engineering — AI in Shopify's engineering, self-reported.
- Shopify says purchases are coming inside ChatGPT as OpenAI retreats on Instant Checkout, Modern Retail, 2026 — the March 2026 change to the ChatGPT channel.

Standards, regulation and official documentation

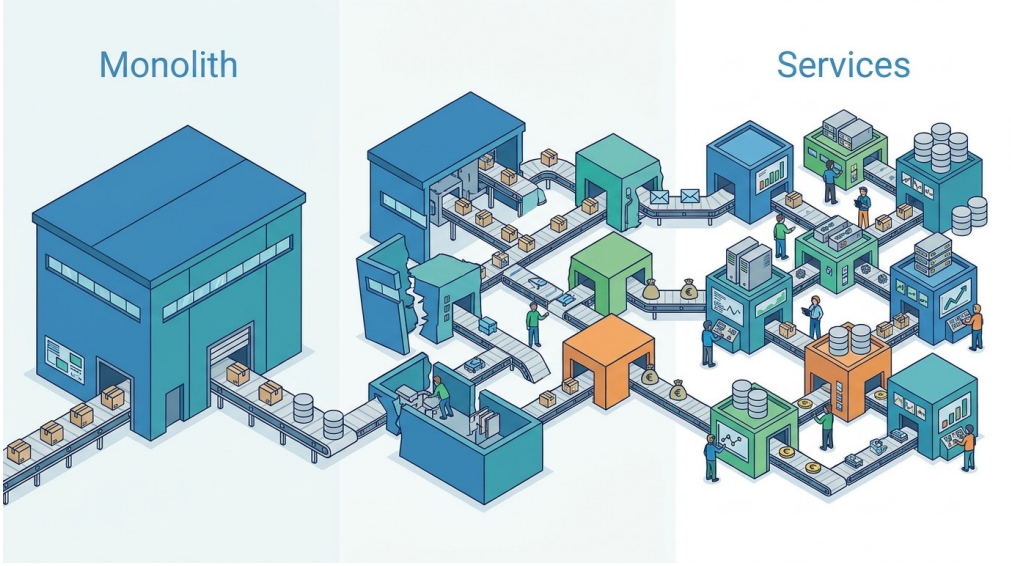
- Introducing the Universal Commerce Protocol, Google, 2026, and ucp.dev, UCP Authors, 2026 — the protocol Shopify's agent endpoints implement.

Research and practitioner sources

- None beyond the first-party and trade sources above; the chapter deliberately relies on Shopify's own dated record.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

47. Case: Amazon, From Monolith to Services to the Everything Store



IN THIS CHAPTER, YOU WILL: Read Amazon's own dated accounts of its architecture and business, separate what they document from what people assume, and decide which lessons a merchant can use.

KEY POINTS:

- **Amazon split its shop into services because it hit a wall, not because services were fashionable.** Its 2006 account dates the scaling wall to 2001; by 2007 one page touched 100 to 150 services. The transferable rule is the one that made the split survivable: every team exposes its data only through a service interface, and nobody reads another team's database.
- **The shopping cart is an availability problem, and the delivery promise is the product.** Amazon's 2007 Dynamo paper encodes a business decision in a data store: an add-to-cart must never be refused, so conflicts are merged later. Its 2024 to 2026 letters put the investment into the network that keeps the promise shown at checkout.

- **A retailer became a platform by opening its operations, not its storefront**. Prime (2005) and Fulfillment by Amazon (2006) took third-party sellers from 3% of physical sales in 1999 to 58% in 2018 and over 60% since. The same machinery is now exported to other shops and, since 2025, to AI agents, on Amazon's terms.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Giant's Boots** — Amazon's service decomposition is the architecture most often copied without its reasons, which were a scaling wall and hundreds of teams.
- **The Spaghetti Junction** — the rule Amazon is remembered for, no direct reads of another team's data store, is the cure for a landscape where every system reaches into every other.
- **The Secret Agent** — Amazon's 2025 position on third-party shopping agents (identify yourself, respect the provider's choice) is the clearest public statement of the rule every merchant will need for automation on its own site.

Amazon is three things to a merchant at once. It is a **channel**: Kestrel & Pine, the fictional outdoor brand this book follows, sells Ridgewalker jackets on Amazon.de through its marketplace connector. It is a **supplier of operations**: about 40% of Kestrel & Pine's German marketplace units are shipped by a marketplace's own fulfillment program, Amazon's among them, from stock sent there in advance. And it is a **competitor** with a delivery promise no brand can match alone.

Amazon is also the company whose architecture is most often held up as the model. "Amazon moved to microservices" and "the API mandate" are repeated in design reviews at companies a thousandth of its size. This chapter goes back to the **dated public record**: Amazon's own papers, blog posts, shareholder letters, press releases and its annual filing with the US Securities and Exchange Commission, the **Form 10-K**. The record is thinner and more specific than the folklore. It says a few clear things about why Amazon split its shop, how it built its cart, how it opened its warehouses to others, and what it invests in now. It says nothing about how today's order, pricing or promise systems are built.

Read the ages of the evidence as carefully as the evidence. The architecture story is from 2006 and 2007: history, nineteen years old, describing systems that have certainly been replaced. The business figures are from filings of 2025 and 2026. The AI features are from 2024 to 2026 and change by the month. Every claim is dated so you can tell which is which.

Where we are in the order's journey. This case sits beside the order's path. Three of Amazon's documented decisions touch steps of Eva's jacket: the cart that never refuses an add (From Cart to Confirmation: Checkout Orchestration), the delivery promise shown at checkout (Getting It There: Carriers, Labels, Tracking and the

Delivery Promise) and the fulfillment program that ships the marketplace variant of Eva's order from Amazon's warehouse rather than from Duiven (Selling on Someone Else's Platform: Marketplaces and Channel Integration).

How to Read a Case Study in This Book

A case study tests the book's argument against a real company, so it follows the evidence rules of Introduction: Why Commerce Architecture Matters more strictly than any other chapter. Every statement about Amazon below is **documented** (the source and its year are in the sentence), **widely reported** (a secondary source, or a primary page that could not be fetched, and the text says so) or an **inference**, labeled as such with its reasoning.

The research was done on 7 November 2026, and several primary pages were unreachable that day: Werner Vogels' 2006 interview in *ACM Queue* (the claims come from a 2007 digest of it), Fast Company's 2004 profile usually cited for the "two-pizza team", Amazon's 2005 Prime press release, and the February 2026 delivery-record release (taken from syndicated coverage).

2001: The Front End Hits a Wall

Start with vocabulary. A **monolith** is one deployable application that contains all of the shop's logic (catalog, cart, checkout, orders) and usually one shared database. A **service-oriented architecture**, or SOA, splits the shop into many separately deployed **services**, each owning one function and talking to the others over the network. In plain words: one big program with one database, versus many small programs that call each other.

Amazon's own account of its move from the first to the second is Werner Vogels' 2006 interview in *ACM Queue*, "A Conversation with Werner Vogels". As digested by High Scalability in September 2007, it describes the early shop as a "two-tier monolith" and dates the turning point: "In 2001 it became clear that the front-end application couldn't scale anymore." Amazon moved "to a fully-distributed, decentralized, services platform" that "grew into hundreds of services and a number of application servers that aggregate the information from the services". A single page request touched "between 100-150 services". Amazon's own 2007 paper on Dynamo, presented at the SOSP conference in October 2007, opens with the same statement in its own words: Amazon "uses a highly decentralized, loosely coupled, service oriented architecture consisting of hundreds of services", and those services have "stringent latency requirements which are in general measured at the 99.9th percentile of the distribution".

That last clause carries the engineering cost of the decision. **Tail latency** at the 99.9th percentile is the response time that only one request in a thousand exceeds. A page assembled from 100 services is only as fast as its slowest call, so Amazon measured

every service on its worst thousandth, not its average. The split bought independent teams and independent scaling; it cost a discipline about time budgets that a monolith never needed.

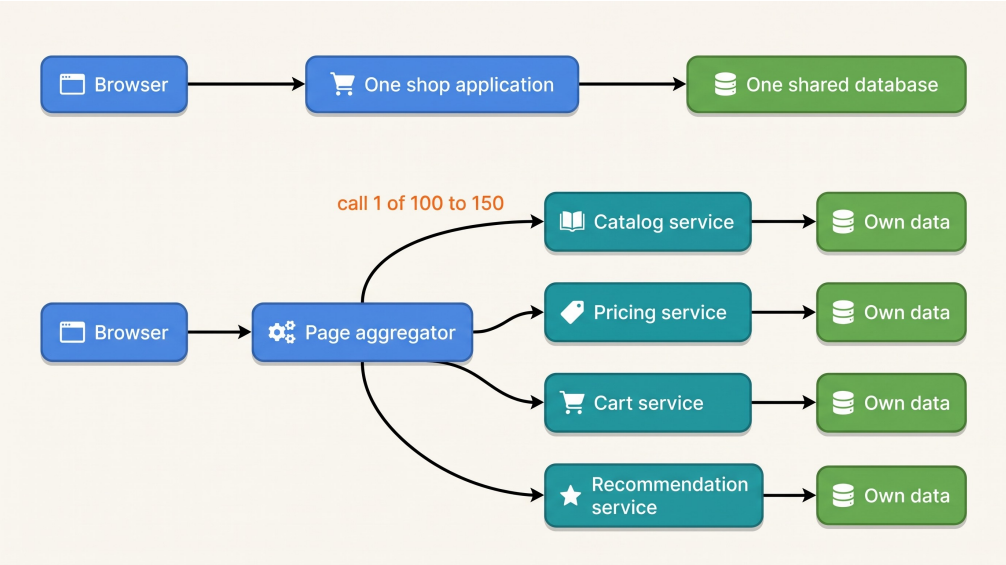


Figure 1: *The two shapes in Amazon's 2006 account: one application on one database until 2001, then a page assembled by an aggregator from hundreds of services that each own their data, where the slowest of 100 to 150 calls sets the page's speed.*

The sequence to take from this is **monolith**, **scaling wall**, **services**, in that order. The record does not say that Amazon decomposed early, or that services were a goal; it says the front end stopped scaling and the decomposition followed. Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says uses this and Zalando's 2015 move as the two dated examples of when a split is justified. The name retellings give the monolith does not appear in any page that could be verified, so it is not used here.

> **INFERENCE, NOT DOCUMENTED**: Amazon's 2001 wall was a wall of both load and people: hundreds of engineers changing one codebase. A merchant at Kestrel & Pine's scale, 293,000 online orders a year with fourteen people in technology, is nowhere near either wall, and nothing in Amazon's record argues for decomposing a shop before it hits one. The lesson transfers at a different altitude: the moment a merchant's product page is assembled from several *systems* (a PIM for content, the ERP for price, the availability service for stock, a reviews app), the merchant inherits Amazon's aggregation problem in miniature, including the tail-latency discipline. That is the Omnichannel stage, and it is where Kestrel & Pine is now.

The Cart That Never Says No

The Dynamo paper is the one place where Amazon explains a system in enough detail to learn from it, and the system it chose to explain is the shopping cart.

The motivation is stated as a business requirement. Customers "should be able to view and add items to their shopping cart even if disks are failing, network routes are flapping, or data centers are being destroyed by tornados". Dynamo targets "an 'always writeable' data store", meaning one "that is highly available for writes". The reasoning is commercial: a refused add-to-cart is a lost sale, while a cart that is briefly out of date is harmless. So the cart "must allow customers to add and remove items from their shopping cart even amidst network and server failures", which "forces us to push the complexity of conflict resolution to the reads".

Two more terms. **Eventual consistency** means that different copies of a piece of data may disagree for a short time and converge later. **Conflict resolution** is what happens when two copies disagree and the system has to produce one answer. Dynamo's rule for the cart is explicit: "an 'Add to Cart' operation can never be forgotten or rejected". If the latest version of the cart is unavailable, a change made to an older version "is still meaningful and should be preserved" but "shouldn't supersede the currently unavailable state of the cart". When versions diverge, they are merged: an added item is never lost, and the paper acknowledges the price, which is that a deleted item can resurface.

What the Dynamo paper documents (2007)	What it does not say
The cart service accepts writes during failures and merges conflicting versions on read	That Amazon runs Dynamo today (the 2007 post says it was internal; the 2012 post says teams moved to managed services)
In 2006–2007 the cart service "served tens of millions requests that resulted in well over 3 million checkouts in a single day"	Anything about the order, pricing or checkout systems behind the cart
Services are measured at the 99.9th percentile	What the budgets were
A business rule (never lose an add) is encoded in a data structure's merge rule	That the same rule is right for orders, payments or stock

The 2012 sequel is as instructive as the paper. Announcing the managed database service DynamoDB in January 2012, Vogels wrote that Dynamo was "originally built to serve the needs of the shopping cart" but "did nothing to reduce the operational complexity of running large database systems"; Amazon's engineers "strongly preferred simplicity

to fine-grained control" and chose managed services instead. A **managed service** is a database run by a provider so the team using it does not operate servers, replication or failover. In plain words: Amazon's own teams would not adopt a correct system that was hard to run. The 2022 USENIX paper on DynamoDB gives the scale of the second generation: during the 66-hour Prime Day event of 2021, calls to DynamoDB peaked "at 89.2 million requests per second" at single-digit-millisecond latency.

> **INFERENCE, NOT DOCUMENTED:** The cart is the one place in a commerce platform where availability beats consistency by design, and the Dynamo merge rule is the book's central idea in miniature: a decision about which promise a system may make (the cart promises to remember every add) encoded where it cannot be forgotten. The opposite rule applies one step later. A checkout that charges twice or an inventory that promises the last Ridgewalker to two customers is the Black Friday Oversell, and those systems need the consistency the cart gives up (Is It in Stock? Inventory, Availability and the Promise You Can Keep, Taking the Money: Payment Providers, Authorization, Capture and Refunds). For Kestrel & Pine the practical reading is short: its hosted platform's cart already behaves this way; a merchant that builds its own storefront should buy a managed, highly available store for the cart rather than reinvent Dynamo; and the 2012 lesson, that operability decides adoption, applies to every system a fourteen-person team is asked to run.

Figure 2: *The cart that never says no: diverging copies are merged so an added item is never lost, at the price that a deleted item can resurface (Dynamo, 2007).*

Teams, Ownership and the Contract as the Only Door

Amazon is as famous for how it organizes engineers as for what they built. Two of the three well-known rules are documented by Amazon; the third is not.

Two-pizza teams. Amazon's own DevOps guidance on the AWS documentation site (undated, accessed November 2026) quotes Jeff Bezos: "We try to create teams that are no larger than can be fed by two pizzas. We call that the two-pizza team rule." It ties the size to ownership: the team should see its service "as an end-to-end venture" rather than treating production "as black boxes where they have no visibility". The phrase is commonly dated to a 2004 Fast Company profile, which could not be fetched for this chapter, so the date is secondary. **You build it, you run it**, the rule that the team that writes a service also operates it in production, appears in the 2007 digest of the Vogels interview alongside the team rule.

The "API mandate". This is the rule most often quoted in architecture reviews, and it exists in public only as one person's recollection. Steve Yegge, a former Amazon engineer, wrote in October 2011 (in a post preserved as a public gist) that "back around 2002 I think, plus or minus a year" Bezos issued a mandate: "All teams will henceforth expose their data and functionality through service interfaces"; "Teams must communicate with each other through these interfaces"; no other communication, "no direct reads of another team's data store"; the technology "doesn't matter"; and "All service interfaces, without exception, must be designed from the ground up to be externalizable". No page from Amazon that the brief could fetch confirms the memo. Treat it as **widely reported**, not as an Amazon document.

What Amazon has confirmed, in its own words, is the outcome. In September 2006 Vogels wrote that Amazon "continues on the path of opening up all of its services for customers and partners to use", and that with Fulfillment by Amazon "we are now also opening up our fulfillment network for everyone to use". Ten years later, in his 2016 retrospective on AWS, he wrote that the lesson of building management APIs first was one "we had already learned from our experiences with Amazon retail".

An **externalizable interface** is an API designed so that it *could* be offered to outside developers, which forces a clean contract even when only internal teams use it. In plain words: build every internal door as if a stranger might walk through it. That test is the useful part of the mandate, whoever wrote it, and Promises Between Systems: API Contracts, Webhooks and Versioning carries it forward with Zalando's written, verifiable guidelines (Case: Zalando, Microservices, Platform Strategy and the Return to Discipline) as the documented version.

> **INFERENCE, NOT DOCUMENTED:** For a merchant, the rule applies to vendors before it applies to teams. Kestrel & Pine's 34 integrations include

agency scripts that read systems directly rather than through contracts, and the ledger's App That Wrote Stock is what happens when a second writer reaches into a system that already has an owner. Waypoint decision W5, a named owner and a contract for every integration, is the merchant-scale form of "no direct reads of another team's data store". At the Omnichannel stage the same rule becomes internal: the availability service of W1 exposes available-to-promise through an API, and nobody reads its tables.

Fulfillment as a Platform

This is the part of Amazon's record that most changes how a merchant should think, because it is about the business model, and the business model is documented in filings.

Three terms first. In the **first-party** model (1P), Amazon buys stock and is the seller. In the **third-party** model (3P), an independent seller sells on Amazon's site and Amazon takes fees. The **seller of record** is the legal party that sells to the consumer and carries the obligations: the invoice, the value-added tax, the consumer's right to return. **Fulfillment by Amazon** (FBA) is the service in which the seller sends stock to Amazon's warehouses and Amazon picks, packs, ships and handles returns.

The dates are clean. Amazon Prime launched in February 2005 at $79 a year for unlimited two-day shipping (widely reported; the press release was unreachable). Fulfillment by Amazon launched as a beta on 19 September 2006 with a press release promising small and medium-sized businesses "access to Amazon's order fulfillment, customer service, customer shipping offers, and underlying website technology"; third-party items "Fulfilled by Amazon" could qualify for Prime shipping. The two were designed to meet: a seller's item in Amazon's warehouse could carry Amazon's promise.

Bezos' 2018 shareholder letter, published in April 2019, gives the result as a series. Third-party sellers' share of physical gross merchandise sales:

Year	1999	2002	2005	2010	2012	2015	2018
Third-party share	3%	17%	28%	34%	42%	51%	58%

The letter's own comment: "Third-party sellers are kicking our first party butt. Badly." It credits two investments, "Fulfillment by Amazon and the Prime membership program", with having "meaningfully improved the customer experience of buying from independent sellers". Amazon's 2022 and 2024 Small Business Empowerment Reports (published May 2023 and May 2025) say "more than 60%" of sales in Amazon's store come from independent sellers; the 2024 report adds that those sellers "have shipped more than 80 billion items through Fulfillment by Amazon since its 2006 launch" and that Multi-Channel Fulfillment helped "more than 300,000 sellers" ship orders from their

other sales channels in the past year. Independent trackers put the unit share at 60 to 62% through 2024 to 2026.

The FY2025 Form 10-K, filed 6 February 2026, defines the model in one sentence: "We offer programs that enable sellers to sell their products in our stores, and fulfill orders using our services. We are not the seller of record in these transactions." Amazon earns "fixed fees, a percentage of sales, per-unit activity fees, interest, or some combination thereof", and designs its stores "to enable hundreds of millions of unique products to be sold by us and by third parties". The same filing gives the scale: net sales of **$716.9 billion** in 2025 (North America $426.3 billion, International $161.9 billion, AWS $128.7 billion), fulfillment expense of **$109.1 billion**, and approximately **1,576,000** full- and part-time employees at 31 December 2025. It does not state a count of fulfillment centers; it reports square footage.

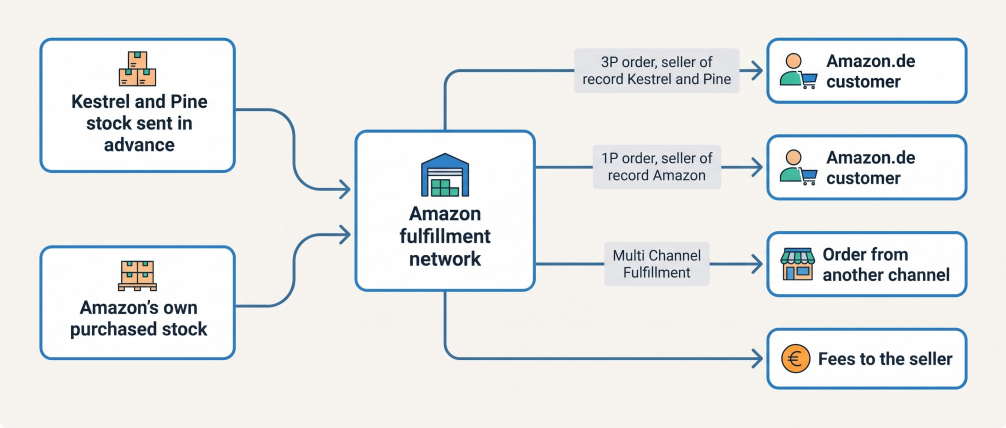


Figure 3: *One fulfillment network serves three flows with different sellers of record: Amazon's own stock, a third-party seller's stock under FBA, and orders from channels outside Amazon under Multi-Channel Fulfillment; the stock pool is shared, the legal and financial flows are not.*

INFERENCE, NOT DOCUMENTED: Amazon does not publish how FBA inventory, 1P inventory and Multi-Channel Fulfillment are kept apart inside its systems, so the diagram shows the documented flows, not the implementation. What the record does establish is that a retailer became a platform *through two operational services*, the promise and the warehouse, rather than through any storefront feature. The architectural consequence of running 1P and 3P side by side is one inventory and fulfillment capability serving two seller-of-record models with different invoices, tax treatment, returns obligations and ledgers. That is the split any merchant faces the day it fulfills a marketplace order from the pool that also serves its own shop, and it is why

Selling on Someone Else's Platform: Marketplaces and Channel Integration

> treats a marketplace order as a different *kind* of order, not a different source of the same one.

For Kestrel & Pine the relationship runs the other way: it is the third-party seller. The stock it sends to Amazon's program (part of the roughly 40% of German marketplace units that marketplace programs ship) is, in Waypoint decision W1, "a tracked external location": stock Kestrel & Pine owns but does not control, counted separately from Duiven, and sold only on the channel that holds it.

The Promise Is the Product

If the 2006–2007 sources are about the shop's internals, the 2024–2026 sources are about one thing: delivery speed. Amazon's recent letters and releases describe the shape of the network, and they describe nothing about the software that computes the promise.

Jassy's 2024 shareholder letter (published April 2025) attributes faster delivery "primarily to our regionalization redesign of our fulfillment network", to new placement algorithms and to same-day fulfillment centers, and confirms the starting scope of Prime: "Prime two-day delivery began with one million products and now covers over 300 million items, with tens of millions available in one day or better." The 2025 letter (released 9 April 2026) reports over one million robots in fulfillment centers, "more than 85 Same Day Fulfillment Centers" that carry the top 90,000 SKUs across the United States, and over $4 billion committed to a rural delivery network. The 3 February 2026 press release (syndicated; the primary page was unreachable) reports over 13 billion items delivered same or next day worldwide in 2025, over 8 billion of them in the United States, and a 70% rise in items delivered in under a day. These figures will be out of date within a year; they are quoted for their shape, not their size.

Three terms. **Regionalization** means holding stock in the region where it will be ordered, so most orders ship from nearby. A **placement algorithm** decides which warehouse should hold which SKU in what quantity. A **delivery promise** is the date or hour shown at checkout, which becomes a commitment the network has to keep. The 10-K's framing of Prime is commercial and candid: shipping offers and Prime benefits are "effective worldwide marketing tools" that Amazon intends "to continue offering them indefinitely".

What the promise needs	Amazon's documented answer (2024–2026)	What a merchant at Kestrel & Pine's scale does
Stock near the customer	Regionalized network; 85+ same-day centers with a curated 90,000-SKU set	One warehouse at Duiven; a static "ordered before 22:00, delivered tomorrow" rule for the Netherlands
Deciding where stock sits	Placement algorithms (named, not described)	Buyers decide by season; marketplace stock sent to Germany in advance
Capacity to keep the promise	Own delivery network, rural investment, robots	PostNL, bpost, DHL and DPD through the shipping service; the cut-off is the carrier's
Measuring the promise	Items delivered same or next day, published yearly	Not measured in 2026 *(ledger: the promise is static)*

INFERENCE, NOT DOCUMENTED: Amazon's public account makes the promise, not the catalog, the product, and the investment goes into the physical network that keeps it. The transferable idea is not the network. It is that **the promise engine, the logic that decides which date to show, is a first-class system** with inputs from stock, the network and carrier capacity, and that a promise is a commitment the fulfillment side has to be able to keep on the busiest hour of the year. Kestrel & Pine's "ordered before 22:00" rule is a promise with no engine behind it: it was true on an ordinary Tuesday and false on the Friday of Black Friday week, when the WMS queue backed up for four hours. Getting It There: Carriers, Labels, Tracking and the Delivery Promise owns the promise; Journey 4: Black Friday, Hour by Hour shows what happens when the promise and the capacity part company. Only Amazon's scale justifies 85 single-purpose same-day centers. The idea that transfers is the *curated subset*: a small set of SKUs for which a fast promise is made and kept, rather than one promise for everything.

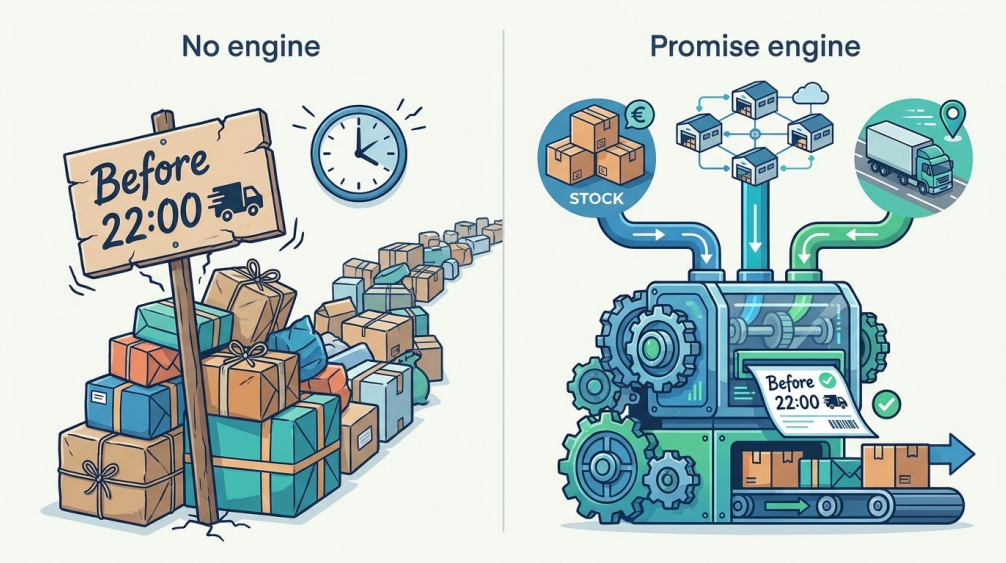


Figure 4: *Kestrel & Pine's "ordered before 22:00" rule is a promise with no engine behind it; the transferable idea is a promise engine fed by stock, the network and carrier capacity.*

Other Shops, Other Agents: 2022 to 2026

Amazon's most recent moves push the same machinery outward, into other merchants' storefronts and into software that shops on a customer's behalf. Everything in this section changes quickly; product names and fees in particular should be checked against the dated pages before being relied on.

Buy with Prime (launched 2022, widely reported) lets a merchant offer Prime delivery and Amazon's checkout on the merchant's own site, with Amazon handling "the payment, processing, storage, picking, packing, and shipping" plus customer service and returns; the product page (accessed November 2026) cites a 16% average lift in revenue per visitor from tests across 167 merchants between July 2023 and June 2024. **Shop Direct** (launched 2025, feeds added 2026) puts products from other stores into Amazon's search results: "over 100 million products from more than 400,000 merchants", synced through existing **product feeds**, structured files of catalog, price and stock, from Feedonomics, Salsify or CEDCommerce.

Rufus, Amazon's generative AI shopping assistant, launched as a beta in the US mobile app in February 2024 (per coverage); Amazon's page records that on 13 May 2026 it was renamed **Alexa for Shopping**. Amazon reports that it "helped over 300 million customers in 2025". The 2026 version adds actions: an automatic purchase when an item reaches a target price (a standing instruction that "stay[s] active for six months or until you cancel", with a 24-hour cancellation window), reorders and scheduled recurring purchases.

Buy for Me (beta, April 2025) goes further. For items Amazon does not sell, the app shows "Shop brand sites directly"; the customer confirms address, taxes, shipping and payment on Amazon's checkout page, and "Amazon completes the purchase on the brand's site". Amazon passes "encrypted name, address, and payment details" to the brand and "cannot see customers' previous or separate orders from other brands' sites"; the brand "handles delivery, returns, exchanges, and customer service" and chooses whether to participate. The catalog grew "from 65K products at launch to over half a million" and later to "tens of millions". Trade coverage of April 2025 reported that Amazon charged brands no commission at the time.

Agentic commerce is the term for software that does not only recommend but acts, placing an order on the customer's behalf. Assistants and Agents: AI in Commerce and What Is Real in 2026 owns it. What this case adds is Amazon's position on agents it does not control. In its statement on Perplexity's Comet browser, Amazon wrote that "third-party applications that offer to make purchases on behalf of customers from other businesses should operate openly" and "respect service provider decisions whether or not to participate", that such apps "have the same obligations" as food-delivery apps or online travel agencies, and that it had "repeatedly" asked Perplexity to remove Amazon from Comet, citing a "significantly degraded shopping and customer service experience". The cease-and-desist letter is dated 31 October 2025; Retail Dive reported on 6 November 2025 that Amazon had sued in Northern California and that Perplexity called the suit a "threat to user choice". The case's status was not checked.

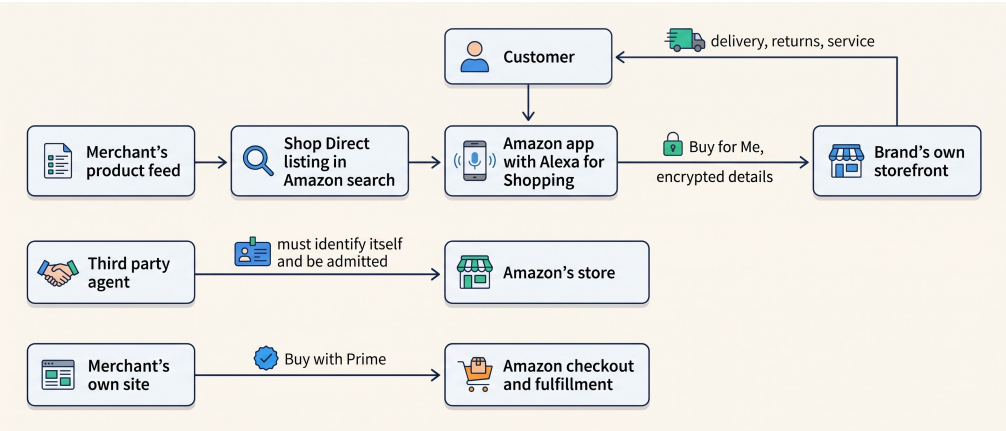


Figure 5: *Three documented directions: Amazon exports its checkout and fulfillment to merchants' sites (Buy with Prime), imports merchants' catalogs into its own surface (Shop Direct, Buy for Me), and admits outside agents only on its terms; the merchant's checkout and warehouse become modules behind someone else's demand.*

INFERENCE, NOT DOCUMENTED: Two patterns are visible. First, Amazon exports its promise-keeping machinery to other storefronts and imports

other storefronts' catalogs into its own, so that a merchant's checkout and fulfillment become interchangeable modules behind a dominant demand surface. Second, Amazon's rule for outside agents, identify yourself and respect the provider's choice to participate, is the rule every merchant will need for bots on its own site: a documented, authenticated channel for automation (feeds, APIs, a checkout protocol) and a way to refuse automation that will not identify itself. That is the Secret Agent pattern from the other side. For Kestrel & Pine it is Waypoint decision W8 in practice: accurate product data, availability and order status by API first, and no shopper-facing agent of its own in 2027.

A Nuance: When One Process Beat a Distributed Design

One Amazon post from 2023 is quoted as the reversal of everything above, and the source supports a much narrower claim.

In March 2023 the Prime Video team responsible for audio and video quality monitoring described replacing a **serverless** design, in which small functions run on demand and a managed workflow engine (AWS Step Functions) calls each step and stores intermediate results in object storage, with a single application process. The reason, as summarized by InfoQ on 3 May 2023 (the original post now redirects to a generic page): "running the infrastructure at a high scale was very expensive", the design "supported only around 5% of the expected load", and moving the data in memory instead of through storage cut operational costs by 90%. The team's own framing was that the "distributed approach wasn't bringing a lot of benefits in our specific use case".

That is a cost-and-coupling argument about one internal pipeline, not evidence about the retail site's architecture. The rule it supports, which Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says develops, is that **a boundary should be sized to the data that crosses it**: when most of the work is moving bytes between steps, the steps belong in one process. Zalando reached a similar conclusion about a stream-based integration in 2025 (Case: Zalando, Microservices, Platform Strategy and the Return to Discipline).

What a Merchant Can Learn, and What Only Amazon Can Do

The table is the author's synthesis; each row names the dated evidence it rests on.

Lesson	Evidence	Transfers to a merchant?
Split along ownership lines, and make the contract the only door	2006 interview digest; 2011 recollection of the mandate	Yes, first as a rule for vendor integrations (no direct reads of another system's data), then for internal services
The cart is an availability problem, not a consistency problem	Dynamo, 2007	Yes; buy, do not build, the highly available store; keep the consistency for stock and payment
Operability decides adoption	DynamoDB, 2012	Yes, at every stage; a fourteen-person team cannot run what Amazon's teams refused to run
Fulfillment opened to others turns a retailer into a platform	FBA 2006; 2018 letter series	Partly: the Group stage; an Omnichannel merchant starts by fulfilling marketplace orders from one pool
The promise is the product; shape the network to it	2024 and 2025 letters; February 2026 release	The idea, yes; the network only at Amazon's scale; the curated fast subset transfers
Agents must identify themselves and respect participation	Perplexity statement, 2025	Yes, as a policy and an authenticated channel
Size a boundary to the data flow across it	Prime Video, 2023	Yes

Only Amazon can do the rest: 85-plus single-purpose same-day centers and a million robots; a 300-million-item two-day catalog; a demand surface that 400,000 merchants feed; a database lineage from Dynamo to DynamoDB. The Giant's Boots pattern is a merchant mistaking the second list for the first.

What Kestrel & Pine takes from the case. Three things, each already a Waypoint decision. From the API rule: every integration gets an owner and a contract, and no script reads another system's data directly (W5). From FBA: the stock in Amazon's program is a tracked external location with its own count, sold only on the channel that holds it, never pooled with Duiven in the availability calculation (W1). From the

promise: "ordered before 22:00, delivered tomorrow" needs an engine behind it before Black Friday 2027, fed by the WMS's real capacity, or it will be withdrawn on the day it matters most (Black Friday Every Year: Peak Load, Resilience and Recovery). What it does not take is the architecture: a hosted platform with an OMS, a PIM and an availability service beside it is the right shape for a €48 million merchant (Build, Buy or Assemble: Choosing and Changing Platforms).

Where the Evidence Stops

Be clear about what this chapter could not establish.

- **Today's internals**. No public, dated description exists of Amazon's current order, pricing, promise or inventory systems. The 2006–2007 architecture is history; the Dynamo paper describes a system Amazon said in 2012 its teams had moved away from. Anything you read about "how Amazon's order system works today" is inference.
- **The API mandate** rests on one first-hand recollection from 2011, never confirmed by Amazon in any page fetched. Its content is consistent with Amazon's documented behavior, which is the most that can be said.
- **The monolith's name**, the 2004 dating of the two-pizza rule, the 2005 Prime release and the February 2026 delivery record are widely reported but were not fetched from primary pages. Fulfillment center counts are not in the 10-K, which reports floor area instead.
- **The agent products** (Alexa for Shopping, Buy for Me, Shop Direct) are described as of November 2026; their names, fees and scope have already changed once in 2026 and will again. The Perplexity litigation's outcome was not checked.
- **Costs**. Amazon publishes fulfillment expense in aggregate and nothing about the unit economics of FBA, Prime or same-day delivery.

What to Take Away

Established.

- Amazon's move from a monolith to hundreds of services followed a scaling wall it dates to 2001, and by 2007 a page touched 100 to 150 services measured at the 99.9th percentile (2006 interview digest; Dynamo paper, 2007). Decomposition followed the wall; it did not precede it.
- The shopping cart was built as an "always writeable" store that merges conflicts on read because a refused add-to-cart is a lost sale (Dynamo, 2007); Amazon's own teams then preferred managed services to a system that was hard to operate (DynamoDB, 2012).

- Amazon became a marketplace through Prime (2005) and Fulfillment by Amazon (2006): third-party sellers went from 3% of physical sales in 1999 to 58% in 2018 and "more than 60%" since (2018 letter; 2022 and 2024 reports). Amazon is "not the seller of record" in those sales (10-K, 2026).
- Amazon's stated rule for outside shopping agents is to operate openly and respect a provider's choice to participate (2025 statement).

Depends on context.

- "No direct reads of another team's data store" is right for a merchant as a rule about vendors and scripts at every stage; it becomes a rule about internal services only once there are internal services to govern, which for most merchants is the Omnichannel stage or later.
- Availability-over-consistency is right for the cart and wrong for stock, payment and orders. A merchant on a hosted platform inherits the right behavior; one building a storefront must choose it deliberately and buy the store rather than build it.
- Fulfilling other sellers' orders from your own pool is a Group-stage move for most merchants; the Omnichannel-stage version is fulfilling your own marketplace orders from one pool with one promise.
- A computed delivery promise becomes necessary with a second stock location or a cut-off that varies by day; a static promise serves one warehouse on ordinary days and fails on peak days.

The author's synthesis.

- The useful Amazon is the narrow one. I would strike "Amazon did microservices" from every design review and replace it with the four documented rules: the contract is the only door, the cart never says no, operability decides adoption, and the promise is the product. Those fit a fourteen-person team; the service count does not.
- The most important Amazon decision for a merchant is not architectural: Amazon opened its *operations* to others and let the storefront follow, which is why a merchant's own warehouse, promise and returns handling are the capabilities worth owning well (Pick, Pack, Ship: Warehouse Management and Fulfillment).

Questions to Ask

1. Which of our integrations read another system's data directly instead of through a documented contract, and who owns each one?
2. If our cart store were unavailable for a minute on Black Friday, would the customer see an error or a slightly stale cart? Which do we want?

3. Where is consistency non-negotiable for us (stock, payment, the order record), and does each of those systems have one writer?
4. Could our team run the systems we are proposing to build, on the busiest night of the year, with the people we have?
5. Which stock do we own but not control (a marketplace program, a 3PL, a store), and is it counted as a separate location in our availability calculation?
6. What is our delivery promise, which system computes it, and what does it know about warehouse capacity and carrier cut-offs today?
7. For which small set of SKUs could we make a faster promise and keep it, instead of one promise for everything?
8. What is our policy for automated buyers and agents on our own site, and do we have an authenticated channel for the ones we want?

Amazon's record ends where its publications end, and the next case has a fuller one. Zalando, the European fashion platform on which Kestrel & Pine also sells, has published its architecture decisions on its own engineering blog for ten years, including the ones it later reversed. Continue with Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.

Sources and Evidence

Company, product and investor sources

- Amazon Architecture — High Scalability, 2007 (18 September 2007), digest of "A Conversation with Werner Vogels", *ACM Queue*, 2006; the two-tier monolith, the 2001 scaling wall, 100–150 services per page, "you build it, you run it". Primary text not fetched; evidence age 19 years.
- Dynamo: Amazon's Highly Available Key-value Store — DeCandia et al., SOSP, 2007 (October); the service-oriented architecture statement, the 99.9th-percentile rule, the always-writeable cart, the merge rule, the 2006–2007 cart volumes.
- Amazon's Dynamo — Werner Vogels, 2007 (2 October); Dynamo was internal and not exposed as a web service.
- Amazon DynamoDB: a fast and scalable NoSQL database service — Werner Vogels, 2012 (18 January); why Dynamo did not spread and teams preferred managed services.
- Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service — Elhemali et al., USENIX ATC, 2022 (July); 89.2 million requests per second during the 66-hour Prime Day 2021 event.
- 10 Lessons from 10 Years of Amazon Web Services — Werner Vogels, 2016 (11 March); the API lesson learned "from our experiences with Amazon retail".
- Fulfilling — Werner Vogels, 2006 (19 September); opening services and the fulfillment network to others.

- Amazon launches Fulfillment by Amazon and WebStore by Amazon — Amazon press release, 2006 (19 September); the FBA launch and Prime eligibility for FBA items.
- Two-Pizza Teams — AWS whitepaper, n.d., accessed 2026-11; the two-pizza rule and end-to-end ownership in Amazon's words.
- Stevey's Google Platforms Rant — Steve Yegge, 2011 (12 October); the only public first-hand recollection of the "API mandate"; not confirmed by Amazon.
- Inside the Mind of Jeff Bezos — Fast Company, 2004; usual dating of the two-pizza phrase; not fetched.
- 2018 Letter to Shareholders — Amazon / Jeff Bezos, 2019 (April); third-party share series 1999–2018 and the credit to FBA and Prime.
- 2024 Letter to Shareholders — Amazon / Andy Jassy, 2025 (April); regionalization, 1 million to 300 million Prime items, delivery reach.
- 2025 Letter to Shareholders — Amazon / Andy Jassy, 2026 (9 April); robots, 85+ same-day centers with 90,000 SKUs, rural network, Amazon Now.
- Form 10-K for fiscal year 2025 — Amazon.com, Inc., SEC EDGAR, 2026 (filed 6 February); the seller-of-record statement, fee types, net sales, fulfillment expense, employees, store counts, floor area, Prime as a marketing tool.
- 2022 Small Business Empowerment Report — Amazon, 2023 (May); "more than 60% of sales" from independent sellers.
- 2024 Small Business Empowerment Report — Amazon, 2025 (May); 80 billion FBA items since 2006, Multi-Channel Fulfillment sellers, sellers over $1 million.
- Amazon Steers Third-Party Seller Share To All-Time High — Marketplace Pulse, 2025; independent tracking of the unit share at 60–62%.
- Amazon Sets New Prime Delivery Speed Record in 2025 — syndicated coverage of Amazon's 3 February 2026 release, 2026; 13 billion same- or next-day items; primary release not fetched.
- Buy with Prime — Amazon, n.d., accessed 2026-11; scope of the service, integrations and the 167-merchant test.
- Amazon announces Rufus — Amazon, 2024 (page updated 2026 with the rename to Alexa for Shopping); launch scope and training data.
- Meet Alexa for Shopping and How Amazon is using generative and agentic AI to transform shopping — Amazon, 2026, accessed 2026-11; usage figures, auto-buy, reorders, Buy for Me catalog growth.
- Buy for Me button on Amazon Shopping app — Amazon, 2025, accessed 2026-11; how the agent purchase works and what the brand keeps.
- Amazon tests 'Buy for Me' agentic AI shopping tool — Digital Commerce 360 / Beth Duckett, 2025 (8 April); launch date and the no-commission note.
- Amazon introduces feeds for Shop Direct — Amazon, 2026, accessed 2026-11; 100 million products, 400,000 merchants, feed partners.
- Statement about Perplexity — Amazon, 2025 (letter dated 31 October); Amazon's rule for third-party agents.

- Amazon sues Perplexity over AI shopping agents — Retail Dive, 2025 (6 November); the lawsuit and Perplexity's response.

Standards, regulation and official documentation

- None cited directly; the seller-of-record obligations referred to in the text are treated in Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation and Tax, Invoices and the Books: From Order to Ledger.

Research and practitioner sources

- Prime Video Switches from Serverless to EC2 and ECS to Reduce Costs by 90% — InfoQ / Rafał Gancarz, 2023 (3 May); summary of the March 2023 Prime Video post, whose original URL now redirects.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

48. Case: Zalando, Microservices, Platform Strategy and the Return to Discipline

IN THIS CHAPTER, YOU WILL: Follow Zalando's own dated accounts from the 2015 microservices move through the corrections that followed, and take from them what a merchant can use about modularity, API contracts and becoming a platform.

KEY POINTS:

- **Zalando split its shop for team autonomy, not for load, and said so.** In 2015 a shared codebase and slow deployments blocked 85-plus teams; the fix was organizational first (small teams, an AWS account each, a paved road called STUPS) and architectural second. The service count followed the team count.
- **The durable export is a written API standard, not the service count.** Zalando's guidelines, open since 2016 and still maintained in 2026, say what a merchant should demand of every vendor: extend compatibly

instead of versioning, never break consumers, and sunset an API only with client consent, a date, headers and usage monitoring.
- **Autonomy was followed by a decade of discipline**. Alignment over raw autonomy (2018), fewer languages (2020), one developer portal (2023) and, in 2025, a synchronous read API replacing a price-and-stock event stream that fell 30 minutes behind at Cyber Week and bred divergent copies of product data. Events propagate change; they are not the system of record.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Giant's Boots** — Zalando's microservices move is copied for its shape; the record shows it was made for 85-plus teams and corrected for a decade afterward.
- **The Pulled Rug** — Zalando's deprecation rules (consent, a sunset date, headers, usage monitoring) are the written cure for the vendor API change that breaks an integration before anyone is told.
- **The Message in a Bottle** — Zalando's 2025 post documents what happens when consumers rebuild their own copy of price and stock from a stream they cannot keep up with: delays and divergent copies.

Zalando is a fashion platform headquartered in Berlin: in 2025 it reported **€17.6 billion** in gross merchandise volume, **62 million** active customers and **278.6 million** orders (FY2025 fact sheet, March 2026). Kestrel & Pine, the fictional outdoor brand this book follows, sells on it in Germany and the Netherlands through its marketplace connector. To a merchant, Zalando is a channel, a possible fulfillment provider, and a company that publishes its API rules.

The reason Zalando is a case in this book is that last point. From 2015 its engineering blog described, in its own words and in near-real time, the rebuilding of its shop as **microservices** owned by **autonomous teams**, the platform tooling it built and open-sourced, the API standard it wrote, and then, over ten years, the corrections: alignment, fewer technologies, one developer portal, and in 2025 a reversal of one integration from events to an API. It is the fullest public record of a European retailer's architecture, including the parts that did not work.

Read the ages of the evidence. The 2015 to 2017 posts are history; the systems they describe have been replaced. The business figures are from 2025 and 2026 filings. The AI material is from 2023 to 2026 and moves quickly. As in Case: Amazon, From Monolith to Services to the Everything Store, every claim is dated in the sentence, inferences about Zalando's internals are labeled, and a section near the end says where the evidence stops. Zalando never announced the end of "Radical Agility"; what this chapter calls the return to discipline is a sequence of its own posts, not a program it declared.

Where we are in the order's journey. This case sits beside the order's path. It bears on three steps of Eva's jacket: the marketplace order that enters Kestrel & Pine's systems

from a platform like this one (Selling on Someone Else's Platform: Marketplaces and Channel Integration), the price and stock that must be correct on the listing when Eva's neighbor buys the same jacket there (Is It in Stock? Inventory, Availability and the Promise You Can Keep), and the API contract the connector depends on (Promises Between Systems: API Contracts, Webhooks and Versioning).

2015: Jimmy, Radical Agility and the Split

Vocabulary first. A **monolith** is one deployable application holding all of the shop's logic with one shared codebase and usually one database. **Microservices** are the opposite: many small, separately deployed services, each owned by one team and talking to the others over the network. An **autonomous team** chooses its own tools and process and owns its services in production. In plain words: one big program everyone edits, versus many small programs each with an owner.

Zalando's monolith was called **Jimmy**. The October 2015 post "From Jimmy to Microservices" says that "all of the Shop teams shared the same code base" and that teams "couldn't move quickly because of Jimmy's slow startup time and our overly complicated deployment processes". Note the stated reason: not load, but **teams blocked by a shared codebase and slow deployments**. In May 2015 a task force called Spearheads, seven engineers and a designer, began rebuilding the Fashion Store, then 15 country sites serving over 16 million active customers, as microservices owned by teams responsible for "programming, quality assurance, deployment, and operations". The team chose to put a router in front of the shop, a Go program called **Skipper**, which it open-sourced.

The organizational side was called **Radical Agility**, introduced in March 2015 and built, by trade accounts of 2016, on autonomy, mastery and purpose, with teams of three to seven people owning a topic end to end. Zalando's own July 2015 post has Eric Bowman, then VP Engineering, at the AWS Summit in Berlin: "We think of teams as a unit of trust"; "we've tried to create the context where people can make mistakes and it's not fatal". By February 2016 a Zalando talk at the MicroXchange conference covered microservices across "our 85+ engineering teams"; by December 2016 InfoQ reported "about 110 teams", more than 1,400 people in Tech across six hubs, and "each team has its own AWS account".

That last fact needed a platform. **STUPS** was Zalando's answer: an **internal platform-as-a-service**, meaning one supported way for every team to deploy to the cloud. Development began in late 2014 and it was available for production to every team in May 2015. InfoQ's April 2016 account describes it as "a convenient and audit-compliant Platform-as-a-Service" on AWS: "Our employees get nearly administrative access to their AWS accounts with almost no restrictions", with near-real-time audit reporting because "Auditors need to know that we're complying with the regulations" (card-

industry, financial-reporting and German rules). In plain words: Zalando gave every team its own cloud account and built the tooling that made that auditable.

The shop itself was rebuilt as **fragments**, pieces of a page owned by different teams and deployed separately, assembled behind the router. One year in, in October 2016, the effort was renamed Project Mosaic: header and footer fragments were live, product-page and wish-list fragments were going live, and the author described a shift "from prototyping to performance optimization" and toward "stable components that other teams can depend on".

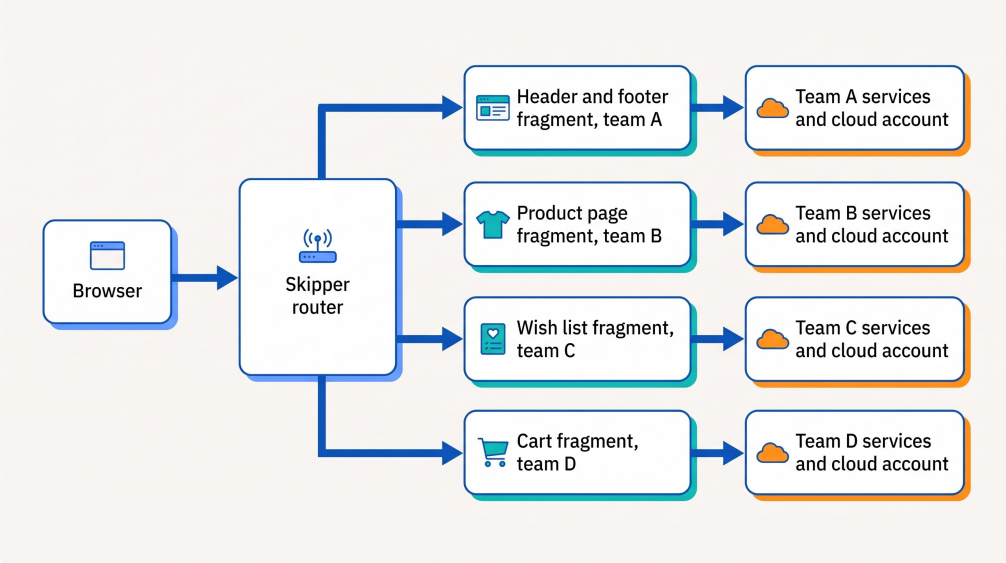


Figure 1: *The 2015–2016 shape as Zalando described it: a router in front, a page assembled from fragments each owned and deployed by one team, and each team on its own cloud account behind a shared, audited platform.*

A smaller post from April 2016 is the most honest about the first months: a team in Dortmund cut its domain into two services, then into eight after modeling the boundaries properly ("Define your Microservices properly and know your boundaries"), and replaced a failed upfront plan with weekly goals.

> **INFERENCE, NOT DOCUMENTED:** Zalando's documented sequence mirrors Amazon's with a thirteen-year lag and a different stated motive. Amazon's 2006 account names a scaling wall; Zalando's 2015 posts name team autonomy. The fix was organizational first (teams, accounts, a paved road) and architectural second, which is why the service count tracked the team count: 85-plus teams in early 2016, about 110 by December, and by 2020 "4,000+ applications" for "1,200+ developers". A merchant with fourteen people in technology, like Kestrel & Pine, is below the team count at which

any of this applies. What transfers is the *order of operations*: decide who owns what before deciding how to split what, which is the argument of Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record and Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.

The Cost of Distributed Data, Stated Early

The clearest early statement of what the split would cost came not from the shop teams but from business intelligence. In September 2015 a Zalando post observed that the monolith's "consolidation of data in a single place makes business intelligence fairly simple", whereas "with each service owning its data, data is spread over a significantly larger number of cloud-based microservices"; article and order data alone were spread across eight PostgreSQL databases. Zalando's response was to deploy Kafka, a message log, on AWS "to make our event data available for downstream systems", and to plan a data lake.

Two terms. An **event** is a message sent the moment something happens (an order placed, a price changed) so other systems can react. An **event bus** is the shared channel on which services publish events and others subscribe. Zalando's event bus was **Nakadi**, started in October 2015: in its own README, "a distributed event bus that implements a RESTful API abstraction on top of Kafka-like queues", with a schema registry so that producers register an event type with a **schema** (the agreed shape of the message) before publishing, and every event is validated against it. By May 2017 a Zalando post on "third-generation microservices" described the intended style: services "process incoming events and data from other services", and data streaming becomes "a central part of the service fabric".

INFERENCE, NOT DOCUMENTED: This is the earliest honest statement in the record of the rule Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation is built on: once each system owns its own data, there is no single place to query, so analytics and reconciliation need a stream of what happened. Kestrel & Pine meets this problem without a single microservice: its thirteen systems hold orders in four places and product data in four, and its data warehouse has three definitions of an order (Waypoint decision W7 picks one). An Omnichannel merchant meets the *data* problem of distribution long before the *team* problem, and should take the event-stream lesson without the service count.

The API Guild and the Guidelines

The piece of Zalando's 2015 work that is still alive, still public and still maintained in 2026 is its API standard. It began as the **API Guild** in March 2015: an informal group of about

20 people who set standards and review APIs as "a sounding board and review resource, not an approval board, to avoid bottlenecks", spending about 10% of their week on it. The September 2015 post explains why it mattered to the business: Zalando's "open platform strategy relies on public APIs used by external partners", so "We cannot afford to break our APIs", and "Ideally, all Zalando APIs should look like the same author created them".

The principle is **API First**: the interface is designed and reviewed, as an OpenAPI document (a machine-readable description of an API), before any code is written. By December 2016 the guidelines were open-sourced; Zalando described API definitions as "a source truth, part of a contract between a service provider and consumers". The current guidelines (undated page, accessed November 2026) use the MUST, SHOULD and MAY of internet standards and say teams "must define APIs first, before coding its implementation, using OpenAPI as specification language" and should "Treat your API as product and act like a product owner".

The rules that matter most to a merchant are the ones about change. In plain words, a **compatible extension** adds to an API without breaking anyone who already uses it; **versioning** publishes a second, different API beside the first; **deprecation** announces that an API will stop; a **sunset** is the date it does.

Rule (Zalando guideline number)	What it says, in plain words	What Kestrel & Pine should demand of a vendor
#106 MUST not break backward compatibility	"Change APIs, but keep all consumers running"	Additive changes only inside a version
#107 SHOULD prefer compatible extensions; #113 SHOULD avoid versioning	Add fields rather than publishing a new version	A changelog of additions, not surprise versions
#108 MUST prepare clients to accept compatible extensions	Clients ignore fields they do not know (the **tolerant reader**)	Its own integrations must not fail on a new field
#114 MUST use media type versioning; #115 MUST not use URL versioning	When a version is unavoidable, put it in the content type, not the address	A stated versioning scheme, whichever it is
#185 MUST obtain approval of clients before API shut down; #186 MUST collect external partner consent on deprecation time span	No sunset until "all clients have given their consent on a sunset date"	A deprecation notice with a date agreed in advance

Rule (Zalando guideline number)	What it says, in plain words	What Kestrel & Pine should demand of a vendor
#187 MUST reflect deprecation in API specifications; #189 SHOULD add Deprecation and Sunset headers	`deprecated: true`, the sunset date and migration guidance in the spec; headers on every response	Machine-readable deprecation, so a script can warn before it breaks
#188 MUST monitor usage of deprecated API scheduled for sunset	The provider watches who is still calling	A warning to the merchant while it is still calling
#191 MUST not start using deprecated APIs	A client never builds on something already marked for removal	The agency checks before building

The companion to the guidelines is the **Tech Radar**, created in 2016 "to inspire and support teams to pick the best technologies for new projects". A Technologist Guild of about 25 senior people assigns technologies to rings by vote: ADOPT ("high confidence in to serve our purpose, also in large scale"), TRIAL, ASSESS and HOLD ("not recommended to be used for new projects"). In 2016 "Teams must use the Tech Radar as one input source for their technology decisions"; that wording changes in 2020.

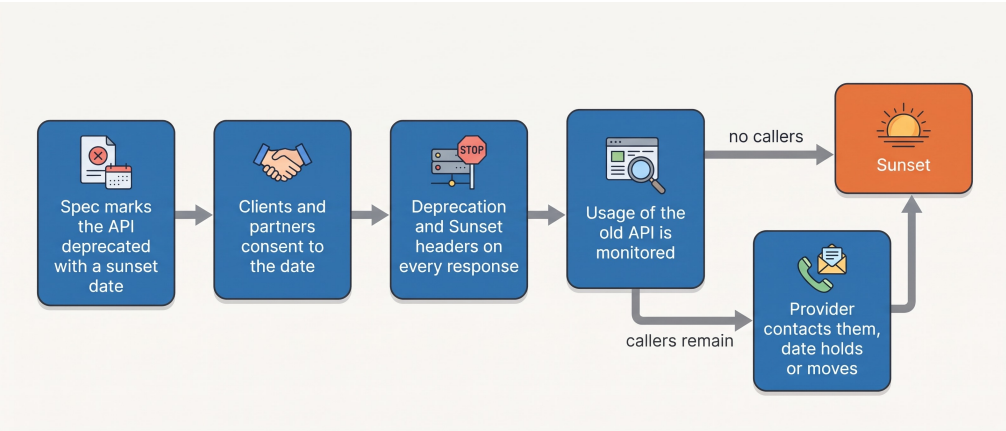


Figure 2: *Deprecation as Zalando's guidelines prescribe it: announce in the specification, obtain consent on a date, signal in headers, monitor who still calls, and only then sunset; every step is a chance to avoid the Pulled Rug.*

Kestrel & Pine learned the value of these rules the expensive way. In February 2027 its marketplace connector retired an API version on a published date; the agency script

that read it had not been updated, and stock to bol was stale for two days (the ledger's Deprecated Endpoint). Every row in the table above describes a step that would have caught it: a machine-readable deprecation, a header the script could have logged, usage monitoring that would have shown the script still calling. Promises Between Systems: API Contracts, Webhooks and Versioning turns the guidelines into the book's contract rules.

The Return to Discipline, 2018 to 2026

No single Zalando post declares that the autonomy era ended. The posts that follow narrow a choice each time, and read in order they tell one story.

Year	What Zalando published	What narrowed
2018	Bowman, by then leading a 500-person group, told InfoQ in September: "Our biggest challenge was that we treated autonomy like an input to teams"; unaligned teams were like "air molecules in a balloon: moving fast but not running together as a group"; autonomy became "an aspiration and a reward, not a gift"; "a set of principles and systems scales better than a set of prescriptions". Teams were embedded across the company with shared objectives and company-wide "Global Projects".	Autonomy without alignment
2018	The agile coaches: "we value principles more than frameworks"; six principles replace a single prescribed method (August).	One mandated process, replaced by principles

Year	What Zalando published	What narrowed
2020	With "1,200+ developers" and "4,000+ applications", Zalando "redefined the ring semantics" of the radar, wrote Technology Selection Principles, gave a Principal Engineering community the decision, required a sponsoring Engineering Head for TRIAL technologies, and stopped new applications in Clojure, Haskell and Rust in favor of Kotlin and TypeScript, because "A broad freedom of choice leads to increased difficulties in supporting software long-term" (July).	Free choice of language
2023	**Sunrise**, an internal developer portal built on Backstage since 2021, replaced "more than 100 disconnected interfaces" with one portal holding "over 40k registered entities (applications, teams, and users)" and 27 integrated tools; old tools were shut down (August).	Many tools, replaced by one paved road
2023	On 21 February the co-CEOs announced the removal of "several hundred overhead roles", about 5% of roughly 17,000 employees, sparing logistics, customer care and outlets (Retail Gazette).	Overhead
2025	The event-driven price and stock pipeline "became a bottleneck, introducing delays and scaling challenges"; it was replaced by a synchronous Product Read API (March).	A stream as the only source of current state

Year	What Zalando published	What narrowed
2026	Connected Retail, the ship-from-store program launched in 2018, is folded into the standard partner program; its technology "will be discontinued by June 30, 2027" (FashionUnited, April).	A separate channel technology

Three terms. A **paved road**, or internal developer platform, is one supported way to build, deploy and find services so teams do not each solve the same problem. A **technology radar with teeth** is a list of allowed technologies teams must follow rather than consult. A **synchronous read API** is one that answers "what is the price now" when asked, instead of making the caller rebuild its own copy from a stream of changes.

The evidence does not support "microservices failed at Zalando". It supports something more specific: **autonomy produced too many technologies**, **too many tools and**, **on one critical path**, **an integration that could not keep up**, and each was fixed by narrowing a choice. Zalando today reports more than 17,000 employees, around 3,000 in technology and more than 7,000 in fulfillment centers (annual report 2025, March 2026).

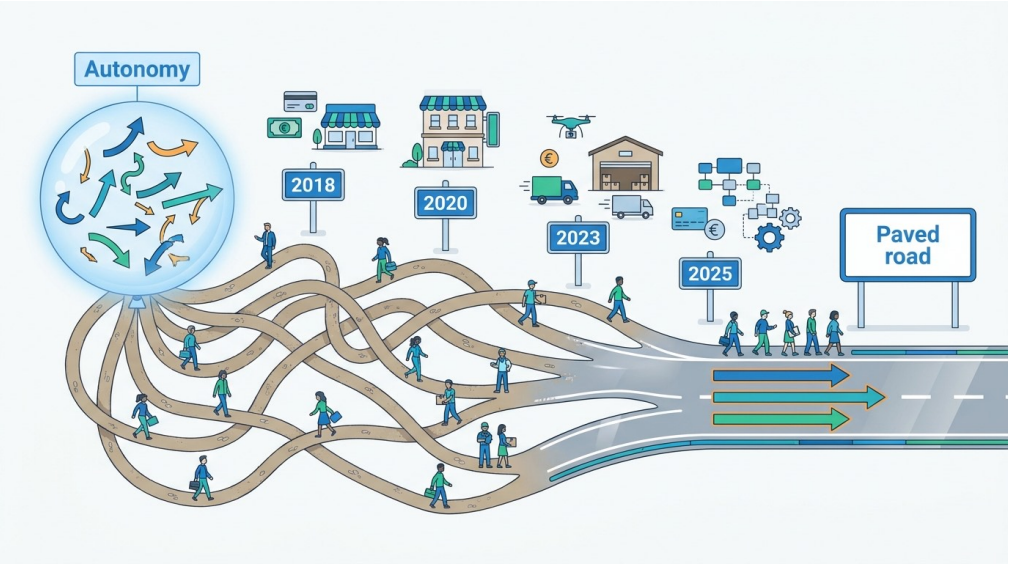


Figure 3: *Zalando's return to discipline narrowed one choice at a time: alignment in 2018, fewer languages in 2020, one developer portal in 2023 and a read API in 2025.*

2025: When the Event Stream Fell Behind

The 2025 post deserves its own section because it is the most useful single document in the case for a merchant, and because it is the Black Friday Oversell told by a platform.

Zalando's price and stock updates were published as events. Consumers, the systems that needed current prices and stock, subscribed to the stream and maintained their own copies. The March 2025 post "From Event-Driven to API" reports what happened at scale: "over 90% of each payload unchanged" was being reprocessed, because an event carried the whole record even when one field had changed; during Cyber Week events "could be delayed by up to 30 minutes"; and teams that could not consume the streams fast enough used other sources, producing "divergent versions of product data". The pipeline was replaced by a synchronous, low-latency **Product Read API** (PRAPI): callers ask for the current price and stock and get an answer now.

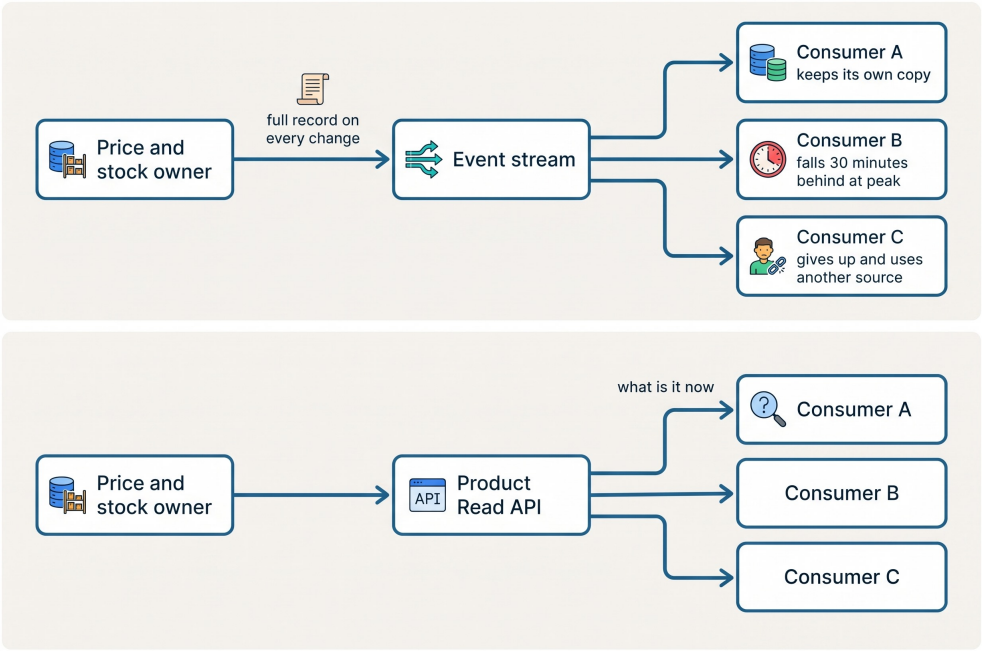


Figure 4: *Before 2025, every consumer rebuilt its own copy of price and stock from a stream of full records and the copies diverged under load; after, consumers ask the owner for the current value, and the stream is no longer the only way to know the truth.*

In plain words: an event is the right way to tell everyone that something changed. It is the wrong way to be the only place the current value can be learned, because then every consumer is a copy, every copy can lag, and on the busiest day the lag is largest. Kestrel & Pine's stock sync chain, WMS to ERP every 15 minutes, ERP to platform hourly,

platform to marketplace connector hourly, is a slower version of the same design: a chain of copies, each one stale by the time it is read, which is how the Ridgewalker kept selling for two hours after it sold out at 19:40 on 27 November 2026.

Figure 5: *Kestrel & Pine's stock sync is a chain of copies, WMS to ERP every 15 minutes, then hourly to the platform and hourly to the connector, which is how the Ridgewalker kept selling for two hours after it sold out.*

> **INFERENCE, NOT DOCUMENTED**: Zalando does not describe PRAPI's internals, its caching or how events and the API now coexist, so the diagram shows the documented before and after, not the implementation. The rule I draw is the one Waypoint decision W1 encodes: keep a **system of record** for stock and price, an **availability service** that answers "can I promise this unit on this channel now" through an API, and use events to propagate change and to feed reconciliation, not as the only source of truth. Events for "what happened"; a read API for "what is it now"; a reconciliation job for "do the copies still agree" (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation). The 30-minute figure is Zalando's; the two-hour figure is Kestrel & Pine's; the shape of the failure is the same.

The Platform Pieces: What Survived

Zalando open-sourced much of what it built in 2015, and the status of those projects in November 2026 is itself evidence.

Project	What it is, in Zalando's words	Status on 7 November 2026
STUPS (for example Senza)	audit-compliant platform-as-a-service on AWS for autonomous teams	Senza archived; last change March 2023
Skipper	"An HTTP router and reverse proxy for service composition"; used "as shop frontend http router with 350000 routes" and as the ingress controller "in more than 100 production clusters"	Active; changed the day before the check
Patroni	"a Python template for building PostgreSQL high availability (HA) clusters", introduced on Zalando's blog in February 2016, now under its own organization	Active
Nakadi	the event bus described above	Archived June 2024: "no longer developed in the open as we've not seen usage outside of Zalando for many years"
RESTful API and Event Guidelines	"A model set of guidelines for RESTful APIs and Events, created by Zalando"	Active; changed September 2026
Tech Radar	the radar page and its generator	Active; changed October 2026
Zalenium	container-based test grid	Archived September 2021: "no new maintainers were found"

INFERENCE, NOT DOCUMENTED: The pieces that survived are the ones with a life outside Zalando: a router, a database failover template and a written standard. The pieces tied to the 2015 account-per-team model (STUPS) or used only internally (Nakadi) were archived. Read the table as a warning against building platform tooling only one company needs.

Archiving Nakadi says nothing about whether Zalando still uses an event bus internally; the README says only that nobody outside did.

A Retailer Becoming a Platform

Zalando's business story runs parallel to its architecture story, and the two meet at the warehouse.

Three terms. In the **partner program**, a brand sells on Zalando as the seller of record (the legal seller who issues the invoice and owes the consumer rights) and Zalando takes a commission, as opposed to Zalando's **wholesale** model in which it buys the stock and sells it itself. **Ship-from-store** sells a physical store's shelf stock online and ships it from the store. **Logistics as a service** offers a retailer's warehouses, carriers and software to other sellers, for orders on and off its own platform.

Year	Move	Documented figures
2011	**Partner Program** launches: brands sell as seller of record through the zDirect portal	June 2021: "more than 600 partners, representing over 2,000 brands"; about a quarter of GMV; a target of 40% by 2023 and half in the long term (targets now dated)
2018	**Connected Retail** launches: stores' stock sold on Zalando, collected by carriers (DHL in Germany, PostNL in the Netherlands), returns back to the store	March 2020: "more than 1,500" stores in Germany and the Netherlands, commissions waived for April–May 2020; January 2021: "more than 2,600"; February 2021: "over 3,000 active stores" in nine markets
2010s–2021	**Zalando Fulfillment Solutions** (ZFS) and Shipping Solutions: Zalando's warehouses and carriers for partners	June 2021: access to 16 European markets, "more than 2,300 market expansions" in 18 months

Year	Move	Documented figures
2022–2023	Multi-channel fulfillment pilot (2022), rebranded **ZEOS** Fulfillment in October 2023: one stock pool for Zalando, own-shop and other-platform orders, open to sellers "whether or not they sell through Zalando"	End of 2025: 25 markets, "over 600 partners selling across 18 sales channels (including Zalando), covering around 85% of the European marketplace volume" (annual report)
2025	**ABOUT YOU** acquired: intention announced 11 December 2024, completed 11 July 2025 at 91.45%, squeeze-out registered 6 November 2025 at €6.50 a share; B2C runs two brands on shared logistics and payment infrastructure	Targeted group synergies "around €100 million per year"
2025	B2B segment defined as ZEOS Fulfillment (including ZFS and multi-channel fulfillment), the commerce platform **SCAYLE** and the integration product **Tradebyte**	FY2025 B2B revenue €1,091.3 million (+14.6%), adjusted EBIT €53.9 million; 330 online shops on SCAYLE; a ZEOS app for Shopify's European merchants
2026	Connected Retail folded into the partner program; same standard fees for all partners from September 2026; technology discontinued by 30 June 2027	"We are not saying goodbye to brick and mortar stores; we are simply updating the technology they use to connect with us"

The FY2025 fact sheet (12 March 2026) gives the scale around this: GMV €17,560.2 million (+14.7%), revenue €12,346.1 million (+16.8%), adjusted EBIT €590.7 million, 278.6 million orders at 4.8 orders per customer and an average basket of €62.8. Reported growth includes ABOUT YOU; Zalando's guidance for 2026 is pro-forma growth of 5 to 10%, the honest measure of the core business. The partner share of group GMV fell to 32.1% from 34.2% because ABOUT YOU "currently operates with a lower partner business share". Zalando's December 2025 strategy note counts "over 60" live merchants on its fulfillment service; the annual report's 600 partners span all B2B services.

> **INFERENCE, NOT DOCUMENTED:** Zalando's platform story is Amazon's in European form and on a European timetable: marketplace (2011), then ship-from-store (2018), then fulfillment for partners, then fulfillment for

anyone on any channel (2023), then the shop software itself (SCAYLE, in the group from 2025). The architectural consequence is the one Case: Amazon, From Monolith to Services to the Everything Store drew: **one stock pool serving several seller-of-record models and several demand channels**, which is the Omnichannel-stage problem (own site, stores, wholesale, marketplaces) at a much larger scale. Two cautions are in the record. Connected Retail shows that a channel built on its own technology gets consolidated once the general program can do the job. And the partner share falling after the ABOUT YOU merger shows that an acquisition imports the acquired company's model mix, not only its revenue (What Changes at Each Scale: From First Store to Group).

AI at Zalando: Discovery First, Then Identity

On 19 April 2023 Zalando announced a beta "fashion assistant powered by ChatGPT" for selected customers in four countries, letting them describe what they want in their own words and refine the results in conversation. The annual report for 2025 says the Zalando Assistant "was used by over 6 million customers" that year, and that Zalando sees "the emerging channel of agentic commerce as a great opportunity" while noting "the rise of AI agents as digital gatekeepers".

The engineering side came in September 2026. Zalando described an internal agentic platform on Kubernetes and open-sourced an **Agentic Identity Broker** that "lets agents act on behalf of users without ever holding provider tokens". Two terms: **delegated access** means a user allows an agent to act with a subset of the user's permissions ("Delegated access is the intersection of both the user and agent permissions"), and **token exchange** means the broker swaps the agent's identity for a short-lived credential scoped to one action, so the agent never holds the user's real credentials. The post adds that consent "should name a business capability … instead of presenting a raw scope string", and that stronger approval is planned for high-risk transactions because users rubber-stamp approvals.

INFERENCE, NOT DOCUMENTED: The 2023 assistant is a search feature: finding products in natural language. The 2026 broker is the infrastructure answer to the question Amazon answered with a policy and a lawsuit: who is this agent, for whom does it act, and what may it do. Read the two cases together. Amazon states the rule for *external* agents; Zalando publishes the identity model for *internal* ones. For Kestrel & Pine the broker is a preview of what Waypoint decision W8 will require before any agent may place or change an order: an identity for the agent, a named person it acts for, and a scope a human can read. Assistants and Agents: AI in Commerce and What Is Real in 2026 owns the subject; the identity model is also treated in the sibling book at Agent Identity, Delegation and Least Privilege.

What Zalando Teaches

The table is the author's synthesis; each row names its evidence.

Lesson	Evidence	Transfers to a merchant?
Decide ownership before decomposition; the service count follows the team count	2015 posts; 110 teams (2016); 4,000+ applications (2020)	The order of operations, yes; the count, only at the Group stage
A written API standard outlives the architecture it was written for	Guild (2015); guidelines active in 2026	Yes, as what to demand of vendors first and what to write for internal APIs later
Deprecate only with consent, a date, headers and monitoring	Guidelines #185–#191	Yes; it is the cure for the Pulled Rug
Distributed data needs an event stream for analytics and reconciliation	2015 BI post; Nakadi	Yes, at the Omnichannel stage, without the service count
An event stream must not be the only source of current state	2025 Product Read API post	Yes; a read API on the system of record, events for propagation
Autonomy needs alignment, a bounded technology set and one paved road	2018, 2020, 2023 posts	At the Group stage; a small team gets this for free from one platform
Build platform tooling only one company needs and it will be archived	STUPS, Nakadi, Zalenium status	Yes, at every stage
A retailer becomes a platform through logistics, then software; consolidation follows	2011–2026 timeline	The pool-serving-many-channels idea, yes; the business, only at scale

What Kestrel & Pine takes from the case. Two things, both already in Waypoint. From the guidelines: the integration standards of W5 should require of every vendor what Zalando requires of itself, a dated deprecation that a script can read, and the agency should check the Sunset header before it builds, so that the Deprecated Endpoint of

February 2027 is the last of its kind. From the 2025 post: the availability service of W1 answers "what is available now" through an API, with stock events feeding it and a weekly reconciliation behind it, so that the two-hour stock lag of Black Friday 2026 is not rebuilt as a faster chain of copies. What Kestrel & Pine does not take is the shape: 110 teams and 4,000 applications are the Giant's Boots for a team of fourteen.

Where the Evidence Stops

- **The current architecture**. Zalando has not published its service inventory, its order or pricing architecture, the internals of ZEOS or how PRAPI and its event streams coexist today. The 2015–2017 posts describe systems that have been replaced.
- **"Radical Agility" ended** nowhere in the public record. No post announces it, and no source describes a 2019 reorganization under that name; the return to discipline is this chapter's reading of a sequence of posts and one leadership change (Bowman left in 2019, by professional-profile sources). The widely reported "expanded too much" wording from the February 2023 announcement was not in the page fetched. The 2016 trade accounts of team size (three to seven) are secondary; Zalando's own posts give "85+" teams.
- **Open-source status** is as of 7 November 2026 and will change; the Tech Radar page shows no last-updated date.
- **Business figures** for 2025 include ABOUT YOU for part of the year; the 2026 guidance of 5–10% pro-forma growth is the comparable number. The "over 60" fulfillment merchants and the "over 600" B2B partners are read here as different scopes; Zalando has not reconciled them in public.
- **The Connected Retail wind-down** and the September 2026 fee change are reported by trade press from Zalando's partner communication; the primary page was not fetched.

What to Take Away

Established.

- Zalando rebuilt its shop as microservices owned by autonomous teams from 2015 because a shared codebase and slow deployments blocked its 85-plus teams, and it built an audited platform (STUPS) so each team could have its own cloud account (Zalando posts, 2015–2016; InfoQ, 2016).
- Its API guidelines, open since 2016 and maintained in 2026, require API-first design in OpenAPI, forbid breaking consumers, prefer compatible extension to versioning, and require client consent, a sunset date, headers and usage monitoring before shutdown.

- Zalando narrowed its choices over the following decade, each in its own posts: alignment over raw autonomy (2018), a bounded language set (2020), one developer portal (2023) and a read API replacing a price-and-stock event stream that lagged up to 30 minutes at Cyber Week (2025).
- Zalando became a platform through logistics: partner program (2011), ship-from-store (2018), fulfillment for partners and then for any channel (ZEOS, 2023), commerce software (SCAYLE, 2025); 32.1% of its 2025 GMV came from partners (annual report, 2026).

Depends on context.

- Team-sized modularity is right when there are enough teams to need it, roughly the Group stage; below that, one platform with named owners for each capability gives the ownership without the operating cost.
- An event bus is necessary once data is spread across systems that must be analyzed and reconciled, which for a merchant is the Omnichannel stage; it is sufficient as a source of current state only while consumers can keep up, which the 2025 post shows they cannot on the busiest week.
- The compatibility and deprecation rules apply at every stage, but in different directions: a Starting or Growing merchant demands them of vendors; an Omnichannel merchant writes them into its own integration contracts; a Group-stage platform owes them to external partners.
- Building a separate technology for one channel (as Connected Retail was) is right while the general program cannot serve it, and becomes a consolidation project once it can; plan the exit when you build it.

The author's synthesis.

- Zalando's most valuable export is not an architecture but a habit: writing down the rules for change, and publishing the corrections. I would adopt the compatibility and deprecation chapters of its guidelines as the baseline for every vendor contract a merchant signs.
- The 2025 post is the book's thesis in a platform's own words: a copy is not a system of record, and the busiest day is when the copies drift furthest. Give every fact an owner with a read API, and let events carry the news.

Questions to Ask

1. Which of our vendors publish a deprecation policy with a sunset date, consent and headers, and which retire versions on a date they choose alone?
2. Do our own scripts and connectors log Deprecation and Sunset headers, and would anyone see the warning?

3. For price and stock, which system can be asked "what is it now", and which systems only hold a copy rebuilt from events?
4. How far behind are our copies on an ordinary day, and how far on the peak hour? Has anyone measured it?
5. If we tolerate unknown fields in every payload we receive, would a vendor adding a field break anything of ours?
6. Do we have an API standard our own team and our agency follow, even a short one, and who reviews against it?
7. Which channel runs on technology built only for that channel, and what is the exit if the general program can serve it later?
8. Before an agent may act on an order, can we name the agent, the person it acts for and the capability it was granted, in words a human can read?

The two cases so far are large companies that wrote their own commerce software. The next asks what happened when vendors tried to sell that shape to everyone else: the composable movement, its promise, its costs and its 2025–2026 turn toward working beside the systems merchants already have. Continue with Case: commercetools, Shopware and the Composable Movement.

Sources and Evidence

Company, product and investor sources

- Radical Agility on AWS (video post) — Zalando Engineering / Lauri Apple, 2015 (7 July); Bowman's "unit of trust" and STUPS as the platform for autonomous teams.
- On APIs and the Zalando API Guild — Zalando Engineering / Frauenstein and Schmeisky, 2015 (3 September); the guild's role, API First, "We cannot afford to break our APIs".
- Data Integration in a World of Microservices — Zalando Engineering / Olaf Melchior, 2015 (21 September); the cost of distributed data and the Kafka deployment.
- From Jimmy to Microservices: Rebuilding Zalando's Fashion Store — Zalando Engineering / Dan Persa, 2015 (16 October); the monolith's name, the stated reasons, Spearheads, Skipper.
- From Monolith to Microservices (video) — Zalando Engineering, 2016 (10 February); "85+ engineering teams".
- Four Lessons with Microservices — Zalando Engineering / Malte Pickhan, 2016 (27 April); two services became eight; weekly goals.
- Zalando Tech Radar — Zalando Engineering / Thomas Frauenstein, 2016 (17 May); the radar's purpose and "one input source".
- Jimmy to Microservices: The Journey One Year Later — Zalando Engineering / Dan Persa, 2016 (5 October); Project Mosaic, fragments live, the shift to stable components.

- Platform Engineering and Third Generation Microservices in Dublin — Zalando Engineering / Bill de hÓra, 2017 (26 May); the event-driven intent.
- Agile: Principles over Frameworks — Zalando Engineering / Samir Keck, 2018 (2 August); principles replacing a prescribed method.
- Technology Choices at Zalando: Tech Radar Update — Zalando Engineering / Bartosz Ocytko, 2020 (15 July); 1,200+ developers, 4,000+ applications, the narrowed language set.
- Sunrise: Zalando's developer platform based on Backstage — Zalando Engineering / Nagel, Reis Puthin, Ocytko, 2023 (3 August); 100+ interfaces replaced, 40k entities.
- From Event-Driven to API — Zalando Engineering / Gallagher and Carta, 2025 (7 March); the 30-minute delays, 90% unchanged payloads, divergent copies, the Product Read API.
- Agentic Platform: Open-Sourcing the Agentic Identity Broker — Zalando Engineering / Brennenstuhl and Jungsbluth, 2026 (25 September); delegated access and token exchange.
- Zalando RESTful API and Event Guidelines and the compatibility and deprecation chapters — Zalando, n.d., accessed 2026-11; rules #106–#115 and #185–#191.
- Zalando Tech Radar — Zalando, n.d., accessed 2026-11; ring definitions.
- STUPS, Skipper, Patroni, Nakadi, Zalenium — Zalando and the Patroni project, GitHub, n.d., accessed 2026-11 (status from the GitHub API on 7 November 2026); project descriptions and archive notices.
- A Decade of Changing the Game for Partners — Zalando, 2021 (24 June); Partner Program figures and targets, ZFS reach.
- Zalando's Connected Retail Program Offers Support to Physical Stores — Zalando, 2020 (31 March); 1,500 stores, carriers, commission waiver.
- Stronger Together: Connecting Online and Offline Retail — Zalando, 2021 (25 January); 2,600 stores, 2018 launch year.
- Zalando Onboards C&A to Connected Retail and Expands Program to Austria — Zalando, 2021 (15 February); 3,000 stores in nine markets.
- Zalando to launch a fashion assistant powered by ChatGPT — Zalando, 2023 (19 April); the assistant beta.
- Zalando and ABOUT YOU successfully complete transaction — Zalando, 2025 (11 July); completion, 91.45%, shared infrastructure, synergies.
- ABOUT YOU and Zalando complete final step of their strategic combination — ABOUT YOU, 2025 (6 November); the squeeze-out and delisting.
- Zalando's strategy in action: the evolution of our B2B growth engine — Zalando, 2025 (10 December); ZEOS components and "over 60" live merchants.
- Zalando SE FY/25 Fact Sheet — Zalando, 2026 (12 March); GMV, revenue, customers, orders, B2B revenue, 2026 guidance.
- Zalando SE Annual Report 2025 — Zalando, 2026 (March); ZEOS reach, SCAYLE shops, partner share, headcount, the assistant's users, agentic outlook.
- ZEOS — Zalando, n.d., accessed 2026-11; the modular service positioning.

Standards, regulation and official documentation

- The guidelines' MUST/SHOULD/MAY follow RFC 2119 — IETF, 1997; the deprecation headers follow the Sunset header convention the guidelines cite. Contract rules as the book uses them are in Promises Between Systems: API Contracts, Webhooks and Versioning.

Research and practitioner sources

- Zalando's STUPS: an open-source PaaS on AWS — InfoQ / Daniel Bryant, 2016 (4 April); STUPS scope and audit model.
- Zalando: APIs, delivery and Radical Agility — InfoQ / Jerome Louvel, 2016 (24 December); 110 teams, 1,400 people, an AWS account per team, API definitions as source of truth.
- Scaling Autonomy at Zalando — InfoQ / Ben Linders, 2018 (30 September); Bowman on autonomy and alignment.
- Zalando embraces radical agility in its bold platform play — Diginomica, 2016; team sizes of three to seven (secondary).
- ZEOS: Zalando Ecommerce Operating System — ChannelX, 2023 (26 October); the 2022 pilot and the October 2023 rebrand.
- Zalando to axe 'hundreds of jobs' amid economic slowdown — Retail Gazette, 2023 (21 February); the overhead reduction.
- Zalando consolidates partnership models: Connected Retail to be discontinued — FashionUnited, 2026 (23 April); the wind-down date and fee change.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

49. Case: commercetools, Shopware and the Composable Movement

Composable

IN THIS CHAPTER, YOU WILL: Test the composable promise against the dated public record of its leading vendors and its own alliance, and learn when assembling commerce from parts is right.

KEY POINTS:

- **The movement's own flagship turned to layers.** commercetools cut staff in February 2025 after missing growth targets, launched AgenticLift in January 2026 "on top of your existing commerce solution", and in July 2026 began selling its order engine and catalog as standalone modules "without replacing their existing platforms".
- **The mid-market leader went the other way.** Shopware, first among the 1,000 largest German shops in the 2026 EHI study, launched Nexus (an orchestration layer that replaces middleware) and Shopware Payments on PayPal's rails in June 2026: integration and payments pulled into the platform, not decomposed out of it.

- **The alliance rewrote its principles, and its own survey names the cost**. Between 2023 and 2026 the MACH Alliance's stated principles went from the four-letter acronym to five principles to three, with microservices and headless gone; its February 2026 report found 37% of respondents naming integration complexity as a primary concern. Composable survives as a per-capability practice around a kept core.

FAILURE PATTERNS THIS CHAPTER ADDRESSES:

- **The Giant's Boots** — composable was defined by vendors whose reference customers were global enterprises with product teams; the same architecture proposed to a €48 million brand is a copy without its reasons.
- **The Burned Bridge** — the composable pitch of 2020–2023 was a replatforming with one cutover; the vendors' 2026 products are the opposite, layers added beside a kept core.
- **The Spaghetti Junction** — every capability bought separately is a join to own; the alliance's own survey says integration complexity is its members' primary concern.

In June 2020 four vendors formed an alliance around an acronym, **MACH**, and a promise: commerce built from independent, best-of-breed services, connected by APIs, each replaceable without touching the rest. By December 2023 the alliance had more than 100 members and a survey saying most technology leaders had increased their share of such infrastructure. Agencies, analysts and conference stages carried the word **composable** into every replatforming conversation, including, in February 2027, the one at Kestrel & Pine.

Then the vendors that made the promise changed what they sold. This chapter follows that change in their own dated words: commercetools, the alliance's founding and flagship vendor, from its 2024 leadership change through its 2025 cuts to its 2026 layers and modules; Shopware, the German open-source platform that leads its home market, pulling integration and payments inside the platform; Adobe and the Magento forks, where "composable" became deployment options; and the MACH Alliance itself, rewriting its principles three times in three years. It ends with what the movement's own survey data say about cost, when composable is right, and where the public evidence stops.

The chapter is a case study, so it reads the record rather than designing a system; the design side (composable as an architecture style, and when to adopt a capability as a separate service) belongs to Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says and Build, Buy or Assemble: Choosing and Changing Platforms, and the market map to Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026. **Composable**, in plain words, means buying commerce as separate

services (catalog, cart, checkout, search, content) from one or several vendors and assembling them, rather than taking one product. **Headless** means the storefront is separated from the commerce engine and talks to it only through an **API** (application programming interface: the contract by which one program calls another). Both are defined again below as the alliance defined them, because the definitions changed.

Where we are in the order's journey. This chapter sits off the order's path. Eva's jacket, the recurring order of Follow the Order: What Happens Between "Buy" and "Delivered", passes through a storefront, a cart, a checkout and an order record; the composable question is whether those four should be one product or four services, and who then owns the joins between them.

What Composable Promised, 2020–2023

On 24 June 2020, EPAM announced that it had joined "the newly formed MACH Alliance" as a founding member with commercetools, Contentstack and Valtech. The release expanded the acronym as "Microservices, API-first, Cloud-Native, and Headless" and described the group as "independent tech companies dedicated to advocating for open, best-of-breed technology ecosystems". That founding document is more than three years old, but it is the authoritative origin, and the four words are the promise as first made.

In plain words, the four meant: **microservices** (the commerce engine itself is built as many small, separately deployed services), **API-first** (every capability is reachable through a documented API before it has any screen), **cloud-native SaaS** (software as a service: the vendor runs it, scales it and upgrades it continuously) and **headless** (no storefront ships with the engine; the merchant builds or buys one). The argument was that a merchant assembling such services could replace any one of them without a replatforming, avoid being locked into one suite, and move faster than a suite's release cycle allowed. On 13 December 2023 the alliance announced it had passed 100 members (Mirakl, inRiver and Salsify among the new ones), restated the goal as ecosystems that are "Microservices based, API-first, Cloud-native SaaS and Headless", and cited its 2023 survey of 500 technology leaders in which 85% said they had increased their share of MACH infrastructure in the previous twelve months. The same release gives two inconsistent applicant figures (over 200 with about half admitted; 225 with about 45%), a small reminder that alliance numbers are marketing numbers.

What the 2020 promise says	What it does not say
Capabilities should be reachable through APIs and replaceable one at a time	Who builds and runs the storefront that no longer ships with the engine

What the 2020 promise says	What it does not say
A merchant should not be locked into one suite's release cycle	Who owns the joins between the services, and what that costs each year
The vendor runs and scales the services	That the merchant's own team must run everything the vendor does not: storefront, integration, identity, content
Best-of-breed beats one suite	For which size of merchant, with which team, the difference pays for itself

The right-hand column is the subject of the rest of this chapter. The promise was true as a description of how large enterprises with product-engineering teams could build; it said nothing about the merchant with fourteen technology staff.

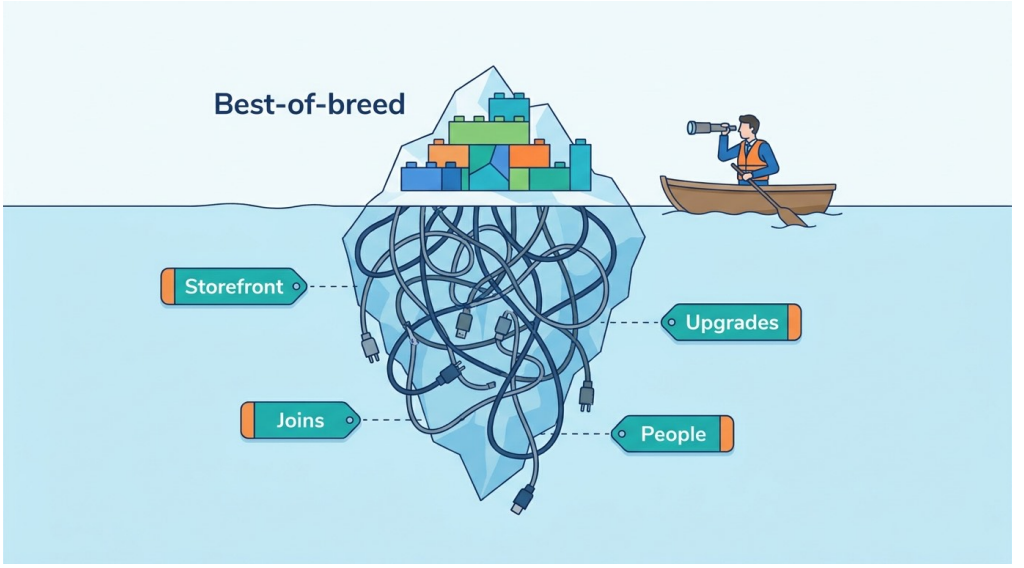


Figure 1: *The 2020 promise showed the services; the storefront, the joins, the upgrades and the people who own them sat below the waterline, assigned to nobody.*

commercetools: From Flagship to Layers

commercetools was founded in Munich in 2006, acquired by the German retail group REWE in 2015, and funded in 2019 ($145 million from Insight Partners at a $300 million valuation) and 2021 ($140 million led by Accel at $1.9 billion), as TechCrunch summarized in February 2025. It is the composable flagship: founding member of the

alliance, a Leader in Gartner's Magic Quadrant for Digital Commerce every year since 2020, and, as of its 22 April 2026 release, past a "$100B GMV run-rate" with more than 600 million orders a year across 36 countries. (**GMV**, gross merchandise volume, is the value of orders through the platform, not the vendor's revenue; a **run-rate** annualizes a recent period and is not a full-year figure.)

The turn is in its releases.

Date	Event (source)	What changed
10 July 2024	Andrew Burton, formerly of Rapid7, became CEO; founder Dirk Hoerig moved to Chief Innovation Officer; the release cited 2023 GMV of over $30 billion, up 45% (commercetools release)	Operator management for a growth-stage vendor
26 February 2025	About 10% of staff cut; CEO Burton's memo said the company had not met its "aggressive revenue growth targets" over several quarters, targets that "were never reset for macroeconomic uncertainty"; the CRO and CFO left the executive team (TechCrunch)	The pure composable pitch no longer carried the growth plan

Date	Event (source)	What changed
21 January 2026	AgenticLift launched as the "first standalone agentic offering" (**agentic commerce** is the 2025–2026 term for selling through AI assistants that browse catalogs and buy on a shopper's behalf), letting companies "layer agentic commerce capabilities on top of your existing commerce solution", including homegrown or legacy systems (commercetools release)	A product for merchants who keep a non-composable core
22 April 2026	Q1 2026 results signed by a new CEO, Doug McNary: $100B run-rate, more than $7 billion of GMV from new enterprise customers in the quarter, same-store sales up 27% (commercetools release)	Growth claimed on new logos and same-store volume
9 June 2026	"Autonomous Commerce" announced as "the new category for the AI era", with Sphere, "an enterprise commerce platform", and a preview of MosAIc, an operations tool (commercetools release)	The vocabulary moves from composable to autonomous

Date	Event (source)	What changed
15 July 2026	Core Commerce ("enterprise-grade cart and order management") and Product Catalog sold as standalone modules that integrate "with their existing technology stack", so enterprises can adopt capabilities "without replacing their existing platforms" (commercetools release)	The flagship sells itself by the piece, beside a kept backend
22 October 2026	A Leader in Gartner's 2026 Magic Quadrant for Digital Commerce Platforms (report dated 19 October 2026) "for the seventh consecutive year" (commercetools release)	Analyst standing intact through the turn

Three things stand out. First, **the vendor's own products now describe the architecture this book recommends for Omnichannel-stage merchants**: a kept core with layers added beside it. AgenticLift and the July 2026 modules are built for stacks that are not composable. Second, **the sales vocabulary moved on**. The June 2026 release does not sell "composable"; it sells "Autonomous Commerce", a category the vendor says it is introducing. A newcomer should notice that the category changed while the product's shape (API services the merchant assembles or attaches) did not. Third, **the numbers need their units**. The April 2026 release says "500+ enterprises"; the July 2026 boilerplate says "more than 600 global brands and enterprises", "over $110 billion in annualized GMV" and "over 700 million orders". The April figures were $100 billion and 600 million orders. Either the business grew about 10% in three months or the units differ; the releases do not say which, and neither should you.

> **INFERENCE, NOT DOCUMENTED**: No commercetools release explains why it moved to standalone modules. The sequence (missed growth targets in 2025, then layers and modules in 2026) is consistent with a vendor finding that full replatforming onto an API-only core sells to fewer merchants than its 2020–2023 pitch assumed, and that Commerce.com's August 2026 remark about B2C replatforming being "softer than historically" describes the same market. That reading is mine.

The CEO sequence also has a gap in the public record: founder Dirk Hoerig to Andrew

Burton (July 2024) is documented; Doug McNary is the CEO quoted from April 2026; the handover between them was not found in any release fetched for this book.

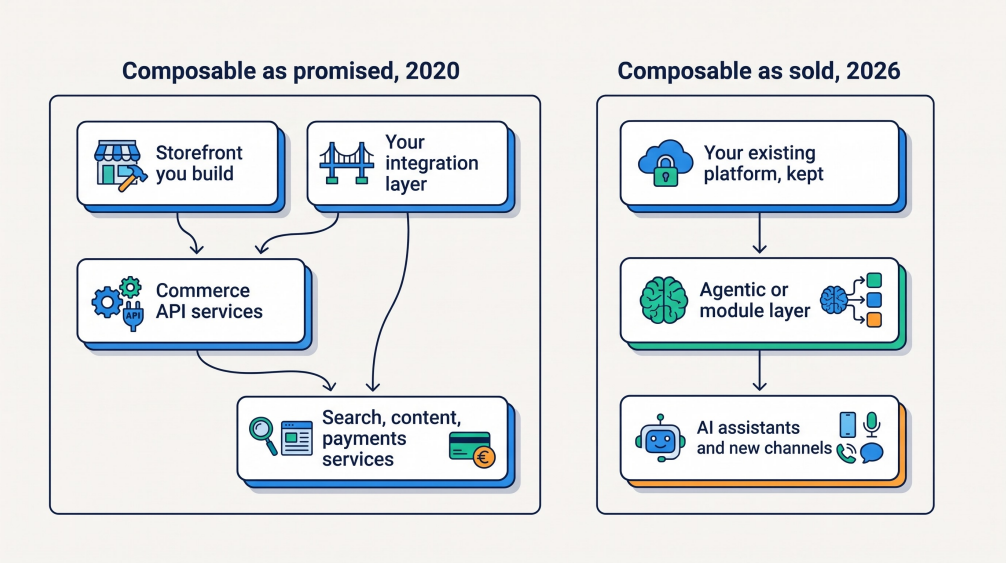


Figure 2: *The 2020 promise asked the merchant to build the storefront and own every join; the 2026 products from the same vendors attach a layer to a platform the merchant keeps.*

Shopware: The Mid-Market Path

Shopware, from Schöppingen in Germany, is the counter-case: an open-source platform whose answer to the composable movement was to pull more into the platform, not less.

Its finances and ownership are documented by its own releases. It was "100% bootstrapped" until February 2022, when Carlyle and PayPal invested USD 100 million, with platform GMV approaching USD 20 billion in 2021 (that release is more than three years old). In late 2025 PayPal raised its stake "from about 11% to more than 40%" by taking over Carlyle's shares, as Shopware's July 2026 boilerplate states and the trade press reported on 23 October 2025 alongside a claim of "more than 50,000 online sellers". On 11 August 2026 Shopware announced that Joachim Weber would become Co-CEO on 17 August and sole CEO on 1 January 2027, with founder Sebastian Hamann becoming Chief Experience Officer and founder Stefan Hamann "Chief Agentic Officer"; the founders keep a majority of shares.

Its market position is specific. The EHI Retail Institute's 2026 report on the 1,000 largest German B2C shops found 12.5% on Shopware, first place for the fifth year (a platform was identified for 60.2% of those shops). W3Techs, which counts websites worldwide,

puts Shopware at 0.2% of sites with a known e-commerce technology; both are true, and the difference is the population counted. Analysts placed it as a Visionary in Gartner's 2025 and 2026 Magic Quadrants for Digital Commerce and a Strong Performer, "best fit for B2B organizations" (B2B: business-to-business, selling to other companies), in Forrester's Commerce Solutions Wave of Q3 2026.

The pricing is the first clue to its path. The Community Edition is free; the paid plans (Rise "from €600 a month", Evolve "from €2,400 a month", Beyond on request, November 2026) are "based on your gross merchandise value", and a fair-usage policy requires a paid plan once the shops linked to one account exceed €1 million of GMV in a rolling year. An open-source platform priced on GMV is a platform that wants to sit in the path of every order, which is the opposite of a service you can swap.

The second clue came on 10 June 2026 at Shopware Community Day in Cologne. Shopware announced **Nexus**, "a native, event-driven orchestration layer" with reusable connectors and AI-assisted mapping, which it positions as replacing middleware, and **Shopware Payments**, powered by PayPal's infrastructure, live in Germany and Austria with the EU and US to follow. The release cited vendor figures that merchants average more than 15 disconnected tools and spend up to 52% of implementation effort wiring them together, and claimed Nexus cuts integration costs by 40%; those are vendor claims without a published method. Nexus is metered: €10 per extra 1,000 executions above the plan's quota (pricing page, November 2026). In plain words, the **iPaaS** (integration platform as a service: the hosted tool that moves data between systems) that a composable merchant buys separately became a Shopware product, and so did the **PSP** (payment service provider: the company that takes the shopper's payment and settles the money).

> **INFERENCE, NOT DOCUMENTED:** Shopware has not said that Nexus and Payments are a response to the composable movement. Read together with the GMV-based pricing and PayPal's stake, they describe a platform that earns on orders and payments and wants the integration layer inside its own metering. That is the same economic shape as a hosted SaaS platform (Case: Shopify, the Modular Monolith That Serves Millions of Merchants), reached from the open-source side; the inference is mine.

What Shopware shows a newcomer is that **"open source" and "composable" are different axes**. Shopware's core is open; its commercial path is to be the hub. A merchant on Shopware at the Omnichannel stage can still buy an OMS or a PIM beside it, but the platform's own direction is to absorb the joins, and the fair-usage line means the hub is paid for once the merchant grows.

Adobe, the Forks and Commerce.com: Composable as a Deployment Option

Two more vendors show the word being absorbed rather than adopted.

Adobe Commerce (the former Magento Commerce, renamed in April 2021 after Adobe's 2018 acquisition of Magento for $1.68 billion) now lists three deployment options on its information center (updated 21 October 2026): on-premises, "on cloud infrastructure" (platform-as-a-service: Adobe runs the hosting, the merchant owns custom code) and "as a Cloud Service" (fully managed SaaS with continuous updates and a headless storefront that Adobe hosts). Beside them, **Adobe Commerce Optimizer** is a SaaS for storefront, catalog and merchandising that "can work alongside an existing Adobe Commerce or third-party commerce backend"; the two SaaS products "don't currently integrate" with each other. Partner reports place both launches in June 2025, after a March 2025 announcement; no Adobe press release was found. Here "composable" has become a layer a suite vendor sells beside any backend, including a competitor's.

The forks tell the open-source side. Community leaders organized in 2021 "concerned about the future of Magento Open Source"; the **Mage-OS** Association was incorporated in Poland in 2022 with a founding board of eight directors and describes its distribution as "independent, community-driven" and "highly compatible" with Magento 2 extensions. Mage-OS 4.0.0 shipped on 30 October 2026, based on Magento 2.4.9; 3.6.0, on 14 October 2026, was a security port of Adobe's October patch. The association "has no shareholders or investors" and lists 216-plus financial contributors and 82-plus professional members (site accessed November 2026). A merchant on Mage-OS has the most freedom of any in this chapter and the longest patch chain: Adobe, then the distribution, then the agency, then the merchant.

Commerce.com is the SaaS side of the same story. BigCommerce Holdings renamed itself Commerce.com on 31 July 2025 and, in Digital Commerce 360's account, described the change as moving "away from an all-in-one platform toward an open, composable system that connects to external AI tools"; its CEO said "this is not a cosmetic change". The engine, BigCommerce, stayed. Its results for the quarter ended 30 June 2026, filed with the SEC on 6 August 2026, showed revenue of $84.5 million, up 0.1%, subscription revenue down 1%, and management saying B2C (business-to-consumer) replatforming "remains softer than historically". A vendor adopted the composable vocabulary and, a year later, reported that the market for moving platforms was weak.

The Alliance Rewrites Its Principles

The clearest evidence of the turn is the alliance's own wording, which changed three times.

Date	The principles as stated (source)
June 2020 and December 2023	"Microservices, API-first, Cloud-Native, and Headless" (EPAM, 2020-06-24); "Microservices based, API-first, Cloud-native SaaS and Headless" (MACH Alliance, 2023-12-13)
5 August 2025	Five guiding principles, Composable, Connected, Incremental, Open and Autonomous, announced with a move to "end-user leadership" by September 2026 (an appointed end-user president, two end-user ambassadors on the board); the acronym is not restated (MACH Alliance via Silicon UK, 2025-08-05)
November 2026	Three principles: "Open is about what the system is made of — and whether humans and agents can see it, trust it, and verify it"; "Composable means capabilities you can assemble, replace, and evolve independently"; "Connected means designed to be optimized in an ecosystem — not in isolation"; the alliance "certifies ISVs, system integrators, and enablers against these principles" (machalliance.org/mach-principles, accessed 2026-11)

Microservices and headless, two of the four letters, are no longer principles. "Incremental", which appeared in August 2025, is the word that most contradicts the 2020–2023 pitch, because incremental adoption around a kept core is what the vendors now sell. The 2026 page gives no reason for the change. My reading, labeled as such: **the acronym outlived its expansion**. The parts of the promise that survive (open, composable, connected) are properties a merchant can ask of any platform, including a hosted SaaS one with good APIs; the parts that were dropped were prescriptions about how the vendor's own software should be built, which were never a merchant's concern.

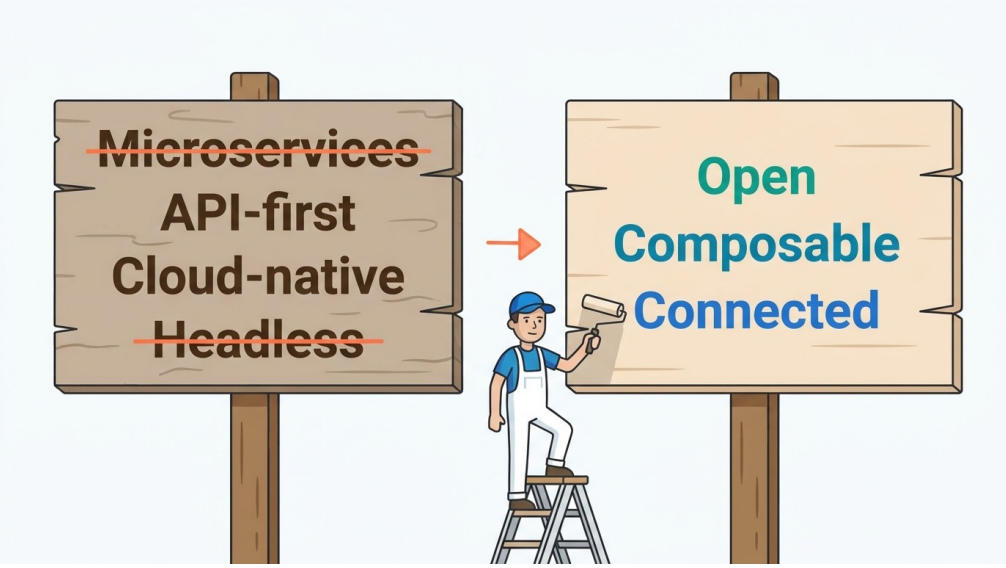


Figure 3: *Between 2020 and November 2026 the alliance went from four words to five principles to three: microservices and headless were dropped, and open, composable and connected remained.*

What the Movement's Own Data Say

The MACH Alliance's Enterprise Technology Report, published 18 February 2026 from a survey of 600 enterprise technology decision-makers in seven markets, reports that 78% of organizations with fully scaled MACH technology see clear return on investment from AI versus 13% in early planning, that 98% of mature composable adopters can support AI at scale versus 33% of early-stage ones, and that 89% say standards and certifications for AI in composable environments are missing. It also reports that **37% cite integration complexity as a primary concern** and 45% flag data privacy and security.

Read it as what it is: a survey commissioned by a vendor alliance, answered by a population that includes its own adopters, with "clear ROI" undefined. The usable finding is the one against interest. Among the people most committed to composable, more than a third name integration complexity as a primary concern. That is the cost line the 2020 promise left out, and the reason the "Spaghetti Junction" pattern is the one most composable merchants meet first.

Shopware's June 2026 figures (more than 15 disconnected tools per merchant; up to 52% of implementation effort spent wiring them) point the same way and carry the same caveat: a vendor selling an integration product has reasons to count integration cost high. No independent, published study with a stated method was found that measures the total cost of a composable assembly against a hosted platform with layers; where this chapter gives costs, they are the author's judgment or Kestrel & Pine illustrations.

What Composable Actually Costs

A composable assembly has the same bill as any platform, plus lines that the brochure assigns to nobody. The table is my synthesis; the amounts are not from any vendor.

Cost line	In a hosted platform with layers	In a composable assembly	Who pays it
The storefront	Included: a theme, customized by an agency	Built or bought separately, then kept current with every framework and browser change	Your storefront team or agency, every year
The joins	Apps and connectors, each with an owner (34 at Kestrel & Pine)	Every pair of services, plus the integration layer that runs them; the alliance's 37%	Your integration team, every day
Upgrades	The vendor upgrades the core for everyone; you upgrade your apps	The vendors upgrade their services; you upgrade everything you built against them, including the storefront	You, on the vendors' schedules
Identity, content, search	Often in the platform or one app each	Separate products, each with a contract, an API version and a renewal date	Your vendor manager
Licenses	Subscription plus a share of sales	Subscription with GMV caps per service (Saleor Cloud: 0.8% above $200,000 a month; commercetools: not public)	The merchant, with more lines to add up

Cost line	In a hosted platform with layers	In a composable assembly	Who pays it
People	Integration engineers and app owners	Product engineers, platform engineers, integration engineers, and someone to own each service	Headcount that must exist before the first sale
Exit	Export data, rebuild theme and apps	Every join is yours to rebuild; the storefront moves with you, which is the real benefit	Paid once, if ever

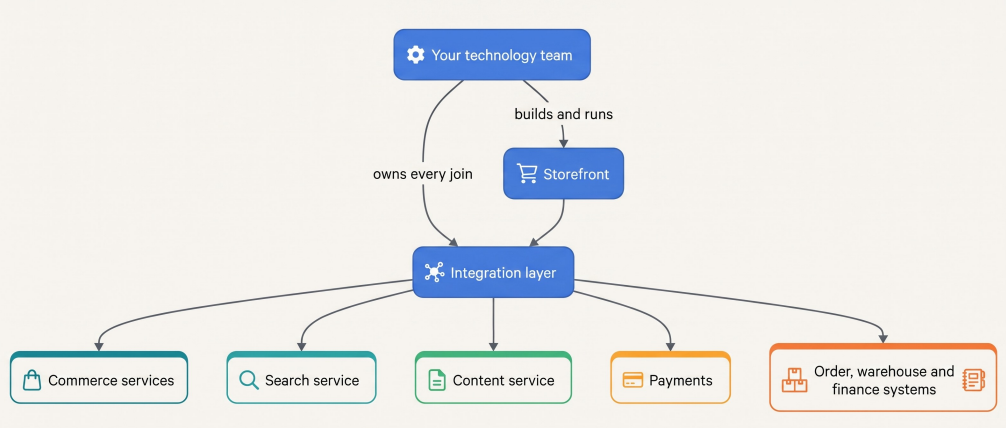


Figure 4: *In a composable assembly the merchant's own team sits at the center, owning the storefront and every join; nothing in the diagram is run by a vendor except the boxes at the edge.*

The storefront line deserves a worked illustration. Kestrel & Pine, the fictional outdoor brand this book follows (Meet Kestrel & Pine), has three storefront and app engineers out of fourteen technology staff and pays an agency €14,000 a month for its theme, apps and small integrations. A built storefront for three countries, with search, content and checkout integrations, is a product in its own right; assume (an illustration, not a ledger fact) that it needs at least the three engineers full time plus the agency, before any of the 34 existing joins are rebuilt against new APIs. The assembly would not have fewer

joins than today; it would have more, and all of them would be Kestrel & Pine's. That is what the fourth column of the table means.

When Composable Is Right

The record does not say composable is wrong. It says it was oversold to the wrong merchants. The conditions under which it is right are specific.

Situation	Composable fits when	It limits when
Scale stage	Group: several brands or countries with a platform team and product teams per journey	Starting and Growing: there is no team to own the joins; Omnichannel: only per capability, around a kept core
Differentiation	The storefront or the checkout is where the business competes, and a built one earns its keep	The storefront is a channel like any other and a theme would do
Team shape	Product-engineering organization: people who build and run services and storefronts	Integration organization: people who own connectors and contracts (Kestrel & Pine's shape)
Vendor count	One owner per service and a vendor manager; contracts with versioned APIs and dated deprecations	Each new service adds a renewal, an API version and an owner nobody has named
Reason to move	A capability the current platform cannot own (a global catalog, an OMS, a B2B engine) is adopted as a service beside it	The reason is an incident that an availability service or an OMS would fix without replatforming

The per-capability path is the one the vendors themselves now sell and the one an Omnichannel merchant can absorb: keep the platform, add a PIM, an OMS, a search service or an agentic layer as separate services with their own owners, and treat each as a composable step. Kestrel & Pine's Waypoint program is that path under another name. The full assembly is a Group-stage choice, and even there the evidence from commercetools' July 2026 modules is that enterprises want to adopt it "without replacing their existing platforms".

What Kestrel & Pine Takes From It

The composable rebuild an outside consultancy pitched to Noor Visser in February 2027 (about €3.2 million over three years; see Build, Buy or Assemble: Choosing and Changing Platforms) proposed exactly the 2020 assembly: an API-only engine, a built storefront, a search service, a content system and an integration layer, for a company with fourteen technology staff and 34 joins it already struggles to own. Waypoint's answer, decisions W1 to W5, is the 2026 version of the same idea: the WMS (warehouse management system) is the system of record for stock, an availability service and an OMS (order management system: one record for every channel's orders) are bought as services beside the platform, a PIM (product information management system) becomes the product record, every integration gets an owner and a contract, and the platform stays. The record in this chapter is why that is not a compromise but the current practice of the movement's own vendors; the 2029 review (W4) is where the Group-stage question is asked again, with figures.

Where the Evidence Stops

- **commercetools' reasons**. No release explains the turn to layers and modules; the link between the February 2025 cuts and the 2026 product line is my inference. The CEO handover from Andrew Burton to Doug McNary is undated in the fetched record, and the April and July 2026 figures ("500+" versus "more than 600"; $100 billion versus $110 billion) are not reconciled by the vendor.
- **Shopware's strategy**. Shopware has not stated that Nexus and Payments respond to composable, nor published a method behind its "40% lower integration costs" claim; its current merchant count ("more than 50,000" in 2025 press) is not on a Shopware page fetched here; PayPal's exact stake is reported as "more than 40%" or "about 41%".
- **Adobe's dates**. The general-availability date of Adobe Commerce as a Cloud Service and Commerce Optimizer rests on partner reports (June 2025); no Adobe press release was found.
- **The alliance's reasons**. The MACH Alliance's principles page gives no reason for dropping microservices and headless; the 2023 release's applicant figures are internally inconsistent; the 2026 survey's method and sample are the alliance's.
- **Costs**. No independent study with a stated method compares the total cost of a composable assembly with a hosted platform plus layers; every cost figure in this chapter is a labeled judgment or a Kestrel & Pine illustration.
- **Analyst placements**. Gartner's 2026 vendor count was not in the fetched release; the placements of Shopify, Salesforce, SAP and Adobe in the 2025–2026 reports were not found.

What to Take Away

Established.

- The composable promise was defined by vendors in June 2020 as microservices, API-first, cloud-native and headless (EPAM release), and by November 2026 the alliance's principles no longer include microservices or headless (machalliance.org, accessed 2026-11).
- The flagship vendor now sells layers and modules for existing stacks: AgenticLift (21 January 2026) and standalone Core Commerce and Product Catalog "without replacing their existing platforms" (15 July 2026), after a February 2025 staff cut tied to missed growth targets (TechCrunch).
- Shopware, first among the 1,000 largest German B2C shops (EHI 2026), moved integration and payments into the platform with Nexus and Shopware Payments (10 June 2026) and prices its plans on GMV (pricing page, November 2026).
- The alliance's own February 2026 survey found 37% of respondents naming integration complexity as a primary concern.

Depends on context.

- A full composable assembly is right for a Group-stage merchant with product and platform teams and a storefront that is itself a competitive product; it is the Giant's Boots for a merchant below that stage or with an integration-shaped team.
- Adopting one capability as a separate service (PIM, OMS, search, an agentic layer) beside a kept platform is right at the Omnichannel stage whenever the platform cannot own that capability well; it limits when the merchant cannot name an owner for the new join.
- Open source and composable are independent choices: Shopware is open and hub-shaped; commercetools is proprietary and service-shaped. Choose on who owns upgrades and joins, not on the license.

The author's synthesis.

- The composable movement was right about properties (open APIs, replaceable capabilities, clear ownership) and wrong about prescriptions (rebuild as services, build your own storefront), because the prescriptions were the vendors' reference architecture, not the merchant's need.
- The 2026 products of commercetools, Adobe and Salesforce are the movement's own correction: the kept core with layers beside it is composable as it should have been sold, and it is the shape most merchants should buy.

Questions to Ask

1. When a vendor or agency says "composable", which of the four 2020 words do they mean, and which of the alliance's three 2026 principles?
2. Who will build and run the storefront that an API-only engine does not ship, and what will that cost each year after launch?
3. For each service in the proposed assembly, who owns the join to the others, its API version and its renewal date?
4. Which capability does our current platform fail to own well, and can it be adopted as one service beside the platform rather than by replacing the platform?
5. Which of the vendor's reference customers is within a factor of ten of our revenue and team size?
6. What does the vendor's own 2026 product line sell to merchants who keep their existing platform, and why would we not buy that instead?
7. What unit, date and definition stand behind every GMV, customer and order figure in the deck, and are the figures in two releases consistent?

The four cases of Part XI end here: a modular monolith that serves millions of merchants, a retailer that became a platform, a microservices pioneer that returned to discipline, and a movement that corrected itself. The reference pages begin with Domain Coverage: From Three Model Families to Forty-Nine Chapters, which maps the research models behind this book to the chapters that cover them.

Sources and Evidence

Company, product and investor sources

- commercetools appoints new CEO, commercetools, 2024 — Andrew Burton's appointment, Dirk Hoerig's move, the 2023 GMV.
- commercetools, a pioneer in headless commerce, lays off dozens of staff, TechCrunch, 2025 — the February 2025 cut, the CEO memo, the funding history.
- commercetools launches AgenticLift, commercetools, 2026 — the "on top of your existing commerce solution" layer.
- commercetools reports strong Q1 2026, commercetools, 2026 — the $100B run-rate, orders, countries, same-store sales, the CEO.
- commercetools introduces "Autonomous Commerce" category and unveils Sphere, commercetools, 2026 — the June 2026 category and platform naming.
- commercetools launches modular commerce offerings, commercetools, 2026 — standalone Core Commerce and Product Catalog, "without replacing their existing platforms", the boilerplate figures.

- commercetools named a Leader in the 2026 Gartner Magic Quadrant for Digital Commerce Platforms, commercetools, 2026 — the seventh consecutive placement, report dated 19 October 2026.
- Shopware raises USD 100 million from Carlyle and PayPal, Shopware, 2022 — the bootstrapped history and 2021 GMV (more than three years old).
- PayPal increases stake in Shopware, Ecommerce News Europe, 2025 — the stake rising to about 41% and the merchant count.
- Shopware named a Strong Performer in Forrester's 2026 commerce evaluation, Shopware, 2026 — the Forrester placement and the boilerplate on PayPal's stake.
- Shopware Community Day 2026, Shopware, 2026 — Nexus, Shopware Payments, the integration-cost claims.
- Shopware launches Shopware Payments in partnership with PayPal, Shopware, 2026 — transaction pricing and onboarding through PayPal.
- New management structure at shopware AG, Shopware, 2026 — the CEO succession and founder roles.
- Shopware remains market leader in Germany 2026 (EHI), Shopware, 2026 — 12.5% of the top 1,000 German B2C shops.
- Shopware a Visionary in the 2025 Gartner Magic Quadrant, Shopware, 2025 — report date and 19-vendor count.
- Shopware pricing, Shopware, n.d., accessed 2026-11 — GMV-based plans, the fair-usage threshold, Nexus metering.
- Adobe Commerce platforms and services, Adobe Experience League, 2026 — the three deployment options and Commerce Optimizer.
- Adobe Commerce as a Cloud Service overview, Adobe Experience League, 2026 — managed SaaS with the headless storefront.
- Demystifying Adobe Commerce as a Cloud Service, Rightpoint, 2025 — the June 2025 launch timing (partner report).
- Magento, Wikipedia, 2026 — the 2018 acquisition and the 2021 rename.
- Mage-OS home, about and founding announcement, Mage-OS Association, 2022–2026 — the distribution, the association, releases 3.6.0 and 4.0.0.
- BigCommerce rebrands as Commerce.com, Digital Commerce 360, 2025 — the rename and the "open, composable system" framing.
- Commerce.com Q2 2026 results, exhibit 99.1, Commerce.com via SEC EDGAR, 2026 — revenue growth of 0.1% and the "softer than historically" remark.
- Saleor pricing, Saleor, n.d., accessed 2026-11 — GMV-capped subscription for an API-only platform.

Standards, regulation and official documentation

- EPAM joins the newly formed MACH Alliance as a founding member, EPAM Systems, 2020 — the founding members and the acronym's expansion (more than three years old; the founding document).

- MACH Alliance surpasses 100 members, MACH Alliance, 2023 — the 2023 formulation, the survey and the applicant figures.
- MACH Alliance advances end-user leadership model, MACH Alliance via Silicon UK, 2025 — the five principles and the governance change.
- MACH principles and MACH explained, MACH Alliance, n.d., accessed 2026-11 — the three principles as stated in 2026.
- MACH Alliance Enterprise Technology Report, MACH Alliance, 2026 — the survey figures, including the 37% integration-complexity finding.

Research and practitioner sources

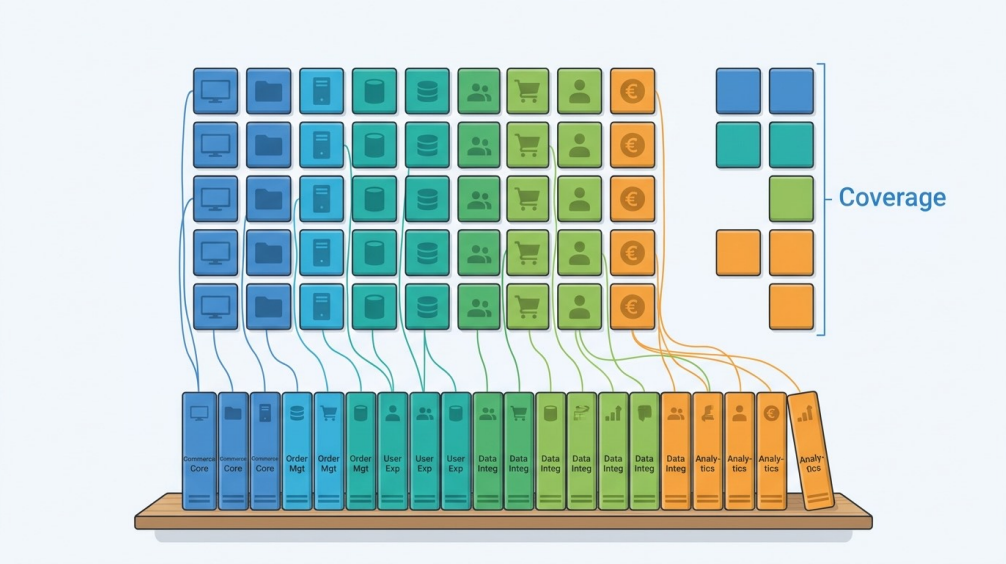
- Usage statistics of e-commerce technologies, W3Techs, 2026 — Shopware's share of websites, contrasted with the EHI population.
- Kibo named a Leader in The Forrester Wave: Commerce Solutions, Q3 2026, Kibo Commerce, 2026 — the ten-vendor count of the wave in which Shopware is a Strong Performer.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Reference

The model-to-chapter coverage map, decision templates and cards, plain-language definitions, and every source the book cites.

Domain Coverage: From Three Model Families to Forty-Nine Chapters



IN THIS SECTION, YOU WILL: Trace every capability of the three research model families behind this book to the chapter that covers it, see where the book departs from the models, and learn which capabilities it leaves out and why.

KEY POINTS:

- **Every capability has a home, and most have two.** The digest behind this page lists 152 distinct capabilities across eleven models. 141 landed in a chapter by plan; this page records where the other eleven, and six thin spots found while checking the written chapters, are mentioned briefly or left out on purpose.
- **The models describe products; the book describes the joins between them.** No model has an integration layer, a scale-stage view, a journey across products or a failure pattern. The book adds Part VI (integration),

Part IX (journeys), Part X (evolution) and the sixteen failure patterns for exactly that reason.

- **The models are maps, not evidence.** They are architectural models inferred from public product behavior, with a modeled seed value for every number. Nothing on this page, and nothing in the book, uses them as a source for a fact about Shopify, Amazon or any specialist vendor.

This book was planned from **capability models, not from a table of contents**. Before the first chapter was written, three families of Productscape domain models were inventoried brick by brick: a model of a merchant commerce platform of the Shopify kind, a model of a retail marketplace of the Amazon kind, and nine models of specialist commerce products (a B2B webshop platform, a returns platform, an EDI network, a marketplace integrator, a commerce ERP, a store POS, a shipping platform, a cloud WMS and the hub that connects them). The chapters were drawn so that every brick lands somewhere, and so that what the models lack is added on purpose rather than by accident.

This page is the audit trail of that planning. It answers four questions: **"Where is my capability covered?"**, **"Where does the book disagree with the models?"**, **"What did the book add?"** and **"What did it leave out?"** The capability map itself is in The Map of a Commerce Platform: Capabilities and Layers and the rule for deciding which system owns which fact is in Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record; this page does not repeat their arguments. It is a lookup table and is meant to be used as one.

Where the Models Come From

A **Productscape domain model** is a structured description of a product domain: the customers and what they care about, the product **bricks** (named business capabilities) and the **modules** inside each brick (screens, APIs, workers, services, adapters), the data each brick keeps, and a stress test of what could change in the business. The models are kept as JSON and Markdown in the author's sibling `productscape-workspace` repository and were compiled into a research digest for this book on 7 November 2026.

Family	Model (short code)	Inspired by	Bricks	Root groups
Merchant platform	`merchant-commerce-platform` (MCP)	The core behavior of a hosted platform of the Shopify kind	29	Storefront and buyer experience; commerce core and operations; money and merchant financial services; growth, channels and AI; platform and ecosystem
Retail marketplace	`online-retail-marketplace` (ORM)	The core behavior of a marketplace of the Amazon kind	30	Shopper experience; seller growth; marketplace foundations
Specialist products (nine models)	`b2b-wholesale-ecommerce-platform` (B2B), `ecommerce-returns-management-platform` (RET), `edi-and-b2b-document-exchange-platform` (EDI), `marketplace-channel-integration-platform` (MKT), `omnichannel-commerce-erp-platform` (ERP), `retail-pos-and-store-management-platform` (POS), `shipping-and-carrier-management-platform` (SHP), `warehouse-management-and-fulfilment-platform` (WMS), `katapull-commerce-hub` (HUB)	The core pages of nine specialist products of the software	25, 15, 22, 27, 23, 28, 24, 26, 10	Per product: a selling layer, a core, an integration layer and a platform foundation; the hub adds a control layer and a shared customer layer

Four rules follow from what the models are:

- **Nothing in a model is a fact about a company**. The models' own notes say that the brick decomposition, module layering, data stores and deployment surfaces are "an architectural model inferred from public product behavior". The book never presents a brick, module or store as the architecture of Shopify, Amazon or any specialist vendor; facts about those companies come only from dated public sources cited in the chapters.
- **Every number in a model is a seed value**. The models' KPI values are, in their authors' words, "illustrative modeling baselines". None is quoted anywhere in the book.
- **The specialist models are used for their capability inventory only**. Which bricks a product has, which data it keeps, which roles it serves. Nothing about the group's or any label's plans, finances, pricing, roadmap or strategy is carried over, and the labels are not named on this page. Where a chapter names a specialist vendor, it does so from a public page verified in a research brief, like any other company.
- **The running example is fictional**. Kestrel & Pine, the fictional outdoor brand this book follows (see Meet Kestrel & Pine), is not a disguised customer of any vendor; its systems are named by role, and none of its figures or incidents belongs to a real company.

How to Read the Tables

The capability tables list the models whose bricks carry a capability by short code, then the chapter that covers it. The first chapter named is the **primary** one: it owns the terms and draws the architecture. Chapters in parentheses cover one part of the capability or use it as an input. Placements were built from the digest's capability inventory and the chapter contracts in the book plan, then checked against each written chapter. For the chapters still in revision when this page was compiled (Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Journey 2: The Jacket Comes Back, Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants) the placements follow the chapter contracts.

Three chapters cover every capability at map level and are not repeated in every row: The Map of a Commerce Platform: Capabilities and Layers (the map and the layers), Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record (the entity–owner table) and What Changes at Each Scale: From First Store to Group (every chapter's scale-stage table gathered into one). Part I, the journeys of Part IX and the cases of Part XI use capabilities as context and appear only where they own a point.

Coverage by Capability

The digest deduplicated the eleven models' bricks into 152 capabilities. The tables below merge closely related ones into a row per chapter topic, so a row may stand for several digest entries.

Part II — Catalog

Capability	Models	Chapter
Product catalog: products, variants, media, attributes, collections, bulk import; specifications, configurators, families and bundles	MCP, ORM, B2B, ERP, POS, WMS, MKT	What Are We Selling? Products, Variants and the Catalog
Supplier article import; barcodes and GS1 identifiers; serialized items; content translation per market	ERP, POS, WMS, MKT, MCP, B2B	What Are We Selling? Products, Variants and the Catalog (The Back Office: ERP Integration and Who Is the Hub)
Price lists, customer-specific net and contract prices, tiered discounts	B2B, ERP, POS, MCP, WMS	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax (Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals)
Price rules from cost and margin; channel prices and head-office pushes; markets with currency, duties and payment methods; marketplace prices from cost and commission	ERP, POS, MCP, MKT	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax (Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Selling on Someone Else's Platform: Marketplaces and Channel Integration)

Capability	Models	Chapter
Promotions and discount rules; promotion planning	MCP, ORM, B2B, ERP, POS, MKT	Discounts, Coupons and Loyalty: Promotions Without Chaos
Loyalty points, customer cards, vouchers, gift cards; paid membership with benefit entitlement	B2B, POS, ERP, ORM	Discounts, Coupons and Loyalty: Promotions Without Chaos (Who Sells What to Whom: Business Models and the Economics That Shape Architecture)
Site search with facets and suggestions; merchandising rules and badges; personalization and recommendations; home feed	MCP, ORM, B2B, ERP	Finding the Product: Search, Navigation and Merchandising
Reviews, ratings and Q&A with moderation	ORM, ERP	Finding the Product: Search, Navigation and Merchandising (Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation)
Multi-location inventory ledger; reservations, allocations, backorders, partial deliveries; transfers and cross-branch lookups	MCP, ERP, POS, WMS, MKT	Is It in Stock? Inventory, Availability and the Promise You Can Keep (The Order Record: Lifecycle, State and the Order Management System, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment)

Capability	Models	Chapter
Available-to-sell per channel with buffers; near-real-time stock publication to channels and pages	MKT, ERP, POS, WMS, B2B, SHP	Is It in Stock? Inventory, Availability and the Promise You Can Keep (Selling on Someone Else's Platform: Marketplaces and Channel Integration)

Part III — Order

Capability	Models	Chapter
Cart and save-for-later; hosted checkout with UI extensions and functions; accelerated checkout with saved buyer identity and wallets	MCP, ORM, B2B, ERP	From Cart to Confirmation: Checkout Orchestration (Apps, Plugins and Functions: Extensibility and Ecosystem Governance, One Customer, One Product: Identity and Master Data Across Systems)
Payment acceptance: methods, authorization, capture, refund, installments; provider connectors; tokenization vault	MCP, B2B, ERP, ORM, POS	Taking the Money: Payment Providers, Authorization, Capture and Refunds

Capability	Models	Chapter
Fraud and risk scoring; chargebacks and disputes; bot protection	MCP, ORM, RET	Taking the Money: Payment Providers, Authorization, Capture and Refunds (Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers); the operator's side in LISTED's Scams, Fakes and Account Takeovers: Fraud Prevention for Listings
Order system of record across channels: edits, payment state, fulfillment orders, refunds; routing and multi-warehouse ship-from selection	MCP, ORM, B2B, ERP, POS, MKT, WMS, SHP	The Order Record: Lifecycle, State and the Order Management System (Getting It There: Carriers, Labels, Tracking and the Delivery Promise)
Tax, VAT, GST and duty calculation; invoicing, credit notes, receivables, dunning; general ledger and multiple administrations	MCP, ERP, POS, B2B	Tax, Invoices and the Books: From Order to Ledger (The Back Office: ERP Integration and Who Is the Hub)
Merchant or seller balance, fees, reserves, payouts, settlement ledger	MCP, ORM	Tax, Invoices and the Books: From Order to Ledger; RECONCILED's Payments-to-Ledger: Draining the Suspense Swamp

Part IV — Fulfillment

Capability	Models	Chapter
Warehouses, locations, containers; purchase orders, goods receipt, put-away; pick-location replenishment	WMS, ERP, POS	Pick, Pack, Ship: Warehouse Management and Fulfillment (The Back Office: ERP Integration and Who Is the Hub)
Batch picking; packing with barcode verification and packaging advice; cycle counting; warehouse rules; scanner app, printers and hardware	WMS, ERP, POS, SHP	Pick, Pack, Ship: Warehouse Management and Fulfillment
Multi-client (3PL) fulfillment: client separation, portal, invoicing	WMS, SHP	Pick, Pack, Ship: Warehouse Management and Fulfillment (Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation)
Shipping rules; carrier and service selection; rate cards and expected cost; booking, labels, scan-to-ship; customs and dangerous-goods documents	SHP, WMS, ERP, POS, MCP, B2B	Getting It There: Carriers, Labels, Tracking and the Delivery Promise (Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation)
Carrier connector network; tracking event hub with one status model; shipment exceptions and carrier scorecards; carrier invoice control	SHP, RET, WMS, MCP, ORM	Getting It There: Carriers, Labels, Tracking and the Delivery Promise (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation)

Capability	Models	Chapter
Delivery promise calculation and promise-risk monitoring; delivery options, pickup points and lockers at checkout; address validation	ORM, SHP, MCP, ERP, B2B	Getting It There: Carriers, Labels, Tracking and the Delivery Promise (Is It in Stock? Inventory, Availability and the Promise You Can Keep, From Cart to Confirmation: Checkout Orchestration)
POS checkout; tenders and terminals; receipts, cashbook, day closing; store hardware; offline operation with idempotent sync	POS, ERP, MCP	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment (Taking the Money: Payment Providers, Authorization, Capture and Refunds, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation)
Click-and-collect, ship-from-store, in-store returns, cross-branch stock; head-office control of chains and franchises	MCP, POS, RET, ERP	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
Returns portal; policy and rules engine; labels, carrier choice and fees; exchanges on live availability; store credit; claims approval gate	RET, SHP, MCP, ORM, B2B	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics (Discounts, Coupons and Loyalty: Promotions Without Chaos)

Capability	Models	Chapter
Return receipt, inspection, grading and disposition; refund orchestration with write-back; withdrawal right; return insights and benchmarks	RET, WMS, ERP, MCP, B2B	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics (Pick, Pack, Ship: Warehouse Management and Fulfillment, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation)
Branded track-and-trace communication; order status page; care cases, escalation, helpdesk context	SHP, RET, ORM, MCP, EDI	Where Is My Order? Notifications, Service and the Customer Record
Customer segmentation and marketing automation with consent	MCP, ERP, RET	Where Is My Order? Notifications, Service and the Customer Record (Whose Data Is It? Privacy, Consent and Retention)

Part V — Channels

Capability	Models	Chapter
Marketplace connector framework with a canonical model and versioned adapters; listing publication with category mapping, validation and content rules; product selections; feeds for shopping and social channels	MKT, MCP, ERP, POS, ORM, B2B	Selling on Someone Else's Platform: Marketplaces and Channel Integration (Promises Between Systems: API Contracts, Webhooks and Versioning, What Are We Selling? Products, Variants and the Catalog)

Capability	Models	Chapter
Repricing and buy box; competitor offers; offer and account health	MKT, ORM	Selling on Someone Else's Platform: Marketplaces and Channel Integration (What Does It Cost? Price Lists, Contract Prices, Currencies and Tax)
Marketplace order import, normalization, routing; status sync within deadlines; marketplace-operated fulfillment and offer switching	MKT, WMS, SHP, ERP, POS, ORM	Selling on Someone Else's Platform: Marketplaces and Channel Integration (The Order Record: Lifecycle, State and the Order Management System)
Seller onboarding, verification and reverification	ORM	Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation (trader traceability); LISTED's Rules of the Marketplace: DSA, DAC7 and Vertical Regulation
Company accounts with buyers, roles, groups, assortments; quotes, quick reorder, order lists; payment on account, terms, credit checks; punchout	B2B, MCP, ERP, WMS, POS, EDI	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals (Taking the Money: Payment Providers, Authorization, Capture and Refunds, Documents Between Companies: EDI, Punchout and E-Invoicing)

Capability	Models	Chapter
Trading partner directory and requirement profiles; onboarding with test interchanges; secure transport (AS2, SFTP, Peppol); translation, validation, routing, status ledger	EDI, ERP, POS	Documents Between Companies: EDI, Punchout and E-Invoicing (Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation)
ORDERS and ORDRSP; DESADV with SSCC; INVOIC and EN 16931 e-invoices with mandates as configuration; archive; web EDI portal; AI-assisted mapping with review	EDI, ERP, POS	Documents Between Companies: EDI, Punchout and E-Invoicing (Journey 3: A Chain Reorders by EDI, From Order to Invoice, Tax, Invoices and the Books: From Order to Ledger)
ERP and back-office connectors; order export; master-data sync; file exchange; accounting export and journal mapping	B2B, MKT, SHP, RET, EDI, ERP, POS	The Back Office: ERP Integration and Who Is the Hub (Tax, Invoices and the Books: From Order to Ledger)

Part VI — Integration

Capability	Models	Chapter
Webhooks; integration runtime with job queue, retries, rate-limit governance and event log; connector registry with credential vault	MKT, SHP, WMS, EDI, RET, POS, HUB, MCP	APIs, Events, Files and Middleware: How Commerce Systems Talk (Black Friday Every Year: Peak Load, Resilience and Recovery, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers)

Capability	Models	Chapter
Event streams for orders, stock, returns and tracking; order events for ERP, marketplaces, shipping and analytics	MCP, ORM, RET, SHP, WMS, ERP, POS, HUB, B2B	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation (The Order Record: Lifecycle, State and the Order Management System)
Public REST or GraphQL API with authentication, throttling and versioning; developer portal	MCP, ORM, MKT, SHP, WMS, RET, EDI, ERP, POS	Promises Between Systems: API Contracts, Webhooks and Versioning
Customer registry and customer file; shopper identity, account graph, entitlements; cross-brand customer registry	POS, MCP, ERP, ORM, HUB	One Customer, One Product: Identity and Master Data Across Systems (Where Is My Order? Notifications, Service and the Customer Record)
Analytics and reporting with a query layer; experimentation and attribution; cross-merchant benchmarks; chain KPIs across a multi-product stack	all	Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics (Whose Data Is It? Privacy, Consent and Retention)

Part VII — Platform

Capability	Models	Chapter
Hosted storefront with themes and content pages; CMS, SEO and sitemaps; headless framework, storefront API, edge hosting	MCP, B2B, ERP, POS	Delivering the Storefront: Themes, Headless, Rendering and Performance (Finding the Product: Search, Navigation and Merchandising)

Capability	Models	Chapter
Multi-tenant accounts: tenants, administrations, branches, shops, warehouses, clients, sub-accounts; multi-shop from one back office; package entitlements and plan metering	all	Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation
App store, review and partner program; sandboxed extensions and functions; no-code workflow automation; embedded white-label partner integrations; app billing and revenue share	MCP, MKT, ERP, RET, WMS, SHP, B2B	Apps, Plugins and Functions: Extensibility and Ecosystem Governance (Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation)
The vendor's own internal structure; hosting with patching and backups	MCP, ORM, B2B	Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says; the hosting duty under shared responsibility in Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers

Part VIII — Trust

Capability	Models	Chapter
Staff accounts, roles, OAuth scopes, API keys, audit of changes; security assurance (management system, penetration tests, incident levels, replicated data centers)	all; EDI, B2B	Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers (Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation)
Consent records; data-sharing policies for third-party apps; anonymization for benchmarks	ORM, MCP, RET	Whose Data Is It? Privacy, Consent and Retention
Fiscal receipts; product-safety attributes on listings; e-invoicing mandates; the withdrawal right; accessibility; recalls and traceability	POS, ERP, MKT, EDI, B2B, ORM, WMS	Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation

Part X — Evolution

Capability	Models	Chapter
Data migration and onboarding tooling with validation	ERP, POS, WMS	Moving Without Burning the Bridge: Replatforming and Migration
Test environments, release management, automated QA	B2B	Who Builds and Who Owns: Teams, Vendors, Agencies and Governance
Implementation, onboarding and support desk sold as part of the product	B2B, RET, EDI, MKT, POS, ERP, WMS	Build, Buy or Assemble: Choosing and Changing Platforms (Who Builds and Who Owns: Teams, Vendors, Agencies and Governance)

Capability	Models	Chapter
Shopper-facing AI assistant; admin assistant acting over store data with approval; agent-facing structured catalog and open commerce protocol	ORM, MCP, RET, EDI	Assistants and Agents: AI in Commerce and What Is Real in 2026 (Finding the Product: Search, Navigation and Merchandising)

Coverage by Model

The same map read from the other side. Brick ids are the models' own keys, kept so a reader of the digest can cross-check; they describe capabilities, not products.

The merchant platform model (MCP)

Root group	Bricks	Chapters
Storefront and buyer experience	`stfr` online store and themes, `hdrg` headless framework, `srch` search, `chkt` checkout and extensibility, `shpy` accelerated checkout and buyer identity, `shop` consumer app	Delivering the Storefront: Themes, Headless, Rendering and Performance, Finding the Product: Search, Navigation and Merchandising, From Cart to Confirmation: Checkout Orchestration, One Customer, One Product: Identity and Master Data Across Systems; the consumer app as a channel in Assistants and Agents: AI in Commerce and What Is Real in 2026

Root group	Bricks	Chapters
Commerce core and operations	`ctlg` catalog, `invt` inventory, `mkts` markets, `b2bc` wholesale, `ordr` orders, `ship` shipping and delivery promise, `rtrn` returns, `posr` POS	What Are We Selling? Products, Variants and the Catalog, Is It in Stock? Inventory, Availability and the Promise You Can Keep, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, The Order Record: Lifecycle, State and the Order Management System, Getting It There: Carriers, Labels, Tracking and the Delivery Promise, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
Money and merchant financial services	`paym` payments, `risk` fraud and chargebacks, `ledg` merchant ledger and payouts, `capl` lending, `taxc` tax and duties, `blng` platform and app billing	Taking the Money: Payment Providers, Authorization, Capture and Refunds, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers, Tax, Invoices and the Books: From Order to Ledger, Apps, Plugins and Functions: Extensibility and Ecosystem Governance; lending left out (below)

Root group	Bricks	Chapters
Growth, channels and AI	`mktg` marketing, `adsn` audience ads, `anly` analytics, `sdkk` admin assistant, `agnt` agentic commerce and global catalog, `chnl` channel sync	Where Is My Order? Notifications, Service and the Customer Record, Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics, Assistants and Agents: AI in Commerce and What Is Real in 2026, Selling on Someone Else's Platform: Marketplaces and Channel Integration; ads mentioned only (below)
Platform and ecosystem	`apis` admin APIs, webhooks and identity, `extn` extensions and workflow automation, `apst` app store and partner program	Promises Between Systems: API Contracts, Webhooks and Versioning, APIs, Events, Files and Middleware: How Commerce Systems Talk, Apps, Plugins and Functions: Extensibility and Ecosystem Governance; the whole model as a case in Case: Shopify, the Modular Monolith That Serves Millions of Merchants and Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation

The retail marketplace model (ORM)

Root group	Bricks	Chapters
Shopper experience: discovery	`home` entry, `sear` search, `rufu` assistant, `pdet` product detail, `revi` reviews, `deal` deals, `reco` recommendations	Finding the Product: Search, Navigation and Merchandising, Discounts, Coupons and Loyalty: Promotions Without Chaos, Assistants and Agents: AI in Commerce and What Is Real in 2026; the case in Case: Amazon, From Monolith to Services to the Everything Store
Shopper experience: conversion and service	`cart`, `chec` checkout and payments, `orde` order tracking, `deli` delivery promise, `retu` returns, `prim` membership, `care` customer care	From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds, The Order Record: Lifecycle, State and the Order Management System, Getting It There: Carriers, Labels, Tracking and the Delivery Promise, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Discounts, Coupons and Loyalty: Promotions Without Chaos, Where Is My Order? Notifications, Service and the Customer Record

Root group	Bricks	Chapters
Seller growth: commercial control	`sell` onboarding, `list` catalog authoring, `bran` brand registry, `price` offer health	Selling on Someone Else's Platform: Marketplaces and Channel Integration (the seller's side), Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation; brand protection left to LISTED (below)
Seller growth: operations and demand	`fbao` marketplace fulfillment, `invn` inventory planning, `supp` seller-fulfilled operations, `payo` payouts, `spon` sponsored ads, `dspm` programmatic audiences, `repr` reporting	Selling on Someone Else's Platform: Marketplaces and Channel Integration, Is It in Stock? Inventory, Availability and the Promise You Can Keep, Tax, Invoices and the Books: From Order to Ledger, Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics; ads mentioned only (below)

Root group	Bricks	Chapters
Marketplace foundations	`iden` identity, `risk` fraud and policy, `auth` listing authenticity, `anal` measurement, `apii` partner APIs	One Customer, One Product: Identity and Master Data Across Systems, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers, Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics, Promises Between Systems: API Contracts, Webhooks and Versioning; the operator's trust function in LISTED's Scams, Fakes and Account Takeovers: Fraud Prevention for Listings

The nine specialist-product models

Shown by capability only: each model's root groups summarized, then the chapters that cover them.

Model	Capabilities	Chapters
B2B webshop platform (B2B)	Storefront, CMS, SEO; catalog and search; company accounts, customer-specific pricing, quotes, reorder; checkout, payments, shipping, promotions, withdrawal; ERP and marketplace integration; building blocks and entitlements, multi-shop, test environments, automated QA, hosting	Delivering the Storefront: Themes, Headless, Rendering and Performance, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, From Cart to Confirmation: Checkout Orchestration, The Back Office: ERP Integration and Who Is the Hub, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance
Returns platform (RET)	Returns portal, country rules, paid returns, labels, tracking; exchanges and store credit; policy engine, automation, claims gate, inspection, resolution, in-store returns; return analytics and benchmarks; platform, ERP, WMS and app connectors	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Journey 2: The Jacket Comes Back, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Where Is My Order? Notifications, Service and the Customer Record, Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics

Model	Capabilities	Chapters
EDI and e-invoicing network (EDI)	Partner directory, profiles, onboarding, AI mapping; transport, translation, validation, routing; orders, despatch advice and SSCC, e-invoicing, archive, vendor managed inventory, web portal; monitoring; ERP and WMS connectors; security assurance	Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice, Tax, Invoices and the Books: From Order to Ledger, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation; VMI left out (below)
Marketplace integrator (MKT)	Connector framework, onboarding, feeds, webshop plugins, back-office connectors, API, embedded partner integrations, app store; catalog hub, selections, content mapping, rules, translation; pricing, repricer, competitor offers; order hub, status sync, stock synchronization, fulfillment manager; reporting, retail-media dashboard, account health; integration runtime	Selling on Someone Else's Platform: Marketplaces and Channel Integration, What Are We Selling? Products, Variants and the Catalog, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Is It in Stock? Inventory, Availability and the Promise You Can Keep, APIs, Events, Files and Middleware: How Commerce Systems Talk, Promises Between Systems: API Contracts, Webhooks and Versioning, Apps, Plugins and Functions: Extensibility and Ecosystem Governance

Model	Capabilities	Chapters
Commerce ERP (ERP)	Webshop and B2B portal, checkout and payments, cloud POS, marketplace bridge, EDI gateway; PIM, pricing, supplier import; quote-to-invoice orders, CRM, returns and repair; multi-warehouse stock, picking, purchasing, shipping, route planning; accounting and BI; tenancy, API hub, migration, workflow automation	The Back Office: ERP Integration and Who Is the Hub, What Are We Selling? Products, Variants and the Catalog, The Order Record: Lifecycle, State and the Order Management System, Tax, Invoices and the Books: From Order to Ledger, Pick, Pack, Ship: Warehouse Management and Fulfillment, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Moving Without Burning the Bridge: Replatforming and Migration; CRM pipeline, repair and route planning left out (below)

Model	Capabilities	Chapters
Store POS and management (POS)	Till, tenders, receipts and day closing, hardware; customer registry, loyalty, customer orders, invoicing, repair; article master, pricing, chain control, stock ledger, stocktaking, purchasing, supplier ordering; store webshop, online order hub, marketplace, shipping and returns connectors; reporting, accounting export, tenancy, onboarding	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Discounts, Coupons and Loyalty: Promotions Without Chaos, What Are We Selling? Products, Variants and the Catalog, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Is It in Stock? Inventory, Availability and the Promise You Can Keep, Selling on Someone Else's Platform: Marketplaces and Channel Integration, Tax, Invoices and the Books: From Order to Ledger, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation

Model	Capabilities	Chapters
Shipping platform (SHP)	Order, ERP and WMS connectors, API, hub bridges; dynamic checkout, stock-aware promise, pickup points; rules, carrier selection, rate cards; booking, documents, scan-to-ship, routing; carrier network, tracking hub; branded communication, exceptions, returns shipping; invoice control, dashboards, CO2; address validation	Getting It There: Carriers, Labels, Tracking and the Delivery Promise, From Cart to Confirmation: Checkout Orchestration, Where Is My Order? Notifications, Service and the Customer Record, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Promises Between Systems: API Contracts, Webhooks and Versioning; CO2 reporting left out (below)
Cloud WMS (WMS)	Store and marketplace connectors, order allocation, B2B portal, API, hub bridges; product master, inventory ledger, locations, cycle counting, lot traceability, purchasing, receiving; picking, packing, shipping, printing, returns; rules, mobile app, collaboration, reports; 3PL client separation, portal and invoicing; plans	Pick, Pack, Ship: Warehouse Management and Fulfillment, Is It in Stock? Inventory, Availability and the Promise You Can Keep, The Order Record: Lifecycle, State and the Order Management System, APIs, Events, Files and Middleware: How Commerce Systems Talk, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation; lot traceability and collaboration thin (below)

Model	Capabilities	Chapters
Commerce hub (HUB)	One brick per sibling product plus a connection registry with cross-platform event routing and a shared customer view with chain KPIs	The shape of a best-of-breed stack in The Map of a Commerce Platform: Capabilities and Layers and Case: commercetools, Shopware and the Composable Movement; the registry in APIs, Events, Files and Middleware: How Commerce Systems Talk; the shared customer view in One Customer, One Product: Identity and Master Data Across Systems

Is Anything Uncovered?

The digest flagged eleven of its 152 capabilities as having no owning chapter contract. Checking the written chapters moved three into a chapter, left the rest where the digest proposed or out of the book, and found six more thin spots.

Capability	Models	Where it stands	Why
Reviews, ratings and Q&A	ORM, ERP	**Covered** in Finding the Product: Search, Navigation and Merchandising as discovery and trust data; the fake-review rule in Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation	Reviews are the first user-generated data a merchant must moderate
Paid membership programs	ORM	**Covered briefly** in Discounts, Coupons and Loyalty: Promotions Without Chaos as a subscription the promotion engine reads as eligibility; the model in Who Sells What to Whom: Business Models and the Economics That Shape Architecture	For a merchant it is a customer attribute with dates and a liability; the operator's side is Amazon's

Capability	Models	Where it stands	Why
Fiscalization of receipts	POS, ERP	**Covered** in Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment as a design constraint and in Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation	A receipt is a legal document in several countries
Retail media and sponsored listings	ORM, MKT, MCP	**Mentioned only**, in Selling on Someone Else's Platform: Marketplaces and Channel Integration, as a cost line and a data stream that never touches the order flow	A revenue line for the operator, a marketing cost for the merchant; neither changes the merchant's architecture
CRM and sales pipeline	ERP	**Mentioned only**: the CRM as a candidate customer master in One Customer, One Product: Identity and Master Data Across Systems	Leads and activities are sales tooling; the book covers the customer record, not the selling process

Capability	Models	Where it stands	Why
Merchant capital lending	MCP	**Left out**	Financial services, not commerce architecture; the platform revenue models behind it are in Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026
Emissions estimates and reporting on deliveries	SHP	**Left out**	A versioned fact per shipment, like a cost; the selection criteria it would join are in Getting It There: Carriers, Labels, Tracking and the Delivery Promise, but the reporting duty was not stable enough at the research cut-off to teach as a requirement
Route planning for own-fleet delivery	ERP	**Left out**	A logistics subject with its own software; Kestrel & Pine ships with carriers

Capability	Models	Where it stands	Why
Repair and workshop management	POS, ERP	**Left out**; repair appears only as a disposition in The Way Back: Returns, Exchanges, Refunds and Reverse Logistics	A service business with its own orders, parts and labor; specialty retail needs it, an apparel brand does not
Vendor managed inventory	EDI	**Left out**; its documents ride on Documents Between Companies: EDI, Punchout and E-Invoicing	Collaboration documents beyond ORDERS, DESADV and INVOIC are rare at the book's scale stages
Collaboration on operational records	WMS	**Left out**	A feature of the warehouse system as a shared workspace; it changes no boundary or contract
Batch, lot and best-before traceability	WMS	**Thin**: recalls in Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation; not modeled in Pick, Pack, Ship: Warehouse Management and Fulfillment	Kestrel & Pine sells apparel without lots; food, cosmetics and pharma readers should treat the lot as a dimension of the stock ledger

Capability	Models	Where it stands	Why
Dropshipping and forwarding orders to suppliers	ERP, SHP	**Left out**; the routing decision in The Order Record: Lifecycle, State and the Order Management System is where it would attach	A fulfillment location the merchant does not control; the running example keeps one warehouse, six stores and a marketplace program
Demand forecasting and replenishment planning	ORM, WMS, ERP, POS	**Left to the ERP**, named as its duty in The Back Office: ERP Integration and Who Is the Hub	Planning is a supply-chain discipline; the commerce platform consumes its result
Merchant onboarding and KYC for payments; seller verification	MCP, ORM	**Thin**: trader traceability in Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation; the provider's contract in Taking the Money: Payment Providers, Authorization, Capture and Refunds	The provider's and the operator's obligation; the merchant meets it as a contract, not a system

Capability	Models	Where it stands	Why
Brand registry, counterfeit defense, listing authenticity	ORM	**Left to LISTED**: Scams, Fakes and Account Takeovers: Fraud Prevention for Listings, Rules of the Marketplace: DSA, DAC7 and Vertical Regulation	The operator's trust function; this book takes the seller's side of a marketplace
Hosting, patching and backups	B2B	**Covered as a duty, not a design**, under shared responsibility in Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers	On a hosted platform these are the vendor's; the merchant's job is to know which, and to check

Where the Book Departs From the Models

The models were drawn one product at a time, from what each product visibly does. That gives them a shape the book deliberately does not copy.

The models as drawn	The book's position	Where argued
No integration layer as a subject: every model carries connectors inside its own bricks; the hub's bridges are the nearest thing to one	Integration is a discipline with its own styles, failure modes, contracts and inventory: Part VI	Part VI — INTEGRATION: Making Separate Systems Behave Like One, APIs, Events, Files and Middleware: How Commerce Systems Talk, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Promises Between Systems: API Contracts, Webhooks and Versioning
No scale-stage view: each model fixes one customer size as a persona	The right answer changes with the size and shape of the business, so every chapter in Parts II–VIII compares four stages	What Changes at Each Scale: From First Store to Group, Who Sells What to Whom: Business Models and the Economics That Shape Architecture
Stress tests instead of failure patterns: shocks (a carrier changes its API, a mandate arrives) and the residue that survives them	Sixteen recurring failures named by their warning signs, because a newcomer meets the failure before the shock. The models' stressors cluster on The Long Fuse, The Pulled Rug, The Lynchpin and The Secret Agent and say little about Schrödinger's Order, The Familiar Stranger, The Burned Bridge or One Size Fits None; the Kestrel & Pine incidents supply those	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong

The models as drawn	The book's position	Where argued
Each model walks its own product's journey; none follows one order across products	Part IX follows one consumer order, one return, one wholesale order and one peak day through every system	Part IX — JOURNEYS: One Order, End to End, Journey 1: Eva's Jacket, From Search to Doorstep, Journey 3: A Chain Reorders by EDI, From Order to Invoice, Journey 4: Black Friday, Hour by Hour
Nothing about choosing, changing or organizing: a product as sold, not how a merchant decides to buy, leave or staff it	Part X: build, buy or assemble; migration; scale stages; teams and vendors; AI readiness	Part X — EVOLUTION: Decisions That Change With Scale, Build, Buy or Assemble: Choosing and Changing Platforms, Moving Without Burning the Bridge: Replatforming and Migration, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance
An order record in seven models (platform, webshop, ERP, POS, integrator, WMS, shipping)	One order record with one state model; every other order is a copy with a declared purpose	Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, The Order Record: Lifecycle, State and the Order Management System
A stock ledger in five models	The warehouse holds the ledger; channels hold snapshots with buffers; one writer per fact	Is It in Stock? Inventory, Availability and the Promise You Can Keep
Checkout and payments as one brick (ERP, ORM)	Checkout is an orchestration under a clock; payment is a state machine whose truth lives at the provider	From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds

The models as drawn	The book's position	Where argued
Fraud as part of payments (MCP)	Card testing, account takeover and skimming are security; chargebacks stay with payments	Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers, Taking the Money: Payment Providers, Authorization, Capture and Refunds
Pricing spread over five models, each with its own rules	One price source, every channel a reader of it; the price as a promise with a history	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
Returns as a product (RET)	Returns as a flow across the portal, the carrier, the warehouse, the payment provider and the books	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Journey 2: The Jacket Comes Back
Regulation as stressors inside each product	A regulatory calendar with an owner per row, read as requirements with dates	Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation

The Models' Personas and the Book's Readers

The eleven models carry some sixty modeled customers. They cluster into eleven roles; each maps to a reader the book is written for and to a seat at Kestrel & Pine where that reader can watch the role at work. The personas are modeled archetypes, not real people or accounts.

Persona cluster (models)	What the models say they want	Reader	Kestrel & Pine seat	Start with
Owner-director of a trading company or brand (B2B, ERP, HUB, POS, WMS, MCP)	Growth without back-office headcount; no rebuild in three years; predictable cost	Leader	Mara Lindqvist, CEO; Teun Baas, product and brand	Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Build, Buy or Assemble: Choosing and Changing Platforms, What Changes at Each Scale: From First Store to Group
E-commerce and channel manager (B2B, ERP, HUB, MKT, POS, MCP)	Content correct everywhere; no rejected listings; conversion on peak days	Product manager	Milan Petrov, e-commerce and marketplaces	What Are We Selling? Products, Variants and the Catalog, Selling on Someone Else's Platform: Marketplaces and Channel Integration, From Cart to Confirmation: Checkout Orchestration

Persona cluster (models)	What the models say they want	Reader	Kestrel & Pine seat	Start with
Operations and warehouse lead (HUB, ERP, MKT, SHP, WMS, RET)	Stock true in every system; cut-offs met at peak; fast, accurate returns	Engineer or product manager in operations	Daniel Okafor, operations	Is It in Stock? Inventory, Availability and the Promise You Can Keep, Pick, Pack, Ship: Warehouse Management and Fulfillment, Journey 4: Black Friday, Hour by Hour
IT, ERP and integration lead (B2B, EDI, HUB, MKT)	Few, stable integrations; changes that do not break the live chain; clear errors and monitoring	Engineer	Noor Visser, engineering; Ruben Alves, integration	APIs, Events, Files and Middleware: How Commerce Systems Talk, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Promises Between Systems: API Contracts, Webhooks and Versioning

Persona cluster (models)	What the models say they want	Reader	Kestrel & Pine seat	Start with
Finance (EDI, ERP, HUB)	Month close without reconciliation; invoices paid first time; true cost per order	Analyst or leader in finance	Sanne de Wit, finance	Tax, Invoices and the Books: From Order to Ledger, The Back Office: ERP Integration and Who Is the Hub, Documents Between Companies: EDI, Punchout and E-Invoicing
Trade buyer and wholesale account manager (B2B, HUB, ERP)	Agreed prices and assortments; fast reorder; delivery certainty	Product manager or consultant on B2B	Lea Moreau, wholesale; Pieter Smit, Bergzicht	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, Journey 3: A Chain Reorders by EDI, From Order to Invoice

Persona cluster (models)	What the models say they want	Reader	Kestrel & Pine seat	Start with
Shopper and parcel recipient (MCP, ORM, RET, SHP, HUB, POS)	One-tap payment; honest stock; a delivery moment; easy returns	Every reader, as the person the promises are made to	Eva Janssen	Follow the Order: What Happens Between "Buy" and "Delivered", Journey 1: Eva's Jacket, From Search to Doorstep, Journey 2: The Jacket Comes Back
Data and merchandising analyst (RET, ORM, MKT)	Which products and reasons drive returns; whether spend is incremental; benchmarks	Analyst	Hanna Berg, data	Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics

Persona cluster (models)	What the models say they want	Reader	Kestrel & Pine seat	Start with
Software, implementation and agency partner (MCP, MKT, SHP, WMS, EDI, HUB)	One API instead of fifty; predictable deprecations; billing without building payments	Consultant or engineer at a vendor or agency	The agency and the ERP partner in the ledger	Promises Between Systems: API Contracts, Webhooks and Versioning, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance

Persona cluster (models)	What the models say they want	Reader	Kestrel & Pine seat	Start with
Marketplace trust and resolution operator (ORM)	Catch fraud and abuse before reports; see at-risk promises early	Consultant or analyst on the operator's side	None; the book takes the seller's side	Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Where Is My Order? Notifications, Service and the Customer Record; LISTED's Scams, Fakes and Account Takeovers: Fraud Prevention for Listings
Counter staff, pickers and packers (POS, WMS)	A till that is never slow; know where to go and what to take; not blamed for mistakes	Engineer or product manager designing floor tools	The Duiven warehouse and the six stores	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Pick, Pack, Ship: Warehouse Management and Fulfillment

Two things the persona lists show. The same four wants recur in nearly every model: stock true in every system, orders that arrive without retyping, changes that do not

break the live chain, and a month close without reconciliation; those are the Truth and Integration families of Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, seen from the customer's chair. And no model has a persona for the **architect**, the role the book speaks in: the models describe what each product's customer wants and leave how the products should fit together to whoever buys them. That is the gap the book is written into.

How to Use This Map

1. **Match by meaning, not by name.** For each system you own, find the row that does the same job and read its primary chapter first.
2. **Read the departures before you copy a decomposition.** An order record in seven systems or a stock ledger in five is not a design; it is the models' shape, and the departures table names the chapters that explain what to do about it.
3. **Check the left-out list against your business.** If you sell food, pharma or cosmetics, lots are not optional; if you run a workshop or your own vans, the book leaves you a sidebar to write.
4. **Use the persona row as a reading route.** Find your seat at Kestrel & Pine and start with the chapters in its row.

The rest of the Reference section turns the map into working tools: Toolkit: Cards and Templates for Commerce Architecture Decisions holds the cards and templates the chapters introduce (the capability map, the entity–owner table, the integration inventory, the reconciliation card and the rest), Glossary: Plain-Language Definitions defines the terms each chapter owns, and Bibliography: Every Source the Book Cites lists every source the chapters cite. Continue with Toolkit: Cards and Templates for Commerce Architecture Decisions.

Sources and Evidence

Company, product and investor sources

- None are cited directly on this page. Facts about Shopify, Amazon and other companies are made, with dated sources, in the chapters and cases the tables point to.

Standards, regulation and official documentation

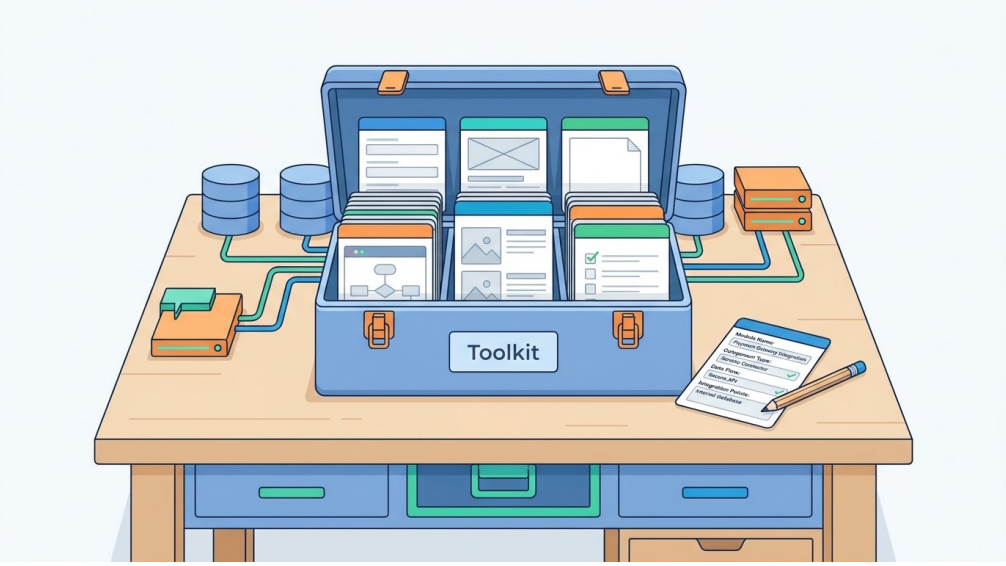
- None are cited directly on this page; regulations and standards are cited in the chapters that rely on them.

Research and practitioner sources

- *Merchant Commerce Platform* Productscape domain model (`productscape-examples/_config/product-domains/platforms/merchant-commerce-platform/`), the author, 2026: 29 bricks in five root groups, inferred from the public behavior of a hosted platform of the Shopify kind. A research map; not a description of any company's architecture.
- *Online Retail Marketplace* Productscape domain model (`productscape-examples/_config/product-domains/marketplaces/online-retail-marketplace/`), the author, 2026: 30 bricks in three root groups, inferred from the public behavior of a marketplace of the Amazon kind. A research map; not a description of any company's architecture.
- The nine *Katapull Group* Productscape domain models (`productscape-examples/_config/product-domains/katapull-group/`), the author, 2026: capability inventories of nine specialist commerce products, authored from public pages. Used for their bricks, data entities and roles only; nothing about any company's plans, finances or strategy is used.
- Productscape digest for this book (`_research/productscape-digest.md`), 7 November 2026: the model-by-model inventory (§1), the cross-model capability inventory with its GAP list (§2), data assets and chapter ownership (§3), personas (§4) and the residuality notes mapped to the failure patterns (§5).
- Book plan and chapter contracts (`PLAN.md`), 7 November 2026: the "Owns" field of each contract and the Reference entry for this page.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Toolkit: Cards and Templates for Commerce Architecture Decisions



IN THIS SECTION, YOU WILL: Find the card that fits the question on your desk, copy its fields, and see it filled in once for Kestrel & Pine, with the chapter that explains every field.

KEY POINTS:

- **Every card names an owner, a system of record and a date.** Those three fields are what turn a diagram into a decision: who is called when it breaks, which system wins when two disagree, and when the card must be looked at again.
- **The cards point at each other.** The capability map says which boxes exist, the entity–owner table says who owns each fact, the integration inventory lists every flow that moves a fact, and each flow has a contract card and a reconciliation card. A decision record changes a row in one of them; nothing else should.
- **Most fields were paid for by an incident.** The writer column came from the App That Wrote Stock, the sunset date from the Deprecated

> Endpoint, the SSCC source from the ASN Chargeback, the summary rule from the Three-Status Order. Write "unknown" rather than guess.

These are **working templates for a merchant whose order passes through more systems than one team can hold in its head**: the records the chapters ask you to keep, from the capability map of The Map of a Commerce Platform: Capabilities and Layers to the decision record of Who Builds and Who Owns: Teams, Vendors, Agencies and Governance. Each is the smallest form I have found that makes a design decision explicit, gives it an owner and leaves evidence that someone else can check. The chapters linked from each card give the reasoning and sources behind every field.

Each card is filled in once for **Kestrel & Pine, the fictional outdoor brand this book follows** (Meet Kestrel & Pine). Every Kestrel & Pine name, figure and date below is invented for teaching and agrees with the chapter that works the full case.

Three conventions run through all sixteen cards.

- **They are kept data, not documents.** A card lives where the people who change the system will see it, in one repository, not in a slide deck.
- **They cite each other by identifier.** A decision record names the entity–owner rows it changes; a contract card names its inventory row.
- **Blank is a finding.** An empty owner cell or a missing sunset date is where the next incident will come from.

Find the Template for the Question on Your Desk

The question in front of you	Tool	Chapter that teaches it
What must the business be able to do, and which system does each job?	Tool 1: Capability map	The Map of a Commerce Platform: Capabilities and Layers
Two systems disagree about a price, a stock level or an order. Which one is right?	Tool 2: Entity–owner table	Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record
How many flows connect our systems, and who owns each one?	Tool 3: Integration inventory	APIs, Events, Files and Middleware: How Commerce Systems Talk

The question in front of you	Tool	Chapter that teaches it
A flow is about to go live, or a vendor has announced a new API version	Tool 4: Integration contract card	Promises Between Systems: API Contracts, Webhooks and Versioning
How do we know a flow actually worked?	Tool 5: Reconciliation card	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation
What may each channel sell, and where does that number come from?	Tool 6: Availability (ATP) card	Is It in Stock? Inventory, Availability and the Promise You Can Keep
Support sees three different statuses for one order	Tool 7: Order state card	The Order Record: Lifecycle, State and the Order Management System
What delivery date may the checkout promise?	Tool 8: Delivery promise card	Getting It There: Carriers, Labels, Tracking and the Delivery Promise
How much stock should each marketplace be told about?	Tool 9: Channel allocation card	Selling on Someone Else's Platform: Marketplaces and Channel Integration
A trading partner orders by EDI or deducts money for a bad despatch advice	Tool 10: EDI partner card	Documents Between Companies: EDI, Punchout and E-Invoicing
Someone wants to install an app, or an app writes data it should not	Tool 11: App governance card	Apps, Plugins and Functions: Extensibility and Ecosystem Governance
Black Friday is coming	Tool 12: Peak plan and degradation ladder	Black Friday Every Year: Peak Load, Resilience and Recovery
A law changes what a field, an invoice or a checkout must do	Tool 13: Regulatory calendar	Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation

The question in front of you	Tool	Chapter that teaches it
Should we build, buy or assemble a capability, or replatform?	Tool 14: Build, buy or assemble decision card	Build, Buy or Assemble: Choosing and Changing Platforms
A system of record is moving from one system to another	Tool 15: Migration plan card	Moving Without Burning the Bridge: Replatforming and Migration
A decision changes who owns a fact, which vendor we depend on or which app may write	Tool 16: Architecture decision record	Who Builds and Who Owns: Teams, Vendors, Agencies and Governance

Two related records are not repeated here because their chapters give them in full: the ownership map and vendor card of Who Builds and Who Owns: Teams, Vendors, Agencies and Governance, and the W7 metric definitions of Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.

How the Records Connect

Most changes touch several cards; they meet at the entity–owner table.

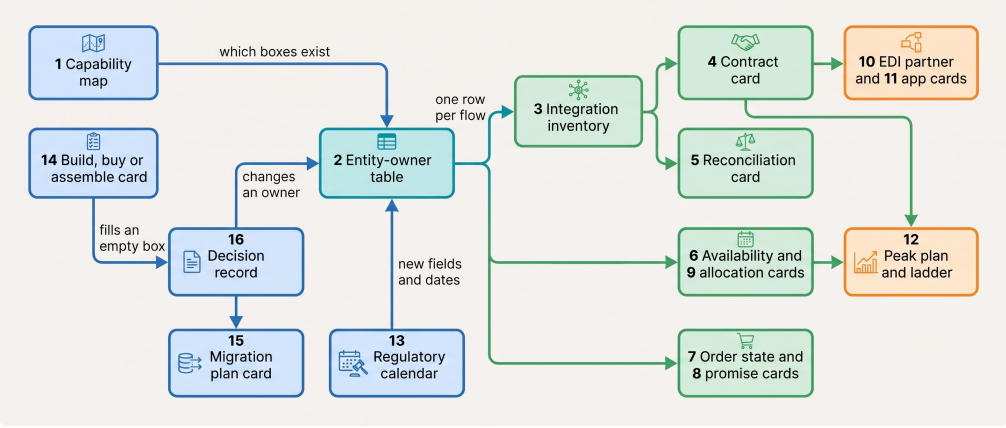


Figure 1: *The capability map says which boxes exist, decision records change who owns each fact, and every flow, partner, app, peak plan and migration is written against the entity–owner table, so a change to one row tells you which other cards to update.*

In plain words: start at the entity–owner table. A fact with no row, or a row with no owner, is the problem.

Tool 1: Capability Map

Purpose. A capability map lists what the business must be able to do, such as "compute what each channel may sell", independent of which system does it today. Placing your systems on it shows the empty boxes (jobs no system owns) and the crowded ones, where incidents start.

The template. One row per capability, grouped in six layers: experience, offer, transaction, operations, partners and back office, foundations.

Field	What to record
Layer and capability	A business ability as a verb phrase ("hold every order in one state model"), never a system, team or vendor
The fact it is the natural home for	The business fact this capability should own; mark computed facts such as availability and the delivery promise
Systems today	Every system that does any part of the job; two or more is an overlap, none is an empty box
Writers	Every system or person that changes the fact
Status and evidence	Covered, overlap or empty, with the incident or number behind it
Owner	One named person accountable for the capability

How to use it. Draw it before reading any vendor's diagram; review it yearly and after any new channel or country.

Kestrel & Pine example (fictional; Noor Visser's first Waypoint working document, February 2027):

Field	Inventory and availability
Fact it is the home for	Available-to-promise per channel, derived from physical stock
Systems today	Stock on hand in the WMS, the ERP, the platform and the POS; **nobody computes availability**
Writers	Six writers of Duiven stock: the WMS feed, manual ERP entry, manual platform entry and three apps; the POS writes store stock separately
Status and evidence	**Empty box** for availability, overlap for stock; the Black Friday Oversell, 1,340 units oversold on 27 November 2026
Owner	Daniel Okafor for stock; Ruben Alves for the availability service once W1 delivers it

The whole map had about two dozen capabilities with thirteen systems placed on them and four empty boxes: product information, order management, availability and customer identity.

Used in: The Map of a Commerce Platform: Capabilities and Layers, Build, Buy or Assemble: Choosing and Changing Platforms, What Changes at Each Scale: From First Store to Group, Domain Coverage: From Three Model Families to Forty-Nine Chapters.

Tool 2: Entity–Owner Table

Purpose. The entity–owner table names, for each business fact, the one **system of record** (the system whose value is the truth when copies disagree), who may write it there, where copies may live and how disagreement is found. It turns "which system is right?" into a lookup.

The template. One row per fact; stock and customers usually need more than one row.

Field	What to record
Fact	A business fact with a lifecycle: product content, consumer price, physical stock per location, order, payment, return, invoice, customer
System of record	The one system whose value wins; per location where the fact is physical
Who may write	People, scans or processes allowed to change it in that system, and only there
Copies live in	Every system that holds a copy

Field	What to record
How copies learn	Event on change, scheduled publication or daily batch, chosen by what a stale copy costs
Disagreement found by	The reconciliation that compares record and copy (Tool 5), with its cadence

How to use it. Write it before buying any hub. Then run a **write audit** each quarter: list every credential, app scope and job that can change each fact, and compare it with the "who may write" cell.

Kestrel & Pine example (fictional; the review of early March 2027, adopted by W1–W3 and the April 2027 price decision):

Fact	System of record	Who may write	How copies learn	Disagreement found by
Physical stock, Duiven	The WMS	Scans, counts and receipts in the WMS only	Stock events (from Q4 2027)	WMS-to-platform reconciliation, weekly; daily in November
Consumer price	The platform's market price lists (NL, BE, DE)	Pricing owner, with effective dates	Price events	Daily price reconciliation across all copies, from May 2027
Order	The OMS (W2, Q4 2027)	Order creation from channels; state changes from events	Order events	Nightly order reconciliation

Used in: Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, Is It in Stock? Inventory, Availability and the Promise You Can Keep, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, What Are We Selling? Products, Variants and the Catalog, One Customer, One Product: Identity and Master Data Across Systems, The Back Office: ERP Integration and Who Is the Hub.

Tool 3: Integration Inventory

Purpose. The integration inventory is one list of every flow that moves data between two systems, with its owner, style and failure handling. It makes **integration debt** (flows whose owner, contract version, failure alert or reconciliation are unknown) visible as a list of rows rather than a feeling.

The template. One row per flow.

Column	What goes in it
Source → target, and what moves	"WMS → ERP: stock on hand per SKU" (a SKU is one sellable variant)
Style and trigger	File, shared database, direct API call or message; scheduled, on event, on request
System of record	Which side wins, from Tool 2
Owner	A named person, not a team or a vendor
Contract version and sunset date	The API version or file format pinned, and when the vendor retires it
Volume and expected rate	Per day, and per minute at peak
Failure handling	Retries, dead-letter queue (a holding queue for messages that failed every retry), alert
Reconciliation	Card reference (Tool 5), or "none"

How to use it. The rule I would adopt anywhere: **no flow goes live without a row, and no row without an owner**. Review it quarterly; a flow whose owner has left is an incident, not a note.

Kestrel & Pine example (fictional; Noor's inventory of February 2027): 34 integrations, only 3 with a reconciliation job. One row:

Column	Value
Source → target	WMS → ERP: stock on hand per SKU
Style and trigger	Scheduled, every 15 minutes, through the iPaaS
System of record	The WMS (W1)
Owner	Ruben Alves
Failure handling	No dead-letter queue

Column	Value
Reconciliation	None; the first stock reconciliation ran in May 2027

Used in: APIs, Events, Files and Middleware: How Commerce Systems Talk, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance, Moving Without Burning the Bridge: Replatforming and Migration, The Back Office: ERP Integration and Who Is the Hub.

Tool 4: Integration Contract Card

Purpose. A contract card is a short written agreement for one flow: what moves, in which direction, how often, which system wins, what happens on a repeat or a failure, and which version of the other side's interface it depends on. It gives every inventory row (Tool 3) a person, a version, a date, an alarm and a check.

The template.

Field	What to record
Flow and owners	Inventory row; source, target, what moves; business and technical owner
Style	Webhook, scheduled push, event, batch, synchronous call
Contract	The interface pinned (API version, message type, file format) and where its changelog is read
Sunset of the pinned version	The vendor's retirement date, and a calendar entry three months ahead
System of record	From Tool 2
Idempotency	The key that makes a repeated message harmless (order number, return id, event id)
Failure handling	Alert time, paging rule, dead-letter queue and its owner; "never exits silently"
Reconciliation	Reference to the reconciliation card (Tool 5)
Expected rate	Volume per run or per minute, and the vendor's rate limit it must fit

How to use it. Write the card before the flow goes live. **Idempotent** handling means a message received twice has the same effect as once; if the card cannot name the key, the first retry will create a duplicate.

Kestrel & Pine example (fictional; the stock push to bol, rewritten by Ruben Alves after the Deprecated Endpoint, in use from March 2027):

Field	Value
Flow	Stock levels, platform to marketplace connector, bol listings
Owner	Milan Petrov (business), Ruben Alves (technical)
Style	Scheduled API push, hourly, until the availability service's events replace it in Q4 2027 (W1)
Contract	The connector's stock API, pinned to its current release; changelog read monthly from the integrations role mailbox
Sunset	Recorded in the inventory; calendar entry three months ahead, assigned to the owner
Idempotency	Last write wins per SKU and location; each push carries its snapshot time so an older push cannot overwrite a newer one
Failure handling	Alert within fifteen minutes; two failed runs in a row page the owner
Reconciliation	Daily comparison of the connector's stock per SKU with the platform's
Expected rate	About 17,000 variants per run, in batches within the connector's limit

It replaced a 2021 agency script whose API version was retired on 8 February 2027, leaving bol with stale stock for two days.

Used in: Promises Between Systems: API Contracts, Webhooks and Versioning, APIs, Events, Files and Middleware: How Commerce Systems Talk, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance, Discounts, Coupons and Loyalty: Promotions Without Chaos, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics.

Tool 5: Reconciliation Card

Purpose. Reconciliation is a job that compares two systems' records on a schedule and reports or repairs the differences. Retries, keys and events reduce disagreement; only reconciliation proves there is none, so **no flow that creates, moves or pays should go live without a card**.

The template.

Field	What to record
What two sets are compared	The record and the copy, and the scope ("all active variants")
Matching key	SKU and location, order number, order number and amount, payout line id
Tolerance	Zero, or a stated threshold, and why
Cadence	How often, and how it changes in peak months
Who gets the exceptions	A named person or team
The fix	Which side is corrected (always the copy, from the system of record) and how the cause is hunted

How to use it. Let the system of record decide the direction of the fix: the copy is overwritten, then the writer that caused the difference is found and removed. A zero tolerance is a choice; for stock it is zero because one wrong variant at peak is one oversell.

Kestrel & Pine example (fictional; the stock reconciliation from May 2027):

Field	Value
Compared	WMS on hand minus reservations vs platform available, per active variant
Matching key	SKU and location
Tolerance	Zero per variant
Cadence	Weekly, Sunday 03:00; daily in November
Exceptions to	Ruben Alves's team and Milan Petrov
The fix	Overwrite the platform from the WMS (W1); find and remove the writer
First result	**2.3%** of active variants, about 390 of 17,000: about 240 from the three apps that wrote stock, about 90 in-flight returns, about 40 manual edits in the platform admin, about 20 unexplained

Used in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Taking the Money: Payment Providers, Authorization, Capture and Refunds, Tax, Invoices and the Books: From Order to Ledger, Black Friday Every Year: Peak Load, Resilience and Recovery, Moving Without Burning the Bridge: Replatforming and Migration.

Tool 6: Availability (ATP) Card

Purpose. **Available-to-promise** (ATP) is the number a channel may still sell: on hand, minus units committed to placed orders, minus reservations, minus a deliberate buffer. The card records, per channel, who supplies each term of that sum and what happens when the computation is down.

The template. One card per channel, or one table with a column per channel.

Field	What to record
Channel	Storefront NL, BE, DE; each marketplace; B2B portal; click-and-collect per store
Locations it may sell from	Warehouse, stores, external locations
On hand from	The counting system per location (WMS, POS, marketplace report)
Committed from	The order-taking system (platform, connector, later the OMS)
Buffer	Units held back per SKU for this channel, and who sets it
Publication	Event within seconds, or a schedule, and the measured lag
Checkout rule	Synchronous check and atomic commit at order creation, or not
If the computation is down	The degraded answer: cached value, wider buffer, "call the store"
Reconciliation	Tool 5 reference

How to use it. The warehouse owns on hand; the order-taking system owns committed; **nobody owns "available" by hand**, because it is a computation. If a person or an app can type a number into it, the card has a second writer.

Kestrel & Pine example (fictional; Ridgewalker shell jacket, women's M, moss, NL storefront, 6 April 2027, before W1 is live):

Field	Value
Location	Duiven
On hand	36, from the WMS
Committed	12 to Bergzicht Outdoor's EDI reorder
Buffer	1 unit for the storefronts (3 for marketplaces)
Available for NL	**23**

Field	Value
Publication today	The hourly chain (WMS → ERP every 15 minutes → platform hourly → connector hourly); events to the availability service from Q4 2027

The same 23 is what Eva Janssen's storefront showed on 13 April 2027.

Used in: Is It in Stock? Inventory, Availability and the Promise You Can Keep, From Cart to Confirmation: Checkout Orchestration, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Journey 1: Eva's Jacket, From Search to Doorstep, Journey 3: A Chain Reorders by EDI, From Order to Invoice.

Tool 7: Order State Card

Purpose. An order is not one state machine but several: the order's lifecycle, the money, the goods on each shipment, and each return. The card writes down which machines exist, where each lives, which system moves it, and how the one status a customer sees is computed from them, so that three systems cannot each publish a different truth.

The template.

Field	What to record
Order number family	One sequence across channels; the channel's own id kept as a reference
Machines	Lifecycle, money, goods (per fulfillment order), return; their states
Who moves each machine	The order system, the PSP's events, the warehouse and carrier, the returns portal and warehouse
Customer-facing summary rule	How "Shipped" or "Delivered" is computed; for a split order, the least-advanced shipment
Split rule	When to split a shipment and when to wait; the message the customer gets
Invoice trigger	Per shipment or per order; one finance decision, recorded once

How to use it. "Fulfilled" is not "delivered" unless the carrier's events are fed back, and a two-parcel order is delivered only when both parcels are.

Kestrel & Pine example (fictional; order #1052877, December 2026, as the OMS handles it under W2):

Field	Value
Order	Trailhead fleece €119 and merino socks €18, €137, paid in full
What went wrong	The fleece shipped 7 December and was delivered 8 December; the socks shipped 14 December. Platform *Delivered*, WMS *In picking*, ERP *Not invoiced*; 11 tickets
Machines under W2	Separate money, goods (per fulfillment order) and return machines in the OMS
Summary rule	Least-advanced shipment: "Partially delivered: 1 of 2 parcels"
Split rule	Two-line D2C order under €150: wait up to one working day for replenishment, then split and emit "ships in two parcels"
Invoice trigger	A "shipment to invoice" event per shipment, idempotent on the shipment id, or one invoice at completion if Finance prefers

Used in: The Order Record: Lifecycle, State and the Order Management System, Where Is My Order? Notifications, Service and the Customer Record, Tax, Invoices and the Books: From Order to Ledger, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Journey 2: The Jacket Comes Back.

Tool 8: Delivery Promise Card

Purpose. The delivery promise is the date the checkout shows. A static sentence ("ordered before 22:00, delivered tomorrow") is a rule with no data behind it; a computed promise is an estimate from the cut-off, the backlog, the stock location and the carrier's measured transit. The card records the inputs and how the promise is checked against what actually happened.

The template. One row per lane (warehouse, carrier service, destination area).

Field	What to record
Lane	Ship-from location, carrier and service, destination area
Cut-off	The last pick wave or carrier collection that still ships today
Backlog input	The age of the oldest order not yet in the warehouse, and the threshold that moves the ship date
Transit	Working days for the lane
Margin	Extra time for the lane's late tail, from measured performance
Measured	Promised versus delivered per lane, and how often it is reviewed
Owner	The person who owns the warehouse's plan, not the theme

How to use it. Start with a transit table and the warehouse's real cut-off, and measure promised against delivered weekly before adding any model.

Kestrel & Pine example (fictional; Eva's order #1072455, Tuesday 13 April 2027, 21:14):

Field	Value
Lane	Duiven → Nijmegen, PostNL standard home delivery
Cut-off	Last pick wave 21:00; the theme's "before 22:00" dated from the Tiel 3PL era
Backlog	None that evening
Transit	One working day; reliable lane, small margin
Static promise	Wednesday 14 April, a day early
Computed promise	**Thursday 15 April**, the day the parcel arrived
Plan	First promise service May 2027, measured weekly per lane; promise API under W8 in 2028
Owner	Daniel Okafor

Used in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise, Pick, Pack, Ship: Warehouse Management and Fulfillment, Journey 1: Eva's Jacket, From Search to Doorstep, Journey 4: Black Friday, Hour by Hour, Assistants and Agents: AI in Commerce and What Is Real in 2026.

Tool 9: Channel Allocation Card

Purpose. The stock figure sent to a marketplace is the quantity that marketplace may sell without asking again. **Channel allocation** is the decision of how much of your

available stock each channel may promise. The card records that policy per channel, with the channel's scorecard and API calendar.

The template. One card per marketplace or sales channel.

Field	What to record
Channel and seller account	Marketplace, countries, account owner
Fulfillment model	Seller-fulfilled, marketplace-fulfilled, or both
External locations	Stock held by the marketplace's program, tracked as its own location
Allocation policy	Shared pool, fixed quantity, buffer, percentage split, or computed ATP per channel
Buffer or share	Units or percentage, and when it changes (peak, hero SKUs)
Scorecard thresholds	The channel's published cancellation, late-shipment and tracking limits
Interface and sunset	API version used and its retirement date (Tool 4)

How to use it. A buffer is an honest interim while stock moves on a schedule; computed ATP per channel is the target once stock moves by events.

Kestrel & Pine example (fictional; marketplaces through the connector, 2027):

Field	Value
Channels	bol (NL/BE), Amazon.de, Zalando (DE, NL); €3.0M in 2026, owner Milan Petrov
External location	About 40% of marketplace units for Germany ship from the marketplace's fulfillment program
Policy from December 2026	The platform's figure minus a **ten-unit buffer** per variant; zero below ten
Policy from Q4 2027 (W1)	Computed ATP per channel from the availability service, buffer 3 units per SKU
Peak rule 2027	Buffer raised to 25 units for hero SKUs when stock events lag (Tool 12, rung 4)
Order path	Every marketplace order created in the OMS from 15 September 2027

Used in: Selling on Someone Else's Platform: Marketplaces and Channel Integration,

Is It in Stock? Inventory, Availability and the Promise You Can Keep, Journey 4: Black Friday, Hour by Hour, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax.

Tool 10: EDI Partner Card

Purpose. EDI (electronic data interchange) is the exchange of orders, despatch advices and invoices between companies' systems as structured documents. Every trading partner publishes its own guideline and enforces it with deductions, so the card holds, per partner, everything needed to keep the messages right, in the merchant's own repository rather than in one consultant's head.

The template.

Field	What to record
Partner and owners	Name, locations, annual value; business owner; internal technical owner who receives partner notices
GLNs	Global Location Numbers for the partner's legal entity and each delivery point
Transport	VAN (a value-added network run by an EDI provider), AS2, web portal, Peppol
Messages and versions	ORDERS, ORDRSP, DESADV (the despatch advice, also called ASN), INVOIC, RECADV; standard and version
Guideline version	The partner's implementation guideline in force, and its next change
Mapping version	Where the mapping lives, who maintains it, an export in your repository
Source of each document	Which system produces the order, the SSCC pallet labels and the DESADV
Compliance rules	Deduction terms, on-time-in-full rules
Open disputes	Amount, deliveries, evidence

How to use it. The one rule the card must enforce: **each document comes from the system that knows its truth.** An SSCC (Serial Shipping Container Code, the number on a pallet label) must come from the system that builds the pallet, so the DESADV must come from the pack confirmation.

Kestrel & Pine example (fictional; Bergzicht Outdoor under W6):

Field	Value
Partner	Bergzicht Outdoor: 38 shops, distribution centre in Zwolle; €2.1M a year; business owner Lea Moreau
GLNs	Recorded per entity and the Zwolle DC; the DESADV is addressed to the GLN the order came from
Transport and messages	The EDI provider's VAN; EDIFACT ORDERS, DESADV, INVOIC; Bergzicht sends no RECADV
Source of each document	Order into the ERP (the OMS from 8 December 2027); SSCC labels from the WMS at pack time; DESADV from the WMS pack confirmation
Terms	Credit limit €450,000 held in the ERP; 30-day payment
Open disputes	€14,200 deducted in October 2026 for nine Q3 2026 deliveries whose SSCC labels did not match the DESADV
Mapping	Built in 2025 by one external consultant; export now in Kestrel & Pine's repository

Used in: Documents Between Companies: EDI, Punchout and E-Invoicing, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, Journey 3: A Chain Reorders by EDI, From Order to Invoice, Pick, Pack, Ship: Warehouse Management and Fulfillment.

Tool 11: App Governance Card

Purpose. An app installed in a hosted platform can read and often write the merchant's data through the scopes (permissions) it was granted. The card names an owner per app, records what it may touch and on which API version, and enforces **one writer per fact**, the rule whose absence produced the App That Wrote Stock.

The template. One card per installed app.

Field	What to record
App, vendor, owner	Public or custom; installed when and by whom; a named business owner
Purpose	One sentence a new colleague would understand
Scopes granted	Each read and write scope
Scopes revoked and why	With date and the decision record (Tool 16)
Facts it writes	Checked against Tool 2's writer column
API version and support end	Pinned version and the vendor's end date
Review date	The next quarterly uninstall review
Exit plan	What replaces it, and when

How to use it. Put custom behavior on the lowest rung that works: configuration, then the platform's own functions and workflows, then a public app, then a custom app with an owner. In the quarterly review, remove any app with no owner, no use in the quarter, or a twin.

Kestrel & Pine example (fictional; the pre-order app, from the April 2027 governance rule):

Field	Value
App, owner	Pre-order app, public, installed 2025; owner Milan Petrov
Purpose	Take pre-orders for next season's Ridgewalker colors
Scopes granted	read_products, read_inventory, write_orders (pre-order tags)
Scopes revoked	write_inventory, June 2027: the WMS feed is the only writer of stock (W1)
API version	2027-01; supported at least to January 2028 *(illustrative)*
Review date	October 2027
Exit plan	Pre-orders handled by the availability service and the OMS from Q4 2027; app removed after the spring 2028 season

Used in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Delivering the Storefront: Themes, Headless, Rendering and Performance, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers, Assistants and Agents: AI in Commerce and What Is Real in 2026.

Tool 12: Peak Plan and Degradation Ladder

Purpose. A peak plan measures every link in the order chain in orders per minute, proves each link with a load test, and decides in a calm month what to switch off in a busy hour. The **degradation ladder** is that switch-off order: features turned off one rung at a time, each by a named owner on a stated trigger, so that checkout, payment and the order path keep working.

The template.

Plan field	What to record
Peak dates and owner	The peak week, the peak day, the incident commander
Forecast	Orders per minute in the peak hour, from last year's peak and this year's promotions; owned by whoever owns the promotion calendar
Capacity per link	Tested orders per minute for checkout, order export, warehouse intake, stock loop, PSP authorizations
Load tests and game day	Dates, rates, marked test orders, failures rehearsed
Freeze	From and to; what it covers
Gauges and alarms	Queue age, dead-letter depth, event lag, decline rate against normal
Morning-after reconciliation	Time, what is compared, who produces it

The ladder is a table with one row per rung: the switch, its one owner (who may flip it without a meeting), the measured trigger and the date it was last flipped in a test, ending with a line for what is **never** switched off.

How to use it. A switch never flipped in a test is a hope; the order is set in a calm month by the people who own the revenue.

Kestrel & Pine example (fictional; the 2027 plan approved with Waypoint in March 2027, Noor Visser as incident commander):

Plan field	Value
Peak	Week of 22–28 November 2027; Black Friday 26 November
Forecast and tests	60 orders a minute (about 1.5 times 2026's 38); every link tested at 80; the OMS-to-WMS link at 120 with marked orders the WMS skips
Go/no-go	Availability service, 15 October 2027; fallback the old chain with the ten-unit marketplace buffer
Freeze	15 November–5 December 2027
Situation room	Friday 16:00–24:00
Reconciliation	08:00 Saturday 27 November, by Hanna Berg's team

Three of the eight rungs:

#	Switch	Owner	Trigger
3	Stock badges from the buffered copy	Ruben Alves	Availability event lag over 2 minutes
5	Delivery promise widened to "within three working days"	Daniel Okafor	Oldest unsent order over 10 minutes, or the Friday pick plan over 80% consumed
8	New-account creation and the newsletter pop-up off	Milan Petrov	Only on Noor's call
—	**Never**: checkout, payment authorization, order creation in the OMS, the order path to the WMS	—	—

Used in: Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.

Tool 13: Regulatory Calendar

Purpose. Most rules that reach a shop are data requirements with a date: a 30-day price history, a refund clock, a manufacturer field on every listing, a structured invoice sent over a network. The calendar reads each rule as a field, a system of record, a test and an owner, so that a deadline is not discovered when it is a few months away (the Long Fuse).

The template. One row per rule, sorted by date.

Column	What to record
Date	Application date from the official source; several dates where the rule phases in
Rule	Official name and number, with a link
Scope check	Why it applies to you, or why it does not (size, country, role)
What it requires	In data terms: which field, clock or document
System of record	Where that field or clock lives (Tool 2)
Test	How you know you comply, and how often it runs
Owner	The person who owns the fact, often not an engineer
Status	*Maintain* (in force), *prepare* (date ahead), *watch* (pending)

How to use it. Plan to the scope and schedule to the date. Review the *watch* rows quarterly; this is not legal advice.

Kestrel & Pine example (fictional rows and owners; the rules and dates are documented in Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation):

Date	Rule	What it requires	Owner	2027 status
13 Dec 2024	GPSR, Regulation (EU) 2023/988, Art. 19	Manufacturer, responsible person, identification and warnings on every offer	Teun Baas	*Maintain*: the PIM blocks publication without the fields from Q3 2027; about 850 third-party variants to clean up
1 Jan 2026	Belgian e-invoicing	Structured invoices over Peppol to Belgian business customers	Sanne de Wit	*Maintain*: delivered-status report; 210 accounts

Used in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Documents Between Companies: EDI, Punchout and E-Invoicing, Whose Data Is It? Privacy, Consent and Retention, What Are We Selling? Products, Variants and the Catalog.

Tool 14: Build, Buy or Assemble Decision Card

Purpose. A merchant covers the capability map with four kinds of parts: a suite, a hosted platform plus apps, assembled specialist services, and custom code. The card compares the realistic options for one capability or one platform decision on five criteria and on three-year total cost, including the lines a vendor's quote leaves out.

The template. Page one is the criteria.

Criterion	What to record per option
Business model	Which channels and stock locations the option must serve
Scale stage	Starting, Growing, Omnichannel or Group, and the signals of the next stage
Team	Engineers needed to own it, and whether you have them
Differentiation	What a customer would notice if you did it better than anyone; build only that
Time and risk	The next peak; the cost of a failed cutover; whether each step is reversible

Page two prices each option over three years: subscriptions and any share of gross merchandise value (GMV); build, implementation and migration; the agency after go-live; people added; revenue at risk during cutover; and the exit cost at the next review.

How to use it. Price every option as incremental cost over today's run rate and label every figure that is not a quote as an assumption.

Kestrel & Pine example (fictional; the February 2027 platform review, incremental cost 2027–2029, illustrative):

	Composable rebuild	Enterprise suite	Stay and surround (W4)
Three-year incremental total	about €3.2M, 18 months	about €3.19M, 12 months	about €1.33M

	Composable rebuild	Enterprise suite	Stay and surround (W4)
Revenue at risk	€540k *(assumption)*	€540k *(assumption)*	about €50k
Exit at the 2029 review	High	High	Moderate
Decision	—	—	**Chosen**: OMS with availability module (€4,500/month, €320k to implement), PIM (€1,800/month, €110k), integration standards €150k; recorded as KP-004

Used in: Build, Buy or Assemble: Choosing and Changing Platforms, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement, What Changes at Each Scale: From First Store to Group.

Tool 15: Migration Plan Card

Purpose. When a system of record moves, two systems hold the same fact for a while and only one may be right. The card plans the move one seam at a time (a channel, a capability or a slice of traffic), with a reconciliation that proves old and new agree and a rehearsed way back, instead of one cutover date with no return (the Burned Bridge).

The template. One card per seam; the data rows once per migration.

Field	What to record
Seam	By channel, by capability or by traffic slice
Phases and dates	Shadow, parallel run, divert this seam, divert all, decommission
Owner of the fact per phase	Old or new
Reconciliation	Tool 5 card; what agrees for how long before going forward
Go/no-go criteria	Written before the date
Rollback	How the seam is routed back, how long it takes, when it was rehearsed
Identifier continuity	Order number family, SKU, GTIN unchanged; last old and first new number recorded
Freeze and peak	Change freeze around the peak; the old path kept operable through one peak
Data migrations	Each data kind with its own check: order history read-only, gift-card balances to the cent

How to use it. Prefer one writer and a copied flow over dual writes (one actor writing both systems).

Kestrel & Pine example (fictional; the OMS introduction under W2, 2027):

Phase	Dates	Reversible by
Shadow copy of all orders	August	Switching the copy off
Parallel run	1–14 September; differences on about 1.4% of orders in week one, mostly status mapping and timing	Nothing to reverse
Marketplace seam	15 September	A rehearsed one-hour route back
D2C seam	13 October	Routing back within the day
Change freeze	15 November–5 December, around Black Friday (26 November)	The freeze itself
Wholesale seam (EDI and manual)	8 December	Routing back to the ERP's order entry

Phase	Dates	Reversible by
Decommission old order paths	January 2028; 2026 orders imported as read-only history	Not needed

About 14 of the 34 integrations were re-pointed.

Used in: Moving Without Burning the Bridge: Replatforming and Migration, The Order Record: Lifecycle, State and the Order Management System, What Are We Selling? Products, Variants and the Catalog, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.

Tool 16: Architecture Decision Record

Purpose. An architecture decision record (ADR) is a one-page note of a decision, the facts that forced it, the options considered and what it changes, kept in the repository so a newcomer can understand a decision instead of blindly accepting or reversing it. Michael Nygard defined the form in 2011; this version adds ownership, vendors and a review date.

The template.

```
# KP-NNN: <Decision in a few words>

Status:       proposed | accepted | deprecated | superseded by KP-NNN
Date:         YYYY-MM-DD
Deciders:     <names>; business owner: <name>

## Context       The incident or finding, as neutral facts with dates and
                 numbers.
## Drivers       What a good answer must achieve; the failure pattern it
                 prevents.
## Options       At least two real options, plus "do nothing".
## Decision      "We will ...", with scope, owner and dates.
## Ownership     Rows of the entity-owner table (Tool 2) whose owner or writer
                 changes.
## Consequences  Better / worse / new work, and who carries it.
## Vendors       Contracts, API versions and exit terms this decision depends
                 on.
## Confirmation  How conformance is checked: reconciliation cards, write
                 audits, gauges.
## Review        A date, or the trigger that reopens it.
```

How to use it. Write one for every decision that changes who owns a fact, which vendor you depend on, or which app may write. Keep it to a page, and supersede rather than edit an accepted record.

Kestrel & Pine example (fictional; Waypoint decision W4, numbered KP-004):

```
# KP-004: Stay on the SaaS platform and surround it

Status:       accepted
Date:         March 2027
Deciders:     Mara Lindqvist, Noor Visser; business owner: Mara Lindqvist

## Context       Two February 2027 replatforming pitches cited the Black Friday
                Oversell (about €96,000). The platform held; the chain behind it
                 failed. Four empty boxes on the capability map.
## Options       Composable rebuild (about €3.2M); enterprise suite (about
↪  €3.19M);
                 stay and surround (about €1.33M); do nothing.
## Decision      We will stay on the SaaS platform. OMS, PIM and availability
↪  are added
                beside it (W1-W3); the platform's own POS is evaluated in 2028.
## Ownership     Order -> OMS (Q4 2027); product content -> PIM (Q3 2027);
                 availability per channel -> availability service (Q4 2027).
## Consequences  Better: no cutover; each step reversible. Worse: more systems
↪  to
                integrate. New work: integration standards (W5), two engineers.
## Confirmation  Order, stock and price reconciliation cards; the 2027 peak
↪  plan.
## Review        The 2029 review.
```

KP-001 to KP-008 are Waypoint decisions W1 to W8.

Used in: Who Builds and Who Owns: Teams, Vendors, Agencies and Governance, Build, Buy or Assemble: Choosing and Changing Platforms, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Delivering the Storefront: Themes, Headless, Rendering and Performance.

Keeping the Cards True

Cards decay unless three habits keep them current.

- **Review on a calendar.** The capability map yearly; the entity–owner table with every decision record; the integration inventory and app cards quarterly; contract cards at every vendor version; partner cards at every guideline change; the peak plan from March; the regulatory calendar's *watch* rows each quarter.
- **Let incidents add fields.** When a review asks a question no card answers, add a field.
- **Keep the set small.** A field filled by habit and read by nobody should go.

The terms these cards use, from ATP and SSCC to idempotency and system of record, are defined in plain language in Glossary: Plain-Language Definitions, and Bibliography: Every Source the Book Cites lists the sources behind them.

Sources and Evidence

The templates are the author's, derived from the chapters linked in each tool, which cite their sources in full. The external forms and texts they build on:

Company, product and investor sources

- Payout reconciliation report, Stripe, n.d., accessed 2026-11 — a vendor's itemized reconciliation, the finance-side model for Tool 5.
- Webhooks, Shopify developer documentation, n.d., accessed 2026-11 — delivery not guaranteed and reconciliation jobs recommended, the reason Tool 4 always points to Tool 5.

Standards, regulation and official documentation

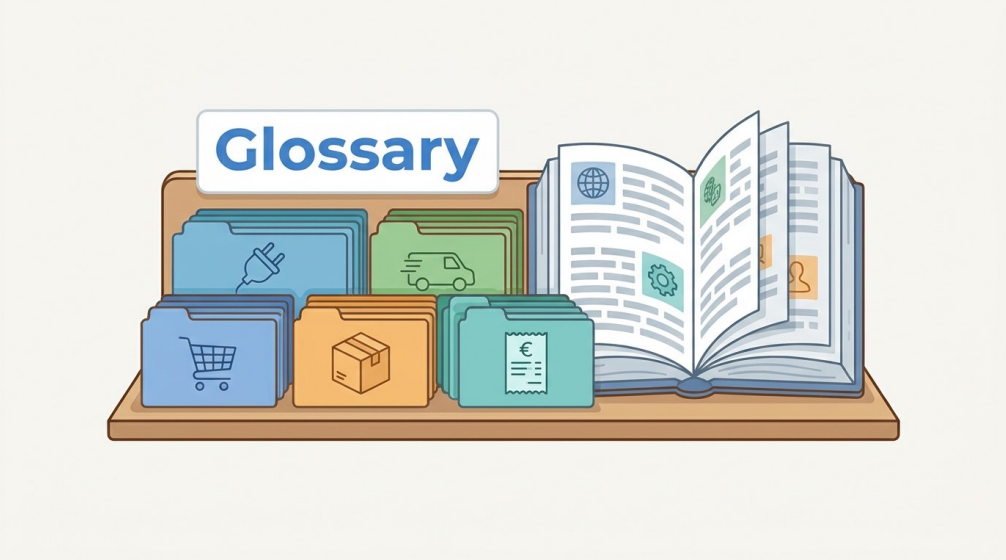
- GS1 General Specifications, Release 26.0, GS1, 2026 — GTIN, GLN and SSCC as used in Tool 10.
- Regulation (EU) 2023/988 (General Product Safety), European Union, 2023 — the Art. 19 row of Tool 13.
- Service Level Objectives and Monitoring Distributed Systems, Google, *Site Reliability Engineering*, 2017 — measured objectives and the gauges of Tool 12.

Research and practitioner sources

- Documenting Architecture Decisions, Michael Nygard, 2011 — the record anatomy (title, context, decision, status, consequences) that Tool 16 extends; older than three years and still the standard reference.
- Feature Toggles (aka Feature Flags), Pete Hodgson, martinfowler.com, 2017 — ops toggles and kill switches as the mechanism of the degradation ladder in Tool 12.
- Strangler Fig Application, Martin Fowler, 2024 (restating a 2004 note), and Patterns of Legacy Displacement, Ian Cartwright, Rob Horn and James Lewis, 2021, revised 2024 — seams, parallel runs and transitional architecture in Tool 15.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Glossary: Plain-Language Definitions



IN THIS SECTION, YOU WILL: Look up any term the book uses, read a one- or two-sentence definition beside the terms it is confused with, and follow the link to the chapter that owns it.

KEY POINTS:

- **Look a term up, then read its neighbors.** Each section collects one part's terms, so the rows around a term carry the contrasts that cause real design errors: product versus variant versus SKU, on hand versus committed versus available-to-promise, authorization versus capture, order versus shipment versus invoice, system of record versus copy.
- **One term, one meaning, one owning chapter.** Every row links to the chapter that owns or best explains the term, and the definitions follow how those chapters define them. When a chapter and this page disagree, one of them needs correcting.
- **Rules, standards and vendor terms date quickly.** Laws, protocols, products and their status are given as of November 2026, following

> the sources cited in the chapters. Check the current source before a definition here goes into a contract, a vendor evaluation or a design decision.

The words that cause the most trouble in commerce are ordinary ones. **"Order" means three different numbers** to the platform, the ERP and the warehouse report at Kestrel & Pine (Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics), and **"in stock" can mean on hand, available or "the last figure the connector received an hour ago"** (Is It in Stock? Inventory, Availability and the Promise You Can Keep). This glossary pins each term to one definition and one owning chapter, so that the chapters, their readers and the AI tools that read them use the words the same way.

How to Use This Glossary

The sections follow the book's structure. The first collects the **six evidence labels**, the **three take-away tiers**, the **four scale stages** and the **sixteen failure patterns** from the Preface; the next eleven follow **Parts I to XI**, from the business of commerce through catalog, order, fulfillment, channels, integration, platform and trust to the journeys, the decisions that change with scale and the public names the case studies rely on. Then come two reference tables: the **standards, protocols and regulations** with their scope (EU, a country, the US or global), and **Kestrel & Pine's fictional names**. The **alphabetical index** at the end lists every term, alias and abbreviation with a link to the section that defines it.

Each section opens with the chapters that own its terms, then a table with 391 rows in all. The definitions are short teaching descriptions for a reader new to the field, not legal text, standards or vendor documentation; the chapter in the last column gives the full treatment, the sources and the worked example. Rows that combine two or more terms with a slash define all of them, because the difference between them is the point. Acronyms are expanded in the term or the definition. Kestrel & Pine, named in some rows and in its own section, is the fictional outdoor brand this book follows (Meet Kestrel & Pine); its names and numbers are illustrations, not facts about any real company.

Preface — Evidence Labels, Take-Away Tiers and Failure Patterns

Every chapter marks what kind of statement it is making, ends with three tiers of take-away and names the recurring ways commerce platforms fail. Introduction: Why Commerce Architecture Matters explains the six evidence labels, the three tiers and the four scale stages; Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go

Wrong tells each of the sixteen failure patterns with its warning sign. The last column names the guide and the chapter that does most of the repair work.

Term	Definition	Chapter
Documented fact	A company's public statement, a filing, a regulation, a standard, a date or an official statistic, linked inline with the year of publication in the sentence. Company evidence older than about three years may describe a system that has since been replaced.	Introduction: Why Commerce Architecture Matters
Industry practice	What merchants and vendors commonly do, phrased as "commonly" or "a typical pattern is", with no invented percentages.	Introduction: Why Commerce Architecture Matters
Reference-architecture inference	How a system is plausibly built, reasoned from engineering principles. When it concerns a real company it is labeled INFERENCE, NOT DOCUMENTED and is never a description of that company's systems.	Introduction: Why Commerce Architecture Matters
Recommendation (evidence label)	The author's judgement as an architect who advises growing merchants, in the first person, with the reasoning and the conditions under which it does not hold.	Introduction: Why Commerce Architecture Matters

Term	Definition	Chapter
Illustration	An example set at Kestrel & Pine, the fictional outdoor brand the book follows. It is labeled fictional in each chapter, and its numbers agree across the book through one scenario ledger.	Introduction: Why Commerce Architecture Matters, Meet Kestrel & Pine
Labeled assumption	An illustrative volume, latency, cost or percentage that is neither documented nor a Kestrel & Pine fact, marked "assume…" in the text or *(assumption)* in a table. Follow the reasoning; do not use it as a benchmark.	Introduction: Why Commerce Architecture Matters
Established / Depends on context / The author's synthesis	The three tiers of every chapter's "What to Take Away": what the field agrees on, with its evidence; recommendations that hold only under a stated condition (scale stage, business model, team size, vendor); and the author's own position, marked as such.	Introduction: Why Commerce Architecture Matters

Term	Definition	Chapter
Scale stage	A shape of business defined by how many channels sell and how many locations hold stock: **Starting** (one channel, under about €2M), **Growing** (two or three channels, about €2M–€20M), **Omnichannel** (stores, wholesale, marketplaces, own warehouse, about €20M–€200M) and **Group** (several brands or units). The revenue bands are markers, not thresholds.	Introduction: Why Commerce Architecture Matters, What Changes at Each Scale: From First Store to Group
Failure pattern	A named, recurring way a commerce platform goes wrong, with a warning sign that can be checked in a design review and a family (Truth, Integration, Platform and change, Governance). The book uses sixteen.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong
The Phantom Stock	Two channels sell the same last unit; "out of stock" cancellations rise in busy weeks. Truth family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Is It in Stock? Inventory, Availability and the Promise You Can Keep
Schrödinger's Order	Support checks three systems to say where an order is, and they disagree. Truth family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, The Order Record: Lifecycle, State and the Order Management System

Term	Definition	Chapter
The Familiar Stranger	Nobody can say whether the store customer and the online customer are the same person, because each system counts customers its own way. Truth family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, One Customer, One Product: Identity and Master Data Across Systems
The Boomerang	Refunds, restocking and credit notes are reconciled by hand after the return arrives, and sometimes twice. Truth family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics
The Price Nobody Can Explain	The same product carries different prices in different channels, and nobody can say which system set which price or why. Truth family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
The Spaghetti Junction	Every pair of systems, sometimes every customer, has its own hand-built connector with its own owner, or none. Integration family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, APIs, Events, Files and Middleware: How Commerce Systems Talk
The Message in a Bottle	Messages between systems are sent once and trusted; customers find the duplicates and the gaps. Integration family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation

Term	Definition	Chapter
The Pulled Rug	A vendor's API or a partner's document format changes and an integration breaks before anyone is told. Integration family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Promises Between Systems: API Contracts, Webhooks and Versioning
The First Snowfall	The busiest day of the year is the first time the system, and its integrations, meet real load. Platform and change family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Black Friday Every Year: Peak Load, Resilience and Recovery
The Giant's Boots	An architecture is copied from a company a hundred times larger, or from a trend, without its reasons. Platform and change family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says
The Burned Bridge	Everything moves to a new platform on one date, with no way back. Platform and change family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Moving Without Burning the Bridge: Replatforming and Migration
The Forked Platform	The platform's core or theme has been customized so deeply that it can no longer be upgraded, and every vendor release is a project. Platform and change family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Apps, Plugins and Functions: Extensibility and Ecosystem Governance

Term	Definition	Chapter
One Size Fits None	Several brands or business units are forced onto one stack before anyone has measured the benefit, or kept apart when they should share. Platform and change family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, What Changes at Each Scale: From First Store to Group
The Lynchpin	A capability customers depend on runs through one vendor, one connector, one agency or one person, with no exit plan. Governance family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance
The Long Fuse	A legal deadline (e-invoicing, accessibility, consumer rules) is discovered when it is months away. Governance family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation
The Secret Agent	A script, app, bot or AI agent changes orders, stock, prices or money without a named owner, its own identity or a person's approval. Governance family.	Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Assistants and Agents: AI in Commerce and What Is Real in 2026

Part I — COMMERCE: What a Commerce Platform Is

The business vocabulary and the first architecture words: what an order is and who promises what, the business models and the numbers that judge them, the capability map, ownership of facts, and the kinds of platform on the market. Follow the Order:

What Happens Between "Buy" and "Delivered" owns the promise and the handoff; Who Sells What to Whom: Business Models and the Economics That Shape Architecture the models and economics; The Map of a Commerce Platform: Capabilities and Layers the capability and the layer; Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record the system of record and the bounded context; Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026 the platform shapes.

Term	Definition	Chapter
Promise	A statement the merchant makes to the customer that some system will later have to make true: "in stock", "€199", "ordered before 22:00, delivered tomorrow". Promises are made early, in seconds, and kept late, over days.	Follow the Order: What Happens Between "Buy" and "Delivered"
Handoff	The moment responsibility for an order passes from one system to another, for example from the platform to the warehouse. Most failures happen at handoffs.	Follow the Order: What Happens Between "Buy" and "Delivered"
Order	The record of what was bought, by whom, at what price and tax, to be delivered where and how, and paid how. It is not a cart, not a shipment and not an invoice.	Follow the Order: What Happens Between "Buy" and "Delivered", The Order Record: Lifecycle, State and the Order Management System

Term	Definition	Chapter
B2C / D2C	**B2C** (business to consumer) is selling to private consumers. **D2C** (direct to consumer) is a brand selling through channels it operates itself, its own web shop and stores, rather than through a retailer.	Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Meet Kestrel & Pine
Wholesale / B2B e-commerce	**Wholesale** is a brand selling to retailers and distributors who resell. **B2B** (business to business) e-commerce is business buyers ordering electronically, through a web portal or by structured messages between systems.	Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals
B2B2C	Business to business to consumer: a brand sells through a partner's consumer channel while keeping control of its product, content and price.	Who Sells What to Whom: Business Models and the Economics That Shape Architecture
Marketplace selling / marketplace operating	Selling on someone else's marketplace as one seller among many, as against running the marketplace and taking a fee from sellers. The two have opposite architectures.	Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Selling on Someone Else's Platform: Marketplaces and Channel Integration

Term	Definition	Chapter
Omnichannel	Stores and digital channels sharing stock, orders and customers, so a shopper can buy in one channel and collect, return or get service in another. Also the name of the third scale stage.	Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
Channel	Any place where a customer can buy and a promise is made: a storefront, a store till, a marketplace listing, the B2B portal, an EDI link, a click-and-collect option.	Is It in Stock? Inventory, Availability and the Promise You Can Keep, What Changes at Each Scale: From First Store to Group
Seller of record	The party that legally sells to the customer: the one whose name is on the invoice, who owes the VAT and who answers for the consumer's rights.	Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Case: Amazon, From Monolith to Services to the Everything Store
GMV (gross merchandise value)	The value of orders passing through a platform, including what the platform does not keep; several vendors price on it. Definitions differ on refunds and fees (Shopify's filing calls it gross merchandise volume and counts it net of refunds), so always read the source's definition.	Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants

Term	Definition	Chapter
AOV (average order value)	Revenue divided by the number of orders. Kestrel & Pine's online AOV in 2026 is €92.	Who Sells What to Whom: Business Models and the Economics That Shape Architecture
Conversion rate / cart abandonment	**Conversion rate** is purchases divided by visits. **Cart abandonment** is the share of carts started and not bought; extra costs at checkout are the top reason shoppers give.	Who Sells What to Whom: Business Models and the Economics That Shape Architecture, From Cart to Confirmation: Checkout Orchestration
Gross margin	Revenue minus the cost of the goods sold, as a share of revenue; markdowns, returns and fees eat into it. Kestrel & Pine's is 54%.	Who Sells What to Whom: Business Models and the Economics That Shape Architecture
Return rate	The share of items, or of orders, that come back; always say which. Kestrel & Pine's online item return rate is 24% and its order return rate 19%.	Who Sells What to Whom: Business Models and the Economics That Shape Architecture, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics
EDI-type sales	Orders received as structured messages between companies' systems rather than through a web page; for business customers in the EU they are the majority of electronic sales.	Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Documents Between Companies: EDI, Punchout and E-Invoicing
Capability	A job the business needs done, named without saying who does it: "calculate the price a given customer pays for a given product today".	The Map of a Commerce Platform: Capabilities and Layers

Term	Definition	Chapter
System / vendor product	A **system** is a piece of software that does one or more capabilities; a **vendor product** is a system someone sells, usually claiming several capabilities at once.	The Map of a Commerce Platform: Capabilities and Layers
Layer	A group of capabilities that share a purpose and a rhythm: the ones a shopper touches, the ones that describe what is for sale, the ones that move money and orders, the ones that move goods, partners and back office, and the foundations.	The Map of a Commerce Platform: Capabilities and Layers
Business fact	Something the business needs to be true about the world: this variant has this GTIN, this customer pays this price, there are 14 units of this size in Duiven.	The Map of a Commerce Platform: Capabilities and Layers, Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record
System of record	The one system whose value of a fact is the truth when copies disagree; every other holder keeps a copy. For master data it is often called the master.	Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, One Customer, One Product: Identity and Master Data Across Systems

Term	Definition	Chapter
Copy / writer	A **copy** is a business fact held in any system other than its system of record. A **writer** is any system, app, script, job or person that can change a fact in a given system; a fact with two writers drifts.	Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record
Bounded context	Eric Evans's term from *Domain-Driven Design* (2003): a boundary inside which a model, and the words in it, have one agreed meaning. "Order" and "stock" mean different things in different contexts.	Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record
Commerce platform	The software that shows products, holds a cart, takes an order and the payment, and hands the order to whoever fulfills it.	Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026
SaaS (software as a service)	Software the vendor runs for many customers at once and rents by the month; you configure it and the vendor upgrades everyone on the same day.	Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms
Suite (enterprise suite)	One vendor's set of commerce, order management and content modules designed to work together, usually implemented by a partner.	Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms

Term	Definition	Chapter
Open source platform	Commerce software whose code is published under a license that lets you run and change it; you or your agency host, extend and upgrade it, and forks (such as Mage-OS from Magento) can appear.	Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026
Composable	Buying commerce as separate services, from one or several vendors, and assembling them yourself rather than taking one suite; every join between them is then yours.	Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms, Case: commercetools, Shopware and the Composable Movement
MACH	The acronym a vendor alliance coined in June 2020 for Microservices, API-first, Cloud-native SaaS and Headless; founders were commercetools, Contentstack, EPAM and Valtech. It names a style, not a product.	Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement
ERP-centered commerce	A storefront attached to the ERP that already owns stock, orders and finance, with the web shop treated as a front end.	Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026

Term	Definition	Chapter
Specialist layer / specialist group	A **specialist layer** is a product for one capability (OMS, PIM, WMS, returns, search, channel, iPaaS, payments) added beside a platform. A **specialist group** is several specialist products under one owner, kept as separate brands but pre-integrated.	Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026

Part II — CATALOG: Products, Prices and Promises

What is for sale, at what price, with which discount, how shoppers find it and whether it is in stock. What Are We Selling? Products, Variants and the Catalog owns product, variant, SKU, GTIN and PIM; What Does It Cost? Price Lists, Contract Prices, Currencies and Tax the price list and the contract price; Discounts, Coupons and Loyalty: Promotions Without Chaos the promotion and stored value; Finding the Product: Search, Navigation and Merchandising the search index, relevance, merchandising rule and facet; Is It in Stock? Inventory, Availability and the Promise You Can Keep stock, available-to-promise and reservations.

Term	Definition	Chapter
Product / variant	A **product** is the sellable concept a shopper browses ("Ridgewalker shell jacket, women's"). A **variant** is one buyable combination of option values (size M, color moss) and must stay stable once it has been sold.	What Are We Selling? Products, Variants and the Catalog, Follow the Order: What Happens Between "Buy" and "Delivered"

Term	Definition	Chapter
SKU (stock-keeping unit)	The merchant's own code for one sellable variant, the thing counted on a shelf and put in a parcel; nobody outside the company understands it, which is its strength. The book also uses SKU for the sellable variant itself.	What Are We Selling? Products, Variants and the Catalog, Meet Kestrel & Pine
GTIN / EAN-13	The **Global Trade Item Number** under the barcode, issued through GS1 so that every company's systems mean the same item by it; never reused for a different item. In European retail it is the 13-digit **EAN-13**.	What Are We Selling? Products, Variants and the Catalog, Documents Between Companies: EDI, Punchout and E-Invoicing
Option / option value	An **option** is a dimension of choice (size); an **option value** is one choice (M). Variants are combinations of option values.	What Are We Selling? Products, Variants and the Catalog
Attribute / attribute set	An **attribute** is a typed field on a product or variant (waterproof rating in millimeters, fabric, weight). An **attribute set** is the list of fields a product of a given kind must carry.	What Are We Selling? Products, Variants and the Catalog
Taxonomy	A fixed tree of categories that a channel understands; each marketplace has its own, and products must be mapped to it.	What Are We Selling? Products, Variants and the Catalog

Term	Definition	Chapter
Bundle (kit) / configurable product	A **bundle** or kit is a product sold as a set of other products (a tent with its footprint). A **configurable product** is the parent that groups purchasable child variants.	What Are We Selling? Products, Variants and the Catalog
PIM (product information management)	The editorial system where product data is created, enriched, translated and approved before it is published to channels; at Kestrel & Pine the system of record for product content from Q3 2027.	What Are We Selling? Products, Variants and the Catalog, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026
Enrichment / syndication	**Enrichment** is adding the marketing, technical and media content a channel needs beyond what the ERP holds. **Syndication** is publishing product content to each channel in its format and under its rules, with a rejection report back.	What Are We Selling? Products, Variants and the Catalog
Feed / structured data	A **feed** is a file or API stream of product records in a channel's format. **Structured data** is product information embedded in the web page itself (schema.org) so that search engines and agents can read it.	What Are We Selling? Products, Variants and the Catalog, Assistants and Agents: AI in Commerce and What Is Real in 2026

Term	Definition	Chapter
Price list / price dimensions	A set of prices that applies to one combination of **market**, **customer group**, **channel** and **validity window**, with a stated **tax basis**. A price without its dimensions cannot be explained.	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
Base price / contract price	A **base price** is the list price for the general public. A **contract price** is agreed with one named account or group and outranks the list.	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals
Tax-inclusive / tax-exclusive	A **tax-inclusive** price is what the consumer pays, with VAT inside it (€199 includes 21% Dutch VAT); a **tax-exclusive** price has tax added later, as wholesale prices do.	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Tax, Invoices and the Books: From Order to Ledger
Pricing engine / price history	The **pricing engine** is the code that applies the price-selection rule to a request at a given moment. The **price history** is every price per SKU, market and channel with effective dates, needed for the 30-day prior-price rule and for audit.	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
Channel price rule	A derived price for a channel that cannot call the pricing engine, such as base price plus an uplift within a floor and a ceiling for a marketplace.	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Selling on Someone Else's Platform: Marketplaces and Channel Integration

Term	Definition	Chapter
Coupon code / automatic discount	A **coupon code** is a promotion the customer unlocks by typing a string; an **automatic discount** applies whenever its conditions are met, with no code.	Discounts, Coupons and Loyalty: Promotions Without Chaos
Eligibility / stacking	**Eligibility** is a promotion's conditions: which products, customers, period, channel and minimum cart. **Stacking** is applying more than one promotion to the same cart or line, which must be decided per campaign.	Discounts, Coupons and Loyalty: Promotions Without Chaos
Discount allocation	Spreading an order-level discount over the lines so that each line, and later each invoice line and return, carries its share.	Discounts, Coupons and Loyalty: Promotions Without Chaos
Gift card / store credit / loyalty points	Stored value the merchant owes: a **gift card** is value the customer paid for in advance, **store credit** is value the merchant granted (often instead of a refund), **loyalty points** are value earned by buying. Each is a liability kept as a ledger.	Discounts, Coupons and Loyalty: Promotions Without Chaos, Tax, Invoices and the Books: From Order to Ledger

Term	Definition	Chapter
Breakage	The part of stored value that will never be redeemed; accounting standards require it to be estimated and recognized in proportion to redemptions, not when cards expire.	Discounts, Coupons and Loyalty: Promotions Without Chaos
Search / navigation / recommendations	The three ways shoppers find products: typing a query, clicking categories and narrowing with filters, and accepting a suggestion such as "often bought together".	Finding the Product: Search, Navigation and Merchandising
Search index / indexing	The **search index** is a separate copy of the catalog built for fast matching; it is always slightly behind the catalog. **Indexing** is the job that copies changes into it.	Finding the Product: Search, Navigation and Merchandising
Relevance / ranking	**Relevance** is the search engine's ordering of results for a query; **ranking** is the same thing seen as a list. Ranking parameters shown to consumers must be explainable.	Finding the Product: Search, Navigation and Merchandising
Facet	A filter with counts ("Color: moss (14)") computed from indexed attributes; it only works if attributes are complete.	Finding the Product: Search, Navigation and Merchandising

Term	Definition	Chapter
Merchandising rule (boost, bury, pin)	A human override of relevance for a query or category: pin a product to the top, push one down, hide one or redirect a query.	Finding the Product: Search, Navigation and Merchandising
Synonyms / typo tolerance / zero-result rate	Synonym sets and typo tolerance let different words find the same product; the **zero-result rate**, the share of searches that return nothing, is the first search metric to watch (9.4% at Kestrel & Pine in Q1 2027).	Finding the Product: Search, Navigation and Merchandising
Vector search / conversational search	**Vector search** matches by meaning rather than exact words; **conversational search** lets a shopper describe a need in sentences. Both still depend on complete product data.	Finding the Product: Search, Navigation and Merchandising, Assistants and Agents: AI in Commerce and What Is Real in 2026
Verified review	A review linked to an order, with a record of how it was collected; moderation rejects on policy, never on sentiment, and incentives may not depend on the rating.	Finding the Product: Search, Navigation and Merchandising

Term	Definition	Chapter
On hand / committed / reserved	**On hand** is the units physically at a location now. **Committed** (also allocated) is the part already claimed by placed orders that have not shipped. **Reserved** is a temporary claim before an order is final, such as a unit in a cart during a limited-stock sale.	Is It in Stock? Inventory, Availability and the Promise You Can Keep, Pick, Pack, Ship: Warehouse Management and Fulfillment
Held back / safety stock / incoming	**Held back** covers units present but deliberately not for sale, including a **safety stock** buffer against counting errors and sync lag. **Incoming** is stock on the way in from a supplier or another location.	Is It in Stock? Inventory, Availability and the Promise You Can Keep
ATP (available-to-promise)	What a channel may still sell: on hand, minus what is committed to unshipped orders, minus what the merchant holds back, computed per channel and location.	Is It in Stock? Inventory, Availability and the Promise You Can Keep, The Map of a Commerce Platform: Capabilities and Layers
Location / stock location	Any place stock can be promised and shipped from: an own warehouse, a store, a 3PL, a marketplace's fulfillment center. A location outside the merchant's control is still a location to track.	Is It in Stock? Inventory, Availability and the Promise You Can Keep, What Changes at Each Scale: From First Store to Group

Term	Definition	Chapter
Stock event	A message sent the moment a stock quantity changes, saying which SKU, at which location, by how much, why and when; the alternative to periodic snapshots.	Is It in Stock? Inventory, Availability and the Promise You Can Keep, Pick, Pack, Ship: Warehouse Management and Fulfillment
Availability service	The component that holds the latest on hand and committed per SKU per location, applies reservations, allocations and buffers, and publishes available-to-promise per channel; at Kestrel & Pine a module of the OMS (Waypoint W1).	Is It in Stock? Inventory, Availability and the Promise You Can Keep, The Order Record: Lifecycle, State and the Order Management System
Reserve-on-cart / reserve-on-order / allocation at fulfillment	The three moments a unit can be claimed: when it enters the cart (with an expiry), when the order is created (the common default) or when the warehouse releases the pick (risking orders that cannot be fulfilled).	Is It in Stock? Inventory, Availability and the Promise You Can Keep
Oversell	Selling more units than exist, usually because two channels read the same stock before either sale reduces it; the root of the Phantom Stock.	Is It in Stock? Inventory, Availability and the Promise You Can Keep, Journey 4: Black Friday, Hour by Hour

Part III — ORDER: Cart, Checkout, Payment and the Order Record

From the draft to the legal record: the cart and its orchestration, the money and its backward flows, the order's states and routing, and tax, invoices and the books. From Cart to Confirmation: Checkout Orchestration owns cart, checkout and the idempotency key at checkout; Taking the Money: Payment Providers, Authorization, Capture and Refunds the PSP, authorization and capture, SCA and refunds; The Order Record: Lifecycle, State and the Order Management System the OMS, state machines and routing; Tax, Invoices and the Books: From Order to Ledger tax calculation, the invoice, the credit note and order-to-ledger.

Term	Definition	Chapter
Cart / abandoned cart	A **cart** is a draft of an order that the shopper can still change; it commits nobody. An **abandoned cart** is one started and not converted within some window.	From Cart to Confirmation: Checkout Orchestration
Checkout / checkout orchestration	**Checkout** is the orchestrated sequence that turns the cart into an order: in seconds the platform calls pricing, promotions, tax, availability, shipping rates and the payment provider, then creates the order.	From Cart to Confirmation: Checkout Orchestration, Follow the Order: What Happens Between "Buy" and "Delivered"
Guest checkout	Buying without creating an account; it raises conversion and creates duplicate customer records that matching must later link.	From Cart to Confirmation: Checkout Orchestration, One Customer, One Product: Identity and Master Data Across Systems

Term	Definition	Chapter
Express wallet	A payment button such as Apple Pay or Google Pay that supplies address and payment in one step, so the merchant never receives the card number.	From Cart to Confirmation: Checkout Orchestration
Idempotency key	A unique token sent with a request so that a retry does nothing twice; it should be derived from the business event ("create order from cart X with payment Y", "refund return R"), not from a page load.	From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds
PSP (payment service provider)	The company that takes the shopper's payment and pays it out to the merchant: one API, many methods, fraud screening, authentication, a balance, payouts and reports.	Taking the Money: Payment Providers, Authorization, Capture and Refunds, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026
Issuer / acquirer / scheme	The **issuer** is the shopper's bank; the **acquirer** is the licensed institution that accepts payments for the merchant (often part of the PSP); the **scheme** or network sets the rules and rails (Visa, Mastercard, iDEAL, SEPA).	Taking the Money: Payment Providers, Authorization, Capture and Refunds

Term	Definition	Chapter
Authorization / capture	**Authorization** is the issuer approving and reserving the money; **capture** is the merchant taking it, often when the parcel ships. An authorization expires, so "capture on shipment" is a promise with a deadline.	Taking the Money: Payment Providers, Authorization, Capture and Refunds, Journey 1: Eva's Jacket, From Search to Doorstep
Settlement / payout	**Settlement** is the scheme or bank moving money to the PSP. A **payout** is the PSP paying the merchant's bank, one batch per period, net of fees, refunds and chargebacks, matched by finance.	Taking the Money: Payment Providers, Authorization, Capture and Refunds, Tax, Invoices and the Books: From Order to Ledger
Refund	Money returned to the shopper on the original payment method, only from what was captured; it is asynchronous and can fail later. One writer per refund and the return id as idempotency key prevent the Double Refund.	Taking the Money: Payment Providers, Authorization, Capture and Refunds, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics
Chargeback	A dispute the cardholder raises with the issuer, which reverses the payment and charges the merchant a fee unless the merchant wins with evidence; not the merchant's decision.	Taking the Money: Payment Providers, Authorization, Capture and Refunds

Term	Definition	Chapter
Interchange	The card schemes' and issuers' part of the card fee; pricing as interchange plus a margin is cheaper at volume but needs reconciliation.	Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Taking the Money: Payment Providers, Authorization, Capture and Refunds
iDEAL / Bancontact / purchase on account	The dominant local methods in Kestrel & Pine's markets: **iDEAL**, a transfer from the shopper's Dutch bank account; **Bancontact**, a Belgian card scheme paid mostly by app; **purchase on account** (*Kauf auf Rechnung*), paying after delivery against an invoice, common in Germany.	Taking the Money: Payment Providers, Authorization, Capture and Refunds
3-D Secure / liability shift	**3-D Secure** is the card networks' protocol for the strong customer authentication EU rules require, usually a confirmation in the banking app. When a 3-D Secure payment is disputed as fraud, liability typically shifts to the issuer; an exemption gives that up.	Taking the Money: Payment Providers, Authorization, Capture and Refunds, From Cart to Confirmation: Checkout Orchestration

Term	Definition	Chapter
Hosted fields / hosted payment page	The PSP draws the card-number boxes inside the merchant's page from its own domain, or redirects to its own page, so the card number never touches the merchant's code; this keeps the merchant's PCI scope small.	Taking the Money: Payment Providers, Authorization, Capture and Refunds, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers
Payment webhook	The PSP's HTTP call to the merchant saying a payment changed state; it is a notification, not a fact, so the receiver fetches the status before acting on it.	Taking the Money: Payment Providers, Authorization, Capture and Refunds, Journey 1: Eva's Jacket, From Search to Doorstep
OMS (order management system)	The operational order record after checkout: every channel's orders in one state model, routing to locations, splits, holds and edits, and the service view. At Kestrel & Pine the order hub from Q4 2027 (Waypoint W2).	The Order Record: Lifecycle, State and the Order Management System, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026
Order header / order lines	The **header** holds number, channel, customer, time, currency, market and legal seller; each **line** holds a SKU, quantity, frozen unit price, allocated discounts and tax.	The Order Record: Lifecycle, State and the Order Management System

Term	Definition	Chapter
State machine	A fixed set of states an object can be in and the allowed moves between them. An order needs separate machines for its lifecycle, its money, its goods (per fulfillment order) and each return, not one status string.	The Order Record: Lifecycle, State and the Order Management System
Sourcing / routing	**Sourcing** is choosing which location fulfills which order line; **routing** is the rule set that makes that choice.	The Order Record: Lifecycle, State and the Order Management System, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
Split shipment	One order shipped in more than one parcel, usually from more than one location or at more than one time; its status must not collapse into one word.	The Order Record: Lifecycle, State and the Order Management System, Pick, Pack, Ship: Warehouse Management and Fulfillment
Order orchestration	The OMS's job of sequencing an order's steps across systems: driving the money, goods and return state machines from events and handing work to warehouses and stores.	The Order Record: Lifecycle, State and the Order Management System, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation
Order number family	One numbering scheme for every channel's orders, so an order has one identity in every system that copies it.	The Order Record: Lifecycle, State and the Order Management System

Term	Definition	Chapter
VAT (value-added tax)	Europe's consumption tax, charged at every sale and reclaimed by business buyers; each sale must carry the right country's rate.	Tax, Invoices and the Books: From Order to Ledger, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
Tax calculation / tax engine	Working out the tax on an order from ship-to address, product tax category and customer status; estimated at checkout, final at invoicing, often by a tax engine or rate table.	Tax, Invoices and the Books: From Order to Ledger
Invoice	The legal document of a sale: required content, a number in sequence per legal entity, never edited, kept for years. The shop issues an order confirmation; the ERP issues the invoice.	Tax, Invoices and the Books: From Order to Ledger, Follow the Order: What Happens Between "Buy" and "Delivered"
Credit note	The invoice's mirror image: it references the invoice, reverses some or all of its lines and carries its own sequential number; a return or a correction produces one.	Tax, Invoices and the Books: From Order to Ledger, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics
E-invoice	An invoice issued, transmitted and received as structured data (XML) that the receiver's software can process; a PDF by e-mail is not one.	Tax, Invoices and the Books: From Order to Ledger, Documents Between Companies: EDI, Punchout and E-Invoicing

Term	Definition	Chapter
Ledger (general ledger)	The book of accounts in which every transaction is recorded twice, as a debit and a credit, so the books always balance; the ERP posts orders, VAT, fees and refunds to it.	Tax, Invoices and the Books: From Order to Ledger, The Back Office: ERP Integration and Who Is the Hub
Reconciliation	Regularly comparing two systems' records of the same thing and explaining every difference: orders against shipments, returns against refunds, PSP payouts against the books.	Follow the Order: What Happens Between "Buy" and "Delivered", Tax, Invoices and the Books: From Order to Ledger, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation

Part IV — FULFILLMENT: From Warehouse to Doorstep and Back

Where promises are kept by people, buildings and vans. Pick, Pack, Ship: Warehouse Management and Fulfillment owns the WMS, the fulfillment order, the pick wave and the 3PL; Getting It There: Carriers, Labels, Tracking and the Delivery Promise the carrier integration, the delivery promise and the tracking event; Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment the POS and the omnichannel flows; The Way Back: Returns, Exchanges, Refunds and Reverse Logistics the RMA, disposition and exchange; Where Is My Order? Notifications, Service and the Customer Record the transactional notification, the order status page and the service view.

Term	Definition	Chapter
WMS (warehouse management system)	The system that runs a warehouse's physical stock and its movements: receiving, put-away, picking, packing and shipping. At Kestrel & Pine the system of record for physical stock at Duiven.	Pick, Pack, Ship: Warehouse Management and Fulfillment
Receiving / put-away / slotting	**Receiving** checks an inbound delivery against the purchase order and books the units in; **put-away** moves them to a storage location with its own barcode; **slotting** decides where each SKU should live.	Pick, Pack, Ship: Warehouse Management and Fulfillment
Pick wave / batch picking	Grouping orders into one picking run, typically by carrier cut-off, channel or zone, so one walk serves many orders. The warehouse works in waves, not order by order.	Pick, Pack, Ship: Warehouse Management and Fulfillment
Packing / cartonization	Choosing the box, packing and weighing; **cartonization** is the software's choice of box size from the items' dimensions.	Pick, Pack, Ship: Warehouse Management and Fulfillment
Shipping confirmation	The warehouse's report that a shipment left, with its tracking number; the moment on hand goes down and the order is fulfilled.	Pick, Pack, Ship: Warehouse Management and Fulfillment

Term	Definition	Chapter
Cycle counting	Counting a small part of the stock on a rolling schedule, instead of one annual full count, to correct the record continuously.	Pick, Pack, Ship: Warehouse Management and Fulfillment, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
Fulfillment order	The instruction to one location: ship these lines of this order, by this service level, with these notes. It is the contract between the OMS and the WMS; the warehouse's accept step locks the order against edits.	Pick, Pack, Ship: Warehouse Management and Fulfillment, The Order Record: Lifecycle, State and the Order Management System
Cut-off / capacity	The **cut-off** is the latest moment an order can reach the warehouse and still ship today: the release of the last wave that makes the last carrier collection. **Capacity** is units or parcels per hour at the day's staffing.	Pick, Pack, Ship: Warehouse Management and Fulfillment, Getting It There: Carriers, Labels, Tracking and the Delivery Promise
3PL (third-party logistics provider)	A company that stores and ships goods on the merchant's behalf, from its own warehouse and usually on its own WMS.	Pick, Pack, Ship: Warehouse Management and Fulfillment, Is It in Stock? Inventory, Availability and the Promise You Can Keep

Term	Definition	Chapter
Marketplace fulfillment program	A marketplace's own warehouse network, stocked in advance by the seller, that picks, packs, ships and handles returns: Fulfillment by Amazon (FBA), Logistics via bol (LvB), Zalando Fulfillment Solutions (ZFS) and ZEOS.	Pick, Pack, Ship: Warehouse Management and Fulfillment, Selling on Someone Else's Platform: Marketplaces and Channel Integration
Carrier / carrier API	The **carrier** moves the parcel (PostNL, bpost, DHL, DPD); the **carrier API** is its interface for creating a shipment, getting a label, closing the day and reading tracking.	Getting It There: Carriers, Labels, Tracking and the Delivery Promise
Label / manifest	The **label** carries the carrier's barcode, the parcel's identity in the carrier's network. The **manifest** closes the day by handing the carrier the list of parcels it is collecting; a label is not a booking.	Getting It There: Carriers, Labels, Tracking and the Delivery Promise
Tracking event	One carrier scan with a time and a place: accepted, at depot, out for delivery, delivered, attempted, exception.	Getting It There: Carriers, Labels, Tracking and the Delivery Promise, Journey 1: Eva's Jacket, From Search to Doorstep
Delivery option / pickup point / locker	What the customer chooses at checkout: home, a **pickup point** (a shop counter), a parcel **locker**, a time window or a date.	Getting It There: Carriers, Labels, Tracking and the Delivery Promise

Term	Definition	Chapter
Lead time / transit time	Days from order to shipment (handling) and from carrier collection to delivery (transit), per lane: an origin, a destination area and a service.	Getting It There: Carriers, Labels, Tracking and the Delivery Promise
Delivery promise / EDD (estimated delivery date)	The date or window shown before purchase, which the network then has to keep. A static promise is written once ("ordered before 22:00, delivered tomorrow"); the **EDD** is the same thing computed from cut-off, backlog and lane performance.	Getting It There: Carriers, Labels, Tracking and the Delivery Promise, Case: Amazon, From Monolith to Services to the Everything Store
Rate shopping / multi-carrier platform	**Rate shopping** picks the cheapest or fastest carrier service per parcel. A **multi-carrier platform** sits between merchant and carriers: one API for labels and tracking, one event vocabulary.	Getting It There: Carriers, Labels, Tracking and the Delivery Promise
POS (point of sale)	The till software that records store sales, takes terminal payments, issues receipts and counts store stock; at Kestrel & Pine a cloud POS and the system of record for store stock.	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Meet Kestrel & Pine

Term	Definition	Chapter
Terminal provider	The company that accepts card payments at the store counter, with its own references and payout files, separate from the online PSP.	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Taking the Money: Payment Providers, Authorization, Capture and Refunds
Fiscalization	A legal requirement that every sale be recorded in a way the tax authority can verify, in some countries with a certified device that signs each transaction (Germany's TSE).	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation
Click-and-collect (BOPIS)	Order online, pick up in store (buy online, pick up in store), from store stock or from warehouse stock sent to the store; it needs store stock visible to availability and a confirmation step.	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
Ship-from-store / return-in-store / endless aisle	**Ship-from-store** sends online orders from a store's stock; **return-in-store** accepts an online order back at the till with one refund path; **endless aisle** lets the till sell stock held elsewhere and ship it to the customer.	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
Offline operation	The store's ability to keep selling when the connection drops; for those hours it is its own system of record, and the replayed sales must be deduplicated.	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment

Term	Definition	Chapter
RFID	Radio tags on every item with readers at the door and counter, which make store stock accurate enough to promise from; common in fashion at Group scale.	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
RMA (return merchandise authorization)	The merchant's record that a return of these items was requested and allowed: the merchant's side of the promise "you may send this back".	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics
Reverse fulfillment / inspection	**Reverse fulfillment** is the warehouse's work to process a return against its RMA; **inspection** is looking at each item and grading it.	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics
Disposition	The warehouse's verdict on a returned item: back to stock, needs processing, discard or missing. It is the first moment the merchant knows what it got back, so refunds and restocking should follow it.	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Journey 2: The Jacket Comes Back
Exchange	A return paired with a new order for a different size, color or item, with any price difference settled as a balance; at Kestrel & Pine a held order released by the disposition.	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Journey 2: The Jacket Comes Back

Term	Definition	Chapter
Refund on inspection	Refunding only after the returned item has been graded, rather than on request, carrier scan or receipt; it costs the customer days and protects against empty-box and wrong-item returns.	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics
Reverse logistics	The trade's name for the whole backward operation: labels, carriers, receiving, grading, resale, refurbishing and recycling.	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics
Returns portal	The customer-facing page and service where a return is requested, a reason chosen, a label issued, and an exchange or refund selected.	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Journey 2: The Jacket Comes Back
Transactional notification	A message triggered by an event on the customer's own order (confirmed, shipped, delivered, refunded), as opposed to marketing; sent by a notification service that records what it sent.	Where Is My Order? Notifications, Service and the Customer Record
Order status page	The self-service page where the customer sees where the order is: a read model of the order record plus the tracking stream.	Where Is My Order? Notifications, Service and the Customer Record

Term	Definition	Chapter
WISMO ("where is my order")	The service contact that asks for delivery status, typically the largest category of tickets; a signal of how well status flows between systems.	Where Is My Order? Notifications, Service and the Customer Record
Service view / self-service	The **service view** shows an agent one order across systems with actions gated on its state. **Self-service** lets customers cancel, change an address or start a return within the same gates.	Where Is My Order? Notifications, Service and the Customer Record

Part V — CHANNELS: Selling Everywhere

Channels where an outside party sets the rules: marketplaces, business buyers, trading partners and the back office. Selling on Someone Else's Platform: Marketplaces and Channel Integration owns the listing, channel allocation and the connector; Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals the company account, contract pricing, quotes, terms and approvals; Documents Between Companies: EDI, Punchout and E-Invoicing EDI, the ASN, the SSCC, punchout and Peppol; The Back Office: ERP Integration and Who Is the Hub the ERP, inventory valuation and the integration hub.

Term	Definition	Chapter
Marketplace	A shop that sells other merchants' products next to its own (Amazon, bol, Zalando), with its own rules, scorecard and penalties.	Selling on Someone Else's Platform: Marketplaces and Channel Integration

Term	Definition	Chapter
Seller-fulfilled / marketplace-fulfilled	In the **seller-fulfilled** model the merchant ships marketplace orders from its own stock; in the **marketplace-fulfilled** model the marketplace ships from stock sent to it in advance, a location outside the merchant's control.	Selling on Someone Else's Platform: Marketplaces and Channel Integration
Listing / offer	A product's presence on a marketplace: content, images, attributes and GTIN (the listing) and price, stock and delivery terms (the offer), each validated by the marketplace.	Selling on Someone Else's Platform: Marketplaces and Channel Integration
Seller performance metrics	The marketplace's scorecard on each seller: order defect rate, seller cancellation rate, late shipment rate, valid tracking rate and on-time delivery, with thresholds that can lead to strikes or suspension.	Selling on Someone Else's Platform: Marketplaces and Channel Integration
Channel allocation	Deciding how much available stock each channel may promise: a shared pool, a fixed quantity, a buffer, a percentage split, or available-to-promise computed per channel.	Selling on Someone Else's Platform: Marketplaces and Channel Integration, Is It in Stock? Inventory, Availability and the Promise You Can Keep

Term	Definition	Chapter
Channel connector (channel integration)	A service that moves listings, stock, prices and orders between a merchant and many marketplaces; at Kestrel & Pine "the marketplace connector".	Selling on Someone Else's Platform: Marketplaces and Channel Integration, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026
Repricing	An automated rule that moves a listing's price within a floor and a ceiling in reaction to competitors.	Selling on Someone Else's Platform: Marketplaces and Channel Integration
Price parity	A marketplace's check that the merchant's price there is not higher than elsewhere; a channel price drifting can trigger a warning.	Selling on Someone Else's Platform: Marketplaces and Channel Integration, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
Age monitor	For each channel, the time since the marketplace last accepted a stock update, with an alert when it grows; the cheapest control against stale stock.	Selling on Someone Else's Platform: Marketplaces and Channel Integration
Company account	A buying organization with several locations, users and roles (who may order, approve or see invoices), instead of one person with one login.	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals

Term	Definition	Chapter
Contract / tiered pricing	**Contract pricing** is a price agreed with one customer or group, usually with an effective period; **tiered** (volume) pricing gives price breaks by quantity.	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
Quote	The seller's offer at a stated price with a validity date that the buyer can accept into an order; the one place a price carries an expiry.	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
Payment terms (net 30)	The agreed time to pay after the invoice, such as net 30 (due 30 days after the invoice); typed data, not free text, and payment becomes a receivable in the ERP rather than a PSP event.	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, From Cart to Confirmation: Checkout Orchestration
Credit limit / approval workflow	The **credit limit** is the seller's control: the most a customer may owe. The **approval workflow** is the buyer's own control: an order above an amount needs a manager.	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals
Pre-season order / in-season reorder	In apparel wholesale, an order placed months ahead for delivery in a window, often on better terms, followed by reorders of what sells.	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals

Term	Definition	Chapter
Punchout (cXML, OCI)	A buyer's procurement system opens the supplier's shop inside its own screens; the cart returns to the procurement system for approval, and the order arrives afterwards as a separate document (cXML OrderRequest or EDI ORDERS). A punchout cart is not an order.	Documents Between Companies: EDI, Punchout and E-Invoicing, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals
EDI (electronic data interchange)	Orders, despatch advices and invoices exchanged between companies' systems as structured messages in an agreed standard, sent over an agreed transport and translated by a mapping per partner.	Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice
Message / standard / transport / mapping	A **message** is a document such as an order; a **standard** defines its fields and order; a **transport** is how the file travels; a **mapping** translates between a partner's guideline and your own records, maintained per partner.	Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice
ORDERS / ORDRSP / INVOIC	EDIFACT messages for the purchase order, the order response (accepted, changed or rejected) and the invoice (X12 850, 855 and 810).	Journey 3: A Chain Reorders by EDI, From Order to Invoice, Documents Between Companies: EDI, Punchout and E-Invoicing

Term	Definition	Chapter
DESADV / ASN (advance shipping notice)	The despatch advice: what is on its way, pallet by pallet, so the receiver can scan instead of count (X12 856). An ASN is only as right as the system that generated it, so it should come from the warehouse at pack time.	Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice, Pick, Pack, Ship: Warehouse Management and Fulfillment
CONTRL / RECADV	**CONTRL** is a syntax receipt ("your message parsed"), saying nothing about meaning; **RECADV** is the buyer's receiving advice of what it counted in.	Journey 3: A Chain Reorders by EDI, From Order to Invoice
GLN (Global Location Number)	The GS1 number that identifies who and where: a legal entity or a physical place such as a distribution center.	Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice
SSCC (Serial Shipping Container Code)	The 18-digit GS1 number on a logistics unit's label that identifies this pallet or carton; the DESADV lists the contents per SSCC.	Pick, Pack, Ship: Warehouse Management and Fulfillment, Documents Between Companies: EDI, Punchout and E-Invoicing
VAN (value-added network) / AS2 / SFTP	Transports for EDI: a **VAN** is a provider-run mailbox network that relays to every partner on it; **AS2** sends signed, encrypted documents over HTTP with a signed receipt; **SFTP** is secure file transfer polled on a schedule.	Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice

Term	Definition	Chapter
Compliance chargeback	A deduction a retail buyer makes from the supplier's payment for a late, incomplete or wrongly documented delivery: a missing ASN, an unscannable label, an ASN that does not match receipt.	Documents Between Companies: EDI, Punchout and E-Invoicing
Peppol / access point / four-corner model	**Peppol** is an open network run by OpenPeppol in which each party connects through an accredited **access point** and reaches any participant: sender, sender's access point, receiver's access point, receiver (the **four-corner model**).	Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice
ERP (enterprise resource planning)	The system that keeps the books: ledger, purchasing, inventory valuation, invoices and credit notes, receivables and payables, and the item master. At Kestrel & Pine it is held to finance only.	The Back Office: ERP Integration and Who Is the Hub, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026
Item master	The ERP's record of each item for purchasing and costing: SKU, GTIN, supplier, cost, purchasing unit, weight and dimensions.	The Back Office: ERP Integration and Who Is the Hub, What Are We Selling? Products, Variants and the Catalog

Term	Definition	Chapter
Inventory valuation / costing method / landed cost	What the stock is worth: a cost per unit under a **costing method** (such as FIFO or weighted average) built from **landed cost**, the purchase price plus freight and duty.	The Back Office: ERP Integration and Who Is the Hub
Integration hub	The one system every other system connects through and that owns the shared records: the ERP, the commerce platform or an OMS, or no hub at all with an iPaaS.	The Back Office: ERP Integration and Who Is the Hub
Master data ownership	The rule that says which system may create and change each kind of slow-changing reference record (items, customers, prices, locations).	The Back Office: ERP Integration and Who Is the Hub, One Customer, One Product: Identity and Master Data Across Systems
ERP implementation partner	The outside firm that configured the ERP and built its connectors; an architectural dependency most merchants do not list.	The Back Office: ERP Integration and Who Is the Hub

Part VI — INTEGRATION: Making Separate Systems Behave Like One

How separate systems move facts, survive repeats and gaps, keep their promises to each other, recognize the same customer and product, and count the same thing. APIs, Events, Files and Middleware: How Commerce Systems Talk owns the styles and the inventory; Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation events, idempotency, the outbox, compensation and reconciliation; Promises Between Systems: API Contracts, Webhooks and Versioning the contract, the version and the webhook; One Customer, One Product: Identity and Master Data Across Systems identity, the

golden record and matching; Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics the warehouse and metric definitions.

Term	Definition	Chapter
API (application programming interface)	The contract by which one program calls another: the operations, fields and errors it accepts and returns.	Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Promises Between Systems: API Contracts, Webhooks and Versioning
Direct API call	The caller asks when it needs the answer and waits for it; right for decisions at checkout, but every callee's outage becomes the caller's.	APIs, Events, Files and Middleware: How Commerce Systems Talk
Webhook	An HTTP request a source system sends to a URL you registered when something happens. It is a notification, not a guarantee: deliveries can be missed, duplicated or out of order, and a failing receiver can lose its subscription.	APIs, Events, Files and Middleware: How Commerce Systems Talk, Promises Between Systems: API Contracts, Webhooks and Versioning
Polling / sweep	The receiver asks on a timer what changed since its last visit; the periodic **sweep** is the safety net under webhooks.	APIs, Events, Files and Middleware: How Commerce Systems Talk, Promises Between Systems: API Contracts, Webhooks and Versioning
Scheduled batch / file transfer	Either side sends a file or batch on a clock; stale by design, and a late or partial file is a quiet failure.	APIs, Events, Files and Middleware: How Commerce Systems Talk

Term	Definition	Chapter
Queue / event stream	A **queue** is a buffer in which work waits until the next system can take it; an **event stream** lets many consumers read published events at their own pace. Delivery is at least once and ordering holds only per key.	APIs, Events, Files and Middleware: How Commerce Systems Talk, Black Friday Every Year: Peak Load, Resilience and Recovery
CDC (change data capture)	Reading a database's own change log with a connector to turn row changes into messages; a row change still has to be translated into a business event.	APIs, Events, Files and Middleware: How Commerce Systems Talk
Event notification / event-carried state transfer	Two meanings of event-driven (Martin Fowler, 2017): the event says only that something changed and receivers call back for details, or the event carries the data and receivers keep their own copy.	APIs, Events, Files and Middleware: How Commerce Systems Talk
iPaaS (integration platform as a service)	A hosted tool that runs data flows between systems: connectors, mapping, retries, errors and monitoring. A hub without owners recreates the tangle in one place.	APIs, Events, Files and Middleware: How Commerce Systems Talk, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026
Point-to-point integration	A direct connector between one pair of systems; their number multiplies with every system added.	APIs, Events, Files and Middleware: How Commerce Systems Talk

Term	Definition	Chapter
Integration inventory	The list of every integration with its owner, style, direction, fields, frequency, system of record and pinned API version; the exit's work list and the first Waypoint document.	APIs, Events, Files and Middleware: How Commerce Systems Talk, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance
Integration contract	A short written agreement for one flow: fields, direction, frequency, source of truth, and what happens on a repeat or a failure.	Who Builds and Who Owns: Teams, Vendors, Agencies and Governance, APIs, Events, Files and Middleware: How Commerce Systems Talk
Event	A message sent the moment something happens (an order placed, a price changed) so other systems can react.	Case: Zalando, Microservices, Platform Strategy and the Return to Discipline, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation
Eventual consistency	The promise that is possible across separate systems: copies may disagree for a while, and if updates stop they all converge on the last value.	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: Amazon, From Monolith to Services to the Everything Store

Term	Definition	Chapter
At-least-once delivery	A message will arrive one or more times; the receiver, not the transport, must make repeats harmless.	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Where Is My Order? Notifications, Service and the Customer Record
Idempotency	Handling the same message twice has the same effect as handling it once: a second "refund order #1072455" must not produce a second refund.	Follow the Order: What Happens Between "Buy" and "Delivered", Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation
Dead-letter queue (DLQ)	Where a message goes after failing a set number of times, so it is neither lost nor retried forever; someone must own and empty it.	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, APIs, Events, Files and Middleware: How Commerce Systems Talk
Outbox	Writing the event to a table in the same database transaction as the change it describes, then publishing it from there, so "the order was saved" and "the event was sent" become one act and no event is lost.	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation

Term	Definition	Chapter
Saga / compensation	A **saga** is a long-lived transaction written as a sequence of smaller ones (Garcia-Molina and Salem, 1987); when a later step fails, **compensation** reverses earlier steps by new business actions, since they cannot be rolled back. A canceled shipped order becomes a return and a refund.	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, The Order Record: Lifecycle, State and the Order Management System
Choreography / orchestration	Two ways to run a multi-step flow: each participant reacts to events (choreography, low coupling, the flow is in nobody's code), or one orchestrator, usually the OMS, decides the next step and shows where every order is.	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation
Two-phase commit	Both systems commit together or not at all; it needs both vendors' cooperation, which SaaS vendors do not give, so commerce flows use events and reconciliation instead.	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation
Reconciliation job	A scheduled job that compares two systems' records by a matching key within a tolerance and sends the differences to a named owner, who repairs the source, not the copy.	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Is It in Stock? Inventory, Availability and the Promise You Can Keep

Term	Definition	Chapter
API contract	The written description of what a system promises to accept and return: operations, fields, errors, rate limits, delivery guarantees and lifetime. A contract has shape, meaning, timing and lifetime.	Promises Between Systems: API Contracts, Webhooks and Versioning
Versioning / API version	Naming generations of a contract so a consumer can say which one it relies on; a vendor's **API version** is a dated edition it promises to keep working for a stated period.	Promises Between Systems: API Contracts, Webhooks and Versioning, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance
Deprecation / sunset	**Deprecation** is the announced, dated withdrawal of part of a contract; the **sunset** is the date it stops working. Missing one is the Deprecated Endpoint.	Promises Between Systems: API Contracts, Webhooks and Versioning
Silent fall-forward	A consumer on a retired or unstated API version is served a newer one without an error, so a field goes missing or changes meaning unnoticed.	Promises Between Systems: API Contracts, Webhooks and Versioning, Apps, Plugins and Functions: Extensibility and Ecosystem Governance

Term	Definition	Chapter
Rate limit	The maximum number of API calls, or units of query cost, a caller may make in a period; above it the provider answers with an error. On a shared platform every app and connector spends the same budget.	Promises Between Systems: API Contracts, Webhooks and Versioning, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation
Sandbox	A copy of a service where nothing real happens, used to test a new API version or an app before production.	Who Builds and Who Owns: Teams, Vendors, Agencies and Governance, Promises Between Systems: API Contracts, Webhooks and Versioning
Schema registry / contract test	A **schema registry** holds the agreed shape of each event type so producers cannot break consumers; a **contract test** checks a provider against its consumers' expectations in the delivery pipeline.	Promises Between Systems: API Contracts, Webhooks and Versioning, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline
Identifier	A value a system uses to tell one record from another: a customer id, an e-mail address, an order number, a barcode number.	One Customer, One Product: Identity and Master Data Across Systems
Customer identity	The set of identifiers that different systems use for the same person, plus the rules that link them; the core of the Familiar Stranger.	One Customer, One Product: Identity and Master Data Across Systems

Term	Definition	Chapter
Master data / MDM	**Master data** is the slowly changing reference data that transactions point to: customers, products, suppliers, locations. **MDM** (master data management) is the discipline and tooling that keeps one owned record of each.	One Customer, One Product: Identity and Master Data Across Systems, The Back Office: ERP Integration and Who Is the Hub
Matching / survivorship	**Matching** decides that two records with no shared identifier refer to the same thing; **survivorship** is the rule that decides which value wins per field when they merge.	One Customer, One Product: Identity and Master Data Across Systems
Golden record	The one assembled record per customer or product, built by merging sources under survivorship rules.	One Customer, One Product: Identity and Master Data Across Systems
Cross-reference / merge log	For each master record, every other system's identifier for it and every link or merge made, kept reversibly so a wrong merge can be undone.	One Customer, One Product: Identity and Master Data Across Systems
CDP (customer data platform)	A product that builds a unified profile per person from every touchpoint for audiences and personalization; a consumer of the customer master, not a replacement for it.	One Customer, One Product: Identity and Master Data Across Systems

Term	Definition	Chapter
Operational / analytical data	**Operational data** is the live state of the business (this order's status); **analytical data** is the history laid out for questions across all records (orders by channel by week).	Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics
Data warehouse	A database built for large analytical queries, loaded with copies of every system's data and modeled into tables analysts can use; a copy, never a system of record.	Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics
Metric definition / metrics layer	A **metric definition** is the written rule for computing a business number (what is counted, which records, which date, which tax treatment). The **metrics layer** defines each metric once, as code, for every dashboard.	Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics
Net sales	Kestrel & Pine's one definition (W7): merchandise value excluding VAT, after discounts, minus returns and cancellations in the month they are refunded, excluding shipping charges and payment fees.	Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics

Term	Definition	Chapter
Sell-through / weeks of cover	**Sell-through** is units sold divided by units received for a season; **weeks of cover** is stock on hand divided by average weekly sales. Both drive buying and markdown decisions.	Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics

Part VII — PLATFORM: How Commerce Platforms Are Built

How the software itself is structured, shared, extended, delivered to the browser and kept up on the busiest day. Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says owns the styles; Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation tenancy, isolation, pods and rate limits; Apps, Plugins and Functions: Extensibility and Ecosystem Governance apps, plugins and functions; Delivering the Storefront: Themes, Headless, Rendering and Performance themes, headless, rendering and Core Web Vitals; Black Friday Every Year: Peak Load, Resilience and Recovery the peak plan, backpressure, graceful degradation and the load test.

Term	Definition	Chapter
Architecture style	The shape of a system as deployed and run: how many separately running programs there are, which data each owns, and how they talk.	Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says
Deployment unit	One program that is built, tested, released and run as a whole; splitting a system means creating more of them.	Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says

Term	Definition	Chapter
Monolith / modular monolith	A **monolith** is one deployable application holding all the shop's logic, usually with one shared database. A **modular monolith** keeps the single deployment but divides the code into named modules with enforced boundaries.	Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: Shopify, the Modular Monolith That Serves Millions of Merchants
Microservices	Many small, separately deployed services, each owned by one team, owning its own data and reached only over the network; they buy team independence at the price of operations and consistency work.	Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline
Tenant / multi-tenancy	A **tenant** is one customer of a shared platform with its own data and configuration (at a commerce platform, one shop). **Multi-tenancy** is one running system serving many tenants.	Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Case: Shopify, the Modular Monolith That Serves Millions of Merchants
Isolation / silo / pool	**Isolation** is how far one tenant's load, data and failures are kept from another's. In the **silo** model tenants get dedicated resources; in the **pool** model they share them, kept apart by a tenant identifier.	Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation

Term	Definition	Chapter
Sharding / shard key	Splitting one database into many slices (**shards**), each holding part of the data; the **sharding key** is the field that decides where a row lives, for example the shop.	Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Case: Shopify, the Modular Monolith That Serves Millions of Merchants
Pod	An isolated group of tenants with their own databases behind shared web servers; Shopify's 2016 design, in which a routing layer sends each request to one pod and shops can be moved between pods.	Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Case: Shopify, the Modular Monolith That Serves Millions of Merchants
Noisy neighbor	A tenant whose load harms others on shared resources; in a pool, one tenant's flash sale is every tenant's load unless the vendor meters it.	Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation
In-process plugin (module, extension)	Custom code that runs inside the platform's own program and can reach anything, including the database; flexible, and re-tested on every release.	Apps, Plugins and Functions: Extensibility and Ecosystem Governance
App / app store	An **app** is a separately hosted program that extends a platform through its APIs and webhooks within granted scopes; an **app store** is where the vendor lists, reviews and bills for third-party apps.	Apps, Plugins and Functions: Extensibility and Ecosystem Governance

Term	Definition	Chapter
Scopes / least scope	The permissions an app is granted (read orders, write inventory); least scope means asking only for what it needs. Apps that may write stock, prices or orders need an owner.	Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers
Extension point (function, call-out)	A place the platform lets custom logic run: a sandboxed serverless function with a fixed input and output, or a synchronous call-out to the merchant's endpoint before saving.	Apps, Plugins and Functions: Extensibility and Ecosystem Governance
Theme / theme code	A **theme** is the set of templates, settings and scripts the platform's own renderer turns into pages; customizing it heavily makes updates merges.	Delivering the Storefront: Themes, Headless, Rendering and Performance, Apps, Plugins and Functions: Extensibility and Ecosystem Governance
Storefront	The software that produces the pages shoppers see; the architectural question is where it is rendered, from which data, and who owns the machinery.	Delivering the Storefront: Themes, Headless, Rendering and Performance
Server-side / client-side rendering	In **server-side rendering** a server builds the complete HTML page; in **client-side rendering** the browser receives JavaScript and builds the page itself from fetched data.	Delivering the Storefront: Themes, Headless, Rendering and Performance

Term	Definition	Chapter
Headless / storefront API	**Headless** means the storefront is a separate application that talks to the commerce engine only through an API, typically a **storefront API**; checkout usually stays with the engine.	Delivering the Storefront: Themes, Headless, Rendering and Performance, Case: commercetools, Shopware and the Composable Movement
Edge / CDN (content delivery network)	Servers close to shoppers that serve cached pages and images (the **CDN**) or run the merchant's front-end code (an **edge runtime**); at peak they absorb most traffic before it reaches the application.	Delivering the Storefront: Themes, Headless, Rendering and Performance, Black Friday Every Year: Peak Load, Resilience and Recovery
Core Web Vitals (LCP, INP, CLS)	Google's three field metrics for page experience, assessed at the 75th percentile of real visits: largest contentful paint (good is 2.5 seconds or less), interaction to next paint (200 milliseconds or less) and cumulative layout shift.	Delivering the Storefront: Themes, Headless, Rendering and Performance
PWA (progressive web app)	A website that can be installed like an app, work offline and send push notifications.	Delivering the Storefront: Themes, Headless, Rendering and Performance
Peak	A period, planned or not, in which traffic and orders are many times their normal level: a seasonal peak, a launch or drop, a flash sale, an unplanned moment.	Black Friday Every Year: Peak Load, Resilience and Recovery

Term	Definition	Chapter
Capacity planning / load test	**Capacity planning** decides before the day how many orders a minute each link must carry; a **load test** sends realistic artificial traffic to find where the chain breaks before customers do.	Black Friday Every Year: Peak Load, Resilience and Recovery
Backpressure	Letting a slow downstream system slow the producers, or letting work pile up in a queue, instead of failing the customer-facing step: accept and persist the order fast, queue everything behind it.	Black Friday Every Year: Peak Load, Resilience and Recovery
Cache / waiting room	A **cache** is a stored copy served instead of recomputing or refetching. A **waiting room** is a virtual queue in front of the storefront that admits shoppers at a set rate.	Black Friday Every Year: Peak Load, Resilience and Recovery, Delivering the Storefront: Themes, Headless, Rendering and Performance
Graceful degradation / degradation ladder	Switching off non-essential features in a planned order so essential ones keep working; the **ladder** lists each switch with an owner and a trigger.	Black Friday Every Year: Peak Load, Resilience and Recovery
Orders per minute	The one unit in which every link of the chain is measured and forecast on a peak day, and the health metric of a commerce system.	Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour

Term	Definition	Chapter
Incident commander / runbook / change freeze	The **incident commander** decides during an incident; a **runbook** is a written list of symptoms, checks and actions; a **change freeze** stops changes to apps, theme and connectors around a peak.	Black Friday Every Year: Peak Load, Resilience and Recovery

Part VIII — TRUST: Security, Privacy and Regulation

What attackers try, what the law asks of customer data, and how rules become requirements. Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers owns card testing, PCI scope, skimming and bots; Whose Data Is It? Privacy, Consent and Retention the controller, consent, requests and retention; Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation the regulatory calendar and the requirement card. The laws themselves are in the standards, protocols and regulations table below.

Term	Definition	Chapter
Card testing	Bots trying stolen card numbers in many small payments or card-save attempts to find which work; the merchant pays a fee for every attempt, declined or not.	Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers

Term	Definition	Chapter
Credential stuffing / account takeover	**Credential stuffing** is the automated injection of stolen username and password pairs; **account takeover** is any route that ends with an attacker logged in as a customer or staff member. Multi-factor authentication is the primary countermeasure.	Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers
Web skimming (Magecart)	Malicious script on a checkout page that copies what the shopper types into the card fields; prevented by a script inventory, integrity checks and change detection on payment pages.	Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers
Coupon abuse / refund abuse / scalping	Abuse that costs margin rather than data: guessing or sharing codes, refunds to another card and empty-box returns, and bots that hoard or buy out scarce stock in a launch.	Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers
Supply-chain risk	An app, plugin, theme or hosted script that is compromised and runs with your privileges; managed through app governance.	Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers, Apps, Plugins and Functions: Extensibility and Ecosystem Governance

Term	Definition	Chapter
PCI scope / SAQ	Which systems touch card data and so fall under PCI DSS; the **SAQ** (self-assessment questionnaire) the merchant signs ranges from SAQ A for a PSP-hosted page or hosted fields to SAQ D for raw card data on its own servers.	Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers, Taking the Money: Payment Providers, Authorization, Capture and Refunds
CSP / SRI	A **Content Security Policy** is an HTTP header that tells the browser which script sources a page may load; **Subresource Integrity** is a hash on a script tag so the browser refuses a changed script.	Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers
Bot management / scraping policy	Controls that tell bots from shoppers per endpoint: allow product pages, challenge cart and checkout, rate-limit the storefront API.	Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers, Delivering the Storefront: Themes, Headless, Rendering and Performance
Personal data	Any information relating to an identified or identifiable person: a name, an e-mail, an order with an address, a cookie id tied to a browser.	Whose Data Is It? Privacy, Consent and Retention

Term	Definition	Chapter
Controller / processor	The **controller** decides why and how personal data is used; a **processor** handles it only on the controller's instructions. Kestrel & Pine is the controller; its SaaS vendors are mostly processors.	Whose Data Is It? Privacy, Consent and Retention
DPA (data processing agreement)	The contract the GDPR requires between a controller and each processor.	Whose Data Is It? Privacy, Consent and Retention
Lawful basis	The legal reason a controller may use personal data at all; for commerce mostly consent, performance of a contract, a legal obligation and legitimate interest.	Whose Data Is It? Privacy, Consent and Retention
Consent / CMP / soft opt-in	**Consent** is a recorded, freely given choice per purpose that must travel to every tool that acts on it. A **consent management platform** (CMP) is the banner and the record behind it. The **soft opt-in** lets a shop e-mail existing customers about similar products until they object.	Whose Data Is It? Privacy, Consent and Retention, One Customer, One Product: Identity and Master Data Across Systems
DSR (data subject request)	A person exercising a right: send me my data, correct it, delete it; it must be verified, fanned out to every system and answered within a month.	Whose Data Is It? Privacy, Consent and Retention, One Customer, One Product: Identity and Master Data Across Systems

Term	Definition	Chapter
Retention / erase, pseudonymize, retain	**Retention** is how long data is kept: no longer than the purpose needs and at least as long as tax law demands. Per system the action is to **erase**, **pseudonymize** (keep the record, remove the person) or **retain**.	Whose Data Is It? Privacy, Consent and Retention
Identity index / request orchestrator	The list of which systems hold a person under which identifiers, and the small workflow that verifies a request, fans it out and records each answer.	Whose Data Is It? Privacy, Consent and Retention
Requirement card / regulatory calendar	A rule turned into one card (scope, data requirement, system of record, channels, test, date, owner) attached to the system-of-record map, and the dated list of rules coming into force.	Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation

Part IX — JOURNEYS: One Order, End to End

The journeys follow one order through every system and name, at each step, the system, the data, the promise and the failure. The vocabulary is defined in the parts above; these rows hold the few terms the journeys add. The Kestrel & Pine orders themselves are in the fictional names section.

Term	Definition	Chapter
Journey (in this book)	The path of one concrete order through every system, told as a sequence diagram and a step table; the book uses "journey" only in this sense.	Part IX — JOURNEYS: One Order, End to End, Journey 1: Eva's Jacket, From Search to Doorstep
Happy path / variant path	The **happy path** is the journey with no failure; a **variant path** branches from it (a refund instead of an exchange, a marketplace or store return, a seller-fulfilled versus marketplace-fulfilled order).	Journey 1: Eva's Jacket, From Search to Doorstep, Journey 2: The Jacket Comes Back
Partner card	One record per EDI trading partner holding its message versions, mapping, identifiers (GLN), transport and contacts; Waypoint W6 requires one per partner.	Journey 3: A Chain Reorders by EDI, From Order to Invoice, Documents Between Companies: EDI, Punchout and E-Invoicing

Part X — EVOLUTION: Decisions That Change With Scale

Choosing, changing and organizing platforms, and where AI fits. Build, Buy or Assemble: Choosing and Changing Platforms owns the four options, total cost of ownership and exit cost; Moving Without Burning the Bridge: Replatforming and Migration the strangler fig, parallel run and cutover; What Changes at Each Scale: From First Store to Group the stage map; Who Builds and Who Owns: Teams, Vendors, Agencies and Governance ownership and the decision record; Assistants and Agents: AI in Commerce and What Is Real in 2026 models, agents and the commerce protocols.

Term	Definition	Chapter
Build / buy / assemble	The four ways to get a commerce capability: a suite, a hosted platform plus apps, a composable assembly of API-only services, or a custom build. Build only what differentiates; buy the rest.	Build, Buy or Assemble: Choosing and Changing Platforms
TCO (total cost of ownership)	Everything a choice costs over its life (licenses, fees, agency, people, integrations, upgrades and the exit), not what the contract says per month.	Build, Buy or Assemble: Choosing and Changing Platforms
Vendor lock-in / exit cost	**Lock-in** is the condition in which leaving a vendor costs more than staying, whatever it does. **Exit cost** is the price of leaving (data, URLs, customizations, apps, integrations, contract), designed in at purchase or never.	Build, Buy or Assemble: Choosing and Changing Platforms
Replatforming	Moving a capability, or the whole shop, from one platform to another; safe when done seam by seam, dangerous on one date.	Moving Without Burning the Bridge: Replatforming and Migration
Strangler fig (strangler migration)	Martin Fowler's gradual approach to replacing a system: route one seam at a time (a channel, a capability, a traffic slice) to the new system until the old one can be retired.	Moving Without Burning the Bridge: Replatforming and Migration, Build, Buy or Assemble: Choosing and Changing Platforms

Term	Definition	Chapter
Transitional architecture	Temporary components built only to make a migration safe and removed afterwards: event interception, diverting a flow, a legacy mimic.	Moving Without Burning the Bridge: Replatforming and Migration
Dark launch / canary release	A **dark launch** runs the new path without showing its result; a **canary release** sends a small share of traffic to it first.	Moving Without Burning the Bridge: Replatforming and Migration
Shadow / parallel run	In a **shadow** the new system receives a copy and nobody reads it; in a **parallel run** both systems process the same input and their results are compared while only the old one is acted on.	Moving Without Burning the Bridge: Replatforming and Migration
Cutover / redirect map	A **cutover** is the moment a seam's traffic is routed to the new owner. The **redirect map** sends every old product and category URL to its new address, or rankings and bookmarks are lost.	Moving Without Burning the Bridge: Replatforming and Migration
ADR (architecture decision record)	A one-page note of a decision, its context and its consequences, kept with the systems it changes.	Who Builds and Who Owns: Teams, Vendors, Agencies and Governance

Term	Definition	Chapter
Ownership map	One row per system, integration and app with its business owner, technical owner, builder or vendor, what it may write, its contract, its pinned API version and its monitor.	Who Builds and Who Owns: Teams, Vendors, Agencies and Governance
Model / LLM	A **model** is software trained on data to produce an output from an input; a **large language model** (LLM) is one trained on text that can read and write natural language.	Assistants and Agents: AI in Commerce and What Is Real in 2026
Assistant / agent	An **assistant** is an LLM in a product that helps a person; an **agent** is an assistant given tools and permission to use them, so it can create a cart, place an order or change a stock figure.	Assistants and Agents: AI in Commerce and What Is Real in 2026
Agentic commerce	Agents that discover, compare and buy on a shopper's behalf, inside a chat product or a marketplace app.	Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Amazon, From Monolith to Services to the Everything Store
Commerce protocol	A published contract (feed, cart, checkout, order, payment token) that lets any agent talk to any merchant without a bespoke integration, as EDI lets any retailer send an order.	Assistants and Agents: AI in Commerce and What Is Real in 2026

Term	Definition	Chapter
MCP (Model Context Protocol)	The transport layer for agents: how an agent discovers and calls tools such as "search catalog" or "create cart".	Assistants and Agents: AI in Commerce and What Is Real in 2026
ACP / UCP / AP2	Agentic commerce protocols as of 2026: **ACP**, OpenAI and Stripe's specification (feed, checkout, delegated payment, later cart and orders); **UCP**, the Google-led consortium standard (catalog to order, MCP built in); **AP2**, the payment layer under UCP, with signed mandates as an audit trail of what the shopper authorized.	Assistants and Agents: AI in Commerce and What Is Real in 2026
Agent identity	Proof of which agent is calling and for whom, the layer the protocols assume; Web Bot Auth, for example, has the agent sign each request with published keys.	Assistants and Agents: AI in Commerce and What Is Real in 2026
Agent readiness	Making a merchant usable by agents without building one: accurate product data, availability and the delivery promise by API, and order status by API (Waypoint W8).	Assistants and Agents: AI in Commerce and What Is Real in 2026

Part XI — PRACTICE: How Others Built It

Public names and programs the four case studies rely on, described only as their dated sources describe them; the case chapters say where the public evidence stops.

Term	Definition	Chapter
MRR / subscription and merchant solutions	Shopify's reporting terms: **MRR** (monthly recurring revenue) is the subscription run-rate at period end; **subscription solutions** are plans, platform fees and apps; **merchant solutions** are payments, lending, labels and other revenue that rises with merchants' sales.	Case: Shopify, the Modular Monolith That Serves Millions of Merchants
Packwerk	Shopify's open-source tool, from 2020, that checks references between packages of its modular monolith and records existing violations so teams can stop new ones without fixing everything at once.	Case: Shopify, the Modular Monolith That Serves Millions of Merchants, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says
Checkout extensibility / Shopify Functions	What replaced Shopify's editable checkout template: fixed slots for app extensions, a branding API, web pixels, and **Functions**, backend logic compiled to WebAssembly and run by the platform.	Case: Shopify, the Modular Monolith That Serves Millions of Merchants, Apps, Plugins and Functions: Extensibility and Ecosystem Governance

Term	Definition	Chapter
Hydrogen / Oxygen	Shopify's headless toolkit (a React-based framework) and the edge hosting that runs it; it ships no checkout.	Case: Shopify, the Modular Monolith That Serves Millions of Merchants, Delivering the Storefront: Themes, Headless, Rendering and Performance
Sidekick / Agentic plan	Shopify's AI assistant in the merchant admin, acting with the merchant's approval, and its plan that syndicates products to AI chat products with checkout in the conversation.	Case: Shopify, the Modular Monolith That Serves Millions of Merchants, Assistants and Agents: AI in Commerce and What Is Real in 2026
Two-pizza team	Amazon's rule that a team should be small enough to be fed by two pizzas, tied to owning a service end to end.	Case: Amazon, From Monolith to Services to the Everything Store, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says
Externalizable interface	An API designed so that it could be offered to outside developers, which forces a clean contract even when only internal teams use it.	Case: Amazon, From Monolith to Services to the Everything Store
Dynamo / conflict resolution	Amazon's 2007 paper on the data store behind its shopping cart, built so an add-to-cart is never refused; **conflict resolution** merges copies that disagree later.	Case: Amazon, From Monolith to Services to the Everything Store

Term	Definition	Chapter
Regionalization	Holding stock in the region where it will be ordered, so most orders ship from nearby; part of how Amazon keeps its delivery promise.	Case: Amazon, From Monolith to Services to the Everything Store
Alexa for Shopping / Buy for Me	Amazon's shopping assistant (Rufus until May 2026) and its agent features that buy on the shopper's behalf, described as of November 2026.	Case: Amazon, From Monolith to Services to the Everything Store, Assistants and Agents: AI in Commerce and What Is Real in 2026
Radical Agility	Zalando's 2015 organizational model of autonomous teams of three to seven people owning a topic end to end, alongside its move to microservices.	Case: Zalando, Microservices, Platform Strategy and the Return to Discipline
Nakadi / event bus	An **event bus** is the shared channel on which services publish events and others subscribe; **Nakadi** was Zalando's, started in 2015, with a schema registry for event types.	Case: Zalando, Microservices, Platform Strategy and the Return to Discipline
API guidelines / tech radar with teeth	Zalando's published rules for designing APIs (additive change over new versions, media-type versioning) and a list of allowed technologies teams must follow rather than consult.	Case: Zalando, Microservices, Platform Strategy and the Return to Discipline, Promises Between Systems: API Contracts, Webhooks and Versioning

Term	Definition	Chapter
PRAPI / synchronous read API	Zalando's Product Read API, which answers "what is the price and stock now" when asked, replacing a pipeline in which each caller rebuilt its own copy from events.	Case: Zalando, Microservices, Platform Strategy and the Return to Discipline
ZEOS / Connected Retail	**ZEOS** is Zalando's fulfillment and logistics service for brands, from one stock pool, whether or not they sell on Zalando; **Connected Retail** let physical shops sell store stock on Zalando, launched 2018 and to be discontinued by 30 June 2027.	Case: Zalando, Microservices, Platform Strategy and the Return to Discipline, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
commercetools / Shopware / Adobe Commerce / Mage-OS	Reference points of the composable movement: an API-only commerce vendor that by 2026 sold layers beside existing stacks; a mid-market platform that absorbed integration (Nexus) and payments; a suite offered in three deployment shapes; and a community fork of Magento open source.	Case: commercetools, Shopware and the Composable Movement, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026

Standards, Protocols and Regulations

The laws, standards and protocols the chapters cite, with their scope (EU, NL, BE, DE, US or global) and a one-line meaning as of November 2026. They are teaching summaries, not legal text: check the official source and the dates of application in the owning chapter

before a row goes into a contract or a design decision.

Term	Scope	Short name	Meaning	Chapter
GDPR	EU	General Data Protection Regulation	The EU's personal-data law: lawful basis, consent, controller and processor duties, data subject rights answered within a month, transfers outside the EU.	Whose Data Is It? Privacy, Consent and Retention, One Customer, One Product: Identity and Master Data Across Systems
ePrivacy Directive	EU	Cookie and e-mail marketing rule	Reading or storing information on a device needs consent unless strictly necessary (Article 5(3)); the soft opt-in for e-mail to existing customers (Article 13(2)).	Whose Data Is It? Privacy, Consent and Retention

Term	Scope	Short name	Meaning	Chapter
SCCs / EU-US Data Privacy Framework	EU	International transfer mechanisms	Standard contractual clauses (Decision (EU) 2021/914) and the 2023 adequacy decision for certified US companies, the routes for personal data leaving the EU.	Whose Data Is It? Privacy, Consent and Retention
Consumer Rights Directive	EU	Right of withdrawal	Directive 2011/83/EU: 14 days to withdraw from a distance contract without reason, refund within 14 days of the notice (may wait for the goods), pre-contract information, risk with the merchant until delivery.	The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Rules Turned Into Require-ments: Consumer, Payment, Invoicing and Platform Regulation

Term	Scope	Short name	Meaning	Chapter
Omnibus Directive	EU	Prior-price and reviews rule	Directive (EU) 2019/2161, applied from 28 May 2022: a price reduction must show the lowest price of the previous 30 days; adds rules on ranking transparency and fake reviews.	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Finding the Product: Search, Navigation and Merchandising, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation

Term	Scope	Short name	Meaning	Chapter
PSD2 / SCA	EU	Strong customer authentication	The second Payment Services Directive (applied 13 January 2018) requires two of knowledge, possession and inherence when a payer initiates an electronic payment; Delegated Regulation 2018/389 (from 14 September 2019) sets the exemptions.	Taking the Money: Payment Providers, Authorization, Capture and Refunds, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation
PSD3 / Payment Services Regulation	EU	Next payments package	Provisionally agreed November 2025 and pending as of November 2026: more PSP liability for fraud, payee-name checks, charge transparency.	Taking the Money: Payment Providers, Authorization, Capture and Refunds, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation

Term	Scope	Short name	Meaning	Chapter
Interchange Fee Regulation	EU	Card fee cap	Regulation (EU) 2015/751 caps interchange on consumer cards at 0.2% (debit) and 0.3% (credit).	Who Sells What to Whom: Business Models and the Economics That Shape Architecture
PCI DSS	global	Payment Card Industry Data Security Standard	The card brands' security standard for anyone who stores, processes or transmits card data, version 4.0.1 (June 2024); requirements 6.4.3 and 11.6.1 govern scripts and changes on payment pages.	Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers, Taking the Money: Payment Providers, Authorization, Capture and Refunds
Digital Services Act	EU	Marketplace trader rules	Regulation (EU) 2022/2065, applying from 17 February 2024: marketplaces must identify and verify traders before they sell, among other platform duties.	Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Selling on Someone Else's Platform: Marketplaces and Channel Integration

Term	Scope	Short name	Meaning	Chapter
GPSR	EU	General Product Safety Regulation	Regulation (EU) 2023/988, from 13 December 2024: every online offer shows the manufacturer, an EU responsible person where needed, and product identification and warnings.	Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, What Are We Selling? Products, Variants and the Catalog
European Accessibility Act	EU	Accessibility of e-commerce	Directive (EU) 2019/882 covers e-commerce services from 28 June 2025, exempting microenterprises; conformance is usually shown against WCAG.	Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Delivering the Storefront: Themes, Headless, Rendering and Performance
WCAG 2.2	global	Web Content Accessibility Guidelines	The W3C Recommendation of 12 December 2024 with levels A, AA and AAA; the practical test for storefront accessibility.	Delivering the Storefront: Themes, Headless, Rendering and Performance

Term	Scope	Short name	Meaning	Chapter
ESPR	EU	Ecodesign for Sustainable Products Regulation	Regulation (EU) 2024/1781 bans large enterprises from destroying unsold apparel and footwear from 19 July 2026, including goods returned under the right of withdrawal.	Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation
Data Act	EU	Data and cloud switching rules	Regulation (EU) 2023/2854, applying from 12 September 2025, with switching rights for cloud and SaaS contracts.	Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation

Term	Scope	Short name	Meaning	Chapter
OSS (One Stop Shop)	EU	Cross-border VAT return	Since 1 July 2021, a merchant whose EU cross-border consumer sales exceed €10,000 a year charges the destination country's VAT and reports it in one quarterly return at home.	Tax, Invoices and the Books: From Order to Ledger, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
IOSS (Import One-Stop Shop)	EU	Import VAT at checkout	Lets a seller charge destination VAT at checkout on imported consign-ments up to €150 and remit it monthly, so the parcel is not stopped for VAT at the border.	Getting It There: Carriers, Labels, Tracking and the Delivery Promise

Term	Scope	Short name	Meaning	Chapter
Reverse charge / VIES	EU	B2B cross-border VAT	On an intra-EU business sale to a valid VAT number (checked in the EU's VIES service) the seller charges 0% and the buyer accounts for the VAT itself.	Tax, Invoices and the Books: From Order to Ledger, From Cart to Confirmation: Checkout Orchestration
ViDA	EU	VAT in the Digital Age	Council Directive (EU) 2025/516: e-invoicing-based digital reporting for cross-border B2B supplies from 1 July 2030.	Documents Between Companies: EDI, Punchout and E-Invoicing, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation

Term	Scope	Short name	Meaning	Chapter
EN 16931 / UBL / Peppol BIS Billing 3.0	EU	E-invoice model, syntax and profile	EN 16931 is the European semantic model of a core invoice; UBL is the XML syntax most e-invoices use; Peppol BIS Billing 3.0 is the usage specification for the Peppol network. XRechnung and ZUGFeRD are German profiles of the same model.	Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice
Belgian B2B e-invoicing mandate	BE	Structured e-invoices	From 1 January 2026, domestic invoices between VAT-registered businesses are structured e-invoices, Peppol BIS at minimum, with fines for lacking the means.	Documents Between Companies: EDI, Punchout and E-Invoicing, Tax, Invoices and the Books: From Order to Ledger

Term	Scope	Short name	Meaning	Chapter
German e-invoicing	DE	E-Rechnung	All domestic businesses must receive e-invoices since 1 January 2025 and issue them from 1 January 2027 (2028 for turnover up to €800,000); EDI procedures are tolerated to the end of 2027.	Documents Between Companies: EDI, Punchout and E-Invoicing
Dutch e-invoicing plan	NL	Planned mandate	Cabinet plan of 11 September 2026: domestic B2B e-invoicing from 1 July 2030 and reporting from 1 July 2031; today only invoices to central government must be electronic.	Documents Between Companies: EDI, Punchout and E-Invoicing, Rules Turned Into Require-ments: Consumer, Payment, Invoicing and Platform Regulation

Term	Scope	Short name	Meaning	Chapter
KassenSichV / TSE	DE	Cash-register security device	Every German electronic till needs a certified technical security device that signs each transaction, sequential numbering, a receipt and registration with the tax office.	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
White cash register (GKS)	BE	Geregistreerd kassasysteem	Belgium's certified-register rule applies only to restaurant and catering services, not to shops.	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
Record retention (7 years / §147 AO)	NL / DE	Bookkeeping retention	Dutch businesses keep business records for at least 7 years; German §147 Abgabenordnung requires 10 years for books and 8 for vouchers. Retention overrides erasure for invoices.	Whose Data Is It? Privacy, Consent and Retention

Term	Scope	Short name	Meaning	Chapter
UN/EDIFACT / EANCOM	global	EDI standard and retail subset	The United Nations EDI standard (ISO 9735) and GS1's EANCOM subset used in European retail, with messages such as ORDERS, DESADV and INVOIC.	Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice
ANSI X12	US	North American EDI standard	The North American counterpart to EDIFACT, with numbered transaction sets (850 order, 856 ship notice, 810 invoice).	Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice
GS1 keys (GTIN, GLN, SSCC)	global	Identification standards	GS1 issues the numbers behind barcodes: GTIN for the item, GLN for the party or place, SSCC for the logistics unit; the General Specifications define them.	What Are We Selling? Products, Variants and the Catalog, Documents Between Companies: EDI, Punchout and E-Invoicing, Pick, Pack, Ship: Warehouse Management and Fulfillment

Term	Scope	Short name	Meaning	Chapter
AS2 / cXML / OCI	global	B2B document protocols	AS2 (RFC 4130) carries signed, encrypted documents over HTTP; cXML and OCI carry punchout sessions and orders between procurement systems and suppliers.	Documents Between Companies: EDI, Punchout and E-Invoicing, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals
South Dakota v. Wayfair	US	Remote sales tax	The 2018 Supreme Court decision letting each state require remote sellers to collect sales tax above a threshold; marketplace-facilitator laws make the marketplace collect.	Tax, Invoices and the Books: From Order to Ledger, Rules Turned Into Require-ments: Consumer, Payment, Invoicing and Platform Regulation
INFORM Consumers Act	US	Marketplace seller verification	Effective 27 June 2023: marketplaces collect and verify bank, tax and contact data from high-volume sellers.	Rules Turned Into Require-ments: Consumer, Payment, Invoicing and Platform Regulation

Term	Scope	Short name	Meaning	Chapter
FTC rule on fake reviews	US	Consumer reviews rule	From 14 August 2024, bans fake or AI-generated reviews, sentiment-conditioned incentives, undisclosed insider reviews and review suppression.	Finding the Product: Search, Navigation and Merchandising, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation
US state privacy laws	US	Opt-out privacy model	At least 19 state laws as of 2025, built on opt-out rather than the EU's opt-in.	Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation
MCP / ACP / UCP / AP2	global	Agentic commerce protocols	The 2025–2026 protocols that let agents call tools, check out and pay; versions and backers change quickly, so date every claim.	Assistants and Agents: AI in Commerce and What Is Real in 2026

Kestrel & Pine Names (Fictional)

Kestrel & Pine is fictional: an outdoor apparel and equipment brand from Arnhem, invented for teaching and introduced in Meet Kestrel & Pine. Its systems are named by role, never by vendor; real marketplaces and carriers appear only as channels. Every name, figure and date below is an illustration that agrees with the book's scenario ledger.

Term	Definition	Chapter
Kestrel & Pine B.V.	The fictional brand the book follows: founded 2016 in Arnhem, €48.0M revenue in 2026 (online €27.0M, wholesale €12.0M, stores €6.0M, marketplaces €3.0M), 310 employees, 14 in technology; in the Omnichannel stage since 2023.	Meet Kestrel & Pine
Waypoint	Kestrel & Pine's 2027–2028 program, approved in March 2027 on Noor Visser's proposal, with €1.1M on top of the running technology spend; its eight decisions W1–W8 are canonical.	Meet Kestrel & Pine, Part X — EVOLUTION: Decisions That Change With Scale
W1 — Stock ownership	The WMS is the system of record for physical stock at Duiven, the POS for store stock, the marketplace program's stock a tracked external location; a bought availability service computes ATP per channel and the hourly sync chain is retired by Q4 2027.	Is It in Stock? Inventory, Availability and the Promise You Can Keep, Meet Kestrel & Pine

Term	Definition	Chapter
W2 — One order record	A bought OMS becomes the order hub by Q4 2027, with one state model and one order number family for every channel; the platform keeps checkout and the storefront order history.	The Order Record: Lifecycle, State and the Order Management System, Meet Kestrel & Pine
W3 — Product information	A PIM becomes the system of record for product content and attributes in Q3 2027; the ERP keeps costs and purchasing; the platform and the connector become consumers.	What Are We Selling? Products, Variants and the Catalog, Meet Kestrel & Pine
W4 — Stay and surround	Kestrel & Pine stays on its SaaS platform and adds OMS, PIM and availability beside it; the platform's own POS is evaluated in 2028 and the decision revisited in 2029.	Build, Buy or Assemble: Choosing and Changing Platforms, Meet Kestrel & Pine
W5 — Integration standards	Every integration has a named owner, a contract, idempotent handling of repeated messages and a reconciliation job; stock, orders and prices move by events, products by scheduled publication, finance by daily batch.	APIs, Events, Files and Middleware: How Commerce Systems Talk, Meet Kestrel & Pine
W6 — EDI compliance	SSCC labels generated by the WMS at pack time; DESADV sent from the WMS confirmation, not the ERP; one partner card per trading partner.	Documents Between Companies: EDI, Punchout and E-Invoicing, Meet Kestrel & Pine

Term	Definition	Chapter
W7 — One definition of an order	The data warehouse adopts one metric definition for order, revenue and return, owned by the Data Lead.	Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics, Meet Kestrel & Pine
W8 — Agent readiness, not agents	Accurate product data, availability and the delivery promise by API, and order status by API are prerequisites; no shopper-facing AI agent is built in 2027.	Assistants and Agents: AI in Commerce and What Is Real in 2026, Meet Kestrel & Pine
The platform	Kestrel & Pine's hosted SaaS commerce platform (enterprise tier since 2023): three localized storefronts in one tenant, the B2B portal, checkout, 41 installed apps.	Meet Kestrel & Pine, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation
The ERP / the WMS / the POS	The cloud mid-market ERP (finance, purchasing, item master, invoicing; since 2023), the standalone cloud WMS at Duiven (since 2023) and the cloud POS in the six stores (since 2019).	Meet Kestrel & Pine
The OMS / the PIM / the availability service	The systems Waypoint adds in 2027: the bought order hub, the product system of record and the availability module of the OMS.	Meet Kestrel & Pine, The Order Record: Lifecycle, State and the Order Management System

Term	Definition	Chapter
The marketplace connector / the returns portal / the shipping service	The channel integration SaaS for bol, Amazon.de and Zalando (since 2021), the returns SaaS that issues labels and triggers refunds (since 2025), and the multi-carrier label and tracking service in the WMS flow.	Meet Kestrel & Pine
The PSP / the terminal provider / the EDI provider / the iPaaS / the data warehouse	The online payment provider (iDEAL, cards, PayPal, Klarna, Bancontact, Apple Pay), the store card-terminal provider, the VAN for Bergzicht's EDI and Peppol, the integration platform carrying 34 integrations, and the data warehouse with its BI tool.	Meet Kestrel & Pine
The agency / the ERP partner	The unnamed outside storefront and apps agency (retained since 2017, €14,000 a month) and the ERP implementation partner that built the platform–ERP connector.	Meet Kestrel & Pine, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance
Duiven warehouse	Kestrel & Pine's 8,500 m² warehouse near Arnhem, opened 2023, run on the WMS; ships about 336,000 parcels a year with a 22:00 next-day cut-off in the Netherlands.	Pick, Pack, Ship: Warehouse Management and Fulfillment, Meet Kestrel & Pine

Term	Definition	Chapter
The six stores	Arnhem (flagship), Utrecht (click-and-collect pilot), Amsterdam, Nijmegen, Eindhoven and Antwerp, each holding about 1,800 variants.	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Meet Kestrel & Pine
Ridgewalker / Trailhead / merino hiking socks	The recurring products: the Ridgewalker shell jacket (€199 consumer, €95 wholesale, landed cost €58), the Trailhead fleece (€119) and merino hiking socks (€18).	Meet Kestrel & Pine
Mara Lindqvist / Teun Baas	The co-founders: Mara is CEO and owns the Waypoint budget (she/her); Teun is Chief Product and Brand and owns the catalog and the seasons (he/him).	Meet Kestrel & Pine
Noor Visser	Head of Engineering since January 2027 (she/her), the reader's stand-in, who proposed Waypoint.	Meet Kestrel & Pine
Daniel Okafor / Sanne de Wit	Head of Operations for the warehouse, stores and carriers (he/him), and Finance Director who owns the ERP, invoicing and month-end (she/her).	Meet Kestrel & Pine

Term	Definition	Chapter
Milan Petrov / Lea Moreau	Ecommerce and Marketplaces Manager who owns the storefronts, apps and connector (he/him), and Head of Wholesale who owns the B2B portal and the EDI relationship (she/her).	Meet Kestrel & Pine
Ruben Alves / Hanna Berg	Lead Integration Engineer since 2022 who owns the iPaaS and the integration inventory (he/him), and Data Lead who owns the data warehouse and metric definitions (she/her).	Meet Kestrel & Pine
Eva Janssen	A customer in Nijmegen (she/her) whose jacket order is the book's recurring order; she also exists as a store loyalty customer, newsletter subscriber and returns contact with no shared identifier.	Meet Kestrel & Pine, Journey 1: Eva's Jacket, From Search to Doorstep
Bergzicht Outdoor / Pieter Smit	A Dutch outdoor chain with 38 shops and a distribution center in Zwolle, the largest wholesale customer at €2.1M a year, ordering by EDI since 2025; Pieter Smit is its buyer (he/him).	Journey 3: A Chain Reorders by EDI, From Order to Invoice, Meet Kestrel & Pine

Term	Definition	Chapter
De Kaap Buitensport	An independent outdoor retailer in Groningen and Assen on net 30 terms with a €15,000 credit limit, whose February 2027 pre-season order shows the B2B flow.	Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals
Order #1072455 (Eva's jacket)	The recurring order: on Tuesday 13 April 2027 at 21:14 Eva orders a women's Ridgewalker, size M, moss (€199) and two pairs of merino socks (€18 each), €235 by iDEAL; delivered 15 April by PostNL, exchanged for size L from 19 April.	Journey 1: Eva's Jacket, From Search to Doorstep, Journey 2: The Jacket Comes Back
Bergzicht's spring reorder	EDI ORDERS received 6 April 2027 for 240 units across 18 SKUs, €19,860 excluding VAT, picked to three SSCC-labeled pallets, delivered to Zwolle and invoiced on 9 April, due 9 May.	Journey 3: A Chain Reorders by EDI, From Order to Invoice
The Black Friday Oversell	Friday 27 November 2026: the Ridgewalker sold out in the warehouse at 19:40 and kept selling for up to two hours through the hourly sync chain; 1,340 units oversold, 1,120 orders canceled or delayed, about €96,000.	Is It in Stock? Inventory, Availability and the Promise You Can Keep, Journey 4: Black Friday, Hour by Hour

Term	Definition	Chapter
The Queue That Waited	On the same Black Friday the iPaaS order flow to the WMS, throttled by the WMS vendor's rate limit, backed up for four hours, so orders reached the warehouse late.	Pick, Pack, Ship: Warehouse Management and Fulfillment, Journey 4: Black Friday, Hour by Hour
The Three-Status Order	Order #1052877 (December 2026) showed Delivered in the platform, In picking in the WMS and Not invoiced in the ERP for six days, causing 11 support tickets.	The Order Record: Lifecycle, State and the Order Management System, Where Is My Order? Notifications, Service and the Customer Record
The Double Refund	January 2027: 212 returns refunded twice, once by the returns portal through the platform and once by support in the PSP console, €19,400 in all.	Taking the Money: Payment Providers, Authorization, Capture and Refunds, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics
The Price Drift	October 2026: the German storefront sold the Ridgewalker at €179 for nine days while other channels charged €199, because a price-list sync failed silently; 410 orders, about €8,200 of margin.	What Does It Cost? Price Lists, Contract Prices, Currencies and Tax
The ASN Chargeback	Q3 2026: Bergzicht deducted €14,200 for despatch advices whose SSCC pallet labels did not match what arrived, because the ERP generated the DESADV before packing.	Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice

Term	Definition	Chapter
The Store Stock Mirage	December 2026: click-and-collect at Utrecht promised 37 items the store had already sold, because store stock reached the ERP only nightly.	Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment
The App That Wrote Stock	March 2026: a pre-order app adjusted inventory in the platform while the WMS feed also wrote it, creating two weeks of phantom reservations on 600 variants.	Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Is It in Stock? Inventory, Availability and the Promise You Can Keep
The Card-Testing Night	August 2026: 48,000 bot attempts with stolen cards in one night against the checkout, €2,900 in PSP fees and a 40-minute account pause.	Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers
The Deprecated Endpoint	February 2027: the marketplace connector retired an API version on a published date; an agency script still used it, and stock to bol was stale for two days.	Promises Between Systems: API Contracts, Webhooks and Versioning, Selling on Someone Else's Platform: Marketplaces and Channel Integration
The Black Friday Stacking Bug	Black Friday week 2026: an automatic 20% off outerwear combined with the 10% welcome code on 2,860 orders because nobody had decided how the two promotions combine.	Discounts, Coupons and Loyalty: Promotions Without Chaos

Term	Definition	Chapter
The Request From Ghent	May 2027: a Belgian customer, Lotte Claes, asks for her data and its deletion across the platform, bol, the returns portal, the newsletter tool and the POS.	Whose Data Is It? Privacy, Consent and Retention
The stock reconciliation	May 2027: the first weekly comparison of the WMS with the platform found 2.3% of active variants (about 390) with a different quantity, most from the three apps that wrote stock.	Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation
The finance-only rule	Waypoint's rule for the back office: the ERP is the system of record for finance only (ledger, invoices and credit notes, receivables and payables, purchasing, inventory valuation), not for stock or orders.	The Back Office: ERP Integration and Who Is the Hub
The stock sync chain	The 2026 chain of copies WMS → ERP (every 15 minutes) → platform (hourly) → marketplace connector (hourly), which let a sale take up to two hours to reach the other channels.	Is It in Stock? Inventory, Availability and the Promise You Can Keep, Meet Kestrel & Pine

Alphabetical Index

- **0–9** — 3-D Secure / liability shift · 3DS · 3PL (third-party logistics provider)
- **A** — Abandoned cart · Access point · Account takeover · ACP · ACP / UCP / AP2 · Acquirer · Across Scale Stages · Adobe Commerce · ADR (architecture decision record) · Advance shipping notice · Age monitor · The agency / the ERP partner

· Agent · Agent identity · Agent readiness · Agentic commerce · Agentic plan · AI agent · Alexa for Shopping · Alexa for Shopping / Buy for Me · Allocated · Analytical data · ANSI X12 · AOV (average order value) · AP2 · API (application programming interface) · API budget · API contract · API guidelines / tech radar with teeth · API mandate · API version · App · App / app store · App ecosystem · App store · The App That Wrote Stock · Application programming interface · Approval workflow · Architecture decision record · Architecture style · AS2 · AS2 / cXML / OCI · ASN · The ASN Chargeback · Assistant · Assistant / agent · At-least-once delivery · ATP (available-to-promise) · Attribute · Attribute / attribute set · Attribute set · Authorization · Authorization / capture · Automatic discount · Availability · Availability service · Available · Available-to-promise · Average order value

- **B** — B2B (business to business) · B2B2C · B2C (business to consumer) · B2C / D2C · Backpressure · Bancontact · Base price · Base price / contract price · Batch · Batch picking · Belgian B2B e-invoicing mandate · Bergzicht Outdoor · Bergzicht Outdoor / Pieter Smit · Bergzicht's spring reorder · BI tool · Bin · Black Friday · The Black Friday Oversell · Black Friday Oversell · The Black Friday Stacking Bug · The Boomerang · Boomerang · Boost · BOPIS · Bot management / scraping policy · Bounded context · Breakage · Buffer (channel) · Build / buy / assemble · Bundle · Bundle (kit) / configurable product · The Burned Bridge · Burned Bridge · Bury · Business fact · Business unit · Buy for Me · Buy online, pick up in store · Buyer roles
- **C** — Cache · Cache / waiting room · Canary release · Cancellation rate · Capability · Capability map · Capacity · Capacity planning · Capacity planning / load test · Capture · Card scheme · Card testing · The Card-Testing Night · Cardholder-data environment · Carrier · Carrier / carrier API · Carrier API · Carrier integration · Cart · Cart / abandoned cart · Cart abandonment · Cartonization · Cartridge · CDC (change data capture) · CDN · CDP (customer data platform) · Change data capture · Change freeze · Channel · Channel allocation · Channel connector (channel integration) · Channel integration platform · Channel price rule · Chargeback · Checkout · Checkout / checkout orchestration · Checkout extensibility · Checkout extensibility / Shopify Functions · Checkout orchestration · Choreography · Choreography / orchestration · Click-and-collect (BOPIS) · CLS · CMP · Commerce platform · Commerce protocol · commercetools · commercetools / Shopware / Adobe Commerce / Mage-OS · Committed · Company account · Compensation · Compliance chargeback · Composable · Composable assembly · Composable commerce · Configurable product · Conflict resolution · Connected Retail · Consent · Consent / CMP / soft opt-in · Consent management platform · Consumer Rights Directive · Content delivery network · Content Security Policy · Contract (API) · Contract / tiered pricing · Contract price · Contract pricing · Contract test · CONTRL · CONTRL / RECADV · Controller · Controller / processor · Conversational search · Conversion rate · Conversion rate / cart abandonment · Copy (of a fact) · Copy / writer · Core Web Vitals (LCP, INP, CLS) · Costing method

· Coupon abuse / refund abuse / scalping · Coupon code · Coupon code / automatic discount · Credential stuffing · Credential stuffing / account takeover · Credit check · Credit limit · Credit limit / approval workflow · Credit note · Cross-reference / merge log · CSP / SRI · CSR · Custom build · Customer data platform · Customer group · Customer identity · Customer master · Cut-off · Cut-off / capacity · Cut-off time · Cutover · Cutover / redirect map · cXML · Cycle counting

- **D** — D2C (direct to consumer) · Daniel Okafor · Daniel Okafor / Sanne de Wit · Dark launch / canary release · Dark launching · Data Act · Data processing agreement · Data subject request · Data warehouse · De Kaap Buitensport · Dead-letter queue (DLQ) · Degradation ladder · Delivery option · Delivery option / pickup point / locker · Delivery promise · Delivery promise / EDD (estimated delivery date) · Delivery-promise service · Deployable · Deployment unit · The Deprecated Endpoint · Deprecation · Deprecation / sunset · DESADV · DESADV / ASN (advance shipping notice) · Despatch advice · Digital Services Act · Direct API call · Direct to consumer · Discount allocation · Disposition · Dispute · Divert the flow · DLQ · Documented fact · The Double Refund · DPA (data processing agreement) · DSR (data subject request) · Duiven · Duiven warehouse · Dutch e-invoicing plan · Dynamo · Dynamo / conflict resolution
- **E** — E-invoice · EAN · EAN-13 · Edge / CDN (content delivery network) · Edge delivery · Edge runtime · EDI (electronic data interchange) · EDI partner card · EDI-type sales · Electronic data interchange · Electronic invoice · Eligibility · Eligibility / stacking · EN 16931 / UBL / Peppol BIS Billing 3.0 · Endless aisle · Enrichment · Enrichment / syndication · Enterprise resource planning · Enterprise suite · ePrivacy Directive · ERP (enterprise resource planning) · The ERP / the WMS / the POS · ERP as hub · ERP implementation partner · ERP partner · ERP-centered commerce · ESPR · Established / Depends on context / The author's synthesis · Estimated delivery date · European Accessibility Act · Eva Janssen · Eva's jacket · Event · Event bus · Event interception · Event notification / event-carried state transfer · Event stream · Eventual consistency · Exchange · Exit cost · Express wallet · Extension point (function, call-out) · Externalizable interface
- **F** — Facet · Failure pattern · The Familiar Stranger · Familiar Stranger · FBA (Fulfillment by Amazon) · FBM · Feed · Feed / structured data · File transfer · The finance-only rule · The First Snowfall · First Snowfall · Fiscal receipt · Fiscalization · Flash sale · The Forked Platform · Forked Platform · Four-corner model · FTC rule on fake reviews · Fulfillment by Amazon · Fulfillment order
- **G** — GDPR · General ledger · German e-invoicing · The Giant's Boots · Giant's Boots · Gift card · Gift card / store credit / loyalty points · GLN (Global Location Number) · Global Location Number · Global Trade Item Number · GMV (gross merchandise value) · Golden record · GPSR · Graceful degradation · Graceful degradation / degradation ladder · Gross margin · Gross merchandise value · Gross merchandise volume · Group (stage) · Growing · GS1 keys (GTIN, GLN, SSCC) · GTIN / EAN-13 · Guest checkout
- **H** — Handoff · Hanna Berg · Happy path · Happy path / variant path · Headless

· Headless / storefront API · Headless storefront · Held back · Held back / safety stock / incoming · Hosted fields · Hosted fields / hosted payment page · Hosted payment page · Hosted platform plus apps · Hosted SaaS platform · Hourly sync chain · Hydrogen · Hydrogen / Oxygen

- **I** — iDEAL · iDEAL / Bancontact / purchase on account · Idempotency · Idempotency key · Idempotent · Identifier · Identity index · Identity index / request orchestrator · Illustration · In-process plugin (module, extension) · Incident commander · Incident commander / runbook / change freeze · Incoming stock · Indexing · Industry practice · INFERENCE, NOT DOCUMENTED · INFORM Consumers Act · INP · Inspection · Integration contract · Integration hub · Integration inventory · Integration platform as a service · Interchange · Interchange Fee Regulation · Interchange-plus · Inventory hoarding · Inventory valuation · Inventory valuation / costing method / landed cost · INVOIC · Invoice · IOSS (Import One-Stop Shop) · iPaaS (integration platform as a service) · Isolation · Isolation / silo / pool · Issuer · Issuer / acquirer / scheme · Item master
- **J** — Journey (in this book)
- **K** — KassenSichV / TSE · Kauf auf Rechnung · Kestrel & Pine · Kestrel & Pine B.V. · Kill switch · Kit
- **L** — Label / manifest · Labeled assumption · Landed cost · Large language model · Largest contentful paint · Late shipment rate · Lawful basis · Layer · LCP · Lea Moreau · Lead time / transit time · Ledger (general ledger) · Legacy mimic · Liability shift · List price · Listing · Listing / offer · LLM · Load test · Location / stock location · The Long Fuse · Long Fuse · Lotte Claes · Loyalty points · LvB · The Lynchpin · Lynchpin
- **M** — MACH · MACH Alliance · Mage-OS · Magecart · Manifest · Mapping (EDI) · Mara Lindqvist · Mara Lindqvist / Teun Baas · Marketplace · Marketplace connector · The marketplace connector / the returns portal / the shipping service · Marketplace fulfillment program · Marketplace selling / marketplace operating · Marketplace-fulfilled · Master (system of record) · Master data / MDM · Master data management · Master data ownership · Matching · Matching / survivorship · MCP (Model Context Protocol) · MCP / ACP / UCP / AP2 · MDM · Merchandising rule (boost, bury, pin) · Merchant solutions · Merchant-fulfilled · Merge · Message / standard / transport / mapping · Message implementation guideline · The Message in a Bottle · Message in a Bottle · Message queue · Metric definition · Metric definition / metrics layer · Metrics layer · MFA (multi-factor authentication) · Microservices · Milan Petrov · Milan Petrov / Lea Moreau · Model / LLM · Model Context Protocol · Modular monolith · Module (platform extension) · Monolith · Monolith / modular monolith · Monthly recurring revenue · MRR · MRR / subscription and merchant solutions · Multi-carrier platform · Multi-tenancy · Multi-tenant
- **N** — Nakadi · Nakadi / event bus · Navigation · Net 30 · Net sales · Net terms · Noisy neighbor · Noor Visser · Notification service

- **O** — OCI · Offer · Offline operation · Omnibus Directive · Omnichannel · Omnichannel (stage) · OMS (order management system) · The OMS / the PIM / the availability service · OMS as hub · On hand · On hand / committed / reserved · One Size Fits None · Open source · Open source platform · Operational / analytical data · Operational data · Option / option value · Order · Order #1052877 · Order #1072455 (Eva's jacket) · Order confirmation · Order Defect Rate · Order header / order lines · Order hub · Order management system · Order number family · Order orchestration · Order routing · Order state model · Order status · Order status page · Order-to-ledger · ORDERS · ORDERS / ORDRSP / INVOIC · Orders per minute · ORDRSP · OSS (One Stop Shop) · Outbox · Oversell · Ownership map · Oxygen
- **P** — Packing / cartonization · Packwerk · Parallel run · Parcel locker · Partner card · Payment service provider · Payment terms (net 30) · Payment webhook · Payout · PCI DSS · PCI scope / SAQ · Peak · Peak plan · Peppol · Peppol / access point / four-corner model · Personal data · The Phantom Stock · Phantom Stock · Pick wave / batch picking · Pickup point · Pieter Smit · PIM (product information management) · Pin · Pin (merchandising) · The platform · Plugin · Pod · Point of sale · Point-to-point integration · Polling · Polling / sweep · Pool model · POS (point of sale) · PRAPI / synchronous read API · Pre-season order / in-season reorder · Pre-season terms · The Price Drift · Price history · Price list · Price list / price dimensions · The Price Nobody Can Explain · Price Nobody Can Explain · Price parity · Pricing engine · Pricing engine / price history · Processor · Product · Product / variant · Product feed · Product information management · Product master · Product Read API · Progressive web app · Promise · Promotion · PSD2 / SCA · PSD3 / Payment Services Regulation · Pseudonymization · PSP (payment service provider) · The PSP / the terminal provider / the EDI provider / the iPaaS / the data warehouse · The Pulled Rug · Pulled Rug · Punchout (cXML, OCI) · Purchase on account · Put-away · PWA (progressive web app)
- **Q** — Queue · Queue / event stream · The Queue That Waited · Quote
- **R** — Radical Agility · Ranking · Rate limit · Rate shopping · Rate shopping / multi-carrier platform · RECADV · Receiving · Receiving / put-away / slotting · Recommendation (evidence label) · Recommendations · Reconciliation · Reconciliation job · Record retention (7 years / §147 AO) · Recurring order · Redirect map · Reference-architecture inference · Refund · Refund on inspection · Refund timing · Regionalization · Regulatory calendar · Relevance · Relevance / ranking · Rendering model · Replatforming · Repricing · The Request From Ghent · Request orchestrator · Requirement card · Requirement card / regulatory calendar · Reservation · Reserve-on-cart · Reserve-on-cart / reserve-on-order / allocation at fulfillment · Reserve-on-order · Reserved · Retention · Retention / erase, pseudonymize, retain · Return merchandise authorization · Return rate · Return request · Return-in-store · Returns portal · Reverse charge / VIES · Reverse fulfillment / inspection · Reverse logistics · Reviews and ratings · RFID · RFQ · Ridgewalker · Ridgewalker / Trailhead / merino hiking socks · Right to erasure · RMA (return merchandise authorization) · Routing · Ruben Alves · Ruben Alves /

Hanna Berg · Rufus · Runbook

- **S** — SaaS (software as a service) · Safety stock · Saga · Saga / compensation · Sales channel · Sandbox · Sanne de Wit · SAQ · SAQ A · Scale stage · Scalping · SCCs / EU-US Data Privacy Framework · Scheduled batch / file transfer · Schema · Schema registry · Schema registry / contract test · Schrödinger's Order · Scopes · Scopes / least scope · Scraping · Seam · Search / navigation / recommendations · Search index · Search index / indexing · The Secret Agent · Secret Agent · Self-service · Sell-through · Sell-through / weeks of cover · Seller of record · Seller performance metrics · Seller-fulfilled · Seller-fulfilled / marketplace-fulfilled · Semantic layer · Serial Shipping Container Code · Server-side / client-side rendering · Serverless function · Service view · Service view / self-service · Settlement · Settlement / payout · SFTP · Shadow / parallel run · Shadow mode · Shard · Sharding / shard key · Sharding key · Shared pool · Ship-from-store · Ship-from-store / return-in-store / endless aisle · Shipping confirmation · Shipping label · Shopify Functions · Shopware · Shopware Nexus · Sidekick · Sidekick / Agentic plan · Silent fall-forward · Silo model · The six stores · Skimming · SKU (stock-keeping unit) · Slotting · Soft opt-in · Software as a service · SoR · Sourcing · Sourcing / routing · South Dakota v. Wayfair · The Spaghetti Junction · Spaghetti Junction · Specialist group · Specialist layer · Specialist layer / specialist group · Split shipment · SSCC (Serial Shipping Container Code) · SSR · Stacking · Staging store · Starting · State machine · Stock event · Stock location · The stock reconciliation · The stock sync chain · Stock-keeping unit · Store credit · The Store Stock Mirage · Stored value · Storefront · Storefront API · Strangler fig (strangler migration) · Strangler migration · Strangler pattern · Strike · Structured data · Subresource Integrity · Subscription solutions · Suite (enterprise suite) · Sunset date · Supplier chargeback · Supply-chain risk · Survivorship · Sweep · Synchronous call · Synchronous call-out · Synchronous read API · Syndication · Synonyms · Synonyms / typo tolerance / zero-result rate · System / vendor product · System of record
- **T** — Tax calculation / tax engine · Tax engine · Tax-exclusive · Tax-inclusive · Tax-inclusive / tax-exclusive · Taxonomy · TCO (total cost of ownership) · Technology radar · Tenant · Tenant / multi-tenancy · Terminal provider · Teun Baas · Theme · Theme / theme code · Third-party logistics · Three tiers of take-away · The Three-Status Order · Throttling · Tiered pricing · Till · Total cost of ownership · Tracking · Tracking event · Trailhead fleece · Transactional notification · Transactional outbox · Transitional architecture · Two-phase commit · Two-pizza team · Typo tolerance
- **U** — UCP · UN/EDIFACT / EANCOM · US state privacy laws · Utrecht store
- **V** — Validity window · Value-added network · Value-added tax · VAN · VAN (value-added network) / AS2 / SFTP · Variance report · Variant · VAT (value-added tax) · Vector search · Vector search / conversational search · Vendor lock-in · Vendor lock-in / exit cost · Verified review · Version pin · Versioning / API version · ViDA · Volume price
- **W** — W1 — Stock ownership · W2 — One order record · W3 — Product information · W4 — Stay and surround · W5 — Integration standards · W6 — EDI compliance · W7

— One definition of an order · W8 — Agent readiness, not agents · Waiting room · Warehouse management system · Wave · Wave release · Waypoint · Waypoint program · WCAG 2.2 · Web Bot Auth · Web skimming (Magecart) · Webhook · Weeks of cover · What to Take Away · Where is my order · White cash register (GKS) · Wholesale · Wholesale / B2B e-commerce · WISMO ("where is my order") · WMS (warehouse management system) · Writer

- **Z** — Zalando API guidelines · ZEOS · ZEOS / Connected Retail · Zero-result rate · ZFS

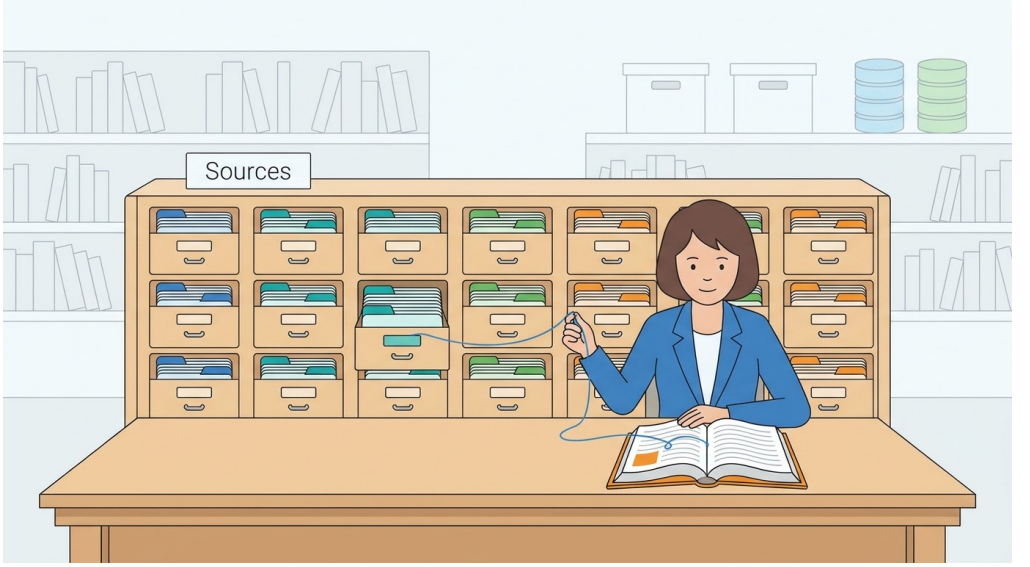
Where to Go Next

The templates that turn many of these terms into working records are collected in Toolkit: Cards and Templates for Commerce Architecture Decisions; the mapping from the research domain models to the chapters is in Domain Coverage: From Three Model Families to Forty-Nine Chapters; and the sources behind every definition, with their publication dates, are listed in Bibliography: Every Source the Book Cites, the last page of the book.

Sources and Evidence

Every definition is taken from the chapter linked in its row, which cites its sources; they are collected in Bibliography: Every Source the Book Cites. Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.

Bibliography: Every Source the Book Cites



IN THIS SECTION, YOU WILL: Find the public source behind any claim in the book, with its date, its evidence type and every chapter that relies on it, by topic or by chapter.

KEY POINTS:

- **Sources are listed because they were cited.** Every entry was harvested from a page's **Sources and Evidence** section, and its **Cited in** line names those pages. Nothing is added for completeness, and a listing does not mean every page of a long document was read.
- **Dates and evidence types tell you how far to trust a claim.** A law records a requirement, a filing records reported numbers, API documentation records an interface as it was read, and an engineering post records what a company said it built in that year. Weigh each claim by the kind and age of the source behind it.
- **Keep documented facts, inferences and Kestrel & Pine distinct.** The sources support the documented facts. Reference architectures

> for real platforms are labeled inferences, the recommendations are my judgement, and Kestrel & Pine, the omnichannel brand the book follows, is fictional: no source here describes it.

This page lists the external sources the chapters cite. The research was conducted in **November 2026**, and most entries were consulted then; where a chapter recorded an access date, a version or a "last updated" date, the entry keeps it. Many sources are in **Dutch**, **German**, **French or Polish** (tax-authority guidance, statutes, retailer supplier manuals and trade press); the citing chapters translated what they use. Much vendor documentation is living and undated, so those entries carry "**n.d.**" **with the month it was read**. The starting map for the research was a set of **Productscape domain models of commerce products, inferred from public product behavior**; they are listed once as the author's working model and are not evidence about any company's systems.

How to Read This Register

R marks a consulted source; the numbers identify, they do not rank. The register holds **816 sources** harvested from the **Sources and Evidence** sections of 53 pages, deduplicated by normalized URL (scheme, `www.`, trailing slash, fragment and `.html` suffix ignored); where chapters cited the same legal text, filing or post under two URLs, the entry keeps both. Identifiers were assigned in topic order for this first edition and stay stable: **new sources append from the next free R-number and are never renumbered**, and a removed source's number is retired rather than reused, so a reference to an identifier keeps meaning the same document.

Each entry gives the identifier, the linked title, the author or publisher, the **publication year as the citing chapter gave it** ("n.d." where the source carries no date, with access or update dates kept where the chapter recorded them), one clause on what the source supports, the evidence type and the chapters that cite it. Company engineering evidence carries its year because old posts may describe systems that have since been replaced; a post more than about three years old is a record of that date, not of today. Where chapters cited one source with slightly different metadata, the entry keeps the wording most chapters used, and where they gave different years it keeps the most specific citation.

Evidence type	Sources	What it is	How to read it
Company engineering	55	Engineering blog posts, conference talks and builder write-ups by Shopify, Zalando, Amazon, Uber, Netflix and others	Documents what the company says it built at that date; it may describe systems since replaced, and it is silent on what was not published
Company filing, report or announcement	87	Annual reports and SEC filings, results statements, shareholder letters, press releases and corporate announcements	Authoritative on the company's reported numbers and events at their date; written for investors, customers and the press, not as design documentation
Product and API documentation	409	Product pages, help centers, partner portals and API documentation of commerce platforms, payment providers, marketplaces, carriers, ERP and integration vendors	Describes the product or interface as offered when it was read; living pages change without notice, and vendor claims are claims
Standard or specification	33	RFCs, W3C, GS1, EDIFACT and X12, Peppol, PCI DSS and EMV, OpenAPI, AsyncAPI and CloudEvents, IFRS, OWASP and the agentic commerce protocols	Authoritative on the specification; an architecture applies it, and versions matter

Evidence type	Sources	What it is	How to read it
Regulation and official source	73	Legal texts, regulator and tax-authority guidance and official statistics in the EU, its member states, the UK and the US	Authoritative on the official position at its date; interpretation for a specific business belongs to its lawyers and tax advisers
Court or competition decision	5	Court judgments and competition- and consumer-authority decisions, including the Amazon marketplace cases	Authoritative on what was decided in that case at that date; facts found in one market do not transfer automatically
Research	16	Peer-reviewed papers, preprints and research reports	Findings hold within the study's method, data and date
Book	5	Books, book chapters and book-length pattern catalogs	Durable concepts and vocabulary; editions matter
Practitioner and press	125	Articles, essays and guides by practitioners, consultancies, law firms and trade bodies, market statistics, and news and retail trade press	Describes common practice, interprets primary sources or records events and claims at a date; secondary, so follow it to the primary source where one exists
Encyclopedia	7	Wikipedia entries, used for definitions, dates and widely reported incidents	Tertiary; the chapters use it only where no primary source was reachable, and the primary source wins where one is cited

Evidence type	Sources	What it is	How to read it
Author's working model	1	The author's Productscape domain models of eleven commerce products, inferred from public product behavior and mapped in Domain Coverage: From Three Model Families to Forty-Nine Chapters	Not public evidence and not a description of any company's systems; they were used to check the book's coverage, never as a source for a fact

The chapters also rely on the book's own research briefs and plan, written in November 2026 to collect and check the company, platform, regulatory and practice facts the chapters use. Those are working notes, not public evidence, and are not listed; wherever a brief's fact matters, the chapter cites the public source behind it, and that source is listed below. Links to the companion books (RECONCILED, LISTED and others) are cross-references to published pages, not external evidence, and are not listed either.

Topic Index

Identifiers run in topic order, so each topic is one contiguous block. The first ten topics follow the book's parts: a source sits under the part whose chapters cite it most, and its **Cited in** line lists every page that uses it. Legal texts, regulator guidance and decisions cited by Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers, Whose Data Is It? Privacy, Consent and Retention or Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation sit together under Part VIII, so the law is found in one place, unless one other part cites them in more pages (VAT and e-invoicing law stays with the tax and EDI chapters that lean on it). The eleventh topic collects the filings, results and announcements of Shopify, Amazon, Zalando, commercetools and Shopware, which chapters across the book cite, together with the sources only the case studies in Part XI use. Engineering posts and product documentation by a case company sit under the part they inform.

Commerce and business models (R1–R94, 94 sources; Part I — COMMERCE: What a Commerce Platform Is). Business models and their numbers, the journey of an order, the capability map, boundaries and systems of record, and the platform landscape a newcomer meets in 2026: market reports, retailer filings, vendor pages and industry statistics. The most widely cited is R4, *MACH Alliance Research: 6x More Organizations Achieve AI ROI with a Composable Foundation*, cited in 7 pages.

Catalog, pricing and search (R95–R185, 89 sources; Part II — CATALOG: Products, Prices and Promises). Product information and variants, price lists and price-indication law, promotions, gift cards and loyalty, search and merchandising, and inventory and availability. The most widely cited is R97, *Directive (EU) 2019/2161 (Omnibus)*, cited in 6 pages.

Checkout, payments, tax and orders (R186–R257, 72 sources; Part III — ORDER: Cart, Checkout, Payment and the Order Record). Carts and checkout, payment providers and payment methods, authorization, capture, refunds and disputes, the order record and its lifecycle, VAT, invoicing and revenue recognition. The most widely cited is R190, *Mollie: Webhooks*, cited in 6 pages.

Fulfillment, shipping, stores, returns and service (R258–R338, 81 sources; Part IV — FULFILLMENT: From Warehouse to Doorstep and Back). Warehouses and fulfillment, carriers and the delivery promise, stores, tills and fiscal rules, returns and refunds, and customer service and notifications. The most widely cited is R263, *Directive 2011/83/EU on consumer rights*, cited in 8 pages.

Marketplaces, B2B, EDI and ERP (R339–R636, 70 sources; Part V — CHANNELS: Selling Everywhere). Marketplaces and channel connectors, B2B commerce with contract prices, quotes and punchout, EDI and e-invoicing with trading partners, and ERP and back-office integration. The most widely cited is R341, *Synchronize customers and companies*, cited in 5 pages.

Integration, contracts, identity and data (R408–R484, 77 sources; Part VI — INTEGRATION: Making Separate Systems Behave Like One). Integration styles and iPaaS, events, consistency and reconciliation, API contracts and versioning, identity and master data, and the commerce data platform. The most widely cited is R409, *Shopify: Webhooks*, cited in 12 pages.

Platform engineering, tenancy, extensibility, storefront and peak (R485–R575, 91 sources; Part VII — PLATFORM: How Commerce Platforms Are Built). Architecture styles from monolith to microservices, multi-tenancy and rate limits, extensibility and app ecosystems, storefront delivery and web performance, and peak load and reliability. The most widely cited is R547, *Feature Toggles (aka Feature Flags)*, cited in 5 pages.

Security, privacy and regulation (R576–R635, 60 sources; Part VIII — TRUST: Security, Privacy and Regulation). Card-data security, bots, account takeover and skimmers, privacy, consent and data transfers, and the consumer, payment, accessibility and platform law (with its guidance and decisions) that turns into requirements. The most widely cited is R579, *FAQ E-Rechnung*, cited in 5 pages.

Journeys (R637–R642, 6 sources; Part IX — JOURNEYS: One Order, End to End). The four end-to-end journeys mostly reuse sources listed under the parts above; this topic holds only the sources a journey alone cites. The most widely cited is R637, *Stripe: Idempotent requests*, cited in 7 pages.

Evolution, organization and AI (R643–R724, 82 sources; Part X — EVOLUTION:

Decisions That Change With Scale). Build, buy or assemble, replatforming and migration, scaling stages, team design and engineering organization, and AI assistants, agents and agentic payment protocols as of 2026. The most widely cited is R652, *Regulation (EU) 2023/2854 (Data Act)*, cited in 5 pages.

Company records and case evidence (R725–R818, 94 sources; Part XI — PRACTICE: How Others Built It). Filings, results, annual reports, shareholder letters and corporate announcements of Shopify, Amazon, Zalando, commercetools and Shopware, and the engineering, press and product sources the four case studies use alone. The most widely cited is R730, *Shopify Inc. Form 10-K for fiscal 2025*, cited in 9 pages.

Commerce and business models

- **R1** · Order status — Shopify Help Center, n.d., accessed 2026-11. Separate status families; no tracking number, no updates. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Follow the Order: What Happens Between "Buy" and "Delivered", Is It in Stock? Inventory, Availability and the Promise You Can Keep, The Order Record: Lifecycle, State and the Order Management System, Where Is My Order? Notifications, Service and the Customer Record, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R2** · Customer notifications — Shopify Help Center, n.d., accessed 2026-11. Separate status families; no tracking number, no updates. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Follow the Order: What Happens Between "Buy" and "Delivered", Where Is My Order? Notifications, Service and the Customer Record, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R3** · Shopify Scripts deprecation — Shopify developer changelog, 2026-04-09. Script editing ends 2026-04-15, execution 2026-06-30. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong.
- **R4** · MACH Alliance Research: 6x More Organizations Achieve AI ROI with a Composable Foundation — MACH Alliance, 2026. 37% cite integration complexity as a primary concern (600 respondents). *Practitioner and press.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Build, Buy or Assemble: Choosing and Changing Platforms, What Changes at Each Scale: From First Store to Group, Case: commercetools, Shopware and the Composable Movement.
- **R5** · How to automate WISMO requests — Gorgias, updated 2023-11-13. WISMO as 18% of requests. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Follow the Order: What

Happens Between "Buy" and "Delivered", Where Is My Order? Notifications, Service and the Customer Record, Journey 1: Eva's Jacket, From Search to Doorstep.

- **R6** · 2025 Retail Returns Landscape — NRF with Happy Returns, 2025-10-15. 19.3% online return rate, 9% fraudulent. *Practitioner and press.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Who Sells What to Whom: Business Models and the Economics That Shape Architecture, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Journey 2: The Jacket Comes Back.
- **R7** · Checkout fulfillment — Stripe, n.d., accessed 2026-11. Fulfillment must not rely on the landing page; the fulfillment function must handle repeated calls with the same session identifier. *Product and API documentation.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered".
- **R8** · iDEAL — Mollie documentation, n.d., accessed 2026-11. Instant charge, automatic capture, no chargeback, 15-minute session timeout. Also cited at: iDEAL. *Product and API documentation.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered", Taking the Money: Payment Providers, Authorization, Capture and Refunds, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R9** · Capture — Adyen, n.d., accessed 2026-11. Authorization reserves funds; capture by default immediately or manually on shipment. *Product and API documentation.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered", Taking the Money: Payment Providers, Authorization, Capture and Refunds, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R10** · Payments lifecycle — commercetools, n.d., accessed 2026-11. Order created with a pending payment; fulfill only on a successful transaction. Also cited at: Checkout payments lifecycle. *Product and API documentation.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered", From Cart to Confirmation: Checkout Orchestration.
- **R11** · Build for fulfillment services — Shopify developer documentation, n.d., accessed 2026-11. The fulfillment request handshake; cancellations after acceptance are requests. *Product and API documentation.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered", Is It in Stock? Inventory, Availability and the Promise You Can Keep, The Order Record: Lifecycle, State and the Order Management System, Pick, Pack, Ship: Warehouse Management and Fulfillment, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R12** · VAT rates and exemptions — business.gov.nl, n.d., accessed 2026-11. Dutch standard VAT rate of 21%. *Regulation and official source.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered", Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Tax, Invoices and the Books: From Order to Ledger, Journey 1: Eva's Jacket, From Search to Doorstep, Journey 3: A Chain Reorders by EDI, From Order to Invoice.

- **R13** · 50 Cart Abandonment Rate Statistics — Baymard Institute, 2026 (updated 28 October 2026). Average documented abandonment rate of 70.22% across fifty studies. *Practitioner and press.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered", Who Sells What to Whom: Business Models and the Economics That Shape Architecture, From Cart to Confirmation: Checkout Orchestration, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R14** · VF Corporation Form 10-K, fiscal 2026 — SEC filing, 2026. Definition of direct-to-consumer; 44% of revenue; gross margin 54.8%. *Company filing, report or announcement.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, What Changes at Each Scale: From First Store to Group.
- **R15** · NIKE, Inc. Form 10-K, fiscal 2026 — NIKE / SEC, 2026. NIKE Direct revenue decline driven by traffic; channel mix; gross margin 42.9%. *Company filing, report or announcement.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture.
- **R16** · Columbia Sportswear Form 10-K, 2025 — Columbia Sportswear / SEC, 2026. Wholesale vs direct-to-consumer channels; gross margin 50.5%. *Company filing, report or announcement.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture.
- **R17** · Amazon GMV surpassed $800 billion in 2025 — Marketplace Pulse, 2026. Analyst estimate of third-party share of units and GMV. *Practitioner and press.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture.
- **R18** · Target Corporation Form 10-K, fiscal 2025 — SEC filing, 2026. Stores fulfilled 97.6% of merchandise sales; Target Circle 360; gross margin 27.9%. *Company filing, report or announcement.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Discounts, Coupons and Loyalty: Promotions Without Chaos, What Changes at Each Scale: From First Store to Group.
- **R19** · Shopify pricing (EUR) — Shopify, n.d., accessed 2026-11. Plan prices in euros, Plus from €2,100, third-party gateway fees by tier, Shopify Payments rates. *Product and API documentation.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms, What Changes at Each Scale: From First Store to Group.
- **R20** · Shopware pricing — Shopware, n.d., accessed 2026-11. Community Edition free to €1 million GMV; Rise from €600; GMV-based pricing. *Product and API documentation.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms, What Changes at Each Scale: From First Store to Group, Case: commercetools, Shopware and the Composable Movement.

- **R21** · Adobe: Cyber Monday hits record $14.25 billion — Adobe, 2025. Cyber Week $44.2 billion; $16 million per minute. *Company filing, report or announcement.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture.
- **R22** · Warby Parker Form 10-K, 2025 — Warby Parker / SEC, 2026. Marketing expense 12.6% of revenue; gross margin 54.0%. *Company filing, report or announcement.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture.
- **R23** · Stripe pricing (Netherlands) — n.d., accessed 2026-11. Card and iDEAL list prices. *Product and API documentation.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture.
- **R24** · Mollie pricing — n.d., accessed 2026-11. Card and iDEAL list prices. *Product and API documentation.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R25** · Adyen pricing — n.d., accessed 2026-11. Card and iDEAL list prices. *Product and API documentation.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Taking the Money: Payment Providers, Authorization, Capture and Refunds, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R26** · Amazon selling fees — Amazon, n.d., accessed 2026-11. Referral fee of 15% for most categories (US page). *Product and API documentation.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R27** · Top 3 signs you need a PIM — Akeneo (guest post), n.d., accessed 2026-11. Signs a PIM is needed. *Product and API documentation.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, What Changes at Each Scale: From First Store to Group.
- **R28** · Hobbycraft to complete delayed digital roll-out by summer — Retail Week, 2013. Relaunch postponed past peak trading (evidence age). *Practitioner and press.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Moving Without Burning the Bridge: Replatforming and Migration.
- **R29** · E-commerce statistics (enterprises) — Eurostat, 2026 (2024 reference year). Web sales vs EDI-type sales; marketplace vs own-site shares; B2B vs B2C web sales. *Regulation and official source.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, What Changes at Each Scale: From First Store to Group.
- **R30** · Quarterly Retail E-Commerce Sales, 2nd Quarter 2026 — US Census Bureau, 2026. 17.1% adjusted share; 18.3% vs 15.8% not adjusted in Q4 and Q3 2025.

Regulation and official source. Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture.

- **R31** · Regulation (EU) 2015/751 on interchange fees — Official Journal, 2015-05-19. Caps of 0.2% and 0.3% for consumer cards, applied from 9 December 2015. *Regulation and official source.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R32** · HDE Online-Monitor — Handelsverband Deutschland, 2026. Online share of German retail 13.5%. *Practitioner and press.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture.
- **R33** · § 12 UStG — gesetze-im-internet.de, consolidated text, accessed 2026-11. Dutch 21% and German 19% standard VAT rates used in the worked example. *Regulation and official source.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Tax, Invoices and the Books: From Order to Ledger.
- **R34** · Average ecommerce conversion rate — Littledata, 2026. Median 1.3%, mean 1.9% across 438 Shopify stores. *Product and API documentation.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture.
- **R35** · The true cost of apparel returns — Coresight Research, 2023. 24.4% online apparel return rate (survey of 100; evidence age). *Practitioner and press.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics.
- **R36** · Forrester: US B2B e-commerce will reach an estimated $3 trillion by 2027 — Forrester, 2022. B2B e-commerce forecast; EDI tracked separately (evidence age). *Company filing, report or announcement.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R37** · Thuiswinkel Markt Monitor 2025 — NielsenIQ for Thuiswinkel.org, 2026. Dutch online spending €35.7 billion, down 1%. *Practitioner and press.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture.
- **R38** · bevh press releases — bevh, 2026. German online goods sales €83.1 billion in 2025. *Company filing, report or announcement.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture.
- **R39** · 8 signs your brand needs an order management system — Opollo, n.d., accessed 2026-11 (vendor content). Channel and location count as OMS triggers. *Practitioner and press.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture.
- **R40** · As AI Agents Transform Commerce, Salesforce Unleashes Its Biggest Agentforce Commerce Release Yet — Salesforce, 2026. A suite covering B2C, B2B, POS and order management in one platform. *Company filing, report or announcement.* Cited in: The Map of a Commerce Platform: Capabilities and

Layers, Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Assistants and Agents: AI in Commerce and What Is Real in 2026.

- **R41** · Adobe Commerce platforms and services — Adobe Experience League, 2026. Three deployment options and Commerce Optimizer working beside an existing backend. *Product and API documentation.* Cited in: The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Build, Buy or Assemble: Choosing and Changing Platforms, Case: commercetools, Shopware and the Composable Movement.
- **R42** · Shopify revenue and GMV in Q4 2025 — Digital Commerce 360, 2026-02-17. *Practitioner and press.* Cited in: The Map of a Commerce Platform: Capabilities and Layers.
- **R43** · Lightspeed fourth quarter and full year fiscal 2026 results — Lightspeed, 2026. Transaction revenue versus subscription revenue. *Product and API documentation.* Cited in: The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms.
- **R44** · Get started with the Shopify Connector — Microsoft Learn, 2026. The ERP connector preinstalled for new Business Central sign-ups. *Product and API documentation.* Cited in: The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, The Back Office: ERP Integration and Who Is the Hub, Build, Buy or Assemble: Choosing and Changing Platforms, What Changes at Each Scale: From First Store to Group.
- **R45** · What is an OMS? — Manhattan Associates, n.d., accessed 2026-11. An OMS vendor's definition of the order-management capability. *Product and API documentation.* Cited in: The Map of a Commerce Platform: Capabilities and Layers, The Order Record: Lifecycle, State and the Order Management System.
- **R46** · Akeneo — Akeneo, n.d., accessed 2026-11. A PIM vendor's definition of the product-information capability. *Product and API documentation.* Cited in: The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R47** · Fluent Commerce order management system — Fluent Commerce, n.d., accessed 2026-11. "your ERP stays the system of record". *Product and API documentation.* Cited in: The Map of a Commerce Platform: Capabilities and Layers, Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, The Order Record: Lifecycle, State and the Order Management System, Where Is My Order? Notifications, Service and the Customer Record.
- **R48** · ChannelEngine — ChannelEngine, n.d., accessed 2026-11. "over 1,300 marketplaces" as an example of a connector-count claim. *Product and API documentation.* Cited in: The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in

2026, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Selling on Someone Else's Platform: Marketplaces and Channel Integration.

- **R49** · Katapull — Katapull, 2026. The specialist-group shape, from public pages only. *Product and API documentation.* Cited in: The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms.
- **R50** · Over Katapull — Katapull, 2026. The specialist-group shape, from public pages only. *Product and API documentation.* Cited in: The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R51** · Katapull breidt Commerce Hub uit met overname van Picqer — Katapull, 2026. The specialist-group shape, from public pages only. *Product and API documentation.* Cited in: The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms.
- **R52** · Vijf softwarebedrijven bundelen krachten in Katapull — Dutch IT Channel, 2025. The specialist-group shape, from public pages only. *Practitioner and press.* Cited in: The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms.
- **R53** · EPAM Joins Newly Formed MACH Alliance as Founding Member — EPAM Systems, 2020. The founding of the MACH Alliance and the acronym's expansion. *Company filing, report or announcement.* Cited in: The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Build, Buy or Assemble: Choosing and Changing Platforms, Case: commercetools, Shopware and the Composable Movement.
- **R54** · OMS vs ERP: the architecture shift reshaping commerce — OneStock, 2026. The OMS as master for availability, the ERP for product data absent a PIM (vendor). *Product and API documentation.* Cited in: Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, The Back Office: ERP Integration and Who Is the Hub, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R55** · Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software* — Addison-Wesley, 2003. The bounded context (older than three years; the concept is unchanged). *Book.* Cited in: Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record.
- **R56** · BoundedContext — Martin Fowler, 2014. Bounded context as DDD's central strategic pattern; unification of a large model is not feasible or cost-effective; "Customer" and "Product" as polysemes (older than three years; the concept is unchanged). *Practitioner and press.* Cited in: Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record.

- **R57** · Shopify Agentic plan — Shopify, n.d., accessed 2026-11. The no-monthly-fee agentic channel plan. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R58** · Commerce.com Q2 2026 results, exhibit 99.1 — Commerce.com via SEC EDGAR, 2026. Revenue, GMV, the "softer than historically" replatforming remark. *Company filing, report or announcement.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Delivering the Storefront: Themes, Headless, Rendering and Performance, Build, Buy or Assemble: Choosing and Changing Platforms, Case: commercetools, Shopware and the Composable Movement.
- **R59** · BigCommerce rebrands as Commerce.com — Digital Commerce 360, 2025. The rename and the three brands. *Practitioner and press.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R60** · Lightspeed announces results of strategic review and Q3 FY2025 results — Lightspeed, 2025. The two growth engines. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms.
- **R61** · Lightspeed eCom pricing (NL) — Lightspeed, n.d., accessed 2026-11. Dutch tiers. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R62** · CCV Shop pricing — CCV Shop, n.d., accessed 2026-11. Flat tiers, "not a percentage of your sales". *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R63** · Spartacus repository — SAP on GitHub, n.d., accessed 2026-11. SAP Commerce Cloud's composable storefront. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R64** · SAP's CX vision gets GA timeline at Connect 2026 — SoftwareReviews, 2026. The cloud ERP edition and the "available now versus demonstrations" warning. *Practitioner and press.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R65** · Salesforce Commerce pricing — Salesforce, n.d., accessed 2026-11. "Contact for pricing" on every edition. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R66** · Demystifying Adobe Commerce as a Cloud Service — Rightpoint, 2025. The June 2025 launch timing (partner report). *Practitioner and press.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R67** · Magento — Wikipedia, 2026. The 2018 Adobe acquisition and the 2021 rename. *Encyclopedia.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement.

- **R68** · Mage-OS home — Mage-OS Association, 2022–2026, accessed 2026-11. The distribution, the association, releases 3.6.0 and 4.0.0. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R69** · about — Mage-OS Association, 2022–2026, accessed 2026-11. The distribution, the association, releases 3.6.0 and 4.0.0. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R70** · WooCommerce pricing — WooCommerce / Automattic, n.d., accessed 2026-11. Free core, hosting and extension costs; the competitor remark on Salesforce pricing. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R71** · PrestaShop — PrestaShop, n.d., accessed 2026-11. The open-source platform named in the category table. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R72** · PayPal increases stake in Shopware — Ecommerce News Europe, 2025. The stake rising to about 41%. *Practitioner and press.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R73** · commercetools, a pioneer in headless commerce, lays off dozens of staff — TechCrunch, 2025. The February 2025 cut and its stated reasons. *Practitioner and press.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms, Case: commercetools, Shopware and the Composable Movement.
- **R74** · Elastic Path — vendor pages, n.d., accessed 2026-11. Positioning, licensing and GMV-capped pricing. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R75** · Medusa — vendor pages, n.d., accessed 2026-11. Positioning, licensing and GMV-capped pricing. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R76** · Saleor pricing — Saleor, n.d., accessed 2026-11. Positioning, licensing and GMV-capped pricing. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms, Case: commercetools, Shopware and the Composable Movement.
- **R77** · Odoo pricing — vendor pages, 2026. Per-user pricing and ERP-centered positioning. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R78** · JTL — vendor pages, 2026. Per-user pricing and ERP-centered positioning. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.

- **R79** · PlentyONE — vendor pages, 2026. Per-user pricing and ERP-centered positioning. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R80** · SuiteCommerce solutions — Oracle NetSuite help, n.d., accessed 2026-11. The three SuiteCommerce products. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R81** · Katapull breidt e-commerce platform uit met overname Returnista — Dutch IT Channel, 2026. The formation and the Returnista acquisition. *Practitioner and press.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R82** · Manhattan Active Omni — Specialist-layer vendor pages, n.d., accessed 2026-11. The definitions, claims and prices in the specialist-layer table. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R83** · Fluent Commerce — Specialist-layer vendor pages, n.d., accessed 2026-11. The definitions, claims and prices in the specialist-layer table. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R84** · Kibo, Forrester Wave Q3 2026 — Kibo Commerce, 2026. The definitions, claims and prices in the specialist-layer table. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R85** · inRiver — Specialist-layer vendor pages, n.d., accessed 2026-11. The definitions, claims and prices in the specialist-layer table. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R86** · Picqer pricing — Picqer, n.d., accessed 2026-11. The definitions, claims and prices in the specialist-layer table. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, What Changes at Each Scale: From First Store to Group.
- **R87** · Loop — vendor sites, n.d., accessed 2026-11. The definitions, claims and prices in the specialist-layer table. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics.
- **R88** · Algolia pricing — Specialist-layer vendor pages, n.d., accessed 2026-11. The definitions, claims and prices in the specialist-layer table. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R89** · Celigo — Specialist-layer vendor pages, n.d., accessed 2026-11. The definitions, claims and prices in the specialist-layer table. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.

- **R90** · Stripe pricing (NL) — Specialist-layer vendor pages, n.d., accessed 2026-11. The definitions, claims and prices in the specialist-layer table. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R91** · MACH principles — MACH Alliance, n.d., accessed 2026-11. The three principles as stated in 2026. *Practitioner and press.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Build, Buy or Assemble: Choosing and Changing Platforms, Case: commercetools, Shopware and the Composable Movement.
- **R92** · Usage statistics of e-commerce technologies — W3Techs, 2026. Website-based usage shares and their limits. *Practitioner and press.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R93** · BigCommerce named a Challenger in the 2025 Gartner Magic Quadrant — GlobeNewswire via Nasdaq, 2025. The 19-vendor count. *Company filing, report or announcement.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026.
- **R94** · Productscape domain models of eleven commerce products, inferred from public product behavior — the author's working models (sibling repository; digest in this book's research notes), 2026. The capability inventory the book's coverage was checked against, mapped in Domain Coverage: From Three Model Families to Forty-Nine Chapters; research maps, not descriptions of any company's systems. *Author's working model.* Cited in: Introduction: Why Commerce Architecture Matters, Domain Coverage: From Three Model Families to Forty-Nine Chapters.

Catalog, pricing and search

- **R95** · Draft order and transfer shipment inventory is moving from reserved to committed — Shopify developer changelog, 2026-08-05. A stock state's meaning changing under its readers. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Is It in Stock? Inventory, Availability and the Promise You Can Keep, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R96** · Inventory (InventoryEntry) — commercetools API documentation, n.d., accessed 2026-11. The 10-second availability consistency window. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Is It in Stock? Inventory, Availability and the Promise You Can Keep, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R97** · Directive (EU) 2019/2161 (Omnibus) — Publications Office of the EU, 2019. The 30-day prior-price rule for price reductions, applied from 28 May 2022. *Regulation and official source.* Cited in: Introduction: Why Commerce Architecture

Matters, Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Discounts, Coupons and Loyalty: Promotions Without Chaos, Finding the Product: Search, Navigation and Merchandising, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.

- **R98** · ACM wil einde aan nepkortingen — ACM, 2022-12-06. Supervision from 2023-01-01. *Court or competition decision.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R99** · New product model (GraphQL Admin API) — Shopify developer documentation, n.d., accessed 2026-11. The product, option and variant model and the `productSet` mutation for an external source of truth. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R100** · The product variant limit is now 2,048 for all merchants — Shopify changelog, 2025-10-15. The variant limit and the warning to apps. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R101** · 2,048 variants — Shopify blog, 2025-10-15. Why the REST API could not carry high-variant products. *Company engineering.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R102** · Variants — Shopify Help Center, n.d., accessed 2026-11. Variants in merchant terms; translations and per-language URLs. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R103** · Markets: languages — Shopify Help Center, n.d., accessed 2026-11. Variants in merchant terms; translations and per-language URLs. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R104** · Standard Product Taxonomy — Shopify, versioned 2026-10, accessed 2026-11. An open-source taxonomy with categories, attributes and values. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R105** · Products — commercetools documentation, n.d., accessed 2026-11. Immutable SKUs, staged and published data, deletion only when unpublished. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R106** · Introduced Modular Catalog in public beta — 2026-06-26. The two catalog models and the 100 and 10,000 variant limits. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R107** · Modular Catalog is now generally available — 2026-09-29, commercetools release notes. The two catalog models and the 100 and 10,000 variant limits. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.

- **R108** · Create a product — Adobe Experience League, updated 2026-10-08. Simple, configurable, grouped, bundle, virtual, downloadable and gift-card product types. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R109** · What is a PIM — Akeneo, n.d., accessed 2026-11. A vendor's definition of PIM, enrichment and syndication. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog, The Back Office: ERP Integration and Who Is the Hub.
- **R110** · pimcore/pimcore — Pimcore, version 2026.1, accessed 2026-11. An open-core PIM and master-data platform. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R111** · Product Content API v10 — bol, n.d., accessed 2026-11. The EAN requirement, the 4,000+ "chunks", mandatory enrichment levels, asynchronous upload reports and the daily data-model update. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R112** · schema.org Product — schema.org, 2026 (v30.1). `sku`, `gtin`, `ProductGroup`, `isVariantOf` and inheritance of features. *Standard or specification.* Cited in: What Are We Selling? Products, Variants and the Catalog, Finding the Product: Search, Navigation and Merchandising, Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R113** · What is a GS1 GTIN — GS1 support, updated 2024-08-30. The GTIN as unique, global and verifiable, issued only by a GS1 member organization. *Standard or specification.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R114** · Global Trade Item Number — Wikipedia, accessed 2026-11. The GTIN-8, -12, -13 and -14 formats and the rule that each trade item and packaging configuration gets its own GTIN (secondary source; GS1's own site blocked fetching). *Encyclopedia.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R115** · Product data specification — Google Merchant Center help, n.d., accessed 2026-11. Required fields, the GTIN disapproval rule, `item_group_id`, availability values and the image minimum from 2027-01-31. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog, What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R116** · google_product_category — Google Merchant Center help, n.d., accessed 2026-11. Required fields, the GTIN disapproval rule, `item_group_id`, availability values and the image minimum from 2027-01-31. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R117** · Google product taxonomy — Google, version 2021-09-21. The 5,595-category tree still in use. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.

- **R118** · Merchant listing structured data — Google Search Central, n.d., accessed 2026-11. Variant URLs and `priceValidUntil`. *Product and API documentation.* Cited in: What Are We Selling? Products, Variants and the Catalog.
- **R119** · B2B catalogs — Shopify Help Center, n.d., accessed 2026-11. Catalogs as price lists, volume pricing, the three-assignment limit below Plus. *Product and API documentation.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R120** · Markets: pricing — Shopify Help Center, n.d., accessed 2026-11. Converted versus fixed market prices and automatic rounding. *Product and API documentation.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax.
- **R121** · Rounding prices — Shopify Help Center, n.d., accessed 2026-11. Converted versus fixed market prices and automatic rounding. *Product and API documentation.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax.
- **R123** · Pricing and discounts overview — commercetools docs, n.d., accessed 2026-11. Price scopes, the eight-step selection fallback, "the cheapest is selected", 100 embedded and 50,000 standalone prices. *Product and API documentation.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R124** · Standalone Prices — commercetools documentation, n.d., accessed 2026-11. Price scopes, the eight-step selection fallback, "the cheapest is selected", 100 embedded and 50,000 standalone prices. *Product and API documentation.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax.
- **R125** · Quotes — commercetools, n.d., accessed 2026-11. Quotes as "legally binding custom offers". *Product and API documentation.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R126** · Tier pricing — Adobe Experience League, 2026 and n.d., accessed 2026-11. Tier precedence, one public shared catalog, "the original price is used". *Product and API documentation.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax.
- **R127** · Shared catalogs — Adobe Experience League, 2026 and n.d., accessed 2026-11. Tier precedence, one public shared catalog, "the original price is used". *Product and API documentation.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax.
- **R128** · Purchase order flow — Adobe, n.d., accessed 2026-11. Tier precedence, one public shared catalog, "the original price is used". *Product and API documentation.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R129** · Europe1PriceFactory — SAP Commerce Javadoc, release 1811, 2018 (evidence age: eight years). Price-row matching and the dimensions of a price row.

Product and API documentation. Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax.

- **R130** · PriceRowModel — SAP Commerce Javadoc, release 1811, 2018 (evidence age: eight years). Price-row matching and the dimensions of a price row. *Product and API documentation.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax.
- **R131** · KBA 2267117: price does not expire after validity end — SAP Support, n.d., accessed 2026-11. Validity compared with the cart creation date. *Product and API documentation.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax.
- **R132** · Notification type values — Amazon, n.d., accessed 2026-11. `PRICING_HEALTH` and `ANY_OFFER_CHANGED`. *Product and API documentation.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R133** · Directive 98/6/EC on consumer protection in the indication of the prices of products — European Parliament and Council, 1998-02-16 (OJ 1998-03-18). The selling price "including VAT and all other taxes". *Regulation and official source.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax.
- **R134** · § 11 PAngV — Bundesministerium der Justiz, Preisangabenverordnung of 2022, accessed 2026-11. The German transposition. *Regulation and official source.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R135** · ACM publiceert boetebesluiten webwinkels voor nepkortingen — ACM, 2024-06-11. The five fines. *Court or competition decision.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R136** · One Stop Shop — European Commission, n.d., accessed 2026-11. OSS from 2021-07-01 and the €10,000 threshold. *Regulation and official source.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Tax, Invoices and the Books: From Order to Ledger, Getting It There: Carriers, Labels, Tracking and the Delivery Promise, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R137** · Proceedings against Amazon terminated — Bundeskartellamt, 2013. The end of price parity clauses in the EU. *Court or competition decision.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax, Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R139** · Proceedings against Amazon (overview) — Bundeskartellamt, accessed 2026-11. The 2026-02-05 prohibition of price control mechanisms and the initial €59 million disgorgement; the page does not state whether the decision is final. *Court or competition decision.* Cited in: What Does It Cost? Price Lists, Contract Prices, Currencies and Tax.
- **R140** · Combining discounts — Shopify Help Center, n.d., accessed 2026-11. Discount classes, combination rules, order of application, the 25-automatic and

5 + 1 code limits. *Product and API documentation.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.

- **R141** · Discounts — Shopify Help Center, n.d., accessed 2026-11. Discounts applied before tax, scheduled rollouts tested against existing discounts. *Product and API documentation.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R142** · Store credit — Shopify Help Center, n.d., accessed 2026-11. Expiry set per issuance. *Product and API documentation.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R143** · Cart Discounts — commercetools docs, n.d., accessed 2026-11. Value types and the proportionate, even and individual allocation modes. *Product and API documentation.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R144** · Limits — commercetools, n.d., accessed 2026-11. Active cart discounts and codes per cart. *Product and API documentation.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos, Finding the Product: Search, Navigation and Merchandising, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation.
- **R145** · Create a cart price rule — Adobe Experience League, n.d., accessed 2026-11. Priority, discard subsequent rules, coupon usage caps, late-evaluated conditions. *Product and API documentation.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R146** · Cart price rules — Adobe Experience League, n.d., accessed 2026-11. Priority, discard subsequent rules, coupon usage caps, late-evaluated conditions. *Product and API documentation.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R147** · Starbucks Form 10-K, fiscal 2025 — Starbucks Corp. via SEC EDGAR, 2025. Stored value liability, breakage, Stars deferral and expiry. *Company filing, report or announcement.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R148** · Unexercised rights (breakage), Revenue guide 7.4 — PwC Viewpoint, n.d., accessed 2026-11. The ASC 606 breakage rules. *Practitioner and press.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R149** · Revenue from customers' unexercised rights — IFRScommunity.com, 2024. Secondary summary of IFRS 15 B44–B47 and material rights. *Practitioner and press.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R150** · Response Transformation and Profit Decomposition for Revenue Uplift Modeling — Gubela, Lessmann and Jaroszewicz, 2019. Revenue uplift as the objective for coupon targeting. *Research.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.

- **R151** · Interpretable Multiple Treatment Revenue Uplift Modeling — Gubela and Lessmann, 2021. Several coupon types, one with positive incremental revenue. *Research.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R152** · Promotion abuse in ecommerce — Ravelin, 2020. Abuse taxonomy and a vendor survey figure. *Practitioner and press.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R153** · What is incrementality — Uniqodo, 2024. Holdout method, vendor content. *Practitioner and press.* Cited in: Discounts, Coupons and Loyalty: Promotions Without Chaos.
- **R154** · Product Search — commercetools docs, n.d., accessed 2026-11. What is indexed, identifiers returned, price filters per scope, the deprecation dates. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R155** · Product Projection Search (deprecated) — commercetools docs, n.d., accessed 2026-11. What is indexed, identifiers returned, price filters per scope, the deprecation dates. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R156** · Shopify B2B integration — Algolia docs, n.d., accessed 2026-11. Daily resync, per-location prices, the 50-location ceiling. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R157** · Personalized pricing for B2B — Algolia docs, n.d., accessed 2026-11. The three B2B price designs. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R158** · Rules overview — Algolia docs, n.d., accessed 2026-11. Merchandising rule consequences, conditions and validity periods. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R159** · Search analytics metrics — Algolia docs, n.d., accessed 2026-11. Metric definitions. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R160** · Algolia launches NeuralSearch — Algolia, 2023. Launch date and a vendor-reported customer result. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R161** · Filters — Shopify Help Center, n.d., accessed 2026-11. Filter limits, boosts, semantic understanding. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R162** · Modifying search — Shopify Help Center, n.d., accessed 2026-11. Filter limits, boosts, semantic understanding. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R163** · Semantic search is now available on more plans — Shopify changelog, 2024. The 24 June 2024 rollout. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.

- **R164** · Constructor — Constructor, n.d., accessed 2026-11. Vendor claims, cited as such. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R165** · Zalando to launch a fashion assistant powered by ChatGPT — Zalando, 2023. The April 2023 beta. *Company filing, report or announcement.* Cited in: Finding the Product: Search, Navigation and Merchandising, Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R166** · The history of Amazon's recommendation algorithm — Amazon Science, 2019. Item-to-item filtering and the Prime Video baseline story (older than three years; historical). *Company engineering.* Cited in: Finding the Product: Search, Navigation and Merchandising, Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R167** · kNN search — Elastic docs, n.d., accessed 2026-11. Vector model constraints, hybrid retrieval, search-time synonyms. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R168** · Elasticsearch: Hybrid search — Elastic docs, n.d., accessed 2026-11. Vector model constraints, hybrid retrieval, search-time synonyms. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R169** · Search with synonyms — Elastic docs, n.d., accessed 2026-11. Vector model constraints, hybrid retrieval, search-time synonyms. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R170** · OpenSearch: Hybrid search — OpenSearch docs, n.d., accessed 2026-11. Hybrid search since 2.11, normalization and reciprocal rank fusion. *Product and API documentation.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R171** · Semantic Product Search — Nigam et al., Amazon, KDD 2019. Why lexical matching falls short and the reported gains. *Research.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R172** · Ecommerce search query types — Baymard Institute, 2024, updated 2026. Failure rates by query type and the search-vs-navigation preference. *Practitioner and press.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R173** · Ecommerce search report and benchmark — Baymard Institute, n.d., accessed 2026-11. Autocomplete and misspelling findings. *Practitioner and press.* Cited in: Finding the Product: Search, Navigation and Merchandising.
- **R174** · Shopify: Inventory management apps — Shopify developer documentation, n.d., accessed 2026-11. The eight quantities, the on-hand sum, `inventorySetQuantities`. *Product and API documentation.* Cited in: Is It in Stock? Inventory, Availability and the Promise You Can Keep, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Journey 1: Eva's Jacket, From Search to Doorstep, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.

- **R175** · Inventory states — Shopify Help Center, n.d., accessed 2026-11. The five merchant-facing states and their arithmetic. *Product and API documentation.* Cited in: Is It in Stock? Inventory, Availability and the Promise You Can Keep.
- **R176** · New On hand and Unavailable inventory states in POS — Shopify changelog, 2023-02-09. Dating of the states and the stated aim of reducing oversells. *Product and API documentation.* Cited in: Is It in Stock? Inventory, Availability and the Promise You Can Keep, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R177** · Order management apps — Shopify developer documentation, n.d., accessed 2026-11. Assigned location and when it may change. *Product and API documentation.* Cited in: Is It in Stock? Inventory, Availability and the Promise You Can Keep, The Order Record: Lifecycle, State and the Order Management System, Pick, Pack, Ship: Warehouse Management and Fulfillment.
- **R178** · Carts (InventoryMode) — commercetools documentation, n.d., accessed 2026-11. Quantities, the three inventory modes, reservation expiry, the 10-second window. *Product and API documentation.* Cited in: Is It in Stock? Inventory, Availability and the Promise You Can Keep.
- **R179** · Inventory APIs overview — commercetools documentation, n.d., accessed 2026-11. Quantities, the three inventory modes, reservation expiry, the 10-second window. *Product and API documentation.* Cited in: Is It in Stock? Inventory, Availability and the Promise You Can Keep.
- **R180** · Introduction to Inventory Management — Adobe Commerce documentation, updated 2026-08-14. Sources, stocks, salable quantity, reservations, sourcing algorithms. *Product and API documentation.* Cited in: Is It in Stock? Inventory, Availability and the Promise You Can Keep.
- **R181** · Source algorithms and reservations — Adobe Commerce documentation, updated 2026-08-20. Sources, stocks, salable quantity, reservations, sourcing algorithms. *Product and API documentation.* Cited in: Is It in Stock? Inventory, Availability and the Promise You Can Keep, The Order Record: Lifecycle, State and the Order Management System, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R182** · StockService, SAP Commerce 2205 Javadoc — SAP, 2022. Reserve, release and actual stock level per warehouse; 2022 documentation, the model is long-lived. *Product and API documentation.* Cited in: Is It in Stock? Inventory, Availability and the Promise You Can Keep.
- **R183** · Picqer product and API pages — Picqer, n.d., accessed 2026-11. The `products.free_stock_changed` webhook. *Product and API documentation.* Cited in: Is It in Stock? Inventory, Availability and the Promise You Can Keep, Pick, Pack, Ship: Warehouse Management and Fulfillment.
- **R184** · ZEOS becomes fulfilment partner of NEXT in continental Europe — Zalando SE, 2024. The shared stock pool; 24 markets and 18 channels. *Company filing, report or announcement.* Cited in: Is It in Stock? Inventory, Availability

and the Promise You Can Keep, Pick, Pack, Ship: Warehouse Management and Fulfillment.

- **R185** · Open-access review citing inventory inaccuracy rates of 20–67% — *Production* journal, 2018. Range across empirical studies. *Research.* Cited in: Is It in Stock? Inventory, Availability and the Promise You Can Keep.

Checkout, payments, tax and orders

- **R186** · The checkout.liquid theme file is being deprecated — Shopify changelog, 2023-02-13. The 2024-08-13 sunset. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, From Cart to Confirmation: Checkout Orchestration, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Journey 1: Eva's Jacket, From Search to Doorstep, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R187** · Refund and cancel payments — Stripe documentation, n.d., accessed 2026-11. Double-refund risk, three-day retries, fulfillment from webhooks. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Taking the Money: Payment Providers, Authorization, Capture and Refunds, Tax, Invoices and the Books: From Order to Ledger, Journey 2: The Jacket Comes Back.
- **R188** · Fulfill orders with Checkout — Stripe documentation, n.d., accessed 2026-11. Double-refund risk, three-day retries, fulfillment from webhooks. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, From Cart to Confirmation: Checkout Orchestration, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R189** · Refunds — Mollie documentation, n.d., accessed 2026-11. Duplicate blocking, "do not retry automatically", ten retries over 26 hours. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Taking the Money: Payment Providers, Authorization, Capture and Refunds, Journey 2: The Jacket Comes Back.
- **R190** · Mollie: Webhooks — Mollie documentation, n.d., accessed 2026-11. Duplicate blocking, "do not retry automatically", ten retries over 26 hours. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Follow the Order: What Happens Between "Buy" and "Delivered", From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R191** · Shopify: checkout.liquid layout reference — n.d., accessed 2026-11. The 28 August 2025 and 26 August 2026 sunsets for thank-you and order-status pages. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout

Orchestration, Journey 1: Eva's Jacket, From Search to Doorstep, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.

- **R192** · Upgrade to checkout extensibility — Shopify Help Center, n.d., accessed 2026-11. Plan gating and deadlines. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration.
- **R193** · Checkout extensibility — n.d., accessed 2026-11. Plan gating and deadlines. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R194** · Script Editor — Shopify Help Center, n.d., accessed 2026-11. Shopify Scripts retired on 30 June 2026. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R195** · Shopify Functions — shopify.dev, n.d., accessed 2026-11. Sandboxed checkout logic and its limits. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R196** · Function APIs — shopify.dev, n.d., accessed 2026-11. Sandboxed checkout logic and its limits. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R197** · Place a hold on a payment method — Stripe documentation, n.d., accessed 2026-11. Fulfillment by webhook, delivery guarantees, authorization windows, idempotency keys, card-testing controls. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R198** · Protect yourself from card testing — Stripe documentation, n.d., accessed 2026-11. Fulfillment by webhook, delivery guarantees, authorization windows, idempotency keys, card-testing controls. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R199** · Handling payment status — Mollie documentation, n.d., accessed 2026-11. Session expiry per method and the id-only webhook with ten retries over 26 hours. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R200** · Apple Pay — Apple developer site, n.d., accessed 2026-11. Express wallet flows and tokens. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration.

- **R201** · Google Pay API for web: overview — Google, n.d., accessed 2026-11. Express wallet flows and tokens. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration.
- **R202** · PaymentTermsType — Shopify Admin API 2026-10, accessed 2026-11. B2B payment-term types. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R203** · Important updates announced for merchants validating to SAQ A — PCI SSC, 2025-01-30. Payment-page script requirements 6.4.3 and 11.6.1 and the 31 March 2025 date. *Standard or specification.* Cited in: From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R204** · Visa Acquirer Monitoring Program fact sheet — Visa, 2025. Enumeration monitoring. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R205** · Commission Delegated Regulation (EU) 2018/389 — Official Journal of the EU, 2018-03-13. Strong customer authentication and its exemptions, applied from 14 September 2019. *Regulation and official source.* Cited in: From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R206** · Checkout flow: average form fields — Baymard Institute, 2024. 11.3 fields and 5.1 steps on average; eight fields suffice. *Practitioner and press.* Cited in: From Cart to Confirmation: Checkout Orchestration.
- **R207** · Adjust authorisation — Adyen documentation, n.d., accessed 2026-11. Authorization and capture semantics, expiry windows, asynchronous refunds, payout batches, dispute stages, the PCI responsibility boundary. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R208** · Refund — Adyen documentation, n.d., accessed 2026-11. Authorization and capture semantics, expiry windows, asynchronous refunds, payout batches, dispute stages, the PCI responsibility boundary. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Tax, Invoices and the Books: From Order to Ledger, Journey 2: The Jacket Comes Back.
- **R209** · Sales day payout — Adyen documentation, n.d., accessed 2026-11. Authorization and capture semantics, expiry windows, asynchronous refunds, payout batches, dispute stages, the PCI responsibility boundary. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization,

Capture and Refunds, Tax, Invoices and the Books: From Order to Ledger, Journey 1: Eva's Jacket, From Search to Doorstep.

- **R210** · Dispute process and flow — Adyen documentation, n.d., accessed 2026-11. Authorization and capture semantics, expiry windows, asynchronous refunds, payout batches, dispute stages, the PCI responsibility boundary. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R211** · Adyen: Webhooks — Adyen documentation, n.d., accessed 2026-11. Authorization and capture semantics, expiry windows, asynchronous refunds, payout batches, dispute stages, the PCI responsibility boundary. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R212** · Batch-level reconciliation — Adyen documentation, n.d., accessed 2026-11. Authorization and capture semantics, expiry windows, asynchronous refunds, payout batches, dispute stages, the PCI responsibility boundary. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Tax, Invoices and the Books: From Order to Ledger.
- **R213** · PCI DSS compliance guide — Adyen Docs, n.d., accessed 2026-11. Authorization and capture semantics, expiry windows, asynchronous refunds, payout batches, dispute stages, the PCI responsibility boundary. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R214** · SAQ A eligibility — Adyen documentation, n.d., accessed 2026-11. Authorization and capture semantics, expiry windows, asynchronous refunds, payout batches, dispute stages, the PCI responsibility boundary. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R215** · Disputes — Stripe documentation, n.d., accessed 2026-11. Authorization windows, refund behavior and fees, dispute mechanics and thresholds, SCA exemptions and liability shift, MITs, idempotency keys, webhook delivery, settlement timing, reconciliation reports, SAQ mapping, card-testing controls. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R216** · Measuring disputes — Stripe documentation, n.d., accessed 2026-11. Authorization windows, refund behavior and fees, dispute mechanics and thresholds, SCA exemptions and liability shift, MITs, idempotency keys, webhook delivery, settlement timing, reconciliation reports, SAQ mapping, card-testing controls. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R217** · Monitoring programs — Stripe documentation, n.d., accessed 2026-11. Authorization windows, refund behavior and fees, dispute mechanics and thresholds, SCA exemptions and liability shift, MITs, idempotency keys, webhook delivery,

settlement timing, reconciliation reports, SAQ mapping, card-testing controls. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.

- **R218** · Strong Customer Authentication — Stripe documentation, n.d., accessed 2026-11. Authorization windows, refund behavior and fees, dispute mechanics and thresholds, SCA exemptions and liability shift, MITs, idempotency keys, webhook delivery, settlement timing, reconciliation reports, SAQ mapping, card-testing controls. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R219** · CITs and MITs — Stripe documentation, n.d., accessed 2026-11. Authorization windows, refund behavior and fees, dispute mechanics and thresholds, SCA exemptions and liability shift, MITs, idempotency keys, webhook delivery, settlement timing, reconciliation reports, SAQ mapping, card-testing controls. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R220** · Payouts — Stripe documentation, n.d., accessed 2026-11. Authorization windows, refund behavior and fees, dispute mechanics and thresholds, SCA exemptions and liability shift, MITs, idempotency keys, webhook delivery, settlement timing, reconciliation reports, SAQ mapping, card-testing controls. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R221** · Payout reconciliation report — Stripe documentation, n.d., accessed 2026-11. Authorization windows, refund behavior and fees, dispute mechanics and thresholds, SCA exemptions and liability shift, MITs, idempotency keys, webhook delivery, settlement timing, reconciliation reports, SAQ mapping, card-testing controls. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Tax, Invoices and the Books: From Order to Ledger, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics, Journey 1: Eva's Jacket, From Search to Doorstep, Toolkit: Cards and Templates for Commerce Architecture Decisions.
- **R222** · Integration security guide — Stripe Docs, n.d., accessed 2026-11. Authorization windows, refund behavior and fees, dispute mechanics and thresholds, SCA exemptions and liability shift, MITs, idempotency keys, webhook delivery, settlement timing, reconciliation reports, SAQ mapping, card-testing controls. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R223** · PCI compliance guide — Stripe documentation, n.d., accessed 2026-11. Authorization windows, refund behavior and fees, dispute mechanics and thresholds, SCA exemptions and liability shift, MITs, idempotency keys, webhook delivery, settlement timing, reconciliation reports, SAQ mapping, card-testing controls.

Product and API documentation. Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.

- **R224** · Place a hold for a payment — Mollie documentation, n.d., accessed 2026-11. iDEAL semantics, hold periods, duplicate-refund blocking, id-only webhooks with ten retries over 26 hours, status model and expiry per method, settlement contents. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R225** · Settlement report — Mollie documentation, n.d., accessed 2026-11. iDEAL semantics, hold periods, duplicate-refund blocking, id-only webhooks with ten retries over 26 hours, status model and expiry per method, settlement contents. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Tax, Invoices and the Books: From Order to Ledger.
- **R226** · Shopify Payments Terms of Service (Ireland) — Shopify, n.d., accessed 2026-11. A platform's payments as a wrapper over licensed PSPs; Payments penetration; country availability. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R227** · Payment processors — Shopify, n.d., accessed 2026-11. A platform's payments as a wrapper over licensed PSPs; Payments penetration; country availability. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R228** · Shopify Payments supported countries — Shopify, n.d., accessed 2026-11. A platform's payments as a wrapper over licensed PSPs; Payments penetration; country availability. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R229** · Annual turnover iDEAL and PIN payments — Dutch Payments Association, 2025-02-18. 2024 iDEAL volumes. *Company filing, report or announcement.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R230** · iDEAL | Wero — Currence iDEAL, n.d., accessed 2026-11. The scheme's current branding. *Product and API documentation.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R231** · 2.5 billion payments with Bancontact and Payconiq in 2024 — Bancontact Payconiq Company, 2025-02-12. Belgian online volumes. *Company filing, report or announcement.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R232** · PayPal remains most popular online payment method in Germany — Ecommerce News reporting EHI's "Online-Payment 2024", 2024. German method

shares for 2023 (secondary source). *Practitioner and press.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.

- **R233** · EMV 3-D Secure — EMVCo, n.d., accessed 2026-11. Frictionless and challenge flows. *Standard or specification.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R234** · Just published: PCI DSS v4.0.1 — PCI Security Standards Council, 2024-06-11. Versions, dates, scope, the SAQ A change. *Standard or specification.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R235** · PCI DSS — PCI SSC, n.d., accessed 2026-11. Versions, dates, scope, the SAQ A change. *Standard or specification.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds.
- **R236** · Study on the payment attitudes of consumers in the euro area (SPACE 2024) — European Central Bank, 2024-12. Online payment shares by method and country. *Regulation and official source.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R237** · Edit orders — Shopify developer documentation, n.d., accessed 2026-11. Calculated orders, unfulfilled lines only, balance collection or refund. *Product and API documentation.* Cited in: The Order Record: Lifecycle, State and the Order Management System, Where Is My Order? Notifications, Service and the Customer Record.
- **R238** · States — commercetools documentation, n.d., accessed 2026-11. Separate state enumerations; custom state machines with optional transitions. *Product and API documentation.* Cited in: The Order Record: Lifecycle, State and the Order Management System.
- **R239** · Kibo order management — n.d., accessed 2026-11. Candidates and suggestions, child shipments, routing criteria, edits validated against state. *Product and API documentation.* Cited in: The Order Record: Lifecycle, State and the Order Management System, Where Is My Order? Notifications, Service and the Customer Record.
- **R240** · Split orders across multiple fulfilment locations — Fluent Commerce documentation, n.d., accessed 2026-11. Split rules, cost and shipment caps, "your ERP stays the system of record". *Product and API documentation.* Cited in: The Order Record: Lifecycle, State and the Order Management System.
- **R241** · Manhattan Active Order Management — Manhattan Associates, n.d., accessed 2026-11. OMS definition and capabilities; Forrester Wave Q1 2025 placement as reported by the vendor. *Product and API documentation.* Cited in: The Order Record: Lifecycle, State and the Order Management System.
- **R242** · IBM Sterling Order Management — IBM, n.d., accessed 2026-11. Omnichannel fulfillment and the call-center add-on. *Product and API documentation.* Cited in: The Order Record: Lifecycle, State and the Order Management System.

- **R243** · Saga — Chris Richardson, microservices.io, n.d., accessed 2026-11. Compensation and orchestration vocabulary used for "cancel after shipment". *Practitioner and press.* Cited in: The Order Record: Lifecycle, State and the Order Management System, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation.
- **R244** · Specify product tax codes and tax behavior — Stripe Tax documentation, n.d., accessed 2026-11. Tax-inclusive versus tax-exclusive pricing and shipping allocation methods. *Product and API documentation.* Cited in: Tax, Invoices and the Books: From Order to Ledger.
- **R245** · AvaTax calculations — Avalara, n.d., accessed 2026-11. What a tax engine takes and returns (vendor pages). *Product and API documentation.* Cited in: Tax, Invoices and the Books: From Order to Ledger.
- **R246** · TaxJar API reference — TaxJar, n.d., accessed 2026-11. What a tax engine takes and returns (vendor pages). *Product and API documentation.* Cited in: Tax, Invoices and the Books: From Order to Ledger.
- **R247** · VAT rates — European Commission, n.d., accessed 2026-11. The 15% minimum standard rate and reduced-rate rules. *Regulation and official source.* Cited in: Tax, Invoices and the Books: From Order to Ledger.
- **R248** · § 14 UStG — gesetze-im-internet.de, consolidated text, accessed 2026-11. German rates, invoice content and numbering, eight-year invoice retention. *Regulation and official source.* Cited in: Tax, Invoices and the Books: From Order to Ledger.
- **R249** · § 14b UStG — gesetze-im-internet.de, consolidated text, accessed 2026-11. German rates, invoice content and numbering, eight-year invoice retention. *Regulation and official source.* Cited in: Tax, Invoices and the Books: From Order to Ledger.
- **R250** · Belgium: other taxes — PwC Worldwide Tax Summaries, 2026. Belgian VAT rates (secondary source; the official page could not be fetched). *Practitioner and press.* Cited in: Tax, Invoices and the Books: From Order to Ledger.
- **R251** · Cross-border VAT — Your Europe, n.d., accessed 2026-11. Reverse charge and VIES. *Regulation and official source.* Cited in: Tax, Invoices and the Books: From Order to Ledger, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R252** · Hoe lang moet u uw administratie bewaren — Belastingdienst, n.d., accessed 2026-11. Seven-year retention of basic records, ten for immovable property. *Regulation and official source.* Cited in: Tax, Invoices and the Books: From Order to Ledger.
- **R253** · About Peppol — OpenPeppol, n.d., accessed 2026-11. The four-corner model, document types 380 and 381. *Standard or specification.* Cited in: Tax, Invoices and the Books: From Order to Ledger, Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R254** · Peppol BIS Billing 3.0 — OpenPeppol, 2026. The four-corner model, document types 380 and 381. *Standard or specification.* Cited in: Tax, Invoices and

the Books: From Order to Ledger, Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.

- **R255** · VAT in the Digital Age (ViDA) — European Commission, n.d., accessed 2026-11. Adoption in March 2025 and the 2030 reporting date. *Regulation and official source.* Cited in: Tax, Invoices and the Books: From Order to Ledger, Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R256** · Unexercised rights (breakage) — RevenueHub, n.d., accessed 2026-11. Gift-card breakage under the US standard, used as a proxy for the IFRS wording. *Practitioner and press.* Cited in: Tax, Invoices and the Books: From Order to Ledger.
- **R257** · Payment reconciliation 101 — Stripe, 2025. Reconciliation as comparing transaction data with the books and investigating differences. *Product and API documentation.* Cited in: Tax, Invoices and the Books: From Order to Ledger.

Fulfillment, shipping, stores, returns and service

- **R258** · Build returns apps — Shopify developer documentation, n.d., accessed 2026-11. Return versus reverse fulfillment order; dispositions. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Journey 2: The Jacket Comes Back.
- **R259** · Zalando stellt Connected Retail ein — TextilWirtschaft, 2026. The program ends by the end of June 2027. *Practitioner and press.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R260** · Inventory Record Inaccuracy: An Empirical Analysis — DeHoratius and Raman, *Management Science* 54(4), 2008. 65% of records inaccurate (dated). *Research.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Is It in Stock? Inventory, Availability and the Promise You Can Keep, Pick, Pack, Ship: Warehouse Management and Fulfillment, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R261** · commercetools: Orders — commercetools API documentation, n.d., accessed 2026-11. Separate order, payment, shipment and return-item states. *Product and API documentation.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered", The Order Record: Lifecycle, State and the Order Management System, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Where Is My Order? Notifications, Service and the Customer Record, Journey 1: Eva's Jacket, From Search to Doorstep, Journey 2: The Jacket Comes Back.

- **R262** · Understanding order status pages — Shopify Help Center, n.d., accessed 2026-11. Five customer-facing states; no updates without a tracking number. *Product and API documentation.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered", Getting It There: Carriers, Labels, Tracking and the Delivery Promise, Where Is My Order? Notifications, Service and the Customer Record, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R263** · Directive 2011/83/EU on consumer rights — Publications Office of the EU, 2011. Delivery within 30 days (Article 18), 14-day withdrawal (Article 9), refund within 14 days (Article 13), passing of risk (Article 20). *Regulation and official source.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered", Is It in Stock? Inventory, Availability and the Promise You Can Keep, Getting It There: Carriers, Labels, Tracking and the Delivery Promise, The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Where Is My Order? Notifications, Service and the Customer Record, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Journey 1: Eva's Jacket, From Search to Doorstep, Journey 2: The Jacket Comes Back.
- **R264** · FulfillmentOrder object — Shopify developer documentation, n.d., accessed 2026-11. Assigned location and when it may change. *Product and API documentation.* Cited in: Is It in Stock? Inventory, Availability and the Promise You Can Keep, The Order Record: Lifecycle, State and the Order Management System, Pick, Pack, Ship: Warehouse Management and Fulfillment, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, Where Is My Order? Notifications, Service and the Customer Record, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R265** · Order routing — Kibo Commerce, n.d., accessed 2026-11. Candidates and suggestions, child shipments, routing criteria, edits validated against state. *Product and API documentation.* Cited in: The Order Record: Lifecycle, State and the Order Management System, Pick, Pack, Ship: Warehouse Management and Fulfillment, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R266** · Fulfillment concept guide — Kibo Commerce, n.d., accessed 2026-11. Candidates and suggestions, child shipments, routing criteria, edits validated against state. *Product and API documentation.* Cited in: The Order Record: Lifecycle, State and the Order Management System, Pick, Pack, Ship: Warehouse Management and Fulfillment, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R267** · Picqer product page — Picqer, n.d., accessed 2026-11. Batch pick lists, labels and tracking back in one step, 25 to 5,000+ orders per day, `picklists.closed` and stock-change webhooks, ~500 requests per minute. *Product and API documentation.* Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment, Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R268** · Odoo Inventory — Odoo, n.d., accessed 2026-11. Wave, cluster and batch picking, cycle counts, barcode scanning in an ERP's warehouse module. *Product*

and API documentation. Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment.

- **R269** · Manhattan Active Warehouse Management — Manhattan Associates, n.d., accessed 2026-11. Order streaming across wholesale, retail and direct orders; wave rebalancing. *Product and API documentation.* Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R270** · Warehouse Management Systems — Infios (formerly Körber Supply Chain Software), n.d., accessed 2026-11. Receiving to shipping scope, Gartner complexity levels 2 to 5, customer count. *Product and API documentation.* Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment.
- **R271** · Warehouse Management — Blue Yonder, n.d., accessed 2026-11. 2026 Gartner Magic Quadrant Leader claim. *Product and API documentation.* Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment.
- **R272** · Fulfillment by Amazon — Amazon, n.d., accessed 2026-11. What FBA does and its fee structure. *Product and API documentation.* Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment, Journey 2: The Jacket Comes Back.
- **R273** · Zalando Fulfillment Solutions reaches milestone — Zalando release via MarketScreener, 2019. ZFS launch in autumn 2017 and 100+ partner brands by June 2019 (older source). *Practitioner and press.* Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment.
- **R274** · ZFS invoice structure — Zalando Partner University, updated 2026-02-16. Fee categories including per-order outbound, per-item-per-day storage and the peak surcharge. *Product and API documentation.* Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment.
- **R275** · Productizing Zalando's capabilities: introducing ZEOS — Zalando SE, 2023. ZEOS as fulfillment for any channel from 12 warehouses. Also cited at: Productizing Zalando's capabilities: introducing ZEOS. *Company engineering.* Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment, Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R276** · SSCC for EDI suppliers — bol, n.d., accessed 2026-11. GS1 labels on all logistic units since 31 December 2021; SSCC in the DESADV. *Product and API documentation.* Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment, Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R277** · Jumbo: Richtlijnen gemechaniseerd DC (guidelines for the mechanised DC) — Jumbo, 2025. DESADV required at pallet level; refusal without DESADV or GS1 barcode. *Product and API documentation.* Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment, Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R278** · Jumbo: Algemene inkoopvoorwaarden (general purchasing conditions) — Jumbo, 2026. €70 per non-compliant message; refusal rights; delivery-performance

penalties. *Product and API documentation.* Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment, Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.

- **R279** · GS1 General Specifications, Release 26.0 — GS1, 2026. Definitions of GTIN, GLN and SSCC; the SSCC as the only identifier of a logistics unit. *Standard or specification.* Cited in: Pick, Pack, Ship: Warehouse Management and Fulfillment, Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice, Toolkit: Cards and Templates for Commerce Architecture Decisions.
- **R280** · PostNL developer portal — PostNL, n.d., accessed 2026-11. The V4 API families (Shipment, Return, Checkout with Locations and TimeFrame, Warehouse with Confirming), the phased rollout from 2025 and the legacy APIs. *Product and API documentation.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R281** · PostNL developer portal and API overview — PostNL, n.d., accessed 2026-11. The V4 API families (Shipment, Return, Checkout with Locations and TimeFrame, Warehouse with Confirming), the phased rollout from 2025 and the legacy APIs. *Product and API documentation.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R282** · PostNL Annual Report 2024 — PostNL, 2025. Delivery quality parcels NL 97% (2023 and 2024); 371 million parcels in 2024. *Company filing, report or announcement.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R283** · Parcel DE Shipping API v2 — DHL Group, updated 2026-11-03. Manifesting rule and 17:45 close-out, `lockerID` for Packstations, DHL Kleinpaket from 1 January 2025, paperless customs documents in production from 3 November 2026. *Product and API documentation.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R284** · Shipment Tracking – Unified API — DHL Group, changelog through 2026-09. Status codes vary by service; unrecognized values undefined; initial rate limit of 250 calls per day. *Product and API documentation.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R285** · DHL developer portal — DHL Group, n.d., accessed 2026-11. The TLS certificate change of 2–4 November 2026. *Product and API documentation.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R286** · Sending tracking notifications (Predict) — DPD Belgium, n.d., accessed 2026-11. Depot arrival and departure notifications. *Product and API documentation.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise, Where Is My Order? Notifications, Service and the Customer Record.
- **R287** · bpack integration manual and Shipping Manager API downloads — bpost, changelog through 2026-05-26. The v3 to v5 move of 17 April 2023; manual version

3.3.35. *Product and API documentation.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.

- **R288** · Sendcloud — Sendcloud, 2026-11-06. Customer, carrier and service-point counts (marketing figures; inconsistent carrier counts noted). *Product and API documentation.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R289** · Sendcloud developer site — n.d., accessed 2026-11. Customer, carrier and service-point counts (marketing figures; inconsistent carrier counts noted). *Product and API documentation.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R290** · nShift — nShift, n.d., accessed 2026-11. Carrier services, pick-up and drop-off point counts (marketing figures). *Product and API documentation.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R291** · Trackers — EasyPost documentation, n.d., accessed 2026-11. The ten tracker statuses, scan history and webhooks. *Product and API documentation.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise, Where Is My Order? Notifications, Service and the Customer Record, Journey 1: Eva's Jacket, From Search to Doorstep.
- **R292** · How Maude leveraged delivery promise to increase conversion rate by 12% — Wonderment, 2024–2026. Vendor-reported conversion effects of showing a delivery date; no peer review. *Practitioner and press.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R293** · Natch case study — Hive, 2024–2026. Vendor-reported conversion effects of showing a delivery date; no peer review. *Practitioner and press.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R294** · Customs formalities for low value consignments — European Commission DG TAXUD, n.d., accessed 2026-11. Import declarations for all goods since 1 July 2021; the reduced data set up to €150. *Regulation and official source.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R295** · Council document ST-15179-2026 — Council of the EU, 2026. The proposal to delete the €150 threshold (status not verified). *Regulation and official source.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R296** · SPEEDY: Framework for sharpening promise time estimates in sub-same-day delivery — Sarkar and Mandayam Comar, Amazon Science, ACM SIGSPATIAL, 2025. Learned promise windows and the A/B test result. *Research.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R297** · Conformal Predictive Distributions for Order Fulfillment Time Forecasting — Ye, Hijazi and Van Hentenryck, arXiv, 2025. Fulfillment time as a distribution; accuracy and late-delivery identification gains over rules. *Research.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R298** · PostNL fails to meet next-day delivery targets for sixth year — DutchNews, 2025. The regulator's letter-delivery figures and fine, cited as the contrast to the

parcel KPI. *Practitioner and press.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.

- **R299** · PostNL calls €7m fine for late deliveries disproportionate — DutchNews, 2026. The regulator's letter-delivery figures and fine, cited as the contrast to the parcel KPI. *Practitioner and press.* Cited in: Getting It There: Carriers, Labels, Tracking and the Delivery Promise.
- **R300** · Shopify POS — Shopify, n.d., accessed 2026-11. The platform-POS feature set (pickup in store, ship to customer, returns at any location, staff roles) and the €79 per location per month POS Pro price. *Product and API documentation.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R301** · Shopify POS pricing — Shopify, n.d., accessed 2026-11. The platform-POS feature set (pickup in store, ship to customer, returns at any location, staff roles) and the €79 per location per month POS Pro price. *Product and API documentation.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R302** · Using Shopify POS offline — Shopify Help Center, n.d., accessed 2026-11. What works offline, deferred capture, device limits and the 24-hour advice. *Product and API documentation.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R303** · Offline payments — Shopify Help Center, n.d., accessed 2026-11. What works offline, deferred capture, device limits and the 24-hour advice. *Product and API documentation.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R304** · Process offline payments — Square Support, n.d., accessed 2026-11. The 72-hour expiry and the merchant's responsibility for offline payments. *Product and API documentation.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R305** · What is unified commerce? — Shopify (Roxanne Voidonicolas), 2026. The vendor definition contrasted with omnichannel. *Product and API documentation.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R306** · Lightspeed Retail POS — Lightspeed, n.d., accessed 2026-11. Stock transfers between stores and customer profiles that follow shoppers between locations. *Product and API documentation.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R307** · Kassasysteem — Vendit, n.d., accessed 2026-11. A Dutch POS that runs from the same database as the web shop (translated). *Product and API documentation.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R308** · Point of sale documentation — Adyen, n.d., accessed 2026-11. In-person and online payments on one platform, the Terminal API and shopper recognition. *Product and API documentation.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R309** · Zalando baut Connected Retail aus — FashionUnited, 2021. The 3,400 active stores figure (evidence age). *Practitioner and press.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.

- **R310** · § 146a Abgabenordnung — German Fiscal Code, current text, accessed 2026-11. The certified technical security device, the receipt obligation and the registration duty. *Regulation and official source.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R311** · Kassensicherungsverordnung — ordinance of 26 September 2017, amended 2026. Transaction numbering, signature counter and receipt content. *Regulation and official source.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R312** · Mitteilungspflicht für elektronische Kassensysteme — Grant Thornton Germany, 2025. The ELSTER registration procedure and the 31 July 2025 deadline. *Practitioner and press.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R313** · Geregistreerd kassasysteem — FPS Finance (Belgium), 2026. The hospitality-only scope, the €25,000 threshold and the GKS 2.0 dates (translated). *Regulation and official source.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R314** · GKS 2.0 mandatory from 1 July 2026 — FPS Finance (Belgium), 2026. The hospitality-only scope, the €25,000 threshold and the GKS 2.0 dates (translated). *Regulation and official source.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R315** · Comunicato stampa: corrispettivi telematici — Agenzia delle Entrate, 2019. The 2019–2020 start of telematic receipts in Italy (translated; evidence age for the start dates only). *Regulation and official source.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R316** · Het kassasysteem in uw bedrijf — Belastingdienst, n.d., accessed 2026-11. Seven-year retention and the voluntary quality mark (translated). *Regulation and official source.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R317** · Keurmerk Betrouwbare Afrekensystemen — n.d., accessed 2026-11. *Practitioner and press.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R318** · Point-to-Point Encryption solutions — PCI Security Standards Council, n.d., accessed 2026-11. The validated P2PE solution listing. *Standard or specification.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R319** · RFID can lead to 100% order accuracy, Auburn study finds — Retail Dive on the Auburn University RFID Lab's Project Zipper, 2018. 69% of audited orders with barcode-data errors against under 0.01% with RFID (evidence age). *Practitioner and press.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R320** · UK retailers responding slowly to needs of online shoppers, says UPS — Post & Parcel on the UPS "Pulse of the Omni-channel Retailer" survey, 2016. 27% of European retailers using stores as online fulfillment centers (evidence age).

Practitioner and press. Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.

- **R321** · Click & collect sales and in-store online ordering — Savills Research, 2018. Click-and-collect and ship-from-store adoption in UK stores and the store managers who did not know how online refunds were booked (evidence age). *Practitioner and press.* Cited in: Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment.
- **R322** · ReverseFulfillmentOrderDispositionType — Shopify developer documentation, n.d., accessed 2026-11. The return, reverse fulfillment order and reverse delivery model, return statuses, dispositions, exchanges with a balance. *Product and API documentation.* Cited in: The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Journey 2: The Jacket Comes Back.
- **R323** · Manage customer returns — Amazon, 2025-04-15. Seller-fulfilled return windows, automatic authorization, prepaid labels, response and refund clocks, A-to-z Guarantee. *Product and API documentation.* Cited in: The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Journey 2: The Jacket Comes Back.
- **R324** · ZEOS becomes fulfilment partner of choice for British retailer NEXT — Zalando, 2024-11-21. Returns centres in a marketplace-run logistics network. *Company filing, report or announcement.* Cited in: The Way Back: Returns, Exchanges, Refunds and Reverse Logistics.
- **R325** · Narvar — n.d., accessed 2026-11. Returns-portal features and customer claims (volatile; marketing sources). *Practitioner and press.* Cited in: The Way Back: Returns, Exchanges, Refunds and Reverse Logistics, Where Is My Order? Notifications, Service and the Customer Record.
- **R326** · Returnista — vendor sites, n.d., accessed 2026-11. Returns-portal features and customer claims (volatile; marketing sources). *Product and API documentation.* Cited in: The Way Back: Returns, Exchanges, Refunds and Reverse Logistics.
- **R327** · Happy Returns — vendor sites, n.d., accessed 2026-11. Returns-portal features and customer claims (volatile; marketing sources). *Product and API documentation.* Cited in: The Way Back: Returns, Exchanges, Refunds and Reverse Logistics.
- **R328** · 2023 Consumer Returns in the Retail Industry — NRF and Appriss Retail, 2023-12-22. US return volumes, online return rates, free returns as a deciding factor, the fraudulent share. *Practitioner and press.* Cited in: The Way Back: Returns, Exchanges, Refunds and Reverse Logistics.
- **R329** · 2024 Consumer Returns in the Retail Industry — NRF and Happy Returns, 2024-12-05. US return volumes, online return rates, free returns as a deciding factor, the fraudulent share. *Practitioner and press.* Cited in: The Way Back: Returns, Exchanges, Refunds and Reverse Logistics.
- **R330** · Using the Shopify integration in Support and Chat — Zendesk, n.d., accessed 2026-11. The helpdesk's order sidebar, refund in any state, cancel only unfulfilled.

Product and API documentation. Cited in: Where Is My Order? Notifications, Service and the Customer Record.

- **R331** · Gorgias — Gorgias, n.d., accessed 2026-11. The one-inbox helpdesk with in-ticket order actions (vendor claims). *Product and API documentation.* Cited in: Where Is My Order? Notifications, Service and the Customer Record.
- **R332** · WhatsApp Business Platform pricing — Meta, accessed 2026-11. Template categories, per-message pricing from 1 July 2025 and the 1 October 2026 change for utility messages in the service window. *Product and API documentation.* Cited in: Where Is My Order? Notifications, Service and the Customer Record.
- **R333** · Service standards — bol partner platform, n.d., accessed 2026-11. The customer-questions standard, response time and strikes. *Product and API documentation.* Cited in: Where Is My Order? Notifications, Service and the Customer Record.
- **R334** · Customer questions — bol partner platform, n.d., accessed 2026-11. The customer-questions standard, response time and strikes. *Product and API documentation.* Cited in: Where Is My Order? Notifications, Service and the Customer Record.
- **R335** · Strike process — bol, n.d., accessed 2026-11. The customer-questions standard, response time and strikes. *Product and API documentation.* Cited in: Where Is My Order? Notifications, Service and the Customer Record, Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R336** · Order Performance program policy (DE) — Amazon, 2026. The Order Defect Rate definition and the under-1% threshold. *Product and API documentation.* Cited in: Where Is My Order? Notifications, Service and the Customer Record, Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R337** · Tokens API use case guide — Amazon, n.d., accessed 2026-11. Personal data available only through a restricted data token. *Product and API documentation.* Cited in: Where Is My Order? Notifications, Service and the Customer Record, Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R338** · WISMO benchmarks — Branvas, 2026. A critical reading of vendor WISMO figures as shares of tickets rather than of orders. *Practitioner and press.* Cited in: Where Is My Order? Notifications, Service and the Customer Record.

Marketplaces, B2B, EDI and ERP

- **R339** · Important Amazon MWS removal dates — Amazon SP-API changelog, 2023–2024. MWS unavailable from 2024-04-01. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R340** · bol partner platform — bol, 2026-11-03. The announcement on 3 November 2026 that Offers API v10 stops on 1 February 2027. *Product and API documentation.* Cited in: Introduction: Why Commerce Architecture Matters, Out of Stock, Out

of Sync: Sixteen Ways Commerce Platforms Go Wrong, Selling on Someone Else's Platform: Marketplaces and Channel Integration.

- **R341** · Synchronize customers and companies — Microsoft Learn, 2026. "decide which system is the system of record for customer data"; mixed directions cause conflicting records. *Product and API documentation.* Cited in: Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, From Cart to Confirmation: Checkout Orchestration, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, The Back Office: ERP Integration and Who Is the Hub, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R342** · ERP Integration: Methods, Benefits, and Examples — Shopify, 2026. "name the system of record for each" field; "Decide in advance what happens when the two systems disagree" (vendor guidance). *Product and API documentation.* Cited in: Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, Tax, Invoices and the Books: From Order to Ledger, Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, The Back Office: ERP Integration and Who Is the Hub.
- **R343** · Usage plans and rate limits — Amazon Selling Partner API, n.d., accessed 2026-11. Token-bucket rate limiting, HTTP 429, "Respond to events rather than polling". Also cited at: Usage plans and rate limits. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation.
- **R344** · Orders API rate limits — Amazon, n.d., accessed 2026-11. `getOrders` at 0.0167 requests a second; v0 deprecated, v2026-01-01 current. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R345** · Orders API — Amazon, n.d., accessed 2026-11. `getOrders` at 0.0167 requests a second; v0 deprecated, v2026-01-01 current. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R346** · Notifications API — Amazon, n.d., accessed 2026-11. Order change, pricing health, listing quantity and FBA inventory notifications. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R347** · Feeds API — Amazon, n.d., accessed 2026-11. Asynchronous feeds, 28-day result retention, Restricted Data Tokens for personal data. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R348** · Amazon seller policies to know — Amazon, 2024. ODR under 1%, late shipment under 4%, cancellation under 2.5%. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R349** · API lifecycle — bol Retailer API, n.d., accessed 2026-11. 12-month deprecation, asynchronous calls, "events as they happen". *Product and API*

documentation. Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.

- **R350** · Rate limiting — bol Retailer API, n.d., accessed 2026-11. 12-month deprecation, asynchronous calls, "events as they happen". *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R351** · Commission — bol, n.d., accessed 2026-11. Fee model, service standards, strikes, fulfillment surcharges. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R352** · Items on time — bol, n.d., accessed 2026-11. Fee model, service standards, strikes, fulfillment surcharges. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R353** · Cancellations — bol, n.d., accessed 2026-11. Fee model, service standards, strikes, fulfillment surcharges. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R354** · Conditions and rates LvB — bol, n.d., accessed 2026-11. Fee model, service standards, strikes, fulfillment surcharges. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R355** · Zalando logistics solutions: an overview — Zalando Partner University, n.d., accessed 2026-11. ZFS, Own Logistics and ZEOS terms. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R356** · The evolution of our B2B growth engine — Zalando SE, 2025. ZEOS launch; 24 markets, 18 channels, over 60 merchants. *Company filing, report or announcement.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R357** · EffectConnect — ChannelEngine support, vendor pages, n.d., accessed 2026-11. The four connector functions and channel counts. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R358** · Rithum — ChannelEngine support, vendor pages, n.d., accessed 2026-11. The four connector functions and channel counts. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R359** · Linnworks — ChannelEngine support, vendor pages, n.d., accessed 2026-11. The four connector functions and channel counts. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R360** · Merchant API: orders — ChannelEngine support, n.d. Acknowledgement "links the ChannelEngine order to your internal system". *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.

- **R361** · Marketplace Connect — Shopify, n.d., accessed 2026-11. Supported marketplaces, pricing, review profile. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R362** · app listing — Shopify, n.d., accessed 2026-11. Supported marketplaces, pricing, review profile. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R363** · Kaufland Global Marketplace conditions — vendor pages, n.d., accessed 2026-11. Subscription and commission figures. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R364** · OTTO Market — vendor pages, n.d., accessed 2026-11. Subscription and commission figures. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R365** · TikTok Shop expands across Europe — TikTok, 2026. Launch in the Netherlands, Belgium, Austria and Poland from 15 June 2026. *Company filing, report or announcement.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R366** · Meta phases out Facebook and Instagram Shops checkout — PPC Land, 2025. Move to website checkout (trade press). *Practitioner and press.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R367** · Mirakl (company site) — Wikipedia, vendor page, n.d., accessed 2026-11. Marketplace operator platform. *Product and API documentation.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R368** · Mirakl (Wikipedia) — Wikipedia, accessed 2026-11. Marketplace operator platform. *Encyclopedia.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration.
- **R369** · Shopify B2B — Shopify Help Center, n.d., accessed 2026-11. Plan availability, catalogs, volume pricing, the three-catalog cap below Plus. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R370** · B2B app development overview — Shopify, n.d., accessed 2026-11. Company, CompanyLocation and CompanyContact definitions; draft orders for review and approval. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R371** · PaymentTermsTemplate — Shopify Admin API 2026-10. Typed payment terms. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R372** · Business Units — commercetools, n.d., accessed 2026-11. The B2B object model, quoting and approval. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R373** · Quote Requests — commercetools, n.d., accessed 2026-11. The B2B object model, quoting and approval. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.

- **R374** · Staged Quotes — commercetools, n.d., accessed 2026-11. The B2B object model, quoting and approval. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R375** · Approval Rules — commercetools, n.d., accessed 2026-11. The B2B object model, quoting and approval. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R376** · Approval Flows — commercetools, n.d., accessed 2026-11. The B2B object model, quoting and approval. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R377** · B2B Commerce Organization — SAP Spartacus documentation, n.d., accessed 2026-11. Units, budgets, cost centers, purchase limits, approval (documents an unmaintained storefront version). *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R378** · B2B Order Approval — SAP Spartacus documentation, n.d., accessed 2026-11. Units, budgets, cost centers, purchase limits, approval (documents an unmaintained storefront version). *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R379** · Implement an Integration of SAP S/4HANA Cloud with SAP Commerce Cloud — SAP Learning, n.d., accessed 2026-11. Master data to commerce, orders to ERP. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, The Back Office: ERP Integration and Who Is the Hub.
- **R380** · B2B for Adobe Commerce guide overview — Adobe, 2026. Company accounts, shared catalogs, purchase orders, "the original price is used". *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R381** · About B2B Edition APIs — BigCommerce, n.d., accessed 2026-11. Company hierarchies, net terms and available credit, the separate API domain, the 30 September 2025 token deprecation. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R382** · Sana Commerce — vendor page, n.d., accessed 2026-11. The ERP-driven web store. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, The Back Office: ERP Integration and Who Is the Hub.
- **R383** · OroCommerce README — Oro Inc., 2026. RFQs, quotes, price lists, order entry. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R384** · Back-office Sales documentation — version 7.0, accessed 2026-11. RFQs, quotes, price lists, order entry. *Product and API documentation.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R385** · B2B ecommerce market (US) — Digital Commerce 360, 2024. Digital B2B sales and total sales, with their definitions. *Practitioner and press.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.

- **R386** · US B2B sales exceed $15 trillion in 2025 — Digital Commerce 360, 2026. Digital B2B sales and total sales, with their definitions. *Practitioner and press.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals.
- **R387** · cXML User's Guide 1.2.037 — cXML.org (SAP Ariba), 2018. The punchout protocol and messages (evidence age; still the current link). *Standard or specification.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, Documents Between Companies: EDI, Punchout and E-Invoicing.
- **R388** · SAP SRM OCI PunchOut — TradeCentric, 2026. OCI's fields and limits compared with cXML (integrator source). *Practitioner and press.* Cited in: Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals, Documents Between Companies: EDI, Punchout and E-Invoicing.
- **R389** · SPS Commerce Form 10-K for 2025 — SEC filing, 2026-02-19. Revenue, recurring share and the "connect once" description. *Company filing, report or announcement.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R390** · EDI Jumbo Supermarkten information — Jumbo, 2026 profile (accessed 2026-11). WebEDI vs integrated routes, document sets, the GLN rule. *Product and API documentation.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R391** · Walmart U.S. gives suppliers new order fulfillment guidelines — Talk Business & Politics, 2017. The 3% on-time-in-full fee at launch (older than three years; thresholds have changed). *Practitioner and press.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R392** · How to dispute Walmart OTIF chargebacks — SPS Commerce, 2024. Current targets as a provider reports them; no official Walmart page found. *Practitioner and press.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R393** · GS1 EANCOM 2002, Syntax 4, Part 1, Edition 2016 Upd. 2021 — GS1, 2021. EANCOM as a subset of UN/EDIFACT (standard older than three years). *Standard or specification.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R394** · Implementation of GS1 EDI standards in 2020 — GS1, 2021. Most-implemented messages and transports (older than three years). *Standard or specification.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R395** · X12 Transaction Sets — X12, n.d., accessed 2026-11. The 850, 855, 856, 810, 846 and 997 descriptions. *Standard or specification.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R396** · EDIFACT — Wikipedia, 2026 (accessed 2026-11). UN/EDIFACT's origin and ISO 9735 (encyclopedia source; unece.org refused fetches). *Encyclopedia.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing.

- **R397** · RFC 4130: Applicability Statement 2 (AS2) — IETF, 2005. What AS2 is (older than three years; still in production use). *Standard or specification.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R398** · Peppol BIS Billing 3.0 release notes — OpenPeppol, 2026. The usage specification of EN 16931, document type codes, release 3.0.21 dates. *Standard or specification.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R399** · What is eInvoicing — European Commission, n.d. (accessed 2026-11). The definition of an electronic invoice. *Regulation and official source.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing.
- **R400** · À partir de quand suis-je concerné par la réforme de la facturation électronique ? — DGFiP, France, 2026. The French dates (translated from French). *Regulation and official source.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing.
- **R401** · eInvoicing in Italy — European Commission, 2026. SdI since 2019, exemption removal 2024, derogation to 2027. *Regulation and official source.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing.
- **R402** · KSeF priority page — Ministry of Finance, Poland, 2026. The KSeF dates (translated from Polish). *Regulation and official source.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing.
- **R403** · E-invoicing — business.gov.nl, n.d. (accessed 2026-11). The existing Dutch obligation toward central government. *Regulation and official source.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing.
- **R404** · Design details: costing methods — Microsoft Learn, 2026. The five costing methods and the no-change rule. *Product and API documentation.* Cited in: The Back Office: ERP Integration and Who Is the Hub.
- **R405** · Handel: voorraad, orders en boekhouding — Exact, n.d. (accessed 2026-11). The mid-market ERP's self-description (translated from Dutch). *Product and API documentation.* Cited in: The Back Office: ERP Integration and Who Is the Hub.
- **R406** · ERP software — AFAS, n.d. (accessed 2026-11). One system, one database (translated from Dutch). *Product and API documentation.* Cited in: The Back Office: ERP Integration and Who Is the Hub.
- **R407** · IAS 2 Inventories — IFRS Foundation, n.d. (accessed 2026-11). Measurement at the lower of cost and net realisable value; FIFO and weighted average. *Standard or specification.* Cited in: The Back Office: ERP Integration and Who Is the Hub.
- **R636** · e-invoicing FAQ: general questions B2B — FPS Finance (Belgium), 2024-09-18, accessed 2026-11. The mandate and the fines. *Regulation and official source.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Documents Between Companies: EDI, Punchout and E-Invoicing, Journey 3: A Chain Reorders by EDI, From Order to Invoice.

Integration, contracts, identity and data

- **R408** · Customer accounts — Shopify developer documentation, n.d., accessed 2026-11. Legacy customer accounts deprecated as of 2026-02-26. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, One Customer, One Product: Identity and Master Data Across Systems.
- **R409** · Shopify: Webhooks — Shopify developer documentation, n.d., accessed 2026-11. Duplicate and unordered deliveries; 8 retries over 4 hours. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Is It in Stock? Inventory, Availability and the Promise You Can Keep, The Order Record: Lifecycle, State and the Order Management System, APIs, Events, Files and Middleware: How Commerce Systems Talk, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Promises Between Systems: API Contracts, Webhooks and Versioning, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 1: Eva's Jacket, From Search to Doorstep, Journey 4: Black Friday, Hour by Hour, Case: Shopify, the Modular Monolith That Serves Millions of Merchants, Toolkit: Cards and Templates for Commerce Architecture Decisions.
- **R410** · Shopify: HTTPS webhook delivery — Shopify, n.d., accessed 2026-11. Duplicate and unordered deliveries; 8 retries over 4 hours. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, APIs, Events, Files and Middleware: How Commerce Systems Talk, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Promises Between Systems: API Contracts, Webhooks and Versioning, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Journey 1: Eva's Jacket, From Search to Doorstep, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R411** · API versioning — Shopify developer documentation, n.d., accessed 2026-11. Quarterly API versions and their support period. *Product and API documentation.* Cited in: Introduction: Why Commerce Architecture Matters, Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, APIs, Events, Files and Middleware: How Commerce Systems Talk, Promises Between Systems: API Contracts, Webhooks and Versioning, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R412** · Receive Stripe events in your webhook endpoint — Stripe, n.d., accessed 2026-11. Double-refund risk, three-day retries, fulfillment from webhooks. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways

Commerce Platforms Go Wrong, Follow the Order: What Happens Between "Buy" and "Delivered", From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds, APIs, Events, Files and Middleware: How Commerce Systems Talk, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Promises Between Systems: API Contracts, Webhooks and Versioning, Journey 1: Eva's Jacket, From Search to Doorstep.

- **R413** · Business Central: Synchronize inventory with Shopify — Microsoft Learn, 2026-10-14. "Business Central acts as the system of record for inventory"; one-way push every 15–30 minutes. *Product and API documentation.* Cited in: Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, Is It in Stock? Inventory, Availability and the Promise You Can Keep, The Back Office: ERP Integration and Who Is the Hub, APIs, Events, Files and Middleware: How Commerce Systems Talk, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation.
- **R414** · Business Central: Synchronize items and inventory with Shopify — Microsoft Learn, 2026-10-16. Choosing the record per entity ("To Shopify" or "From Shopify"). *Product and API documentation.* Cited in: Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, What Are We Selling? Products, Variants and the Catalog, The Back Office: ERP Integration and Who Is the Hub, APIs, Events, Files and Middleware: How Commerce Systems Talk, One Customer, One Product: Identity and Master Data Across Systems.
- **R415** · Business Central: Synchronize and fulfill sales orders with Shopify — Microsoft Learn, 2026. Orders originate in the shop and are imported into the ERP. *Product and API documentation.* Cited in: Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, Tax, Invoices and the Books: From Order to Ledger, Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment, The Back Office: ERP Integration and Who Is the Hub, APIs, Events, Files and Middleware: How Commerce Systems Talk, One Customer, One Product: Identity and Master Data Across Systems.
- **R416** · Walmart's Real-Time Inventory System using Kafka — Suman Pattnaik on Confluent's blog, 2020. One consumer per item-store partition (older than three years; author's own views). *Product and API documentation.* Cited in: Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record, Is It in Stock? Inventory, Availability and the Promise You Can Keep, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R417** · Shopify: REST Admin API reference (legacy notice) — Shopify, n.d., accessed 2026-11. REST Admin API legacy as of 1 October 2024; deprecated fields removed on a schedule. *Product and API documentation.* Cited in: APIs, Events, Files and Middleware: How Commerce Systems Talk, Promises Between Systems: API Contracts, Webhooks and Versioning, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.

- **R418** · Debezium features — Debezium project, n.d. (version 3.7), accessed 2026-11. Change data capture as a product category. *Product and API documentation.* Cited in: APIs, Events, Files and Middleware: How Commerce Systems Talk.
- **R419** · MuleSoft Anypoint pricing — n.d., accessed 2026-11. iPaaS metering units and the public price points quoted. *Product and API documentation.* Cited in: APIs, Events, Files and Middleware: How Commerce Systems Talk.
- **R420** · Boomi pricing — n.d., accessed 2026-11. iPaaS metering units and the public price points quoted. *Product and API documentation.* Cited in: APIs, Events, Files and Middleware: How Commerce Systems Talk.
- **R421** · Workato pricing — n.d., accessed 2026-11. iPaaS metering units and the public price points quoted. *Product and API documentation.* Cited in: APIs, Events, Files and Middleware: How Commerce Systems Talk.
- **R422** · Celigo pricing — n.d., accessed 2026-11. iPaaS metering units and the public price points quoted. *Product and API documentation.* Cited in: APIs, Events, Files and Middleware: How Commerce Systems Talk.
- **R423** · Make pricing — n.d., accessed 2026-11. iPaaS metering units and the public price points quoted. *Product and API documentation.* Cited in: APIs, Events, Files and Middleware: How Commerce Systems Talk.
- **R424** · n8n pricing — n.d., accessed 2026-11. iPaaS metering units and the public price points quoted. *Product and API documentation.* Cited in: APIs, Events, Files and Middleware: How Commerce Systems Talk.
- **R425** · Cloud-based integration (iPaaS) — Wikipedia, relaying Gartner's definition retrieved 2015, accessed 2026-11. The iPaaS definition; the Gartner page itself was not reachable. *Encyclopedia.* Cited in: APIs, Events, Files and Middleware: How Commerce Systems Talk.
- **R426** · Integration Styles — Gregor Hohpe and Bobby Woolf, *Enterprise Integration Patterns*, 2003 (site updated 2023). The four styles and the criteria for choosing; the source is more than twenty years old and the styles are unchanged. *Book.* Cited in: APIs, Events, Files and Middleware: How Commerce Systems Talk.
- **R427** · What do you mean by "Event-Driven"? — Martin Fowler, 2017. Event notification, event-carried state transfer, event sourcing and CQRS. *Practitioner and press.* Cited in: APIs, Events, Files and Middleware: How Commerce Systems Talk.
- **R428** · Stripe: Rate limits — Stripe documentation, n.d., accessed 2026-11. Exponential backoff with randomness on `429`. *Product and API documentation.* Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Promises Between Systems: API Contracts, Webhooks and Versioning, Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics, Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour.
- **R429** · Shopify: Idempotent requests — Shopify developer documentation, n.d., accessed 2026-11. `idempotencyKey` on GraphQL Admin API mutations. *Product*

and API documentation. Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation.

- **R430** · Shopify: API rate limits — Shopify, n.d., accessed 2026-11. One-second backoff. *Product and API documentation.* Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation.
- **R431** · CloudEvents specification — CNCF, accessed 2026-11. The vendor-neutral event envelope. *Standard or specification.* Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R432** · CloudEvents (CNCF project page) — CNCF, n.d. (graduated 2024-01-25), accessed 2026-11. The vendor-neutral event envelope. *Practitioner and press.* Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R433** · Kafka design — Apache Software Foundation, Kafka 4.4 documentation, accessed 2026-11. At-least-once by default; exactly-once within Kafka; ordering per partition. *Product and API documentation.* Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation.
- **R434** · Amazon SQS dead-letter queues — AWS documentation, n.d., accessed 2026-11. `maxReceiveCount`, retention advice, redrive. *Product and API documentation.* Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour.
- **R435** · Eventually Consistent — Werner Vogels, 2008. The definition of eventual consistency and its variants; eighteen years old, definitions unchanged. *Company engineering.* Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation.
- **R436** · Transactional outbox — Chris Richardson, microservices.io, n.d., accessed 2026-11. The outbox and relay; choreography vs orchestration; compensating transactions; lack of isolation. *Practitioner and press.* Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation.
- **R437** · Sagas — Hector Garcia-Molina and Kenneth Salem, Proc. ACM SIGMOD 1987. The original definition; thirty-nine years old. *Research.* Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation.
- **R438** · Exponential backoff and jitter — AWS Architecture Blog, 2015. The simulation showing jitter halves contending calls. *Company engineering.* Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour.
- **R439** · Monitoring Distributed Systems — Google SRE book, 2017. The four golden signals; symptoms vs causes; black-box vs white-box. *Book.* Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black

Friday, Hour by Hour, Toolkit: Cards and Templates for Commerce Architecture Decisions.

- **R440** · GraphQL Admin API rate limits — Shopify, n.d., accessed 2026-11. Cost points by plan, mutation cost, query maximum. *Product and API documentation.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R441** · Versioning — Stripe documentation, n.d., accessed 2026-11. Account pinning, `Stripe-Version` header, webhook endpoint versions, current version 2026-10-28.endive. *Product and API documentation.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R442** · APIs as infrastructure: future-proofing Stripe with versioning — Brandur Leach, Stripe, 2017. Version change modules, compatibility with every version since 2011 (evidence age: nine years). *Company engineering.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R443** · Introducing Stripe's new API release process — Stripe, 2024. Monthly non-breaking releases and two major releases a year. *Company engineering.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R444** · RESTful API Guidelines — Zalando SE, changelog to 2026-03-16, accessed 2026-11. API First, compatibility rules #106 to #115, deprecation rules #185 to #191. *Company engineering.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R445** · On APIs and the Zalando API Guild — Zalando Engineering, 2015. The guild's size, time budget and review-not-approval role; "We cannot afford to break our APIs". *Company engineering.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R446** · Zalando's APIs and delivery under Radical Agility — InfoQ, 2016. API definitions as source of truth and contract. *Company engineering.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R447** · Stevey's Google Platforms Rant — Steve Yegge, 2011. The recollected Amazon service-interface mandate; widely reported, not an Amazon document. *Practitioner and press.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning, Case: Amazon, From Monolith to Services to the Everything Store.
- **R448** · RFC 9745: The Deprecation HTTP Response Header Field — IETF, Dalal and Wilde, 2025. The Deprecation header and its relation to Sunset (RFC 8594). *Standard or specification.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.

- **R449** · AIP-180: Backwards compatibility — Google, created 2019, updated 2025. What counts as a compatible and a breaking change. *Standard or specification.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R450** · Standard Webhooks specification 1.0.0 — n.d., accessed 2026-11. Headers, signed content, replay tolerance, retry guidance, steering committee and implementations. *Product and API documentation.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R451** · Standard Webhooks (project site) — n.d., accessed 2026-11. Headers, signed content, replay tolerance, retry guidance, steering committee and implementations. *Standard or specification.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R452** · OpenAPI Specification v3.2.1 — OpenAPI Initiative, 2026. The HTTP API description standard. *Standard or specification.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R453** · AsyncAPI Specification 3.0.0 — AsyncAPI Initiative, 2023. Message-driven API descriptions. *Standard or specification.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R454** · AsyncAPI 3.0.0 release notes — AsyncAPI Initiative, 2023. Message-driven API descriptions. *Standard or specification.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R455** · JSON Schema specification — JSON Schema, n.d., accessed 2026-11. Version 2020-12. *Standard or specification.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R456** · Consumer-Driven Contracts: A Service Evolution Pattern — Ian Robinson, 2006. Provider, consumer and consumer-driven contracts (evidence age: twenty years; the idea is unchanged). *Practitioner and press.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R457** · Pact documentation — Pact Foundation, updated 2026-08-25. Code-first consumer-driven contract testing. *Product and API documentation.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning.
- **R458** · customerMerge mutation — Shopify developer documentation, API version 2026-10, accessed 2026-11. The published survivorship rules, asynchronous job and preview. *Product and API documentation.* Cited in: One Customer, One Product: Identity and Master Data Across Systems.
- **R459** · customerEmailMarketingConsentUpdate mutation — Shopify developer documentation, n.d., accessed 2026-11. Consent state, opt-in level, and "when and where" consent was given or withdrawn. *Product and API documentation.* Cited in: One Customer, One Product: Identity and Master Data Across Systems.
- **R460** · What is a CDP? — CDP Institute, 2026 wording, accessed 2026-11. The definition and the "primary responsibility" for customer identity. *Practitioner and press.* Cited in: One Customer, One Product: Identity and Master Data Across Systems.

- **R461** · Identity Resolution — Twilio Segment documentation, last modified 2026-06-04. Identity graph, external ids, merge protection. *Product and API documentation.* Cited in: One Customer, One Product: Identity and Master Data Across Systems.
- **R462** · What is master data management? — Informatica, n.d., accessed 2026-11. The golden record and MDM domains. *Product and API documentation.* Cited in: One Customer, One Product: Identity and Master Data Across Systems.
- **R463** · What Is a Golden Record in MDM? — Benjamin Bourgeois, Profisee, 2025-08-26. Survivorship rule types: source priority, recency, completeness. *Product and API documentation.* Cited in: One Customer, One Product: Identity and Master Data Across Systems.
- **R464** · What is a GTIN? — GS1 US, n.d., accessed 2026-11. GTIN-8, -12, -13 and -14 and their uses. *Practitioner and press.* Cited in: One Customer, One Product: Identity and Master Data Across Systems.
- **R465** · DAMA Body of Knowledge — DAMA International, n.d., accessed 2026-11. The practitioner framework for master data governance, revised 2024. *Practitioner and press.* Cited in: One Customer, One Product: Identity and Master Data Across Systems.
- **R466** · Record linkage — Wikipedia, accessed 2026-11. The history (Dunn 1946, Newcombe 1959, Fellegi and Sunter 1969), thresholds, blocking; a secondary source, used because the 1969 paper is paywalled. *Encyclopedia.* Cited in: One Customer, One Product: Identity and Master Data Across Systems.
- **R467** · Key Concepts and Architecture — Snowflake documentation, n.d., accessed 2026-11. Separation of storage, compute and services. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R468** · What is BigQuery? — Google Cloud documentation, n.d., accessed 2026-11. Serverless warehouse with separated compute and storage. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R469** · Fivetran pricing — Fivetran, n.d., accessed 2026-11. Monthly active rows, free tier, sync frequency by plan. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R470** · Shopify source connector — Airbyte documentation, changelog to 2026-11-03. Streams, incremental sync by update time, lookback window, deleted-product stream, rate-limit backoff. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R471** · Bulk operations: queries — Shopify developer documentation, n.d., accessed 2026-11. Asynchronous bulk export. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R472** · ShopifyQL sales schema, API version 2026-10 — Shopify developer documentation, 2026. Definitions of orders, gross sales, discounts, sales reversals, net

sales and total sales. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.

- **R473** · dbt Semantic Layer — dbt Labs documentation, updated 2026-09-29. Metrics defined once and refreshed everywhere. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R474** · What is LookML? — Google Cloud documentation, n.d., accessed 2026-11. A modeling language for dimensions and aggregates. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R475** · Cube introduction — Cube documentation, n.d., accessed 2026-11. Modeling, access control, caching and APIs. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R476** · Introduction to Secure Data Sharing — Snowflake documentation, n.d., accessed 2026-11. Sharing without copying. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R477** · Share data securely across organizations — Databricks documentation, updated 2026-10-27. An open sharing protocol across platforms (product naming in flux). *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R478** · GA4 recommended events reference — Google Analytics developer documentation, n.d., accessed 2026-11. The purchase event, transaction id, value and refund. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R479** · Measure ecommerce — Google Analytics developer documentation, n.d., accessed 2026-11. The purchase event, transaction id, value and refund. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R480** · BigQuery Export schema and limits — Google Analytics Help, n.d., accessed 2026-11. Raw export differs from reports; intraday export is best-effort. *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R481** · Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics — Armbrust, Ghodsi, Xin and Zaharia, CIDR 2021. The two-tier problem and the lakehouse proposal. *Research.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R482** · The missing piece of the modern data stack — Benn Stancil, 2021. The case for a metrics layer. *Practitioner and press.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.
- **R483** · How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh — Zhamak Dehghani, 2019. Domain ownership of data as a product. *Practitioner and press.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.

- **R484** · Why is GA4 under- or over-reporting? — Elevar help center, n.d., accessed 2026-11 (vendor help page; widely reported causes of the platform-versus-analytics gap). *Product and API documentation.* Cited in: Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics.

Platform engineering, tenancy, extensibility, storefront and peak

- **R485** · How we prepare Shopify for BFCM — Shopify Engineering, 2025. Load tests at 150% of the previous peak, 200 million requests per minute, 80,000+ checkouts per minute. *Company engineering.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R486** · From Event-Driven to API: Rethinking Price and Stock Updates — Zalando Engineering, 2025. Event delays of up to 30 minutes during Cyber Week and the move to a synchronous read API. *Company engineering.* Cited in: Introduction: Why Commerce Architecture Matters, Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R487** · Sunrise: Zalando's developer platform based on Backstage — Zalando Engineering, 2023. "more than 100 disconnected interfaces" replaced. *Company engineering.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R488** · Goodbye Microservices: From 100s of Problem Children to 1 Superstar — Segment (Twilio), 2018. 140+ services consolidated into one (dated). Also cited at: Goodbye Microservices: From 100s of Problem Children to 1 Superstar. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, What Changes at Each Scale: From First Store to Group.
- **R489** · Prime Video Switches from Serverless to EC2 and ECS to Reduce Costs by 90% — InfoQ, 2023. 90% cost reduction; the original post has been removed. *Company engineering.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: Amazon, From Monolith to Services to the Everything Store.

- **R490** · Adobe Commerce lifecycle policy — Adobe Experience League, n.d., accessed 2026-11. Three years of standard support; cloud enforcement from 2027-06-01. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R491** · Adobe Commerce as a Cloud Service overview — Adobe Experience League, 2026. Managed SaaS with continuous updates. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Delivering the Storefront: Themes, Headless, Rendering and Performance, Case: commercetools, Shopware and the Composable Movement.
- **R492** · API Extensions — commercetools documentation, n.d., accessed 2026-11. Synchronous call-outs and the two-second default timeout. *Product and API documentation.* Cited in: From Cart to Confirmation: Checkout Orchestration, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
- **R493** · Cloud, Load, and Modular Code: What 2022 Looks Like for Shopify — Shopify Engineering, 2022. 13 billion webhooks. *Company engineering.* Cited in: The Order Record: Lifecycle, State and the Order Management System, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Delivering the Storefront: Themes, Headless, Rendering and Performance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R494** · App requirements checklist — shopify.dev, n.d., accessed 2026-11. Apps using APIs deprecated within 90 days cannot be submitted. *Product and API documentation.* Cited in: Promises Between Systems: API Contracts, Webhooks and Versioning, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Delivering the Storefront: Themes, Headless, Rendering and Performance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R495** · Deconstructing the Monolith — Shopify Engineering, 2019. The componentization decision, the reasons for rejecting microservices, Wedge and the tax-engine payoff. *Company engineering.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, What Changes at Each Scale: From First Store to Group, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R496** · Under Deconstruction: The State of Shopify's Monolith — Shopify Engineering, 2020. 2.8 million lines, 37 components, circular dependencies, services split "only for good reasons". *Company engineering.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R497** · Enforcing Modularity in Rails Apps with Packwerk — Shopify Engineering, 2020. The violation list that blocks new boundary breaks. *Company engineering.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the

Evidence Says, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.

- **R498** · A Packwerk Retrospective — Shopify Engineering, 2024. What worked and what did not after four years; the quoted lessons. *Company engineering.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R499** · How Shopify Reduced Storefront Response Times with a Rewrite — Shopify Engineering, 2020. The one hot path carved out for latency, with verifier and fallback. *Company engineering.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Delivering the Storefront: Themes, Headless, Rendering and Performance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R500** · From Jimmy to Microservices: Rebuilding Zalando's Fashion Store — Zalando Engineering, 2015. The monolith's name and the motive for the split. *Company engineering.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Moving Without Burning the Bridge: Replatforming and Migration, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R501** · Zalando Engineering Principles (deprecated repository) — Zalando, n.d., accessed 2026-11. The 2015 principles and the 2020 deprecation note. *Product and API documentation.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says.
- **R502** · Technology Choices at Zalando: Tech Radar Update — Zalando Engineering, 2020. The narrowed radar and the quoted reason. *Company engineering.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R503** · Amazon Architecture (digest of "A Conversation with Werner Vogels", ACM Queue 2006) — High Scalability, 2007. The 2001 scaling wall, hundreds of services, 100–150 services per page; the primary interview text was not fetchable. *Practitioner and press.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: Amazon, From Monolith to Services to the Everything Store.
- **R504** · ACM Queue Interview with Werner Vogels — AWS News Blog, 2006. "you build it, you run it". *Company engineering.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R505** · Two-Pizza Teams — AWS whitepaper, n.d., accessed 2026-11. The team-size rule and its ownership rationale. *Product and API documentation.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: Amazon, From Monolith to Services to the Everything Store.
- **R506** · Dynamo: Amazon's Highly Available Key-value Store — DeCandia et al., 2007. The shopping cart's daily checkout volume. *Research.* Cited in: Monolith,

Modular Monolith, Microservices, Composable: What the Evidence Says, Case: Amazon, From Monolith to Services to the Everything Store.

- **R507** · Completing the Netflix Cloud Migration — Netflix, 2016. The 2008–2016 timeline and "hundreds of microservices". *Company engineering.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says.
- **R508** · MACH Alliance Surpasses 100 Members — MACH Alliance, 2023. The 2023 restatement. *Company filing, report or announcement.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: commercetools, Shopware and the Composable Movement.
- **R509** · MACH Alliance Advances End-User Leadership Model — MACH Alliance via Silicon UK, 2025. Five principles without the acronym. *Company filing, report or announcement.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Case: commercetools, Shopware and the Composable Movement.
- **R510** · commercetools General Concepts — commercetools, n.d., accessed 2026-11. The Project as the unit and its limits. *Product and API documentation.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R511** · Medusa Architecture Overview — Medusa, n.d., accessed 2026-11. A layered single application with modules. *Product and API documentation.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says.
- **R512** · How Do Committees Invent? — Melvin Conway, 1968. The quoted thesis. *Research.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R513** · Microservices — James Lewis and Martin Fowler, 2014. The definition, decentralized data, complexity moving into the connections. *Practitioner and press.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says.
- **R514** · MonolithFirst — Martin Fowler, 2015. Boundaries are hard to get right at the start. *Practitioner and press.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, What Changes at Each Scale: From First Store to Group.
- **R515** · Microservice Trade-Offs — Martin Fowler, 2015. The three costs quoted in the citing chapter. *Practitioner and press.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says.
- **R516** · Team Topologies: Key Concepts — Matthew Skelton and Manuel Pais, n.d., accessed 2026-11. The team types and the complexity premise. *Practitioner and press.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What

the Evidence Says, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.

- **R517** · A Pods Architecture to Allow Shopify to Scale — Shopify Engineering, 2018. Sharding in 2015, pods in 2016, the router and the Pod Mover; eight years old, pre-cloud. *Company engineering.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R518** · MySQL Database Shard Balancing at Terabyte Scale — Shopify Engineering, 2021. The shop as sharding unit, the routing table, Ghostferry moves, the four-times imbalance and the two flash-sale merchants. *Company engineering.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R519** · API limits — Shopify, n.d., accessed 2026-11. The cost model, plan tiers, per-app-per-store scope, the one-second backoff and the stability clause. *Product and API documentation.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation.
- **R520** · REST Admin API rate limits — Shopify, n.d., accessed 2026-11. The cost model, plan tiers, per-app-per-store scope, the one-second backoff and the stability clause. *Product and API documentation.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R521** · Storefront API reference — shopify.dev, n.d., accessed 2026-11. Buyer traffic without a fixed limit, bot limits, the per-minute checkout throttle. *Product and API documentation.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Delivering the Storefront: Themes, Headless, Rendering and Performance, Black Friday Every Year: Peak Load, Resilience and Recovery, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R522** · Faster checkout for customers during flash sales — Shopify Changelog, 2018. Payment processing separated from order creation under load; eight years old. *Product and API documentation.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R523** · Shopify Markets — Shopify Help Center, n.d., accessed 2026-11. One shop with several markets; the merchant-of-record variant and its geography. *Product and API documentation.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation.
- **R524** · Managed Markets — Shopify Help Center, n.d., accessed 2026-11. One shop with several markets; the merchant-of-record variant and its geography. *Product and API documentation.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation.
- **R525** · Platform Multitenant Architecture — Salesforce Architects, n.d., accessed 2026-11. The metadata-driven pool, the OrgID and governor limits. *Product and*

API documentation. Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation.

- **R526** · API rate limits — BigCommerce, n.d., accessed 2026-11. Request quotas by plan and the response headers. *Product and API documentation.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation.
- **R527** · Extensions concept — Shopware, n.d., accessed 2026-11. Plugins with database access unsupported on the cloud edition; apps over APIs. *Product and API documentation.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
- **R528** · Silo, pool, and bridge models — AWS Well-Architected SaaS Lens, n.d., accessed 2026-11. The tenancy models and the noisy-neighbor definition. *Product and API documentation.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation.
- **R529** · Noisy neighbor — AWS Well-Architected SaaS Lens, n.d., accessed 2026-11. The tenancy models and the noisy-neighbor definition. *Product and API documentation.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation.
- **R530** · Ghostferry — Shopify, n.d., accessed 2026-11. The open-source tool behind shop moves, with its own caveat that the model is not a proof of the implementation. *Product and API documentation.* Cited in: Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R531** · Shopify Unite 2021 announcements — Shopify Partners blog, 29 June 2021. Theme app extensions and app blocks, uninstalls removing ghost code, Functions in beta. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Delivering the Storefront: Themes, Headless, Rendering and Performance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R532** · Access scopes — shopify.dev, n.d., accessed 2026-11. Scope examples, the 60-day order window, protected customer data. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R533** · Revenue share — shopify.dev, n.d., accessed 2026-11. The 0% to $1,000,000 lifetime threshold and 15% above. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R534** · Privacy law compliance — shopify.dev, n.d., accessed 2026-11. The three mandatory compliance webhooks and the 30-day completion rule. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Whose Data Is It? Privacy, Consent and Retention, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.

- **R535** · Shopify Flow — Shopify Help Center, n.d., accessed 2026-11. The workflow tool and its plan gating. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
- **R536** · Web performance — Shopify Help Center, n.d., accessed 2026-11. Theme code and apps named as what can slow a site. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R537** · App marketplace requirements — BigCommerce developer documentation, n.d., accessed 2026-11. "Apps configured with all scopes aren't approved", checkout and PCI reviews, clean uninstall. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
- **R538** · Plugins (interceptors) — Adobe Commerce PHP developer guide, n.d., accessed 2026-11. Before, after and around plugins, ordering and the performance warning. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
- **R539** · Out-of-process extensibility — Adobe Commerce developer documentation, n.d., accessed 2026-11. The in-process versus App Builder trade-off in Adobe's words. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
- **R540** · Upgrade best practices — Adobe Experience League, n.d., accessed 2026-11. Custom work and modules as the drivers of upgrade complexity. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
- **R541** · Upgrade Compatibility Tool overview — Adobe Experience League, updated 8 October 2026. What the tool checks and reports. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
- **R542** · Extension concept — SAP Help Portal (SAP Commerce 2205), n.d., accessed 2026-11. "a flexible, modular architecture based on extensions". *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
- **R543** · Customize with cartridges — Salesforce Trailhead, n.d., accessed 2026-11. The cartridge path and the rule never to edit the base storefront. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
- **R544** · Apps overview — Saleor documentation, n.d., accessed 2026-11. Apps as external services over webhooks and the API. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
- **R545** · Getting started building extensions — WooCommerce developer documentation, n.d., accessed 2026-11. "backwards compatibility between WooCommerce releases is not guaranteed"; the 3.6M+ active stores reach figure. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.

- **R546** · Hooks — WordPress Plugin Handbook, n.d., accessed 2026-11. Actions and filters as the in-process mechanism under WooCommerce. No cross-vendor standard governs commerce extensibility; the privacy obligations that reach apps are covered in Whose Data Is It? Privacy, Consent and Retention. *Product and API documentation.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance.
- **R547** · Feature Toggles (aka Feature Flags) — Pete Hodgson, martinfowler.com, 2017. The toggle taxonomy used to decide whether a change belongs in configuration. *Practitioner and press.* Cited in: Apps, Plugins and Functions: Extensibility and Ecosystem Governance, Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance, Toolkit: Cards and Templates for Commerce Architecture Decisions.
- **R548** · Developer highlights from our Winter '23 Edition — Shopify Partners blog, 9 February 2023. Oxygen hosting on the Basic, Shopify, Advanced and Plus plans at no extra cost. *Product and API documentation.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R549** · Hydrogen — shopify.dev, n.d., accessed 2026-11. Hydrogen as a toolkit, the APIs it covers, no checkout UI, the React Router 7 starter and Oxygen. *Product and API documentation.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R550** · Theme performance best practices — shopify.dev, n.d., accessed 2026-11. Core Web Vitals as a ranking signal, the Lighthouse 60 minimum, app scripts. *Product and API documentation.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R551** · Stencil — BigCommerce developer documentation, n.d., accessed 2026-11. The Handlebars-based theme engine. *Product and API documentation.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R552** · PWA Kit — Salesforce Commerce Cloud, GitHub, n.d., accessed 2026-11. The React storefront on a Managed Runtime. *Product and API documentation.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R553** · Adobe Commerce Storefront — Adobe Experience League, n.d., accessed 2026-11. Drop-in components and document-based authoring. *Product and API documentation.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R554** · Next.js Commerce — Vercel, GitHub, n.d., accessed 2026-11. The server-rendered template, its platform forks, and its archiving. *Product and API documentation.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.

- **R555** · Incremental Static Regeneration — Vercel documentation, updated 28 August 2026. Stale-while-revalidate, the e-commerce use case, request collapsing and failure behavior. *Product and API documentation.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R556** · Unified Data Layer — Alokai documentation, n.d., accessed 2026-11. Normalizing product data across platforms. *Product and API documentation.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R557** · CMRC Q2 earnings call flags softer B2C demand, outlook cut — Nasdaq, 2026. *Practitioner and press.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R558** · Web Vitals — web.dev (Google), updated 31 October 2024. The three metrics, their thresholds and the 75th-percentile rule. *Practitioner and press.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R559** · Interaction to Next Paint becomes a Core Web Vital on March 12 — web.dev, 2024. INP replacing first input delay. *Practitioner and press.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R560** · Understanding Core Web Vitals and Google search results — Google Search Central, n.d., accessed 2026-11. "highly recommend" and alignment with ranking systems, without a weight. *Product and API documentation.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R561** · What are Progressive Web Apps? — web.dev, n.d., accessed 2026-11. The PWA definition. *Practitioner and press.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R562** · Web Content Accessibility Guidelines (WCAG) 2.2 — W3C Recommendation, 12 December 2024. Levels, principles and the current edition. *Standard or specification.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R563** · Milliseconds Make Millions — Deloitte Ireland for Google, 24 March 2020. The 0.1-second study and its caveats; six years old. *Practitioner and press.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R564** · The business impact of Core Web Vitals — web.dev, n.d., accessed 2026-11. The self-reported case studies quoted as illustrations. *Practitioner and press.* Cited in: Delivering the Storefront: Themes, Headless, Rendering and Performance.
- **R565** · Scale and performance testing at Shopify — Shopify Engineering, 2023. Full-scale testing with CDN bypass and cache-busting, and the abort switch. *Company engineering.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R566** · End-to-end load testing Zalando's production website — Zalando Engineering, 2019. Recorded journeys replayed at 130,000 requests per second; situation rooms; what broke on replay (seven years old). *Company engineering.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour.

- **R567** · How Zalando prepares for Cyber Week — Zalando Engineering, 2020. 7,200 orders per minute in 2019, production-readiness reviews, test orders that fulfillment skips, pausing non-essential processing, the Situation Room (six years old). *Company engineering.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R568** · The magic behind the world's largest online shopping festival — Alibaba Cloud, 2020. 583,000 orders per second at the 2020 peak, "orders" undefined. *Company filing, report or announcement.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery.
- **R569** · Adobe: Cyber Monday 2024 recap — Adobe, 2024. $13.3 billion and $15.8 million per minute in the peak hours. *Company filing, report or announcement.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery.
- **R570** · Amazon internal documents: what caused the Prime Day crash — CNBC, 2018. The fallback front page and the international traffic cut (a press account, eight years old). *Practitioner and press.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery.
- **R571** · Cloudflare Waiting Room — Cloudflare, 2021. What a waiting room is and how it is configured. *Company engineering.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour.
- **R572** · Waiting Room configuration settings — Cloudflare Docs, n.d., accessed 2026-11. What a waiting room is and how it is configured. *Product and API documentation.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery.
- **R573** · How does Queue-it work — Queue-it, n.d., accessed 2026-11. Threshold-triggered virtual queues with adjustable outflow. *Product and API documentation.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery.
- **R574** · Using load shedding to avoid overload — AWS Builders' Library, 2019. Overload, goodput and prioritizing requests. *Company engineering.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 4: Black Friday, Hour by Hour.
- **R575** · Service Level Objectives — Google SRE book, 2017. SLI, SLO, error budget and the four golden signals. *Book.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance, Toolkit: Cards and Templates for Commerce Architecture Decisions.

Security, privacy and regulation

- **R576** · Polyfill supply chain attack hits 100K+ sites — Sansec, 2024-06-25. The script domain sold in February 2024. *Practitioner and press.* Cited in: Out of Stock, Out

of Sync: Sixteen Ways Commerce Platforms Go Wrong, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.

- **R577** · Regulation (EU) 2016/679 (GDPR) — Official Journal text, 2016. Article 12(3), one month to answer a data-subject request. *Regulation and official source.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, One Customer, One Product: Identity and Master Data Across Systems, Whose Data Is It? Privacy, Consent and Retention.
- **R578** · Directive (EU) 2019/882 (European Accessibility Act) — Official Journal, 2019-06-07. Application from 2025-06-28. *Regulation and official source.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Delivering the Storefront: Themes, Headless, Rendering and Performance, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R579** · FAQ E-Rechnung — German Federal Ministry of Finance, accessed 2026-11. Receiving since 2025-01-01, issuing transition to end 2027. *Regulation and official source.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Tax, Invoices and the Books: From Order to Ledger, Documents Between Companies: EDI, Punchout and E-Invoicing, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R580** · Guidance for PCI DSS e-commerce requirements effective after 31 March 2025 — PCI Security Standards Council, 2025-03-26. Mandatory from 2025-03-31. *Standard or specification.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R581** · FTC announces final rule banning fake reviews and testimonials — FTC, 2024-08-14. The rule's prohibitions. *Regulation and official source.* Cited in: Finding the Product: Search, Navigation and Merchandising, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R582** · Directive (EU) 2015/2366 (PSD2) — Official Journal, 2015-12-23. The SCA definition (Article 4(30)) and requirement (Article 97); applied from 13 January 2018. *Regulation and official source.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R583** · Regulation (EU) 2024/886 on instant credit transfers — Official Journal, 2024-03-19. Receive and send deadlines, free payee verification. *Regulation and official source.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R584** · Payment services deal: more protection from online fraud and hidden fees — European Parliament, 2025-11-21. PSD3 and PSR content and status. *Regulation and official source.* Cited in: Taking the Money: Payment Providers,

Authorization, Capture and Refunds, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.

- **R585** · State of play of legislative files — Council of the EU, 2026-10-29. PSD3 and PSR content and status. *Regulation and official source.* Cited in: Taking the Money: Payment Providers, Authorization, Capture and Refunds, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R586** · §147 Abgabenordnung — gesetze-im-internet.de, current text, accessed 2026-11. German retention of books and records. *Regulation and official source.* Cited in: Tax, Invoices and the Books: From Order to Ledger, Whose Data Is It? Privacy, Consent and Retention.
- **R587** · South Dakota v. Wayfair, Inc. — US Supreme Court via Cornell LII, 2018. Economic nexus. *Regulation and official source.* Cited in: Tax, Invoices and the Books: From Order to Ledger, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R588** · Kabinet kiest voor invoering e-facturatie en rapportage voor bedrijven — Rijksoverheid, 2026. The Dutch plan for 1 July 2030. *Regulation and official source.* Cited in: Tax, Invoices and the Books: From Order to Ledger, Documents Between Companies: EDI, Punchout and E-Invoicing, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R589** · Regulation (EU) 2022/2065 (Digital Services Act) — Publications Office of the EU, 2022. Article 30 trader traceability; application from 17 February 2024. *Regulation and official source.* Cited in: Selling on Someone Else's Platform: Marketplaces and Channel Integration, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R590** · Period of tolerance during the first three months of 2026 — FPS Finance (Belgium), 2025-12-18. The Belgian mandate, scope, format, fines and tolerance. *Regulation and official source.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R591** · e-invoicing FAQ: specific questions about e-invoicing — n.d., accessed 2026-11. The Belgian mandate, scope, format, fines and tolerance. *Regulation and official source.* Cited in: Documents Between Companies: EDI, Punchout and E-Invoicing, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R592** · Best practices for managing secret API keys — Stripe Docs, n.d., accessed 2026-11. Restricted keys, vaults, rotation, exposure handling. *Product and API documentation.* Cited in: Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R593** · Script security for ecommerce — Adyen Docs, n.d., accessed 2026-11. CSP, script inventory, SRI and violation reports as the 11.6.1 alert source. *Product and*

API documentation. Cited in: Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.

- **R594** · Security — Shopify, n.d., accessed 2026-11. Level 1 certification extending to all stores, and the merchant's own steps. *Product and API documentation.* Cited in: Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R595** · PCI compliance — Shopify, n.d., accessed 2026-11. Level 1 certification extending to all stores, and the merchant's own steps. *Product and API documentation.* Cited in: Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R596** · Two-step authentication — Shopify Help Center, n.d., accessed 2026-11. Who can enforce MFA, SMS retirement, payouts on hold. *Product and API documentation.* Cited in: Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R597** · Shared responsibility — Adobe Experience League, n.d., accessed 2026-11. The merchant's and Adobe's duties on Adobe Commerce cloud infrastructure. *Product and API documentation.* Cited in: Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R598** · CosmicSting — Sansec, 2024, accessed 2026-11. The Adobe Commerce flaw, its fix dates and the breach campaigns. *Practitioner and press.* Cited in: Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R599** · 2025 Bad Bot Report — Imperva, 2025. Vendor figures on automated traffic, quoted as such. *Practitioner and press.* Cited in: Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R600** · OAT-001 Carding — OWASP Automated Threats to Web Applications, n.d., accessed 2026-11. The threat identifiers used in the citing chapter. *Standard or specification.* Cited in: Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R601** · Credential stuffing — OWASP, n.d., accessed 2026-11. Definition and MFA as countermeasure. *Standard or specification.* Cited in: Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R602** · British Airways data breach — Wikipedia, citing the ICO penalty notice of 16 October 2020, accessed 2026-11. The entry point, the script, the numbers and the penalty. Eight years after the incident; the ICO's own pages could not be fetched. *Encyclopedia.* Cited in: Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers.
- **R603** · Protected customer data — Shopify developer documentation, n.d., accessed 2026-11. Minimum data, purpose limitation, consent, retention, encryption and level-2 controls for apps. *Product and API documentation.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R604** · Attorney General Bonta announces settlement with Sephora — California Attorney General, 2022-08-24. Trackers as a sale, Global Privacy Control, the US$1.2 million settlement. *Regulation and official source.* Cited in: Whose Data Is

It? Privacy, Consent and Retention, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.

- **R605** · Latombe's challenge to EU-US data transfers fails — Lewis Silkin, 2025-09-03. The General Court judgment and the advice to keep SCCs as a fallback. *Practitioner and press.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R606** · Digital Omnibus reshapes EU cookie rules, leaves banner fatigue largely intact — Osborne Clarke, 2025-12-10. A reading of the November 2025 proposal. *Practitioner and press.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R607** · Directive 2009/136/EC — Official Journal text, 2009. The amended Article 5(3) device rule of the ePrivacy Directive. *Regulation and official source.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R608** · Directive 2002/58/EC (ePrivacy), consolidated 2009 — EUR-Lex, consolidated text of 19 December 2009. Article 13(2), the soft opt-in; text more than three years old, national transpositions differ. *Regulation and official source.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R609** · Guidelines 07/2020 on the concepts of controller and processor — European Data Protection Board, version 2.0, 2021-07-07. Purposes, essential and non-essential means. *Regulation and official source.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R610** · Guidelines 05/2020 on consent — European Data Protection Board, 2020-05-04. Cookie walls, scrolling. *Regulation and official source.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R611** · Guidelines 2/2023 on the technical scope of Art. 5(3) of the ePrivacy Directive — European Data Protection Board, version 2.0, 2024-10-07. Pixels, URL tracking, IP-only tracking. *Regulation and official source.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R612** · Cookies — Autoriteit Persoonsgegevens (Dutch), n.d., accessed 2026-11. Functional and limited analytical cookies, tracking cookies, cookie walls. *Regulation and official source.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R613** · Cookies en andere traceringsmiddelen — Gegevensbeschermingsautoriteit (Belgium, Dutch), 2023-11-17. Exempt categories, pre-ticked boxes, purpose grouping. *Regulation and official source.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R614** · Keeping business records — business.gov.nl (Netherlands Enterprise Agency), n.d., accessed 2026-11. At least seven years; ten for immovable property; digital files accessible. *Regulation and official source.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R615** · Commission Implementing Decision (EU) 2023/1795 (EU-US Data Privacy Framework) — Official Journal text, 2023. The adequacy decision. *Regulation and official source.* Cited in: Whose Data Is It? Privacy, Consent and Retention.

- **R616** · Commission Implementing Decision (EU) 2021/914 (standard contractual clauses) — Official Journal text, 2021. *Regulation and official source.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R617** · Case C-703/25 P, Latombe v Commission — Official Journal C/2025/6610, 2025. The pending appeal. *Court or competition decision.* Cited in: Whose Data Is It? Privacy, Consent and Retention, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R618** · California Consumer Privacy Act — California Department of Justice, n.d., accessed 2026-11. Thresholds, rights, Global Privacy Control. *Regulation and official source.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R619** · US State Privacy Legislation Tracker — IAPP, updated 2026-09-08. Nineteen state laws. *Practitioner and press.* Cited in: Whose Data Is It? Privacy, Consent and Retention.
- **R620** · US State Privacy Legislation Tracker chart — IAPP, 2025. Nineteen state laws. *Practitioner and press.* Cited in: Whose Data Is It? Privacy, Consent and Retention, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R621** · 2025 Year-End Digital Accessibility Lawsuit Report — UsableNet, 2025-12. US lawsuit counts, quoted as a vendor figure. *Practitioner and press.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R622** · Council Directive (EU) 2025/516 (ViDA) — Publications Office of the EU, 2025. Application from 1 July 2030. *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R623** · Regulation (EU) 2023/988 (General Product Safety) — European Union, 2023. Art. 4, 19 and 22(9). *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Toolkit: Cards and Templates for Commerce Architecture Decisions.
- **R624** · Regulation (EU) 2024/1781 (Ecodesign for Sustainable Products) — Publications Office of the EU, 2024. Art. 13, 24 and 25 and Annex VII (text verified 2026-11). *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R625** · Regulation (EU) 2025/40 (Packaging and Packaging Waste) — Publications Office of the EU, 2025. Application date, Art. 12 and 45. *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R626** · LUCID Packaging Register — Zentrale Stelle Verpackungsregister, n.d., accessed 2026-11. *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R627** · Directive (EU) 2022/2555 (NIS2) — Publications Office of the EU, 2022. Scope and reporting clocks. *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.

- **R628** · Cyberbeveiligingswet in force 15 August 2026 — Rijksoverheid, 2026-07-07. *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R629** · Regulation (EU) 2024/2847 (Cyber Resilience Act) — Publications Office of the EU, 2024. *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R630** · Regulation (EU) 2024/1689 (AI Act) — Publications Office of the EU, 2024 and 2026. Art. 50 and the revised dates. *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R631** · Regulation (EU) 2026/1744 (Digital Omnibus on AI) — Official Journal of the EU, 2026. Art. 50 and the revised dates. *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R632** · Informing Businesses about the INFORM Consumers Act — FTC, n.d., accessed 2026-11. Seller thresholds and penalties. *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R633** · ADA Title II web rule — US Department of Justice, 2024, updated 2026. Scope limited to governments. *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R634** · FTC Relaunches Negative Option Rulemaking — Kelley Drye via JD Supra, 2026-03-13. The vacated click-to-cancel rule. *Practitioner and press.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation.
- **R635** · Structured electronic invoices between companies are compulsory from 2026 — FPS Finance Belgium, 2024. The mandate and the fines. *Regulation and official source.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Tax, Invoices and the Books: From Order to Ledger, Documents Between Companies: EDI, Punchout and E-Invoicing, Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Journey 3: A Chain Reorders by EDI, From Order to Invoice.

Journeys

- **R637** · Stripe: Idempotent requests — Stripe documentation, n.d., accessed 2026-11. Idempotency keys and saved responses on retry. *Product and API documentation.* Cited in: Follow the Order: What Happens Between “Buy” and “Delivered”, From Cart to Confirmation: Checkout Orchestration, Taking the Money: Payment Providers, Authorization, Capture and Refunds, Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Journey 1: Eva’s Jacket, From

Search to Doorstep, Journey 2: The Jacket Comes Back, Journey 3: A Chain Reorders by EDI, From Order to Invoice.

- **R638** · Customer invoices — Odoo, version 18.0 documentation (2024–2025), accessed 2026-11. Sequential numbering on confirmation, immutability, credit notes as the only correction. *Product and API documentation.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered", Tax, Invoices and the Books: From Order to Ledger, Journey 1: Eva's Jacket, From Search to Doorstep, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R639** · Credit notes and refunds — Odoo, version 18.0 documentation (2024–2025), accessed 2026-11. Sequential numbering on confirmation, immutability, credit notes as the only correction. *Product and API documentation.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered", Tax, Invoices and the Books: From Order to Ledger, Journey 1: Eva's Jacket, From Search to Doorstep, Journey 2: The Jacket Comes Back, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R640** · IFRS 15 Revenue from Contracts with Customers — IFRS Foundation, issued 2014, effective 2018. Revenue recognized when the performance obligation is satisfied. *Standard or specification.* Cited in: Follow the Order: What Happens Between "Buy" and "Delivered", Discounts, Coupons and Loyalty: Promotions Without Chaos, Tax, Invoices and the Books: From Order to Ledger, Journey 1: Eva's Jacket, From Search to Doorstep, Journey 2: The Jacket Comes Back, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R641** · Invoice requirements — Belastingdienst, n.d., accessed 2026-11. Dutch invoice content. *Regulation and official source.* Cited in: Tax, Invoices and the Books: From Order to Ledger, Journey 1: Eva's Jacket, From Search to Doorstep, Journey 3: A Chain Reorders by EDI, From Order to Invoice.
- **R642** · Starbucks Does Not Use Two-Phase Commit — Gregor Hohpe, 2004. Asynchronous processing, correlation, write-off, retry and compensation; twenty-two years old, still the clearest analogy. *Book.* Cited in: Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation, Black Friday Every Year: Peak Load, Resilience and Recovery, Journey 1: Eva's Jacket, From Search to Doorstep, Journey 3: A Chain Reorders by EDI, From Order to Invoice.

Evolution, organization and AI

- **R643** · Target's website crashes again — MPR News (AP), 2011-10-25. The Missoni crash of 2011-09-13 and the second outage (dated). *Practitioner and press.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Build, Buy or Assemble: Choosing and Changing Platforms, Moving Without Burning the Bridge: Replatforming and Migration.
- **R644** · Finish Line pulls a U-turn online after losing $3 million — Retail Dive, 2013-01-08. The 17-day reversal (dated). *Practitioner and press.* Cited in: Out

of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Moving Without Burning the Bridge: Replatforming and Migration.

- **R645** · Visa press release on agentic transactions in Europe — Visa, 2026. 31 issuers and 4 merchants live. *Company filing, report or announcement.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R646** · Agentic commerce: new AI tools and the Universal Commerce Protocol — Google, 2026. Launch of the agent checkout protocol. *Practitioner and press.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R647** · Agentic Commerce Protocol — OpenAI and Stripe, 2025–2026, specification releases through 2026-04-17. Beta status, governance, release dates. *Product and API documentation.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R648** · Shopify Plus plan — Shopify Help Center, n.d., accessed 2026-11. Plus base price and the variable platform fee. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms, What Changes at Each Scale: From First Store to Group.
- **R649** · BigCommerce pricing — Commerce.com, n.d., accessed 2026-11. GMV-threshold plans, plan GMV definition and payment-provider fees. *Product and API documentation.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms, What Changes at Each Scale: From First Store to Group.
- **R650** · One decade in Zalando tech — Henning Jacobs, 2020. The 2010–2020 sequence, team and application counts, "six languages", the costs. *Company engineering.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, What Changes at Each Scale: From First Store to Group, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R651** · Introducing Domain-Oriented Microservice Architecture — Uber Engineering, 2020. 2,200 services, "networked monoliths", the domain and gateway remedy. *Practitioner and press.* Cited in: Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, What Changes at Each Scale: From First Store to Group, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R652** · Regulation (EU) 2023/2854 (Data Act) — EU Publications Office, 2023. Switching rights and dates. *Regulation and official source.* Cited in: Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation, Build, Buy or Assemble: Choosing and Changing Platforms, Moving Without Burning the Bridge: Replatforming and Migration, What Changes at Each Scale:

From First Store to Group, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.

- **R653** · Ecommerce Replatforming and Migration Guide for 2026 — Shopify Enterprise blog, 2026-03-18. The vendor's list of migration, ongoing and hidden costs (vendor content). *Product and API documentation.* Cited in: Build, Buy or Assemble: Choosing and Changing Platforms, Moving Without Burning the Bridge: Replatforming and Migration.
- **R654** · Shopify for Enterprise — Shopify, n.d., accessed 2026-11. The "33% lower TCO" and "20% faster migrations" claims and the $50M–$500M GMV survey sample. *Product and API documentation.* Cited in: Build, Buy or Assemble: Choosing and Changing Platforms, What Changes at Each Scale: From First Store to Group.
- **R655** · Shopify Terms of Service — Shopify, 2026. "as is", no warranty of uninterrupted service, modification without notice. *Product and API documentation.* Cited in: Build, Buy or Assemble: Choosing and Changing Platforms, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R656** · Target parts ways with Amazon.com — InternetNews, 2009-08-07. The build decision and the 2011 outages (evidence older than three years). *Practitioner and press.* Cited in: Build, Buy or Assemble: Choosing and Changing Platforms, Moving Without Burning the Bridge: Replatforming and Migration.
- **R657** · Zalando consolidates partnership models, Connected Retail to be discontinued — FashionUnited, 2026-04-23. The 30 June 2027 discontinuation with notice. *Practitioner and press.* Cited in: Build, Buy or Assemble: Choosing and Changing Platforms, Moving Without Burning the Bridge: Replatforming and Migration, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R658** · Magento 1 reaches EOL — Help Net Security, 2020. The forced migration (widely reported). *Practitioner and press.* Cited in: Build, Buy or Assemble: Choosing and Changing Platforms, Moving Without Burning the Bridge: Replatforming and Migration.
- **R659** · Documenting Architecture Decisions — Michael Nygard, 2011. The ADR format used for W4. *Practitioner and press.* Cited in: Build, Buy or Assemble: Choosing and Changing Platforms, Who Builds and Who Owns: Teams, Vendors, Agencies and Governance, Toolkit: Cards and Templates for Commerce Architecture Decisions.
- **R660** · Marks & Spencer Q1: general merchandise sales down — CNBC (Reuters), 2014-07-08. The 8.1% fall in M&S.com sales used as the revenue-at-risk assumption (evidence older than three years). *Practitioner and press.* Cited in: Build, Buy or Assemble: Choosing and Changing Platforms, Moving Without Burning the Bridge: Replatforming and Migration.
- **R661** · Target's website crashes under demand for Italian knitwear — MPR News (AP), 2011-09-13. The Missoni outage, Target's "extremely popular" notice, and the second outage (evidence older than three years). *Practitioner and press.* Cited in: Moving Without Burning the Bridge: Replatforming and Migration.

- **R662** · Comment: M&S fall in online sales highlights the risks of replatforming — Retail Week, 2014-07-10. The trade commentary (paywalled beyond the lead). *Practitioner and press.* Cited in: Moving Without Burning the Bridge: Replatforming and Migration.
- **R663** · Jimmy to microservices: the journey one year later — Zalando Engineering, 2016-10-05. The router-fronted, fragment-by-fragment migration (evidence older than three years; the system has changed since). *Company engineering.* Cited in: Moving Without Burning the Bridge: Replatforming and Migration, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R664** · Strangler Fig Application — Martin Fowler, 2024 (restating a 2004 note). The gradual approach and why wholesale replacement plans fail. *Practitioner and press.* Cited in: Moving Without Burning the Bridge: Replatforming and Migration, Toolkit: Cards and Templates for Commerce Architecture Decisions.
- **R665** · Patterns of Legacy Displacement — Ian Cartwright, Rob Horn and James Lewis, 2021, revised 2024. Transitional architecture, event interception, divert the flow, legacy mimic, dark launching, canary release, parallel run, and the warning about feature parity and big-bang releases. *Practitioner and press.* Cited in: Moving Without Burning the Bridge: Replatforming and Migration, Toolkit: Cards and Templates for Commerce Architecture Decisions.
- **R666** · IT Spending and Staffing Benchmarks 2025/2026, Chapter 8: Retail — Avasant, 2025. The retail IT-spend benchmark exists but is paywalled; the 2–4% range in the text is judgement. *Practitioner and press.* Cited in: What Changes at Each Scale: From First Store to Group.
- **R667** · How to earn as a Shopify Partner — Shopify Help Center, n.d., accessed 2026-11. Partner revenue share and referral payouts. *Product and API documentation.* Cited in: Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R668** · Shopify Plus Partner Program — Shopify Help Center, n.d., accessed 2026-11. Partner tiers by attributed GMV and shops; older tiers retiring 1 January 2027. *Product and API documentation.* Cited in: Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R669** · Development stores — Shopify developer documentation, n.d., accessed 2026-11. Test stores cannot process real transactions; cap of 250. *Product and API documentation.* Cited in: Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R670** · Shopify CEO: prove AI can't do the job before asking for headcount — CNBC, 2025. The April 2025 memo. *Practitioner and press.* Cited in: Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R671** · Adobe Commerce Cloud Pro architecture — Adobe Experience League, n.d., accessed 2026-11. Integration, staging and production environments. *Product and API documentation.* Cited in: Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R672** · Stripe sandboxes and test mode — Stripe documentation, n.d., accessed 2026-11. Isolated test environments where no payments are processed. *Product and API*

documentation. Cited in: Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.

- **R673** · Astound Commerce enlists Hobbycraft for e-commerce replatforming project — Retail Technology Innovation Hub, 2022. An agency-led replatforming, agency-sourced. *Practitioner and press.* Cited in: Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R674** · Announcing the 2024 DORA report — Google Cloud, 2024. AI adoption associated with lower delivery throughput and stability. *Product and API documentation.* Cited in: Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R675** · State of AI-assisted Software Development 2025 — DORA, 2025. AI as an amplifier of existing strengths and weaknesses. *Research.* Cited in: Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R676** · Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity — METR (arXiv), 2025. The randomized trial showing a 19% slowdown. *Research.* Cited in: Who Builds and Who Owns: Teams, Vendors, Agencies and Governance.
- **R677** · Stripe and OpenAI launch Instant Checkout — Stripe newsroom, 2025. The launch, US Etsy sellers, the scoped payment token. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R678** · Product Feed Spec — OpenAI developers, n.d., accessed 2026-11. Required fields, US targeting. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R679** · OpenAI shifts checkout plans — Digital Commerce 360, 2026. The March 2026 retreat and the "integrity of transactions" argument. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R680** · Shopify says purchases are coming inside ChatGPT — Modern Retail, 2026. The March 2026 retreat and the "integrity of transactions" argument. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R681** · Announcing Agent Payments Protocol (AP2) — Google Cloud, 2025. Mandates and partners. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R682** · AP2 repository — n.d., accessed 2026-11. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R683** · Universal Cart at Google Marketing Live — Google, 2026. UCP launch, geography, "hundreds of thousands" enabled. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R684** · Google Shopping holiday updates — Google, 2026. UCP launch, geography, "hundreds of thousands" enabled. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.

- **R685** · ucp.dev — UCP Authors, 2026. Capabilities, transports, councils, absence of version tags. *Standard or specification.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R686** · UCP MAINTAINERS — n.d., accessed 2026-11. Capabilities, transports, councils, absence of version tags. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R687** · Universal Commerce Protocol repository — n.d., accessed 2026-11. Capabilities, transports, councils, absence of version tags. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R688** · Fall '26 Edition — Shopify, 2026. Agentic Storefronts, Sidekick app extensions, the 4.2x figure. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R689** · Agentic storefronts — Shopify Help Center, n.d., accessed 2026-11. Buyer geography and product exclusions. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R690** · ChatGPT channel — Shopify Help Center, n.d., accessed 2026-11. Buyer geography and product exclusions. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R691** · Google AI Mode and Gemini — Shopify Help Center, n.d., accessed 2026-11. Buyer geography and product exclusions. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R692** · Build for agents — shopify.dev, n.d., accessed 2026-11. Agent profiles, trust tiers, cache limits, the MCP-to-UCP replacement. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R693** · Catalog — shopify.dev, n.d., accessed 2026-11. Agent profiles, trust tiers, cache limits, the MCP-to-UCP replacement. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R694** · Storefront MCP — shopify.dev, n.d., accessed 2026-11. Agent profiles, trust tiers, cache limits, the MCP-to-UCP replacement. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R695** · Expanding your AI horizons, Summer '25 — Shopify, 2025. Sidekick's capabilities and review step. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.

- **R696** · Sidekick — Shopify Help Center, n.d., accessed 2026-11. Sidekick's capabilities and review step. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R697** · Summer '23 Edition — Shopify, 2023. Sidekick's capabilities and review step. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R698** · MCP Server — Shopware developer docs, n.d., accessed 2026-11. Copilot's agentic mode and the experimental MCP server. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R699** · Adobe Commerce commits to agentic standards — Glenbrook (relay), 2026. Vendor programs one layer down. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R700** · Adyen Agentic — Adyen, 2026. Vendor programs one layer down. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R701** · Klarna Agentic Product Protocol — Klarna, 2025. Vendor programs one layer down. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R702** · PayPal and Perplexity launch Instant Buy — PayPal, 2025. Vendor programs one layer down. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R703** · Amazon Rufus / Alexa for Shopping — Amazon, n.d., accessed 2026-11. The 13 May 2026 rename and Auto-Buy. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R704** · Buy for Me — Retail Dive, 2025. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R705** · Amazon and Perplexity — Engadget, 2025. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R706** · Amazon blocks Meta's Muse — The Star (Bloomberg), 2026. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R707** · Perplexity wins the appeal — EcomCrew, 2026. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R708** · Klarna AI assistant first month — Klarna, 2024. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R709** · Klarna rehires humans — Entrepreneur (Bloomberg), 2025. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R710** · Amazon generative AI listings — Amazon, n.d., accessed 2026-11. Vendor claims, labeled as such. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.

- **R711** · Stripe Radar — Stripe, n.d., accessed 2026-11. Vendor claims, labeled as such. *Product and API documentation.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R712** · Visa, BILL and Billtrust B2B pilot — Visa, 2026. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R713** · Cloudflare and payments companies on Trusted Agent Protocol — Cloudflare, 2025. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R714** · Mastercard unveils Agent Pay — Mastercard, 2025. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R715** · Europe is building the foundations for trusted agentic commerce — Mastercard, 2026. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R716** · Worldline and ING complete Europe's first agentic payment in production — Worldline, 2026. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R717** · Generative AI holiday shopping traffic 2025 — Digital Commerce 360 (Adobe data), 2026. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R718** · PSD3/PSR final compromise texts are published — William Fry, 2026. Status of the files. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R719** · HTTP Message Signatures for automated traffic (draft-ietf-webbotauth-httpsig-protocol-01) — IETF Web Bot Auth working group, 2026. *Standard or specification.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R720** · Forget IPs: using cryptography to verify bot and agent traffic — Cloudflare, 2025. *Company engineering.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R721** · Agentic payments: a new challenge for Europe's payments ecosystem — Osborne Clarke, 2026. No special regime under PSD2. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R722** · Agentic AI and consumer protection — CERRE (C. Busch), 2026. The consumer-law questions. *Practitioner and press.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R723** · DeepAR — Salinas, Flunkert and Gasthaus, 2017. Forecasting methods, not current systems. *Research.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R724** · Deep learning based forecasting: a case study from the online fashion industry — Kunz et al., 2023. Forecasting methods, not current systems. *Research.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026.

Company records and case evidence

- **R725** · Black Friday Cyber Monday 2025 data — Shopify, 2025. $14.6 billion and $5.1 million per minute. *Company filing, report or announcement.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Who Sells What to Whom: Business Models and the Economics That Shape Architecture, From Cart to Confirmation: Checkout Orchestration, Black Friday Every Year: Peak Load, Resilience and Recovery, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R726** · Shopify completes sale of Shopify Logistics to Flexport — Shopify, 2023. A vendor exiting a capability. *Company filing, report or announcement.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, The Map of a Commerce Platform: Capabilities and Layers, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms.
- **R727** · Zalando and ABOUT YOU successfully complete transaction — Zalando, 2025-07-11. Shared logistics and payment infrastructure, about €100 million of synergies. *Company filing, report or announcement.* Cited in: Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R728** · Zalando SE Annual Report 2025 — Zalando, 2026. Partner business definition and 32.1% share of GMV; average basket €62.8; minimum order fee; gross margin 39.9%. *Company filing, report or announcement.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, What Changes at Each Scale: From First Store to Group, Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R729** · Amazon.com, Inc. Form 10-K for fiscal 2025 — Amazon / SEC, 2026. Not the seller of record for third-party sales; third-party seller services $172.2 billion; subscription services $49.6 billion. *Company filing, report or announcement.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Discounts, Coupons and Loyalty: Promotions Without Chaos, Case: Amazon, From Monolith to Services to the Everything Store.
- **R730** · Shopify Inc. Form 10-K for fiscal 2025 — U.S. SEC, 2026. GMV definition; most merchants on Basic and Grow, most GMV from Plus and enterprise. *Company filing, report or announcement.* Cited in: Who Sells What to Whom: Business Models and the Economics That Shape Architecture, Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Taking the Money: Payment Providers, Authorization, Capture and Refunds, Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics, Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says, Many Merchants,

One Platform: Multi-Tenancy, Pods and Isolation, Apps, Plugins and Functions: Extensibility and Ecosystem Governance, What Changes at Each Scale: From First Store to Group, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.

- **R731** · Shopify Q4 2025 financial results — Shopify, 2026. 2025 revenue; GMV as reported by. *Company filing, report or announcement.* Cited in: The Map of a Commerce Platform: Capabilities and Layers.
- **R732** · Shopware Community Day 2026 — Shopware, 2026. Nexus, Shopware Payments, MCP coverage and the integration-cost claims. *Company filing, report or announcement.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R733** · Shopware remains market leader in Germany 2026 (EHI) — Shopware, 2026. 12.5% of the top 1,000 German B2C shops. *Company filing, report or announcement.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R734** · Shopware a Visionary in the 2025 Gartner Magic Quadrant — Shopware, 2025. Report date and vendor count. *Company filing, report or announcement.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R735** · commercetools reports strong Q1 2026 — commercetools, 2026. The $100B run-rate, orders, countries, the CEO. *Company filing, report or announcement.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R736** · commercetools launches AgenticLift — commercetools, 2026. The "on top of your existing commerce solution" layer. *Company filing, report or announcement.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Build, Buy or Assemble: Choosing and Changing Platforms, Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R737** · commercetools launches modular commerce offerings — commercetools, 2026. Standalone Core Commerce and Product Catalog modules; the "more than 600" boilerplate. *Company filing, report or announcement.* Cited in: Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026, Case: commercetools, Shopware and the Composable Movement.
- **R738** · Zalando Q3 2026 results — Zalando, 2026. 22 million unique Assistant users. *Company filing, report or announcement.* Cited in: Finding the Product: Search, Navigation and Merchandising, Assistants and Agents: AI in Commerce and What Is Real in 2026.
- **R739** · Letter to shareholders 2025 — Zalando SE, 2026. The shared stock pool; 24 markets and 18 channels. *Company filing, report or announcement.* Cited in: Is It

in Stock? Inventory, Availability and the Promise You Can Keep, Pick, Pack, Ship: Warehouse Management and Fulfillment.

- **R740** · Shopify launches Global ERP Program — Shopify, 2021. The program and its launch partners (older than three years). *Company filing, report or announcement.* Cited in: The Back Office: ERP Integration and Who Is the Hub.
- **R741** · Shopify merchants achieve record-breaking $14.6 billion in Black Friday Cyber Monday sales — Shopify investor press release, 2025. BFCM 2025 sales, peak sales per minute, edge, app-server and API requests per minute, and the unaudited caveat. *Company filing, report or announcement.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R742** · Black Friday Cyber Monday 2024 data — Shopify News, 2024. BFCM 2024 load figures for comparison. *Company filing, report or announcement.* Cited in: Black Friday Every Year: Peak Load, Resilience and Recovery, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R743** · The agentic commerce platform — Shopify, 2026. Agentic Storefronts, Sidekick app extensions, the 4.2x figure. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R744** · Winter '26 Edition — Shopify, 2025. Agentic Storefronts, Sidekick app extensions, the 4.2x figure. *Company filing, report or announcement.* Cited in: Assistants and Agents: AI in Commerce and What Is Real in 2026, Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R745** · Form 8-K exhibit 99.1, Q3 2026 results — Shopify Inc. via SEC EDGAR, 2026. Latest quarter. *Company filing, report or announcement.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R746** · Form 10-Q, quarter ended 30 September 2026 — Shopify Inc. via SEC EDGAR, 2026. Latest quarter. *Company filing, report or announcement.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R747** · Horizontally scaling the Rails backend of Shop app with Vitess — Shopify Engineering, 2024. Shop app only. *Company engineering.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R748** · How Shopify is re-architecting with YugabyteDB — Yugabyte (vendor blog), 2026. Unconfirmed vendor account. *Company engineering.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R749** · Adventures in Garbage Collection — Shopify Engineering, 2023. Deploy cadence. *Company engineering.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R750** · Shopify: observability at global scale — ClickHouse (vendor write-up), 2026. *Company engineering.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.

- **R751** · Shopify Scripts will be deprecated on June 30, 2026 — shopify.dev changelog, 2026. The timeline and plan gating. *Product and API documentation.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R752** · Shop Pay — Shopify, n.d., accessed 2026-11. Vendor claims. *Product and API documentation.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R753** · Shop Pay help — n.d. *Product and API documentation.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R754** · Reserved to committed — 2026. Inventory states. *Product and API documentation.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R755** · BFCM 2023 — Shopify News and Investors, 2023. Peak figures and caveats. *Company filing, report or announcement.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R756** · Internal memo: Shopify CEO declares AI non-optional — Digital Commerce 360, 2025. AI in Shopify's engineering, self-reported. *Practitioner and press.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R757** · Under the River — Shopify Engineering, 2026. AI in Shopify's engineering, self-reported. *Company engineering.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R758** · Back to native — 2026, Shopify Engineering. AI in Shopify's engineering, self-reported. *Company engineering.* Cited in: Case: Shopify, the Modular Monolith That Serves Millions of Merchants.
- **R759** · Amazon's Dynamo — Werner Vogels, 2007 (2 October). Dynamo was internal and not exposed as a web service. *Company engineering.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R760** · Amazon DynamoDB: a fast and scalable NoSQL database service — Werner Vogels, 2012 (18 January). Why Dynamo did not spread and teams preferred managed services. *Company engineering.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R761** · Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service — Elhemali et al., USENIX ATC, 2022 (July). 89.2 million requests per second during the 66-hour Prime Day 2021 event. *Research.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R762** · 10 Lessons from 10 Years of Amazon Web Services — Werner Vogels, 2016 (11 March). The API lesson learned "from our experiences with Amazon retail". *Company engineering.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R763** · Fulfilling — Werner Vogels, 2006 (19 September). Opening services and the fulfillment network to others. *Company engineering.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R764** · Amazon launches Fulfillment by Amazon and WebStore by Amazon — Amazon press release, 2006 (19 September). The FBA launch and Prime eligibility

for FBA items. *Company filing, report or announcement.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.

- **R765** · Inside the Mind of Jeff Bezos — Fast Company, 2004. Usual dating of the two-pizza phrase; not fetched. *Practitioner and press.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R766** · 2018 Letter to Shareholders — Amazon / Jeff Bezos, 2019 (April). Third-party share series 1999–2018 and the credit to FBA and Prime. *Company filing, report or announcement.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R767** · 2024 Letter to Shareholders — Amazon / Andy Jassy, 2025 (April). Regionalization, 1 million to 300 million Prime items, delivery reach. *Company filing, report or announcement.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R768** · 2025 Letter to Shareholders — Amazon / Andy Jassy, 2026 (9 April). Robots, 85+ same-day centers with 90,000 SKUs, rural network, Amazon Now. *Company filing, report or announcement.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R769** · 2022 Small Business Empowerment Report — Amazon, 2023 (May). "more than 60% of sales" from independent sellers. *Company filing, report or announcement.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R770** · 2024 Small Business Empowerment Report — Amazon, 2025 (May). 80 billion FBA items since 2006, Multi-Channel Fulfillment sellers, sellers over $1 million. *Company filing, report or announcement.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R771** · Amazon Steers Third-Party Seller Share To All-Time High — Marketplace Pulse, 2025. Independent tracking of the unit share at 60–62%. *Practitioner and press.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R772** · Amazon Sets New Prime Delivery Speed Record in 2025 — syndicated coverage of Amazon's 3 February 2026 release, 2026. 13 billion same- or next-day items; primary release not fetched. *Practitioner and press.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R773** · Buy with Prime — Amazon, n.d., accessed 2026-11. Scope of the service, integrations and the 167-merchant test. *Product and API documentation.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R774** · Amazon announces Rufus — Amazon, 2024 (page updated 2026 with the rename to Alexa for Shopping). Launch scope and training data. *Company filing, report or announcement.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R775** · Meet Alexa for Shopping — Amazon, 2026, accessed 2026-11. Usage figures, auto-buy, reorders, Buy for Me catalog growth. *Company filing, report or announcement.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.

- **R776** · How Amazon is using generative and agentic AI to transform shopping — Amazon, 2026, accessed 2026-11. Usage figures, auto-buy, reorders, Buy for Me catalog growth. *Company filing, report or announcement.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R777** · Buy for Me button on Amazon Shopping app — Amazon, 2025, accessed 2026-11. How the agent purchase works and what the brand keeps. *Company filing, report or announcement.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R778** · Amazon tests 'Buy for Me' agentic AI shopping tool — Digital Commerce 360 / Beth Duckett, 2025 (8 April). Launch date and the no-commission note. *Practitioner and press.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R779** · Amazon introduces feeds for Shop Direct — Amazon, 2026, accessed 2026-11. 100 million products, 400,000 merchants, feed partners. *Company filing, report or announcement.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R780** · Statement about Perplexity — Amazon, 2025 (letter dated 31 October). Amazon's rule for third-party agents. *Company filing, report or announcement.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R781** · Amazon sues Perplexity over AI shopping agents — Retail Dive, 2025 (6 November). The lawsuit and Perplexity's response. *Practitioner and press.* Cited in: Case: Amazon, From Monolith to Services to the Everything Store.
- **R782** · Radical Agility on AWS (video post) — Zalando Engineering / Lauri Apple, 2015 (7 July). Bowman's "unit of trust" and STUPS as the platform for autonomous teams. *Company engineering.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R783** · Data Integration in a World of Microservices — Zalando Engineering / Olaf Melchior, 2015 (21 September). The cost of distributed data and the Kafka deployment. *Company engineering.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R784** · From Monolith to Microservices (video) — Zalando Engineering, 2016 (10 February). "85+ engineering teams". *Company engineering.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R785** · Four Lessons with Microservices — Zalando Engineering / Malte Pickhan, 2016 (27 April). Two services became eight; weekly goals. *Company engineering.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R786** · Zalando Tech Radar (2016 announcement) — Zalando Engineering / Thomas Frauenstein, 2016 (17 May). The radar's purpose and "one input source". *Company engineering.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R787** · Platform Engineering and Third Generation Microservices in Dublin — Zalando Engineering / Bill de hÓra, 2017 (26 May). The event-driven intent.

Company engineering. Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.

- **R788** · Agile: Principles over Frameworks — Zalando Engineering / Samir Keck, 2018 (2 August). Principles replacing a prescribed method. *Company engineering.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R789** · Agentic Platform: Open-Sourcing the Agentic Identity Broker — Zalando Engineering / Brennenstuhl and Jungsbluth, 2026 (25 September). Delegated access and token exchange. *Company engineering.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R790** · Zalando RESTful API Guidelines (GitHub repository) — Zalando, n.d., accessed 2026-11. Zalando, n.d., accessed 2026-11; rules #106–#115 and #185–#191. *Product and API documentation.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R791** · Zalando Tech Radar (live radar) — Zalando, n.d., accessed 2026-11. Ring definitions. *Company engineering.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R792** · STUPS — Zalando and the Patroni project, GitHub, n.d., accessed 2026-11 (status from the GitHub API on 7 November 2026). Project descriptions and archive notices. *Product and API documentation.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R793** · Skipper — Zalando and the Patroni project, GitHub, n.d., accessed 2026-11 (status from the GitHub API on 7 November 2026). Project descriptions and archive notices. *Product and API documentation.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R794** · Patroni — Zalando and the Patroni project, GitHub, n.d., accessed 2026-11 (status from the GitHub API on 7 November 2026). Project descriptions and archive notices. *Product and API documentation.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R795** · Nakadi — Zalando and the Patroni project, GitHub, n.d., accessed 2026-11 (status from the GitHub API on 7 November 2026). Project descriptions and archive notices. *Product and API documentation.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R796** · Zalenium — Zalando and the Patroni project, GitHub, n.d., accessed 2026-11 (status from the GitHub API on 7 November 2026). Project descriptions and archive notices. *Product and API documentation.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R797** · A Decade of Changing the Game for Partners — Zalando, 2021 (24 June). Partner Program figures and targets, ZFS reach. *Company filing, report or announcement.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R798** · Zalando's Connected Retail Program Offers Support to Physical Stores — Zalando, 2020 (31 March). 1,500 stores, carriers, commission waiver. *Company*

filing, report or announcement. Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.

- **R799** · Stronger Together: Connecting Online and Offline Retail — Zalando, 2021 (25 January). 2,600 stores, 2018 launch year. *Company filing, report or announcement.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R800** · Zalando Onboards C&A to Connected Retail and Expands Program to Austria — Zalando, 2021 (15 February). 3,000 stores in nine markets. *Company filing, report or announcement.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R801** · ABOUT YOU and Zalando complete final step of their strategic combination — ABOUT YOU, 2025 (6 November). The squeeze-out and delisting. *Company filing, report or announcement.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R802** · Zalando SE FY/25 Fact Sheet — Zalando, 2026 (12 March). GMV, revenue, customers, orders, B2B revenue, 2026 guidance. *Company filing, report or announcement.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R803** · ZEOS — Zalando, n.d., accessed 2026-11. The modular service positioning. *Product and API documentation.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R804** · RFC 2119 — The guidelines' MUST/SHOULD/MAY follow, IETF, 1997. The deprecation headers follow the Sunset header convention the guidelines cite. Contract rules as the book uses them are in Promises Between Systems: API Contracts, Webhooks and Versioning. *Standard or specification.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R805** · Zalando's STUPS: an open-source PaaS on AWS — InfoQ / Daniel Bryant, 2016 (4 April). STUPS scope and audit model. *Company engineering.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R806** · Scaling Autonomy at Zalando — InfoQ / Ben Linders, 2018 (30 September). Bowman on autonomy and alignment. *Company engineering.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R807** · Zalando embraces radical agility in its bold platform play — Diginomica, 2016. Team sizes of three to seven (secondary). *Practitioner and press.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R808** · ZEOS: Zalando Ecommerce Operating System — ChannelX, 2023 (26 October). The 2022 pilot and the October 2023 rebrand. *Practitioner and press.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.
- **R809** · Zalando to axe 'hundreds of jobs' amid economic slowdown — Retail Gazette, 2023 (21 February). The overhead reduction. *Practitioner and press.* Cited in: Case: Zalando, Microservices, Platform Strategy and the Return to Discipline.

- **R810** · commercetools appoints new CEO — commercetools, 2024. Andrew Burton's appointment, Dirk Hoerig's move, the 2023 GMV. *Company filing, report or announcement.* Cited in: Case: commercetools, Shopware and the Composable Movement.
- **R811** · commercetools introduces "Autonomous Commerce" category and unveils Sphere — commercetools, 2026. The June 2026 category and platform naming. *Company filing, report or announcement.* Cited in: Case: commercetools, Shopware and the Composable Movement.
- **R812** · commercetools named a Leader in the 2026 Gartner Magic Quadrant for Digital Commerce Platforms — commercetools, 2026. The seventh consecutive placement, report dated 19 October 2026. *Company filing, report or announcement.* Cited in: Case: commercetools, Shopware and the Composable Movement.
- **R813** · Shopware raises USD 100 million from Carlyle and PayPal — Shopware, 2022. The bootstrapped history and 2021 GMV (more than three years old). *Company filing, report or announcement.* Cited in: Case: commercetools, Shopware and the Composable Movement.
- **R814** · Shopware named a Strong Performer in Forrester's 2026 commerce evaluation — Shopware, 2026. The Forrester placement and the boilerplate on PayPal's stake. *Company filing, report or announcement.* Cited in: Case: commercetools, Shopware and the Composable Movement.
- **R815** · Shopware launches Shopware Payments in partnership with PayPal — Shopware, 2026. Transaction pricing and onboarding through PayPal. *Company filing, report or announcement.* Cited in: Case: commercetools, Shopware and the Composable Movement.
- **R816** · New management structure at shopware AG — Shopware, 2026. The CEO succession and founder roles. *Company filing, report or announcement.* Cited in: Case: commercetools, Shopware and the Composable Movement.
- **R817** · founding announcement — Mage-OS Association, 2022–2026, accessed 2026-11. The distribution, the association, releases 3.6.0 and 4.0.0. *Product and API documentation.* Cited in: Case: commercetools, Shopware and the Composable Movement.
- **R818** · MACH explained — MACH Alliance, n.d., accessed 2026-11. The three principles as stated in 2026. *Practitioner and press.* Cited in: Case: commercetools, Shopware and the Composable Movement.

Sources by Chapter

Each line lists the identifiers a page cites in its **Sources and Evidence** section, in identifier order, with pages in reading order. Pages that cite no external source (most part introductions) are omitted.

- Introduction: Why Commerce Architecture Matters (5): R94, R97, R340, R411, R486.

- Out of Stock, Out of Sync: Sixteen Ways Commerce Platforms Go Wrong (46): R1, R2, R3, R4, R5, R6, R95, R96, R97, R98, R186, R187, R188, R189, R190, R258, R259, R260, R339, R340, R408, R409, R410, R411, R412, R485, R486, R487, R488, R489, R490, R576, R577, R578, R579, R580, R635, R636, R643, R644, R645, R646, R647, R725, R726, R727.
- Follow the Order: What Happens Between "Buy" and "Delivered" (19): R1, R2, R5, R7, R8, R9, R10, R11, R12, R13, R190, R261, R262, R263, R412, R637, R638, R639, R640.
- Who Sells What to Whom: Business Models and the Economics That Shape Architecture (33): R6, R12, R13, R14, R15, R16, R17, R18, R19, R20, R21, R22, R23, R24, R25, R26, R27, R28, R29, R30, R31, R32, R33, R34, R35, R36, R37, R38, R39, R725, R728, R729, R730.
- The Map of a Commerce Platform: Capabilities and Layers (19): R4, R19, R29, R40, R41, R42, R43, R44, R45, R46, R47, R48, R49, R50, R51, R52, R53, R726, R731.
- Who Owns the Truth: Boundaries, Bounded Contexts and Systems of Record (11): R40, R47, R54, R55, R56, R341, R342, R413, R414, R415, R416.
- Suites, SaaS, Open Source and Composable: The Platform Landscape in 2026 (65): R4, R19, R20, R24, R25, R29, R40, R41, R43, R44, R46, R48, R49, R50, R51, R52, R53, R57, R58, R59, R60, R61, R62, R63, R64, R65, R66, R67, R68, R69, R70, R71, R72, R73, R74, R75, R76, R77, R78, R79, R80, R81, R82, R83, R84, R85, R86, R87, R88, R89, R90, R91, R92, R93, R491, R648, R649, R726, R730, R732, R733, R734, R735, R736, R737.
- What Are We Selling? Products, Variants and the Catalog (21): R99, R100, R101, R102, R103, R104, R105, R106, R107, R108, R109, R110, R111, R112, R113, R114, R115, R116, R117, R118, R414.
- What Does It Cost? Price Lists, Contract Prices, Currencies and Tax (23): R48, R97, R98, R115, R119, R120, R121, R123, R124, R125, R126, R127, R128, R129, R130, R131, R132, R133, R134, R135, R136, R137, R139.
- Discounts, Coupons and Loyalty: Promotions Without Chaos (22): R18, R97, R98, R123, R134, R135, R140, R141, R142, R143, R144, R145, R146, R147, R148, R149, R150, R151, R152, R153, R640, R729.
- Finding the Product: Search, Navigation and Merchandising (25): R97, R112, R144, R154, R155, R156, R157, R158, R159, R160, R161, R162, R163, R164, R165, R166, R167, R168, R169, R170, R171, R172, R173, R581, R738.
- Is It in Stock? Inventory, Availability and the Promise You Can Keep (23): R1, R11, R95, R96, R174, R175, R176, R177, R178, R179, R180, R181, R182, R183, R184, R185, R260, R263, R264, R409, R413, R416, R739.
- From Cart to Confirmation: Checkout Orchestration (26): R10, R13, R186, R188, R190, R191, R192, R193, R194, R195, R196, R197, R198, R199, R200, R201, R202, R203, R204, R205, R206, R341, R412, R492, R637, R725.
- Taking the Money: Payment Providers, Authorization, Capture and Refunds (50): R8, R9, R25, R31, R187, R189, R190, R197, R198, R199, R203, R204, R205, R207, R208, R209, R210, R211, R212, R213, R214, R215, R216, R217, R218, R219, R220, R221, R222,

R223, R224, R225, R226, R227, R228, R229, R230, R231, R232, R233, R234, R235, R236, R412, R582, R583, R584, R585, R637, R730.

- The Order Record: Lifecycle, State and the Order Management System (19): R1, R11, R45, R47, R177, R181, R237, R238, R239, R240, R241, R242, R243, R261, R264, R265, R266, R409, R493.
- Tax, Invoices and the Books: From Order to Ledger (34): R12, R33, R136, R187, R208, R209, R212, R221, R225, R244, R245, R246, R247, R248, R249, R250, R251, R252, R253, R254, R255, R256, R257, R342, R415, R579, R586, R587, R588, R635, R638, R639, R640, R641.
- Pick, Pack, Ship: Warehouse Management and Fulfillment (22): R11, R177, R183, R184, R260, R264, R265, R266, R267, R268, R269, R270, R271, R272, R273, R274, R275, R276, R277, R278, R279, R739.
- Getting It There: Carriers, Labels, Tracking and the Delivery Promise (24): R136, R262, R263, R267, R280, R281, R282, R283, R284, R285, R286, R287, R288, R289, R290, R291, R292, R293, R294, R295, R296, R297, R298, R299.
- Stores Are Systems Too: Point of Sale and Omnichannel Fulfillment (29): R181, R259, R260, R264, R265, R266, R300, R301, R302, R303, R304, R305, R306, R307, R308, R309, R310, R311, R312, R313, R314, R315, R316, R317, R318, R319, R320, R321, R415.
- The Way Back: Returns, Exchanges, Refunds and Reverse Logistics (14): R6, R35, R87, R258, R261, R263, R322, R323, R324, R325, R326, R327, R328, R329.
- Where Is My Order? Notifications, Service and the Customer Record (22): R1, R2, R5, R47, R237, R239, R261, R262, R263, R264, R286, R291, R325, R330, R331, R332, R333, R334, R335, R336, R337, R338.
- Selling on Someone Else's Platform: Marketplaces and Channel Integration (38): R26, R48, R132, R137, R259, R275, R335, R336, R337, R339, R340, R343, R344, R345, R346, R347, R348, R349, R350, R351, R352, R353, R354, R355, R356, R357, R358, R359, R360, R361, R362, R363, R364, R365, R366, R367, R368, R589.
- Selling to Businesses: Accounts, Contract Prices, Quotes and Approvals (27): R36, R119, R125, R128, R202, R341, R342, R369, R370, R371, R372, R373, R374, R375, R376, R377, R378, R379, R380, R381, R382, R383, R384, R385, R386, R387, R388.
- Documents Between Companies: EDI, Punchout and E-Invoicing (30): R253, R254, R255, R276, R277, R278, R279, R387, R388, R389, R390, R391, R392, R393, R394, R395, R396, R397, R398, R399, R400, R401, R402, R403, R579, R588, R590, R591, R635, R636.
- The Back Office: ERP Integration and Who Is the Hub (15): R44, R54, R109, R341, R342, R379, R382, R404, R405, R406, R407, R413, R414, R415, R740.
- APIs, Events, Files and Middleware: How Commerce Systems Talk (18): R409, R410, R411, R412, R413, R414, R415, R417, R418, R419, R420, R421, R422, R423, R424, R425, R426, R427.
- Built to Disagree and Recover: Events, Eventual Consistency and Reconciliation (22): R190, R221, R243, R409, R410, R412, R413, R416, R428, R429, R430, R431, R432, R433, R434, R435, R436, R437, R438, R439, R637, R642.

- Promises Between Systems: API Contracts, Webhooks and Versioning (27): R409, R410, R411, R412, R417, R428, R431, R432, R440, R441, R442, R443, R444, R445, R446, R447, R448, R449, R450, R451, R452, R453, R454, R455, R456, R457, R494.
- One Customer, One Product: Identity and Master Data Across Systems (13): R408, R414, R415, R458, R459, R460, R461, R462, R463, R464, R465, R466, R577.
- Counting the Same Thing Everywhere: Data Platforms, Definitions and Analytics (23): R221, R416, R428, R467, R468, R469, R470, R471, R472, R473, R474, R475, R476, R477, R478, R479, R480, R481, R482, R483, R484, R728, R730.
- Monolith, Modular Monolith, Microservices, Composable: What the Evidence Says (37): R4, R53, R91, R144, R269, R486, R487, R488, R489, R492, R493, R495, R496, R497, R498, R499, R500, R501, R502, R503, R504, R505, R506, R507, R508, R509, R510, R511, R512, R513, R514, R515, R516, R650, R651, R728, R730.
- Many Merchants, One Platform: Multi-Tenancy, Pods and Isolation (23): R41, R86, R144, R343, R409, R410, R440, R510, R517, R518, R519, R520, R521, R522, R523, R524, R525, R526, R527, R528, R529, R530, R730.
- Apps, Plugins and Functions: Extensibility and Ecosystem Governance (30): R174, R186, R194, R195, R196, R410, R411, R490, R491, R492, R494, R527, R531, R532, R533, R534, R535, R536, R537, R538, R539, R540, R541, R542, R543, R544, R545, R546, R547, R730.
- Delivering the Storefront: Themes, Headless, Rendering and Performance (26): R58, R491, R493, R494, R499, R521, R531, R536, R548, R549, R550, R551, R552, R553, R554, R555, R556, R557, R558, R559, R560, R561, R562, R563, R564, R578.
- Black Friday Every Year: Peak Load, Resilience and Recovery (24): R409, R428, R434, R438, R439, R485, R521, R522, R547, R565, R566, R567, R568, R569, R570, R571, R572, R573, R574, R575, R642, R725, R741, R742.
- Protecting the Shop: Card Data, Account Takeover, Bots and Skimmers (18): R198, R213, R222, R223, R234, R576, R580, R592, R593, R594, R595, R596, R597, R598, R599, R600, R601, R602.
- Whose Data Is It? Privacy, Consent and Retention (21): R534, R577, R586, R603, R604, R605, R606, R607, R608, R609, R610, R611, R612, R613, R614, R615, R616, R617, R618, R619, R620.
- Rules Turned Into Requirements: Consumer, Payment, Invoicing and Platform Regulation (35): R97, R205, R263, R578, R579, R580, R581, R582, R583, R584, R585, R587, R588, R589, R590, R591, R604, R617, R620, R621, R622, R623, R624, R625, R626, R627, R628, R629, R630, R631, R632, R633, R634, R635, R652.
- Journey 1: Eva's Jacket, From Search to Doorstep (41): R1, R2, R5, R8, R9, R11, R12, R13, R25, R95, R96, R136, R174, R186, R188, R190, R191, R199, R209, R220, R221, R229, R236, R260, R261, R262, R263, R264, R281, R282, R283, R291, R409, R410, R412, R637, R638, R639, R640, R641, R642.
- Journey 2: The Jacket Comes Back (13): R6, R187, R189, R208, R258, R261, R263, R272, R322, R323, R637, R639, R640.
- Journey 3: A Chain Reorders by EDI, From Order to Invoice (35): R12, R54, R202, R251, R253, R254, R255, R269, R276, R277, R278, R279, R341, R389, R390, R391, R392,

R393, R394, R395, R397, R398, R579, R583, R588, R590, R591, R635, R636, R637, R638, R639, R640, R641, R642.

- Journey 4: Black Friday, Hour by Hour (13): R409, R428, R434, R438, R439, R485, R522, R547, R565, R566, R567, R571, R574.
- Build, Buy or Assemble: Choosing and Changing Platforms (29): R4, R19, R20, R41, R43, R44, R49, R51, R52, R53, R58, R60, R73, R76, R91, R643, R648, R649, R652, R653, R654, R655, R656, R657, R658, R659, R660, R726, R736.
- Moving Without Burning the Bridge: Replatforming and Migration (15): R28, R500, R643, R644, R652, R653, R656, R657, R658, R660, R661, R662, R663, R664, R665.
- What Changes at Each Scale: From First Store to Group (21): R4, R14, R18, R19, R20, R27, R29, R44, R86, R488, R495, R514, R648, R649, R650, R651, R652, R654, R666, R728, R730.
- Who Builds and Who Owns: Teams, Vendors, Agencies and Governance (27): R194, R411, R440, R490, R504, R510, R512, R516, R545, R547, R567, R575, R650, R651, R652, R655, R659, R667, R668, R669, R670, R671, R672, R673, R674, R675, R676.
- Assistants and Agents: AI in Commerce and What Is Real in 2026 (64): R40, R112, R115, R165, R166, R205, R630, R631, R645, R646, R647, R677, R678, R679, R680, R681, R682, R683, R684, R685, R686, R687, R688, R689, R690, R691, R692, R693, R694, R695, R696, R697, R698, R699, R700, R701, R702, R703, R704, R705, R706, R707, R708, R709, R710, R711, R712, R713, R714, R715, R716, R717, R718, R719, R720, R721, R722, R723, R724, R732, R736, R738, R743, R744.
- Case: Shopify, the Modular Monolith That Serves Millions of Merchants (71): R57, R174, R176, R186, R191, R193, R195, R196, R226, R227, R228, R409, R410, R411, R417, R440, R485, R493, R494, R495, R496, R497, R498, R499, R517, R518, R520, R521, R522, R530, R531, R532, R533, R534, R548, R549, R550, R565, R646, R680, R685, R688, R689, R690, R691, R692, R693, R694, R695, R696, R697, R725, R730, R741, R742, R743, R744, R745, R746, R747, R748, R749, R750, R751, R752, R753, R754, R755, R756, R757, R758.
- Case: Amazon, From Monolith to Services to the Everything Store (29): R447, R489, R503, R505, R506, R729, R759, R760, R761, R762, R763, R764, R765, R766, R767, R768, R769, R770, R771, R772, R773, R774, R775, R776, R777, R778, R779, R780, R781.
- Case: Zalando, Microservices, Platform Strategy and the Return to Discipline (41): R165, R356, R444, R445, R446, R486, R487, R500, R502, R657, R663, R727, R728, R782, R783, R784, R785, R786, R787, R788, R789, R790, R791, R792, R793, R794, R795, R796, R797, R798, R799, R800, R801, R802, R803, R804, R805, R806, R807, R808, R809.
- Case: commercetools, Shopware and the Composable Movement (34): R4, R20, R41, R53, R58, R59, R66, R67, R68, R69, R72, R73, R76, R84, R91, R92, R491, R508, R509, R732, R733, R734, R735, R736, R737, R810, R811, R812, R813, R814, R815, R816, R817, R818.
- Domain Coverage: From Three Model Families to Forty-Nine Chapters (1): R94.
- Toolkit: Cards and Templates for Commerce Architecture Decisions (10): R221, R279, R409, R439, R547, R575, R623, R659, R664, R665.

Kestrel & Pine is fictional; architectural recommendations, labeled inferences and labeled assumptions are the author's judgement.